Архив статей wasm.ru. Том 8: Исследование программ

Собрание русскоязычных статей сайта wasm.ru о программировании, ассемблере, реверс-инжиниринге и компьютерной безопасности. Том 8 из 11. Разделы: 18. Исследование программ. Все приложенные файлы находятся в wasm_files.zip.

Больше крутых материалов в моём телеграм канале @grogrigs

18. Исследование программ

Идентификация IF - THEN - ELSE

Автор: Крис Касперски

Дата: 2002-06-16

> Значение каждого элемента текста определяется
> контекстом его употребления. Текст не описывает мир,
> а вступает в сложные взаимоотношения с миром.
> Тезис аналитической философии

Существует два вида алгоритмов - *безусловные* и *условные*. Порядок действий безусловного алгоритма всегда постоянен и не зависит от входных данных. Например: "a=b+c". Порядок действий условных алгоритмов, напротив, зависит от входных данных. Например: "**если** с не равно нулю, **то**: a=b/c; **иначе**: вывести сообщение об ошибке".

Обратите внимание на выделенные жирным шрифтом ключевые слова "если", "то" и "иначе", называемые **операторами условия** или **условными операторами**. Без них не обходится ни одна программа (вырожденные примеры наподобие "Hello, World!" - не в счет). Условные операторы - сердце любого языка программирования. Поэтому, чрезвычайно важно уметь их правильно идентифицировать.

В общем виде (не углубляясь в синтаксические подробности отдельных языков программирования) оператор условия схематично изображается так:

```
IF (условие) THEN { оператор_1; оператор_2;} ELSE { оператор_a; оператор_b;}
```

Задача компилятора - преобразовать эту конструкцию в последовательность машинных команд, выполняющих **оператор_1**, **оператор_2**, если условие истинно и, соответственно - **оператор_a**, **оператор_b**; если оно ложно. Однако микропроцессоры серии 80x86 поддерживают весьма скромный набор условных команд, ограниченный фактически одними условными переходами. Программистам, знакомым лишь с IBM PC, такое ограничение не покажется чем-то неестественным, между тем, существует масса процессоров, поддерживающих префикс условного выполнения инструкции. Т.е. вместо того, чтобы писать:

```
TEST ECX,ECX
JNZ xxx
MOV EAX,0x666
```

там поступают так:

```
TEST ECX,ECX
IFZ MOV EAX,0x666
```

"**IFZ**" - и есть префикс условного выполнения, разрешающий выполнение следующей команды только в том случае, если установлен флаг нуля.

В этом смысле микропроцессоры 80x86 можно сравнить с ранними диалектами языка Бейсика, не разрешающими использовать в условных выражениях никакой другой оператор кроме "GOTO". Сравните:

```
IF A=B THEN PRINT "A=B"  10 IF A=B THEN GOTO 30
20 GOTO 40
30 PRINT "A=B"
40 ... // прочий код программы

Старый диалект "Бейсика"        Новый диалект "Бейсика"
```

```
Листинг 139
```

Если вы когда-нибудь программировали на старых диалектах Бейсика, то, вероятно, помните, что гораздо выгоднее выполнять GOTO если условие ложно, а в противном случае продолжать нормальное выполнение программы. (Как видите, вопреки расхожему мнению, навыки программирования на Бейсике отнюдь не бесполезны, особенно - в дизассемблировании программ).

Большинство компиляторов (даже не оптимизирующих) инвертируют истинность условия, транслируя конструкцию

```
IF (условие) THEN {оператор_1; оператор_2}
```

в следующий псевдокод:

```
IF (NOT условие) THEN continue
оператор_1;
оператор_2;
continue:
...

Листинг 140
```

Следовательно, для восстановления исходного текста программы, нам придется вновь инвертировать условие и "подцепить" блок операторов {оператор_1; оператор_2} к ключевому слову THEN. Т.е. если откомпилированный код выглядит так:

```
10 IF A<>B THEN 30
20 PRINT "A=B"
30 ...// прочий код программы

Листинг 141
```

Можно с уверенностью утверждать, что в исходном тексте присутствовали следующие строки: "IF A=B THEN PRINT "A=B"". А если, программист, наоборот, проверял переменные A и B на неравенство, т.е. "IF A<>B THEN PRINT "A<>B""? Все равно компилятор инвертирует истинность условия и сгенерирует следующий код:

```
10 IF A=B THEN 30
20 PRINT "A<>B"
30 ...// прочий код программы

Листинг 142
```

Конечно, встречаются и дебильные компиляторы, страдающие многословием. Их легко распознать по безусловному переходу, следующему сразу же после условного оператора:

```
IF (условие) THEN do
GOTO continue
do:
оператор1;
оператор2;
continue:
...

Листинг 143
```

В таком случае инвертировать условие не нужно. Впрочем, если это сделать, ничего страшного не произойдет, разве что код программы станет менее понятным, да и то не всегда.

Рассмотрим теперь как транслируется полная конструкция

```
IF (условие) THEN {оператор_1; оператор_2;} ELSE {оператор_a; оператор_b;}
```

Одни компиляторы поступают так:

```
IF (условие) THEN do_it
// Ветка ELSE
```

```
оператор_a;
оператор_b
GOTO continue

do_it:
//Ветка IF
оператор_1;
оператор_2;
continue:
```

А другие так:

```
IF (NOT условие) THEN else
//Ветка IF
оператор_1;
оператор_2;
GOTO continue

else:
// Ветка ELSE
оператор_a;
оператор_b
continue:

Листинг 144
```

Разница межу ними в том, что вторые инвертируют истинность условия, а первые - нет. Поэтому, не зная "нрава" компилятора, определить: как выглядел подлинный исходный текст программы - невозможно! Однако это не создает проблем, ибо условие всегда можно записать так, как это удобно. Допустим, не нравится вам конструкция "IF (c<>0) THEN a=b/c ELSE PRINT "Ошибка!"" пишите ее так: "IF (c==0) THEN PRINT "Ошибка!" ELSE a=b/c" и - ни каких гвоздей!

Типы условий: Условия делятся на простые (элементарные) и сложные (составные). Пример первых - "if (a==b)...", вторых "if ((a==b) && (a!=0))...". Очевидно, что любое сложное условие можно разложить на ряд простых условий. Вот с простых условий мы и начнем.

Существуют два основных типа элементарных условий: условия отношений ("меньше", "равно", "больше", "меньше или равно", "не равно", "больше или равно", соответственно обозначаемые как: "<", "==", ">", "<=", "!=", ">=") и логические условия ("И", "ИЛИ", "НЕ", "И исключающее ИЛИ", в Си-нотации соответственно обозначаемые так: "&", "|", "!", "^"). Известный хакерский авторитет Мэтт Питрек приплетает сюда и проверку битов, однако несколько некорректно смешивать в одну кучу людей и коней, даже если они чем-то и взаимосвязаны. Поэтому, о битовых операциях мы поговорим отдельно.

Если условие истинно, оно возвращает булево значение TRUE, соответственно, если ложно - FALSE. Внутренне (физическое) представление булевых переменных зависит от конкретной реализации и может быть любым. По общепринятому соглашению, FALSE равно нулю, а TRUE не равно нулю. Часто (но не всегда) TRUE равно единице, но на это нельзя полагаться! Так, код "IF ((a>b)!=0)E" абсолютно корректен, а: "IF ((a>b)==1)E" привязан к конкретной реализации и потому нежелателен.

Обратите внимание: "IF ((a>b)!=0)E" проверяет на неравенство нулю отнюдь не значения самих переменных a и b, а именно - результата их сравнения. Рассмотрим следующий пример: "IF ((666==777)==0) printf("Woozl!")" - как вы думаете, что отобразится на экране, если его запустить? Правильно - "Woozl"! Почему? Ведь ни 666, ни 777 не равно нулю! Да, но ведь 666 != 777, следовательно, условие (666==777) - ложно, следовательно равно нулю. Кстати, если записать "IF ((a=b)==0)E" получится совсем иной результат - значение переменной b будет присвоено переменной a, и потом проверено на равенство нулю.

Логические условия чаще всего используются для связывания двух или более элементарных условий отношения в составное. Например, "IF ((a==b) && (a!=0))E". При трансляции программы

компилятор всегда выполняют развертку составных условий в простые. В данном случае это происходит так: "IF a==b THEN IF a=0 THENE" На втором этапе выполняется замена условных операторов на оператор GOTO:

```
IF a!=b THEN continue
IF a==0 THEN continue
...// код условия
:continue
...// прочий код
```

Листинг 145

Порядок вычисления элементарных условий в сложном выражении зависит от прихотей компилятора, гарантируется лишь, что условия, "связанные" операцией логического "И" проверяются слева направо в порядке их объявления в программе. Причем, если первое условие ложно, то следующее за ним вычислено не будет! Это дает возможность писать код наподобие следующего: "if ((filename) & (f=fopen(&filename[0],"rw")))E" - если указатель filename указывает на невыделенную область памяти (т.е. попросту говоря содержит нуль - логическое FALSE), функция fopen не вызывается и ее краха не происходит. Такой способ вычислений получил название "быстрых булевых операций" (теперь-то вы знаете, что подразумевается под "быстротой").

Перейдем теперь к вопросу идентификации логических условий и анализу сложных выражений. Вернемся к уже облюбованному нами выражению "if ((a==b) && (a!=0))E" и вглядимся в результат его трансляции:

```
IF a!=b THEN continue  -------!
IF a==0 THEN continue ---!    !
...// код условия        !    !
:continue             <--! <---
...// прочий код
```

Листинг 146

Легко видеть - он выдает себя серией условных переходов к одной и той же метке, причем, - обратите внимание, - выполняется проверка на неравенство каждого из элементарных условий, а сама метка расположена позади кода условия.

Идентификация логической операции "ИЛИ" намного сложнее в силу неоднозначности ее трансляции. Рассмотрим это на примере выражения "if ((a==b) || (a!=0))...". Его можно разбить на элементарные операции и так:

```
IF a==b THEN do_it -------------!
IF a!=0 THEN do_it -----!       !
goto continue -----!    !       !
:do_it             ! <--! <-----!
...// код условия  !
:continue     <-!
...// прочий код
```

Листинг 147

и так:

```
IF a==b THEN do_it  -----------!
IF a==0 THEN continue--!       !
:do_it                 ! <-----!
...// код условия      !
:continue  <-----------!
...// прочий код
```

Листинг 148

Первый вариант обладает весьма запоминающийся внешностью - серия проверок (без инверсии условия) на одну и ту же метку, расположенную перед кодом условия, а в конце этой серии - безусловный переход на метку, расположенную позади кода условия.

Однако оптимизирующие компиляторы выкидывают безусловный переход, инвертируя проверку последнего условия в цепочке и, соответственно меняя адрес перехода. По неопытности эту конструкцию часто принимают за смесь OR и AND. Кстати, о смещенных операциях - рассмотрим результат трансляции следующего выражения: "if ((a==b) || (a==c) && a(!=0))...":

```
IF a==b THEN check_null
IF a!=c THEN continue
check_null:
IF a==0 THEN continue
...// код условия
continue:
...// прочий код
```

Листинг 149

Как из непроходимого леса элементарных условий получить одно удобочитаемое составное условие? Начинаем плясать от печки, т.е. от первой операции сравнения. Смотрите, если условие a==b окажется истинно, оно "выводит из игры" проверку условия a!=c. Такая конструкция характерна для операции OR - т.е. достаточно выполнения хотя бы одного условия из двух для "срабатывания" кода. Пишем в уме или карандашом: "if ((a==b) || ...)", далее - если условие (a!=c) истинно, все дальнейшие проверки прекращаются, и происходит передача управления на метку, расположенную позади условного кода. Логично предположить, что мы имеем дело в последней операцией OR в цепочке сравнений - это ее "почерк". Значит, мы инвертируем условие выражения и продолжаем писать: "if ((a==b) || (a==c)E)". Последний бастион - проверка условия "a==0". Выполнить условный код, миновав его не удаться, - следовательно, это не OR, а AND! A AND всегда инвертирует условие срабатывания, и поэтому, оригинальный код должен был выглядеть так: "if ((a==b) || (a==c) && (a!=0))". Ура! У нас получилось!

Впрочем, как любил поговаривать Дмитрий Николаевич, не обольщайтесь - то, что мы рассмотрели - это простейший пример. В реальной жизни оптимизирующие компиляторы такого понаворочают...

___Впрочем, для ломания головы вполне хватит и не оптимизирующих, но прежде, чем перейти к изучению конкретных реализаций, рассмотрим на последок две "редкоземельные" операции NOT и XOR.

__NOT - одноместная операция, поэтому, она не может использоваться для связывания, однако,

Наглядное представление сложных условий в виде дерева. Конструкцию, состоящую из трех - четырех элементарных условий, можно проанализировать и в уме (да и то, если есть соответствующие навыки), но хитросплетения пяти и более условий образуют самый настоящий лабиринт - его с лету не возьмешь. Неоднозначность трансляции сложных условий порождает неоднозначность интерпретации, что приводит к многовариантному анализу, причем с каждым шагом в голове приходится держать все больше и больше информации. Так недолго и крышей поехать или окончательно запутаться и получить неверных результат.

Выход - в использовании двухуровневой системы ретрансляции. На первом этапе элементарные условия преобразуются к некоторой промежуточной форме записи, наглядно и непротиворечиво отображающей взаимосвязь элементарных операций. Затем осуществляется окончательная трансляция в любую подходящую нотацию (например, Си, Бейсик или Pascal).

Единственная проблема - выбрать удачную промежуточную форму. Существует множество решений, но в книге по соображениям экономии бумажного пространства, мы рассмотрим только одно - деревья.

Изобразим каждое элементарное условие в виде узла, с двумя ветвями, соответствующим состояниям: условие истинно и условие ложно. Для наглядности обозначим "ложь" равнобедренным треугольником, а "истину" - квадратом и условимся всегда располагать ложь на левой, а истину на правой ветке. Получившуюся конструкцию назовем "гнездом" (nest).

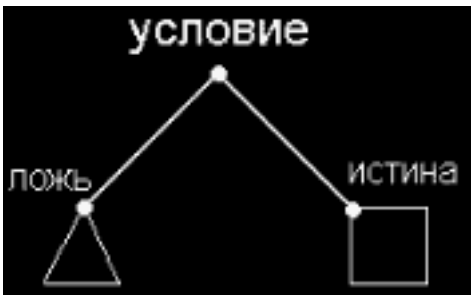


Рисунок 22. 0x015 Схематическое представление гнезда (nest)

Гнезда могут объединяться в деревья, соединясь узлами с ветками другого узла. Причем, каждый узел может соединяться только с одним гнездом, но всякое гнездо может соединяться с несколькими узлами. Непонятно? Не волнуйтесь, сейчас со всем этим мы самым внимательным образом разберемся.

Рассмотрим объединение двух элементарных условий логической операцией "AND" на примере выражения "((a==b) && (a!=0))". Извлекаем первое слева условие (a==b), "усаживаем" его в гнездо с двумя ветвями: левая соответствует случаю, когда a!=b (т.е. условие a==b - ложно), а правая, соответственно, - наоборот. Затем, то же самое делаем и со вторым условием (a!=0). У нас получаются два очень симпатичных гнездышка, - остается лишь связать их меж собой операцией логического "AND". Как известно, "AND" выполняет второе условие только в том случае, если истинно первое. Значит, гнездо (a!=0) следует прицепить к правой ветке гнезда (a==b). Тогда - правая ветка гнезда (a!=0) будет соответствовать истинности выражения "((a==b) && (a!=0))", а обе левые ветки - его ложности. Обозначим первую ситуацию меткой "do_it", а вторую - "continue". В результате дерево должно принять вид, изображенный на рис. 23.

Для наглядности отметим маршрут из вершины дерева к метке "do_it" жирной стрелкой. Как видите, в пункт "do_it" можно попасть только одним путем. Вот так графически выглядит операция "AND".

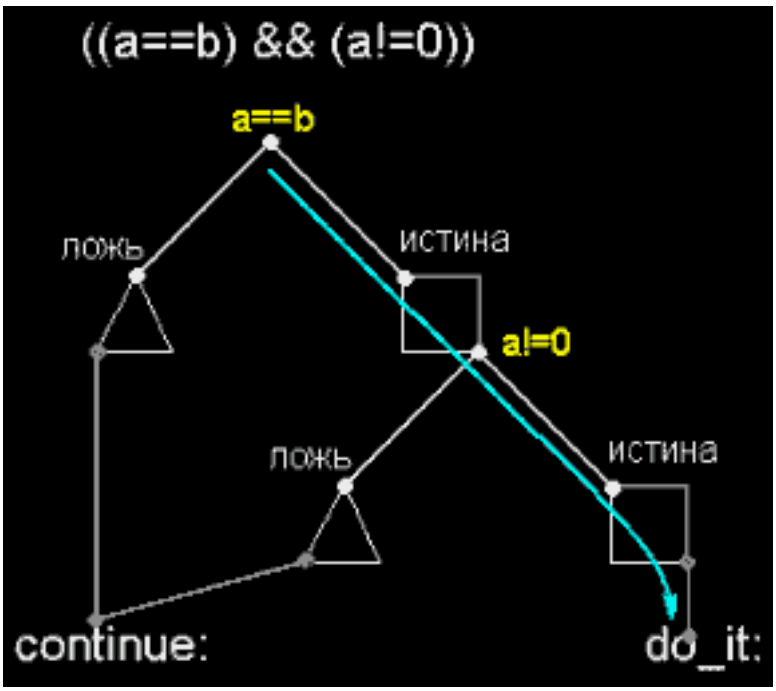


Рисунок 23. 0x016 Графическое представление операции AND в виде двоичного дерева. Обратите внимание - в пункт do_it можно попасть только одним путем!

Перейдем теперь к операции логического "OR". Рассмотрим конструкцию "((a==b) || (a!=0))". Если условие "(a==b)" истинно, то и все выражение считается истинным. Следовательно, правая ветка гнезда "(a==b)" связана с меткой "do_it". Если же условие же "(a==b)" ложно, то выполняется

проверка следующего условия. Значит, левая ветка гнезда "(a==b)" связана с гнездом "(a!=b)". Очевидно, если условие "(a!=b)" истинно, то истинно и все выражение "((a==b) || (a!=0))", напротив, если условие "(a!=b)" ложно, то ложно и все выражение, т.к. проверка условия "(a!=b)" выполняется только в том случае, если условие "(a==b)" ложно. Отсюда мы заключаем, что левая ветка гнезда "(a!=b)" связана с меткой "continue", а правая - с "do_it". (см. рис. 24). Обратите внимание - в пункт "do_it" можно попасть двумя различными путями! Вот так графически выглядит операция "OR".

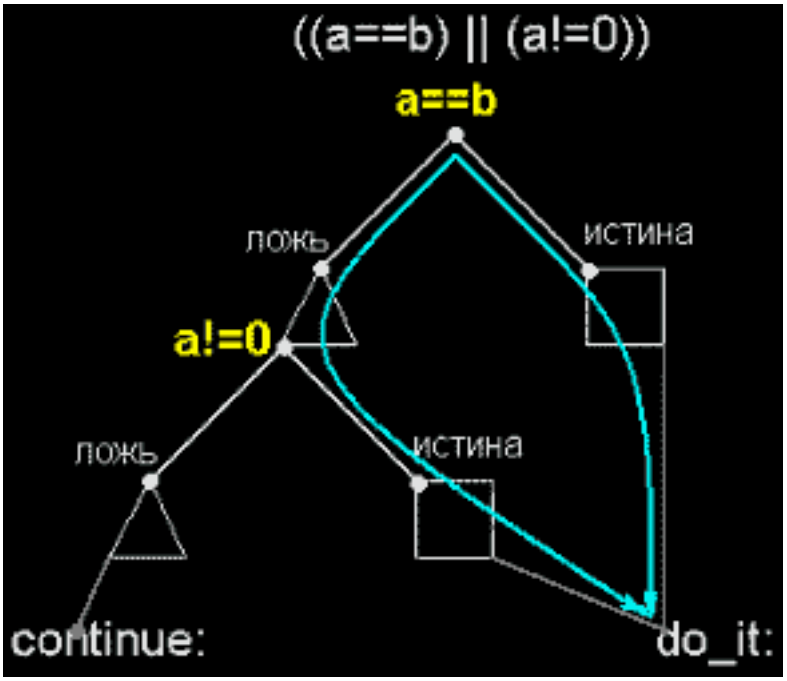


Рисунок 24. 0x017 Графическое представление операции OR в виде двоичного дерева. Обратите внимание - в пункт do_it можно попасть двумя различными путями!

До сих пор мы отображали логические операции на деревья, но ведь деревья создавались как раз для противоположной цели - преобразованию последовательности элементарных условий к интуитивно понятному представлению. Займемся этим? Пусть в тексте программы встретится следующий код:

```
IF a==b THEN check_null
IF a!=c THEN continue
check_null:
IF a==0 THEN continue
...// код условия
continue:
...// прочий код

Листинг 150
```

Извлекаем условие (a==b) и сажаем его в "гнездо", - смотрим: если оно ложно, то выполняется проверка (a!=c), значит, гнездо (a!=c) связано с левой веткой гнезда (a==b). Если же условие (a==b) истинно, то управление передается метке check_null, проверяющей истинность условия (a==0), следовательно, гнездо (a==0) связано с правой веткой гнезда (a==b). В свою очередь, если условие (a!=c) истинно, управление получает метка "continue", в противном случае - "check_null". Значит, гнездо (a!=0) связано одновременно и с правой веткой гнезда (a==b) и с левой веткой гнезда (a!=c).

Конечно, это проще рисовать, чем описывать! Если вы все правильно зарисовали, у вас должно получится дерево очень похожее на изображенное на рисунке 25.

Смотрите: к гнезду "(a==0)" можно попасть двумя путями - либо через гнездо (a==b), либо через цепочку двух гнезд (a==b) а (a!=c). Следовательно, эти гнезда связаны операцией OR. Записываем: "if ((a==b) || !(a!=c)E.)". Откуда взялся NOT? Так ведь гнездо (a==0) связано с левой веткой гнезда

(a!=c), т.е. проверяется ложность его истинности! (Кстати, "ложность истинности" - очень хорошо звучит). Избавляемся от NOT, инвертируя условие: "if ((a==b) || (a==c)E.)E". Далее - из гнезда (a==0) до пункта do_it можно добраться только одним путем, значит, оно связано операцией AND. Записываем: "if (((a==b) || (a==c)) && !(a==0))E". Теперь избавляемся от лишних скобок и операции NOT. В результате получается: "if ((a==b) || (a==c) && (a!=0)) {// Код условия}"

Не правда ли все просто? Причем вовсе необязательно строить деревья вручную, - при желании можно написать программу, берущую эту работу на себя.

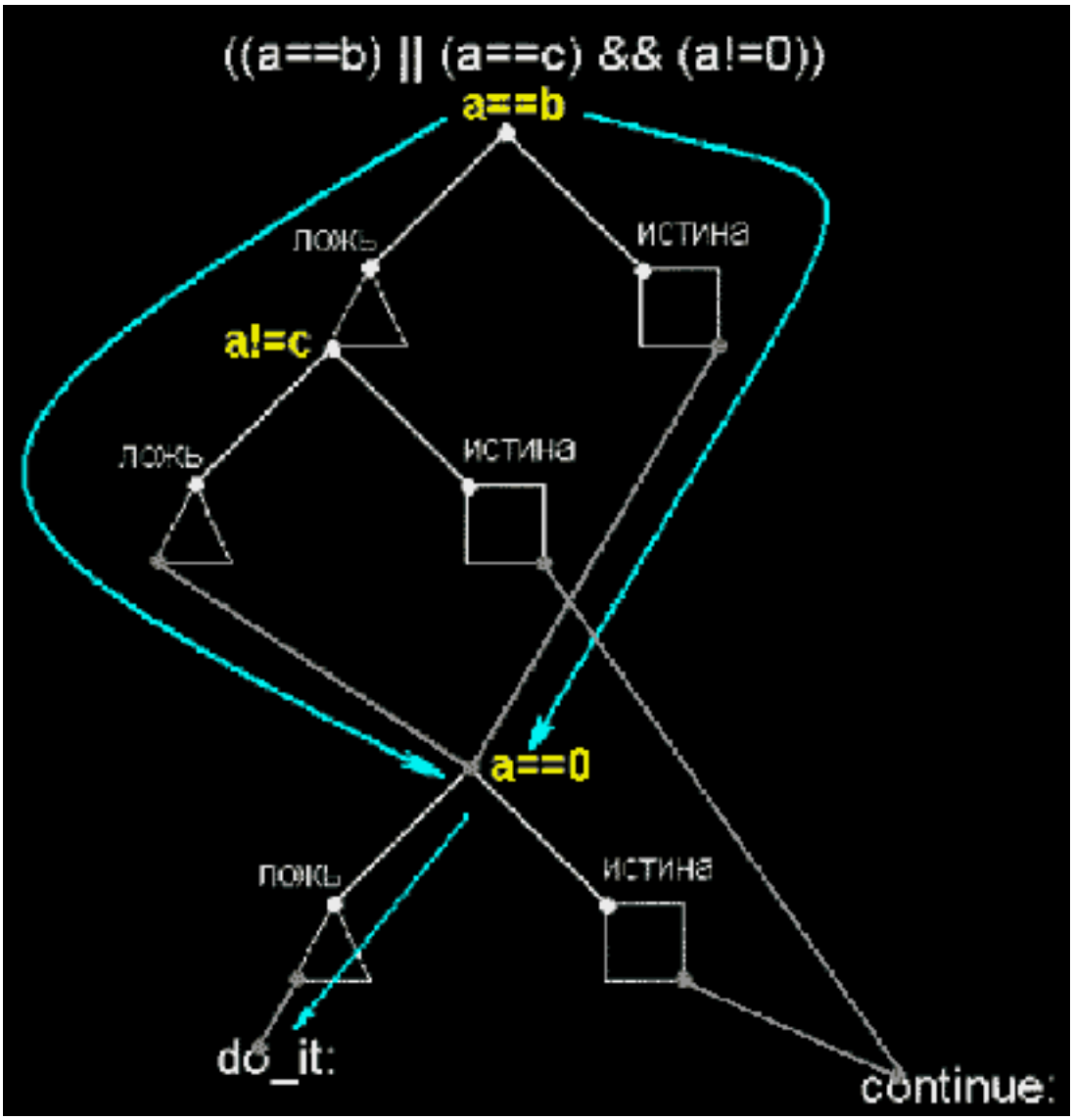


Рисунок 25. 0x018 Графическое представление сложного выражения

Исследование конкретных реализаций. Прежде чем приступать к отображению конструкции "IF (сложное условие) THEN оператор_1:оператор_2 ELSE оператор_a:оператор_b" на машинный язык, вспомним, что, во-первых, агрегат "IF - THEN - ELSE" можно выразить через "IF - THEN", во-вторых, "THEN оператор_1:оперратор_2" можно выразить через "THEN GOTO do_it", в-третьих, любое сложное условие можно свести к последовательности элементарных условий отношения. Таким образом, на низком уровне мы будем иметь дело лишь с конструкциями "IF (простое условие отношения) THEN GOTO do_it", а уже из них, как из кирпичиков, можно сложить что угодно.

Итак, условия отношения, или другими словами, результат операции сравнения двух чисел. В микропроцессорах Intel 80x86 сравнение целочисленных значений осуществляется командой CMP, а вещественных - одной из следующих инструкций сопроцессора: FCOM, FCOMP, FCOMPP,

FCOMI, FCOMIP, FUCOMI, FUCOMIP. Предполагается, что читатель уже знаком с языком ассемблера, поэтому не будем подробно останавливаться на этих инструкциях и рассмотрим их лишь вкратце.

::CMP. Команда CMP эквивалентна операции целочисленного вычитания SUB, за одним исключением - в отличие от SUB, CMP не изменяет операндов, а воздействует лишь на флаги основного процессора: флаг нуля, флаг переноса, флаг знака и флаг переполнения.

Флаг нуля устанавливается в единицу, если результат вычитания равен нулю, т.е. операнды равны друг другу.

Флаг переноса устанавливается в единицу, если в процессе вычитания произошел заем из старшего бита уменьшаемого операнда, т.е. уменьшаемое меньше вычитаемого.

Флаг знака равен старшему - знаковому - биту результата вычислений, т.е. результат вычислений - отрицательное число.

Флаг переполнения устанавливается в единицу, если в результате вычислений "залез" в старший бит, приводя к потере знака числа.

Для проверки состояния флагов существует множество команд условных переходов, выполняющихся в случае, если определенный флаг (набор флагов) установлен (сброшен). Инструкции, использующиеся для анализа результата сравнения целых чисел, перечислены в таблице 16.

В общем случае конструкция "IF (элементарное условие отношения) THEN do_it" транслируется в следующие команды процессора:

```
CMP A,B
Jxx do_it
continue:
```

Между инструкциями "CMP" и "Jxx" могут находиться и другие команды, не изменяющие флагов процессора, например "MOV", "LEA".

Условие	Тип	Zero flag	Carry Flag	Sing Flag	Инструкция 1	Инструкция 2	Инструкция 3		
a == b		1	?	?	JZ	JE			
a != b		0	?	?	JNZ	JNE			
a < b	беззнаковое	?	1	?	JC	JB	JNAE		
a < b	знаковое	?	?	!=OF	JL	JNGE			
a > b	беззнаковое	0	0	?	JA	JNBE			
a > b	знаковое	0	?	==OF	JG	JNLE			
a >=b	беззнаковое	?	0	?	JAE	JNB	JNC		
a >=b	знаковое	?	?	==OF	JGE	JNL			
a <= b	беззнаковое	(ZF == 1) \	\	(CF == 1)		?	JBE	JNA	
a <= b	знаковое	1	?		JLE	JNG			

Таблица 16. Соответствие операций отношения командам процессора

::сравнение вещественных чисел. Команды сравнения вещественных чисел FCOMxx (см. таблицу 18) в отличие от команд целочисленного сравнения воздействуют на регистры сопроцессора, а не основного процессора. На первый взгляд - логично, но весь камень преткновения в том, что

инструкций условного перехода, управляемых флагами сопроцессора, не существует! К тому же, флаги сопроцессора непосредственно недоступны, - чтобы прочитать их статус необходимо выгрузить регистр состояния сопроцессора SW в память или регистр общего назначения основного процессора.

Хуже всего - анализировать флаги вручную! Если при сравнении целых чисел можно и не задумываться: какими именно флагами управляется условный переход, достаточно написать, скажем: "CMP A,B; JGE do_it". ("Jump [if] Great [or] Equal" - прыжок, если A больше или равно B), то теперь этот номер не пройдет! Правда, можно схитрить и скопировать флаги сопроцессора в регистр флагов основного процессора, а затем использовать "родные" инструкции условного перехода из серии Jxx.

Конечно, непосредственно скопировать флаги из сопроцессора в основной процессор нельзя и эту операцию приходится осуществлять в два этапа. Сначала флаги FPU выгружать в память или регистр общего назначения, а уже оттуда заталкивать в регистр флагов CPU. Непосредственно модифицировать регистр флагов CPU умеет только одна команда - POPF. Остается только выяснить - каким флагам сопроцессора, какие флаги процессора соответствуют. И вот что удивительно - флаги 8й, 10й и 14й сопроцессора совпадают с 0ым, 2ым и 6ым флагами процессора - CF, PF и ZF соответственно (см. таблицу 17). То есть - старшей байт регистра флагов сопроцессора можно безо всяких преобразований затолкать в младший байт регистра флагов процессора и это будет работать, ноЕ при этом исказятся 1й, 3й и 5й биты флагов CPU, никак не используемые в текущих версиях процессора, но зарезервированные на будущее. Менять значение зарезервированных битов нельзя! Кто знает, вдруг завтра один из них будут отвечать за самоуничтожение процессора? Шутка, конечно, но в ней есть своя доля истины.

К счастью, никаких сложных манипуляций нам проделывать не придется - разработчики процессора предусмотрели специальную команду - SAHF, копирующую 8й, 10й, 12й, 14й и 15й бит регистра AX в 0й, 2й, 4й, 6й и 7й бит регистра флагов CPU соответственно. Сверяясь по таблице 17 мы видим, что 7й бит регистра флагов CPU содержит флаг знака, а соответствующий ему флаг FPU - признак занятости сопроцессора!

Отсюда следует, что для анализа результата сравнения вещественных чисел использовать знаковые условные переходы (JL, JG, JLE, JNL, JNLE, JGE, JNGE) нельзя! Они работают с флагами знака и переполнения, - естественно, если вместо флага знака им подсовывают флаг занятости сопроцессора, а флаг переполнения оставляют в "подвешенном" состоянии, условный переход будет срабатывать не так, как вам бы этого хотелось! Применяйте лишь беззнаковые инструкции перехода - JE, JB, JA и др. (см. таблицу 16)

Разумеется, это не означает, что сравнивать знаковые вещественные значения нельзя, - можно, еще как! Но для анализа результатов сравнения обязательно всегда использовать только беззнаковые условные переходы!

Регистр	**7 / 15**	**6 / 14**	**5 / 13**	**4 / 12**	**3 / 11**	**2 / 10**	**1 / 9**	**0 / 8**	
CPU, биты	7	6	5	4	3	2	1	0	
CPU, флаги	SF	ZF	--	AF	--	PC	--	CF	
FPU, биты	15	14	13	12	11	10	9	8	
FPU, флаги	Busy!	C3(**ZF**)	TOP	TOP	TOP	C2(**PF**)	C1	C0(**CF**)	**Таблица 17. Соответствие флагов CPU и FPU**

Таким образом, вещественная конструкция "IF (элементарное условие отношения) THEN do_it" транслируется в одну из двух следующих последовательностей инструкций процессора:

```
fld    [a]        fld    [a]
fcomp  [b]        fcomp  [b]
fnstsw ax         fnstsw ax
sahf              test   ah, bit_mask
jxx    do_it      jnz    do_it
```

Листинг 151

Первый вариант более нагляден, зато второй работает быстрее. Однако, такой код (из всех известных мне компиляторов) умеет генерировать один лишь Microsoft Visual C++. Borland C++ и хваленый WATCOM C испытывают неопределимую тягу к инструкции SAHF, чем вызывают небольшие тормоза, но чрезвычайно упрощают анализ кода, - ибо, встретив команду наподобие JNA, мы и спросонок скажем, что переход выполняется когда a <= b, а вот проверка битвой маски "TEST AH, 0x41/JNZ do_it" заставит нас крепко задуматься или машинально потянуться к справочнику за разъяснениями (см. таблицу 16)

Команды семейства FUCOMIxx в этом смысле гораздо удобнее в обращении, т.к. возвращают результат сравнения непосредственно в регистры основного процессора, но - увы - их "понимает" только Pentium Pro, а в более ранних микропроцессорах они отсутствуют. Поэтому, вряд ли читателю доведется встретиться с ними в реальных программах, так что не имеет никакого смысла останавливаться на этом вопросе. Во всяком случае, всегда можно обратится к странице 3-112 руководства "Instruction Set Reference", где эти команды подробно описаны.

Инструкция	**Назначение**	**Результат**
FCOM	Сравнивает вещественное значение, находящееся на вершине стека сопроцессора, с операндом, находящимся в памяти или стеке FPU	флаги FPU
FCOMP	То же самое, что и FCOM, но с выталкиванием вещественного значения с вершины стека	флаги FPU
FCOMPP	Сравнивает два вещественных значения, лежащих на вершине стека сопроцессора, затем выталкивает их из стека	флаги FPU
FCOMI	Сравнивает вещественное значение, находящееся на вершине стека сопроцессора с другим вещественным значением, находящимся в стеке FPU	флаги CPU
FCOMIP	Сравнивает вещественное значение, находящееся на вершине стека сопроцессора с другим вещественным значением, находящимся в стеке FPU, затем выталкивает верхнее значение из стека	флаги CPU
FUCOMI	Неупорядоченно сравнивает вещественное значение, находящееся на вершине стека сопроцессора с другим вещественным значением, находящимся в стеке FPU	флаги CPU
FUCOMIP	Неупорядоченно сравнивает вещественное значение, находящееся на вершине стека сопроцессора с другим вещественным значением, находящимся в стеке FPU, затем выталкивает верхнее значение из стека	флаги CPU

Таблица 18. Команды сравнения вещественных значений

Флаги FPU	Назначение	Назначение	Битовая маска
OE	Флаг переполнения	Overfull Flag	#0x0008
C0	Флаг переноса	Carry Flag	#0x0100
C1	---		#0x0200
C2	Флаг четности	Partite Flag	#0x0400
C3	Флаг нуля	Zero Flag	#0x4000

Таблица 19. Назначение и битовые маски флагов сопроцессора

Отношение	Состояние флагов FPU	SAHF	Битовая маска
a<b	C0 == 1	JB	#0x0100 == 1
a>b	C0 == 0 C3 == 0	JNBE	#0x4100 == 0
a==b	C3 == 1	JZ	#0x4000 == 1
a!=b	C3 == 0	JNZ	#0x4000 == 0
a>=b	C0 == 0	JNB	#0x0100 == 0
a<=b	C0 == 1 C3 === 1	JNA	#0x4100 == 1

Таблица 20. Состояние регистров флагов для различных операций отношения. 'a' - левый, а 'b' правый операнд команды сравнения вещественных значений

Компилятор	Алгоритм анализа флагов FPU
Borland C++	копирует флаги сопроцессора в регистр флагов основного процессора
Microsoft Visual C++	тест битовой маски
WATCOM C	копирует флаги сопроцессора в регистр флагов основного процессора
Free Pascal	копирует флаги сопроцессора в регистр флагов основного процессора

Таблица 21. "Характер" некоторых компиляторов

Условные команды булевой установки. Начиная с 80386 чипа, язык микропроцессоров Intel обогатился командой условной установки байта - SETxx, устанавливающей свой единственный операнд в единицу (булево TRUE), если условие "xx" равно и, соответственно, сбрасывающую его в нуль (булево FALSE), если условие "xx" - ложно.

Команда "SETxx" широко используются оптимизирующими компиляторами для устранения ветвлений, т.е. избавления от условных переходов, т.к. последние очищают конвейер процессора, чем серьезно снижают производительность программы.

Команда	Команда	Команда	Отношение	Тип	Условие		
SETA	SETNBE		a>b	беззнаковое	CF == 0 && ZF == 0		
SETG	SETNLE		a>b	знаковое	ZF == 0 && SF == OF		
SETAE	SETNC	SETNB	a>=b	беззнаковое	CF == 0		
SETGE	SETNL		a>=b	знаковое	SF == OF		

Команда	Команда	Команда	Отношение	Тип	Условие		
SETB	SETC	SETNAE	a<b	беззнаковое	CF == 1		
SETL	SETNGE		a<b	знаковое	SF != OF		
SETBE	SETNA		a<=b	беззнаковое	CF == 1 \	\	ZF == 1
SETLE	SETNG		a<=b	знаковое	ZF == 1 \	\	SF != OF
SETE	SETZ		a==b	---	ZF == 1		
SETNE	SETNZ		a!=0	---	ZF == 0		

Таблица 22. Условные команды булевой установки

Прочие условные команды. Микропроцессоры серии 80x86 поддерживают множество условных команд, в общем случае не отображающихся на операции отношения, а потому и редко использующиеся компиляторами (можно даже сказать - вообще не использующиеся), но зато часто встречающиеся в ассемблерных вставках. Словом, они заслуживают хотя бы беглого упоминания.

::Команды условного перехода. Помимо описанных в таблице 16, существует еще восемь других условных переходов - JCXZ, JECXZ, JO, JNO, JP (он же JPE), JNP (он же JPO), JS и JNS. Из них только JCXZ и JECXZ имеют непосредственное отношение к операциям сравнения. Оптимизирующие компиляторы могут заменять конструкцию "CMP [...]CX, 0\JZ do_it" на более короткий эквивалент "J[...]CX do_it", однако, чаще всего они (в силу ограниченности интеллекта и лени своих разработчиков) этого не делают.

Условные переходы JO и JNS используются в основном в математических библиотеках для обработки чисел большой разрядности (например, 1024 битых целых).

Условные переходы JS и JNS помимо основного своего предназначения часто используются для быстрой проверки значения старшего бита.

Условные переходы JP и JNP вообще практически не используются, ну разве что в экзотичных ассемблерных вставках.

Команда	Команда	Переход, если…	Флаги
JCXZ		регистр CX равен нулю	CX == 0
JECXZ		регистр ECX равен нулю	ECX == 0
JO		переполнение	OF == 1
JNO		нет переполнения	OF == 0
JP	JPE	число бит младшего байта результата четно	PF == 1
JNP	JPO	число бит младшего байта результата нечетно	PF == 0
JS		знаковый бит установлен	SF == 1
JNS		знаковый бит сброшен	SF == 0

Таблица 23. Вспомогательные условные переходы

::Команды условной пересылки. Старшие процессоры семейства Pentium (Pentium Pro, Pentium II, CLERION) поддерживают команду условной пересылки CMOVxx, пересылающей значение из источника в приемник, если условие xx - истинно. Это позволяет писать намного более эффективный код, не содержащий ветвлений и укладывающийся в меньшее число инструкций.

Рассмотрим конструкцию "IF a<b THEN a=b". Сравните: как она транслируется с использованием условных переходов (1) и команды условной пересылки (2).

```
CMP A,B    CMP A, B
JAE continue:   CMOVB A, B
MOV A,B
continue:
1)    2)

Листинг 152
```

К сожалению, ни один из известных мне компиляторов на момент написания этих строк, никогда не использовал CMOVxx при генерации кода, однако, выигрыш от нее настолько очевиден, что появления усовершенствованных оптимизирующих компиляторов следует ожидать в самом ближайшем будущем. Вот почему эта команда включена в настоящий обзор. В таблице 24 дана ее краткое, но вполне достаточное для дизассемблирования программ, описание. За более подробными разъяснениями обращайтесь к странице 3-59 справочного руководства "Instruction Set Reference" от Intel.

Команда	Команда	Команда	Отношение	Тип	Условие		
CMOVA	CMOVNBE		a>b	беззнаковое	CF == 0 && ZF == 0		
CMOVG	CMOVNLE		a>b	знаковое	ZF == 0 && SF == OF		
CMOVAE	CMOVNC	CMOVNB	a>=b	беззнаковое	CF == 0		
CMOVGE	CMOVNL		a>=b	знаковое	SF == OF		
CMOVB	CMOVC	CMOVNAE	a<b	беззнаковое	CF == 1		
CMOVL	CMOVNGE		a<b	знаковое	SF != OF		
CMOVBE	CMOVNA		a<=b	беззнаковое	CF == 1 \	\	ZF == 1
CMOVLE	CMOVNG		a<=b	знаковое	ZF == 1 \	\	SF != OF
CMOVE	CMOVZ		a==b	---	ZF == 1		
CMOVNE	CMOVNZ		a!=0	---	ZF == 0		

Таблица 24. Основные команды условной пересылки

Булевы сравнения. Логической лжи (FALSE) соответствует значение ноль, а логической истине (TRUE) - любое ненулевое значение. Таким образом, булевы отношения сводятся к операции сравнения значения переменной с нулем. Конструкция "IF (a) THEN do_it" транслируется в "IF (a!=0) THEN do_it".

Практически все компиляторы заменяют инструкцию "CMP A, 0" более короткой командой "TEST A,A" или "OR A,A". Во всех случаях, если A==0, устанавливается флаг нуля и, соответственно, наоборот.

Поэтому, встретив к дизассемблером тексте конструкцию a la "TEST EAX, EAX\ JZ do_it" можно с уверенностью утверждать, что мы имеем дело с булевым сравнением.

Идентификация условного оператора "(условие)?do_it:continue" Конструкция "a=(условие)?do_it:continue" языка Си в общем случае транслируется так: "IF (условие) THEN a=do_it ELSE a=continue", однако результат компиляции обоих конструкций вопреки распространенному мнению, не всегда идентичен.

В силу ряда обстоятельств оператор "?" значительно легче поддается оптимизации, чем ветвление "IF - THEN - ELSE". Покажем это на следующем примере:

```
main()
{
int a;  // Перерменная специально не инициализирована
```

```
int b;  // чтобы компилятор не заменил ее константой

a=(a>0)?1:-1; // Условный оператор

if (b>0)  // Ветвление
b=1;
else
b=-1;

return a+b;
}
```

Листинг 153

Если пропустить эту программу сквозь компилятор Microsoft Visual C++, на выходе мы получим такой код:

```
push ebp
mov ebp, esp
; Открываем кадр стека

sub esp, 8
; Резервируем место для локальных переменных

; // Условный оператор ?
; Начало условного оператора ?
xor eax, eax
; Обнуляем EAX

cmp [ebp+var_a], 0
; Сравниваем переменную a с нулем

setle al
; Поместить в al значение 0x1, если var_a <= 0
; Соответственно, поместить в al значение 0, если var_a>0

dec eax
; Уменьшить EAX на единицу
; Теперь,  если var_a > 0, то EAX := -1
;   если var_a <=0, то EAX := 0

and eax, 2
; Сбросить все биты, кроме второго слева, считая от одного
; Теперь,  если var_a > 0, то EAX := 2
;   если var_a <=0, то EAX := 0

add eax, 0FFFFFFFFh
; Отнять от EAX 0x1
; Теперь,  если var_a > 0, то EAX := 1
;   если var_a <=0, то EAX := -1
mov [ebp+var_a], eax
; Записать результат в переменную var_a
; Конец оператора ?
; Обратите внимание: для трансляции условного оператора не потребовалось ни
; одного условного перехода, - компилятор сумел обойтись без ветвлений!


; // Ветвление
; Начало ветвления IF - THEN - ELSE
cmp [ebp+var_b], 0
; Сравнение переменной var_b с нулем

jle short else
; Переход, если var_b <= 0

; Ветка "var_b > 0"
mov [ebp+var_b], 1
; Записываем в переменную var_b значение 1

jmp short continue
; Переход к метке continue
```

```
; Ветка "var_b > 0"
else:     ; CODE XREF: _main+1D j
mov [ebp+var_b], 0FFFFFFFFh
; Записываем в переменную var_b значение -1

continue:     ; CODE XREF: _main+26 j
; Конец ветвления IF-THEN-ELSE
; Обратите внимание - представление ветвления "IF-THEN-ELSE" намного компактнее
; условного оператора "?", однако, содержит в себе условные переходы, ощутимо
; снижающие быстродействие программы

mov eax, [ebp+var_a]
; Загружаем в EAX значение переменной var_a

add eax, [ebp+var_b]
; Складываем значение переменной var_a со значением переменной var_b
; и помещаем результат в EAX

mov esp, ebp
pop ebp
; Закрываем кадр стека
retn

Листинг 154
```

Таким образом, мы видим, что нельзя апории утверждать, будто бы результат трансляции условного оператора "?" всегда эквивалентен результату трансляции конструкции "IF-THEN-ELSE". Однако тот же Microsoft Visual C++ в режиме агрессивной оптимизации в обоих случаях генерирует идентичный код. Смотрите:

```
_main  proc near
push ecx
; Резервируем место для локальных переменных a и b
; Поскольку, они никогда не используются вместе, а только поочередно,
; компилятор помещает их в одну ячейку памяти

mov edx, [esp+0]     ; команда N1 оператора ?
; Загрузка в EDX значения переменной a

xor eax, eax     ; команда N2 оператора ?
; Обнуляем EAX
; Поскольку, команда setle al изменяет содержимое одного лишь al, и не трогает
; остальную часть регистра, нам приходится очищать его самостоятельно

test edx, edx     ; команда N3 оператора ?
; Проверка переменной a на равенство нулю

mov edx, [esp+0]     ; команда N1 ветвления IF
; Загрузка в EDX значения переменной b

setle al      ; команда N4 оператора ?
; Поместить в al значение 0x1, если a <= 0
; Соответственно, поместить в al значение 0, если a>0

dec eax      ; команда N5 оператора ?
; Уменьшить EAX на единицу
; Теперь,  если a > 0, то EAX := -1
;   если a <=0, то EAX := 0

xor ecx, ecx     ; команда N2 ветвления IF
; Обнулить ECX

and eax, 2      ; команда N6 оператора ?
; Сбросить все биты, кроме второго слева, считая от одного
; Теперь,  если a > 0, то EAX := 2
;   если a <=0, то EAX := 0

dec eax      ; команда N7 оператора ?
; Уменьшить EAX на единицу
; Теперь,  если a > 0, то EAX := 1
```

```
;   если a <=0, то EAX := -1


test edx, edx     ; команда N3 ветвления IF
; Проверка переменной b на равенство нулю

setle cl      ; команда N4 ветвления IF
; Поместить в cl значение 0x1, если b <= 0
; Соответственно, поместить в cl значение 0, если b>0

dec ecx      ; команда N5 ветвления IF
; Уменьшить ECX на единицу
; Теперь,  если b > 0, то ECX := -1
;   если b <=0, то ECX := 0

and ecx, 2      ; команда N6 ветвления IF
; Сбросить все биты, кроме второго слева, считая от одного
; Теперь,  если b > 0, то ECX := 2
;   если b <=0, то ECX := 0

dec ecx      ; команда N7 ветвления IF
; Уменьшить ECX на единицу
; Теперь,  если b > 0, то ECX := -1
;   если b <=0, то ECX := 0

add eax, ecx
; Сложить переменную a с переменной b

pop ecx
; Закрыть кадр стека
retn
_main  endp
```

Листинг 155

Компилятор некоторым образом перемешал команды, относящиеся к условному оператору "?", с командами ветвления "IF-THEN-ELSE" (это было сделано для лучшего спаривания инструкций), однако, если их сравнить, то выяснится - реализации обеих конструкций абсолютно идентичны друг другу!

Однако с точки зрения языка условный оператор "?" выгодно отличается от ветвления тем, что может непосредственно использоваться в выражениях, например:

```
main()
{
int a;
printf("Hello, %s\n", (a>0)?"Sailor":"World!");
}
```

Листинг 156

Попробуйте так же компактно реализовать это с помощью ветвлений! Но на самом деле, это удобство лишь внешнее, а компилятор транслирует приведенный пример так:

```
main()
{
int a;
char *p;
static char s0[]="Sailor";
static char s1[]="World";
if (a>0) p=s0; else p=s1;

printf("Hello, %s\n", p);
}
```

Листинг 157

Откомпилируйте оба листинга и дизассемблируйте полученные файлы, - они должны быть идентичны. Таким образом, при декомпиляции Си/Си++ программ в общем случае невозможно

сказать использовалось ли в них ветвление или условный оператор, однако, все же есть некоторые зацепки, помогающие восстановить истинный вид исходного текста в некоторых частных случаях.

Например, маловероятно, чтобы программист строил свой листинг, как показано в последнем примере. Зачем вводить статические переменные и сложным образом манипулировать с указателем, когда проще поступить использовать условный оператор вместо ветвления?

Таким образом, если условный оператор гладко ложиться в декомпилируемую программу, а ветвление не лезет в нее никаким боком, то, очевидно, что в исходном тексте использовался именно условный оператор, а не ветвление.

Идентификация типов. Условные команды - ключ к идентификации типов. Поскольку, анализ результата сравнения знаковых и беззнаковых переменных осуществляется различными группами инструкций, можно уверенно и однозначно отличить signed int от unsigned int.

Вообще же, идентификация типов - тема отдельного разговора, поэтому не будет отклоняться в сторону, а рассмотрим ее чуточку позже в одноименной главе.

16 разрядный режим. Одна из неприятных особенностей 16-разрядного режима - ограниченная "дальнобойность" команд условного перехода. Разработчики микропроцессора в стремлении добиться высокой компактности кода, отвели на целевой адрес всего один байт, ограничив тем самым длину прыжка интервалом в 255 байт. Это, так называемый, короткий (short) переход, адресуемый относительным знаковым смещением, отсчитываемым от начала следующий за инструкцией перехода командой (см. рис 26). Такая схема адресации ограничивает длину прыжка "вперед" (т.е. "вниз") всего 128 байтами, а "назад" (т.е. "вверх") и того меньше - 127! (Прыжок вперед короче потому, что ему требуется "пересечь" и саму команду перехода). Этих ограничений лишен ближний (near) безусловный переход, адресуемый двумя байтами и действующий в пределах всего сегмента.

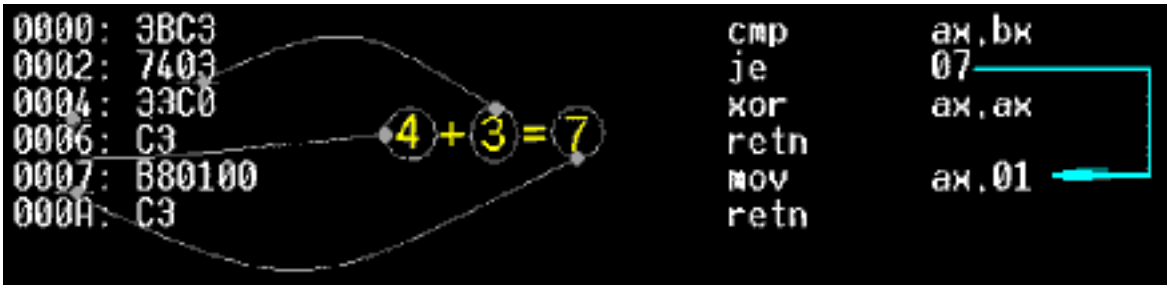


Рисунок 26. 0x019 Внутреннее представление короткого (short) перехода

Короткие переходы усложняют трансляцию ветвлений - ведь не всякий целевой адрес находится в пределах 128 байт! Существует множество путей обойти это ограничение. Наиболее популярен следующий примем: если транслятор видит, что целевой адрес выходит за пределы досягаемости условного перехода, он инвертирует условие срабатывания и совершает короткий (short) переход на метку continue, а на do_it передает управление ближним (near) переходом, действующим в пределах одного сегмента (см. рис. 27)

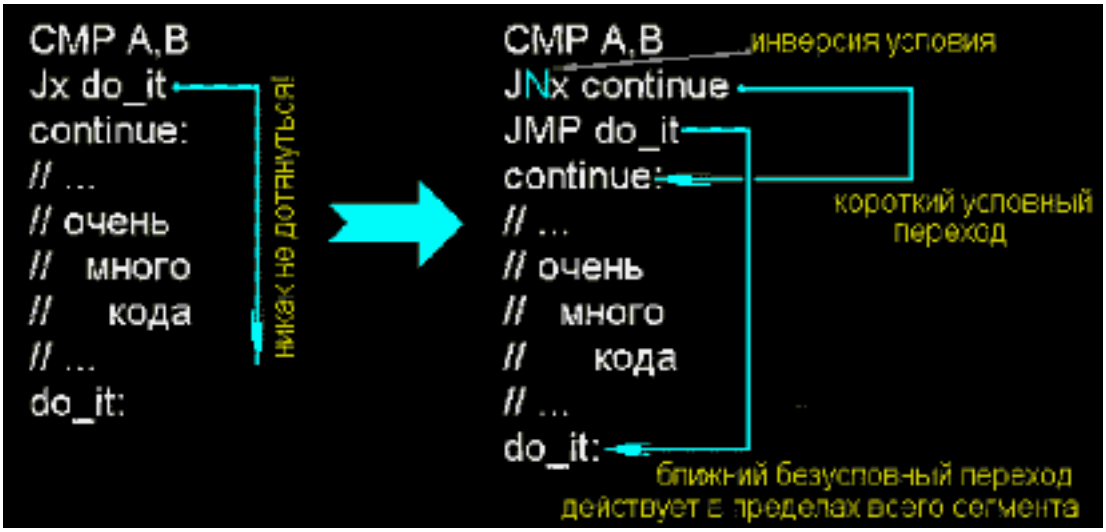


Рисунок 27. 0x01A Трансляция коротких переходов

Аналогичным образом можно выкрутиться и в тех ситуациях, когда целевой адрес расположен совсем в другом сегменте - достаточно лишь заменить ближний безусловный переход на дальний. Вот, собственно, и все.

К великому счастью разработчиков компиляторов и не меньшей радости хакеров, дизассемблирующих программы, в 32-разрядном режиме условный переход "бьет" в пределах всего четырех гигабайтного адресного пространства и все эти проблемы исчезают, как бородавки после сеанса Кашпировского.

Листинги примеров. А теперь для лучшего уяснения материала, рассмотренного в этой главе, давайте рассмотрим несколько "живых" примеров, откомпилированных различными компиляторами. Начнем с исследования элементарных целочисленных отношений:

```
#include <stdio.h>

main()
{
int a; int b;
if (a<b) printf("a<b");
if (a>b) printf("a>b");
if (a==b) printf("a==b");
if (a!=b) printf("a!=b");
if (a>=b) printf("a>=b");
if (a<=b) printf("a<=b");
}
```

Листинг 158

Результат компиляции этого примера компилятором Microsoft Visual C+ должен выглядеть так:

```
main  proc near  ; CODE XREF: start+AF p

var_b  = dword ptr -8
var_a  = dword ptr -4

push ebp
mov ebp, esp
; Открываем кадр стека

sub esp, 8
; Резервируем память для локальных переменных var_a и var_b

mov eax, [ebp+var_a]
; Загружаем в EAX значение переменной var_a

cmp eax, [ebp+var_b]
```

```
; Сравниваем значение переменной var_a со значением переменной var_b

jge short loc_40101B
; Если var_a >= var_b то переход на continue иначе - печать строки
; Обратите внимание, что оригинальный код выглядел так:
; if (a<b) printf("a<b");
; Т.е. условие отношения было инвентировано компилятором!
; Знаковая операция JGE говорит о том, что и сравнивыемые переменные
; var_a и var_b - так же знаковые

; // BETKA DO_IT
push offset aAB_4 ; "a<b"
call _printf
add esp, 4
; Печать строки "a<b"

; // BETKA CONTINUE
loc_40101B:    ; CODE XREF: main+C j
mov ecx, [ebp+var_a]
; Загружаем в ECX значение переменной var_a

cmp ecx, [ebp+var_b]
; Сравниваем значение переменной var_a с переменной var_b

jle short loc_401030
; Переход, если var_a <= var_b, иначе - печать строки
; Следовательно, строка печатается, когда !(var_a <= var_b), или
; var_a > var_b. Тогда исходный код программы должен выглядеть так:
; if (a>b) printf("a>b");

push offset aAB_3 ; "a>b"
call _printf
add esp, 4
;

loc_401030:    ; CODE XREF: main+21 j
mov edx, [ebp+var_a]
; Загружаем в EDX значение переменной var_a

cmp edx, [ebp+var_b]
; Сравниваем значение переменной var_a с переменной var_b

jnz short loc_401045
; Переход если var_a!=var_b, иначе печать строки
; Следовательно, оригинальный код программы выглядел так:
; if (a==b) printf("a==b");

push offset aAB ; "a==b"
call _printf
add esp, 4

loc_401045:    ; CODE XREF: main+36 j
mov eax, [ebp+var_a]
; Загружаем в EAX значение переменной var_a

cmp eax, [ebp+var_b]
; Сравниваем значение переменной var_a со значением переменной var_b

jz short loc_40105A
; Переход, если var_a==var_b, иначе - печать строки.
; Следовательно, оригинальный код программы выглядел так:
; if (a!==b) printf("a!=b");

push offset aAB_0 ; "a!=b"
call _printf
add esp, 4

loc_40105A:    ; CODE XREF: main+4B j
mov ecx, [ebp+var_a]
; Загружаем в ECX значение переменной var_a
cmp ecx, [ebp+var_b]
```

```
; Сравниваем значение переменной var_a с переменной var_b

jl short loc_40106F
; Переход, если var_a < var_b, иначе - печать строки
; Следовательно, оригинальный код программы выглядел так:
; if (a>=b) printf("a>=b");

push offset aAB_1 ; "a>=b"
call _printf
add esp, 4

loc_40106F:    ; CODE XREF: main+60 j
mov edx, [ebp+var_a]
; Загружаем в EDX значение переменной var_a

cmp edx, [ebp+var_b]
; Сравниваем значение переменной var_a с переменной var_b

jg short loc_401084
; Переход если var_a>var_b, иначе печать строки
; Следовательно, оригинальный код программы выглядел так:
; if (a<=b) printf("a<=b");

push offset aAB_2 ; "a<=b"
call _printf
add esp, 4

loc_401084:    ; CODE XREF: main+75 j
mov esp, ebp
pop ebp
; Закрываем кадр стека

retn
main  endp

Листинг 159
```

А теперь сравним этот, 32-разрядный код, с 16-разрядным кодом, сгенерированном компилятором Microsoft C++ 7.0 (ниже, для экономии места приведен лишь фрагмент):

```
mov ax, [bp+var_a]
; Загрузить в AX значение переменной var_a

cmp [bp+var_b], ax
; Сравнить значение переменной var_a со значением переменной var_b

jl loc_10046
; Переход на код печати строки, если var_a < var_b

jmp loc_10050
; Безусловный переход на continue
; Смотрите! Компилятор, не будучи уверен, что "дальнобойности" короткого
; условного перехода хватит для достижения метки continue, вместо этого
; прыгнул на метку do_it, расположенную неподалеку - в гарантированной
; досягаемости, а передачу управления на continue взял на себя
; безусловный переход
; Таким образом, инверсия истинности условия сравнения имело место дважды
; первый раз при трансляции условия отношения, второй раз - при генерации
; машинного кода. A NOT на NOT можно сократить!
; Следовательно, оригинальный код выглядел так:
; if (a<b) printf("a<b");

loc_10046:    ; CODE XREF: _main+11 j
mov ax, offset aAB ; "a<b"
push ax
call _printf
add sp, 2

loc_10050:    ; CODE XREF: _main+13 j
; // прочий код
Листинг 160
```

А теперь заменим тип сравниваемых переменных с int на float и посмотрим, как это повлияет на сгенерированный код. Результат компиляции Microsoft Visual C++ должен выглядеть так (ниже приведен лишь фрагмент):

```
fld [ebp+var_a]
; Загрузка значения вещественной переменной var_a на вершину стека сопроцессора

fcomp [ebp+var_b]
; Сравнение значение переменной var_a с переменной var_b
; с сохранением результата сравнения во флагах сопроцессора

fnstsw ax
; Скопировать регистр флагов сопроцессора в регистр AX

test ah, 1
; Нулевой бит регистра AH установлен?
; Соответственно: восьмой бит регистра флагов сопроцессора установлен?
; А что у нас храниться в восьмом бите?
; Ага, восьмой бит содержит флаг переноса.

jz short loc_20
; Переход, если флаг переноса сброшен, т.е. это равносильно конструкции jnc
; при сравнении целочисленных значений. Смотрим по таблице 16 - синоним jnc
; команда jnb.
; Следовательно, оригинальный код выглядел так:
; if (a<b) printf("a<b");

push offset $SG339 ; "a<b"
call _printf
add esp, 4
loc_20:     ; CODE XREF: _main+11 j
```

Листинг 161

Гораздо нагляднее код, сгенерированный компилятором Borland C++ или WATCOM C. Смотрите:

```
fld [ebp+var_a]
; Загрузка значения вещественной переменной var_a на вершину стека сопроцессора

fcomp [ebp+var_b]
; Сравнение значение переменной var_a с переменной var_b
; с сохранением результата сравнения во флагах сопроцессора

fnstsw ax
; Скопировать регистр флагов сопроцессора в регистр AX

sahf
; Скопировать соответствующие биты регистра AH во флаги основного процессора

jnb short loc_1003C
; Переход, если !(a<b), иначе печать строки printf("a<b")
; Теперь, не копаясь ни в каких справочных таблицах, можно восстановить
; оригинальный код:
; if (a<b) printf("a<b");

push offset unk_100B0 ; format
call _printf
pop ecx
loc_1003C:    ; CODE XREF: _main+F j
```

Листинг 162

Теперь, "насобачившись" на идентификации элементарных условий, перейдем к вещам по настоящему сложным. Рассмотрим следующий пример:

```
#include <stdio.h>

main()
{
```

```
unsigned int a; unsigned int b; int c; int d;
if (d) printf("TRUE"); else if (((a>b) && (a!=0)) || ((a==c) && (c!=0))) printf("OK\n");
if (c==d) printf("+++\n");
}
```

Листинг 163

Результат его компиляции должен выглядеть приблизительно так:

```
_main  proc near

var_d  = dword ptr -10h
var_C  = dword ptr -0Ch
var_b  = dword ptr -8
var_a  = dword ptr -4

push ebp
mov ebp, esp
; Открытие кадра стека

sub esp, 10h
; Резервирование места для локальный переменных

cmp [ebp+var_d], 0
; Сравнение значение переменной var_d с нулем

jz short loc_1B
; Если переменная var_d равна нулю, переход к метке loc_1B, иначе
; печать строки TRUE. Схематически это можно изобразить так:
;                       var_d == 0
;                      /          \
;                 loc_1B          printf("TRUE");

push offset $SG341 ; "TRUE"
call _printf
add esp, 4
jmp short loc_44
; "Ага", говорим мы голосом Пяточка, искушающего Кенгу!
; Вносим этот условный переход в наше дерево
;
;                       var_d == 0
;                      /          \
;                 loc_1B          printf("TRUE");
;                                     |
;                                  loc_44

loc_1B:     ; CODE XREF: _main+A j
mov eax, [ebp+var_a]
; Загружаем в EAX значение переменной var_a

cmp eax, [ebp+var_b]
; Сравниваем переменную var_a с переменной var_b

jbe short loc_29
; Если var_a меньше или равна переменной var_b, то переход на loc_29
; Прививаем новое гнездо к нашему дереву, попутно обращая внимание не то, что
; var_a и var_b - беззнаковые переменные!
;
;                       var_d == 0
;                      /          \
;                 loc_1B          printf("TRUE");
;                    |                 |
;             var_a  <= var_b        loc_44
;            /             \
;         continue          loc_29

cmp [ebp+var_a], 0
; Сравниваем значение переменной var_a с нулем

jnz short loc_37
; Переход на loc_37, если var_a не равна нулю
```

```
;
;                       var_d == 0
;                      /           \
;                  loc_1B          printf("TRUE");
;                     |                 |
;              var_a  <= var_b        loc_44
;                    /             \
;          var_a !=0           loc_29
;         /         \
;    continue       loc_37

loc_29:     ; CODE XREF: _main+21 j
; Смотрите - в нашем дереве уже есть метка loc_29! Корректируем его!
;
;                       var_d == 0
;                      /           \
;                  loc_1B          printf("TRUE");
;                     |                 |
;              var_a  <= var_b        loc_44
;                    /             \
;          var_a !=0           loc_29
;          /         \          |
;         |          |          |
;          \       loc_37       |
;           \                   |
;            \------------------+

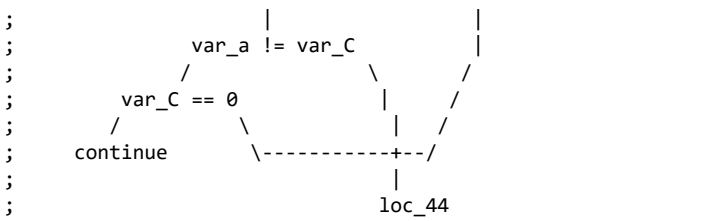
mov ecx, [ebp+var_a]
; Загружаем в ECX значение переменной var_a

cmp ecx, [ebp+var_C]
; Сравниваем значение переменной var_a с переменной var_C

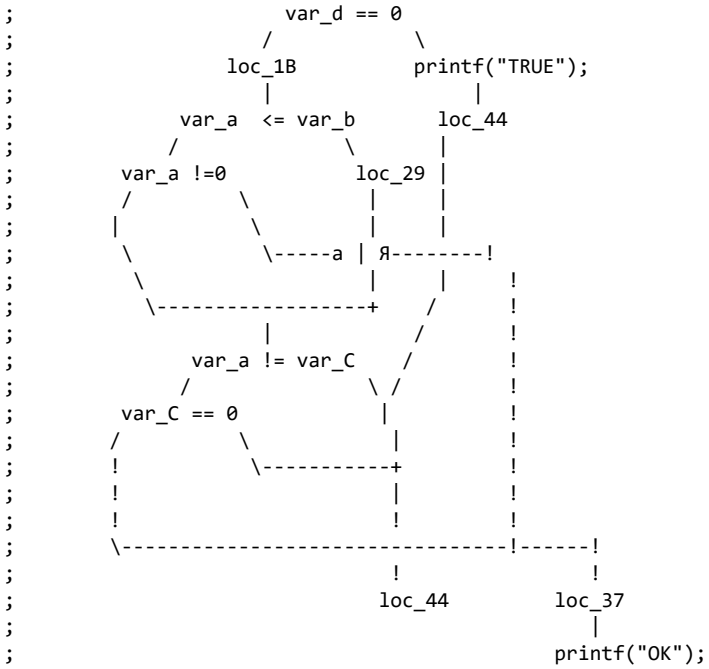
jnz short loc_44
; переход, если var_a != var_C
;
;                       var_d == 0
;                      /           \
;                  loc_1B          printf("TRUE");
;                     |                 |
;              var_a  <= var_b        loc_44
;                    /             \
;          var_a !=0           loc_29
;          /         \          |
;         |          |          |
;          \       loc_37       |
;           \                   |
;            \------------------+
;                     |
;                var_a != var_C
;               /              \
;           continue            loc_44

cmp [ebp+var_C], 0
; Сравнение значения переменной var_C с нулем

jz short loc_44
; Переход на loc_44 если var_C == 0
;
;                       var_d == 0
;                      /           \
;                  loc_1B          printf("TRUE");
;                     |                 |
;              var_a  <= var_b        loc_44
;                    /             \     |
;          var_a !=0           loc_29    |
;          /         \          |        |
;         |          |          |        |
;          \       loc_37       |        |
;           \                   |        |
;            \------------------+        |
```

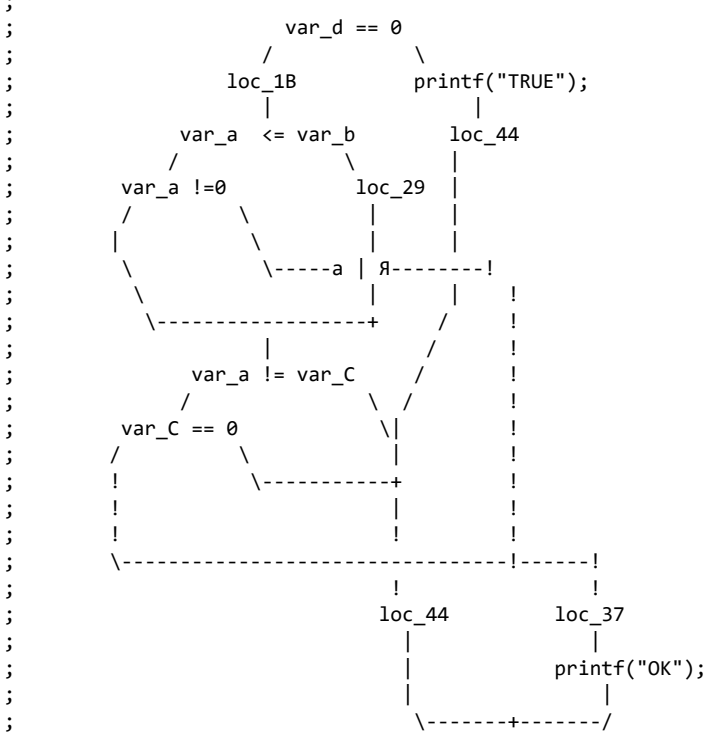


```
loc_37:     ; CODE XREF: _main+27 j
; Смотрим - метка loc_37 уже есть в дереве! Прививаем!
;
```



```
push offset $SG346 ; "OK\n"
call _printf
add esp, 4

loc_44:     ; CODE XREF: _main+19 j _main+2F j ...
; Смотрите - ветки loc_44 и loc_37 смыкаются!
;
```



```
;                                       |
;                                       |

mov edx, [ebp+var_C]
; Загружаем в EDX значение переменной var_C

cmp edx, [ebp+var_d]
; Сравниваем значение var_C со значением переменной var_D

jnz short loc_59
; Переход, если var_C != var_D

push offset $SG348 ; "+++\n"
call _printf
add esp, 4
;                        var_d == 0
;                       /           \
;                   loc_1B          printf("TRUE");
;                     |                 |
;               var_a  <= var_b       loc_44
;                    /              \       |
;          var_a !=0           loc_29  |
;          /         \          |      |
;         |           \         |      |
;          \           \-----a | Я--------!
;           \                  |      |    !
;            \------------------+     /     !
;                     |              /      !
;                var_a != var_C     /       !
;                    /              \  /     !
;          var_C == 0               |        !
;         /           \             |        !
;         !            \-----------+        !
;         !                        |        !
;         !                        !        !
;         \--------------------------------!------!
;                                  !              !
;                                loc_44         loc_37
;                                  |              |
;                                  |            printf("OK");
;                                  |              |
;                                   \-------+-------/
;                                           |
;                                           |
;                                       var_C != var_D
;                                      /              \
;                              printf("+++")           !
;                                                     конец


loc_59:     ; CODE XREF: _main+4A j
  mov esp, ebp
  pop ebp
  retn
_main  endp

Листинг 164
```

В итоге вырастает огромное разлапистое дерево, в котором на первый взгляд просто невозможно разобраться. Но, как говориться, глаза страшатся, а руки делают. Первым делом, оптимизируем дерево: избавимся от "перекрученных" ветвей, инвертировав условие в гнезде, и выкинем все метки - теперь, когда скелет дерева построен, они уже не нужны. Если все сделать правильно, дерево должен выглядеть так:

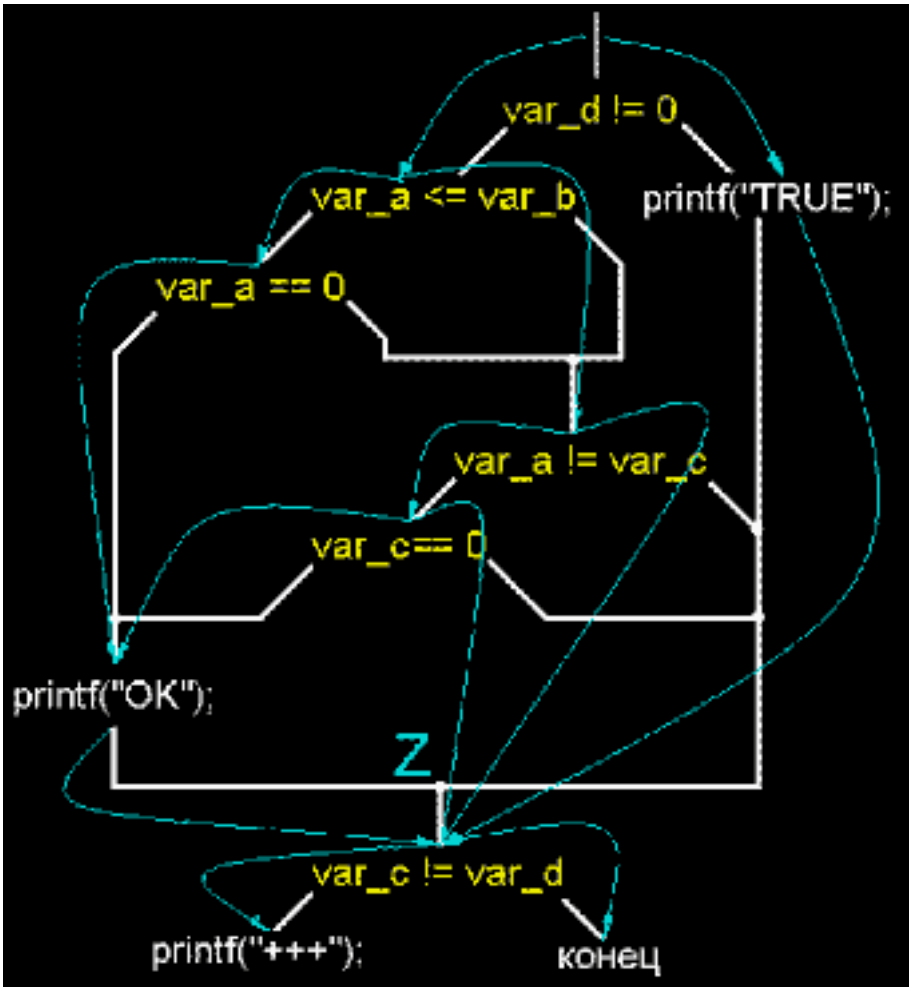


Рисунок 28. 0x01B Логическое древо

Сразу же бросается в глаза, что все пути проходят точку "Z", сплетающую все ветви воедино. Это значит, что мы имеем дело с двумя самостоятельными деревьями, представленными собственными конструкциями "IF". Замечательно! Такой поворот событий весьма упрощает анализ - раз деревья независимые, то и анализироваться они могут независимо! Итак, начинаем с верхнего из них...

От гнезда "var_d !=0" отходят две ветки - правая ведет к "printf("OK")" и далее к завершению конструкции "IF - THEN [ELSE]", а левая, прежде чем выйти к точке "Z", минует целое полчище гнезд. В переводе на русский язык ситуация выглядит так: "если переменная var_d не равна нулю, то печатаем "OK" и сваливаем, иначе выполняем дополнительные проверки". Проще говоря: "IF (var_d !=0) THEN printf("OK") ELSE E". Т.е. левая ветка гнезда (var_d != 0) есть ветка "ELSE". Изучим ее?

От гнезда (var_a <= var_b) к узлу "printf("OK")" ведут два пути: !(var_a <= var_b) а !(var_a ==0) и !(var_a != var_c) а !(var_c == 0). Где есть альтернатива - там всегда есть OR. Т.е. либо первый путь, либо второй. В то же время, узлы обоих путей последовательно связаны друг с другом, - значит, они объедены операций AND. Таким, образом, эта ветка должна выглядеть так: "IF ((var_a > var_b) && (var_0 != 0)) || (var_a == var_c) && (var_c != 0)) printf("OK")", прививаем "ELSE" к первому IF и получаем. "IF (var_d !=0) THEN printf("OK") ELSE IF((var_a > var_b) && (var_0 != 0)) || (var_a == var_c) && (var_c != 0)) printf("OK")"

Ну, а разбор второго дерева вообще тривиален: "IF (var_c==var_d) printf("+++")". Итак, исходный текст дизассемблируемой программы выглядел так:

```
u_int a; u_int b; ?_int c; ?_int d;
```

```
if (d) printf("TRUE");
else
if (((a>b) && (a!=0)) || ((a==c) && (c!=0))) printf("OK\n");

if (c==d) printf("+++\n");
```

Листинг 165

Тип переменных a и b мы определили как unsigned int, т.к. они результат сравнения анализировался беззнаковой условной командой - jnb. А вот тип переменных c и d, увы, определить так и не удалось. Однако это не умоляет значимости того факта, что мы смогли ретранслировать сложное условие, в котором без деревьев было бы немудрено и запутаться...

> - Больше всего следует опасаться идей, которые переходят в дела.
> Френк Херберт "Мессия дюны"

Оптимизация ветвлений: Какое коварство - под флагом оптимизации сделать каждую строчку кода головоломкой. Тьфу-ты, тут ящика пива не хватит, чтобы с этим справиться (а с этим лучше справляться вообще без пива - на трезвую голову). Итак, предположим, встретился вам код следующего содержания. На всякий случай, чтобы избавить вас от копания по справочникам (хотя, покопаться в них лишний раз - только на пользу) отмечу, что команда SETGE устанавливает выходной операнд в 1, если флаги состояния SF и OF равны (т.е. SF==OF). Иначе выходной операнд устанавливается в ноль.

```
mov eax, [var_A]
xor ecx,ecx
cmp eax, 0x666
setge cl
dec ecx
and ecx, 0xFFFFFC00
add ecx, 0x300
mov [var_zzz],ecx
```

Листинг 166

На первый взгляд этот фрагмент заимствован из какого-то хитро-запутанного защитного механизма, но нет. Перед вами результат компиляции следующего тривиального выражения: if (a<0x666) zzz=0x200 else zzz=0x300, которое в не оптимизированном виде выглядит так:

```
mov eax,[var_A]
cmp eax,0x666
jge Label_1
mov ecx, 0x100
jmp lable_2
Label_1:
mov ecx, 0x300
Lable_2:
mov [var_zzz],ecx
```

Листинг 167

Чем же компилятору не понравился такой вариант? Между прочим, он даже короче. Короче-то, он короче, но содержит ветвления, - т.е. внеплановые изменения нормального хода выполнения программы. А ветвления отрицательно сказываются на производительности, хотя бы уже потому, что они приводят к очистке конвейера. Конвейер же в современных процессорах очень длинный и быстро его не заполнишь... Поэтому, избавление от ветвлений путем хитроумных математических вычислений вполне оправдано и горячо приветствуется. Попутно это усложняет анализ программы, защищая ее от всех посторонних личностей типа хакеров (т.е. нас с вами).

Впрочем, если хорошенько подумать.... Начнем пошагово исполнять программу, мысленно комментируя каждую строчку.

```
mov eax, [var_A]
```

```
; eax == var_A

xor ecx,ecx
; ecx=0;

cmp eax, 0x666
; if eax<0x666 { SF=1; OF=0} else {SF=0; OF=0}

setge cl
; if eax<0x666 (т.е. SF==1, OF ==0) cl=0 else cl=1

dec ecx
; if eax<0x666 ecx=-1 else ecx=0

and ecx, 0xFFFFFC00
; if eax<0x666 (т.е. ecx==-1) ecx=0xFFFFFC00 (-0x400) else ecx=0;

add ecx, 0x300
; if eax<0x666 (т.е. ecx=-0x400) ecx=0x100 else ecx=0x300;

mov [esp+0x66],ecx
```

Листинг 168

Получилось! Мы разобрались с этим алгоритмом и успешно реверсировали его! Теперь видно, что это довольно простой пример (в жизни будут нередко попадаться и более сложные). Но основная идея ясна, - если встречаются команда SETxx - держите нос по ветру: пахнет условными переходами! В вырожденных случаях SETxx может быть заменена на SBB (вычитание с заемом). По этому поводу решим вторую задачу:

```
SUB EBX,EAX
SBB ECX,ECX
AND ECX,EBX
ADD EAX,ECX
```

Листинг 169

Что этот код делает? Какие-то сложные арифметические действия? Посмотрим...

```
SUB EBX,EAX
; if (EBX<EAX) SF=1 else SF=0

SBB ECX,ECX
; if (EBX<EAX) ECX=-1 else ECX=0

AND ECX,EBX
; if (EBX<EAX) ECX=EBX else ECX=0

ADD EAX,ECX
; if (EBX<EAX) EAX=EAX+(EBX-EAX) else EAX=EAX
```

Листинг 170

Раскрывая скобки в последнем выражении (мы ведь не забыли, что от EBX отняли EAX?) получаем: if (EBX<EAX) EAX=EBX, - т.е. это классический алгоритм поиск минимума среди двух знаковых чисел. А вот еще один пример:

```
CMP EAX,1
SBB EAX,EAX
AND ECX,EAX
XOR EAX,-1
AND EAX,EBX
OR  EAX,ECX
```

Листинг 171

Попробуйте решить его сами и только потом загляните в ответ:

```
CMP EAX,1
```

```
; if (EAX!=0) SF=0 else SF=1

SBB EAX,EAX
; if (EAX!=0) EAX=-1 else EAX=0

AND ECX,EAX
; if (EAX!=0) ECX=ECX else ECX=0

XOR EAX,-1
; if (EAX!=0) EAX=0 else EAX=-1

AND EAX,EBX
; if (EAX!=0) EAX=0 else EAX=EBX

OR  EAX,ECX
; if (EAX!=0) EAX=ECX else EAX=EBX
```

Листинг 172

Да... после таких упражнений тов. Буль будет во сне сниться! Но... таковы уж издержки цивилизации. К слову сказать, подавляющее большинство компиляторов достаточно лояльно относятся к условным переходам и не стремятся к их тотальному изгнанию. Так что... особо напрягаться при анализе оптимизированного кода не приходится (правда, к ручной оптимизации это не относится - профессиональные разработчики выкидывают переходы первую очередь).

Идентификация SWITCH - CASE - BREAK

Автор: Крис Касперски

Дата: 2002-06-16

> *"…когда вы видите все целиком, то у вас нет выбора, вам не из чего выбирать. Тогда вы имеете два пути одновременно, следуете одновременно этим двум направлениям"*
> Ошо "Пустая лодка" Беседы по высказываниям Чжуан Цзы

Для улучшения читабельности программ в язык Си был введен оператор множественного выбора – switch. В Паскале с той же самой задачей справляется оператор CASE, кстати, более гибкий, чем его Си-аналог, но об их различиях мы поговорим попозже.

Легко показать, что switch эквивалентен конструкции "IF (a == x1) THEN оператор1 ELSE IF (a == x2) THEN оператор2 IF (a == x2) THEN оператор2 IF (a == x2) THEN оператор2 ELSE …. оператор по умолчанию". Если изобразить это ветвление в виде логического дерева, то образуется характерная "косичка", прозванная так за сходство с завитой в косу прядью волос – см. рис. 29

Казалось бы, идентифицировать switch никакого труда не составит, – даже не стоя дерева, невозможно не обратить внимания на длинную цепочку гнезд, проверяющих истинность условия равенства некоторой переменной с серией непосредственных значений (сравнения переменной с другой переменной switch не допускает).

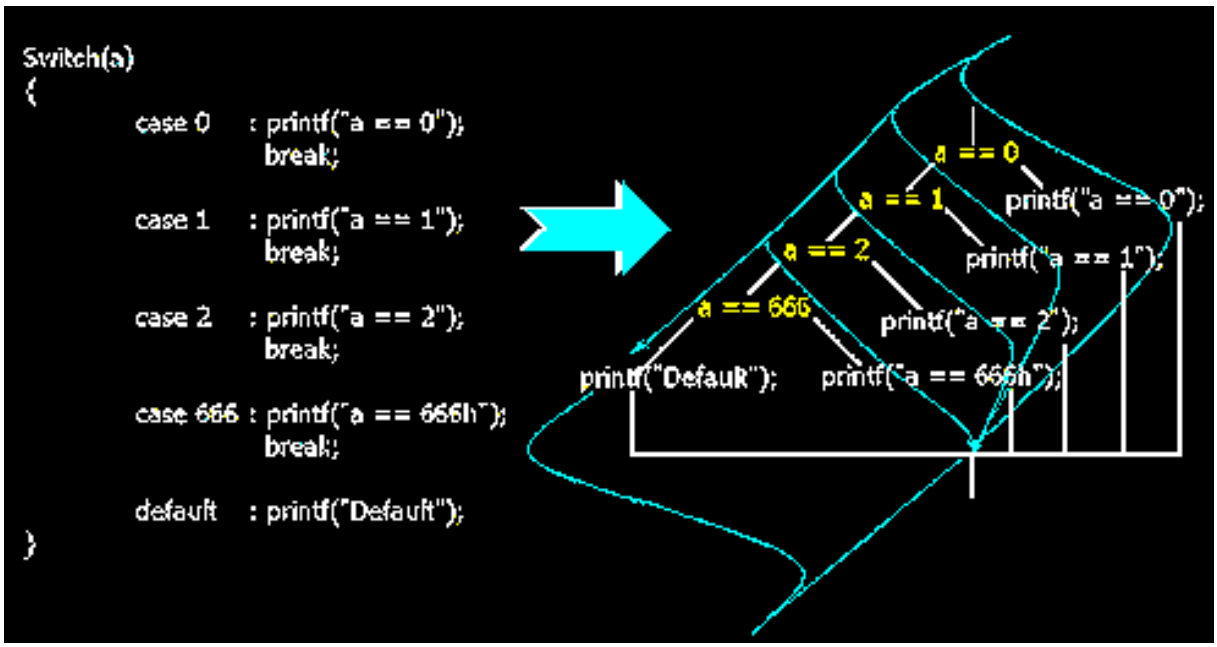


Рисунок 29 0x01C Трансляция оператора switch в общем случае

Однако в реальной жизни все происходит совсем не так. Компиляторы (даже не оптимизирующие) транслируют switch в настоящий "мясной рулет", доверху нашпигованных всевозможными операциями отношений. Давайте, откомпилируем приведенный выше пример компилятором Microsoft Visual C++ и посмотрим, что из этого выйдет:

```
main  proc near  ; CODE XREF: start+AF p

var_tmp  = dword ptr -8
var_a  = dword ptr -4

push ebp
mov ebp, esp
```

```
; Открываем кадр стека

sub esp, 8
; Резервируем место для локальных переменных

mov eax, [ebp+var_a]
; Загружаем в EAX значение переменной var_a

mov [ebp+var_tmp], eax
; Обратите внимание – switch создает собственную временную переменную!
; Даже если значение сравниваемой переменной в каком-то ответвлении CASE
; будет изменено, это не повлияет на результат выборов!
; В дальнейшем во избежании путаницы, мы будем условно называть
; переменную var_tmp переменной var_a

cmp [ebp+var_tmp], 2
; Сравниваем значение переменной var_a с двойкой
; Хм-хм, в исходном коде CASE начинался с нуля, а заканчивался 0x666
; Причем же тут двойка?!

jg short loc_401026
; Переход, если var_a > 2
; Обратите на этот момент особое внимание – ведь в исходном тексте такой
; операции отношения не было!
; Причем, этот переход не ведет к вызову функции printf, т.е. этот фрагмент
; кода получен не прямой трансляцией некой ветки case, а как-то иначе!

cmp [ebp+var_tmp], 2
; Сравниваем значение var_a с двойкой
; Очевидный "прокол" компилятора – мы же только что проделывали эту
; операции, и с того момента не меняли никакие флаги!

jz short loc_40104F
; Переход к вызову printf("a == 2"), если var_a == 2
; OK, этот код явно получен трансляцией ветки CASE 2: printf("a == 2")

cmp [ebp+var_tmp], 0
; Сравниваем var_a с нулем

jz short loc_401031
; Переход к вызову printf("a == 0"), если var_a == 0
; Этот код получен трансляцией ветки CASE 0: printf("a == 0")

cmp [ebp+var_tmp], 1
; Сравниваем var_a с единицей

jz short loc_401040
; Переход к вызову printf("a == 1"), если var_a == 1
; Этот код получен трансляцией ветки CASE 1: printf("a == 1")

jmp short loc_40106D
; Переход к вызову printf("Default")
; Этот код получен трансляцией ветки Default: printf("a == 0")

loc_401026:    ; CODE XREF: main+10 j
; Эта ветка получает управление, если var_a > 2
cmp [ebp+var_tmp], 666h
; Сравниваем var_a со значением 0x666

jz short loc_40105E
; Переход к вызову printf("a == 666h"), если var_a == 0x666
; Этот код получен трансляцией ветки CASE 0x666: printf("a == 666h")

jmp short loc_40106D
; Переход к вызову printf("Default")
; Этот код получен трансляцией ветки Default: printf("a == 0")

loc_401031:    ; CODE XREF: main+1C j
; // printf("A == 0")
push offset aA0 ; "A == 0"
call _printf
```

```
add esp, 4
jmp short loc_40107A
; ^^^^^^^^^^^^^^^^^^^^^ - а вот это оператор break, выносящий управление
; за пределы switch – если бы его не было, то начали бы выполняться все
; остальные ветки CASE, не зависимо от того, к какому значению var_a они
; принадлежат!

loc_401040:    ; CODE XREF: main+22 j
; // printf("A == 1")
push offset aA1 ; "A == 1"
call _printf
add esp, 4
jmp short loc_40107A
; ^ break

loc_40104F:    ; CODE XREF: main+16 j
; // printf("A == 2")
push offset aA2 ; "A == 2"
call _printf
add esp, 4
jmp short loc_40107A
; ^ break

loc_40105E:    ; CODE XREF: main+2D j
; // printf("A == 666h")
push offset aA666h ; "A == 666h"
call _printf
add esp, 4
jmp short loc_40107A
; ^ break

loc_40106D:    ; CODE XREF: main+24 j main+2F j
; // printf("Default")
push offset aDefault ; "Default"
call _printf
add esp, 4

loc_40107A:    ; CODE XREF: main+3E j main+4D j ...
; // КОНЕЦ SWITCH
mov esp, ebp
pop ebp
; Закрываем кадр стека
retn
main  endp
```
Листинг 173

Построив логическое дерево (см. "Идентификация IF – THEN – ELSE"), мы получим следующую картину (см. рис. 30). При ее изучении бросается в глаза, во-первых, условие "a >2", которого не было в исходной программе, а во-вторых, изменение порядка обработки case. В то же время, вызовы функций printf следуют один за другим строго согласно их объявлению. Зачем же компилятор так чудит? Чего он рассчитывает этим добиться?

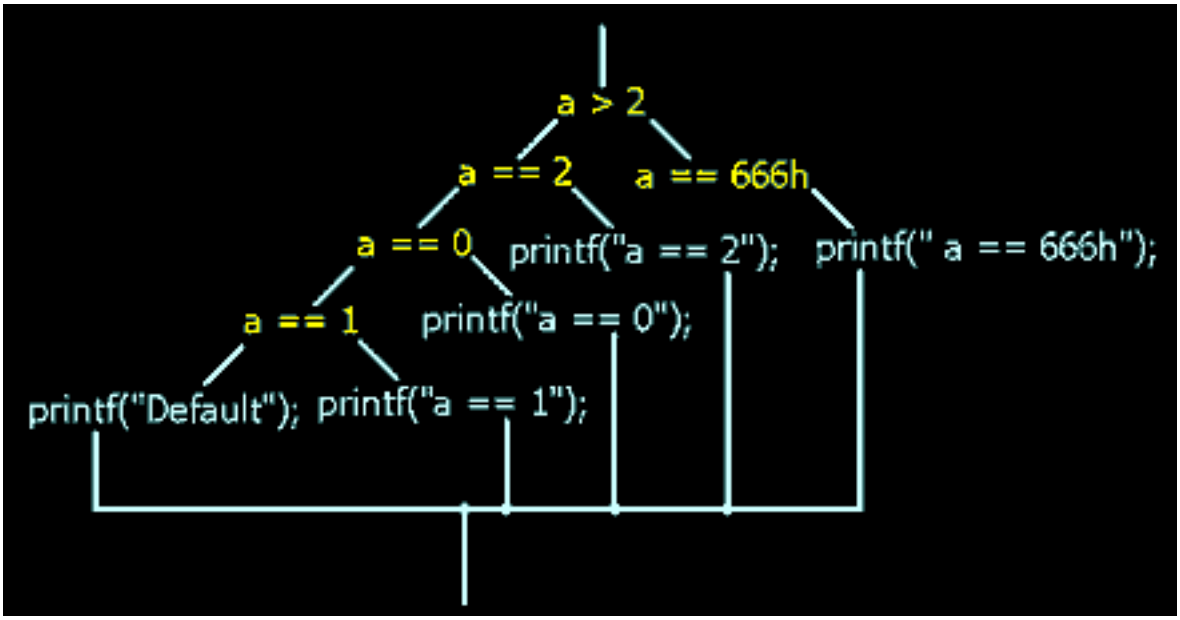


Рисунок 30 0x01D Пример трансляция оператора switch компилятором Microsoft Visual C

Назначение гнезда (a > 2) объясняется очень просто – последовательная обработка всех операторов case крайне непроизводительная. Хорошо, если их всего четыре-пять штук, а если программист натолкает в switch сотню - другую case? Процессор совсем запарится, пока их все проверит (а по закону бутерброда нужный case будет в самом конце). Вот компилятор и "утрамбовывает" дерево, уменьшая его высоту. Вместо одной ветви, изображенной на рис. 30, транслятор в нашем случае построил две, поместив в левую только числа не большие двух, а в правую – все остальные. Благодаря этому, ветвь "666h" из конца дерева была перенесена в его начало. Данный метод оптимизации поиска значений называют "методом вилки", но не будет сейчас на нем останавливаться, а лучше разберем его в главе "*Обрезка длинных деревьев*".

Изменение порядка сравнений – право компилятора. Стандарт ничего об этот не говорит и каждая реализация вольна поступать так, как ей это заблагорассудится. Другое дело – case-обработчики (т.е. тот код, которому case передает управление в случае истинности отношения). Они обязаны располагаться так, как были объявлены в программе, т.к. при отсутствии закрывающего оператора break они должны выполняться строго в порядке, замышленном программистом, хотя эта возможность языка Си используется крайне редко.

Таким образом, идентификация оператора switch не сильно усложняется: *если после уничтожения узлового гнезда и прививки правой ветки к левой (или наоборот) мы получаем эквивалентное дерево, и это дерево образует характерную "косичку" – мы имеем дело с оператором множественного выбора или его аналогом.*

Весь вопрос в том: правомерны ли мы удалять гнездо, не нарушит ли эта операция структуры дерева? Смотрим – на левой ветке узлового гнезда расположены гнезда (a == 2), (a == 0) и (a == 1), а на левом – (a==0x666) Очевидно, если a == 0x666, то a != 0 и a != 1! Следовательно, прививка правой ветки к левой вполне безопасна и после такого преобразования дерево принимает вид типичный для конструкции switch (см. рис. 31)

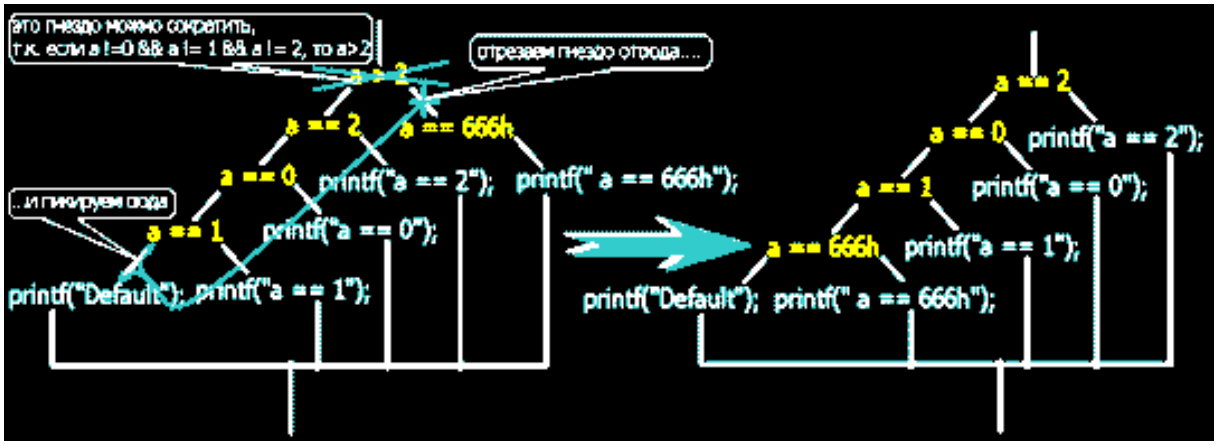


Рисунок 31 0x01E Усечение логического дерева

Увы, такой простой прием идентификации срабатывает не всегда! Иные компиляторы такого наворотят, что волосы в разных местах дыбом встанут! Если откомпилировать наш пример компилятором Borland C++ 5.0, то код будет выглядеть так:

```
; int __cdecl main(int argc,const char **argv,const char *envp)
_main  proc near  ; DATA XREF: DATA:00407044 o

push ebp
mov ebp, esp
; Открываем кадр стека
; Компилятор помещает нашу переменную a в регистр EAX
; Поскольку она не была инициализирована, то заметить этот факт
; не так-то легко!

sub eax, 1
; Уменьшает EAX на единицу! Что бы этого значило, хвост Тиггера?
; Никакого вычитания в нашей программе не было!

jb short loc_401092
; Если EAX < 1, то переход на вызов printf("a == 0")
; (мы ведь помним, что CMP та же команда SUB, только не изменяющая операндов?)
; Ага, значит, этот код сгенерирован в результате трансляции
; ветки CASE 0: printf("a == 0");
; Внимание! задумайтесь: какие значения может принимать EAX, чтобы
; удовлетворять условию этого отношения? На первый взгляд, EAX < 1,
; в частости, 0, -1, -2,… СТОП! Ведь jb - это беззнаковая инструкция
; сравнения! А -0x1 в беззнаковом виде выглядит как 0xFFFFFFFF
; 0xFFFFFFFF много больше единицы, следовательно, единственным подходящим
; значением будет ноль
; Таким образом, данная конструкция - просто завуалированная проверка EAX на
; равенство нулю! (Ox! и хитрый же этот Borland - компилятор!)
;

jz short loc_40109F
; Переход, если установлен флаг нуля
; Он будет он установлен в том случае, если EAX == 1
; И действительно переход идет на вызов printf("a == 1")

dec eax
; Уменьшаем EAX на единицу

jz short loc_4010AC
; Переход если установлен флаг нуля, а он будет установлен когда после
; вычитания единицы командой SUB, в EAX останется ровно единица,
; т.е. исходное значение EAX должно быть равно двум
; И точно - управление передается ветке вызова printf("a == 2")!

sub eax, 664h
; Отнимаем от EAX число 0x664

jz short loc_4010B9
```

```
; Переход, если установлен флаг нуля, т.е. после двукратного уменьшения EAX
; равен 0x664, следовательно, исходное значение - 0x666

jmp short loc_4010C6
; прыгаем на вызов printf("Default"). Значит, это - конец switch

loc_401092:    ; CODE XREF: _main+6 j
; // printf("a==0");
push offset aA0 ; "a == 0"
call _printf
pop ecx
jmp short loc_4010D1

loc_40109F:    ; CODE XREF: _main+8 j
; // printf("a==1");
push offset aA1 ; "a == 1"
call _printf
pop ecx
jmp short loc_4010D1

loc_4010AC:    ; CODE XREF: _main+B j
; // printf("a==2");
push offset aA2 ; "a == 2"
call _printf
pop ecx
jmp short loc_4010D1

loc_4010B9:    ; CODE XREF: _main+12 j
; // printf("a==666");
push offset aA666h ; "a == 666h"
call _printf
pop ecx
jmp short loc_4010D1

loc_4010C6:    ; CODE XREF: _main+14 j
; // printf("Default");
push offset aDefault ; "Default"
call _printf
pop ecx

loc_4010D1:    ; CODE XREF: _main+21 j _main+2E j ...
xor eax, eax
pop ebp
retn
_main  endp
```
Листинг 174

Код, сгенерированный компилятором, модифицирует сравниваемую переменную в процессе сравнения! Оптимизатор посчитал, что DEC EAX короче, чем сравнение с константой, да и работает шустрее. Вот только нам, хакером, от этого утешения ничуть не легче! Ведь прямая ретрансляция кода (см. *"Идентификация IF - THEN - ELSE"*) дает конструкцию вроде: "if (a-- == 0) printf("a == 0"); else if (a==0) printf("a == 1"); else if (--a == 0) printf("a == 2"); else if ((a-=0x664)==0) printf("a == 666h); else printf("Default")", - в которой совсем не угадывается оператор switch! Впрочем, почему это "не угадывается"?! Угадывается, еще как! Где есть длинная цепочка "IF-THEN-ELSE-IF-THEN-ELSE…" там и до switch-а недалеко! Узнать оператор множественного выбора будет еще легче, если изобразить его в виде дерева - смотрите (см. рис. 32) вот она, характерная "косичка"!

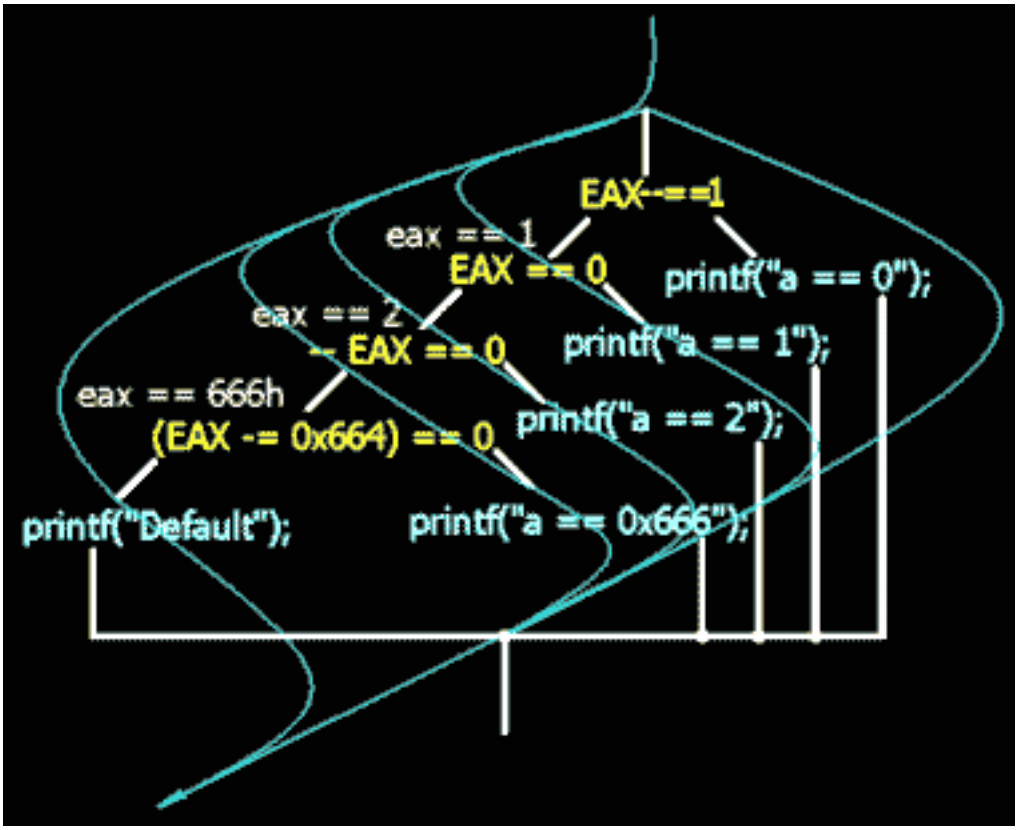


Рисунок 32 0x01F Построение логического дерева с гнездами, модифицирующими саму сравниваемую переменную

Другая характерная деталь - case-обработчики, точнее оператор break традиционно замыкающий каждый из них. Они-то и образуют правую половину "косички", сходясь все вместе с точке "Z". Правда, многие программисты питают паралогическую любовь к case-обработчикам размером в два-три экрана, включая в них помимо всего прочего и циклы (о них речь еще впереди - см. "*Идентификация for\while*"), и ветвления, и даже вложенные операторы множественно выбора! В результате правая часть "косички" превращается в непроходимый таежный лес, сквозь который не проберется и стадо слонопотамов. Но даже если и так - левая часть "косички", все равно останется достаточно простой и легко распознаваемой!

В заключение темы рассмотрим последний компилятор - WATCOM C. Как и следует ожидать, здесь нас подстерегают свои тонкости и "вкусности". Итак, откомпилированный им код предыдущего примера должен выглядеть так:

```
main_  proc near  ; CODE XREF: __CMain+40 p
push 8
call __CHK
; Проверка стека на переполнение

cmp eax, 1
; Сравнение регистровой переменной EAX, содержащей в себе переменную a
; со значением 1

jb short loc_41002F
; Если EAX == 0, то переход к ветви с дополнительными проверками

jbe short loc_41003A
; Если EAX == 1 (т.е. условие bellow уже обработано выше), то переход
; к ветке вызова printf("a == 1");

cmp eax, 2
; Сравнение EAX со значением 2

jbe short loc_410041
```

```
; Если EAX == 2 (условие EAX <2 уже было обработано выше),  то переход
; к ветке вызова printf("a == 2");

cmp eax, 666h
; Сравнение EAX со значением 0x666

jz short loc_410048
; Если EAX == 0x666, то переход к ветке вызова printf("a == 666h");

jmp short loc_41004F
; Что ж, ни одно из условий не подошло - переходит к ветке "Default"


loc_41002F:    ; CODE XREF: main_+D j
; // printf("a == 0");
test eax, eax
jnz short loc_41004F
; Совершенно непонятно - зачем здесь дополнительная проверка?!
; Это ляп компилятора - она ни к чему!

push offset aA0 ; "A == 0"
; Обратите внимание - WATCOM сумел обойтись всего одним вызовом printf!
; Обработчики case всего лишь передают ей нужный аргумент!
; Вот это действительно - оптимизация!
jmp short loc_410054

loc_41003A:    ; CODE XREF: main_+F j
; // printf("a == 1");
push offset aA1 ; "A == 1"
jmp short loc_410054

loc_410041:    ; CODE XREF: main_+14 j
; // printf("a == 2");
push offset aA2 ; "A == 2"
jmp short loc_410054

loc_410048:    ; CODE XREF: main_+1B j
; // printf("a == 666h");
push offset aA666h ; "A == 666h"
jmp short loc_410054

loc_41004F:    ; CODE XREF: main_+1D j main_+21 j
; // printf("Default");
push offset aDefault ; "Default"

loc_410054:    ; CODE XREF: main_+28 j main_+2F j ...
call printf_
; А вот он наш printf, получающий аргументы из case-обработчиков!

add esp, 4
; Закрытие кадра стека

retn
main_  endp
```
Листинг 175

В общем, WATCOM генерирует более хитрый, но, как ни странно, весьма наглядный и читабельный код.

::Отличия switch от оператора case языка Pascal. Оператор CASE языка Pascal практически идентичен своему Си собрату - оператору switch, хотя и близнецами их не назовешь: оператор CASE выгодно отличается поддержкой *наборов и диапазонов значений.* Ну, если обработку наборов можно реализовать и посредством switch, правда не так элегантно как на Pascal (см. листинг 176), то проверка вхождения значения в диапазон на Си организуется исключительно с помощью конструкции "IF-THEN-ELSE". Зато в Паскале каждый case-обработчик принудительно завершается неявным break, а Си-программист волен ставить (или не ставить) его по своему усмотрению.

```
CASE a OF     switch(a)
begin      {
1 : WriteLn('a == 1');   case 1 : printf("a == 1");
  break;
2,4,7 : WriteLn('a == 2|4|7');  case 2 :
case 4 :
case 7 : printf("a == 2|4|7");
  break;
9 : WriteLn('a == 9');   case 9 : printf("a == 9");
  break;
end;
```
Листинг 176

Однако оба языка накладывают жесткое ограничение на выбор сравниваемой переменной: она должна принадлежать к перечисленному типу, а все наборы (диапазоны) значений представлять собой константы или константные выражения, вычисляемые на стадии компиляции. Подстановка переменных или вызовов функций не допускается.

Представляет интерес посмотреть: как Pascal транслирует проверку диапазонов и сравнить его с компиляторами Си. Рассмотрим следующий пример:

```
VAR
a : LongInt;
BEGIN

CASE a OF
2  : WriteLn('a == 2');
4, 6  : WriteLn('a == 4 | 6 ');
10..100  : WriteLn('a == [10,100]');
END;
END.
```
Листинг 177

Результат его компиляции компилятором Free Pascal должен выглядеть так (для экономии места приведена лишь левая часть "косички"):

```
mov eax, ds:_A
; Загружаем в EAX значение сравниваемой переменной

cmp eax, 2
; Сравниваем EAX со значением 0x2

jl loc_CA  ; Конец CASE
; Если EAX < 2, то - конец CASE

sub eax, 2
; Вычитаем из EAX значение 0x2

jz loc_9E  ; WriteLn('a == 2');
; Переход на вызов WriteLn('a == 2') если EAX == 2

sub eax, 2
; Вычитаем из EAX значение 0x2

jz short loc_72 ; WriteLn('a == 4 | 6');
; Переход на вызов WriteLn(''a == 4 | 6') если EAX == 2 (соотв. a == 4)

sub eax, 2
; Вычитаем из EAX значение 0x2

jz short loc_72 ; WriteLn('a == 4 | 6');
; Переход на вызов WriteLn(''a == 4 | 6') если EAX == 2 (соотв. a == 6)

sub eax, 4
; Вычитаем из EAX значение 0x4

jl loc_CA  ; Конец CASE
; Переход на конец CASE, если EAX < 4 (соотв. a < 10)

sub eax, 90
```

```
; Вычитаем из EAX значение 90

jle short loc_46 ; WriteLn('a = [10..100]');
; Переход на вызов WriteLn('a = [10..100]') если EAX <= 90 (соотв. a <= 100)
; Поскольку, случай a > 10 уже был обработан выше, то данная ветка
; срабатывает при условии a>=10 && a<=100.

jmp loc_CA  ; Конец CASE
; Прыжок на конец CASE - ни одно из условий не подошло
```
Листинг 178

Как видно, Free Pascal генерирует практически тот же самый код, что и компилятор Borland C++ 5.x, поэтому его анализ не должен вызвать никаких сложностей.

__::IDA распознает switch

::Обрезка (балансировка) длинных деревьев. В некоторых (хотя и редких) случаях, операторы множественного выбора содержат сотни (а то и тысячи) наборов значений, и если решать задачу сравнения "в лоб", то высота логического дерева окажется гигантской до неприличия, а его прохождение займет весьма длительное время, что не лучшим образом скажется на производительности программы.

Но, задумайтесь: чем собственно занимается оператор switch? Если отвлечься от устоявшейся идиомы "*оператор SWITCH дает специальный способ выбора одного из многих вариантов, который заключается в проверке совпадения значения данного выражения с одной из заданных констант и соответствующем ветвлении*", то можно сказать, что switch - оператор *поиска* соответствующего значения. В таком случае каноническое switch - дерево представляет собой тривиальный алгоритм последовательного поиска - самый неэффективный алгоритм из всех.

Пусть, например, исходный текст программы выглядел так:

```
.
switch (a)
{
case 98 : …;
case 4  : …;
case 3  : …;
case 9  : …;
case 22 : …;
case 0  : …;
case 11 : …;
case 666: …;
case 096: …;
case 777: …;
case 7  : …;
}
```
Листинг 179

Тогда соответствующее ему не оптимизированное логическое дерево будет достигать в высоту одиннадцати гнезд (см. рис. 33 слева). Причем, на левой ветке корневого гнезда окажется аж десять других гнезд, а на правой - вообще ни одного (только соответствующий ему case - обработчик).

Исправить "перекос" можно разрезав одну ветку на две и привив образовавшиеся половинки к новому гнезду, содержащему условие, определяющее в какой из веток следует искать сравниваемую переменную. Например, левая ветка может содержать гнезда с четными значениями, а правая - с нечетными. Но это плохой критерий: четных и нечетных значений редко бывает поровну и вновь образуется перекос. Гораздо надежнее поступить так: берем наименьшее из всех значений и бросаем его в кучу A, затем берем наибольшее из всех значений и бросаем его в кучу B. Так повторяем до тех пор, пока не рассортируем все, имеющиеся значения.

Поскольку оператор множественного выбора требует уникальности каждого значения, т.е. каждое число может встречаться в наборе (диапазоне) значений лишь однажды, легко показать, что: а) в

обеих кучах будет содержаться равное количество чисел (в худшем случае - в одной куче окажется на число больше); б) все числа кучи А меньше наименьшего из чисел кучи В. Следовательно, достаточно выполнить только одно сравнение, чтобы определить в какой из двух куч следует искать сравниваемое значения.

Высота нового дерева будет равна ((N+1/2)+1), где N - количество гнезд старого дерева. Действительно, мы же ветвь дерева надвое и добавляем новое гнездо - отсюда и берется N/2 и +1, а (N+1) необходимо для округления результата деления в большую сторону. Т.е. если высота не оптимизированного дерева достигала 100 гнезд, то теперь она уменьшилась до 51. Что? Говорите, 51 все равно много? А что нам мешает разбить каждую из двух ветвей еще на две? Это уменьшит высоту дерева до 27 гнезд! Аналогично, последующее уплотнение даст 16 ' 12 ' 11 ' 9 ' 8… и все! Более плотная упаковка дерева невозможна (подумайте почему - на худой конец постройте само дерево). Но, согласитесь, восемь гнезд - это не сто! Полное прохождение оптимизированного дерева потребует менее девяти сравнений!

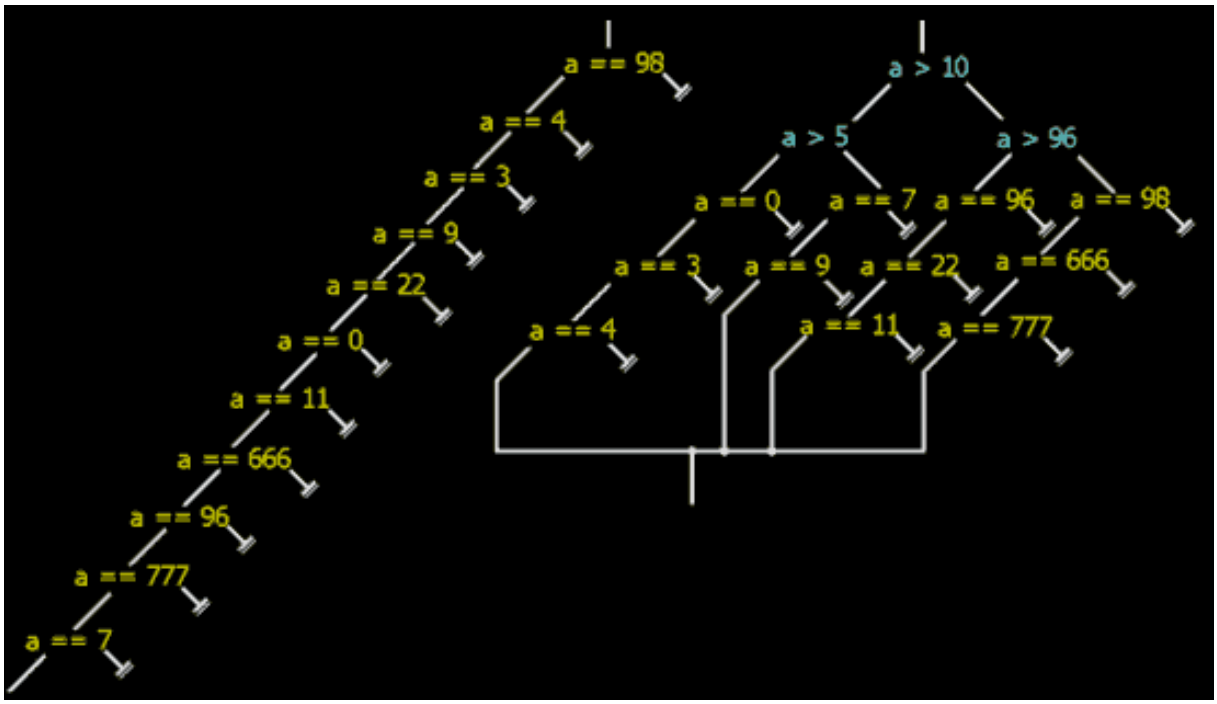


Рисунок 33 0x21 Логическое дерево до утрамбовки (слева) и после (справа)

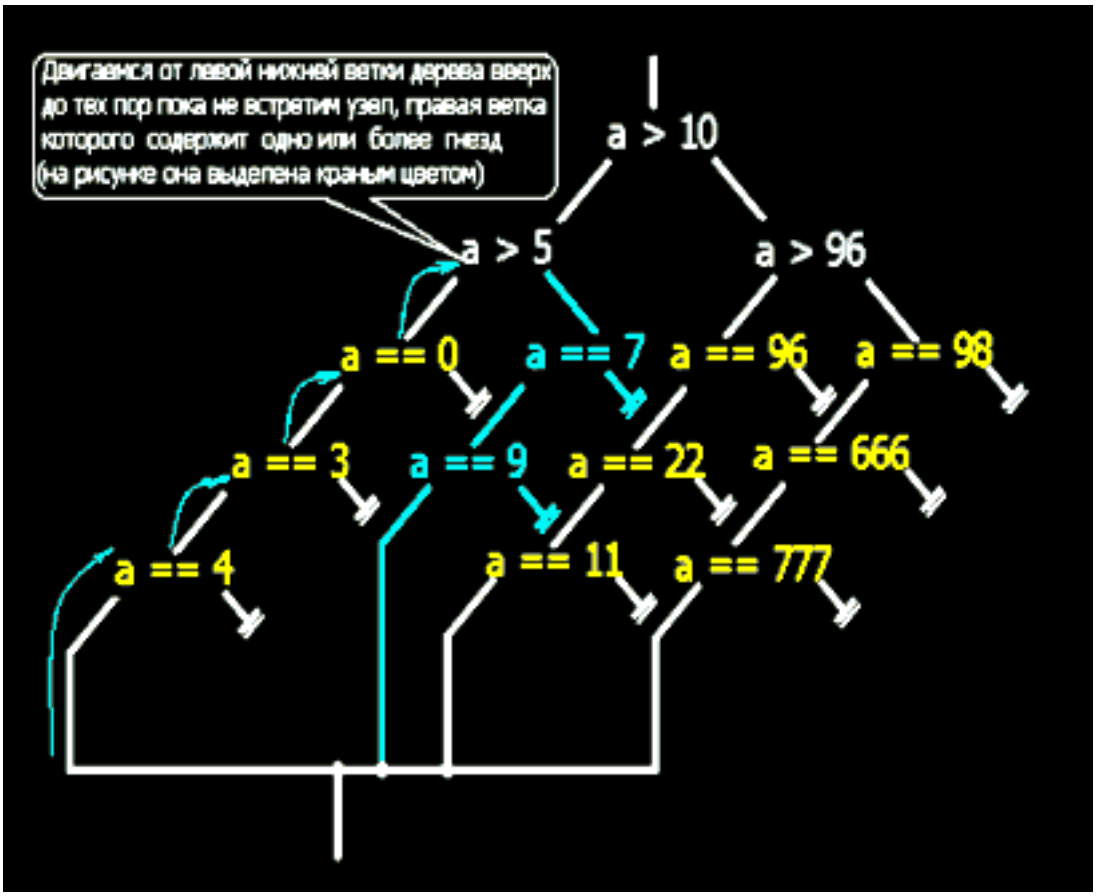
"Трамбовать" логические деревья оператора множественного выбора умеют практически все компиляторы - даже не оптимизирующие! Это увеличивает производительность, но затрудняет анализ откомпилированной программы. Взгляните еще раз на рис. 33 - левое несбалансированное дерево наглядно и интуитивно - понятно. После же балансировки (правое дерево) в нем Тиггер хвост обломит.

К счастью, балансировка дерева допускает эффективное обращение. Но прежде, чем засучить рукава и приготовиться к лазанью по деревьям (а Тиггеры по деревьям лазают лучше всех!) введем понятие *балансировочного узла*. Балансировочный узел не изменяет логики работы двоичного дерева и являются факультативным узлов, единственная функция которого укорачивание длины ветвей. Балансировочный узел без потери функциональности дерева может быть замещен любой из своих ветвей. Причем каждая ветвь балансировочного узла должна содержать одно или более гнезд.

Рассуждая от противного - все узлы логического дерева, правая ветка которых содержит одно или более гнезд, могут быть замещены на эту самую правую ветку без потери функциональности дерева, то данная конструкция представляет собой оператор switch. Почему именно правая ветка?

Так ведь оператор множественного выбора в "развернутом" состоянии представляет цепочку гнезд, соединенных левыми ветвями друг с другом, а на правых держащих case-обработчики, - вот мы и пытаемся подцепить все правые гнезда на левую ветвь. Если это удается, мы имеем дело с оператором множественного выбора, а нет - с чем-то другим.

Рассмотрим обращение балансировки на примере следующего дерева (см. рис. 34 слева). Двигаясь от левой нижней ветви, мы будем продолжать взбираться на дерево до тех пор, пока не встретим узел, держащий на своей правой ветви одно или более гнезд. В нашем случае - это узел (a > 5). Смотрите: если данный узел заменить его гнездами (a==7) и (a == 9) функциональность дерева не нарушиться! (см. рис. 34 посередине). Аналогично узел (a > 10) может быть безболезненно заменен гнездами (a > 96), (a == 96), (a == 22) и (a == 11), а узел (a > 96) в свою очередь - гнездами (a == 98), (a == 666) и (a == 777). В конце -концов образуется классическое switch-дерево, в котором оператор множественного выбора распознается с первого взгляда.



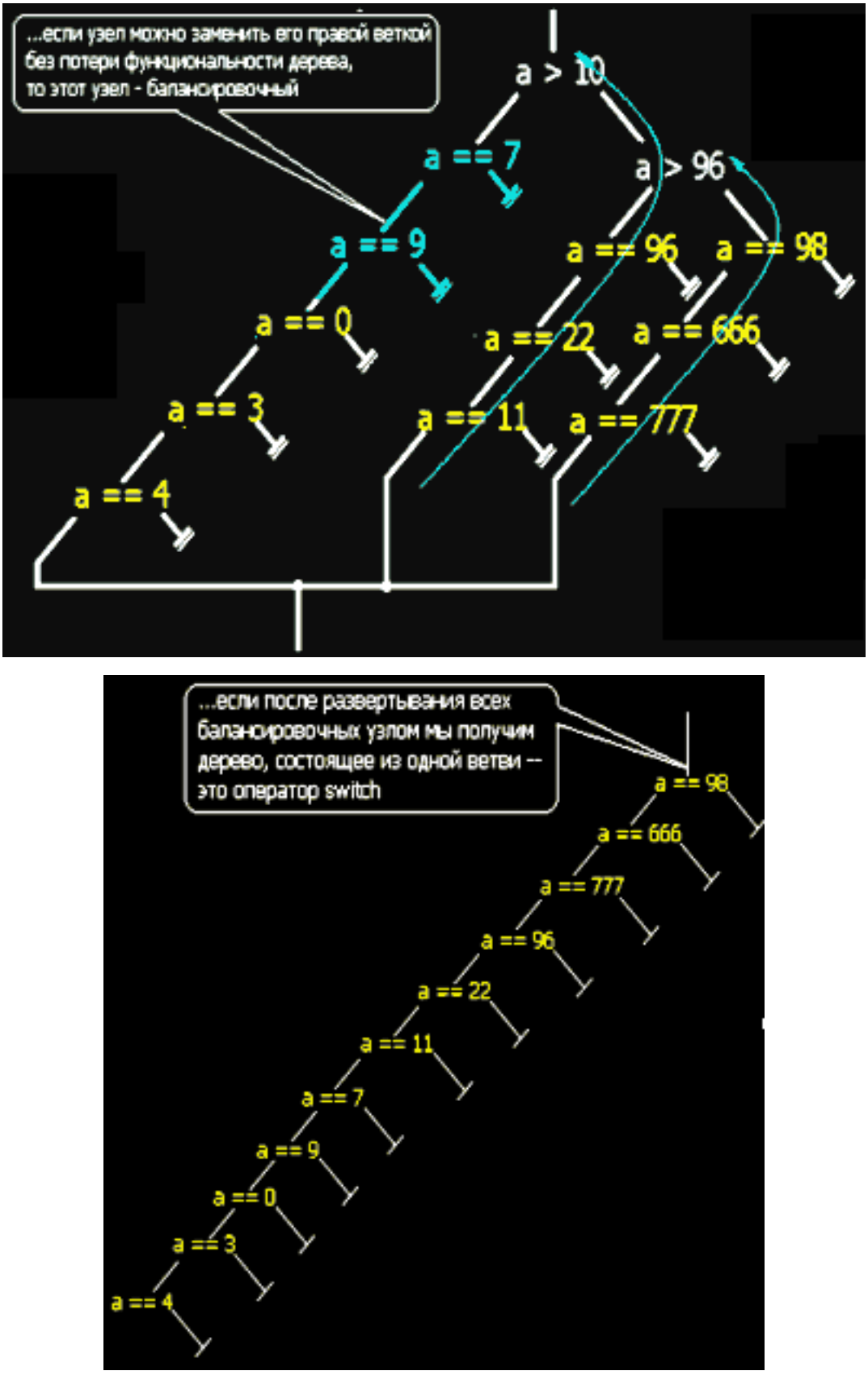


Рисунок 34 0x22 Обращение балансировки логического дерева

Сложные случаи балансировки или оптимизирующая балансировка. Для уменьшения высоты "утрамбовываемого" дерева хитрый трансляторы стремятся замещать уже существующие гнезда балансировочными узлами. Рассмотрим следующий пример: (см. рис. 35). Для уменьшения высоты

дерева транслятор разбивает его на две половины - в левую идут гнезда со значениями меньшие или равные единицы, а в правую - все остальные. Казалось бы, на правой ветке узла (a > 1) должно висеть гнездо (a == 2), ан нет! Здесь мы видим узел (a >2), к левой ветки которого прицеплен case-обработчик :2! А что, вполне логично - если (a > 1) и !(a > 2), то a == 2!

Легко видеть, что узел (a > 2) жестко связан с узлом (a > 1) и работает на пару с последним. Нельзя выкинуть один из них, не нарушив работоспособности другого! Обратить балансировку дерева по описанному выше алгоритму без нарушения его функциональности невозможно! Отсюда может создаться мнение, что мы имеем дело вовсе не с оператором множественного выбора, а чем-то другим.

Чтобы развеять это заблуждение придется предпринять ряд дополнительных шагов. Первое - у switch-дерева все case-обработчики всегда находятся на правой ветви. Смотрим - можно ли трансформировать наше дерево так, чтобы case-обработчик 2 оказался на левой ветви балансировочного узла? Да, можно: заменив (a > 2) на (a < 3) и поменяв ветви местами (другими словами выполнив инверсию). Второе - все гнезда switch-дерева содержат в себе условия равенства, - смотрим: можем ли мы заменить неравенство (a < 3) на аналогичное ему равенство? Ну, конечно же, можем - (a == 2)!

Вот, после всех этих преобразований, обращение балансировки дерева удается выполнить без труда!

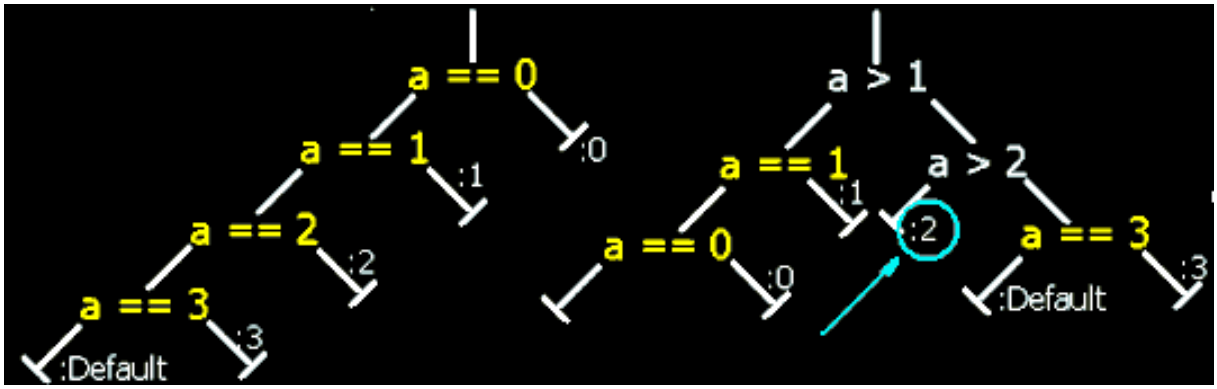


Рисунок 35 0x23 Хитрый случай балансировки

Ветвления в case-обработчиках. В реальной жизни case-обработчики прямо-таки кишат ветвлениями, циклами и прочими условными переходами всех мастей. Как следствие - логическое дерево приобретает вид ничуть не напоминающий оператор множественного выбора, а скорее смахивающий на заросли чертополоха, так любимые И-i. Понятное дело - идентифицировав case-обработчики, мы могли бы решить эту проблему, но как их идентифицировать?!

Очень просто - за редкими клиническими исключениями, case-обработчики не содержат ветвлений относительно сравниваемой переменной. Действительно, конструкции "switch(a) …. case 666 : if (a == 666) …." или "switch(a) …. case 666 : if (a > 66) …." абсолютно лишены смысла. Таким образом, мы можем смело удалить из логического дерева все гнезда с условиями, не касающимися сравниваемой переменной (переменной коневого гнезда).

Хорошо, а если программист в порыве собственной глупости или стремлении затруднить анализ программы "впаяет" в case-обработчики ветвления относительно сравниваемой переменной?! Оказывается, это ничуть не затруднит анализ! "Впаянные" ветвления элементарно распознаются и обрезаются либо как избыточные, либо как никогда не выполняющиеся. Например, если к правой ветке гнезда (a == 3) прицепить гнездо (a > 0) - его можно удалить, как не несущее в себе никакой информации. Если же к правой ветке того же самого гнезда прицепить гнездо (a == 2) его можно удалить, как никогда не выполняющееся - если a == 3, то заведомо a != 2!

Идентификация циклов

Автор: Крис Касперски

Дата: 2002-06-16

> "Связь между элементами системы носит
> трансуровневый характер и проявляет себя в виде
> повторяющихся единиц разных уровней (мотивов)"
> Тезис классического постструктурализма
> в его отечественном изводе

Циклы - единственная (за исключением неприличного "GOTO") конструкция языков высокого уровня, имеющая ссылку "назад", т.е. в область более младших адресов. Все остальные виды ветвлений - будь то IF - THEN - ELSE или оператор множественного выбора switch всегда направлены "вниз" - в область старших адресов. Вследствие этого, логическое дерево, изображающее цикл, настолько характерно, что легко опознается с первого взгляда.

Существуют три основных типа цикла: циклы с условием вначале , циклы с условием в конце (см. рис. 36 в центре) и циклы с условием в середине (см. рис. 36 справа). Комбинированные циклы имеют несколько условий в разных местах, например, в начале и в конце одновременно.

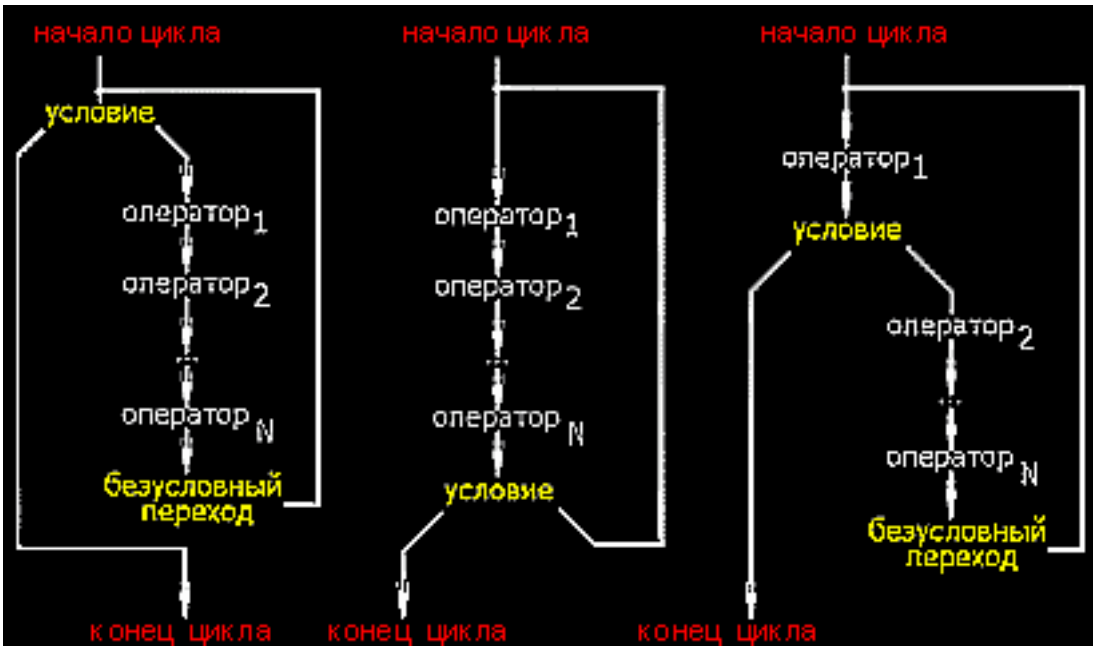


Рисунок 36.0x024 Логическое дерево цикла с условием вначале (слева) и условием в конец (справа).

В свою очередь условия бывают двух типов: *условия завершения цикла* и *условия продолжения цикла* . В первом случае:если условие завершения истинно происходит переход в конец цикла, иначе - его продолжение.Во втором: если условие продолжения цикла ложно происходит переход в конец цикла, в противном случае - его продолжения. Легко показать, что условия продолжения цикла представляют собой инвертированные условия завершения. Таким образом, со стороны транслятора вполне достаточно поддержки условий одного типа. И действительно, операторы циклов while,do и for языка Си работают исключительно с условиями продолжения цикла. Оператор while языка Pascal так же работает с условием продолжения цикла, и исключение составляет один лишь repeat-until ожидающий условие завершения цикла.

::*Циклы с условиями в начале (так же называемые циклами с преусловием)*. В языках Си и Pascal поддержка циклов с преусловием обеспечивается оператором "while (условие)", где "условие" - условие продолжения цикла. Т.е. цикл "while (a <10) a++;" выполняется до тех пор, пока условие (a > 10) остается истинным. Однако транслятор при желании может инвертировать условие продолжение цикла на условие завершения цикла. На платформе Intel 80x86 такой трюк экономит от одной до двух машинных команд. Смотрите: на листинге 180 слева приведен цикл с условием завершения, а справа - с условием продолжения. Как видно, цикл с условием завершения на одну команду короче! Поэтому, практически все компиляторы (даже не оптимизирующие) всегда генерируют левый вариант. (А некоторые, особо одаренные, даже умеют превращать циклы с предусловием в еще более эффективные циклы с пост-условием - см. "*Циклы с условием в конце*").

```
while:          while:
CMP A, 10       CMP A, 10
JAE end         JB continue
INC A           JMP end
JMP while       continue:
end:            INC A
                JMP while
                end:
```

Листинг 180.Слева показан цикл с условием завершения цикла, а справа - тот же цикл, но с условием продолжения цикла. Как видно, цикл с условием завершения на одну команду короче.

Цикл с условием завершения не может быть непосредственно отображен на оператор while. Кстати, об этом часто забывают начинающие, допуская ошибку "что вижу, то пишу": "while (a >= 10) a++". С таким условием данный цикл вообще не выполниться ни разу! Но как выполнить инверсию условия и при этом гарантированно не ошибиться? Казалось бы, что может быть проще, - а вот попросите знакомого хакера назвать операцию, обратную "больше". Очень может быть (даже наверняка!) ответом будет... "меньше". А вот и нет, - правильный ответ "меньше или равно". Полный перечень обратных операций отношений можно найти в таблице 25, приведенной ниже

Логическая операция	Обратная логическая операция
==	!=
!=	==
>	<=
<	>=
<=	>
>	<=

Таблица 25 Обратные операции отношения

::Циклы с условием в конце (так же называемые циклами с пост-условием). В языке Си поддержка циклов с пост-условием обеспечивается парой операторов do - while, а в языке Pascal - repeat\until. Циклы с пост-условием без каких либо проблем непосредственно отображаются с языка высокого уровня на машинный код и, соответственно, наоборот. Т.е. в отличие от циклов с предусловием, инверсии условия не происходит.

Например: "do a++; while (a<10)" в общем случае компилируется в следующий код (обратите внимание: в переходе использовалась та же самая операция отношения, что и в исходном цикле, - красота и никаких ошибок при декомпиляции):

```
repeat: <---------!
      INC A       !
      CMP A, 10   !
      JB repeat---!
         end
```

Листинг 181

Вернувшись страницей назад, сравним код цикла с пост-условием с кодом цикла с предусловием. Не правда ли, цикл с условием в конце компактнее и быстрее? Некоторые компиляторы (например, Microsoft Visual C++) умеют транслировать циклы с предусловием в циклы с пост-условием. На первый взгляд - это вопиющая самодеятельность компилятора, - если программист хочет проверять условие в начале, то какое право имеет транслятор ставить его в конце?! На самом же деле, разница между "до" или "после" не столь велика и значительна. Если компилятор уверен, что цикл выполняется хотя бы один раз, то он вправе выполнять проверку когда угодно. Разумеется, при этом необходимо несколько скорректировать условие проверки: "while (a<b)" не эквивалентно "do ... while (a<b)", т.к. в первом случае при (a == b) уже происходит выход из цикла, а во втором цикл выполняется еще одну итерацию. Однако этой беде легко помочь: увеличим a на единицу ("do ... while ((a+1)<b)") или вычтем эту единицу из b ("do ... while (a<(b-1))") и... теперь все будет работать!

Спрашивается: и на кой все эти извращения, значительно раздувающие код? Дело в том, что блок статического предсказания направления ветвлений Pentium-процессоров оптимизирован именно под переходы, направленные назад, т.е. в область младших адресов. Поэтому, циклы с постусловием должны выполняться несколько быстрее аналогичных им циклов с предусловием.

::Циклы со счетчиком. Циклы со счетчиком (for) не являются самостоятельным типом циклов, а представляют собой всего лишь синтаксическую разновидность циклов с предусловием. В самом деле, "for (a = 0; a < 10; a++)" в первом приближении это то же самое, что и: "a = 0; while (a < 10) {...;a++;}". Однако, результаты компиляции двух этих конструкций не обязательно должны быть идентичны друг другу!

Оптимизирующие компиляторы (да и значительная часть не оптимизирующих) поступают хитрее, передавая после инициализации переменной-счетчика управление на команду проверки условия выхода из цикла. Образовавшаяся конструкция, во-первых, характерна и при анализе программы сразу бросается в глаза, а, во-вторых, не может быть непосредственно отображена на циклы while языка высокого уровня. Смотрите:

```
MOV A, xxx          ; Инициализация переменной "счетчика"
JMP conditional     ; Переход к проверке условия продолжения цикла
repeat:             ; Начало цикла
...                 ; // ТЕЛО
...                 ; //      ЦИКЛА
ADD A, xxx [SUB A, xxx]; Модификация счетчика
conditional:        ; Проверка условия продолжения цикла
CMP A, xxx          ; ^
Jxx repeat          ; Переход в начало цикла, если условие истинно
```

Листинг 182

Непосредственный прыжок вниз может быть результат компиляции и цикла for, и оператора GOTO, но GOTO сейчас не в моде и используется крайне редко, а без него оператор условного перехода "IF - THEN" не может прыгнуть непосредственно в середину цикла while! Выходит, изо всех "кандидатов" остается только цикл for.

Некоторые, особо продвинутые компиляторы (Microsoft Visual C++, Borland C++, но не WATCOM C), поступают хитрее: анализируя код они еще на стадии компиляции пытаются определить: выполняется ли данный цикл хотя бы один раз и, если видят, что он действительно выполняется, превращают for в типичный цикл с постусловием:

```
MOV A, xxx      ; Инициализация переменной "счетчика"
repeat:         ; Начало цикла
...             ; // ТЕЛО
...             ; //      ЦИКЛА
ADD A, xxx [SUB A, xxx]; Модификация счетчика
CMP A, xxx      ; Проверка условия продолжения цикла
```

```
Jxx repeat      ; Переход в начало цикла, если условие истинно
```

Листинг 183

Наконец, самые крутые компиляторы (из которых автор на вскидку может назвать один лишь Microsoft Visual C++ 6.0) могут даже заменять циклы с приращением на циклы с убыванием при условии, что параметр цикла не используется операторами цикла, а лишь прокручивает цикл определенное число раз. Зачем это компилятору? Оказывается, циклы с убыванием гораздо короче - однобайтовая инструкция DEC не только уменьшает операнд, но и выставляет Zero-флаг при достижении нуля. В результате, в команде CMP A, xxx отпадает всякая необходимость.

```
MOV A, xxx      ; Инициализация переменной "счетчика"
repeat:         ; Начало цикла
...             ; // ТЕЛО
...             ; //      ЦИКЛА
DEC A           ; Декремент счетчика
JNZ repeat      ; Повтор, пока A != 0
```

Листинг 184

Таким образом, в зависимости от настроек и характера компилятора, циклы for могут транслироваться и в циклы с предусловием, и в циклы с постусловием, начинающими свое выполнение с проверки условия продолжения цикла. Причем, условие продолжения может инвертироваться в условие завершения, а возрастающий цикл может "волшебным" образом превращаться в убывающий.

Такая неоднозначность затрудняет идентификацию циклов for, - надежно отождествляются лишь циклы, начинающиеся с проверки постусловия (т.к. они не могут быть отображены на do без использования GOTO). Во всех остальных случаях никаких строгих рекомендаций по распознаванию for дать невозможно.

Скажем так: если логика исследуемого цикла синтаксически удобнее выражается через оператор for, то и выражайте ее через for! В противном случае используйте while или do (repeat\until) для циклов с пред- и пост- условием соответственно.

И в заключение пара слов о "кастрированных" циклах - язык Си позволяет опустить инициализацию переменной цикла, условие выхода из цикла, оператор приращения переменной или все это вместе. При этом for вырождается во while, и становится практически неотличимым от него.

::Циклы с условием в середине. Популярные языки высокого уровня непосредственно не поддерживают циклы с условием в середине, хотя необходимость в них возникает достаточно часто. Поэтому, программисты их реализуют на основе уже имеющихся циклов while (while\do) и оператора выхода из цикла break. Например:

```
while(1)                repeat:
{                               ...
    ...                         CMP xxx
    if (условие) break;         Jxx end
    ...                         ...
}                       JMP repeat
                        end:
```

Листинг 185

Компилятор (если он не совсем Осел - Иi в смысле) разворачивает бесконечный цикл в безусловный переход JMP, направленный, естественно назад (ослы генерируют код like - "MOV EAX, 1\CMP EAX,1\JZ repeat"). Безусловный переход, направленный назад, весьма характерен - за исключением бесконечного цикла его может порождать один лишь оператор GOTO, но GOTO уже давно не в моде. А раз у нас есть бесконечный цикл, то условие его завершения может находиться лишь в середине этого цикла (сложные случаи многопоточных защит, модифицирующих из соседнего потока безусловный переход в NOP, мы пока не рассматриваем).

Сделать это будет нетрудно - оператор break транслируется в переход на первую команду, следующую на JMP repeat, а сам break получает управление от ветки IF (условие) - THEN - [ELSE]. Условие ее срабатывания и будет искомым условием завершения цикла. Вот, собственно, и все.

::Циклы с множественными условиями выхода. Оператор break позволяет организовать выход из цикла в любом удобном для программиста месте, поэтому, любой цикл может иметь множество условий выхода беспорядочно разбросанных по его телу. Это ощутимо усложняет анализ дизассемблируемой программы, т.к. возникает риск "прозевать" одно из условий завершения цикла, что приведет к неправильному пониманию логики программы.

Идентифицировать же условия выхода из цикла очень просто - они всегда направлены "вниз" т.е. в область старших адресов и указывают на команду, непосредственно следующую за инструкций условного (безусловного) перехода, направленного "вверх" - в область младших адресов. (см. так же "*Циклы с условием в середине*").

::Циклы с несколькими счетчиками. Оператор "запятая" языка Си позволяет осуществлять множественную инициализацию и модификацию счетчиков цикла for. Например: "for (a=0, b=10; a != b; a++, b--)". А как насчет нескольких условий завершения? И "ветхий" и "новый " заветы (первое и второе издание K&R соответственно), и стандарт ANSI C, и руководства по C, прилагаемые к компиляторам Microsoft Visual C, Borland C, WATCOM C на этот счет хранят "партизанское" гробовое молчание.

Если попробовать скомпилировать следующий код "for (a=0, b=10; a >0, b <10 ; a++, b--)" он будет благополучно "проглочен" практически всеми компиляторами без малейших ругательств с их стороны, но ни один их них не откомпилирует данный пример *правильно*. Логическое условие (a1,a2,a3,...an) лишено смысла и компиляторы без малейших колебаний и зазрений совести отбросят все, кроме самого правого выражения an. Оно-то и будет единолично пределять условие продолжение цикла. Один лишь WATCOM вяло ворчит по этому поводу: *"Warning! W111: Meaningless use of an expression:* the line contains an expression that does nothing useful. In the example "i = (1,5);", the expression "1," is meaningless. This message is also generated for a comparison that is useless"

Если условие продолжения цикла зависит от нескольких переменных, то их сравнения следует объединить в одно выражение посредством логических операций OR, AND и др. Например: "for (a=0, b=10; (a >0 && b <10) ; a++, b--)" - цикл прерывается сразу же, как только одно из двух условий станет ложно; "for (a=0, b=10; (a >0 || b <10); a++, b--)" - цикл продолжается до тех пор, пока истинно хотя бы одно условие из двух.

В остальном же циклы с несколькими счетчиками транслируются аналогично циклам с одним счетчиком, за исключением того, что инициализируется и модифицируется не одна, а сразу несколько переменных.

::Идентификация continue. Оператор continue приводит к непосредственной передаче управления на код проверки условия продолжения (завершения) цикла. В общем случае он транслируется в безусловный jump, в циклах с предусловием направленный вверх, а в циклах в постусловием - вниз. Код, следующий за continue, уже не получает управления, поэтому continue практически всегда используется в условных конструкциях.

Например: "while (a++ < 10) if (a == 2) continue;..." компилируется приблизительно так:

```
repeat:     ; Начало цикла while
INC A       ; a++
CMP A, 10   ; Проверка условия завершения цикла
JAE end     ; Конец, если a >= 10
CMP A,2     ; if (a == 2) ...
JNZ woo     ; Переход к варианту "иначе", если a != 2
JMP repeat  ; <== continue
woo:        ; // ТЕЛО
...         ; //       ЦИКЛА
```

```
JMP repeat  ; Переход в начало цикла
```

Листинг 186

::Сложные условия. До сих пор, говоря об условиях завершения и продолжения цикла, мы рассматривали лишь элементарные условия отношения, в то время как практически все языки высокого уровня допускают использование составных условий. Однако составные условия можно схематично изобразить в виде абстрактного "черного ящика" с входом/выходом и логическим двоичными деревом внутри. Построение и реконструкция логических деревьев подробно рассматриваются в главе *"Идентификация IF - THEN - ELSE"* здесь же нас интересует не сами условия, а организация циклов.

::Вложенные циклы. Циклы - понятное дело - могут быть и вложенными. Казалось бы, какие проблемы? Начало каждого цикла надежно определяется по перекрестной ссылке, направленной вниз. Конец цикла - условный или безусловный переход на его начало. У каждого цикла только одно начло и только один конец (хотя условий выхода может быть сколько угодно, но это - другое дело). Причем, циклы не могут пересекаться - если между началом и концом одного цикла встречается начало другого цикла, то этот цикл - вложенный.

Но не все так просто: тут есть два подводных камня. Первый: *оператор continue в циклах с предусловием*, второй - *сложные условия продолжения цикла с постусловием*. Рассмотрим их подробнее.

Поскольку, continue в циклах с предусловием, транслируется в безусловный переход, направленный "вверх", он становится практически неотличим от конца цикла. Смотрите:

```
while(условие1)
{
    ...
    if (условие2) continue;
    ...
}

транслируется в:

NOT условие1 выхода из цикла---------!  <-!  <-----!
...                                  !    !        !
если НЕ условие2 GOTO continue ---!  !    !        !
безусловный переход в начало ------)--)---!        !
continue:                   <-----!  !             !
...                                  !             !
безусловный переход в начало ---------)-------------!
конец всего <------------------------!
```

Два конца и два начала вполне напоминают два цикла, из которых один вложен в другой. Правда, начала обоих циклов совмещены, но ведь может же такое быть, если в цикл с пост условием вложен цикл с предусловием? На первый взгляд да, но если подумать, то... ай-ай-ай! А ведь *условие1* выхода из цикла прыгает аж за второй конец! Если это предусловие вложенного цикла, то оно прыгало бы за первый конец. А если *условие1* - это предусловие материнского цикла, то конец вложенного цикла не смог бы передать на него управление. Выходит, это не два цикла, а один. А первый "конец" - результат трансляции оператора continue.

С разбором сложных условий продолжения цикла с постусловием дела обстоят еще лучше. Рассмотрим такой пример:

```
do
{
...
} while(условие1 || условие2);

Результат его трансляции в общем случае будет выглядеть так:

...                 <---! <-!
```

```
условие продолжения1 ---!   !
условие прололжения2 -------!

Ну, чем не:

do
{
    do
    {
        ...
    }while(условие1)

}while(условие2)
```

Строго говоря, предложенный вариант является логически верным, но синтаксически некрасивым. Материнский цикл крутит в своем теле один лишь вложенный цикл и не содержит никаких других операторов. Так зачем он тогда, спрашивается, нужен? Объединить его с вложенным циклом в один!

Дизассемблерные листинги примеров. Давайте для закрепления сказанного рассмотрим несколько живых примеров.

Начнем с самого простого - с циклов while\do:

```
#include <stdio.h>

main()
{
    int a=0;
    while(a++<10) printf("Оператор цикла while\n");

    do {
    printf("Оператор цикла do\n");
    } while(--a >0);
}
```

Листинг 187 Демонстрация идентификации циклов while\do

Результат компиляции этого примера компилятором Microsoft Visual C++ 6.0 с настройками по умолчанию должен выглядеть так:

```
main    proc near       ; CODE XREF: start+AF.p

var_a        = dword    ptr -4

    push   ebp
    mov    ebp, esp
    ; Открываем кадр стека

    push   ecx
    ; Резервируем память для одной локальной переменной

    mov    [ebp+var_a], 0
    ; Заносим в переменную var_a значение 0x0

loc_40100B:             ; CODE XREF: main_401000+29.j
;                       ^^^^^^^^^^^^^^
    ; Перекрестная ссылка, направленная вниз, говорит о том, что это начло цикла
    ; Естественно: раз перекрестная ссылка направлена вниз, то переход,
    ; ссылающийся на этот адрес, будет направлен вверх!

    mov    eax, [ebp+var_a]
    ; Загружаем в EAX значение переменной var_a

    mov    ecx, [ebp+var_a]
    ; Загружаем в ECX значение переменной var_a
    ; (недальновидность компилятора - можно было бы поступить и короче MOV ECX,EAX)

    add    ecx, 1
    ; Увеличиваем ECX на единицу
```

```
    mov    [ebp+var_a], ecx
    ; Обновляем var_a

    cmp    eax, 0Ah
    ; Сравниваем старое (до обновления) значение переменной var_a с числом 0xA

    jge    short loc_40102B
    ; Если var_a >= 0xA - прыжок "вперед", непосредственно за инструкцию
    ; безусловного перехода, направленного "назад"
    ; Раз "назад", значит, - это цикл, а, поскольку, условие выхода из цикла
    ; проверяется в его начале, то это цикл с предусловием
    ; Для его отображения на цикл while необходимо инвертировать условие выхода
    ; из цикла на условие продолжения цикла (Т.е. заменить >= на <)
    ; Сделав это, мы получаем:
    ; while (var_a++ < 0xA)...
    ;

    // Начало тела цикла
    push   offset aOperatorCiklaW ; "Оператор цикла while\n"
    call   _printf
    add    esp, 4
    ; printf("Оператор цикла while\n")

    jmp    short loc_40100B
    ; Безусловный переход, направленный назад, на метку loc_40100B
    ; Между loc_40100B и jmp short loc_40100B есть только одно условие
    ; выхода из цикла - jge short loc_40102B, значит, исходный цикл
    ; выглядел так:
    ; while (var_a++ < 0xA) printf("Оператор цикла while\n")

loc_40102B:                ; CODE XREF: main_401000+1A.j
                           ; main_401000+45.j
                           ; ^^^^^^^^^^^^^^^
    ; // Это начало цикла с пост-условием
    ; // Однако на данном этапе мы этого еще не знаем, хотя и можем догадываться
    ; // благодаря наличию перекрестной ссылки, направленной вниз

    ; Ага, никакого условия в начале цикла не присутствует, значит, это цикл
    ; с условием в конце или середине
    push   offset aOperatorCiklaD ; "Оператор цикла do\n"
    call   _printf
    add    esp, 4
    ; printf("Оператор цикла do\n")
    ; // Тело цикла

    mov    edx, [ebp+var_a]
    ; Загружаем в EDX значение переменной var_a

    sub    edx, 1
    ; Уменьшаем EDX на единицу

    mov    [ebp+var_a], edx
    ; Обновляем переменную var_a

    cmp    [ebp+var_a], 0
    ; Сравниваем переменную var_a с нулем

    jg     short loc_40102B
    ; Если var_a > 0, то переход в начало цикла
    ; Поскольку, условие расположено в конце тела цикла, этот цикл - do:
    ; do printf("Оператор цикла do\n"); while (--a > 0)
    ;
    ; // Для повышения читабельности дизассемблерного текста рекомендуется
    ; // заменить префиксы loc_ в начале цикла на while и do (repeat) в циклах
    ; // с пред- и пост- условием соответственно

    mov    esp, ebp
    pop    ebp
    ; Закрываем кадр стека
    retn
main    endp
```

Листинг 188

Совсем другой результат получится если включить оптимизацию. Откомпилируем тот же самый пример с ключом "/Ox" (максимальная оптимизация) и посмотрим на результат, выданный компилятором:

```
main        proc near        ; CODE XREF: start+AF.p
    push   esi
    push   edi
    ; Сохраняем регистры в стеке

    mov    esi, 1
    ; Присваиваем ESI значение 0x1
    ; Внимание - взгляните на исходный код - ни одна из переменных не имела
    ; такого значения!

    mov    edi, 0Ah
    ; Присваиваем EDI значение 0xA. Ага, это константа для проверки условия
    ; выхода из цикла

loc_40100C:                ; CODE XREF: main+1D.j
                           ; ^^^^^^^^^^^^^^^^^^^
    ; Судя по перекрестной ссылке, направленной вниз, этот - цикл!

    push   offset aOperatorCiklaW ; "Оператор цикла while\n"
    call   _printf
    add    esp, 4
    ; printf("Оператор цикла while\n")
    ; ...тело цикла while? (растерянно так)
    ; Постой, постой! А где же предусловие?!

    dec    edi
    ; Уменьшаем EDI на один

    inc    esi
    ; Увеличиваем ESI на один

    test    edi, edi
    ; Проверяем EDI на равенство нулю

    ja     short loc_40100C
    ; Переход в начало цикла, пока EDI != 0
    ; Так... (задумчиво) Компилятор в порыве оптимизации превратил неэффективный
    ; цикл с предусловием в более компактный и быстрый цикл с пост-условием
    ; Имел ли он на это право? А почему нет?! Проанализировав код, компилятор понял
    ; что данный цикл выполняется, по крайней мере, один раз, следовательно,
    ; скорректировав условие продолжения, его проверку можно вынести в конец цикла
    ; Поэтому-то начальное значение переменной цикла равно единице, а не нулю!
    ; Т.е. while ((int a = 0) < 10) компилятор заменил на
    ; do ... while (((int a = 0)+1) < 10) ==
    ; do ... while ((int a=1) < 10)
    ;
    ; Причем, что интересно, он не сравнивал переменную цикла с константой,
    ; а поместил константу в регистр и уменьшал его до тех пор, пока тот не стал
    ; равен нулю! Зачем? А затем, что так короче, да и работает быстрее
    ; Что ж, это все хорошо, но как нам декомпилировать этот цикл?
    ; Непосредственное отображение на язык Си дает следующую конструцию:
    ; var_ ESI = 1; var _EDI = 0xA;
    ; do {
    ;;printf("Оператор цикла while\n"); var_EDI--; var_ESI++;
    ; } while(var_EDI > 0)
    ;
    ; Правда, коряво и запутано? Что-ж, тогда попытаемся избавится от одной
    ; из двух переменных. Это действительно возможно, т.к. они модифицируются
    ; синхронно, и var_EDI = 0xB - var_ESI
    ; OK, выполняем подстановку:
    ; var_ ESI = 1; var _EDI = 0xB - var_ESI ; (== 0xA;)
    ; do {
    ;;printf("Оператор цикла while\n"); var_EDI--; var_ESI++;
    ;                                   ^^^^^^^^^^
```

```
    ; Это мы вообще сокращаем, т.к. var_EDI уже выражена через var_ESI
    ; } while((0xB - var_ESI) > 0); (== var_ESI > 0xB)
    ;
    ; Что, ж уже получается нечто осмысленное:
    ;
    ; var_ ESI = 1; var _EDI == 0xA;
    ; do {
    ;;    printf("Оператор цикла while\n"); var_ESI++;
    ; } while(var_ESI > 0xB)
    ; На этом можно и остановится, а можно и пойти дальше, преобразовав цикл
    ; с пост-условием в более наглядный цикл с предусловием
    ;
    ; var_ ESI = 1; var _EDI == 0xA; <-- var_EDI не используется,
    ;можно сократить
    ; while (var_ESI < = 0xA) {
    ;;    printf("Оператор цикла while\n"); var_ESI++;
    ; }
    ; Но и это не предел выразительности: во-первых var_ESI < = 0xA эквивалентно
    ; var_EDI < 0xB, а во-вторых, поскольку, переменная var_ESI используется лишь
    ; как счетчик, ее начальное значение можно безбоязненно привести к нулевому
    ; значению, а операцию инкремента внести в сам цикл:

    ; var_ ESI = 0;
    ; while (var_ESI++ < 0xA) <-- вычитаем единицу из левой
    ; и правой половины
    ; printf("Оператор цикла while\n");
    ;
    ; Ну, разве не красота?! Сравните этот вариант с первоначальным -
    ; насколько он стал яснее и понятнее

loc_40101F:                ; CODE XREF: main+2F.j
;                           ^^^^^^^^^^^^^^^^^^^
; Перекрестная ссылка, направленная вниз, говорит о том, что это - начало цилка

    ; // Предусловия нет - значит, это цикл do

    push   offset aOperatorCiklaD ; "Оператор цикла do\n"
    call   _printf
    add    esp, 4
    ; printf("Оператор цикла do\n");

    dec    esi
    ; Уменьшаем var_ESI

    test    esi, esi
    ; Проверка ESI на равенство нулю

    jg     short loc_40101F
    ; Продолжать цикл, пока var_ESI > 0
    ;
    ; OK. Этот цикл легко и непринужденно отображается на язык Си:
    ; do printf("Оператор цикла do\n"); while (--var_ESI > 0 )

    pop    edi
    pop    esi
    ; Восстанавливаем сохраненные регистры

    retn
main        endp
```

Листинг 189

Несколько иначе оптимизирует циклы компилятор Borland C++ 5.x. Смотрите:

```
_main        proc near        ; DATA XREF: DATA:00407044.o

    push   ebp
    mov    ebp, esp
    ; Открываем кадр стека

    push    ebx
```

```
    ; Сохраняем EBP в стеке

    xor    ebx, ebx
    ; Присваиваем регистровой переменной EBX значение ноль
    ; Как легко догадаться - EBX и есть "a"

    jmp    short loc_40108F
    ; Безусловный прыжок вниз. Очень похоже на цикл for...

loc_401084:                ; CODE XREF: _main+19.j
;                           ^^^^^^^^^^^^^^^^^^^^
; Перекрестная ссылка, направленная вниз - значит, это начало какого-то цикла

    push   offset aOperatorCiklaW ; "Оператор цикла while\n"
    call   _printf
    pop    ecx
    ; printf("Оператор цикла while\n")

loc_40108F:                ; CODE XREF: _main+6.j
    ; А вот сюда был направлен самый первый jump
    ; Посмотрим: что же это такое?

    mov    eax, ebx
    ; Копирование EBX в EAX

    inc    ebx
    ; Увеличение EBX

    cmp    eax, 0Ah
    ; Сравнение EAX со значением 0xA

    jl     short loc_401084
    ; Переход в начало цикла, если EAX < 0xA
    ; Вот так-то Borland оптимизировал код! Он расположил условие в конце цикла,
    ; но, чтобы не транслировать цикл с предусловием в цикл с постусловием,
    ; просто начал выполнение цикла с этого самого условия!
    ;
    ; Отображение этого цикла на язык Си дает:

    ; for (int a=0; a < 10; a++) printf("Оператор цикла while\n")
    ;
    ; и, хотя подлинный цикл выглядел совсем не так, наш вариант нечем не хуже!
    ; (а может даже и лучше - нагляднее)

loc_401097:                ; CODE XREF: _main+29.j
;                           ^^^^^^^^^^^^^^^^^^^^
    ; Начало цикла!

    ; Условия нет - значит, это цикл с постусловием

    push   offset aOperatorCiklaD ; "Оператор цикла do\n"
    call   _printf
    pop    ecx
    ; printf("Оператор цикла do\n")

    dec    ebx
    ; --var_EBX

    test    ebx, ebx
    jg     short loc_401097
    ; Продолжать цикл, пока var_EBX > 0
    ; do printf("Оператор цикла do\n"); while (--var_EBX > 0)

     xor    eax, eax
    ; return 0

    pop    ebx
    pop    ebp
    ; Восстанавливаем сохраненные регистры

    retn
```

```
_main        endp
```

Листинг 190

Остальные компиляторы генерируют аналогичный или даже еще более примитивный и очевидный код, поэтому не будем подробно их разбирать, а лишь кратно опишем используемые ими схемы трансляции.

Компилятор Free Pascal 1.x ведет себя аналогично компилятору Borland C++ 5.0, всегда помещая условие в конец цикла и начиная с него выполнение while-циклов.

Компилятор WATCOM C не умеет преобразовывать циклы с предусловием в циклы с постусловием, вследствие чего располагает условие выхода из цикла в начале while-циклов, а в их конец вставляет безусловный jump. (Классика!)

Компилятор GCC вообще не оптимизирует циклы с предусловием, генерируя самый неоптимальный код. Смотрите:

```
      mov    [ebp+var_a], 0
      ; Присвоение переменной a значения 0

      mov    esi, esi
      ; Э... на редкость умный код! При его виде трудно не упасть со стула!

  loc_401250:                ; CODE XREF: sub_40123C+34.j
  ;                          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  ; Начало цикла

      mov    eax, [ebp+var_a]
      ; Загрузка в EAX значения переменной var_a

      inc    [ebp+var_a]
      ; Увеличение var_a на единицу

      cmp    eax, 9
      ; Сравнение EAX со значением 0x9

      jle    short loc_401260
      ; Переход, если EAX <= 0x9 (EAX < 0xA)

      jmp    short loc_401272
      ; Безусловный переход в конец цикла
      ; Стало быть, предыдущий условный переход - переход на его продолжение
      ; Какой неоптимальный код! Зато нет инверсии условия продолжения цикла,
      ; что упрощает дизассемблирование

      align 4
      ; Выравнивание перехода по адресам, кратным четырем, ускорят код, но заметно
      ; увеличивает его размер (особенно, если переходов очень много)

  loc_401260:                ; CODE XREF: sub_40123C+1D.j
      add    esp, 0FFFFFFF4h
      ; Вычитание из ESP значения 12 (0xC)

      push   offset aOperatorCiklaW ; "Оператор цикла while\n"
      call    printf
      add    esp, 10h
      ; Восстанавливаем стек (0xC + 0x4 ) == 0x10

      jmp    short loc_401250
      ; Переход в начало цикла

  loc_401272:
      ; Конец цикла
```

Листинг 191

Разобравшись с while\do, перейдем к циклам for. Рассмотрим следующий пример:

```
#include <stdio.h>

main()
{
    int a;
    for (a=0;a<10;a++)    printf("Оператор цикла for\n");
}
```

Листинг 192 Демонстрация идентификации циклов for

Результат компиляции Microsoft Visual C++ 6.0 с настройками по умолчанию будет выглядеть так:

```
main        proc near        ; CODE XREF: start+AF.p

var_a        = dword    ptr -4

    push   ebp
    mov    ebp, esp
    ; Открываем кадр стека

    push   cx
    ; Резервируем память для локальной переменной

    mov    [ebp+var_a], 0
    ; Присваиваем локальной переменной var_a значение 0

    jmp    short loc_401016
    ; Непосредственный переход на код проверки условия продолжения цикла -
    ; характерный признак for

loc_40100D:                ; CODE XREF: main+29.j
;                            ^^^^^^^^^^^^^^^^^^^
; Перекрестная ссылка, направленная вниз говорит о том, что это начало цикла

    mov    eax, [ebp+var_a]
    ; Загрузка в EAX значения переменной var_a

    add    eax, 1
    ; Увеличение EAX на единицу

    mov    [ebp+var_a], eax
    ; Обновление EAX
    ; Следовательно, исходный код выглядел так:
    ; ++a

loc_401016:                ; CODE XREF: main+B.j
    cmp    [ebp+var_a], 0Ah
    ; Сравниваем var_a со значением 0xA

    jge    short loc_40102B
    ; Выход из цикла, если var_a >= 0xA

    push    offset aOperatorCiklaF ; "Оператор цикла for\n"
    call    _printf
    add     esp, 4
    ; printf("Оператор цикла for\n")

    jmp    short loc_40100D
    ; Безусловный переход в начало цикла
    ;
    ; Итак, что мы имеем?
    ; инициализация переменной var_a
    ; переход на проверку условия выхода из цикла ----------!
    ; инкремент переменной var_a <---------------------!    !
    ; проверка условия относительно var_a <------------ ! ---!
    ; прыжок на выход из цикла, если условие истинно---!----!
    ; вызов printf                                    !    !
    ; переход в начало цикла --------------------------!    !
    ; конец цикла <-----------------------------------------!
    ;
    ; Проверка на завершения, расположенная в начале цикла, говорит о том, что
```

```
    ; это цикл с предусловием, но непосредственно выразить его через while
    ; не удается - мешает безусловный переход в середину цикла, минуя код
    ; инкремента переменной var_a
    ; Однако этот цикл с легкостью отображается на оператор for, смотрите:
    ; for (a = 0; a < 0xA; a++) printf("Оператор цикла for\n")
    ;
    ; Действительно, цикл for сначала инициирует переменную - счетчик,
    ; затем проверяет условие продолжение цикла
    ; (оптимизируемое компилятором в условие завершение), далее выполняет
    ; оператор цикла, модифицирует счетчик, вновь проверяет условие и т.д.
    ;

loc_40102B:                ; CODE XREF: main+1A.j
    mov    esp, ebp
    pop    ebp
    ; Закрываем кадр стека

    retn
main        endp
```

Листинг 193

А теперь задействуем оптимизацию и посмотрим, как видоизмениться наш цикл:

```
main        proc near        ; CODE XREF: start+AF.p
    push   esi
    mov    esi, 0Ah
    ; Инициализируем переменную - счетчик
    ; Внимание! В исходном коде начальное значение счетчика равнялось нулю!

loc_401006:                ; CODE XREF: main+14.j
    push   offset aOperatorCiklaF ; "Оператор цикла for\n"
    call   _printf
    add    esp, 4
    ; printf("Оператор цикла for\n")
    ; Выполняем оператор цикла! Причем безо всяких проверок!
    ; Хитрый компилятор проанализировал код и понял, что цикл выполняется
    ; по крайней мере один раз!

    dec    esi
    ; Уменьшаем счетчик, хотя в исходном коде программы мы его увеличивали!
    ; Ну, правильно - dec \ jnz намного короче INC\ CMP reg, const\ jnz xxx
    ; Ой и мудрит компилятор! Кто же ему давал право так изменять цикл?!
    ; А очень просто - он понял, что параметр цикла в самом цикле используется
    ; только как счетчик, и нет никакой разницы - увеличивается он
    ; с каждой итерацией или уменьшается!

    jnz    short loc_401006
    ; Переход в начало цикла если ESI > 0
    ;
    ; М да, по внешнему виду это типичный
    ; a = 0xa; do printf("Оператор цикла for\n"); while (--a)
    ;
    ; Если вас устраивает читабельность такой формы записи - оставляйте ее, а нет:
    ; for (a = 0; a < 10; a++) Оператор цикла for\n")
    ;
    ; Постой, постой! На каком основании автор выполнил такое преобразование?!
    ; А на том самом - что и компилятор: раз параметр цикла используется только
    ; как счетчик, законна любая запись, выполняющая цикл ровно десять раз -
    ; остается выбрать ту, которая удобнее (с эстетической точки зрения)
    ; Никто же не будет утверждать, что
    ; for (a = 10; a > 0; a--) более привычно чем for (a = 0; a < 10; a++)?

    pop    esi
    retn
main        endp
```

Листинг 194

А что скажет нам товарищ Borland C++ 5.0? Компилируем и смотрим:

```
_main        proc near        ; DATA XREF: DATA:00407044.o
```

```
    push   ebp
    mov    ebp, esp
    ; Открываем кадр стека

    push   ebx
    ; Сохраняем EBX в стеке

    xor    ebx, ebx
    ; Присваиваем регистровой переменной EBX значение 0

loc_401082:                ; CODE XREF: _main+15.j
;                           ^^^^^^^^^^^^^^^^^^^^^
; Начало цикла

    push   offset aOperatorCiklaF ; format
    call   _printf
    pop    ecx
    ; Начинаем цикл с выполнения его тела
    ; OK, Borland понял, что цикл выполняется по крайней мере раз

    inc    ebx
    ; Увеличиваем параметр цикла

    cmp    ebx, 0Ah
    ; Сравниваем EBX со значением 0xA

    jl     short loc_401082
    ; Переход в начало цикла, пока EBX < 0xA

    xor    eax, eax
    pop    ebx
    pop    ebp
    retn
_main        endp
```

Листинг 195

Видно, что Borland C++ 5.0 не дотягивает до Microsoft Visual C++ 6.0 - понять, что цикл выполняется один раз он понял, а вот реверс счетчика ума уже не хватило. Аналогичным образом поступает и большинство других компиляторов, в частности WATCOM C.

Теперь настала очередь циклов с условием в середине или циклов, завершаемых вручную оператором break. Рассмотрим следующий пример:

```
#include <stdio.h>

main()
{
    int a=0;
    while(1)
    {
        printf("1й оператор\n");
        if (++a>10) break;
        printf("2й оператор\n");
    }

    do
    {
    printf("1й оператор\n");
    if (--a<0) break;
    printf("2й оператор\n");
    }while(1);
}
```

Листинг 196 Демонстрация идентификации break

Результат компиляции Microsoft Visual C++ 6.0 с настройками по умолчанию должен выглядеть так:

```
main        proc near        ; CODE XREF: start+AF.p
var_a        = dword    ptr -4
```

```
    push   ebp
    mov    ebp, esp
    ; Открываем кадр стека

    push   ecx
    ; Резервируем место для локальной переменной

    mov    [ebp+var_a], 0
    ; Присваиваем переменной var_a значение 0x0

loc_40100B:                ; CODE XREF: main+3F.j
;                           ^^^^^^^^^^^^^^^^^^^^
; Перекрестная ссылка, направленная вниз - цикл

    mov    eax, 1
    test   eax, eax
    jz     short loc_401041
    ; Смотрите! Когда optimize disabled, - компилятор транслирует безусловный
    ; цикл "слишком буквально", т.к. присваивает EAX значение 1 (TRUE)
    ; и затем педантично проверяет ее на равенство нулю
    ; Если в кои веки TRUE будет равно FALSE - произойдет выход из цикла
    ; Словом, все эти три инструкции - глупый и бесполезный код цикла
    ; while (1)

    push   offset a1iOperator ; "1й оператор\n"
    call   _printf
    add    esp, 4
    ; printf("1й оператор\n")

    mov    ecx, [ebp+var_a]
    ; Загружаем в ECX значение переменной var_a

    add    ecx, 1
    ; Увеличивем ECX на единицу

    mov    [ebp+var_a], ecx
    ; Обновляем var_a

    cmp    [ebp+var_a], 0Ah
    ; Сравниваем var_a со значением 0xA

    jle    short loc_401032
    ; Переход, если var_a <= 0xA
    ; Но куда этот переход? Во-первых, переход направлен вниз, т.е. это уже
    ; не переход к началу цикла, следовательно и условие - не условие цикла, а
    ; результат компиляции конструкции IF - THEN
    ; Второе - переход прыгает на первую команду, следующую за инструкцией
    ; безусловного перехода, направленного вверх - в начало цикла
    ; Следовательно, jmp    short loc_401041 осуществляет выход из цикла, а
    ; jle short loc_401032 - продолжает его выполнение

    jmp    short loc_401041
    ; OK, - это переход на завершение цикла. А кто у нас завершает цикл?
    ; Ну, конечно же, break! Следовательно, окончательная декомпиляции выглядит так
    ; if (++var_a > 0xA) break
    ; Мы инвертировали "<=" в ">", т.к. JLE передает управление на код продолжения
    ; цикла, а ветка THEN в нашем случае - на break

loc_401032:                ; CODE XREF: main+2E.j
;                           ^^^^^^^^^^^^^^^^^^^^
; Перекрестная ссылка направлена вверх - следовательно, это не начало цикла

    push   offset a2iOperator ; "2й оператор\n"
    call   _printf
    add    esp, 4
    ; printf("2й оператор\n")

    jmp    short loc_40100B
    ; Прыжок в начало цикла. Вот мы и добрались до конца цикла
    ; Восстанавливаем исходный код:
    ; while(1)
```

```
    ; {
    ;     printf("1й оператор\n");
    ;     if (++var_a > 0xA) break;
    ;     printf("2й оператор\n");
    ; }
    ;

loc_401041:                ; CODE XREF: main+12.j main+30.j ...
;                                          ^^^^^^^^^^
; Перекрестная ссылка, направленная вниз, говорит, что это начало цикла

    push   offset a1iOperator_0 ; "1й оператор\n"
    call   _printf
    add    esp, 4
    ; printf("1й оператор\n")

    mov    edx, [ebp+var_a]
    sub    edx, 1
    mov   [ebp+var_a], edx
    ; --var_a

    cmp    [ebp+var_a], 0
    ; Сравниваем var_a со значением 0x0

    jge    short loc_40105F
    ; Переход вниз, если var_a >= 0
    ; Смотрите: оператор break цикла do ничем не отличается от break цикла while!
    ; Поэтому, не будем разглагольствовать, а сразу его декомпилируем!
    ; if (var_a < 0) ...

    jmp    short loc_401075
    ; ...break

loc_40105F:                ; CODE XREF: main+5B.j
    push   offset a2iOperator_0 ; "2й оператор\n"
    call   _printf
    add    esp, 4
    ; printf("2й оператор\n")

    mov    eax, 1
    test   eax, eax
    jnz    short loc_401041
    ; А это - проверка продолжения цикла

loc_401075:                ; CODE XREF: main+5D.j
    mov    esp, ebp
    pop    ebp
    ; Закрываем кадр стека

     retn
main        endp
```

Листинг 197

Что ж, оператор break в обоих циклах выглядит одинаково и элементарно распознается (правда, не с первого взгляда, но отслеживанием нескольких переходов - да). А вот с бесконечными циклами не оптимизирующий компилятор подкачал, транслировав их в код, проверяющий условие, истинность (не истинность) которого очевидна. А как поведет себя оптимизирующий компилятор?

Давайте откомпилируем тот же самый пример компилятором Microsoft Visual C++ 6.0 с ключом "/Ox" и посмотрим:

```
main        proc near        ; CODE XREF: start+AF.p
    push    esi
    ; Сохраняем ESI в стеке

    xor     esi, esi
    ; Присваиваем ESI значение 0
    ; var_ESI = 0;
loc_401003:                ; CODE XREF: main+23.j
```

```
;                           ^^^^^^^^^^^^^^^^^^^^
; Перекрестная ссылка, направленная вперед
; Это - начало цикла

    push   offset a1iOperator ; "1й оператор\n"
    call   _printf
    add    esp, 4
    ; printf("1й оператор\n")
    ;
    ; Ага! Проверки на дорогах нет, значит, это цикл с постусловием
    ; (или условием в середине)

    inc    esi
    ; ++var_ESI

    cmp    esi, 0Ah
    ; Сравниваем var_ESI со значением 0xA

    jg     short loc_401025
    ; Выход из цикла, если var_ESI > 0xA
    ; Поскольку, данная команда - не последняя в теле цикла,
    ; это цикл с условием в середине
    ; if (var_ESI > 0xA) break

    push   offset a2iOperator ; "2й оператор\n"
    call   _printf
    add    esp, 4
    ; printf("2й оператор\n")

    jmp    short loc_401003
    ; Безусловный переход в начало цикла
    ; Как видно, оптимизирующий компилятор выкинул никому ненужную проверку
    ; условия, упростив код и облегчив его понимание:
    ; Итак:
    ; var_ESI = 0
    ; for (;;)     <-- вырожденный for представляет собой бесконечный цикл
    ; {
    ;     printf("1й оператор\n");
    ;     ++var_ESI;
    ;     if (var_ESI > 0xA) break;
    ;     printf("2й оператор\n");
    ; }

loc_401025:                ; CODE XREF: main+14.j
;                           ^^^^^^^^^^^^^^^^^^^^
; Это не начало цикла!

    push   offset a1iOperator_0 ; "1й оператор\n"
    call    _printf
    add    esp, 4
    ; printf("1й оператор\n")
    ; Хм, как же это не начало цикла?! Очень похоже!

    dec    esi
    ; --var_ESI

    js     short loc_401050
    ; Выход из цикла, если var_ESI < 0

    inc    esi
    ; Увеличиваем var_ESI на единицу
    ; М-м-м... (задумчиво)...

loc_401036:                ; CODE XREF: main+4E.j
;                           ^^^^^^^^^^^^^^^^^^^^
; А вот это начало цикла!

    push   offset a2iOperator_0 ; "2й оператор\n"
    call   _printf
    ; printf("2й оператор\n")
    ; Только странно, что начало цикла начинается с его, с позволения сказать,
```

```
    ; середины...

    push   offset a1iOperator_0 ; "1й оператор\n"
    call   _printf
    add    esp, 8
    ; printf("1й оператор\n")
    ;
    ; ???!!! Что за чудеса творятся? Во-первых, вызов первого оператора второго
    ; цикла уже встречался ранее, во-вторых, не может же следом за серединой цикла
    ; следовать его начало?!

    dec    esi
    ; --var_ESI

    jnz    short loc_401036
    ; Продолжение цикла, пока var_ESI != 0

loc_401050:                ; CODE XREF: main+33.j
    ; Конец цикла
    ; Да... тут есть над чем подумать!
    ; Компилятор нормально "перевалил" первую строку цикла
    ; printf("1й оператор\n")
    ; а затем "напоролся" на ветвление:
    ; if (--a<0) break
    ; Хитрые парни из Microsoft знают, что для супер - конвейерных процессоров
    ; (коими и являются чипы Pentium) ветвления все равно, что чертополох для
    ; Тиггеров. Кстати, Си-компиляторы под процессоры серии CONVEX вообще
    ; отказываются компилировать циклы с ветвлениями, истощенно понося
    ; умственные способности программистов. А вы еще IBM PC ругаете ;-)
    ; Вот и приходится компилятору исправлять ляпы программиста, что он делать
    ; в принципе не обязан, но за что ему большое человеческое спасибо!
    ; Компилятор как бы "прокручивает" цикл, "слепляя" вызовы функций printf
    ; и вынося ветвления в конец
    ; Образно исполняемый код можно представить трассой, а процессор - гонщиком
    ; Чем длиннее участок дороги без поворотов, тем быстрее его проскочит гонщик!
    ; Выносить условие из середины цикла в его конец компилятор вполне правомерен,
    ; ведь переменная, относительно которой выполняется ветвление,
    ; не модифицируется ни функцией printf, ни какой другой
    ; Поэтому, не все ли равно где ее проверять? Конечно же не все равно!!!
    ; К моменту когда условие (--a < 10) становится истинно, успевает выполниться
    ; первый printf, а вот второй - уже не получает управления
    ; Вот для этого-то компилятор и поместил код проверки условия следом за
    ; первым вызовом первой функции printf, а затем изменил порядок вызова
    ; printf в теле цикла. Это привело к тому, что на момент выхода из цикла
    ; по условию первый printf выполняется на один раз больше, чем второй
    ; (т.к. он встречается дважды)
    ; Остается разобраться с увеличением var_ESI - что бы это значило?
    ; Давайте рассуждать от противного: что произойдет, если выкинуть
    ; команду INC ESI? Поскольку, счетчик цикла при первой итерации цикла
    ; декрементируется дважды, возникнет недостача и цикл выполниться на раз
    ; короче. Что бы этого не произошло, var_ESI искусственно увеличивается
    ; на единицу
    ; Ой, и не просто во всей этой головоломке разобраться, а представьте:
    ; насколько сложно реализовать компилятор, умеющий проделывать такие фокусы!
    ; А еще кто-то ругает автоматическую оптимизацию. Да уж! Конечно, руками-то
    ; можно и круче оптимизировать(особенно понимания смысл кода), но ведь эдак
    ; и мозги вывихнуть будет можно! А компилятор, даже будучи стиснут со всех
    ; сторон кривым кодом программиста, за доли секунды успевает его довольно
    ; прилично окультурить

    pop    esi
    retn
main        endp
```

Листинг 198

Компиляторы Borland C++ и WATCOM при трансляции бесконечных циклов заменяют код проверки условия продолжения цикла на безусловный переход, но вот, увы, оптимизировать ветвления, вынося их в конец цикла так, как это делает Microsoft Visual C++ 6.0 они не умеют...

Теперь, после break, рассмотрим: как компиляторы транслирует его "астральный антипод", - оператор continue. Возьмем следующий пример:

```
#include <stdio.h>

main()
{
    int a=0;
    while (a++<10)
    {
        if (a == 2) continue;
        printf("%x\n",a);
    }

    do
    {
        if (a == 2) continue;
        printf("%x\n",a);
    } while (--a>0);
}
```

Листинг 199 Демонстрация идентификации continue

Результат его компиляции компилятором Microsoft Visual C++ 6.0 с настройками по умолчанию будет выглядеть так:

```
main        proc near        ; CODE XREF: start+AF.p

var_a        = dword    ptr -4

    push   ebp
    mov    ebp, esp
    ; Открываем кадр стека

    push   ecx
    ; Резервируем место для локальной переменной

    mov    [ebp+var_a], 0
    ; Присваиваем локальной переменной var_a значение 0

loc_40100B:                ; CODE XREF: main+22.j main+35.j
;                                       ^^^^^^^^^^^^^^^^^^^
; Две перекрестные ссылки, направленные вперед, говорят о том, что это либо
; начало двух циклов (один из которых - вложенный), либо переход в начало
; цикла оператором continue

    mov    eax, [ebp+var_a]
    ; Загружаем в EAX значение var_a

    mov    ecx, [ebp+var_a]
    ; Загружаем в ECX значение var_a

    add    ecx, 1
    ; Увеличиваем ECX на единицу

    mov    [ebp+var_a], ecx
    ; Обновляем переменную var_a

    cmp    eax, 0Ah
    ; Сравниваем значение переменной var_a до увеличения с числом 0xA

    jge    short loc_401037
    ; Выход из цикла (переход на команду, следующую за инструкцией, направленной
    ; вверх - в начало цикла) если var_a >= 0xA

    cmp   [ebp+var_a], 2
    ; Сравниваем var_a со значением 0x2

    jnz    short loc_401024
    ; Если var_a != 2, то прыжок на команду, следующую за инструкцией
    ; безусловного перехода, направленной вверх - в начало цикла
```

```
    ; Очень похоже на условие выхода из цикла, но не будет спешить с выводами!
    ; Вспомним - в начале цикла нам встретились две перекрестные ссылки
    ; Безусловный переход "jmp short loc_40100B" как раз образует одну из них
    ; А кто "отвечает" за другую?
    ; Чтобы ответить на этот вопрос необходимо проанализировать остальной код цикла

    jmp    short loc_40100B
    ; Безусловный переход, направленный в начало цикла - это либо конец цикла,
    ; либо continue
    ; Предположим, что это конец цикла. Тогда что же представляет собой
    ; "jge short loc_401037"? Предусловие выхода из цикла? Не похоже - в таком
    ; случае они прыгало бы гораздо "ближе" - на метку loc_401024
    ; А может, "jge short loc_401037" предусловие одного цикла, а
    ; "jnz short loc_401024" - постусловие другого, вложенного в него?
    ; Вполне возможно, но маловероятно - в этом случае постусловие представляло бы
    ; собой условие продолжения, а не завершения цилкла
    ; Поэтому, с некоторой долей неуверенности, мы можем принять конструкцию
    ; CMP var_a, 2 \ JNZ loc_401024 \ JMP loc_40100B за if (a==2) continue

loc_401024:                ; CODE XREF: main+20.j
    mov    edx, [ebp+var_a]
    push   edx
    push   offset asc_406030 ; "%x\n"
    call   _printf
    add    esp, 8
    ; printf("%x\n",var_a)

    jmp    short loc_40100B
    ; А вот это - явно конец цикла, т.к. jmp short loc_40100B - самая
    ; последняя ссылка на начало цикла
    ; Итак, подытожим, что мы имеем:
    ; Условие, расположенное в начале цикла, крутит этот цикл до тех пор, пока
    ; var_a < 0xA, причем инкремент параметра цикла происходит до его сравнения
    ; Затем следует еще одно условие, возвращающее управление в начало цикла, если
    ; var_a == 2. Строй замыкает оператор цикла printf и безусловный переход в его
    ; начало. Т.е.
    ;
    ; Начало цикла:             <-----------! <--!
    ; Инкремент переменной var_a            !    !
    ; условие "далекого" выхода -------!    !    !
    ; условие "ближнего" продолжения --)----!    !
    ; тело цикла                       !         !
    ; безусловный переход в начало ----)---------!
    ; конец цикла                <----!
    ;
    ; Условие "ближнего" продолжение не может быть концом цикла, т.к. тогда условию
    ; "далекого" выхода пришлось выйти аж из надлежащего цикла, на что ни break,
    ; ни другие операторы не способны. Таким образом, условие ближнего продолжения
    ; может быть только оператором continue и на языке Си всю эту конструкция
    ; будет выглядеть так:
    ; while(a++<10)               // <-- инкремент var_a и условие далекого выхода
    ; {
    ;     if (a == 2) continue;    // <-- условие ближнего продолжения
    ;     printf(%x\n",var_a);     // <-- тело цикла
    ; }                            // <-- безусловный переход на начало цикла

loc_401037:                ; CODE XREF: main+1A.j main+5D.j
;                                           ^^^^^^^^^
; Начало цикла

    cmp    [ebp+var_a], 2
    ; Сравниваем переменную var_a со значением 0x2

    jnz    short loc_40103F
    ; Если var_a != 2, то продолжение цикла

    jmp    short loc_401050
    ; Переход к коду проверки условия продолжения цикла
    ; Это бесспорно "continue" и вся конструкция выглядит так:
    ; if (a==2) continue
loc_40103F:                ; CODE XREF: main+3B.j
```

```
    mov    eax, [ebp+var_a]
    push   eax
    push   offset asc_406034 ; "%x\n"
    call   _printf
    add    esp, 8
    ; printf("%x\n", var_a)

loc_401050:                    ; CODE XREF: main+3D.j
    mov    ecx, [ebp+var_a]
    sub    ecx, 1
    mov    [ebp+var_a], ecx
    ; --var_a

    cmp    [ebp+var_a], 0
    ; Сравнение var_a с нулем

    jg    short loc_401037
    ; Пока var_a > 0 продолжать цикл. Похоже на постусловие верно? Тогда:
    ; do
    ; {
    ;     if (a==2) continue;
    ;     printf("%x\n", var_a);
    ; } while (--var_a > 0);
    ;
    mov    esp, ebp
    pop    ebp
    retn
main        endp
```

Листинг 200

А теперь посмотрим, как повлияла оптимизация ("/Ox") на вид циклов:

```
main        proc near        ; CODE XREF: start+AF.p
    push   esi
    mov    esi, 1

loc_401006:                    ; CODE XREF: main+1F.j
;                                ^^^^^^^^^^^^^^^^^^^
; Начало цикла

    cmp    esi, 2
    jz     short loc_401019
    ; Переход на loc_401019, если ESI == 2

    push   esi
    push   offset asc_406030 ; "%x\n"
    call   _printf
    add     esp, 8
    ; printf("%x\n", ESI)
    ; Прим: эта ветка выполняется только если ESI !=2
    ; Следовательно, ее можно изобразить так:
    ; if (ESI != 2) printf("%x\n", ESI)

loc_401019:                ; CODE XREF: main+9.j
    mov    eax, esi
    inc    esi
    ; ESI++;

    cmp    eax, 0Ah
    jl     short loc_401006
    ; Продолжение цикла пока (ESI++ < 0xA)
    ; Итого:
    ; do
    ; {
    ;     if (ESI != 2) printf("%x\n", ESI);
    ; } while (ESI++ < 0xA)
    ;
    ; А что, выглядит вполне читабельно, не правда ли? Ни чуть не хуже, чем
    ; if (ESI == 2) continue
    ;
```

```
loc_401021:                ; CODE XREF: main+37.j
;                                       ^^^^^^^^
; Начало цикла

    cmp    esi, 2
    jz     short loc_401034
    ; Переход на loc_401034, если ESI == 2

    push   esi
    push   offset asc_406034 ; "%x\n"
    call   _printf
    add    esp, 8
    ; printf("%x\n",ESI);
    ; Прим. эта ветка выполняется лишь когда ESI != 2

loc_401034:                ; CODE XREF: main+24.j
    dec    esi
    ; --ESI

    test   esi, esi
    jg     short loc_401021
    ; Условие продолжение цикла - крутить кака ESI > 0
    ; Итого:
    ; do
    ; {
    ;     if (ESI != 2)
    ;    {
    ;        printf("%x\n", ESI);
    ;     }

    ; } while (--ESI > 0)
    ;

    pop    esi
    retn
main        endp
```

Листинг 201

Остальные компиляторы сгенерируют приблизительно такой же код. Общим для всех случаев будет то, что на циклах с предусловием оператор continue практически неотличим от вложенного цикла, а на циклах с постусловием continue эквивалентен элементарному ветвлению.

Наконец, настала очередь циклов for, вращающих несколько счетчиков одновременно. Рассмотрим следующий пример:

```
main()
{
    int a; int b;
    for (a = 1, b = 10; a < 10, b > 1; a++, b --)
    printf("%x %x\n", a, b);
}
```

Листинг 202 Демонстрация идентификации циклов for с несколькими счетчиками

Результат его компиляции компилятором Microsoft Visual C++ 6.0 должен выглядеть так:

```
main        proc near        ; CODE XREF: start+AF.p

var_b        = dword    ptr -8
var_a        = dword    ptr -4

    push   ebp
    mov    ebp, esp
    ; Открываем кадр стека

    sub    esp, 8
    ; Резервируем память для двух локальных переменных

    mov    [ebp+var_a], 1
```

```
    ; Присваиваем переменной var_a значение 0x1

    mov    [ebp+var_b], 0Ah
    ; Присваиваем переменной var_b значение 0xA

    jmp    short loc_401028
    ; Прыжок на код проверки условия выхода из цикла
    ; Это характерная черта не оптимизированных циклов for

loc_401016:                    ; CODE XREF: main+43.j
;                                          ^^^^^^^^^
; Перекрестная ссылка, направленная вниз, говорит о том, что это - начало цикла
; А выше мы уже выяснили, что тип цикла - for

    mov    eax, [ebp+var_a]
    add    eax, 1
    mov    [ebp+var_a], eax
    ; var_a++

    mov    ecx, [ebp+var_b]
    sub    ecx, 1
    mov    [ebp+var_b], ecx
    ; var_b--

loc_401028:                    ; CODE XREF: main+14.j
    cmp    [ebp+var_b], 1
    jle    short loc_401045
    ; Выход из цикла, если var_b <= 0x1
    ; Обратите внимание: выполняется проверка лишь одного (второго слева) счетчика!
    ; Выражение (a1,a2,a3,...an) компилятор считает бессмысленным и берет лишь an
    ; молчаливо отбрасывая все остальное
    ; (из известных мне компиляторов на это ругается один WATCOM)
    ; В данном случае проверяется лишь условие (b > 1), а (a < 10) игнорируется!!!

    mov    edx, [ebp+var_b]
    push   edx
    mov    eax, [ebp+var_a]
    push   eax
    push   offset aXX    ; "%x %x\n"
    call   _printf
    add    esp, 0Ch
    ; printf("%x %x\n", var_a, var_b)

    jmp    short loc_401016
    ; Конец цикла
    ; Итак, данный цикл можно представить как:
    ; while(1)
    ; {
    ;     var_a++;
    ;     var_b--;
    ;     if (var_b <= 0x1) break;
    ;    printf("%x %x\n", var_a, var_b)
    ; }
    ;
    ; Но по соображениям удобочитаемости имеет смысл скомпоновать это код в for
    ;for (var_a=1,var_b=0xA;var_b>1;var_a++,var_b--) printf("%x %x\n",var_a,var_b)
    ;

loc_401045:                     ; CODE XREF: main+2C.j
    mov    esp, ebp
    pop    ebp
    ; Закрываем кадр стека

    retn
main    endp
```

Листинг 203

Оптимизированный вариант программы рассматривать не будем, т.к. это не покажет нам ничего нового. Какой бы компилятор мы не выбрали - выражения инициализации и модификации

счетчиков будут обрабатываться вполне корректно в порядке их объявления в тексте программы, а вот множественные выражения продолжения цикла не умеет правильно обрабатывать ни один компилятор!

Идентификация математических операторов

Автор: Крис Касперски

Дата: 2002-06-16

> *"…если вы обессилены, то не удивительно, что вся ваша жизнь - не развлечение. У вас… так много вычислений, расчетов, которые необходимо сделать в вашей жизни, что она просто не может быть развлечением."*
> Ошо "Пустая Лодка"
> Беседы по высказываниям Чжуан Цзы

Идентификация оператора "+". В общем случае оператор "+" транслируется либо в машинную инструкцию ADD, "перемалывающую" целочисленные операнды, либо в инструкцию FADDx, обрабатывающую вещественные значения. Оптимизирующие компиляторы могут заменять "ADD xxx, 1" более компактной командой "INC xxx", а конструкцию "c = a + b + const" транслировать в машинную инструкцию "LEA c, [a + b + const]". Такой трюк позволяет одним махом складывать несколько переменных, возвратив полученную сумму в любом регистре общего назначения, - не обязательно в левом слагаемом как это требует мнемоника команды ADD. Однако, "LEA" не может быть непосредственно декомпилирована в оператор "+", поскольку она используется не только для оптимизированного сложения (что, в общем-то, побочный продукт ее деятельности), но и по своему непосредственному назначению - вычислению эффективного смещения. (подробнее об этом см. *"Идентификация констант и смещений", "Идентификация типов"*). Рассмотрим следующий пример:

```
main()
{
    int a, b,c;
    c = a + b;
    printf("%x\n",c);
    c=c+1;
    printf("%x\n",c);
}
```

Листинг 204 Демонстрация оператора "+"

Результат его компиляции компилятором Microsoft Visual C++ 6.0 с настройками по умолчанию должен выглядеть так:

```
main  proc near  ; CODE XREF: start+AF p

var_c  = dword ptr -0Ch
var_b  = dword ptr -8
var_a  = dword ptr -4

push ebp
mov ebp, esp
; Открываем кадр стека

sub esp, 0Ch
; Резервируем память для локальных переменных

mov eax, [ebp+var_a]
; Загружаем в EAX значение переменной var_a

add eax, [ebp+var_b]
; Складываем EAX со значением переменной var_b и записываем результат в EAX

mov [ebp+var_c], eax
; Копируем сумму var_a и var_b в переменную var_c, следовательно:
; var_c = var_a + var_b
mov ecx, [ebp+var_c]
```

```
push ecx
push offset asc_406030 ; "%x\n"
call _printf
add esp, 8
; printf("%x\n", var_c)

mov edx, [ebp+var_c]
; Загружаем в EDX значение переменной var_c

add edx, 1
; Складываем EDX со значением 0x1, записывая результат в EDX

mov [ebp+var_c], edx
; Обновляем var_c
; var_c = var_c +1

mov eax, [ebp+var_c]
push eax
push offset asc_406034 ; "%x\n"
call _printf
add esp, 8
; printf("%\n",var_c)

mov esp, ebp
pop ebp
; Закрываем кадр стека

retn
main  endp
```

Листинг 205

А теперь посмотрим, как будет выглядеть тот же самый пример, скомпилированный с ключом "/Ox" (максимальная оптимизация):

```
main  proc near  ; CODE XREF: start+AF p

push ecx
; Резервируем место для одной локальной переменной
; (компилятор посчитал, что три переменные можно ужать в одну и это дейст. так)

mov eax, [esp+0]
; Загружаем в EAX значение переменной var_a

mov ecx, [esp+0]
; Загружаем в EAX значение переменной var_b
; (т.к .переменная не инициализирована загружать можно откуда угодно)

push esi
; Сохраняем регистр ESI в стеке

lea esi, [ecx+eax]
; Используем LEA для быстрого сложения ECX и EAX с последующей записью суммы
; в регистр ESI
; "Быстрое сложение" следует понимать не в смысле, что команда LEA выполняется
; быстрее чем ADD, - количество тактов той и другой одинаково, но LEA
; позволяет избавиться от создания временной переменной для сохранения
; промежуточного результата сложения, сразу направляя результат в ESI
; Таким образом, эта команда декомпилируется как
; reg_ESI = var_a + var_b

push esi
push offset asc_406030 ; "%x\n"
call _printf
; printf("%x\n", reg_ESI)

inc esi
; Увеличиваем ESI на единицу
; reg_ESI = reg_ESI + 1

push esi
```

```
push offset asc_406034 ; "%x\n"
call _printf
add esp, 10h
; printf("%x\n", reg_ESI)

pop esi
pop ecx
retn
main  endp
```

Листинг 206

Остальные компиляторы (Borland C++, WATCOM C) генерируют приблизительно идентичный код, поэтому, приводить результаты бессмысленно - никаких новых "изюминок" они в себе не несут.

Идентификация оператора "-". В общем случае оператор "- " транслируется либо в машинную инструкцию SUB (если операнды - целочисленные значения), либо в инструкцию FSUBx (если операнды - вещественные значения). Оптимизирующие компиляторы могут заменять "SUB xxx, 1" более компактной командой "DEC xxx", а конструкцию "SUB a, const" транслировать в "ADD a, -const", которая ничуть не компактнее и ни сколь не быстрей (и та, и другая укладываться в один так), однако, хозяин (компилятор) - барин. Покажем это на следующем примере:

```
main()
{
    int a,b,c;

    c = a - b;
    printf("%x\n",c);

    c = c - 10;
    printf("%x\n",c);
}
```

Листинг 207 Демонстрация идентификации оператора "-"

Не оптимизированный вариант будет выглядеть приблизительно так:

```
main  proc near  ; CODE XREF: start+AF p

var_c  = dword ptr -0Ch
var_b  = dword ptr -8
var_a  = dword ptr -4

push ebp
mov ebp, esp
; Открываем кадр стека

sub esp, 0Ch
; Резервируем память под локальные переменные

mov eax, [ebp+var_a]
; Загружаем в EAX значение переменной var_a

sub eax, [ebp+var_b]
; Вычитаем из var_a значением переменной var_b, записывая результат в EAX

mov [ebp+var_c], eax
; Записываем в var_c разность var_a и var_b
; var_c = var_a - var_b

mov ecx, [ebp+var_c]
push ecx
push offset asc_406030 ; "%x\n"
call _printf
add esp, 8
; printf("%x\n", var_c)

mov edx, [ebp+var_c]
; Загружаем в EDX значение переменной var_c
```

```
sub edx, 0Ah
; Вычитаем из var_c значение 0xA, записывая результат в EDX

mov [ebp+var_c], edx
; Обновляем var_c
; var_c = var_c - 0xA

mov eax, [ebp+var_c]
push eax
push offset asc_406034 ; "%x\n"
call _printf
add esp, 8
; printf("%x\n",var_c)

mov esp, ebp
pop ebp
; Закрываем кадр стека
retn
main  endp
```

Листинг 208

А теперь рассмотрим оптимизированный вариант того же примера:

```
main  proc near  ; CODE XREF: start+AF p

push ecx
; Резервируем место для локальной переменной var_a

mov eax, [esp+var_a]
; Загружаем в EAX значение локальной переменной var_a

push esi
; Резервируем место для локальной переменной var_b

mov esi, [esp+var_b]
; Загружаем в ESI значение переменной var_b

sub esi, eax
; Вычитаем из var_a значение var_b, записывая результат в ESI

push esi
push offset asc_406030 ; "%x\n"
call _printf
; printf("%x\n", var_a - var_b)

add esi, 0FFFFFFF6h
; Добавляем к ESI (разности var_a и var_b) значение  0xFFFFFFF6
; Поскольку, 0xFFFFFFF6 == -0xA, данная строка кода выглядит так:
; ESI = (var_a - var_b) + (- 0xA) = (var_a - var_b) - 0xA

push esi
push offset asc_406034 ; "%x\n"
call _printf
add esp, 10h
; printf("%x\n", var_a - var_b - 0xA)

pop esi
pop ecx
; Закрываем кадр стека

retn
main  endp
```

Листинг 209

Остальные компиляторы (Borland, WATCOM) генерируют практически идентичный код, поэтому здесь не рассматриваются.

Идентификация оператора "/". В общем случае оператор "/" транслируется либо в машинную инструкцию "DIV" (беззнаковое целочисленное деление), либо в "IDIV" (целочисленное деление со знаком), либо в "FDIVx" (вещественное деление). Если делитель кратен степени двойки, то "DIV" заменяется на более быстродействующую инструкцию битового сдвига вправо "SHR a, N", где a - делимое, а N - показатель степени с основанием два.

Несколько сложнее происходит быстрое деление знаковых чисел. Совершенно недостаточно выполнить арифметический сдвиг вправо (команда арифметического сдвига вправо SAR заполняет старшие биты с учетом знака числа), ведь если модуль делимого меньше модуля делителя, то арифметический сдвиг вправо сбросит все значащие биты в "битовую корзину", в результате чего получиться 0xFFFFFFFF, т.е. -1, в то время как правильный ответ - ноль. Вообще же, деление знаковых чисел арифметическим сдвигом вправо дает округление в **большую** сторону, что совсем не входит в наши планы. Для округления знаковых чисел в меньшую сторону необходимо перед выполнением сдвига добавить к делимому число 2^N- 1, где N - количество битов, на которые сдвигается число при делении. Легко видеть, что это приводит к увеличению всех сдвигаемых битов на единицу и переносу в старший разряд, если хотя бы один из них не равен нулю.

Следует отметить: деление очень медленная операция, гораздо более медленная чем умножение (выполнение DIV может занять свыше 40 тактов, в то время как MUL обычно укладываться в 4), поэтому, продвинутые оптимизирующие компиляторы заменяют деление умножением. Существует множество формул подобных преобразований, вот, например, она (самая популярная из них):

a/b = 2^N/b * a/2^N', где N' - разрядность числа. Выходит, грань между умножением и делением очень тока, а их идентификация довольно сложна. Рассмотрим следующий пример:

```
main()
{
    int a;
    printf("%x %x\n",a / 32, a / 10);
}
```

Листинг 210 Идентификация оператора "/"

Результат его компиляции компилятором Microsoft Visual C++ с настройками по умолчанию должен выглядеть так:

```
main  proc near  ; CODE XREF: start+AF p

var_a  = dword ptr -4

push ebp
mov ebp, esp
; Открываем кадр стека

push ecx
; Резервируем память для локальной переменной

mov eax, [ebp+var_a]
; Копируем в EAX значение переменной var_a

cdq
; Расширяем EAX до четверного слова EDX:EAX

mov ecx, 0Ah
; Заносим в ECX значение 0xA

idiv ecx
; Делим (учитывая знак) EDX:EAX на 0xA, занося частное в EAX
; EAX = var_a / 0xA

push eax
; Передаем результат вычислений функции printf

mov eax, [ebp+var_a]
```

```
; Загружаем в EAX значение var_a

cdq
; Расширяем EAX до четверного слова EDX:EAX

and edx, 1Fh
; Выделяем пять младших бит EDX

add eax, edx
; Складываем знак числа для выполнения округления отрицательных значений
; в меньшую сторону

sar eax, 5
; Арифметический сдвиг вправо на 5 позиций
; эквивалентен делению числа на 2^5 = 32
; Таким образом, последние четыре инструкции расшифровываются как:
; EAX = var_a / 32
; Обратите внимание: даже при выключенном режиме оптимизации компилятор
; оптимизировал деление

push eax
push offset aXX ; "%x %x\n"
call _printf
add esp, 0Ch
; printf("%x %x\n", var_a / 0xA, var_a / 32)

mov esp, ebp
pop ebp
; Закрываем кадр стека

retn
main  endp
```

Листинг 211

А теперь, засучив рукава и глотнув пустырника (или валерьянки) рассмотрим оптимизированный вариант того же примера:

```
main  proc near  ; CODE XREF: start+AF p

push ecx
; Резервируем память для локальной переменной var_a

mov ecx, [esp+var_a]
; Загружаем в ECX значение переменной var_a

mov eax, 66666667h
; Так, что это за зверское число?!
; В исходном коде ничего подобного и близко не было!

imul ecx
; Умножаем это зверское число на переменную var_a
; Обратите внимание: именно умножаем, а не делим.
; Однако притворимся на время, что у нас нет исходного кода примера, потому
; ничего странного в операции умножения мы не видим

sar edx, 2
; Выполняем арифметический сдвиг всех битов EDX на две позиции вправо, что
; в первом приближении эквивалентно его делению на 4
; Однако ведь в EDX находятся старшее двойное слово результата умножения!
; Поэтому, три предыдущих команды фактически расшифровываются так:
; EDX = (66666667h * var_a) >> (32 + 2) = (66666667h * var_a) / 0x400000000
;
; Понюхайте эту строчку - не пахнет ли паленым? Как так не пахнет?! Смотрите:
; (66666667h * var_a) / 0x400000000 = var_a * 66666667h / 0x400000000 =
; = var_a * 0,10000000003492459654808044433594
; Заменяя по всем правилам математики умножение на деление и одновременно
; выполняя округление до меньшего целого получаем:
; var_a * 0,1000000000 = var_a * (1/0,1000000000) = var_a/10
;
; Согласитесь, от такого преобразования код стал намного понятнее!
```

```
; Как можно распознать такую ситуацию в чужой программе, исходный текст которой
; неизвестен? Да очень просто - если встречается умножение, а следом за ним
; сдвиг вправо, обозначающий деление, то каждый нормальный математик сочтет
; своим долгом такую конструкцию сократить, по методике показанной выше!

mov eax, edx
; Копируем полученное частное в EAX

shr eax, 1Fh
; Сдвигаем на 31 позицию вправо

add edx, eax
; Складываем: EDX = EDX + (EDX >> 31)
; Чтобы это значило? Нетрудно понять, что после сдвига EDX на 31 бит вправо
; в нем останется лишь знаковый бит числа
; Тогда - если число отрицательно, мы добавляем к результату деления один,
; округляя его в меньшую сторону. Таким образом, весь этот хитрый код
; обозначает ни что иное как тривиальную операцию знакового деления:
; EDX = var_a / 10
; Не слишком ли много кода для одного лишь деления? Конечно, программа
; здорово "распухает", зато весь этот код выполняется всего лишь за 9 тактов,
; в то время как в не оптимизированном варианте аж за 28!
; /* Измерения проводились на процессоре CLERION с ядром P6, на других
; процессорах количество тактов может отличается */
; Т.е. оптимизация дала более чем трехкратный выигрыш, браво Microsoft!

mov eax, ecx
; Вспомним: что находится в ECX? Ох, уж эта наша дырявая память, более дырявая
; чем дуршлаг без дна… Прокручиваем экран дизассемблера вверх. Ага, в ECX
; последний раз разгружалось значение переменной var_a

push edx
; Передаем функции printf результат деления var_a на 10

cdq
; Расширяем EAX (var_a) до четверного слова EDX:EAX

and edx, 1Fh
; Выбираем младшие 5 бит регистра EDX, содержащие знак var_a

add eax, edx
; Округляем до меньшего

sar eax, 5
; Арифметический сдвиг на 5 эквивалентен делению var_a на 32

push eax
push offset aXX ; "%x %x\n"
call _printf
add esp, 10h
; printf("%x %x\n", var_a / 10, var_a / 32)

retn
main  endp
```

Листинг 212

Ну, а другие компиляторы, насколько они продвинуты в плане оптимизации? Увы, ни Borland, ни WATCOM не умеют заменять деление более быстрым умножением для чисел отличных от степени двойки. В подтверждении тому рассмотрим результат компиляции того же примера компилятором Borland C++:

```
_main  proc near  ; DATA XREF: DATA:00407044 o

push ebp
mov ebp, esp
; Открываем кадр стека

push ebx
; Сохраняем EBX
```

```
mov eax, ecx
; Копируем в EAX содержимое неинициализированной регистровой переменной ECX

mov ebx, 0Ah
; Заносим в EBX значение 0xA

cdq
; Расширяем EAX до четверного слова EDX:EAX

idiv ebx
; Делим ECX на 0xA (долго делим - тактов 20, а то и больше)

push eax
; Передаем полученное значение функции printf

test ecx, ecx
jns short loc_401092
; Если делимое не отрицательно, то переход на loc_401092

add ecx, 1Fh
; Если делимое положительно, то добавляем к нему 0x1F для округления

loc_401092:    ; CODE XREF: _main+11 j

sar ecx, 5
; Сдвигом на пять позиций вправо делим число на 32

push ecx
push offset aXX ; "%x %x\n"
call _printf
add esp, 0Ch
; printf("%x %x\n", var_a / 10, var_a / 32)

xor eax, eax
; Возвращаем ноль

pop ebx
pop ebp
; Закрываем кадр стека

retn
_main  endp
```

Листинг 213

Идентификация оператора "%". Специальной инструкции для вычисления остатка в наборе команд микропроцессоров серии 80x86 нет, - вместо этого остаток вместе с частным возвращается инструкциями деления DIV, IDIV и FDIVx (см. *идентификация оператора "/"*).

Если делитель представляет собой степень двойки (2^N = b), а делимое беззнаковое число, то остаток будет равен N младшим битам делимого числа. Если же делимое - знаковое, необходимо установить все биты, кроме первых N равными знаковому биту для сохранения знака числа. Причем, если N первых битов равно нулю, все биты результата должны быть сброшены независимо от значения знакового бита.

Таким образом, если делимое - беззнаковое число, то выражение a % 2^N транслируется в конструкцию: "AND a, N", в противном случае трансляция становится неоднозначна - компилятор может вставлять явную проверку на равенство нулю с ветвлением, а может использовать хитрые математические алгоритмы, самый популярный из которых выглядит так: DEC x\ OR x, -N\ INC x. Весь фокус в том, что если первые N бит числа x равны нулю, то все биты результата кроме старшего, знакового бита, будут гарантированно равны одному, а OR x, -N принудительно установит в единицу и старший бит, т.е. получится значение, равное, -1. А INC -1 даст ноль! Напротив, если хотя бы один из N младших битов равен одному, заема из старших битов не происходит и INC x возвращает значению первоначальный результат.

Продвинутые оптимизирующие компиляторы могут путем сложных преобразований заменять деление на ряд других, более быстродействующих операций. К сожалению, алгоритмов для быстрого вычисления остатка для всех делителей не существует и делитель должен быть кратен k * 2^t, где k и t - некоторые целые числа. Тогда остаток можно вычислить по следующей формуле:

a % b = a % k*2^t = a -((2^N/k * a/2^N) & -2^t)*k

Да, эта формула очень сложна и идентификация оптимизированного оператора "%" может быть весьма и весьма непростой, особенно учитывая патологическую любовь оптимизаторов к изменению порядка команд.

Рассмотрим следующий пример:

```
main()
{
    int a;
    printf("%x %x\n",a % 16, a % 10);
}
```

Листинг 214 Идентификация оператора "%"

Результат его компиляции компилятором Microsoft Visual C++ с настройками по умолчанию должен выглядеть так:

```
main  proc near  ; CODE XREF: start+AF p

var_4  = dword ptr -4

push ebp
mov ebp, esp
; Открываем кадр стека

push ecx
; Резервируем память для локальной переменной

mov eax, [ebp+var_a]
; Заносим в EAX значение переменной var_a

cdq
; Расширяем EAX до четвертного слова EDX:EAX

mov ecx, 0Ah
; Заносим в ECX значение 0xA

idiv ecx
; Делим EDX:EAX (var_a) на ECX (0xA)


push edx
; Передаем остаток от деления var_a на 0xA функции printf

mov edx, [ebp+var_a]
; Заносим в EDX значение переменной var_a

and edx, 8000000Fh
; "Вырезаем" знаковый бит и четыре младших бита числа
; в четырех младших битах содержится остаток от деления EDX на 16

jns short loc_401020
; Если число не отрицательно, то прыгаем на loc_401020

dec edx
or edx, 0FFFFFFF0h
inc edx
; Последовательность сия, как говорилось выше характера для быстрого
; расчета отставка знакового числа
; Следовательно, последние шесть инструкций расшифровываются как:
; EDX = var_a % 16
loc_401020:    ; CODE XREF: main+19 j
```

```
push edx
push offset aXX ; "%x %x\n"
call _printf
add esp, 0Ch
; printf("%x %x\n",var_a % 0xA, var_a % 16)

mov esp, ebp
pop ebp
; Закрываем кадр стека
retn
main  endp
```

Листинг 215 Любопытно, что оптимизация не влияет на алгоритм вычисления остатка. Увы, ни Microsoft Visual C++, ни остальные известные мне компиляторы не умеют вычислять остаток умножением. *Идентификация оператора *.* В общем случае оператор * транслируется либо в машинную инструкцию "MUL" (беззнаковое целочисленное умножение), либо в "IMUL" (целочисленное умножение со знаком), либо в "FMULx" (вещественное умножение). Если один из множителей кратен степени двойки, то "MUL" ("IMUL") обычно заменяется командой битового сдвига влево "SHL" или инструкцией "LEA", способной умножать содержимое регистров на 2, 4 и 8. Обе последних команды выполняются за один такт, в то время как MUL требует в зависимости от модели процессора от двух до девяти тактов. К тому же LEA за тот же такт успевает сложить результат умножение с содержимым регистра общего назначения и/или константой в придачу. Это позволяет умножать на 3, 5 и 9 просто добавляя к умножаемому регистру его значение. Ну, разве это не сказка? Правда, у LEA есть один недочет - она может вызывать остановку AGI, в конечном счете "съедающую" весь выигрыш в быстродействии на нет.

Рассмотрим следующий пример:

```
main()
{
    int a;
    printf("%x %x %x\n",a * 16, a * 4 + 5, a * 13);
}
```

Листинг 216 Идентификация оператора *

Результат его компиляции компилятором Microsoft Visual C++ с настройками по умолчанию должен выглядеть так:

```
main  proc near  ; CODE XREF: start+AF p

var_a  = dword ptr -4

push ebp
mov ebp, esp
; Открываем кадр стека

push ecx
; Резервируем место для локальной переменной var_a

mov eax, [ebp+var_a]
; Загружаем в EAX значение переменной var_a

imul eax, 0Dh
; Умножаем var_a на 0xD, записывая результат в EAX

push eax
; Передаем функции printf произведение var_a * 0xD

mov ecx, [ebp+var_a]
; Загружаем в ECX значение var_a

lea edx, ds:5[ecx*4]
; Умножаем ECX на 4 и добавляем к полученному результату 5, записывая его в EDX
; И все это выполняется за один такт!
push edx
```

```
; Передаем функции printf результат var_a * 4 + 5

mov eax, [ebp+var_a]
; Загружаем в EAX значение переменной var_a

shl eax, 4
; Умножаем var_a на 16

push eax
; Передаем функции printf произведение var_a * 16

push offset aXXX ; "%x %x %x\n"
call _printf
add esp, 10h
; printf("%x %x %x\n", var_a * 16, var_a * 4 + 5, var_a * 0xD)

mov esp, ebp
pop ebp
; Закрываем кадр стека
retn
main  endp
```

Листинг 217 За вычетом вызова функции printf и загрузки переменной var_a из памяти на все про все требуется лишь *три* такта процессора. А что будет, если скомпилировать этот пример с ключиком "/Ox"? А будет вот что:

```
main  proc near  ; CODE XREF: start+AF p

push ecx
; Выделяем память для локальной переменной var_a

mov eax, [esp+var_a]
; Загружаем в EAX значение переменной var_a

lea ecx, [eax+eax*2]
; ECX = var_a * 2 + var_a = var_a * 3

lea edx, [eax+ecx*4]
; EDX = (var_a * 3)* 4 + var_a = var_a * 13!
; Вот так компилятор ухитрился умножить var_a на 13,
; причем всего за один (!) такт. Да, обе инструкции LEA прекрасно спариваются
; на Pentium MMX и Pentium Pro!

lea ecx, ds:5[eax*4]
; ECX = EAX*4 + 5

push edx
push ecx
; Передаем функции printf var_a * 13 и var_a * 4 +5

shl eax, 4
; Умножаем var_a на 16

push eax
push offset aXXX ; "%x %x %x\n"
call _printf
add esp, 14h
; printf("%x %x %x\n", var_a * 16, var_a * 4 + 5, var_a * 13)

retn
main  endp
```

Листинг 218

Этот код, правда, все же не быстрее предыдущего, не оптимизированного, и укладывается в те же три такта, но в других случаях выигрыш может оказаться вполне ощутимым.

Другие компиляторы так же используют LEA для быстрого умножения чисел. Вот, к примеру, Borland поступает так:

```
_main  proc near  ; DATA XREF: DATA:00407044 o

lea edx, [eax+eax*2]
; EDX = var_a*3

mov ecx, eax
; Загружаем в ECX неинициализированную регистровую переменную var_a

shl ecx, 2
; ECX = var_a * 4

push ebp
; Сохраняем EBP

add ecx, 5
; Добавляем к var_a * 4 значение 5
; Borland не использует LEA для сложения. А жаль…

lea edx, [eax+edx*4]
; EDX = var_a + (var_a *3) *4 = var_a * 13
; А вот в этом Borland и MS единодушны :-)

mov ebp, esp
; Открываем кадр стека
; Да, да… вот так посреди функции и открываем…
; Выше, кстати, "потерянная" команда push EBP

push edx
; Передаем printf произведение var_a * 13

shl eax, 4
; Умножаем ((var_a *4) + 5) на 16
; Что такое?! Да, это глюк компилятора, посчитавшего: раз переменная var_a
; неинициализирована, то ее можно и не загружать…

push ecx
push eax
push offset aXXX ; "%x %x %x\n"
call printf
add esp, 10h
xor eax, eax
pop ebp
retn
_main  endp
```

Листинг 219

Хотя "визуально" Borland генерирует более "тупой" код, его выполнение укладывается в те же три такта процессора. Другое дело WATCOM, показывающий удручающе отсталый результат на фоне двух предыдущих компиляторов:

```
main  proc near
push ebx
; Сохраняем EBX в стеке

mov eax, ebx
; Загружаем в EAX значение неинициализированной регистровой переменной var_a

shl eax, 2
; EAX = var_a * 4

sub eax, ebx
; EAX = var_a * 4 - var_a = var_a * 3
; Вот каков WATCOM! Сначала умножает "с запасом", а потом лишнее отнимает!

shl eax, 2
; EAX = var_a * 3 * 4 = var_a * 12

add eax, ebx
; EAX = var_a * 12 + var_a = var_a * 13
; Вот так, да? Четыре инструкции, в то время как "ненавистный" многим
```

```
; Microsoft Visual C++ вполне обходится и двумя!

push eax
; Передаем printf значение var_a * 13

mov eax, ebx
; Загружаем в EAX значение неинициализированной регистровой переменной var_a

shl eax, 2
; EAX = var_a * 4

add eax, 5
; EAX = var_a * 4 + 5
; Ага! Пользоваться LEA WATCOM то же не умеет!

push eax
; Передаем printf значение var_a * 4 + 5

shl ebx, 4
; EBX = var_a * 16

push ebx
; Передаем printf значение var_a * 16

push offset aXXX ; "%x %x %x\n"
call printf_
add esp, 10h
; printf("%x %x %x\n",var_a * 16, var_a * 4 + 5, var_a*13)

pop ebx

retn
main_  endp
```

Листинг 220 В результате, код, сгенерированный компилятором WATCOM требует шести тактов, т.е. вдвое больше, чем у конкурентов. *::Комплексные операторы.* Язык Си\Си++ выгодно отличается от большинства своих конкурентов поддержкой комплексных операторов: x= (где x - любой элементарный оператор), ++ и - -.

Комплексные операторы семейства "a x= b" транслируются в "a = a x b" и они идентифицируются так же, как и элементарные операторы (см. *"элементарные операторы"*).

Операторы "++" и "--": в префиксной форме они выражаются в тривиальные конструкции "a = a +1" и "a = a - 1" не представляющие для нас никакого интереса, но вот постфиксная форма - дело другое.

Продолжение следует...

Убираем баннеры в "Штирлице" или Баг в защите

Автор: Broken Sword

Дата: 2002-08-14

Штирлиц шел по лесу и напоролся на сук. - "Шли бы вы домой, девочки, война все-таки!"

Из личного дела:

```
Имя:            Штирлиц 4.01
Вес:            738 кило
Дата рождения:  2001 год
Отец:           Всеволод Лукьянин
```

Prelude

Наверное, нет такого человека, который никогда бы ничего не слышал о Штирлице. И речь в данной статье пойдет не о В. Тихонове, и не об анекдотах, а о лучшем в своем роде расшифровщике-трансляторе, вернее, о доведении оного до совершенства :).

Любому из нас приходилось получать загадочные письма из сети, содержащие в себе прелюбопытные наборы значков и черточек. Разнообразие стандартов и кодировок породило на свет огромное количество перекодировщиков. Во всем этом безобразном разнообразии отчетливо обозначились несколько особо продвинутых программулин. Мне почему то особенно запомнились только две – Bred и Штирлиц. И если первая ничем не достает и не просит на "покушать", то вторая изрядно выводит из себя своим "рекламным окошком", которое по заявлению самого автора "не мешает работать программе" (Лукьянин вообще парень с юмором). Но, ближе к делу...

С чего начинается родина…

Родина, безусловно, начинается с отладчика, и не с простого, а с большой буквы. Как вы уже догадались – это SoftIce, Многие из вас уже испугались и начали выводить кресты, однако я просто советую вам поставить на свою машину именно этот отладчик и пользуясь данным руководством в конце пути вы просто пожмете ему руку и... больше никуда не отпустите. Еще ОЧЕНЬ пригодится дизасмер (я рекомендую ТОЛЬКО WDASM) и HIEW (который, как окажется в последствии, совершенно не понадобится, но в педагогических целях его неплохо бы заиметь).

Больше нам реально ничего не понадобится... Ах да, разве что еще ваш любимый выходной набор "мозги + прямые руки". И конечно, листик с ручкой (запомните, ни один серьезный взлом не обходится без листика с ручкой, а еще лучше завести для этого дела специальную тетрадь, но это уже для извращенцев).

Go Go Go...!

Итак, поехали! Запускаем Shtirlitz.exe для того чтобы окончательно понять, чего же мы все таки хотим. А хотим мы убрать этот чертов баннер, ато он $%@##$ уже достал внатуре !!!

Где же он ?! Вот он, сидит себе в углу, моргает. Подожди же, зараза...

SoftIce у нас уже конечно же запущен…

Сразу же жмем ctrl-D, осмотримся, чего же у нас творится в данный момент... Оказавшись в любимом окне набиваем:

```
addr
```

В ответ получим загадочную последовательность запущенных на данный момент процессов. Где-то в самом конце списка (на всякий случай, перемещение по списку – клавиши [вверх] и [вниз]) мы просто обязаны увидеть строку такого содержания:

```
      Addr   PID  NAME
…… …… …… ……
xxxxxxxx xxxxxxxx  xxxx  Shtirlitz
```

где вместо "x"-ов будут hex-цифры.

Вот он, родимый!!! Привяжемся же к нему всей душой!!!

Набиваем:

```
addr xxxx
```

где вместо иксов подставляем соотв. значение из колонки PID.

Теперь, повторно набрав в строке addr убеждаемся, что Штирлиц стал текущим процессом (т.е. голубым :).

Далее. Добившись того, чтобы Shtirlitz стал текущим процессом, можно смело узнать про него много всего полезного. Набиваем:

```
hwnd Shtirlitz
```

(Shtirlitz, кстати, в данном случае берется из колонки NAME)

И что мы видим ? А видим мы длинный список инфы про нашего пациента. Чего мы вообще хотим? Убрать баннеры... Смотрим, чего тут можно нарыть по нашей проблеме... Ага!!! Вот оно!!! Первой же строкой!!! И не стыдно...?

Shtirlitz.banner ... запомним.

Теперь запускаем WDASM. Грузим Shtirlitz.exe. Пошарим по файлу (теперь мы знаем чего искать). В строке поиска пишем banner. Ждем... Вуаля!

```
* Reference To: GDI32.GetStockObject, Ord:00FAh
                                  |
:00401B1C FF15B4E54400             Call dword ptr [0044E5B4]
:00401B22 50                       push eax
:00401B23 6800080000              push 00000800

* Possible StringData Ref from Data Obj ->"Shtirlitz.Banner"
                                  |
:00401B28 6804414400              push 00444104
:00401B2D B938BA4400             mov ecx, 0044BA38
:00401B32 E8BF6B0000              call 004086F6
:00401B37 84C0                     test al, al
:00401B39 7473                      je 00401BAE
:00401B3B 57                        push edi
:00401B3C 56                        push esi
:00401B3D 8B742418                 mov esi, dword ptr [esp+18]
:00401B41 53                       push ebx
:00401B42 55                        push ebp
:00401B43 6800004086               push 86400000
:00401B48 56                       push esi

* Reference To: USER32.AdjustWindowRect, Ord:0001h
                                  |
:00401B49 FF156CE94400             Call dword ptr [0044E96C]
:00401B4F 8B7E08                   mov edi, dword ptr [esi+08]
```

```
:00401B52 8B5E0C                  mov ebx, dword ptr [esi+0C]
:00401B55 2B3E                     sub edi, dword ptr [esi]
:00401B57 2B5E04                  sub ebx, dword ptr [esi+04]
:00401B5A 55                      push ebp

* Reference To: USER32.GetSystemMetrics, Ord:012Ch
                                  |
:00401B5B 8B2D64E94400            mov ebp, dword ptr [0044E964]
:00401B61 FFD5                     call ebp
:00401B63 83E814                  sub eax, 00000014
:00401B66 6A01                     push 00000001
:00401B68 894608                  mov dword ptr [esi+08], eax
:00401B6B FFD5                    call ebp
:00401B6D 8B4E08                  mov ecx, dword ptr [esi+08]
:00401B70 83C0EC                  add eax, FFFFFFEC
:00401B73 89460C                  mov dword ptr [esi+0C], eax
:00401B76 2BCF                     sub ecx, edi
:00401B78 2BC3                     sub eax, ebx
:00401B7A 890E                     mov dword ptr [esi], ecx
:00401B7C 894604                  mov dword ptr [esi+04], eax

* Reference To: KERNEL32.GetTickCount, Ord:0145h
                                  |
:00401B7F FF1598E74400            Call dword ptr [0044E798]
:00401B85 8B4C2410                mov ecx, dword ptr [esp+10]
:00401B89 89415C                  mov dword ptr [ecx+5C], eax
:00401B8C 33C0                     xor eax, eax
:00401B8E 50                       push eax
:00401B8F 50                       push eax
:00401B90 50                       push eax
:00401B91 56                       push esi
:00401B92 6800004086              push 86400000
:00401B97 68D8B64400               push 0044B6D8

* Possible StringData Ref from Data Obj ->"Shtirlitz.Banner"
                                  |
:00401B9C 6804414400               push 00444104
:00401BA1 6888000000               push 00000088
:00401BA6 E81D700000              call 00408BC8
:00401BAB 5B                       pop ebx
:00401BAC 5E                       pop esi
:00401BAD 5F                       pop edi

* Referenced by a (U)nconditional or (C)onditional Jump at Address:
|:00401B39(C)
|
:00401BAE 5D                       pop ebp
:00401BAF 59                       pop ecx
:00401BB0 C20800                  ret 0008
```

Всеволод Лукьянин не только с юмором, а еще и очень хитер! Видите, как ловко он вплел показ баннеров в код?! После непродолжительного разглядывания данного куска становиться очевидно :), что от результата, возвращаемого процедурой в строке

```
:00401B32 E8BF6B0000     call 004086F6
```

зависит относится ли данный вызов непосредственно к показу рекламы или нет (если нет, то возвращаемый результат равен 0, и соответственно после

```
:00401B37 84C0           test al, al
```

мы перепрыгиваем "через овечку" прямо в конец куска (перелетаем полностью через весь показ баннеров). Сам переход происходит в строке

```
:00401B39 7473           je 00401BAE
```

Записывай пока на бумажку адрес 00401B39.

Теперь напряжемся и вместе подумаем: как же сделать так, чтобы "перелет через показ баннера" происходил в любом случае? Выхода здесь два: либо загружать al перед je нулем посредством xor-а (mov не меняет флаги), либо просто изменить "74 на EB" (74 – код команды je, EB – код команды jmp; эти коды, наряду с CD (int) и др. кодами ты должен знать как "отче наш").

Конечно же, мы выбираем второй вариант...

Где наш HIEW? Вот и он. Запускаем. Ищем и меняем байт по адресу 00401B39h. Кстати, небольшое...

Лирическое отступление

Для того, чтобы не бегать в HIEW по всему коду в поисках адреса 00401B39 мы в режиме decode (выбирается по F4) жмем F5 и набираем его в строке для быстрого перехода. Набрав 401B39 нас сразу же пошлют подальше, хотя реально это смещение в окне HIEW мы действительно можем наблюдать белым по синему. Так в чем же дело? А дело в том, что это не те смещения... :) Все эти адреса генерятся из заголовка EXE файла, а РЕАЛЬНОЕ смещение в файле, как оно есть, можно увидеть, нажав в любимом HIEW-е комбинацию клавиш alt+F1. Все. Теперь все смещения – реальные, на любое из них можно прыгнуть по F5. Но куда же нам, все таки, in hell прыгать? Возвращаемся в любимый WDASM. Наводим курсор на строку

```
00401B39 7473     je 00401BAE
```

А вот внизу то что нам нада:

```
Line:1734 Pg 35 and 36 of 2788 Code Data ... @offset 00000F39 in file...
```

Видишь @offset in file... ? Все. Теперь в HIEW прыгаем на F39 и меняем "74 на EB".

ЗОЛОТОЕ ПРАВИЛО: всегда, перед тем как править файл в HIEW ВСЕГДА сохраняй его копию!

И до чего же мы докатились ? Запускаем Shtirlitz, смотрим...

Вот зараза...

Такой подлости мы явно не ожидали... "Файл поврежден, переустановите программу с дистрибутива". Что же произошло? А произошло то, что shtirlitz при загрузке проверяет сам себя на вшивость (или "на измену"). Как он это делает? А очень просто! Считывает сам себя в память (ф-ция ReadFile), затем с первого байта начинает сканировать сам себя, попутно манипулируя своим "телом" через регистры. Короче говоря, вычисляет контрольную сумму всех своих байт. (Контрольная сумма на деле оказывается не простой суммой, а результатом махинаций с регистрами, XOR-ами и MOV-ами). Вникать в алгоритм нет никакого желания да и не нужно нам это пока... Но запомните: взломать прогу "красиво" можно только полностью проникнувшись подобными алгоритмами.

Итак, восстанавливаем из сохраненной копии Shtirlitz.exe и опять в SoftIce. Просто взять и установить бряк на ReadFile будет глупо, т.к. данная API-шка после запуска shtirlitz.exe вызывается тучу раз, и какой именно нужен нам – непонятно. Будем действовать старым дедовским методом. Лезем в WDASM. Смотрим, есть ли чего нить, что отвечает за загрузку файла. В строке поиска поочередно перебираем возможные варианты: сначала openfile. Найдено GetOpenFileName. Не то. CreateFile. О! То что нада! CreateFileA!

Несколько слов по поводу данной апишки... Все что нам нужно знать про нее – это то, что перед ее вызовом последним в стек ляжет адрес на имя загружаемого файла. Вот его то нам и нужно отлавливать и контролировать, во избежание вызова "лже-CreateFile".

Back to the SoftIce... Набиваем:

```
bpx CreateFileA
```

Запускаем Штирлица и ждем срабатывания бряка... Есть! Смотрим, чего у нас лежит в стеке:

```
dd esp
```

В окне данных видим:

```
00407289    00910BB0 ...
```

Когда SI отлавливает вызов API, управление передается на первую команду этой самой API, поэтому сейчас мы оказались на самом первом байте кода ф-ции CreateFile.

Чего же это там у нас в окне данных... Ну первое число – понятно, откуда произошел вызов, оно нам не нада, а вот второе – это и есть указатель на имя открываемого файла (у вас это может быть другое число). Лезем по нему:

```
d 00910BB0
```

Облом... Shtirlitz.loc какой то... А нам нужен Shtirlitz.exe. Но ничего! Ждем следующего вызова CreateFile… Жмем F5. И вот буквально тут же! Повторяем уже знакомые операции (dd esp), вот оно второе число: 00910910. Заглянем и туда (d 910910) – yes! Вот он, родимый! Shtirlitz.exe!!! Ну все!!! ВОТ ТЕПЕРЬ можно ставить бряк на ReadFile! НЕ ВЫХОДЯ ИЗ SI пишем:

```
bpx ReadFile
```

Теперь жмем F5, ждем... Вот он долгожданный бряк по ReadFile! Смотрим стек (dd esp). Видим:

```
00405B00 00000060 00E90020 000A9400 ...
```

нас интересуют первые четыре числа. Первое число – адрес возврата (не нада), второе число – хэндл файла (не нужен), третье число – уже интереснее (адрес, куда надо считывать файл), четвертое – сколько байт нада считывать.

Четвертое число вообще очень интересное... Кстати, ebx тоже равен этому же числу... Запомним это. Смотрим на калькуляторе: A9400h=693246d. А теперь угадай с трех раз, сколько весит файл Shtirlitz.exe? Теперь можно точно сказать, что происходит подсчет контрольной суммы ВСЕГО файла целиком... Но ты не сиди – записывай все числа на бумажку. С комментариями, ато потом заблудишься.

Все. Пришло время заняться делом. Как только мы оказались в начале апишки ReadFile и посмотрели стек – жмем F12 (сам увидишь как это связано с первым числом). Теперь поехали трейсить! Запомни: видишь call – жми F10, чтобы не влезать в него. В других случаях – жмешь F8. Теперь главное вовремя остановиться... :). По ходу пьесы тебе часто придется вываливаться из SI, хотя бы даже для заглядывания в данную статью, поэтому перед выходом тебе ПРИДЕТСЯ набивать bc* (убрать все бряки). Так вот, для возврата тебе придется опять два раза отлавливать CreateFile, потом ReadFile... запаришься! Поэтому вспомни, что ebx был равен четвертому числу – а это ключ!!! Зная это, просто пишешь:

```
bpx ReadFile if ebx==A9400
```

и ты в нужном месте в нужное время...!

Итак, трейсим... через несколько F8 и парочки F10 мы попадаем в загадочную область:

```
00423B1E:   movzx...
            ...
            inc esi
```

```
            cmp esi, ebx
            mov [ebp-08], ecx
00423B5E:   jb 00423B1E
00423B60:   ...
```

Это какой то заговор... Обрати внимание на тот факт, что esi в начале цикла равен нулю, а ebx – A9400 (или размеру файла Штирлиц.exe); я нарочно выделил inc esi и последующее сравнение с ebx – тут даже койоту ясно, что идет подсчет контрольной суммы всего файла. Ну что ж, любители острых ощущений могут просидеть так пару дней, давя F8, я предпочитаю (и вам советую) сделать так:

```
bpx 00423B60
```

Это есть бряк на следующую после jb команду, а сюда управление вернется только после того, как выполнится A9400 итераций. (вообще желательно выписать на бумажку значения BCEX регистров и ячейки ebp-08 в этом месте, т.к. и ежу понятно – это единственные правильные значения, которые могут иметь место (ведь если бы мы "исправили" файл в HIEW, то значения регистров на выходе оказались бы совершенно иными). Вот значения регистров для "девственного" shtirlitz.exe:

```
EAX=00000016
ECX=000048D2
EDX=00000020
EDI=00E90020
dword [ebp-08]=ECX=000048D2
```

Кстати, как многие уже догадались 48D2 – это и есть контрольная сумма, запомним это число.

Что теперь? Пока ничего, трейсим дальше... :) Здесь нам встретятся два call-a, перелетаем их. Ждем ret-a... И вот мы вернулись непонятно куда – на

```
00423744:  cmp ax,si
00423747:   jz 0042376A
```

Любознательные могут сами залезть в те два call-a, я же просто скажу: после того, как мы оказались на 00423744 в esi всегда будет лежать 48D2h (помните его?), а вот в ax – то, что получилось из нашего shtirlitz.exe, и если он только не равен 48D2... то... сами понимаете.

Ну что, опять в 00423747 замена jz на jmp ? И сколько это будет продолжаться ? Пробуем... (00423747=22B47 in file)

Запускаем... Ну вот!!! Теперь еще круче!!! Вообще ничего!!! Мда...

Пришло время пораскинуть мозгами. Очевидно, что единственное, что поменялось в самом процессе работы программы – значение регистров после вычисления контрольной суммы. С одной стороны, весь цикл (00423B1E-00423B5E) можно заменить на код типа такого:

```
mov ecx,48D2h
mov edx,20h
...
```

т.е. галимо подогнать значения всех регистров, зная какими они выглядят в "девственном" Штирлице.exe. Но, во-первых, это уродливо, а во вторых - это нас не спасет... (можете попробовать). Стоит только отметить, что обращение к области памяти с E90020h (там, где лежит копия файла shtirlitz.exe) происходит еще минимум 4 раза!!! Разбираться и отлавливать их все нет никакого желания, Всеволод очень неплохо постарался...

Поэтому пойдем по другому пути... Помнишь, в самом начале мы заменили по адресу

```
:00401B39 7473      je 00401BAE
```

je на jmp ? Так вот, что если править код не в самом файле, а непосредственно в памяти??? Тогда никакой возни с контрольной суммой не возникнет!!! Отлично! Заодно можно проверить, по тому ли пути мы вообще идем... Итак, берем "девственный" shtirlitz.exe, ставим в SI бряк на readfile при ebx==A9400, запускаем shtirlitz.exe... Теперь, правим байт:

```
eb 401B39 eb
```

Жмем F5 Yes! Никаких баннеров!!! Мы победили... Но... Не будем же мы в самом деле каждый раз при запуске Штирлица править байт в памяти !!!

Но это уже и не нужно!!! Запустите shtirlitz.exe и убедитесь в этом сами!!! Вот он пресловутый "баг в защите"!!!

Postlude

Существует еще как минимум два способа взлома данной программы. Сам автор на своем сайте в форуме признался, что если запустить Штирлица более 200 раз или с параметром –banoff в командной строке, то баннеры исчезнут... Но эти ли способы достойны настоящих хакеров?

Вот как раз с первым способом (запустить 200 раз) и связан баг, который мы обнаружили.

FAQ

• После того, как я пишу hwnd я ничего не вижу!
А addr XXXX ты не забыл? Еще это может быть из за несовместимости версии SoftIce-а с установленной версией винды. Так, например, в SI из DriverStudio 2.5 в XP наблюдается именно этот баг, хотя по заявлениям SI 2.5 полностью поддерживает XP. В XP безгючно (почти!) работает ТОЛЬКО SI из DriverPack 2.6 (и вообще именно этот пак я и рекомендую всем)

• Хочу исправить байт в HIEW, а оно ругается: "read-only mode".
Закрой WDASM.

• Теперь ругается SoftIce: Symbol not defined.
Вот горе ты мое... Ищи файл winice.dat (в NT лежит он здесь: c:\windows\system32). Ищи строку: EXP=C:\WINDOWS\system32\comdlg32.dll. После нее дописывай таким же манером пути к нужным библиотекам (где лежат нужные API-шки). Какие это такие нужные библиотеки – см. в WDASM-е в самом начале идет список, впиши их все – не ошибешься. И будь внимателен с названием API-шек (SI различает CreateFile и CreateFileA)

• Опять SoftIce пищит: Syntax error!
Внимательно набирай команды!!! "bpx readfile if ebx=A9400" - ошибка!!! Должно быть два значка "=".

*Ломать – не строить!
(C) Народная мудрость.*

Защита файлов в Excel

Автор: Chingachguk / HI-TECH

Дата: 2002-08-15

Как создать защищенный файл в Excel

Предположим, вы работаете с файлом (таблицами) в Excel и при этом хотите, чтобы кроме вас (или доверенных лиц) никто не мог бы иметь доступа к нему. Для этого вы можете защитить файл паролем (Файл - Сохранить как - Параметры - Ввод пароля). После всех подтверждений файл будет сохранен. При последующих попытках открыть его, Excel будет предлагать ввести пароль, и если тот окажется неверен, то вы (или кто-то другой) содержимое файла увидеть не сможете. Примечание: речь идет о версии Excel 97.

Что происходит с файлом при сохранении его с паролем?

Создадим новый файл. В первое поле таблицы занесем отладочную строку, например 50 букв "A". Сохраним файл с паролем. Пусть он будет тоже характерным, например "123456789012345" (максимальная длина, которую позволяет ввести программа - 15 символов). Создадим также файл с той же строкой из букв "A", но защищать его паролем не будем.

Теперь попытаемся сравнить оба файла. Длины защищенного и незащищенного файлов не отличаются. Попытаемся найти отличия в их содержимом. Проще сверять файлы не визуально, а с помощью небольшой программы, которая считывает в два буфера содержимое файлов, а потом побайтно сравнивает их, выводя несовпадающие байты и их номер в файле в протокол.

Оказыватеся, что файлы Excel имеют что-то вроде 512-байтного заголовка, который абсолютно одинаков для защищенного и незащищенного файлов. Далее, начиная с байта в позиции 0214h в файле начинаются отличия:

```
0214h: защищенный: 86h, незащищенный: E1h
0216h: защищенный: 00h, незащищенный: 02h
0218h: защищенный: 2Fh, незащищенный: B0h
...
```

и так далее. Попробуем найти нашу тестовую строку в файле. Оказывается, ее там нет. Как нет и пароля в явном виде. Что ж, на такой легкий вариант никто и не рассчитывал. Один из простейших способов зашифровать данные - это выполнить операцию xor (исключающее ИЛИ) над байтами шифруемой информации. В самом простом случае на каждый байт накладывается одна и та же xor-маска. Для расшифровки достаточно повторить ту же самую операцию xor с той же маской - поскольку эта операция при выполнении четное число раз приводит к начальному результату:

```
(ЛюбойБайт xor Mask) xor Mask = ЛюбойБайт
```

Убедимся, что Excel не так прост. Для этого достаточно пройти все 256 байтов масок, накладывая их на все байты файла и создавая новый файл. Неудачным поиском тестовой строки по таким новым файлам легко проверить, что программа все еще хитрее нас.

Итак, мы имеем множество отличающихся байтов в защищенном и незащищенном файлах. Точнее, их более 60%. Очевидно, что сам пароль сейчас как "иголка в стоге сена", поэтому нам не хватает дополнительных экспериментальных данных, для того чтобы хотя бы знать, где примерно

находится то, во что Excel превратил исходный пароль.

Напрашивается следующий опыт - сохранить множество одинаковых файлов с одним и тем же паролем и содержимым (скажем, все с той же тестовой строкой). Создадим, к примеру, 10 таких файлов. Сравнивать их опять же удобно по вышеприведенному принципу, немного усложнив алгоритм - сначала читаем в некий буфер один из файлов, а потом сравниваем второй файл с первым, несовпадающие байты фиксируем, для остальных файлов просто выводим байты в несовпадающих позициях. В протокол выведем строки типа:

```
Номер_байта Байт1 Байт2 ... Байт10,
```

где Байт1 Байт2 ... Байт10 - байты 10 файлов, причем Байт1 всегда отличен от Байт2.

Что же обнаруживается в результате такого сравнения? Оказывается, что очень значительная доля одинаковых с точки зрения пароля и содержащейся в них информации файлов, зашифрованных одной и той же копией Excel, оказалась различной (около 30%)! Выявленное многообразие шифрования только на первый взгляд кажется осложняющим дальнейший анализ. В самом деле, вот у нас есть блоки байтов, которые сильно отличаются от файла к файлу, а вот есть наборы байтов, которые остаются неизменными, причем их расположение и длина от файла к файлу не меняются. Тут может быть два варианта - либо измененный "пароль" принадлежит неизменной части (частям), либо переменной. Остальные варианты пока можно отбросить из-за их сложной реализации с точки зрения программиста.

Преположим, что измененный "пароль" принадлежит к постоянной части (частям) файла. Но это находится в противоречии с тем фактом, что значительная часть файла, а следовательно и информация, по крайней мере частично, изменяется. Ведь если "пароль" постоянен, то неоткуда взять дополнительный источник, вносящий изменения - дата или время создания файлов на эту роль не годятся, ибо при копировании защищенных файлов они могут быть изменены.

Сделав такое допущение, будем искать характерные изменяющиеся блоки в протоколе. Вероятно, что длина их должна быть порядка длины пароля, как вероятно и то, что расположение их находится в "любимых" программистами местах файла - либо в начале файла, либо в его конце.

Остановимся пока на первом варианте, ибо мы уже обнаружили, что структура файла сделана по принципу заголовок (информация о файле) - информативное содержание, а не наоборот.

Раз сам заголовок не изменяется, перейдем к несовпадающим байтам после него. Сразу можно заметить, что с позиции 0222h начинается постоянная по длине (48 байтов) изменяющаяся группа байтов. Далее можно обнаружить и другие такие группы. Например, для пароля длиной в один символ на месте данных файла (тестовой строки) находятся видоизменяющиеся группы длиной 14 байтов, перемежаемые постоянными группами длиной 8 байтов (найти в зашифрованном файле позицию тестовой строки легко - достаточно узнать ее позицию в незашифрованном файле).

Как же можно "вручную" выбрать из таких групп ту, которая содержит в искаженном виде пароль? Предположим, что программа сначала считывает такую группу, запрашивает пароль, сверяет его по внутреннему алгоритму с этой группой, и только в случае успешной проверки продолжает распаковывать файл. Для примера изменим любой один байт из первой группы в 48 байтов и попробуем открыть его в Excel.

Оказывается, в этом случае Excel стабильно выдает сообщение - неверный пароль! А вот если мы изменим другие переменные байты, скажем, в позиции 025Ch, он выдаст сообщение типа "Не могу прочитать файл", или вообще появится традиционное "Программа выполнила :".

На этом месте явно наступило время обдумать полученные "в лоб" результаты и обогатиться новыми знаниями. Ведь нам совершенно не известно, как программа обрабатывает эти 48 байтов вместе с паролем.

Криптография и все-все-все

Пусть мы пока не знаем, как поступает программа с этими 48 байтами, но можно предположить, что существует некоторая однозначная функция проверки пароля и этого блока:

```
F = F(пароль, 48 байтов с позиции 0222h).
```

Например, CRC (контрольная сумма) 48 байтов - скажем, просто их побайтное сложение по два должно образовывать последовательность из 16 байтов. Эти байты, например, представляют собой сам пароль. Или не сам пароль, а пароль, на который наложена 16-байтная постоянная хог-маска, взятая из последних 16 байтов.

В случае использования алгоритмов, подобных этому, сразу возникает вопрос - а что, если кто-то узнает (скажем, украдет или подсмотрит исходный код Excel, или просмотрит его под отладчиком) принцип действия алгоритма ? Тогда любой пароль элементарно вычисляем - достаточно проделать вышеописанные операции над этими 48 байтами, повторив действия программы, чтобы получить пароль для любого файла. Иначе говоря, существует функция G, обратная к функции F:

```
Пароль = G(48 байтов с позиции 0222h).
```

Очевидно, что в этом случае хакерами становились бы уже в детском саду. Да и парни из команд, подобных MS, знают, с кем имеют дело. В чем же тут хитрость?

Обогатимся, так сказать, знаниями, которые накопило к настоящему времени человечество в области выстраивания заборов и капканов. Скорее всего, "против нас играют" не программеры, а какие-нибудь ученые-математики с их заумными формулами и доказательствами.

Наберем в поисковике чего-нибудь со словами crypto. Крипто - это криптография, так по науке называются способы защиты данных. Искать лучше на русскоязычных сайтах - лишних трудностей нам не нужно. Вот, скажем, попался нам www.cryptography.ru - сайт МГУ по этой самой криптграфии.

Начинаем читать статью "Введение в криптографию" под редакцией В.В.Ященко. Оказывается, многие современные алгоритмы шифрования (да, скорее всего, и наш) строится вот по какому принципу - если используется некий алгоритм создания и проверки некоторой измененной информации (пароля), то не должно существовать однозначной функции, позволяющей получить сам пароль по этой зашифрованной информации. То есть, все что можно - это брать пароль и сравнивать его каким-нибудь способом с зашифрованными данными (функция F), а вот из самой зашифрованной информации однозначно пароль получить нельзя (обратная функция G). Причем отсутствие этой самой обратной функции строго не доказано (!), но сейчас все же полагают, что способы выбора функций F достаточно надежны. Проверяют надежность того или иного способа многократными попытками взлома (!).

Хорошо, поверим в то, что не существует способа сразу вычислить пароль. Но ведь никто нам не мешает узнать алгоритм проверки программой пароля, т.е. узнать функцию F. Узнав эту функцию, мы можем методом перебора поискать пароль, "подкидывая" все множество возможных паролей на вход функции F - как только сравнение окажется верным, мы нашли пароль. Такой алгоритм прост до неприличия, но в чем же хитрость защиты - ведь перебором однозначно можно найти нужный пароль? Смысл в том, что время, необходимое на перебор всех-всех вариантов огромно, хотя и конечно.

Пусть процедура проверки одного пароля занимает 1 мкс. Пусть также мы ищем пароль длиной 10 символов. Число возможных символов, которые можно ввести с клавиатуры, будет примерно 100. Тогда число всех возможных вариантов пароля составит:

```
10010 = 1020 вариантов.
```

А время, необходимое на перебор всех вариантов, будет равно:

```
1020 x 10-6 = 10^14 секунд
```

Увы, мы живем явно меньше. Тем не менее, интересно все же узнать как именно Excel применяет такие алгоритмы шифрования.

Алгоритмы шифрования, используемые в Excel

Итак, перед нами явно какой-то из этих алгоритмов - или их комбинация. В той же статье В.В.Ященко читаем, что Word и другие MS-продукты используют некие алгоритмы RC4 и MD5. Использовать они их начали не сразу, раньше можно было, например, импортировать данные из Excel-файла в Access из любого защищенного файла, пароль при этом почему-то никто не спрашивал.

Что же такое RC4, MD5 и тому подобные штучки? Схематично их алгоритмы можно представить таким образом - шифруемая информация (последовательность битов) должна быть, с одной стороны, скрыта во множестве "мусора" (тривиальный пример - сложение каждого байта строки с байтами строки-константы), а с другой стороны - рассеяна по полученному множеству таким образом, чтобы связи между шифруемыми битами были потеряны, но не окончательно - ведь еще предстоит проверка (например, меняем четные байты пароля на нечетные).

Возьмем, к примеру, алгоритм шифрования RC4. Он состоит из подготовительной части и самого шифрования.

Пусть у нас есть пароль "Password". Формируем две таблицы длиной 256 байтов. Первая (S-таблица) будет вначале содержать числа 0, 1, 2,.., 255, а вторую (K-таблицу) заполним паролем примерно так:

```
K = "PasswordPassword..."
```

(пароль ведь может быть меньше 256 байтов). После этого введем индексы i и j, j вначале равен 0. Выполним 256 замен между таблицами:

```
i = 0,1.. 255, j = (j + S[i] + K[i]) and 255,  xchg S[i],S[j].
```

Полученная таблица S называется таблицей подстановок.

Теперь пусть есть некоторая информация, которую надо зашифровать - массив байтов Info[0,1,2:]. Определим два счетчика Q1 и Q2 с начальными значениями 0. При шифрации каждого байта Info[] выполняем следующие действия:

```
Q1 = (Q1 + 1) and 255, Q2 = (Q2 + S[Q2]) and 255,
xchg S[Q1], S[Q2], T = (S[Q1] + S[Q2]) and 255,
Gamma = S[T],
```

и, наконец - вот оно:

```
Info[..] = Info[..] xor Gamma.
```

Таким образом, мы формируем таблицу подстановок S и с ее помощью проводим операции "исключающее ИЛИ" (xor) над элементами шифруемой информации. Как уже было сказано ранее, четное число операций xor над байтом дает исходный байт. Поэтому достаточно провести еще раз ту же самую операцию над зашифрованным алгоритмом RC4 массивом Info[], чтобы получить исходный Info[].

С алгоритмом MD5 все гораздо сложнее. Но случайно мне под руку подвернулся его исходник, правда, на Си, но читать можно. MD5 - это по сути не алгоритм шифрования, а так называемая хэш-функция. Слово громкое, но это всего лишь сверхуникальная последовательность из 16 байтов для любой последовательности байтов любой длины. Грубо говоря, это некая особая выжимка из

массива чисел. Работает даже тогда, когда массив пуст. Алгоритм поражает своей тяжестью - именно не сложностью! В нем, так же как и в RC4, есть секция инициализации, которая состоит в заполнении 4 двойных слов (32 бита) - пусть это будет массив state[1..4] - следующими числами:

```
67452301 ; state[1]:=MagValue1
EFCDAB89 ; state[2]:=MagValue2
98BADCFE ; state[3]:=MagValue3
10325476 ; state[4]:=MagValue4,
```

а также в инициализации двух 32-битных счетчиков нулевыми значениями. Разобраться в MD5 гораздо сложнее, чем в RC4 - да это нам и не нужно. Нам пока важно иметь общее представление о механизмах, которые мы будем искать в Excel. А найдя их, мы найдем уже готовый код проверки пароля от MS.

"Микроскоп" SoftIce и Excel в роли "инфузории-туфельки"

Итак, мы имеем смутное представление о том, что же мы хотим найти в Excel. Посмотрим в отладчике SoftIce на то, как программа работает с файлом и введенным паролем. Запускаем SoftIce. Выбираем в браузере зашифрованный файл. Нажимаем Ctrl-D. Сейчас файл будет открыт, и из него будет прочитано несколько блоков файла и, видимо, в том числе наш блок из 48 байтов. Чтение из файла осуществляется через api-функции SetFilePointer (Установить указатель в файле) и ReadFile (Прочитать блок байт из файла). Находятся эти функции в модуле kernel32.dll. Формат вызова у этих функций примерно такой:

```
SetFilePointer(FileHandle: DWORD;
               Pos: DWORD;
               Rezerv: DWORD;
               FromWhat: DWORD): DWORD;
```

Где:

• FileHandle - дескриптор файла;
• Pos - указатель в файле;
• Rezerv - (резерв?) передают NULL;
• FromWhat - с начала файла (FILE_BEGIN = 0;), с конца файла (FILE_CURRENT = 1;), с текущей позиции (FILE_END = 2;).

```
ReadFile(FileHandle: DWORD;
         AddrBuffer: DWORD;
         BuffSize: DWORD;
         AddrForBytesRead: DWORD;
         Rezerv: DWORD): DWORD;
```

Где:

• FileHandle - дескриптор файла;
• AddrBuffer - адрес буфера чтения;
• BuffSize - сколько байтов читать;
• AddrForBytesRead - сколько байтов будет прочитано.

Устанавливаем брейкпойнты на эти две функции:

```
bpx SetFilePointer
bpx ReadFile
```

и продолжаем выполнение программы (Ctrl-D). После клика на нашем файле в браузере мы почти сразу же "выпадаем" в отладчик на первый breakpoint по SetFilePointer. Ага, смотрим переданные параметры и видим, что произошел следующий вызов:

```
SetFilePointer( Handle: ...; Pos: 0; Rez: 62E2D4; FromWhat: FILE_BEGIN)
```

Т.е. мы собираемся читать что-то с начала файла. Пропускаем этот и еще несколько вызовов, пока не обнаруживаем вот это:

```
015F:7FF66B88 call ReadFile
( ReadFile( Handle: ...; AddrBuffer: 64107Ah; BuffSize: 10h;
AddrForBytesRead: 62B8F0h; Rezerv: 0;) ).
```

Т.е читаем 16 байтов в буфер 64107Ah. Выполняем эту функцию:

```
G @SS:ESP,
```

и видим, что прочли как раз первые 16 байтов с позиции 0222h. Далее мы видим еще два интересующих нас вызова:

```
015F:7FF66B88 call ReadFile
( ReadFile( Handle: :; AddrBuffer: 62BA54h; BuffSize: 10h;
AddrForBytesRead: 62B8F0h; Rezerv: 0;) )
```

и

```
015F:7FF66B88 call ReadFile
( ReadFile( Handle: :; AddrBuffer: 62BA44h; BuffSize: 10h;
AddrForBytesRead: 62B8F0h; Rezerv: 0;) ).
```

А это, собственно, и есть чтение второго и третьего блока, по 16 байтов каждый, из тех самых 48 байтов.

Продолжая дальше выполнять программу, мы видим, как на экране появляется приглашение к вводу пароля. Введем характерный пароль, чтобы потом можно было легко искать его в памяти. После подтверждения ввода повторно "вываливаемся" в отладчик на тех же самых чтениях 48 байтов в той же самой последовательности.

Итак, мы получили 3 вызова с одной и той же точки(:7FF66B88) на чтение 16 байтов 48-байтного блока. Это явно подпрограмма, одна и та же подпрограмма. Поэтому нужно найти код, который делает что-то вроде этого:

```
; Прочесть первые 16 байтов
; Что-то сделать
; Прочесть вторые 16 байтов
; Что-то сделать
; Прочесть третьи 16 байтов
; Что-то сделать
; Видимо, сравнить пароль
```

Последовательно возвращаясь из вложенных процедур с точки ":7FF66B88", находим именно такой код (buff2, buff3 и buff4 - 1-й, 2-й и 3-й блоки по 16 байтов):

```
308B7793:
; ...
 lea edx,[edi+116h] ; <- 0064107A - buff2
; Прочитать buff2
308B77A3:
 call 3080AF24 ; Прочесть buff2
; ...
 call [esi+10h]  ; Выполнить код 308B75D5
308B77B5:
; ...
 call [esi+08]  ; Вызвать 308B74FD
308B77BF:
; ...
; Прочитать buff3
 lea edx,[ebp-14h] ; <-0062BA54 - buff3 !!!
 call 3080AF24  ; Прочесть там buff3
 test eax,eax  ; Ошибка ?
 jz 308B778A  ; Нет
; ...
308B77DA:
 call esi   ; Вызвать 308B7548
```

```
; ...
; Прочитать buff4
 lea edx,[ebp-24h] ; <-0062BA44 - buff4 !!!
 call 3080AF24  ; Прочесть там buff4
; ...
 call esi   ; Вызвать 308B7548
; ...
 call 309AD4D2  ; MD5Init( MD5_CTX - контекст [0062B9DC] )
; ...
 call 309AD4FC  ; MD5Update( MD5_CTX - контекст MD5, буф., дл.буф. )
 lea ecx,[ebp-8Ch] ; <-0062B9DC - адрес контекста
 call 309AD5A9  ; MD5Final( контекст: 0062B9DC, digest: +58h)
; Сравнить вычисленные данные.
308B781B:
 mov ecx,ebx ; <-10h
 lea edi,[ebp-24h] ; <- buff4, уже отксореный ранее по RC4
 lea esi,[ebp-34h] ; <- 0062BA34 - digest к 0062B9DC из 10h байтов
 xor eax,eax
 rep cmpsb   ; Сравнить 3 блок из 16 байтов в 222h файла.
; jnz ...   ; Пароль не верен?
```

Вот примерно и ясен каркас подпрограммы проверки пароля.

Остается пройтись по подпрограммам, которые вызываются из этого участка кода (начиная с чтения первых 16 байтов и до последней проверки сравнением двух строк длиной по 16 символов).

Полный текст процедуры проверки пароля Excel'ом приводится в приложении(ниже) с некоторыми комментариями.

Оказывается, что алгоритм проверки довольно-таки запутанный. Можно сказать, что это дикая смесь RC4 и MD5. Схематично он выглядит так:

• прочесть первые 16 байтов из 48;
• получить MD5-хеш от пароля;
• от этого хеш'а и первых 16 байтов образовывать новый MD5-хеш, фактически от ("MD5(пароль)+16 байтов" x 16) ;
• еще раз(!) от полученного хеш'а и пароля образовать новый MD5-хеш;
• полученный хеш использововать как пароль для формирования таблицы подстановок RC4;
• прочесть вторые 16 байтов из 48;
• выполнить RC4-расшифровку этих 16 байтов, таблица подстановок уже получена;
• прочесть третьи 16 байтов из 48;
• выполнить RC4-расшифровку и этих 16 байтов;
• от уже расшифрованных RC4 двух 16 байтов получить MD5-хеш;
• и только сейчас сравнить его с расшифрованным по RC4 третьим блоком из 16 байтов!!!

Где-то внутри этого алгоритма также происходит формирование буфера, необходимого для расшифровки самого файла - например, той же таблицы подстановок для RC4.

Время выполнения проверки одного пароля на P-120, windows98 составляет в среднем около 800 мкс. Т.е для проверки всех вариантов паролей длиной 2 символа необходимо:

```
~100x100x0.0008 = 8 секунд!
```

А как же без "дырок" ?

Итак, мы создали простой код, подбирающий пароли к файлам Excel методом "грубой силы". Очевидно, перебор всех паролей достаточно серьезной длины (например, 10 символов) займет огромное время. Попытаемся проанализировать оригинальный код Excel на предмет "багов", наличие которых может существенно сократить время подбора пароля.

Обратим внимание на тот факт, что сам пароль используется в процедуре проверки ВСЕГО ОДИН РАЗ, а именно - в подпрограмме 308B75D5: сначала всего лишь вычисляется MD5-хэш от пароля:

```
; Подпрограмма 308B75D5
P308B75D5 proc
 push ebp
 mov ebp,esp
 sub esp,68h  ; Выделить локальный буффер в 68h=104байта
 push ebx  ; 10h
 push esi
 push edi
 lea ecx,[ebp-68h] ; Адрес локального буффера в 68h
 call P309AD4D2  ; MD5Init
 mov eax,[ebp+0Ch]
; movz ecx,word ptr [eax] ; ECX=5, если пароль длиной 5
        xor ecx,ecx
 mov cx,word ptr [eax]
 shl ecx,1  ; x 2
 push ecx
 lea edx,[eax+02] ; <- АДРЕС ПАРОЛЯ
 lea ecx,[ebp-68h] ; Адрес локального буффера в 68h
 call P309AD4FC ; MD5Update(лок.контекст[0062B958],Пароль,Длина пароля)
        lea ecx,[ebp-68h] ; Адрес лок.контекст[0062B958]
 call P309AD5A9         ; MD5Final
```

Назовем этот хэш hp.

А что происходит дальше ? А дальше - самое интересное, программа формирует следующий хэш, но только от 5 байт первого, hp:

```
 lea ecx,[ebp-68h]
 call P309AD4D2  ; MD5Init
 mov ebx,[ebp+08]
 push 10h
 lea esi,[ebx+116h]
 pop edi
L308B7616:
 push 5   ; Длина буфера для MD5Update РАВНА ПЯТИ !
 lea edx,[ebp-10h]
 lea ecx,[ebp-68h]
 call P309AD4FC  ; MD5Update
 push 10h  ; Длина буфера для MD5Update
 mov edx,esi        ; buff2 - буфер для MD5Update
 lea ecx,[ebp-68h]
 call P309AD4FC  ; MD5Update
 dec edi   ; 10h->0Fh->...
 jnz L308B7616  ; Вызывать 10h раз MD5Update
 lea ecx,[ebp-68h]
 call P309AD5A9  ; MD5Final
```

В первом приближении дальнейший код зависит ТОЛЬКО от этих пяти байтов hp ! Да, в начале формируется 16-ти байтный хэш hp от пароля, что никак не ослабляет мощность введенного пароля, но вот его усечение до пяти символов - это уже что-то. Давайте проверим эту гипотезу. Создадим некий файл, защищенный паролем "1". Откроем его, установив брейкпойнт в точке 308B7616:

```
bpx cs:308B7616
```

Только что сформирован хэш hp, он находится по адресу [ebp-10h]:

```
d ss:ebp-10
```

И мы видим первые пять байтов хэша от пароля "1":

```
06 D4 96 32 C9 ...
```

Запомним их и продолжим выполнение программы. Естественно, раз пароль был верным, файл нормально откроется. А теперь соль эксперимента: откроем еще раз этот же файл, только вот

вместо правильного пароля введем какую-нибудь чепуху. Опять попадем в отладчик в той же точке 308B7616 и заменим первые пять байт нового хэша(видно, что он отличается от прежнего) на те самые первые пять байт от оригинального пароля(data 0 > заменить пять байт по адресу [ebp-10h]). И что мы видим ? Файл открылся как ни в чем не бывало !

Итак, мы нашли очевидный баг. Для подбора пароля, оказывается, нет нужды перебирать все пароли длиной от 1 до 15, достаточно перебрать первые 5 байтов хэша hp ! А это уже совсем другое число вариантов: 256^5 ~= 10^12 ! Теперь алгоритм перебора может быть переписан в следующем виде: вызываем нашу процедуру обработки пароля, передаем ей какой-нибудь пароль (произвольный !), но дописываем небольшой код, подменяющий создаваемые оригинальным кодом первые пять байт hp на перебираемые значения:

```
.data
; Тестовый "пароль"
TestFiveBytes db 000h,000h,000h,000h,000h

@@FindPassword:
; ...
  push 0826BB25Ch
  push offset Password
  push offset LocBuffer
  call P308B774C  ; Вызов главного кода
  jz @@PasswordFound
  xor ebx,ebx
@@NextLetter:
  inc byte ptr TestFiveBytes[ebx]
  jnz @@FindPassword
  inc ebx
  cmp ebx,MaxPassLen
  jb  @@NextLetter
; ...
  call P309AD5A9 ; MD5Final
; Подставляем переборочный хэш, причем ТОЛЬКО 5 байт !!!
  pushad
  lea edi,[ebp-10h] ; <-62B9B0 - UN1- адрес буфера для MD5Update
  mov esi,offset TestFiveBytes
  mov ecx,5
  cld
  rep movsb
  popad
; ...
  pop edi
L308B7616:
```

Теперь нам надо перебирать "всего" 256^5 вариантов. Много это или мало ? Во-первых, существенно меньше первичной реализации нашего подбирателя. Число вариантов составляет примерно 10^12, при скорости подбора 10000 хэшей в секунду необходимо 10^8 секунд, что составляет примерно 40 месяцев, что не так уж и много для особо желающих, а, что самое главное - вполне обозримое время.

Если подключить к делу сотни мощных компьютеров, то подбор может занять вообще несколько дней, что не говорит в пользу защиты от Excel.

Следует добавить, что сам пароль можно будет найти после нахождения верного хэша путем атаки на MD5 перебором и сравнением первых 5-ти байт получаемых хэшей с найденными ранее. А также потенциальное наличие в алгоритме защиты ДУБЛИРУЮЩИХСЯ паролей (с вероятностью примерно 10^-12) - т.е. пароли, у которых первые пять байт MD5-хэша равны, для Excel будут эквиваленты !

Реализация подборщика паролей

Итак, попробуем создать более-менее быстрый подборщик паролей. Еще раз в двух словах о том, что он должен делать.

Как уже было написано выше, мы можем откинуть часть общего алгоритма проверки пароля и перебирать либо первые пять байт хэша от пароля, либо первые пять байт входного текста для одного из следующих MD5-хэшей: именно этот хэш будет подан в качестве ключа на вход RC4. Т.е., нам необходимо:

• перебирать в цикле комбинацию из пяти байт: 00 00 00 00 00 -> FF FF FF FF FF;
• сформировать от очередной комбинации + 4 нуля MD5-хэш;
• полученный хеш использоовать как пароль для формирования таблицы подстановок RC4;
• прочесть вторые 16 байтов из 48;
• выполнить RC4-расшифровку этих 16 байтов, таблица подстановок уже получена;
• прочесть третьи 16 байтов из 48;
• выполнить RC4-расшифровку и этих 16 байтов;
• от уже расшифрованных RC4 двух 16 байтов получить MD5-хеш;
• и сейчас сравнить его с расшифрованным по RC4 третьим блоком из 16 байтов.
• если сравнение окажется верным, нам необходимо запомнить найденные пять байт;
• далее при открытии файла ввести ЛЮБОЙ пароль и подставить найденные байты в нужное место.

Начнем, пожалуй.

```
; Это программа - подборщик паролей к файлам Excel.
; На самом деле подбирает не сам пароль, а первые 5 байт некоторого хэша MD5.
; Затем необходимо при открытии файла разыскать в теле кода Excel процедуру
; формирования этого хэша(вызывается при открытии файла, защищенного паролем)
; и подставить вместо ПЕРВЫХ ПЯТИ байт полученного хэша найденные(пароль,
; введенный в диалоговом окне, при этом НЕВАЖЕН !)
; Как это сделать ? ;) Проще всего в софтайсе установить туда бряк и заменить
; его по-быстрому ручками. Можно пропатчить Excel, именно это место, таким образом
; чтобы вводить с клавиатуры не сам пароль, а именно хэш(найденный, первые пять байт)
; Ну - это дело вкуса ;0) Адрес в Excel от Office-97
; находится в :308B74FD(чуть ниже, ВТОРОЙ ВЫЗОВ ПОСЛЕ ВВОДА ЛЮБОГО ПАРОЛЯ).
; А теперь о том, как устроен алгоритм проверки пароля(все это есть в коде, но слова
; не помешают):
; Итак, вначале читается пароль с диалогового окна и 16 байт из файла по
; адресу 222h(buff2). Далее образуется MD5-хэш от пароля(16 байт, кто не в курсе).
; От полученного хэша(ВНИМАНИЕ !) h1 берутся первые пять байт
; (вот тут-то и нужна замена и подбор !) и дополняются 16-тью байтами
; из файла, потом полученная последовательность дублируется 16 раз
; и от нее получаем хэш h2 = MD5((h1(0..4)+buff2)x16).
; Далее от этого хэша и 4-х нулей формируем еще один хэш - h3,
; причем ОПЯТЬ БЕРЕМ первые пять байт(в принципе, тут тоже можно
; перебирать хэш, БУДЕТ ДАЖЕ в 2-3 раза БЫСТРЕЕ !).
; ИМЕННО ЭТО ТУТ И ДЕЛАЕТСЯ.
; Затем этот h3 используется как ключ для RC4.
; Потом читаются два буфера из файла - с адресов 232h и 242h
; - назовем из buff3 и buff4. Оба расшифровываются по полученной
; таблице RC4 и от расшифрованного buff3 получаем хэш h4.
; Проверка верного пароля состоит в равенстве h4 и только
; что расшифрованного по RC4 buff4.
; Скорость подборки такая: тестирование проводилось
; на P120, 16MerRAM, win98, используются команды cli/sti.
; Код оптимизирован(мной) - было на мощном VC++, стало на asm.
; Один пароль занимает ~41.5 мкс.(~24000 вар/сек).
; При реальном использовании разумно будет дописать
; код заданием ИНТЕРВАЛА ПЕРЕБОРА(например, ищем с такого-то
; значения по такое-то).
; Ну вот и все. Удачи !
; Coded by Chingachguk, 2002.
```

Это было вступление. Лирическое :)

```
.386
.model flat,stdcall
option casemap :none   ; case sensitive
include \masm32\include\windows.inc
include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib
```

Инклюды, библиотеки ... подключаем. Компилировал на masm32.

Далее идет описание макросов, использующихся в вычислении MD5-хэша.

```
; Макрос PFF. Необходим для вычисления MD5. Выполняет следующее:
; _a = ((((_b and _c) or (not(_b) and _d)) + a + _ac + _x) rol _s) + _b
PFF macro  _a,_b,_c,_d,_x,_s,_ac
  lea  ecx,[_a+_ac]
  mov  _a,_b
  and  _a,_c
  add  ecx,_x
  mov  edx,_b
  not  edx
  and  edx,_d
  or   _a,edx
  add  _a,ecx
  rol  _a,_s
  add  _a,_b
endm
; Макрос PGG. Необходим для вычисления MD5. Выполняет следующее:
; _a = (((_b and _d) or (_c and (not(_d)) + _a + _ac + _x) rol _s) + _b
PGG macro  _a,_b,_c,_d,_x,_s,_ac
  mov  ecx,_c
  mov  edx,_d
  not  edx
  and  ecx,edx
  mov  edx,_b
  and  edx,_d
  or   edx,ecx
  lea  _a,[_a+edx+_ac]
  add  _a,_x
  rol  _a,_s
  add  _a,_b
endm
; Макрос PHH. Необходим для вычисления MD5. Выполняет следующее:
; _a = ((((_b xor _c) xor _d) + _a + _ac + _x) rol _s) + _b
PHH macro  _a,_b,_c,_d,_x,_s,_ac
  mov  ecx,_b
  xor  ecx,_c
  xor  ecx,_d
  lea  _a,[_a+ecx+_ac]
  add  _a,_x
  rol  _a,_s
  add  _a,_b
endm
; Макрос PII. Необходим для вычисления MD5. Выполняет следующее:
; _a = (((_c xor (_b or (not(_d)))) + _a + _ac + _x) rol _s) + b
PII macro  _a,_b,_c,_d,_x,_s,_ac
  mov  ecx,_b
  mov  edx,_d
  not  edx
  or   ecx,edx
  mov  edx,_c
  xor  edx,ecx
  add  _a,edx
  add  _a,_x
  add  _a,_ac
  rol  _a,_s
  add  _a,_b
endm
```

Данные программы, мессаджи ...

```
.data
szDlgTitle    db "Check for Excel locked file",0
szMsg         db "  Welcome !!! ",0
PasswordTitle db "Check you Password",0
EXCELfile     db "C:\ASS_R\XLS\STATIS\1.xls",0 ; Имя файла для подбора
Unknown       db 1000 dup(?)
EndUn         db 0
descr         dd ?
DebugFileOp   db "Debug Message",0
DebugOpOK     db "File not open !",0
; Буффера для чтения блоков защиты из файла
buffer1       db 10h dup(?)  ; Буфер для 222h-231h
buffer2       db 10h dup(?)  ; Буфер для 232h-241h
buffer3       db 10h dup(?)  ; Буфер для 242h-251h
bRead         dd ?
```

А вот эта вещь является специфической. Используется MD5 в качестве добавления в процедуре MD5_Final как аргумент для MD5_Update.

```
; Буфер для передачи его MAIN
StrangeBuffer db 80h, 15 dup(0)
              db 16 dup(0)
              db 16 dup(0)
              db 16 dup(0)
LocBuffer     db 4000h dup(0)
```

Пошел код. Приветственное сообщение...

```
.code
start:
; MessageBox
    push MB_OK
    push offset szDlgTitle
    push offset szMsg
    push 0
    call MessageBox
; Open Excel file
    push OF_READ
    push offset Unknown
    push offset EXCELfile
    call OpenFile
    mov  descr,eax          ; Save file descriptor
    cmp  eax,0ffffffffh
    jnz  @@FileOpen
; File not open
    push MB_OK
    push offset DebugFileOp
    push offset DebugOpOK
    push 0
    call MessageBox
    jmp  @@Exit
@@FileOpen:
; Read File Block
    push FILE_BEGIN
    push NULL
    push 222h
    push descr
    call SetFilePointer     ; iniHandle, Pointer, NULL, FILE_BEGIN
    push NULL
    push offset bRead
    push 10h*3
    push offset buffer1
    push descr
    call ReadFile           ; iniHandle, addr d_buffer, 2, addr bRead, NULL
    push descr
    call CloseHandle        ; Close excel file
```

Только что мы прочли необходимые нам 16 x 2 байтов из файла - это блок защиты - в переменные buffer2 & buffer3.

```
; Вызов главного кода проверки.
.data
; Тестовый "пароль".
; На самом деле это первые пять байт хэша от (см. выше).
; Реально, конечно, нужно начинать с пяти нулей ! :)
; На этих байтах я тренировался, так что не обращайте внимания ;)
; Найдите Sice-ом место, куда нужно их подставить, введите правильный
; пароль на файл и смотрите, что у вас там ...
TestFiveBytes db 09Eh,0D1h,02Fh,084h,07Dh
              db 4 dup(0) ; Эти четрые нуля ОБЯЗАТЕЛЬНЫ !!!
LogFileName db "pass_ex.txt",0
.code
```

Откроем файл, куда будем писать результат.

```
 invoke CreateFile,offset LogFileName,GENERIC_WRITE,FILE_SHARE_WRITE,0,CREATE_ALWAYS,\
 FILE_ATTRIBUTE_NORMAL,0
 mov  descr,eax          ; Save file descriptor
 cmp  eax,0ffffffffh
 jnz  @@LogFileOpen
.data
OpenError db "File log.txt not open !",0
.code
 push MB_OK+MB_ICONERROR
 push offset szDlgTitle
 push offset OpenError
 push 0
 call MessageBox
 jmp  @@Exit
@@LogFileOpen:
.data?
StartRC4Table db 100h+4 dup(?)
.code
```

Создадим исходную RC4 - таблицу замен: 0,1,..255 - так немного быстрее.

```
; Create Start RC4 Table
  xor  eax,eax
  mov  ecx,offset StartRC4Table
@@StartFillRC4:
  mov  [ecx+eax],al
  inc  eax
  cmp  eax,100h
  jb   @@StartFillRC4
  mov  dword ptr [ecx+100h],0
```

Собственно, небольшие пояснения по отладочным кодам...

```
; Будем выполнять проверку тестового пароля, введя
; начальное значение-99 - всего в данном примере выполнится 100
; раз код проверки пароля
  sub  word ptr TestFiveBytes,99
  cli
@@FindPassword:
```

Зовем процедуру проверки. Передаем ей адрсе буфера, где она будет развлекаться: строить RC4-таблицу, и т.д.

```
  push offset LocBuffer ; <- буфер для RC4 & хэша MD5 от пароля
  call P308B774C  ; Вызов главного кода
  test eax,eax
  jz   @@PasswordFound
```

Выполняем переход к очередному подборочному хэшу...

```
  xor  ebx,ebx
@@NextLetter:
  inc  byte ptr TestFiveBytes[ebx]
```

```
  jnz  @@FindPassword
  inc  ebx
  cmp  ebx,5
  jb   @@NextLetter
  jmp  @@Exit
@@PasswordFound:
```

Нашли "пароль" - сохраним его и выдадим сообщение.

```
  sti
  push NULL
  push offset bRead
  push 5h
  push offset TestFiveBytes
  push descr
  call WriteFile
.data
PasswordMsg   db "You Password found ! See pass_ex.txt file.",0
.code
  push MB_OK
  push offset PasswordTitle
  push offset PasswordMsg
  push 0
  call MessageBox
@@Exit:
.data
EndWork db "All done ... ;)",0
.code
  sti
  push MB_OK
  push offset szDlgTitle
  push offset EndWork
  push 0
  call MessageBox
  invoke CloseHandle,descr
  push 0
  call ExitProcess
```

Процедура проверки, которая вызывает этот код, работает с переменными в стеке, куда просит нас в очередной раз записать блоки защиты из файла. Мы их один раз прочли и передаем ей.

```
; "Чтение" buff3 & buff4
Read_buff3_buff4 proc
  push edi
  mov  edi,edx
  mov  esi,offset buffer2
  mov  ecx,10h*2/4
  rep  movsd
  pop  edi
  ret
Read_buff3_buff4 endp
```

Собственно, начало процедуры проверки пассворда.

```
; Точка входа в проверку кода, адрес буфера в стеке.
P308B774C proc
  push ebp
  mov  ebp,esp
; Выделим место для локальных переменных: контекста MD5, ... etc
  sub  esp,08Ch
  mov  edi,[ebp+08h] ; <- адресом буфера
  lea  ecx,[ebp-8Ch]
  call P309AD4D2       ; MD5Init
```

Подставляем переборочный "хэш", причем ТОЛЬКО 5 байт !!!

```
  push 5+4        ; Длина буфера для MD5Update
  mov  edx,offset TestFiveBytes   ; <- адрес переборочного хэша
  lea  ecx,[ebp-8Ch]
  call P309AD4FC       ; MD5Update
  lea  ecx,[ebp-8Ch]
```

```
  call P309AD5A9       ; MD5Final
```

Сформировали хэш от переборочных пяти байт плюс 4 нуля. Теперь создадим таблицу замен RC4 на основе полученного хэша.

```
  lea  eax,[ebp-34h] ; <- есть заполненный digest
  push eax             ; Ключ 10h для RC4 - только что созданный хэш
  mov  edx,10h
  lea  ecx,[edi+04]    ; <-[Переданный буфер+4]
  call RC4Table_10h    ; Инициализировать таблицу подстановок RC4 - Длина ключа 10h !
```

Сформировали RC4-table. Теперь "прочтем" блоки защиты и расшифруем их на основе только что полученной таблицы замен RC4.

```
; Прочитать buff3 & buff4
  lea  edx,[ebp-24h]    ; buff3
  call Read_buff3_buff4
; Откcорить buff3 & buff4
  lea  eax,[ebp-24h]    ; <- buff3,buff4
  push eax ; Адрес buff3
  mov  edx,10h*2        ; Len of buff3 & buff4
  lea  ecx,[edi+04]     ; RC4Table
  call P309AD416
```

Сформировать MD5-хэш от откcоренного buff3

```
  lea  ecx,[ebp-8Ch]
  call P309AD4D2 ; MD5Init
  push 10h  ; 10h - длина буфера
  lea  edx,[ebp-24h] ; buff3 !!! - адрес буфера
  lea  ecx,[ebp-8Ch] ; контекст MD5
  call P309AD4FC ; MD5Update
  lea  ecx,[ebp-8Ch] ; адрес контекста
  call P309AD5A9 ; MD5Final
```

Сравнить вычисленные данные.

```
  mov  ecx,10h/4
  lea  edi,[ebp-14h] ; <- buff4, уже откcореный ранее по RC4
  lea  esi,[ebp-34h] ; <-0062BA34 -  digest к 0062B9DC из 10h байт
  xor  eax,eax
  repe cmpsd       ; Сравнить 3 блок из 16 байт в 222h файла.
  jz   EXITMAIN  ; Пароль не верен ?
  inc  eax
EXITMAIN:
  leave
  ret  04h  ; Вернуться назад...
P308B774C endp
; Служебные подпрограммы. RC4-формирование таблицы ... MD5_Final
```

Подпрограмма P309AD389. Инициализировать таблицу подстановок RC4.

```
Next_RC4 macro
  add  al,dl ; +key[i]
  mov  bl,[ecx+esi] ; S[i]
  shr  edx,8 ; Next key[i]
  add  al,bl
  mov  bh,[ecx+eax] ; bh=S[j]
  mov  [ecx+eax],bl ; S[j]=S[i]
  mov  [ecx+esi],bh ; S[j]=S[i]
  inc  esi
endm
Next4_RC4 macro
  Next_RC4
  Next_RC4
  Next_RC4
  add  al,dl ; +key[i]
  mov  bl,[ecx+esi] ; S[i]
  add  al,bl
  mov  bh,[ecx+eax] ; bh=S[j]
  mov  [ecx+eax],bl ; S[j]=S[i]
```

```
  mov  [ecx+esi],bh ; S[j]=S[i]
  inc  esi
endm
; edx=len_key, ecx=addr of RC4Table, stack = key offset
; НО ТОЛЬКО ЕСЛИ КЛЮЧ ДЛИНОЙ 10h !!!
RC4Table_10h proc
  pushad
  mov  esi,offset StartRC4Table
  mov  ebx,ecx
  mov  edi,ecx
  mov  ecx,100h/4
  rep  movsd
  movsw ; Два нулевых счетчика после 0,1,...255
  mov  ecx,ebx
  xor  esi,esi ; i
  xor  eax,eax ; j
  mov  ebp,[esp+7*4+8]
@@FillRC4:
; Key[1..4]
  mov  edx,[ebp]
  Next4_RC4
; Key[5..8]
  mov  edx,[ebp+4]
  Next4_RC4
; Key[9..12]
  mov  edx,[ebp+8]
  Next4_RC4
; Key[13..16]
  mov  edx,[ebp+0Ch]
  Next4_RC4
  cmp  esi,100h
  jb   @@FillRC4
  popad
  ret  4
RC4Table_10h endp
```

Подпрограмма - расшифровать байты по таблице подстановок RC4.

```
;*** EDX=len of buff, ecx=offset RC4 Table, stack = buff_addr
P309AD416 proc
  push ebp
  mov  ebp,esp
  push ebx
  push esi
  push edi
  mov  edi,[ebp+8] ; edi=Адрес буффера
  lea  ebp,[edi+edx] ; buff_addr+len_buff
  xor  ebx,ebx
  xor  edx,edx
  mov  bl,[ecx+100h] ; Q1
  mov  dl,[ecx+101h] ; Q2
@LOOP:
  xor  eax,eax
  inc  bl
  mov  al,[ecx+ebx] ; S[Q1]
  add  dl,al
  mov  ah,[ecx+edx] ; S[Q2]
  mov  [ecx+edx],al
  mov  [ecx+ebx],ah
  add  al,ah
  and  eax,0FFh
  mov  al,[ecx+eax]
  xor  [edi],al ; xor buffer[edi],gamma
  inc  edi
  cmp  edi,ebp
  jl   @LOOP
  mov  bh,dl
  mov  [ecx+100h],bx ; Запомнить счетчик 1,2
  pop  edi
  pop  esi
  pop  ebx
```

```
  pop  ebp
  ret  4
P309AD416 endp
```

Подпрограмма MD5_Init - инициализация контекста MD5.

```
; Подпрограмма MD5Init( MD5_CTX - контекст MD5 ).
P309AD4D2 proc   ; Инициализация 24 байт контекста по ECX.
  mov dword ptr [ecx],00000000h         ; count[1]:=0;
  mov dword ptr [ecx+04h],00000000h ; count[2]:=0;
  mov dword ptr [ecx+08h],67452301h ; state[1]:=MagValue1
  mov dword ptr [ecx+0Ch],0EFCDAB89h    ; state[2]:=MagValue2
  mov dword ptr [ecx+10h],98BADCFEh ; state[3]:=MagValue3
  mov dword ptr [ecx+14h],10325476h ; state[4]:=MagValue4
  ret
P309AD4D2 endp
```

Подпрограмма MD5Update - добавление текста к MD5-хэшу.

```
; Подпрограмма MD5Update( MD5_CTX - контекст MD5, буфер, длина буфера )
; MD5_CTX - контекст MD5 - в ECX (его адрес)
; буфер - в EDX (его адрес)
; длина буфера - в стеке (первое слово за адресом возврата)
P309AD4FC proc ; Что-то делаем с 24 байтами по ECX
  push ebp
  mov  ebp,esp
  push edi
  mov  esi,ecx       ; Адрес переданного по ECX контекста
  mov  ebp,[ebp+8h]   ; Длина буффера, переданного нам > EBP
  mov  eax,ebp
  mov  ebx,edx       ; адрес буффера -> EBX
  mov  edx,[esi]      ; 4 байта из переданного в ECX контекста
  mov  ecx,edx       ; Положить их в ecx
  lea  eax,[edx+eax*8]  ; Длина массива * 8 + dd из контекста
  and  ecx,01F8h      ; Первые 4 байта из контекста and 01F8h
  shr  ecx,3       ; разделить их на 8
  cmp  eax,edx       ; Сравнить: (Длина массива * 8 + dd из контекста) и (dd из контекста)
  jae  L309AD522      ; Вышли за пределы ?
  inc  dword ptr [esi+04] ; Увеличить следующий dd из контекста
L309AD522:
  mov  [esi],eax      ; в контекст положить вычисленный eax
  mov  eax,ebp        ; Длина буффера, переданного нам > EAX
  shr  eax,1Dh       ; Оставить 31,30 и 29 биты ? - 00,00,..
  add  [esi+04],eax   ; Добавить к следующему dd контекста
  dec  ebp            ; Уменьшить длину буффера, переданного нам
  js   L309AD59F      ; Равна 0 ? -> На выход
L309AD53A:
  mov  al,[ebx]       ; Взять байт переданного буффера
  inc  ebx            ; Указать на следующий байт
  mov  [ecx+esi+18h],al ; Положить его в [4 байта из переданного в ((ECX and 01F8h) shr 3 +
; + адрес переданного массива в 24 байта + следующие в нем 24 байта)
  inc  ecx       ; Прирастить разделенный на 8 первый элемент массива
  cmp  ecx,40h       ; Он вдруг стал 64 ?
  jnz  L309AD593      ; Да, да ...
  lea  edx,[esi+18h]
  lea  ecx,[esi+08h]
  call P309AD671      ; МАКРОСЫ
  xor  ecx,ecx
L309AD593:
  dec  ebp            ; Длина буффера, переданного нам
  jns  L309AD53A
L309AD59F:
  pop  edi
  pop  ebp
  ret  4
P309AD4FC endp
```

Подпрограмма MD5_Final формирование конечного хэша по контексту.

```
; (MD5_CTX - контекст, digest - наполняемый буфер 10h)
; ECX - адрес контекста MD5
P309AD5A9 proc  ; ECX=адрес контекста
```

```
  push edi
  mov  eax,[ecx+4] ; Взять второе слово контекста
  mov  esi,ecx       ; Адрес контекста
  mov  [esi+18h+3Ch],eax ; <- второе слово контекста
  mov  ecx,[esi]       ; Первое слово контекста
  mov  [esi+18h+38h],ecx ; <- Первое слово контекста
  and  ecx,1F8h       ; с 1-м словом
  shr  ecx,3  ; /8
  mov  eax,38h
  cmp  ecx,eax
  jl   L309AD5D5
  mov  eax,78h
L309AD5D5:
  sub  eax,ecx
  mov  edx,offset StrangeBuffer
  push eax
  mov  ecx,esi       ; Адрес буфера
  call P309AD4FC       ; MD5Update
  lea  edx,[esi+18h]
  add  esi,8h
  mov  ecx,esi
  call P309AD671       ; Макроподстановки MD5
  lea  edi,[esi+50h]
  movsd
  movsd
  movsd
  movsd
  pop  edi
  ret
P309AD5A9 endp
```

Подпрограмма макроподстановок в MD5.

```
; Подпрограмма 309AD671 - Макроподстановки MD5
; edx - адрес массива x[16] из long-ов
; ecx - адрес массива s[4] из long-ов
;// Константы для MD5Transform.
S11 equ 7
S12 equ 12
S13 equ 17
S14 equ 22
S21 equ 5
S22 equ 9
S23 equ 14
S24 equ 20
S31 equ 4
S32 equ 11
S33 equ 16
S34 equ 23
S41 equ 6
S42 equ 10
S43 equ 15
S44 equ 21
s0 equ eax  ; Элемент s[0]
s1 equ ebx  ; Элемент s[1]
s2 equ esi  ; Элемент s[2]
s3 equ ebp  ; Элемент s[3]
x0 equ dword ptr [edi]    ; Элемент x[0]
x1 equ dword ptr [edi+4]  ; Элемент x[1]
x2 equ dword ptr [edi+8]  ; Элемент x[2]
x3 equ dword ptr [edi+0Ch]; Элемент x[3]
x4 equ dword ptr [edi+10h]; Элемент x[4]
x5 equ dword ptr [edi+14h]; Элемент x[5]
x6 equ dword ptr [edi+18h]; Элемент x[6]
x7 equ dword ptr [edi+1Ch]; Элемент x[7]
x8 equ dword ptr [edi+20h]; Элемент x[8]
x9 equ dword ptr [edi+24h]; Элемент x[9]
x10 equ dword ptr [edi+28h] ; Элемент x[10]
x11 equ dword ptr [edi+2Ch] ; Элемент x[11]
x12 equ dword ptr [edi+30h] ; Элемент x[12]
x13 equ dword ptr [edi+34h] ; Элемент x[13]
```

```
x14 equ dword ptr [edi+38h] ; Элемент x[14]
x15 equ dword ptr [edi+3Ch] ; Элемент x[15]
P309AD671 proc
    pushad
    mov  edi,edx
    mov  s0,[ecx]       ; s[0]
    mov  s1,[ecx+4]     ; s[1]
    mov  s2,[ecx+8]     ; s[2]
    mov  s3,[ecx+0Ch]   ; s[3]
    push ecx
;static void MDTransformF (UINT4 s[4, UINT4 x16])
    PFF s0,s1,s2,s3,x0,S11,0d76aa478h ; //  1
    PFF s3,s0,s1,s2,x1,S12,0e8c7b756h ; //  2
    PFF s2,s3,s0,s1,x2,S13,0242070dbh ; //  3
    PFF s1,s2,s3,s0,x3,S14,0c1bdceeeh ; //  4
    PFF s0,s1,s2,s3,x4,S11,0f57c0fafh ; //  5
    PFF s3,s0,s1,s2,x5,S12,04787c62ah ; //  6
    PFF s2,s3,s0,s1,x6,S13,0a8304613h ; //  7
    PFF s1,s2,s3,s0,x7,S14,0fd469501h ; //  8
    PFF s0,s1,s2,s3,x8,S11,0698098d8h ; //  9
    PFF s3,s0,s1,s2,x9,S12,08b44f7afh ; // 10
    PFF s2,s3,s0,s1,x10,S13,0ffff5bb1h ; // 11
    PFF s1,s2,s3,s0,x11,S14,0895cd7beh ; // 12
    PFF s0,s1,s2,s3,x12,S11,06b901122h ; // 13
    PFF s3,s0,s1,s2,x13,S12,0fd987193h ; // 14
    PFF s2,s3,s0,s1,x14,S13,0a679438eh ; // 15
    PFF s1,s2,s3,s0,x15,S14,049b40821h ; // 16
; static void MDTransformG (UINT4 s[4, UINT4 x16])
    PGG s0,s1,s2,s3,x1,S21, 0f61e2562h ; // 17
    PGG s3,s0,s1,s2,x6,S22, 0c040b340h ; // 18
    PGG s2,s3,s0,s1,x11,S23,0265e5a51h ; // 19
    PGG s1,s2,s3,s0,x0,S24, 0e9b6c7aah ; // 20
    PGG s0,s1,s2,s3,x5,S21, 0d62f105dh ; // 21
    PGG s3,s0,s1,s2,x10,S22,002441453h ; // 22
    PGG s2,s3,s0,s1,x15,S23,0d8a1e681h ; // 23
    PGG s1,s2,s3,s0,x4,S24, 0e7d3fbc8h ; // 24
    PGG s0,s1,s2,s3,x9,S21, 021e1cde6h ; // 25
    PGG s3,s0,s1,s2,x14,S22,0c33707d6h ; // 26
    PGG s2,s3,s0,s1,x3,S23, 0f4d50d87h ; // 27
    PGG s1,s2,s3,s0,x8,S24, 0455a14edh ; // 28
    PGG s0,s1,s2,s3,x13,S21,0a9e3e905h ; // 29
    PGG s3,s0,s1,s2,x2,S22, 0fcefa3f8h ; // 30
    PGG s2,s3,s0,s1,x7,S23, 0676f02d9h ; // 31
    PGG s1,s2,s3,s0,x12,S24,08d2a4c8ah ; // 32
; static void MDTransformH (UINT4 s[4, UINT4 x16])
    PHH s0,s1,s2,s3,x5,S31, 0fffa3942h ; // 33
    PHH s3,s0,s1,s2,x8,S32, 08771f681h ; // 34
    PHH s2,s3,s0,s1,x11,S33,06d9d6122h ; // 35
    PHH s1,s2,s3,s0,x14,S34,0fde5380ch ; // 36
    PHH s0,s1,s2,s3,x1,S31, 0a4beea44h ; // 37
    PHH s3,s0,s1,s2,x4,S32, 04bdecfa9h ; // 38
    PHH s2,s3,s0,s1,x7,S33, 0f6bb4b60h ; // 39
    PHH s1,s2,s3,s0,x10,S34,0bebfbc70h ; // 40
    PHH s0,s1,s2,s3,x13,S31,0289b7ec6h ; // 41
    PHH s3,s0,s1,s2,x0,S32, 0eaa127fah ; // 42
    PHH s2,s3,s0,s1,x3,S33, 0d4ef3085h ; // 43
    PHH s1,s2,s3,s0,x6,S34, 004881d05h ; // 44
    PHH s0,s1,s2,s3,x9,S31, 0d9d4d039h ; // 45
    PHH s3,s0,s1,s2,x12,S32,0e6db99e5h ; // 46
    PHH s2,s3,s0,s1,x15,S33,01fa27cf8h ; // 47
    PHH s1,s2,s3,s0,x2,S34, 0c4ac5665h ; // 48
; static void MDTransformI (UINT4 s[4, UINT4 x16])
    PII s0,s1,s2,s3,x0,S41, 0f4292244h ; // 49
    PII s3,s0,s1,s2,x7,S42, 0432aff97h ; // 50
    PII s2,s3,s0,s1,x14,S43,0ab9423a7h ; // 51
    PII s1,s2,s3,s0,x5,S44, 0fc93a039h ; // 52
    PII s0,s1,s2,s3,x12,S41,0655b59c3h ; // 53
    PII s3,s0,s1,s2,x3,S42, 08f0ccc92h ; // 54
    PII s2,s3,s0,s1,x10,S43,0ffeff47dh ; // 55
    PII s1,s2,s3,s0,x1,S44, 085845dd1h ; // 56
    PII s0,s1,s2,s3,x8,S41, 06fa87e4fh ; // 57
```

```
    PII s3,s0,s1,s2,x15,S42,0fe2ce6e0h ; // 58
    PII s2,s3,s0,s1,x6,S43, 0a3014314h ; // 59
    PII s1,s2,s3,s0,x13,S44,04e0811a1h ; // 60
    PII s0,s1,s2,s3,x4,S41, 0f7537e82h ; // 61
    PII s3,s0,s1,s2,x11,S42,0bd3af235h ; // 62
    PII s2,s3,s0,s1,x2,S43, 02ad7d2bbh ; // 63
    PII s1,s2,s3,s0,x9,S44, 0eb86d391h ; // 64
; Добавить результат к первичным значениям !!!
    pop ecx
    add [ecx],s0       ; s[0]
    add [ecx+4],s1     ; s[1]
    add [ecx+8],s2     ; s[2]
    add [ecx+0Ch],s3   ; s[3]
    popad
    ret
P309AD671 endp
```

Вот, собственно, и все. Совсем немного, правда ? :)

Оценка написанного кода подборщика

Итак, выше приведен код, подбирающий некоторые пять байт, которые, если их найти, необходимо подставить при открытии защищенного файла Excel в процедуре по адресу :308B74FD перед формированием хэша MD5, второй вызов после ввода любого пароля.

Оценим его эффективность. Скорость подбора составлет ~2x10^4 вариантов/сек на P120, общее число вариантов - 10^12. Т.е. при условии, что только последний вариант окажется удачным, необходимо

```
10^12 / 2x10^4 ~= 5x10^7 секунд,
```

или

```
5*10^7 / 3600 ~= 1,5x10^4 часов,
```

или

```
1.5*10^4 / 24 ~= 625 дней ~= 21 месяц.
```

В среднем, конечно, будет примерно 11 месяцев. Что ж, реальный срок, хотя ОЧЕНЬ большой.

Что может улучшить время подбора ?

Во-первых, техника. Если взять компьютеры 1.6 ГГц(уже есть такие), то ориентировочно можно принять повышение производительности в 10 раз, это дает полное время перебора

```
~ 62 дней (два месяца).
```

Во-втрорых, более умный перебор. Что мы на самом деле подбираем ? Фактически, MD5-хэш. А хэш - это не просто какая-то последовательность байт, это практически ПСЕВДОСЛУЧАЙНАЯ последовательность. Т.е. не имеет практического смысла перебирать варианты вида 00 00 00 00 00 или 22 22 22 22 22 как явно неслучайные. Или (что надежнее) перебирать их в последнюю очередь.

Как же можно оценить случайность очередного варианта ? Например, посчитав число единичных и нулевых бит. В псевдослучайной последовательности они равновероятны, т.е. всего 5x8 = 40 бит, примерно половина должна быть нулевой, половина - единичными. То же самое можно сказать и про двойки бит, т.е число сочетаний 00,01,10 и 11 должно быть примерно равно числу всех двоек в пяти байтах = 40/2 = 20 деленному на четыре - 5. А также число четверок бит 0000,0001..1111

должно быть в среднем равно (40/4)/16 ~= 1-2.

Насколько велики могут быть отклонения числа битов, двоек, четверок от среднего ожидаемого ? Если принять, что распределение числа конкретного варианта нулевых(единичных) бит описывается распределением Гаусса со средним значением 20, то около 67% случаев будет составлять диапазон

```
20 +/- sqrt(20) ~= 20 +/- 5
```

Соответственно, число каждого варианта двоек бит будет лежать в диапазоне

```
5 +/- sqrt(5) ~= 5 +/- 3
```

А число каждого варианта четверок бит будет лежать в диапазоне

```
1-2 +/- sqrt(1-2) ~= 1-2 +/- 1-2
```

Вот число нулевых/единичных бит, пар и четверок реальных пяти байт подобранного хэша:

```
Число 0/1 - 18/22
Число пар - 4/6/4/6
Число четверок - 0/1/1/0/1/0/0/1/1/1/0/0/0/2/1/1
```

Мною был добавлен фильтр на перебираемые значения 5 байт, заключался он в проверке числа пар и четверок. Эффективность составила 2-4 раза в сравнении с "тупым" перебором.

В-общем, тут еще есть над чем подумать. Проще, разумеется, включить парк компьютеров и запустить параллельный подбор на непересекающихся диапазонах искомых пяти байт. Удачи!

Защита файлов программами pkzip/pkunzip

Автор: Chingachguk / HI-TECH

Дата: 2002-09-10

Как создать защищенный файл, используя pkzip

Популярная утилита сжатия данных pkzip(pkunzip-распаковщик) позволяет не только уменьшать размер файла(ов), но и зашифровывать данные. Например, для установки защиты на файл my_file.txt вы можете запустить pkzip со следующими ключами:

pkzip -s protect.zip my_file.txt

После этого pkzip запросит у Вас пароль(дважды) и после всех подтверждений файл будет упакован с установленной защтой. При последующих попытках открыть файл protect.zip pkunzip будет предлагать ввести пароль, и если тот окажется неверен, то Вы (или кто-то другой) вместо содержимого файла скорее всего получите сообщение "PKUNZIP: (W14) Warning! Incorrect password for file: MY_FILE.TXT".

Примечание: речь идет о dos-версии 2.06 ("PKUNZIP (R) FAST! Extract Utility Version 2.06 01-24-94"). Однако методы, изложенные в данном документе, применимы с минимальными ограничениями и к файлам, созданным WinZip.

Методика исследования защиты

Для анализа защиты pkzip-а будем использовать как программные средства (отладчик), так и некий документ APPNOTE.TXT - в нем содержится очень подробное описание (видимо, от авторов) алгоритмов pkzip и pkunzip (на английском языке). Я нашел этот документ по адресу http://www.halyava.ru/document/txt_form.htm. Сразу скажу, что многое о защите pkzip можно почерпнуть из этого файла, но некоторые тонкости и непосредственную реализацию алгоритмов - только путем анализа выполнимого кода. Поэтому сначала приводится исследование pkunzip без помощи APPNOTE.TXT, и только после того, как будут обнаружены все базовые процедуры защиты, будет приведен конспект из этого документа. Наименования процедур, данных будут сопоставлены с наименованиями, данными авторами APPNOTE.TXT.

Далее на основе полного знания о алгоритме защиты будет проведена попытка оценить его стойкость. Исходя из этой оценки появится возможность "взлома" в случае успешной словарной атаки и будет приведен ее алгоритм.

Поскольку pkzip является dos-программой, то будем "играть по правилам" и выберем в качестве инструмента исследования dos-отладчик td.exe.

Разумно также будет привести поверхностный анализ защиты вообще без каких-либо инструментов (отладчиков, дизассемблеров...).

Предварительный анализ (без отладчика)

Попытаемся получить информацию на основе анализа файлов, упакованных и незащищенных (защищенных) паролем, используя обычный редактор файлов.

Создадим файл из одного символа. Упакуем его как с паролем, так и без него. Сразу обратим внимание на тот факт, что файлы различаются по размеру ровно на 12 байт (у меня размеры файлов составили 115 и 127 байт). Сравнив побайтно содержимое обоих файлов, обнаруживаем, что они отличаются наличием в защищенном файле блока из 12 байт, находящихся сразу после имени упакованного файла (имя файла находится по смещению 001Eh), тогда как в незащищенном файле следом за именем следует символ из упакованного файла. Естественно, что в защищенном файле нет этого символа, по крайней мере после 12-ти байтного блока (предполагаем, что файлы

отличаются только вставкой 12-ти байт).

Проверим эту версию. Допустим, эти 12 байт (назовем их Hash) относятся к за/расшифровке файла, тогда как следом за ними начинается упакованная информация. Выбор названия достаточно произволен и связан со следующими соображениями: полагаем вначале, что алгоритм проверки пароля pkunzip заключается в получении от введенного пользователем пароля некоторой хэш-суммы и сравнения ее с хэш-суммой от оригинального пароля, сохраненной в файле.

Действительно, если изменить любой байт из Hash и попробовать получить упакованный файл используя верный пароль, то получим сообщение о неверном пароле. А если изменить первый байт после Hash, то файл "нормально" распакуется, только первый символ в нем окажется отличным от исходного.

Итак, мы получили некий ориентир при анализе защиты - это 12 байт Hash. Если при разборе pkzip/pkunzip нам встретится код, работающий с ними, то можно с большой вероятностью отнести его к блоку за/расшифровки.

Исследования с помощью отладчика

Будем исследовать как программа pkunzip расшифровывает информацию и на основании чего выдает сообщение о неверном пароле при помощи отладчика td.exe. Однако нам придется выполнить некоторые подготовительные действия, прежде чем мы сможем работать в отладчике обычным способом: ставить точки останова и т.д., поскольку pkunzip обработан утилитой сжатия PKLITE (в этом можно убедиться, если посмотреть на первые байты pkunzip в редакторе) а также содержит некий цикл расшифровки кода распаковщика PKLITE.

Если загрузить распаковщик в отладчике ("td pkunzip -s protect.zip"), и протрассировать несколько первых его команд, то довольно скоро можно увидеть некий цикл расшифровки блока кода:

```
; cx=0796h (MagValue?), dx=0138h
; si=di=03E4h, ds=es=cs
0164:
  nop, nop
  dec  dx ; Уменьшить счетчик циклов
  jz   0178     ; Расшифровка закончена ?
  mov  ax,[di] ; Получить слово кода
  sub  di,2     ; Указать на предидущее слово кода
  xchg cx,ax
  xor  ax,cx    ; Расшифровать слово кода
  mov  [si],ax ; И записать обратно
  sub  si,2
  jmp  0164
0178:
```

Итак, этот цикл расшифровывает код, находящийся в интервале начальный сегмент:[0178h..03E4h]. Причем пока его работа не закончена полностью, мы не можем ставить точки останова (F4) в этом интервале, поскольку цикл расшифровки уничтожит эти точки, и мы будем вынуждены трассировать этот довольно значительный цикл пошагово F7(F8). Также мы не сможем ставить точки останова прямо в оригинальный код pkunzip, поскольку он станет доступен только после его распаковки подпрограммой PKLITE. Рассмотрим более детально, как это происходит: дождемся сначала расшифровки кода от адреса 0178:

```
; Самокопирующая процедура PKLITE
0178:
  pop dx, pop es, push es
  inc  byte ptr [018B] ; Заменить retf 018B -> retf
  mov  cx,012Dh
  xor  di,di, push di, mov si,018Ch, cld
  rep  movsw ; Копировать тело распаковщика PKLITE в последний сегмент
018B:
  retf 8CFD ; Перейти на код распаковки
```

Таким образом, мы видим, как стартовый цикл расшифровки декриптует код PKLITE, который в свою очередь перебрасывает свое тело в сегмент после упакованного оригинального кода pkunzip и переходит на него. Поскольку при исследовании pkunzip предполагается многократная его загрузка в отладчике, то проще не выполнять каждый раз процедуру декриптования, а сразу (небольшой программкой, повторяющей действия декриптора) расшифровать код, не забыв удалить код декриптора (он исказит нам расшифрованный код). Алгоритм примерно следующий: найти в pkunzip начало зашифрованного кода (байты E6h,59h,BCh,.. по смещению в файле 00D8), декриптовать их аналогично оригинальному декриптору, дезактивировать код декриптора (например, заменить команду декриптора mov [si],ax на два nop'a: 89h,04h->90h,90h).

Если после такого изменения pkunzip'a загрузить его в отладчик, то можно сразу ставить breakpoint на любую команду PKLITE с адреса 0178 кроме модифицирцуемой команды retf 8CFD - например, на rep movsw.

Трассируя далее уже код PKLITE переходим вслед за ним в последний сегмент и видим, как он начинает распаковывать код pkunzip:

```
; Начало кода PKLITE
0000:
  std
  mov  bx,ds ; <- seg PSP
  push bx
  ...
```

Трассировать код PKLITE также довольно утомительное занятие. Хотелось бы сразу установить прерывание в то место, когда его код полностью отработал и осталось только перейти на распакованный код pkunzip. Как правило, межсегментный переход реализуется с помощью команд retf, jmp far, причем он должен располагаться сразу после всех циклов распаковки PKLITE. Действительно, пролистав вниз код, обнаруживаем такой переход:

```
; Завершение кода PKLITE
020E:
  xor  bx,bx
  mov  cx,bx, mov dx,bx, mov bp,bx, mov si,bx, mov di,bx
021A:
  retf ; Переход на оригинальный код pkunzip
  ...
```

Теперь достаточно устанавливать breakpoint на этот retf, чтобы разом пропускать всю работу PKLITE.

Получив таким образом возможность "обычным образом" исследовать в отладчике pkunzip, приступим к определению механизма проверки пароля и расшифровки данных. Задача сводится к поиску в значительном потоке выполняющихся команд тех, которые непосредственно относятся к проверке пароля и (желательно для полноты картины) расшифровке данных. Выбор варианта решения зависит в некоторой степени от вкусов исследователя, я же остановлюсь на последовательной локализации интересующего кода ориентируясь на характерные для него проявления.

Так, например, нас интересует код проверки пароля. Делаем предположение, что он находится между кодом, получающим пароль от пользователя и выдачей сообщения о неверном пароле (если специально ввести таковой). Так, к примеру, после перехода на распакованный PKLITE'ом код pkunzip мы видим следующий порядок вызовов процедур:

```
; В основном сегменте(cs) pkunzip после его распаковки
cs:4BE2:
  mov  ah,30h
  int  21h
  ...
4C83:
  call 5718
  call 558E
```

```
  xor  bp,bp
  ...
  call 20B6
  ...
  call 4DB0
  ret
```

Как видно, перед нами серия call-ов, видимо отвечающих стартовому коду программы. Пройдем все вызовы командой F8 (Step over - не заходя в код подпрограммы). Оказывается, подпрограммы 5718, 558E и 4DB0 "ничего" не делают, в то время как 20B6 запрашивает у нас пароль, выдает сообщение о неверном пароле и проч. Поэтому мы можем на данном этапе локализовать область исследований подпрограммой 20B6 и в следующей итерации исследовать только ее код. Далее аналогично исследуя набор вызовов этой подпрограммы:

```
; Внутри процедуры 20B6 pkunzip
20B6:
  ...
  call 1EEC
  ...
  call 70D4:0004 ; Дальний вызов из вспомогательного модуля
  ...
  call 071E
  ...
```

Обнаруживаем, что процедура 071E запрашивает пароль - в первом приближении интересующий нас код находится внутри 071E или далее. Установить способ получения пароля можно следующим способом: простая резидентная программа, перехватившая некий набор функций прерывания int 21h или int 16h должна "всплыть", если pkunzip будет использовать одну или несколько функций из этого набора. В данном случае pkunzip'ом используется функция 08h int 21h. Сжимая далее круг поиска, обнаруживаем следующий интересный кусок кода:

```
; Проверка правильности пароля в pkunzip
1880:
  call 7566
  les  bx,[bp+4], test byte ptr es:[bx+0A],8
  jz   1893
  mov  al,es:[bx+0F]
  jmp  189B
1893:
  mov  ax,es, mov si,bx, mov al,es:[si+15h]
189B:
  sub  ah,ah, mov [bp-6],ax, mov al,[BCA9], cmp ax,[bp-6]
  jz   18B2
  lea  bx,[04A9] ; bx->"Incorrect password for file..."
  ...
  call 0136
```

Процедура 0136 выдает сообщение в случае неверного пароля, в то время как в случае верного пароля выполняется переход на метку 18B2. Анализируя условный переход, делаем вывод, что после подпрограммы 7566 инициализируется дальний указатель на некоторую структуру (es:bx) и выполняется сравнение между элементом этой структуры (либо +0F, либо +15h) и некоторой глобальной переменной [BCA9]. Что же представляют собой используемые в данном месте программы элементы структуры ? Простейшим действием является проверка на наличие в zip-файле элементов этой структуры. Действительно, байты структуры со смещением 0F и 15h совпадают с байтами файла по смещению 11h и 17h соответственно ! А в элементе со смещением 0Ah, похоже, находится какой-то битовый флаг, управляющий выбором тестового байта.

Итак, небольшая часть кода проверки пароля известна. Однако стоило ли исследовать машинный код, если, как уже было сказано в начале, многое о защите можно было почерпнуть из документа APPNOTE.TXT ? Приведем цитату, относящуюся к описанию защитного механизма и сравним ее непосредственно с обнаруженным алгоритмом:

```
; Описание структуры заголовка файлов *.zip и алгоритма проверки пароля
```

```
; из APPNOTE.TXT
...
A. Local file header:

   local file header signature     4 bytes  (0x04034b50)
   version needed to extract       2 bytes
   general purpose bit flag        2 bytes
   compression method              2 bytes
   last mod file time              2 bytes
   last mod file date              2 bytes
   crc-32                          4 bytes
   compressed size                 4 bytes
   uncompressed size               4 bytes
   filename length                 2 bytes
   extra field length              2 bytes

...After the header is decrypted,  the last 1 or 2 bytes in Buffer
should be the high-order word/byte of the CRC for the file being
decrypted, stored in Intel low-byte/high-byte order.  Versions of
PKZIP prior to 2.0 used a 2 byte CRC check; a 1 byte CRC check is
used on versions after 2.0.  This can be used to test if the password
supplied is correct or not...
```

Итак, из документа следует, что для проверки пароля необходимо сравнить последний байт(слово) некоего расшифрованного буфера с старшим байтом(словом) crc, содержащимся в заголовке файла. Однако мы видим, что в зависимости от некоторго бита (по всей видимости, находящимся в элементе "general purpose bit flag" заголовка) сравнение может быть проведено совсем не с crc, а с старшим байтом времени ("last mod file time")...

Остается найти код, формирующий "второй параметр" сравнения - байт в ячейке [BCA9]. В отладчике td нет возможности установить "breakpoint at memory access" (точку прерывания при обращении к памяти), но можно применить все тот же метод "последовательной локализации", только индикатором будет служить изменение (визуальное) ячейки [BCA9]. Для этого необходимо указать отладчику в окне "data" отображать указанную ячейку и, сделав это в момент начала выполнения кода pkunzip'a, следить за ней во время трассировки. Разумным предположением в смысле экономии усилий будет начать поиск с "близлежащего" от места проверки правильности пароля - а именно, подпрограммы 7566. Именно в ней мы и находим сначала следующий код, использующий пароль:

```
; Подпрограмма использования пароля для изменения им 12 байтного буфера (buff12)
7554:
  mov  al,[si] ; ds:si-> введенный пароль (си-строка)
  call add_seg:07CA
  inc  si
  cmp  byte ptr [si],0
  jnz  7754
; А также подпрограмма модификации байтом(в al) 12 байтного буфера (buff12)
add_seg:082A:
  cli, xor ecx,ecx
  movzx edx,al ; al= очередной символ пароля
  push di, push ds, pop es, cld, mov di,0BDF4h
  mov   eax,[di]
  xor   dl,al
  shr   eax,8
  xor   eax,cs:[edx*4+ecx]
  stosd
  mov   dl,al
  mov   eax,[di]
  add   eax,edx
  imul  eax,eax,08088405h
  inc   eax
  stosd
  shr   eax,18h
  mov   edx,[di]
  xor   al,dl
  shr   edx,8
  xor   edx,cs:[eax*4+ecx]
```

```
  mov   [di],edx
  sti, pop di, retf
```

Грубо говоря, каждый символ пароля последовательно передается на вход процедуре add_seg:082A, которая использует его для последовательного изменения трех двойных слов в буфере [0BDF4]. К моменту начала добавления пароля буфер инициализируется значениями 12345678h, 23456789h и 34567890h (оригинальный код не приводится). Мы назовем эти три двойные слова "buff12". При этом используется добавочная таблица размером в 100h двойных слов (см. команды типа xor eax,cs:[edx*4+ecx], размер следует из того, что индекс (edx) всегда меньше 256). Небольшой промах компилятора (xor ecx,ecx, use ecx как базу) не помешает нам понять расположение этой таблицы в памяти - а именно, в сегменте кода дополнительного модуля. Видимо, к основной программе был подключен модуль с функциями криптования, выполненными в far call стиле и активно использующими инструкции 386 процессора. Действуя так же, как и при поиске вышеприведенного кода, пытаемся обнаружить процедуру инициализации добавочной таблицы (назовем ее XOR-таблицей) - хотя бы ориентируясь на факт ее заполнения. Если проявить чрезмерную осторожность и начать следить за ее изменением с самого начала работы программы, то можно обнаружить любопытный факт: таблица заполняется дважды и оба раза ... неодинаково ! Первый раз (оригинальный код пропущен) это делается на базе инструкций 8086 процессора, далее следует проверка текущего типа CPU и таблица переинициализируется с использованием инструкций 386 процессора:

```
; Подпрограмма инициализации XOR-таблицы.
Build_XorTable proc near
  std
  mov  di,offset XorTable+400h-4h
  mov  dx,0FFh
  mov  ebx,0EDB88320h
@@FillTable:
  mov   cx,8
  movzx eax,dx
@@Loop8:
  shr   eax,1
  sbb   esi,esi
  and   esi,ebx
  xor   eax,esi
  dec   cx
  jnz   @@Loop8
  stosd
  dec   dx
  jns   @@FillTable
  retn
Build_XorTable endp
```

Можно предположить гипотетическую ситуацию: файл был заархивирован с паролем на машине 8086, а расшифровывается на 386-ой. Тогда он ... не может быть расшифрован !

Чтож, любопытно, однако перейдем к финальной части проверки пароля. Нам осталось выяснить, как формируется байт в [BCA9] и как при этом используется вышеприведенный набор из трех двойных слов, измененных паролем. Прежде всего замечаем, что немного ранее кода изменения двойных слов buff12 (метка 7554) в буфер [BCA9-(0Ch-1)] заносятся те самые двенадцать байт из zip-файла (Hash), которые после изменения двойных слов паролем также модифицируются подпрограммой 2938:

```
; Подпрограмма использования файлового Hash после использования пароля.
2938: push bp, mov bp,sp...
2948:
  mov  ax,[BDFC] ; Фактически, младший байт третьего двойного слова
  or   al,2
  mov  bx,ax
  xor  bl,1
  mul  bx
  mov  al,ah
  xor  ah,ah
```

```
mov  es,bp
xor  al,es:[di] ; es:di->[BCA9-(0Ch-1)]=Hash
stosb
call add_seg:07CA ; Опять изменить buff12 байтом из al
dec  si
jnz  2948
```

Таким образом Hash преобразуется (дешифруется) и последний байт его сравнивается либо с старшим байтом crc, либо со старшим байтом временем файла.

Последнее, что осталось рассмотреть - это способ расшифровки потока данных. При его поиске можно, например, опираться на следующие предположения: вероятно, код расшифровки начинает работать после проверки правильности пароля (условный переход на метку 18B2), вероятно, что расшифровкой занимаются подпрограммы дополнительного модуля, аналогичные или совпадающие с приведенными выше - как только в программе встретится дальний вызов подпрограммы из этого модуля, это должно вызвать подозрения. Собственно говоря, процедура расшифровки Hash сама по себе уже является кандидатом на "расшифровщик" потока данных. Действительно, если "на всякий случай" установить breakpoint в ее начало , то он сработает ! Дампируя содержимое памяти по адресу в es:di (адрес дешифруемых байт), идентифицируем их с байтами файла, лежащими следом за Hash файла: процедура дешифровки данных - это тот код, который ранее расшифровал Hash.

Итоговое описание алгоритма проверки пароля с выдержками из авторского описания (APPNOTE.TXT)

Итак, приводим описание алгоритма проверки пароля и дешифровки данных.

Сначала программа запрашивает пароль. Это осуществляется при помощи функции 08h прерывания 21h. Для нас важно то, что эта функция не позволяет вводить символы с следующими кодами: 0,3,8,13,16,19,27.

Инициализируются три двойных слова следующими значениями: 12345678h, 23456789h и 34567890h:

```
...
Step 1 - Initializing the encryption keys
----------------------------------------
Key(0) <- 305419896
Key(1) <- 591751049
Key(2) <- 878082192
```

При рассмотрении защиты под отладчиком Keys мы назвали buff12.

Используя пароль, изменяем эти Keys (оригинальный код приведен выше и носит название "Подпрограмма использования пароля для изменения им 12 байтного буфера (buff12)" и "Подпрограмма модификации байтом 12 байтного буфера (buff12)"):

```
loop for i <- 0 to length(password)-1
    update_keys(password(i))
end loop

Where update_keys() is defined as:

update_keys(char):
  Key(0) <- crc32(key(0),char)
  Key(1) <- Key(1) + (Key(0) & 000000ffH)
  Key(1) <- Key(1) * 134775813 + 1
  Key(2) <- crc32(key(2),key(1) >> 24)
end update_keys

Where crc32(old_crc,char) is a routine that given a CRC value and a
character, returns an updated CRC value after applying the CRC-32
algorithm described elsewhere in this document.
```

Способ получения crc32(XOR-таблицы) приведен выше и носит название "Подпрограмма инициализации XOR-таблицы".

Дешифруется буфер из 12 байт, расположенных в zip-файле сразу за именем упакованного файла (ранее мы дали ему название Hash):

```
Step 2 - Decrypting the encryption header
-----------------------------------------
The purpose of this step is to further initialize the encryption
keys, based on random data, to render a plaintext attack on the
data ineffective.

Read the 12-byte encryption header into Buffer, in locations
Buffer(0) thru Buffer(11).

loop for i <- 0 to 11
    C <- buffer(i) ^ decrypt_byte()
    update_keys(C)
    buffer(i) <- C
end loop

Where decrypt_byte() is defined as:

unsigned char decrypt_byte()
    local unsigned short temp
    temp <- Key(2) | 2
    decrypt_byte <- (temp * (temp ^ 1)) >> 8
end decrypt_byte
```

Далее осуществляется проверка правильности пароля. Для этого осуществляется сравнение последнего, 12-го расшифрованного байта и старшего байта crc32 (лежит по смещению 14 в файле) упакованного файла, если третий бит байта ("general purpose bit flag", лежит по смещению 6 в файле) сброшен, иначе берется страший байт времени (смещение 10).

Дешифрация данных производится аналогично расшифровке 12-ти байт. Поток защищенных данных (сначала упакованных, затем шифрованых) подается на вход процедуре дешифровки аналогично потоку 12-ти байт безо всяких дополнительных действий.

Словарная атака

Обсудим возможность нахождения неизвестного пароля к произвольному защищеному zip-файлу.

Итак, перед нами следующая последовательность действий pkunzip'а при расшифровке файла:

- Получение пароля от пользователя;

- Инициализация Key, изменение Key паролем;

- Использование Key для расшифровки "12-byte encryption header";

- Сравнение последнего расшифрованного байта и crc(времени) файла. Если они совпадают, пароль признается pkunzip'ом верным.

- Использование Key для расшифровки данных в файле.

Вначале исследуем последнию стадию - расшифровку данных. Ведь конечной целью является получение не пароля, а самих данных. Входными данными для алгоритма дешифрации является поток данных из файла и Key в некотором произвольном состоянии.

Здесь возможны два варианта, зависящие от нашего знания о шифруемой информации. Если мы не обладаем никакими данными о информации, то тут мы зашли в тупик. Допустим, однако, мы имеем некоторое представление о том, что зашифровано в файле. Откуда мы его можем получить ? Сначала необходимо понять, что она представляет собой. Вспомним, что основная функция pkunzip - это упаковка файлов и информация может быть сначала упакована и затем закриптована, либо наоборот. Оказывается, pkunzip криптует упакованные данные ! В случае значительного по

объему входного файла первыми байтами упакованных данных будет что-то вроде начала таблицы символов для распаковки. В общем случае pkunzip может использовать до восьми методов сжатия ! Интерес представляет метод stored - сохранение информации без сжатия. Этот метод может быть применен в случае небольшого объема информации либо принудительно по желанию пользователя. Метод упаковки можно получить из поля "compression method" заголовка файла (0 - The file is stored). Не будем подробно рассматривать алгоритм атаки в этом маловероятном случае и перейдем к рассмотрению возможности получения пароля.

Прежде всего замечаем, что все множество различных шифровок любой информации совпадает количественно с множеством состояний 12 байт Key, которые можно получить, используя все множество доступных для ввода паролей. Максимальная длина пароля составляет примерно 64 символа, что, несмотря на некоторое количество запрещенных для ввода символов, дает огромное в сравнении с числом всех возможных вообще состояний Key (256^12). Интуитивно ясно, что если алгоритм изменения Key паролем не содержит явных ошибок, то множество возможных состояний Key по порядку величины совпадает с числом комбинаций 12 байт (256^12). Даже если число возможных состояний Key, полученных от всего множества возможных паролей составит 256^8 ~ 10^19, то, даже имея однозначное правило проверки пароля (или соответствующего ему состояния Key), невозможно за приемлимое время перебором всех вариантов обнаружить верный пароль. Если же число возможных состояний состовляет, например, 256^5 ~ 10^12 вариантов, то при скорости проверки паролей в 10^5 паролей в секунду имеет смысл выполнять такой подбор. Необходимо только знать критерий проверки пароля, оставляющий конечное число кандидатов в пароли (чтобы можно было проверить их вручную за конечное время) - например, 10000 или немногим более.

Вряд ли, однако, число состояний Key состовляет менее 256^6, что делает бессмысленным полный перебор всех возможных паролей (Key) даже при наличии критерия проверки пароля. Косвенным доказательством служит следующий опыт. Будем подавать на вход процедуре update_keys различные пароли и следить за выпадением первых четырех байт (двойное слово) в интервале от 0 до 255 (или любом другом). Например, в моем опыте подавались пароли из символов от "A" и до конца таблицы длиной от единицы и выше. В результате после подачи ~7*10^8 таких паролей выпало не менее ~40 Key со значением первого двойного слова в интервале 0..255, из чего можно сделать оценку (очень грубо), что существует не менее 40/256 ~1/8 состояний Key от максимально возможного (256^12).

Альтернативой алгоритму перебора является получение пароля по следам его использования. В нашем случае пароль используется единожды при изменении начального состояния Key и единственное место, если не считать расшифровку данных, где он проявляет свои свойства - это заданное значение последнего байта расшифрованного "encryption header".

Поток состояний Key, сформированных множеством возможных паролей, способен образовать практически псведосучайные последовательности в виде расшифровки encryption header, следовательно, примерно на каждые 256 входящих паролей один окажется верным с точки зрения алгоритма проверки. Следовательно, даже если бы число возможных состояний Key было конечным, порядка 256^5, мы бы смогли уменьшить число возможных Key не более чем в 256 раз. Вот если бы мы знали исходное состояние encryption header, то мы бы могли практически однозначно проверить любой пароль, либо попытаться получить пароль по результату шифрования исходного состояния. Авторами подчеркивается, что исходное состояние encryption header - это псевдосучайный набор байт.

Так ли это на самом деле ? Проверим это утверждение и найдем отладчиком код, формирующий случайные байты encryption header. Для этого нам придется трассировать уже pkzip (модуль упаковки), правда, по сходной схеме с pkunzip. Вот этот код:

```
; Процедура формирования случайных байт header
; es=0, ds:bx=header(в начале все байты равны 0)
```

```
; в si - псевдослучайное число
  and  si,07FEh
  mov  cx,56h
  mov  dx,07FEh-8
  jmp  short @@L6034
@@L6032:
  sub  si,dx
@@L6034:
  cmp  si,dx
  jnb  @@L6032
  lods word ptr es:[si]
  sbb  [bx],ax
  lods word ptr es:[si]
  adc  [bx+2],ax
  lods word ptr es:[si]
  sbb  [bx+4],ax
  lods word ptr es:[si]
  adc  [bx+6],ax
  lods word ptr es:[si]
  sbb  [bx+8],ax
  lods word ptr es:[si]
  adc  [bx+0Ah],ax
  loop @@L6034
```

В чем смысл этого кода ? header представляется в виде шести слов, изначально равных нулю, каждое из которых командами sbb или adc модифицируется 86 раз. Команды sbb(adc) вычитают(добавляют) с учетом флага переноса содержимое памяти в регионе 0000h...0800h. В этом регионе памяти находится таблица векторов прерываний (первые 400h байт), затем идут области данных BIOS и DOS). Значения слов региона зависят от установленных драйверов, версии OS. Но самое главное, что в этой области находится счетчик тиков системного таймера и буфер клавиатуры, которые используются примерно в 50% случаев. Правда, существует выделенный случай, когда в точности известна конфигурация машины, на которой была осуществлена упаковка.

Таким образом, в общем случае невозможно обращение пароля по следам его использования и невозможен полный перебор всех вариантов паролей (Key). Разумеется, необходимо проверить выделенные случаи: например, первый зашифрованный файл в архиве - распространенный read.me и т.д. Следовательно, следует вывод о том, что единственным возможным путем атаки является перебор ограниченного числа паролей с расчетом на то, что искомый пароль окажется среди рассматриваемых - атака по словарю.

Как организовать словарную атаку ?

Ключевым моментом является определение критерия проверки правильности пароля. Как было сказано выше, критерий, применяемый программой для выдачи сообщения о неверном пароле (сверка последнего байта расшифрованного encryption header с crc или временем файла) уменьшит число проверяемых паролей не более чем в 256 раз, что явно мало для более или менее солидной подборки паролей.

Но что же произойдет, если мы перебором возможных паролей найдем такой "верный" пароль ? Достаточно перебрать пароли числом >> 256, чтобы найти пароль, удовлетворяющий критерию программы. Попытаемся расшифровать им защищенный файл. Сообщения о неверном пароле не появилось, но зато pkunzip выдаст сообщение о неверном crc распакованного файла ! В ряде случаев может появиться сообщение о том, что разрушена таблица символов или еще что-то в этом роде. Оказывается, распаковав файл, pkunzip проверит его crc. Причем используется crc32. Вероятность совпадения для такого crc составляет 1/2^32, что практически однозначно при проверке паролей числом 256^4 определит нужный. Плюс стандартный критерий уменьшит число паролей в 256 раз, но и это еще не все ! Что будет, если pkunzip начнет распаковывать неверно расшифрованные данные ? Наверняка в их структуре будет что-то не так, и это тоже является критерием проверки пароля ! Так, например, при методе упаковки "deflating" первый бит упакованных данных - признак последнего блока, а если следующие два бита равны единице, то

это является признаком разрушенного блока.

Итак, алгоритм словарной атаки примерно ясен: получаем очередной пароль, изменяем им key, расшифровываем encryption header, сверяем последний байт. Если сравнение успешно, то распаковываем файл так же, как это делает pkunzip, при ошибках отбрасываем пароль. Считаем crc файла и сверяем его с эталонным. Этим методом мы из каждых 256^5 или более паролей оставим примерно один "верный".

Осталась реализация. Очевидно, что универсальный алгоритм подбора должен включить в себя все методы распаковки, которые может использовать pkunzip. Это довольно утомительно, в результате мы напишем еще один распаковщик. Единственным его достоинством будет относительно большая скорость распаковки, достигаемая за счет буферизации данных. А если шифрованный файл более 64K, то будет необходимо держать несколько буферов (речь идет о DOS) и на этом пути pkunzip постепенно догоняет нас. Так не лучше ли задачу распаковки и проверки crc возложить на него, а пароли подбирать только с точки зрения стандартнго критерия ?

Действительно, pkunzip поддерживает возможность передать пароль в командной строке ! Например, текущий проверяемый пароль - "TEST". Для его проверки необходимо запустить pkunzip следующим образом:

pkunzip -sTEST -o test.zip

"-o" означает записывать распаковываемый файл поверх старого, если таковой уже имеется, символы после "-s" трактуются как пароль. Правда, нельзя передавть принципиально возможные пароли с символами " " (пробел) или "-". Следовательно, необходимо выполнять запуск pkunzip'a из подборщика, например с помощью функции ax=4B00h прерывания int 21h. Ну а как же получить результат проверки от работающего pkunzip'a ? Понятно, что он выдаст предупреждение на дисплей, но ведь речь идет о массовых запусках. Оказывается, в случае ошибочной распаковки pkunzip устанавливает значение errorlevel (код возврата) в отличное от нуля значение. Получить код возврата после выполнения программы можно через функцию ah=4Dh прерывания int 21h.

Быстродействие ? На компьютере P-120 составляет около 3000 вариантов в секунду, что позволит просмотреть словарь объемом 3 миллиона слов всего за 1000 секунд или 20 минут.

Небольшие тонкости. В ряде случаев в архиве лежит более одного файла. В этом случае можно применять ключ "-x", позволяющий отбирать нужные. Также в случае работы под dos архивы, содержащие файлы с длинными именами, вызывают проблемы с оценкой кода возврата. В этом случае достаточно сократить имена вручную прямо в файле (в двух местах), не сдвигая остальные данные. Увеличить скорость перебора более чем в два раза можно перенаправлением вывода pkunzip'a с экрана на фиктивное устройство ("> nul").

Исходник здесь.

(C) Chingachguk /HI-TECH

Реконструирование Java-программ

Автор: FCA00000 / SET#25, пер. Aquila

Дата: 2002-09-21

Декомпиляция Java-программ В данной статье я собираюсь рассказать об основах дизассемблирования программ, написанных на Java. **Что такое Java?**

Термин "Java" включает в себя несколько различных концепций:

• Язык программирования
• Спецификации языка
• Среда выполнения и API

Вот небольшое пояснение к вышесказанному: после многочисленных экспериментов по созданию языка программирования для своих инженеров SUN решила изобрести современный (по ее утверждению) язык, основанный на принципах объектно-ориентированного программирования, очень модному в то время. В архитектуру ООП входят (среди прочих) такие концепции как наследование и полиморфизм.

Это означает, что объекты расширяют и/или реализуют другие объекты, используя функции вышестоящих класов, от которых они наследуются, что в результате ведет к повышению реюзабельности кода (панацея программирования).

В то время, как SUN выпускала спецификации языка, она решила, что компиляция программы, написанной на Java, будет происходить в байткод, который будет выполняться специальным интерпретатор. Этот интерпретатор называется виртуальной JAVA-машиной (JVM).

Java как API задает типы данных и вызовы объектов. Это позволяет расширить интерфейс классами и избавиться он необходимости использования статических библиотек, так как все объекты полностью динамические. В API входят самые различные библиотеки, в которые входит множество функций: математических, для работы со списками, массивами, работы с сетью, безопасностью, графикой...

Что представляет из себя JAVA?

Каждый объект в JAVA находится в отдельном файле. Один или более объектов можно сгруппировать в пакет (pakage'и). Файлы группируются в Java-архив (JAR) или в архив, сходный по формату с ZIP.

Каждый класс объявляется определенным образом. Например:

```
public class MyClase
{
}
```

У каждого класса может быть (а может и не быть) один или несколько конструкторов, которые отличаются друг от друга количеством или типом аргументов:

```
public class MiClase
{
        public MiClase()
        {
        }

        public MiClase(int i)
        {
        }
}
```

У каждого класса может быть (а может и не быть) один или больше методов.

Также классы бывают разных типов: публичными, частными, финальными и виртуальными... Методы могут быть публичными, частными, статическими, финальными, ...

Класс может расширять функциональность другого.

Базовые типы аргументов методов и переменных могут быть следующие: boolean, char, byte, short, int, long, float, double. Эти типы являются системными. Им соответствуют следующие классы: java.lang.Integer, java.lang.String, java.net.ContentHandler, java.rmi.NotBoundException, ...

Важным классом является java.lang.Class. Экземпляры класса Class представляют классы и интерфейсов Java-приложения. JVM предоставляет механизм установки каждого объекта, как экземпляра класса Class.

Таким образом, то, насколько быстро будет работать программа, зависит не только от того, как написана программа, как она скомпилирована или вычислительной мощности компьютера, но и от того, какие виды оптимизации умеет выполнять JVM: техники кэширования, дублирование объектов, таблиц, сборка мусора, индексация поиска методов, ...

Первые шаги

Предположим, что у нас есть программа, написанная на Java, которую необходимо скомпилировать. Для этого нам необходимо иметь компилятор Java. В частности, SUN бесплатно распространяет JDK, который включает в себя компилятор javac.

Вызываем

```
javac  MiClase.java
```

Он запускается следующим образом:

```
javac  MiClase.java
```

На выходе получаем файл 'MiClase.class', который содержит Java-байткод.

Теперь, чтобы эта программа была переведена в машинные коды и выполнена, необходим Java-интерпретатор (который также включен в JDK). Код, генерируемый интерпретатором, зависит от процессора, на котором выполняется программа.

Запустить программу можно следующим образом:

```
java MiClase
```

Введение в суть дела

Всю необходимую информацию относительно спецификации JavaTM Virtual Machine, можно получить на сайте java.sun.com.

Архив каждого класса начинается со структуры типа ClassFile

```
{
    u4 magic;
    u2 minor_version;
    u2 major_version;
    u2 constant_pool_count;
    cp_info constant_pool[constant_pool_count-1];
    u2 access_flags;
    u2 this_class;
    u2 super_class;
    u2 interfaces_count;
    u2 interfaces[interfaces_count];
    u2 fields_count;
    field_info fields[fields_count];
    u2 methods_count;
    method_info methods[methods_count];
    u2 attributes_count;
    attribute_info attributes[attributes_count];
```

```
}
```

Например, поле 'magic' занимает 4 байта и всегда равно 0xCAFEBABE. В общем случае данные этой структуры представляют массивы элементов размером в слово, при этом каждому массиву предшествует поле, указывающее размер этого массива. Например, methods_count содержит в себе количество методов (функций или подпрограмм), которые содержит класс, в то время как methods[methods_count] является массивом структур типа method_info. Эта информация генерируется компилятором и предназначается для JVM.

Поскольку эта информация показывает, сколько класс занимает места в памяти, JVM имеет механизм верификации классов.

Этот процесс выполняется следующим образом:

Фаза 1:
- поле magic верно

Фаза 2:
- у класса с типом final (финальный) не может быть наследников
- класс наследуется от другого, хотя бы java.lang.Class или java.lang.Object
- область констант непрерывна
- у всех поля и методы принадлежат верным классам, имена и типы полей и методов также верны.

Фаза 3:

- запук программы возможно только, если:

• присутствуют все необходимые операторы
• куча содержит данные верных типов
• переменные имеют определенное значение (включая void и null)
• вызовы методов используют правильные аргументы
• у всех опкодов верные аргументы

Фаза 4:

- некоторые проверки фазы 3 выполняются во время запуска класса.

Как и все языки, основывающиеся на байткодах, JVM имеет ряд регистров и структур для внутреннего использования. Например, программный счетчик (pc) содержит адрес выполняющейся JVM инструкции для каждого треда. Помните, что по определению при выполнении JAVA-программы создается несколько тредов, выполняющихся одновременно. Также существуют различные кучи данных (стек) по одной для каждого из выполняющихся тредов. Существует одна уникальная куча, использующаяся для хранения объектов.

Другой важной зоной является область методов, в которой хранятся байткоды классов. И еще одна зона с константами. И еще одна с "родными"(native)-методами (процедуры, написанные не на JAVA, а на другом языке, как правило C, и предназначающиеся для вызовов в Java и вызовов методов DLL; соответственно, эти процедуры хранятся не в байткодах).

Вопрос о переполнении буфера и самомодифицирующемся коде остается открытым, но не забудьте, что в JVM встроенны различные механизмы безопасности, например, проверка того, чтобы сегменты кода и данных не пересекались, а также проверки прав класса, модифицирующего другой.

Более подробное рассмотрение

Опкоды организованы по типам:

• Присвоение и чтение
• Арифметические инструкции

• Конвертирование типов
• Создание объектов и манипулирование ими
• Управление кучей
• Контроль передачи данных
• Вызов методов
• Исключения

Поскольку многие из этих инструкций оперируют данными различных типов, для выполнения одной операции может быть несколько опкодов, делающих одно и то же, но предназначенных для работы с разными операндами.

Чтобы байткоды могли быть интерпретированы какой-либо JVA, они должны соответствовать определенным спецификациям (также установленным SUN'ом), в которых задаются возможные инструкции и их операнды.

Вот таблица возможных опкодов:

```
<++>java/opcodes.txt
00 (0x00) nop
01 (0x01) aconst_null
02 (0x02) iconst_m1
03 (0x03) iconst_0
04 (0x04) iconst_1
05 (0x05) iconst_2
06 (0x06) iconst_3
07 (0x07) iconst_4
08 (0x08) iconst_5
09 (0x09) lconst_0
10 (0x0a) lconst_1
11 (0x0b) fconst_0
12 (0x0c) fconst_1
13 (0x0d) fconst_2
14 (0x0e) dconst_0
15 (0x0f) dconst_1
16 (0x10) bipush
17 (0x11) sipush
18 (0x12) ldc
19 (0x13) ldc_w
20 (0x14) ldc2_w
21 (0x15) iload
22 (0x16) lload
23 (0x17) fload
24 (0x18) dload
25 (0x19) aload
26 (0x1a) iload_0
27 (0x1b) iload_1
28 (0x1c) iload_2
29 (0x1d) iload_3
30 (0x1e) lload_0
31 (0x1f) lload_1
32 (0x20) lload_2
33 (0x21) lload_3
34 (0x22) fload_0
35 (0x23) fload_1
36 (0x24) fload_2
37 (0x25) fload_3
38 (0x26) dload_0
39 (0x27) dload_1
40 (0x28) dload_2
41 (0x29) dload_3
42 (0x2a) aload_0
43 (0x2b) aload_1
44 (0x2c) aload_2
45 (0x2d) aload_3
46 (0x2e) iaload
47 (0x2f) laload
48 (0x30) faload
49 (0x31) daload
```

```
50 (0x32) aaload
51 (0x33) baload
52 (0x34) caload
53 (0x35) saload
54 (0x36) istore
55 (0x37) lstore
56 (0x38) fstore
57 (0x39) dstore
58 (0x3a) astore
59 (0x3b) istore_0
60 (0x3c) istore_1
61 (0x3d) istore_2
62 (0x3e) istore_3
63 (0x3f) lstore_0
64 (0x40) lstore_1
65 (0x41) lstore_2
66 (0x42) lstore_3
67 (0x43) fstore_0
68 (0x44) fstore_1
69 (0x45) fstore_2
70 (0x46) fstore_3
71 (0x47) dstore_0
72 (0x48) dstore_1
73 (0x49) dstore_2
74 (0x4a) dstore_3
75 (0x4b) astore_0
76 (0x4c) astore_1
77 (0x4d) astore_2
78 (0x4e) astore_3
79 (0x4f) iastore
80 (0x50) lastore
81 (0x51) fastore
82 (0x52) dastore
83 (0x53) aastore
84 (0x54) bastore
85 (0x55) castore
86 (0x56) sastore
87 (0x57) pop
88 (0x58) pop2
89 (0x59) dup
90 (0x5a) dup_x1
91 (0x5b) dup_x2
92 (0x5c) dup2
93 (0x5d) dup2_x1
94 (0x5e) dup2_x2
95 (0x5f) swap
96 (0x60) iadd
97 (0x61) ladd
98 (0x62) fadd
99 (0x63) dadd
100 (0x64) isub
101 (0x65) lsub
102 (0x66) fsub
103 (0x67) dsub
104 (0x68) imul
105 (0x69) lmul
106 (0x6a) fmul
107 (0x6b) dmul
108 (0x6c) idiv
109 (0x6d) ldiv
110 (0x6e) fdiv
111 (0x6f) ddiv
112 (0x70) irem
113 (0x71) lrem
114 (0x72) frem
115 (0x73) drem
116 (0x74) ineg
117 (0x75) lneg
118 (0x76) fneg
119 (0x77) dneg
120 (0x78) ishl
```

```
121 (0x79) lshl
122 (0x7a) ishr
123 (0x7b) lshr
124 (0x7c) iushr
125 (0x7d) lushr
126 (0x7e) iand
127 (0x7f) land
128 (0x80) ior
129 (0x81) lor
130 (0x82) ixor
131 (0x83) lxor
132 (0x84) iinc
133 (0x85) i2l
134 (0x86) i2f
135 (0x87) i2d
136 (0x88) l2i
137 (0x89) l2f
138 (0x8a) l2d
139 (0x8b) f2i
140 (0x8c) f2l
141 (0x8d) f2d
142 (0x8e) d2i
143 (0x8f) d2l
144 (0x90) d2f
145 (0x91) i2b
146 (0x92) i2c
147 (0x93) i2s
148 (0x94) lcmp
149 (0x95) fcmpl
150 (0x96) fcmpg
151 (0x97) dcmpl
152 (0x98) dcmpg
153 (0x99) ifeq
154 (0x9a) ifne
155 (0x9b) iflt
156 (0x9c) ifge
157 (0x9d) ifgt
158 (0x9e) ifle
159 (0x9f) if_icmpeq
160 (0xa0) if_icmpne
161 (0xa1) if_icmplt
162 (0xa2) if_icmpge
163 (0xa3) if_icmpgt
164 (0xa4) if_icmple
165 (0xa5) if_acmpeq
166 (0xa6) if_acmpne
167 (0xa7) goto
168 (0xa8) jsr
169 (0xa9) ret
170 (0xaa) tableswitch
171 (0xab) lookupswitch
172 (0xac) ireturn
173 (0xad) lreturn
174 (0xae) freturn
175 (0xaf) dreturn
176 (0xb0) areturn
177 (0xb1) return
178 (0xb2) getstatic
179 (0xb3) putstatic
180 (0xb4) getfield
181 (0xb5) putfield
182 (0xb6) invokevirtual
183 (0xb7) invokespecial
184 (0xb8) invokestatic
185 (0xb9) invokeinterface
186 (0xba) xxxunusedxxx1
187 (0xbb) new
188 (0xbc) newarray
189 (0xbd) anewarray
190 (0xbe) arraylength
191 (0xbf) athrow
```

```
192 (0xc0) checkcast
193 (0xc1) instanceof
194 (0xc2) monitorenter
195 (0xc3) monitorexit
196 (0xc4) wide
197 (0xc5) multianewarray
198 (0xc6) ifnull
199 (0xc7) ifnonnull
200 (0xc8) goto_w
201 (0xc9) jsr_w
зарезервированные опкоды:
202 (0xca) breakpoint
254 (0xfe) impdep1
255 (0xff) impdep2
<-->
```

Как видно из вышеприведенного, язык достаточно краток, чтобы его можно было реализовать на системах с небольшим количеством ресурсов, таких как встроенные микропроцессоры, микроконтроллеры, ... , в то же время предусмотрена возможность эффективного выполнения на машинах типа RISC.

Установленные факты:

- Некоторые опкоды не используются. Их обработка, если они встретятся в каком-то классе, будет зависеть от реализации JVM.

- В общем случае каждый опкод имеет форму ixxxx, dxxxx, lxxxx, fxxxx, в зависимости от типа данных, с которым он работает (целые числа, двойные, длинные или плавающие).

- Для сохранения значения в куче используется aload_n, где 0 <= n <= 3. Хочу сказатель, что хотя существует опкод для сохранения числа 2, но для сохранения номера отдельный опкод aload отсутствует. Отдельные опкоды были выделены для наиболее часто используемых значений: 0, 1, 2, 3.

Инструменты

Первым и основным является JAVA-компилятор. Легче всего достать JDK от SUN и, хотя у него не слишком много возможностей и сервисов по сравнению с другими компиляторами, генерируемый им код более-менее приемлем.

Где: java.sun.com

Для того, чтобы совершать путешествия внутрь ява-программ, следует ознакомиться с соответствующими спецификациями на http://java.sun.com/docs/books/vmspec/2nd-edition/html/.

Поскольку мы предполагаем, что у вас установлен JAVA-компилятор, по-видимому, у вас установлен и декомпилятор. Он называется javap и вызывается с помощью команды 'javap -c MiClase', что отобразит на экране дизассемблированный код.

Единственное, что плохо - если встречается ссылка на класс, местонахождение которого невозможно обнаружить, процесс дизассемблирования прерывается.

И, поскольку мы находимся в андерграундном мире, необходимо найти инструмент, умеющий декомпилировать Java-программы. Наиболее часто используемый - это JAD, а лично мне нравится DJ Java Decompiler, который можно найти на http://members.fortunecity.com/neshkov/dj.html.

Генерируемый декомпилятором код выдается на языке JAVA, а не опкоды, хотя если встречаются невалидные опкоды или какое-нибудь другое несоответствие с языком Java, скомпилировать декомпилированный код не получится.

Задачу декомпиляции затрудняет то, что существуют программы для усложнения скомпилированного кода насколько это возможно. Найти такие программы можно, например, на www.zelix.com или www.condensity.com.

Jasmin - компилятор опкодов. Он трансформирует опкоды в классы. На вход он принимает то, что мы получили с помощью javap.

jas - другой JAVA-компилятор. Он позволяет генерировать динамический код и модифицировать программу на лету, что является полезной техникой для создания мутирующих программ, чтобы спрятать реальный код.

D-java - это еще один JAVA-дизассемблер, написанный на языке C (доступны исходники). Он так же прост, как и javac, но он несколько более устойчив.

Больше информации вы можете получить здесь: http://www.meurrens.org/ip-Links/java/codeEngineering/#tocDecompilersToJava

Практика

Теперь, когда вам известна основные теоретические сведения, мы можем начать с простого примера. Жертвой будет программа под названием BEA-WebLogic. Это приложение работает в Windows так же как и в UNIX, так как полностью написано на JAVA, и представляет собой JAR-архив, в котором упакованны все соответствующие классы. В своей основе это сервер веб-приложений, поэтому в него включен веб-сервер, сервлеты и EJB. Загрузить BEA-WebLogic можно с www.bea.com или www.beasys.com. Использовалась версия 5.1. Уже вышла 6.0, но для того, чтобы понять технику, хватит и 5.1.

Будучи проинсталлированной в c:\weblogic, система лицензий устанавливает в c:\weblogic\WebLogicLicense.xml типа XML (т.е. текстовый) со примерно следующими строками

```
<LICENSE PRODUCT="WebLogic" IP="127.0.0.1"
      UNITS="1" EXPIRATION="never" KEY="XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX" />
```

Игра с приложением

Модифицирование KEY приводит к LicenseKeyInvalidException, которое находится в классе, располагающемся в файле c:\weblogic\classes\weblogic\common\LicenseKeyInvalidException.class

Сейчас мы должны найти все файлы, которые обращаются к этому классу, то есть содержать строку LicenseKeyInvalidException, что дает нам c:\weblogic\classes\weblogic\t3\svr\T3Srvr.class.

Декомпилируем его с помощью DJ Java Decompiler и получаем

```
private void handleLicenseException(LicenseException licenseexception)
{
.......
if(licenseexception.getClass().isInstance(new LicenseKeyInvalidException()))
 {
 if(licenseExceptions == null)
  {
  licenseExceptions = new StringBuffer("");
  licenseExceptions.append("\n" + licenseexception.getMessage());
  return;
  }
 licenseExceptions.append("\nAnd also: " + licenseexception.getMessage());
 }
 .......
}
```

после чего вызывается

```
private boolean checkAccess()
{
.......
Object obj = null;
try
        {
        FileInfo fileinfo = ServerUtil.findFile("WebLogic" + logSuffix());
        fileinfo.checkAccess(0, logVal());
        x86 = true;
```

```
        }
catch(LicenseException licenseexception)
        {
        handleLicenseException(licenseexception);
        }
.......
}
```

после чего вызывается

```
public static void main(String args[], String s, String s1)
{
.......
t3srvr.checkAccess();
.......
}
```

Также в c:\weblogic\classes\weblogic\common\internal\FileInfo.class встречаем

```
public void checkAccess(int i, String s) вызывается исключение LicenseException
{
.......
if(!getFullName(s).equals(getPermissions()))
 throw new LicenseKeyInvalidException("License key incorrect for "
 + toString());
.......
else
        return;
}
```

Пачкаем руки

Хоть и не очень сильно.

Один из наиболее часто применяемых методов - это не вызывать checkAccess в main, хотя также можно не вызывать исключение LicenseException в fileinfo.checkAccess или узнать, какой метод сохраняет KEY.

Благодаря JAD'у мы можем отредактировать полученный исходный код: удаляем вызов checkAccess и компилируем новый T3Srvr.java.

Делайте так:

```
jad -d T3Srvr.class > T3Srvr.java
комментируем строку 't3srvr.checkAccess();'
javac T3Srvr.java
```

Запускаем программу.

И вуаля! Код работает.

Если, в силу какой-либо причины, сгенерированный JAD'ом код нельзя скомпилировать, остается другой способ: дизассемблирование вручную. Нам нужно избежать вызова checkAccess.

Дизассемблирование с помощью D-java или javap дает:

```
Method public static void main(String[],String,String)
 0 iconst_0
 1 istore_3
 2 invokestatic #595 <Method weblogic.kernel.Kernel.setIsServer():void>
 5 new #353 <Class weblogic.t3.srvr.T3Srvr>
 8 dup
 9 invokespecial #678 <Method weblogic.t3.srvr.T3Srvr.<init>():void>
12 putstatic #732 <Field weblogic.t3.srvr.T3Srvr.theT3Server:
                        weblogic.t3.srvr.T3Srvr>
15 sipush 26160
18 invokestatic #491 <Method weblogic.html.HtmlElement.setAnchorMode(int):
                             void>
21 invokestatic #525 <Method weblogic.t3.srvr.T3Srvr.getT3Srvr():
                             weblogic.t3.srvr.T3Srvr>
24 astore local4
```

```
26 aload local4
28 aload_1
29 putfield #435 <Field weblogic.t3.srvr.T3Srvr.logval:String>
32 aload local4
34 aload_2
35 putfield #805 <Field weblogic.t3.srvr.T3Srvr.logsuffix:String>
38 invokestatic #804 <Method weblogic.t3.srvr.T3Srvr.configure():
                            weblogic.t3.services.Config>
41 pop
42 aload local4
44 invokespecial #485 <Method weblogic.t3.srvr.T3Srvr.checkAccess():boolean>
47 pop
48 aload local4
50 invokevirtual #650 <Method weblogic.t3.srvr.T3Srvr.start():void>
53 goto 70
.......
```

Вызов, который нам нужно избежать, находится в строке 44. У инструкции invokespecial опкод 0xB7, а 485=0x01E5. Посему загружаем T3Srvr.class в наш любимый шестнадцатиричный редактор, ищем последовательность B701E5, которая оказывается по смещению 8B27 и заменяем ее на 000000 (три NOP'а). И остается перекомпилировать.

Другой пример

Как и в других языках, написанный на Java код важно привести в порядок и сделать более читаемым. Для этого есть специальные программы, форматирующие исходный код.

Одной из таких программ является Jindent, который можно получить на www.jindent.de. Шароварная версия неудобна, так как не форматирует файлы объемом более 400 строк. Мы попытаемся убрать это ограничение.

Программа вызывается следующим образом:

```
java -jar Jindent.jar Jindent MiClase
```

но так как в MiClase.java больше 400 строк, то получаем ошибку

```
Error: ".\MiClase.java" exceeds 400 lines of code.
Parsing terminated.
```

Это весьма огорчительно.

Код находится в файле Jindent.jar, который, как нам хорошо известно, является не более чем Java ARchive, поэтому мы можем использовать jar -xvf Jindent.jar. Переходив в директорию jindent\, где находятся все архивы. Ищем строку "exceeds 400 lines of code", но не находим. Ищем "exceeds" и снова не находим.

В чем же дело? По-видимому, код был прогнан через обфускатор. У нас есть архивы под названием a.class, b.class, c.class, ..., чье название нам никак не помогает, а просмотр каждого класса будет весьма утомительным.

Распаковываем файлы и пишем

```
java Jindent MiClase.java
```

Чтобы получить немного больше информации пишем

```
java -verbose:class Jindent MiClase.java

.......
[Loaded java.awt.LightweightDispatcher$2 from c:\jdk1.3\JAVA2\lib\rt.jar]
[Loaded sun.awt.ScreenUpdater from c:\jdk1.3\JAVA2\lib\rt.jar]
[Loaded sun.awt.ScreenUpdater$1 from c:\jdk1.3\JAVA2\lib\rt.jar]

Parsing from file  ".\MiClase.java".
[Loaded jindent.m]
[Loaded java.awt.geom.Rectangle2D$Double from c:\jdk1.3\JAVA2\lib\rt.jar]
[Loaded java.awt.geom.GeneralPath from c:\jdk1.3\JAVA2\lib\rt.jar]
```

```
[Loaded jindent.y]

Error: ".\MiClase.java" exceeds 400 lines of code.
Parsing terminated.
```

Похоже, что m.class отвечает за загрузку файла, а класс y.class жалуется на превышение лимита. С помощью нашего друга JAD'a декомпилируем m.class. У нас есть некоторое количество функций с разными именами. Например, у нас есть 15 функций с именем a, но с различным количеством и типом аргументов. Это так называемые перегруженные методы, что для нас не слишком хорошо. В частности, последняя функция:

```
private static String a(String s1)
{
        char ac[];
        int i1;
        int j1;
        ac = s1.toCharArray();
        i1 = ac.length;
        j1 = 0;
        goto _L1
_L9:
        ac;
        j1;
        JVM INSTR dup2 ;
        JVM INSTR caload ;
        j1 % 5;
        JVM INSTR tableswitch 0 3: default 76
    //          0 52
    //          1 58
    //          2 64
    //          3 70;
        goto _L2 _L3 _L4 _L5 _L6
_L3:
        0x30;
        goto _L7
_L4:
        102;
        goto _L7
_L5:
        9;
        goto _L7
_L6:
        80;
        goto _L7
_L2:
        44;
_L7:
        JVM INSTR ixor ;
        (char);
        JVM INSTR castore ;
        j1++;
_L1:
        if(j1 < i1) goto _L9; else goto _L8
_L8:
        return new String(ac);
}
```

Как видим, опкоды невозможно перекомпилировать с помощью простого вызова javac. Но так как они закоментированы, это не важно. Просто изменяем последнюю функцию в файле m.java следующим образом:

```
private static String a(String s1)
{
        return new String("funcion   a   ha sido llamada "+s1);
}
```

Теперь, поскольку теперь не вызывается ничего из того, что вызывалось раньше, нам не о чем беспокоиться.

И помещаем в каждый методод следующую строку

```
System.out.println("estoy en la funcion xxxx ");
```

в каждый метод.

Запускаем программу и видим, что последние строки выполняются вместе с методом o() и после метода b().

```
public void b()
{
    System.out.println("estoy en la funcion b() ");
    r = null;
    b = null;
    k = null;
    l = null;
}
```

Похоже, что очищаются какие-то указатели. Ничего особенного.

С другой стороны

```
public int o()
{
    System.out.println("estoy en la funcion o() ");
    int i1 = 0;
    for(int j1 = 0; j1 < s; j1++)
        if(r[j1] == '\n')
            i1++;
    System.out.println("estoy en la funcion o() . Retorno i1="+i1);
    return i1;
}
```

Запускаем программу и в результатет получаем

```
    "estoy en la funcion o() . Retorno i1=2000"
```

Случайно в нашем файле оказывается 2000 строк. Все становится ясным: функция o() возвращает количество строк в файле.

В редких случаях эта функция не вызывается из этого модуля. Тогда делаем printStackTrace(), пока не будет возвращен i1, получаем:

```
    at jindent.m.o(m.java, Compiled Code)
    at jindent.JindentParser.a(JindentParser.java)
    at jindent.JindentParser.c(JindentParser.java, Compiled Code)
    at jindent.JindentParser.d(JindentParser.java)
    at jindent.JindentParser.invoke(JindentParser.java, Compiled Code)
    at Jindent.main(Jindent.java)
```

До того, как пропатчить, исследуем программу еще немного.

Дизассемблируем JindentParser.class и ищем, где происходит первый вызов o().

```
void i(k k1)
{
    if(k1.n())
    {
        int i1 = k1.o();
        C.setVariable(E("p\016]xvl?P}a"), k1.p());
        for(int j1 = 0; j1 < i1; j1++)
        {
            String s1 = k1.i(j1);
            s1 = e(s1);
            g(s1);
        }
    }
}
```

В цикле переменной i1 присваивается результат o(). Пока лучше не патчить o() (В любом случае, мы несколько смущены. Откуда мы знаем, что k1 - это объект типа m?).

второй вызов:

```
строка 11255
void a(Reader reader, Writer writer) throws JindentException
{
mW();
cA = new m(reader, 1, 1, e, u);
.........
if(cA.o() > bi - 512)
 {
 bY();
 throw new JindentException(E("K\005Yxp\"\016Qnag\017Z-02[\taml\016Z-kdKJb`gE"));
 }
.........
```

Это очень интересно. cA - это один экземпляр объекта типа m, и где-то в конце вызывается m.o().

```
          if(cA.o() > bi - 512)
```

где bi равно ?

Переходим к определению переменных и (нам везет) находим константу bi:

```
    bi = 912;
```

Ах, теперь все понятно: bi - 512 = 912 - 512 = 400, все просто!

третий вызов (строка 8158):

```
void a(String s1, String s2, String s3) throws JindentException
{
Object obj = null;
.........
cA = new m(filereader, 1, 1, e, u);
if(cA.o() > bi - 512)
 {
 bY();
 throw new JindentException("\"" + s2 + E(" KLugg\016M~$6[\031-hk\005L~$m\r\tnkf\016\007"));
 }
.........
```

Теперь просто необходимо пропатчить значение bi JindentParset.class, чтобы утилита принимала файлы размером до 65535-512.

Другая техника состоит в использовании отладчика. В JDK входит один, хотя он предоставляет большого количества сервисов. Большую часть информации он выдает, если класс был скомпилирован с опцией отладки.

С другой стороны, две наиболее используемые среды разработки, VisualAge и VisualCafe имеют возможность посмотреть, что делает класс без необходимости иметь исходный код.

Стоит упомянуть о SUN'овском Hotspot'e, прекрасой JVM с JIT'овским компилятором с прилагающимся исходным кодом. Он может не просто выполнять инструкции, но создавать файл, который затем запустит.

Туда включен графический пошаговый отладчик.

Есть еще много тем, касающихся Java, которых было бы интересно коснуться.

Например:

• реализация JVM для новичков
• функционирование Sandbox и как подгружать классы с помощью другого ClassLoader'a
• генерация java-кода на лету для повышения безопасности
• вызовы DLL и библиотек S.O. с помощью jni

• сервлеты, RMI, EJB

Наконец, Java - это язык, который быстро развивается, для которого уже написано много приложений и еще больше появится в будущем (все оказалось не так радужно - прим.пер.)

Если предсказания известных аналитиков софтверного рынка оправдаются, 90% процентов приложений будут интегрированы с Web, а это значит, что они будут переведены на Java (автор статьи не подозревал, что Miscrosoft замутит .Net и связанную с ней ересь :) - прим.пер.). Как бы то ни было, можно назвать такие примеры как Oracle, SmartCard, Vantive, Lotus и IBM.

Обзор некоторых защит на основе FLEXlm SDK

Автор: Volodya

Дата: 2003-01-30

"_… ибо его оружие - мозг - не требовало особого
ухода, спецхранения и работало в любых условиях._"
В. Головачев "Схрон"

Автор: Volodya

Итак, я предполагаю, что читатель имеет твердое представление об ассемблере, неплохо владеет Си, прекрасно разбирается в том, как компилировать и линковать программы. Также, очевидно, необходимо уметь пользоваться Visual Studio.

Данная статья написана как довесок к моей основной статье о профилировщиках. Здесь рассказывается, как устранить некоторые проблемы инсталляции и работы программ , использующих для своего функционирования набор API-функций, предоставляемых пакетом FlexLm. Так уж получилось, что подавляющее большинство профилировщиков, средств покрытия кода (т.е. средств, показывающих, сколько веток if и других циклов выполняется при нормальном течении хода проги, это очень полезно, т.к. показывает, насколько ваша прога "готова" к нестандартным изменениям ситуации, как тщательно вы подготовились к неприятностям, т.к. если ветки, которые получают управление редко, небрежно написаны, то в результате можно получить ой-ой-ой!) и выявления утечек памяти используют именно FlexLm. Среди них – Intel Compiler/Vtune environment, все продукты семейства DevPartner от Numega, и все опробованные мной продукты от Rational.

Рассмотрим сперва Intel Compiler, единственный реальный конкурент MS cl.exe. Закачать его можно с <ftp://download.intel.com/software/products/>. Последняя версия на момент написания -W_CC_P_7.0.073.exe. «Вес» пакета – 74 Мб.

Итак, закачиваем и инсталлируем (помните, в обязательном порядке должно иметь место наличие присутствия Visual Studio 6/Net). При попытке инсталляции получаем что-то вроде FlexLm license error. Теперь мы и подобрались к сути проблемы.

Итак, FlexLm – это творчество компании GLOBEtrotter Software. Фиговина эта требует присутствия .lic файла (действие которого может быть ограниченным во времени), без которого работать не хочет ну никак. Компания, как это модно сейчас, выпускает FlexLm SDK, где имеются API-функции для работы с lic-файлом, которые надо втиснуть в исходный код, скомпилировать с соответствующими .h-файлами и слинковать с соответствующими библиотеками.

Только в качестве иллюстрации:

```
#include "lmpolicy.h"    //компиляция с нужным файлом оглавления

LP_HANDLE *lp_handle;

status = lp_checkout(LPCODE, policy, feature, version, num_lic,
                           license_file_path, &lp_handle) //здесь зовем нужные функции
```

Библиотеки для разных случаев используются разные. Скажем, для линковки мультипоточного варианта (multithread) будут использоваться одни библиотеки, если приложение представлено одним единственным процессом – другие. Ниже дан пример из FlexLm SDK версии 8.0:

FLEXlm Object File — FLEXlm Client Libraries — MSVC++ Libraries

Standard — lm_new.obj — lmgr.lib — libcmt.lib(/MT) oldnames.lib kernel32.lib user32.lib netapi32.lib advapi32.lib gdi32.lib comdlg32.lib comctl32.lib wsock32.lib

Add for CRO — libcrvs.lib libsb.lib

Add for static FLEXlock — flock.lib

For /MD — Replace lm_new.obj with lm_new_md. obj — Replace Standard lmgr.lib with lmgr_md.lib — Replace Standard libcmt.lib with msvcrt.lib (/MD)

For CRO /MD — Add to Standard: libcrvs_md.lib libsb_md.lib — Replace Standard libcmt.lib with msvcrt.lib (/MD)

Видно, что здесь есть много интересного. Сразу появляется желание скачать этот самый SDK и маленько с ним разобраться. Ничего невозможного - http://linux20368.dn.net/crackz/Flexlm.htm, т.к. с родного сайта компании вы уже ничего не скачаете - доступ закрыт, однако.

Теперь о том, как это ломать. Варианта два – 1-й - разобраться с форматом .lic файла, подправить оригинальные API-функции, словом вдумчиво ковырять FlexLm SDK (это уже и было проделано и в сети есть много англоязычных статей на эту тему), 2-й – лично мне больше по душе – ковырять конкретное приложение, которое неумело использует функции FlexLm. Есть идиоты, которые вообще выносят эти API-функции в отдельную .dll, вместо статической линковки, и это несмотря на предупреждения, красным по белому:

Caution: Use of the FLEX *lm* DLL on Windows remains strongly discouraged

А ведь если функции вынесены в dll, то кракеру поймать их ничего не стоит. Пример не за горами.

Итак, теперь к делу.

Выкачиваем компилятор. Пробуем запустить – ругается, нужен lic-файл. Его, ессно, нет. Начинаем копать. Существует достаточно много подходов ко взлому конкретного приложения. Кто к каким привык. Стандартом, как правило, считается постановка bpx на что-нибудь вроде MessageBoxExW или DialogBoxIndirectParamA, затем исследуется дизассемблированный код ВЫШЕ точки «всплыва» отладчика и определяется, какой из условных переходов, привел к таким ругательствам со стороны проги.

MessageBoxExW
Для постановки точки останова следует всегда выбирать MessageBoxExW вместо MessageBoxA или тому подобных. Дизассемблируйте user32.dll и увидите, что MessageBoxA, W, ExA являются просто переходниками к ExW. Такое имеет место быть только в чисто юникодовских системах, таких как Windows NT, 2000, XP. К 95/98 это не относится, а если что-то такое и имеет место быть, то с точностью до наоборот. Данное правило подходит не только к MessageBox, оно соблюдается и для многих других WinAPI функций – ASCII-варианты являются просто переходниками к юникодовским

Еще один вариант – это прогонять программу под каким-нибудь API-монитором, вроде BoundsChecker от NuMega. Реальных конкурентов у него нет, одни слишком медленные и тупые, вторые, может и быстрые, но для хакера совершенно непригодны. На этом этапе, если приложение использует какие-то API-функции из определенного рода библиотек (например, flexlm-библиотек), то это все станет очевидным.

Можно даже и не пользоваться API-монитором, т.е. динамическим анализом программы. Вполне хватит и статического, если не используется явное связывание (explicit linking). Для статического анализа лучше всего использовать dumbin или dependency walker. Оба от MS.

Explicit linking
Динамическую библиотеку можно вызывать двумя способами – явным и неявным связыванием. Неявное связывание – наиболее широко используемый способ. Здесь с самого начала компилятору строго говорится, что вот так-то и так-то, вот это вот считать dll и поступать с ней вот так, а не иначе. Явное связывание – это явная задача в исходном коде инструкций вида LoadLibraryA для загрузки конкретной библиотеки (например, kernel32.dll), а потом GetProcAddress для возвращения адреса конкретной функции (например, IsDebuggerPresent), а потом что-то вроде call [адрес, возвращенный GetProcAddress]. Ясно, что неявное связывание оставляет следы в таблице импорта, которые легко обнаружить, например, dumpbin. Явное связывание в таблице импорта не отображается. Подробнее – Джеффери Рихтер «Windows для профессионалов»

Очевидно, что любой из видов анализа рано или поздно приведет вас к анализу дизассемблированного кода. Лучше всего использовать IDA. Аналогов ей нет все равно. Вообще, я лично с презрением отношусь ко всяким заявлениям типа «следует юзать только HIEW, потому что настоящие кульные хацкеры юзают только HIEW и ничего больше». Надо уметь пользоваться всем, что под руку попадается.

Так вот, не важно, какой из вышеперечисленных методов используется, так или иначе, мы с вами вышли на файл ChkLic.dll, который экспортирует одну, ну жутко интересную функцию - _CheckValidLicense. Дальше выбора два – либо делать патч непосредственно в dll, либо в зовущем файле – setup.exe. Код выглядит так:

```
Setup.exe - 40,960 bytes

_text:0040175D loc_0_40175D:                           ; CODE XREF: _WinMain@16+744j
; явное связывание в наглядном варианте. В таблице импорта setup.exe следов вызова
; функции не будет, однако рапорт BoundsChecker покажет, что имел место вызов .dll и
; вызывалась соответствующая функция
_text:0040175D                 push    offset aChklic_dll    ; lpLibFileName
_text:00401762                 call    ds:LoadLibraryA
_text:00401768                 mov     esi, eax
_text:0040176A                 push    offset a_checkvalidlicense ; lpProcName
_text:0040176F                 push    esi                ; hModule
_text:00401770                 call    ds:GetProcAddress
_text:00401776                 cmp     esi, ebx
_text:00401778                 jz      loc_0_40192E
_text:0040177E                 cmp     eax, ebx
_text:00401780                 jz      loc_0_40192E
_text:00401786                 push    ebx
_text:00401787                 call    eax  ; _CheckValidLicense
_text:00401789                 test    eax, eax
_text:0040178B                 jnz     loc_0_4019A3  ;заменить на jz
```

Можно считать, что с первой линией обороны мы справились. Заметьте, что защита flexlm пакету не помогла вообще, мы даже и не узнали, есть ли она там. Авторам надо было хоть немного читать то, что написано в SDK. А ведь ситуация могла бы и изменится, если бы использовалась статическая линковка. Но на каждую хитрую жопу есть хрен с винтом. Выход очень прост. Далее, в файлах icl.exe (интеловский компилятор для IA-32) и ecl.exe (IA-64 вариант) уже никакие dll не используются, и, все равно, это ничем не помогло разработчикам. Здесь можно работать по следующей методике. Сперва нам надо узнать, какая версия flexlm используется, использовать технологию IDA FLAIR, исследовать код вблизи мест проверки lic-файла и взломать прогу.

IDA FLAIR
Ильфак Гюильфанов написал замечательную статью о том как в IDA реализован механизм определения библиотечных функций – «Как IDA распознает стандартные функции?». Статью можно найти на pilorama.com.ru, повторяться не хочу.

Для определения версии flexlm можно использовать любой текстовый фильтр, либо тот, который найдет требуемое слово в бинарнике (Search and Replace – kickme.to/FOSI), либо тот, который вывалит все текстовые строки бинарника (мой собственный, например – wasm.ru – filter, "исходники").

Так или иначе:

```
24/12/2002 5:33:52 PM
Search String: flexlm
Replace String:
Path: C:\Downloads\comp\
File Mask: *.*

Processing file : C:\Downloads\comp\chklic.exe

Offset 0x680dd - $Id: flexlm.c,v 1.10.2.1 2002/10/18 20:06:49 mtoguchi Exp $
Offset 0x6e781 - yyyyyyyyyyyy@(#) FLEXlm v7.2i (liblmgr.a),
```

```
Copyright (C) 1988-2001 Globetrotter Software, Inc.
```

Верхняя строчка оставлена для хохмы, это вообще одуреть. Строки отчета cvs попали в бинарник! Это ж какое качество компилятора! Ладно, зато теперь нам точно известно, какая версия использовалась. Это дает возможность использовать средства FLAIR для получения файла сигнатур, который поможет элементарно найти нужные строки кода в дизассемблированном листинге. Ниже приведен пример создания файла сигнатур для FlexLM SDK 8.0.

```
E:\flexlm\v8.0\i86_n3>pcf lmgr.lib lmgr
lmgr.lib: skipped 0, total 142

E:\flexlm\v8.0\i86_n3>pcf lmgr_md.lib lmgr_md
lmgr_md.lib: skipped 0, total 142

E:\flexlm\v8.0\i86_n3>sigmake -n"FlexLm Win32 v8.0
Standard&Multi-thread" lmgr.pat+lmgr_md.pat lmgr.sig
```

В первой строке запускается парсер для lib-файлов coff-формата (elf-формат идет под Linux). В результате имеем pat-файл. В данном случае запускаем парсер второй раз для получения мультипоточного варианта библиотеки. Потом используется утилита sigmake для объединения полученных pat-файлов в один sig-файл, ключ –n дает файлу название, под которым его видит конечный пользователь. Процесс опознавания библиотечных функций не безупречен, часто появляется файл коллизий. Опять таки, отсылаю к статье Ильфака. Файл коллизий редактируется очень просто. В этом файле представлены функции, которые имеют сходные CRC. Теперь уже нам с вами нужно решать, что с ними делать (ставить "-" для появления имени функции в виде комментария, "+" для обычного вида, не ставить ничего для полного исключения функции из файла сигнатур). Я предпочитаю включать ту, что выполняет более общее действие. Пример:

```
+_l_swap                                                58              734E
558B4C24108BEC83EC60538B450CC745FC0000000056578B34888B7C880485F6
_swap                                                   58              734E
558B4C24108BEC83EC60538B450CC745FC0000000056578B34888B7C880485F6
```

Здесь поставлен знак «+» перед функцией _l_swap, так как думается мне, что эта функция более напоминает при анализе бинарника функцию flexlm SDK. Аналогично расставляются знаки и для других пар (или большего количества) функций, вызвавших коллизии.

После редактирования .exc файла просто повторите:

```
E:\flexlm\v8.0\i86_n3>sigmake -n"FlexLm Win32 v8.0
Standard&Multi-thread" lmgr.pat+lmgr_md.pat lmgr.sig
```

Утилита сама поймет, что делать с файлом коллизий на этот раз. В завершение процесса нужно сжать файл сигнатур утилитой zipsig и поместить его в директорию /sig папки IDA.

Дизассемблируем файл icl.exe и ecl.exe. Применяем полученный файл сигнатур (я приложил sig-файлы для FlexLM SDK v6.0/7.2i/8.0). У меня было опознано свыше девятисот(!) библиотечных функций. Теперь мы сравнительно легко придем к нужному результату:

```
icl.exe - 684,032 bytes


_text:0040105A loc_0_40105A:                            ; CODE XREF: _main+350j
_text:0040105A                  mov     eax, dword_0_488B24
_text:0040105F                  call    sub_0_4083F4
_text:00401064                  mov     eax, offset aFfrpsz  ; "FFrpsZ"
_text:00401069                  mov     edx, 21h
; легко можно увидеть, что эта функция "скатывается" к FlexLMMainCheck
; думаю, название снимает все вопросы
_text:0040106E                  call    sub_0_404C0E  ;mov eax,0
_text:00401073                  test    eax, eax
_text:00401075                  jnz     short loc_0_40107F


ecl.exe - 692,224 bytes
```

```
_text:0040305A loc_0_40305A:                           ; CODE XREF: _main+350j
_text:0040305A                 mov     eax, dword_0_48AB44
_text:0040305F                 call    sub_0_40BD51
_text:00403064                 mov     eax, offset aFfrpsz  ; "FFrpsZ"
_text:00403069                 mov     edx, 21h
_text:0040306E                 call    sub_0_40837E  ;mov eax,0
_text:00403073                 test    eax, eax
_text:00403075                 jnz     short loc_0_40307F
```

Теперь у нас есть компилятор. Самое время взломать профилировщик.

Используя методы, описанные выше, мы легко сможем обнаружить, что ругательство при запуске VTune выдает AnProdInfo.dll, опираясь на ресурсы из AnProdInfoRes.dll. Дополнительно, чтобы облегчить себе жизнь, нужно использовать команду Soft-Ice – stack – она покажет последовательность вызовов функций и мы сможем вычислить искомую.

STACK – проход стека, FPO и т.п.

Кадр стека (т.е. адресация по ebp либо инструкции enter/leave) - великая вещь, своим появлением он обязан относительно "скудным" возможностям адресации процессоров до 386-го (тогда с адресацией было туговато, не всякий регистр подходил), где в качестве индексного регистра разрешалось использовать только si,di,bx,bp. Теперь в качестве индексного можно использовать любой 32-разрядный регистр, например esp. Тогда кадр стека становится ненужным (т.к. используется адресация переменных по esp ± xxh - такой подход называется frame pointer omission (FPO)), а это есть хорошо для прикладных программистов и не есть хорошо для нас с Вами, т.к. при esp-подходе теряется большая часть символьной информации (например, команда S-Ice stack становится бесполезной, если в функции отсутствует кадр стека). Если разработчик программы - лопух и забыл убрать отладочную информацию, то мы сможем выловить с помощью dumpbin /FPO информацию о количестве переданных параметров и т.п. Пример: с помощью Search&Replace я задал искать слово .debug (есть иногда такая секция в PE-файле) в директории Windows\system. Нашлось файлов 5. Вот часть дампа со случайно взятого файла: Уже сейчас мы можем выяснить размер процедуры, количество переданных параметров, количество локальных переменных и т.п. Если Вы не очень уютно себя чувствуете с кадром стека - почитайте Рэндалла Хайда и проиграйтесь с отладчиком - многое прояснится. Может (и справедливо!) возникнуть еще один вопрос: "Хорошо, я знаю адрес процедуры, количество параметров и прочую ерунду. А КАК мне сделать, чтобы это узнал и Soft-Ice?" Ответ см. ниже - он есть, а пока методику поняли? Ищем секцию отладки в PE-файле (F8,F6 в HIEW) - если есть, она будет называться как-нибудь вроде .debug. Далее извлекаем ее (см. ниже), переводим в понятный S-Ice'у формат (см. ниже) и работаем дальше.

```
dumpbin.exe /FPO sh30w32.dll >fpo.txt
Dump of file sh30w32.dll
File Type: DLL
FPO Data (222)
                                                 Use Has  Frame
RVA      Proc Size   Locals  Regs   Prolog  BP  SEH  Type   Params
00001000        96        0     2        2   N   N    fpo        4  _MemPoolShrink@4
00001060       144        8     4       21   Y   N    fpo        0
```

Результат:

```
W_VT_P_61_0011.EXE - закачать с intel.com, размер - свыше 100 мб.

AnProdInfo.dll

.text:602341C0                 call    sub_60232466 ->xor eax,eax
.text:602341C5                 xor     ebx, ebx
.text:602341C7                 add     esp, 0Ch
.text:602341CA                 cmp     eax, ebx
.text:602341CC                 mov     [esi], eax
.text:602341CE                 jz      loc_6023429D
.text:602341D4                 mov     [ebp+0Ch], ebx
```

```
.text:602341D7                cmp     eax, 0FFFFFFEBh
.text:602341DA                mov     [ebp-4], ebx
.text:602341DD                jz      short loc_602341F2
.text:602341DF                cmp     eax, 0FFFFFFF6h
.text:602341E2                jz      short loc_602341EB
.text:602341E4                push    0D1h
.text:602341E9                jmp     short loc_602341F7
```

«Однако за время пути собака могла подрасти». Пока писалась эта статья, я совершенно случайно накопал еще один update от Intel на тему Vtune –

<ftp://download.intel.com/software/products/vtune/downloads/>.

Что он делает, в двух словах и не рассказать. В своей статье о профилировщиках я его не описываю тоже, отчасти потому, что работать с ним страшновато. Это надстройка над JavaMachine, да еще к ней прибабахан MySQL-сервер. Для чего все это нужно, мне сказать сложновато. Естественно, все это дело тормозит жутко, а потому вдумчиво его ковырять на предмет радостных сообщений защиты мне было недосуг.

Поэтому здесь я скорее предпочту описать методики, как можно поймать интересующий код, чем давать готовые решения, хотя парочка решений, конечно же, будет.

Итак, немного поиграв с пакетом, имеем стандартный MessageBox со стандартными фразами. Что здесь можно сделать?

Вещь первая. Если строки сообщения защиты находятся в ресурсах PE-файла, то надо воспользоваться редактором ресурсов (например, Restorator 2.52 – wasm.ru), чтобы узнать их порядковый номер. Пример:

```
123      The GUI license for VTune(TM) Enterprise Analyzer for Web
            Applications is invalid.  Please contact technical support.
```

Здесь, "123" – это произвольный (т.е. разработчик брал от фонаря) порядковый номер ресурса, который в дальнейшем будет использован функцией LoadString. Номер этот будет передан функции через стек (таков закон). Т.е. будет использоваться инструкция push + шестнадцатеричное представление "123" (0x7B). Такой шаблон легко найти в HIEW – в режиме ассемблерного поиска надо ввести push ?7B (отметьте знак вопроса – он означает, что перед 7B может идти что угодно). Чаще всего, вхождений будет несколько, однако что стоит найти правильное?

```
.text:10024387                push    7Bh             ; uID
.text:10024389                mov     [esp+4Ch+var_4], esi
.text:1002438D                mov     ecx, [eax+8]
.text:10024390                push    ecx             ; hInstance
.text:10024391                lea     ecx, [esp+50h+var_34]
.text:10024395                call    sub_10010A10   ;к LoadString
```

А дальше, опять таки нужно воспользоваться командой stack в Soft-Ice, чтобы понять, какая последовательность вызовов функций привела к столь плачевному результату.

Вещь вторая. Коль уж вхождение найдено, часто оказывается необходимым ставить bpx не на MessageBox и ему подобные, а на конкретные адреса. Т.к. VTune – продукт, содержащий множество .dll, это автоматически означает, что будет производится перерасчет адресов загрузки .dll (подробнее – моя статья о профилировщиках или многочисленные статьи Мэтта Питрека и Джона Робинса в сети). А это, в свою очередь, означает, что адреса, которые мы увидим в IDA/WDasm/HIEW и т.п. НЕ будут соответствовать адресам в памяти, а предсказать адрес, по которому dll будет загружена, достаточно сложно, т.к. точно не известен порядок, в котором приложение загружает эти dll. Однако выход здесь прост. Старый кракерский трюк с int 3. int 3 имеет опкод 0CCh. Влепите этот опкод в требуемое место программы (не забыв выровнять количество байт и выписать эти самые байты, потому что потом их придется восстанавливать в Soft-Ice). Теперь для того, чтобы Sof-Ice отловил этот опкод, надо активировать опцию i3here on.

i3here on

Soft-Ice является отладчиком класса last-chance notification. Это означает, что он предоставляет отладчикам класса first-chance notification (таким, как отладчик VisualStudio) шанс обработать исключительную ситуацию, перед тем как брать бразды правления в свои руки. Что такое "последнее предупреждение" и "первое предупреждение" в двух словах не рассказать, это связано с реализацией SEH (structured exception handling) в Windows и знакомо специалистам по C++ как блоки try/catch. Да, кстати, почитайте в документации к Soft-Ice, что такое опция faults off. Она во многом определяет поведение Soft-Ice во время исключительных ситуаций. Есть также неплохая книжка Джеффери Рихтера – “Windows для профессионалов”. Там тоже очень неплохо рассказывается об исключениях. Интересующихся низкоуровневой организацией SEH в Windows (на уровне внутренних структур данных) отсылаю к статьям Мэтта Питрека.

После срабатывания точки останова и «всплыва» Soft-Ice необходимо поправить EIP (r eip = требуемое значение) так, чтобы он показывал ПРЯМО на int 3, а не на инструкцию ПОСЛЕ нее (там, где, собственно, и произошел всплыв). После того как поправили EIP, с помощью инструкции "a" или "e" поправьте байты по тому адресу (на те самые, которые выписывали, помните?).

Рано или поздно, исследуя внутренности VTune Enterprise можно наткнуться на функцию ?IsJavaEnabled@CTahoeLicenseMgr@@QAE_NXZ. Это одна из функций защиты приложения. Функция экспортируется библиотекой VTEGUIShared.dll и "спускается" к

```
    ?IsFeatureEnabled@CTahoeLicenseMgr@@QAE_NW4SPFeatureEnum@@@Z
.text:1002D780                 mov     eax, [ecx+4]
.text:1002D783                 mov     ecx, [esp+arg_0]
.text:1002D787                 and     eax, ecx
; Очень изящный и приятный код. Это пример хорошей оптимизации. Смотрите, вообще нет
; условных переходов, которые для современных CPU не представляют собой ничего хорошего.
; Вместо этого используется инструкция вычитания с заемом - более подробно см. статьи
; Касперски. А здесь, недолго мороча голову:
.text:1002D789                 neg     eax  --> mov eax,1; nop
.text:1002D78B                 sbb     eax, eax
.text:1002D78D                 neg     eax
.text:1002D78F                 retn    4
```

Вроде все методики расписаны. Единственная вещь, которая может вызвать легкое смущение – это странный вид имени импортируемой функции. Это – наглядный пример искажения ("замангления") имен.

Искажение имен (mangling)

Также иногда встречается термин "украшение". Искажение имен применяется компилятором для разрешения своих внутренних конфликтов. Часто сложно отличить имена функций друг от друга. Проблема становится особенно актуальной в ООП-языков программирования, где становится возможной перегрузка функций (когда функции имеют одинаковое название, но выполняют более-менее разные действия) и даже операторов. Вот компилятор и изворачивается. Каждый компилятор (даже одного и того же языка) имеет собственную схему искажения имен. Схема искажения имен компилятора Borland сравнительно неплохо описана П.В. Румянцевым в своей "Работа с файлами в Win32". Схема искажения имен компилятора от MS, насколько я знаю, нигде особо не описывалась (хотя, может, стоит статьи Питрека поглядеть?) Однако есть утилита undname.exe, входящая в состав VisualStudio. Вся эта утилита, позволяющая, как и IDA, "размангля́ть" символьные имена, является переходником к функции UnDecorateSymbolName из imagehlp.dll. Функция вполне документирована, однако, представляет собой всего лишь переходник к уже недокументированной функции __unDName из msvcrt.dll. Советую изучить код функции, если вас интересует проблема искажения имен.

Теперь можно поговорить о взломе продуктов компании Rational Software (www.rational.com). Это ближайшие конкуренты фирмы NuMega, однако продукты их подходят только для программиста, а не для хакера. Но ведь и мы с вами не только тем занимаемся, что программы ломаем?

Основной пакет – PurifyPlus. Состоит из Purify/Quantify/PureCoverage. Все они тесно интегрируются с VS.NET. Purify – средство выявления утечек памяти и других мелких гадостей, Quantify – весьма неплохой профилировщик, PureCoverage – средство покрытия кода. Всем этим программам есть аналоги от NuMega (TrueTime, TrueCoverage и т.п.), однако в последнее время нужный инструмент от NuMegane всегда легко найти. С сайта скачать невозможно ничего (в отличие от rational.com, пока!), в сети выловить тоже тяжеловато.

PurifyPlus активно использует flexlm (на момент написания статьи – версия 7.0f), о чем сам и говорит. Взломать эту защиту не составляет никакой сложности. Нетрудно установить, что

ругательство ведется из rscommonui.dll. Вообще забавно наблюдать, как все FlexAPI-функции засовываются в одну-единственную на всю программу несчастную функцию, да еще и возвращающую результат типа bool. А потом еще говорят, что flex криво написан. Он-то, может, и криво написан, но используют его еще кривее! Что ж стоит подделать ответ функции, поменять нолик на единичку или наоборот? Да ничего! И никаких больше проверок! А ведь метод, который я предлагаю здесь, неверен в принципе! Здесь мы боремся со следствием, а причина (.lic-файл) остается нетронутой. А борьба со следствием чревата тем, что всегда есть риск проворонить где-нибудь какой-нибудь подлый call и получить уже полные старания фантазии разработчика, вплоть до самоуничтожения программы. Однако сейчас код код пишется настолько неаккуратно, что многие защиты можно взломать, не прилагая особых умственных усилий. В самом деле, все чаще встречаются программы, использующие для защиты либо примитивнейшие самопальные алгоритмы, возвращающие 1 или 0 в результате, либо использующие пакеры/криптовщики, но это уже совсем другая песня…

Патч:

```
RationalPurify.2002.05.20.468.000.exe

purifyw.exe - 3,240,020 bytes

_text:00407F33 loc_0_407F33:                           ; CODE XREF: sub_0_407F20+Aj
_text:00407F33                 call    sub_0_4E2060
_text:00407F38                 call    sub_0_5DBD5F
_text:00407F3D                 test    eax, eax
_text:00407F3F                 jz      short loc_0_407F68
_text:00407F41                 call    sub_0_4E26C0
_text:00407F46                 test    eax, eax
_text:00407F48                 jz      short loc_0_407F55
_text:00407F4A                 call    sub_0_4E20F0    ; funny proc - mov eax,1
_text:00407F4F                 test    eax, eax
_text:00407F51                 jnz     short loc_0_407F68
_text:00407F53                 pop     esi
_text:00407F54                 retn


RationalQuantify.2002.05.20.468.000.exe

quantifyw.exe - 2,621,528 bytes

_text:00428C23 loc_0_428C23:                           ; CODE XREF: sub_0_428C10+Aj
_text:00428C23                 call    sub_0_491130
_text:00428C28                 call    sub_0_56E8EF
_text:00428C2D                 test    eax, eax
_text:00428C2F                 jz      short loc_0_428C58
_text:00428C31                 call    sub_0_4917D0
_text:00428C36                 test    eax, eax
_text:00428C38                 jz      short loc_0_428C45
_text:00428C3A                 call    sub_0_4911C0 ; funny proc - mov eax,1
_text:00428C3F                 test    eax, eax
_text:00428C41                 jnz     short loc_0_428C58
_text:00428C43                 pop     esi
_text:00428C44                 retn


RationalPureCoverage.2002.05.20.468.000.exe

coveragew.exe - 2,527,320 bytes

_text:00405773 loc_0_405773:                           ; CODE XREF: sub_0_405760+Aj
_text:00405773                 call    sub_0_489800
_text:00405778                 call    sub_0_559E7F
_text:0040577D                 test    eax, eax
_text:0040577F                 jz      short loc_0_4057A8
_text:00405781                 call    sub_0_489EA0
_text:00405786                 test    eax, eax
_text:00405788                 jz      short loc_0_405795
```

```
_text:0040578A                call    sub_0_489890 ; funny proc - mov eax,1
_text:0040578F                test    eax, eax
_text:00405791                jnz     short loc_0_4057A8
_text:00405793                pop     esi
_text:00405794                retn
```

Вот, собственно, и все о защите FlexLm. То есть, о защите писать еще можно долго и нудно, однако о крайне бездарном ее применении в приложениях, я думаю, вы уже имеете весьма неплохое представление. За сим разрешите откланяться.

Volodya
wasm team.

P.S. Автор выражает искреннюю благодарность всем членам wasm team за пинки по делу и вылавливание мелких и крупных блох. Персональное спасибо alexela и cybermaniac.

Примеры реальных взломов: Вступление

Автор: Крис Касперски

Дата: 2003-04-02

(Фрагмент книги "Техника и философия хакерских атак 2.000")

В настоящей книге все атаки рассматривались исключительно на примерах, специально написанных для демонстрации того или иного алгоритма программ 'CrackMe'. При этом многие из них были слишком искусственными и далекими от реальных защитных механизмов. Это было удобно для изложения материала, но не отражало реальных существующих защит.

Поэтому я решил включить в приложения некоторые примеры реальных взломов. Все эти программы широко распространены и отражают средний уровень защиты условно-бесплатных программ. Заметим, что он существенно ниже, чем многие из предложенных в книге реализаций.

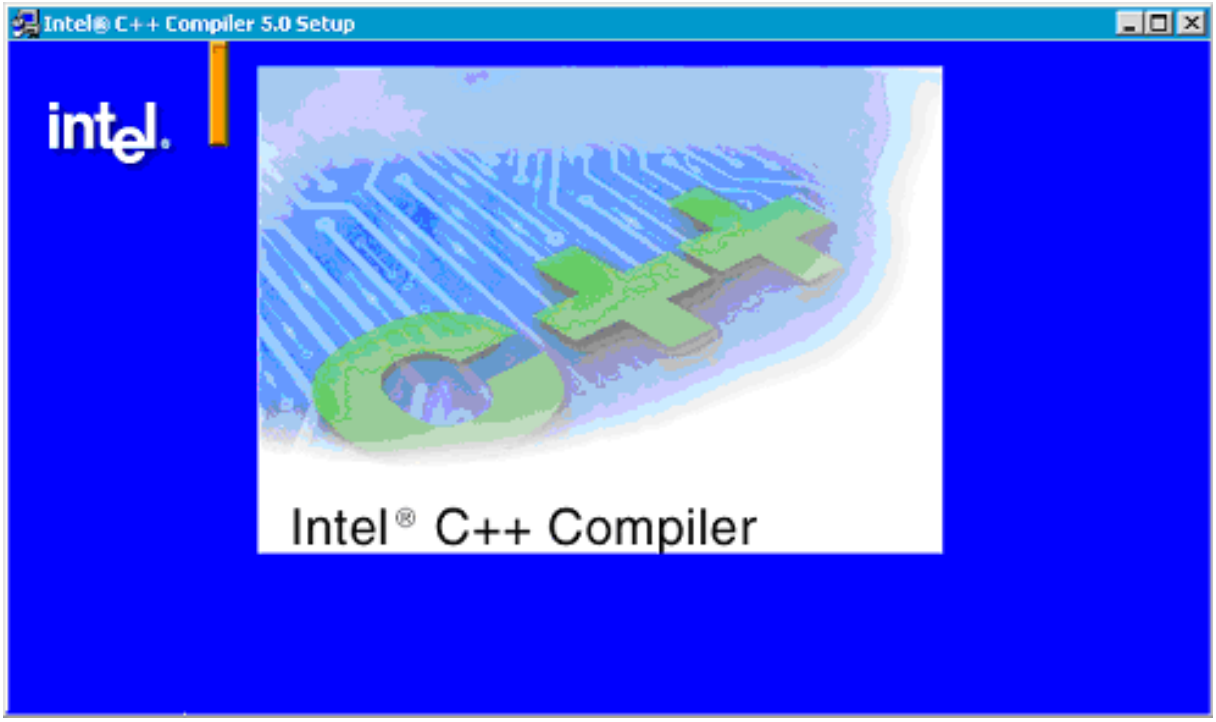
Напоминаю, что взлом в той или иной мере конфликтует с российским и международным законодательством. Поэтому необходимо помнить, что взлом не освобождает от регистрации и может быть использован только в образовательных целях, но никак не для получения какой-либо выгоды. В этом случае конфликтов с законодательсвом не возникнет.

- Компилятор Intel C++ 5.0.1
- Intel Fortran 4.5
- Intel C++ 7.0 Compiler
- Record Now
- UniLink v1.03 от Юрия Харона

Примеры реальных взломов: компилятор Intel C++ 5.0.1

Автор: Крис Касперски

Дата: 2003-04-02



Прежде, чем приступать к обсуждению аспектов стойкости защиты компилятора Intel C++ 5.0.1, считаю своим долгом заявить, что я глубоко восхищен этим великолепным программным продуктом и ломать его на мой взгляд по меньшей мере кощунственно. Впрочем, сегодня только ленивый не найдет в Сети кряк (один только Google по запросу "Intel C++ crack" выдает свыше 12 тысячи ссылок!), так что никакого вреда от данной публикации не будет.

Немного грустных новостей для начала. Приобрести легальную версию данного компилятора для жителей России оказывается чрезвычайно затруднительно. И вопрос упирается даже не в то "сколько он стоит" (а стоит он, если мне не изменяет память что-то в районе тысячи долларов), - компания Intel просто игнорирует данный сегмент рынка. Обращения в российское представительство компании с просьбой предоставить (за деньги!) данный компилятор для его же описания (читай - рекламы и продвижения) в книге "Техника оптимизации программ" положительных результатов не дали, - даже после того, как к этому вопросу подключились прямо-таки скажем не мелкие отечественные издательства BHV и Солон - Р. Ладно, не хотят продавать - ну и не надо! Благо хоть с сервера компании можно свободно утянуть 30-дневный триал. Негусто конечно, но для сравнительного тестирования - вполне достаточно (а для других целей мне этот компилятор и не нужен!).

Впрочем, все оказалось не так просто! С web-сервера компилятор за просто так не отдался, - после заполнения регистрационной формы меня вежливо поблагодарили и сообщили, что сейчас ко мне на мыло упадет письмо с триальной лицензией и инструкцией по ее установке. Это "сейчас" заняло у севера аж несколько дней (такое впечатление, что анкеты просматриваются вручную). ОК! Лицензия получена! Начинаем скачивать файл…. Как это так докачка не поддерживается?! А вот не поддерживается и все! Учитывая, что у меня лишь хлипкий Dial-Up по каналу в 19.200 (да и тот по межгороду) скачать полста мегабайт без единого разрыва просто нереально. К тому же, работа над

книгой уже близиться к завершению и вносить в нее еще один компилятор (а, значит, - переписывать кучу текста заново) мне становится просто в лом. Да и Intel C++ это далеко не самый популярный в кругах российских программистов компилятор и книга без него как ни будь уж переживет (хотя, посмотреть как Intel оптимизирует код под свои процессоры очень хотелось, да и документация по компилятору вдохновляла) .

Разозлившись на весь свет (и на парней из Intel в частности) я отправился на ftp-сервер компании, откуда наскоро, всего за каких-то три дня, слил полнофункциональную (хотя и шибко несвежую) версию компилятора, находящуюся по следующему адресу: ftp://download.intel.com/software/products/downloads/C5.0.1-15.exe. (приятно, что ftp докачку исправно поддерживал и многократные разрывы никаких проблем не вызывали). Польстившись на размер, я скачал именно пятую версию компилятора, которая была в полтора раза легче шестой (под которую у меня имелась неиспользованная триальная лицензия) и аж в два раза компактнее седьмой - новейшей на момент написания этих строк - версии, ломать которую из "политических" соображений я все равно бы не рискнул, так зачем же ее зря качать?

Теперь, собственно, мы и подходим к известному философскому вопросу: этично ли ломать программный продукт уважаемой тобой компании или без этого можно обойтись? Да если бы без этого было возможно обойтись, я бы - честное слово - без тени сожаления выложил за этот замечательный продукт пачку вечнозеленых, но, увы… компания не проявляет ко мне как покупателю никакого интереса и, кроме как ломать, ничего другого просто не остается!

Итак, инсталлируем Intel C++ и, предварительно скопировав в просроченную лицензию от шестой версии в папку \Intel\Licenses, запускаем головной файл программы:

```
...\Program Files\Intel\C501\Compiler50\ia32\bin>icl.exe
Intel(R) C++ Compiler for 32-bit applications, Version 5.0.1   Build 010525Z
Copyright (C) 1985-2001 Intel Corporation.  All rights reserved.

icl: error: could not checkout FLEXlm license
checkout failed: No such feature exists (-5,357)
```

Как и следовало ожидать "could not checkout FLEX lm license" ("не могу проверить FLEX lm лицензию") - компилятор ругается и прекращает свою работу. Ага, стало быть, программа защищена FLEX'ом, - достаточно известным в хакерских кругах менеджером лицензий от компании Globetrotter Inc, представляющим собой достаточно продвинутую защиту интегрированного типа. Разработчик защищаемого приложения получает в свое распоряжение SDK, содержащее как тривиальные функции проверки валидности ключевого файла (лицензии), так и развитые средства динамической шифровки файла. При грамотном подходе к защите запустить защищенную программу без наличия соответствующей ей лицензии доподлинно невозможно. Если часть программы зашифрована, - пытаться расшифровать ее без ключа - дохлое дело. Правда не факт, что парни из Intel действительно использовали шифрование, к тому же, зашифрованные фрагменты иногда удается восстановить по косвенным данным. Это смотря, что еще зашифровано!

Разумеется, при наличии триальной лицензии шифровка снимается без труда, но в том-то все и дело, что триальной лицензии у меня не было! Тем не менее надежда меня не покидала и, перекусив для смелости батоном докторской колбасы, сдобренной значительным количеством кетчупа, я запустил свой любимый дизассемблер IDA, и… не знаю у кого как, а у меня вид консольной IDA, распахнутой на весь экран, всегда вызывает чувство благоговения. ОК, ну-ка посмотрим где скрываются те текстовые строки, которые выводятся при отсутствии лицензии на экран. Результат: ни "No such feature exists", ни "could not checkout" в ASCII-строках (т. е. тех строках, что сумел распознать автоматический анализатор IDA) не найдено. Хорошо, зайдем с другого конца: нажимам \<F4> для переключения в hex-режим и давим \<ALT-T> для поиска текстовых строк в "сыром" виде. Что ж, на этот раз поиск "could not checkout" увенчался успехом!

```
.data1:0042D9C0  63 6F 75 6C 64 20 6E 6F-74 20 63 68 65 63 6B 6F "could not checko"
```

```
.data1:0042D9D0  75 74 20 46 4C 45 58 6C-6D 20 6C 69 63 65 6E 73 "ut FLEXlm licens"
.data1:0042D9E0  65 00 00 00 63 6F 75 6C-64 20 6E 6F 74 20 6C 6F "e...could not lo"
.data1:0042D9F0  63 61 74 65 20 46 4C 45-58 6C 6D 20 72 65 67 69 "cate FLEXlm regi"
.data1:0042DA00  73 74 72 79 20 6B 65 79-00 00 00 00 63 6F 75 6C "stry key....coul"
```

Нажимаем \<F4> еще один раз для возврата в режим дизассемблера, подводим курсор к адресу 42D9C0h и нажимаем A для преобразования цепочки байт в ASCII-строку. В результате мы получаем:

```
.data1:0042D9C0 aCouldNotChecko db 'could not checkout FLEXlm license',0
```

А как узнать: кто же выводит строку-ругательство на экран? Нет ничего проще! Вновь переключившись в режим дизассемблера по \<F4>, давим \<ALT-T> для поиска последовательности "C0 D9 40 00" - адрес строки, представленный в обратном (с учетом порядка следования старших байтов) виде. Опа! Мы видим код наподобие следующего:

```
.data:00420CE8                db 50h
.data:00420CE9                db 0DEh ; ¦
.data:00420CEA                db  42h ; B
.data:00420CEB                db    0 ;
.data:00420CEC                db    1 ;
.data:00420CED                db    0 ;
.data:00420CEE                db    0 ;
.data:00420CEF                db    0 ;
.data:00420CF0                db 2Ch
.data:00420CF1                db 0DEh ; ¦
.data:00420CF2                db  42h ; B
.data:00420CF3                db    0 ;
.data:00420CF4                db    2 ;
.data:00420CF5                db    0 ;
.data:00420CF6                db    0 ;
.data:00420CF7                db    0 ;
```

Косвенный вызов строки! Ну, собственного, этого и следовало ожидать (иначе с чего бы это автоматических анализатор IDA их не распознал?). Хорошо, преобразуем двойные слова в смещения, руководствуясь при этом тем, что число "42h" должно выпадать на младший байт старшего слова (иначе, адрес ссылки уйдет за диапазон предельно допустимых значений) и получаем:

```
.data:00420DE8 dd offset aCouldNotLoca_0 ; "could not locate FLEXlm registry direct"
.data:00420DEC dd 21h
.data:00420DF0 dd offset aCouldNotLocate ; "could not locate FLEXlm registry key"
.data:00420DF4 dd 22h
.data:00420DF8 dd offset aCouldNotChecko ; "could not checkout FLEXlm license"
.data:00420DFC dd 23h
```

Попробуем теперь найти ту су…, в общем тот код, что обращается к указателю на ругательную строку, расположенному по адресу 420CE8h? Не надо спешить! По виду полученной таблицы смещений можно с уверенностью заключить, что прямого обращения к ее элементам не будет. Можно предположить, что числа, стоящие возле ссылок на строки - это коды ошибок, а сами строки - соответствующие тексты сообщений. Если так, то с вероятностью близкой к единице, разработчиками программы использовалась относительная адресация, т. е. для вычисления эффективного адреса элемента ее смещение в таблицы суммируются с базовым адресом таблицы, - единственным адресом, который загружается явно.

Прокручивая экран дизассемблера вверх, мы внезапно натыкаемся на длинную последовательность нулей, интерпретируемую нами как начало таблицы:

```
.data:00420CDE              db    0 ;
.data:00420CDF              db    0 ;
.data:00420CE0 off_420CE0   dd offset unk_42DE80    ; DATA XREF: sub_403370+5E^r
.data:00420CE4 dword_420CE4 dd 0                    ; DATA XREF: sub_403370+19^r
.data:00420CE4                                      ; sub_403370+39^r
.data:00420CE8              dd offset aCouldNotFindDi ; "could not find directory"
.data:00420CEC              dd 1
```

Ага! Есть две перекрестных ссылки! Это хорошо! Теперь поднимемся по ним вверх, - прямиком к вызывающему их коду? Можно, конечно, поступить и так, но есть и более универсальное решение: запустив Soft-Ice, мы устанавливаем точку останова на чтение ячейки 420DE8h (если вы еще не забыли - это адрес элемента таблицы, ссылающийся на искомую ругательную строку). Теперь - кто бы к ней не обращался, Soft-Ice обязательно всплывет, и ведь действительно он всплывает! Пару раз отдав команду "P RET", поднимающую нас из дебрей глубоко вложенных процедур поближе к свету. Наконец, мы взбирается на вершину стека и очередной "P RET" приводит к завершению программы. ОК, повторяем все заново, делая на этот раз на один "P RET" меньше. Записываем любой из близлежащих адресов (пусть это будет для определенности адрес 4031C4h) и натравливаем на него IDA.

```
.text:004031C4      call    lc_checkout
.text:004031C9      test    eax, eax
.text:004031CB      jz      short loc_403215
.text:004031CD      cmp     eax, 0FFFFFFF6h
.text:004031D0      jz      loc_41B000
.text:004031D6      cmp     eax, 0FFFFFFB7h
.text:004031D9      jz      loc_41B01A
.text:004031DF
.text:004031DF loc_4031DF:                              ; CODE XREF: .text1:0041B015vj
.text:004031DF                                          ; .text1:0041B026vj
.text:004031DF      mov     [esp+240h+var_240], 23h
.text:004031E6      call    sub_405B00
.text:004031EB      mov     eax, dword_424C9C
.text:004031F0      mov   [esp+240h+var_240], eax
.text:004031F3      mov [esp+240h+var_23C], offset aCheckoutFailed ; "checkout failed"
.text:004031FB      call  lc_perror
.text:00403200      mov   eax, dword_424C9C
.text:00403205      mov   [esp+240h+var_240], eax
.text:00403208      call  lc_get_errno
.text:0040320D      mov   [esp+240h+var_240], eax
.text:00403210      call  sub_405BA0
.text:00403215
.text:00403215 loc_403215:                              ; CODE XREF: sub_403000+1CB^j
.text:00403215      mov   eax, dword_424C9C
.text:0040321A      mov   edx, dword_421E3C
.text:00403220      mov   [esp+240h+var_240], eax
.text:00403223      mov   [esp+240h+var_23C], edx
.text:00403227      call  lc_auth_data
.text:0040322C      mov   edx, eax
.text:0040322E      mov   eax, dword_424C9C
.text:00403233      call  sub_40A6F8
.text:00403238
```

Вот это да! - восклицаем мы, пришибленно уставившись на экран. Многое мы ожидали от IDA, но вот чтобы она так запросто представила символьные имена защитных функций, все говорящие за себя: lc_chekout, lc_perror, lc_auth_data… Черт, возьми, как? Вдохновленные смутной надеждой мы неуверенно подгоняем курсор к lc_chekout и нажимаем на \<ENTER>.

```
.idata:0041D12C ; Imports from LMGR327A.dll
.idata:0041D12C ;
.idata:0041D12C extrn __imp_lc_init:dword ; DATA XREF: lc_init^r
.idata:0041D130 extrn __imp_lc_expire_days:dword
.idata:0041D130        ; DATA XREF: lc_expire_days^r
.idata:0041D134 extrn __imp_lc_free_job:dword ; DATA XREF: lc_free_job^r
.idata:0041D138 extrn __imp_lc_checkin:dword ; DATA XREF: lc_checkin^r
.idata:0041D13C extrn __imp_lc_auth_data:dword ; DATA XREF: lc_auth_data^r
.idata:0041D140 extrn __imp_lc_get_errno:dword ; DATA XREF: lc_get_errno^r
.idata:0041D144 extrn __imp_lc_perror:dword ; DATA XREF: lc_perror^r
.idata:0041D148 extrn __imp_lc_checkout:dword ; DATA XREF: lc_checkout^r
.idata:0041D14C extrn __imp_lc_set_attr:dword ; DATA XREF: lc_set_attr^r
.idata:0041D150
```

Святой Кондратий! И это они еще называют защитой?! Все защитные функции вынесены в отдельную динамическую библиотеку (наверное, чтобы взломщику разбираться было легче?) -

LMGR327A.DLL, в названии которой угадывается "Library ManaGeR", причем, это штатные функции FLEX lm, описание которых можно найти в его же SDK (хоть SDK на FLEX ln с компилятором и не поставляется, найти его в Сети - плевое дело).

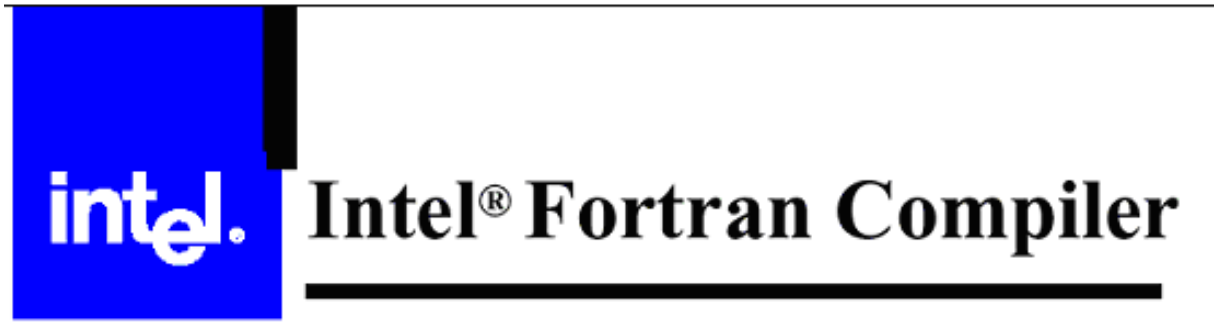
Отыскав в текущем каталоге этот самый LMGR327A.DLL, мы открываем его HIEW'ов на предмет полного переписывания функции lc_checkout. Ну, насчет "переписывания" автор, ясное дело, загнул. Всего-то и требуется, - заставить lc_checkout всегда возвращать нуль, для чего первые две команды ее тела должны выглядеть приблизительно так: "XOR EAX, EAX / RETN". Записываемся и с дрожью в сердце, запускам icl.exe на выполнение. Критическая ошибка приложения? А чего мы хотели?! Ведь теперь функция lc_auth_data получает неверные данные и гробит все к черту. Впрочем, не будет спешить. Беглое исследование процедуры sub_40A6F8 как будто не выявляет никаких следов шифрования и поэтому ее можно смело удалить, не забыв тоже самое "на всякий пожарный" случай проделать и с lc_auth_data (самое простое - впихнуть в ее начало RETN). Сохраняемся, запускам icl.exe и… компилятор работает! Все! Больше тут нечего ломать!

Самое забавное, что размер защитного механизма (413 Кб) в два с половиной раза превышает размер защищенной с его помощью программы (176 Кб)! Как говорится - no comment.

Примеры реальных взломов: Intel Fortran 4.5

Автор: Крис Касперски

Дата: 2003-04-03



Логотип Intel Fortran Compiler

Ситуация с этим компилятором вкратце такова. В процессе работы над третьим томом "Образа мышления IDA" я исследовал большое количество компиляторов на предмет особенностей их кодогенерации и вытекающих отсюда трудностей восстановления исходного кода. Не избежал этой участи и "Intel Fortran Compiler", обнаруженный на диске "Научись сам программировать на FORTRAN". Краткая аннотация на буклете гласила "Intel FORTRAN Compiler 4.5 - новейшая версия знаменитого компилятора. Для регистрации программы смотрите поддиректорию CRACK". Ну, на счет "новейшего" составили диска явно приврали, т .к. на тот момент уже вышла седьмая версия, да и CRACK оказался некорректным. Вместо того, чтобы ломать защиту, он ломал сам компилятор, необратимо его гробя. К счастью, оригинальный ifl.exe на диске все-таки имелся и это давало возможность заставить работать компилятор мне самому. В конце концов использовать в коммерческих целях этот, бесспорно, замечательный программный продукт я все равно не собирался, а для серии тестовых прогонов не то, что месяца (положенного мне по праву) даже нескольких дней было вполне предостаточно, поэтому с этической точки зрения ничего кощунственного я не совершал (просто мне очень уж не хотелось тянуть ~160 метров из Интернета, с моим междугородним Интернетом это действительно проблематично).

Итак, запускаем оригинальный файл компилятора на выполнение и лицезрим как он спускает на нас Полкана (ругается в смысле):

```
KPNC$C:\Program Files\Intel\compiler45\bin>ifl1.exe >1
Intel(R) Fortran Compiler Version 4.5 000403
Copyright (C) 1985-2000 Intel Corporation.  All rights reserved.
Evaluation Copy
ifl1: error: The evaluation period has expired.

    The evaluation period for this trial version of the
    Intel(R) Fortran Compiler has expired.  For product ordering
    information, please refer to the product release notes or visit the
    Intel Developer web site at the following URL:

               http://developer.intel.com/vtune
```

Ни слова о FLEX lm! (см. "Компилятор Intel C++ 5.0.1") и файл LMGxxx.DLL отсутствует. Странно! Похоже, что Fortran Compiler защищен иначе, что, собственно и не удивительно, поскольку их делали разные группы.

Что ж, запускаем IDA и натравливаем на нее исполняемый файл, который, кстати, занимает всего 176,128 Кб, что с точностью до байта соответствует размеру Intel C++ 5.1 Compiler. Странно! Но, как бы там ни было, ASCII-строки "The evaluation period has expired" автоматический анализатор IDA в тексте дизассемблируемого файла так и не нашел. Что ж, тогда мы сделаем это сами. <F4>,

<ALT-T>, "The evaluation period" и…

```
.data1:0042A220  54 68 65 20 65 76 61 6C-75 61 74 69 6F 6E 20 70 "The evaluation p"
.data1:0042A230  65 72 69 6F 64 20 68 61-73 20 65 78 70 69 72 65 "eriod has expire"
.data1:0042A240  64 2E 0A 0A 20 20 20 20-54 68 65 20 65 76 61 6C "d.00    The eval"
.data1:0042A250  75 61 74 69 6F 6E 20 70-65 72 69 6F 64 20 66 6F "uation period fo"
.data1:0042A260  72 20 74 68 69 73 20 74-72 69 61 6C 20 76 65 72 "r this trial ver"
.data1:0042A270  73 69 6F 6E 20 6F 66 20-74 68 65 0A 20 20 20 20 "sion of the0    "
```

Теперь, вновь нажимаем <ALT-T> для поиска последовательности "20 A2 42 00" - адрес начала строки, заданный в обратном виде. Результат не заставляет себя долго ждать:

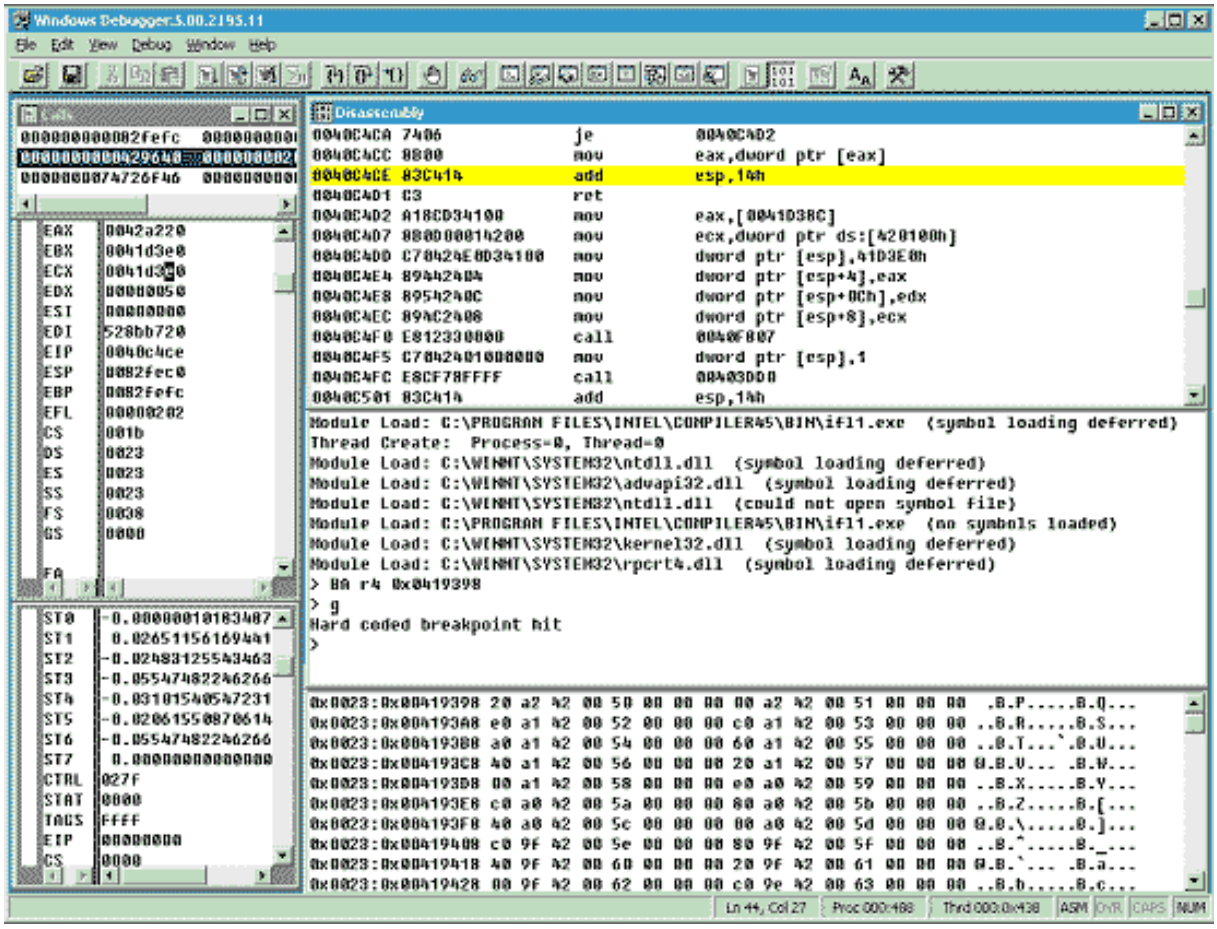
```
.data:00419390  60 A3 42 00 4F 00 00 00-20 A2 42 00 50 00 00 00 "`rB.O... вB.P..."
.data:004193A0  00 A2 42 00 51 00 00 00-E0 A1 42 00 52 00 00 00 ".вB.Q...рбB.R..."
.data:004193B0  C0 A1 42 00 53 00 00 00-A0 A1 42 00 54 00 00 00 "LбB.S...абB.T..."
.data:004193C0  60 A1 42 00 55 00 00 00-40 A1 42 00 56 00 00 00 "`бB.U...@бB.V..."
.data:004193D0  20 A1 42 00 57 00 00 00-00 A1 42 00 58 00 00 00 " бB.W....бB.X..."
```

Переключаемся обратно в дизассемблер, трижды жмем <D> для преобразования цепочки байт в двойное слово, затем <O> для перевода его в смещение и… в результате таких манипуляций получаем приблизительно такую же таблицу, как и в нашем предыдущем случае с Intel C++

```
.data:00419390 dd offset aSNoteTheEvalua ; "%s: NOTE: The evaluation period for thi"
.data:00419394 dd 4Fh
.data:00419398 dd offset aTheEvaluationP ; "The evaluation period has expired.\n\n   "
.data:0041939C dd 50h
.data:004193A0 dd offset aCommandLineErr ; "Command line error"
.data:004193A4 dd 51h
.data:004193A8 dd offset aCommandLineWar ; "Command line warning"
.data:004193AC dd 52h
```

А посему и действовать мы будем точно так же: поставим бряк на адрес 0419390h и дождемся пока отладчик не получит управления. Кстати, на счет отладчика. В момент написания этих строк у автора как раз закачивалась седьмая версия компилятора Intel C++ и от использования soft-ice пришлось воздержаться (в момент своей активации soft-ice полностью "замораживает" операционную систему, что пагубно влияет на Интернет, а точнее на установленные TCP/IP соединения). И вместо soft-ice автор решил для разнообразия использовать Microsoft WDB, который кстати, справился со своей задачей ничуть не хуже.

Запускам WDB на выполнение, нажимаем <Ctrl?E>, указываем имя загружаемого файла, переходим в окно команд ("Command Window") и устанавливаем точку остановка на адрес 0419398h для чего отдаем команду "BA r4 0x0419398" (что расшифровывается как: "Break on Access of Read 4 bytes long"). Затем для продолжения выполнения программы пишем "G" и с полсекунды ждем…



Внешний вид отладчика MS WBD в процессе ломания программы

Ага, отладчик говорит "Hard coded breakpoint hit" ("сработала аппаратная точка останова") и приостанавливает выполнение отлаживаемой программы. Сама же отлаживаемая программа к этому моменту уже успела вывести на экран:

```
Intel(R) Fortran Compiler Version 4.5 000403
Copyright (C) 1985-2000 Intel Corporation.  All rights reserved.
Evaluation Copy
if11: error:
```

Обратите внимание на строки, выделенные жирным шрифтом! Очевидно, они свидетельствуют о том, что мы попали не в самое начало защитной процедуры, а где-то в ее середину. Кстати, а что у нас там лежит на стеке? Смотрим (~Viewa Stack, см. рис. 0x003). Всего три адреса, - довольно неглубокий уровень вложения не так ли? Причем (обратив свой взор к окну дизассемблера), сейчас уровень вложения еще понизится, т. к. следующей командой мы выходим из этой процедуры:

```
0040C4CC 8B00            mov         eax,dword ptr [eax]
0040C4CE 83C414          add         esp,14                  _
0040C4D1 C3              ret
```

Теперь неспешно трассируем код, попеременно поглядывая то на дизассемблированный листинг, то на консоль отлаживаемой программы. Следующая трассируемая функция (внутрь которой мы не заходим, а "заглатываем" ее одним нажатием <F10>) выводит на экран "The evolution period has expired", но не завершает программу, а продолжает ее выполнение. Что ж! Тогда и мы продолжим (трассировку)! Вызов функции 040F5FEh проходит без каких либо внешних проявлений и, так и не поняв, зачем она собственно нужна, мы поднимается на еще один уровень вверх, куда нас забрасывает завершающий функцию RET.

```
00403C7C E833880000       call        0040C4B4
; отсюда ^^^^^^^^^^^^^^^^^^^^ мы только что вышли
```

```
00403C81 89442404         mov         dword ptr [esp+4],eax
00403C85 891C24           mov         dword ptr [esp],ebx
00403C88 896C2408         mov         dword ptr [esp+8],ebp
00403C8C E8A8BB0000       call        0040F839
; эта процедура выводит "The evaluation period has expired."

00403C91 C70424C04C4200   mov         dword ptr [esp],424CC0h
00403C98 895C2404         mov         dword ptr [esp+4],ebx
00403C9C E85DB90000       call        0040F5FE
; эта процедура ничего не делает

00403CA1 83C414           add         esp,14h
00403CA4 5B               pop         ebx
00403CA5 5D               pop         ebp
00403CA6 C3               ret
…и таким Макаром мы трассируем код до тех пор, пока не наткнемся на следующую конструкцию:

0040105A E8E1800000       call        00409140 ; отсюда мы только что вышли по RETN
0040105F 0FB6C0           movzx       eax,al
00401062 85C0             test        eax,eax
00401064 0F84C4000000     je          0040112E
```

что в ней необычного? А то, что это первая встретившаяся нам материнская процедура, которая анализирует код возврата дочерней функции. В нашем случае регистр EAX содержит значение "ноль" и, стало быть, следующий условный переход выполняется. Но не тот ли это переход который нам нужен? Что ж, сейчас мы это узнаем - нажимаем клавишу <F10> еще несколько раз… Опля! Наш условный переход перебрасывает нас на ту ветку программы, которая спустя несколько команд скоропостижно сдыхает, захлопывая окно программы. А что произойдет, если команду "JE" в строке 401064h заменить на противоположную (или, как вариант, просто удалить этот условный переход)? Пробуем…

Компилятор по прежнему смачно ругается на "evaluation expired", но… он работает! Работает!! Работает!!! По соображением экономии экранного места (в самом деле, ругательство занимает чуть ли не половину экрана и смотрится крайне некрасиво) мы забиваем вызов процедуры 0409140h командами NOP. Проверяем - сработало ли? Ну… это смотря как посмотреть. Трехэтажный мат действительно исчез, но вот лаконичная строка "Evaluation Copy" так и осталась. Найдем что за код ее выводит? Зачем? - лучше найти саму эту строку и тем же HIEW'ом ее переписать во что ни будь более привычное, например: "hacked by mother-fucker guy". Переписываем, и… пользуемся компилятором в свое удовольствие, не забывая, однако о том, что по истечении 30-дневного срока вы будете должны его стереть, в противном случае вы поступите очень и очень нехорошо, да и незаконно.

Примеры реальных взломов: Intel C++ 7.0 Compiler

Автор: Крис Касперски

Дата: 2003-04-04

…компилятор Intel C++ 7.0 докачался глубокой ночью, часу где-то в пятом утра. Спать хотелось неимоверно, но и любопытство: была ли усилена защита или нет, тоже раздирало. Решив, что до тех пор пока не разберусь с защитой я все равно не усну, я, открыв новую консоль, и переустановив системные переменные TEMP и TMP на каталог C:\TEMP, наскоро набил неприлично длинное имя инсталлятора W_CC_P_7.0.073.exe в командной строке (необходимость в установке переменных TEMP и TMP объясняется тем, что в Windows 2000 они по умолчанию указывают на очень глубоко вложенный каталог, а инсталлятор Intel C++ - да и не только он - не поддерживает путей такого огромного размера).

Сразу же выяснилось, что политика защиты была кардинально пересмотрена и теперь наличие лицензии проверялось уже на стадии установки программы (в версии 5.x установка осуществлялось без проблем). ОК, даем команду dir и смотрим на содержимое того, с чем нам сейчас предстоит воевать:

```
>dir
Содержимое папки C:\TMP\IntelC++Compiler70
17.03.2003  05:10       <DIR>          html
17.03.2003  05:11       <DIR>          x86
17.03.2003  05:11       <DIR>          Itanium
17.03.2003  05:11       <DIR>          notes
05.06.2002  10:35                45 056 AutoRun.exe
10.07.2001  12:56                    27 autorun.inf
29.10.2002  11:25                 2 831 ccompindex.htm
24.10.2002  08:12               126 976 ChkLic.dll
18.10.2002  22:37               552 960 chklic.exe
17.10.2002  16:29                28 663 CLicense.rtf
17.10.2002  16:35                   386 credist.txt
16.10.2002  17:02                34 136 Crelnotes.htm
19.03.2002  14:28                 4 635 PLSuite.htm
21.02.2002  12:39                 2 478 register.htm
02.10.2002  14:51                40 960 Setup.exe
02.10.2002  10:40                   151 Setup.ini
10.07.2001  12:56                   184 setup.mwg
              19 файлов      2 519 238 байт
               6 папок     886 571 008 байт свободно
```

Ага! Программа установки setup.exe занимает всего сорок с хвостиком килобайт. Очень хорошо! В такой объем серьезную защиту навряд ли спрячешь, а если даже так - этот крохотный файл ничего не стоит проанализировать целиком - до последнего байта дизассемблерного листинга. Впрочем, не факт, что защитный код расположен именно в setup.exe, он может находится и в другой месте, вот например… ChkLic.dll/ChkLic.exe, занимающими в совокупности немногим менее семисот килобайт. Постой, какой такой ChkLic? Это сокращение от Check License что ли?! Гм, у ребят из Intel очевидно серьезные проблемы с чувством юмора. Уж лучше бы они назвали этот файл "Hack Me" честное слово! Ладно, судя по объему, ChkLic это тот самый FLEX lm и есть, а с ним мы уже сталкивались (см. "Intel C++ 5.0 Compiler") и приблизительно представляем как его ломать.

Даем команду "dumpbin /EXPORTS ChkLic.dll" для исследования экспортируемых функций и… крепко держимся за Клаву, чтобы не упасть со стула:

```
Dump of file ChkLic.dll

File Type: DLL

  Section contains the following exports for ChkLic.dll

           0 characteristics
```

```
3DB438B4 time date stamp Mon Oct 21 21:26:12 2002
    0.00 version
       1 ordinal base
       1 number of functions
       1 number of names

ordinal hint RVA      name

      1    0 000010A0 _CheckValidLicense
```

Черт побери! Защита экспортирует всего одну-единственную функцию с замечательным именем CheckValidLicense. "Замечательным" - потому, что назначение функции становится понятным ее названия и появляется возможность избежать кропотливого анализа дизассемблерного кода. Ну вот, отбили весь интерес… уж лучше бы они ее по ординалу экспортировали что ли, или, по крайней мере, окрестили ее каким ни будь отпугивающим именем типа DES Decrypt.

…размечтались! Ладно, вернемся к нашим баранам. Давайте рассуждать логически: если весь защитный код сосредоточен непосредственно в ChkLic.dll (а, судя по "навесному" характеру защиты, это действительно так), то вся "защита" сводится к вызову CheckValidLicense из Setup.exe и проверке возвращенного ею результата. Поэтому для "взлома" достаточно лишь пропадчить ChkLic.dll, заставляя функцию ChekValidLicense всегда возвращать… да, кстати, что она должна возвращать? Точнее: какое именно возвращаемое значение соответствует успешной проверки лицензии? Нет, не торопитесь дизассемблировать setup.exe для определения, ведь возможных вариантов не так уже и много: либо FALSE, либо TRUE. Вы делаете ставку на TRUE? Что ж, в каком-то смысле это логично, но с другой стороны: а почему мы, собственно, решили, что функция CheckValidLicense возвращает именно флаг успешности операции, а не код ошибки? Ведь должна же она как-то мотивировать причины отказа устанавливать компилятор: файл с лицензией не найден, файл поврежден, лицензия просрочена и так далее? Хорошо, попробуем возвратить ноль, а если это не прокатит, возвратим единицу.

ОК, пристегивайтесь, поехали! Запускаем HIEW, открываем файл ChkLic.dll (если же он не открывается - трижды помянув сусликов, временно скопируем его в корневую или любую другую директорию, не содержащую в своем имени спецсимволов, которые так не нравятся hiew'y). Затем, обратившись еще раз к таблице экспорта, полученной с помощью dumpbin, определяем адрес функции CheckValidLicense (в данном случае 010A0h) и через <F5>, "10A0" переходим в ее начало. Теперь, - режем по "живому", перезаписывая поверх старого кода "XOR EAX, EAX/RETN 4". Почему именно "REN 4", а не просто "RET"? Да потому, что функция поддерживает соглашение stdcall, о чем можно узнать взглянув в HIEW'e на ее эпилог (просто пролистывайте экран дизассемблера вниз до тех пор, пока не встретите RET).

Проверяем… Это работает!!! Несмотря на отсутствие лицензии, инсталлятор, не задавая лишних вопросов, начинает установку! Стало быть, защита пала. Ой, не верится нам, что все так просто и, чтобы не сидеть, тупо уставившись в монитор в ожидании завершения процесса инсталляции программы, мы натравливаем на setup.exe свой любимый дизассемблер IDA. Первое, что бросается в глаза, отсутствие CheckValidLicense в списке импортируемых функций. Может быть, она файл ChkLic.exe как-то запускает? Пробуем найти соответствующую ссылку среди автоматически распознанных строк: "~View aNames", "ChkLic"… ага, строки "Chklic.exe" здесь вообще нет, но зато обнаруживается "Chklic.dll". Ага, понятно, значит, библиотека ChkLic загружается явной компоновкой через LoadLibrary. И переход по перекрестной ссылке подтверждает это:

```
.text:0040175D                push    offset aChklic_dll ; lpLibFileName
.text:00401762                call    ds:LoadLibraryA
.text:00401762 ; загружаем ChkLic.dll ^^^^^^^^^^^^^^^^
.text:00401762 ;
.text:00401768                mov     esi, eax
.text:0040176A                push    offset a_checkvalidlic ; lpProcName
.text:0040176F                push    esi             ; hModule
```

```
.text:00401770                 call    ds:GetProcAddress
.text:00401770 ; получаем адрес функции CheckValidLicense
.text:00401770 ;
.text:00401776                 cmp     esi, ebx
.text:00401778                 jz      loc_40192E
.text:00401778 ; если такой библиотеки нет, то выходим из программы установки
.text:00401778 ;
.text:0040177E                 cmp     eax, ebx
.text:00401780                 jz      loc_40192E
.text:00401780 ; если такой функции в библиотеке нет, то выходим из установки
.text:00401780 ;
.text:00401786                 push    ebx
.text:00401787                 call    eax
.text:00401787 ; вызываем функцию ChekValidLicense
.text:00401787 ;
.text:00401789                 test    eax, eax
.text:0040178B                 jnz     loc_4019A3
.text:0040178 ; если функция возвратила не ноль, то выходим из программы установки
```

Невероятно, но эта до ужаса примитивная защита построена именно так! Причем, полуметровый файл ChkLic.exe вообще не нужен! И чего ради стоило тащить его из Интернета? Кстати, если вы надумаете сохранять дистрибьютив компилятора (внимание: я не говорил "распространять"!), то для экономии дискового места ChkLic.* можно стереть: либо пропадчив setup.exe, навсегда отучив его к ним обращаться, либо же просто создав свою собственную ChkLic.dll, экспортирующую stdcall функцию CheckValidLicence вида: int CheckValidLicence(int some_flag) { return 0;}

Так-с, пока мы все это обсуждали, инсталлятор закончил установку компилятора и благополучно завершил свою работу. Интересно ли запустится ли компилятор или все самое интересное только начинается? Лихорадочно спускаемся вниз по разветвленной иерархии вложенных папок, находим icl.exe, который как и следовало ожидать, находится в каталоге bin, нажимаем <ENTER> и… Компилятор естественно не запускается, ссылаясь на то, что "icl: error: could not checkout FLEX lm license", без которой он не может продолжить свою работу.

Выходит, что Intel применила многоуровневую защиту и первый уровень оказался грубой защитой от дураков. Что ж! Мы принимаем этот вызов и, опираясь на свой предыдущий опыт, машинально ищем файл LMGR*.DLL в каталоге компилятора. Бесполезно! На этот раз такого файла здесь не оказывается, зато выясняется, что icl.exe сильно прибавил в весе, перевалив за отметку шестиста килобайт… Стоп! А не прилинковали ли разработчики компилятора этот самый FLEX lm статической компоновкой? Смотрим: в Intel C++ 5.0 сумма размеров lmgr327.dll и icl.exe составляла 598 Кб, а сейчас одни лишь icl.exe занимает 684 Кб. С учетом поправки на естественное старческое "ожирение", цифры очень хорошо сходятся. Значит, все-таки FLEX lm! Ой-ой! А ведь теперь, - без символических имен функций, ломать защиту будет намного труднее… Впрочем, не будем раньше времени паниковать! Давайте думать, только спокойно! Навряд ли команда разработчиков полностью переписала весь код, взаимодействующей с этой "конвертной" защитой. Скорее всего, ее "усовершенствование" одной лишь сменой типа компоновки и закончилось. А раз так, то шансы взломать программу по прежнему велики!

Памятуя о том, что в прошлый раз защитный код находится в функции main, мы, определив ее адрес, просто устанавливаем точку останова и, дождавшись всплытия отладчика, тупо трассируем код, попеременно поглядывая то на отладчик, то на окно вывода программы: не появилась ли там ругательное сообщение? При этом, все встретившиеся нам условные переходы, мы отмечаем на отдельном листке бумаги (или откладываем в своей собственной памяти, если вы так хотите), не забыв указать выполнялся ли каждый условный переход или нет… Стоп! Что-то заболтались мы с вами, а ведь ругательное сообщение уже выскочило! ОК, хорошо! Посмотрим, какой условный переход ему соответствовал. Наши записи показывают, что последним встретившимся переходом, был условный переход JNZ, расположенный по адресу 0401075h и "реагирующий" на результат, возращенной процедурой sub_404C0E:

```
.text:0040106E                 call    sub_404C0E
```

```
.text:00401073                 test    eax, eax
.text:00401075                 jnz     short loc_40107F
.text:00401077                 mov     al, 1
.text:00401079                 mov     byte ptr [esp+40h+var_18], al
.text:0040107D                 jmp     short loc_4010BA
.text:0040107F ; ---------------------------------------------------------------------------
.text:0040107F
.text:0040107F loc_40107F:                             ; CODE XREF: _main+75^j
.text:0040107F                 mov     eax, offset aFfrps ; "FFrps"
.text:00401084                 mov     edx, 21h
.text:00401089                 call    sub_404C0E
.text:0040108E                 test    eax, eax
.text:00401090                 jnz     short loc_40109A
```

Очевидно, что sub_404C0E и есть та самая защитная процедура, которая осуществляет проверку лицензии на ее наличие. Как ее обхитрить? Ну, тут много вариантов… Во-первых, можно, вдумчиво и скрупулезно проанализировать содержимое sub_404C0E на предмет выяснения: что именно и как именно она проверяет. Во-вторых, можно просто заменить JNZ short loc_40107F на JZ short loc_40107F или даже NOP, NOP. В-третьих, команду проверки результата возврата TEST EAX, EAX можно превратить в команду установки нуля: XOR EAX, EAX. В-четвертых, можно пропадчить саму sub_404C0E, чтобы она всегда возвращала ноль. Не знаю, как вы, но мне больше всех приглянулся способ номер три. Меняем два байта и запускаем компилятор. Если никаких других проверок его "лицензионности" в защите нет, то программа заработает и, соответственно, наоборот. (Как мы помним, в пятой версии таких проверок было две). Поразительно, но компилятор больше не ругается и работает!!! Действительно, как и следовало ожидать, его разработчики ничуть не усилили защиту, а, напротив, даже ослабили ее!

Примеры реальных взломов: Record Now

Автор: Крис Касперски

Дата: 2003-04-07

…и угораздило же меня приобрести "писец" (то бишь CD-RW) в OEM-поставке! И ведь спрашивал продавца: а где, позвольте, тут пишущий софт или, по крайней мере, драйвера? На что продавец, удивленно так пожимая плечами, ответил: какие драйвера? Втыкаете - работает. А пишущие программы подходят любые, вот купите в соседнем магазине диск с Neuro CD. Мне, обладателю retail-"писца" от PHILPS, еще тогда это показалось странным, поскольку, я хорошо помнил, что диск с драйверами в коробке PHILPS'a был, а Easy CD Creator - непосредственно сам пишущий софт, - располагался совсем на другом диске. Но ведь как-то же справляются с OEM-продукцией другие люди, подумал я, и… купил.

Наскоро воткнув новехонький 40-скоростной NEC в свой компьютер, я был немало удивлен, когда Neuro CD наотрез отказался признать его "писцом". Не помог тут и Easy CD Creator, взятый с Филечкиного CD. Провозившись битый час и, ничего ровным счетом так и не выяснив, я, зверски разозленный на продавца, решил сделать ход конем, установив NEC на компьютер с "девяносто восьмой" Windows, вернув PHILPS'a себе. Никаких изменений! Собравшись было отдавать привод назад продавцу, я неожиданно вспомнил, что в одном из последних номеров Компьютер Пресс был обзор пишущих программ, причем демонстрационные версии всех этих программ содержались на прилагаемом к журналу компакт-диске. Из всех программ NEC'овый писец опознала лишь одна: Record NOW, которая, к счастью не имела никаких функциональных ограничений, исключая, правда 30-днейвый триальный период. Причем, программа оказалась такой уютной и удобной, что расставаться с ней мне не захотелось, но и расставаться со своими деньгами мне не хотелось тоже.

Как выглядит защита? При каждом запуске программа выводит противный nag-screen, напоминающий сколько дней ей еще "жить" осталось, и тем самым страшно нервирующий: Хорошо, ищем фразу "Number of days remaining in evaluation" во всех файлах программы и, если наша искалка поддерживает уникод, быстро выясняется, что данный текст содержится в файле lockers.dll, открыв который любым редактором ресурсов, мы обнаруживаем в нем тот самый заветный диалог! Остается выяснить: кто же выводит этот диалог на экран? Ищем строку "lockres.dll" во всех файлах программы. ОК, это lockout.dll. Да… и эти разработчики не в ладах с юмором. Запускам dumpbin и смотрим список экспортируемых функций:

```
Dump of file lockout.dll

File Type: DLL

  Section contains the following exports for lockout.dll

           0 characteristics
    3C855E8D time date stamp Wed Mar 06 03:10:53 2002
        0.00 version
           1 ordinal base
          23 number of functions
          23 number of names

    ordinal hint RVA      name

          3    0 0000CFF0 ?DESDecrypt@@YAKPBDPAD0@Z
          4    1 0000CC40 ?DESEncrypt@@YAKPBDPAD0@Z
          1    2 00003520 EvalModeTest
          2    3 00003930 EvalModeTestVB
          6    4 0000B230 _ezLICENSE_Check_Delphi@16
          7    5 0000B1A0 _ezLICENSE_Check_VB@16
          9    6 0000BC20 _ezLICENSE_ChkExpire_Delphi@16
```

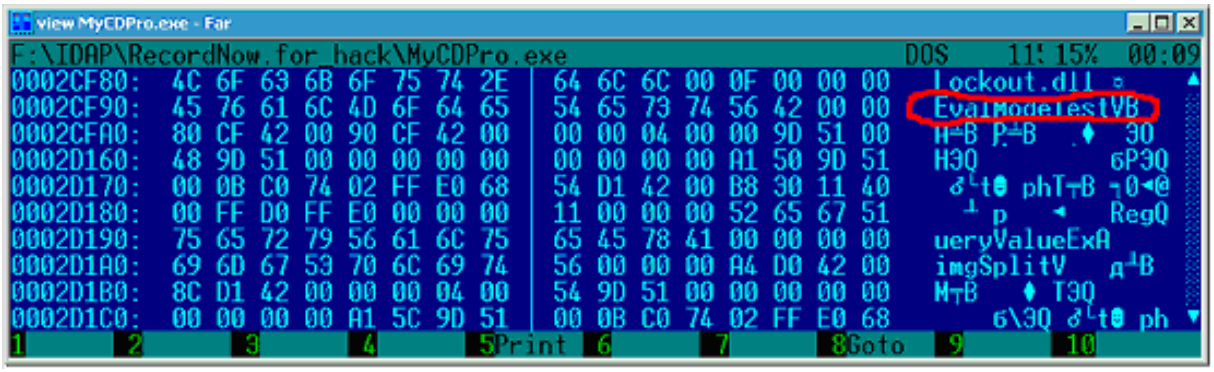
```
10    7 0000BB90 _ezLICENSE_ChkExpire_VB@16
12    8 00009DB0 _ezLICENSE_ChkFileCRC_Delphi@8
13    9 00009D40 _ezLICENSE_ChkFileCRC_VB@8
15    A 0000BA30 _ezLICENSE_Clear_Delphi@12
16    B 0000B9B0 _ezLICENSE_Clear_VB@12
18    C 0000A320 _ezLICENSE_GetRestNumber_Delphi@16
19    D 0000A290 _ezLICENSE_GetRestNumber_VB@16
22    E 0000A6C0 _ezLICENSE_Upgrade_Delphi@20
23    F 0000A610 _ezLICENSE_Upgrade_VB@20
 5   10 0000B2C0 ezLICENSE_Check
 8   11 0000BCA0 ezLICENSE_ChkExpire
11   12 00009E20 ezLICENSE_ChkFileCRC
14   13 0000BAA0 ezLICENSE_Clear
17   14 0000A3B0 ezLICENSE_GetRestNumber
20   15 00009C30 ezLICENSE_GetVersion
21   16 0000A770 ezLICENSE_Upgrade
```

Сурово! Во-первых, обращает на себя пара функция DES Encrypt/DES Decrypt, что-то (как и следует из ее названия) зашифровывающая/расшифровывающая. Во-вторых, тройственный подход к наименованию функций, наводит на мысль, что мы имеем дело с "конвертной" защитой, разработанной независимо от защищенной с ее помощью программы, и поддерживающий все основные языки программирования: Си/Си++, Дельфи и конечно же, Visual Basic, узнаваемый по суффиксу VB. В-третьих, такое обилие всевозможных проверочных функций предвещает, что исследование защиты и защищенной программы окажется делом отнюдь не легким! Причем, в те три сотни килобайт, которые занимает файл lockout.dll можно много всяких ловушек и хитростей понапхать, так что на скорый успех нам рассчитывать не приходится. Но… глаза страшатся, а руки делают. Начнем с того, что посмотрим: какие именно функции защитной библиотеки использует программа.

…вот тебе и раз! Защищенная-то программа состряпана на Визуальном Бейсике, о чем красноречиво свидетельствует единственная явно загружаемая ею библиотека MSVBVM60.DLL! Ах, так?! Хорошо, пойдем напролом. Просто удаляем lockout.dll из каталога программы и подсовываем ей любую другую DLL, предварительно переименованную в данную. Запускаем программу. На экране незамедлительно появляется сообщение об ошибке: среда Visual Basic'a ругается, что не может найти функцию EvalModeTestVB. Что ж, это уже кое-что! Загружаем lockout.dll в дизассемблер, находим в нем эту самую "Eval", быстро выясняем, что она является "переходником" к EvalModeTest, которая… которая… Ой-ой-ой, которая занимает до черта килобайт и содержит в себе крайне запутанный с большим количеством глубоко вложенных друг в друга процедур программный код. Да чтобы проанализировать все это и месяца не хватит! А кто сказал, что этот код вообще следует анализировать?! Достаточно просто подсунуть нужный код возврата и все! Весь вопрос в том: какой именно код нужный. Бегло просмотр содержимого функции показал, что существуют как минимум три различных кода возврата: "0", "2" и "3". Если это так, то скорее всего одному из них соответствует состояние "программа не зарегистрирована, но лицензия еще не истекала", "программа не зарегистрирована и лицензия уже истекла", и, наконец, "программа зарегистрирована". Что ж, на перебор трех вариантов не уйдет много времени! Взяв в руки HIEW, переписываем код защитной функции "с нуля": XOR EAX, EAX/RETN.

Возвращаем lockout.dll на ее прежнее место, запускаем Record NOW и… не можем поверить свои глазам, - программа исправно работает! "Исправно" - в том смысле, что nag?screen уже не выводится и по истечении положенных тридцати дней писец по-прежнему живет, а не умирает.

Хорошо, а если бы разработчик защищенного приложения, не поленился бы воткнуть проверку на успешность загрузки функции EvalModeTestVB и при ее отсутствии немотивированно прекращал свою работу? Смогли бы мы тогда узнать: какие функции библиотеки lockout используется, а какие нет? Уговорили! Взломаем программу другим путем! Погоняем курсор к MyCDPro.exe и, нажав на <F3>, пытаемся найти lockout.dll прямым контекстным поиском. Вот, пожалуйста:



Поиск ссылки на lockout.dll в защищенной программе

Прямым текстом: "lockout.dll" и рядышком с ней EvalModeTestVB. Имена остальных защитных функций в исследуемой программе отсутствуют. Самое забавное, что в модуле lockout.dll присутствует огромное количество строк типа: "'User has turned back their clock, so calculating days based on last and init", "The CRC file is valid", "Failed to update the Last Accessed time", - т. е. защита составлена довольно грамотно и в состоянии как следует за себя постоять. Если, конечно, разработчик защищаемого приложения использовал все, предоставленные ей возможности, сполна. Увы, этого не произошло и на этот раз…

Примеры реальных взломов: UniLink v1.03 от Юрия Харона

Автор: Крис Касперски

Дата: 2003-04-08

Баста! Надоело! Все эти уродские защиты… (см. описания четырех предыдущих взломов) только портят настроение и еще, - чего доброго - вызывают у читателей смутное сомнение: а не специально ли автор подобрал такие простые программы? Может быть, он - автор - вообще не умеет ничего серьезного ломать?! Уметь-то он (вы уж поверьте) умеет, но публично описывать взлом "серьезных" программ - боязно, а в "несерьезных" хороших защит мне как-то и не попадалось. Хотя, стоп! Ведь есть же такой программный продукт как UniLink, созданный опытнейшим системщиком Юрием Хароном (хорошо известным всем членам туссовски FIDO7.SU.C-CPP; если же вы никогда не заглядывали туда ранее, не поленитесь, сходите на Google, поднимите архив конференции и почитайте, - уверяю вас, вы не пожалеете). Достаточно сказать, что один лишь bag-list на UniLink - настоящая кладезь информации, перечисляющая больше количество ошибок операционной системы и ее окружения.

Наша цель - отучить UniLink ругаться на trial expired при запуске (из уважения к Харону необходимо отметить, что взлом проводится исключительно из спортивного интереса и природного любопытства, какие либо корыстные цели тут не причем - линкер абсолютно бесплатен и может быть свободно скачен по следующему адресу: ftp://ftp.styx.cabel.net/pub/UniLink/ulnbXXXX.zip, где XXXX - номер версии). Цитирую со слов Харона "Любая бета через полтора месяца начнёт "ругаться", что мол она expired :). Сделано это, просто как напоминание. в силу заинтересованности в том, что бы тестировались последнии билды". Так что, ломая линкер помните, что взлом еще не освобождает от beta-тестирования ;-).

Несмотря на бесплатность линкера, Харон очень неплохо его защитил. Во всяком случае у меня на полный анализ защиты (включая развернутое описания взлома и отвлечения на повседневную текучку) ушла добрая неделя! Сейчас, когда пишутся эти строки, даже жалко, что защита так быстро сломалась и то интересное, во что еще можно вонзить свои зубы, закончилось. Впрочем, лучше отложим всю эту ностальгию до лучших времен в сторону, и вспомним, как эта неделя "эротических развлечений с защитой" собственно и начиналась…

…привычным движением руки загружаем исполняемый файл линкера в свою любимую IDA и… IDA грязно ругается по поводу того, что… "can't find translation for virtual address 00000000, continue?". Хм, ну что нам еще остается делать, - покорно жмем "Yes", чтобы сделать "continue". Увы! Наш фокус не увенчался успехом - на экране возникает еще одно ругательство "File read error at 0004C7AC (may be bad PE structure), continue?". Обречено жмем "Yes" и… …IDA просто исчезает. Да-да! Именно исчезает, даже не успев перед смертью выдать сообщение о критической ошибке!!!

Интересный формат файла, однако! Пытаясь выяснить что же в нем содержится такое нехорошее, что так не понравилось IDA, мы решаем натравить на него утилиту dumpbin. Щас! Разбежались - при попытке вывести таблицу импорта, dumpbin выдает сообщение о внутренней ошибке "DUMPBIN: error: Internal error during DumpImports", и только что успев скинуть контекст аварийно прекращает свою работу. Вот, значит, как?! Ну, защита, держись! Сейчас мы заглянем внутрь тебя "вручную", - каким ни будь низкоуровневым инструментом, ну, например, HIEW'ом…

Облом-с! При попытке сделать "prepare import data" HIEW скручивает дулю и, выдав нам на прощание трогательно красное окошко с надписью "Import name No free memory" банально виснет. Конкурирующий с ним QVIEW умирает и вовсе без каких либо пояснений. Утилита "PEDUMP" от Мэта Питтрека (известнейшего исследователя недр Windows) хоть и не виснет, но выдает

сообщение о критической ошибке приложения и автоматически прибивается операционной системой. Так, чем еще можно исследовать внутренний формат PE-файла? На ум приходит efd (Executable File Dumper) от Ильфака, но даже эта утилита не справляется - выдав сообщение "Can't find translation for 000002F6 (758.)" она просто прекращает свою работу. Dump PE от Clive Turvey поступает аналогично. Дизассемблер De Win от Милюкова - виснет. Win DASM не виснет, но и не дизассемблирует. Даже знаменитый PROCDUMP распаковывать этот файл отказывается, правда позволяет сделать rebuild PE-заголовка, однако, после такой операции полученный файл становится неработоспособным. В, общем, этот список можно продолжать бесконечно…

Кошмар! Защиты, срывающие крышу отладчику, - это я еще понимаю, но вот чтобы так агрессивно сопротивляется дизассемблеру! Причем, не какой-то одному, конкретно взятому дизассемблеру, а всем дизассемблерам сразу. И в это же самое время защита ухитряется работать в любой Windows-совестимой операционной системе, включая NT и w2k, а, значит, никаких грязных хаков не использует. Харон по определению гений!

Вот мы и столкнулись с тем самым случаем, когда приходится дизассемблировать не готовым дизассемблером, а своими собственными руками и головой! Тяпнув для храбрости пивка, запускаем Иду и загружаем нашего подопытного в бинарном режиме, то есть без анализа заголовков файла. Файл, естественно, успешно загружается. Теперь, открываем свой MSDN на странице "Microsoft Portable Executable and Common Object File Format Specification" и вдумчиво читаем все, что в там написано. Без четкого представления о структуре и порядке загрузке PE-файлов Харонову защиту нам ни за что не сломать. Если чтение фирменных спецификаций вызывает проблемы, попробуйте обратится к сторонним источникам. В том же MSDN содержится масса статей, посвященных исследованию PE?формата, в частности: "The Portable Executable File Format from Top to Bottom" by Randy Kath, русский перевод которой ("Исследование переносимого формата исполнимых файлов сверху вниз") легко найти в Сети. На худой конец можно обойтись и одним лишь заголовочным файлом WINNT.H, входящим в штатный комплект поставки любого windows-компилятора (но разобраться с "голым" WINNT.H сумеет лишь гений!)

Наша задача состоит в том, чтобы вручную проанализировать все заголовки, все секции и все поля исследуемого файла, пытаясь определить: что же такого необычного есть в каждом из них. Спрашиваете: "необычное" - это вообще как? Навскидку можно предположить по крайней мере три варианта: а) защита использует документированные, но малоизвестные возможности PE?файлов, не поддерживаемые распространенными дизассемблерами; б) защита использует недокументированные особенности (и/или поля) PE?файлов, не поддерживаемые дизассемблерами, но корректно обрабатываемые операционной системой; в) разночтения спецификаций PE?формата привели к тому, что разработчики ОС трактовали отдельные поля заголовков по-своему, а разработчики дизассемблеров - по-своему, в результате чего появилась возможность создать такой извращенный файл, корректно загрузить который сумеет одна лишь система, а все остальные исследовательские программы конкретно обломаются на его анализе.

Из пункта "а" со всей очевидностью следует, что для анализа защищенного файла одной лишь документации явно недостаточно, ведь нам требуется не только убедиться в соответствии всех полей исследуемого файла фирменной спецификации, но и выяснить: насколько эти поля вообще типичны. Другими словами нам необходим практический опыт работы с PE-файлами, а если его нет, - что ж, возьмите несколько заведомо неизвращенных PE-файлов и основательно проштудируйте их от пола до потолка.

С пунктом "б" справится сложнее. Допустим, в фирменной спецификации такое-то поле помечено как неиспользуемое, а в защищенном файле здесь прописано некоторое значение. Как быть? (Дизассемблировать загрузчик операционной системы не предлагать). Да очень просто! Берем hiew старой версии - той, которая ничего не знает о PE и никак его не анализирует - и перебиваем "неиспользуемое" поле нулями или любым другим значением, пришедшимся нам по вкусу. Если это не нарушит работоспособности защищенного файла, - по всей видимости это поле действительно

не используется и, соответственно, наоборот.

Пункт "в" еще более сложен. Никакие прямолинейные решения тут не действуют и все, что нам остается - вдумчиво читать каждую букву исходной спецификации и… нет! не стремиться "понять" ее, а пытаться представить себе: как она вообще должна быть понята, чтобы загрузчик операционной системы работал, а дизассемблер - нет. Дайте волю своему воображению, напрягите интуицию - весь многих тонкостей PE-форматов составители документации просто не описали. С другой стороны, сами разработчики ОС данный формат не с потолка брали и по тем же самым спецификациям его и реализовывали. Задумайтесь: а как бы вы реализовали загрузку PE-файла в память? Какие бы комбинации свойств PE-файла вы могли бы использовать для его защиты?

Первое, что нам приходит в голову: инициализация некоторых критических ячеек памяти посредством добавления их адреса в таблицу перемещаемых элементов. А что, это мысль! Особенно привлекательной в этом плане выглядит таблица перемещаемых элементов из old exe - заглушки, расположенной перед PE-файлов и большинством дизассемблеров просто игнорируемой. Но обращает ли системный загрузчик внимание на эти элементы или нет, - вот ведь в чем вопрос! Хорошо, давайте посмотрим на восстановленный old exe заголовок, извлеченный нами из защищенного файла.

```
seg000:00000000 ; OLD EXE HEADER
seg000:00000000 cc     db 'MZ'
seg000:00000002 e_cblp     dw 405
seg000:00000004 e_cp     dw 1
seg000:00000006 e_crlc     dw 0
seg000:00000008 e_cparhdr    dw 4
seg000:0000000A e_minalloc    dw 33
seg000:0000000C e_maxalloc    dw 33
seg000:0000000E e_ss     dw 16h
seg000:00000010 ccaaa     dw 512
seg000:00000012 e_csum     dw 0
seg000:00000014 e_ip     dw 106
seg000:00000016 e_cs     dw 0
seg000:00000018 e_lfarlc    dw offset RelocationTable
seg000:0000001A e_ovno     dw 0
seg000:0000001C ae_res     db 'UniLink!'
seg000:00000024 e_OEMid     dw 0
seg000:00000026 e_OEMinfo    dw 1
seg000:00000028 e_res2     db 14h dup(0)
seg000:0000003C e_lfanew    dd offset IMAGE_NT_SIGNATURE_PE ; "PE"
```

Баста карапузики! Нас обломали! Никаких перемещаемых элементов в DOS-заглушке нет, о чем поле e_ovno красноречиво и свидетельствует (в дизассемблерном листинге оно выделено жирным шрифтом). Да и во всех остальных отношениях, old exe заголовок выглядит вполне корректным и приличным. Ладно, лиха беда начало! Отталкиваясь от значения поля e_lfanew, переходим по содержащемуся в нем смещению на заголовок PE-файла.

```
seg000:00000198 ; NEW EXE HEADER
seg000:00000198 IMAGE_NT_SIGNATURE_PE db 'PE',0,0 ; DATA XREF: seg000:0000003C
seg000:0000019C Machine               dw 14Ch ; IMAGE_FILE_MACHINE_I386
seg000:0000019E NumberOfSection       dw 3  ; три секции
seg000:000001A0 TimeDateStamp         dd 3D4EE158h ; временная метка
seg000:000001A4 PointerToSymbolTable  dd 0  ; указатель на таблицу символов
seg000:000001A8 NumberOfSymbols       dd 0  ; кол-во символов ноль, т.е. нет
seg000:000001AC SizeOfOptionalHeader  dw 0C0h ; размер опционального заголовка
seg000:000001AC ;           а вот это   ^^^^ уже интересно: зная, за концом
seg000:000001AC ; опционального заголовка сразу же следуют заголовки сегментов,
seg000:000001AC ; пытаемся проверить корректность этого поля "на глаз":
seg000:000001AC ; складываем 0x1B0 (начало опционального заголовка) с 0xC0
seg000:000001AC ; (указанный размер заголовка) и получаем 0x270.
seg000:000001AC ; смотрим - по этому смещению в файле расположено слово ".text",
seg000:000001AC ; значит, размер заголовка указан правильно.
seg000:000001AC ; Но… в то же самое время 0xC0 - это крайне нетипичный размер для
seg000:000001AC ; опционального заголовка и все, исследуемые мной файлы, содержали
```

```
seg000:000001AC ; совсем другое значение, - а именно 0xE0.
seg000:000001AC ; за счет чего же "наш" заголовок оказался меньше? очевидно,
seg000:000001AC ; защищенный файл содержит урезанный массив data directory, что
seg000:000001AC ; теоретически должно восприниматься всеми дизассемблерами нормально,
seg000:000001AC ; но вот полной увечности у нас в этом нет. Как быть? Представляется
seg000:000001AC ; логичным найти (или создать) PE-файл с урезанной data directory
seg000:000001AC ; и натравить на него дизассемблер (ту же IDA) - интересно зависнет
seg000:000001AC ; он или нет? А вот как создать такой файл, не имея под руками
seg000:000001AC ; соответствующего линкера? Просто пропадчить заголовок в готовом
seg000:000001AC ; PE-файле нельзя, т. к. за концом data directory загрузчик ожидает
seg000:000001AC ; увидеть каталог сегментов, а при "искусственном" уменьшении размера
seg000:000001AC ; заголовка там окажется "хвост" от data directory, что приведет
seg000:000001AC ; дизассемблер в сильное замешательство. "вырезать" кусочек
seg000:000001AC ; data directory из файла так же невозможно, ведь при этом посыплются
seg000:000001AC ; все смещения, что так же приведет к непредсказуемой реакции
seg000:000001AC ; дизассемблера при попытке анализа такого файла. А если… Постойте-ка!
seg000:000001AC ; ведь можно просто сдвинуть каталог сегментов на место
seg000:000001AC ; "освободившихся" после усечения заголовка элементов data directory?!
seg000:000001AC ; а знаете, это должно сработать! OK, вооружившись hiew'ом усекаем
seg000:000001AC ; размер заголовка любого заведомо нормального файла до 0xC0 и
seg000:000001AC ; перемещаем каталог сегментов на 0x20 байт "вверх". Запускаем сам
seg000:000001AC ; фал. Работает? Работает! Загружаем файл в дизассемблер… Работает!!!
seg000:000001AC ; OK, значит, размер заголовка в 0xC0 действительно допустим
seg000:000001AC ; продолжаем анализ….
seg000:000001AC ;
seg000:000001AE Characteristics        dw 30Fh ; IMAGE_FILE_RELOCS_STRIPPED|
seg000:000001AE     ; IMAGE_FILE_EXECUTABLE_IMAGE|
seg000:000001AE      ; IMAGE_FILE_LINE_NUMS_STRIPPED|
seg000:000001AE      ; IMAGE_FILE_32BIT_MACHINE |
seg000:000001AE     ; IMAGE_FILE_DEBUG_STRIPPED
seg000:000001AE ; атрибуты файла несколько нетипичны. обычно встречается 0x10F,
seg000:000001AE ; а не 0x30F (т.е. в нормальных файлах отсутствует флаг
seg000:000001AE ; IMAGE_FILE_DEBUG_STRIPPED даже когда они не содержат никакой
seg000:000001AE ; отладочной инфы), но с другой стороны, так даже и правильнее.
seg000:000001AE ; Эксперименты показывают, что исправление 0x10F на 0x30F в
seg000:000001AE ; остальных файлах (ес-но без дебужной инфы) проходит безболезненно,
seg000:000001AE ; значит, собака зарыта не здесь
```

Вот мы и выяснили, что PE-заголовок защищенного файла не содержит абсолютно ничего интересно, и если кто и завешивает HIEW и срывает IDA крышу, то уж точно не он. Что ж, сделав короткий перерыв (для "пивка"), продолжим наше утомительное исследование формата PE-файла, на сей раз взявшись за так называемый опциональный заголовок (optional header), следующий за концом PE-заголовка.

```
seg000:000001B0 ; ОПЦИОНАЛ ХИДЕР
seg000:000001B0 ; ==============
seg000:000001B0 Magic                   dw 10Bh ; NORMAL EXE (все OK)
seg000:000001B2 MajorLinkerVersion      db 1  ; версия линкера
seg000:000001B3 MinorLinkerVersion      db 3  ; версия линкера
seg000:000001B4 SizeOfCode              dd 49817h ; размер кода
seg000:000001B4     ; выглядит вполне нормально.
seg000:000001B4     ; т. е. при длине exe-файла в
seg000:000001B4     ; 0x4C7AA байт, потребности в
seg000:000001B4     ; 0x49817 байт вполне
seg000:000001B4     ; удовлетворяются
seg000:000001B4
seg000:000001B8 SizeOfInitializedData   dd 3008h ; размер секции
seg000:000001B8     ; инициализированных данных
seg000:000001B8     ; выглядит вполне нормально
seg000:000001B8
seg000:000001BC SizeOfUninitializedData dd 0  ; нет секции
seg000:000001BC     ; неинициализированных данных
seg000:000001C0 AddressOfEntryPoint    dd 46673h ; адрес точки входа
seg000:000001C4 BaseOfCode    dd 1000h ; базовый адрес сегмента кода,
seg000:000001C4      ; забегая вперед, отметим,
seg000:000001C4     ; что этот адрес в точности равен
seg000:000001C4     ; адресу сегмента .text, так что
seg000:000001C4     ; тут все законно
seg000:000001C4
seg000:000001C8 BaseOfData              dd 4B000h ; базовый адрес сегмента данных,
```

```
seg000:000001C8      ; проверка подтверждает его
seg000:000001C8      ; корректность
seg000:000001C8
seg000:000001CC ImageBase               dd 400000h ; image base абсолютно нормальный
seg000:000001D0 SectionAlignment        dd 1000h ; выравнивание секций по границе
seg000:000001D0     ; в 4K6, что OK
seg000:000001D0
seg000:000001D4 FileAlignment           dd 200h ; выравнивание файла по границе
seg000:000001D4      ; в 512 байт, что OK
seg000:000001D8 MajorSysVersion         dw 4  ; версия требуемой системы, OK
seg000:000001DA MinorSysVersion         dw 0  ; OK
seg000:000001DC MajorImageVersion       dw 1  ; версия приложения, OK
seg000:000001DE MinorImageVersion       dw 0  ; OK
seg000:000001E0 MajorSubsystemVersion   dw 4  ; версия подсистемы, OK
seg000:000001E2 MinorSubsystemVersion   dw 0  ; OK
seg000:000001E4 Win32VersionValue       dd 0  ; OK
seg000:000001E8 SizeOfImage             dd 52000h ; размер образа файла в памяти
seg000:000001E8      ; выглядит вполне достоверно
seg000:000001E8
seg000:000001EC SizeOfHeaders           dd 400h ; размер всех заголовков, OK
seg000:000001F0 CheckSum                dd 0  ; нет контрольной суммы, OK
seg000:000001F4 Subsystem               dd 3  ; кол-во секций, OK
seg000:000001F4      ; (дальше мы их все найдем)
seg000:000001F4
seg000:000001F8 SizeOfStackReserve      dd 100000h ; кол-во резервируемой памяти
seg000:000001F8      : под стек, OK
seg000:000001F8
seg000:000001FC SizeOfStackCommit       dd 2000h ; кол-во выделенной под стек
seg000:000001FC      ; памяти, OK
seg000:000001FC
seg000:00000200 SizeOfHeapReserve       dd 100000h ; кол-во резервируемой под кучу
seg000:00000200      ; памяти, OK
seg000:00000200
seg000:00000204 SizeOfHeapCommit        dd 1000h ; кол-во выделенной под кучу
seg000:00000204     ; памяти, OK
seg000:00000204
seg000:00000208 LoaderFlags             dd 0  ; не используется, OK
seg000:0000020C NumberOfRvaAndSizes     dd 0Ch ; кол-во элементов в
seg000:0000020C     ; IMAGE_DATA_DIRECTORY
```

…и опциональный заголовок не содержит ничего интересного, но вот IMAGE DATA DIRECTORY, расположенная за ним следом, - дело другое и буквально с третий по счету строки мы выходим на след защиты:

```
seg000:00000210 IMAGE_DATA_DIRECTORY   dd 0  ; EXPORT dir
seg000:00000214                        dd 0
seg000:00000218
seg000:00000218 Import Table
seg000:00000218                        dd offset IMPORT_TABLE ;
```

Вот она - ссылка на таблицу импорта, - ту самую таблицу, которая приводит к буйному замешательству огромное количество дизассемблеров и срывает крышу всем PE-утилитам вместе взятым. Посмотрим на нее?

```
seg000:0004B000 IMPORT_TABLE              dd 94010F0Eh  ; DATA XREF: seg000:00000218^o
seg000:0004B000                                         ; flags
seg000:0004B004                           dd 4000696h   ; date start
seg000:0004B008                           dd 54414C46h  ; foward index
seg000:0004B00C                           dd offset unk_39A39
seg000:0004B010                           dd 8965410h   ; import addres
seg000:0004B014
```

Пошла вода в хату! Оказывается, в таблице импорта вместо нормальных полей содержится какой-то голимый "мусор", который кое-что проясняет. С такой таблицей импорта дизассемблеры работать просто не могут и… если проверка корректности содержимого таблицы импорта отсутствует, они - виснут, в противном же случае, - аварийно прерывают свою работу с сообщением об ошибке.

Но это совершенно не объясняет как с такой защитой ухитряется работать загрузчик операционной системы? Уж не имеем ли мы дело с некоторыми недокументированными особенностями? Или, быть может, по этим "мусорным" адресам в оперативной памяти расположено что-то особенное? Последнее навряд ли! Поскольку защита успешно функционирует во всех windows-подобных системах, представляется сомнительным, что содержимое данных адресов всегда и везде одно и то же (кстати, беглая проверка отладчиком, это допущение с треском опровергает). Недокументированные возможности? Хм, непохоже… да если так - где прикажите искать реально импортируемые адреса?! Ладно, двигаемся дальше, может быть нам и повезет…

```
seg000:00000268 ; Bound Import
seg000:00000268                         dd offset bound_import_table
seg000:0000026C                         dd 1Ch
```

Ага! Держи Тигру за хвост! Защита использует документированное, но малоизвестное поле bound import, - представляющее собой альтернативный механизм импорта функций из DLL. Смотрим, что у нас там…

```
seg000:000002E8 ; bound import table
seg000:000002E8 TimeDateStamp          dd 0FFFFFFFFh ; DATA XREF: seg000:0000268
seg000:000002EC OffsetModuleName       dw 0Eh  ; относительное смещение
seg000:000002EC       ; строки, содержащей имя
seg000:000002EC       ; импортируемой DLL
seg000:000002EC       ; 0x2E8 + 0xE == 0x2F6
seg000:000002EC       ; где мы обнаруживаем
seg000:000002EC       ; "kernel32.dll", что
seg000:000002EC       ; очевидно, уже не мусор!
seg000:000002EC
seg000:000002EE NumberOfModuleForward   dw 0   ; ничего не импортируем?!
seg000:000002F0 Reserverd               dw 0
seg000:000002F2                         dd 0
seg000:000002F6 aKernel32_dll           db 'kernel32.dll',0 ; DATA XREF: seg000:049E0C
```

Вот это уже явно не мусор, а вполне удобоваримая таблица импорта, загружающая динамическую библиотеку kernel32.dll, и импортирующая…. Как это так - никаких функций?! Странно… Но ведь защита все-таки работает (пусть час от часу становится все менее и менее понятно как). Хорошо, давайте рассуждать логически. Программ, не импортирующих никаких функций, под Windows NT существовать в принципе не может. Даже если защита использует native API (т. е. обращается к системным функциям напрямую через прерывание 2Eh), операционный загрузчик окажется не в состоянии загрузить такое приложение, поскольку ему необходимо, чтобы на адресное пространство загружаемого процесса была спроецирована библиотека kernel32.dll. Это в Windows 9x, где системные библиотеки автоматически отображаются на адресные пространства процессов, "голые" файлы работают безо всяких проблем, а в NT, отображающий только явно загруженные библиотеки, такой фокус уже не проходит. А, знаете, это многое объясняет! Теперь становится понятно в частности почему таблица импорта не содержит в себе ни одной функции, - они просто не нужны! Ссылка на kernel32.dll присутствует лишь затем, чтобы спроецировать эту библиотеку на адресное пространство процесса, как этого требует системный загрузчик. Хорошо, но как быть с "мусором" в стандартной таблице импорта? Как ни крути, а такие извращения системный загрузчик скорее удавится, чем обработает… Увы, нам нечего ответить на этот вопрос и, скрепя сердце, его вновь приходится откладывать, надеясь, что последующий анализ отделит свет от тьмы и все расставит по своим местам…

```
seg000:00000270 ; НАЧАЛО СЕГМЕНТОВ
seg000:00000270 a_text                db '.text',0,0,0
seg000:00000278 vir_size_text         dd 49817h ; размер секции text в памяти
seg000:0000027C virt_addr_text        dd 1000h ; адрес проекции на память
seg000:00000280 szRawData_text        dd 49810h ; размер в файле
seg000:00000284 pRawData_text         dd 400h ; смещение начала секции в файле
seg000:00000288 pReloc_text           dd 0
seg000:0000028C pLineNum_text         dd 0
seg000:00000290 nReloc_text           dw 0
seg000:00000292 nLineNum_text         dw 0
```

```
seg000:00000294 FLAG_TEXT             dd 60000020h ; code | executable | readable
```

Вот мы и добрались до каталога сегментов! IMAGE HEADER секции ".text" выглядит вполне типично и никаких подозрений у нас не вызывает, но вот следующая за ним секция ".data" очень многое прояснеет…

```
seg000:00000298 a_data              db '.data',0,0,0
seg000:000002A0 vir_size_data       dd 3008h ; размер секции .data в памяти
seg000:000002A4 vir_addr_data       dd 4B000h ; адрес проекции на память
seg000:000002A8 szRawData_data      dd 14h  ; размер в файле
seg000:000002AC pRawData_data       dd 49E00h ; смещение в файле
seg000:000002B0 pReloc_data         dd 0
seg000:000002B4 pLineNum_data       dd 0
seg000:000002B8 nReloc_data         dw 0
seg000:000002BA nLineNum_data       dw 0
seg000:000002BC FLAG_DATA           dd 0C0000040h ; readable | writeable
```

Ну и что здесь интересного? - спросит иной читатель. А вот что - присмотритесь повнимательнее куда именно грузится содержимое данной секции. Если верить выделенной жирным шрифтом строке, - то по адресу IMAGE_BASE + 0x4B000. Ничего не напоминает? Во-первых, адрес 0x4B000 "волшебным" образом совпадает с адресом "мусорной" таблицы импорта (те, кто поимел сект с защитой этот адрес надолго запомнят, кстати Харону не мешало бы его немножко замаскировать, чтобы он не так бросался в глаза). Во-вторых, изобразив процесс проецирования секций графически (см. рис. 0x05) мы с удивлением обнаружим, что секция .data расположена не следом за секцией .text (как это обычно и бывает), а находится внутри нее. Действительно, давайте подсчитаем: виртуальный адрес секции .text равен 0x1000, а ее размер - 0x49817, и последний байт секции приходится на адрес 0x59817, что превышает виртуальный адрес секции .data, равный 0x4B000.

Так вот оно что! Поскольку, секции отображаются на память в порядке их перечисления в каталоге (недокументированно, но факт!), то содержимое секции .data затирает область адресов 0x4B000 - 0x4E008! А что там у нас расположено?! ТАБЛИЦА ИМПОРТА!!! В дисковом файле по смещению 0x4B000 действительно расположен чистейшей воды мусор (и это косвенно подтверждается тем, что изменения первых 0x14 байт работу программы не нарушают), а истинная таблица импорта расположена непосредственно в секции .data, которой соответствует смещение 0x49E00 дискового файла. Заглянем: что у нас там?!

```
seg000:00049E00 RealImportTable           dd offset IAT ; OriginalFirstThunk
seg000:00049E04 TimeDateStamp1            dd 1
seg000:00049E08 ForwarderChain            dd 0FFFFFFFFh ; no forward
seg000:00049E0C Name                      dd offset aKernel32_dll ; "kernel32.dll"
seg000:00049E10 FirstThunk                dd offset IAT
```

Вот, это действительно похожее на таблицу импорта со ссылкой на IAT. Кстати, не мешает посмотреть, что за функции импортирует IAT. Подгоняем курсор к "IAT" и, нажав, на смотрим:

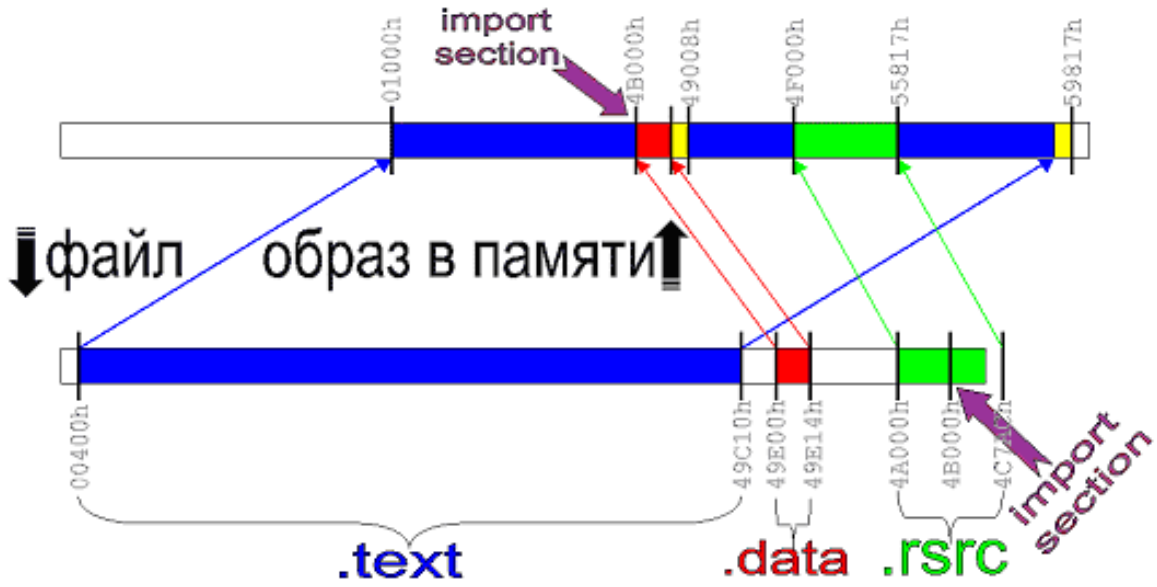
```
seg000:0004B014 IAT                       dd 47440600h  ; DATA XREF: seg000:00049E00^o
seg000:0004B014                                         ; seg000:00049E10^o
seg000:0004B018                           dd 50554F52h
seg000:0004B01C                           dd 69A8Bh
seg000:0004B020                           dd 0FF03FF11h
seg000:0004B024                           db    2 ;
seg000:0004B025                           db  4Ch ; L
```

Мать родная! Ну почему ты не родишь меня обратно?! Опять вместо символических имен или на худой конец - ординалов, нам попадается этот проклятый мусор! Хотя, - подождите минуточку - давайте попробуем определить что будет расположено по данному адресу после загрузки программы. Возвращаясь к описанию секции .data, мы обнаруживаем, что упустили один очень важный момент. Виртуальный размер секции .data (0x3008 байт) намного больше ее физического размера (0x14 байт) и потому, регион 0x4B014 - 49E008 будет заполнен нулями, а ведь "мусорная" IAT как раз и расположена по адресу 0x4B014! Следовательно, после загрузки ее содержимое

окажется заполнено одними нулями, что соответствует пустой таблице импорта функций. Фу-х! Невероятно, но мы действительно в этом разобрались!!! Кстати, подобный прием и широко используется авторами упаковщиков исполняемых файлов.

```
seg000:000002C0 seg000:000002C0 b_rsrc     db '.rsrc',0,0,0
seg000:000002C8 vir_size_rsc    dd 27ACh ; размер секции rsrc в памяти
seg000:000002CC vir_addr_rsc    dd 4F000h ; адрес проекции на память
seg000:000002D0 szRawData_rsc   dd 27ACh ; размер в файле
seg000:000002D4 pRawData_rsc    dd 4A000h ; смещение секции в файле
seg000:000002D8 pReloc_rsc      dd 0
seg000:000002DC pLineMun_rsc    dd 0
seg000:000002E0 nReloc_rsc      dw 0
seg000:000002E2 nLineNum_rsc    dw 0
seg000:000002E4 FLAG_RSC        dd 50000040h ; initalized data |
seg000:000002E4     ; shareable | readable
```

Аналогичным образом поступает и секция .rsrc, внедрясь в середину секции .text (но секцию .data она не перекрывает), причем, для ослепления некоторых дизассемблеров тут используется еще один хитрый примем: указанный "физический" размер секции .rsrc "вылетает" за пределы дискового файла. Системному загрузчику - хоть бы что, а вот некоторые исследовательские утилиты от этого и крышей поехать могут.



Динамическое замещение таблицы импорта в процессе загрузки PE-файла

Настало время проверить наши предположения на практике. Давайте загрузим эту извращенную программу отладчиком и посмотрим что содержится в памяти по адресу IMAGE_BASE + 0x4B000 = 0x44B000: мусор или нормальная таблица импорта? Отладчик soft-ice (как это и следовало ожидать) обламывается с отладкой этого извращенного файла, просто проскакивая точку входа, а вот WDB сполна оправдывая репутацию фирмы Microsoft (это не ирония!), пусть и не без ругательств, но все-таки загружает наш подопытный файл и послушно останавливается в точке входа.

```
Module Load: F:\IDAP\HARON\ulink.exe  (symbol loading deferred)
Thread Create:  Process=0, Thread=0
Module Load: C:\WINNT\SYSTEM32\ntdll.dll  (symbol loading deferred)
Module Load: C:\WINNT\SYSTEM32\kernel32.dll  (symbol loading deferred)
Module Load: C:\WINNT\SYSTEM32\ntdll.dll  (could not open symbol file)
Module Load: F:\IDAP\HARON\ulink.exe  (could not open symbol file)
Module Load: C:\WINNT\SYSTEM32\kernel32.dll  (could not open symbol file)
Stopped at program entry point
```

Обратите внимание на выделенную жирным шрифтом строку: отладчику показалось, что отлаживаемая программа импортирует некоторые функции… из самой себя! Но мы-то, излазившие

защищенный файл вдоль и поперек, хорошо знаем, что за исключением kernel32.dll, никаких других экспортируемых и/или импортируемых библиотек здесь нет и такое поведение отладчика, судя по всему, объясняется все тем же самым "мусором". ОК, переключаем свое внимание на окно с дампом памяти, заставляя ее отобразить содержимое таблицы импорта:

```
0x0023:0x0044B000 14 b0 04 00 01 00 00 00 ff ff ff ff f6 02 00 00 ................
0x0023:0x0044B010 14 b0 04 00 00 00 00 00 00 00 00 00 00 00 00 00 ................
0x0023:0x0044B020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ................
```

Ура! Открываем на радостях пиво! Содержимое памяти доказательно подтверждает, что загрузка файла действительно происходит именно так, как мы и предполагали! Хорошо, но что же нам теперь делать? То бишь, найти-то причину помешательства дизассемблеров мы нашли, но вот как ее нейтрализовать? Ну, это не вопрос! Достаточно лишь скопировать 0x14 байт памяти с адреса 0x49E00 по адресу 0x4B000 и скорректировать ссылку на IAT, направив ее на любое, заполненное нулями, место.

…HIEW теперь заглатывает защищенную программу и даже не пикает! А IDA… а IDA по прежнему отказываться обрабатывать этот файл и с завидным упорством слетает. В чем же причина? Вы, конечно, будете смеяться, но истинный виновник есть ни кто иной как Microsoft! Если бы не ее жутко прогрессивная платформа NET… А, впрочем, чего это я разворчался? Сами смотрите:

```
(o) Microsoft.Net assembly [pe.ldw]
( ) Portable executable for IBM PC (PE) [pe.ldw]
( ) MS-DOS executable (EXE) [dos.ldw]
( ) Binary file
```

Вот это да! С роду такого не было! Чтобы IDA да не правильно опознала формат файла!!! Перемещаем радио-кнопку на одну позицию вниз (ведь мы имеем дело отнюдь не с Microsoft Net assembly, а с PE!) и… IDA успешно открывает файл. Причем, с восстановлением таблицы импорта можно было и не возиться, - IDA просто ругнулась на мусор и все! Но кто ж знал?! Задним умом все мы крепки…

Короче, возвращаясь к нашим баранам (в данном случае - к терпеливо ожидающему нас отладчику) в точке входа дизассемблерный текст выглядит так:

```
00446673 55              push        ebp
00446674 68AECF4200      push        42CFAEh
00446679 8BDC            mov         ebx,esp
0044667B 2403            and         al,3
0044667D 7203            jb          00446682
0044667F FE4302          inc         byte ptr [ebx+2]
00446682 D7              xlat        byte ptr [ebx]
00446683 27              daa
00446684 81042453970000  add         dword ptr [esp],9753h
0044668B 1AC9            sbb         cl,cl
0044668D 9F              lahf
0044668E FF33            push        dword ptr [ebx]
00446690 FC              cld
00446691 C3              ret
```

Не очень-то это похоже на осмысленный код программы! Может быть, это снова мусор? Маловероятно, - ведь отладчик использует штатный системный загрузчик PE-файлов и потому показывает образ файла таким, какой он в действительности есть, ну… если, конечно, защита тем или иным образом не противостоит отладке. Ладно, отставив разговорчики в строю, начинам трассировать код и… с первых же строк впадаем в некоторое замешательство. Защита опрашивает начальное значение регистра EAX, которое (если верить отладчику!) как будто бы равно нулю, но полной уверенности в этом у нас нет, - еще со времен старушки MS-DOS многие отладчики славились тем, что самостоятельно инициализировали регистры после загрузки, чем и выдавали себя (в частности, при нормальной загрузке файла регистр SI содержал в себе адрес первой исполняемой команды, а при загрузке под отладчиком Turbo Debugger и иже с ним, был равен нулю). Вообще-то, закладываться на "предопределенные" значения регистров - дурной тон. Никто

не гарантирует, что в следующих версиях Windows что ни будь не изменится, и если такое вдруг произойдет, то защита откажет в работе, обломав не только хакеров, но и легальных пользователей. Впрочем, начальное значение регистра EAX (AX) по жизни равно нулю, и с некоторой натяжной за это можно зацепиться.

Далее защита непонятно зачем увеличивает старшее слово, только что закинутое в стек, на единицу и вызывает абсолютно бесполезные команды XLAT, DAA, ADD, SBB и… загружает регистр флагов в EAX. Уж не пытает ли она этим самым обнаружить флаг трассировки? Затем делает RETN для передачи управления по адресу: (0x42CFAE + 0x10000) + 0x9753 == 0x446701

```
.text:00446701         mov     edi, esi
.text:00446703         mov     esi, ebx
.text:00446705         sub     dword ptr [esi], 1006Fh
.text:0044670B         lodsw
.text:0044670D         bswap   eax
.text:0044670F         inc     byte ptr [esi]
.text:00446711         lodsb
.text:00446712         mov     ah, al
.text:00446714         lodsb
.text:00446715         bswap   eax
.text:00446717         mov     ebp, eax
.text:00446719         movzx   ecx, cl
.text:0044671C         push    dword ptr [ebp+6Bh]
.text:0044671F         lea     eax, [esi-8]
.text:00446722         xchg    eax, fs:[ecx]
.text:00446725         mov     edx, eax
.text:00446727         inc     edx
.text:00446728         jz      short loc_44672D
.text:0044672A         mov     edx, [eax+4]
.text:0044672D
.text:0044672D loc_44672D:          ; CODE XREF: .text:00446728 j
.text:0044672D         xchg    eax, [esp]
.text:00446730         pushf
.text:00446731         lea     ebx, [eax+21ADFh]
.text:00446737         jnz     short loc_446745
.text:00446739         lea     edi, [edi+0ACh]
.text:0044673F         mov     dword_44CAF8, edi
.text:00446745
.text:00446745 loc_446745:          ; CODE XREF: .text:00446737 j
.text:00446745         bts     dword ptr [esi-0Ch], 8
.text:0044674A         jb      short loc_446753
.text:0044674C         popf
.text:0044674D         call    $+5
.text:00446752         retf
```

…отладчик доходит лишь до RETF и после этого сразу же "дохнет". К тому же, остается совершенно непонятным, что же собственно делает этот запутанный и витиеватый код? При желании, конечно, с ним можно разобраться, но… нужно ли? Ведь отладить нашу подопытную мы все равного не сможем, во всяком случае в WDB.

Хорошо, зайдем с другого конца. Предположим, что программа работает с операционной системой не напрямую (через native API), а через подсистему win32 (win32 API). Тогда, установив точку останова на любую API-функцию, вызываемому программой, мы автоматически попадем в гущу "нормального" программного кода, уже распакованного (расшифрованного?) защитой. Весь вопрос в том: какие именно API-функции вызывает программа. Ну, пусть это будет GetVersion, - с вызова которой начинается стартовый код практически любой программы. Запускаем soft-ice, нажимаем , даем команду "bpx GetVersion", выходим из отладчика, вызываем unlink.exe и… ничего не происходит! - Отладчик не всплывает! Выходит, исследуемая нами программа не использует GetVersion! Что ж, удаляем предыдущую точку останова и пытаемся "забрейкать" CreateFileA (ну должен ли линкер как-то открывать файлы!!!). Так, , bpx CreateFileA, х… Ура! Это срабатывает! Отладчик перехватывает вызов защищенной программы и, после выхода из тела CreateFileA по команде P RET (в CreateFileA для нас действительно нет ничего интересного), мы оказывается в следующем коде:

```
001B:00416DEB  CALL    [USER32!CharToOemBuffA]
001B:00416DF1  PUSH    00000104
001B:00416DF6  LEA     EAX,[ESP+08]
001B:00416DFA  PUSH    EAX
001B:00416DFB  LEA     EDX,[ESP+0C]
001B:00416DFF  PUSH    EDX
001B:00416E00  CALL    [KERNEL32!GetShortPathNameA]
001B:00416E06  TEST    EAX,EAX
001B:00416E08  JZ      00416E2B
001B:00416E0A  LEA     EDX,[ESP+04]
001B:00416E0E  PUSH    00
001B:00416E10  PUSH    27
001B:00416E12  PUSH    03
001B:00416E14  PUSH    00
001B:00416E16  PUSH    01
001B:00416E18  PUSH    80000000
001B:00416E1D  PUSH    EDX
001B:00416E1E  CALL    [KERNEL32!CreateFileA]
001B:00416E24  MOV     EBX,EAX
001B:00416E26  CMP     EBX,-01
001B:00416E29  JNZ     00416E35
001B:00416E2B  CALL    [KERNEL32!GetLastError]
001B:00416E31  MOV     ESI,EAX
001B:00416E33  JMP     00416E5B
```

Обратите внимание: несмотря на отсутствие таблицы импорта, программа каким-то загадочным образом все-таки импортирует из kernell32.dll все, необходимые ей API-функции. Очень хорошо! Секс с native API и прочими извратами программистской хитрости отменяется! И мы остаемся в среде привычной нам подсистемы win32 API. Как именно осуществляется импорт - вот это уже другой вопрос! Кстати, давайте заглянем в одну такую функцию дизассемблером:

```
.text:00416E18           push 80000000h
.text:00416E1D           push edx
.text:00416E1E           call dword_44CC20 ; в отладчике это было KERNEL32!CreateFileA
.text:00416E24           mov  ebx, eax
.text:00416E26           cmp  ebx, 0FFFFFFFFh
.text:00416E29           jnz  short loc_416E35
…
.data:0044CC14 dword_44CC14   dd ?                    ; DATA XREF: sub_416DA0+AD^r
.data:0044CC14                                        ; sub_416DA0+F9^r ...
.data:0044CC18 dword_44CC18   dd ?                    ; DATA XREF: .text:0041A10E^r
.data:0044CC1C dword_44CC1C   dd ?                    ; DATA XREF: .text:0041A1AA^r
.data:0044CC20 dword_44CC20   dd ?                    ; DATA XREF: sub_416DA0+7E^r
.data:0044CC20                                        ; sub_416F3C+AB^r
.data:0044CC24 dword_44CC24   dd ?                    ; DATA XREF: sub_416DA0+DF^r
.data:0044CC24                                        ; sub_416F3C+128^r
.data:0044CC28 dword_44CC28   dd ?                    ; DATA XREF: sub_416F3C+1AE^r
.data:0044CC28                                        ; sub_417158+F1^r ...
.data:0044CC2C dword_44CC2C   dd ?                    ; DATA XREF: sub_419DD8+3C^r
.data:0044CC2C                                        ; sub_41AD20+12E^r ...
.data:0044CC30 dword_44CC30   dd ?                    ; DATA XREF: .text:004014C4^r
.data:0044CC34 dword_44CC34   dd ?                    ; DATA XREF: sub_419DD8+31^r
.data:0044CC34                                        ; .text:0041A3E5^r ...
.data:0044CC38 dword_44CC38   dd ?                    ; DATA XREF: sub_419DD8+1E^r
.data:0044CC38                                        ; .text:0041A3A4^r ...
```

Смотрите! В дисковом файле адресов импортируемых функций просто нет и таблица импорта судя по всему заполняется защитой динамически. А это значит, что в дизассемблере мы просто не сможем разобраться: какая именно функция в какой точке программы вызывается. Или… все-таки сможем?! Достаточно просто скинуть импорт работающей программы в дамп, а затем просто загрузить его в IDA! Затем, отталкиваясь от адресов экспорта, выданных "dumpbin /EXPORTS kernel32.dll", мы без труда приведем таблицу импорта в нормальный вид. Итак, прокручивая экран дизассемблера вверх, находим где у этой таблицы расположено ее начало или нечто на него похожее (если мы ошибемся - ничего странного не произойдет, просто часть функций останется нераспознанными и когда мы с ними столкнемся лицом к лицу, эту операцию придется повторять

вновь). Вот, кажется, мы нашли, что искали, смотрите:

```
.data:0044CC09                                        ; sub_43E6D4+22A^r ...
.data:0044CC0A                db ? ; unexplored
.data:0044CC0B                db ? ; unexplored
.data:0044CC0C                db ? ; unexplored
.data:0044CC0D                db ? ; unexplored
.data:0044CC0E                db ? ; unexplored
.data:0044CC0F                db ? ; unexplored
.data:0044CC10                db ? ; unexplored
.data:0044CC11                db ? ; unexplored
.data:0044CC12                db ? ; unexplored
.data:0044CC13                db ? ; unexplored
.data:0044CC14 dword_44CC14   dd ?                    ; DATA XREF: sub_416DA0+AD^r
.data:0044CC14                                        ; sub_416DA0+F9^r ...
.data:0044CC18 dword_44CC18   dd ?                    ; DATA XREF: .text:0041A10E^r
.data:0044CC1C dword_44CC1C   dd ?                    ; DATA XREF: .text:0041A1AA^r
.data:0044CC20 dword_44CC20   dd ?                    ; DATA XREF: sub_416DA0+7E^r
.data:0044CC20                                        ; sub_416F3C+AB^r
.data:0044CC24 dword_44CC24   dd ?                    ; DATA XREF: sub_416DA0+DF^r
.data:0044CC24                                        ; sub_416F3C+128^r
.data:0044CC28 dword_44CC28   dd ?                    ; DATA XREF: sub_416F3C+1AE^r
.data:0044CC28                                        ; sub_417158+F1^r ...
.data:0044CC2C dword_44CC2C   dd ?                    ; DATA XREF: sub_419DD8+3C^r
.data:0044CC2C                                        ; sub_41AD20+12E^r ...
```

Условимся считать адрес 0044CC14h началом. Используя точку останова на CreateFileA вновь вламываемся в программу и, отключив окно "data" командой wd, скидываем таблицу импорта в хистори: "d 44CC14". Выходим из Айса, запускаем NuMega Symbol Loader и записываем историю команд в файл winice.log (или любой другой по вашему вкусу). И как со всем этим нам теперь работать? Рассмотрим это на примере функции "call dword_44CC78". Прежде всего мы должны выяснить, какое значение находится в загруженной программе по адресу: 0x44CC87. Открываем winice.log по и смотрим:

```
0010:0044CC78 77E8668C  77E8F51E  77E93992  77E8DBF8      .f.w...w.9.w...w
0010:0044CC88 77E93F05  77E85493  77E87BE4  77E87D16      .?.w.T.w.{.w.}.w
0010:0044CC98 77E8C0A6  77E8AF8E  77E8878A  77E8BDE8      ...w...w...w...w
0010:0044CCA8 77E94911  77E9499C  77E9138C  77E8D019      .I.w.I.w...w...w
```

Теперь, обратившись к таблице экспорта kernel32.dll, определяем: а) базовый адрес ее загрузки (в данном случае: 0x77E80000); б) имя функции, сумма RVA и IMAGE BASE которой совпадает со значением 0x77E8668C. Вычитаем из 0x77E8668C базовый адрес загрузки - 0x77E80000 и получаем: 0x668C. Ищем строку 0x668C простым контекстным поиском и…

```
302  12D 0000668C GetLastError
```

…это оказывается ни кто иной, как GetLastError, что и требовалось доказать. Конечно, восстанавливать весь импорт вручную - крайне скучно и утомительно. Но кто нам сказал, что мы должны это делать именно вручную?! Ведь дизассемблер IDA поддерживает скрипты, что позволяет автоматизировать всю рутинную работу (подробнее о языке скрпитов можно прочитать в книге "Образ мышления - дизассемблер IDA" от Криса Касперски, то есть, собственно, меня).

ОК, еще один барьер успешно взять. Воодушевленные успехом и доверху наполненные выпитым во время хака пивом, мы продолжаем! В плане возвращения к нашим баранам, сосредоточим свои усилия на загрузчике таблице импорта, расположенном по всей видимости где-то недалеко от точки входа. Несмотря на то, что soft-ice по-прежнему упорно проскакивает Entry Point, обламываясь с загрузкой защищенного файла (впрочем, другие версии soft-ice с этим справляются на ура), мы можем легко обхитрить защиту просто воткнув в точку входа бряк поинт. Поскольку, бряк поиск должен устанавливаться во вполне определенном контексте, используем уже известную нам нычку с CreateFileA. Итак, "bpx CreateFileA", , запускаем unlink и, когда soft-ice "всплывает" даем: "bpx 0x446673" (адрес точки входа), выходим из soft-ice и… запускаем ulink вновь. Отладчик тут же всплывает:

```
001B:00446673  55                PUSH    EBP
001B:00446674  68AECF4200        PUSH    0042CFAE
001B:00446679  8BDC              MOV     EBX,ESP
001B:0044667B  2403              AND     AL,03
001B:0044667D  7203              JB      00446682
001B:0044667F  FE4302            INC     BYTE PTR [EBX+02]
001B:00446682  D7                XLAT
001B:00446683  27                DAA
```

Знакомые места! Трассируем код до тех пор пока на не встретится подозрительный RETF (от RET FAR - далекий возврат), передающий управление по следующему адресу:

```
001B:77F9FB90  8B1C24            MOV     EBX,[ESP]
001B:77F9FB93  51                PUSH    ECX
001B:77F9FB94  53                PUSH    EBX
001B:77F9FB95  E886B3FEFF        CALL    77F8AF20
001B:77F9FB9A  0AC0              OR      AL,AL
001B:77F9FB9C  740C              JZ      77F9FBAA
001B:77F9FB9E  5B                POP     EBX
001B:77F9FB9F  59                POP     ECX
```

Судя по адресу, этот код принадлежит непосредственно самой операционной системе (а точнее - NTDLL.DLL) и представляет собой функцию KiUserExceptionDispatcher. Но что это за функция? Ее описание отсутствует в SDK, но поиск по MSDN обнаруживает пару статей Мета Питтрека, посвященных механизмам функционирования SEH и функции KiUserExceptionDispatcher в частности.

Структурные исключения! Ну конечно же! Какая защита обходится без них! Ладно, разберемся, ворчим мы себе под нос, продолжая трассировку защиты дальше. Увы! В той же точке, где WDB терял над программой контроль, soft-ice просто слетает. Ах, вот значит как!!! Ну, защита, держись!!!

(продолжение следует)

Защита программы Ulead COOL 3D v3.5 (trial)

Автор: Chingachguk / HI-TECH

Дата: 2003-04-13

Что собираемся делать ?

Объект исследования : программа Ulead COOL 3D, триальная защита

Инструменты : Soft-Ice (под '98), немного мозгов

Цель : изучение win32-программ, обладающих защитой от отладки

Примечание : ничего особо нового, простая техника, SEH, etc...

Ниже я собираюсь исследовать защиту этой проги, которая позволяет пользователю использовать ее функциональность только в течении календарного месяца (30 дней) или купить возможность ее использования неограниченно во времени (вид входного окошка после инсталляции, инсталлятор UC3D35TBYB_E.exe):



Что она вообще делает ? Примерно - это какой-то графический редактор (вид после нажатия кнопки "TRY"):



Таким образом в первом приближении сразу после инсталляции данная программа позволяет полноценно работать, но в стартовом окошке напоминает о том, что до конца использования осталось 30 дней ("Remaining Days - 30") и предлагает, видимо, купить разрешение на полное использование ("BUY").

Поисследуем реакцию программы на изменение системной даты. Поменяем текущую дату на один день вперед:



Ага, программа предупреждает нас, что осталось на 1 день меньше. Если перевести дату ровно на месяц вперед, то кнопка "TRY" становится недоступна и число оставшихся дней равно 0:



Кнопка "TRY" недоступна. Попробуем поймать программу на том, как она закрывает для доступа кнопку и поисследовать, чем она при этом руководствуется. Win32-приложение обычно делает это при помощи API-функции EnableWindow:

EnableWindow(Id_Объекта_Окна,0/1-закрыть/открыть Объект)

Устанавливаем на машине дату таковой, чтобы программа решила, что осталось 0 дней для использования, устанавливаем breakpoint в SoftIce:

bpx EnableWindow

запускаем программу, получаем break и видим в отладчике место вызова (оказалось, код принадлежит некоей xsystem.dll, которую можно найти в том каталоге, куда инсталлировалась программа Ulead COOL 3D):

```
026D7F5F:
  mov  edi,User32!EnableWindow
  push edi
  mov  edi,esp
  pushad
  lea  esi,[ebp+2C63h]
  lodsb
  cmp  al,90h
  jnz  026D7F8A
  push 03E8h
  push dword ptr [edi+8]
  call [ebp+2B50h]
  push 0 ; FALSE - disable button "TRY"
  push eax
  call [ebp+2B54h] ; Точка вызова EnableWindow
026D7F8A:
  popad
  ...
```

Итак, программа анализирует первый байт некоторой структуры (lea esi,[ebp+2C63h], lodsb, cmp al,90h), сравнивает его с 90h, если он оказывается таковым, то кнопка "TRY" становится недоступной (переход на метку 026D7F8A). Попробуем проверить гипотезу о том, что все, что нужно для "освобождения" программы от проверки даты - это сделать прямо здесь достпуной кнопку "TRY". Попробуем сделать это прямо в отладчике: установим break на адрес 026D7F5F:

bpx LoadLibraryA ; Ждем загрузки xsystem.dll

...

bc 0 ; clear breakpoint at LoadLibraryA

bpx 026D7F5F ; Set breakpoint

Ctrl+D...

и попытаемся дождаться срабатывания break'а и "руками" поменять в отладчике значение регистра al сразу после lodsb: но вместо того, чтобы получить break на этом (026D7F5F) адресе, мы получаем GPF... Видно (в Sice'e), что машина пытается исполнять какой-то мусор в xsystem.dll.

...Так мы выяснили, что программа не только контролирует системную дату, но и сопротивляется нашим попыткам ее трейсить ;) Наверное, программа как-то динамически модифицирует свой код - например по адресу 026D7F5F - и break, который мы пытаемся установить, мы устанавливаем в то место, которое впоследствии будет изменено. Скажем, самораспаковывающийся или саморасшифровывающийся код может быть причиной тому. Попробуем как-то обойти это ограничение и вспоминаем про то, что программа неминуемо должна узнать системную дату - ведь ей необходимо ее сравнить с чем-то типа даты инсталляции. Win32-приложение обычно делает это при помощи API-функции GetSystemTime:

GetSystemTime(offset структуры типа SYSTEMTIME)

```
SYSTEMTIME STRUCT
  wYear             WORD      ?
  wMonth            WORD      ?
  wDayOfWeek        WORD      ?
  wDay              WORD      ?
  wHour             WORD      ?
```

```
  wMinute           WORD      ?
  wSecond           WORD      ?
  wMilliseconds     WORD      ?
SYSTEMTIME ENDS
```

Устанавливаем break на эту API-функцию:

bpx GetSystemTime

И обнаруживаем, что вызов произошел из следующего места (опять xsystem.dll):

```
026D6DE3:
  lea  esi,[ebp-100h]
  push esi
  call [ebp-4] ; call GetSystemTime
  pop  ebx
026D6DEE:
  movzx edx,word ptr [esi] ; d esi -> D3 07 03 00 05 00 1C
  ; edx= 07D3h
  shl  edx,8
  mov  dl,[esi+02] ; Месяц
  shl  edx,8
  mov  dl,[esi+06] ; День
  ; edx=07D3031C
  push edx
```

Уже видно, как программа работает с датой: код по адресу ~026D6DE3 получает косвенным call'ом дату от Win, собирает в 32 битах edx год (07D3h=2003), месяц (03) и день (1Ch=28) - 28 марта 2003 года в данном случае, делает что-то еще и управление попадает к коду в ~026D7F5F, который уже знает, в валидном ли интервале дат мы находимся и закрывает кнопку "TRY", если это не так. Но мы помним, что нужно все же попробовать подменить результат манипуляций с датой прямо перед принятием решения о disable кнопки "TRY". Ранее нам сделать это не удалось - видимо, после загрузки dll в память ее (?-пока не знаем) код самомодифицируется и мешает ставить полноценные break'и. Однако есть надежда, что в момент выполнения кода по адресу ~026D6DE3 - кода, спрашивающего дату у Win - код в ~026D7F5F расшифрован и доступен для отладки. Повторяем попытку установить break на 026D7F5F, но уже после того, как мы попали в 026D6DE3 и опять получаем GPF ! Почему, ведь код уже явно виден в отладчике ? Может, дело в том, что адрес API EnableWindow прописывается динамически прямо в команду mov edi,const_addr ? Пробуем установить break на команду загрузки регистра esi адресом структуры (lea esi,[ebp+2C63h]) и ... оказываемся в нужном месте:

```
026D7F5F:
  mov  edi,User32!EnableWindow
  ...
026D7F68:
  lea  esi,[ebp+2C63h]
  lodsb ; al = 90h если запрещенный период, иначе всегда 6Ah
  cmp  al,90h
  jnz  026D7F8A
  push 03E8h
  push dword ptr [edi+8]
  call [ebp+2B50h]
  push 0 ; FALSE - disable button "TRY"
  push eax
  call [ebp+2B54h] ; Точка вызова EnableWindow
026D7F8A:
  popad
  ...
```

Ура !? Что ж, проверим: модифицируем (в отладчике) регистр al после лодсб и отпускаем программу...



...Так, кнопка "TRY" доступна (хотя Remaining Days = 0). Жмем ее и видим всего лишь вот этот MessageBox:



Вот и познакомились ;) Оказывается, защиту зовут "xLok". А разрешить жать на кнопку - не значит получить доступ к функциональности ;(

Но что если данными для проверки валидности даты для кода в ~026D7F5F является не байт, а слово или даже что-то большее ? На такую мысль может навести пара команд "lea esi,[ebp+2C63h], lodsb" вместо "cmp byte ptr [ebp+2C63h],90h", хотя это может быть "приколом" компиллятора. Внимательнее поизучаем байты в структуре, адресуемой по [ebp+2C63h] для двух дат: для даты в интервале работы и вне ее:

```
Работа возможна:  6A 01 FF 73 08 FF 95 54 ...
Работа запрещена: 90 90 FF 73 08 E8 9D 00 ...
```

Установим дату вне интервала работы, опять остановимся в отладчике в точке 026D7F68, поменяем "неверные" байты на "верные" (90 90 FF ... -> 6A 01 FF ...) и "отпустим" программу. MessageBox'a от xLok'a больше нет, но есть банальный GPF...

Следовательно, с большой вероятностью можно заключить, что защита выполняет (динамически) расшифровку какой-то части рабочего кода и снятие всех ее проверок после расшифровки приводит к неизбежному краху, так как этот код расшифровывается неверно вне интервала работы программы. Видимо, защита использует дату (точнее, не саму дату, а некоторую функцию от нее, которая постоянна на интервале работы программы) как ключ шифрования части своего кода.

Проверим это: в этом случае будет достаточно подменить системную дату, которую защита спрашивает у Win. В отладчике это легко сделать: опять устанавливаем break на GetSystemTime, возвращаемся в xsystem.dll к коду в 026D6DEE и "патчим" значение структуры SYSTEMTIME (или значение edx чуть ниже) датой инсталляции... Все OK, программа отлично работает и ничего не запрещает.

Здорово, вроде бы осталось поменять команды инициализации edx (код в 026D6DEE и ниже) на что-то вроде таких:

```
026D6DEE:
; Было так:
; movzx edx,word ptr [esi] ; d esi -> D3 07 03 00 05 00 1C
; shl  edx,8
; mov  dl,[esi+02] ; Месяц
; shl  edx,8
;  mov  dl,[esi+06] ; День
; А будет так:
  mov  edx,Const_Valid_Date
  nop
  ...
  nop
  push edx
```

и все будет работать ? Пробуем найти опкоды оригинальных инструкций в DLL:

```
  Команды             Опкод
movzx edx,w [esi]    0F B7 16
shl   edx,8          C1 E2 08
mov   dl,[esi+2]     8A 56 02
...
```

и не находим их ! Этого следовало ожидать еще тогда, когда стало ясно, что код самомодифицирующийся. Что делать дальше ? Может быть, имеет смысл попробовать найти расшифровшик, проанализировать его алгоритм и "пропатчить" нужные байты в соответствии с алгоритмом ? Пусть алгоритм дешифровщика состоит в наложении xor-последовательности на байты DLL:

```
  Команды             Опкод    Шифро-Опкод в DLL  (Опкод) xor (Шифро-Опкод в DLL)
movzx edx,w [esi]    0F B7 16     B5 2A 1A              BA 9D 0C
shl   edx,8          C1 E2 08     C4 45 2E              05 A7 26
...
```

В этом случае не составило бы труда подготовить те "новые" опкоды, которые мы собираемся поместить на место старых - нужно выполнить над ними операцию с той же xor-последовательностью - как это делает пока неизвестный нам расшифровщик:

```
  Команды             Опкод_новый   (Опкод) xor (Шифро-Опкод в DLL) Результат
mov edx,07D3031Ch    BA 1C 03 D3 07       BA 9D 0C 05 A7             00 81 0F...
nop
...
```

Помещаем в файл xsystem.DLL по смещению (в файле) 6DEE подготовленные таким образом байты и пытаемся смотреть, что же сделал с ними расшифровщик - а в результате видим такой вот MessageBox:



Да, защите удалось не только скрыть результат работы расшифровщика (если он вообще отработал) но и определить модификацию собственного кода. Неплохо ! Как же ему это удалось ? Попробуем "отловить" его на обращении к памяти, содержащей измененные коды:

bpm 026D6DEE

и, действительно, наблюдаем следующий код:

```
026D510C:
  push esi
  push ecx
  push eax
  ...
026D5115:
  mov  ecx,49A0h
  xor  edx,edx
  xor  eax,eax
@@GetCRC:
  lodsb ; esi=026D6DEFh
  add  edx,eax
  dec  ecx
  jnz  @@GetCRC
  ...
  xchg edx,eax ; eax->1FBFFAh
  pop  edx
  pop  ecx
  pop  esi
  add  dword ptr [esp],5
026B515A:
  ret
```

Очень интересный код ! Похоже на подсчет CRC (возвращает в eax), причем регион подсчета CRC включает и критичные для нас адреса в ~026D6DEE. Следовательно, прежде чем воевать с их расшифровщиком, нам нужно победить подсчет контрольной суммы. Открыты ли байты подпрограммы подсчета CRC в DLL ?

```
026D510C:
  Команда / Опкод
  push esi / 56h
  push ecx / 51h
  push eax / 52h
  ...
```

Ищем их в файле DLL по смещению 510Ch и находим ! Следовательно, мы можем без особой боязни заменить оригинальную подпрограмму подсчета CRC на что-то вроде:

```
@@NewCRCProc:
  mov  eax,1FBFFAh
  add  dword ptr [esp],5
  ret
@@EndOfNewCRC:
  db (026D515Bh-026D510Ch) - (@@EndOfNewCRC-@@NewCRCProc) dup(90h)
@@NewCRCProcDone:
```

При этом мы вернем нужное где-то дальше CRC (в eax) без всякого подсчета - что зря процессор гонять, хитро вернемся назад (add dword ptr [esp],5) и забьем все остальное место nop'ами. Патчим таким образом DLL и получаем ... GPF ! Оба-на, защита перестала детектировать сосбтвенную модификацию но и программа "рухнула" ;(Неужели мы неправильно составили новый код подсчета CRC ?... Стоп, а ведь мы еще раньше поменяли байты, относящиеся к получению даты (в ~026D6DEE) ! Установим break на API GetSystemTime, вернемся к коду получения даты в DLL и видим, что расшифровщик явно сделал не то, что мы ожидали:

```
  Команды_новые   Результат работы расшифровщика / Опкод
026D6DEE:
mov edx,07D3031Ch   sbb  byte ptr [esi] / 1D 82 1E 1D
nop                 mov  bl,7Fh / B3 7F
...
```

Следовательно, алгоритм расшифровщика производит расшифровку байт в ~026D6DEE в зависимости от исходных байт. Т.е. это не простое наложение XOR-последовательности; расшифровка каждого байта зависит от того, как был расшифрованы все предидущие. Алгоритм расшифровщика нам по-прежнему неизвестен и мы не можем даже пытаться осуществлять атаку на его шифр. Откатим пока эти изменения, оставив лишь получение CRC и продолжим следить в отладчике за ходом выполнения программы защиты после возврата из подпрограммы подсчета CRC:

```
026D5366:
  sub  eax,1FBfFAh ; если пришло верное CRC=1FBfFAh, то в eax будет 0
  pop  ebx
  jz   026D5382
  ...
026D5382:
  mov  [ebp+3C16h],ebx
  mov  edx,[ebp+3C36h]
  push dword ptr fs:[0]
  mov  fs:[0],esp
  push eax
  push ebp
  jmp  026D53A1
  ...
026D53A1:
  pushfd
  invalid (0F1h)
  db 0BEh,0F0h,...
```

Если продолжить в отладчике пошагово трассировать код с 026D53A2, то налетаешь на GPF. Но сразу бросаются в глаза явно вычурные для win32-кода команды работы с сегментными регистрами (mov fs:[0],esp). Если уж приложение взялось за это, то явно не с добрыми намерениями ;) На самом деле таким образом в windows пользовательским программам дозволяется установить свой обработчик исключительных ситуаций и в случае таковых получить управление и попытаться их обработать (интересно, кому-нибудь удавалось обрабатывать их инчае как выдать сообщение "Программа выполнила ... и будет закрыта" ?...). Далее мы увидим, что иногда такой обработчик может выполнять некоторые "полезные" действия.

В двух словах о таких обработчиках. Эта штука называется "SEH" - structure exeption handler. Для того, чтобы обработчик правильно получил управление, необходимо:

• поместить в стек смещение обработчика;
• сохранить смещение старого обработчика на том же стеке;
• записать по селектору из FS, смещению 0 содержимое esp

Сам обработчик вызвается в C-формате, при этом получая кучу параметров - указатель на структуру регистров в момент исключения и т.п.:

```
SEHproc proc C pExcept: dword, pFrame: dword, pContext: dword, pDispatch: dword
    PrintException pExcept
```

```
    ...
    ret
SEHproc endp
```

Но самое главное - это то, что он может вернуться в достаточно произвольное место кода (включая и вызвавшее исключение), при этом уведомив диспечер о том, что исключение им обработано (например, mov eax, ExceptionContinueExecution перед выходом) и то, что он имеет право записи в сегмент команд (это необходимо для исправления "дефектного" кода). Таким образом, обработчик исключения защиты явно собирается именно этим заниматься: подать управление на неверный опкод, поймать exeption, дешифровать некоторые байты, вернуться опять на неверный опкод (не обязательно первый), и так до тех пор, пока необходимый код не будет полностью расшифрован.

Где же защита разместила свой обработчик ? Нет нужды копаться в командах, предшествующих коду установки SEH (~026D5382), можно просто посмотреть стек в этот момент (скажем, после выполнения mov fs:[0],esp):

d esp

Оказывается, адрес у обработчика равен 026D5040, а вот и он сам:

```
026D5040:
  pushad
  mov  edx,0C8EC918Bh
  call 026D504B
026D504B:
  pop  edi ; edi-> 026D504B, получение текущего смещения
  mov  esi,36h
  add  esi,edi ; esi=026D5081h
  mov  ecx,94h
  push esi
  sub  esi,9
  mov  edi,esi
  xor  eax,eax
026D5061:
  lodsb
  xor  eax,edx
  rol  edx,5
  imul edx,edx,0FB712715h
  add  eax,0AB358CDFh
  stosb
  dec  ecx
  jnz  026D5061
026D5078:
  db 0F4h,041h,0ECh,076h,03Ah...
026D5081:
  ...
```

Итак, обработчик SEH сразу после получения управления начинает что-то расшифровывать с адреса 026D5078 и ниже довольно незамысловатым алгоритмом, немного странно реализованным (вместо add eax,0AB358CDFh достаточно add al,0DFh). Размер байт для дешифровки невелик (mov ecx,94h). Дожидаемся возможности установить break в 026D5078 - после первого же stosb там появлется pop esi и после срабатывания break'a видим весь расшифрованный код:

```
026D5078:
  pop  esi
  mov  eax,[esp+28h]
  mov  [eax+4],esi ; <-026D5081h
  popad
026D5081:
  push ebp
  mov  ebp,esp
  push esi,edi,ebx,ecx,edx
  mov  eax,[ebp+10h]
  mov  edi,[eax+0B8h]
026D5093:
  mov  edx,2
026D5097:
```

```
  jmp  026D5099
026D5099:
  call 026D509E
026D509E:
  pop  ebx ; ebx<-текущее смещение 026D509E
  add  ebx,0FFFFFFE3h ; ebx=026D5081h
  add  edx,[eax+0B4h]
  mov  [ebx+12h],edx
  ...
  mov  ebx,0BB51B5E3h
  test esi,esi
  jz   026D50CE
  xor  [esi],bl
026D50CE:
  cmp  byte ptr [edi],09Dh
  jz   026D50DF
  xor  [edi],bl
  or   dword ptr [eax+0C0h],100h
026D50DF:
  xor  ebx,ebx
  mov  [eax+4],ebx, mov  [eax+8],ebx, mov  [eax+10h],ebx
  mov  dword ptr [eax+18h],101h
  mov  [edx],edi
  xor  eax,eax
  push eax, push eax, dec eax, push eax, mov eax,ofs Kernel!FlushInstructionCashe
  call eax
  xor  eax,eax
  pop  edx,ecx,ebx,edi,esi,ebp
  ret  ; Последняя расшифрованная команда
026D510Ch:
```

Что же любопытного в раскрывшемся коде ? Прежде всего обратим внимание на характерный код в 026D5081 - push ebp, mov ebp,esp. Это явно оформлено начало какой-то процедуры, и она заканчивается в 026D510B. Командами

```
  mov  eax,[esp+28h]
  mov  [eax+4],esi ; <-026D5081h
```

декриптор определил адрес **нового** обработчика SEH - теперь это будет код в 026D5081. При следующих исключениях всегда будет вызываться именно эта процедура, что-то расшифровывающая командами:

```
  xor  [esi],bl
026D50CE:
  cmp  byte ptr [edi],09Dh
  jz   026D50DF
  xor  [edi],bl
  or   dword ptr [eax+0C0h],100h
026D50DF:
```

Но в данный момент код получения даты (в ~026D6DEE) еще не расшифрован. Может быть, только что расшифрованная SEH-подпрограмма 026D5081 и есть ее декриптор ? Устанавливаем break на начало 026D5081:

bpx 026D5081

и наблюдаем, как раз за разом управление переходит к SEH-обработчику. Он явно расшифровывает какой-то код и хотелось бы остановить программу в тот момент, когда он полностью закончит свою работу. Довольно утомительно было бы жать Ctrl-D столько раз, поэтому я написал небольшой код, заменяющий собой распаковщик SEH-обработчика:

```
; Новые команды по адресу 026D5040 (первоначальный SEH-обработчик)
@@NewBytes:
  pushad
  call @@GetOfsNewBytes
@@GetOfsNewBytes:
  pop  esi
  add  esi,offset @@ArtBytes - @@GetOfsNewBytes
```

```
  mov  eax,[esp+28h]
  mov  [eax+4],esi
  popad
  db   0EBh ; jmp на адрес 026D5081
  db   (@@NewBytes+40h) - $
@@ArtBytes:
  pushad
  call @@GetOfs
@@GetOfs:
  pop  eax
  inc  word ptr [eax+(offset CounterCalls - offset @@GetOfs)]
  mov  ax,word ptr [eax+(offset CounterCalls - offset @@GetOfs)]
  popad
  db   0EBh ; jmp на адрес 026D5081
  db   (@@NewBytes+40h) - $
CounterCalls dw   0
@@DoneNewBytes:
LastNops db (@@DoneOldBytes - @@OldBytes) dup (90h) ; nop only
```

Новый код в отличии от старого не декриптует SEH-обработчик (это можно сделать прямо в файле xsystem.dll и не обременять код графического редактора дешифрацией), а устанавливает новый адрес SEH-обработчика - это будет подпрогрммка @@ArtBytes, которая только лишь вычисляет число call'ов SEH'а и передает jmp'ом управление на старый адрес 026D5081. Теперь в Sice'е можно ставить условный break на команде mov ax,word ptr [eax+(offset CounterCalls - offset @@GetOfs)]:

bpx IF (AX==0x100)

Заранее, конечно, число call'ов SEH'а неизвестно, но простым методом деления "отрезка" пополам можно довольно быстро добраться до конца работы SEH-обработчика. Трейсим последний вызов по SEH-обработчика и видим, что:

- Даже когда он закончил свою работу, код получения даты (в ~026D6DEE) нерасшифрован;

- Последний свой xor SEH-обработчик выполнил с адресом 026D540Ch:

```
  xor  [esi],bl ; edi=026D540Ch, там 75h, bl=3Eh
026D50CE:
  cmp  byte ptr [edi],09Dh
  ...
```

Таким образом, SEH-обработчик-дешифровщик не расшифровал код получения даты. Но что за байты появились по адресу 026D540Ch ?...

```
026D540C:
  popfd
  call 026D5412 ; get current offset
026D5412:
  pop  esi
  add  esi,02Eh
  mov  edi,esi
  mov  ecx,0CF9h
  mov  edx,9548E9F7h
026D5425:
  lodsd
  xor  eax,edx
  add  eax,54ADF121h
  xor  eax,ecx
  rol  edx,5
  imul edx,edx,70C967BEh
  xor  edx,ecx
  stosd
  dec  ecx
  jnz  026D5425
  jmp  026D5450
026D5440:
  ...
```

О, старые знакомые ! :) Опять переносимый (call 026D5412, pop esi) код дешифрует нижележащие байты. Ставим break в конец цикла, дожидаемся его окончания и с нетерпением смотрим ... нет, не байты после 026D5440, а байты получения даты - ~026D6DEE. Они расшифрованы, теперь - ура - мы знаем кто их декриптует и этот кто-то (026D540C - последний дешифровщик) крайне мал, чтобы возится с его расшифровщиком (обработчик SEH'a) - проще вбить его, уже нам известный на то же место, где сейчас лежит его зашифрованный собрат. Только тогда нужно будет дезактивировать его расшифровщик (обработчик SEH'a) - а то он все испортит ;) Как же это сделать ? Вернемся к моменту инсталляции SEH'a:

```
  ...
026D5382:
  mov  [ebp+3C16h],ebx
  mov  edx,[ebp+3C36h]
  push dword ptr fs:[0]
  mov  fs:[0],esp
  push eax
  push ebp
  jmp  026D53A1
  ...
026D53A1:
  pushfd
  invalid (0F1h)
  db 0BEh,0F0h,...
```

Что будет, если вместо invalid-команды, намеренно сделанной для генерации исключения, сделать переход на дешифровщик в 026D540C, который теперь, как мы планируем, будет записан нами в открытом виде прямо в DLL ? SEH-обработчик явно не вызовется, осталось опасность потерять стек при таком прыжке. Но парная команда к pushfd - последняя команда перед вызовом SEH-обработчика через исключение - popfd:

```
026D540C:
  popfd
  call 026D5412 ; get current offset
  ...
```

как будто бы явно написана разработчиками защиты с целью восстановить состояние процесса после отработки SEH'a. Так и поступим. Остался последний момент: где разместить код, который поменяет пресловутые команды получения даты на одну спокойную mov edx,Const_Valid_Date и парочку nop'ов ?

Ниже привожу свое решение, возможно немного надуманное. Поскольку этот код хотелось бы разместить после отработки расшифровщика 026D540C, т.е. тогда, когда уже код от 026D5440 и далее расшифрован, то он должен быть после последней команды расшифровывающего цикла:

```
026D540C:
  ...
  dec  ecx
  jnz  026D5425
  ; Здесь должен быть код, вставляющий команду mov edx,Const_Valid_Date
  jmp  026D5450
026D5440:
  ...
```

Вместо двух байт короткого jmp'a сложновато сделать такое, но если немного пооптимизировать код расшифровщика и вспомнить, что после удаления старого кода подсчета CRC осталась куча места, то можно написать следующее:

Этой командой мы закроем выполнение SEH'a:

```
; Этот jump мы поместим на место всяких invalid'ов, чтобы SEH не вызывался
; Сам SEH-обработчик оставим в покое
@@NewJumpAt53A2:
  jmp  $+(540Ch-53A2h)
@@NewJumpAt53A2Done:
```

Это код нового, открытого декриптора по адресу 026D540C:

```
@@Decryptor2:
  popfd
  call @@GetOfsDecryptor2
@@GetOfsDecryptor2:
  pop  esi
  ;; db   081h,0C6h,02Eh,000h,000h,000h ; add  esi,2Eh - в оригинале
  ;; был именно вариант из 6-ти байт
  add  esi,2Eh
  mov  edi,esi
  mov  ecx,0CF9h
  mov  edx,9548E9F7h
@@DecryptLast:
  lodsd
  xor  eax,edx
  add  eax,54ADF121h
  xor  eax,ecx
  rol  edx,5
  imul edx,edx,70C967BEh
  xor  edx,ecx
  stosd
;;  dec  ecx
;;  jnz  @@DecryptLast
  loop @@DecryptLast
  ; Переходим на кусок кода, который разместили внутри подпрограммы
  ; подсчета CRC
  mov  si,510Ch+(offset @@PatchAt026D6DEE-offset @@NewCRCProc)
  jmp  esi
@@Decryptor2Done:
```

Это новый код подсчета CRC вместе с подпрограммой модификации кода получения даты в 026D6DEE:

```
;
; New subprogram 026D510Ch - calculate CRC + patch code at 026D6DEE
;
@@NewCRCProc:
  mov  eax,1FBFFAh
  add  dword ptr [esp],5
  ret
@@PatchAt026D6DEE:
;
; Необходимо:
; 1. Пропатчить байты в ~026D6DEE - подменить дату
; 2. Пропатчить байты в ~026D6E0B - подменить дату для вычисления Remaining Days
; 3. Вернуться в 026D5450
;
; Сделать mov  edx,07D3031Ch ; Year + Month + Day -> 0BAh,01Ch,003h,0D3h,007h
  mov  si,6DF7h
  mov  byte ptr [esi],0BAh ; mov edx,...
@@PatchCommand:
  mov  dword ptr [esi+1],07D3031Ch ; Year + Month + Day
  mov  byte ptr [esi+5],90h ; nop
; Пропатчить получение стратовой даты для вычисления Remaining Days -
; - сделать mov  edx,07D3031Ch вместо xor eax,7895ADEBh по адресу 6E0Bh
  mov  si,6E0Bh
  mov  byte ptr [esi],0B8h ; mov eax,...
@@PatchCommandRemainingDays:
  mov  dword ptr [esi+1],07D3031Ch ; Year + Month + Day
; Возвращаемся
  mov  si,5450h
  jmp  esi
```

Вот вроде бы и все ;) Хотя далее есть еще несколько пугающих кусков типа:

```
026D6C52:
  lea  esi,[ebx-100h]
  ...
  call 026D7623
  int  62h ; !!!
```

```
add  eax,0FCfC4C4h
...
```

которые заставили меня лишний раз заглянуть в Ральфа Брауна, но особой опасности они не представляют ;)

Немного извращений или граффити

После реализации подмены системной даты и многократных смен системной даты в целях выяснинения особенностей защиты программы можно наблюдать интересный эффект:

xsystem.dll уже изменена, все работает, но вот с числом оставшихся дней что-то не то. Как бы не мешает, но некрасиво... Любопытен тот факт, что обратная замена исходной DLL дела не меняет: число оставшихся дней меняется, но как-то "криво". Есть и такой "баг": если даже взять нетронутую версию программы, перевести дату на некоторое число дней вперед, запустить программу, а следом вернуть дату на место, то при первом запуске программа не позволяет работать (число оставшихся дней равно 0, кнопка "TRY" недоступна).

Анализ кода xsystem.dll показал, что после того, как системная дата получена программой, выполняется следующий код:

```
026D6E03:
  mov  ecx,eax
  mov  eax,[ebp+3202h]
  xor  eax,7895ADEBh
  test eax,eax
  ...
```

- из некоторой области памяти достается число, которое "расшифровывается" xor-ом и далее (код здесь не приводится) трактуется как дата начала работы программы (инсталляции). Поэтому для завершенности изменений в программе можно поменять команду "xor eax,7895ADEBh" на "mov eax,Const_Valid_Date". В этом случае программа будет выдавать 30 дней до конца trial'a. Но можно немного поизвращаться и оставить число Remaining Days вот таким:



Исходный текст патчера

Ниже приводится текст почти полный текст программы, выполняющей описанные выше действия. Некоторые вспомогательные подпрограммы типа GetCL пропущены.

```
; ***
; Патч проги Ulead COOL 3D 3.5 ;)
; ***
.386
.model flat,stdcall
option casemap :none   ; case sensitive
include \masm32\include\windows.inc
include \masm32\include\kernel32.inc
includelib \masm32\lib\kernel32.lib

; -----------------------------------
; Read text at end of module for usage
  GetCL PROTO :DWORD,:DWORD
; -----------------------------------
.data?
bRead         dd ? ; Байт прочитано
bWrite        dd ? ; Байт записано
hConsole dd ?
.code
start:
  cld
; Получаем хэндл консоли
  invoke GetStdHandle,STD_OUTPUT_HANDLE
  mov  dword ptr hConsole,eax
; ***
; Открываем лог
; ***
.data
LogFileName   db "patch_xsystem01.log",0
.data?
LogDescr      dd ?
.code
; Открыть лог, если есть он уже
  invoke CreateFile,offset LogFileName,GENERIC_WRITE,FILE_SHARE_WRITE+FILE_SHARE_READ,\
      0,OPEN_EXISTING,FILE_ATTRIBUTE_NORMAL,0
  mov  LogDescr,eax          ; Save file descriptor
  inc  eax
  jnz  @@LogFileOpen
; Видимо, нет. Тогда попробовать создать
  invoke CreateFile,offset LogFileName,GENERIC_WRITE,FILE_SHARE_WRITE+FILE_SHARE_READ,\
      0,CREATE_ALWAYS,FILE_ATTRIBUTE_NORMAL,0
  mov  LogDescr,eax          ; Save file descriptor
  inc  eax
  jnz  @@LogFileOpen
.data
OpenErrorMsg db "Log file not created !",0Ah,0
.code
  push offset OpenErrorMsg
  call Write_Log
  jmp  Exit
@@LogFileOpen:
; Встать в конец лога
  invoke SetFilePointer,LogDescr,0,NULL,FILE_END

  jmp  @@SkipBytes

;
; New subprogram 026D510Ch - calculate CRC + patch code at 026D6DEE
;

@@NewCRCProc:
  mov  eax,1FBFFAh
  add  dword ptr [esp],5
  ret
@@PatchAt026D6DEE:
;
; Необходимо:
; 1. Пропатчить байты в ~026D6DEE - подменить дату
```

```
; 2. Вернуться в 026D5450
;
; Сделать mov  edx,07D3031Ch ; Year + Month + Day -> 0BAh,01Ch,003h,0D3h,007h
  mov  si,6DF7h
  mov  byte ptr [esi],0BAh ; mov edx,...
@@PatchCommand:
  mov  dword ptr [esi+1],07D3031Ch ; Year + Month + Day
  mov  byte ptr [esi+5],90h ; nop
; Пропатчить получение стратовой даты для вычисления Remaining Days -
; - сделать mov  edx,07D3031Ch вместо xor eax,7895ADEBh по адресу 6E0Bh
  mov  si,6E0Bh
  mov  byte ptr [esi],0B8h ; mov eax,...
@@PatchCommandRemainingDays:
  mov  dword ptr [esi+1],07D3031Ch ; Year + Month + Day
; Возвращаемся
  mov  si,5450h
  jmp  esi

@@EndOfNewCRC:
  db (026D515Bh-026D510Ch) - (@@EndOfNewCRC-@@NewCRCProc) dup(90h)
@@NewCRCProcDone:

@@Decryptor2:
  popfd
  call @@GetOfsDecryptor2
@@GetOfsDecryptor2:
  pop  esi
  ;; db   081h,0C6h,02Eh,000h,000h,000h ; add  esi,2Eh
  add  esi,2Eh
  mov  edi,esi
  mov  ecx,0CF9h
  mov  edx,9548E9F7h
@@DecryptLast:
  lodsd
  xor  eax,edx
  add  eax,54ADF121h
  xor  eax,ecx
  rol  edx,5
  imul edx,edx,70C967BEh
  xor  edx,ecx
  stosd
;;  dec  ecx
;;  jnz  @@DecryptLast
  loop @@DecryptLast
;;  jmp  $+(5450h-543Eh)
  mov  si,510Ch+(offset @@PatchAt026D6DEE-offset @@NewCRCProc)
  jmp  esi
@@Decryptor2Done:

@@NewJumpAt53A2:
  jmp  $+(540Ch-53A2h)
@@NewJumpAt53A2Done:

@@SkipBytes:

;
; Читаем дату, которую надо будет записать в файл
;

.data
Year  dw 2003
Month db 03
Day   db 28
.data?
PatchDate db 128 dup(?)
.code
  invoke GetCL,1,offset PatchDate
  cmp  eax,1
  jz   @@DatePresent
.data
NoDateMsg db 0Ah,"No specifed date in command line ! Patch with 2003/03/28",0Ah,0
```

```
.code
@@NoDateSpec:
  push offset NoDateMsg
  call Write_Log
  jmp  @@DateDone
@@DatePresent:
  mov  esi,offset PatchDate
  call Str_Len
  test ecx,ecx
  jz   @@NoDateSpec
  cld
; Year
  lodsd
  mov  ebx,eax
  mov  ecx,4
  xor  ebp,ebp
@@GetYear:
  imul ebp,ebp,10
  movzx eax,bl
  shr  ebx,8
  sub  al,'0'
  add  ebp,eax
  loop @@GetYear
  mov  eax,ebp
  mov  Year,ax
  lodsb
; Month
  xor  eax,eax
  xor  ebx,ebx
  lodsw
  sub  ax,'00'
  mov  bl,ah
  mov  ah,10
  mul  ah
  add  bl,al
  mov  Month,bl
  lodsb
; Day
  xor  eax,eax
  xor  ebx,ebx
  lodsw
  sub  ax,'00'
  mov  bl,ah
  mov  ah,10
  mul  ah
  add  bl,al
  mov  Day,bl
; Display patched date
.data
PatchDateHead db 0Ah,"Patch date: "
PatchDateStr  db "????/??/??",0Ah,0h
.code
  movzx eax,word ptr Year
  mov  ebx,1000
  mov  edi,offset PatchDateStr
  call DecChar
  movzx eax,byte ptr Month
  mov  ebx,10
  mov  edi,offset PatchDateStr+5
  call DecChar
  movzx eax,byte ptr Day
  mov  ebx,10
  mov  edi,offset PatchDateStr+8
  call DecChar
  push offset PatchDateHead
  call Write_Log

@@DateDone:

;
; Открываем DLL
```

```
;
.data
IniFileName db "xsystem.dll",0
.data?
IniDescr  dd ?
.code
  invoke CreateFile,offset IniFileName,GENERIC_WRITE or GENERIC_READ,\
    FILE_SHARE_READ,0,OPEN_EXISTING,FILE_ATTRIBUTE_NORMAL,0
  mov  IniDescr,eax          ; Save file descriptor
  cmp  eax,0ffffffffh
  jnz  @@DestFileOpen
.data
DestFileNotOpenMsg db "xsystem.dll not open (not exist ?)! Process aborted !",0Ah,0
.code
  push offset DestFileNotOpenMsg
  call Write_Log
  jmp  Exit
@@DestFileOpen:

;
; Патчим DLL
;

;
; Новая процедура без CRC
;

; Встать в нужное место
  invoke SetFilePointer,IniDescr,0510Ch,NULL,FILE_BEGIN
; Копируем новый код в сегмент данных для установки даты
.data?
Data_@@NewCRCProc db (@@NewCRCProcDone-@@NewCRCProc) dup(?)
.code
  mov  ecx,@@NewCRCProcDone-@@NewCRCProc
  mov  esi,offset @@NewCRCProc
  mov  edi,offset Data_@@NewCRCProc
  lea  ebx,[edi+(@@PatchCommand-@@NewCRCProc)+3]
  lea  edx,[edi+(@@PatchCommandRemainingDays-@@NewCRCProc)+3]
  rep  movsb
  mov  ax,Year
  shl  eax,16
  mov  ah,Month
  mov  al,Day
  mov  [ebx],eax
.data
YearCurr  dw 2005
MonthCurr db 04
DayCurr   db 13
.code
  mov  ax,YearCurr
  shl  eax,16
  mov  ah,MonthCurr
  mov  al,DayCurr
  mov  [edx],eax
; Записываем новый код подсчета CRC
  invoke WriteFile,IniDescr,offset Data_@@NewCRCProc,(026D515Bh-026D510Ch),offset bWrite,NULL
  cmp  dword ptr bWrite,(026D515Bh-026D510Ch)
  jz  @@PatchCRCOK
.data
DestFileNotPatchedCRCMsg db "xsystem.dll (CRC) not patched: I/O error !",0Ah,0
.code
  push offset DestFileNotPatchedCRCMsg
  call Write_Log
@@PatchCRCOK:

;
; Записать декриптор2 в незашифрованном виде (в 026D540Ch)
;

; Встать в нужное место
  invoke SetFilePointer,IniDescr,0540Ch,NULL,FILE_BEGIN
```

```
  invoke WriteFile,IniDescr,offset @@Decryptor2,(5440h - 540Ch),offset bWrite,NULL
  cmp  dword ptr bWrite,(5440h - 540Ch)
  jz   @@WriteDec2OK
.data
DestFileNotWriteDec2Msg db "xsystem.dll: decryptor2 not writed I/O error !",0Ah,0
.code
  push offset DestFileNotWriteDec2Msg
  call Write_Log
@@WriteDec2OK:

;
; Записать переход на новый декриптор (2)
;

; Встать в нужное место
  invoke SetFilePointer,IniDescr,053A2h,NULL,FILE_BEGIN
  mov  ebp,offset @@NewJumpAt53A2Done - offset @@NewJumpAt53A2
  invoke WriteFile,IniDescr,offset @@NewJumpAt53A2,ebp,offset bWrite,NULL
  cmp  dword ptr bWrite,ebp
  jz   @@WriteDecJump2OK
.data
DestFileNotWriteDecJump2Msg db "xsystem.dll: decryptor2(jump) not writed I/O error !",0Ah,0
.code
  push offset DestFileNotWriteDecJump2Msg
  call Write_Log
@@WriteDecJump2OK:

; ***
; Выход из программы: закрытие дескрипторов и т.п.
; ***
Exit:
.data
EndWorkMsg db 0Ah,"All done... ;)",0Ah,0
.code
  push offset EndWorkMsg
  call Write_Log
; Закрыть открытые файлы
  cmp  dword ptr IniDescr,-1
  jz   @@SkipCloseIniFile
  invoke CloseHandle,IniDescr
@@SkipCloseIniFile:
; Закрыть лог
  invoke CloseHandle,LogDescr
  invoke ExitProcess,0
; ***
; Подпрограмма записи в лог и на консоль конца строки (0Ah)
; ***
Write_EndStr_ToLog proc
.data
EndStr db 0Ah,0
.code
  push offset EndStr
  call Write_Log
  ret
Write_EndStr_ToLog endp
; ***
; Подпрограмма записи в лог и на консоль одновременно
; ***
; [ebp+8] - адрес строки для записи
Write_Log proc
  push ebp
  mov  ebp,esp
; Получить длину строки
  mov  esi,dword ptr [ebp+8] ; Адрес строки
  call Str_Len
  test ecx,ecx
  jz   @@ExitWrite_Log
; Запись на консоль
  push ecx
  push NULL
  push offset bWrite
```

```
  push ecx ; Длина строки
  push dword ptr [ebp+8] ; Адрес строки
  push hConsole
  call WriteFile
  pop  ecx
; Запись в протокол
  push NULL
  push offset bWrite
  push ecx ; Длина строки
  push dword ptr [ebp+8] ; Адрес строки
  push LogDescr
  call WriteFile
@@ExitWrite_Log:
  pop  ebp
  ret  4
Write_Log endp
; ***
; Get lenght of string
; esi=addr of string; result: ecx=length
; ***
Str_Len proc
  xor  ecx,ecx
  push eax
  push esi
@@GetStrLen:
  lodsb
  test al,al
  jz   @@ExitStrLen
  inc  ecx
  jmp  @@GetStrLen
@@ExitStrLen:
  pop  esi
  pop  eax
  ret
Str_Len endp
; ************ DecChar: Подпрограмма форматирования строки из числа ***********
; eax=number to digit, edi=offset result string in format 00000000(@n[ebx])
; ebx=начальный делитель
DecChar proc
  pushad
  pushfd
  cld
@GetDec:
  xor  edx,edx
  div  ebx
  add  al,'0'
  stosb
  push edx
  mov  eax,ebx
  xor  edx,edx
  mov  ebx,10
  div  ebx
  mov  ebx,eax
  pop  eax
  test ebx,ebx
  jnz  @GetDec
  popfd
  popad
  ret
DecChar endp
 ...
```

Фрагмент из первого издания книги "Техника и философия сетевых атак"

Автор: Крис Касперски

Дата: 2003-06-11

Новый рубеж

Мир давно привык к тому, что популярные технологии вовсе не обязательно бывают хорошими. Именно так произошло и в области защиты условно-бесплатного программного обеспечения. Наибольшее распространение получила защита, основанная на регистрации клиента. Слабость этого механизма в том, что регистрационный код, генерируемый на основе имени пользователя, может быть проверен единственно возможным способом: аналогичной генерацией и последующей сверкой.

Т.е. имеются два полностью идентичных генератора - у автора в виде отдельного приложения и в защитном механизме. Таким образом, остается только извлечь из защищенного приложения эту процедуру и обеспечить удобный обмен данными с пользователем. Иначе говоря, написать собственный генератор регистрационных номеров.

Все, что способен сделать автор защиты, - затруднить анализ и извлечение защитного механизма. Первое осуществляется оригинальными приемами программирования, специальными антиотладочными приемами; а второе - "размазыванием" кода по десяткам процедур, активным использованием глобальных переменных и взаимодействия с разными фрагментами кода.

Наверное, излишне говорить, что запутывание алгоритма малоэффективно и больше похоже на "ребячество", антиотладочные приемы бессильны против современных отладчиков; кроме того, их очень трудно полноценно реализовать на языках высокого уровня.

Чаще всего нет никакой нужды тратить время на анализ генератора, когда его можно просто "выкусить" и перенести в свой код, а потом передать необходимые параметры. Однако этому легко помешать. Действительно, если организовать генератор не в виде локальной процедуры, заключающей в себе весь необходимый код, а в виде множества процедур со сложным взаимодействием и неочевидным обменом данных, то без анализа архитектуры защиты (и выделения всех относящихся к ней компонентов) копирование кода невозможно.

Поэтому предпочтительнее все же первый метод. Кроме того, он не вступает в противоречие с законодательными ограничениями, тогда как любой фрагмент чужого кода в вашей программе называется плагиатом и законом уже наказывается. Чтить же уголовный кодекс хакеры должны в первую очередь.

Рассмотрим простую реализацию данного механизма защиты на примере программы file://CD/SRC/CRACK04/Crack04.exe

До сих пор мы пользовались дизассемблером для изучения кода программ. Но это не единственный возможный подход к задаче. Не меньшим успехом у хакеров пользуются отладчики. Однако отладка более агрессивный способ исследования. Необходимо постоянно помнить, что "операция" осуществляется "вживую" и возможны любые нюансы. Антиотладочный код может "завесить" систему или сделать то, чего вы никак не ожидаете. Однако становятся доступными многие возможности, о реализации которых в дизассемблерах можно только мечтать. Например, контрольные точки останова, которыми мы чуть позже с успехом и воспользуемся.

Самым популярным на сегодняшний день отладчиком является Soft-Ice от NuMega. Это очень мощный профессиональный инструмент. Новички часто испытывают большие трудности при его настройке, поэтому в приложении подробно описывается, как это сделать.

Разумеется, никто не ограничивает свободу читателя в выборе отладчика, однако в настоящее время не существует программ, которые могли бы составить реальную конкуренцию Софт-Айсу. Это вовсе не означает, что другие программы не пригодны для взлома. Большинство из них могут решать рядовые задачи с не меньшим успехом, а узкоспециализированные - в своей области заметно обгоняют Айс. Но уникальность Айса в том, что он покрывает рекордно широкий круг задач и платформ. Кроме того, очень приятен и удобен. Я не знаю ни одного другого отладчика, поддерживающего командную строку.

Однако обо всех преимуществах не расскажешь в двух строках, поэтому рассмотрим его в действии. Запустим исследуемое приложение. Программа просит нас ввести имя и регистрационный номер. Попробуем набрать что-нибудь "от балды".

Разумеется, ничего не получается, и таким способом, скорее всего, программу зарегистрировать никогда не удастся. На это и рассчитывал автор защиты. Однако у нас есть приимущество. Знания ассемблера позволяют заглянуть внутрь кода и проанализировать его алгоритм.

Конкретно нас интересует механизм генерации регистрационных номеров. Как обнаружить его в изучаемом коде? Один из самых легких способов - отследить обращение к введенной строке. Код, читающий ее, очевидно, либо непосредственно входит в генератор, либо лежит в непосредственной близости. Остается только узнать, по какому адресу строка расположена в памяти.

Хорошая задачка! Откуда же узнать этот адрес? Неужели придется утомительно анализирать код? Разумеется, нет. Существуют гораздо более оригинальные приемы. Начнем с того, что содержимое окна редактирования надо как-то считать. Для этого нужно послать окну сообщение WM_GETTEXT и адрес буфера, куда этот текст следует принять. Однако этот способ не снискал популярности, и программисты обычно используют функции API. В SDK можно найти по крайней мере две функции, пригодные для этой цели, - GetWindowText и GetDlgItemText. Причем первая используется гораздо чаще.

Перехват процедуры чтения содержимого окна позволяет узнать, по какому адресу в памяти располагается введенная строка, и поставить на последнюю точку останова - так, чтобы любой код, обращающийся к этой области вызывал отладочное исключение. Это быстро позволит нам найти защитный механизм в сколь угодно большой программе так же быстро как и в маленькой.

Итак, нам нужно установить точку останова на вызываемую функцию. Чтобы узнать, какую, вновь заглянем в список импорта crack04.exe Как мы помним, это приложение использует MFC, а следовательно, крайне маловерятно, чтобы программист, писавший его, воспользовался напрямую Win32 API, а не библиотечной функцией.Вероятнее всего CWnd::GetWindowText, Попробуем найти ее среди списка импортируемых функций. Для этого можно воспользоваться любой утилитой (например IDA) или даже действовать вручную. Так или иначе, мы обнаружим, что ординал этой функции 0xF22. Этого достаточно, чтобы установить точку останова и перехватить чтение введенной строки.

Однако легко видеть, что CWnd::GetWindowText это всего лишь "переходник" от Win32 API GetWindowTextA. Поскольку нам нужно выяснить только сам адрес строки, то все равно перехватом какой функции мы это сделаем, т.к. и та и другая работают с одним и тем же буфером. Это применимо не только к MFC, но и к другим библиотекам. В любом случае на самом низком уровне приложений находятся вызовы Win32 API, поэтому нет никакой нужды досконально изучать все существующие библиотеки, достаточно иметь под рукой SDK. Однако это никак еще не означает, что можно вообще не интересоваться архитектурой высокоуровневых библиотек. Приведенный пример оказался "прозрачен" только потому, что GetWindowTextA передавался указатель на тот же самый буфер, в котором и возвращалась введенная строка. Но разве не может быть иначе? GetWindowTextA передается указатель локального буфера, который затем копируется в

результирующий. Поэтому полезно хотя бы бегло ознакомиться с архитектурой популярных библиотек.

Но давайте, наконец, перейдем к делу. Для этого вызовем отладчик и (если это Софт_Айс) дадим команду bpx GetWindowTextA. Попутно укажем, откуда взялась буква 'A'. Она позволяет отличить 32-разрядные функции, работающие с unicode строками (W), от функций, работающих с ANSI строками (A). Нам это помогает отличать новые 32-разрядные фуккции от одноименных 16-разрядных. Подробности можно найти в SDK.

После этого введем свое имя и произвольный регистрационный номер и нажмем Enter. Если отладчик был правильно настроен, то он тут же "всплывет". В противном случае нужно обратиться к приложению в конце книги.

Сейчас мы находимся в точке входа в функцию GetWindowTextA. Как узнать адрес переданного ей буфера? Разумеется, через стек. Рассмотрим ее прототип:

```
int GetWindowText(
    HWND hWnd,        // handle to window or control with text
    LPTSTR lpString,  // address of buffer for text
    int nMaxCount     // maximum number of characters to copy
    );
```

Следовательно, стек будет выглядить так:

```
  ----------------------¬0x0
DWORD ¦          EIP        ¦
  +---------------------+ 0x4
DWORD ¦      nMaxCount      ¦
  +---------------------+ 0x8
DWORD ¦       lpString      ¦
  +---------------------+ 0xC
  ¦                     ¦
```

Переведем окно дампа для отображения двойных слов командой DD и командой d ss:esp+8 выведем искомый адрес. Запомним его (запишем на бумажке) или выделим мышью и скопируем в буфер. Теперь дождемся выхода из процедуры (p ret) и убедимся, что прочитанная строка соответствует введеному имени. (Вполне возможно, что программа сперва читает регистрационный номер и только потом имя).

Теперь необходимо поставить точку останова на начало строки или на весь диапазон. Первое может не сработать, если защита игнорирует несколько первых символов имени, а второе замедляет работу. Обычно сначала выбирают первое, а если оно не сработало (что бывает крайне редко), то второе.

Двойное слово lpString это указатель на строку. Однако это только 32-битное смещение. Но относительно какого сегмента? Разумеется, DS. Поэтому установка точки останова может выгядеть так: bpx ds:xxxxx r. Первый код, читающий строку, на самом деле не принадлежит к отлаживаемой программе. В этом можно убедиться, если несколько раз дать команду p ret, - до тех пор, пока мы не выйдем из функции MFC42!0F22. Как мы помним это ординал CWnd::GetWindowText. Теперь любой обращающийся к строке код будет принадлежать непосредственно защите. Мы, вероятно, уже находимся в непосредственной близости от защитного механизма, но иногда бывает так, что программист читает строку в одном месте программы, а использует результат совсем в другом. Поэтому дождемся повторного отладочного исключения. Рассмотрим код, вызвавший его:

```
015F:004015F7  8A0C06              MOV     CL,[EAX+ESI]
```

Используемая адресация наталкивает нас на мысль, что eax, возможно, параметр цикла, а вся эта конструкция посимвольно читает строку. Очень похоже, что в самом центре генератора серийного номера. Если мы посмотрим чуть-чуть ниже, то в глаза бросится очень любопытная строка:

```
015F:0040164B  51                  PUSH    ECX
015F:0040164C  52                  PUSH    EDX
```

```
015F:0040164D  FF15D0214000        CALL    [MSVCRT!_mbscmp]
```

Вероятно, она сравнивает введенный нами и сгенерированный регистрационный номер! Переведем курсор на нее и дадим команду here. И последовательно дадим команды d ds:ecx и d ds:edx. В одном случае мы увидим свою строку, а во втором - истинный регистрационный номер. Выйдем из отладчика и попытаемся ввести его в программу. Получилось! Нас признали зарегистрированным пользователем!

Вся эта операция не должна была занять больше пары минут. Обычно для подобных защит больше и не требуется. С другой стороны, на ее написание автор потратил в лучшем случае минут пять-десять. Это очень плохой баланс между накладными расходами на создание защиты и ее стойкостью.

Вышеописанная технология доступна для понимания чрезвычайно широкого круга людей и не требует даже поверхностного знания ассемблера и операционной системы. Любопытно, что большинство кракеров под Windows вообще смутно предсталяют себе "внутренности" последней и знают API куда хуже прикладных программистов. Воистину, тут подходит фраза: "умение снять защиту еше не означает умения ее поставить".

На этом фоне популярность такого подхода выглядит загадочной. Нельзя сказать, что авторы защит не представляют, насколько легко ее вскрыть. Подтверждением являются просьбы (особенно у российских программистов) к кракерам не ломать защиту, а зарегистрироваться и способствоать развитию отечественного рынка. Иной раз настолько красноречивые и длинные, что за время, потраченное на сочинение подобных опусов, можно было бы значительно улучшить реализацию защиты, что несомненно дало бы гораздо больший эффект.

Однако мы не закончили взлом программы. Да, мы узнали регистрационный код для нашего имени, но понравится ли это остальным пользователям? Ведь каждый из них хочет зарегистрировать программу на СЕБЯ. Кому будет приятно видеть чужое имя?

Вернемся к коду, сравнивающему эти строки:

```
015F:00401643  8B4C2410            MOV     ECX,[ESP+10]
015F:00401647  8B54240C            MOV     EDX,[ESP+0C]
015F:0040164B  51                  PUSH    ECX
015F:0040164C  52                  PUSH    EDX
015F:0040164D  FF15D0214000        CALL    [MSVCRT!_mbscmp]
015F:00401653  83C408              ADD     ESP,08
015F:00401656  85C0                TEST    EAX,EAX
015F:00401658  5E                  POP     ESI
015F:00401659  6A00                PUSH    00
015F:0040165B  6A00                PUSH    00
015F:0040165D  7507                JNZ     00401666
```

Давайте заменим в строке 0040164C 0x52 на 0x51, тогда защита будет сравнивать строку с ней самой. Разумеется, сама с собой строка не совпадать никак не может. Конечно, можно заменить JNZ на JMP или JZ, но это будет не так оригинально.

Замечу, что этот способ срабатывает очень редко. Чаще всего проверка будет не одна и в самых неожиданных местах. Достаточно вспомнить, что регистрационные данные запоминаются защитой в реестре или внешнем файле. Блокировав первую проверку, мы добьемся того, что позволим защите сохранить неверные данные. Очень вероятно, что при их загрузке автор предусмотрел проверку на валидность. Ее можно отследить аналогичным образом, перехватив вызовы функций, манипулирующих с реестром, однако это было бы очень утомительно. Впрочем, не так утомительно, как может показаться на первый взгляд. В самом деле, не интересуясь механизмом ввода данных, можно отследить все вызовы процедуры генерации. Возможны по крайней мере два варианта. Автор либо использовал вызов одной и той же процедуры из разных мест, либо дублировал ее по необходимости. В первом случае нас выручат перекрестные ссылки (наиболее полно их умеет отслеживать sourcer), во втором - сигнатурный поиск. Крайне маловероятно, что

автор использовал не один, а несколько вариантов процедуры генератора. Но даже в этом случае не гарантировано отсутствие совпадающих фрагментов. И уж тем более на языках высокого уровня. Далеко не каждый программист знает, что (! a) ? b=0 : b=1 и if (!a) b=0; else b=1 генерируют идентичный код. Поэтому написать одну и ту же процедуру, но так, чтобы ни в одном из вариантов не было повторяющихся фрагментов кода, представляется очень нетривиальной задачей.

Другим возможным решением помимо изменения кода программы (которое далеко не приветствуется законом) может быть написание собственного генератора регистрационных номеров или, в просторечии, ключеделки. Для этого необходимо проанализировать алгоритм генератора и написать аналогичный самостоятельно.

Вернемся немного назад:

```
015F:004015F7  8A0C06              MOV     CL,[EAX+ESI]
015F:004015FA  660FBE440601        MOVSX   AX,BYTE PTR [EAX+ESI+01]
015F:00401600  660FBEC9            MOVSX   CX,CL
015F:00401604  0FAFC1              IMUL    EAX,ECX
015F:00401607  25FFFF0000          AND     EAX,0000FFFF
015F:0040160C  B91A000000          MOV     ECX,0000001A
015F:00401611  99                  CDQ
015F:00401612  F7F9                IDIV    ECX
015F:00401614  8D4C240C            LEA     ECX,[ESP+0C]
015F:00401618  80C241              ADD     DL,41
015F:0040161B  88542414            MOV     [ESP+14],DL
015F:0040161F  8B542414            MOV     EDX,[ESP+14]
015F:00401623  52                  PUSH    EDX
015F:00401624  E805030000          CALL    0040192E
                                   ^^^^^^^^^^^^^^^^
015F:00401629  8B442408            MOV     EAX,[ESP+08]
```

Если мы попытаемся заглянуть в процедуру 0x040192E, то вероятнее всего утонем в условных переходах и вложенных вызовах. Сложность и витиеватость кода наталкивают на мысль, что это библиотечная процедура. Но какая? Дело в том, что отладчик не был правильно настроен и экспортировал только системные функции. Исследуемое приложение активно использует MFC42.DLL, поэтому для загрузки символьной информации о функциях последнего необходимо его явно загрузить. Это делается директивой EXP в файле winice.dat Посмотрим, что у нас получилось:

```
015F:0040161B  88542414            MOV     [ESP+14],DL
015F:0040161F  8B542414            MOV     EDX,[ESP+14]
015F:00401623  52                  PUSH    EDX
015F:00401624  E805030000          CALL    MFC42!ORD_03AC

^^^^^^^^^^^^^^^^^^^^
```

Несмотря на то что символьная информация по-прежнему отсутствует, изучение кода значительно облегчилось. По крайней мере, теперь выделены все библиотечные функции. Даже если бы мы не знали, как получить имя через ординал (а мы это уже знаем), все равно объем анализируемого кода значительно бы уменьшился. Вы же не будете исследовать библиотечную функцию? В любом случае можно догадаться о ее назначении по входным и выходным параметрам.

Однако отладчики не предназначены для подробного анализа кода. Гораздо удобнее изучать логику программы с помощью дизассемблера. Найти же требуемый фрагмент очень просто. Достаточно вспомнить, что адрес уже известен. Переместим курсор на строку .text:0040161B, для чего в IDA дадим с консоли команду Jump(MK_FP(0,0x40161B)) и прокрутим экран немного вверх, пока не встретим следующие строки:

```
.text:004015D3                call    j_?GetWindowTextA!!AMPER!!CWnd!!AMPER!!!!AMPER!!QBEXAA
.text:004015D8                mov     eax, [esp+4]
.text:004015DC                mov     ecx, [eax-8]
.text:004015DF                cmp     ecx, 0Ah
.text:004015E2                jge     short loc_0_4015EF
```

Очевидно, последний условный переход выполняется, когда длина введенной строки больше девяти символов. Для понимания этого необходимо знать, что CString хранит свою длину в двойном слове, находящемся до начала строки. Итак, непосредственно относящийся к защите код начинается с адреса 0x4015EF. Рассмотрим его:

```
.text:004015EF loc_0_4015EF:
.text:004015EF                 push    esi
.text:004015F0                 xor     esi, esi
.text:004015F2                 dec     ecx
.text:004015F3                 test    ecx, ecx
.text:004015F5                 jle     short loc_0_401636
```

Это типичный цикл for. Заглянем в его тело:

```
.text:004015F7 loc_0_4015F7:
.text:004015F7                 mov     cl, [esi+eax]
```

Загрузка очередного символа строки. Поскольку eax - содержит базовый адрес, то очевидно, что esi - смещение в строке. Выше видно, что начальное значение его равно нулю. Логично, что строка обрабатывается от первого до последнего символа, хотя часто бывает и наоборот.

```
.text:004015FA                 movsx   ax, byte ptr [esi+eax+1]
```

MOVe and Sign eXtension (пересылка со знаковым расширением) загружает байт в регистр AX, автоматически расширяя его до слова.

```
.text:00401600                 movsx   cx, cl
```

Обратим внимание на несовершенство компилятора. Эту команду можно было записать более экономно как movsx cx, [esi+eax]

```
.text:00401604                 imul    eax, ecx
```

Подставим всесто регистров их смысловые значения и получим String[idx]*String[idx+1].

```
.text:00401607                 and     eax, 0FFFFh
```

Преобразуем eax к машинному слову.

```
.text:0040160C                 mov     ecx, 20h
.text:00401611                 cdq
```

CDQ - Convert Double word to Quad word - Преобразование двойного слова в счетверенное слово

```
.text:00401612                 idiv    ecx
.text:00401614                 lea     ecx, [esp+28h+var_1C]
.text:00401618                 add     dl, 41h
```

Поскольку 0x41 - это код символа 'A', то, вновь выполнив смысловую подстановку, получим: *dl = (String[idx]*String[idx+1]) % 0x20 + 'A'. Т.е автор вычисляет хеш-сумму строки. Обратим внимание, что она будет инъективна для интервала 'A'-"* и, более того, нечувствительна к регистру!

Этот код можно назвать "кодом черной магии". С первого взгляда не понятно как он работает и чем обусловлена нечувствительность к регистру. Обычно для этого программист сначала переводит все буквы в заглавные и только потом начинает разбор строки. Или делает это на лету явным сравнением типа cmp xx, 'a'.

Оригинальные приемы всегда ценятся хакерами, особенно когда они позволяют сократить немного байт и тактов процессора.

```
.text:0040161B                 mov     byte ptr [esp+28h+var_14],dl
.text:0040161F                 mov     edx, [esp+28h+var_14]
.text:00401623                 push    edx
.text:00401624                 call    CString::operator+=(char)
```

Очередной перл компилятора. Можно было не вводить локальную переменную, а непосредственно передать dl (предварительно расширив его до двойного слова) в стек, что повысило бы скорость

обработки за счет избавления от обращений к памяти.

```
.text:0040162D                inc     esi
```

Перемещаем указатеь idx на следующий символ в строке.

```
.text:0040162E                mov     ecx, [eax-8]
.text:00401631                dec     ecx
.text:00401632                cmp     esi, ecx
.text:00401634                jl      short loc_0_4015F7
```

Очевидно, что эти строки также относятся к циклу for. Поэтому уже можно восстановить исходный код генератора.

```
for (int idx=0;idx>String.GetLength()-1;idx++)
   RegCode+= ((WORD) sName[a]*sName[a+1] % 0x20) + 'A';
```

Теперь нетрудно написать собственный генератор регистрационных номеров. Это можно сделать на любом симпатичном вам языке, например на ассемблере. На диске находится один вариант (file://CD/SRC/CRACK04/key_gen.asm). Без текстовых строк исполняемый файл занимает менее пятидесяти байт и еще оставляет простор для оптимизации. Ключевая процедура может выглядеть так:

```
Reprat:                          ;
      LODSW                     ; Читаем слово
       MUL     AH               ; Password[si]*Password[si+1]
       XOR     DX,DX            ; DX == NULL
       DIV     BX               ; Password[si]*Password[si+1] % 0x20

       ADD     DL,'A'           ; Переводим в символ
       XCHG    AL,DL
       STOSB                    ; Записываем результат
       DEC     SI
       LOOP    Reprat
```

Испытаем написанный генератор. Заметим, что в key_gen.asm есть одно несущественное упущение. Он не проверяет минимальную длину строки. Но на деле это не вызывает больших неудобств, зато экономит пяток байт кода.

Генератор успешно работает и вычисляет правильные регистрационные номера. Теперь можно начинать его публичное распространение. Отметим, что последнее совершенно не запрещено законом. И ничьих прав не ущемляет. Использование же генераторов все же вызывает конфликтную ситуацию, т.к. пользователь вводит поддельный регистрационный номер. С другой стороны, это недоказуемо, т.к. сгенерированные номера ничем не отличаются от настоящих. Тем не менее я категорически не советую уповать на это. Лицензиозные соглашения пишутся не для того, чтобы их нарушать. Точно так же и создание собственного генератора не должно побуждать к его использованию, отличному от познавательного. Перечислите автору требуемую сумму или откажитесь от использования программы. Истинный хакер так и поступит. В этом и заключается его отличие от кракеров. Хакер по определению первоклассный специалист, который всегда заработает на необходимое программное обеспечение (или, если он действительно хакер, то напишет свое).

Я понимаю, что такая трактовка может встретить возражение. Действительно, зачем что-то ломать, если хакер все равно должен приобретать лицензиозный софт? Но разве в этом есть что-то нелогичное? Хакер - это взрослый ребенок, удовлетворяющий свое любопытство. Конечно, очень трудно, обладая такими знаниями и навыками, удержаться от соблазна нарушить закон. Более того, я не знаю ни одного человека, который поступал бы именно так. Увы, хакерство действительно оказывается тесно связанным с криминалом. Это, к сожалению, так.

Фрагмент из второго издания книги "Техника и философия хакерских атак 2.000"

Автор: Крис Касперски

Дата: 2003-06-11

Дао регистрационных защит (рабочий вариант)

> _…идем мы [Andrew Dolgov] с Сергеем Кожиным (кто не в курсе - это автор parmatosser'a) на пойнтовку к нему. Такой диалог: Я: Ты бы дал мне нормальный ключ, а то этот пиратский генератор как-то не катит. Он: Нафиг? Я сам им пользуюсь, он меньше и рработает быстрее.
> фидошное_

Мир давно привык к тому, что популярные технологии далеко не всегда оказываются хорошими. Вот и в сфере условно-бесплатного программного обеспечения наибольшее распространение получили защиты, генерирующие регистрационный номер на основе имени пользователя (регистрационные защиты). Суть этого механизма заключается в том, что на основе некоторой функции f(name) разработчик преобразует регистрационное имя клиента в регистрационный номер и за некоторую плату отсылает его клиенту. Защита же в свою очередь проделывает с регистрационным именем ту же самую операцию, а затем сравнивает сгенерированный регистрационный номер с регистрационным номером, введенным пользователем. Если эти номера совпадают, то все ОК и, соответственно, wrong reg num в противном случае (см. рис. 0x00F).

Таким образом, защитный механизм содержит в себе полноценный генератор регистрационного кода и все, что требуется хакеру: найти процедуру генерации и, подсунув ей свое собственное имя, просто подсмотреть возращенный результат! Другая слабая точка: компаратор, т. е. процедура, сравнивающая введенный и эталонный регистрационный номера. Если на оба плеча компаратора подать один и тот же регистрационный номер (не важно введенный пользователем или сгенерированный защитой), он, со всей очевидностью, скажет "ОК" и защита примет любого пользователя как родного. Еще один способ взлома: проанализировав алгоритм генератора отладчиком и/или дизассемблером, хакер сможет создать свой собственный генератор регистрационных номеров.

Все, чем может досадить хакеру автор защиты, - затруднить анализ и реконструкцию алгоритма генерации. Первое осуществляется оригинальными приемами программирования, противостоящих отладке и/или дизассемблированию, а второе - "размазыванием" кода по десяткам процедур, активным использованием глобальных переменных и запутанным взаимодействием различных фрагментами кода.

Стоит ли и говорить, что запутывание алгоритма малоэффективно и отдает "ребячеством", а подавляющее большинство антиотладочных приемов бессильно против современных отладчиков; кроме того, далеко не все антиотладочные приемы удается реализовать на языках высокого уровня. Спускаться же на уровень ассемблера практически никто из разработчиков не хочет.

Причем, если генератор реализован в одной-двух процедурах (а чаще всего генераторы реализуются именно так!), хакеру нет никакой нужды тратить время на его анализ, и можно прибегнуть к тупому "выкусыванию" кода генератора и копированию его тела в свою собственную программу-оболочку, позволяющую передавать генератору произвольные параметры, в роли которых обычно выступают имена пользователя, company name и прочие регистрационные данные. Впрочем, "выдергиванию" кода легко помешать. Действительно, если рассредоточить код генератор по множеству служебных функций со сложным взаимодействием и неочевидным

обменом данных, то без кропотливого анализа всей защиты, выделение всех, относящихся к ней компонентов, окажется невозможным! (Кстати, с точки зрения закона создание собственных генераторов намного более предпочтительнее, чем несанкционированное выдирания фрагментов "живого" кода из чужой программы).

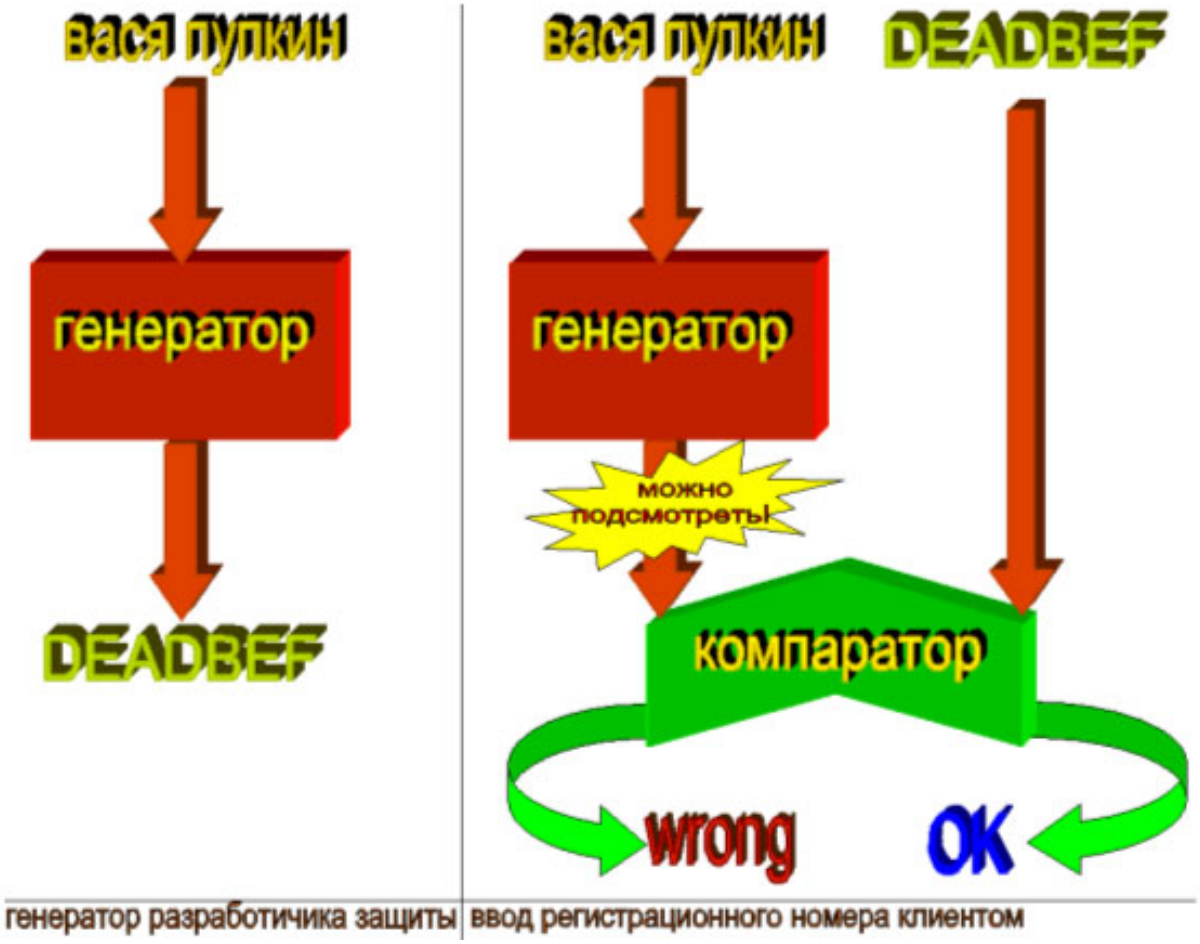


Рисунок 2 0x00F принцип работы регистрационной защиты

Рассмотрим простую реализацию данного защитного механизма на примере программы crackme.58DD2D69h. До сих пор для изучения защитного кода мы пользовались одним лишь дизассемблером, но это не единственный возможный подход к задаче. Не меньшим успехом у хакеров пользуются и отладчики. Отметим, что отладка - более агрессивный способ исследования: в этом случае взлом программы осуществляется "в живую" и со стороны защиты возможны любые "подлянки". Антиотладочный код может запросто "завесить" вашу систему и вообще, выкинуть то, чего вы от него никак не ожидаете. С другой стороны, отладчик обладает многими замечательными (в плане взлома) возможностями, о реализации которых в дизассемблерах пока приходится только мечтать. В первую очередь это относится к точкам останова (по-английски break point), с которыми мы чуть позже с успехом и воспользуемся.

Самым популярным среди хакеров отладчиком был, есть и остается отладчик Soft-Ice от компании NuMega, представляющий собой профессионально-ориентированный инструмент, и потому вызывающий большие трудности у новичков в его освоении. Однако потраченные усилия стоят того! Разумеется, никто не ограничивает свободу читателя в выборе инструментария, - вы можете использовать Microsoft Windows Debugger, Borland Turbo Debugger, Intel Enhanced Debugger, DeGlucker или любой другой отладчик по своему вкусу . Рядовые задачи они решают не хуже айса, а узкоспециализированные отладчики (такие, например, как CUP и Exe Hack) в своих областях даже обгоняют soft-ice. Но уникальность "Айса" как раз и заключается в том, что он покрывает рекордно широкий круг задач и платформ. Существуют его реализации для MS-DOS (ну вдруг кому

ни будь понадобится старушка!), Windows 3.1, Windows 9x и Windows NT. Все эти версии Айса несколько различаются межу собой по набору и синтаксису команд, однако эти отличия не столь принципиальны, чтобы вызывать какие либо проблемы. На всякий случай: здесь описывается soft-ice 2.54 под Windows NT.

Итак, загружаем отладчик (под NT это можно сделать в любое время, а в Windows 9x только на стадии загрузки компьютера) и запускаем ломаемое приложение, которое немедленно запрашивает у нас имя и регистрационный номер. Поскольку, регистрационный номер нам доподлинно не известен, приходится набрать что-нибудь "от балды".

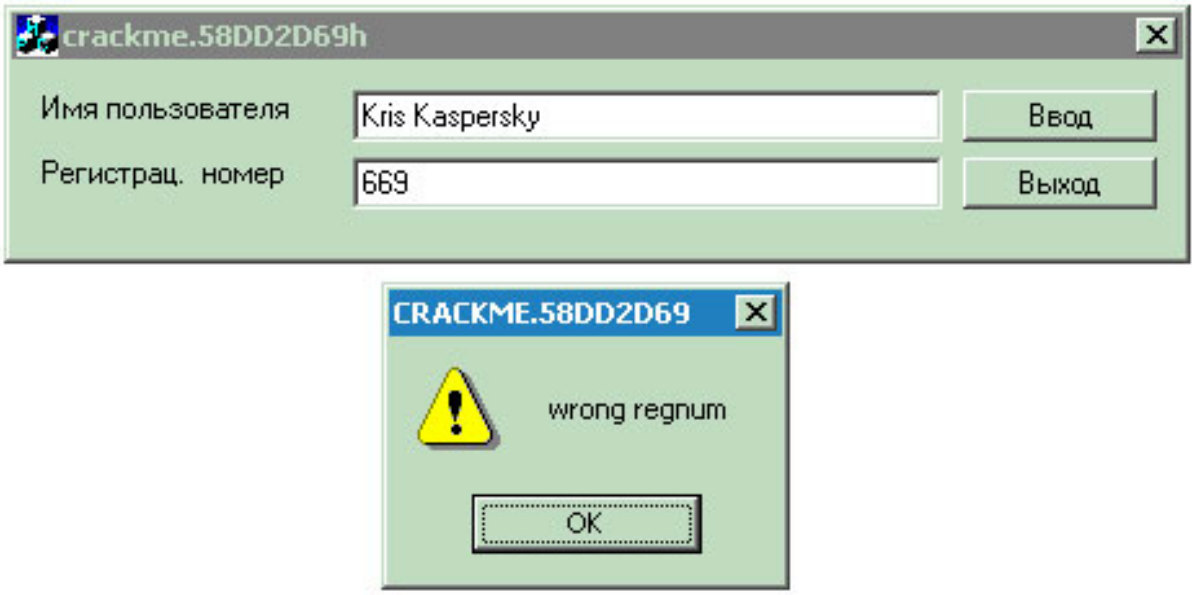


Рисунок 3 0x00A реакция защиты на неверно введенный регистрационный номер

Защита, обложив нас матом, сообщает, что "regnum" есть "wrong" и никакой регистрации нам не видать! А чего мы ждали?! Угадать регистрационный номер ни с первой, ни со второй, ни даже с тысячной попытки нереально (регистрационные номера по обыкновению до безобразия длинны) и тупым перебором взломать программу нам не удастся. На это, собственно, и рассчитывал автор защиты. Однако у нас есть преимущество: знание ассемблера позволяет нам заглянуть внутрь кода и проанализировать алгоритм генерации регистрационных номеров. То есть, атаковать защиту не в лоб, а, обойдя укрепленные позиции, напасть с тыла.

Сразу же возникает вопрос: как определить местонахождение генератора, не прибегая к полному анализу исследуемой программы? Давайте представим себе, что генератор это взяточник, а мы - ОБХСС. Роль денег будет играть регистрационное имя, вводимое пользователем. Код, позарившийся на взятку, очевидно и будет самим генератором! То есть, в основе взлома по сути своей лежит перехват обращения к исходным регистрационным данным, избежать которого защита в принципе не может (телепатических возможностей существующие процессоры увы лишены).

Для осуществления такого перехвата нам потребуется всего лишь установить на регистрационное имя так называемую точку останова (break point). Процессор на аппаратном уровне будет контролировать этот регион памяти и при первой же попытке обращения к нему, прервет выполнение программы, сообщая отладчику адреса машинной команды, рискнувшей осуществить такой доступ. Естественно, для установки точки останова требуется знать точное расположение искомой строки в памяти. Спрашиваете, как мы его найдем? Начнем с того, что содержимое окна редактирования надо как-то считать. В Windows это осуществляется посылкой окну сообщения WM_GETTEXT с указанием адреса буфера-приемника. Однако, низкоуровневая работа с сообщениями - занятие муторное и непопулярное. Гораздо чаще программисты используют API-функции, предоставляющие приятный и удобный в обращении высокоуровневый интерфейс. В

Platform SDK можно найти по крайней мере две таких функции: GetWindowText и GetDlgItemText. Статистика показывает, что первая из них встречается чуть ли не на порядок чаще, что и не удивительно, т. к. она более универсальна чем ее "коллега".

Перехватив вызов функции, читающей содержимого окна, мы сможем подсмотреть значение переданного ей указателя на буфер, в который и будет скопирована наша строка. Очевидно, что это и есть тот самый адрес, на который мы стремимся установить точку останова! Теперь любой код, обращающийся к этой области, вызовет отладочное исключение и "разбудит" отладчик. Благодаря этому мы обнаружим защитный механизм в сколь угодно большой программе так же быстро как и в маленькой.

Спрашиваете, как мы сможем перехватить вызов функции? Да все с помощью той же самой точки останова! Единственное, что нам для этого потребуется - адрес самой функции. Но вот какой именно функции? Как уже было сказано выше, функций, пригодных для чтения текста из окна редактирования, существует по меньшей мере две. Программист мог использовать либо ту, либо другую, либо вообще третью…

Поскольку исследуемое нами приложения написано на Microsoft Visual C++ с применением библиотеки MFC (что видно по копирайтам, содержащимся в теле файла, и содержимому таблицы импорта), то представляется достаточно маловероятным, чтобы программист, разрабатывающий его, использовал прямые вызовы win32 API. Скорее всего, он, как истинный поклонник объективно ориентированного программирования, сосредоточился исключительно на MFC-функциях, и употребил CWnd::GetWinowText или производные от него методы. К сожалению, неприятной особенностью библиотеки MFC является отсутствие символических имен функций в таблице экспорта и она экспортирует их лишь по порядковому номеру (так же называемому ординалом - от английского ordinal). При наличии сопутствующих библиотек мы без труда определим какому именно ординалу соответствует то или иное имя, однако, вся проблема как раз и заключается в том, что далеко не всегда такие библиотеки у нас есть. Ведь не можем же мы устанавливать на свой компьютер все версии всех компиляторов без разбора?!

Зацепку дает тот факт, что CWnd::GetWindowText по сути своей является сквозным "переходником" от win32 API функции GetWindowTextA. Поскольку все, что нам сейчас требуемся, - это выяснить адрес регистрационной строки, то не все ли равно перехватом какой именно функции это делать? Материнская функция-обертка работает с тем же самым буфером, что и дочь. Это типичное не только для MFC, но и для подавляющего большинства других библиотек. В любом случае на нижнем уровне приложений находятся вызовы win32 API и поэтому нет никакой нужды досконально изучать все существующие библиотеки. Достаточно иметь под рукой SDK! Однако не стоит так же бросаться и в другую крайность, отвергая идею изучения архитектуры высокоуровневых библиотек вообще. Приведенный пример оказался "прозрачен" лишь благодаря тому, что функции GetWindowTextA передается указатель на тот же самый буфер, в котором и возвращалась введенная строка. Но в некоторых случаях функции GetWindowTextA передается указатель на промежуточный буфер, который впоследствии копируется в целевой. Так что ознакомление (хотя бы поверхностное) с архитектурой популярных библиотек очень полезно. **[врезка] как узнать имя функции по ординалу**

Если динамическая библиотека экспортирует свои функции по ориналу и только по ординалу, то непосредственно определить имена функций невозможно, поскольку их там нет. Однако при наличии соответствующей библиотеки (обычно поставляющейся вместе со средой разработки) наша задача значительно упрощается. Ведь как-то же определяют линкеры ординалы функций по их именам! Так почему же нам не проделать обратную операцию? Давайте воспользуемся уже полюбившейся нам утилитой DUMPBIN из комплекта поставки Platform SDK, запустив ее с ключом /HEADERS и, естественно, именем анализируемой библиотеки. В частности, для определения ординала функции CWnd::GetWindowText мы должны найти в каталоге \Microsoft Visual Studio\VC98\MFC\Lib файл MFC42.lib и натравить на него DUMPBIN:

```
> dumpbin /HEADERS MFC42.lib > MFC42.headers.txt
> type MFC42.headers.txt | MORE
  Version      : 0
  Machine      : 14C (i386)
  TimeDateStamp: 35887C4E Thu Jun 18 06:32:46 1998
  SizeOfData   : 00000033
  DLL name     : MFC42.DLL
  Symbol name  : ?GetWindowTextA@CWnd@@QBEXAAVCString@@@Z
: (public: void __thiscall CWnd::GetWindowTextA(class CString &)const )
  Type         : code
  Name type    : ordinal
  Ordinal      : 3874
```

…затем в образовавшемся файле находим нужное нам имя и смотрим всю информацию по нему и, среди всего прочего - ординал (в данном случае: 3874h)

Но вернемся к нашим баранам. Нажатием вызываем soft-ice и даем ему команду "bpx GetWindowTextA" Откуда, спрашиваете взялась буква 'A'? Это суффикс, указывающий на ее принадлежность к ANSI-строкам. Функции, обрабатывающие Unicode-строки, имеют префикс 'W' (в Windows 9x они не реализованы и представляют собой лишь "заглушки", а ядро Windows NT, наоборот, работает исключительно с уникодом и уже ANSI - функции представляют собой переходники; более подробно об этом можно прочитать в Platform SDK), выходим из отладчика повторным нажатием или аналогичной по действию командой "x" и вводим в ломаемое приложение свое имя и произвольный регистрационный номер, подтверждая серьезность своих намерений нажатием . Если отладчик был правильно настроен, то он тут же "всплывает". В противном случае вам следует внимательно изучить прилагаемое к нему руководство или на худой конец его русский перевод, который без труда можно найти в сети.

В общем, будет считать, что все перипетии борьбы с отладчиком уже позади и сейчас мы находимся в точке входа в функцию GetWindowTextA. Как узнать адрес переданного ей буфера? Разумеется, через стек. Рассмотрим ее прототип, приведенный в SDK:

```
int GetWindowText(
    HWND hWnd,        // handle to window or control with text
    LPTSTR lpString,  // address of buffer for text
    int nMaxCount     // maximum number of characters to copy
    );
```

Поскольку, все win32 API функции придерживаются соглашения stdcall и передают свои аргументы слева направо, то стек, на момент вызова функции, будет выглядеть так:

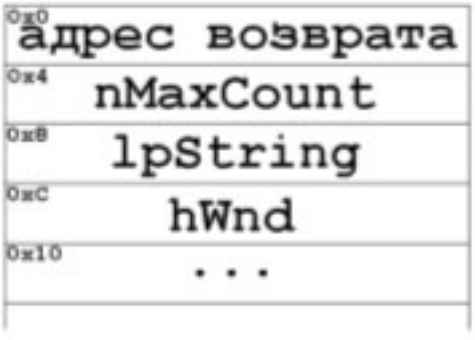


Рисунок 4 0x00B состояние стека на момент вызова функции GetWindowText

Переведем окно дампа в режим отображения двойных слов командой "DD" и командой "d ss:esp + 8" заставим его отобразить искомый адрес. Запомним его (запишем на бумажке) или выделим мышью и скопируем в буфер (последние версии soft-ice поддерживают мышь). В частности, на компьютере автора содержимое стека выглядело так:

```
:dd
:d ss:esp+8
0023:0012F9EC 002F4018  0000000F  00402310  004015D8      .@/......#@...@.
```

```
0023:0012F9FC 0012FA04  0012FE14  002F4018  6C361C58      .........@/.X.6l
0023:0012FA0C 6C361C58  0012F9F8  0012FB44  00401C48      X.6l....D...H.@.
0023:0012FA1C 00000002  6C2923D8  00402310  00000111      .....#)l.#@.....
```

Выделенное жирным шрифтом число и есть адрес буфера, готового принять прочитанную из окна строку. Посмотрим, что у нас там? Переключившись из режима двойных слов в режим байтов командой "DB", мы говорим отладчику "D SS:2F4018" и… ну конечно же видим вокруг себя один мусор, что и не удивительно, ведь функция GetWindowTextA еще и начинала своего выполнения! Что ж, приказываем Айсу выйти из функции ("P RET") и… вот она, наша строка!

```
:db
:d ss:2f4018
:p ret
0023:002F4018 4B 72 69 73 20 4B 61 73-70 65 72 73 6B 79 00 00  Kris Kaspersky..
0023:002F4028 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
0023:002F4038 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
0023:002F4048 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
```

Теперь установим точку останова на адрес начала строки (в листинге, приведенном выше он обведен рамкой) или на всю строку целиком. Заметим, что обоим решениям присущи свои недостатки: если защита игнорирует несколько первых символов имени, то первый примем просто не сработает. С другой стороны, точки останова на диапазон адресов аппаратно не поддерживаются и отладчик вынужден прибегать к хитрым манипуляциям с атрибутами страницы, заставляя процессор генерировать исключение при всякой попытке доступа к ней, а затем вручную анализировать - произошло ли обращение к контролируемой области или нет. Естественно, это значительно снижает производительность и отлаживаемое приложение исполняется со скоростью, которой не позавидует и черепаха! Поэтому, к этому трюку имеет смысл прибегать лишь тогда, когда не сработал первый (а не срабатывает он крайне редко).

Уничтожив ставшей ненужной точку останова на GetWindowText (команда "bc *") мы устанавливаем новую точку останова "bpm ss:2F4018" (разумеется, на вашем компьютере адрес строки может быть и другим) и покидаем отладчик нажатием . Не желая коротать свои дни в одиночестве, отладчик тут же всплывает, сигнализируя нам о том, что некий код попытался обратиться к нашей строке:

```
001B:77E9736D REPNZ SCASB
001B:77E9736F NOT ECX
001B:77E97371 DEC ECX
001B:77E97372 OR DWORD PTR [EBP-04],-01
```

Судя по адресу, мы имеем дело с некоторой системной функцией (ибо они традиционно размешаются в верхних адресах), но вот с какой именно? Сейчас выясним! Долго ли умеючи! Наскоро набив на клавиатуре трехбуквенное сочетание "mod" мы заставляем отладчик вывести список всех модулей системы на экран:

```
:mod
hMod Base     PEHeader Module Name      File Name
     80400000 804000C8 ntoskrnl         \WINNT\System32\ntoskrnl.exe
     77E10000 77E100D8 user32           \WINNT\system32\user32.dll
     77E80000 77E800D0 kernel32         \WINNT\system32\kernel32.dll
     77F40000 77F400C8 gdi32            \WINNT\system32\gdi32.dll
     77F80000 77F800C0 ntdll            \WINNT\system32\ntdll.dll
     78000000 780000D8 msvcrt           \WINNT\system32\msvcrt.dll
```

Очевидно, что адрес 77E9736Dh принадлежит динамической библиотеке kernel32.dll, а точнее, - функции lstrlenA, которая, как и следует из ее названия определяет длину строки. Поскольку, в определении длины для нас нет ничего интересного, мы безо всякого зазрения совести оставляем этот код жить на бозе и вновь выходим из отладчика, позволяя ему продолжить поиски защитного кода.

Следующее всплытие отладчика оказывается более информативным, смотрите (внимание: в силу архитектурных особенностей x86 процессоров, отладочное исключение возникает не до, а после выполнения команды, "зацепившей" точку останова, а потому отладчик подсвечивает не ее саму, а

следующую за ней команду):

```
001B:004015F7  MOV     CL,[EAX+ESI]  ; эта команда "зацепила" breakpoint
001B:004015FA  MOVSX   AX,BYTE PTR [EAX+ESI+01] ; здесь отладчик получил управление
001B:00401600  MOVSX   CX,CL
001B:00401604  IMUL    EAX,ECX
001B:00401607  AND     EAX,0000FFFF
001B:0040160C  AND     EAX,8000001F ; STATUS_BEGINNING_OF_MEDIA
001B:00401611  JNS     00401618
001B:00401613  DEC     EAX
```

Используемая адресация наталкивает нас на мысль, что EAX, возможно, параметр цикла, а вся эта конструкция посимвольно читает строку. Очень похоже, что в мы находимся самом "сердце" защитного механизма - генераторе серийного номера. Если мы посмотрим чуть-чуть ниже, то в глаза бросится очень любопытная строка (в тексте она выделена жирным шрифтом) :

```
001B:0040164E PUSH ECX
001B:0040164F PUSH EDX
001B:00401650 CALL [MSVCRT!_mbscmp]
001B:00401656 ADD ESP,08
001B:00401659 TEST EAX,EAX
001B:0040165B POP ESI
001B:0040165C PUSH 00
001B:0040165E PUSH 00
001B:00401660 JNZ 00401669
001B:00401662 PUSH 00403030
001B:00401667 JMP 0040166E
```

Вероятно, здесь-то защита и сравнивает введенный пользователем регистрационный номер с только что сгенерированным эталоном! Переведем курсор на строку 401650h и дадим команду "HERE", обозначающую буквально "сюда!" Теперь последовательно дадим команды "D DS:ECX" и "D DS:EDX", посредством которых мы сможем подсмотреть содержимое указателей, передаваемых функции в качестве аргументов. Скорее всего, один из них принадлежит введенной нами строки, а другой - сгенерированному защитой регистрационному номеру.

```
:d ecx
0023:002F40B8 36 36 36 00 00 00 00 00-00 00 00 00 00 00 00 00  666.............
0023:002F40C8 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................

:d edx
0023:002F4068 47 43 4C 41 41 4C 54 51-51 5B 57 52 54 00 35 38  GCLAALTQQ[WRT.58
0023:002F4078 44 44 32 44 36 39 2E 2E-2E 00 00 00 00 00 00 00  DD2D69..........
```

Итак, наше предположение на счет "введенного регистрационного номера" полностью подтверждается, и шансы на то, что абракадабра "GCLAALTQQ[WRT" и есть эталонный регистрационный номер весьма велики (обратите внимание на завершающий ее нуль, отсекающий остаток строки "…58DD2D69", который по невнимательности можно принять за саму строку.

Выйдем из отладчика и попытаемся ввести "GCLAALTQQ[WRT" в программу… Защита, благополучно проглотив регистрационный номер, выводит диалог с победной надписью "ОК". Получилось! Нас признали зарегистрированным пользователем! Вся операция не должна была занять порядка пары-тройки минут. Обычно для подобных защит большего и не требуется. С другой стороны, на их написание автор потратил как минимум полчаса. Это очень плохой баланс между накладными расходами на создание защиты и ее стойкостью. Тем не менее, использование таких защит вовсе не лишено смысла (ведь не все же пользователи - хакеры). Нельзя сказать, что создатели защит совсем уж не представляют насколько их легко вскрыть. Косвенным это подтверждением этого являются убедительные просьбы не ломать защиту, а зарегистрироваться и способствовать развитию отечественного рынка (что особенно характерно для российских программистов). Иной раз они бывают настолько красноречивы и длинны, что за время, потраченное на сочинение подобных опусов, можно было бы значительно усилить защиту.

Вышеописанная технология взлома доступна невероятно широкому кругу людей и не требует даже поверхностного знания ассемблера и операционной системы. Просто ставим точку останова на GetWindowText, затем еще одну на строковой буфер и, дождавшись всплытия отладчика, пытаемся найти в каком месте происходит сравнение введенного регистрационного номера со сгенерированным на основе имени эталоном. Любопытно, но большинство кракеров довольно смутно представляют себе "внутренности" операционной системы и знают API куда хуже прикладных программистов. Воистину "умение снять защиту еще не означает умения ее поставить". Чего греха таить! И автор этой книги сначала научился ломать и лишь затем программировать.

Однако мы не закончили взлом программы. Да, мы узнали регистрационный код для нашего имени, но понравится ли это остальным пользователям? Ведь каждый из них хочет зарегистрировать программу на себя. Кому приятно видеть чужое имя?! Вернемся к коду, сравнивающему строки введенного и эталонного регистрационного номера. Если мы заменим в строке 0040164Eh команду PUSH ECX (опкод 52h) на команду PUSH EDX (опкод 51h), то защита станет сравнивать эталонный регистрационный номер с… самим эталонным регистрационным номером! Разумеется, не совпадать с самим собой регистрационный номер просто не может и какие бы строки мы не вводили, защита воспримет их как правильные. Другой путь - заменить условный переход JNZ в строке 401660h (в тексте он выделен квадратиком) на безусловный переход JZ (тогда защита будет "проглатывать" любые регистрационные номера, кроме правильных), или же забить его любой незначащей командой подходящего размера, например SUB EAX, EAX (тогда будут "проглатываться" любые регистрационные номера, включая правильные), хотя последнее и неоригинально. Запускаем HIEW, переводим его в ASM-режим двойным нажатием , переходим по адресу 401660h (, ".401660") и меняем "jne 1669" на "je 1669", скидываем изменения в файл и запускаем программу. Вводим в нее любую понравившуюся вам комбинации и… это работает!!!

Замечу, что это не самый лучший способ взлома и в ряде случаев он не срабатывает. Типичные защитные механизмы имеют как минимум два уровня обороны. На первом осуществляется проверка корректности введенного регистрационного номера и, если он воспринимается защитой как правильный, то данные пользователя заносятся в реестр или дисковый файл. Затем, при перезапуске программы, защитный механизм извлекает пользовательские данные из места их постоянного хранения и проверяет: а соответствует ли имя пользователя его регистрационному номеру?

Блокировав первую проверку, мы добьемся лишь того, что позволим защите сохранить неверные данные, но наш обман будет немедленно раскрыт как только программа попытается загрузить поддельные данные! Конечно, второй "укрепрайон" защитного механизма можно разбить тем же самым способом, которым мы воспользовались для захвата первого (только на этот раз вместо перехвата функции GetWindowText следует установить точки останова на функции, манипулирующие с файлом и реестром), однако это очень утомительно. Другой, и все такой же утомительный, путь - отследить все вызовы процедуры генерации регистрационного номера по перекрестным ссылкам (если одна и та же процедура вызывалась из разных мест защитного механизма), либо же по ее сигнатуре (если создатель защиты дублировал процедуру генерации). Действительно, крайне маловероятно, чтобы разработчик использовал не один, а несколько независимых вариантов генератора. Но даже в последнем случае очень трудно избежать отсутствия совпадающих фрагментов (во всяком случае на языках высокого уровня). Далеко не каждый программист знает, что "(!a) ? b = 0 : b = 1" и "if (a) b=1; els b=0" в общем случае компилируются в идентичный код. Реализовать один и тот же алгоритм так, чтобы ни в одном из вариантов не присутствовало повторяющихся фрагментов кода, представляется достаточно нетривиальной задачей! Тем не менее, выделение уникальной последовательности, присущей одному лишь защитному коду, - задача ничуть не менее нетривиальная, особенно если в защите присутствует множество проверок, расположенных в самых неожиданных местах.

К счастью, помимо изменения двоичного кода программы (которое, кстати, не очень-то приветствуется законом), существует и другая стратегия взлома: создание собственного генератора регистрационных номеров или, в просторечии, ключеделки. Для осуществления своего замысла хакеру необходимо проанализировать алгоритм оригинального генератора и затем написать аналогичный самостоятельно. Преимущества такого подхода очевидны: во-первых, ключеделка вычисляет действительно правильный регистрационный номер и сколько бы раз защита его ни проверяла - менее правильным он все равно не станет. Во-вторых, с юридической точки зрения создание собственного генератора регистрационных номеров более мягкое преступление, чем модификация защитного кода программы. Правда, возможность наказания за нелегальное использование ПО у законников все равно остается, так что, право же, не стоит так рисковать. Но не будем углубляться в дебри юриспруденции, - пусть трактовкой законов занимаются судьи и адвокаты, нам же - хакерам - лучше сосредоточить свои усилия на машинном коде. Вернемся немного назад, в то самое место, где отладчик зафиксировал обращение к первому байту строки, содержащей имя пользователя, и прокрутим экран дизассемблера немного вверх, до тех пор, пока не встретим начало цикла генератора, определяющееся наименьшим адресом условного (безусловного) перехода, направленного назад (подробнее см. "Фундаментальные основы хакерства" by me главы "Идентификация циклов" и "Идентификация условных операторов").

```
001B:004015EF PUSH ESI
001B:004015F0 XOR ESI,ESI
001B:004015F2 DEC ECX
001B:004015F3 TEST ECX,ECX
001B:004015F5 JLE 00401639
001B:004015F7 MOV CL,[EAX+ESI] ; эта команда обратилась к строке
001B:004015FA MOVSX AX,BYTE PTR [EAX+ESI+01]
001B:00401600 MOVSX CX,CL
001B:00401604 IMUL EAX,ECX
001B:00401607 AND EAX,0000FFFF
001B:0040160C AND EAX,8000001F
001B:00401611 JNS 00401618  ; адрес направлен "вниз", это не цикл
001B:00401611    ; а оператор "IF"
001B:00401613 DEC EAX
001B:00401614 OR EAX,-20
001B:00401617 INC EAX
001B:00401618 ADD AL,41
001B:0040161A LEA ECX,[ESP+0C]
001B:0040161E MOV [ESP+14],AL
001B:00401622 MOV EDX,[ESP+14]
001B:00401626 PUSH EDX
001B:00401627 CALL 0040192E
001B:0040162C MOV EAX,[ESP+08]
001B:00401630 INC ESI
001B:00401631 MOV ECX,[EAX-08]
001B:00401634 DEC ECX
001B:00401635 CMP ESI,ECX
001B:00401637 JL 004015F7  ; "наивысший" адрес из всех
001B:00401637    ; 4015F7 - начало цикла генератора
001B:00401637    ; 401637 - конец  цикла генератора
001B:00401639 LEA EAX,[ESP+10]
001B:0040163D LEA ECX,[EDI+60]
001B:00401640 PUSH EAX
001B:00401641 CALL 00401934
001B:00401646 MOV ECX,[ESP+10]
001B:0040164A MOV EDX,[ESP+0C]
001B:0040164E PUSH ECX
001B:0040164F PUSH EDX
001B:00401650 CALL [MSVCRT!_mbscmp] ; ђ тут сравниваются строки
     ;    очевидно это конец генератор
```

Прежде нем приступать к восстановлению алгоритма генерации регистрационных номеров, отметим, что отладчики вообще-то не предназначены для декомпиляции кода и нам лучше прибегнуть к помощи дизассемблера. Найти же в дизассемблерном листинге требуемый фрагмент очень просто, - ведь адрес процедуры генератора нам уже известен. Для быстрого перемещения к

исследуемому коду в IDA достаточно отдать к консоли команду Jump(0x4015EF) , а в HIEW'e - , ".4015EF". Так или иначе мы встретим следующие строки (а еще лучше, если из мазохистских соображений, мы будем анализировать этот код под отладчиком, поскольку дизассемблер - особенно IDA - доступен не всем):

```
001B:004015EF PUSH ESI
001B:004015F0 XOR ESI,ESI
001B:004015F2 DEC ECX
001B:004015F3 TEST ECX,ECX
001B:004015F5 JLE 00401639
```

Регистр ESI здесь инициализируется явно (ESI ^ ESI := 0), а вот чему равен ECX?! Прокручиваем экран отладчика вверх до тех пор, пока не встретим машинную команду, присваивающую ECX то или иное значение:

```
001B:004015D8 MOV EAX,[ESP+04]
001B:004015DC MOV ECX,[EAX-08]
001B:004015DF CMP ECX,0A
001B:004015E2 JGE 004015EF
```

Ага, здесь в ECX пересылается значение ячейки по адресу [EAX-08], но что это за ячейка и куда указывает сам EAX? Что ж, под отладчиком (в отличии от дизассемблера) его содержимое очень просто подсмотреть! Достаточно дать команду "D EAX" и область памяти на которую указывает EAX немедленно отобразится в окне дампа:

```
:d eax
0023:002F4018 4B 72 69 73 20 4B 61 73-70 65 72 73 6B 79 00 00  Kris Kaspersky..
0023:002F4028 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
0023:002F4038 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
0023:002F4048 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
```

Да это же только что введенная нами строка! А в регистр ECX тогда загружается что? Смотрим: так, значение ECX равно 0Eh или 14 в десятичной системе исчисления. Очень похоже на длину этой строки (как известно, MFC -строки, точнее объекты класса CString хранят свою длину в специальном 32-разрядном поле, родимым пятном которого как раз и является смещение на 8 байт влево относительно начала самой строки). Действительно, имя "Kris Kaspersky" как раз и насчитывает ровно 14 символов (считая вместе с пробелом). Тогда становятся понятными две следующие машинные команды: CMP ECX,0Ah/JGE 4015EFh, осуществляющие контроль строк на соответствие минимально допустимой длине. При попытке ввода имени, состоящего из девяти или менее символов, программа откинет его как непригодное для регистрации. Это важный момент! Многие хакеры игнорируют подобные тонкости алгоритма и создают не вполне корректные генераторы, не осуществляющие таких проверок вообще. Как следствие - пользователь вводит свое короткое имя в генератор (например, "KPNC"), получает регистрационный код, подсовывает его защите и… обложив матом хакера, вводит в генератор другое имя - на сей раз подлиннее. А если защита имеет ограничение на предельно допустимую длину? Сколько так пользователю придется мотаться между защитой и генератором?

Ладно, оставим вопросы профессиональной этики и вернемся к коду генератора, черкнув в лежащем справа от Клавы листке белой бумаги, что EAX указывает на имя пользователя, а ECX содержит его длину.

```
001B:004015F2 DEC ECX
001B:004015F3 TEST ECX,ECX
001B:004015F5 JLE 00401639
```

Здесь: мотаем цикл до тех пор, пока не будут обработаны все символы строки (читатели, знакомые с "Фундаментальными основами хакерства" уже наверняка распознали в этой конструкции цикл for).

Теперь заглянем в тело цикла, спустившись еще на одну строчку вниз:

```
001B:004015F7 MOV CL,[EAX+ESI]
```

Здесь происходит загрузка очередного символа строки (и именно этот код вызвал всплытие отладчика при установленной точке останова, так что, надеюсь, вы его все еще помните). Поскольку, EAX - указатель на имя, то ESI с большой степени вероятности - параметр цикла. Правда, немного странно, что очередной символ строки помешается в младший байт регистра ECX, который судя по всему представляет собой счетчик цикла, но это все потом… Пока же мы нам известно лишь то, что начальное значение ESI равно нулю, а потому строка скорее всего обрабатывается от первого до последнего символа (хотя некоторые защиты поступают и наоборот).

```
001B:004015FA MOVSX AX,BYTE PTR [EAX+ESI+01]
```

MOVe whith Signed eXtension (пересылка со знаковым расширением) загружает следующий байт строки в регистр AX, автоматически расширяя его до слова и загаживая тем самым указатель на саму строку с именем. На редкость уродливый код! Но дальше - больше.

```
001B:00401600 MOVSX CX,CL
```

Преобразуем первый прочитанный символ строки к слову (обратим внимание, что здесь и далее под "первым" и "вторым" символом мы будем понимать отнюдь не NameString[0] и NameString[0], а NameString[ESI] и NameString[ESI + 1] соответственно, а сам ESI условно обозначим как index или, сокращенно, idx). Обратим внимание на несовершенство компилятора. Эту команду можно было записать более экономно как MOVSX CX, [ESI+EAX]

```
001B:00401604 IMUL EAX,ECX
```

Подставим вместо регистров их смысловые значения, мы получаем: EDX:EAX := NameString[idx] * String[idx + 1]

```
001B:00401607 AND EAX,0000FFFF
```

Преобразуем EAX к машинному слову, откидывая старшие 16 бит.

```
001B:0040160C AND EAX,8000001F
```

Выделяем пять младших бит от оставшегося слова (почему именно пять? просто переведите 1Fh в двоичную форму и сами увидите). Так же, выделяется и старший, знаковый, бит слова, однако, он всегда равен нулю, так как его принудительно сбрасывает предыдущая команда. Зачем же тогда его компилятор так старательно выделает? Осел он - вот почему. Программист присваивает результат беззнаковой переменной, вот компилятор и понимает его буквально!

```
001B:00401611 JNS 00401618
```

Если знаковый бит не установлен (ха! а с какой такой радости ему быть установленным?!), то прыгаем на 401618h. Ну что ж! Прыгаем, так прыгаем, избавляя себя от "радости" анализа нескольких никогда не исполняющихся команд защитного кода:

```
001B:00401618 ADD AL,41
001B:0040161A LEA ECX,[ESP+0C]
001B:0040161E MOV [ESP+14],AL
001B:00401622 MOV EDX,[ESP+14]
```

Первая машинная команда добавляет к содержимому регистра AL константу 41h (литера 'A' в символьном представлении) и полученная сумма перегоняется в регистр EDX, минуя по пути локальную переменную [ESP + 14].

С конструкцией LEA ECX, [ESP + 0Ch] разобраться несколько сложнее. Во-первых, ячейка [ESP +0Ch] явным образом не инициализируется в программе, а, во-вторых, значение регистра ECX ни здесь, ни далее не используются. Если бы оптимизирующие компиляторы не выкидывали все лишние операции присвоения (т .е. такие, чей результат не используется), мы бы просто списали эту команду на ляп разработчика защитного механизма, но сейчас такая стратегия уже не проходит. К тому же это удачный повод для знакомства с плавающими фреймами, без умения работать с

которыми невозможно побороть практически ни одну современную защиту.

Для начала давайте вспомним устройство "классического" кадра стека. При выходе в функцию компилятор сохраняет в стеке прежнее значение регистра EBP (а так же при желании и всех остальных регистров общего назначения, если они действительно должны быть сохранены), а затем приподнимает регистр ESP немного "вверх", резервируя тем самым то или иное количество памяти для локальных переменных. Область памяти, расположенная между сохраненным значением регистра EBP и новой вершиной стека, и называется кадром. Начальный адрес только что созданного кадра копируется в регистр EBP, и этот регистр используется в качестве опорной точки для доступа ко всем локальным переменным. По мере разбухания стека поверх кадра могут громоздиться и другие данные, заталкиваемые туда машинными командами PUSH и PUSHF (например: аргументы функций, временные переменные, сохраняемые регистры и т. д.). Достоинство этой системы заключается в том, что для доступа к локальным переменным нам достаточно знать всего лишь одно число - смещение переменной относительно вершины кадра стека. Благодаря этому, машинные команды, обращающиеся к одной и той же локальной переменной, из какой бы точки функции они ни шли, выглядят одинаково. То есть, нам не требуется никаких усилий, чтобы догадаться, что MOV EAX, [EBP + 69h] и MOV [EBP + 69h], ECX в действительности обрабатывают одну локальную переменную, а не две. Между прочим, вы зря смеетесь! Хотите получить кукурузный початок в зад? Ну так получайте! (Знаю, что больно, но ведь я же предупреждал!).

Поскольку регистров общего назначения в архитектуре IA-32 всего семь, то отдавать даже один из них на организацию поддержки фиксированного кадра стека по меньшей мере не логично, тем более, что локальные переменные можно адресовать и через ESP. Ну и в чем же разница? - спросите вы. А разница между тем принципиальна! В отличии от EBP, жестко держащего верхушку кадра за хвост, значение ESP изменяется всякий раз, когда в стек что-то вложат или, наоборот, что-то вытащат оттуда. Рассмотрим это на следующем примере: MOV EAX, [ESP+10h]/PUSH EAX/MOV ECX, [ESP + 10h]/PUSH ECX/MOV [ESP + 18h], EBP, - как вы думаете, к каким локальным переменным здесь происходит обращение? На первый взгляд, значение ячейки [ESP + 10h] дважды засылается в стек, а затем в ячейку [ESP +18h] копируется содержимое регистра EBP. На самом же деле тут все не так! После засылки в стек содержимого регистра EAX, указатель вершины стека приподнимается на одно двойное слово вверх и дистанция между ним и локальными переменными неотвратимо увеличивается! Следующая машинная команда - MOV ECX, [ESP + 10h] на самом деле копирует в регистр ECX содержимое совсем другой ячейки! А вот [ESP + 18h] после засылки ECX указывает на ту же самую ячейку, что вначале копировалась в регистр EAX. Ну и как теперь насчет "посмеяться"?

Такие оптимизированные кадры стека по-русски называются "плавающими", а в англоязычной литературе обычно обозначаются аббревиатурой FPO - Frame Pointer Omission. Это едва ли не самое страшное проклятие для хакеров. Основной камень преткновения заключается в том, что для определения смещения переменной в кадре мы должны знать текущее состояние регистра ESP, а узнать его можно лишь путем отслеживания всех предшествующих ему машинных команд, манипулирующих с указателем верхушки стека и, если мы случайно упустим хоть одну из них, вычисленный таким трудом адрес локальной переменной окажется неверным! Следовательно, неверным окажется и результат дизассемблирования!!! Вернемся к нашему примеру LEA ECX, [ESP + 0Ch]. Будем прокручивать экран "CODE" отладчика вверх до тех пор, пока не обнаружим пролог функции или не накопим по меньшей мере 0Ch байт, закинутых на стек командами PUSH (в квадратных скобках показано смещение соответствующих ячеек относительно вершины стека на момент вызова нашего LEA).

```
001B:00401580 PUSH FF   [ +24h]
001B:00401582 PUSH 00401C48  [ +20h]
001B:00401587 MOV EAX,FS:[00000000]
001B:0040158D PUSH EAX   [ +1Ch]
```

```
001B:0040158E MOV FS:[00000000],ESP
001B:00401595 SUB ESP,10   [ +18h] (40161A:04h)
001B:00401598 PUSH EDI   [ +08h]
001B:00401599 MOV EDI,ECX
…
001B:004015CD PUSH EAX   [ +04h]
…
001B:004015EF PUSH ESI   [ +00h]
```

Ну, что Шура, я Вам могу сказать, - если считать, что SUB ESP, 10h открывает фрейм функции, то LEA ECX, [ESP + 0Ch] лежит по смещению 04h от его начала, - аккурат посередине. А что у нас здесь? Листаем код ниже (в квадратных скобках показано смещение соответствующих ячеек относительно начала кадра стека):

```
001B:00401595 SUB ESP,10   [ +00h]
001B:00401598 PUSH EDI   [ +20h]
001B:00401599 MOV EDI,ECX
001B:0040159B LEA ECX,[ESP+04]  [ +00h]
001B:0040159F CALL 40190Ah
001B:004015A4 LEA ECX,[ESP+0C]  [ +08h]
001B:004015A8 MOV DWORD PTR [ESP+1C],00h
001B:004015B0 CALL 40190Ah
001B:004015B5 LEA ECX,[ESP+08]  [ +04h]
001B:004015B9 MOV BYTE PTR [ESP+1C],01
001B:004015BE CALL 40190Ah
```

Ага! Вот теперь мы видим, что указатель на локальную переменную, расположенную по смещению 04h от начала кадра стека (далее просто var_04h) передается функции 40190Ah очевидно для ее, переменной, инициализации. Но вот что делает эта загадочная функция? Если, находясь в отладчике, нажать для входа в ее тело, мы обнаружим следующий код:

```
001B:0040190A JMP [00402164h]
```

Узнаете? Ну да, это характерный способ вызова функций из динамических библиотек. Но вот какая функция какой именно библиотеки сейчас вызывается? Ответ хранит ячейка 402164h, содержащая непосредственно сам вызываемый адрес. Посмотрим ее содержимое?

```
:dd
:d 402164
0010:00402164 6C29198E  6C294A70  6C2918DD  6C298C74      ..)lpJ)l..)lt.)l
```

Остается только узнать какому модулю принадлежит адрес 6C9198Eh. Не выходя из soft-ice даем ему команду "mod" и смотрим (протокол, приведенный ниже по понятным соображениям сильно сокращен):

```
Base  PEHeader Module Name  File Name
10000000 10000100 pdshell  \WINNT\system32\pdshell.dll
6C120000 6C1200A8 mfc42loc  \WINNT\system32\mfc42loc.dll
6C290000 6C2900F0 mfc42  \WINNT\system32\mfc42.dll
6E380000 6E3800C8 indicdll  \WINNT\system32\indicdll.dll
```

Легко видеть, что адрес 6C29199Eh принадлежит модулю MFC42.DLL, что совершенно неудивительно ввиду того, что данная программа действительно интенсивно использует библиотечку MFC. Чтобы не вычислять принадлежность всех остальных функций вручную давайте просто загрузим символьную информацию из MFC42.DLL в отладчик. Запустив NuMega "Symbol Loader" (если только вы еще не сделали этого ранее), выберите команду "Load Exports" в меню "File", а затем, перейдя в папку "\WINNT\System32\" дважды щелкните по строке с именем "MFC42.DLL". Теперь, тот же самый код под отладчиком будет выглядеть так:

```
001B:004015B5 LEA ECX,[ESP+08]
001B:004015B9 MOV BYTE PTR [ESP+1C],01
001B:004015BE CALL MFC42!ORD_021B
```

Умница soft-ice определил не только название динамической библиотеки, экспортирующей вызываемую функцию, но и ее ординал! Что же касается имени функции, его можно вычислить с

помощью DUMPBIN и библиотеки MFC42.lib. Даем команду "DUMPBIN /HEADRES MFC42.LIB >MFC42.headrs.txt" и затем в образовавшемся файле простым контекстным поиском ищем строку "Ordinal : 539", где "539" наш ординал 021Bh записанный в десятичном виде (именно так выдает ординалы этот dumpbin). Если все идет пучком, мы должны получить следующую информацию:

```
Version      : 0
Machine      : 14C (i386)
TimeDateStamp: 35887C4E Thu Jun 18 06:32:46 1998
SizeOfData   : 00000020
DLL name     : MFC42.DLL
Symbol name  : ??0CString@@QAE@PBG@Z (__thiscall CString::CString(unsigned short *))
Type         : code
Name type    : ordinal
Ordinal      : 539
```

Так, это конструктор объекта типа CString, а указатель, передаваемый ему, стало быть и есть тот самый this, что указывает на свой экземпляр CString! Следовательно, var_4 - это локальная переменная типа "MFC-строка". Теперь, не грех вернуться к изучению прерванной темы (а прервали мы ее на строке 40161Ah, где осуществлялась загрузка указателя на var_4 в регистр ECX посредством машинной команды LEA; регистр же EDX, как мы помним, содержит в себе результат умножения двух символов исходной строки, преобразованный в литерал):

```
001B:00401626 PUSH EDX
001B:00401627 CALL MFC42!ORD_03AB
```

Следующими двумя командами мы заталкиваем полученный литерал в стек, передавая его в качестве второго аргумента функции MFC42!ORD_03AB (первый аргумент функций типа __thiscall передается через регистр ECX, содержащий указатель на экземпляр соответствующего объекта, с которым мы сейчас и манипулируем). Преобразовав ординал в символьное имя функции, мы получаем "оператор +=", что очень хорошо вписывается в обстановку окружающей действительности. Другими словами, здесь осуществляется посимвольное наращивание строки var_4 генерируемыми налету литералами.

```
001B:0040162C MOV EAX,[ESP+08]
```

Что у нас в [ESP + 8]? Прокручивая экран с дизассемблерным листингом вверх, находим, что здесь лежит самая первая ячейка из принадлежащих кадру стека. Условимся называть ее var_0. Давайте определим, что же за информация в ней находится?

```
001B:00401595 SUB ESP,10    ; [ +00h]
001B:00401598 PUSH EDI    ; [ +04h]
…
001B:004015C3 LEA EAX, [ESP+04]   ; var_0
001B:004015C7 LEA ECX,[EDI+000000A0]
001B:004015CD PUSH EAX    ; [ +08h]
001B:004015CE MOV BYTE PTR [ESP+20],02
001B:004015D3 CALL MFC42!ORD_0F21   ; CWnd::GetWindowText
```

Кое-что начинает уже проясняться. Переменная var_0 содержит указатель на MFC-строку, бережно хранящую в себе регистрационное имя пользователя.

```
001B:00401630 INC ESI
```

Указатель текущего символа перемещается на одну позицию вправо (ведь вы помните, что в ESI содержится именно указатель на текущий обрабатываемый символ регистрационной строки, верно?).

```
001B:00401631 MOV ECX,[EAX-08]   ; EAX := var_4
001B:00401634 DEC ECX
001B:00401635 CMP ESI,ECX
001B:00401637 JL 004015F7
```

Первая машинная команда из четырех загружает длину регистрационной MFC-строки в регистр ECX, команда "DEC" уменьшает ее на единицу, а "CMP ESI, ECX" сравнивает полученное значение

с индексом текущего обрабатываемого символа регистрационной строки. И, до тех пор, пока индекс не достигнет предпоследнего символа строки, условный переход "JL" прыгает на адрес 4015F7h, мотая цикл.

```
001B:00401639 LEA EAX,[ESP+10]
001B:0040163D LEA ECX,[EDI+60]
001B:00401640 PUSH EAX
001B:00401641 CALL MFC42!ORD_0F21
001B:00401646 MOV ECX,[ESP+10]
001B:0040164A MOV EDX,[ESP+0C]
001B:0040164E PUSH ECX
001B:0040164F PUSH EDX
001B:00401650 CALL [MSVCRT!_mbscmp]
```

По факту завершения цикла, защита сравнивает только что сгенерированную ей строку с регистрационным номером, введенным пользователем и, в зависимости от результатов этого сравнения, пользователь либо признается легальным чувяком, либо получает от ворот поворот.

Брр! Вы еще не запутались?! Что ж, тогда давайте подытожим все вышесказанное краткими комментариями к защитному коду:

```
:ESI            = 0 (индекс)   [index];
:[ESP+08h], EAX - на регистрационную строку [NameString];
:[ESP+0Ch]      - на генерируемую строку [GenString]
001B:004015F7 MOV CL,[EAX+ESI]  ; CL := (char) NameString[index]
001B:004015FA MOVSX AX,BYTE PTR [EAX+ESI+1];AX := (uint)((char) NameString[index+1])
001B:00401600 MOVSX CX,CL   ;
001B:00401604 IMUL EAX,ECX   ; EAX := EAX * ECX
001B:00401607 AND EAX,0000FFFF  ; EAX := LOW_WORD(EAX)
001B:0040160C AND EAX,8000001F  ; EAX := EAX ^ 1Fh
001B:00401611 JNS 00401618  ; GOTO 401618h
001B:00401618 ADD AL,41   ; EAX := EAX + 'A'
001B:0040161A LEA ECX,[ESP+0C]  ; ECX := &GenString
001B:0040161E MOV [ESP+14],AL  ; tmp := AL
001B:00401622 MOV EDX,[ESP+14]  ; EDX := tmp
001B:00401626 PUSH EDX   ;
001B:00401627 CALL 0040192E  ; GetString += EDX
001B:0040162C MOV EAX,[ESP+08]  ; EAX := &NameString
001B:00401630 INC ESI   ; index++
001B:00401631 MOV ECX,[EAX-08]  ; ECX := NameString->GetLength()
001B:00401634 DEC ECX   ; ECX--
001B:00401635 CMP ESI,ECX   ;
001B:00401637 JL 004015F7  ; if (index < ECX) GOTO 4015F7h
```

Вот теперь - другое дело и нам уже ничего не стоит восстановить исходный код генератора.

```
for (int idx=0;idx>String.GetLength()-1;idx++)
 RegCode+= ((WORD) sName[a]*sName[a+1] % 0x20) + 'A';
```

Остается лишь написать собственный генератор регистрационных номеров. Это можно сделать на любом симпатичном вам языке, например на ассемблере. На диске находится один вариант (file://CD/SRC/crackme.58DD2D69h/HACKGEN/KeyGen.asm). Ключевая процедура может выглядеть так:

```
; ГЕНЕРАЦИЯ РЕГИСТРАЦИОННОГО НОМЕРА
; ===========================================================================
 MOV ECX, [Nx]  ; ECX := strlen(NameString)
 SUB ECX, 2  ; выкусываем перенос строки
 DEC ECX  ; уменьшаем длину строки на единицу
 MOV EBX, 20h  ; магическое число
 LEA ESI, hello ; указатель на буфер с именем пользователя
 LEA EDI, buf_in ; ^ указатель на буфер для генерации

; ЯДРО ГЕНЕРАТОРА
; ===========================================================================
gen_repeat:  ;<<<-----------------------------------------; CORE
 LODSW  ; читаем слово    ; CORE
 MUL AH ; AX := NameString[ESI]*NameString[ESI+1] ; CORE
 XOR EDX, EDX ; EDX := NULL    ; CORE
```

```
  DIV EBX ; DX := NameString[ESI]*NameString[ESI+1] % 1Ah ; CORE
  ADD EDX, 'A' ; переводим в символ   ; CORE
    ;     ; CORE
  XCHG EAX, EDX ;     ; CORE
  STOSB  ; записываем результат   ; CORE
  DEC ESI ; на символ назад    ; CORE
LOOP gen_repeat ; ---- цикл ---------------------------------->>> ; CORE
```

Испытаем написанный генератор. Запустив откомпилированный файл KeyGen.exe на выполнение, введем в качестве регистрационного имени какую ни будь текстовую строку (например, свое собственное имя или псевдоним), - не пройдет и секунды как генератор выдаст подходящий regnum в ответ. В частности, имени "Kris Kaspersky" соответствует следующий регистрационный код: "GCLAALTQQ[WRT"

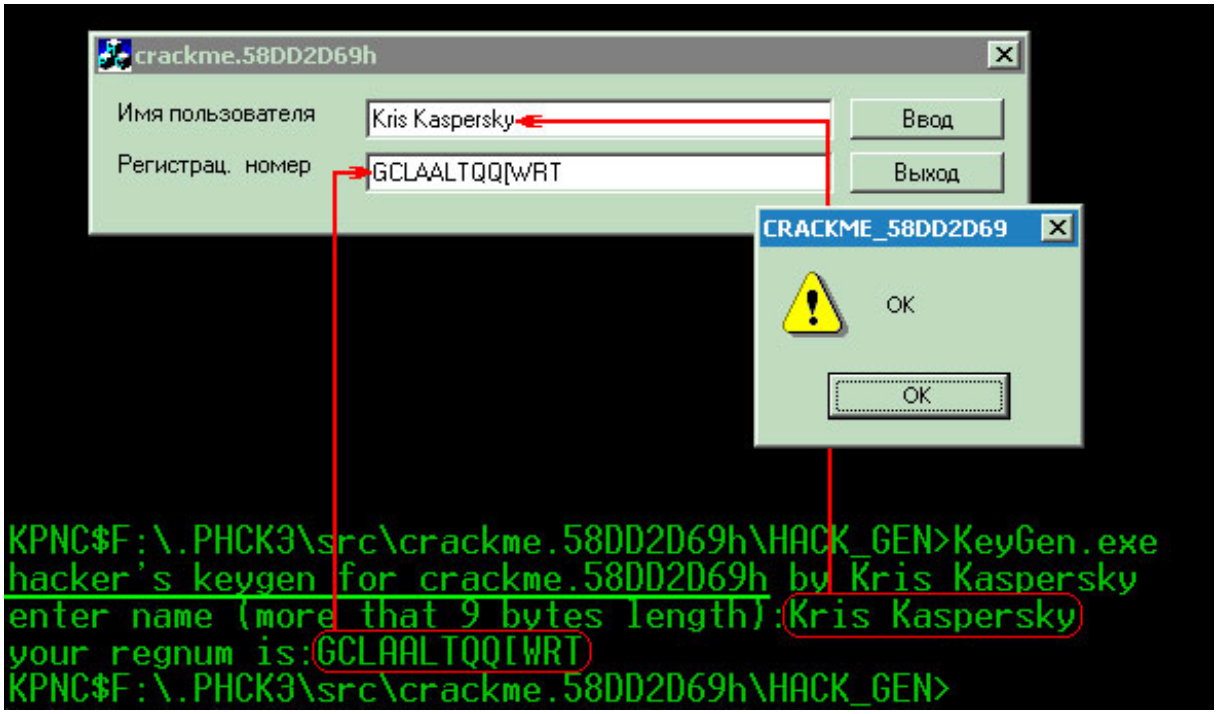


Рисунок 5 0x00C демонстрация работы ключеделки

Генератор успешно работает и вычисляет правильные регистрационные номера. Однако, вводить регистрационный номер вручную не только утомительно, но и неэлегантно. Да, можно скопировать его и через буфер обмена, но все равно возня будет еще та. В конечном итоге, компьютер на то и придуман, чтобы служить пользователю, но не наоборот. Идеальный crack - это такой crack который не докучает пользователю теми вопросами, ответ на которые знает сам, равно как и не требует от последнего никаких действий, которые он может выполнить и самостоятельно. Единственное, что требует такой crack - своего запуска. Короче, хорошая программа должна заботиться о себе сама!

Первое, что приходит на ум: просто пропадчить защитный код на диске или в памяти. В предыдущей главе мы как раз разбирали как это сделать. Однако, падчики, во-первых, просто вопиюще незаконны, во-вторых, крайне чувствительны к версии билда. Генераторы регистрационных номеров, напротив, весьма мирно уживаются с уголовным кодексом, поскольку они не подделывают, а именно генерируют регистрационный номер на основе имени, введенного пользователем (см. эпиграф) и их написание столь же "незаконно", сколько открытие мастерской по изготовлению дубликатов ключей например. К тому же алгоритм генерации регистрационного номера если и изменяется, то во всяком случае не в каждой версии программы .

Во времена старушки MS-DOS эта проблема решалась перехватом прерывания int 16h с целью эмуляции ввода с клавиатуры. Ломалка, грубо говоря, прикидывалась пользователем и

подсовывала защищенной программе сначала имя, а затем и сгенерированный регистрационный номер. От самого же пользователя не требовалось ничего, кроме запуска такой программы. Ну разве не красота? К сожалению, с переходом на Windows прямой контроль над прерываниями оказался безвозвратно утерян и все трюки старой Лисы перестали работать…

Но, "мало того, что их сосед в жилом доме свинью держит, так он еще и круглосуточно над ней измывается..." . Незадачливого музыканта подвела хорошая межквартирная слышимость (читай: хреновая звукоизоляция). Так вот, Windows с точки зрения безопасности - та же хрущоба и слышимость в ней о-го-го! Архитектура подсистемы пользовательского интерфейса, достающаяся NT/9x в наследство от незаконно рожденной Windows 1.0, неотделима от концепции сообщений (messages) - эдакой собачей будке, перенесенной с заднего двора на самое видное место. Любой процесс в системе может посылать сообщения окнам любого другого процесса, что позволяет ему управлять этими окнами по своему усмотрению. Хотите "подсмотреть" содержимое чужого окна? Пожалуйста! Пошлите ему SendMessage с WM_GETTEXT и все дела! Хотите послать окну свою строку с приветствием? Нет проблем, - SendMessage в купе с WM_SETTEXT спасут отца русской демократии! Аналогичным образам вы можете нажимать на кнопки, двигать мышь, раскрывать пункты меню, словом полностью контролировать работу приложения. Самое интересное, что уровень привилегий при этом никак не проверяется, - процесс с гостевыми правами может свободно манипулировать окнами, принадлежащими процессу-администратору. Знаете, в NT/w2k есть такое забавное окошко "запуск программы от имени другого пользователя", обычно используемое для запуска привилегированных приложений из сеанса непривилегированного пользователя? Ну вот например захотели проверить вы свой жесткий диск на предмет целостности файловой структуры, а перезапускать систему под "Администратором" вам лень (точнее, просто не хочется закрывать все активные приложения). На первый взгляд никакой угрозы для безопасности в этом нет, ведь "запуск программы от имени другого пользователя" требует явного ввода пароля! А вот получи треска гранату, - любое злопакостное приложение сможет перехватить ваш пароль только так! Причем, речь идет не о какой-то непринципиальной недоработке, которая легко устранима простой заплаткой (в просторечии называемой "падчем").

Нет! Все так специально и задумывалось. Не верите? Откроем Рихтера "…система отслеживает сообщения WM_SETTEXT и обрабатывает их не так, как большинство других сообщений. При вызове SendMessage внутренний код функции проверяет, не пытаетесь ли вы послать сообщение WM_SETTEXT. Если это так, функция копирует строку из вашего адресного пространства в блок памяти и делает его доступным другим процессам. Затем сообщение посылается потоку другого процесса. Когда поток-приемник готов к обработке WM_SETTEXT, он определяет адрес общего блока памяти (содержащего новый текст окна) в адресном пространстве своего процесса. Параметру lParam пристраивается значение именного этого адреса, и WM_SETTEXT направляется нужной оконной процедуре. Не слишком ли накручено, а?" Выходит, разработчики оконной подсистемы искусственно и крайне неэлегантно обошли подсистему защиты Windows, разделяющую процессы по их адресным пространствам. Естественно, это делалось отнюдь не с целью диверсии, - просто запрети Microsoft посылку сообщений между процессами куча существующих приложений (написанных большей частью под Windows 3.x) тут же перестала бы работать! А значит, эмуляция ввода с клавиатуры жила, жива и будет жить!

Единственное, что нужно знать - так это дескриптор (handle) окна, которого вы хотите "осчастливить" своим сообщением. Существует множество путей получить эту информацию. Можно например воспользоваться API-функцией FindWindow, которая возвращает дескриптор окна по его названию (текстовой строке, красующейся в заголовке) или тупо переворошить все окна одно за другим, в надежде что рано или поздно среди них встретиться подходящее. Перечисление окон верхнего уровня осуществляется функцией EnumWindows, а дочерних окон (к которым диалоговые элементы управления как раз и принадлежат) - EnumChildWindows.

Собственно, получить дескриптор главного окна ломаемого приложения - не проблема, ведь мы знаем его имя, которое в большинстве случаев однозначно идентифицирует данное окно среди прочих запущенных приложений. С дочерними окнами справиться не в пример сложнее. Ладно, кнопки еще можно распознать по их надписи (получаем дескрипторы всех дочерних окон вызовом EnumChildWindows, а затем посылаем каждому из них сообщение WM_GETTEXT с требованием сказать как кого зовут, после чего нам останется лишь сопоставить дескрипторы кнопок с их названиями). К сожалению с окнами редактирования такой фокус не пройдет, ибо по умолчанию они вообще не содержат в себе никакой информации, - вот и разбирайся это окно для ввода регистрационного имени или номера?

На помощь приходит тот факт, что порядок перечисления окон всегда постоянен и не меняется от одной операционной системе к другой. То есть, определив назначения каждого из дочерних окон экспериментально (или с помощью шпионских средств типа Spyxx из комплекта SDK) мы можем жестко прописать их номера в своей программе. Например, применительно к crackme.58DD2D69h это может выглядеть так: запускаем наш любимый soft-ice и даем команду "HWND" для выдачи списка всех окон, включая дочерние, зарегистрированных в системе.

```
0B0416    #32770 (Dialog)              6C291B81    43C CRACKME_
  0B0406    Button                      77E18721    43C CRACKME_
  0B040A    Static                      77E186D9    43C CRACKME_
  0D0486    Edit                        6C291B81    43C CRACKME_
  0904C6    Static                      77E186D9    43C CRACKME_
  0D0412    Edit                        6C291B81    43C CRACKME_
  0A047C    Button                      77E18721    43C CRACKME_
```

Ага! Вот они окна редактирования (см. текст выделенный жирным шрифтом), - третье и пятое по счету дочернее окно в списке перечисления. Одно из них наверняка принадлежит строке регистрационного имени, а другое - регистрационного номера. Но как узнать какое кому? Воспользовавшись ключом xc, заставим sof-ice выдать более подробную информацию по каждому из окон:

```
HWND -xc
 Hwnd  : 0D0486    (A0368EF8)
 Class Name : Edit
 Module  : CRACKME_
 Window Proc : 6C291B81 (SuperClassed from: 77E19896)
 Win Version : 0.00
 Parent  : 0B0416    (A0368A88)
 Next  : 0904C6    (A0368FB8)
 Style  :
 Window Rect : 387, 546, 615, 566 (228 x 20)
 Client Rect : 2, 2, 226, 18 (224 x 16)
 …
 Hwnd  : 0D0412    (A03690A8)
 Class Name : Edit
 Module  : CRACKME_
 Window Proc : 6C291B81 (SuperClassed from: 77E19896)
 Win Version : 0.00
 Parent  : 0B0416    (A0368A88)
 Next  : 0A047C    (A0369168)
 Style   :
 Window Rect : 387, 572, 615, 592 (228 x 20)
 Client Rect : 2, 2, 226, 18 (224 x 16)
```

Как легко установить по координатам вершин окон, первое из них находится на 26 пикселей выше другого (546 против 572), следовательно первое окно - окно регистрационного имени, а второе - окно регистрационного номера.

Теперь, когда порядковые номера окон редактирования известны можно накрапать следующую несложную программку:

```
// ПЕРЕЧИСЛЕНИЕ ДОЧЕРНИХ ОКОН crackme
// ==========================================================================
// получаем хэндлы всех интересующих нас окон
```

```
// (порядок окон определяем либо экспериментально, либо тестовым прогоном
// с отладочным выводом информации по каждому из окон)
BOOL CALLBACK EnumChildWindowsProc(HWND hwnd,LPARAM lParam)
{
 static N = 0;

 switch(++N)
 {
  case 3: // окно с именем пользователя
    username = hwnd;
    break;

  case 4: // text со строкой "reg. num."
    hackreg = hwnd;
    break;

  case 5: // окно для ввода регистрационного номера
    regnum = hwnd;
    break;

  case 6: // конопка ввода
    input_but = hwnd;
    return 0;
 }
 return 1;

}
```

Теперь перейдем непосредственно к технике эмуляции ввода. Ну, ввод/вывод текста в окна редактирования больших проблем не вызывает: WM_SETTEXT/WM_GETTEXT и все пучком, а вот "программно" нажать на кнопку несколько сложнее. Но ведь вам же хочется, чтобы программа не только ввела в соответствующие поля всю необходимую регистрационную информацию, но и самостоятельно долбанула по , чтобы закончить ввод?!

Как показывает практика, посылка сообщения BM_SETSTATE элементу управления типа "кнопка" не приводит к ее нажатию. Почему? Наша ошибка заключается в том, что для корректной эмуляции ввода мы во-первых, должны установить фокус (WM_SETFOCUS), а после перевода кнопки в состояние "нажато" этот фокус убить (WM_KILLFOCUS), ведь, как известно даже желторотым пользователям, кнопки срабатывают не в момент их нажатия, но в момент отпускания. Не верите? Поэкспериментируйте с любым приложениям и убедитесь в справедливости сказанного. Кстати, забавный трюк: если под NT/w2k в сообщение WM_KILLFOCUS передать недействительный дескриптор окна, получающего на себя бразды правления, то операционная система по понятным соображениям не передаст фокус несуществующему окну, но у активного окна фокус все-таки отберет. Windows 9x, напротив, оставляет фокус активного окна неизменным! Вот такая разница между двумя операционными системами. Еще одна делать на последок. Если в роли убийцы фокуса выступает функция SendMessage по поток, эмулирующий ввод, блокируется вплоть до того момента, пока обработчик нажатия кнопки не возвратит циклу выборки сообщений своего управления. Чтобы этого не произошло, - используйте функцию PostMessage, которая посылает убийцу фокуса и, не дожидаясь от него ответа, как ни в чем не бывало продолжает выполнение.

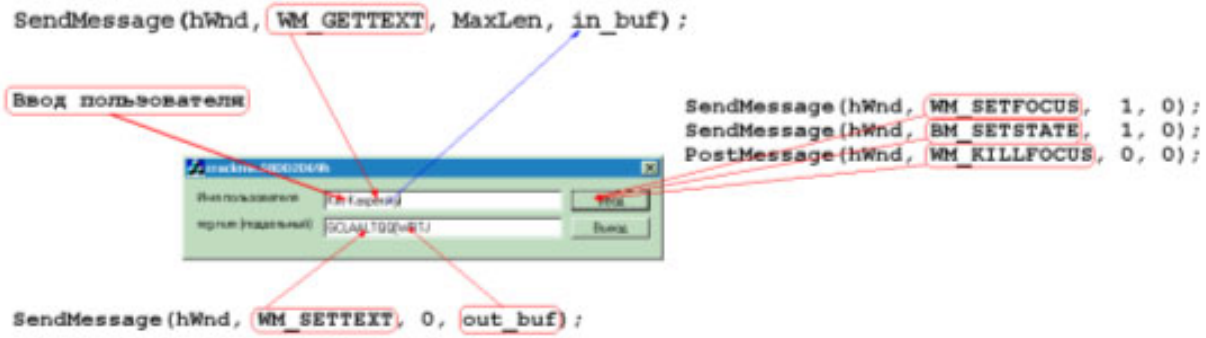


Рисунок 6 0x00D "автоматическое" считывание имени пользователя, ввод регистрационного номера и эмуляция нажатия на клавишу "ввод"

Испытаем наш автоматический регистратор? (file://CD/SRC/crack-me58DD2D69h/HACKGEN2/autocrack.c). Запустив защищенную программу и при желании заполнив поле имени пользователя (если его оставить пустым, автоматический регистратор использует имя по умолчанию), мы дрожащей от волнения рукой запускаем autocrack.exe… Держите нас! Это сработало! Вот это автоматизация! Вот это хакерство! Вот это мы понимаем! **[врезка] как сделать исполняемые файлы меньше**

Даже будучи написанным на чистом ассемблере, исполняемый файл генератора регистрационных номеров занимает целых 16 килобайт! Хорошенький монстр, нечего сказать! Хакерам, чей первый компьютер был IBM PC с процессором Pentium-4, может показаться, что 16 килобайт это просто фантастически мало, однако еще в восьмидесятых годах существовали компьютеры с объемом памяти равным этому числу! Впрочем, зачем нам так далеко ходить, - откроем первое издание настоящей книги: "Без текстовых строк исполняемый файл [генератора] занимает менее пятидесяти байт и еще оставляет простор для оптимизации". Сравните пятьдесят байт и шестнадцать килобайт, - переход с MS-DOS на Windows увеличил аппетит к памяти без малого в триста раз!

Вообще-то, с чисто потребительской точки зрения никакой проблемы в этом нет. Размеры жестких дисков сегодня измеряются сотнями гигабайт и лишний десяток килобайт особой погоды не делает. К тому же, наш исполняемый файл замечательно ужимается pkzip'ом до семисот с небольшим байт, что существенно для его передачи по медленным коммуникационным сетям, - да только где такие нынче найдешь?!

С чисто же эстетической точки зрения держать у себя такой файл действительно нехорошо. Обиднее всего, что на 99% генератор состоит из воздуха и воды, - нулей, пошедших на вырывание секций по адресам, кратным 4Кб. Три секции (кодовая секция .text, секция данных .data и таблица импорта .itable) плюс PE-заголовок, - вместе они эти самые 16 Кб и создают. Полезного же кода в исполняемом файле просто пшик - немногим менее двухсот байт. Конечно, двести это не пятьдесят и с переходом на Windows мы все равно проигрываем и в компактности, и в скорости, но все-таки кое-какой простор для оптимизации у нас имеется.

Начнем с того, что прикажем линкеру использовать минимальную кратность выравнивания из всех имеющихся, - составляющую всего четыре байта. Указывав в командной строке ключ "/ALIGN:4" мы сократим размер исполняемого файла с 16.384 до 1.032 байт! Согласитесь, что с таким размером уже можно жить!

Причем, это далеко не предел оптимизации! При желании можно: а) выкинуть MS-DOS stub, который все равно бесполезен; б) подчистить IMAGE_DIRECTORY; в) использовать незадействованные поля OLD EXE/PE-заголовков для хранения глобальных переменных; г) объединить секции .text, .data, .rdata в одну общую секцию, сведя тем самым эффективную кратность выравнивая к одному и высвободив еще трохи места за счет ликвидации двух секций. Словом, возможности для самовыражения под Windows все-таки имеются!

SentinelLM!Вступление

Автор: Quantum

Дата: 2003-07-01

Данная статья представляет собой первый выпуск элементарного пособия -- а-ля SentinelLM for Dummies -- которое должно помочь начинающему хакеру перейти в разряд хакеров подготовленных! Sentinel License Manager (далее просто SLM) является продвинутой системой защиты коммерческих приложений от взлома (пардон, имеется ввиду реверсинг в чисто познавательных целях). Разработчик SLM -- фирма Reinbow Technologies (www.reinbow.com).

Правда, SLM не является гарантией от... нашего с вами наивного любопытства. Дело в том, что некоторые софтверные гиганты крайне безграмотно используют защитный потенциал Sentinel, упрощая жизнь и без того скучающего хакера, и предоставляют удобные примеры для подобных статей.

Прежде всего, как распознать наличие защиты SLM? Обычно, пресловутые клиенты Sentinel поставляются вместе с комплектом драйверов Rainbow под названием вроде lserv.XXX, ldriv.XXX и т.п. Правда, наиболее бросающимся в глаза отличием можно считать присутствие электронного ключа для внешнего подключения (LPT, USB). Ключ (он же -- dongle) зачастую пугает новичков. Его всё-таки стоит бояться в серьёзных и грамотно построенных защитах (есть и такие!), но в случаях подобных тому, который мы в дальнейшем рассмотрим, гн. Донгл -- безобидный кусок электронно-пластикового лома размером 5,5 на 4 см. Ещё одна важная особенность SLM и её основная уязвимость -- это наличие текстового файла с лицензионной информацией. Имя файла произвольное, а содержимое примерно следующего типа:

```
Q843R7MXTCCQRM2VQKDSJLXEDJSR88N33SM4QZ# "XXX" version "n"
MR2M6XP5A9WJAVMYV4W7BO5TA5LAHSQ8KDSXX7# "XXX" version "n"
2Q36N2ZXJ5ANS95HMPGOEYX46HPSTB9X7V274Z# "XXX" version "n"
IKSI28479A7A7W823RS3K37SBEC7IESE9973H9# "XXX" version "n"
ZI1U2U245D66K6JNKQWJLQ4MUG7U2AI3GIHBQAENMY# "XXX" version "n"
SR78DMVBR8NGDOXW9ZAUDLBAPXHNKORQPUWWET2FBA# "XXX" version "n"
XNMMO7CB326SGKVT3GO7W2BKJGTQGMOK6A9BXXC9SM# "XXX" version "n"
25X2OCB28WO6R3XYIF473Z2PFY73XOTZ5O872JH5CY# "XXX" version "n"
```

После version "n" может идти expires и другие подробности лицензируемой фичи. На самом деле, парсер SLM воспринимает символ '#' как начало комментариев, т.е. он просто игнорирует всё остальное до конца строки.

Примечание: лицензионный файл может и отсутствовать.

Итак, настало время назвать почётного таргета нашего исследования. Пора рискнуть привлечь внимание антипиратских организаций... Сегодня на операционном столе побывает один из самых популярных IDE для VLSI дизайнеров на букву 'L'. Больше не скажу -- не камикадзе я :-) Так вот, для тех кто в танке: CD с 30-дневной полнофункциональной демо можно заказать на сайте TR. Если мой таргет не совпадает с вашим, не отчаивайтесь -- данное руководство применительно к большинству клиентов SLM (уже проверил на 5-и различных таргетах).

• Внимание! Перечень операционного инвентаря:
• SentinelLM SDK 7.1 (где достать -- писали в форуме wasm.ru)
• Сигнатуры SentinelLM для IDA Pro (лежат в разделе "инструменты")
• IDA Pro собственной персоной (даже не спрашивайте ;-)
• SoftIce (куда же без него?)
• MASM32
• Hiew
• ... удобное кресло перед компом!

Перед тем как приступить к делу, предлагаю разделить процесс на 5 этапов:

1. Получение Vendor Id нашей жертвы.
2. Генерация S/N для Vendor Id из предыдущего пункта.
3. Долгое и нудное -- но одноразовое -- кряканье над SDK.
4. Успешное получение новой постоянной плавающей лицензии.
5. Осторожное проделывание отверстия в донгле для последующего превращение оного в стильный брелок.

Вот, собственно, и всё. Качайте SDK и прочий инструментарий, внимательно изучайте лицензионный файл, читайте доки о внутреннем устройстве донгла (для общего развития)... Дописываю первую главу о Vendor Id. Жду комменты и пожелания.

SentinelLM!Vendor_Id

Автор: Quantum

Дата: 2003-07-01

Vendor Id (далее просто VId) -- уникальное 16-битное значение в диапазоне 0x0000 -- 0xFFFF, присваиваемое каждому из клиентов Sentinel. Таким образом, Borland (aka Inprise) имеет свой VId; у Tanner Research тоже имеется по крайней мере один идентификатор. Инсталлятор SentinelLM SDK косвенно запрашивает VId и помещает его в некоторые исполнимые файлы SDK при установке. Значит нам необходимо будет узнать соответствующий VId нашей жертвы, как для установки SDK, так и для корректной генерации новых лицензий.

Откуда нам достать VId нашей жертвы? Можно спросить у синьора Донгла, если таковой имеется. Вторая ячейка слева -- искомый VId (серьёзно ;-). Умелые электронщики могут попробовать попытать Донгл под беспощадным программатором EEPROM'ов. Более простой подход -- чтение Донгла через порт. Я не советую вам делать ни то ни другое. Ещё спалите Донгл и тогда уж точно: кроме как брелок он вам больше ничем не послужит. Кстати, у обоих методов есть существенный недостаток -- Донгла-то может и не быть, если у вас копия совсем пиратская. Самый универсальный подход -- спросить у самой жертвы. Она уж точно знает, просто сказать стесняется. Да вот беда -- дистрибутив-то состоит из нескольких исполнимых файлов (экзешников и DLL-ок) -- поди узнай, кто из них за регинфу отвечает... Если ваша интуиция молчит, я вам подскажу. Читайте дальше.

Сентинельная защита, как такова, распространяется Reinbow Technologies ввиде оболочки API, т.е. комплекса драйверов, DLL, документации и прочих любимых программистами мелочей. Таким образом, пользовательская программа может вызвать некоторую API-функцию для проверки наличия донгла, запросить целостность лицензионной информации через другую API-функцию и т.д. и т.п. Для доступа ко всем этим функциям, т.е. для подключения к API, Reinbow предлагает два-три способа:

1. прямое использование lsapiw32.dll (распространяется вместе с дистрибутивом);
2. статическая линковка lsapiw32.lib (lsapiw32.dll НЕ фигурирует в дистрибутиве);
3. нездоровые комбинации из 1 и 2.

В любом случае, SLM должен будет вызвать специальную функцию для получения своего VId. Наша цель -- перехватить эту самую функцию и подсмотреть значение EAX.

Анализируем дистрибутив нашей жертвы на предмет наличия lsapiw32 или подобной DLL. Нашли? Если да, то грузите её в IDA. В противном случае имеет смысл предположить, что lsapiw32 статически прилинкована к главному исполнимому модулю программы. Как бы то ни было, наложение сигнатуры W32MCST1 (sentinel static lib) в IDE должно обнаружить БОЛЬШОЕ количество сентинельных функций. Если не обнаружит, попробуйте загрузить другой модуль и т.д. Терпение постепенно превращается в опыт.

Далее, жмите на Ctrl+L и удивляйтесь: зачем SLM столько вспомогательных функций? Попытаемся угадать функцию, возвращающую VId. Первый, но не единственный, кандидат -- _computeVendorCode. Далее приводится примерное содержание функции:

```
.DATA
  Error       db "SentinelLM",0
  SentinelLM  db "%s error: Illegal vendor identification.",0Ah,0

.CODE
_computeVendorCode PROC param:WORD
  LOCAL lVar:DWORD
  invoke _DencVendId,param + 18h,ADDR lVar
  test eax,eax
```

```
    jnz @@1
    invoke _VLMwriteFrontendMsgs,0,OFFSET Error,OFFSET SentinelLM
    mov eax,0FFFFFFFCh
    jmp @@2
  @@1:
    mov eax,lVar
  @@2:
    ret
  _computeVendorCode ENDP
```

Обратите внимание на то, что _computeVendorCode является враппером к _DencVendId, которая, в свою очередь, вызывает _VLM_deMorphId. Особого внимания заслуживают и строковые константы, так? ;-)

Пора перейти к софтайсу... Создаём MAP-файл (если хотите, можете убрать все символы кроме _computeVendorCode или _DencVendId). Ищем утилиту msym.exe в softice_path/util16 и перегоняем наш MAP в SYM ("MSYM file.MAP"). Полученный SYM транслируем в NMS с помощью nmsym.exe в softice_path/ ("NMSYM file.sym") и загружаем в Symbol Loader ("NMSYM /SYM:file.NMS"). Далее, если _computeVendorCode находится в DLL, загружаем её в Symbol Loader ("Open module...") и смело жмём на "Load". Потом заходим в SoftIce (Ctrl+D) и ставим бряк на _computeVendorCode (bpx _computeVendorCode). Запускаем наше подопытное приложение (*.EXE) обычным способом и терпеливо ждём появления дружелюбного отладчика. Дождались? F12 и записываем содержимое EAX на бумажку!!!

А что если _computeVendorCode находится в EXE? Тогда чуть сложнее... После загрузки NMS, грузим *.EXE через Symbol Loader (!) и жмём на "Load". При появлении окна отладчика клацаем по F8 для попадания в код приложения. Задаём команду TASK и находим имя нашего подопытного процесса. Передаём полученное имя в качестве параметра команде MAP32 (вроде "MAP32 myprog") и смотрим на поля "Obj Name" и "Address". Например:

```
:MAP32 myprog
Owner     Obj Name  Obj#  Address        Size      Type
MYPROG    .text     0001  01AF:00401000  000ECF70  CODE  RO
MYPROG    .data     0002  01B7:004EE000  00001BE4  IDATA RW
MYPROG    .rsrc     0003  01B7:00508000  00026800  IDATA RO
```

С полученными данными можете приступать к релокации символов NMS:

```
:SYMLOC 1 1AF 401000
:SYM
.text(01AF:00401000, 002772F7 bytes)
      01AF:006782F3 _computeVendorCode
```

'1' в SYMLOC обозначает порядковый номер сегмента кода в MAP-файле. "1AF 401000" соответствует адресу "01AF:00401000". В данном случае, имеется ввиду именно сегмент CODE (aka .text), так-как в коде находится _computeVendorCode (не среди данных или ресурсов). Команда SYM показывает местоположение нашей функции, тем самым подтверждая успешную релокацию символов. Далее, ставим бряк на _computeVendorCode и закрываем окно отладчика. При повторном появлении оного жмём на F12 и записываем VId.

Как и прежде, жду ваши комментарии. До следующей главы. Отдыхайте.

Примечание

Если вы не совсем поняли или совсем не поняли содержание последних абзацев, советую поискать и почитать туториалы по IDA и SoftIce. Последние лучше всего напечатать перед изучением.

SentinelLM!SDK_Keygen

Автор: Quantum

Дата: 2003-07-04

В предыдущей главе мы славно потрудились и получили VId нашей жертвы. Наша следующая цель -- подсунуть этот VId инсталлеру Sentinel SDK (вы уже скачали 80-тонный архивчик?) и получить генератор лицензионных цепочек ИМЕННО для нашей жертвы. Правда, тут возникает одно НО, которое и служит темой для данной главы. Дело в том, что установщик не запрашивает VId напрямую, а вместо него просит ввести некий серийник (Serial Number)...

Отладчик сразу проясняет ситуацию:

1. Инсталлятор разжимает библиотеку MODBIN.DLL
2. Введённый серийник передаётся функции MODBIN!GetVendorId (не смейтесь :-)
3. Функция возвращает VId соответствующий серийнику или -1, если таковой не найден.

Мммда... таким образом VId косвенно передаётся инсталлеру. Значит, если хотите играть по правилам, тогда соизвольте подобрать серийник для нашего VId. В противном случае можете попробовать обойти стадию с MODBIN и насильно запихнуть VId прямо в SDK. Я, честно говоря, такого делать не пробовал и вам не советую! Итак, приступим к написанию ключегенератора для SentinelLM SDK 7.1.

Внимание! Не спешите компилировать прилагаемые исходники, сначала дочитайте главу!

Во-первых, покопайтесь над установщиком и вытяните из него MODBIN.DLL. При желании можете дизассемблировать библиотеку и постараться понять принцип хеширования (серийник -> VId). Таким образом вы убедитесь в сложности построения обратной функции (VId -> серийник). Гораздо проще обойтись прямым перебором всех возможных комбинаций до получения искомого VId. На самом деле время перебора не так уж и велико (1,5 - 2 часа на среднестатистическом компе), но я предлагаю несколько иной подход, а именно: сгенерировать файл со всеми ВАЛИДНЫМИ парами (VId, серийник) для использования в ключегенераторе в качестве словаря. Размер такого файла, в зависимости от формата, может составить около 382Кб. Далее приводится исходник первой стадии генератора:

```
.386
.model flat, stdcall
option casemap:none

include    windows.inc
include    kernel32.inc
includelib kernel32.lib
include    user32.inc
includelib user32.lib

.DATA
modbin      db "modbin",0                   ; Имя библиотеки
GetVendorId db "GetVendorId",0              ; Импортируемая ф-ция
Err1        db "MODBIN.DLL не найден",0
Err2        db "GetVendorId не найден",0
Err3        db "Ошибка при создании файла",0
sLog        db "slm_gen",0                  ; Имя файла-словаря
sOK         db "Программа готова к старту!",0
sDone       db "Программа завершилась успешно!",0
sCaption    db "MODBIN кряк",0
seed        dd 0FFFFFFFFh                   ; текущий серийник

.DATA?
hLib        dd ?   ; Хендл 'MODBIN.DLL'
procAddr    dd ?   ; Адрес ф-ции 'MODBIN!GetVendorId'
hFile       dd ?   ; Хендл файла-словаря
```

```
VId         dw ?   ; Текущий VId
strBuf      db 128 dup (?)

.CODE
start:
  ; 1. Задать текущий каталог
  ; -------------------------
  invoke GetModuleFileName,0,OFFSET strBuf,128
  test eax,eax
  jz @F
  mov edi,OFFSET strBuf
  add edi,eax
  xchg eax,ecx
  mov al,'\'
  std
  repnz scasb
  jnz @F
  mov BYTE PTR [edi + 1],0
  cld      ; <- это важно в NT
  invoke SetCurrentDirectory,OFFSET strBuf
@@:
  ; 2. Загрузить MODBIN.DLL
  ; -----------------------
  invoke LoadLibraryEx,OFFSET modbin,0,0
  test eax,eax
  jnz @F
  invoke MessageBox,eax,OFFSET Err1,eax,MB_ICONSTOP
  jmp _end_1
@@:
  mov hLib,eax
  invoke GetProcAddress,eax,OFFSET GetVendorId
  test eax,eax
  jnz @F
  invoke MessageBox,eax,OFFSET Err2,eax,MB_ICONSTOP
  jmp _end_2
@@:
  mov procAddr,eax
  ; 3. Создать файл-словарь
  ; -----------------------
  invoke CreateFile,OFFSET sLog,GENERIC_WRITE,\
    FILE_SHARE_READ,0,CREATE_ALWAYS,FILE_ATTRIBUTE_NORMAL,0
  cmp eax,INVALID_HANDLE_VALUE
  jne @F
  invoke MessageBox,0,OFFSET Err3,0,MB_ICONSTOP
  jmp _end_2
@@:
  mov hFile,eax
  invoke MessageBox,0,OFFSET sOK,OFFSET sCaption,MB_ICONINFORMATION
  ; 4. Приступим!
  ; -------------
_mainloop:
  ; Переводим текущий серийник в строку
  ; Содрано из DWTOA + чуть-чуть подоптимизировано
  dec seed
  jz _end_3
  mov eax,seed
  mov edi,OFFSET strBuf
  mov ecx,429496730
  mov esi,edi
_wloop:
  mov ebx,eax
  mul ecx
  mov eax,edx
  lea edx,[edx*4+edx]
  add edx,edx
  sub ebx,edx
  add bl,'0'
  mov [edi],bl
  inc edi
  test eax,eax
  jnz _wloop
```

```
  mov BYTE PTR [edi],0
  .WHILE (esi < edi)
    dec edi
    mov al,[esi]
    mov ah,[edi]
    mov [edi],al
    mov [esi],ah
    inc esi
  .ENDW
  push OFFSET strBuf
  call procAddr       ; Вызываем MODBIN!GetVendorId
  pop ecx             ; Выравниваем стек
  test eax,eax
  js _mainloop        ; -1 значит левый серийник
  mov VId,ax
  push seed
  pop DWORD PTR strBuf
  ; Добавляем в файл пару (VId, серийник)
  invoke WriteFile,hFile,OFFSET VId,6,OFFSET modbin,0
  jmp _mainloop
_end_3:
  invoke CloseHandle,hFile
  ; Усё... справились!
  invoke MessageBox,0,OFFSET sDone,OFFSET sCaption,MB_ICONINFORMATION
_end_2:
  invoke FreeLibrary,hLib
_end_1:
  invoke ExitProcess,eax
END start
```

По-моему, разбирать предыдущий исходник не стоит во избежание личных обид :-) В общем, запускайте его и ожидайте... ожидайте... ожидайте сообщения об успешном завершении программы. Полученный 382-килобайтный словарь (slm_gen) послужит нам многие годы и внесённый в него труд даром не пропадёт (надеюсь). Следующий шаг -- написать парсер для нашего словаря:

```
.386
.model flat, stdcall
option casemap:none

include    windows.inc
include    kernel32.inc
includelib kernel32.lib
include    user32.inc
includelib user32.lib

DlgProc PROTO :DWORD,:DWORD,:DWORD,:DWORD

.DATA
file_not_found db "Словарь не найден!",0
dwFormat       db "%u",13,10,0

.DATA?
hInst  dd ?
hSN    dd ?   ; Хендл edit'а с серийником
hFile  dd ?   ; Хендл словаря
hMap   dd ?   ; Хендл словаря в памяти
hView  dd ?   ; Указатель на маппированные данные
fSize  dd ?   ; Размер словаря
szBuf  db 128 dup (?)

.CODE
start:
  invoke GetModuleHandle,0
  mov hInst,eax
  invoke DialogBoxParam,eax,200,0,OFFSET DlgProc,0
  invoke ExitProcess,eax

DlgProc PROC hDlg:DWORD,uMsg:DWORD,wParam:DWORD,lParam:DWORD
  mov eax,uMsg
```

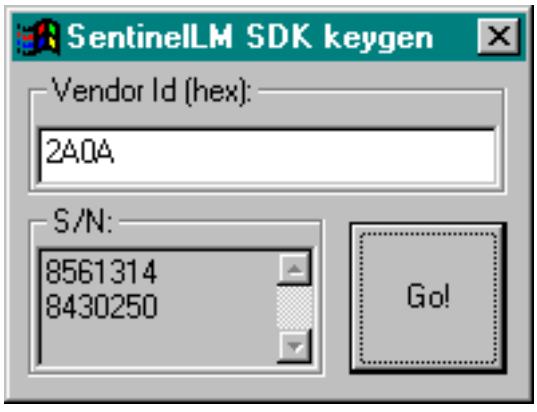
```
  cmp eax,WM_COMMAND
  jne no_wm_comm
  mov eax,wParam
  cmp ax,102
  je @F
  xor eax,eax
  ret
@@:
  ; 1. Получаем введённый VId
  ; -------------------------
  mov DWORD PTR szBuf,0
  invoke SetDlgItemText,hDlg,101,OFFSET szBuf
  invoke GetDlgItemText,hDlg,100,OFFSET szBuf,5
  test eax,eax
  jnz @F
  ret
@@:
  ; 2. Переводим VId из шестнадцатиричной строки в 16-битное слово
  ; ---------------------------------------------------------------
  mov edx,DWORD PTR szBuf
  mov ecx,4
  xor eax,eax
hex2bin:
  test dl,dl
  jz endloop
  shl eax,4
  sub dl,48
  cmp dl,9
  jle @F
  sub dl,7
  cmp dl,0Fh
  jle @F
  sub dl,32
@@:
  or al,dl
  shr edx,8
  loop hex2bin
endloop:
  ; 3. Ищем VId в словаре
  ; ---------------------
  mov ecx,fSize
  shr ecx,1
  jecxz endsrch
  push ebx      ; Сохранить EBX (для NT)
  push edi      ; Сохранить EDI (для 9x)
  mov edi,hView
  cld
search:
  scasw
  jne @F
  ; 4. Добавляем очередной серийник и продолжаем поиск
  ; ---------------------------------------------------
  call appendSN
@@:
  dec ecx
  dec ecx
  jle done      ; наш словарь кто-то обрезал :-)
  add edi,4
  loop search
done:
  pop edi
  pop ebx
endsrch:
  ret
no_wm_comm:
  cmp eax,WM_INITDIALOG
  jne no_wm_init
  invoke GetModuleFileName,hInst,OFFSET szBuf,127
  ; Убираем расширение
  add eax,OFFSET szBuf - 4
  mov BYTE PTR [eax],0
```

```
  invoke CreateFile,OFFSET szBuf,GENERIC_READ,\
    FILE_SHARE_READ,0,OPEN_EXISTING,FILE_ATTRIBUTE_NORMAL,0
  cmp eax,INVALID_HANDLE_VALUE
  jne @F
  invoke MessageBox,0,OFFSET file_not_found,0,MB_ICONSTOP
  invoke EndDialog,hDlg,0
  ret
@@:
  mov hFile,eax
  invoke GetFileSize,eax,0
  mov fSize,eax
  ; Отображаем словарь в память
  invoke CreateFileMapping,hFile,0,PAGE_READONLY,0,0,0
  mov hMap,eax
  invoke MapViewOfFile,eax,FILE_MAP_READ,0,0,0
  mov hView,eax
  invoke GetDlgItem,hDlg,101
  mov hSN,eax
  ret
no_wm_init:
  cmp eax,WM_CLOSE
  jne no_wm_close
  invoke UnmapViewOfFile,hView
  invoke CloseHandle,hMap
  invoke CloseHandle,hFile
  invoke EndDialog,hDlg,0
  ret
no_wm_close:
  xor eax,eax
  ret
DlgProc ENDP

; Показать найденный серийник
appendSN PROC uses edi ecx eax
  mov eax,[edi]
  invoke wsprintf,OFFSET szBuf,OFFSET dwFormat,eax
  invoke SendMessage,hSN,WM_GETTEXTLENGTH,0,0
  invoke SendMessage,hSN,EM_SETSEL,eax,eax
  invoke SendMessage,hSN,EM_REPLACESEL,0,OFFSET szBuf
  ret
appendSN ENDP

END start
```

Ну всё, компилируем вместе с RC-скриптом и наш новый генератор серийных номеров для SentinelLM SDK готов к употреблению!



Попробуйте задать ему 2A0A (это последний VId в нашем словаре) и получите целых ДВА результата (8561314 и 8430250). Почему два? Видимо SDK разделяет все VId на две категории:

1. [0000 - 7FFF] для обычных клиентов
2. [8000 - FFFF] для крутых разработчиков (вроде нас с вами)

Первый серийник соответсвует обычному VId (2A0A) а второй наделяет SDK дополнительными фичами. Так как большинству клиентским программам (например, нашей жертве) совершенно безразличен MSB (старший бит) её VId, вам будет всё равно какой из этих серийников использовать. Кстати, иногда может быть вплоть до восьми серийных номеров на один VId (больше восьми я ещё не встречал).

Далее, просто ставим SDK и проверяем, что VId соответствует нами заданному. Только не спрашивайте: "А как это проверить?" Медитируйте над первой главой. Обратите внимание на то, что SDK отказывается на нас работать... Не переживайте -- в следующей главе мы его уломаем :-)

SentinelLM!WIscGen_Crk_1

Автор: Quantum

Дата: 2003-07-26

Это первая часть последней главы из цикла о SentinelLM, но уже после её прочтения вы должны быть в состоянии "зарегистрировать" и/или отучить от заглушки в порту (aka донгл) некоторые простенькие программы, продавшие душу Rainbow Tech.

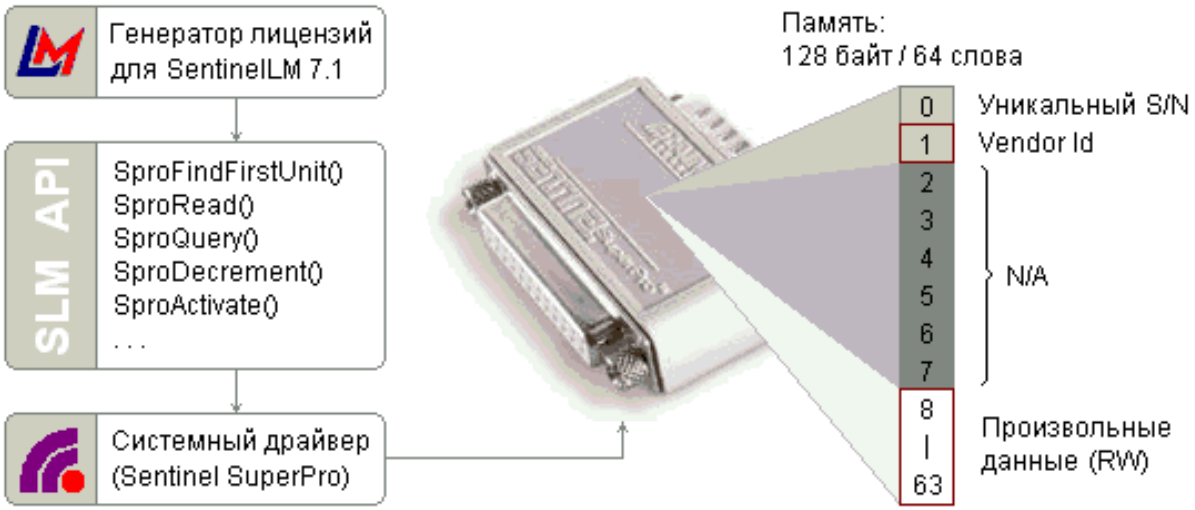
Итак, на данный момент имеем установленный SentinelLM SDK 7.1, эксклюзивно настроенный на наш таргет, т.е. в нём жёстко прописан VId нашей жертвы. Для того, чтобы изменить этот VId можно просто переустановить SDK или найти и пропатчить эти значения в бинарниках, что вовсе не тривиально. После нескольких неудачных попыток поменять VId, я решил всегда переустанавливать SDK и вам советую так поступать.

Пока я усердно работал над этой главой, вы уже прочитали объёмные PDF'ы из документации и узнали, что генератор лицензий SLM (он же WIscGen.exe) так просто нас не наградит халявными лицензиями для нашей жертвы. Оказывается, что он сам защищён SLM и требует наличия донгла! Смешно и обидно... Но не отчаивайтесь -- сегодняшний практикум многому вас научит и поможет устранить защиту SLM даже в патологически сложных случаях, в которых кроме ратификации лицензионного файла программа активно использует EEPROM-память донгла. У меня даже есть готовый патч для "исправления" WIscGen, но я настаиваю на необходимости хотя бы один раз поработать над этим вручную. Чем не ритуал посвящения в Sentinel? :-)

Кроме двух PDF'ов, входящих в комплект поставки SDK, вам нужно прочитать ещё один ОЧЕНЬ важный документ: "SentinelSuperPro 6.0 Developers Guide" (6,4 Мб), который можно скачать на сайте www.rainbow.com. Документ посвящён API Sentinel SuperPro, с помощью которой SLM "общается" с донглами семейства SuperPro. Особого внимания заслуживает глава 14: "API Function Reference".

Перед штурмом WIscGen, хочу добавить пару слов о принципе работы этой программы (особенно для тех лентяев, которые -- как и я в своё время -- не прочитали документацию ;-) Маркетинговая политика радужной компании основана вовсе не на продаже CD с SentinelLM SDK в красивых упаковках. Эти капиталисты взимают дань со своих клиентов за КАЖДУЮ выпущенную лицензию. Справедливости ради стоит заметить, что многие другие компании тоже так поступают... В общем, вместе с SDK компания выдаёт (вроде, первый раз бесплатно) своеобразный электронный счётчик, который автодекрементируется при каждом употреблении. При создании постоянных лицензий для одного PC, счётчик уменьшается на определённое количество единиц; при выпуске корпоративных лицензий для нескольких PC -- теряется соответственно большее количество единиц. Всё зависит от типа лицензии, срока годности и т.д. Эти единицы теряются безвозвратно, даже если пользователь по ошибке создаст бракованную лицензию. При полном истощении счётчика, приходится идти к Rainbow и покупать новый. Вам это не напоминает картриджи для принтеров HP? Так вот, счётчик этот реализован в виде нескольких ячеек памяти в ранее упомянутом донгле и WIscGen "видит" эти ячейки через API, которая общается с донглом через драйвер.

На следующем изображении упрощённо показана зависимость WIscGen от донгла. На большее у нас с Paint'ом не хватило воображения :-)



Обратите внимание на ячейку "Vendor Id" (или "Developer Id") -- это тот самый VId!!! Таким образом, каждый лицензионный донгл "привязан" к своему хозяину и отказывается работать на "чужих" владельцев. Так, по крайней мере, считают в Rainbow. Ячейка "Уникальный S/N" -- это вовсе не то, о чём вы сразу подумали. Нам данный S/N абсолютно безразличен (не путать с серийником для SDK). Область "N/A" не доступна приложениям и, в основном, представляет собой зарезервированные ячейки. В области "Произвольные данные" находится, например, вышеупомянутый счётчик.

SLM API в данном случае реализована статически, т.е. нужные функции внедрены в WIscGen.exe. Наличие lsapiw32.dll не должно сбивать вас с толку. Наша задача -- пропатчить генератор лицензий, вернее пропатчить SLM API для того, чтобы генератор "видел" несуществующий донгл и признавал его своим. Потом мы "заполним" счётчик и получим неистощимый запас лицензий на зависть всем клиентам Rainbow!

Единственный известный мне рабочий метод взлома WIscGen -- это метод описанный в статье CyberHeg "Removing need for dongle in Sentinel LM Wlscgen". Вообще, SentinelLM!WIscGen_CRK_1 можно считать частичным переводом статьи CyberHeg.

Итак, запускаем WIscGen.exe и убеждаемся в его неработоспособности без донгла. Я настоятельно рекомендую вам сделать паузу в чтении этой статьи и помедитировать над кодом программы как в IDA (не забудьте наложить сигнатуру статической SLM API), так и под софтайсом. Можете запомнить текст сообщения об ошибке и попытаться оттрассировать его под отладчиком. Попробуйте угадать какие функции из SLM API могут/должны вызываться для обнаружения донгла.

Сдаётесь? Так быстро? Ладно... в IDA > Names ищем те самые функции из "SentinelSuperPro 6.0 Developer's Guide". При помощи дизассемблера, отладчика и хекс-редактора мы постепенно заставим некоторые из этих функций работать с несуществующим донглом :-) По мере необходимости мы будем заглядывать в SoftIce (как обычно, MAP > SYM > NMS и т.д.)

RNBOsproFindFirstUnit(packet, VendorID)

Первым делом ставим BPX на RNBOsproFindFirstUnit(), так-как эта функция предназначена для поиска донгла с заданным VId. Для большей уверенности можете поставить бряк сразу на все 16 функций из этой API. Наше предположение оказывается верным и SoftIce выскакивает на входе в RNBOsproFindFirstUnit(). Трассируем до RETN и получаем 3 в аккумуляторе (EAX = 3), что означает отсутствие донгла и объясняет сообщение об ошибке. Не теряя времени, идём в IDA и правим это безобразие:

```
00435B00 _RNBOsproFindFirstUnit@8 proc near
00435B00             push   ebx
00435B01             push   esi
```

```
00435B02             mov    eax, [packet]
00435B06             or     eax, eax
00435B08             jnz    short loc_435B13
00435B0A             mov    ax, 2
00435B0E             pop    esi
00435B0F             pop    ebx
00435B10             retn   8
                     . . .
```

При успешном выполнении RNBOsproFindFirstUnit возвращает SP_SUCCESS, т.е. ноль. Блиц-патч:

```
00435B00 _RNBOsproFindFirstUnit@8 proc near
00435B00             push   ebx
00435B01             push   esi
00435B02             mov    eax, [packet]
00435B06 33C0        xor    eax, eax      ; Обнуляем eax (SP_SUCCESS)
00435B08 90          nop                  ; Забиваем переход
00435B09 90          nop
00435B0A 90          nop                  ; Забиваем mov ax, 2
00435B0B 90          nop
00435B0C 90          nop
00435B0D 90          nop
00435B0E             pop    esi
00435B0F             pop    ebx
00435B10             retn   8
                     . . .
```

Элементарно, не правда ли? Вместо исправления кода внутри функции RNBOsproFindFirstUnit можно поправить условные переходы после каждого её вызова, но гораздо проще и быстрее поступить вышеописанным образом. По ходу дела, не забывайте осуществлять изменения в самом файле WIscGen.exe при помощи HIEW, WinHex или подобной утилиты.

RNBOsproRead(packet, address, *data)

После успешного "обнаружения" донгла с помощью RNBOsproFindFirstUnit, WIscGen пытается прочитать 16-битные ячейки донгловой памяти через RNBOsproRead. Второй параметр функции задаёт относительное местоположение запрашиваемой ячейки. Полученное значение возвращается через указатель в третьем параметре. Так-как данная функция возвращает принципиально различные значения через указатель, нам её так просто не пропатчить. Опираясь на статьи Goatass и CrackZ, CyberHeg предложил внедрить в конец RNBOsproRead массив из 128 байт для эмуляции памяти донгла. Естественно, код самой функции нужно исправить и заменить вызов драйвера донгла на чтение локального, так сказать, массива. "Правильные" значения ячеек в данном массиве определяются путём долгих мучений с отладчиком. В большинстве случаев, WIscGen проверяет значение считанной ячейки вскоре после возврата из RNBOsproRead и вы можете его перехватить и добавить в массив. Правда, иногда проверка значений происходит иным, менее элементарным образом. Далее представлен исправленный вариант RNBOsproRead:

```
00435CD0 _RNBOsproRead@12 proc near
00435CD0             push   esi
00435CD1             push   edi
00435CD2             mov    eax, [packet]
00435CD6             or     eax, eax
00435CD8             jnz    @F
00435CDA             mov    ax, 2
00435CDE             pop    edi
00435CDF             pop    esi
00435CE0             retn   0Ch
00435CE3 @@:         push   eax
00435CE4             call   sub_42F940
00435CE9             mov    esi, eax
00435CEB             cmp    word ptr [esi], 7242h
; Здесь начинаются изменения:
```

```
00435CF0 90          nop
00435CF1 90          nop
00435CF2 55          push   ebp
00435CF3 E800000000  call   $+5
00435CF8 5D          pop    ebp             ; Получить EIP
; (1Eh = 00435D16 - 00435CF8)
00435CF9 8D551E      lea    edx, [ebp+1Eh]  ; Адрес начала
                                            ; нашего массива
00435CFC 5D          pop    ebp             ; Восстановить стек
00435CFD 0FB74C2410  movzx  ecx, [address]
; Каждая ячейка представляет собой 16-битное слово, т.е. 2 байта
; Поэтому ECX следует умножить на 2
00435D02 D1E1        shl    ecx, 1
00435D04 0FB7040A    movzx  eax, [edx+ecx]  ; Помещаем в EAX значение
                                            ; считанное из ячейки
00435D08 8B7C2414     mov    edi, [*data]
00435D0C 668907       mov    [edi], ax       ; Сохранить результат
00435D0F 33C0         xor    eax, eax        ; Обнуляем eax (SP_SUCCESS)
00435D11 5F          pop    edi
00435D12 5E          pop    esi
00435D13 C20C00      retn   0000C
; А вот и наш массив:
00435D16 0000        dw     00000h           ; №0
00435D18 0000        dw     00000h
00435D1A 0000        dw     00000h
00435D1C 0000        dw     00000h
00435D1E 0000        dw     00000h
00435D20 0000        dw     00000h
00435D22 0000        dw     00000h
00435D24 0000        dw     00000h
00435D26 FFFF        dw     0FFFFh           ; №8
00435D28 FFFF        dw     0FFFFh           ; №9
00435D2A 0000        dw     00000h
00435D2C FFFF        dw     0FFFFh           ; №11
00435D2E 0000        dw     00000h
00435D30 0000        dw     00000h
00435D32 0000        dw     00000h
00435D34 0007        dw     00700h           ; №15
00435D36 0000        dw     00000h
. . .                                       ; Забить нулями
00435D5C 0000        dw     00000h
00435D5E FFFF        dw     0FFFFh           ; №36
00435D60 0000        dw     00000h
. . .                                       ; Забить нулями
00435D94 0000        dw     00000h           ; №63
```

Обратите внимание на 4 ячейки с 0FFFFh и одну с 0700h. Эти значения были получены путём трассирования каждого вызова RNBOsproRead.

Если снова запустить WlscGen, программа должна заметно тормознуть (!) и показать окно с приглашением "Please Login". Эта задержка при запуске служит явным признаком того, что нам ещё рано праздновать. Действительно, пора заняться RNBOsproQuery, RNBOsproFindNextUnit и RNBOsproDecrement. Хотя, стоп!!! Я по себе чувствую, что вы подустали :-) Поэтому оставим эти три API для следующей главы. Я вам даже больше скажу: с RNBOsproFindNextUnit и RNBOsproDecrement вы должны быть в состоянии разобраться сами! Это будет своеобразным домашним заданием :-) Отдыхайте.

Об упаковщиках в последний раз: Часть первая - теоретическая

Автор: Volodya / HI-TECH, NEOx / UINC.RU

Дата: 2003-08-11

_Сложное – то, что делается мгновенно,
невозможное – то, что требует лишь
немногим большего времени._

Рецензент: Dr.Golova/UINC.RU
Главный корректор: Edmond/HI-TECH

1. Лирическое отступление

Решено было разбить эту статью на две-три части. Первая часть собственно теоретическая. Она дает абсолютно необходимый минимум теории, который позволит успешно распаковывать файлы. В статье также описывается формат XML, который затем переводится в CSS-HTML при помощи XSLT-процессора. Таким образом, наряду со знанием структуры PE-файла вы, по прочтении статьи, будете иметь самые базовые представления о XML/CSS/XSLT, Perl/Java и других страшных технологиях. Мы достаточно неплохо осознем, что тема уже достаточно замусолена, поэтому нудного пересказа PE-документации здесь не будет. Скорее, вам стоит воспринимать эту статью как некий коллектор, попытку обобщения и анализа всей доступной информации о пакерах, PE-файлах, антиотладке и т.п. А поскольку есть возможность перевода статей с испанского и французского языков (помимо стандартного английского), то должно получится что-то неплохое. Вторая часть будет чисто практической. Мы рассмотрим UPX, Aspack, Asprotect, Crunch, Armadillo и вскользь упомянем о других, менее популярных пакерах. Все опыты, приведенные в первой части проводились на calc.exe – стандартном калькуляторе из поставки Windows 2000. Вторая часть тоже будет проходить вместе с calc.exe. Реальные случаи будете рассматривать сами. Последнее, но немаловажное замечание – ВСЕ, что здесь обсуждается, валидно ТОЛЬКО для Windows 2k/XP/2003 (последняя – с натяжкой!). 9x НЕ рассматривается как анахронизм. Замечание важно, т.к. 9x и NT по-разному трактуют оперативную память, загружают библиотеки, работают с процессами, обрабатывают исключения и т.п.

Для того чтобы понимать, что здесь написано, введем пару допущений. Допущение номер раз: вы знаете ассемблер; №2: вы знаете, что такое исполняемый файл, т.е. файл загружается с диска в оперативную память с помощью механизмов ОС, при этом ОС определенным образом может при загрузке корректировать информацию внутри файла, если это необходимо; №3: вы слышали о языке программирования Perl; №4 – вы имеете очень приличное представление о базовых типах языка C – структурах, массивах и т.п., последнее – пятое – вы имеете хоть и очень смутное, но все же какое-то представление о том, что такое PE-формат исполняемого файла.

2. Основные концепции

Теперь, наконец, давайте рассмотрим формат **PE** -файла. Скажем сразу, что в наши намерения не входило давать здесь пересказ технической документации **MS**. Файл со спецификациями **PE/COFF** при-ложен, на wheaty.net есть статьи Питрека на эту тему. Полагаем, в русской части Интернета сравнительно просто найти саму книгу Питрека. Из русской литературы можно предложить Румянцева, где неплохо описаны поля, и даже есть исходники, хотя их качество нам не нравится, и статью о загрузчике **PE** -файлов на RSDN.ru – там все расписано очень неплохо, есть и исходники, но они на Delphi, а этот язык мы не знаем и не любим. Однако алгоритм, по справедливому замечанию автора, везде одинаков.

Да, чуть не забыли. Для хорошо знающих английский, есть абсолютно великолепная вещь, совершенно свободно распространяемая - http://www.iecc.com/linker/ - слейте ее целиком, и обязательно прочтите, просто изумительно! Книга будет продублирована и на WASM.RU, ибо вполне стоит того! Практическим примером к этой книге может служить техническая документация с сайта http://www.cs.virginia.edu/~lcc-win32/ - *Якоб Навия* очень неплохо расписывает многие тонкости по созданию компилятора и линкера. Еще дополнительно прилагается англоязычное описание **PE** -формата (версия 1.7) от **Lumnificer** , где подробно расписываются все поля, и даже объясняется, как создавать **PE** -файл ручками, т.е. расписывается работа примитивного линкера!

Так же, посоветуем изучить исходный код программы **Wine** – эмулятора **Windows** для **Linux**. **Wine** умеет загружать **pe** -файлы под **Linux** , причем код **pe_image.c** написан очень развернуто. А там еще мно-о-го примеров есть… Сайт - http://www.winehq.com.

Теперь очень быстренько пробежимся по списку литературы, описывающего работу **NTDLL.dll** – **Windows loader'a**.

Нашими маяками в этом бушующем море станут статьи Питрека.
А также, основанная на этом статья **Russell Osterlund** с сайта http://www.smidgeonsoft.com, название статьи – "*Windows 2000 Loader: What Happens Inside Windows 2000: Solving the Mysteries of the Loader* " – лежит на MSDN. Лучше пока еще ничего не написано. Примеры для статьи скачивайте только с **MS** , родной сайт лучше не трогать. Причем после слива можно смело удалить все **dsp/dsw/ncb/suo/sln** файлы, оставив только **vcproj**. Причина в том, что все файлы проектов повреждены, во всяком случае, **VS.NET 2003** их открыть не смогла. Другое дело **vcproj** , который, по сути, является простым **xml** -файлом. Подредактируйте их немного (неправильно прописаны пути **Output Directory** и т.п.), и держите в уме, что проект **Forwarder** требует **lib** -файл проекта **Forwarded** , поэтому второй должен собираться раньше.

Во всей остальной части статьи мы будем жестко придерживаться терминов, принятых **Microsoft**. Все определения структур взяты из **winnt.h**. Эта мера кажется очень разумной, т.к. на данный момент существует дикая неразбериха в терминах - существует целая куча слов, для которых нет четких определений, например, секция импорта, таблица импорта, **IAT** и т.п. Каждый писатель под этими словами понимает что-то свое, в результате чего имеем полный беспредел.

Теперь взгляните на рисунок ниже:

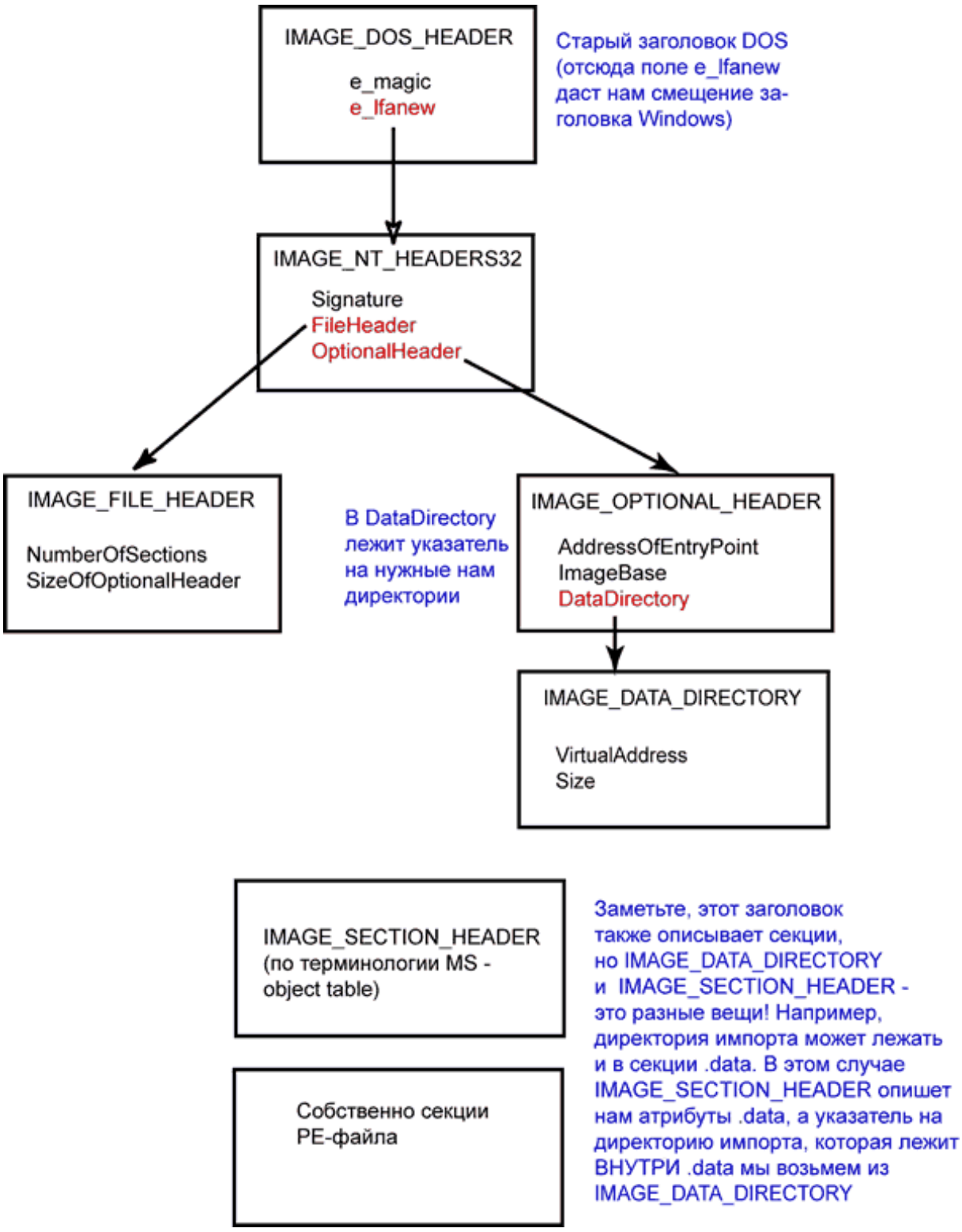


Рис. 1.

Мы предполагаем, что вы оторветесь от чтения этой статьи и проведете несколько минут с этим рисунком и официальной документацией от **MS** , держа в уме, что **PE** файл, по сути, просто совокупность секций, которые описываются в заголовке.
Единственное что, стоит объяснить несколько нюансов.

2.1 RVA/VA и иже с ними

RVA переводится как *Relative Virtual Address* - относительный виртуальный адрес. Его относительность заключается в том, что он отсчитывается от адреса загрузки, который может быть, а может и не быть, равен **ImageBase**.
Обязательно учтите, что **RVA** имеет РАЗНЫЙ смысл для бинарных файлов (exe, dll, sys) и объектных файлов (**obj** , **lib**). Вторые мы здесь не рассматриваем, а для первых – файл загружается в память и **RVA** какого-либо элемента вычисляется так:

```
RVA = VA – адрес загрузки,
```

где **VA** (*Virtual Address*) – виртуальный адрес элемента в памяти, а адрес загрузки берется из поля Op-tionalHeader.ImageBase, в том случае, если он равен ImageBase, либо вычисляется лоадером.
Обратите внимание на то, что элементарные просмотрщики файла, например, **QVIEW/WinHEX** и т.п. отображают абсолютные offset-ы, **HIEW** способен переключаться (Alt+F1) между показом **VA** и внутрифайловых смещений (local/global mode), а **IDA** вообще частично эмулирует работу **ntdll.dll** – Windows-loader'a, поэтому показывает только **VA** (если вы не укажете иное, например, загрузив файл как "binary").
Давайте рассмотрим пример для практического усвоения материала. Точка входа **calc.exe** (также часто называемая **EP** – entry point) выглядит так:

```
.01012420: 55 8B ;VA
```

А в виде абсолютных внутрифайловых смещений – вот так:

```
00011A20: 55 8B
;обратите внимание на отсутствие точки!
;HIEW требует точку, если хотите VA!
```

Давайте попытаемся разобраться, какой же механизм пересчета **VA** во внутрифайловые смещения.

Алгоритм примерно выглядит так:

1) По **VA** рассчитать **RVA**

```
VA = RVA + ImageBase
```

2) Для **RVA** найти секцию, в которой мы находимся (о секциях мы поговорим чуть ниже).

3) Пересчитать **RVA** во внутрифайловый offset по формуле:

```
offset = RVA - IMAGE_SECTION_HEADER.VirtualAddress +
   + IMAGE_SECTION_HEADER.PointerToRawData
```

Теперь давайте посчитаем ручками. В качестве примера возьмем адрес точки входа calc.exe - **0x01012420**.

Сразу смотрим в HIEW на значение **ImageBase** -

```
 ¦ Image base 01000000 ¦ Subsystem GUI ¦

   VA = RVA + ImageBase;
  01012420 = RVA + 1000000;
  RVA = 12420;
  //что можно сразу увидеть из OptionalHeader.EntryPoint
```

Теперь определяемся, где же у нас лежит этот **RVA**. Ага, **RVA** лежит в секции **.text**. Обратите вни-мание на то, что лежит он ближе к концу секции:

Number — Name — VirtSize — RVA — PhysSize — Offset — Flag

1 — .text — 000124EE — 00001000 — 00012600 — 00000600 — 60000020

Теперь **VA** во внутрифайловое смещение:

```
offset = 12420 - 1000 + 600 = 11A20;
```

Заметим, что процесс этот уже давным-давно автоматизирован. Есть такие утилиты как **LordPE** и **PE Tools** , а в них есть такая вещь как **FLC** (*File Location Calculator*), который по данному **VA** отобразит **RVA** и внутрифайловый **offset**.
Имея на руках некоторые **API** -функции - **GetModuleHandle** , **ImageRvaToVa** и **ImageRvaToSection** , несложно написать программу, которая быстренько переведет любой **VA** во внутрифайловый оффсет. Например, аналог функции от **MS ImageRvaToSection** может выглядеть где-то так:

```
 /***********************
 * Return value: Возвращается структура IMAGE_SECTION_HEADER, для данного RVA
 * Parameters: Первый – это структура IMAGE_FILE_HEADER
 * Parameters: Второй – это RVA для которого нужно найти секцию
 ***********************/

PIMAGE_SECTION_HEADER RvaToSection(PIMAGE_FILE_HEADER pFH, DWORD dwRVA)
{
  UINT                  i;
  IMAGE_SECTION_HEADER  *pSH;

  pSH = (PIMAGE_SECTION_HEADER)((DWORD)(pFH + 1)
       + pFH->SizeOfOptionalHeader);
  for (i = 0; i < pFH->NumberOfSections; i++)
  {
   if (dwRVA >= pSH->VirtualAddress
    && dwRVA < pSH->VirtualAddress
      + pSH->Misc.VirtualSize)
    return pSH;
   ++pSH;
  }
  return NULL;
}
```

Примеры функций **RtlImageRvaToVa** /**RtlImageRvaToSection** /**RtlImageDirectoryEntryToData** есть в файле **loader.c** из **wine**.

Например, есть у вас есть **dll** , загруженная по адресу, отличному от **ImageBase**. Вы нашли защиту в **Soft-Ice** , и хотите пропатчить это место HIEW'вом - тут-то и пригодится умение.

2.2 О секциях PE-файла

PE-файлы были спроектированы для работы в ОС со страничной адресацией памяти. Известно, что Windows делит память на страницы различного размера, поэтому и PE-файл разбит на секции. Это очень важно понимать, поскольку секции PE-файла в памяти и на диске - это вовсе не одно и то же! В том случае, если сами данные в секции имеют общий объем менее размера кластера - секция на диске выравнивается (т.е., дополняется нулями, **int3** - **CCh** или nop – **90h**) под размер кластера (не путать с сек-тором – кластеры состоят из многих секторов диска!), который обычно равен **512** байтам (или другое значение – поле **OptionalHeader.FileAlignment**).

Когда файл загружается, то секция попадает в страницу памяти большего размера – 4 кб (или дру-гое значение - **OptionalHeader.SectionAlignment**), и выравнивается вновь, если это необходимо, уже только нулями. Обратите вниманиме, мы говорим "секция" – т.е., секция **.text** , **CODE** , **.data** , **DATA** , **.aspack** или что-либо еще. Каждая секция будет проецироваться на одну или несколько страниц памяти.

Не путайте понятие "*Секция* " (**section**) и "*Директория* " (directory). Директории (импорта, экспорта, ресурсов, исключений или чего-то еще) находятся **ВНУТРИ** секций. Заметьте, секция определяет разбиение **PE** -файла на части, но секции глубоко все равно, какой тип данных будет находится

внутри нее. Для того, чтобы охарактеризовать тип, у нас есть директории! Директории характеризуют данные, содержащиеся в **PE** , по их типу и функциональности. Таким образом, можно говорить, что секция является физическим разбиением **PE** файла на состовляющие, а директория - логическим.

Иногда размер секции может занимать много кластеров, и отображаться Windows на несколько страниц памяти. Общее количество секций должно в точности соответствовать полю **FileHeader.NumberOfSections**. Теоретически, с этим шутить не стоит, так как шаг в сторону – стреляю. Такой файл не будет опознан **ntdll.dll** , поэтому на поле **NumberOfSections** можно смело(?) опираться. Однако и здесь нам уже радость испортили! Некоторые крипторы, например, **telock** , заменяют это значение в **ПАМЯТИ** , например, на **0xFFA4**. Угадайте, что случится, когда вы попытаетесь запустить дамп с такого файла? А что будет, если утилита, просматривающая содержимое полей секций в цикле, будет ориентироваться на такую переменную как на счетчик цикла, нечто вроде такого:

```
for (i=0; i<= FileHeader.NumberOfSections; i++)
//где FileHeader.NumberOfSections == 0xFFA4
{
//Здесь обрабатываем содержимое секции
//А теперь представим, что секции закончились. О-о-о-о-й!
//да, попробуйте такой файл посмотреть HIEW. Забавная реакция!
}
```

Как вариант, можно использовать значение **NumberOfSections** с диска. Другую альтернативу мы рассмотрим, после того как ознакомимся со структурой секции изнутри.

LordPE , **HIEW** на этот трюк покупаются! Все версии **IDA** - тоже! **PE Tools** – нет! Прекрасная защита от сброса дампа на диск (имеется в виду, дамп сбросить можно, а вот заставить его заработать потом.... тоже можно, но чуть посложнее)! Хотя, с другой стороны, такой файл не будет загружен самой Windows. Цитирую Евгения Сусликова: "*не должно быть хиеву умнее виндов* ". Что ж, вполне справедливо.
Для каждой секции в заголовке PE-файла указан ее размер на диске и в памяти. Структура выглядит так:

```
#define IMAGE_SIZEOF_SHORT_NAME 8
typedef struct _IMAGE_SECTION_HEADER {
//имя не более 8 символов
    BYTE    Name[IMAGE_SIZEOF_SHORT_NAME];
    union
 {
            DWORD   PhysicalAddress;
            DWORD   VirtualSize;
//помеченные красным значения характеризуют секцию в памяти
//VirtualSize – общий размер секции в памяти.
//если это число БОЛЬШЕ размера секции на диске
// – секция дополняется нулями
 } Misc;
    DWORD   VirtualAddress;
//RVA первого байта секции в памяти
//синий шрифт – данные о секции на диске
    DWORD   SizeOfRawData;
//Размер ИНИЦИАЛИЗИРОВАННЫХ данных на диске
//Ой, какое интересное поле! Поговорим о нем чуть ниже.
//Вкупе с VirtualSize им часто пользуются протекторы
    DWORD   PointerToRawData;
//очень полезное поле – offset от начала файла до первого байта секции
    DWORD   Characteristics;
//характеристики секции – это поле тоже очень любят менять,
// поговорим ниже и о нем
} IMAGE_SECTION_HEADER, *PIMAGE_SECTION_HEADER;
```

Особый интерес в этой структуре вызывают два поля – **VirtualAddress** и **SizeOfRawData**. Все остальные тоже очень интересны, но эти два широко используются протекторами для сокрытия от

наших глаз того, что должно быть скрыто. К примеру, вам, вероятно, часто доводилось видеть нечто вроде такого в **IDA** :

```
.data1:005B6FFD                 db    0 ;
.data1:005B6FFE                 db    0 ;
.data1:005B6FFF                 db    0 ;
.data1:005B7000                 db    ? ;
.data1:005B7001                 db    ? ;
.data1:005B7002                 db    ? ;
.data1:005B7003                 db    ? ;
```

в то время как самый обычный hex-редактор выдавал вполне осмысленные цифры. Проблема и ее решение достаточно просты. Обратите пристальное внимание на поля **VirtualSize** и **SizeOfRawData** той секции, где есть такой подарочек. Значение **SizeOfRawData** должно быть меньше значения поля **VirtualSize** :

Number — Name — VirtSize — RVA — PhysSize — Offset — Flag

1 — .text — 00195C5C — 00001000 — 00000000 — 00000000 — 60000020

2 — .data — 00008DD4 — 00197000 — 00000000 — 00000000 — C0000040

3 — .text1 — 00010000 — 001A0000 — 0000F000 — 00001000 — 60000020

4 — .data1 — 00020000 — 001B0000 — 00007000 — 00010000 — C0000040

5 — .pdata — 000B0000 — 001D0000 — 000B0000 — 00017000 — C0000040

6 — .rsrc — 00001000 — 00280000 — 00001000 — 000C7000 — 40000040

Именно поэтому и появляются знаки вопроса – **IDA** ориентируется только на raw-байты (хотя, в случае секции кода с этим можно поспорить). Чтобы исправить положение, надо просто приравнять значение **SizeOfRawData** значению **VirtualSize**. И, вуаля:

```
.data1:005B7000 aPdata000_0     db 'PDATA000'
.data1:005B7008                 db    2 ;
.data1:005B7009                 db    1 ;
```

Заметим также, что ни **VirtualSize** , ни **SizeOfRawData** различных секций перекрываться не могут. Таким образом подправленный файл просто не будет загружен.
Возможен и еще, например, такого рода трюк. Положим, есть файл, у которого ручками создана несуществующая директория отладки (*Debug Directory*):

```
#define IMAGE_DIRECTORY_ENTRY_DEBUG 6
```

Структура директории выглядит так:

```
typedef struct _IMAGE_DEBUG_DIRECTORY {
    DWORD   Characteristics;
    DWORD   TimeDateStamp;
    WORD    MajorVersion;
    WORD    MinorVersion;
    DWORD   Type;
    DWORD   SizeOfData;
    DWORD   AddressOfRawData;
    DWORD   PointerToRawData;
} IMAGE_DEBUG_DIRECTORY, *PIMAGE_DEBUG_DIRECTORY;
```

Положим, мы задали заведомо огромные значения полей **SizeOfData** , **AddressOfRawData** и **PointerToRawData**. Лоадера эти поля абсолютно не интересуют (ему важна лишь валидность параметров в **IMAGE_DATA_DIRECTORY**). Прямо сказано, что эти поля касаются только отладчика (или дизассемблера). Так вот, **IDA** на таком файле (до версии 4.3 включительно) слетит наглухо! Такого рода трюк (© **Dr. Golova**) был применен в программе, которую рассматривал **Sten** в своей статье – "*Исследование подаруночка* " на reversing.net, если кто помнит.

Популярны также трюки с директориями импорта-экспорта. Некоторые поля (**OriginalFirstThunk** , например) забивают **0xFFFFFFFF** или чем-нибудь не менее гадким - **RVA OriginalFirstThunk** должно, согласно действиям лоадера (*Win2k SP4*), укладываться в диапазон

```
cmp     ecx, [eax+IMAGE_NT_HEADERS.OptionalHeader.SizeOfHeaders]
jb      loc_77F9373F ;если меньше SizeOfHeaders
cmp     ecx, [eax+IMAGE_NT_HEADERS.OptionalHeader.SizeOfImage]
jnb     loc_77F9373F ;и больше либо равно SizeOfImage
```

если это не так – файл загружен не будет. В этих случаях **IDA** говорит "*Can't find a translation for virtual address…* ", и … раньше слетала, а теперь продолжает работу. **HIEW** в своей работе на **winnt.h** не опирается вообще, он был создан раньше. Валидность адресов директории импорта/экспорта проверяется валидностью **VA**. Мы поговорим о подобных приемах немного позднее.

Флаги секции – тоже вещь небезынтересная! Здесь мы не будем углубляться в детали. Скажем лишь, что флаги секции могут преобразовываться лоадером в атрибуты страниц и сегментов, биты CR-регистров и т.д. Это достаточно сложная тема, требующая неплохого понимания принципов работы защищенного режима. Мы бы порекомендовали Рендалла Хайда "*Art of Assembly* " и Михаила Гука. Заодно можно почитать Фроловых, а также ознакомится со статьями **Broken Sword** на WASM.RU. После этого можно смело утверждать, что вы будете является одним из очень твердых специалистов по защищенному режиму процессоров x86 :). Такое знание не будет бесполезным. Например, понимание принципов защиты страниц виртуальной памяти помогает практически однозначно идентифицировать **Aspack**. Сбросьте флаг разрешения записи у секции кода в программе, предположительно запакованной **Aspack** , и взгляните на результаты. Почему такое происходит – мы поясним во второй части. Обязательно обратите внимание - это поле **НИКОГДА** не должно быть равным нулю. Как это можно использовать - см. ниже.

Сейчас уместным было бы посоветовать на какое-то время отвлечься от статьи, и попробовать поэкспериментировать с секциями файла. Да, мы сделаем это немного позднее, во второй и третьей частях, когда будем чистить файл от мусора, оставленного крипторами, однако, то, что сделано собственными руками, едва ли когда-нибудь забудется. Например, попробуйте поставить поле **VirtualSize** какой-нибудь секции в ноль (как, например, это делает линкер от **Watcom**). Также учтите, что количество страниц в памяти не обязательно будет соответствовать количеству секций в **PE** -файле. Во-первых, размер, во-вторых – сама Windows просто не загрузит страницу памяти до тех пор, пока она не нужна. Только если страница нужна, тогда она будет подгружена.

Еще нюанс. Все упаковщики, пакеры и крипторы очень нежно относятся к секции **.rsrc**. Секция кода, директрия импорта – все это безжалостно калечится, однако секцию ресурсов трогать боятся. При-чина проста. В исходном коде **UPX** можно найти следующие комментарии:

```
// after some windoze debugging I found that the name of the sections
// DOES matter :( .rsrc is used by oleaut32.dll (TYPELIBS)
// and because of this lame dll, the resource stuff must be the
// first in the 3rd section - the author of this dll seems to be
// too idiot to use the data directories... M$ suxx 4 ever!
// ... even worse: exploder.exe in NiceTry also depends on this to
// locate version info
```

И действительно. В файле **oleaut32.dll** можно найти следующие строки, сохранившиеся там со времен 95-го по 2k и выше:

```
.77A078C7: 7825              js         .077A078EE  -----v (2)
.77A078C9: 8D4580            lea         eax,[ebp][-80]
.77A078CC: 687CB7A377      push        077A3B77C  -----v (3)
.77A078D1: 50                push        eax
.77A078D2: FF15F8229B77      call        lstrcmpiA ;KERNEL32.dll

.77A3B770:  44 00 49 00-52 00 00 00-2A 00 00 00-2E 72 73 72  D I R   *   .rsr
.77A3B780:  63 00 00 00-00 00 00 00-74 79 70 65-6C 69 62 00  c       typelib
```

И из-за откровенно неумного, недальновидного, …, не будем перечислять, поведения программиста, кракеры получили большой подарок, а авторы пакеров – подарок поменьше. Спасибо, Microsoft! И абсолютно без всякой иронии! Так что, если автор хочет, чтобы его файл имел красивую иконку, то он должен предоставить ресурсы файла в наглядном виде. Конечно, не все так просто, секция ресурсов то-же безжалостно калечится, однако, вероятность того, что название секции будет сохранено наряду с иконкой, очень велика. Что до остального…. Разберемся с этим чуточку попозже.

Сказанное выше справедливо для всех приложений, имеющих ресурсы (очевидно, часто, но не всегда, консольные приложения к таковым не относятся).

В заключение, рассмотрим алгоритм подсчета количества секций PE-файла. Как уже упоминалось, поле **FileHeader.NumberOfSections** не может служить достаточно надежным проводником. Тогда что? Пока мы используем этот алгоритм (к слову, тоже ненадежный), но, возможно, после выхода статьи, он уже не будет работать в силу вполне определенных причин. Тогда нам с вами останется последний верный и надежный способ – считать значение из файла диске. Многие утилиты это позволяют. Итак, производим подсчет секций, и смотрим на поля **PointerToRelocations** , **PointerToLinenumbers** , **NumberOfRelocations** , **NumberOfLinenumbers**. Если все эти поля равны нулю, а поле **Characteristics - HET** , значит - это секция.

Алгоритм прост:

```
/**********************
 * Return value: настоящее значение NumberOfSections
 * Parameters: pMem - возвращаемое значение
 * от CreateFileMapping, либо от GetModuleHandle
 **********************/

WORD GetRealNumberOfSections(PVOID pMem)
{
 PIMAGE_DOS_HEADER       pDosh;
    PIMAGE_NT_HEADERS       pNTh;
PIMAGE_SECTION_HEADER   pSh;

    WORD  iRealNumOfSect = 0;

 // Устанавливаем SEH
      __try
{
 // Считывает IMAGE_DOS_HEADER
  pDosh = (PIMAGE_DOS_HEADER)pMem;
  // Считываем IMAGE_NT_HEADERS
  pNTh = (PIMAGE_NT_HEADERS)((DWORD)pDosh + pDosh->e_lfanew);
  // Получаем указатель на первую секцию
  pSh = IMAGE_FIRST_SECTION32(pNTh);

  // Считываем секции по очереди
  for(word i = 0; i < (pNTh->FileHeader.NumberOfSections); i++)
  {
   if(!pSh->PointerToRelocations &&
    !pSh->PointerToLinenumbers &&
    !pSh->NumberOfRelocations &&
    !pSh->NumberOfLinenumbers &&
    pSh->Characteristics)
    // Увеличиваем счётчик секций
    iRealNumOfSect++;
   else
    return iRealNumOfSect;
   ++pSh; // Переходим к следующей секции
  }
  return iRealNumOfSect;
 }
 __except(EXCEPTION_EXECUTE_HANDLER)
 {
  // Ошибка “Access Violation!”
  return 0;
```

```
  }
 }
```

Уже упоминалось, что линкер **Watcom** устанавливает значение поля **VirtualSize** в нуль. Тогда возникает вопрос - откуда брать **VirualSize**? Рекомендуется использовать значение поля **SizeOfRawData** , выровненное по **SectionAlignment**. Ниже указан макрос на **C++** для выравнивания секции по **SectionAlignment** :

```
#define RALIGN(dwToAlign, dwAlignOn) ((dwToAlign % dwAlignOn == 0) ?
 dwToAlign : dwToAlign - (dwToAlign % dwAlignOn) + dwAlignOn)
```

Используется так: **VirtualSize** = **RALIGN**(**VirtualSize** , **SectionAlignment**);

Еще немного об **ImageBase**. Понятия « *директория кода* » не существует. Есть только *секция кода* , которая может называться, как душе угодно. Начало секции вычисляется из **IMAGE_OPTIONAL_HEADER.BaseOfCode**. В случае, если значение поля **AddressOfEntryPoint** не укладывается в диапазон,

```
BaseOfCode<=AddressOfEntryPoint<=SizeOfCode
```

то это внимательному человеку может говорить о многом. Во-первых, файл, вероятно, запакован (почему так – см. часть вторую). Во-вторых, теперь вам придется сушить мозги с его распаковкой. Некоторые утилиты, например, **OllyDbg** , это дело подмечают, о чем вежливо предупреждают. Мало ли что?

Теперь перейдем к директории импорта.

2.3 Директория импорта

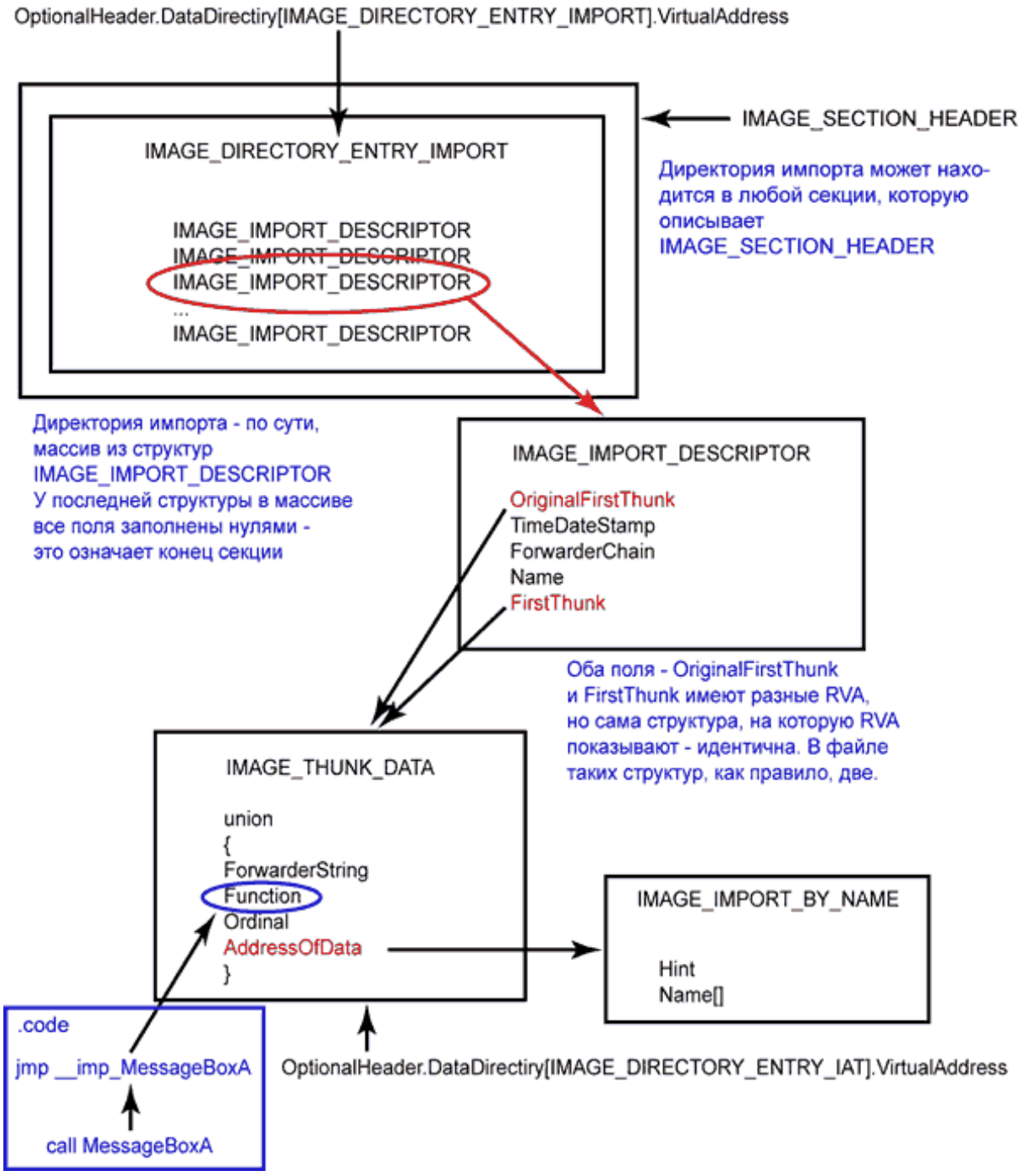


Рис. 2.

Для закрепления увиденного материала, опять таки, рекомендуется немного оторваться от чтения, и что-то поклацать. Все пояснения полей есть и в англо- и в русскоязычной документации.

Однако перед тем как перейти к прикладным аспектам, давайте немного разберемся с путаницей в терминологии. Прежде всего, массив из структур **IMAGE_IMPORT_DESCRIPTOR** иначе как директорией импорта называть некорректно. Структура эта содержит ряд полей, среди которых есть **OriginalFirstThunk** и **FirstThunk**. Очень важно понимать состояние структур, на которые эти поля показывают. Структуры называются **IMAGE_THUNK_DATA** – их две!

```
OriginalFirstThunk ? IMAGE_THUNK_DATA №1
```

```
/*обратите внимение, этой структуры может не быть,
 точнее, адреса будут заполнены нулями – это можно считать багом
 борландовского линкера.
 MS утилита bind.exe отказывается обрабатывать такой файл,
 почему так – см. ниже*/

FirstThunk ? IMAGE_THUNK_DATA №2
```

До загрузки образа в память **OriginalFirstThunk** и **FirstThunk** содержат **RVA** (которые, естественно, тоже различаются) на эти структуры, являющиеся ни чем иным, как таблицами адресов импорта (**IAT** – *import address table*). Но и здесь уже тоже наизобретали велосипедов! **IMAGE_THUNK_DATA** на которую указывает **OriginalFirstThunk ntdll.dll** при загрузке в нормальных условиях не обрабатывается (хотя внутренняя функция **ntdll.dll LdrpSnapIAT** может читать и оттуда, см. ниже), а адреса в структуре, на которую указывает **FirstThunk** действительно меняются (патчатся лоадером), поэтому **OriginalFirstThunk** называют еще **import lookup table** (*import name table*), а **FirstThunk** – вот это уже настоящая **IAT** , для которой есть свой **#define** :

```
#define IMAGE_DIRECTORY_ENTRY_IAT           12
```

Была когда-то и **СЕКЦИЯ** импорта, звали **.idata** , но сейчас она, пожалуй, издохла. Во всяком случае, сейчас этот зверь – редкий. Разве что линкер специально попросите...

Забавно, что даже **LordPE** , написанный действительно талантливым программистом, и тот не избежал этого бага. Поглядите потом, что вытворяет! Кнопочка – **Directories** , а директорию импорта называет **IAT** , а **offset** вообще обалденный выдает! **PETools** зато отображает корректно и термины корректные.

Теперь рассмотрим саму структуру и манипуляции с полями. Итак:

```
typedef struct _IMAGE_IMPORT_DESCRIPTOR {
    union {
// 0 for terminating null import descriptor
        DWORD   Characteristics;
// RVA to original unbound IAT (PIMAGE_THUNK_DATA)
        DWORD   OriginalFirstThunk;
    };
    DWORD   TimeDateStamp; // 0 if not bound,
                           // -1 if bound, and real date\time stamp
                           // in IMAGE_DIRECTORY_ENTRY_BOUND_IMPORT (new BIND)
                           // O.W. date/time stamp of DLL bound to (Old BIND)
    DWORD   ForwarderChain;// -1 if no forwarders,
                           //положительное число в противном случае
// RVA на имя dll
    DWORD   Name;
// RVA to IAT (if bound this IAT has actual addresses)
    DWORD   FirstThunk;
} IMAGE_IMPORT_DESCRIPTOR;
```

Перед тем, как начинать разбирать структуру, очень важно четко представлять, что в последнее время развелось много всяких наворотов на бедную **IAT**.

NTDLL.dll (функция **LdrpWalkImportDescriptor**) может читать **IAT** из:

```
1) OptionalHeader.DataDirectory[IMAGE_DIRECTORY_ENTRY_IMPORT] -
из IMAGE_IMPORT_DESCRIPTOR.OriginalFirstThunk,
если удовлетворяются определенные условия.

2) OptionalHeader.DataDirectory[IMAGE_DIRECTORY_ENTRY_IAT] –
т.е., из IMAGE_IMPORT_DESCRIPTOR.FirstThunk - это стандарт

3) OptionalHeader.DataDirectory[IMAGE_DIRECTORY_ENTRY_BOUND_IMPORT]
 - все файлы Windows имеют такую привязку
```

Есть и 4-й **#define** :

```
#define IMAGE_DIRECTORY_ENTRY_DELAY_IMPORT   13
// Несмотря на #define в winnt.h определений полей структуры вы там не найдете.
```

```
// Структура определена в delayimp.h в директории VC.
// Малоизвестен тот факт, что линкер
// от MS умеет также ассемблировать код!
// Так вот, именно это здесь и используется.
// В исполняемый файл просто-напросто запихиваются
// дополнительные клочки кода.
```

NTDLL.dll это не патчит, т.к. здесь уже используется совсем другой механизм. Фактически, это аналог **LoadLibrary** /**GetProcAddress** в одной упаковке. Более подробно - как всегда, Мэтт Питрек.

Здесь тоже понаворочено более, чем достаточно. Внутри **Delay Load Directory Table** есть стандартная таблица импорта, может выполняться привязка, т.е. возникать **Bound Delay Import Table** , есть **Unload Delay Import Table** и т.п. Мало не покажется!

LordPE понимает, что такое директория отложенного импорта. **PE Tools** пока выдает только **VirtualAddress** и **Size** (т.е. стандартные описания директории), **HIEW** (включая 6.85) вообще не умеет работать с директориями, **IDA** , хм.., **IDA** знает все.

Теперь у лоадера есть выбор – из какого же поля, собственно читать? Сначала обрабатывается цепочка директорий. Функция **LdrpWalkImportDescriptor** обрабатывает **ТОЛЬКО** директорию привязанного импорта, если **VirtualAddress** директории **IMAGE_DIRECTORY_ENTRY_BOUND_IMPORT** не равен нулю, и временные пометки (**TimeDateStamp**), зашитые внутри модуля при его создании, соответствуют таковым в библиотеках, к которым этот модуль привязан (системные библиотеки **Windows** , **dll** приложения и т.п.).

В случае отсутствия **IMAGE_DIRECTORY_ENTRY_BOUND_IMPORT** (или невыполнения некоторых условий, например, загрузки экспортируемого модуля не по адресу **ImageBase**) читается уже стандартный **IMAGE_IMPORT_DESCRIPTOR**.
Так же уже не вполне ясна роль поля **IMAGE_IMPORT_DESCRIPTOR.TimeDateStamp** , которое может иметь три значения:

1. **0** – стандартный **IMAGE_DIRECTORY_ENTRY_IMPORT**.
2. **xxxxxxxxh** – из **IMAGE_IMPORT_DESCRIPTOR**. **FirstThunk** , но с некоторыми оговорками, см. ниже.
3. **-1** – лоадер читает из **IMAGE_DIRECTORY_ENTRY_BOUND_IMPORT**. Как уже говорилось, в этом случае лоадеру совершенно безразлично (при соблюдении определенных условий), есть ли **IMAGE_DIRECTORY_ENTRY_IMPORT** вообще или нет! Если попытка чтения **BOUND_IMPORT** проваливается в силу, например, несовпадения **TimeDateStamp** у **exe** файла и **dll** , к которым он привязан, то тогда читается стандартная **IMAGE_DIRECTORY_ENTRY_IMPORT** , но как при этом обрабатывается **–1** – неясно. Во всяком случае, в коде **LdrpWalkImportDescriptor НЕТ** сравнения **TimeDateStamp** с -1, поэтому можно предположить, что достаточно существования **IMAGE_DIRECTORY_ENTRY_BOUND_IMPORT**.

Теперь продолжим исследование. Начнем с поля **Name**. Это **RVA** на **ASCIIZ** -строку нужной библиотеки. Совершенно неоправданно на нее опираются во многих алгоритмах, где массив из структур **IMAGE_IMPORT_DESCRIPTOR** обрабатывается в цикле. С одной стороны да, если имя библиотеки не указано, то откуда изволите функцию импортировать, а с другой стороны – есть поле **TimeDateStamp** , которое может быть равно **–1** , точнее, есть директория **BIAT** (есть и такое название у директории привязанного импорта), и, если она присутствует, о старой директории импорта можно забыть! И можно вытворять, что душе угодно с массивом из **IMAGE_IMPORT_DESCRIPTOR** , лоадера это нисколько не волнует! Т.е. утилита, обрабатывающая такой файл, это дело должна учитывать, т.е. частично эмулировать работу лоадера. **PE Tools** это умеет. **LordPE** - тоже.

При **TimeDateStamp == 0** мы имеем дело со старой доброй стандартной **IAT**. До загрузки **OriginalFirstThunk** и **FirstThunk** имеют одинаковую структуру (**IMAGE_THUNK_DATA**), которая

физически продублирована в разных местах **PE** -файла. После загрузки **FirstThunk** содержит реальные адреса функций, **OriginalFirstThunk** остается такой же. Лоадер патчит адреса в **FirstThunk** , меняя атрибуты страницы на время выполнения этой операции.

При **TimeDateStamp** равном **xxxxxxxxh** имеем интересный случай со «старой привязкой». Важно понимать, что в этом случае лоадер не делает ничего! Пара проверок на валидность и образ можно запускать! Добиться эффекта можно, запустив утилиту **bind** с ключом **–o**.

```
bind –o –u filename
// без ключа -u изменения в образ записаны не будут,
// если интересно, что
// происходит - используйте ключ -v. В этом случае утилита просто
// озвучит свои действия, но файл не изменит
```

Поле **OriginalFirstThunk** не затрагивается, а поле **FirstThunk** содержит настоящие адреса **API** -функций. Т.е. **IMAGE_IMPORT_BY_NAME** смысла более не имеет, т.к. **DWORD** в **IMAGE_THUNK_DATA** теперь, по сути, представляет адрес **API** -функции в памяти, а не ссылку на **IMAGE_IMPORT_BY_NAME**.

Импорт может просто привязываться, а может и происходить так называемый “ *API forwarding* ”, что можно перевести как "перенаправление API". Это очень изящная штука, которая помогает **MS** скрывать разницу между **OS**. Классический пример с **HeapAlloc**. Определена в **kernel32.dll** , но в действительности **КОДА** ее там и близко нет! Вместо этого вызов сразу перенаправляется на **ntdll.dll** , где упирается в функцию **RtlAllocateHeap**! Т.е. где-то так:

calc.exe - > — kernel32.dll -> — ntdll.dll

(call HeapAlloc) — (немедленный форвард) — (RtlAllocateHeap)

(NTDLL.RtlAllocateHeap)

Этого эффекта можно добиться, употребляя следующее выражение при компиляции dll:

```
#pragma comment(linker, "/export:HeapAlloc=NTDLL.RtlAllocateHeap")
```

Аналогичного эффекта можно добиться и с помощью **def** -файла:

```
EXPORTS
        HeapAlloc=NTDLL.RtlAllocateHeap
```

Exe -файл, использующий **dll** с таким редиректом, должен загружать ее динамически – через **LoadLibrary** /**GetProcAddress**(“ *HeapAlloc* ”) - если импорт по имени, либо по ординалу с помощью макроса **MAKEINTRESOURCE**. Если хотите линковать статически - тоже нет проблем. Код **dll** :

```
#include <windows.h>

BOOL APIENTRY DllMain(
  HANDLE hModule,
  DWORD  ul_reason_for_call,
  LPVOID lpReserved)
{
    return TRUE;
}

void F1(void)
{
 MessageBeep(0xFFFFFFFF);
}
```

def -файл для **dll** (можно обойтись и без него, если использовать директиву **pragma**):

```
LIBRARY Forward
EXPORTS F1 @1
EXPORTS Hi = User32.MessageBoxA @2 NONAME
```

И, наконец, код **exe** , который будет использовать такую хитрую **dll** :

```
#include <windows.h>

extern "C" int __stdcall Hi(int, char*, char*, int);

int main(void)
{
 Hi(0, "Lya-lya-lya", "Test", 0);
 return 0;
}
```

Обратите внимание, в самой **dll** нет и следа кода форварда, это не является необходимым. Директива **Hi = User32.MessageBoxA @2 NONAME** предписывает компилятору перенаправить вызов **Hi** , присвоить этой функции ординал номер два, и сделать так, чтобы в результирующей **dll** имени функции **HE** было (заметьте, имя функции сохранится в lib, но не будет существовать в **dll**). В **exe** файле две директивы extern "C" и __stdcall помогут нам избежать проблем с манглингом функций.

Файл с **API-forwarding** будет иметь таблицу экспорта с функцией, имеющей нестандартный **RVA** , значение которого не будет укладываться в лимит размера директории. Алгоритм обработки таких форвардов заложен в функцию **LdrpSnapIAT** и, как нам кажется, **Russell** в своей статье здесь допустил ошибку. Смотрите:

```
  NTSTATUS
 LdrpSnapIAT
...
{
…
  // there are forwarders
  if (-1 != pImageImportDescriptor->ForwarderChain)
  {
   IATFirstThunkEntry =
    pLoadingItem->ImageBase +
    pImageImportDescriptor->FirstThunk +
    (pImageImportDescriptor->ForwarderChain / 4);
…

}
```

Реальный кусочек кода из лоадера (Windows 2000, SP4):

```
.text:77F99897       mov     eax, [ecx+IMAGE_IMPORT_DESCRIPTOR.ForwarderChain]
.text:77F9989A       cmp     eax, 0FFFFFFFFh ; -1
.text:77F9989D       jz      no_forwarder
.text:77F998A3       mov     ebx, [edi]
.text:77F998A5       lea     edx, [ebx+eax*4]
.text:77F998A8       mov     eax, [ecx]
.text:77F998AA       add     eax, edx
.text:77F998AC       mov     esi, [ecx+IMAGE_IMPORT_DESCRIPTOR.FirstThunk]
.text:77F998AF       add     esi, edx
```

Т.е. не деление, а умножение! Для каждого **IMAGE_IMPORT_DESCRIPTOR** такой код вызывается единожды. **ForwarderChain** здесь является индексом первого форварда.

Продолжаем. Если файл привязан к **dll** , которая загружена по адресу, отличному от **ImageBase** , привязка перестает быть валидной. В этом случае лоадер будет читать **IAT** , если не валидна и она – образ загружен не будет.

Время загрузки файла немного уменьшается – это происходит за счет того, что **NTDLL.dll** уже не патчит адреса внутри модуля, все уже сделано.
При **TimeDateStamp** равном **–1** (т.е. **0xFFFFFFFF**) имеем новый стиль привязки.

Лоадер сначала проверяет **IMAGE_DIRECTORY_ENTRY_BOUND_IMPORT** , а только уж потом обращается к **IMAGE_IMPORT_DESCRIPTOR** (если обращается!). Проведите простой эксперимент: привяжите файл – утилита **bind –u filename** , потом удалите **ВСЕ** содержимое **IMAGE_IMPORT_DESCRIPTOR** , или наполните его мусором. Запустите файл. Ну как? Еще раз

повторим. Если не выполняется строгое равенство **TimeDateStamp** , если файл загружен не на **ImageBase** , если **API** адрес не найден и т.п, то мы опять читаем **IAT**. Поэтому **bind** отказывается производить привязку образов, не имеющих **OriginalFirstThunk** – если что-то пойдет не так, мы ведь не сможем восстановиться!

IMAGE_DIRECTORY_ENTRY_BOUND_IMPORT имеет свою структуру:

```
typedef struct _IMAGE_BOUND_IMPORT_DESCRIPTOR {
    DWORD   TimeDateStamp; //очень разумная мера!
/*Откуда, например, лоадер может знать,
 а не изменилась ли версия dll, к которой привязаны адреса файла?
 Только сравнивая «временные штампы» библиотек.
 Если данный TimeStamp совпадает с таковым у библиотеки – все OK,
 нет – тогда плохо - выставляется флажок ошибки,
 и лоадер начинает работу по чтению IID*/
    WORD    OffsetModuleName;
//offset (а НЕ RVA!!!) на имя библиотеки от начала директории
    WORD    NumberOfModuleForwarderRefs;
/*
 Заметьте, это поле – счетчик,
 указатель количества структур типа IMAGE_BOUND_FORWARDER_REF,
 которые следуют ПОСЛЕ данной структуры.
 Строение их абсолютно идентично таковому у IMAGE_BOUND_IMPORT_DESCRIPTOR
 за исключением поля NumberOfModuleForwarderRefs, которое зарезервировано
*/
// Array of zero or more IMAGE_BOUND_FORWARDER_REF follows
} IMAGE_BOUND_IMPORT_DESCRIPTOR,  *PIMAGE_BOUND_IMPORT_DESCRIPTOR;
```

Обратите внимание, что структуры **IMAGE_BOUND_IMPORT_DESCRIPTOR** и **IMAGE_BOUND_FORWARDER_REF** в файле перемешаны. Форвард означает, что **API** как бы «перенаправляется» в другую **dll**. В ту самую, где, собственно и определен. В этом случае первая **dll** служит как бы переходником, не более того.

«Живой» пример на нашем неизменном **calc.exe** выглядит так:

```
; 1-dec-1999 7:37:27
HEADER:01000238 bound_import_directory dd 3844D037h
HEADER:0100023C            dw offset на "SHELL32.dll"
;поле содержит 0x38, что прямиком отсылает на к 01000270 (0x38+0x38 == 0x70)
;в IDA жмем G, потом +38 и усе ?
HEADER:0100023E            dw 0
HEADER:01000240            dd 37F2C227h ; 30-sep-1999 1:51:35
HEADER:01000244            dw offset на "MSVCRT.dll"
HEADER:01000246            dw 0
...
HEADER:01000266            dw 0
HEADER:01000268 bound_import_directory_terminator dd 0
HEADER:0100026C            dw 0
HEADER:0100026E            dw 0
HEADER:01000270 bound_SHELL32_dll   db 'SHELL32.dll',0
HEADER:0100027C bound_MSVCRT_dll    db 'MSVCRT.dll',0
HEADER:01000287 bound_ADVAPI32_dll  db 'ADVAPI32.dll',0
HEADER:01000294 bound_KERNEL32_dll  db 'KERNEL32.dll',0
HEADER:010002A1 bound_GDI32_dll     db 'GDI32.dll',0
HEADER:010002AB bound_USER32_dll    db 'USER32.dll',0
```

Также обязательно необходимо учитывать, что перебазирование **dll** , к которой привязан данный исполняемый файл (приложение можно "*привязать* " и к своим библиотекам, **bind** это позволяет), также повлечет за собой чтение из "*старой* " директории импорта.

Теперь остановимся на одной грустной-грустной для нас с вами вещи. Положим, мы сдампили программу каким-нибудь дампером, например, **LordPE**. Положим, также, что некоторые опции утилиты были отключены. Что в этом случае у нас будет с директорией импорта? Предположим, имен и ординалов у нас нет!

Очевидно, файл будет рабочим только при соблюдении одного-единственного непреложного условия – все **API** -адреса в **IAT** (*FirstThunk*) должны быть строго валидными. Шаг в сторону – и такой файл можно просто выбросить. Или нет? Здесь мы подходим к понятию "*восстановление таблицы импорта* ".

Под восстановлением мы понимаем воссоздание массива из структур **IMAGE_IMPORT_DESCRIPTOR** , обязательно с именами и ординалами. Один из возможных алгоритмов поиска выглядит так. Предполагается, что файл загружен в память и старая директория импорта не является исковерканной.

```
/************************
 * Функция ищет старую таблицу импорта, и возвращает указатель
 * на первый IMAGE_IMPORT_DESCRIPTOR
 * Первый параметр это - указатель на память (Memory Mapped Files)
 * Второй параметр это - размер памяти
 ************************/

// Процесс этот ресурсоемкий
LPVOID FindOldImportTable(LPVOID pMem, DWORD dwSize)
{
    CHAR *szStr; //Тут хранится имя модуля
    HMODULE hModule;
    PIMAGE_IMPORT_DESCRIPTOR pIID = {0};

    for(DWORD dwRVA = 0; dwRVA < dwSize; dwRVA++)
    {
        // Устанавливаем SEH, так как указатель pIID->Name,
        // иногда ведёт за пределы файла
       __try
        {
            // Получаем структуру IMAGE_IMPORT_DESCRIPTOR по адресу dwRVA
            pIID = (PIMAGE_IMPORT_DESCRIPTOR)((DWORD)pMem + dwRVA);
            // Если pIID->Name равно хоть чему нибудь, идём дальше
            if(pIID->Name)
            {
                // Получаем имя модуля по этому адресу
                szStr = (CHAR*)ImageRvaToVa(
             ImageNtHeader(pMem),
             pMem, pIID->Name,
             NULL);
                if(!IsBadStringPtr(szStr, 40))
                {
                    // Если строка больше 4-х символов, идём дальше.
                    // Почему 4-м? Потому, что ".dll"
                    if(lstrlen(szStr) > 4)
                    {
                        // Загружаем Dll, если загрузилась идём дальше
                        hModule = LoadLibrary(szStr);
                        if(hModule != NULL)
                        {
                            FreeLibrary(hModule); // Выгружаем Dll
       // Небольшая проверка на валидность структуры
       if(pIID->TimeDateStamp == 0xFFFFFFFF ||
       pIID->TimeDateStamp == 0x00000000)
                     // Возвращаем указатель
                     // на первый IMAGE_IMPORT_DESCRIPTOR
                            return pIID;
                        }
                    }
                }
            }
        }
        __except(EXCEPTION_EXECUTE_HANDLER)
        {
            // "Access violation" однако. Все равно, продолжаем...
            continue;
        }
    }
    return NULL;
```

```
}

/************************
 * Функция делает простой rebuild, и возвращает размер новой таблицы импорта
 * Первый параметр это – указатель на память (Memory Mapped Files)
 * Второй параметр это - адрес возвращённый функцией FindOldImportTable
 ************************/

DWORD RebuildIT(LPVOID pMem, LPVOID pStart)
{
 PIMAGE_DOS_HEADER         pDos = {0};
 PIMAGE_NT_HEADERS         pNT = {0};
 PIMAGE_IMPORT_DESCRIPTOR  pIID = {0};
 DWORD                     dwITSize = sizeof(IMAGE_IMPORT_DESCRIPTOR);

 pDos = (PIMAGE_DOS_HEADER)pMem;
 pNT = (PIMAGE_NT_HEADERS)((DWORD)pMem + pDos->e_lfanew);
 pIID = (PIMAGE_IMPORT_DESCRIPTOR)((DWORD)pStart);

 if(!pIID)
  return 0;

 while (pIID->Name)
 {
  dwITSize += sizeof(IMAGE_IMPORT_DESCRIPTOR);
  pIID->TimeDateStamp  = 0x00000000;
  pIID->ForwarderChain = 0x00000000;
  ++pIID;
 }

 return dwITSize;
}

//Пример использования:

DWORD  dwMapSize; // Размер памяти
LPVOID m_pMap;    // Указатель на память (Memory Mapped Files)
PIMAGE_DOS_HEADER pDos = {0};
PIMAGE_NT_HEADERS pNT = {0};

...

// Ищем старую таблицу импорта
IMAGE_IMPORT_DESCRIPTOR* pIID=
(PIMAGE_IMPORT_DESCRIPTOR)FindOldImportTable(m_pMap, dwMapSize);
if(pIID)
{
      // Если найдена, делаем rebuild
 DWORD dwSize = RebuildIT(m_pMap, pIID);
 if(dwSize > 0)
 {
  pDos = (PIMAGE_DOS_HEADER)m_pMap;
  pNT = (PIMAGE_NT_HEADERS)((DWORD) m_pMap + pDos->e_lfanew);
            // Устанавливаем RVA таблицы
   pNT->OptionalHeader.DataDirectory[IMAGE_DIRECTORY_ENTRY_IMPORT].VirtualAddress
                = (DWORD)pIID - (DWORD)m_pMap;
         // Устанавливаем размер таблицы
   pNT->OptionalHeader.DataDirectory[IMAGE_DIRECTORY_ENTRY_IMPORT].Size
                = dwSize;
 }
}

...
```

Заметим, что данный алгоритм будет работать только на самых простых пакерах (например **PECompact**), с более продвинутыми, работать уже **НЕ** будет. Пошла сейчас мода коверкать директорию импорта. И здесь пакеры потихоньку выигрывают, т.к. искалечить всегда легче, чем восстановить. Никаких новых алгоритмов и идей здесь более обсуждаться не будет, т.к. это знание пойдет только во вред ибо может быть использовано авторами пакеров. К нашему глубокому

сожалению в сети существуют некоторые проекты по восстановлению директории импорта с открытым исходным кодом, однако это большой недостаток. Уж лучше иметь работающий **ImpRec** , который худо-бедно но обновляется, чем открытый исходный код с устаревшей, бесполезной идеей.

Теперь рассмотрим, как работать со структурами **PE** -файла в **IDA**.

Простите за банальность – **IDA** – это очень мощный дизассемблер. Практически все операции идут под «капотом». Дизассемблировав **PE** -файл, вы даже не увидите его заголовков, **IDA** автоматически прогуляется по всей таблице импорта, все сделано за вас. Это и хорошо, и плохо. Хорошо это тогда, когда нет ни времени, ни желания ковыряться в секциях, просто быстро поломать. Плохо это тогда, когда попадается хитро подправленный файл, который **IDA** в автоматическом режиме взять не может.

Для примера давайте быстро вручную пробежимся по структурам секции импорта **calc.exe**. Для начала нам надо найти начало секции таблицы импорта. Итак, быренько запускаем какой-нибудь **PE** -редактор, например, **PE Tools**. В **PE Editor** давим кнопочку **Directories**. Как вы помните, это массив из **#define** , где секция импорта идет второй

```
#define IMAGE_DIRECTORY_ENTRY_IMPORT 1
```

Ее **RVA - 12A40**. **ImageBase** файла – **OptionalHeader**. **ImageBase** – **01000000** , следовательно, в **IDA** давим **G** , потом вводим **1012A40** (**01000000** + **12A40**). Вот прямо с этого места у нас и начинается массив из структур типа **IMAGE_IMPORT_DESCRIPTOR**.

Выглядит так:

```
.text:01012A40                db 0C0h ; L
.text:01012A41                db  2Bh ; +
.text:01012A42                db    1 ;
.text:01012A43                db    0 ;
.text:01012A44                db 0FFh ;
.text:01012A45                db 0FFh ;
.text:01012A46                db 0FFh ;
...
```

Обратите внимание на название директории еще раз. Директория будет располагаться внутри секции. Более того, ничто не мешает слить большинство секций файла в одну. Однако мы отвлеклись.

Для того чтобы указать **IDA** , что тут у нас структура и надо бы преобразовать поля, лезем в **Structures**. Там создаем **Ins** структуру тютелька в тютельку по документации из **winnt.h** :

```
0000 IMAGE_IMPORT_DESCRIPTOR struc
; (sizeof=0x14, standard type)
0000 OriginalFirstThunk dd ?
0004 TimeDateStamp   dd ?
0008 ForwarderChain  dd ?
000C Name            dd ?
0010 FirstThunk      dd ?
0014 IMAGE_IMPORT_DESCRIPTOR ends
```

Далее остается применить эту структуру к смещению 1012A40. Примерно так:

```
.text:01012A40            dd 12BC0h               ; OriginalFirstThunk
.text:01012A40            dd 0FFFFFFFFh           ; TimeDateStamp
.text:01012A40            dd 0FFFFFFFFh           ; ForwarderChain
.text:01012A40            dd 12CE6h               ; Name
.text:01012A40            dd 10F4h                ; FirstThunk
```

И так до тех пор пока не попадется **IMAGE_IMPORT_DESCRIPTOR** , ВСЕ поля которого содержат нули. Аналогичную песню с определением новых структур придется спеть и для **IMAGE_THUNK_DATA** (а, в случае **TimeDateStamp == 0** и для **IMAGE_IMPORT_BY_NAME**) и т.д., и т.п. А может и не придется! Ведь **IDA** хранит список стандартных структур! Поэтому, *Add struct type*

> *Add standard structure* > *Search*. Но неплохо бы это все дело автоматизировать еще больше! Что ж, не мы с вами такие ленивые. Топаем на datarescue.com/idabase/idadown.htm и сливаем оттуда **PE utilities** от **Atli Mar Gudmundsson**. Необходимо поместить их в **%IDADIR%\idc** , а потом **F2** и файл **pe_sections.idc**. Если до этого вы не просто читали, а и пытались что-то делать ручками, то будете приятно удивлены.

2.4 Директория ресурсов

Очевидно, что в этой директории программисты хранят иконки, кнопочки, и прочие маленькие радости пользователя. Уже упоминалось, что директория ресурсов обязательно должна размещаться в секции **.rsrc** , причем имя секции должно быть в точности таким – "*.rsrc* ". Причины уже объяснялись.

Теперь поговорим о восстановлении ресурсов. Для чего это бывает необходимо и каковы здесь технические детали?

Взгляните:

```
typedef struct _IMAGE_RESOURCE_DIRECTORY_ENTRY {
    union {
        struct {
            DWORD NameOffset:31;  //сюда
//Обратите внимание - offset от начала секции.
            DWORD NameIsString:1;
        };
...
} IMAGE_RESOURCE_DIRECTORY_ENTRY, *PIMAGE_RESOURCE_DIRECTORY_ENTRY;

typedef struct _IMAGE_RESOURCE_DATA_ENTRY {
    DWORD   OffsetToData;   //и вот сюда
// Несмотря на название - это самый типичный RVA
    DWORD   Size;
    DWORD   CodePage;
    DWORD   Reserved;
} IMAGE_RESOURCE_DATA_ENTRY, *PIMAGE_RESOURCE_DATA_ENTRY;
```

Здесь хотелось бы процитировать **Randy Kath** с его статьей "The Portable Executable File Format from Top to Bottom": "*The two fields OffsetToData and Size indicate the location and size of the actual resource data. Since this information is used primarily by functions once the application has been loaded, it makes more sense to make the OffsetToData field a relative virtual address. This is precisely the case. Interestingly enough, all other offsets, such as pointers from directory entries to other directories, are offsets relative to the location of the root node.* "

Да, справедливо то, что полностью уничтожить директорию нельзя и теперь мы знаем почему. Однако, положим, вам пришлось проделать некие хитрые манипуляции с вашим конкретным файлом, например, после распаковки убрать более ненужные секции упаковщика. ОК, вы их убрали и лишились ресурсов, т.к. **RVA** перестал быть валидным! Значит, надо ресурсы восстановить, т.е. пересчитать смещения. Для этой цели у нас популярным средством является **Resource Rebuilder v1.0** by **Dr.Golova** , а у зарубежных коллег, пожалуй, **PE Rebuilder 0.96**. О сугубо практическом применении можно почитать здесь - http://xtin.org. Статья - Отрезание секций, перемещение ресурсов, автор **Hex**.

Восстановление ресурсов, к счастью, не самая сложная процедура. Вкратце, основные принципы таковы:

1. Получаем из **PE** заголовка секцию с именем " *.rsrc* ".
2. Считывается дерево каталогов. Сами данные ресурсов пока можно и не считывать, а считать их на стадии создания новой секции ресурсов.

3. Создаём новую секцию ресурсов.
1. Подсчитываем необходимый размер будущей секции.
2. Далее записываем все ресурсы (каталоги/имена/данные) в эту секцию.
3. Затем делается ещё один проход и корректируем все смещения в каталогах.
4. Корректируем **PE** заголовок, если это необходимо.

Здесь алгоритма не будет, так как он очень простой, но большой.

Так же стоит помнить, что и в памяти пакерам не следует грохать заголовок PE-файла (во всяком случае, определенную его часть), т.к. ресурсные **API** , например, **LoadResource** , просто перестанут работать. Вероятно, ситуацию можно свести к следующему предположению: если хендл чего-то из **PE** -файла был получен, то далее эту часть **PE** -заголовка можно убирать, чтобы затруднить нам жизнь.

Словом, пакеры оставляют все сигнатуры, **MZ** поле **e_lfanew** , **OptionalHeader**. **DataDirectory**[x] и т.п., но часть жизненно важных данных может быть искажена. Поэтому лучше не играть в кошки-мышки, а просто-напросто в опциях дампера (**PE Tools** , **LordPE**) активировать режим вставки заголовка файла с диска (Full Dump: Paste header from disk).

2.5 Директория перемещаемых элементов

Теперь, в довесок к изуродованию директории импорта, секции кода, директории ресурсов и т.п., появляется возможность калечить директорию перемещаемых элементов (*relocations* , *fixups*). Например, такой фокус производит **UPX** (но там открыт исходный код, следовательно, нужно ожидать такого подарочка от многих пакеров в скором времени).

Информация из директории перемещаемых элементов применяется тогда, когда адрес загрузки файла отличается от **ImageBase**. Для **exe** -файлов такое встречается достаточно редко (но не так, чтобы и очень – вспомните **VTune**), а для **dll** – это норма. Так вот, директория перемещаемых элементов содержит информацию о том, как патчить данные, расположенные по указанному адресу, чтобы было возможно нормальное продолжение работы. В противном случае, операнд инструкции будет указывать в "космос".

Структура директории выглядит просто:

```
typedef struct _IMAGE_BASE_RELOCATION {
//с этого адреса лоадер должен начинать падчить адреса
    DWORD   VirtualAddress;
//размер fixup-блока для данной страницы памяти
    DWORD   SizeOfBlock;
//  WORD    TypeOffset[1]; //тип фиксапа
} IMAGE_BASE_RELOCATION;
```

Давайте разберем на живом примере, ЧТО может быть подвержено перемещению и как с этим работать. Итак, создадим простейший файл:

```
#include <stdio.h>

void main(void)
{
 char hello[] = "Name";

 printf ("%s", hello);
}
```

Только попросим при этом линкер создать исполняемый файл с опцией **/FIXED:NO** , что означает автоматическое создание директории перемещаемых элементов (как правило, это секция **.reloc** , хотя и не обязательно). Теперь дизассемблируем в **IDA** , применяем скрипт от *Atli Mar*

Gudmundsson и смотрим:

```
loc:1002B000      ; DATA XREF: HEADER:100000D8o
; HEADER:10000298o

.reloc:1002B000 relocs_0001_page dd 11000h
.reloc:1002B004 relocs_0001_size dd 54h
.reloc:1002B008 relocs_0001_toff dw 3A4Fh, 3A58h, 3A64h, 3A78h, 3A9Bh,
...
; ограничено VirtualSize секции
```

Итак, поле **relocs_0001_page** , очевидно, соответствует полю **VirtualAddress** (напоминаем, что релоки применяются к виртуальным страницам, отсюда и слово **page**), **relocs_0001_size** - **SizeOfBlock** , **relocs_0001_toff** - чуть поинтереснее. Как видим, это массив из word, где старшие четыре бита - это тип релока (в даном случае - **#define IMAGE_REL_BASED_HIGHLOW 3**), а оставшиеся **12** надо сложить с **VirtualAddress** , чтобы получить **RVA** данных для обработки. Ну-ка, давайте посмотрим, куда указывает самое первый **word** из массива - **3A4F**. Итак, A4F + 11000:

```
.text:10011A4E            mov     eax, dword ptr ds:aName ; "Name"
.text:10011A53            mov     [ebp-0Ch], eax
.text:10011A56            mov     cl, byte ptr ds:aName+4
.text:10011A5C            mov     [ebp-8], cl
.text:10011A5F            lea     eax, [ebp-0Ch]
.text:10011A62            push    eax
.text:10011A63            push    offset aS_0             ; "%s"
.text:10011A68            call    j__printf
```

Ба! Да прямо в центр инструкции **mov** , т.е. адрес начала массива &hello[0] подвержен обработке! И так много чего. Релокейшенам подвержены строковые литералы, статические переменные, ссылки на неинициализированные статические данные, абсолютные **call/jmp** , да мало ли что еще! Словом, кошмар!

Релоки обрабатываются лоадером при загрузке – функция - **LdrRelocateImage**. Обработка релоков происходит перед обработкой таблицы импорта (не всегда!). Функция **LdrpMapDll** зовет **LdrRelocateImage** , которая, в свою очередь, определяет заголовок файла, вычисляет где расположена директория релоков и зовет **LdrProcessRelocationBlock**.

LdrRelocateImage вызывается когда **ZwMapViewOfSection** (бывшая **NtMapViewOfSection**) возвращает **STATUS_CONFLICTING_ADDRESSES**. В случае загрузки по адресу, отличному от **ImageBase** модуль загружается полностью (обратите внимание, пакер в любом случае обязан пройтись по всем страницам, поэтому запакованный модуль действительно загружается полностью), релоки накладываются сразу, и потом секция с релоками выгружается из памяти (флаг секции **IMAGE_SCN_MEM_DISCARDABLE**) в своп и в дальнейшем используются только от туда, т.к. из dll Windows более ничего не читает. Что-то отдаленно похожее алгоритмически можно найти в **Wine** - функция **map_image** из файла **virtual.c**. Реальный код из лоадера выглядит так (Win 2000 SP4):

```
again:
mov     ecx, [eax+IMAGE_BASE_RELOCATION.SizeOfBlock]
lea     edx, [eax+_IMAGE_BASE_RELOCATION.TypeOffset]
sub     [ebp+HMODULE], ecx
; RtlImageDirectoryEntryToData перетирает это значение.
; Теперь оно содержит размер всех релоков
; эквивалентно вычитанию восьми
add     ecx, 0FFFFFFF8h
; -IMAGE_BASE_RELOCATION = sizeof(Type/Offset entries)
push    ebx  ; Bias
push    edx  ; Type/Offset entries
shr     ecx, 1          ; Each Type/Offset entry is a word
push    ecx  ; Number of Type/Offset entries
mov     ecx, edi ; HMODULE
;т.е. прибавить RVA
add     ecx, [eax+IMAGE_BASE_RELOCATION.VirtualAddress]
push    ecx  ;т.е. имеем VA
```

```
call    _LdrProcessRelocationBlock@16
test    eax, eax ; указатель на следующий блок либо ноль
jz      error
cmp     [ebp+HMODULE], 0
jnz     short again
```

Как вы видите, будут обработаны все блоки перемещаемых элементов, а уж есть ли релоки во ВСЕХ секциях модуля – это совсем другая песня. Сам алгоритм **LdrProcessRelocationBlock** достаточно тривиален и очень хорошо расписан во многих источниках (www.RSDN.ru, http://mitglied.lycos.de/yoda2k и т.п.), где есть исходники и на **C/C++** и на **Delphi**.

Заметим, что применение релоков чревато не только увеличением времени загрузки, а и частыми **page fault** - вследствие применения релоков к секции получаем ее копию в памяти - механизм **copy-on-write** - подробнее **MSDN** - *Ruediger R. Asche - Rebasing Win32 DLLs: The Whole Story*.

Итак, мы имеем еще один неутешительный вывод - после применения релоков пакер может эту директорию смело убирать, что он и делает.

Восстановление **relocation directory** – дело непростое. Стандартной техникой со времен **DOS** считалась загрузка файла по двум разным базовым адресам, и поиск различий в них (например, по этому принципу работает утилита **ReloX** от **mackT**). Для загрузки одного и того же файла его достаточно переименовать, иначе лоадер откажется загружать файл повторно (**LdrpCheckForLoadedDll**). Заметим, что директория перемещаемых элементов все равно сохраняется под пакером, пусть и в его формате, следовательно, нет принципиальных сложностей в загрузке модуля по двум разным базовым адресам.

Однако в **PE Tools** используется совершенно безумный алгоритм статического анализа файла, изобретенный **NEOx'ом** после нескольких бутылок пива (водки?). Итак, основной принцип восстановления релоков – это поиск предположительных ссылок, которые в той или иной степени могут быть подвержены релокам. Ссылки находятся следующим образом:

1) Сканируем все секции от начала до конца (читать данные **DWORD** -ами, смещаясь на один байт в цикле).

2) Из полученного значения отнимаем **OptionalHeader.ImageBase** (это нужно для того, чтобы вычислить адрес ссылки)

```
dwOpcode -= pPeh->OptionalHeader.ImageBase;
```

3) Проверить является ли этот адрес ссылкой, а не кодом. Проверяем – если полученное значение больше **OptionalHeader.BaseOfCode** , но меньше **OptionalHeader.SizeOfImage** , значит это ссылка, если нет – читаем следующий **DWORD**.

```
if(dwOpcode > pPeh->OptionalHeader.BaseOfCode)
  if(dwOpcode < pPeh->OptionalHeader.SizeOfImage)
  {
  ...
  //здесь тоже не все так просто!
  }
```

4) На основе этих данных генерируем новые релоки.

2.6 NTDLL.DLL – Windows Loader

Также необходимо разобраться, что же происходит при загрузке exe/dll в память. Сразу оговорюсь, что процесс этот не будет рассмотрен вообще – не любим повторов.

Для начала необходимо достаточно четко представлять себе архитектуру распределения памяти в Win NT+. Думается, что лучше, чем это сделал Джеффери Рихтер в своей книге "*Windows для*

профессионалов " это не объяснил никто. Такое понимание сильно облегчает жизнь. Например, если вы встречаете нечто вроде адреса 7xxxxxxx, то можете не сомневаться – это адрес какой-нибудь **API** функции из системных библиотек. Не плохо бы представлять себе и границу стека. ПРИБЛИЗИТЕЛЬНО регион стека в любом вьювере адресного пространства легко опознать по атрибуту страницы **PAGE_GUARD**. Приблизительно потому, что этот атрибут страницы смещается по стеку во время выполнения.

Загляните в MSDN - статья Q147314 – описание флагов Session Manager. Среди них есть и такой, который требует от NTDLL.dll "*озвучивать* " свои действия, т.е. выдавать сообщения в окно отладчика (WinDBG, отладчика VS и т.п.). Для того чтобы не морочить себе голову запоминанием флагов, MS создала утилиту **GFlags.exe** , которая поставляется вместе с комплектом отладочных утилит с сайта **MS**. Скачать это дело можно совершенно свободно. Мы бы порекомендовали активировать флаг "озвучивания" действий лоадера, т.к. это будет исключительно полезно, например, в тех случаях, когда сдампленная программа не запускается, а лоадер говорит: "*Application failed to initialize properly* ". В этом случае вы сможете быстро найти ту функцию ntdll.dll, которая отказалась "*принять* " файл и, зная алгоритм ее работы, у вас уже будет какой-то ключик. Это адова работа, но в некоторых случаях такая методика помогает.

Учтите, что **ntdll.dll** автоматически проецируется на адресное пространство любого Windows-приложения.

Знание работы лоадера часто является абсолютно необходимым. Допустим, у вас есть вопрос, ответа на который вы не нашли ни здесь, ни где-либо еще. Что остается? Дизассемблирование лоадера! Здесь тоже не придется действовать вслепую, уже давно проторены дорожки.

Поверхностно алгоритм работы лоадера расписан на http://www.iecc.com/linker/. Так же алгоритм работы можно приблизительно уяснить из исходных кодов **Wine** и **ReactOS** (http://www.reactos.com - своеобразный аналог **Wine** , только цели чуть другие). Во всем этом деле полезным будет справочник по Native API - http://undocumented.ntinternals.net. Также многие функции лоадера, например **RtlImageHeader** и т.п., прокомментированы на более высоком уровне в **MSDN** – библиотека **imagehlp.dll**.

Для облегчения анализа обязательно сгрузите себе файлы символов – **pdb/dbg**.

mdsl.microsoft.com
Современные утилиты (*VS. NET 2003/Driver Studio 2.7*+) имеют специальные средства, которые облегчают нам жизнь при копании в недрах Windows. Раньше приходилось топать на http://www.microsoft.com/, и сливать символы ручками, конвертировать их ручками и т.п. Теперь все достаточно просто. В состав **DS 2.7** и старше входит утилита **Symbol Retriever** , которая позволяет сливать символы со специального сервера **MS**. http://msdl.microsoft.com/download/symbols. Естественно, что и **VS .NET 2002/2003** способна обращаться к этому серверу (еще бы!). Достаточно задать в переменных окружения _NT_SYMBOL_PATH, и присвоить ей значение типа _SRV*Symbols**Symbols*_http://msdl.microsoft.com/download/symbols. Более подробно - MSDN - статьи Джона Роббинса.

Самое главное при дизассемблировании лоадера – начинать плясать от определения структур данных. При дизассемблировании файла в **IDA** вставьте структуры **winnt.h**(метод описан выше), и начинайте превращать код из неудобоваримого:

```
mov ax, [edi+4]
```

в

```
mov ax, [edi+IMAGE_BOUND_IMPORT_DESCRIPTOR.OffsetModuleName]
```

Удачи! :)

3. 64-bit PE-files

Ну, не так страшен черт, как его малюют. PE файлы изначально проектировались очень разумно, поэтому изменения, связанные с переходом на 64-битную архитектуру, минимальны. С другой стороны – то же все не слишком безоблачно, в первую очередь, из-за типа int64. Но давайте по порядку.

Итак, 64-bit PE файлы еще называются как PE32+. Отличить их от обычного файла можно по сигнатуре. Сигнатура расположена в поле OptionalHeader.Magic и равна

```
#define IMAGE_NT_OPTIONAL_HDR32_MAGIC      0x10b //PE
#define IMAGE_NT_OPTIONAL_HDR64_MAGIC      0x20b //PE32+
```

Для определения типа файла можно использовать эту функцию, которой необходимо передать адрес на проекцию файла:

```
/**********************
 * Return value: TRUE если файл 64-х битный, FALSE если файл 32-х битный
 * Parameters: pMem - возвращаемое значение от CreateFileMapping,
 * либо от GetModuleHandle
 **********************/

BOOL IsPEPlus(LPVOID m_Map)
{
 PIMAGE_NT_HEADERS pNT;
 pNT = ImageNtHeader(m_Map);
 if(pNT)
 return pNT->OptionalHeader.Magic == IMAGE_NT_OPTIONAL_HDR64_MAGIC ?
     TRUE : FALSE;
 else
  return FALSE;
}
```

Различий же в строении самих структур (т.е. секций, директорий и т.п.) нет. Естественно, что **DWORD** заменяется на **ULONGLONG** , т.е. unsigned int64 (ха-ха!). Разумеется, добавлены новые типы флагов для совместимости, например, с **AMD64** и т.п. В директории импорта теперь появляется структура **IMAGE_THUNK_DATA64** с типами **ULONGLONG** , изменения коснулись и ресурсов.

Особо останавливаться на этом не будем.

Такие модули начнут попадаться все чаще и чаще, однако особой паники, думается нам, в этом нет. **IDA** вполне готова, **HIEW** грустно говорит об "*Unknown header magic* " и редактировать поля более не позволяет, **LordPE** и **PE Tools** на высоте.

Единственное, что смущает во всем этом деле – так это сам тип **int64** , а, вернее, его полное отсутствие на 32-битных машинах. Компиляторы просто-напросто эмулируют **int64** путем помещения двух **DWORD** в стек, причем, понятия unsigned int64 пока нет вообще! Словом, у **MS** ещё не все ладно с поддержкой самого типа. А в будущем… Конечно, проблему решат. Например, переходом на Windows IA-64.

4. Страшное слово XML

Данную главу вы можете смело пропустить на данный момент времени, т.к. создание плагина под PE Tools для генерирования XML-отчета пока только планируется. С другой стороны, данная глава дает очень краткий экскурс в эту несложную, но достаточно специфическую тему, а знание никогда не бывает бесполезным, в особенности в свете того, что имеется тенденция использовать **XML** -файлы в качестве файлов конфигурации.

Для того чтобы вывести само строение **PE** -файла необходим какой-то очень наглядный формат вывода. Например, такой, в котором мы сможем описывать его структуру и составляющие. Самыми известными являются **XML** и **ASN.1**. Последний имеет некоторые преимущества над **XML** (например, платформо-независимые правила бинарного перекодирования - **BER** , **DER** и т.п., хотя с появлением **XER** - правил бинарного кодирования для **XML** все немного изменилось), имеет куда более длинную историю развития, но не слишком понятен, громоздок, и, самое главное, многим просто НЕ симпатичен. Поэтому после работы данной программы вы получите **XML** -файл, описывающий структуру заголовков **PE** -файла (некоторые поля), и подробно раскрывающий содержание таблицы импорта.

XML – extensible markup language

Формат появился на свет не так уж и давно, однако сразу завоевал бешеную популярность. Его активно используют по всему миру как формат обмена данными между программами (различными ОС, приложениями и т.п.). Формат этот платформо-независимый, еще бы, ведь это чистый текст. От HTML внешне он отличается только тем, что название тегов в HTML жестко фиксировано (например, ТОЛЬКО тег <TITLE> и ничто другое не может стоять для вывода информации в строку титула). В XML названия тегов задаю я сам. Получается весьма наглядно и интуитивно понятно. Например: Сравните, как много информации нам дает XML, и как мало – HTML. Да, в первом случае все, быть может, чуть нагляднее представлено браузером, однако во втором – намного более информативно. На каждый XML-документ налагаются достаточно жесткие требования. Как минимум, он должен быть well-formed. Мы бы перевели это как «корректно оформленный» - нельзя делать, например, так: Далее, для каждого XML-документа должен существовать (ну, скажем, ЖЕЛАТЕЛЬНО, чтобы существовал) некий словарь, на соответствие которому мы сможем проверить, а правильно ли составлен документ XML (валидность – valid/invalid), т.е. законны или незаконны применяемые теги. Для будущего плагина, без сомнения, будет выбрана проверка в виде XML-Schemas (есть и другие, однако это – самая лучшая). XML хорош и тем, что позволяет трансформировать документ в другие типы документов при помощи xsl(xslt)-файла. Последний тоже является xml-документом, который представляет собой совокупность директив для программы – XSLT-процессора, принимающей на вход оригинальный XML-файл, и преобразующей его согласно XSLT-файлу. Вывод осуществляется в другой файл или в STDOUT. В качестве замечательного примера такой программы назову XMLSpy (www.altova.com), который позволяет не только применить XSLT к выбранному документу, но и проделать много других вещей, например, проверить на well-formness, валидность и т.п. Более подробно об этом мы поговорим ниже.

```
HTML:
         <html>
         <head>
         <title>PE file format</title>
         </head>
         <body>
         <table>
         <TR>216</TR> <!—А что такое 216? -!>
         </table></body>
         </html>

XML:

<PE>
   <IMAGE_DOS_HEADER>
   <e_lfanew>216</e_lfanew>
   <!—Ах, да это же содержимое одного из полей заголовка! -!>
   </IMAGE_DOS_HEADER>
</PE>

<PE>
   <IMAGE_DOS_HEADER>
</PE>
  </IMAGE_DOS_HEADER>
 <!--должен соблюдаться уровень вложенности!-->
```

Теперь немного о самой будущей программе. **NEOx** дает возможность создания плагинов под **PE Tools** , однако, пока совершенно не предоставляет никакого **API** для удобства работы, а поскольку мы люди ленивые, то пока нету API, нет и плагинов.

Сама программа будет написана на **C** , т.е. на компилируемом языке, и для своего запуска не требует ничего (кроме **Windows**). Однако преобразование **XML** формата в **XSLT** , выполняемое на **C** либо даже **C++** - занятие нетривиальное! Поэтому для преобразования с использованием **XSLT** (или **XSLT** : **FO** для психов) традиционно используются другие языки. Например, несравненный **Perl** либо богатейшая библиотека классов **Java**. Есть, конечно, и **MS XML SDK** , однако в некоторых случаях лично у нас вызывается жуткое отвращение, в особенности когда MS пытается обозвать свои разработки "*стандартом* ". **XML** специально разработан так, чтобы разработчики на разных операционных системах могли свободно обмениваться данными. Это гарантируется самой спецификацией **XML**. MS это пытается нарушить, вводя свои правила. Поэтому, **MS XML SDK** здесь рассмотрен не будет! Хотя над самой темой обработки XML на C/C++, вероятно, стоит подумать.

Для того, чтобы иметь возможность конвертировать **XML** в **HTML** при помощи **Perl** , вам надо скачать Win32-Perl-интерпретатор (www.activestate.com), проинсталлировать его (а иначе, зачем было качать?), сделать определенные установки и можно начинать работать.

После того, как скачали и поставили дистрибутив **Perl** , самое время воспользоваться **PPM**. **PPM** – это *Perl Package Manager* – утилита, позволяющая управлять дополнительными модулями. В данном случае нам потребуется модуль **Win32API-File** , который обеспечивает доступ к некоторым **Win32API** -функциям работы с файлами. По умолчанию **PPM** выполняет поиск в так называемых « *репозиториях* » - одна из них расположена на вашем диске, вторая – в сети на сайте http://www.activestate.com (который, кстати, сильно проигрывает по свежести многим другим зеркалам, позже, мы поговорим об этом).

PPM 3.0.1
PPM – пакет откровенно глюкавый. Что поделать – цена новых необкатанных технологий. Ждем новых версий (почаще заглядывайте на activestate). Посему - НЕ инсталлируйте **Perl** в директорию, имя которой содержит пробелы (вроде Program Files) – **PPM** помрет в муках. Инсталлируйте его куда-нибудь вроде *C:\Perl* , *D:\Perl* или что-то в этом роде. Массу нервов сбережете! Или, как вариант, можно попробовать **VPM** из состава *Perl Dev Kit*. Если уж совсем модуль ставиться не хочет, хоть плачь, то и тогда есть выход – загрузите его вручную, и запустите nmake –install. Только, к сожалению, и здесь камней более чем достаточно.

Ну вот, **XML** -файл готов. Самое время превратить его в **HTML**. **XSL** -файл у нас есть (см. приложенный пакет), осталось найти утилиту, которая преобразует полученный **XML** -файл согласно инструкциям из **XSL** -файла. Мы возьмем три примера:

1. **XMLSpy**
2. **VisualXSLT** + command-line Perl parser
3. Saxon + command-line Java-parser. Сразу скажем, что наиболее просто и приятно использовать XMLSpy. Вес пакета ~ 30 Мб, загрузите его с http://www.altova.com.

XMLSpy – открываете два документа – полученный в результате работы утилиты **xml** -файл и приложенный **XSLT** -файл. **F10** – и наслаждаетесь полученным **HTML** -файлом. Можете сохранить. Все. Да, если взбредет в голову проверить на валидность – **F8** , на корректность – **F7**. Впрочем, если что-то пойдет не так, вы об этом быстро узнаете.

VisualXSLT + command-line Perl parser – немного более болезненно. **VisualXSLT** – это модуль к **VS.NET**. Т.е. у вас должна стоять *Visual Studio*. Сам пакет брать с *activestate*. В установках проекта помечаете формат выходного файла, **F5** и все. Если парсер запускается из коммандной строки то тогда у вас должен быть собственно сам парсер (приложен). Для работы приложенного парсера требуется библиотека **LibXSLT**. Ее нужно загрузить из интернета:

http://theoryx5.uwinnipeg.ca/ppmpackages/

Есть такое себе зеркало **activestate** , которое ведет Randy Kobes (randy@theoryx5.uwinnipeg.ca) – хороший мужик и зеркало очень приличное. Сгружайте оттуда. Заходим в PPM -

```
ppm> rep
Repositories:
[1] ActiveState PPM2 Repository
[2] ActiveState Package Repository

ppm> rep add "Randy"
http://theoryx5.uwinnipeg.ca/cgi-bin/ppmserver?urn:/PPMServer58
Repositories:
[1] ActiveState PPM2 Repository
[2] ActiveState Package Repository
[3] Randy

ppm> search XML-LibXSLT
...

ppm> install XML-LibXSLT
...
Successfully installed XML-LibXSLT version 1.53 in ActivePerl 5.8.0.806.
```

Да, если поставили Перл в директорию, имя которой содержит пробелы – несколько приятных часов вам обеспечено %).

Если же модуль по каким-либо причинам ставиться не хочет, тогда топайте на cpan.org и сгружайте его оттуда вручную (если он там есть). Потом вручную настраивайте makefile и потом nmake -install.

Make, чтоб ее...

Что тут говорить... Ходят шутки, что даже сам создатель утилиты как-то в слезах признавался, что он больше никогда ничего подобного не сотворит. Видать, боялся, что побьют. И действительно, более неудачное решение трудно себе представить. По сути, make из *nix (и ее аналог от MS – nmake) – это утилита, которая берет на вход **makefile** из той директории, откуда ее запустили и прорабатывает инструкции, в нем содержащиеся. После чего вы либо с радостью видите готовый exe-файл (что бывает редко), либо с ругательствами принимаетесь разбираться в формате make-файла (что бывает гораздо чаще). Некоторые сторонники make говорят, что это кросс-платформенный вариант, так вот - это ложь. **Makefile** неизбежно использует ряд платформо-привязанных команд (имя компилятора, директивы командного процессора и т.п.). Кроме того, файл нельзя отлаживать (nmake –n не в счет). В данной статье не будет обсуждаться, что лучше make, однако, посмотрите, например, **Ant**. А пока, поскольку и **Perl** использует make (**nmake**) – на всякий случай приложен маленький симпатичный турториал по makefile, изучив его, вы сможете поправить makefile, если что-то пойдет не так. Также держите в уме, что makefile в процессе создания **XS** -модуля генерируется автоматически скриптом **makefile.pl** (точнее, модулем, подключенным к этому скрипту) на основе данных, собранных во время инсталляции дистрибутива. Из этого вытекает, что, хотя генерация **makefile** абсолютно платформо-независима, но ДАННЫЕ, на основе которых он собирается, ЯВЛЯЮТСЯ зависимыми. Поэтому, скажем, если вы поставили Perl, а потом поменяли, скажем, **VS 6.0** на 2002 или 2003, вам не просто придется пересоздавать makefile, но переинсталлировать сам **Perl**.

Теперь вкратце пробежимся по разным **XSLT** -преобразователям.

Saxon + command-line Java-parser

Для этой бандуры необходим **JDK** – брать с java.sun.com – он называется J2SE. Сливайте, ставьте и вперед. После установки надо выставить переменную окружения **CLASSPATH** – для W2k это делается через окошко «Система». Сама **CLASSPATH** должна выглядеть так: *.;C:\путь_к_JDK\lib\tools.jar*. Обратите внимание на точку и точку с запятой в начале строки! Также добавьте в **PATH** путь к JDK. Теперь можно запускать парсер. Парсера звать **Saxon** – очень навороченная Java-игрушка. Запускать так:

```
java -jar saxon7.jar имя_файла.xsl имя_файла2.xml
```

Но **Saxon** большой и качать его долго. Поэтому приложен парсер поменьше и попроще – **Stylizer**. Скомпилированный файл называется Stylizer.class. Запускать так:

```
java Stylizer
```

Обратите внимание на отсутствие расширения!

Существуют и другие утилиты – например, **Xalan**. Однако и тех, что здесь перечислены – хватит за глаза.

Единственная вещь, которая может вызывать смущение – это размер **XML** -файла. Хотя трудно представить себе такой **PE** -файл, отчет на который займет метров 50 в виде **XML** , однако… Тогда все **xslt** -преобразователи благополучно слетят либо с **OutOfMemoryException** , если это Ява, либо просто помрут в муках, если это C/C++ (потому что чучундры, которые их писали, просто не могли себе представить, что malloc/new может вернуть ноль, кстати, выброс исключения new введен стандартом ANSI-ISO именно поэтому – слишком мало людей озаботились проверить код возврата!). Для столь больших файлов применяется совсем другая схема – пишется разовый **SAX** -парсер, настроенный на конкретные цели, однако обсуждение этой темы здесь совсем не к месту. Словом, если вам не встретятся файлы отчетов свыше мегабайт, скажем, 30, то и волноваться нечего.

Ну вот, вроде и все. По идее, теперь вы должны представлять строение **PE** -файла, уметь играться с **Perl** и **Java** , а также приблизительно понимать **XML** , и уметь применять **XSLT**. Также, наверняка, вам очень хочется сказать нам что-то теплое, ласковое, домашнее за ваши муки при чтении.
Хотелось бы заметить, что сам характер материала, излагаемого здесь, должен подтолкнуть вас к самостоятельным исследованиям. Нельзя верить тому, что пишут, проверяйте сами! Наверняка большая часть алгоритмов, здесь описанных, в скором времени работать перестанет! Поэтому скажем еще раз - нечего плеваться, если нашли оговорку или опечатку, они неизбежны, несмотря на все наши старания. Все надо проверять!

Теперь самое время расстаться с нашими игрушками и рассмотреть реальные случаи.

---

5. Благодарности

Ну, традиционный пункт, который все авторы старательно пишут, а все читатели старательно игнорируют. Можете игнорировать и вы, кусать никто не будет.

Первым пишу я, **Volodya**.

Хотелось бы выразить искреннюю благодарность и признательность **Евгению Сусликову** за данные консультации по RVA/VA и файловым оффсетам. Я также горжусь знакомством со **sla-md** , который дал мне очень подробные объяснения о преобразовании флагов секций PE-файла в соответствующие атрибуты, объяснил некоторые сетевые трюки и вообще, просто является гениальным человеком. Щіре дякую **Quantum'y** за успешную попытку статической линковки API-forward-функций. Так же обязательно стоит поблагодарить **Russell Osterlund** за некоторые пояснения по работе лоадера. Самые теплые слова (эх, как звучит!) надо сказать моей любимой команде - **HI-TECH** (wasm.ru) - **Edmond** , **Aquila** , **Broken Sword** , **Four-F** - спасибо за ценные замечания, указания, пожелания, вылавливание мелких и крупных блох в моих рассуждениях. Спасибо!

Вторым пишу я, **NEOx**.

Хотелось бы выразить особую благодарность **Доктору Голове** за его консультацию по поводу восстановления директории ресурсов, а так же всей команде **uinC**.

Итак, что дальше.

Вторая часть статьи будет рассказывать об алгоритмах работы элементарного пакера, самых легких случаях нахождения **OEP** , распаковке ручками Aspack/UPX/telock/pelock/crunch и т.п. Будет упомянут механизм **SEH** и некоторые антиотладочные приемы.

Третья часть посвящена очень сложным защитным системам – Armadillo, Obsidium, Asprotect.

Засим разрешите распрощаться до следующей части. *Ave*.

SentinelLM!WIscGen_CRK_2

Автор: Quantum

Дата: 2003-09-10

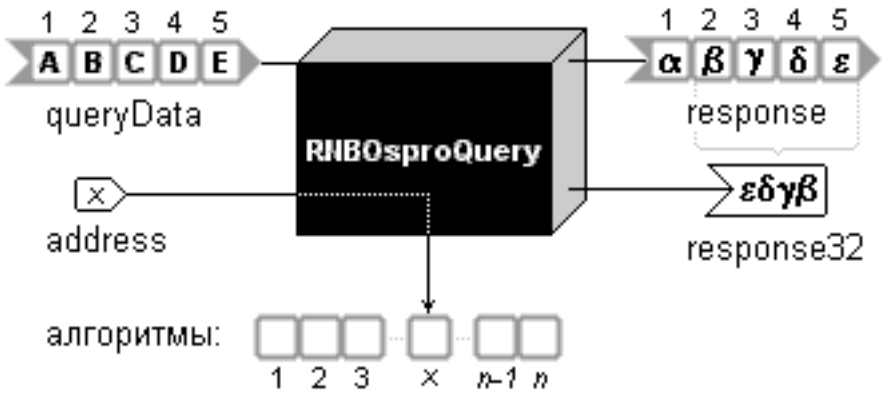
Дорогие коллеги, предлагаю вашему вниманию заключительную статью из цикла о SentinelLM. Вы уже знаете, что от полного счастья нас отделяет одна единственная функция SLM API. Итак, пришло время нанести смертельный удар по SLM SDK, т.е. пропатчить функцию RNBOsproQuery.

Я наивно полагаю, что вы уже изучили "SentinelSuperPro 6.0 Developer's Guide" и прекрасно представляете себе работу этой функции, но "повторение -- мать учения" (© русский народ). RNBOsproQuery уподобается чёрному ящику, в который поступают произвольные данные и выходят данные не менее произвольные. Количество байт во входном массиве равно количеству байт в выходном. Последние 4 байта выходного массива дублируются в 32-битной ячейке памяти.

RNBOsproQuery(packet, address, *queryData, *response, *response32, length)

address -- это номер ячейки донгла, с которой начинается алгоритм; *queryData* -- входной массив; *response* -- выходной массив; *response32* -- последние 4 байта выходных данных, представленные в виде 32-битной переменной; length -- длинна (в байтах) как входного, так и выходного массивов.

Чтоб окончательно вас запутать, далее представлен "графический эквивалент" описания RNBOsproQuery:



А что всё-таки там за алгоритм кроется в недрах донгла? Каким образом выходные данные связаны со входными? Кто застрелил Дж. Ф. Кеннеди? Что было вначале: яйцо или курица? Надеюсь, вы теперь понимаете почему хак RNBOsproQuery порой представляет собой изрядную сложность даже для опытного реверсера.

CyberHeg в своей статье предлагает очень интересный, но довольно сложный, подход к решению этой проблемы. Правда, в случае с WIscGen.exe обращения к RNBOsproQuery осуществляются в стиле

```
if(!isQueryOK()) goto fig_vam;
```

т.е. пропатчить этот код не просто, а очень просто! Причём RNBOsproQuery вызывается всего лишь 4 раза и обработка результата почти одинакова во всех четырёх случаях. Итак, грузим WIscGen.exe в IDA, применяем соответствующие сигнатуры и задаём поиск вызовов RNBOsproQuery. Вот вам и первый результат:

```
00423611 loc_423611:
00423611             xor    ebx, ebx
00423613             push   ebx
00423614             call   _time
00423619             add    esp, 4
```

```
0042361C            push   eax
0042361D            call   _srand
00423622            add    esp, 4
00423625            call   _rand     ; Генерируем случайное число (X)
0042362A            cdq
0042362B            mov    ecx, 0Ah
00423630            idiv   ecx       ; dx = X%10 (остаток от деления)
00423632            movzx  eax, dx
00423635            imul   eax, 9
00423638            mov    [var_A], dx   ; Сохраняем случайное число
                    . . .
00423659            lea    ecx, [var_18]
0042365C            lea    eax, [var_94]
00423662            lea    edx, [var_D0]
00423668            push   4                    ; length
0042366A            push   ecx                  ; *response32
0042366B            push   eax                  ; *response
0042366C            lea    ecx, [var_4D4]
00423672            push   edx                  ; *queryData
00423673            push   0Ch                  ; address
00423675            push   ecx                  ; packet
00423676            call   _RNBOsproQuery@24
0042367B            lea    ecx, [var_58]
0042367E            lea    edx, [var_94]
00423684            push   ecx
00423685            push   4
00423687            push   edx
                    ; Конвертируем response32 в строку
00423688            call   sub_42394F
0042368D            add    esp, 0Ch
00423690            lea    ecx, [var_58]
                    ; Извлекаем наше случайное число
00423693            movzx  eax, [var_A]
00423697            imul   eax, 9
0042369A            push   4
0042369C            add    eax, OFFSET a3a70b51a
004236A1            push   eax
004236A2            push   ecx
004236A3            call   _strncmp              ; Оно или не оно?
004236A8            add    esp, 0Ch
004236AB            mov    edi, eax
004236AD            test   edi, edi
004236AF            jz     short loc_4236C4      ; Оно!!!
```

Что делает этот код? Ладно, объясняю:

1. генерируем случайное число стандартным сишным методом;
2. колдуем над этим числом и переводим его в массив;
3. подсовываем этот массив в RNBOsproQuery для обработки алгоритмом 0Ch (адрес начальной ячейки алгоритма);
4. результат обработки конвертируем в ASCII-строку;
5. используем случайное число из пункта 1 для адресации фиксированного в экзешнике массива со строковыми константами;
6. сравниваем обе строки;
7. если строки совпадают, продолжаем (последний переход JZ).

Тот фиксированный массив в программе является своеобразным рангом алгоритма 0Ch, т.е. он хранит все возможные результаты данного алгоритма. Я рекомендую вам пройтись по данному участку кода под отладчиком и убедиться в тривиальности всей проблемы. Что нам мешает исправить тот последний переход? Ну, вообще говоря, правильнее будет поксорить регистр EAX на месте вызова _strncmp, как будто строки равны. Заодно можно убрать вызов RNBOsproQuery чтоб лишний раз не беспокоить драйвер. Тривиальный патч:

```
00423668            push   4
0042366A            push   ecx
0042366B            push   eax
```

```
0042366C             lea    ecx, [var_4D4]
00423672             push   edx
00423673             push   0Ch
00423675             push   ecx
00423676 83C418      add    esp, 18h    ; Восстанавливаем стек
00423679 90          nop                ; вместо вызова RNBOsproQuery
00423679 90          nop
0042367B             lea    ecx, [var_58]
0042367E             lea    edx, [var_94]
00423684             push   ecx
00423685             push   4
00423687             push   edx
00423688             call   sub_42394F
0042368D             add    esp, 0Ch
00423690             lea    ecx, [var_58]
00423693             movzx  eax, [var_A]
00423697             imul   eax, 9
0042369A             push   4
0042369C             add    eax, OFFSET a3a70b51a
004236A1             push   eax
004236A2             push   ecx
004236A3 33C0        xor    eax, eax     ; обнуляем EAX
004236A5 90          nop                 ; вместо вызова strncmp
004236A6 90          nop
004236A7 90          nop
004236A8             add    esp, 0Ch
004236AB             mov    edi, eax
004236AD             test   edi, edi
004236AF             jz     short loc_4236C4
```

Остальные 3 вызова RNBOsproQuery фиксятся аналогичным образом. Справедливости ради стоит заметить что подобных глюков (иначе и не назовёшь явное сравнение через _strncmp) вы больше не увидите. RNBOsproQuery кроет в себе громадный потенциал, который по достоинству ценят лишь немногие пользователи SLM, хакеры и мы с вами. Чего только нельзя сделать используя ячейки с алгоритмами, содержание которых недоступно реверсеру?! Даже идея со случайным числом, хеш-алгоритмами и массивом строк можно было бы назвать удачной, если б не то [нецензурный эпитет] сравнение сразу после RNBOsproQuery.

Ну вот и всё: WlscGen.exe к употреблению готов! Попробуйте создать пробную лицензию (задайте параметры по вашему усмотрению) и убедитесь в работоспособности генератора. Получилось? Тогда достаньте ваш любимый патчегенератор (Patch Creation Wizard, например) и сделайте универсальный патч для WlscGen.exe (не фиксить же его вручную при каждой смене VId?!). Не распространяйте ваш патч потом в сети, ладно?

Давайте подытожим наши знания. Для этого рассмотрим два типичных случая использования SLM.

1. SentinelSuperPro API

> Пример: WlscGen.exe.
> Документация: "SentinelSuperPro 6.0 Developer's Guide".
> Приложение использует функции работы с донглом: RNBOsproRead, RNBOsproWrite, RNBOsproQuery и т.д. Задача исследователя заключается в обнаружении и последующем обезвреживании данных функций, как мы и поступили с WlscGen. Данный случай можно назвать низкоуровневой защитой (приложение явно опирается на архитектуру донгла).

2. SLM Client API и иже с ним

> Пример: мой таргет и все-все-все.
> Документация: "SentinelLM Programmer's Reference Manual", "SentinelLM Developer's Guide".
> Приложение использует высокоуровневые функции, вроде LSRequest. Всю грязную работу (чтение лицензионного файла, общение с донглом и т.д.) выполняет API. Обратите внимание на то, что API может и не обращаться к донглу если лицензионный файл того не

требует. Типичный подход к решению проблемы требует получения VId жертвы и некоторой информации о самой лицензии (тип лицензии, версия лицензируемой фичи, наименование фичи и т.д.) Полученная информация передаётся в WlscGen и создаётся новая лицензия (без триального срока... без донгла... :-) Стоп! А откуда взять эту дополнительную информацию о самой лицензии? В самом простом случае её можно выудить из оригинального лицензионного файла. Имя по умолчанию для лиц. файлов -- lservrc (без расширения). Вот, например, демо-версия моей жертвы досталась мне с файлом следующего содержания (некоторая информация удалена по понятным причинам):

```
# XXX for: Quantum Sent with Order: YYY
N2V3R7MXTCCQRM2VQGDSJLXEDJSR88N33SM4QZ# "BPR" version "0"
1Z26N2ZXJ5ANS95HMPGOEYX46HPSTB9Y7V274Z# "DRC" version "0"
WA18DMVBR8NGDOXW9ZAUDLBAPXHNKORQPUWWET2FBA# "BPR" version "91"
5QM2OCB28W63R3XYIF473Z2PFY73XOTZ5O872JH5CY# "DRC" version "91"
```

Итого: 4 лицензии. Вся необходимая WlscGen'у информация любезно предоставлена в каждой строчке после '#'. А что если файла нет? Или он есть но без полной информации? Тогда чуть сложнее... Придётся задействовать IDA + SoftIce и перехватить вызовы LSRequest. Вот вам типичный пример использования LSRequest (фрагмент взят из первой попавшейся под руку демки):

```
00463804            push   eax
00463805            mov    ecx, esi
00463807            mov    BYTE PTR [3BCh+var_4], 0Bh
0046380F            call   sub_461810
00463814            mov    ebp, [eax]
00463816            lea    ecx, [3B8h+var_330]
0046381D            push   OFFSET unk_4E10B0
00463822            push   ecx
00463823            mov    BYTE PTR [3C0h+var_4], 0Ch
0046382B            call   sub_464090
00463830            mov    eax, [eax]
00463832            push   ebx                  ; *handle
00463833            push   0                    ; challenge
00463835            lea    edx, [3C8h+var_39C]
00463839            push   0                    ; log comment
0046383B            push   edx                  ; tokens
0046383C            push   edi                  ; feature version
0046383D            push   ebp                  ; feature name
0046383E            push   eax                  ; publisher
0046383F            push   0                    ; licsystem
00463841            call   _LSRequest
00463846            add    esp, 28h
```

Параметры LSRequest напоминают искомую нами информацию... Да, так оно и есть:

LSRequest(licsystem,publisher,feature name,feature version, tokens,log comment,challenge,*handle)

В вышеописанном фрагменте не фигурируют прямые указатели на строковые константы, передаваемые в LSRequest. Проще всего будет перехватить эти параметры в SoftIce. Ловим LSRequest примерно также как и computeVendorCode в первой главе. Потом извлекаем интересующие нас параметры из стека и радуемся жизни.

Теперь вы скажете: "А какого @#% мы столько возились с WlscGen, ведь можно было просто найти и пофиксить LSRequest в коде нашей жертвы?!". Хмм... Я предпочитаю оставить этот вопрос вам на засыпку. Помедитируйте над ним :-)

Существуют и другие вариации SLM, как-то: сетевой сервер SLM, новомодный SentinelLM Shell (тоже входит в SDK). Но это уже темы для отдельных статей. Короче, бросайте всё и бегите добивать ваш таргет!

Неужели это всё?!

Наверное я не затронул и десятой доли того, что входит в теорию взлома SentinelLM. Вы дочитываете последнюю главу вступления в SLM! Если вы смогли удачно воплотить в код то, что я тут описал, плюс поняли суть самой защиты и её снятия, значит я не зря потратил время. Как говаривал Фокс Мольдер, "Я хочу верить" :-)

Для дальнейшего изучения рекомендую кучу / уйму туториалов по взлому конкретных таргетов под защитой Sentinel. Особенно хороши статьи CyberHeg, CrackZ и Goatass, которые можно найти... в сети. Нет, всё это добро конечно лежит на сайте CrackZ, но сам сайт CrackZ постоянно кочует. Тут (http://66.98.132.48/krobar/collections/crackz.zip) можно качнуть оффлайновую версию всего сайта. На Krobars (http://krobars.reverse-engineering.info/) можно найти многие статьи по SLM и донглам вообще. Ещё эти статьи прозеркалены на других сайтах, даже в переводе. На испанском, например, это добро можно скачать с сайта Karpoff (http://hackindex.com/~karpoff/Manuales.htm#Dongles).

Теперь о главном...

Премного благодарствую всем писателям статей по Sentinel (too many и всех не перечислить), а также коллективу WASM.RU, включая уважаемых членов форума. Отдельной благодарности заслуживают volodya (щедрый поощритель начинающих писателей) и главные редакторы (вы их и сами знаете :-). Большое спасибо и вам, дорогой читатель и коллега. Да пребудет с вами сила Дзена и свобода мысли!

Неявный самоконтроль как средство создания не ломаемых защит

Автор: Крис Касперски

Дата: 2003-10-17

Введение

Основная ошибка подавляющего большинства разработчиков защитных механизмов состоит в том, что они дают явно понять хакеру, что защита еще не взломана. Если защита сообщает "неверный ключевой файл (пароль)", то хакер ищет тот код, который ее выводит и анализирует условия, которые приводят к передаче управления на данную ветку программы. Если защита в случае неудачной аутентификации блокирует некоторые элементы управления и/или пункты меню, – хакер либо снимает такую блокировку в "лоб", либо устанавливает точки останова (в просторечии называемые бряками) на API-функции, посредством которых такое блокирование может быть осуществлено (как правило это EnableWindows), после чего он опять-таки оказывается в непосредственной близости от защитного механизма, который ничего не стоит проанализировать и взломать. Даже если защита не выводит никаких ругательств на экран, а просто тихо "кончает", молчаливо выходя из программы, то хакер либо ставит точку останова на функцию exit, либо тупо трассирует программу и, дождавшись момента передачи управления на exit, анализирует один или несколько последующих условных переходов в цепи управления, – какой-то из них непосредственно связан с защитой!

В некоторых защитных механизмах используется контроль целостности программного кода на предмет выявления его изменений. Теперь, если хакер подправит несколько байтиков в программе, защита немедленно обнаружит это и взбунтуется. Святая простота! – воскликнет хакер, – и отключит самоконтроль защиты, действуя тем же самым способом, что описан выше. По наблюдениям автора, типичный самоконтроль выявляется и нейтрализуется за несколько минут. Наиболее сильный алгоритм защиты: использовать контрольную сумму критических участков защитного механизма для динамической расшифровки некоторых веток программы ломаются уже не за минуты, а за часы (в редчайших случаях – дни). Алгоритм взлома выглядит приблизительно так: а) подсмотрев контрольную сумму в оригинальной программе, хакер переписывает код функции CalculateCRC, заставляя ее всегда возвращать это значение, не выполняя реальной проверки; б) если защита осуществляет подсчет контрольной суммы различных участков программы и/или разработчик использовал запутанный самомодифицирующийся код, трудно предсказуемым способом меняющий свою контрольную сумму, то хакер может изменить защиту так, чтобы она автоматически само, восстанавливалась после того, как все критические участки будут пройдены; в) отследив все вызовы CalculateCRC, хакер может просто снять динамическую шифровку, расшифровав ее вручную, после чего надобность в CalculateCRC пропадает. Стоит отметить, что независимо от способа своей реализации любой самоконтроль элементарно обнаруживается установкой точек останова на те участки защитного механизма, которые были изменены. Остальное – дело техники. Можно сколь угодно усложнять алгоритм подсчета контрольной суммы, – напичкивать его антиатладочными приемами, реализовать его на базе собственных виртуальных машин (то есть интерпретаторов), некоторые из которых, такие, например, как Стрелка Пирса, достаточно трудно проанализировать. Но… если такие меры и остановят взломщика, то ненадолго.

Техника неявного контроля Ошибка традиционного подхода заключается в его предсказуемости. Любая явная проверка чего бы то ни было, независимо от ее алгоритма – это зацепка! Если хакер локализует защитный код, то все – пиши пропало. Единственный надежный способ отвадить его от взлома – "размазать" защитный код по всей программе с таким расчетом, чтобы нейтрализовать

защиту без полного анализа всей программы целиком – было заведомо невозможным. К сожалению, существующие методики "размазывания" либо многократно усложняют реализацию программы, либо крайне неэффективны. Некоторые программисты вставляют в программу большое количество вызовов *одной и той же защитной функции*, идущих из различных мест, наивно полагая тем самым, что хакер будет искать и анализировать их все. Да как бы не так! Хакер ищет ту самую защитную функцию и правит ее. К тому же, зная смещение вызываемой функции, найти, отследить ее вызовы можно без труда! Даже если встраивать защитную функцию непосредственно в место ее вызова, – хакер сможет найти все такие места тупым поиском по сигнатуре. Пускай, оптимизирующие компиляторы, несколько меняют тело inline-функций с учетом контекста конкретного вызова – эти изменения не принципиальны. Реализовать же несколько десятков *различных защитных функций* – слишком накладно, да и фантазии у разработчика не хватит и хакер, обнаружив и проанализировав пару – тройку защитных функций, настолько проникнется "духом" и ходом мысли разработчика, что все остальные найдет без труда.

Между тем существует и другая возможность – неявная проверка целостности своего кода. Рассмотрим следующий алгоритм защиты: пусть у нас имеется зашифрованная (а еще лучше упакованная) программа. Мы, предварительно скопировав ее в стековый буфер, расшифровываем (распаковываем) ее и… используем освободившийся буфер под локальные переменные защищенной программы. С точки зрения хакера, анализирующего дизассемблерный код, равно как и гуляющего по защите отладчиком, все выглядит типично и "законно". Обнаружив защитный механизм (пусть для определенности это будет тривиальная парольная проверка), хакер правит соответствующий условный переход и с удовлетворением убеждается, что защита больше не ругается и программа работает.

Как будто бы работает! – через некоторое время выясняется, что после взлома работа программы стала неустойчивой, – то она неожиданно виснет, то делает из чисел "винегрет", то… Почесав репу, хакер озадаченно думает: а как это вообще ломать? На что ставить точки останова? Ведь не анализировать же весь код целиком! Весь фокус в том, что некоторые из ячеек буфера, ранее занятого зашифрованной (упакованной) программой при передаче их локальным переменным не были проинициализированы! Точнее, они были проинициализированы теми значениями, что находились в соответствующих ячейках оригинальной программы. Как нетрудно догадаться, именно эти ячейки и хранили критичный к изменениям защитный код, а потому и неявно контролируемый нашей программой. Теперь я готов объяснить зачем вся эта котовасия с шифровкой (упаковкой) нам вообще понадобилась: если бы мы просто скопировали часть кода программы в буфер, а затем "наложили" на него наши локальные переменные, то хакер сразу бы заинтересовался происходящим и, бормоча под нос "что.то здесь не так", вышел бы непосредственно на след защиты. Расшифровка нам понадобилась лишь для усыпления бдительности хакера. Вот он видит, что код программы копируется в буфер. Спрашивает себя "а зачем?" и сам же себе отвечает: "для расшифровки!". Затем, дождавшись освобождения буфера с последующим затиранием его содержимого локальными переменными, хакер (даже проницательный!) теряет к этому буферу всякий интерес. Далее – если хакер поставит контрольную точку останова на модифицированный им защитный код, то он вообще не обнаружит к ней обращения, т. к. защита контролирует именно зашифрованный (упакованный) код, содержащийся в нашем буфере. Даже если хакер поставит точку останова на буфер, он быстро выяснит, что: а) ни до, ни в процессе, ни после расшифровки (распаковки) программы содержимое модифицированных им ячеек не контролируется (что подтверждает анализ кода расшифровщика/распаковщика – проверок целостности там действительно нет); б) обращение к точке останова происходит лишь тогда, когда буфер затерт локальными переменными и (по идее!) содержит другие данные.

Правда, ушлый хакер может обратить внимание, что после "затирания" значение этих ячеек осталось неизменным. Совпадение? Проанализировав код, он сможет убедиться, что они вообще

не были инициализированы и тогда защита падет! Однако, мы можем усилить свои позиции: достаточно лишь добиться, чтобы контролируемые байты попали в "дырки", образующиеся при выравнивании структуры (этим мы отвечает хакеру на вопрос: а чего это они не инициализированы?), а затем скопировать эту структуру целиком (вместе с контролируемыми "дырками"!) в десяток-другой буферов, живописно разбросанных по всей программе. Следить за всеми окажется не так-то просто: во-первых, не хватит контрольных точек, а, во-вторых, это просто не придет в голову.

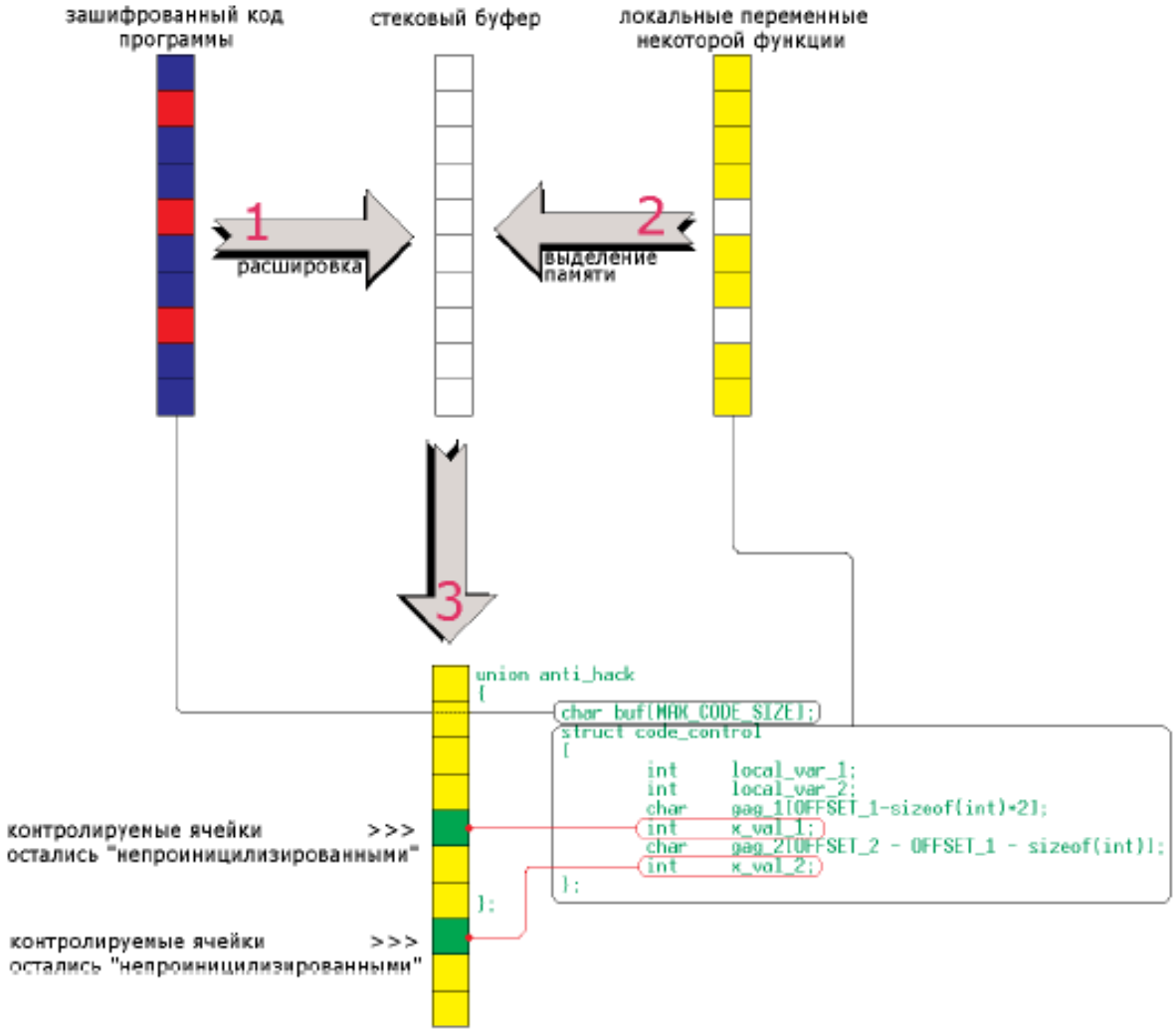


Рисунок 1.

Рисунок 1

Практическая реализация

Правила хорошего тона обязывают нас проектировать защитные меха. низмы так, чтобы они никогда, ни при каких обстоятельствах не могли нанести какой бы то ни было вред легальному пользователю. Даже если вам очень. очень хочется наказать хакера, ломающего вашу программу, форматировать диск в случае обнаружения модификации защитного кода, категорически недопустимо! Во.первых, это просто незаконно и попадает под статью о умышленном создании деструктивных программ, а во.вторых… задумайтесь, что произойдет, если искаже. ние файла произойдет в результате действий вируса или некоторого сбоя? Если вы не хотите, чтобы пострадали невинные, вам придется отказаться от всех форм вре. да, в том числе и преднамеренном нарушении стабильности работы самой защи. щенной программы.

Стоп! Ведь выше мы говорили как раз об обратном. Единственный путь сде. лать защиту трудно ломаемой, – не выдавая никаких ругательных сообщений, по которым нас можно засечь, молчаливо делать "винегрет" из обрабатываемых дан. ных. А теперь, выясняется, что делать этого по этическим (и юридическим!) сооб. ражением нельзя. На самом деле, если хорошо подумать, то все эти ограничения легко обойти. Что нам мешает оснастить защиту явной проверкой целостности своего кода? Хакер найдет и нейтрализует ее без труда, но это и не страшно, по. скольку истинная защита находится совершенно в другом месте, а вся эта бутафо. рия нужна лишь затем, чтобы предотвратить последствия непредумышленного ис. кажения кода программы и поставить пользователя в известность, что все данные нами гарантии (как явные, так и предполагаемые) ввиду нарушения целостности оригинального кода, аннулируются. Правда, при обсуждении защиты данного ти. па, некоторые коллеги мне резонно возразили, а что, если в результате случайного сбоя окажутся изменены и контролируемые ячейки, и сама контрольная сумма? Защита сработает у легального пользователя!!! Ну что мне на это ответить? Случай. но таких "волшебных" искажений просто не бывает, их вероятность настолько близка к нулю, что… К тому же, в случае срабатывания защиты мы ведь не форма. тируем легальному пользователю диск, а просто нарушаем нормальную работу про. граммы. Путь и предумышленно, все равно, если в результате того или иного сбоя был искажен исполняемый файл, то о корректности его работы более говорить не приходится. Ну хорошо, если вы так боитесь сбоев, можно встроить в защиту хоть десяток явных проверок, – трудно нам что ли?! Ладно, оставим этические проблемы на откуп тем самым пользователям, ко. торые приобретают титул "лицензионных" исключительно через крак, и перейдем к чисто конкретным вещам. Простейший пример реализации данной защиты приве. ден в листинге 1. Для упрощения понимания и абстрагирования от всех технических деталей, здесь используется простейшая схема аутентификации, "ломать" которую совершенно необязательно: достаточно лишь подсмотреть оригинальный пароль, хранящийся в защищенном файле прямым текстом. Для демонстрационного приме. ра такой прием с некоторой натяжкой допустим, но в реальной жизни, вам следует быть более изощренными. По крайней мере следует добиться того, чтобы ваша защи. та не ломалась изменением одного единственного байта, поскольку в этом случае да. же неявный контроль будет легко выявить. Следует так же отметить, что контроли. ровать все критические байты защиты – не очень.то хорошая идея, т. к. хакер сможет это легко обнаружить. Если защита требует для своего снятия хотя бы десяти моди. фикаций в различных местах, три из которых контролируются, то с вероятностью ~70% факт контроля не будет обнаружен. Действительно, среднестатистический ха. кер следить за всеми модифицированными им байтами просто не будет. Вместо это. го он, в надежде что тупая защита контролирует целостность своего кода целиком, будет следить за обращениями к одной, ну максимум двум.трем, измененным им ячейкам и… с удивлением обнаружит, что защита их вообще не контролирует. После того, как контрольные точки выбраны, вы должны определить их сме. щение в откомпилированном файле. К сожалению, языки высокого уровня не по. зволяют определять адреса отдельных машинных инструкций и, если только вы не пишите защиту на ассемблерных вставках, то у вас остается один.единственный путь – воспользоваться каким ни будь дизассемблером (например, IDA).

Допустим, критическая часть защиты выглядит так и нам необходимо про. контролировать целостность условного оператора if, выделенного жирным синим шрифтом:

```
int
my_func
()
 {
   if (check_user())
    {
      fprintf(stderr, "passwd ok\n");
    }
   else
    {
      fprintf(stderr, "wrong passwd\n");
```

```
      exit(.1);
     }
    return 0;
}
```

Загрузив откомпилированный файл в дизассемблер, мы получим следую. щий код (чтобы быстро узнать которая из всех процедур и есть my_func, опирайтесь на тот факт, что большинство компиляторов располагает функции в памяти в по. рядке их объявления, т. е. my_func будет вторая по счету функция):

```
.text:00401060 sub_401060 proc near
; CODE XREF: sub_4010A0+AFvp

.text:00401060 call sub_401000
.text:00401065 test eax, eax

.text:00401067 jz short loc_40107E

.text:00401069 push offset aPasswdOk
; "passwd ok\n"

.text:0040106E push offset unk_407110
.text:00401073 call _fprintf
.text:00401078 add esp, 8
.text:0040107B xor eax, eax
.text:0040107D retn
.text:0040107E ; ---------------------------------------------------------------
.text:0040107E
.text:0040107E loc_40107E:
; CODE XREF: sub_401060+7^j

.text:0040107E push offset aWrongPasswd
; "wrong passwd\n"

.text:00401083 push offset unk_407110
.text:00401088 call _fprintf
.text:0040108D push 0FFFFFFFFh
; int

.text:0040108F call _exit
.text:0040108F sub_401060 endp
```

Как нетрудно сообразить, условный переход, расположенный по адресу 0x401067 и есть тот самый "if", который нам нужен. Однако, это не весь if, а только малая чего часть. Хакер может и не трогать условного перехода, а заменить инструкцию test eax, eax на любую другую инструкцию, сбрасывающую флаг нуля. Так же он может модифицировать защитную функцию sub_401000, которая и осуществляет проверку пароля. Словом, тут много разных вариантов и на этом несчастном условном переходе свет клином не сошелся, а потому для надежного распознавания взлома нам потребуются дополнительные проверки. Впрочем, это уже детали. Главное, что мы определили смещение контролируемого байта. Кстати, а почему именно байта?! Ведь мы можем контролировать хоть целое двойное слово, расположенное по данному смещению! Особого смысла в этом нет, просто так проще.

Чтобы не работать с непосредственными смещениями (это неудобно и вообще некрасиво), давайте загоним их в специально на то предназначенную структуру следующего вида:

```
union anti_hack
 {
   // буфер, содержащий оригинальный код программы
   char buf[MAX_CODE_SIZE];
   // локальные переменные программы
   struct local_var
    {
      int local_var_1;
      int local_var_2;
```

```
    };
   // неявно контролируемые переменные программы
   struct code_control
    {
      char gag_1[OFFSET_1];
      int x_val_1;
      char gag_2[OFFSET_2 . OFFSET_1 . sizeof(int)];
      int x_val_2;
    };
  };
```

Массив buf – это тот самый буфер, в который загружается оригинальный код программы для его последующей расшифровки (распаковки). Поверх массива накладываются две структуры: local_val, содержащая в себе локальные переменные, которые в процессе своей инициализации затирают соответствующие им ячейки buf'a и тем самым создают впечатление, что прежнее содержимое буфера стало теперь ненужным и более уже не используется. Количество локальных переменных может быть любым, главное – следить за тем, чтобы они не перекрывали контрольные точки программы, изменять содержимое которых нельзя. В приведенном выше примере, по соображениям наглядности, контрольные точки выне. сены в отдельную структуру code_control, два массива которой gag_1 и gag_2 используются лишь для того, чтобы переменные x_val_1 и x_val_2 были размещены компилятором по необходимым нам адресам. Как нетрудно долгаться: константа OFFSET_1 задает смещение первой контрольной точки, а OFFSET_2 – второй. Достоинство такой схемы заключается в том, что при добавлении или удалении локальных переменных в структуру local_var, структура code_contorl остается неизменной. Напротив, если объединить локальные переменные и контрольные точки одной общей крышей, то размеры массивов gag_1 и gag_2 станут зависеть от количества и размера используемых нами локальных переменных:

```
union anti_hack
 {
   char buf[MAX_CODE_SIZE];
   struct code_control
    {
      int local_var_1;
      int local_var_2;
      char gag_1[OFFSET_1.sizeof(int)*2];
      int x_val_1;
      char gag_2[OFFSET_2 . OFFSET_1 . sizeof(int)];
      int x_val_2;
    };
 };
```

Код, выделенный жирным шрифтом, как раз и отвечает за то, чтобы размер массива.пустышки gag_1, компенсировал пространство, занятое локальными пе. ременными. Такая ручная "синхронизация" крайне ненадежна и служит источни. ком потенциальных ошибок. С другой стороны, теперь мы можем не беспокоить. ся, что локальные переменные случайно затрут контрольные точки, т. к. если такое произойдет, длина массива gag_1 станет отрицательной и компилятор тут же выска. жет нам все, что он о нас думает. Поэтому, окончательный выбор используемой конструкции остается за вами.

Теперь – пару слов о расшифровке (распаковке) нашей программы. Во.пер. вых, нет нужды расшифровывать всю программу целиком, – достаточно расши. фровать лишь сам защитный механизм, а то и его критическую часть. Причем, са. ма процедура расшифровки должна быть написано максимально просто и незамы. словато. Поверьте, лишние уровни защиты здесь совершенно ни к чему. Хакер все равно их вскроет за очень короткое время, и, самое главное, чем круче окажется за. щита, тем внимательнее будет вести себя хакер. Мы же, напротив, должны убедить его, что шифровка это – так, защита от детишек, и "настоящая" защита спрятана где-то совсем в другом месте (пусть ищет то, чего нет!).

Правда, тут есть одна проблема. По умолчанию Windows запрещает модификацию кодовой секции PE.файла и потому непосредственная расшифровка кода невозможна! Первая же попытка записи

ячейки, принадлежащей секции .text, вызовет аварийное завершение программы. Можно, конечно, обхитрить Windows, создав свою собственную секцию, разрешающую операции чтения, исполнения и записи одновременно, или, как еще более изощренный вариант, исполнять расшифрованный код непосредственно в стеке, однако, мы пойдем другим путем и просто отключим защиту кодового сегмента от его непредумышленной модификации. Достоинство этого приема в том, что он очень просто реализуется, а недостаток – ослабление контроля за поведением программы. Если в результате тех или иных ошибок, наша программа пойдет в разнос и начнет затирать свой собственный код, операционная система будет бессильна ее остановить, поскольку мы сами отключили защиту! С другой стороны, в тщательно протестированной программе вероятность возникновения подобной ситуации исчезающе мала и в ряде случаев ею можно смело пренебречь. Во всяком случае, в примере, приведенном ниже, мы поступим именно так (речь ведь все равно идет не о технике расшифровке, а о неявном контроле за модификацией кода).

Остается лишь обмолвится парой слов о способах определения диапазона адресов, принадлежащих защитному коду. Поскольку, большинство компиляторов размешают функции в памяти в порядке их объявления в программе, адрес начала защитного кода совпадает с адресом первой, относящейся к нему функции, а адрес конца равен адресу первой не принадлежащей к нему функции (т. е. первой функ. ции, расположенной на его "хвостом").

Теперь, разобравшись с расшифровкой, переходим к самому интересному – неявному контролю за критическими точками нашего защитного механизма. Пусть у нас имеется контрольная точка x_val_1, содержащая значение x_original_1, тогда для его неявной проверки можно "обвязать" некоторые вычислительные выражения следующим кодом: some_var = some_var + (x_val_1 – x_original_1). Если кон. трольная ячейка x_val_1 действительно содержит свое эталонное значение x_original_1, то разность двух этих чисел равна нулю, а добавление нуля к чему бы то ни было, никак не изменяет его значения. Грубо говоря, x_val_1 уравновешивается противоположным ему по знаку x_origial_1 и за это данный алгоритм называют "алгоритмом коромысла" или "алгоритмом весов". Можно ли быстро обнаружить такие "весы" беглым просмотром листинга программы? Не спешите отвечать "нет", поскольку правильный ответ – "да". Давайте рассуждать не как разработчики защитного механизма, а как хакеры: вот в процессе взлома мы изменили такие-то и такие.то ячейки программы, после чего она отказала в работе. Существует два "тупых" способа контроля своей целостности: контроль по адресам и контроль по содержимому. Для выявления первого хакер просто ищет адрес "хакнутой" им ячейки в коде программы. Если его нет (а в данном случае его и нет!), он предпринимает попытку обнаружить ее содержимое! А вот содержимое контролируемой ячейки в точности равно x_original_1 и тривиальный контекстный поиск за доли секунды выявит все вхождения! Чтобы этого не произошло и наша защита так просто не сдалась, следует либо уменьшить протяженность контролируемых точек до байта (байт – слишком короткая сигнатура для контекстного поиска), либо не хранить x_original_1 в прямом виде, а получать его на основе некоторых математических вычислений. Только не забываете, что оптимизирующие компиляторы все кон. стантные вычисления выполняют еще на стадии компиляции и: (#define x_orginal_1 0xBBBBBA; some_var += (x_val_1 – 1 – x_original_1);) на самом деле не усилит защиту! Поэтому, лучше вообще отказаться от алгоритма "весов", тем более, что он элементарно "вырезается" в случае его обнаружения. Надежнее изначально спроектировать алгоритм программы так, чтобы она осмысленно использовала x_original, а не уравновешивала его "противовесом". Приведенный ниже пример умышленно ослаблен в целях демонстрации как можно использовать эту уязвимость для облегчения взлома.

Исходный текст

```
#include

#define PASSWD "+++"
#define MAX_LEN 1023
#define MAX_CODE_SIZE (0x10*1024)
#define OFFSET_1 0x42
```

```
#define OFFSET_2 0x67
#define x_original_1 0xc01b0574
#define x_original_2 0x44681574
Крис Касперски 10
#define x_original_all 0x13D4C04B
#define x_crypt 0x66

int check_user()
{
  char passwd[MAX_LEN];
  fprintf(stderr,"enter password:");
  fgets(passwd, MAX_LEN, stdin);
  return ~strcmp(passwd, PASSWD);
}

int my_func()
{
  if (check_user())
   {
     fprintf(stderr, "passwd ok\n");
   }
  else
   {
     fprintf(stderr, "wrong passwd\n");
     exit(.1);
   }
  return 0;
}
main()
{
  int a, b = 0;
  #pragma pack(1)
  union anti_hack
   {
     char buf[MAX_CODE_SIZE];
     struct code_control
      {
        int local_var_1;
        int local_var_2;
        char gag_1[OFFSET_1.sizeof(int)*2];
        int x_val_1;
        char gag_2[OFFSET_2 . OFFSET_1 . sizeof(int)];
        int x_val_2;
      };
   };
union anti_hack ZZZ;

// TITLE

  fprintf(stderr, "crackeme.0xh by Kris Kaspersky\n");

// расшифровка кода
  // ===========================================================================

  // копируем расшифровываемый код в буфер

  memcpy(&ZZZ,  _user, (int) &main . (int)  _user);

// расшифровываем в буфере

  for (a = 0; a < (int) &main . (int)  _user; a++)
   {
     (*(char *) ((int) &ZZZ + a)) ^= x_crypt;
   }

// копируем обратно

  memcpy( _user, &ZZZ, (int) &main . (int)  _user);

// явная проверка изменения кода
  // ===========================================================================
```

```
  for (a = 0; a < (int) &main . (int)  _user; a++)
    {
      b += *(int *) ((int)  _user + a);
    }
  if (b != x_original_all)
    {
      fprintf(stderr, ".ERR: invalid CRC (%x) hello, hacker\n", b);
      return 0;
    }

// явная проверка "валидности" пользователя
  // ============================================================================

  my_func();

// нормальное выполнение программы
  // ============================================================================

  // скрытый контроль
  ZZZ.local_var_1 = 2;
  ZZZ.local_var_2 = 2;x_original_2;
  sprintf(ZZZ.gag_1, "%d * %d = %d\n", ZZZ.local_var_1,
          ZZZ.local_var_2,
          ZZZ.local_var_1*ZZZ.local_var_2+((x_original_1^ZZZ.x_val_1)
    +(x_original_2^ZZZ.x_val_2)));
  printf("DEBUG: %x %x\n", ZZZ.x_val_1, ZZZ.x_val_2);
  fprintf(stderr, "%s",ZZZ.gag_1);
}
```

Как это ломают?

Если все сделано правильно, то полученный исполняемый файл не рушиться при его запуске, а победоносно выводит на экран: "crackeme.0xh by Kris Kaspersky\nenter password:" и ждет ввода пароля. Договоримся не обращать внимание на пароль, прямым текстом хранящийся в программе и попробуем взломать защиту другим, более универсальным путем, а именно: изучением алгоритма ее работы под дизассемблером. Запускаем нашу любимую ИДУ и, дождавшись окончания процесса дизассемблирования, смотрим что у нас там. Ага, текстовые строки "passwd ok" и "wrong passwd" в сегменте данных действительно есть, но вот перекрестных ссылок, ведущих к коду, выводящему их, что-то не видно. Странно, ну да лиха беда начало! Запускам любой отладчик (например, WDB) и устанавливаем на адрес строки "wrong passwd" точку останова: "BA r4 407054". Даем команду "GO" для продолжения выполнения программы, вводим любой, пришедший нам на ум пароль, и… отладчик тут же всплывает, показывая адрес машинной команды, обращающейся к первому символу строки. Но что нам это дает? Ведь мы, судя по всему, находится в теле библиотечной функции out, осуществляющей вывод на консоль и в ее коде для нас нет ничего интересного. С другой стороны – эту функцию кто-то вызывает! Кто именно? Ну мало ли! Функция printf к примеру, код которой для нас ничуть не более интересен… Конечно, поднимаясь по цепочке вызовов вверх (окно call stack вам в помощь!), мы рано или поздно достигнем защитного кода, вызвавшего эту функцию, но вот как нам быстро определить где защитный код, а где библиотечные функции? Да очень просто! Та функция, один из аргументов которой представляет собой непосредственное смещение нашей строки, очевидно, и есть функция защитного кода! Последовательно щелкая мышкой по адресам возврата, перечисленных в окне "call stack", мы наконец находим:

```
0040106E
 6854704000   push
407054h

00401073 6810714000   push 407110h
00401078 E88A010000   call 00401207
0040107D 6AFF         push 0FFh
```

Смещение, выделенное жирным шрифтом, – есть ни что иное, как смещение искомой строки, соответственно, адрес 0x40106E (так же выделенный жирным шрифтом) лежит где-то в гуще

защитного кода. А ну-ка, глянем сюда дизассемблером – чего это вдруг ИДА не создала перекрестную ссылку к строке?

```
.text:00401000 dword_401000     dd 62668AE7h, 31306666h, 2616560Eh, 17760E66h, 968E6626h
.text:00401000
; DATA XREF: sub_401090+23vo

.text:00401000
; sub_401090+28vo ...

.text:00401000                  dd 0E666667h, 662616B6h, 724222EBh, 6665990Eh, 0E38E3666h
.text:00401000                  dd 0E5666667h, 26D972A2h, 0EB662616h, 0DF6E4212h, 66666663h
.text:00401000                  dd 0C095B455h, 6939A4EDh, 0E738A6F2h, 666266A2h, 0F6F6A566h
.text:00401050                  dword_401050 dd 9999CD8Eh, 12A6E399h, 162E0E73h, 760E6626h,
    8E662617h
.text:00401050
; CODE XREF: sub_401090+AFvp

.text:00401050                  dd 666667F9h, 556EA2E5h, 320EA5A6h, 0E662616h, 66261776h
.text:00401050                  dd 6667EC8Eh, 8E990C66h, 666664FDh, 556AA2E5h, 0F6F6A5A6h
.text:00401050                  dd 0F6F6F6F6h
```

Вот это номер! Она вообще не посчитала это за код, объявив его массивом! Хорошо, заставим ее дизассемблировать этот фрагмент вручную. Переместив курсор к самому началу массива, нажимаем <U> для его удаления, а затем <C> для превращения байтовой цепочки в код.

```
text:00401000
; sub_401090+28vo ...

text:00401000               out 8Ah, eax
; DMA page register 74LS612:

text:00401000
; Channel 7 (address bits 17.23)

text:00401002               bound sp, [esi+66h]
text:00401006               xor [ecx], dh
text:00401008
text:00401008 loc_401008:
; CODE XREF: .text:0040102Dvj

text:00401008               push cs
text:00401009               push esi
text:0040100A               push ss
text:0040100B               db 26h, 66h
text:0040100B               push cs
text:0040100E               jbe short loc_401027
text:00401010               db 66h
text:00401010               mov ss, es:[esi+0E666667h]
text:00401018               mov dh, 16h
text:0040101A               db 26h, 66h
text:0040101A               jmp short small near ptr unk_401040
```

Хм! Что за ерунда у нас получается?! Вновь переключившись на отладчик, мы убеждаемся, что тот же самый код в нем выглядит вполне нормально:

```
00401000 81EC00040000    sub esp,400h
00401006 56              push esi
00401007 57              push edi
00401008 6830704000      push 407030h
0040100D 6810714000      push 407110h
00401012 E8F0010000      call 00401207
```

Такое впечатление, что защитный механизм зашифрован… А почему бы в самом деле и нет? Возвращаясь к дизассемблеру, щелкаем по перекрестной ссылке и видим:

```
.text:004010AE              mov eax, offset sub_401090
.text:004010AE
; загружаем в регистр EAX непосредственное смещение процедуры sub_401090,
.text:004010AE ; чем и выдаем наш бесхитростный расшифровщик с головой если бы целевой
```

```
.text:004010AE ; адрес вычислялся бы на основе некоторых математических операций, то выявить
.text:004010AE ; расшифровщик было бы сложнее (но по аппаратным контрольным точкам –
.text:004010AE ; все равно возможно)

.text:004010B3             mov esi, offset loc_401000
.text:004010B3
; загружаем в регистр esi непосредственное смещение процедуры loc_401000
.text:004010B3 ;

.text:004010B8             sub eax, offset loc_401000
.text:004010B8
; вычисляем длину зашифрованного фрагмента
.text:004010B8 ;

.text:004010BD             lea edi, [esp+14h]
.text:004010BD
; устанавливаем EDI на локальный буфер esp+14h
.text:004010BD ;

.text:004010C1             mov ecx, eax
.text:004010C3             add esp, 8
.text:004010C6             mov edx, ecx
.text:004010C8             shr ecx, 2
.text:004010CB             repe movsd
.text:004010CD             mov ecx, edx
.text:004010CF             and ecx, 3
.text:004010D2             repe movsb
.text:004010D2
; копируем фрагмент [0x40100 – 0x401090) в локальный буфер
.text:004010D2 ;

.text:004010D4             xor ecx, ecx
.text:004010D6             test eax, eax
.text:004010D8             jle short loc_4010EA
.text:004010DA
; есть что расшифровывать? ;.)
.text:004010DA ;

.text:004010DA loc_4010DA:
; CODE XREF: sub_401090+58vj

.text:004010DA do{
.text:004010DA             mov dl, [esp+ecx+0Ch]
.text:004010DE             xor dl, 66h
.text:004010E1             mov [esp+ecx+0Ch], dl
.text:004010E1
; производим над каждым байтом зашифрованного кода операцию XOR 0x66

.text:004010E1
;

.text:004010E5             inc ecx
.text:004010E5
; берем следующий байт
.text:004010E5 ;

.text:004010E6             cmp ecx, eax
.text:004010E8             jl short loc_4010DA
.text:004010E8
; } while (ecx < eax)

.text:004010EA
.text:004010EA             loc_4010EA: ; CODE XREF: sub_401090+48^j
.text:004010EA             mov ecx, eax
.text:004010EC             lea esi, [esp+0Ch]
.text:004010F0             mov edx, ecx
.text:004010F2             mov edi, offset loc_401000
.text:004010F7             shr ecx, 2
.text:004010FA             repe movsd
.text:004010FC             mov ecx, edx
.text:004010FE             and ecx, 3
```

```
.text:00401101             repe movsb
.text:00401101
; записываем расшифрованные данные обратно постой, как записываем обратно?!
.text:00401101 ; ведь модификация секции .text обычно запрещена?! но ведь "обычно" еще
.text:00401101 ; не "всегда", верно? смотрим атрибуты секции: Flags E0000020: Text
.text:00401101 ; Executable Readable Writable. ага! защита от записи была вручную отключена
.text:00401101 ; разработчиком! поэтому перезапись расшифрованного фрагмента
.text:00401101 ; происходит без ошибок и препирательств со стороны Windows
```

Теперь, когда алгоритм расшифровки установлен (см. выделенную жирным шрифтом строку), мы можем самостоятельно расшифровать его. Для этого нажимаем <F2> в окне IDA и вводим следующий скрипт:

```
auto a;
for (a=0x401000; a < 0x401090; a++)
 {
   PatchByte(a, Byte(a) ^ 0x66);
 }
```

Нажав <Ctrl.Enter> для его выполнения, мы становится свидетелями успешной расшифровки кода защитного механизма. Теперь с ним можно беспрепятственно работать без всяких преград. Кстати, посмотрим, создала ли IDA перекрестные ссылки к строкам "passwd ok" и "wrong passwd"…

```
.text:00401050             sub_401050 proc near
; CODE XREF: sub_401090+AFvp

.text:00401050             call sub_401000
.text:00401055             test eax, eax
.text:00401057
jz short loc_40106E

.text:00401059             push offset aPasswdOk
; "passwd ok\n"

.text:0040105E             push offset unk_407110
.text:00401063             call _fprintf
.text:00401068             add esp, 8
.text:0040106B             xor eax, eax
.text:0040106D             retn
.text:0040106E ; ---------------------------------------------------------------
.text:0040106E loc_40106E:
; CODE XREF: sub_401050+7^j

.text:0040106E             push offset aWrongPasswd
; "wrong passwd\n"

.text:00401073             push offset unk_407110
.text:00401078             call _fprintf
.text:0040107D             push 0FFFFFFFFh
; int

.text:0040107F             call _exit
.text:0040107F             sub_401050 endp
.text:0040107F
```

Держи нас за хвост! Перекрестные ссылки действительно созданы и ведут к приведенному выше коду, который слишком прост, чтобы его комментировать. Смотрите: подпрограмма loc_40106E, выводящая надпись "wrong passwd" на экран и прерывающая выполнение программы вызовом функции _exit, имеет перекрестную ссылку, sub_401050+7, ведущую к условному переходу jz short loc_401064 (в листинге он выделен жирным шрифтом), который, судя по всему, и есть тот самый условный переход, что нам нужен! Забив его машинными командами NOP, мы, очевидно, добьемся того, что защита перестанет "ругаться" на неверные пароли и любой введенный пароль воспримет как правильный.

Ну что, запустим HIEW и запишем по адресу .401057 последовательность "90h 90h"? Не спешите, не все так просто! Ведь исходная программа зашифрована и записанные нами команды NOP после

расшифровки превратятся неизвестно во что. Какой из этого выход? Да очень простой: записав последовательность 90h 90h в HIEW'е мы тем же самым HIEW'ом ее и зашифруем! ОК, приступаем. Итак, <Enter> для перевода HIEW'а в hex.режим, <F5> и ".401057" для перехода по требуемому адресу, <F3> для входа в режим редактирования, 90, 90 – забивает условный переход, <Left Arrow> (четыре раза) для перемещения курсора на начало редактируемого фрагмента, <F8>, <"66"> и еще раз <F8> для шифровки. Наконец, <F9> для сохранения внесенных изменений.

Победно запускаем взломанный файл и…

```
crackeme.0xh by Kris Kaspersky
.ERR: invalid CRC (d7988417) hello, hacker
```

…и тут выясняется, что защита отнюдь не так непроходима тупа, как нам это показалось вначале! Судя по надписи она как.то контролирует целостность своего кода и прекращает работу в случае его изменения. Что ж! Открываем очередное пиво и продолжаем взлом. Можно поступить двояко: или поискать перекрестную ссылку на строку ".ERR: invalid CRC" или же установить контрольную точку на модифицированный нами условный переход. Кинем монетку, если выпадет орел – ищем перекрестную ссылку, ну а если монета упадет решкой – используем контрольную точку. Так, а где у нас монетка? Нету монетки?! Ну тогда, как истинные хакеры, мы быстренько пишем собственный генератор случайных чисел и… решка! (Если у вас выпал орел, значит, нам с вами не по пути).

```
> BA r4 0x407054
> G
Hard coded breakpoint hit
```

Отладчик WDB сообщает, что сработала контрольная точка останова. Пропускаем ее, – это защита копирует код программы в локальный буфер для его последующей расшифровки (это следует из того, что мы всплыли на инструкции movs). Следующее всплытие отладчика соответствует обратной операции – копированию уже расшифрованного кода на место постоянного проживания. А вот третье по счету всплытие уже интересно:

```
0401109    BA00104000   mov edx,401000h
0040110E   8B3C0A       mov edi,dword ptr [edx+ecx]
00401113   41           inc ecx
00401114   3BC8         cmp ecx,eax
00401116   7CF1         jl 00401109

00401118   81FB80EC0040 cmp ebx,4000EC80h

0040111E   741F         je 0040113F
```

Тривиальный алгоритм подсчета контрольной суммы буквально сам броса. ется в глаза. "Или автор защиты полный идиот, или же он специально хотел быть обнаруженным" – ворчим мы себе под нос, попутно размышляя, что лучше: скорректировать контрольную сумму или же просто заменить условный переход в строке 0x40111E на безусловный, так чтобы он вообще не контролировал свою целостность? Ладно, будем приучать себя к аккуратности. Подгоняем курсор к строке 0x401118 и даем команду "Run to cursor", не забыв предварительно заблокировать установленную точку останова (иначе отладчик просто зациклиться) и смотрим: какое значение содержит в себе регистр EBX. Как следует из окна "Registers" оно равно 0xd7988417, в то время как оригинальная контрольная сумма защищенного файла была – 0x4000EC80 (см. строку 0x401118 приведенного выше листинга). Запускаем HIEW и переписываем ее по живому, меняя "cmp ebx, 4000EC80h" на "cmp ebx, 0xd7988417". Проверяем! wow! Это работает! Выломанный файл успешно запускается и, молчаливо проглотив любой введенный пароль, смиренно сообщает "passwd ok" и продолжает нормальное выполнение программы. Обмыв это дело на радостях двойным количеством пива, хакер раздает выломанную программу всем, нуждающимся в ней пользователям и…

…в процессе эксплуатации взломанной программы выясняется, что ведет она себя мягко выражаясь не совсем адекватно. В частности, в нашем случае она выводит на экран: "2 * 2 = 34280". Вот это номер! Поскольку доверять такому взлому со всей очевидностью нельзя, лучше

всего не испытывать судьбу, а приобрести легальную копию программы (особенно, если дело касается бухгалтерского ПО, ошибки которого зачастую несопоставимы с его стоимостью). Но все-таки, хотя бы в плане спортивного интереса, – можно ли взломать такую программу или нет? Условимся, что мы не будем анализировать код, вычисляющий дважды два, поскольку в реальном, полновесном приложении очень легко добиться, чтобы ошибка проявлялась не в месте ее возникновения, а в совсем другой ветке программы, делая тем самым обратную трассировку невозможной.

Первое, что попытается сделать любой здравомыслящий хакер – поискать смещение и/или содержимое модифицированных им ячеек, надеясь что они хранятся в программе прямым текстом. Причем, следует помнить о том, что некото. рые защиты контролируют не сам модифицированный байт, а некоторую протяжную область к которой он принадлежит. В частности, если контролируется целост. ность первого байта условного перехода, то разработчик защиты может схитрить, обратившись к двойному слову, расположенному на три байта "выше". Что ж!

```
00401111 03DF           add ebx,edi
```

Сказано – сделано. Ищем… Быстро выясняется, что ничего похожего на смещение модифицированного нами перехода в защищенной программе нет, но вот его оригинальное содержимое на наше удивление все.таки обнаруживается:

```
.text:00401090 arg_3F   = dword ptr 43h
.text:00401090
arg_53
   = dword ptr
57h

.text:00401144          mov ecx, [esp+0Ch+
arg_53
]
.text:00401148          mov edx, [esp+0Ch+arg_3F]
.text:0040114C          xor ecx, 48681574h
.text:00401152          xor edx, 5EC0940Fh
.text:00401158          mov eax, 2
```

Мало того! Рядом с ним валяется указатель 0x57, который "волшебным" образом совпадает с относительным смещением модифицированного нами байта, отсчитываемого от начала тела первой зашифрованной процедуры (развитие зрительной памяти невероятно ускоряет взлом программ). Так вот ты какой, северный олень! Буквально за одну-две секунды мы вышли на след защитного кода, который по замыслу автора мы ни за что не должны были обнаружить! А обнаружили мы его только "благодаря" тому обстоятельству, что и смещение, и содержимое контрольной точки хранилось в программе в открытом виде. Вот если бы оно вычислялось на лету на основе запутанных математических операций… впрочем, не будет повторяться, мы об этом уже говорили.

Хорошо, условимся считать, что поиск по содержимому не дал результатов и хакер остался с защитой один на один. Что он еще может предпринять? А вот что – аппаратная точка останова на модифицированный байт! Да, конечно, мы уже устанавливали ее, но ранее слишком быстро отсекали "лишние" срабатывания. Теперь же настало время заняться этим вопросом вплотную. Вновь запустив порядком затосковавший за это время WDB, мы даем ему уже знакомую команду "ba r4 0x401057" (не обязательно набивать ее на клавиатуре, достаточно лишь нажать стрелку вверх и отладчик сам извлечет ее из истории команд). Первое срабатывание приходится на следующий код:

```
04010C8           C1E902 shr ecx,2
004010CB          F3A5 rep movs dword ptr [edi],dword ptr [esi]
004010CD          8BCA mov ecx,edx
```

Узнаете? Ну да, были мы здесь недавно и все тщательно проанализировали, так и не обнаружив ничего интересного. Идем дальше? Стоп! А точку останова на буфер.приемник кто будет ставить?

ОК, отдаем отладчику следующую команду: "ba r4 (edi.4)". Почему (edi.4)? Так ведь точки останова срабатывают после выполнения соответствующей им команды, т. е. на момент всплытия отладчика, регистр EDI указывает на следующее двойное слово, а совсем не на то, которые содержит только что скопированный в буфер код.

Очередное всплытие отладчика приводит нас к коду расшифровщика, уже знакомому нам и не содержащему абсолютно ничего интересного. Не тратя на него понапрасну свое драгоценное время мы отдаем команду "G" и… через серию последовательных всплытий отладчика, отождествляем расшифровку защитного кода, его обратное копирование, явную проверку контрольной суммы и, наконец, сталкивается с малопонятным на первый взгляд кодом, про который можно сказать лишь одно: он использует значение тех самых ячеек защитного кода, которые мы варварски "модернизировали":

```
0040113F   E80CFFFFFF        call 00401050

00401148   8B54244F          mov edx,dword ptr [esp+4Fh]

0040114C   81F174156848      xor ecx,48681574h
00401152   81F20F94C05E      xor edx,5EC0940Fh
00401158   B802000000        mov eax,2
0040115D   8D4C1104          lea ecx,[ecx+edx+4]
00401161   8D54240C          lea edx,[esp+0Ch]
```

Конечно, в данном демонстрационном примере алгоритм "балансировки" распознается без особого труда и серьезных умственных усилий, но как бы там ни было, аппаратные точки останова позволили выявить тот самый код, что осуществляет неявный контроль целостности защиты. Кстати, аппаратных контрольных точек всего четыре, а количество буферов, в которые можно запихать "клоны" копий оригинального кода программы – не ограничено много. Словом, если чуть-чуть постараться, можно очень сильно умерять хакерский пыл – за всеми буферами так просто не уследить, придется анализировать огромное количество кода, лишь часть из которого непосредственно относится к защитному механизму, а все остальное – мусор. Чтобы еще больше запутать хакера можно осуществлять неявный контроль целостности не при каждом запуске программы, а, скажем, на осно. ве датчика случайных чисел, – один раз, эдак, из десяти. "Плавающая" защита – что может быть хуже?! Да, теоретически можно и ее сломать, но: во-первых, даже трудно себе представить сколько на нее придется угробить времени, а, во-вторых, никто не даст и кончика хвоста на отсечение, что выявлены и нейтрализованы все уровни защиты. Ведь аппаратные точки срабатывают лишь в момент обращения к ним, а дизассемблирование бессильно выявить адреса, получаемые на основе сложных математических манипуляций с указателями.

Но все-таки давайте доломаем нашу защиту. В данном конкретном случае мы можем нейтрализовать защитный механизм, просто заменив команду xor ecx, 48681574h на xor ecx, 48689090h, т. е. просто скорректировав "балансир". Однако, при взломе реальной программы хакер должен убедиться, что корректируемый им балансир и не балансирует что-то еще.

Написание Plugin модулей для IDA. Основы.

Автор: dragon

Дата: 2003-11-05

На данный момент IDA является пожалуй самым мощным дизассемблером, обладающим огромным количеством возможностей. Но всё это сводилось бы на нет, если бы ни ещё одна - возможность расширения стандартных функций посредством дополнительных модулей, в простонародии - плагинов. Рассмотрим теперь подробнее как же эти плагины писать.

Ну во первых для всего этого вам потребуется сам дизассемблер, который IDA называется, а также пакет SDK для вашей версии. Главная часть SDK - это директория include со включаемыми файлами, в которых содержаться все константы и прототипы функций, которые используются при написании плагинов, а также библиотека импорта - ida.lib. Мы здесь будем рассматривать компиляторы, которые совместимы с VC++, а именно Intel C++. Выбрал я его потому, что в SDK нет его встроенной поддержки(например для компиляторов Borland C++ и самого VC++ в SDK входят файлы инсталляции), а это позволяет лучше понять рассматриваемую тему. Файл ida.lib является библиотекой импорта от ida.wll(расположен в директории с IDA). Все функции оттуда экспортируются по ординалам, а сопоставление имён ординалам производится в ida.lib. Вот почему лучше использовать SDK от той же версии, что и IDA, т.к. ординалы экспортируемых функций могут меняться от версии к версии. Ну ладно, допустим у вас всё есть, приступим.

Что же представлят собой этот самый плагин? А представляет он собой, как ни странно, обычную DLL, у которой экспортируется следующая структура под именем PLUGIN:

```
struct plugin_t
{
 int version;        //Показывает версию IDA, для которой пишеться плагин, обычно это поле равно
    IDP_INTERFACE_VERSION - константе, показывающей, какой версии SDK. Если значение этого поля не
    совпадёт с используемой версией IDA, то плагин загружен не будет.
  int flags        //Проще будет перевести, это просто флаги. Проще всего заполнить это поле нулём,
    но можно сказать, какие здесь есть возможности.
```

PLUGIN_MOD - Установка этого флага значит, что плагин меняет базу с листингом.(Честно говоря мало плагинов, которые её не меняют). Ставиться этот флаг затем, что если в модуле процессора(здесь не рассматривается) запрещается изменять базу, то плагин нельзя будет использовать. В общем в модуле x86 всё разрешается, вообще этот флаг нам не нужен, ровно как и остальные.

PLUGIN_DRAW - Установка этого флага указывает IDA на то, что после вызова плагина надо перерисовать(т.е. обновить) своё окно.

PLUGIN_SEG - Плагин может вызываться, если адрес, на котором стоит курсор, принадлежит какому-нибудь сегменту. Тоже не особо нужен.

PLUGIN_UNL - Если установить этот флаг, то после использования плагин будет выгружаться из памяти. Это конечно хорошо для отладки, но нам не очень подходит.

Так как это флаги, то их можно комбинировать с помощью логического или(or по нашему или | в языке C), а также через простое сложение. Но проще, как я уже говорил, заполнить это поле красивым словом NULL.

```
    void (idaapi* init)(void);        //А это функция инициализации, вызывается при попытке загрузить
      плагин, и если эта функция вернёт что-то плохое, то плагин не загрузиться.
    void (idaapi* term)(void);        //Функция, вызываемая при выгрузке плагина. Обычно эта пустышка,
       используется редко.
    void (idaapi* run)(int arg);        //Наверное самое важное поле - рабочая функция, которая
      запускается при каждом вызове плагина и содержит основной его код.
    char* comment;        //комментарий, используется как подсказка.
```

```
  char* help;        //чтож, help есть help.
  char* wanted_name;        //Предпочитаемое имя плагина.
  char* wanted_hotkey;        //Предпочитаемая горячая клавиша. Вообще, вне зависимости от того,
    какие там клавиши предпочитает автор плагина, истинные значения имени и горячей клавиши
    записываются в файл plugin.cfg, о котором речь пойдёт ниже.
};
```

Вообще то в include файле loader.hpp эта структура объявлена как класс, но для тех, кто не знает, скажу, что в C++ класс и структура в сущности одно и тоже.Ну вроде как настало время и исходник привести. Я привожу здесь простейший плагин, который в базе будет менять следующие бесполезные группы инструкций:

```
pusha
popa

push eax
push edx
rdtsc
pop edx
pop eax

push edx
push eax
rdtsc
pop eax
pop edx
```

на "более понятные" инструкции nop. Вот и исходник:

```
#pragma comment(linker,"/NODEFAULTLIB")
#pragma comment(linker,"/Entry:Dllmain")    //Что-то не очень мне нравиться Runtime-библиотека,
    забирает 40kb, а толку от неё почти никакого, особенно в плагине. А если уж её отключать, то и
    точку входа надо переобозначать.

#define __NT__
#define __IDP__    //А вот это надо прописать ОБЯЗАТЕЛЬНО, а то плагин работать не будет.

typedef unsigned char BYTE;    //Стандартные include-файлы здесь не подключаются, а тип BYTE лишним
    не будет.

#include <ida.hpp>
#include <idp.hpp>
#include <bytes.hpp>
#include <loader.hpp>
#include <kernwin.hpp>    //"Джентльменский набор" подключаемых файлов из IDA SDK

  //Переобозначенная точка входа
bool Dllmain(void* hInstDLL, int reason, int reserved)
{return true;}

char comment[] = "Experimental plugin";
char help[] = "Experimental plugin";
char wanted_name[] = "ExpPlugin";
char wanted_hotkey[] = "F11";

BYTE rdtsc1[]={0x52, 0x0F, 0x31, 0x5A, 0x58};
BYTE rdtsc2[]={0x50, 0x0F, 0x31, 0x58, 0x5A};    //Это просто опкоды инструкций

  //Процедура инициализации плагина
```

В зависимости от возвращаемого значения IDA может загрузить и не загрузить плагин. Вот, что она может возвращать

PLUGIN_SKIP - Плагин не будет загружен.

PLUGIN_OK - Ну раз OK, то есстественно плагин будет загружен.

PLUGIN_KEEP - Плагин не только будет загружен, но и будет постоянно висеть в памяти

К примеру, чтобы плагин работал только с файлами PE формата, надо в процедуру инициализации прописать следующий код:

```
if (inf.filetype != f_PE) return PLUGIN_SKIP;
return PLUGIN_OK;
```

Структура inf здесь также не рассматривается, для изучения подробностей см. SDK. В данном же случае плагин будет загружаться всегда, поэтому код таков:

```
int init(void)
{return PLUGIN_OK;}

    //В unload - процедуре тоже не производиться никаких действий, это пустышка.
void term(void)
{return;}

    //Ну и наконец главная процедура
```

Перед рассмотрением главной процедуры можно для интереса рассмотреть функции для чтения и записи в базу, а также для получения текущего адреса, т.е. адреса, на котором стоит курсор в IDA:

get_byte

get_word

get_long - Чтение байтов из базы. Принимают один параметр - адрес, который отображается в листинге.

patch_byte

patch_word

patch_long - Ну соответственно запись байтов в базу. Параметров два, первый параметр - адрес, второй - значение

Как можно догадаться, функции, у которых на конце написано byte работают(т.е. принимают и возвращают) байт, word - слово, a long - двойное слово.

get_screen_ea - Функция для получения текущего адреса, который она и возвращает.

```
void run(int arg)
{BYTE cb1;
  int i, tmp0;

 ea_t CA = get_screen_ea();

 cb1=get_byte(CA);

    //pusha
    //popa
 if (cb1==0x60 && get_byte(CA+1)==0x61)    //Опкод команды pusha - 60h, а команды popa - 61h
   {patch_word(CA, 0x9090);
     return;}

    //rdtsc
 if (cb1==0x50)    //Первый байт последовательности - опкод команды push eax
   {tmp0=1;
    for (i=0; i<5; i++)    //Остальные байты в массиве, проверяются в цикле
    if (get_byte(CA+i+1)!=rdtsc1[i]) {tmp0=0; break;}


    if (tmp0==1)
      {patch_long(CA, 0x90909090);
        patch_word(CA+4, 0x9090);
        return;}}

 if (cb1==0x52)
   {tmp0=1;
     for (i=0; i<4; i++)
```

```
      if (get_byte(CA+i+1)!=rdtsc2[i]) {tmp0=0; break;}


      if (tmp0==1)
        {patch_long(CA, 0x90909090);
          patch_word(CA+4, 0x9090);
          return;}}
};

    //Теперь помещаем здесь описанную ранее экспортируемую структуру.
    //Прописать extern "C" необходимо, чтобы линкер не извратил имя экспортируемой
    //структуры
extern "C" plugin_t PLUGIN = {
  IDP_INTERFACE_VERSION,
  0,
  init,
  term,
  run,
  comment,
  help,
  wanted_name,
  wanted_hotkey
};
```

Теперь посмотрим, как его надо компилировать. Вот bat файл:

```
icl -Gz /I <Путь к папке с include-файлами из SDK> plugin.cpp ida.lib /link /subsystem:windows /DLL
    /def:plugin.def /out:exp.plw
del "<Путь к папке с IDA>\plugins\exp.plw"
copy exp.plw "<Путь к папке с IDA>\plugins\"
PAUSE
```

Как я уже говорил, пример для Intel C++, если у вас VC++, то исходник менять не надо, а командную строку для компиляции придёться переделать (просто-напросто замените icl на cl). Теперь поподробнее.

-Gz говорит интеловскому компилятору, что передача параметров в функции по умолчанию будет stdcall.

/I - Путь к include директории IDA SDK.

plugin.cpp - исходник с плагином.

ida.lib - библиотека импорта из SDK, должна быть в одной папке с исходником.

После параметра /link идут опции линкера, комментировать нет необходимости. Кроме def-файла. Раз уж здесь есть исходник и bat-файл, то уж придёться приводить и def- файл.

```
LIBRARY exp
EXPORTS PLUGIN
```

T.e def файл указывает, что надо экспортировать структуру PLUGIN. В общем теперь всё, плагин готов к использованию, осталось лишь добавить его в IDA. Для начала неплохо бы скопировать его в папку с plugins, которая находится в папке с IDA. Но этого мало, надо ещё его сконфигурировать. Вся конфигурация плагинов храниться в файле plugins.cfg в той же папке plugins. Обычно вначале этого файла идут комментарии, а в конце записи уже установленных плагинов. Чтобы добавить наш exp.plw(плагины для работы под Windows имеют расширение plw), надо добавить в конец файла четыре параметра в следующем порядке:

```
Имя_плагина  Имя_файла  Горячая_клавиша  Аргумент.
```

Аргумент этот передаётся в функцию run, лично мне кажеться, что толку от него никакого, поэтому установим его в 0. Горячую клавишу можно установить F11, вроде как она не используется IDA. Значит добавляемая нами строчка должна выглядеть так:

```
    expplugin    exp    F11    0
```

Точку с запятой впереди ставить не рекомендую, т.к. это комментарий. Ну в общем можно считать, что работа закончена. Осталось только запустить IDA, открыть какой-нибудь файл, где встречаются инструкции, которые наш плагин обрабатывает(например можно самому быстренько на асме написать и скомпилировать), и проверить. Если всё сделано правильно, то при нажатии F11 или вызова плагина через меню, инструкции должны заменяться nop'ами. Разумеется курсор должен стоять на начале этих scrambled инструкций, если их можно так назвать, всё-таки это мусорные инструкции. Вот собственно и всё.

Об упаковщиках в последний раз. Часть вторая

Автор: Volodya / HI-TECH, NEOx / UINC.RU

Дата: 2003-11-06

_“That people seeking education should have the opportunity to find it.”
Nick Parlante
“Binary Trees”_

Рецензент:
Dr.Golova/UINC.RU

Сорецензенты:
Four-F/HI-TECH , **Quantum** , **Sten** , **Fixer**

1. Требования повышаются!

2. Вместо введения
3. Как Windows работает с секциями PE-файла
4. А запакован ли файл?
5. Тонкости PE-формата
6. OEP и иже с ним
7. Дампер процессов
8. Практический пример: UPX
9. Практический пример: Aspack
10. SEH с точки зрения кракера
11. Немного об антиотладке – в преддверии telock
12. Практический пример: teLock
13. Список литературы
14. Выводы...

Требования повышаются!

Итак, мы продолжаем. Подразумевается, что вы ознакомились с первой частью статьи и теперь владеете минимумом теории. Однако мы еще более повышаем требования. Теперь, для успешного и полного понимания **SEH** (Structured Exception Handling) и некоторых приемов, тут продемонстрированных, вам придется иметь базовое представление о **C++**. Мы не будем особо далеко лезть в дебри родовых классов, виртуальных функций и т.п., но самые основы **ООП** , такие как наследование, вам необходимо (по крайней мере, желательно!) знать.

Предполагается, также, что вы имеете джентльменский набор знаний по защищенному режиму - т.е. знаете, что такое **IDT/GDT/PDE/PTE** и прочие страшные аббревиатуры.

Кроме того, вам придется хлебнуть информации о недрах **Windows** для чего ОЧЕНЬ предлагается прочесть какую-нибудь хорошую книжку по системному программированию под Windows – например, *Марка Руссиновича* или *Свена Шрайбера* (список рекомендуемой литературы в конце статьи). Также по ходу дела мы постараемся давать линки на статьи (к сожалению, подавляющая масса литературы подобного характера на английском языке, на русском тоже что-то есть, но не слишком много).

Как вы уже, должно быть, догадались, все статьи этого цикла будут напирать на теорию. Мы предпочтем рассмотреть вопрос "почему", а не вопрос "как". Обязательно учтите, что здесь мы не рассматриваем 9x!

Практические примеры требуют воспроизводимости. Это означает, что вы должны иметь возможность воспроизвести примеры, показанные здесь, т.к. возможны недопонимания и все следует проверять практически. Во всех примерах упаковщиков используется один-единственный файл – **calc.exe** из поставки Windows 2000.

Вместо введения

написано совместно с Four-F

Что представляет собой процесс в **Windows**? Как запускается программа? Какие объекты создает и поддерживает операционная система? Что такое и как происходит диспетчеризация системных вызовов? Все это - достаточно важные вопросы, которые надо четко себе представлять. Всю оставшуюся часть статьи мы будем глубоко погружаться в мир недокументированных функций Windows, ввиду чего вам настоятельно рекомендуется ознакомится с предложенной литературой. Это статья слишком мала, чтобы детально рассматривать все внутренние системные структуры данных, это не было целью, но вот некоторые нюансы, которые помогут вам сориентироваться в этом, ох каком нелегком, деле, мы здесь продемонстрируем.

Известно, что процессоры **Intel** предлагают операционной системе 4 кольца защиты, из которых Windows использует только два - кольцо-0 и кольцо-3. Часть структур, сопровождающих программу, создается операционной системой в режиме ядра (ring-0) и программе ring-3 они недоступны. Другая часть создается в кольце-3 и, теоретически, доступна программе, однако документация отсутствует, примеров кода нет, словом MS постаралась, чтобы мы знали как можно меньше о внутренностях операционной системы.

Все трюки, продемонстрированные ниже, нельзя исполнить без файлов символов.

Символы отладки и эталонный отладчик

Если вам в силу тех или иных причин необходимо исследовать Windows, то сразу же возникает необходимость в настройке среды для работы. Очевидно, одного джентельменского набора из IDA/Soft-Ice/HIEW/IceExt и дампера процессов 3-кольца типа PE Tools тут уже недостаточно. Это шаг выше. Поэтому вам потребуется обзавестись символами отладки, которые можно сгрузить с сайта MS по этому адресу: http://www.microsoft.com/whdc/ddk/debugging/symbolpkg.mspx установить и корректно настроить их. Обязательно учтите – символы должны ТОЧНО соответствовать вашей ОС и установленному SP. Для этого лучше использовать технологии MS. NuMega Symbol Retriever показал себя нестабильно работающей утилитой. Поэтому не поленитесь, если есть такая возможность, сгрузить файлы ручками, прогнать их через NMS-транслятор и проверить командной table, все ли у нас в порядке. Кстати, если по каким-то, одним разработчикам понятным причинам, Soft-Ice ну никак не желает откликаться на bpx, возможно, установка nms-символов на ntoskrnl сможет помочь. IDA, включая версию 4.5, невероятно глючно накладывает информацию из pdb файла на свою базу, поэтому соизвольте сгрузить либо с сайта datarescue, либо с wasm.ru изумительный и очень шустрый плагин - PDB Plus. Показал себя безукоризненно. Было бы неплохо также иметь какую-то утилиту, способную извлекать информацию из PDB-файла. Есть и такие – pdbdump – http://sourceforge.net/projects/pdbdump. Просто удивительно, сколько полезной информации можно извлечь из PDB. Нам с вами еще не раз предстоит в этом убедится на протяжении статьи. Наиболее полезный файл из всех – ntosrknlsym.pdb. Следующая потенциальная проблема – это выбор отладчика, который был бы способен показывать

внутренности ОС с достаточно большой степенью достоверности. Разумеется, нативный отладчик должен лучше понимать "свою" ОС, чем это делают все остальные. Разработчики Soft-Ice не раз подчеркивали, что все структуры реверсированы, а это не всегда самый надежный способ. Поэтому эталонным отладчиком можно смело считать отладчик самой MS – MS kd. MS kd тоже не идеален и грешит сокрытием информации (например, об объектах - проблема была описана Шрайбером, который предложил и решение для некоторых частных случаев), но это лучше, чем подавляющее большинство утилит. Известно, что kd требует установки двух машин, соединенных между собой. Марк Руссинович разработал утилиту LiveKd (доступна на sysinternals.com и wasm.ru), которая позволяет запускать kd на одной и той же машине. Для отладки, разумеется, можно и нужно применять Soft-Ice, однако, когда возникает необходимость подглядеть какую-то структуру или адрес функции - лучше kd нет ничего (кроме, ессно, ручек и дизассемблера). kd доступен по адресу: http://www.microsoft.com/whdc/ddk/debugging/default.mspx Установка не должна вызвать проблем. Поместите LiveKd в ту же директорию, программа все сделает сама.

Очень важными структурами пользовательского режима являются **TIB** , **TEB и** PEB **. Последний мы в этой статье затрагивать не будем, а касательно двух первых - Thread Environment Block и** Thread Information Block **- необходимо развеять кое-какую путаницу. Здесь мы хотели бы вам сказать, что читать НЕ стоит! Не стоит читать статью Питрека - http://www.microsoft.com/msj/archive/S2CE.aspx и главы «Обработка исключений в реальном и защищенном режимах», «Как противостоять трассировке» и «Как противостоять контрольным точкам останова» и «Как обнаружить отладку средствами Windows» из книги Криса Касперски «Фундаментальные основы хакерства». Почему так? По поводу статьи Питрека – уж слишком она устарела. По поводу Касперски – автор не озаботился проверить, а соответствует ли то, что он написал, действительности, дочитайте данную статью до конца и станет ясно почему. Теперь выдержка из статьи Питрека: "The Windows 95 code calls it a Thread Information Block (** TIB **). In Windows NT, it's called the** Thread Environment Block **(** TEB **)." Утверждение неверно. Структура** TIB **существует и в Windows NT+ и называется** NT_TIB **(полностью документирована в winnt.h**), а структура **TEB**(недокументирована) включает в себя структуру **TIB** , т.е. является ее надмножеством. Где-то так (заметьте, полные описания структур мы не приводим, для этого есть исходники к книге Шрайбера на wasm.ru и исходный код ReactOS – ссылки в конце статьи):

```
  typedef struct _NT_TIB {
/*0*/  struct _EXCEPTION_REGISTRATION_RECORD *ExceptionList;
//fs:[0] - рассматривается в главе о SEH
/*остальные поля, за исключением поля *Self,
// здесь не рассматриваются - есть статьи Питрека*/
...
/*0x18*/ struct _NT_TIB *Self; //fs:[18]
/*вот из-за этого поля вся и путаница - это указатель на начало структуры TIB,
// и, одновременно, на начало структуры TEB*/
} NT_TIB;
typedef NT_TIB *PNT_TIB;

typedef struct _TEB {
/*0*/  NT_TIB TIB; //теперь становится очевидным, что TEB вмещает TIB!
/*0x1C*/ PVOID EnvironmentPointer;
/*обратите внимание, смещение этого поля - 1Ch,
// т.к. перед ним идет вся структура TIB*/
...
/*0x2C*/ PPVOID ThreadLocalStorage; //будет рассмотрен подробнее чуть попозже
/*0x30*/ PPEB Peb; /*см. ниже*/
/*0x34*/ DWORD LastErrorValue;
}
```

Ну, поскольку, рисунок всегда нагляднее, вот:

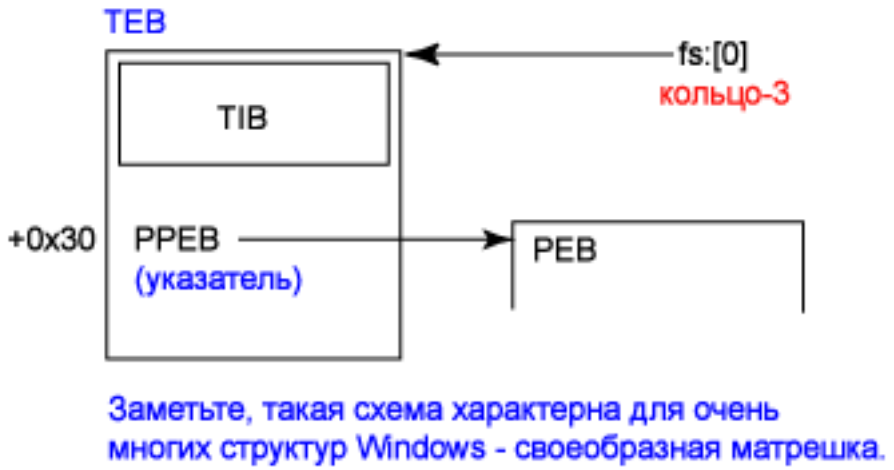


рис 1

А если и рисунка мало, тогда смотрите на дамп из **kd** :

```
kd> !teb
TEB at 7FFDE000
    ExceptionList:    6d474  ;ExceptionList – первое поле TIB
...
    PEB Address:      7ffdf000
;а это указатель на родимый PEB – по адресу видно, что это структура кольца-3
...
```

А теперь глянем в Шрайбера (**w2k_def.h**) касательно **PEB**. Оговоримся сразу – нас не интересует большинство полей, глянем только на два:

```
typedef struct _PEB
        {
...
/*002*/ BOOLEAN              BeingDebugged;
/*используется функций IsDebuggerPresent – см. ниже*/
...
/*00C*/ PPROCESS_MODULE_INFO ProcessModuleInfo; /*а вот это – ошибка!*/
...
/*1E8*/} PEB, * PPEB;
```

Теперь **ReactOS** (**teb.h**):

```
typedef struct _PEB
{
...
   UCHAR BeingDebugged;                            /* 02h */
...
   PPEB_LDR_DATA Ldr;                              /* 0Ch */
...
} PEB;
```

Видите, структуры различаются. Кто же прав? Ответ нам даст **kd** :

```
kd> !peb
PEB at 7FFDF000
    InheritedAddressSpace:    No
    ReadImageFileExecOptions: No
    BeingDebugged:            No
    ImageBaseAddress:         01000000
    Ldr.Initialized: Yes
    Ldr.InInitializationOrderModuleList: 71f38 . 76660
    Ldr.InLoadOrderModuleList: 71ec0 . 76650
    Ldr.InMemoryOrderModuleList: 71ec8 . 76658
```

Но даже более того! Не стоит полностью доверять и **kd**. То, что он показывает – верно на 100%, но есть одно маленькое но – он показывает **НЕ ВСЕ**. Часть структур просто скромно умалчивается.

Однако есть одна вещь, которая никогда не солжет – дамп **pdb** -файла. Мы уже упоминали о pdbdump – давайте им и воспользуемся (не забудьте слить MS DIA SDK с wasm.ru или обзавестись Visual Studio .NET 2002+):

```
struct _PEB_LDR_DATA {
/*некорректно названа у Шрайбера*/
  // non-static data --------------------------------
  /*<thisrel this+0x0>*/ /*|0x4|*/ unsigned long Length;
  /*<thisrel this+0x4>*/ /*|0x1|*/ unsigned char Initialized;
  /*<thisrel this+0x8>*/ /*|0x4|*/ void* SsHandle;
/*смысл полей ниже одинаков – они все показывают на одну и ту же структуру,
 просто упорядочены по-разному*/
  /*<thisrel this+0xc>*/ /*|0x8|*/
struct _LIST_ENTRY InLoadOrderModuleList;
  /*<thisrel this+0x14>*/ /*|0x8|*/
struct _LIST_ENTRY InMemoryOrderModuleList;
  /*<thisrel this+0x1c>*/ /*|0x8|*/
struct _LIST_ENTRY InInitializationOrderModuleList;
};
```

Уходим еще глубже - в двусвязные списки **LIST_ENTRY**. Структура определена в **winnt.h** как

```
typedef struct _LIST_ENTRY {
/*в случае одного-единственного элемента в списке
 Flink/Blink показывают сами на себя*/
   struct _LIST_ENTRY *Flink;  //forward
   struct _LIST_ENTRY *Blink;  //backward
} LIST_ENTRY, *PLIST_ENTRY, *RESTRICTED_POINTER PRLIST_ENTRY;
```

Указатели, в нашем случае, показывают на структуру **LDR_DATA_TABLE_ENTRY** :

```
struct _LDR_DATA_TABLE_ENTRY {
/*на http://undocumented.ntinternals.net/UserMode/Structures/LDR_MODULE.html
 структура ошибочно названа
_LDR_MODULE*/
  /*<thisrel this+0x0>*/ /*|0x8|*/ struct _LIST_ENTRY InLoadOrderLinks;
  /*<thisrel this+0x8>*/ /*|0x8|*/ struct _LIST_ENTRY InMemoryOrderLinks;
  /*<thisrel this+0x10>*/ /*|0x8|*/ struct _LIST_ENTRY InInitializationOrderLinks;
  /*<thisrel this+0x18>*/ /*|0x4|*/ void* DllBase;
  /*<thisrel this+0x1c>*/ /*|0x4|*/ void* EntryPoint;
  //давайте поиграемся с этим полем
  /*<thisrel this+0x20>*/ /*|0x4|*/ unsigned long SizeOfImage;
  /*<thisrel this+0x24>*/ /*|0x8|*/ struct _UNICODE_STRING FullDllName;
  /*<thisrel this+0x2c>*/ /*|0x8|*/ struct _UNICODE_STRING BaseDllName;
  /*<thisrel this+0x34>*/ /*|0x4|*/ unsigned long Flags;
  /*<thisrel this+0x38>*/ /*|0x2|*/ unsigned short LoadCount;
  /*<thisrel this+0x3a>*/ /*|0x2|*/ unsigned short TlsIndex;
  /*<thisrel this+0x3c>*/ /*|0x8|*/ struct _LIST_ENTRY HashLinks;
  /*<thisrel this+0x3c>*/ /*|0x4|*/ void* SectionPointer;
  /*<thisrel this+0x40>*/ /*|0x4|*/ unsigned long CheckSum;
  /*<thisrel this+0x44>*/ /*|0x4|*/ unsigned long TimeDateStamp;
  /*<thisrel this+0x44>*/ /*|0x4|*/ void* LoadedImports;
};
```

Для чего мы заставляем вас проходить через это? Смотрите на код, он теперь должен иметь немного больше смысла:

```
void main(void)
{
 __asm
 {
  mov eax, fs:[30h] ;Teb.Peb
  mov eax, [eax+0Ch]     ;Peb.Ldr - PEB_LDR_DATA
  ;не совсем корректно доступаться по указателю списка
         ;не проверив его сначала,
  ;но для ясности мы проверку опустим
  mov eax, [eax+0Ch] ;Ldr.InLoadOrderModuleList.Flink – сам на себя
  lea ebx, [eax+20h] ;LDR_DATA_TABLE_ENTRY.SizeOfImage
  add [ebx], 88h
  ;LDR_DATA_TABLE_ENTRY.SizeOfImage += 0x88
  ;число выбрано просто так
```

```
 }
}
```

Потрассируйте такой код и посмотрите в **LordPe** , что он вам покажет в поле **SizeOfImage**... Посмотрели? Угадайте, что будет, если дампер будет пробовать читать память, которой НЕТ? А теперь прогоните через **PE Tools** … Ну как результат?

Обратите внимание, что структура **TEB** (и входящая в нее структура **TIB**) создается **Windows** для каждого потока в момент его порождения и "сопровождает" поток до прекращения выполнения. Структура эта в кольце-3 доступна через регистр **fs**. Обратите внимание - мы говорим в кольце-3, т.к. содержимое, доступное через **fs** , РАЗЛИЧАЕТСЯ в ring-3 и ring-0. Почему так – ответ вы найдете в **GDT**. К Soft-Ice прилагается шикарная книженция – “Using Soft-Ice”. Есть там и глава – «Exploring Windows NT” где рассматривается, что куда и как отображается, и приведен примерчик.

В остальной части этой статьи мы будем употреблять термин "**TEB** " только тогда, когда смещение в **fs** -регистре превысит размер структуры **TIB**. Пример из книги Джона Роббинса "Отладка приложений": "*реализация GetCurrentThreadId (из Windows 2000) получает сначала линейный адрес TIB -блока и затем, в позиции со смещением 0x24 (в TIB -блоке) - фактический идентификатор (ID) потока*". Нет такого смещения в **TIB** -структуре! Нет и не было никогда. А вот в **TEB** - есть. Удивительно, как много пользы от таких простых знаний. Положим, вас заинтересовала работа функций **GetCurrentThreadId** , **GetLastError** и **IsDebuggerPresent** :

```
GetCurrentThreadId:
mov     eax, large fs:18h ;NT_TIB.Self – линейный адрес структуры TEB,
                          ; расположенной в ring-3
mov     eax, [eax+24h]  ;TEB.Cid.UniqueThread

GetLastError:
mov     eax, large fs:18h  ;NT_TIB.Self
mov     eax, [eax+34h]   ;TEB.LastErrorValue


IsDebuggerPresent
mov     eax, large fs:18h  ;NT_TIB.Self
mov     eax, [eax+30h]  ;TEB.Peb – извлекается УКАЗАТЕЛЬ на структуру
movzx   eax, byte ptr [eax+2] ;Peb.BeingDebugged
```

Продолжаем. Следующее, что мы рассмотрим здесь - это интерфейс **2Eh** и таблицы системных вызовов. Мы попытаемся отследить путь вызова процедуры вплоть до ядра Windows. За теорией - к Руссиновичу. Не имеете возможность купить эту книгу - вот линк на статью по теме http://www.sysinternals.com/ntw2k/info/ntdll.shtml. Обязательно учтите, что int 2E в XP+ отсутствует! Вместо этого используется команда **sysenter**.

Итак, мы предполагаем - вам известно, что такое **Native API**. Когда, скажем, вызывается функция kernel32.dll CreateFile, что происходит потом? Управление передается в ntdll.dll, где имеем код вида:

```
.text:77F83DA8 _NtCreateFile@44 proc near              ; CODE XREF: .text:77FA0B3Cp
;вот это и есть пример самой настоящей Native API функции
.text:77F83DA8
.text:77F83DA8 arg_0           = byte ptr  4
.text:77F83DA8
.text:77F83DA8                 mov     eax, 20h        ; NtCreateFile
.text:77F83DAD                 lea     edx, [esp+arg_0]
.text:77F83DB1                 int     2Eh
.text:77F83DB3                 retn    2Ch
.text:77F83DB3 _NtCreateFile@44 endp
```

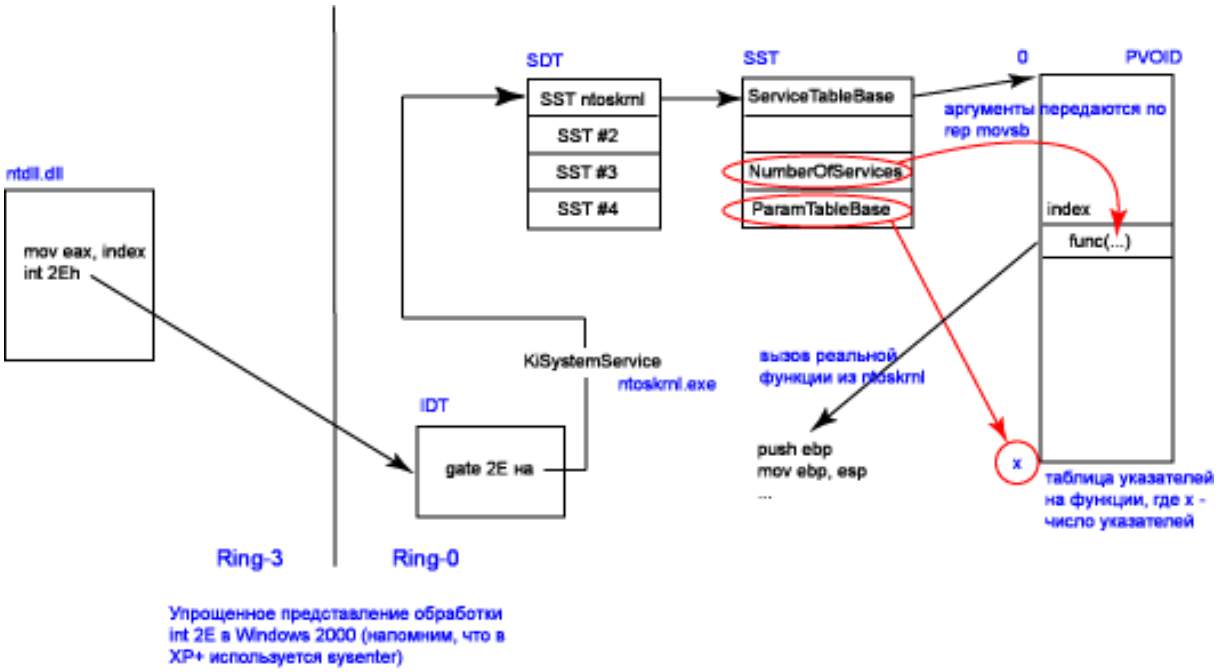
Что происходит потом, когда выполняется **int 2Eh**? Поскольку это прерывание, то оно имеет свой обработчик в **IDT**. Обратите внимание - все функции **ntdll.dll** , обращающиеся к ядру, используют int 2E (в Win 2k, в XP+ используется специальная команда PII+ **sysenter**). Как же обработчик определяет, что делать дальше? Для этого полезем в **Soft-Ice** :

```
:idt 2e
Int   Type     Sel:Offset     Attributes Symbol/Owner
IDTbase=80036400  Limit=07FF
002E  IntG32   0008:804655CD  DPL=3  P   _KiSystemService
```

Т.о. обработчик называется **_KiSystemService** и сидит в **ntoskrnl.exe**. Дальше имело бы смысл привести дизассемблированный листинг этой функции, но все это уже сделано за нас - **Peter Kosyh** в своем замечательном сборничке очень подробно расписал что к чему - сборничек можно слить с wasm.ru. Глава - "Интерфейс системных вызовов". Там предельно подробно рассказывается, как обработчик находит нужную функцию. Единственное что, имеет смысл привести описания структур **SDT** /**SST** , в которых обработчик ее ищет, и рисуночек:

```
typedef struct _SERVICE_DESCRIPTOR_TABLE {
/*SDT доступна через идентификатор ntoskrnl - KeServiceDescriptorTable,
 заметьте, мы не рассматриваем здесь KeServiceDescriptorTableShadow
 - это далеко выходит за рамки статьи
 - подробнее см. великолепную книгу Шрайбера*/
/*0*/  SYSTEM_SERVICE_TABLE ntoskrnl;  //SST для ntoskrnl.exe
/*0x10*/ SYSTEM_SERVICE_TABLE win32k;    //SST для win32k.sys
/*0x20*/ SYSTEM_SERVICE_TABLE iis;
//SST для MS IIS Server (заполнено, ТОЛЬКО если установлен IIS)
/*0x30*/ SYSTEM_SERVICE_TABLE unused;    //не используется
} SERVICE_DESCRIPTOR_TABLE;

typedef struct _SYSTEM_SERVICE_TABLE{
/*в ядре также есть идентификтор KiServiceTable, который является, по сути,
SERVICE_DESCRIPTOR_TABLE.ntoskrnl, остальные здесь не рассматриваются*/
/*0*/ PVOID ServiceTableBase;             //указатель на начало таблицы,
                                          //содержащей адреса функций
/*4*/ PVOID ServiceCounterTable(0);
/*поле содержит количество вызовов той или иной системной функции
 и используется только в т.н. checked build версиях ОС,
 где KiSystemService занимается его заполнением*/
/*8*/ unsigned int NumberOfServices;
/*количество записей в таблице - учтите,
 что индекс функции (eax) НЕ должен превышать это значение*/
/*0xC*/ PVOID ParamTableBase;
/*если вам интересно, как KiSystemService узнает,
 сколько параметров принимает функция на стороне ядра,
 то количество таковых берется как раз отсюда*/
} SYSTEM_SERVICE_TABLE;
```



Упрощенное представление обработки int 2E в Windows 2000 (напомним, что в XP+ используется sysenter)

рис 2

Для чего мы все это вам рассказываем? Положим, вас жутко заинтересовал механизм работы **NtCreateSection** (**ZwCreateSection**). Никаких проблем. Вы идете в **ntdll.dll** и находите ее вызов. Хм... Он скатывается к **int 2Eh**... Что дальше? Ладно, в этом случае все просто - функция экспортируется ядром - **ntoskrnl.exe** , следовательно, ничто не мешает прийти с дизассемблером и туда. А что вы скажете по поводу **NtContinue** (в eax - 1Ch)? Точно также - **int 2Eh**. А вот в таблице экспорта ядра такой функции нет. Тупик? Нет. Используя знания о структуре **SDT** можно легко отследить место расположения **NtContinue** в ядре, а затем найти эту функцию в **ntoskrnl.exe** на диске. Примерно так:

```
:exp KeService
;проверяем наличие такого символа, достаточно частичного имени
ntoskrnl
0008:8046DFA0 KeServiceDescriptorTable ;ага, Soft-Ice знает этот символ
:dd KeServiceDescriptorTable
0023:8046DFA0 804742B8  00000000  000000F8  8047469C      .BG.........FG.
/*теперь вам уже известно строение SDT:
(отображена первая SST, принадлежащая ntoskrnl.exe)
804742B8 - соответствует ServiceTableBase
00000000 - соответствует ServiceCounterTable
000000F8 - соответствует NumberOfServices
8047469C - соответствует ParamTableBase
*/
...
:dd KiServiceTable
/*
ServiceTableBase - это, по сути, массив из указателей на функции - void*
*/
0023:804742B8 8049DD52  804AF6C1  804B043A  8050D5B8      R.I...J.:.K...P.
0023:804742C8 804B0470  8045CEA2  8050F7BE  8050F7FE      p.K...E...P...P.
0023:804742D8 80494A38  8050A9F2  804ADED8  804FD82D      8JI...P...J.-.O.
...
:u *(KiServiceTable+1c*4)          ;ну и где у нас в ServiceTableBase
                                   ;лежит элемент с индексом 1Ch?
_NtContinue
0023:804692A0  55                  PUSH      EBP
;полагаем, пересчитать этот адрес в памяти в реальное смещение
;в файле не составит труда - общая методика
;такова: просто используйте команду mod, вычтите Base address из вашего VA,
;если это необходимо, и можете брать ;дизассемблер и идти в гости
0023:804692A1  8B1D24F1DFFF        MOV       EBX,[P0BootThread]
0023:804692A7  8B553C              MOV       EDX,[EBP+3C]
0023:804692AA  899328010000        MOV       [EBX+00000128],EDX
0023:804692B0  8BEC                MOV       EBP,ESP
0023:804692B2  8B4500              MOV       EAX,[EBP+00]
```

Все, описанное выше, можно сделать еще проще. Разумеется, такой мощный инструмент как Soft-Ice имеет средство для просмотра **SDT** - это команда **ntcall** (обязательно учтите, что **ntcall** покажет только функции, принадлежащие **ntoskrnl**). Единственное что, использование таких команд освобождает от необходимости знать некоторые тонкости работы, что не всегда хорошо, т.к., к примеру, **SST** может быть использована для антиотладочных процедур почти на самом низком из всех возможных уровней (ниже - только драйвер). Пример - статья Тима Роббинса (*Tim Robbins*) - http://www.wiretapped.net/~fyre/sst.html.
Также очень неплохо было бы ознакомится с
http://www.windowsitlibrary.com/Documents/Book.cfm?DocumentID=356 - Undocumented Windows NT - очень сильная книжка, хотя немного и устарела. Возможно, имеет смысл скомпоновать ее главы в виде .chm-формата и поместить на wasm.

Также необходимо предельно кратко ознакомиться со структурами режима ядра. Мы и здесь вполне осознаем, сколь много было сделано и описано Руссиновичем и Шрайбером, поэтому километровые листинги структур здесь даны не будут, мы рассмотрим лишь наиболее общие, а также наиболее ценные, в практическом плане, вопросы.

Мы уже упоминали, что содержимое, доступное через **fs** , различно для **ring-0** и **ring-3**. Как вы теперь понимаете, в ring-3 **fs:[0]** показывает на структуру **TEB**. А что же мы видим, к примеру, в этом случае в кольце-0:

```
mov     eax, large fs:124h
mov     al, [eax+134h]
```

На что показывает **fs:124h** при **DPL** = 0? На что показывает **fs:0** при **DPL** = 0? Все это подробнейшим образом описывает Шрайбер - его книга действительно великолепна. Мы здесь описания структур приводить не будем, достаточно слить некоторые файлы из раздела инструментов на wasm.ru и обзавестить **DDK**. Однако, предельно кратко, в виде рисунка, показать некоторые вещи стоит. Так нагляднее:

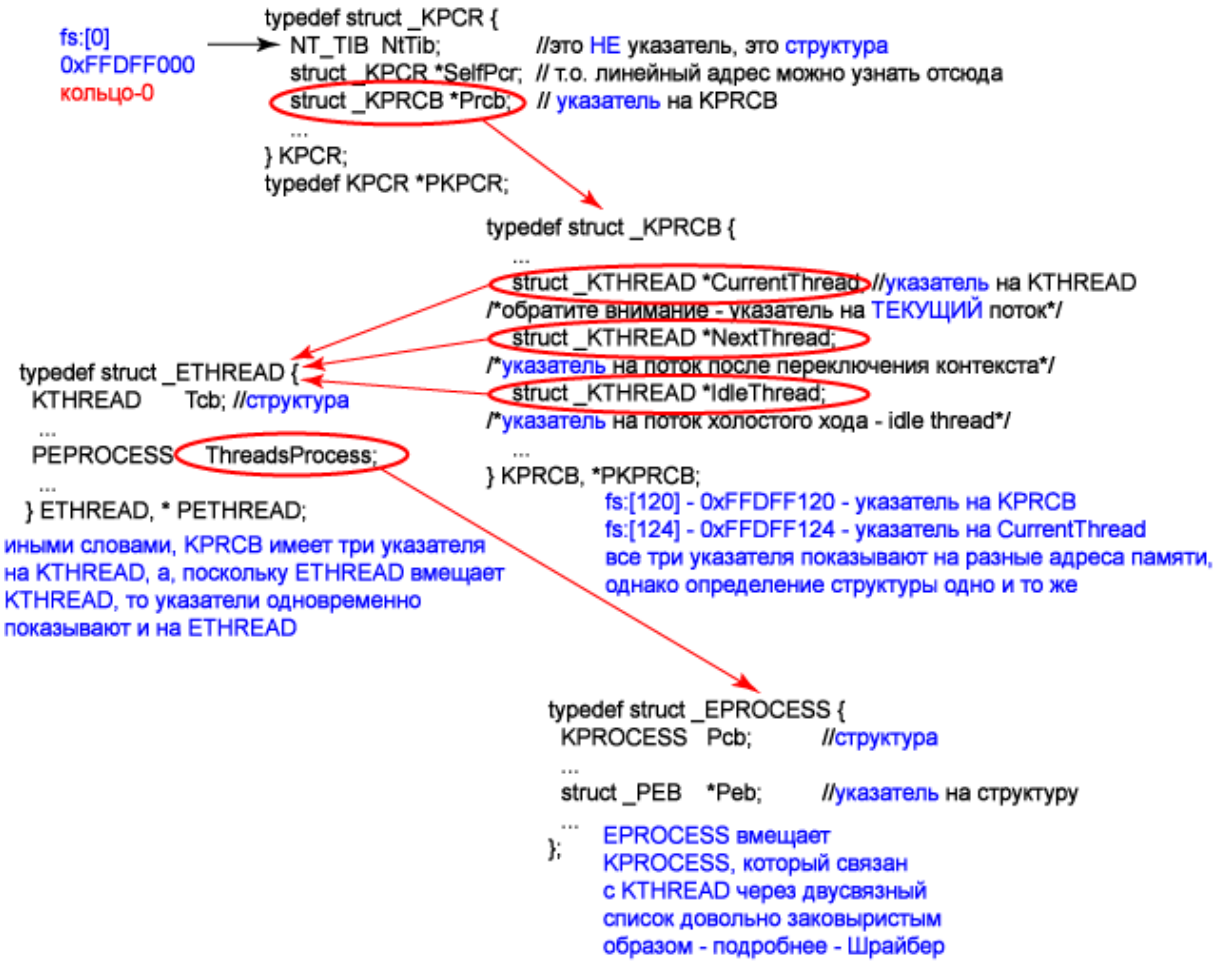


рис 3

Таким образом вы легко сможете преобразовать примеры выше в удобоваримый код, с которым можно сравнительно легко работать дальше:

```
mov     eax, large fs:124h  ;KTHREAD
mov     al, [eax+KTHREAD.PreviousMode]
```

Помните, что Windows отображает адрес **0xFFDFF000** на **fs:[0]** (гляньте в **GDT**!). К примеру, по **fs:[50]** будет лежать **KPCR.DebugActive** , по **fs:[120]** - **KPRCB** (0xFFDFF020), по **fs:[13C]** - структура **CONTEXT**. Единственное, что может смутить - по 0xFFDFF020 лежит указатель на **KPRCB** , значение которого равно 0xFFDFF120:

```
mov     eax, ds:0FFDFF020h ;KPRCB по адресу FFDFF120
inc     dword ptr [eax+4A8h]
;KPRCB.KeExceptionDispatchCount – да,
;увы и ах, документированная в ntddk.h структура KPRCB, конечно, что-то
```

```
;описывает, но опять-таки НЕ ПОЛНОСТЬЮ!
;лезьте в pdbdump и отчет о файле ntoskrnlsym.pdb – там много чего есть!
```

Еще, вероятно, вас может смутить поле **NT_TIB** в составе **KPCR**. Как же так – ведь структура **NT_TIB** принадлежит кольцу-3! А вот и не совсем так. И кольцо-0 и кольцо-3 владеют каждый по **NT_TIB** – по одной на брата. Т.о. и в кольце-0 и в кольце-3 **fs:[0]** показывает на **TIB** , только в первом случае TIB входит в KPCR и ни о каком TEB речи нет, а вот во втором случае **TIB** входит в **TEB**. Вот вам и дамп из kd в доказательство:

```
kd> !pcr
PCR Processor 0 @ffdff000     ;адресочки-то нулевого колечка, однако
        NtTib.ExceptionList: f4347c68
        NtTib.StackBase: f4347df0
        NtTib.StackLimit: f4344000
...
```

Учитывайте эти нюансы и все будет ОК.

Очень многое осталось за бортом этого, предельно короткого, обзора. Однако для успешного понимания оставшейся части статьи этого более чем достаточно, при условии, что все понято. Если нет - Руссинович и Шрайбер. Особенно Шрайбер! Только читая его, обязательно учитывайте ГОД написания книги. Например, цитата: "*Внутреннее строение структур WIN32_PROCESS и WIN32_THREAD - это еще одна пока что непознанная область Windows 2000, исследовать которую только предстоит* ". Уже не совсем так. У Шрайбера код выглядит так:

```
typedef struct _EPROCESS
        {
/*000*/ KPROCESS Pcb;
...
/*214*/ struct _WIN32_PROCESS *Win32Process;
/*как видите, многие поля названы просто по порядку, не более*/
/*218*/ DWORD                   d218;
/*21C*/ DWORD                   d21C;
/*220*/ DWORD                   d220;
/*224*/ DWORD                   d224;
/*228*/ DWORD                   d228;
/*22C*/ DWORD                   d22C;
/*230*/ PVOID                   Wow64;
/*234*/ DWORD                   d234;
/*238*/ IO_COUNTERS             IoCounters;
/*268*/ DWORD                   d268;
/*26C*/ DWORD                   d26C;
/*270*/ DWORD                   d270;
/*274*/ DWORD                   d274;
/*278*/ DWORD                   d278;
/*27C*/ DWORD                   d27C;
/*280*/ DWORD                   d280;
/*284*/ DWORD                   d284;
/*288*/ }
        EPROCESS,
     * PEPROCESS,
    **PPEPROCESS;
```

А теперь посмотрите, СКОЛЬКО информации нам дает **pdbdump** , написанный позже:

```
  /*<thisrel this+0x214>*/ /*|0x4|*/ void* Win32Process;
  /*<thisrel this+0x218>*/ /*|0x4|*/ struct _EJOB* Job;
  /*<thisrel this+0x21c>*/ /*|0x4|*/ unsigned long JobStatus;
  /*<thisrel this+0x220>*/ /*|0x8|*/ struct _LIST_ENTRY JobLinks;
  /*<thisrel this+0x228>*/ /*|0x4|*/ void* LockedPagesList;
  /*<thisrel this+0x22c>*/ /*|0x4|*/ void* SecurityPort;
  /*<thisrel this+0x22c>*/ /*|0x4|*/ struct _UNICODE_STRING* AuditImageName;
  /*<thisrel this+0x230>*/ /*|0x4|*/ struct _WOW64_PROCESS* Wow64Process;
  /*<thisrel this+0x238>*/ /*|0x8|*/ union _LARGE_INTEGER ReadOperationCount;
  /*<thisrel this+0x240>*/ /*|0x8|*/ union _LARGE_INTEGER WriteOperationCount;
  /*<thisrel this+0x248>*/ /*|0x8|*/ union _LARGE_INTEGER OtherOperationCount;
  /*<thisrel this+0x250>*/ /*|0x8|*/ union _LARGE_INTEGER ReadTransferCount;
  /*<thisrel this+0x258>*/ /*|0x8|*/ union _LARGE_INTEGER WriteTransferCount;
```

```
/*<thisrel this+0x260>*/ /*|0x8|*/ union _LARGE_INTEGER OtherTransferCount;
/*<thisrel this+0x268>*/ /*|0x4|*/ unsigned long CommitChargeLimit;
/*<thisrel this+0x26c>*/ /*|0x4|*/ unsigned long CommitChargePeak;
/*<thisrel this+0x270>*/ /*|0x8|*/ struct _LIST_ENTRY ThreadListHead;
/*<thisrel this+0x278>*/ /*|0x4|*/ struct _RTL_BITMAP* VadPhysicalPagesBitMap;
/*<thisrel this+0x27c>*/ /*|0x4|*/ unsigned long VadPhysicalPages;
/*<thisrel this+0x280>*/ /*|0x4|*/ unsigned long AweLock;
```

Мы надеемся, что это достаточно эффектный пример. **Soft-Ice** может использовать свои имена (чего только стоят названия **KTEB** и **UTEB** – кого угодно запутать можно), **kd** может скрыть часть информации (введите команду !processfields и посмотрите как мало она дает), кода **ReactOS** , временами, выдают ТАКОЕ... Мы можем доверять лишь **pdb** -файлу и дизассемблеру. Не верьте именам структур, если они недокументрованы – любой их назовет как угодно, придерживайтесь имен самой MS – **pdb** -файлы не соврут.

Ну, а если вы истинный, то есть, ленивый (это синонимы) программист, то уже должны думать про себя: «Неужели мне, каждый раз, когда я вижу **ebx+134h** , придется каждый раз делать такие комментарии в IDA (да, кстати, недокументированные функции полностью отсутствуют в этом дизассемблере!)?». Ну, разумеется, нет! Все уже сделано за вас. Озаботьтесь загрузить себе **idc** -скрипты, описывающие некоторые структуры нулевого кольца с http://www.alkor.ru/~00077500/kb/winnt.htm или, как всегда, с wasm.ru (в последнем случае картина более полная, т.к. Four-F создал замечательный здоровенный idc-скрипт).

Ничто также не мешает вам самим перевести **.h** файлы в **idc** -скрипты. Частично проблема решена Леонидом Лисовским (*Leonid Lisovsky*) в его скрипте h2enum (слить с сайта **datarescue**), однако более разумным кажется приспособить готовый лексический анализатор для этих целей, к примеру, **lex** , совместить его с перловским скриптом, который будет подставлять нужные функции **IDA** и все. Почитать о **lex** и **yacc** можно, например, тут: http://www.codeproject.com/cpp/introlexyacc.asp. Хм. Добровольцы?

Также обязательно ознакомьтесь с набором команд **IceExt** (скачать с http://stenri.pisem.net либо с wasm.ru) – уникальный плагин для Soft-Ice, способный не только выполнять анти-антиотладку, но, к примеру, могущий также показать список **PTE** , содержание теневой **SDT** и т.п.

Мы довольно прилично осознаем, что от такого введения недолго и в обморок упасть. Поэтому не торопитесь. Вы, должно быть, уже сообразили, что эту статью нельзя прочесть с наскока – это не бульварное чтиво. В помощь при разборе ассемблерщики пусть возьмут себе уникальный **KmdKit** by Four-F и внимательно разберутся с файлом **w2kundoc.inc**. Программисты на C пусть возьмут основательно подправленный **Volodya** файл Шрайбера **w2k_def.h**. Оба доступны с wasm.

Как Windows работает с секциями PE-файла

написано совместно с Four-F

В первой части мы представили и должным образом дополнили работу *Rustell Osterlund* о работе **ntdll.dll** и тех проверках на валидность **PE** -файла, которые она выполняет. Теперь пришло время двинуться дальше и рассмотреть, какие ограничения на формат накладывает само ядро Windows. Это означает, что придется идти глубоко – в **ntoskrnl.exe**. В принципе, новички могут пропустить эту главу, так как она способна запросто привести в ужас кого угодно, кроме самих создателей **Windows**. Единственное что – в самом ее конце мы описываем практическое использование полученных знаний и реализацию оных в **PE Tools**.

Продвинутые читатели наверняка знают некоторые подробности об объектах Windows, поэтому смягчать выражения мы особо не будем. Единственный объект, который нас интересует – это объект «секция». Объект полностью недокументирован. Лишь в главе 7 книги Руссиновича есть

легкое упоминание о такой вещи и симпатичный рисуночек.

Итак, объект "секция" не стоит путать с термином "секция" из PE-файла, это вовсе не одно и тоже. Section object создается в ЕДИНСТВЕННОМ экземпляре на файл. Доказательство: функция NtCreateSection, вызывается ОДИН раз - в коде лоадера (**ntdll.dll**) из **LdrpCreateDllSection** (эта, в свою очередь, из **LdrpMapDll**), и в коде CreateProcess также единожды, в последовательности:

```
...
  NtOpenFile(...);
  ...
  NtCreateSection(...);
  ...
  NtQuerySection(...);
...
```

Подробнее – *Петр Косых* aka *gloomy*. Материалы можно слить с wasm.ru. Кода **ReactOS** , довольно часто упоминаемые нами в этой статье, по данному поводу можно просто выбросить – там чушь. **NtCreateSection** является лишь тонкой прослойкой вокруг **MmCreateSection** , которая и выполняет всю работу по заполнению объекта «секция», работе с **PTE** , а, точнее, с гиперпространством, проверке валидности, переводу флагов PE-секций в атрибуты структур **SUBSECTION** (см. ниже) через хитрые массивы ядра и т.п. Внутри **MmCreateSection** могут вызываться три функции:

```
MiCreateDataFileMap
MiCreatePagingFileMap
MiCreateImageFileMap
```

Сначала вызывается **MiCreatePagingFileMap**. Далее, **Windows** на основании флагов из структуры **CONTROL_AREA** (см. рисунок ниже) решает как ей быть дальше – либо работать с файлом как с данными через **MiCreateDataFileMap** , либо как с исполняемым файлом через **MiCreateImageFileMap** , принимающей указатель на **FILE_OBJECT**. Весь процесс этот достаточно сложный, но, быть может, этот рисунок поможет немного разобраться (маленько улучшенная копия оного из книги Соломона-Руссиновича):



рис 4

Все семейство Mi*-функций, активно используемых в Mm-функциях, невероятно интересно. Однако целиком мы его рассматривать не будем. Внутри **MiCreateImageFileMap** заголовок PE-файла (ТОЛЬКО заголовок) безусловно отображается на гиперпространство по адресу 0x0C050000 (mov eax, 0C0500000h) функцией **MiMapImageHeaderInHyperSpace** (в функцию жестко зашито значение для отображения). После этого отображенный заголовок принимаются активно проверять – функция **MiVerifyImageHeader**. А вот эта функция уже безусловно интерестна для нас с вами:

```
NTSTATUS MiVerifyImageHeader(PIMAGE_NT_HEADERS pPE, ...)
{
 DWORD FileAlign;

 if(pPE->Signature != IMAGE_NT_SIGNATURE)
 {
  if(pPE->Signature != IMAGE_OS2_SIGNATURE)
   return STATUS_INVALID_IMAGE_PROTECT; //0C0000130h
  else
  {
      /*...код для проверки формата NE...
      не рассматривается, здесь используются два других параметра функции
      и функция MiCheckDosCalls*/
   if (NE is invalid)
    return STATUS_INVALID_IMAGE_WIN_16; //0C0000131
  }
 }
 if (!pPE->FileHeader.Machine)
 {
  if(!pPE->FileHeader.SizeOfOptionalHeader)
   return STATUS_INVALID_IMAGE_PROTECT;
 }
   //IMAGE_FILE_EXECUTABLE_IMAGE
 if (!(pPE->FileHeader.Characteristics & 2))
  return STATUS_INVALID_IMAGE_FORMAT; //0C000007Bh
 if(!(pPE & 3))
   //проверка на выравнивание на границу DWORD
  return STATUS_INVALID_IMAGE_FORMAT;
 if (pPE->OptionalHeader.Magic != IMAGE_NT_OPTIONAL_HDR32_MAGIC)
  return STATUS_INVALID_IMAGE_FORMAT;
 FileAlign = pPE->OptionalHeader.FileAlignment;
   //проверка на кратность 512 байтам
```

```
 if (!(FileAlign & 0x1FF))
 {
  if(FileAlign != pPE->OptionalHeader.SectionAlignment)
   return STATUS_INVALID_IMAGE_FORMAT;
 }
 if (!FileAlign)
  return STATUS_INVALID_IMAGE_FORMAT;
   //проверка на степень двойки
 if (!(FileAlign & (FileAlign-1)))
  return STATUS_INVALID_IMAGE_FORMAT;
 if (pPE->OptionalHeader.SectionAlignment < FileAlign)
  return STATUS_INVALID_IMAGE_FORMAT;
 if (pPE->OptionalHeader.SizeOfImage > 0x77000000)
  return STATUS_INVALID_IMAGE_FORMAT;
 return
   (pPE->FileHeader.NumberOfSections > 0x60) ?
   (STATUS_INVALID_IMAGE_FORMAT):(0);
}
```

В принципе, этот псевдокод особых проблем вызвать не должен. Особое внимание обратите на проверку выравнивания файла и секций.

Для вступления этого вполне достаточно. С оставшейся частью, при большом желании, наличии времени и обладании **DDK** и интернетом, расправиться не так уж и сложно. Код, правда, насыщен функциями для работы с **IRQL** и спин-блокировками, однако Соломон и Руссинович достаточно подробно осветили этот вопрос. Также рекомендуем почитать статью **Matt** - http://www.tuningsoft.com/documents/irql.htm - « *Understanding IRQL* ». Заметьте, **MiVerifyImageHeader** не единственное место, где **MiCreateImageFileMap** решает валиден ли образ или нет, однако оставшийся код активно оперирует с гиперпространством, что выводит обсуждение этого вопроса далеко за рамки данной статьи. И соваться туда стоит не раньше, чем прочтете (как следует!) главы Руссиновича о памяти, и всю доступную литературу о **PTE** /**PDE** /**PFN**.

Что до практического применения полученных знаний - **NtCreateSection** будет использоваться в качестве проверки валидности PE-файла в PE Tools (вероятно, с версии 1.6). Опция - "Validate PE". Псевдокод может выглядеть где-то так:

```
/*как вы помните из первой части, ntdll.dll БЕЗУСЛОВНО отображается
на адресное пространство каждого Win32-приложения Windows*/
GetModuleHandle(“ntdll.dll”);
...
/*вызывать только динамически – через GetProcAddress
т.к. нам нужна именно платформенная специфичность,
поэтому никаких статических линковок с ntdll.lib*/
ZwCrSec = GetProcAddress(..., “ZwCreateSection”);
if (STATUS_CODE = ZwCrSec)
 /*значит, ошибка, будем думать, что случилось*/
else
 /*все параметры секций PE-файла валидны,
     содержимое директорий - ?*/
...
```

Если NtCreateSection вернула что-либо отличное от нуля – файл валидным не является – это невероятно надежный источник проверки валидности файла! Если неуверенно себя чувствуете с нативными приложениями (т.е. программами, использующими **ntdll.dll** напрямую с помощью статической линковки, или динамически, через **GetProcAddress**), то вот замечательный линк: http://www.osr.com/ntinsider/1996/native.htm

А запакован ли файл?

написано совместно с Fixer

В виду того, что эта статья позиционируется для взломщика с некоторым опытом, кажется немного наивным поднимать здесь такой вопрос. И в самом деле, для того, чтобы определить, а запакован ли файл, нужен только опыт. Дизассемблируется мусор, редакторы ресурсов отказываются работать и т.д., и т.п. Словом, все симптомы налицо и не о чем тут говорить. Это так. Возражений нет. Однако в рамках этой главы нам бы хотелось показать довольно оригинальный прием программной детекции упакованного файла. Для этого определим понятие "энтропия".

Энтропия

Боже, как только это слово в термодинамике не обругали. И мера порядка системы, и мера рассеивания энергии, уж чего только там не было! Без сомнения, настоящего физика от нашего определения покоробит, а настоящий математик искренне возмутится. Тем не менее, по дилетантски определим слово "энтропия" как некоторую меру эффективности хранения информации. Для иллюстрирования понятия продемонстрируем следующий пример. Условимся считать файл всего лишь строкой байт, конец которой определяется каким-то загадочным образом самой операционной системой. Положим, у нас есть строка: abcdcdaad Подсчитаем количество вхождений каждого байта. Вот так: Таким образом, мы можем сказать, что частота (т.е. вероятность - мы лезем в статистику) появления данного байта в этой конкретной последовательности составляет: где 9 - общая длина последовательности. Определим теперь энтропию каждого символа по формуле: где log2 - логарифм по основанию 2. Таким образом, имеем: Просуммировав сумму всех энтропий, получаем: Теоретически, это означает, что данную строку мы могли бы хранить в компьютерной памяти, используя лишь 17 бит информации. Реально же используется 72 - т.е. символов у нас 9, каждый символ - это байт, а байт - 8 бит. Итого 72 = 8*9. Остается сделать последний штрих - подсчитать остаток от деления общего количества бит на "энтропийные" биты (72/17 = 4,23). Выполнив его, мы увидим, что эффективность хранения информации невысока - фактически, имеем разницу в ЧЕТЫРЕ раза. Отношение к упаковке это имеет самое прямое. Когда секция кода файла сжата, то суммарная энтропия при подсчете возрастет, т.е. приблизится к стандартной сумме всех бит - это будет говорить о повышенной эффективности хранения информации, т.е. возможном сжатии секции кода (остаток от деления уменьшится).

```
a = 3
b = 1
c = 2
d = 3
------
9 total

a = 3/9 = 0.33...
b = 1/9 = 0.11...
c = 2/9 = 0.22...
d = 3/9 = 0.33...,

entropy = |log2(frequency_of_given_byte)|,

a: |log2(3/9)| = 1,584962500721156181453738943947 8
b: |log2(1/9)| = 3,169925001442312362907477887895 6
c: |log2(2/9)| = 2,169925001442312362907477887895 6
d: |log2(3/9)| = 1,584962500721156181453738943947 8,

1,584962500721156181453738943947 8 A
1,584962500721156181453738943947 8 A
1,584962500721156181453738943947 8 A
3,169925001442312362907477887895 6 B
2,169925001442312362907477887895 6 C
2,169925001442312362907477887895 6 C
1,584962500721156181453738943947 8 D
1,584962500721156181453738943947 8 D
1,584962500721156181453738943947 8 D
-----------------------------------------------
17,019550008653874177444867327367 бит информации
```

Алгоритм может выглядеть так:

```
long *ArrFreq;
double *aEntropy;
double msgEntropy=0.0;
ArrFreq=new long[256];
aEntropy=new double[256];
ZeroMemory(ArrFreq,256*sizeof(long));
ZeroMemory(aEntropy,256*sizeof(double));
BYTE *pBuff=(BYTE*)Offset;
long Max=0;
DWORD i = 0;
// подсчитаем каждый байт в сегменте (кода, данных и т.п.)
for (;i<pSegment->Size;i++)
{
 ArrFreq[pBuff[i]]++;
}
BYTE OpCode=0xff;

for (i=0;i<255;i++)
{
 if (ArrFreq[i]>Max)
 {
  Max = ArrFreq[i];
  OpCode=(BYTE)i;
 }
 // хранит вероятность появления символа
 double prob=(double)ArrFreq[i]/(double)pSegment->Size;
 if (prob)
 {
  //подсчитаем энтропию для i-го байта
  aEntropy[i]=(-log(prob)/log(2))*(double)ArrFreq[i];
  // и в общую сумму!
  msgEntropy+=aEntropy[i];
 }
}

// теперь в битах
double DataSize=(double)pSegment->Size*8.0;

// теперь делим, для вычисления остатка
double CompressionRatio=DataSize/msgEntropy;
```

Данный кусочек кода был любезно предоставлен Manuel Jimenez - автором **BDASM** (www.bdasm.com) - очень перспективного и быстрого дизассемблера, из которого в будущем может получится достойный соперник **IDA**! Разумеется, код нельзя просто скомпилировать, однако общий подход он даст.

А особо любознательным расскажем, что идея эта, естественно, отнюдь не нова. Давным-давно криптографы определили понятие "гаммирование" - т.е. наложение какой-либо последовательности байт на текст, чтобы исказить его до неудобочитаемости. Шифр Вернама, перестановки Цезаря - все это призвано было защитить файл от чужих глаз. Однако криптоаналитики придумали подход, который можно назвать "частотным анализом". Т.е., делается предположение о том, что зашифрованный файл содержит в себе осмысленные предложения из такого-то или такого-то языка. И зная вероятности появления символов алфавита в тексте (кстати, как вы думаете, а какая самая часто встречающаяся буква в русском алфавите?), можно попытаться примерно таким же алгоритмом угадать, а что же спрятано за маской? Доказано, что гаммирование принципиально нельзя сломать при условии равенства (и достаточно скрупулезного подбора!) длины последовательности (гаммы), длине шифруемой последовательности. Но даже если это и не так, то, ответьте, что мне мешает заархивировать файл, а уж потом наложить гамму, пусть и неустойчивую? Архивация полностью уничтожит вероятностные распределения букв под маской, делая дешифровку невозможной. Так? А вот и не так! Популярных архиваторов не так уж и много! Опытный криптоаналитик, увидев подобную белиберду, первым делом попробует сжать файл. Как, не сжимается? Ах так! Ну мы тогда...

Впрочем, кажется, мы увлеклись. Итак, определение того, что файл упакован, не займет много времени. Теперь зададим вопрос: "А ЧЕМ упакован файл?". Ответ на подобный вопрос нужен не одним нам с вами. Мировым стандартом (не побоимся этой фразы) считается составление сигнатуры и поиска этой сигнатуры в файле. С точки зрения алгоритмики имеем поиск подстроки в строке.

Приготовление сигнатуры – вещь не сложная и во многом должна определяться квалификацией того человека, который эту сигнатуру составляет. В Pe Tools для этой цели разработана утилита SignMan, следующая версия которой будет основана на очень простом принципе: побайтовом сравнении файлов, запакованных ОДНИМ И ТЕМ ЖЕ упаковщиком с разными опциями:

```
SetFilePointer(на оффсет, введенный пользователем,
             т.к. отсчитывать от точки входа – неверное решение!);

... //отвести буферы и т.д., и т.п.

for(int i = 0; i < до какого-то значения, заданного пользователем; i++)
{

if(byFromFile1[i] == byFromFile2[i])
   {
 /*хорошо – в отчет*/
   }
}
```

Сигнатуры хранятся в текстовой форме в файле **Sign.txt**. Практика показала, что это решение удачно. У многих людей, заинтересованных в судьбе утилиты, нашлось свободное время, чтобы отправить баг-репорт с правками сигнатур и, можно заявить, что файл, на сегодняшний день является достаточно тщательно проверенным не только авторами утилиты, но и многочисленными пользователями. Хотя, разумеется, это не означает, что багов там нет...

Словом, примем сигнатуру для поиска достаточно надежным средством. Осталось определится с тем, как ее искать. Мы трактуем файл как последовательность байт. **PE Sniffer** , по версию 1.5.x. включительно, является пока еще утилитой-ребенком и от него, ни в коем случае, пока нельзя ожидать многого. Поэтому утилита должна быть переписана с учетом быстрого и в достаточной степени надежного поиска. Итак, поиск может быть разделен на две категории:

1) Точный поиск подстроки в строке –

Байт-в-байт в точке входа – тривиальное **strcmp** с минимальными трюками по пропуску плавающих байт. Медленно, наивно, не всегда работает. Скажем, ничто не мешает переписать точку входа PE-файла на свой, достаточно безобидный код, который, скажем, делает не больше, чем **xchg eax,eax** – и этого хватит, чтобы сигнатура не была опознана. Учтите, что автоматический алгоритм детекции секции кода может быть неверным (причины см. в первой части), поэтому надежнее сначала просто спросить у пользователя, какую секцию PE он считает секцией кода. Что мы имеем здесь в более продвинутом плане? Первое - поиск по принципу бинарного дерева – реализован в **IDA** во **FLIRT** -алгоритме. Более подробно об этом можно почитать в статье Ильфака Гильфанова о **FLIRT**. Второе - поиск **Бойера-Мура** в файле. Рекомендуется прочесть некоторые документы с http://algolist.manual.ru/ или ознакомиться со статьей http://www.rsdn.ru/article/alg/textsearch.xml на **RSDN**. Хочется сказать спасибо автору статьи - Андрею Боровскому – за такие объяснения, какими они должны быть. Понять алгоритм можно только посидев с карандашом над ним, разрисовав палочки и черточки. Эта статья относится именно к такому классу. Приведенные в ней алгоритмы в несколько более эффективном варианте, переписанные на C, будут использованы в **PE Sniffer**. В этом случае с файлом работают при помощи **MMF** -функций.

2) Неточный поиск подстроки в строке –

Поиск Бойера-Мура достаточно быстр за счет построения таблицы смещений и делает меньше сравнений, чем тривиальное **strcmp**. Однако он подходит лишь для точного поиска образца (или, в

улучшенных вариантах, допускает лишь минимальные отклонения, согласно простейшим регулярным выражениям). Здесь же мы ни в коей мере не можем быть уверены, что подстрока (сигнатура) не будет самым злостным образом искажена. Давайте рассмотрим пару примеров и на их основе попытаемся сформулировать ряд правил для написания движка.

Пример: **libc.lib** – стандартная библиотека языка C. Известно, что код программы начинается не с main, а с * **mainCRTStartup** (одной из четырех). Поэтому ничто не препятствует поменять код процедуры с такого, например:

```
posvi = (OSVERSIONINFOA *)_alloca(sizeof(OSVERSIONINFOA));
posvi->dwOSVersionInfoSize = sizeof(OSVERSIONINFOA);
(void)GetVersionExA(posvi);
_osplatform = posvi->dwPlatformId;
_winmajor = posvi->dwMajorVersion;
_winminor = posvi->dwMinorVersion;
```

на вот такой:

```
posvi = (OSVERSIONINFOA *)_alloca(sizeof(OSVERSIONINFOA));
goto here;

now_here:
_osplatform = posvi->dwPlatformId;
_winmajor = posvi->dwMajorVersion;
_winminor = posvi->dwMinorVersion;
goto keep_on;

here:
posvi->dwOSVersionInfoSize = sizeof(OSVERSIONINFOA);
(void)GetVersionExA(posvi);
goto now_here;

keep_on:
...
```

Выражено коряво, но смысл вы схватить должны. Это полностью исказит сигнатуру и мы более не сможем сказать, что же это за компилятор. Причем и противопоставить-то такому приему практически нечего... К счастью, такое практически невозможно в общем случае проделать с исполняемым файлом, поэтому рассмотренный код относится, скорее, к патологии.

Реально же имеем следующее:

1) Для каждого пакера существует устойчивая кодовая последовательность именуемая в дальнейшем сигнатурой для которой действуют следующие правила (здесь под элементом сигнатуры подразумевается байт)

>
> a) Элементы сигнатуры не могут меняться местами
> b) Элементы сигнатуры не могут быть заменены на другие

2) Между отдельными элементами может присутствовать "*шумовой* " код (если мы для сигнатуры выберем 1, 5, 10 байты сгенерированные пакером, для уменьшения количества ложных срабатываний, необходимо учесть минимальное расстояние на котором могут встретиться эти элементы сигнатуры друг от друга), а между некоторыми такой код появиться не может (двух и более байтные команды).

Что-то наподобие такого и будет реализовано в новой, уже не совсем детской версии **PE Sniffer**. Хотя и здесь решение для общего случая, скорее всего, НЕ существует. Уж слишком злопакостно можно исказить сигнатуру при желании, и ни CRC, ни хог-сумма строки, ни побайтовое сравнение по хитрым правилам не помогут. Скажем, пункты 1.a/1.b явно дискуссионны, чего только стоят т.н. **stolen bytes** в **Asprotect**. Поэтому если имеете собственное суждение – не стесняйтесь его высказать.

Возможно, самые отчаянные захотят обсудить экзотические методы детекции – применение нейронных сетей, генетические алгоритмы, **fuzzy logic** и т.п. – это прекрасно. Дерзайте! Учтите, что и **OEP** (см. главу об **OEP**) можно искать в памяти по такому же принципу – поиском сигнатур компилятора. Так, к примеру, поступает **PEiD** в своем **genoep.dll** – просто-напросто в дампе программы выполняется поиск компиляторных сигнатур, при этом, кстати, опять таки, не учитывается понятие «украденные байты» - **stolen bytes** – защитный прием некоторых пакеров (**Asprotect**), при котором куски кода из **OEP** нагло утаскиваются.

В заключение было бы интерестно рассмотреть – а как же действуют профессиональные антивирусы? Ведь как-то же они предполагают наличие вируса, хотя, зачастую и неверно, давая ложные срабатывания или просто не узнавая его. Однако мир антивирусов практически закрыт от конечного пользователя. И кто-то еще ругает Windows за закрытость кода?

Как правило, в открытую печать не попадает сколь-нибудь ценной технической информации... Что тут говорить. Тем не менее, мы предложим пару линков:

1. «Heuristic Techniques in AV Solutions: An Overview»
http://www.securityfocus.com/infocus/1542 - в качестве вступления смотрися неплохо.
2. «Stripping Down An Av Engine»
2. http://www.nai.com/common/media/vil/pdf/imuttik_VB_%20conf_2000.pdf – в качестве вступления к следующему линку тоже ничего.
3. http://clamav.elektrapro.com/ - попытка антивируса с открытым исходым кодом (язык C). Функция эвристики (пока только ребенок) находится в файле **matcher.c** и использует обобщение алгоритма Кнута-Морриса-Пратта в поиске в односвязном списке – функция **cl_scanbuff**. Есть и еще несколько подобных антивирусов, но у их основателей хватило ума писать их на **Java** … Что тут сказать... И это там, где важна скорость и низкоуровневые трюки...
4. http://www.peterszor.com/ - на удивление приличные статьи о детекции метаморфов, пара статей о Win32 высокого класса. Это достаточно хороший уровень. Очень рекомендуется!

Тонкости PE-формата

Вы решили потратить деньги на приобретение пакера. ОК, выбор ваш. Положим, вы не уверены, что способны написать хорошую защиту, просто нет времени или еще что-то. Тогда проверьте пакер! Хороший криптор должен не только перезагружать Windows если кто-то подошел к монитору, он должен и корректно обрабатывать многие тонкости и сложности PE-формата.

Одним из достаточно простых, но достаточно забавных тестов на качество написанного пакера может быть следующий, почти гениальный, клочок кода:

```
__declspec(thread ) int i = 1;
__declspec(thread ) int m = 0;

void main(void)
{
/*для нас совершенно неважно, какой именно код тут используется,
он написан просто так, чтобы что-то написать;
что действительно важно, так это объявления переменных.
 Статические объявления заставляют компилятор
создать секцию .tls в результирующем файле*/
printf("%ld\n", i*m);
}
```

Основной смысл такого упражнения – в создании секции **.tls** в результирующем PE-файле (заметьте, мы говорим о статическом **tls** – с **tls** можно работать и динамически – подробнее – Джеффери Рихтер). **.tls** -секция (если она существует) обрабатывается лоадером при загрузке –

вызываются callback-функции и т.п. Все это достаточно подробно описано Питреком в

http://www.microsoft.com/msj/0999/hood/hood0999.aspx

и облегчать жизнь писателям пакеров у нас желания нет. Однако факт остается фактом – многие коммерческие упаковщики не учитывают инициализацию tls-цепочек лоадером, в результате чего запакованный файл падает. Среди таковых и Aspack 2.12, который такой файл даже обработать не может! А люди еще за это и деньги платят...

Более того, можно сделать еще веселее! Положим, мы имеем дело с **dll** (**tls** -цепочки используются, в основном, именно в **dll**) – как прикажете обрабатывать секцию **tls** , которая подвержена перемещению, т.е. появляются **fixup** -элементы? В случае **exe** -файла аналогичного результата можно добиться опцией MS-линкера **/FIXED:NO**. Так это вообще фантастика! **UPX** гарантировано обрабатывает такие вещи, а вот некоторые остальные, не будем показывать пальцами...

Вот такими и является большинство упаковщиков. С миру по нитке накопировано антиотладочных приемов (зачастую, беззастенчиво спертых из кода вирусов или толковых статей кракеров), все это достаточно неумело отлажено, кое-как обрабатывается PE-файл, при этом совершенно не учитываются многие нюансы и тонкости формата и за это просят деньги, утверждая, что пакер работает превосходно и надежно спасает от недалеких кракеров.

OEP и иже с ним

Для этой главы неплохо было бы выбрать самый простой из всех возможных упаковщик, на котором и проиллюстрировать некоторые закономерности работы. Такой, к счастью, есть. Называется **PE Deminisher** и доступен для закачки с wasm.ru.

Итак, пакуем наш старый добрый calc.exe и что мы имеем:

```
   Name   VirtSize   RVA    PhysSize  Offset    Flag
.text    000124EE 00001000 0000782B 00000600 E0000020
.data    000010C0 00014000 000003E1 00008000 C0000040
.rsrc    00002B98 00016000 00002C00 00008400 40000040
.teraphy 00001000 00019000 0000041A 0000B000 C0000040
```

Очевидно, три секции являются нормальными, четвертая принадлежит упаковщику. Так же очевидно, что, в этом случае, остальные секции (за исключением **.rsrc**) являются сжатыми по какому-либо алгоритму (в данном случае это apLib, но нам это не важно). Что находится в четвертой секции и почему она расположена именно четвертой? Вот слегка обрезанный отчет **HIEW** о данном файле:

```
Name        RVA      Size
Import       00019391 00000089  ;импорт перенаправлен
Resource     00016000 00002B98  ;ресурсы оставлены
Debug        00001210 0000001C  ;старый трюк – см. первую часть
Import Table 00001000 0000020C
;недоработка данного упаковщика – IID
;перенаправлена в секцию .teraphy, IAT оставлена, но не валидна
```

Использование директории отладки является старым добрым антиотладочным приемом, мы это уже описывали и повторяться неинтересно. Ресуры тоже уже мало кого удивляют. Любопытнее выглядит изменение **RVA** директории импорта в секцию пакера. Если глянуть на заголовок (**Optional Header**), то можно видеть, что и точка входа переориентирована в новую секцию, и количество секций, соответственно, увеличено. Разумеется, изменилось поле **SizeOfImage** , иначе файл валидным не будет. Кому интересно видеть все мелочи – воспользуйтесь функцией **Compare** из **Pe Tools**. Нам же интерестнее ответить на вопрос: так почему же секция идет четвертой. Если чуточку подумать над проблемкой, то ответ ясен – так легче. Положим, можно

поставить и третьей, если не лень пересчитывать ресурсы как директорию и как секцию. А вот первой – ни-ни. Так как слишком уж это дело будет хлопотное... Но если кому-то не лень, что тогда? Смотрите – теоретически невозможно поставить секцию пакера первой – это потребует коррекции ссылок в секции кода и коррекции ссылок между секциями кода и данных и т.п., да и не только. Откуда получить такую информацию? Если файл содержит **IMAGE_DIRECTORY_ENTRY_BASERELOC** , то тогда, используя информацию оттуда, такое дело возможно, однако методика для общего случая работать не будет.

Что в этом плане можем извлечь мы. Да очень простой, старый и почти безотказно работающий трюк. Только давайте сначала четко определимся с терминами. Итак: **OEP** – **original entry point** – это не та самая точка входа которая записана в заголовке PE файла (**OptionalHeader.AddressOfEntryPoint**). **OEP** - это **VA** , куда упаковщик передаст управление после полной распаковки файла. Т.е. это - оригинальная точка входа которая была в заголовке PE файла до упаковки. А точка входа в запакованном файле называется **EP** (*Entry Point*). Так вот, нетрудно заметить, что прыжок после распаковки всегда будет происходить из области больших адресов в область меньших адресов. На этом механизме и были построены многие OEP-трейсеры – revirgin и icedump в их числе. Очевидно, что если кто-нибудь (гм, например, мы) не поленится написать драйвер, который даже будет не сколько трассировать приложение, сколько просто смотреть за **EIP** , когда тот будет выходить за пределы секции (секций) упаковщика. Положим, протектор сможет делать ложные прыжки, положив оные в конструкцию **try/catch** (см. ниже) – но отчет утилиты покажет это все человеку, а уж человек элементарно разберется – какой прыжок ложный, а какой нет. С другой стороны, пакер вполне может применять засечки количество тактов процессора – rdtsc и, чуть что не так, начинать орать. Словом, тут есть над чем подумать...

Работать с **dll** примерно так же просто. Достаточно давно разработана методика **Break & Enter**. Смысл ее состоит во влеплении опкода **CC** (о самом опкоде см. ниже) прямо в **EP** программы. Известно, что **Soft-Ice Symbol Loader** часто просто проскакивает мимо EP. Поэтому **LordPE** и **PE Tools** лепят CC-байт прямо в **EP** , предварительно запоминая оригинальный. Все, что остается пользователю – ввести bpint 3 и восстановить старый байт после всплытия **Soft-Ice**. Скоро будет написан плагин под **PE Tools** в виде лоадера **dll** , т.к. **dll** , с нашей с вами точки зрения, мало чем отличается от **exe**.

Что до директории импорта – тоже достаточно просто понять, что оригинальная директория импорта остается нетронутой лоадером (он ее просто не видит). Вместо этого пакер сам, после расшифровки содержимого файла, находит эту директорию и в цикле, перед передачей управления на **OEP** , с помощью **GetProcAddress** , наполняет ее валидными для данной системы адресами и производит перерасчет **RVA** на **VA** (см практический пример с **Aspack**). Обязательно следует заметить, что и тут прогресс ушел далеко вперед. Современные крипторы уже не используют **GetProcAddress**. Уж слишком легко нам поставить на нее брейкпоинт и разобраться в логике пакера (см. практические примеры).

bpx на функции API

Команда bpx Soft-Ice использует зарезервированный Intel опкод CC. Интеловские талмуды говорят нам следующее: «The INT 3 instruction generates a special one byte opcode (CC) that is intended for calling the debug exception handler. (This one byte form is valuable because it can be used to replace the first byte of any instruction with a breakpoint, including other one byte instructions, without over-writing other code).». Не путайте также опкод CD 03 с CC – они используются в разных случаях – «Note that the “normal” 2-byte opcode for INT 3 (CD03) does not have these special features. Intel and Microsoft assemblers will not generate the CD03 opcode from any mnemonic, but this opcode can be created by direct numeric code definition or by self-modifying code.». Поставить точку останова на API и получить результат – это прекрасно для нас с вами, но не слишком хорошо для авторов упаковщиков. Вот пример немного наивного кода для детекции bpx в самом начале LoadLibraryA: Код прост, но хорошо отражает суть. Его можно выразить и чуть иначе, например, так: а можно и еще парой

десятков вариаций. Для его обхода опытные люди используют трюк с bpx API-name + x, где x – число, которое приходится на начало другой инструкции. Важно заметить, что + x может быть не абы каким числом. Это обязательно должно быть поле операнда, иначе исключения #DB не произойдет, произойдет другое исключение :) Скажем, часто предлагается ставить нечто вроде bpx GetProcAddress + 3. В этом случае Soft-Ice превратит символьное имя GetProcAddress в VA (гм, а может и не в VA, бог его знает, что там внутри Soft-Ice происходит – ведь большинство не задумывается ПОЧЕМУ команда bpx на какую-нибудь API-функцию типа MessageBox срабатывает в любом адресном пространстве, т.е. контекстно-неспецифична – под Windows для первых двух гигабайт это попахивает черной магией!), добавит к этому адресу 3 и воткнет туда CC. Будьте готовы – авторам упаковщиков элементарно проверить каждую вызываемую функцию на наличие CC, а тогда ... Тогда можно еще использовать bpm или уходить на уровень NativeAPI. К примеру, для GetProcAddress цепочка выглядит так: Последняя основательно прокомментирована в статье о DLL-лоадере, рассмотренной нами в первой части.

```
mov  eax,[KernelBase]
push offset LoadLibraryA
push eax
call GetProcAddress
cmp  byte ptr [eax],0cch
je   Found_Hook

mov edi, offset на собственную IAT – сканируем первый байт всех адресов
mov al, 0CCh
repnz scasb  ;код скорее, схематичен,
             ;чем реален, но в сети есть настоящие примеры

GetProcAddress -> LdrGetProcedureAddress -> LdrpGetProcedureAddress
```

Вместо этого используется более изощренная технология – прямое сканирование директории экспорта целевых dll. Более подробно об этом можно почитать в статье «Win32 Assembly Components» написанной LSD Team. Статью можно скачать с wasm.ru или с сайта команды – http://lsd-pl.net/projects.html#windowsassembly. Что мы можем сказать по этому поводу? М-м-м... Опять таки, если время сканирования экспорта опирается на **rdtsc** – тут тяжелее. В общем случае – почему бы, наконец, не раскачать **Sten** 'а на написание нормальной **bpr** -команды, которую, к сожалению, убрали из **Soft-Ice**... Присутствие такой **bpr** -команды позволит ставить точки останова на большие диапазоны памяти – например, на диапазон директории экспорта, и отслеживать обращающиеся к этому диапазону инструкции. Команда должна быть контекстно-специфичной.

Теперь еще один вопрос. Когда надо дампить программу? Почему в подавляющем большинстве статеек по пакерам сказано, что нужно вводить образ в бесконечный цикл? Давайте проясним ситуацию.

Как ни странно, ответ на вопрос – когда нужно дампить программу, часто заключается в самой запакованной программе. Все зависит от того, насколько разумно она проектировалась программистом. Как известно, язык **C** позволяет создавать статические и глобальные переменные. Оставим спор по поводу необходимости их применения в стороне, а сами зададим вопрос: « *а чем от этого плохо нам?* ». Рассмотрим следующий пример:

```
#include "windows.h"
static void* h_heap = 0; /*вот в этом вся и соль*/
void main(void)
{
  if (!h_heap)
    h_heap=HeapCreate(0,0x1000,0);

  HeapAlloc(h_heap, HEAP_ZERO_MEMORY, 0x500);

  MessageBox(0, "I said NOW!", "Dump me NOW!", MB_OK);

  HeapDestroy(h_heap);
}
```

Программа запускается и работает абсолютно нормально, но стоит сдампить ее как раз на **MessageBox** – последствия не заставят себя ждать. Причина в том, что статические и глобальные переменные инициализируются **КОМПИЛЯТОРОМ**! Следовательно, при нормальном развитии событий переменная имеет свой нолик еще в секции **PE** -файла и проверка проходит нормально. В сдампленной программе переменная уже заполнена функцией HeapCreate и, следовательно, при следующем запуске будет нам с вами радость.

Так что, в общем случае, рекомендация должна звучать примерно так: секции данных лучше дампить сразу после раскриптовки/распаковки, ибо протектор может их подпортить еще до **OEP**. И это подводит нас к очень важному выводу: не надо полагаться на **ImpRec** , **OEP** - **finder** 'ы, **TRW** с его **makepe** или что-либо еще – с каждым пакером пока приходится работать индивидуально, т.к. общее решение проблемы пока не разработано.

Дампер процессов

Основой получения списка процессов является функция **NtQuerySystemInformation**. Читатель может заворчать – вот, опять **Native API**. Да, именно так. Только учитывайте, что бывают РАЗНЫЕ уровени недокументированности! Скажем, весьма и весьма вероятно, что реализация некоторых **Mi** *-функций, рассмотренная нами ранее, может и, скорее всего, будет, варьироваться от ОС к ОС и даже от **SP** к **SP** , в то время как некоторые **Native API** функции весьма стабильны и едва ли изменятся в будущем. Во всяком случае, прототип, название и смысл должны сохранится. А смысл этой функции настолько велик, что Шрайбер назвал ее «**кладезем** » информации о системе. Также Гарри Неббет (*Gary Nebbet*) неплохо осветил эту тему в 1 главе своей книги. Кое-что из этой информации мы повторим здесь. Приступим:

```
NTSYSAPI NTSTATUS NTAPI NtQuerySystemInformation (
    IN SYSTEM_INFORMATION_CLASS SystemInformationClass,
    OUT PVOID SystemInformation,
    IN ULONG SystemInformationLength,
    OUT PULONG ReturnLength OPTIONAL
    );


typedef enum _SYSTEM_INFORMATION_CLASS {
    SystemBasicInformation,
    SystemProcessorInformation,
    SystemTimeOfDayInformation,
    SystemPathInformation,
    SystemProcessInformation,
    SystemCallCountInformation,
    SystemDeviceInformation,
    SystemProcessorPerformanceInformation,
    SystemFlagsInformation,
    SystemCallTimeInformation,
    SystemModuleInformation,
    SystemLocksInformation,
    SystemStackTraceInformation,
    SystemPagedPoolInformation,
    SystemNonPagedPoolInformation,
    SystemHandleInformation,
    SystemObjectInformation,
    SystemPageFileInformation,
    SystemVdmInstemulInformation,
    SystemVdmBopInformation,
    SystemFileCacheInformation,
    SystemPoolTagInformation,
    SystemInterruptInformation,
    SystemDpcBehaviorInformation,
    SystemFullMemoryInformation,
```

```
    SystemLoadGdiDriverInformation,
    SystemUnloadGdiDriverInformation,
    SystemTimeAdjustmentInformation,
    SystemSummaryMemoryInformation,
    SystemNextEventIdInformation,
    SystemEventIdsInformation,
    SystemCrashDumpInformation,
    SystemExceptionInformation,
    SystemCrashDumpStateInformation,
    SystemKernelDebuggerInformation,
    SystemContextSwitchInformation,
    SystemRegistryQuotaInformation,
    SystemExtendServiceTableInformation,
    SystemPrioritySeperation,
    SystemPlugPlayBusInformation,
    SystemDockInformation,
    SystemPowerInformation,
    SystemProcessorSpeedInformation,
    SystemCurrentTimeZoneInformation,
    SystemLookasideInformation
} SYSTEM_INFORMATION_CLASS;
```

Где, **SystemInformationClass** – тип требуемой информации, нас интересует только информация о процессах, т.е. **SystemInformationClass** = **5** , **SystemInformation** – указатель на буфер данных, **SystemInformationLength** – размер буфера данных, **ReturnLength** – размер записанных в буфер данных. Если выделенного буфера недостаточно, то в параметр **ReturnLength** будет возвращён требуемый размер буфера. Рекомендуемый размер = **sizeof**(**SYSTEM_PROCESS_INFORMATION**) * **1024** , так как вряд ли у кого то будет запущенно **1024** процесса.

Далее если не произошло ошибок, в буфер **SystemInformation** будет передана структура **SYSTEM_PROCESS_INFORMATION** , описывающая отдельный процесс в списке. Первым членом структуры (**NextEntryOffset**) будет смещение на следующий процесс в списке, если же оно равно NULL, значит это последний процесс в списке.

```
typedef struct _SYSTEM_PROCESS_INFORMATION {
    ULONG NextEntryOffset;
    ULONG NumberOfThreads;
    LARGE_INTEGER SpareLi1;
    LARGE_INTEGER SpareLi2;
    LARGE_INTEGER SpareLi3;
    LARGE_INTEGER CreateTime;
    LARGE_INTEGER UserTime;
    LARGE_INTEGER KernelTime;
    UNICODE_STRING ImageName;
    KPRIORITY BasePriority;
    HANDLE UniqueProcessId; // PID
    HANDLE InheritedFromUniqueProcessId;
    ULONG HandleCount;
    ULONG SpareUl2;
    ULONG SpareUl3;
    ULONG PeakVirtualSize;
    ULONG VirtualSize;
    ULONG PageFaultCount;
    ULONG PeakWorkingSetSize;
    ULONG WorkingSetSize;
    ULONG QuotaPeakPagedPoolUsage;
    ULONG QuotaPagedPoolUsage;
    ULONG QuotaPeakNonPagedPoolUsage;
    ULONG QuotaNonPagedPoolUsage;
    ULONG PagefileUsage;
    ULONG PeakPagefileUsage;
    ULONG PrivatePageCount;
    SYSTEM_THREAD_INFORMATION TH[1];
} SYSTEM_PROCESS_INFORMATION, *PSYSTEM_PROCESS_INFORMATION;
```

Простой пример использования функции, подразумевается, что буфер выделен, и **NtQuerySystemInformation** вызвалась без ошибок:

```
SYSTEM_PROCESS_INFORMATION *pSysInfo;
...

NtQuerySystemInformation(…);

while(1)
{
 // Здесь уже можно использовать структуру
 ...

 // Проверям – последний это процесс в списке? Если да, то
 // выходим из цикла
if(!pSysInfo->NextEntryOffset)
 break;

// Переходим к следующему процессу
pSysInfo = (SYSTEM_PROCESS_INFORMATION)((PVOID)pSysInfo + pSysInfo->NextEntryOffset);
}
```

Внимание: важный момент, перед вызовом **NtQuerySystemInformation** рекомедуется установить своему приложению привилегию отладки программ. Это нужно для получения списка системных процессов (*System Idle process*). Но это возможно только под правами Администратора. Всё это проделывает функция **EnableDebugPrivilege**. Чтобы установить привилегию, первый и единственный параметр должен быть равен **TRUE** , чтобы убрать **FALSE**.

```
BOOL EnableDebugPrivilege(BOOL bEnable)
{
 HANDLE hToken;
 BOOL   bOk = FALSE;
 if(OpenProcessToken(GetCurrentProcess(), TOKEN_ADJUST_PRIVILEGES, &hToken))
 {
  TOKEN_PRIVILEGES tp;
  tp.PrivilegeCount = 1;
  LookupPrivilegeValue(NULL, SE_DEBUG_NAME, &tp.Privileges[0].Luid);
  tp.Privileges[0].Attributes = bEnable ? SE_PRIVILEGE_ENABLED : 0;
  AdjustTokenPrivileges(hToken, FALSE, &tp, sizeof(tp), NULL, NULL);
  bOk = (GetLastError() == ERROR_SUCCESS);
  CloseHandle(hToken);
 }
 return bOk;
}
```

Получение списка загруженных процессом модулей

Для получения списка модулей служит функция **RtlQueryProcessDebugInformation**.

```
RtlQueryProcessDebugInformation(HANDLE hPID, DWORD dwInfo, PVOID pRtlBuff);
```

где,

• **hPID** – идентификатор процесса (**PID**),
• **dwInfo** – код запроса требуемой информации,
• **pRtlBuff** – буфер куда будет передана информация о модулях процесса. Ниже приведена структура этого буфера:

```
typedef struct _DEBUGMODULEINFO
{
 DWORD ImageBase;
 DWORD ImageSize;
 DWORD unknown1;
 USHORT DllSequenceNum
 USHORT NumDlls;
 DWORD GrantedAccess;
 CHAR Name[MAX_PATH];
 DWORD unknown;
} DEBUGMODULEINFO,PDEBUGMODULEINFO;
```

Для работы с функцией **RtlQueryProcessDebugInformation** необходимо создать (**RtlCreateQueryDebugBuffer**) рабочий буфер для хранения информации о модулях. И после окончания работы, вызвать функцию **RtlDestroyQueryDebugBuffer** , чтобы освободить этот самый буфер.

```
PDWORD RtlCreateQueryDebugBuffer(DWORD, DWORD);
DWORD RtlDestroyQueryDebugBuffer(PDWORD);
```

Пример:

```
QUERYDEBUGBUFFER *pModuleInfo; // Информация о модулях
DWORD dwNtStatus;              // Код возврата функции
                               // RtlQueryProcessDebugInformation
DWORD dwPID;                  // PID процесса
// Выделяем буфер
DWORD *pRtlBuffer = RtlCreateQueryDebugBuffer(NULL, NULL);

if(!pRtlBuffer)
{
 // Error!
}
// Запрашиваем информацию о модулях
dwNtStatus = RtlQueryProcessDebugInformation((HANDLE *)dwPID, 0x01, pRtlBuffer);

if(!dwNtStatus)
{
 pModuleInfo = (QUERYDEBUGBUFFER*)pRtlBuffer;

 // Производим перечисление модулей процесса
for(int i = 0; i < pModuleInfo->dwNumNames; i++)
{
 printf("ImageBase: 0x%0.8Xl", pModuleInfo[i]->ImageBase);
 printf("ImageSize: 0x%0.8Xl", pModuleInfo[i]->ImageSize);
 ...
}
}
else if(dwNtStatus == DEBUG_ACCESS_DENIED)
{
// Error
}

// Освобождаем рабочий буфер
RtlDestroyQueryDebugBuffer(pModuleInfo);
```

Определение ImageBase и ImageSize

ImageBase и **ImageSize** определяются достаточно легко. Нужно сделать перечисление модулей того процесса, для которого определяются эти значения. И в СПИСКЕ МОДУЛЕЙ найти имя нужного процесса, как правило оно идёт первым. Далее взять значения из структуры **DEBUGMODULEINFO** :

```
QUERYDEBUGBUFFER *pModuleInfo;
...
pModuleInfo->ImageBase;
pModuleInfo->ImageSize;
```

НО! Если процесс использует антидамповые приёмы (один из которых можно посмотреть во введении – нечего было его проскакивать!), то, как правило, **ImageSize** имеет неправильное значение. Поэтому рекомендуется его считать из **PE** заголовка процесса, но можно и из файла на диске (последнее даже надежнее). Ниже приведён пример функции считывающей **ImageSize** из заголовка:

```
//Первый параметр - это PID процесса, второй  - ImageBase.
DWORD GetRealSizeOfImage(DWORD dwPID, PVOID pModBase)
```

```
{
 IMAGE_DOS_HEADER pDosh = {0};
 IMAGE_NT_HEADERS pNT = {0};

 HANDLE hProcess = OpenProcess(PROCESS_VM_READ, FALSE, dwPID);
 if(hProcess)
 {
  ReadProcessMemory(hProcess,
                        pModBase,
                        &pDosh,
                        sizeof(IMAGE_DOS_HEADER),
                        NULL);
  if(IMAGE_DOS_SIGNATURE == pDosh.e_magic)
   ReadProcessMemory(hProcess,
                           (PBYTE)pModBase + pDosh.e_lfanew,
                            &pNT,
                           sizeof(IMAGE_NT_HEADERS), NULL);
  CloseHandle(hProcess);
  if(pNT.Signature == IMAGE_NT_SIGNATURE)
   return pNT.OptionalHeader.SizeOfImage;
 }
 return NULL;
}
```

Однако даже наличие таких методик в **PE Tools** все равно не спасет от защиты-драйвера (например, **Extreme-Protector**). Поэтому, видно, пришло время и **PE Tools** переходить на драйверный движок, который мало в чем будет опираться на структуры кольца-3. Так правильнее.

Практический пример: UPX

написано совместно с Quantum

UPX является практически единственным исключением в своем роде - полностью открыт исходный код. Это обстоятельство пытаются использовать многие авторы упаковщиков, и распаковщиков тоже. Часть просто молча ворует исходники (GPL-лицензия таки налагает некоторые забавные ограничения, хоть исходный код и открыт), другая часть пытается использовать это знание нам во вред, забывая - что **OpenSource** - это палка о двух концах. В данной главе принципиально не будет приведено ни единой строчки кода дизассемблера. Зачем? Все есть в кодах **UPX**. Итак, скачивайте кода с http://upx.sourceforge.net и в директории **stub** находите файл **l_w32pe.asm**. Не забудьте заглянуть и в **stub.asm**. В самом начале данного файла есть забавное предупреждение - 5 минут смеха обеспечены! Тем не менее, ассемблерный листинг стаба проще читается под отладчиком, так как исходники набиты всякими директивами препроцессору C (невидимыми ассемблеру), которые явно мешают уловить суть алгоритма.

UPX полностью пересобирает **PE** -файл, меняя все, что можно, за исключением ресурсов. К ним **UPX** относится достаточно бережно. Все остальное программа переводит в свой внутренний формат и сжимает по алгоритму **UCL** (кстати, вовсе не обязательно именно **UCL** , есть еще прогрессивный NRV) http://www.oberhumer.com/opensource/ucl.

Как программа переводит эти данные в свой формат описано в файле **p_w32pe.cpp** (для каждого поддерживаемого формата - свой **cpp** -файл со своими методами). Масса полезной информации находится также в файлах **packer.cpp** , **packhead.cpp** и **compress.ch** (очень занимательное чтиво). Однако нас это интересует мало, тем более не всякий открытый исходный код - панацея, т.к. за спасибо можно получить только исходники компрессора **UCL** , а последняя бета-версия **UPX** использует компрессор **NRV** (не **GPL** , однако). Получается, что основная часть упаковщика остаётся за кадром...

В результирующем файле (сколько бы секций ни было в оригинальном) всегда будут только три секции (для версий, отличных от 1.24-1.90, правило, возможно, соблюдаться не будет – кода мы не изучали) -

1. UPX0 - кладутся tls
2. UPX1 - fixup-элементы, импорт, экспорт, код и т.п.
3. UPX2 - ресурсы, однако в действительности этой секции, как правило, нет, т.к. автор утилиты прекрасно знал о том, что имя секции .rsrc очень много значит.

Код стаба - **l_w32pe.asm** - разжимает секции и обрабатывает директории импорта и **fixup** -элементов. Заметьте, повторим еще раз - идет обработка директории импорта! Часто встречается утверждение, что, мол, **UPX** переводит все в свой внутренний формат и, посему, лучше использовать **ImpRec** и иже с ним. Переводить-то, утилита переводит, да только потом разжать-то ведь надо, и перевести назад – в валидный для ОС формат. Внимательно рассмотрим код **l_w32pe.asm** по обработке импортов:

```
/*код стаба построен достаточно хитро - он насыщен инструкциями вроде %endif;
 __PEMAIN01__, где невидима для ассемблера,
  но прекрасно видима препроцессором Си
  - т.о. этот код может (и будет!) различаться для dll/exe и т.п.*/
                pushad
                mov     esi, 'ESI0'                ; VA секции UPX1
                lea     edi, [esi + 'EDI0']        ; VA секции UPX0
...
                push    edi
...
                pop     esi
...
                lea     edi, [esi + 'BIMP']        ;распакованные имена функций
                                                   ;во внутреннем формате UPX

next_dll:
; проверить на конец массива имен – DWORD = 0
                mov     eax, [edi]
                or      eax, eax
; хоть метка и называется imports_done, правильнее - names_done
                jz      imports_done
                mov     ebx, [edi+4]
; имена dll во внутреннем формате UPX
                lea     eax, [eax + esi + 'IMPS']
                add     ebx, esi
; как мы постоянно упоминаем – имена лежат во внутреннем формате и edi
; показывает на два хитрых DWORD’а,
; на основании второго из них ([edi+4]) вычисляется VA,
; куда GetProcAddress будет класть полученные адреса
                push    eax
                add     edi, 8
                call    [esi + 'LOAD'] ; LoadLibraryA
                xchg    eax, ebp  ; ebp - хендл
next_func:
                ; имя функции
                mov     al, [edi]
; как мы уже упоминали – формат внутренний,
; строки разделены нулями
                inc     edi
                or      al, al
                jz      next_dll
                mov     ecx, edi

                push    edi             ; имя функции из dll
                dec     eax             ; 0 - разделитель
                repne
                scasb                   ; встать на начало следующего имени

                push    ebp   ; хендл
                call    [esi + 'GETP'] ; GetProcAddress
                or      eax, eax
```

```
                jz      imp_failed
                mov     [ebx], eax      ; начинаем готовить массив адресов
                add     ebx, 4          ; увеличить VA на sizeof(DWORD)
                jmps    next_func
imp_failed:
  ; к ExitProcess
imports_done:
  ; к OEP
```

То, что UPX, фактически, распространяется с открытым кодом, да ещё и для различных форматов исполнимых файлов (Win16, Win32, Posix, MS-DOS и т.д.) серьёзно ограничивает его антиотладочные возможности... Цель UPX заключается в максимальном сжатии файла, но не в противостоянии хакерским усилиям по его распаковке. UPX.EXE поддерживает модификатор -d в командной строке для распаковки своих же файлов и почти всех предыдущих версий UPX включительно. На вопрос "как распаковать UPX?" можно лениво ответить, что мол "UPX -d packed.exe" (зря вы дампер приготовили). Так даже не интересно...

Стоп! Мы забываем про утилиты для защиты UPX, так называемые скрамблеры (scramblers). Скрамблеры пытаются немного замаскировать запакованные файлы, чтобы обмануть UPX. Но мы-то знаем, что перед нами файл, запакованный UPX'ом, потому что нам об этом сообщил идентификатор (sniffer) файлов или в дизассемблере "на глаз" был подмечен стаб UPX'a.

Здесь мы НЕ будем учиться распаковывать UPX. Существуют утилиты, которые прекрасно справляются с этой задачей, без особого участия со стороны пользователя. Здесь мы попытаемся понять принципы защиты UPX от распаковки, которые активно используются скрамблерами, вроде UPX-SCRAMBLER и HidePX. Данные утилиты призваны уберечь поднаготную запакованного экзешника от посторонних глаз, но делают они это не очень эффективно. Скачайте себе любую из них, или обе, или какой-нибудь другой скрамблер и пропустите через него ваш calc.exe. Что, уже не получается распаковать через upx -d? Как уже упоминалось раньше, существуют мощные распаковщики, которым скрамблеры погоды не строят, но наша цель - преодолеть защиту скрамблеров своими руками.
Не поленитесь сравнить calc.exe до и после прохождения через скрамблер. Можете воспользоваться WinHex / File Manager / Compare или другой подобной утилитой (PE Tools и LordPE умеют сравнивать и поля PE-формата – опция – “Compare”). Внимательно изучите листинг расхождений в обоих файлах и вы скоро поймёте, что первостепенные различия связаны с именами секций и сигнатурой UPX. UPX-SCRAMBLER заменяет UPX0 на code и UPX1 на text. HidePX затирает имя UPX0 и заменяет UPX1 на .rdata.

Если бы всё дело было только в именах секций, то для восстановления calc.exe можно было бы просто восстановить имена секций в любом PE-редакторе. Имена секций восстановлены, но там ещё и с сигнатурой что-то не так... Что такое сигнатура в данном случае? Здесь есть два понятия, которые необходимо различать. Под сигнатурой, с точки зрения последовательности байт кода, сама UPX понимает следующее:

```
/*код взят из файла p_w32pe.cpp метод canUnpack класса PackE32Pe*/
/*этот метод очень важен – именно тут UPX делает проверки
 на количество секций и их имена, проверятся байтовая сигнатура и,
 если что-то не так, бросается исключение с надписью
 «file is modified/hacked/protected; take care!!!»*/

bool PackW32Pe::canUnpack()
{
 ...
      static const unsigned char magic[] = "\x8b\x1e\x83\xee\xfc\x11\xdb";
      // mov ebx, [esi];    sub esi, -4;    adc ebx,ebx

}
```

Очевидно, изменив код этого метода, несложно добится того, что для программы перестанет иметь значение имя секции (количество секций лучше не трогать), об отсутствии или неверном **offset** ’е

сигнатуры она станет лишь предупреждать, а не бросать исключение, но это лишь малая часть айсберга, т.к. существует ВТОРАЯ сигнатура! Под второй сигнатурой понимается структура, начинающаяся с “**UPX**!” (см. таблицу), которую **UPX** помещает перед сжатой частью файла. Помните, мы говорили, что **UPX** полностью перестраивает формат файла и граница старого сжатого файла начинается со второй сигнатуры. И, если испорчена она (что и делают скрамблеры), то тогда **UPX** просто слетит с внутренним исключением. Очевидно, снятие ВТОРОЙ сигнатуры и есть самое главное препятствие. Препятствие ли?

Для нахождения/просмотра/восстановления сигнатуры **UPX** можно воспользоваться хекс-редактором, но лучше - плагином **Uncover UPX** для **PE Tools**. Плагин прилагается к данной главе, так что можете сразу копировать его в каталог **Plugins** и **PE Tools** автоматически поместит его в соответствующее меню.

Основная черта данной утилиты заключается в автоматическом пересчёте контрольной суммы (поле CRC) при изменении остальных полей сигнатуры. Плагин также умеет частично или даже полностью восстанавливать сигнатуру после применения скрамблера. В случае с **HidePX** вам потребуется ввести (исправить) некоторые значения. Итак, что там за поля такие в сигнатуре? Учтите, что все значения отображаются в обратном порядке байт, т.е. в формате **little endian** , например: **12345678** -> **78563412** , **ABCDEF** -> **EFCDAB** , **ABCD** -> **CDAB** , **AB** -> **AB**.

Поле	**Размер (в байтах)**	**Значение**
Magic	4	Последовательность ASCII-символов 'UPX!'
Version	1	Версия упаковщика, например: 0C значит 1.24, 0D - это последняя на данный момент бета 1.90. Если вы пользуетесь UPX v1.90 и подопытный экзешник не очень старый (после 2001), можете спокойно прописать сюда 0D.
Format	1	Для интересующих нас экзешников в формате PE32, это поле всегда равно 09.
Method	1	Наиболее распространённые методы сжатия - это NRV и UCL. Обоим соответствует значение 02.
Level	1	Степень сжатия.
U_adler	4	Контрольная сумма части экзешника в распакованном виде*.
C_adler	4	Контрольная сумма части экзешника в запакованном виде*.
U_len	4	Размер части экзешника в распакованном виде*.
C_len	4	Размер части экзешника в запакованном виде*.
U_file_size	4	Размер распакованного экзешника.
Filter	2	Об этом чуть позже!
CRC	1	Контрольная сумма сигнатуры. Плагин показывает её в виде 16-битного значения, потому что перед CRC идёт дополнительный байт выравнивания.

• Под частью экзешника подразумевается та часть, которая подлежит сжатию/разжатию. В общем, кроме самого **UPX** ’а никто больше не умеет определять точные границы этой *части*.

Сигнатура **UPX** для старых **NE** , линуксовых **ELF** и т.д. представлена иначе, но нам интересен только формат **PE32**.

Uncover UPX самостоятельно восстанавливает сигнатуру после **UPX-SCRAMBLER** , так как данный скрамблер уничтожает сигнатуру частично, но что делать если сигнатура утеряна полностью, как в случае с **HidePX**? Тогда **Uncover UPX** заполнит её значениями по умолчанию и нам придётся немного ему помочь, если в том возникнет нужда.

Поле **magic** , понятное дело, менять не стоит. Поле version обычно оставляется как есть (0C или 0D). Format оставьте со значением 09. Method в **99%** случаев равен **02**. В **level** можете поместить любое отличное от нуля значение, так-как распаковщик не обращает внимания на уровень сжатия. Тоже самое относится к полю **u_file_size**.

Значения в **u_adler** и **c_adler** - это контрольные суммы, рассчитанные по алгоритму Марка Адлера (http://www.cdrom.com/pub/infozip/zlib/). Можете посмотреть исходники данного алгоритма, но они нам не помогут... Предполагается, что у нас нет распакованного варианта экзешника. Значит и подсчитать его контрольную сумму, даже зная алгоритм, мы не можем... На самом деле, обе контрольные суммы не влияют на процесс распаковки, т.е. можно просто отключить проверку данных значений внутри **UPX**. Вы уже скачали исходники **UPX** и **UCL**? Кстати, в исходниках **UCL** есть пример реализации алгоритма М. Адлера, но не будем отвлекаться. В исходниках **UPX** , в файле **packer.cpp** есть код следующего содержания:

```
void Packer::decompress(const upx_bytep in, upx_bytep out,
                        bool verify_checksum)
{
    // verify_checksum = true, т.е. этот код всегда выполняется
    if (verify_checksum)
    {
        unsigned adler = upx_adler32(in,ph.c_len);
        if (adler != ph.c_adler)
            throwChecksumError();
    }

    // Тут происходит вызов 'настоящего' распаковщика
    unsigned new_len = ph.u_len;
    int r = upx_decompress(in,ph.c_len,out,&new_len, ph.method);
    if (r != UPX_E_OK || new_len != ph.u_len)
        throwCompressedDataViolation();

    // опять эти адлеры...
    if (verify_checksum)
    {
        unsigned adler = upx_adler32(out,ph.u_len);
        if (adler != ph.u_adler)
            throwChecksumError();
    }
}
```

Если отключить обе проверки (до и после распаковки), про адлеры можно будет забыть. Следующий код заодно отключает проверку **c_len** и **u_len** , т.е. гонимся за четырьмя зайцами и успешно их ловим:

```
void Packer::decompress(const upx_bytep in, upx_bytep out,
                        bool verify_checksum)
{
    // Тут происходит вызов 'настоящего' распаковщика
    int r = upx_decompress(in,ph.c_len,out,&ph.u_len,ph.method);
}
```

Правда, даже если вы сможете перекомпилировать этот код, вы получите версию UPX без поддержки NRV. Было бы куда лучше внести эти исправления в последнюю версию UPX (1.90 на данный момент), которая поддерживает сразу UCL и NRV. Где наш дизассемблер? Стоп, перед устранением этого бага в UPX, не забудьте его распаковать (он сам собой и запакован)

Распакованный **UPX.EXE** (около **327** Кб) грузим в **HIEW** и задаём поиск **56578B7C241484DB8BF1**. Кстати, данная последовательность применительна и к предыдущей версии **UPX**. Узнаёте следующий код?

```
53         push  ebx
8A5C2410   mov   bl,[esp][10]
55         push  ebp
56         push  esi
57         push  edi
8B7C2414   mov   edi,[esp][14]
84DB       test  bl,bl
8BF1       mov   esi,ecx
74XX       je    XXX  ; это тот if (verify_checksum)
```

Меняем 74 на EB и первая проверка адлеров решена! Чуть дальше вы встретите вторую проверку контрольной суммы:

```
7405       je    XXX
E8XXXXFFFF call  XXX
84DB       test  bl,bl
74XX       je    XXX  ; это второй if (verify_checksum)
```

Исправляем на безусловный переход, как в первом случае и вторая проверка тоже решена! Осталось исправить throwCompressedDataViolation(). Для этого следуем за первым исправленным переходом и вскоре видим вызов функции с пятью параметрами - это **upx_decompress** :

```
8B561C     mov   edx,[esi][1C]  ; это ph.u_len
8B4614     mov   eax,[esi][14]
8B6C2418   mov   ebp,[esp][18]
8D4C241C   lea   ecx,[esp][1C]  ; а это new_len
8954241C   mov   [esp][1C],edx
8B5620     mov   edx,[esi][20]
50         push  eax            ; ph.method
51         push  ecx            ; &new_len
55         push  ebp            ; out
52         push  edx            ; ph.c_len
57         push  edi            ; in
E8XXXXXXXX call  XXX            ; upx_decompress
```

Функции вместо **new_len** нужно подсунуть адрес **ph.u_len**. Это можно организовать заменив 8D4C *24* 1C на 8D4C *26* 1C. Сразу за этим вызовом видим код примерно следующего содержания:

```
85C0       test  eax,eax
75XX       jne   XXX      ; если r == UPX_E_OK
8B44XXXX   mov   eax,XXX
8B4EXX     mov   ecx,XXX
3BC1       cmp   eax,ecx
74XX       je    XXX      ; если new_len == ph.u_len
```

Исправьте второй переход с условного на безусловный. Всё, теперь нам море по колено! Можете вписывать в поля **u_adler** и **c_adler** всё, что хотите (хоть **FFFFFFFF** , чтобы не путаться с **little endian**) Для полного отключения проверки валидности файла можете ещё удалить сравнение имени первой секции. Это сравнение очень просто найти в... Нет уж, ищите сами! В противном случае, будете и дальше править имена секций в **PE Editor**.

Что до **u_len** и **c_len** , то проверку валидности данных полей вы уже отключили но, в отличии от адлеров, выбор значений **u_len** и **c_len** налагает некоторую ответственность. Дело в том, что **UPX** резервирует два буфера в памяти: один размером с **u_len** для временного хранения распакованного файла, другой размером с **c_len** для чтения запакованного содержимого файла. Понятно, что если задать слишком маленькое значение для **u_len** , то распакованный файл просто не поместится в буфер. С другой стороны, слишком большое значение заставит **UPX** потреблять больше динамической памяти. Для **u_len** вполне подойдёт значение **000FFFFF** (**FFFF0F00** в **little endian**) для большинства упакованных экзешников.

С **c_len** чуть сложнее. Опять же, слишком маленькое значение вызовет конфликтную ситуацию с динамической памятью, но слишком большое, кроме излишнего расхода памяти, отрицательно воздействует на сам процесс распаковки. Иначе говоря, фокус с **000FFFFF** не пройдёт. Надо подобрать более близкое значение. К счастью, в версии 1.24 выдаётся одно сообщение об ошибке, когда значение меньше правильного и другое - когда больше. В версии 1.90 эта фича отсутствует, но на данный момент **HidePX** не поддерживает 1.90, а **UPX-SCRAMBLER** не портит значение **c_len**. В версии 1.24 даже не нужно задавать абсолютно точное значение c_len - небольшая погрешность спокойно поглощается распаковщиком.

В заключение стоит упомянуть поле **filter**. В нём обычно хранится значение **260X** , где **X** может быть 0, реже – 1, а еще реже – что-нибудь другое, например, 6, для старых версий UPX. Данное поле заслуживает особого внимания, так как неправильное значение фильтра приводит к неправильной распаковке экзешника, т.е. файл распаковывается, но не запускается! В общем, зря скрамблеры пренебрегают этой записью. Немного подправив значение фильтра можно защитить файл куда эффективнее, хотя от дамперов это всё равно не поможет, но всё-таки...

Теперь давайте проведем два практических примера. Наконец-то!

Итак, **UPX-SCRAMBLER**. Скачиваем с wasm специальные upx by **Quantum** /**Volodya**. Результат:

```
C:\Downloads\PE\upx>upx1_24.exe -d s.calc.exe
распаковано мгновенно!
```

Теперь **HidePX**.

```
C:\Downloads\PE\upx>upx1_24.exe -d p.calc.exe
upx1_24: p.calc.exe: CantUnpackException: fillPackHeader: Seems like HidePX...
```

Поможем нашему **UPX**. Загружаем **PE Tools** , запускаем **Quantum** 'овский плагин. Жмем одну-единственную кнопочку – **Fix**. Повтор:

```
C:\Downloads\PE\upx>upx1_24.exe -d p.calc.exe
распаковано мгновенно!
```

На тот маловероятный случай, что что-то пойдет не так... Хм, а зачем мы вам столько писали, а?

В заключение статьи надо сказать, что некоторые упаковщики довольно бездарно пытаются замаскироваться под **UPX**. Например, это делает **telock**. Только вот все они не учитывают одной маленькой тонкости - **UPX** 1.24+ создает ТОЛЬКО ДВЕ секции - **UPX1** и **UPX0** - независимо от того, сколько на самом деле секций в файле. А **telock** этого попросту не учитывает, создавая в некоторых случаях несколько **UPX** -секций, что сразу бросается в глаза. Гораздо более точным критерием в данном случае можно считать присутствие характерного стаба, ведь он действительно нужен для того, что бы экзешник мог сам себя распаковать в памяти.

В самое заключение главы. Помните, в самом начале мы говорили, что здесь не будет ни строчки дизассемблированного листинга? Так вот, мы соврали. Имеет смысл разобрать один скользкий момент с **HIEW**. Передача управления на **OEP** в **UPX** отображается **HIEW** (по 6.85 включительно) так:

```
.0101AFF1: 8903                    mov         [ebx],eax
.0101AFF3: 83C304                  add         ebx,004 ;"¦"
.0101AFF6: EBE1                    jmps       .00101AFD9  -----^ (3)
.0101AFF8: FF9690BC0100            call        d,[esi][0001BC90]
.0101AFFE: 61                      popad
.0101AFFF: E91C74FFFF              jmp         0FFFFE820 ;на OEP
```

Почему же **jmp** по адресу **0x101AFFF** имеет такой странный операнд? Давайте спросим автора **HIEW** – **SEN** 'а. Ответ: «...но такого **VA** в файле нет, он появится потом, когда **UPX** память выделит для этого **VA** , а в файле ничего нет, поэтому **hiew** просто отсчитывает в глобальных адресах смещение и показывает как есть». Так что это ни в коем случае не баг утилиты. Просто автор очень не хочет включать поддержку многочисленных частных случаев, благодаря чему HIEW был и,

пожалуй, так и остается одним из самых быстрых дизассемблеров на сегодяшний день. А уж эта возможность поиска по ассемблерной маске с * и ? – так это вообще фантастика...

Практический пример: Aspack

Что делает новичок, увидев программу, запакованную **Aspack**? Ну, берется **Soft-Ice** , берется дампер процессов и, обязательно, **Imprec**. Потом над всем этим инструментарием начинают интенсивно издеваться – зацикливать на **OEP** , дампить целиком, посекционно или еще как, немедленно запускать Imprec и вставлять полученный дамп директории импорта в файл. Ну что ж. Можно и так. Работает. Только давайте усложним задание. Положим, дампер процессов не имеет никакого движка по перестройке импорта, а **Imprec** и иже с ним у нас просто нет. Что тогда?

А вот тут-то и надо рассматривать сам алгоритм работы упаковщика. К счастью, есть и такие статьи. Например: «Исследование алгоритма работы упаковщика ASPack v1.08.03» - довольно толковая статья (когда будете читать – хм, вы уже знаете, что такое **KTEB**). Несмотря на то, что рассматривается старая версия, в новой (2.12) не так уж и много изменений с нашей точки зрения. Для кросс-проверки и некоторого дополнения сведений можно также проглядеть статью «ASPack 'Spelunking'».

В связи с тем, что есть такие великолепные материалы, мы не станем рассматривать код **ASPack** подробно. Для начала учтите – что это не криптор, это самый обычный упаковщик. Здесь нет ни антиотладки, ни сколь-нибудь сложных приемов противостояния дизассемблерам. Лишь в самом начале есть нечто, робко напоминающее полиморфный код:

```
;версия 2.12
.aspack:01019001                pusha
;КРЕПКО запомните эту инструкцию! Проникнитесь! Она нам ох как пригодится!
;да, меж прочим, дизассемблирована эта команда неверно!
;В 32-битном режиме это pushad.
.aspack:01019002                call    loc_101900A
.aspack:01019002 ; ---------------------------------------------------------
.aspack:01019007                db 0E9h ; щ
.aspack:01019008 ; ---------------------------------------------------------
.aspack:01019008                jmp     short loc_101900E
.aspack:0101900A ; ---------------------------------------------------------
.aspack:0101900A
.aspack:0101900A loc_101900A:                           ; CODE XREF: start+1p
.aspack:0101900A                pop     ebp
.aspack:0101900B                inc     ebp
;++ebp = eip – хороший пример позиционно-независимого кода – PIC
.aspack:0101900B start          endp
.aspack:0101900B
.aspack:0101900C                push    ebp
.aspack:0101900D
.aspack:0101900D locret_101900D:                        ; CODE XREF: start+7u
.aspack:0101900D                retn
;учтите – это не антиотладка, это лишь достаточно красивый
;пример вывертов с ассемблером – смотреть приятно!
.aspack:0101900E ; ---------------------------------------------------------
.aspack:0101900E
.aspack:0101900E loc_101900E:                           ; CODE XREF: start+7j
.aspack:0101900E                call    loc_1019014
.aspack:01019014 loc_1019014:                           ; CODE XREF: .aspack:0101900Ep
.aspack:01019014                pop     ebp
;вопрос на засыпку – на что показывает ebp?
;здесь очень удобна возможность IDA переходить по G “+”
; – просто ставьте курсор на нужное смещение и вперед.
```

Итак, ясно, что **ebp** показывает на начало кода/данных. В связи с этим весь остальной код теперь достаточно четок и ясен. Полагаем, понять, что делает этот код, после фокуса с **ebp** , уже

предельно просто:

```
.aspack:01019035     lea     eax, [ebp+42Eh]       ;offset на строку kernel32.dll
.aspack:0101903B     push    eax
.aspack:0101903C     call    dword ptr [ebp+0F4Dh] ;вызывается GetModuleHandleA
; явное получение хендла нужно для вызова некоторых
; дополнительных функций – VirtualAlloc/VirtualFree и т.п.
```

Фокус с

```
.aspack:0101906C                lea     eax, [ebp+77h]
.aspack:0101906F                jmp     eax
```

уже тоже сбить не должен. Куда передаст управление регистровый jmp, вы уже можете найти и без отладчика.

Теперь давайте посмотрим, как правильно надо сбрасывать дамп из программ, запакованных **Aspack**. Здесь предельно четко нужно понимать, что нам нужен файл ДО обработки его самим упаковщиков, т.е. СРАЗУ ЖЕ после разжатия. Такой момент существует не для всех упаковщиков, однако для Aspack он проявляется предельно наглядно.

Итак, **Aspack** выполняет разжимание секций по **LZ** +Хаффман-подобному алгоритму с использованием **VirualAlloc** для отведения памяти под временный буфер и **VirtualFree** для высвобождения этого буфера. В буфер вбрасывается разжатое содержимое, которое копируется двойными словами (хорошая оптимизация!) (и докопируется побайтно при необходимости) на то место, что принадлежит ему по праву – на оригинальный **RVA** секции. После полного разжатия и копирования **Aspack** принимается обрабатывать директорию импорта (см. ниже), директорию перемещаемых элементов, вычисляет и предает управление на **OEP**. Наша задача – сбросить дамп **ДО** обработки импорта, **fixup** -элементов и т.п. Сделаем мы это как раз здесь:

```
.aspack:0101916D        mov     ecx, eax        ; счетчик
.aspack:0101916F        mov     edi, [esi]
.aspack:01019171        add     edi, [ebp+422h] ; ImageBase + section RVA
.aspack:01019177        mov     esi, [ebp+152h] ; внутренние таблицы упаковщика
.aspack:0101917D        sar     ecx, 2
          ; копирование двойнми словами – edi - приемник
.aspack:01019180        rep movsd
.aspack:01019182        mov     ecx, eax
.aspack:01019184        and     ecx, 3
.aspack:01019187        rep movsb               ; докопировать хвостик
.aspack:01019189        pop     esi
.aspack:0101918A        push    MEM_RELEASE     ; DWORD dwFreeType
.aspack:0101918F        push    0               ; в точности после VirtualAllloc
.aspack:01019191        push    dword ptr [ebp+152h] ; LPVOID lpAddress
.aspack:01019197        call    dword ptr [ebp+551h] ; VirtualFree
.aspack:0101919D        add     esi, 8
.aspack:010191A0        cmp     dword ptr [esi], 0
.aspack:010191A3        jnz     unpack_loop ; для каждой секции
.aspack:010191A9        push    MEM_RELEASE ;---> это и есть наша цель
.aspack:010191AE        push    0
.aspack:010191B0        push    dword ptr [ebp+156h]
.aspack:010191B6        call    dword ptr [ebp+551h] ; VirtualFree
```

Адрес **0x010191A9** здесь как раз и является « *заветным* ». Зацикливая программу на этом адресе мы можем быть уверены, что дамп является полностью рабочим и не подвергнут ни обработке импорта, ни чему-либо еще. Ввести программу в бесконечный цикл можно по-разному, к примеру, введите ассемблерную команду **jmp eip** , воспользуйтесь командой **!dump** в **IceExt** , выберите **SuspendThread** , словом, вам и карты в руки. Мы же здесь, разумеется, объясним как использовать **PE Tools**.

Итак, после того как **jnz** не сработает – все секции распакованы и можно приступать. Полный дамп файла с опциями **Full Dump: fix header, Full Dump: rebuild image. DumpFix, ValidatePE, RebuildPE**. В дампе ручками в PE Editor: **Optional Header >** пересчитываем: **SizeOfImage, SizeOfHeaders, Checksum** (при помощи кнопки "?"). Уменьшаем размер. Делаем **RebuildPE** , с

опциями: **DumpFix** , **ValidatePE** , **RebuildPE**.

Файл ПОЧТИ валиден. Осталась мелочь – параметры директории импорта и правка **EP**. Нам более не нужен код пакера, поэтому **EP** должна быть переориентирована назад – на **OEP** и директория импорта должна быть поправлена. Для этого залезем в алгоритм **Aspack** еще раз. Вспоминаем первую часть и действие функции **LdrpSnapIAT** , что в **ntdll.dll**. Все пакеры должны эмулировать ее действие, заключающееся (не только) в превращении RVA полей директории импорта в **VA** – взгляните на код:

```
.aspack:01019278       mov     esi, 12A40h     ; RVA на директорию импорта
;обратите внимание – вычисляется упаковщиком
;при обработке файла – гляньте на RVA в нормальном файле
;остается в файле в открытом виде, что очень удобно для поиска по константе
.aspack:0101927D       mov     edx, [ebp+422h] ; ImageBase
.aspack:01019283       add     esi, edx    ; в VA (VA = RVA + ImageBase)
.aspack:01019285
.aspack:01019285 process_IID:              ; CODE XREF: .aspack:01019395j
.aspack:01019285       mov     eax, [esi+IMAGE_IMPORT_DESCRIPTOR.Name]
.aspack:01019288       test    eax, eax
.aspack:0101928A       jz      finish      ; конец директории импорта?
.aspack:01019290       add     eax, edx    ; RVA на имя dll -> в VA на имя dll
.aspack:01019292       mov     ebx, eax
.aspack:01019294       push    eax
.aspack:01019295       call    dword ptr [ebp+0F4Dh] ; GetModuleHandle
.aspack:0101929B       test    eax, eax
.aspack:0101929D       jnz     short module_already_loaded
.aspack:0101929F       push    ebx
.aspack:010192A0       call    dword ptr [ebp+0F51h]  ; LoadLibraryA
...
.aspack:010192F5       push    ebx              ; или имя, или ординал
.aspack:010192F6       push    dword ptr [ebp+545h]    ; handle
.aspack:010192FC       call    dword ptr [ebp+0F49h]   ; GetProcAddress
.aspack:01019302       test    eax, eax
.aspack:01019304       pop     ebx           ; *PIMAGE_THUNK_DATA32
.aspack:01019305       jnz     short addr_ok ; функция вернула валидный адрес
...
.aspack:01019376 addr_ok:                ; CODE XREF: .aspack:01019305j
.aspack:01019376                         ; .aspack:0101935Ej
.aspack:01019376       mov     [edi+IMAGE_THUNK_DATA.u1], eax
                       ; +sizeof(IMAGE_THUNK_DATA32)
.aspack:01019378       add     dword ptr [ebp+549h], 4
.aspack:0101937F       jmp     loc_10192B6
; после заполнения IMAGE_THUNK_DATA реальными
; адресами функций и замене RVA на VA
; директория теряет смысл для нас
```

Итак, RVA директории импорта ясен, осталось узнать размер. Делаем так:

```
:g 1019278             ;встать на mov esi, 12A40h
:d esi+edx             ;видим начало IID и визуально определяем
                       ; конец
001B:01012A40 C0 2B 01 00 FF FF FF FF-FF FF FF FF E6 2C 01 00  .+.........ж,..
001B:01012A50 F4 10 00 00 60 2B 01 00-FF FF FF FF FF FF FF FF  ....`+..........
001B:01012A60 08 2E 01 00 94 10 00 00-CC 2A 01 00 FF FF FF FF  ....”....*......
;он выглядит так:
:d 010134D0
0023:010134D0 65 01 47 65 74 57 69 6E-64 6F 77 54 65 78 74 57  e.GetWindowTextW
0023:010134E0 00 00 55 53 45 52 33 32-2E 64 6C 6C 00 00 00 00  ..USER32.dll....
0023:010134F0 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
:? 10134f0-1012a40     ;вычисляем длину
<ulong> = 0xAB0, 2736, "\n°"
```

Параметры директории должны выглядеть так: **RVA** – 0x12A40, **Size** – 0xAB0. Теперь **OEP** :

```
.aspack:0101939A     mov     eax, 12420h     ; OEP RVA –
                                             ; и OEP хранится в чистом виде
.aspack:0101939F     push    eax
.aspack:010193A0     add     eax, [ebp+422h] ; OEP VA = OEP RVA + ImageBase
.aspack:010193A6     pop     ecx
.aspack:010193A7     or      ecx, ecx
```

```
.aspack:010193A9     mov     [ebp+3A8h], eax
;заметьте, в коде aspack много подобных инструкций –
;это его отличительная черта, сбросьте флажок Writable у
;секции aspack в вашем файле и немедленно увидите результат
.aspack:010193AF     popa
.aspack:010193B0     jnz     short loc_10193BA
.aspack:010193B2     mov     eax, 1
.aspack:010193B7     retn    0Ch             ; оригинальная dll не имела
                                             ; точки входа
.aspack:010193BA ; ---------------------------------------------------------------
.aspack:010193BA
.aspack:010193BA loc_10193BA:              ; CODE XREF: .aspack:010193B0j
.aspack:010193BA     push    0
;операнд инструкции push заполняется динамически,
;инструкцией по адресу 010193A9
.aspack:010193BF     retn                  ;на OEP
```

Меняем RVA OEP на 0x12420. Все. Дамп рабочий и полностью готов. В качестве домашнего задания – попытайтесь удалить секции Aspack. Если с тем, что здесь написано, есть внутренние сомнения – почитайте дополнительно очень хорошую статью: «unpacking files with a .aspr section», а мы, тем временем, рассмотрим один оригинальный, хотя и не новый, прием. Надеемся, вы еще помните, что мы советовали обратить внимание на **pusha** в начале этой главы. Вы уже знаете, что дампить программу нужно не на **OEP** , а, желательно, задолго до него. В случае **Aspack** , где в открытом виде остается директория импорта, перемещаемых элементов, распаковываются секции, до **OEP** добираться нет нужды. Однако, возможно представить ситуации, когда времени просто нет и нужно добежать. Тогда используйте трюк с **bpm esp-4**. Немедленно после pushad ставьте эту точку останова. Действие pushad можно глянуть в интеловских талмудах в виде псевдокода. Если лень глядеть, вот:

```
Temp ? (ESP);
Push(EAX);  // 0x1C
Push(ECX);  // 0x18
Push(EDX);  // 0x14
Push(EBX);  // 0x10
Push(Temp);  // 0x0C
Push(EBP);  // 0x08
Push(ESI);  // 0x04
Push(EDI);  // 0x00
```

Точка останова сработает поблизости от **OEP** – на **popad** – при вытаскивании регистров назад. Вот здесь:

```
.aspack:010193AF    popa   ;-> точка останова сработает здесь
.aspack:010193B0    jnz     short loc_10193BA
.aspack:010193B2    mov     eax, 1
.aspack:010193B7    retn    0Ch    ; оригинальная dll не имела точки входа
.aspack:010193BA ; ---------------------------------------------------------
.aspack:010193BA
.aspack:010193BA loc_10193BA: ; CODE XREF: .aspack:010193B0j
.aspack:010193BA    push    0
.aspack:010193BF    retn      ;на OEP
```

Заметим напоследок, что трюк с **bpm esp-4** сейчас работает не так уж и часто. Многие крипторы специально осложняют нам жизнь. В некоторых случаях это можно обойти, к примеру, поставив точку останова еще ниже, но с некоторыми крипторами не помогает и это. К примеру, вполне возможно поставить в блок **try/catch** пример на исчерпание стека в бесконечном цикле, а обработчик исключения должен это дело разобрать. Для успешного понимания таких защит необходимо четко разбираться в **SEH** – структурных исключениях, что и рассматривается в следующей главе.

SEH с точки зрения кракера

Рискуя повторится, тем не менее, скажем еще раз - для успешного отдирания криптора от файла необходимо неплохо разбираться в принципах работы операционной системы. Одним из таковых, и достаточно интересным, являются **SEH** - *structured exception handling* - обработчики исключений. Как всегда, мы не собираемся начинать здесь с азов. Также учтите, мы расписываем только исключения, пришедшие из **user mode** , т.е. **ring-3**. Если честно - сеть исключительно бедна сколь-нибудь хорошей информацией, приятное исключение, как всегда, **Мэтт Питрек** со своей изумительной статьей http://www.microsoft.com/msj/0197/exception/exception.aspx. Если хочется солидной теории – можно глянуть http://www.codeproject.com/cpp/exceptionhandler.asp Также немножечко об этом рассказывает Руссинович, как всегда, на высоте *Gary Nebbett* , да еще с миру по нитке можно найти в качестве предисловия к некоторым книгам по разработке драйверов. Информации по использованию **SEH** для облегчения жизни нам при отдирании пакера вообще практически нет - чуть-чуть в некоторых китайских статьях, да на форуме fravia (http://66.98.132.48/forum/) что-то проскакивало. Итак, предполагается, что вы внимательнейшим(!) образом прочли статью Питрека. Теперь взгляните на рисунок:

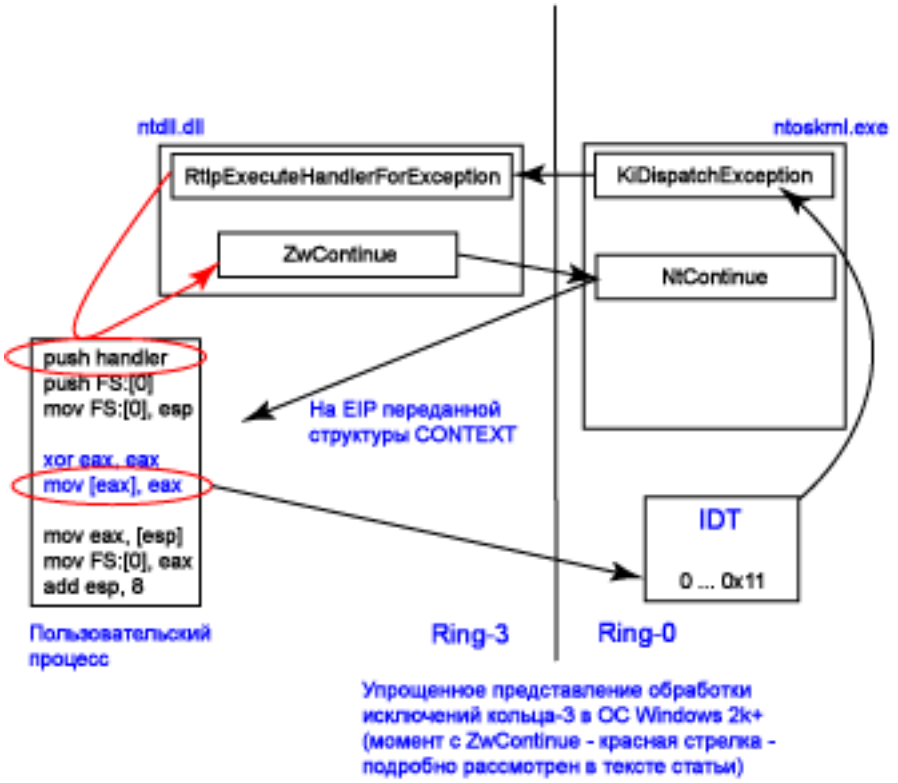


рис 5

Если что-то не ясно - не волнуйтесь, мы детально рассмотрим большинтсво случаев на протяжении этой главы. С оставшимися вы уже сумеете расправиться самостоятельно.

Не имеет смысла особо глубоко вдаваться в тонкости обработки исключений процессором Intel и реализацией этого дела в Win 2k+ (мы рассмотрим это в практическом примере). Достаточно сказать, что в один прекрасный момент времени управление получает функция **KiDispatchException** (ntoskrnl.exe), псевдокод которой можно найти в исходниках **ReactOS** (см. первую часть статьи) или в книге Неббета. А уж эта функция и будет решать, что делать - либо отладчику управление передавать, либо **SEH** -фреймам программы, либо еще что-то. Обработчики **SEH** , по сути, являются односвязанным списком (Windows никуда и не убегала от фундаментальных типов хранения данных). А это означает многое... Например, ничто не мешает

перестроить список, добавив туда свой обработчик. Или, к примеру, слегка изменить тело узла, список это позволяет. Правда, **SEH** -списки контекстно-специфичны, ну да и что с того? Список **SEH** -обработчиков немножко нестандартный, вместо указателя на следующий узел, он содержит указатель на узел предыдущий (своего рода стек на основе списка), вот так:

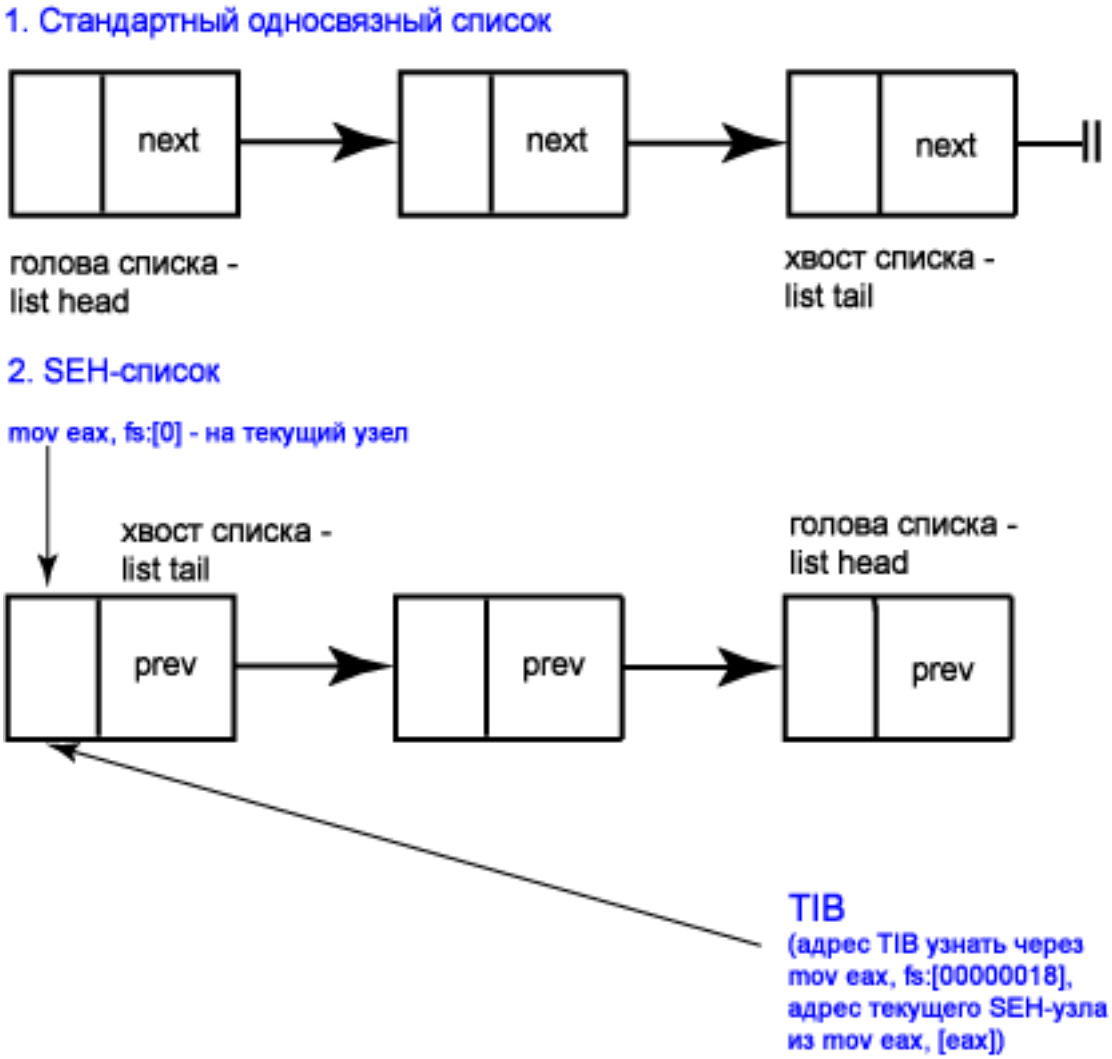


рис 6

Можно с полным на то основанием считать использование заведомо сбойных участков кода в **SEH** -кадрах самым настоящим антиотладочным приемом, который одинаково хорошо работает и как против **ring-0** отладчика, и как против **ring-3** отладчика. Автор упаковщика встраивает код, который заведомо даст сбой, в **SEH** -кадр. Примерно так (код из статьи Питрека очень нагляден):

```
__asm
{/* построить запись EXCEPTION_REGISTRATION:*/
    push    handler      // адрес функции-обработчика исключения
    push    FS:[0]       // адрес предыдущей функции-обработчика
    mov     FS:[0],ESP   // новая запись EXCEPTION_REGISTRATION
}

__asm
{
   db F1;   //такого опкода нет,
            // следовательно, произойдет исключение UD
}

__asm
{/* удалить нашу EXECEPTION_REGISTRATION: */
    mov     eax,[ESP]      // получить указатель на предыдущую запись
```

```
    mov     FS:[0], EAX    // установить в списке эту запись
    add     esp, 8         // удалить нашу EXECEPTION_REGISTRATION из стека
}
```

Что произойдет, когда управление получит код во втором **__asm** блоке? Произойдет исключение **UD** - *invalid opcode* (которое, как известно, сопоставлено с прерыванием **int 06** в **IDT**), и управление получит обработчик **_KiTrap06** , что в **ntoskrnl.exe** :

```
:idt
Int   Type     Sel:Offset     Attributes Symbol/Owner
IDTbase=80036400  Limit=07FF
...
0005  IntG32   0008:80466D06  DPL=0  P   _KiTrap05
0006  IntG32   0008:80466E6A  DPL=0  P   _KiTrap06

:u 08:80466E6A
_KiTrap06
0008:80466E6A  F744240800000200    TEST      DWORD PTR [ESP+08],00020000
0008:80466E72  0F849F030000        JZ        80467217
0008:80466E78  60          PUSHAD
0008:80466E79  B823000000          MOV       EAX,00000023
0008:80466E7E  668ED8              MOV       DS,AX
0008:80466E81  668EC0              MOV       ES,AX
```

Далее управление уйдет в **KiDispatchException** - мы не рассматриваем этот момент подробно. Потом управление уйдет к кольцо-3 - **KiUserExceptionDispatcher** (**ntdll.dll**), чтобы никогда более не вернутся в кольцо-0 (ложь, подробнее см. ниже). Функция **RtlDispatchException** подхватит управление дальше. Учтите, что возврат в функцию **KiUserExceptionDispatcher** может и не произойти (это ОЧЕНЬ важный момент, подробнее ниже). В нашем примере адрес обработчика выглядит так:

```
:xframe
xFrame    xHandler    xTable        xScope
------    --------    ------        ------
0x12FFE0    0x7C4FF0B4    0x7C4E8EC8    00
     try/except (0000) filter=0x7C519C1C, handler=0x7C519C2D

0x12FFB0    0x411177    0x424078    00
     try/except (0000) filter=0x411E73, handler=0x411E8E
;именно 411177 и является адресом обработчика!
```

Давайте немного разберемся с тем, что здесь написано. Команду Soft-Ice **xframe** мы описывать не будем - для этого есть документация. В данном конкретном случае команда вывела отчет обо всех **SEH** -фреймах, зарегистрированных для данного контекста (не забываем, что мы имеем дело с односвязным списком). Обратите внимание, самый первый **SEH** начинается из **TIB**. В SoftIce выполните команду **thread -x** *имя_процесса* :

```
;почему бы не взять файл, запакованный telock
:thread -x telock
             Extended Thread Info for thread 5B4
    KTEB:      FDA94A20  TID:       5B4  Process:   telock(488)
  ...
    UTEB:      7FFDE000  Except Frame:  0006FFE0  Last Err: 0000007E
;трассируем дальше и доходим до установки нового узла в списке SEH
:thread -x telock
             Extended Thread Info for thread 5B4
    KTEB:      FDA94A20  TID:       5B4  Process:   telock(488)
  ...
    UTEB:      7FFDE000  Except Frame:  0006FF74  Last Err: 0000007E
;обратите внимание - адрес изменился!
;Давно известен главный недостаток односвязного списка – невозможность
;быстрого доступа к его произвольному элементу, например,
;хвосту списка, поэтому разработчики Windows поступили
;правильно – держится указатель на хвост, что повышает эффективность
;при добавлении новых элементов
```

Поле **UTEB** показывает адрес памяти, по которому в ring-3 расположен TIB, а поле Except Frame показывает адрес ПОСЛЕДНЕГО узла в списке **SEH** (к **SEH** можно также подступится не только через FS:[0], но и через **TIB** (FS:[18]) в которой, помимо указателя на хвост списка **SEH** , хранится линейный адрес самой структуры TIB, который можно использовать для доступа к ее полям). Потрассируйте программу, использующую SEH, и обратите внимание на изменение адреса в этом поле.

Самый первый **SEH** -обработчик ставит недокументированная функция **BaseProcessStart** (описывается и у Питрека, и у Рихтера), а роль последнего оплота выполняет функция **kernel32.dll UnhandledExceptionFilter**. Таким образом мы видим, что НИ ОДНО приложение Windows не существует без обработчика исключительных ситуаций по умлочанию (**xframe** даже определила его тип - try/except, что точно соответствует действительности). Некоторые вирусы используют этот факт для поиска kernel32.dll в памяти. Soft-Ice ставит API-hook (т.е., просто-напросто, лепит опкод CC) на **UnhandledExceptionFilter**. Немного забавно то, что отладчик нулевого кольца вынужден ставить перехват функции кольца-3, но, как мы уже упоминали, исключение 3-го кольца сначала обрабатывается в ядре, а потом возвращается назад. Т.е. когда управление попадает в ring-3, и начинают вызываться последовательно обработчики **SEH** , то управление в **ring-0** , ВОЗМОЖНО, уже не возвращается. Если ни один из обработчиков ошибку не обработал, то вызывается **UnhandledExceptionFilter** , которая в системе без Soft-Ice после вывода посмертного сообщения просто прибивает процесс. Некоторые пакеры (например, **Ultra Protect**) проверяют наличие CC в начале этой функции - вот и еще один метод поимки **SoftIce** (**IceExt** отсекает этот метод, просто запрещая ставить **SoftIce** CC на эту функцию).

Сам обработчик, в нашем случае может просто увеличить **EIP** и вернуть управление назад, так, чтобы перескочить **F1**. И вот теперь мы, собственно, добрались до кульминационного момента. Когда **Windows** вызывает обработчик исключения, ему (обработчику) становится доступной структура **CONTEXT** , дающая возможность менять все общедоступные регистры процессора, включая **DR0-DR7**.

Итак, заглянув в документацию **Intel** , вы теперь знаете, что единственная, доступная команда для работы с **DR** -регистрами - это **mov**. И в 3-кольце инструкция **mov drx, x** недоступна! Следовательно, запрос должен уйти в ядро. И этот факт путает многих. Часто встречается утверждение, что обработчик **SEH** работает в нулевом кольце. Чушь (9x не рассматривается)! Но вот структура **CONTEXT** , возможно измененная обработчиком, действительно предается в **ntoskrnl** , где и происходит все самое важное. Еще раз обратимся к статье Питрека:

```
// заметьте, RtlDispatchException может НЕ вернуть управление

  if ( RtlDispatchException( pExceptRec, pContext ) )

 retValue = NtContinue( pContext, 0 );

  else

 retValue = NtRaiseException( pExceptRec, pContext, 0 );
```

Вопрос: когда **RtlDispatchException** возвратит управление, а когда нет? Ответ заключается в самом обработчике прерывания. Если обработчик имеет вид:

```
...

do_something;

...

ret;
```

то, очевидно, управление будет передано назад. Если же имеем обработчик вида:

```
...
```

```
do_something;

...

jmp;
```

то **RtlDispatchException** в **ntdll.dll** управление не вернет! Что это означает? Да то, что если обработчик, к примеру, намеренно изменил **DR** -регистры через структуру **CONTEXT** , то и что с того? Да ничего и не произойдет. Кроме того, есть еще и другая причина. За обработчики, не возвращающие управление в **ntdll** , надо, в буквальном смысле слова, отрывать уши! Почему? Да потому что при таком раскладе список **SEH** перестанет быть сбалансированным - появятся две новые записи, а именно –

1) запись от **RtlpExecuteHandlerForException** , которая тоже использует **try/catch** ,

2) запись пользовательского обработчика. Т.е. список будет выглядеть так:

```
:xframe
xFrame    xHandler    xTable        xScope
------    --------    ------        ------
0x06FFE0    0x7C4FF0B4    0x7C4E8EC8    00
     try/except (0000) filter=0x7C519C1C, handler=0x7C519C2D

0x06FF74    0x101A9FE    0x42F042E    16886162
//неубранная запись пользовательского обработчика
// (scope table inaccessible)


0x06FBE8    0x77F951B6    0x06FF74    457852

//неубранная запись обработчика ntdll в RtlpExecuteHandlerForException
//"я бы этому дяде с большими ушами, уши бы-то пооткрутил!"
// (scope table inaccessible)
```

Итак, вывод. Если обработчик намеренно меняет **DR** -регистры, то управление должно вернуться в **ntdll.dll** и быть передано функции **NtContinue**. Теперь мы вплотную подошли к этой любопытной функции. Из функции, реально, документирован только прототип. В кодах **ReactOS** по поводу **NtContinue** наблюдается полное отсутствие всякого присутствия. Следующий псевдокод получен путем дизассемблирования и анализа, как всегда:

```
#define KernelMode  0
#define UserMode    1

/*Нас ни в малейшей степени не интересует возня с
IRQL и спин-блокировками, поэтому далее демонстрируется только
действительно важный, с нашей точки зрения, код.
Так же учтите, что опущены многие проверки и просто куски кода - в нашу
задачу не входило давать полный разбор NtContinue!*/

NtContinue(PCONTEXT pcontext, bool bAlert)
/*NtContinue из ntoskrnl.exe – Windows 2000 SP 4*/
{
    /*указатель на структуру KTRAP_FRAME - принадлежит KTHREAD
    в кодах ReactOS структура KTRAP_FRAME определена в ke.h*/
    PKTRAP_FRAME pframe;
    /*KTHREAD можно использовать либо из кодов ReactOS, либо
    из кодов w2k_def.h, созданных Шрайбером и выложенных на wasm*/
    KTHREAD kth; /*mov ebx, ds:0FFDFF124h ; KTHREAD*/
    /*однако KPCR/KPRCB предпочтительнее использовать из DDK - как-никак
    официальная документация, хоть и не полная!*/
    KPCR kp;

    if (!KiContinue(pcontext, 0, pframe))
    {
        if (bAlert == TRUE)
            KeTestAlertThread(kth.PreviousMode);

        /*KiServiceExit2*/
```

```
    __asm cli;
    if(pframe->Eflags & 0x20000 || !(pframe->Cs & 1))
        while(kth.ApcState.UserApcPending)
        {
            ... /*KiDeliverApc*/
        }

    kp.NtTib.ExceptionList = pframe->ExceptionList;
    if (!(kp.DebugActive & 0x0F)) /*mov ebx, large fs:50h ; KPCR.DebugActive*/
    {
        /*вот это невероятно важный момент - см. текст статьи!*/
        if(pframe->Eflags & 0x20000 || !(pframe->Cs & 1))
        /*важность настолько велика,
         что не поленимся привести ассемблерный листинг:
        .text:00465942        test    [ebp+KTRAP_FRAME.EFlags], 20000h
        .text:00465949        jnz     short to_dr2
        ;to_dr2 соответствует коду в блоке _asm
        .text:0046594B        test    [ebp+KTRAP_FRAME.SegCs], 1
        .text:00465952        jz      short loc_4658F5
        */
        {
            _asm
            {
       /*иными словами, все то, что трудолюбиво наделано
       в KeContextToKframes запихивается в DR-регистры именно тут,
       т.к.единственная дозволенная инструкция
       для работы с DR-регистрами - mov!*/
       mov     ebx, 0
       mov     esi, [ebp+KTRAP_FRAME.Dr0]
       mov     edi, [ebp+KTRAP_FRAME.Dr1]
       mov     dr7, ebx
       mov     dr0, esi
       mov     ebx, [ebp+KTRAP_FRAME.Dr2]
       mov     dr1, edi
       mov     dr2, ebx
       mov     esi, [ebp+KTRAP_FRAME.Dr3]
       mov     edi, [ebp+KTRAP_FRAME.Dr6]
       mov     ebx, [ebp+KTRAP_FRAME.Dr7]
       mov     dr3, esi
       mov     dr6, edi
       mov     dr7, ebx
            }
        }
    }
    else
        ... /*перезагрузка сегментных регистров
    и общих регистров из KTRAP_FRAME*/
    __asm iretd;

}
else /*KiServiceExit*/
{
    __asm cli;
    if(pframe->Eflags & 0x20000 || !(pframe->Cs & 1))
        while(kth.ApcState.UserApcPending)
        {
            /*KiDeliverApc*/
        }
    kp.NtTib.ExceptionList = pframe->ExceptionList;
    if (!(kp.DebugActive & 0x0F)) /*mov ebx, large fs:50h ; KPCR.DebugActive*/
    {
        if(pframe->Eflags & 0x20000 || !(pframe->Cs & 1))
        {
            __asm
            {
       mov     ebx, 0
       mov     esi, [ebp+KTRAP_FRAME.Dr0]
       mov     edi, [ebp+KTRAP_FRAME.Dr1]
       mov     dr7, ebx
       mov     dr0, esi
       mov     ebx, [ebp+KTRAP_FRAME.Dr2]
```

```
           mov     dr1, edi
           mov     dr2, ebx
           mov     esi, [ebp+KTRAP_FRAME.Dr3]
           mov     edi, [ebp+KTRAP_FRAME.Dr6]
           mov     ebx, [ebp+KTRAP_FRAME.Dr7]
           mov     dr3, esi
           mov     dr6, edi
           mov     dr7, ebx
                }
            }
        }
        else
            ... /*перезагрузка сегментных регистров
                и общих регистров из KTRAP_FRAME*/

        __asm sti;
        __asm sysexit;
        __asm iretd;
    }
}

KiContinue(PCONTEXT pcontext, ???, PKTRAP_FRAME pframe)
{
    KTHREAD kth; /*mov eax, large fs:124h*/

    if (kth.PreviousMode == UserMode)
        KiContinuePreviousModeUser(pcontext, 0, pframe, kth.PreviousMode);
    else
        KeContextToKframes(pframe, 0, pcontext, pcontext->ContextFlags);
}

KiContinuePreviousModeUser(PCONTEXT context,
                  ???, PKTRAP_FRAME pframe,
                  bool PreviousMode
                  )
{
    PCONTEXT buff;

    if(!(context & 3))    //выровнено на границу DWORD?
        ExRaiseDatatypeMisalignment();
    else
    {
        /*довольно глупая проверка*/
if((pContext + sizeof(CONTEXT)) > MmUserProbeAddress)
            ExRaiseAccessViolation();
        memmove(buff,pcontext,sizeof(CONTEXT));
        KeContextToKframes(pframe, 0, buff, pcontext->ContextFlags);
    }
}

/*функция производит большую работу по трудолюбивому перекладыванию содержимого
из структуры CONTEXT в структуру KTRAP_FRAME, попутно делая громадное количество
проверок, поэтому код достаточно тривиален и полностью рассмотрен не будет, нас
интересует, главным образом, тот клочок KeContextToKframes, что отвечает за
перенос DR-содержимого из CONTEXT в KTRAP_FRAME*/

KeContextToKframes(PKTRAP_FRAME pframe, ???, PCONTEXT pcontext, unsigned flags)
/*к слову сказать, функция SetThreadContext, в конечном итоге, тоже приходит сюда
 – все пути ведут в Рим*/
{

    ...
    if(pcontext->ContextFlags)
        (pcontext->Dr0 > MmHighestUserAddress)
            ?(pframe->Dr0 = 0):
            (pframe->Dr0 = pcontext->Dr0);
/*
.text:00430777        mov     ecx, [esi+CONTEXT.Dr0]
.text:0043077A        cmp     MmHighestUserAddress, ecx
.text:00430780        sbb     eax, eax
.text:00430782        not     eax
```

```
.text:00430784        and     eax, ecx
*/
    if(pcontext->ContextFlags)
        (pcontext->Dr1 > MmHighestUserAddress)
             ?(pframe->Dr1 = 0):
             (pframe->Dr1 = pcontext->Dr1);
    if(pcontext->ContextFlags)
        (pcontext->Dr2 > MmHighestUserAddress)
             ?(pframe->Dr2 = 0):
             (pframe->Dr2 = pcontext->Dr2);
    pframe->Dr6 = pcontext->Dr6 & 0xE00F;
/*1110000000001111 – B0-B3, BS, BD, BT-флаги*/
    pframe->Dr7 = pcontext->Dr7 & 0x155;
/*101010101 – L0-L3, LE-флаги – особое внимание на LE-флаг
 – контекстные точки останова*/
/*
.text:004305AA        mov     eax, [esi+CONTEXT.Dr7]
.text:004305AD        and     ax, 155h
...
.text:004305B3        mov     [ebx+KTRAP_FRAME.Dr7], eax
*/
/*чтобы понять смысл – глава 15 3-го тома Intel Manual
 – “debugging and performance monitoring”*/

//хм, медвежья услуга, однако!
KTHREAD.DebugActive = CONTEXT.Dr7 & 0x55 ? 1 : 0;

/*
.text:004305B8        mov     eax, large fs:124h
.text:004305BE        mov     ecx, eax
.text:004305C0        mov     eax, [esi+CONTEXT.Dr7]
.text:004305C3        and     al, 55h
.text:004305C5        neg     al
.text:004305C7        sbb     eax, eax
.text:004305C9        neg     eax
.text:004305CB        mov     [ecx+KTHREAD.DebugActive], al
*/
    ...
}
```

Если приведенный выше кусок псевдокода привел вас в ужас - ничего. Все вполне можно уложить в пару фраз. Итак, так или иначе, управление будет передано в **KeContextToFrames** , единственная задача которой - переложить содержимое полученной извне (это может быть и кольцо-0 в случае драйвера!) структуры **CONTEXT** в структуру **KTRAP_FRAME**. Заметьте, структура **KTRAP_FRAME** еще не означает **DR** -регистры. Код, перекладывающий содержимое **KTRAP_FRAME** , полученное после **KeContextToFrames** , расположен в **NtContinue** , а точнее, либо в области **KiServiceExit** , либо в области **KiServiceExit2**. Внимательнейшим образом рассмотрите инлайн-ассемблер и, самое главное, предшествующую ему проверку.

Что это означает для нас с вами? Смотрите, у пакера нет выбора. Если пакеру необходимо изменить **DR** -регистр, он ДОЛЖЕН передать управление в ядро, где программа кольца-3 не в силах проконтролировать установку точек останова, и момент передачи управления на код, работающий с **DR** -регистрами, может быть нами подправлен как нам заблагорассудится!

Очевидно, что если переделать в абсолютный **jmp** один условный переход, то **DR** -регистры останутся неискаженными, следовательно, мы свободно сможем пользоваться нашей любимой командой **bpm**. И все – наполнение мусором **DR** -регистров (как это любит делать **asprotect**) нам теперь до лампочки. Неужели все так просто? К сожалению, нет. Да, действительно, нет никаких принципиальных сложностей в изменении условных переходов (скажем, можно привинтить в **IceExt** команду, которая будет ставить подобную защиту при необходимости и снимать ее, когда отпадет надобность), но и авторы упаковщиков даже не даром свой хлеб едят. Что вы скажете о защите, которая сама ставит точки останова, используя **DR** -регистры, т.е. трассирует сама себя? Т.е., если **DR** -содержимое очищено или оставлено без изменений с помощью трюка с **NtContinue** , точка

останова не сработает и программа узнает, что дело плохо. Вероятно, в общем случае, разработать решение этой проблемы не удастся... Подумайте, возможно, именно вы сумеете разобраться как можно проэмулировать этот трюк, если это вообще реализуемо.

Теоретическая глава почти закончена. Единственная вещь, не рассмотреная нами и представляющая определенный практический интерес, называется **scope table**. Как уже говорилось, **Visual C++** ставит дополнительные данные в **EXCEPTION_REGISTRATION** - добавляются некоторые структуры (конкретнее - см. файл **exsup.inc**). Далее обратите внимание на код **startup** процедуры в **crt0.c** (стандартном startup-коде) - он обернут в **_try/catch**. Причем этот **_try** уже идет с дополнительными структурами, т.е. появляется **scope table**. Скажем, довольно несложно разработать драйвер, который будет трассировать пакер вплоть до момента появления **scope table** , которое должно означать, что начал исполняться пользовательский код. С другой стороны, что помешает пакеру установить ложную **scope table**... С третьей стороны - можно просто выдать пользователю отчет, а уж он пусть сам разбирается, что к чему... С четвертой стороны, есть еще **Borland** и **Watcom** со своей реализацией, с пятой ... И так до бесконечности.

Спешим «порадовать» еще одной «приятной» новостью – фирма **MS** в **Windows XP** ввела так называемые векторные исключения – **VEH** , имеющие приоритет над **SEH** , т.е., выполняющиеся перед ними. Умница Питрек и тут не подкачал – читайте его статью.

Пока нам не доводилось встречать программы, которые бы игрались с этим приемом, однако, когда придет и их время, полагаем, у вас теперь найдется, что им сказать.

Немного об антиотладке – в преддверии telock

написано совместно с Four-F

Практически в любой статье по снятию упаковщика рассказано о трюке применения **int 1** в SEH-кадре и том, что это помогает обнаружить **Soft-Ice**. Однако ни слова не сказано о том, ПОЧЕМУ И КАК это происходит. Давайте попробуем разобраться. Хотелось бы сразу сказать, что лезть придется глубоко – в ядро **Windows**. Итак, поехали!

Очевидно, чтобы избежать антиотладочных фокусов, нужно понимать, что такое отладка и какие средства в нашем распоряжении есть. А это требует чертовски хорошего понимания защищенного режима и обработки прерываний защищенного режима. Значит – интеловские талмуды!

Итак, вполне понятно, что **int 1** в качестве антиотладки можно использовать только в **SEH** -кадрах, иначе не выживет сама программа. Стандартно можно поступить так:

```
push    handler
push    FS:[0]
mov     FS:[0],ESP
int 1
;само по себе использование int1
; – это еще НЕ антиотладка, антиотладка будет дальше
pop dword prt FS:[0]
add     esp, 4
```

Более изощренный код, в котором адрес обработчика исключения уже не столь очевиден, может выглядеть так:

```
CODE:00401000        call    here
...
here:
; в этом случае весь фокус сводится к действию call,
; загоняющей в стек адрес возврата –
; обработчик расположен по адресу 401005 – см. ниже.
CODE:0040101C        push    dword ptr fs:0
CODE:00401022        mov     fs:0, esp
```

```
CODE:00401028        int     1
```

Так же учтите, что удаление **SEH** -кадра тоже можно проделывать не только через pop, а и прямыми манипуляциями со стеком – mov esp … Чтобы не поплыть во всем этом, надо четко представлять себе состояние стека в обработчике прерывания. На эту тему есть одна очень хорошая статья – http://win32assembly.online.fr/Exceptionhandling.html - не поленитесь внимательно посмотреть картинки, или смотрите программу на C ниже в этой же главе. Кстати, переполнение стека – стандартный способ проникновения в систему – можно вызывать и из обработчика **SEH**. Читните http://www.peterszor.com/blended.pdf - очень познавательно.

А мы, тем временем, продолжим разбор обработчика с **int 1**. Очень внимательно рассмотрите этот код:

```
      push xHandler
      assume fs:nothing
      push dword ptr fs:[0]
      mov    fs:[0], esp
      assume fs:error

  Int1::
      int 1

      ; Here if i1here is on
  ExceptionNotCatched:
      MessageBox - Exception not catched = i1here is on!
      jmp на выход

  NticeFound::
      MessageBox - NtIce found
      jmp на выход

  NticeNotFound::
      MessageBox - NtIce not found
      jmp на выход

      ; Remove seh-frame
      assume fs:nothing
      pop fs:[0]
      add esp, sizeof DWORD
      assume fs:error

  ;
  ; обработчик исключения - xHandler
  ;

  mov ecx, pContext
  mov eax, [pContext.Eip]    ;Eip структуры CONTEXT
  cmp eax, Int1
  ;вот в этом cmp весь и смысл!
  ;Поведение процессора(!) будет различаться при установленном Soft-Ice и без него!
  ;см. текст статьи!
  jnz els
  ; SoftIce неактивен
  mov [pContext.Eip], NticeNotFound
  jmp rt
  els::
  ; SoftIce активен
  mov [pContext.Eip], NticeFound
  rt::
  ; передать управление на NtContinue
  xor eax, eax
  ret
```

В системе БЕЗ Soft-Ice **int 1** имеет DPL = 0 (если с **IDT** чувствуете себя неуверенно – хорошая ссылка: http://www.acm.uiuc.edu/sigops/roll_your_own/i386/idt.html):

```
[o] INT 00000001 Location: 00000008:80466786 DPL: 00000000 Type: 32bit Interrupt Gate
```

```
; адрес показывает на KiTrap01, что в ntoskrnl
```

Быстренько вспоминаем, что происходит, когда задача с меньшим **DPL** пытается обратится к сегменту с большим **DPL**? Правильно – нарушение защиты **#GP**! В системе БЕЗ **Soft-Ice int 1** имеет **DPL** = 0 и, в случае намеренного (хм, или ненамеренного) сбоя в программе кольца-3 (**DPL** = 3), будет вызван обработчик **KiTrap0D** , выполняющий следующие действия:

```
/*KiTrap0D*/
...
; вычислить адрес сбойной инструкции
; и положить его на вершину стека
.text:00467ECF        mov     eax, [esp+KTRAP_FRAME.SegCs]
.text:00467ED3        shl     eax, 4
.text:00467ED6        add     eax, [esp+KTRAP_FRAME.Eip] ; cs+eip
.text:00467EDA        xor     edx, edx
.text:00467EDC        mov     ecx, ss:[eax]
.text:00467EDF        mov     dl, cl
.text:00467EE1        mov     dl, ss:OpcodeIndex[edx]
; массив из DWORD – наверняка будет
; любопытен для любого специалиста по опкодам
.text:00467EE8        jmp     ss:off_473F04[edx*4]
...
```

в конечном результате которых мы имеем код **0xC0000005**(!) в **ExceptionRecord- >ExceptionCode**! Еще раз – произошло **int 1** , а имеем – **access violation**!

Что же произойдет в системе, где **Soft-Ice** установлен?

```
[o] INT 00000003 Location: 00000008:F404EF87 DPL: 00000003 Type: 32bit Interrupt Gate
; изменился DPL обработчика и АДРЕС – об адресе см. ниже
```

Т.о. DPL обработчика **int 1** изменился. Что это значит? В системе с Soft-Ice результат будет другим. Ice меняет DPL int 1 с 0 на 3, в результате чего **access violation** более НЕ происходит, **KiTrap0D** НЕ вызывается, а имеем **KiTrap01** – **0x80000004** - **STATUS_SINGLE_STEP**! Поведение процессора тоже будет отличаться. Теперь **EIP** показывает не на инструкцию, вызвавшую **access violation** , а на инструкцию, следующую после нее! Более подробно – интеловские мануалы – разница между **trap** и **fault**.

```
/*KiTrap01*/
.text:004667EF        and     [ebp+KTRAP_FRAME.EFlags], 0FFFFFEFFh

;eip показывает на инструкцию ПОСЛЕ int 1
.text:004667F6        mov     ebx, [ebp+KTRAP_FRAME.Eip]
.text:004667F9        mov     eax, STATUS_SINGLE_STEP
.text:004667FE        jmp     to_CommonDispatchException
;далее управление уйдет на KiDispatchException,
;которая передаст его по подправленному адресу назад
```

Ну а далее, можно либо проверять флаг, либо переориентировать **EIP** через структуру **CONTEXT**.

Нет нужды верить на слово. Все элементарно можно проверить. Вот программа:

```
#include <windows.h>
#include <stdio.h>

EXCEPTION_DISPOSITION
__cdecl
_except_handler(
    struct _EXCEPTION_RECORD *ExceptionRecord, //+0x4
    void * EstablisherFrame,           //+0x8
    struct _CONTEXT *ContextRecord,             //+0xC
    void * DispatcherContext )            //+0x10
{

    printf("STATUS: %x, EIP: %x\n",
           ExceptionRecord->ExceptionCode,
           ContextRecord->Eip
           );
```

```
/*в случае Soft-Ice EIP показывает на mov eax,[ESP] и статус 80000004,
 без Soft-Ice EIP показывает на int 1 и статус c0000005*/

//раскомментируйте строчку ниже для системы БЕЗ Soft-Ice
//ContextRecord->Eip += 2; /*проскочить int 1 и встать на mov eax,[ESP]*/

    return ExceptionContinueExecution;
}


int main(void)
{
    unsigned long handler = (unsigned long)_except_handler;

    __asm
    {
        push    handler
        push    FS:[0]
        mov     FS:[0],ESP
        int 1
        mov     eax,[ESP]
        mov     FS:[0], EAX
        add     esp, 8
    }

    return 0;

}
```

Защитится от этого можно элементарно – пропатчив **IDT** или поставив IceExt, который восстановит **DPL** int 1 в прежнее состояние.

Что до **int 3** – то само по себе использование **int 3** в **SEH** -кадрах не есть антиотладка – статус будет всегда одним и тем же – **STATUS_BREAKPOINT**. Специфичной антиотладкой **int 3** становится тогда, когда к нему добавляются переданные в ax, dx и т.п. параметры. Например, известный трюк с командой **HBOOT**. Если в **Soft-Ice** выполнить данную команду, произойдет мнгновенная перезагрузка. Того же эффекта можно достичь через **backdoor** интерфейс, что, к примеру, описано у **Hex'a** (проект *xtin.org*). Если помните, мы говорили, что **int 3** обязательно должен иметь опкод **CC** и никакого другого. Windows явно рассчитывает на это. Кусок кода из **KiTrap03** тому примером:

```
/*KiTrap03*/
.text:00466AE2        mov     ebx, [ebp+KTRAP_FRAME.Eip]
.text:00466AE5        dec     ebx
;заметьте, предполагается использование ОДНОБАЙТНОЙ инструкции – eip –
;и это еще раз подтверждает, что MS не
;готова к использованию CD 03 в качестве опкода,
;а лишь понимает CC – это явно указано во 2 томе интела
.text:00466AE6        mov     ecx, 3
.text:00466AEB        mov     eax, STATUS_BREAKPOINT
.text:00466AF0        call    CommonDispatchException
;далее управление уйдет на KiDispatchException,
;которая передаст его по подправленному адресу назад, и, если
;опкод был CD 03 – это вызовет повторное исключение
```

Кстати, Soft-Ice тоже лжет. Выполните команду **idt** и вы увидите следующее:

```
...
0001  IntG32   0008:80466786  DPL=3  P   _KiTrap01
0002  IntG32   0008:0000145E  DPL=0  P
0003  IntG32   0008:80466A5E  DPL=3  P   _KiTrap03
...
```

т.е., якобы, ничего не изменилось. В действительности же имеем:

```
[o] INT 00000001 Location: 00000008:F404EF69 DPL: 00000003
                Type: 32bit Interrupt Gate
[o] INT 00000002 Location: 00000008:F404EF78 DPL: 00000000
                Type: 32bit Interrupt Gate
```

```
[o] INT 00000003 Location: 00000008:F404EF87 DPL: 00000003
               Type: 32bit Interrupt Gate
```

т.е. Soft-Ice перехватывает **int 1/int 3** (кстати, не только их!) на свои заглушки, а это значит - не стоит всегда верить и **Soft-Ice**. И вообще, начитавшись всего этого, недолго и параноиком стать. Все врут.

Практический пример: tElock

Эта цель отнюдь не является легкой. В качестве антиотладки очень активно используются **SEH** -кадры, набитые всякой гадостью. Однако, используемые приемы однотипны. Так, вся защита программы построена на одном веселом принципе. Смотрите:

```
0008:01019DFE  8B642408            MOV       ESP,[ESP+08]
0008:01019E02  EB0D                JMP       01019E11
0008:01019E04  33DB                XOR       EBX,EBX

;начинаем плясать отсюда

0008:01019E06  64FF33              PUSH      DWORD PTR FS:[EBX]
0008:01019E09  648923              MOV       FS:[EBX],ESP
0008:01019E0C  F1          INVALID
0008:01019E0D  F7F3                DIV       EBX
;это вызывает исключение и управление переходит в ядро,
;что нам, в данном случае, ни к чему
;поэтому используем команду xframe для определения
;адреса обработчика
; – она работает безукоризненно
;обработчик, в данном случае, расположен по адресу 01019DFE
; – внимательно посмотрите куда ведет безусловный ;переход!
0008:01019E0F  EBE8                JMP       01019DF9
0008:01019E11  EB02                JMP       01019E15
;опять в обработчик! Если вы внимательно читали главу о SEH,
;то уже должно быть ясно,
;что оставлять в списке SEH всякое барахло нельзя,
;поэтому программа должна убрать обработчик
0008:01019E13  FF20                JMP       [EAX]
0008:01019E15  83F833              CMP       EAX,33
;ну очень полезная инструкция – и таких в коде много – автор пытается запутать нас,
;перемешивая реальный код с мусором
0008:01019E18  2BC9                SUB       ECX,ECX
0008:01019E1A  648F01              POP       DWORD PTR FS:[ECX]
0008:01019E1D  59          POP       ECX
0008:01019E1E  EB01                JMP       01019E21
;обработчик корректно удален из списка, можно продолжать дальше
```

Что мы можем этому противопоставить? Очевидно в ядро, наш старый добрый знакомый, нам заходить ни к чему. Поэтому имеет смысл переориентировать **EIP** на безопасный клочок внутри **SEH** -кадра и продолжать двигаться дальше. Но это только в теории. На практике, **Soft-Ice** очень не любит такие конструкции, поэтому проходит через **SEH** -фреймы пешком очень не рекомендуется. Поскольку строение и принципы работы **SEH** теперь понятны, постарайтесь всегда узнавать адрес обработчика ДО вхождения в кадр и сразу переориентируйте **EIP** на него. Только не стоит забывать о такой вещи как балансировка стека и значения регистров. Иначе очень быстро допереориентируетесь...

Мы рассмотрим пример самотрассировки в **telock** – это высший пилотаж на **DR** -регистрах, а не просто тупое заполнение их мусором. Итак, приложение третьего кольца действительно может записать что-то в **DR** -регистры, но только не прямо, а опосредованно - через **SetThreadContext** или **SEH**. Значит, мы должны ожидать доступа по **CONTEXT.drx** и возврата управления в ядро. И это не заставит себя ждать. Вперед:

```
seg000:0101907A        call    $+5
seg000:0101907F        pop     ebp             ; ebp == eip
seg000:01019080        lea     eax, [ebp+46h]  ; eax = eip+0x46
seg000:01019083        push    eax             ; смещение на обработчик
seg000:01019084        xor     eax, eax
seg000:01019086        push    dword ptr fs:[eax]
seg000:01019089        mov     fs:[eax], esp
;лишь команды, помеченные жирным,
;являются значимыми – остальное мусор, но,
;зато, как используется!
seg000:0101908C        int     3               ; Trap to Debugger
seg000:0101908D        nop
seg000:0101908E        mov     eax, eax
seg000:01019090        stc                     ;первая hardware breakpoint
seg000:01019091        nop
seg000:01019092        lea     eax, ds:1234h[ebx*2]
seg000:01019099        clc                      ;вторая hardware breakpoint
seg000:0101909A        nop
seg000:0101909B        shr     ebx, 5
seg000:0101909E        cld                      ;третья hardware breakpoint
seg000:0101909F        nop
seg000:010190A0        rol     eax, 7
seg000:010190A3        nop                      ;четвертая hardware breakpoint
seg000:010190A4        nop
seg000:010190A5        xor     ebx, ebx         ;исключение #DE
seg000:010190A7        div     ebx
seg000:010190A9        pop     dword ptr fs:0
seg000:010190AF        add     esp, 4
seg000:010190B2        mov     si, 4647h
seg000:010190B6        mov     di, 4A4Dh
seg000:010190BA        mov     al, [ebp+99h]
seg000:010190C0        jmp     work_with_PE
;передать управление дальше – на работу с PE-файлом,
;здесь не рассматривается
```

int 3 вызывает исключение **STATUS_BREAKPOINT**. Управление переходит в ядро и возвращается на обработчик. В данном случае его адрес - **10190C5**. Заглянем туда:

```
;указатель на структуру ExceptionRecord
seg000:010190C5       mov     eax, [esp+4]
;указатель на структуру CONTEXT
seg000:010190C9       mov     ecx, [esp+0Ch]
seg000:010190CD       inc     [ecx+CONTEXT.Eip]
seg000:010190D3       mov     eax, [eax+EXCEPTION_RECORD.ExceptionCode]
seg000:010190D5       cmp     eax, STATUS_INTEGER_DIVIDE_BY_ZERO
seg000:010190DA       jnz     short loc_1019100
;первое исключение – int 3 не даст нам этот код возврата,
;поэтому в первый раз этот прыжок будет выполнен
;этот код еще долго не получит управления...
seg000:010190DC       inc     [ecx+CONTEXT.Eip]
;EIP теперь показывает на pop dword ptr fs:0 –
; корректный выход из SEH

; ExceptionContinueExecution
seg000:010190E2       xor     eax, eax

; Dr0-3 <- 0
seg000:010190E4       and     [ecx+CONTEXT.Dr0], eax
seg000:010190E7       and     [ecx+CONTEXT.Dr1], eax
seg000:010190EA       and     [ecx+CONTEXT.Dr2], eax
seg000:010190ED       and     [ecx+CONTEXT.Dr3], eax
seg000:010190F0       and     [ecx+CONTEXT.Dr6], 0FFFF0FF0h
; 11111111111111110000111111110000 - флаги B0-B3,
; BD, BS, BT - занулить
seg000:010190F7       and     [ecx+CONTEXT.Dr7], 0DC00h
; 1101110000000000 – флаг GD в 1, R/W в 1
seg000:010190FE       jmp     short ret
seg000:01019100 ; ---------------------------------------------
seg000:01019100
seg000:01019100 loc_1019100:                  ; CODE XREF: seg000:010190Dj
seg000:01019100       cmp     eax, STATUS_SINGLE_STEP
```

```
; опять таки, и это не следствие CC –
; эта ветка должна сработать 4 раза - после отработки всех точек останова
; т.к. процессор возбуждает int 1
seg000:01019105       jz      short loc_1019113
seg000:01019107       cmp     eax, STATUS_BREAKPOINT
; а вот это то, что произойдет после int 3
; случилось что-то, что мы не можем обработать ExceptionContinueSearch
seg000:0101910C       jz      short fill_dr
seg000:0101910E       push    1
seg000:01019110       pop     eax
seg000:01019111       jmp     short ret
seg000:01019113 ; ---------------------------------------------
seg000:01019113
seg000:01019113 loc_1019113:                  ; CODE XREF: seg000:01019105j
seg000:01019113     call    loc_1019119
seg000:01019113 ; ---------------------------------------------
seg000:01019118       db    4 ;
;этот байт здесь поставлен не случайно –
;модифицируется инструкцией по адресу 0101911A – self-modifying code
seg000:01019119 ; ---------------------------------------------
seg000:01019119
seg000:01019119 loc_1019119:                  ; CODE XREF: seg000:01019113p
seg000:01019119       pop     eax             ; eax = eip
seg000:0101911A       inc     byte ptr [eax]  ; изменить опкод по 01019118
seg000:0101911C       sub     eax, eax        ; ExceptionContinueExecution
seg000:0101911E       jmp     short ret
seg000:01019120 ; ---------------------------------------------
seg000:01019120
seg000:01019120 fill_dr:                ; CODE XREF: seg000:0101910Cj
;итак, ebp, как вы помните, используется тут
;для относительной адресации от адреса 101907f – см. вход в SEH
;обязательно проверьте эти значения –
;вернитесь к коду внутри SEH-кадра и посмотрите где сработают точки
;останова
seg000:01019120       mov     eax, [ecx+CONTEXT.Ebp] ;
seg000:01019126       lea     eax, [eax+24h]   ; ebp+24
seg000:01019129       mov     [ecx+CONTEXT.Dr0], eax    ;ставим DR0 на ebp+24
seg000:0101912C       mov     eax, [ecx+CONTEXT.Ebp]
seg000:01019132       lea     eax, [eax+1Fh]   ; ebp+1F
seg000:01019135       mov     [ecx+CONTEXT.Dr1], eax    ;ставим DR1 на ebp+1F
seg000:01019138       mov     eax, [ecx+CONTEXT.Ebp]
seg000:0101913E       lea     eax, [eax+1Ah]  ; ebp+1A
seg000:01019141       mov     [ecx+CONTEXT.Dr2], eax    ;ставим DR2 на ebp+1A
seg000:01019144       mov     eax, [ecx+CONTEXT.Ebp]
seg000:0101914A       lea     eax, [eax+11h]  ; ebp+11
seg000:0101914D       mov     [ecx+CONTEXT.Dr3], eax    ;ставим DR3 на ebp+11
seg000:01019150       xor     eax, eax
seg000:01019152       and     [ecx+CONTEXT.Dr6], 0FFFF0FF0h ;
; 11111111111111110000111111110000 – флаги B0-B3, BD, BS, BT - занулить
seg000:01019159       mov     [ecx+CONTEXT.Dr7], 155h
; 101010101 – G0, G1, G2, G3 - занулить
seg000:01019160
seg000:01019160 ret:            ; CODE XREF: seg000:010190FEj
seg000:01019160                 ; seg000:01019111j ...
seg000:01019160       retn      ; возвратить управление в ядро на NtContinue
                                ; код действия Windows – в eax
work_with_PE:
...
```

Если вы все еще с нами – продолжаем! Итак, обработчик **int 3** записывает в DR значения ВНУТРИ ТОГО ЖЕ САМОГО кадра **SEH**. Управление ОПЯТЬ передается в этот же самый злополучный кадр! Четыре раза должны отработать DR-регистры. После четвертого раза происходит исключение типа деление на ноль. И тут-то получает управление ветка, которую можно назвать «на дорожку». Правится **EIP** , во все **DR** -регистры прописываются нули, ставятся маски на управляющие DR6-7 и управление идет дальше – на метку **work_with_PE**. И это все! Больше работы с **DR** -регистрами в коде telock нет!

Немножко по поводу **DR** -регистров и **Soft-Ice** , который действительно является чудом. **Dr** -регистры явно не отображаются и принудительно могут быть выведены командой CPU. Записать в DR-регистры что-нибудь можно лишь опосредовано – через bpm с явным указанием одного из четырех (DR0-DR3). Кроме того, умничка **Soft-Ice** умеет отображать модельно-специфичные регистры – **MSR**. Для этого есть одноименная команда и более усовершенствованный вывод сообщений при срабатывании **bpm** :

```
Break due to BP 01: BPMB USER32!MessageBoxA X DR3  (ET=19.91 milliseconds)
  MSR LastBranchFromIp=0101A2D0 <- полезный адрес!
    MSR LastBranchToIp=77E38098
```

Читаем в интеловских талмудах – том 3 глава 15 “ *Debugging and performance monitoring* ”: “ *The LastBranchToIP and LastBranchFromIP MSRs are 32-bit registers for recording the instruction pointers for the last branch, interrupt, or exception that the processor took prior to a debug exception being generated* ”.

Так это же вообще шикарно! LastBranchFromIp – это « *откуда* », **LastBranchToIp** – это у нас «куда». Как это можно использовать? Ну... Встретите защиту вида:

```
/*здесь намеренное использование FPO – frame pointer omission –
 т.е., ebp отсутствует и команда ws не поможет*/
/*более подробно см. статью «Playing with the stack»
на http://www.codeproject.com или, к примеру, сие:
http://www.codeguru.com/misc/stackdumper.html */


MessageBoxA(“Да пошел ты...”); //угадайте, что произойдет,
                               // ДАЖЕ если у вас стоит bpx MessageBoxA?
ExitProcess();
```

и все сразу станет ясно...

Ну а что дальше? Дальше – еще веселее. Безумные **SEH** -кадры с инсталляцией головы списка наподобие:

```
seg000:01019B77                 mov     [esp+4], eax
seg000:01019B7B                 mov     fs:0, esp
```

и **int 68h** внутри. Однако, смеем надеяться, более проблем они вам не доставят! Да, они головоломны, да, код перенасыщен мусором, да, обилие зашифрованных сегментов – но дорогу осилит идущий! Самое главное – чисто пройти головоломный трюк с **DR**. Все. После него вы спокойно можете ставить **bpm**. И не слушайте всяких рассказов, что, мол, **bpm** в **telock** ставить нельзя. Можно!

Смело ставьте **bpm** *GetProcAddress x* (**telock** использует его напрямую, без всяких сканирований директории экспорта):

```
seg000:0101A574              call    dword ptr [ebp+40BAE0h] ; GetProcAddress
seg000:0101A57A
seg000:0101A57A loc_101A57A:            ; CODE XREF: seg000:0101A565j
seg000:0101A57A              inc     eax
seg000:0101A57B              dec     eax
seg000:0101A57C              jnz     short good_addr
...
seg000:0101A5B1 good_addr:              ; CODE XREF: seg000:0101A57Cj
seg000:0101A5B1              mov     [edi], eax
```

Собственно, не имеет смысла описывать этот цикл. Важно отметить – нет нужды во многочисленных плагинах под **ImpRec** , возьмите директорию отсюда. Остерегайтесь ставить **bpx** на **API** напрямую, используйте **bpm** или старый трюк с **bpx API+x** , т.к. **telock** проверяет вызываемые функции на CC в начале (разумеется, точки останова на **Native API** никто проверить не сможет). **bpm LoadLibraryA** и **bpm GetProcAddress** – наши отправные позиции. Используйте их просто как зацепку, т.к. нам нужна ЧИСТАЯ, НЕИЗМЕНЕННАЯ директория импорта. Куски кода

можно просто дампить **IceExt** по ходу дела и бинарные сегменты дизассемблировать прямо в **IDA**. Дамп – разбор кода, дамп – разбор кода. Можно, скажем, поступить и еще оригинальнее – известно, что в один прекрасный момент времени директория (да и не только она) является полностью разжатой – просто выполните в **Soft-Ice** поиск имен библиотек или предполагаемых функций (например, «kernel32.dll»), в памяти и разберитесь на глаз с диапазонами директории). Словом, дорогу осилит идущий.

На вашу совесть также мы оставляем трюк с мьютексами. После прочтения Джеффери Рихтера он не должен быть источником проблем.

Список литературы

Эта статья никогда не была бы написана, если бы не громадный ворох литературы, который был основательно перелопачен. В подавляющем большинстве – это англоязычные книги, лишь для нескольких из них был выполнен перевод. Поэтому, как это не прискорбно, знание языка ОБЯЗАТЕЛЬНО, если хотите быть в потоке.

1. Соломон, Руссинович, “Inside Windows 2000, 3rd edition” – это теория. Но какая! Здесь есть все. И, если сочетать ее с практическими книгами, вроде 2 и 3 – то результат будет. Есть русскоязычный перевод.
2. Шрайбер “Недокументированные возможности Windows 2000” – сугубо практическая книга, ценность которой... Мама родная – там все есть. И есть русскоязычный перевод.
3. Gary Nebbett “Windows NT/2000 Native API Reference” – English only. Перевода нет. Книга описывает прототипы и очень многие структуры данных Native API. Есть практические примеры использования Native API. Опять таки – a must.
4. Джеффери Рихтер “Windows для профессионалов” – практическая книга. Очень сильная и очень полезная для любого. Есть русскоязычный перевод.

Список литературы ни в коем случае не ограничивается лишь этими 4-мя книгами. Мы старались давать как можно больше линков по ходу статьи – они стоят того, чтобы их прочесть.

Что до литературы по распаковке – такой, к сожалению, нет принципиально. Со статьями тоже напряженка. В подавляющем большинстве статей расписывается лишь как давить на кнопочки, или, что еще лучше – приводятся километровые ассемблерные листинги без комментариев, превращая статью в нечто нечитабельное. Здесь хочется похвалить Касперски, который очень удачно выбирает размер листингов в своих книгах – 10-20 очень тщательно подобранных строк – самое то!

В заключение хотелось бы сказать о третьей части. Готова она будет не скоро и при ее написании мы будем учитывать, что, наконец-то, появились две великолепнейшие статьи - по **Asprotect** by **seeQ** и по **Armadillo** 3.10 by **dragon**. Обе статьи доступны на **xtin.org** и являются редким исключением среди большинства статей по упаковщикам. Авторы проделали прекрасную работу. Спасибо им. Теперь вроде все. Да, если есть замечания, пожелания – или **NEOx** на **uin.ru** или я на форуме wasm’а готовы их выслушать.

Выводы

Ну что ж. Вот и закончена очередная часть. Будем надеятся, вы усвоили две главные вещи.

Вещь 1 : программа исполняется не в вакууме! Windows создает и поддерживает огромное количество сложнейших структур данных, поддерживающих каждый процесс, каждый поток. Чем

больше ваше понимание об этих механизмах – тем легче ломать. Применение ImpRec/PE Tools и иже с ними не освобождает от понимания принципов, лежащих в основе этих и многих других утилит.

Вещь 2 : с пакером надо РАБОТАТЬ. Наплюйте на файловые анализаторы, generic-анпакеры (например, такие как проект GUW by Christoph Gabler - http://wave.prohosting.com/mackt/main.htm) - это все хорошо, но разработать общее решение пока не удается. Уж больно много всего учитывать надо. Требуется больше времени и куда больше усилий, чтобы разработать общую схему, если она существует.

Благодарности

Информационное поле статьи было чудовищным и неподъемным для одиночки. Мне помогали очень многие люди и всем им я хочу сказать – без вас этой статьи бы НЕ БЫЛО! Мы с **Four-F** плечом к плечу прошли через структуры кольца-3 и кольца-0 и когда я начинал захлебываться в этом – меня было кому поддержать! **Sten** всегда откликался на призывы о помощи и мы вели долгие беседы о **SEH**. **Dr.Golova** долго и терпеливо вразумлял неуча о трюке с **NtContinue** , а также долго и нудно многократно вычитывал эту статью в поисках ошибок. **Fixer** и **Alephz** были со мной во время написания главы о сигнатурах. **Alephz** любезно предоставил исходный код **CC** – утилиты-поисковика сигнатур криптоалгоритмов в исполняемом файле (доступна для скачивания с **wasm.ru**), а **Fixer** выработал ряд правил для описания сигнатуры. Умничка **Quantum**... В одиночку **UPX** я бы не поднял никогда. А сколько времени он потратил, трудолюбиво переводя с испанского все турториалы Рикардо Нарвахи и сколько времени было потрачено в попытках извлечь оттуда какие-нибудь ценные методики или приемы... **The Svin** и мр. **Хемуль Советикус** вместе с masquer крепко пинали за декомпилирование. Broken Sword был моим консультантом по DR-регистрам, а **Edmond** сумел сверстать эти 40 страниц ужаса. **Asterix** пинался по поводу **Aspack** , вынуждая нас с **NEOx** ’ом снова и снова переписывать ту несчастную главу, а **seeQ** , гм, **seeQ** просто сказал много хороших слов, в которых я нуждался после пинков **Asterix**. Спасибо также **Kai Zheng** , который любезно согласился перевести китайскую статью по **telock** и директории импорта. Скажите этим людям спасибо! Они много сделали для вас, читатель!

Теоретические основы крэкинга: Введение

Автор: CyberManiac

Дата: 2003-12-12

> *Что тебя смутит - то ложь*
> *Е. Летов*

За пять лет более или менее активного занятия крэкингом (а именно столько времени прошло с того знаменательного момента, как я открыл счет взломанным программам) мне удалось выработать набор методов, позволяющих сравнительно быстро найти подход к большинству программ. Возьму на себя смелость утверждать, что именно эта система наиболее доступна для начинающих, поскольку она базируется не на раскрытии алгоритма работы защиты в целом, а на более простом процессе анализа сравнительно небольших участков кода и установлении взаимосвязи между ними.

Несмотря на то, что крэкеры, в большинстве своем, работают под одной очень известной операционной системой и пользуются сходными инструментами, я буду делать упор не на описание этих инструментов и правил работы с ними, а на универсальные идеи (хотя в качестве примеров я, разумеется, буду приводить то, что наиболее доступно для самостоятельных экспериментов). Думаю, что Вы сами определитесь, какие инструменты Вам больше по душе.

Предлагаемую мной систему не следует воспринимать как единственно правильную, это не "истина в последней инстанции", но именно эта система помогла мне "победить" немало программ, так что ее эффективность проверена и подтверждена практикой. Вообще, моя точка зрения относительно использования тех или иных методов такова: одна лишь практическая эффективность может служить критерием того, какие идеи и технические приемы могут и должны применяться крэкером в его "профессиональной" деятельности. Именно поэтому я настоятельно рекомендую отрешиться от любых утверждений о "некрасивости" тех или иных приемов борьбы с защитами и ориентироваться лишь на достижение конечной цели - независимо от того, является ли этой целью раскрытие алгоритма работы какого-либо куска кода, или же простое снятие триальных ограничений в программе.

Так или иначе, статей от том, "что делать", то есть как взломать конкретную программу или тип защиты, во много раз больше, чем руководств "как и почему надо делать именно это". Образно говоря, статьи о том, "что делать" - это уровень начальной школы, "почему был выбран именно такой прием" - уровень выпускных классов. Но ведь есть еще и "высшая школа" - изучение идей, которые не привязаны к каким-либо программам и типам защит, но могут быть адаптированы для решения любой конкретной задачи. Выше - только "научная работа", то есть область чистого творчества, генерация оригинальных идей, и никакой "учебник" в этой области принципиально невозможен. По моему глубокому убеждению, основная проблема, возникающая у начинающих крэкеров, заключается в огромном количестве пособий уровня "делай, как я", совершенно не объясняющих, почему автор пособия поступил именно так. В результате человек "на ура" раскалывает новые версии одной и той же программы, но теряется перед аналогичной защитой, реализованной слегка иначе. Разумеется, существуют и весьма качественные "учебники", предлагающие именно *систему*, а не только набор технических приемов (те же +ORC Cracking Tutorialz или руководство от the Keyboard Caper'a) - но абсолютное большинство их них написаны на английском языке. Для каждого человека естественно желание получать необходимую информацию на своем родном языке (а для меня, как и для многих других, родным языком является русский). А поскольку в последние годы на территории бывшего СССР искусство борьбы с защитами движется семимильными шагами, рано или поздно должны были появиться русскоязычные тексты, систематизирующие опыт крэкинга. Именно такой материал я и старался

написать, а насколько хорошо это у меня получилось - решать вам.

Теоретические основы крэкинга: Глава 0. Дзен-крэкинг. Теория.

Автор: CyberManiac

Дата: 2003-12-13

> *Я знаю, что ничего не знаю*
> *Сократ*

Идея, лежащая в основе дзен-крэкинга (именно это название широко используется на сайте Fravia для обозначения той системы крэкинга, о которой я рассказываю), звучит так: "я не знаю, как это работает, но я знаю, как это сломать". Разумеется, речь не идет об абсолютном незнании того, как функционирует программа - знание команд ассемблера, способов передачи параметров в функции и процедуры, назначения системных вызовов ОС, особенностей кодогенерации определенных компиляторов и многого другого, несомненно, является обязательным. Более того, это основы, без которых любое изучение внутренностей программ в принципе невозможно - как нельзя извлечь информацию из книги, не понимая языка, на котором она написана. "Не знаю, как работает" следует понимать в том смысле, что очень часто для успешного взлома программы совершенно необязательно проводить доскональный анализ всех защитных процедур. Иметь возможность сказать: "я знаю, для чего нужен каждый байт в этой программе" - это, конечно, хорошо, но на практике вполне успешно работает модель "черного ящика", когда нам известно назначение отдельных процедур, взаимосвязь между ними и то, какие эффекты вызывает подача тех или иных параметров на входы "черного ящика".

Вообще, в крэкинге существует два пути. Первый - путь глубокого анализа, изучения и достижения понимания того, как работает программа. Этот путь весьма надежен, но для получения гарантированного результата он требует много времени, усилий и практического опыта. В наше время главным критерием является эффективность и скорость взлома, а не "правильность", которая интересна лишь "гуру" и вечно недовольным теоретикам. К тому же, если Вы только начали обучаться крэкингу, у Вас может просто не оказаться нужных знаний и опыта, чтобы знать, в каком направлении нужно двигаться. Итак - налицо парадокс: чтобы приобрести опыт, нужно ломать программы, причем ломать успешно, и чем больше - тем лучше, а ломать их не получается из-за недостатка опыта. Но существует второй путь - исследовать программы, исходя из предположений, которые, в свою очередь, строятся на наблюдении за внешними эффектами, производимыми программой. То есть Вы не начинаете сразу выяснять, что и как происходит в недрах кода программы, а сначала делаете предположения: "на что это может быть похоже", "как это может быть реализовано" и "как бы я добился такого эффекта, будь я автором программы". Как ни странно, при использовании этого метода, успех зависит не только от знаний, но и от того, насколько богато Ваше воображение. Эффективность дзен-крэкинга опирается, прежде всего, на наблюдения и смелые предположения. Поэтому не надейтесь, что авторы защит будут применять избитые приемы, которые можно аккуратно переписать на бумажку и составить "инструкцию по взлому". Ожидайте неожиданного!

Однако Вы не можете строить предположения о работе программы на пустом месте - Вам потребуется некий стартовый набор знаний. Поэтому Вы должны собрать как можно больше информации о самой программе - выяснить, упакована она или нет, какие ограничения содержатся в незарегистрированной программе и как выглядит процесс регистрации; узнать, что будет, если Вы попытаетесь использовать программу дольше, чем это предусмотрено триальными ограничениями; проанализировать, какие текстовые строки и ресурсы содержатся внутри программы; поинтересоваться, к каким файлам и ключам реестра программа обращается при запуске, и многое

другое. Не поленитесь заглянуть в справочную систему программы - там Вы можете, к примеру, найти описания различий между зарегистрированной и незарегистрированной версией. Значительная часть этой информации Вам скорее всего не пригодится, но Вы не можете знать заранее, какой путь окажется наиболее удобным и какие знания о программе Вам понадобятся, а какие - нет. Более того, почти наверняка все необходимые данные о работе программы Вы с первой попытки не соберете, и уже в процессе изучения кода Вам придется возвращаться к этому этапу, чтобы узнать, например, при каких условиях программа в заданной точке вызывает функцию создания файла, куда считываются введенные серийные номера, сколько раз и откуда программа вызывает функцию проверки регистрации и т.п. А потому - собирайте информацию!

Когда Вы сгенерировали идеи о том, как именно могут работать интересующие Вас механизмы в программе, начинается этап проверки правильности Ваших предположений. А значит, следующая Ваша задача - добраться до кода, который эти предполагаемые Вами механизмы мог бы реализовывать. Для этого Вам нужно проанализировать все предполагаемые варианты и найти, к чему можно "прицепиться" в каждом случае. Иными словами, Вы должны представить, чем может отличаться интересующий Вас кусок кода от множества других кусков, и чем более явными будут эти отличия, тем легче Вам будет этот код найти. Например, если программа выводит nag-screen, можно "прицепиться" к функциям создания и отображения окон; если предполагается проверка CRC файла, результатом будет либо многократное чтение небольших блоков, либо загрузка или отображение всего файла в память; если в заголовке окна программы большими буквами написано UNREGISTERED, можно поискать эту строчку в программе и выяснить, откуда и при каких условиях происходит обращение к этой надписи.

И, наконец, Вам придется разработать практические приемы, при помощи которых можно добраться до интересующих Вас кусков кода, выбрать подходящий инструмент из своего арсенала и правильно его применить.

Как это работает на практике? Представьте себе программу, которая делает нечто. Например, отказывается запускаться после 30 дней использования, выдавая стандартное окошко с сообщением (широко известное как MessageBox). Чтож, у нас есть первое наблюдение. Простой перевод системного времени не помогает - обмануть программу и заставить ее работать дольше, чем положено, не удается. Это второе наблюдение. Из него следует, что программа проверяет текущую дату не (или не только) на основе показаний внутренних часов Windows. Предполагаем, что программа либо уже сделала пометку "больше не запускаться" где-нибудь в реестре или на диске, либо все-таки определяет текущее время, но каким-либо хитрым способом. Например, читая дату последнего доступа или модификации какого-либо файла. У нас уже есть целых два смелых предположения, которые можно брать за основу в дальнейшем расследовании вредительской деятельности защиты. Если программа не просто "задумывается" при запуске, но еще и шуршит винчестером, вероятность второго варианта сильно повышается. Теперь начинаем проверять эти варианты. В первом случае нам однозначно проще докопаться до истины, установив точки останова на все виды MessageBox'ов и выяснять, какой из условных переходов позволяет избежать появления этого сообщения. Во втором случае в качестве отправной точки можно использовать всевозможные GetFileTime, CompareFileTime (а чем не способ - сравнить дату создания файла программы, т.е. дату инсталляции с датой последней модификации какого-либо файла) и FindFirstFile/FindNextFile (они ведь тоже способны читать временные характеристики файлов).

Абсолютное большинство защит, как бы аккуратно они не были реализованы, все-таки имеют "ахиллесову пяту". Эта уязвимость может быть запрятана глубоко в недрах кода, размазана по нескольким десяткам процедур или же быть совершенно неочевидной - но она есть. Стоит только ее обнаружить и нанести точный удар - и защита развалится, как карточный домик. Следовательно, залогом успешного взлома является отыскание уязвимых мест в защите.

Теперь, когда мы знаем, что нам нужно искать, осталось только определить, как выглядят эти уязвимости. Наиболее "удобные" для крэкера дыры - это прежде всего глобальные переменные, в

которых хранится состояние программы ("зарегистрирована - не зарегистрирована"), функции, возвращающее критичное для защиты значение (число запусков или дней до истечение триала, результат проверки серийного номера на правильность) и процедуры, выводящие сообщение об успешной или неуспешной попытке регистрации, а также об истечении триального срока программы. Одним из величайших "шедевров", встреченным мной, была глобальная переменная в секции инициализированных данных. По умолчанию ее значение было равно нулю, и менялось на единицу если серийный номер, извлекаемый из реестра, был верен. Исправление одного-единственного бита превратило программу в зарегистрированную. Другим перспективным приемом, который, правда, эффективен в основном против ограничений максимального/минимального значения какого-либо числового параметра (количества обрабатываемых документов, числа запусков и т.п.) является поиск константы, с которой производится сравнение, и модификация либо самой константы, либо условия проверки. Более подробно о том, как обращаться с переменными и константами, я расскажу в соответствующей главе.

"Регистрация" программ, где защитная функция возвращает единственный результат несколько сложнее - требуется лишь выявить все точки, в которых функция возвращает какое-либо значение, и подправить это значение. Нужно только помнить, что куски кода вроде

```
mov eax,0
…
ret
```

могут встречаться в теле функции в нескольких экземплярах, и обезвредить нужно их все. Да еще то, что возвращаемое значение совершенно не обязательно является ноликом или единичкой.

Другая проблема для программистов защит, которая сильно облегчает жизнь крэкерам (вот уж воистину "что русскому хорошо, то немцу - смерть") это "проблема условного перехода". Эта проблема заключается в том, реализовать проверку какого-либо условия, не использовав команду условного перехода, не так уж просто. Зато если этот условный переход есть, его легко можно исправить на такой же, но с противоположным условием - обычно для этого достаточно изменить ровно один бит. Несмотря на техническую простоту, правка условных переходов все-таки менее предпочтительна, чем модификация функций. Причина этого в том, что условных переходов, имеющих отношение к защите, в программе может быть довольно много, и их поиск требует особой внимательности. К тому же, если защита вместо обычных функций использует inline-функции или макросы, разбросанные по всей программе, защитный механизм выглядит как длинный и внешне однородный кусок кода. Поиск "плохих" переходов внутри такого "монолитного" кода может быть довольно затруднителен. С другой стороны, если в таких защитных вставках используются вызовы каких-либо "нормальных" (не inline) функций, особенно функций WinAPI, найти такие идентичные куски становится не так уж сложно. В таком блоке кода почти наверняка найдутся последовательности команд, по которым такие защитные вставки можно идентифицировать - так что поможет либо поиск в двоичном файле программы с использованием маски, либо в дизассемблированном тексте - с использованием регулярных выражений. Если запастись терпением, можно даже проверить все подозрительные участки программы вручную, это вполне реально, если таких участков в программе не больше двух десятков.

Теперь Вам известны наиболее часто встречающиеся в защитах дыры, в выявлении которых заключена половина успеха крэкера. Пришло время рассмотреть трудности и "подводные камни", которые могут ожидать Вас в нелегком кэкерском труде. Прежде всего это проблема неверной интерпретации собранной информации. Например, Вы обнаружили, что программа поддерживает использование плагинов и при запуске сканирует все файлы с расширением DLL в собственной директории. Вы вполне логично предполагаете, что программа строит список плагинов для дальнейшей загрузки и подключения. А потом Вы можете очень долго искать механизм определения даты первого запуска - и не найти его. Потому что он уже отработал - как раз при

поиске плагинов. Как такое может быть? Да очень просто: в комплект программы включается как минимум один плагин. Далее - обычная привязка к дате последней модификации файла этого плагина; саму дату модификации файла несложно получить в ходе поиска через FindFirst/FindNext. Вот так иногда авторы защит прячут свой вредительский код, что называется, "на самом видном месте".

Другой пример неверной интерпретации собранных данных встречается еще чаще; более того, это неизбежное следствие подхода, используемого в дзен-крэкинге. Если Вы нашли условный переход, который начисто "выключает" все сообщения о незарегистрированности программы, из этого не обязательно следует, что программа будет вести себя в точности как зарегистрированная. Убедиться в стопроцентной надежности (или принципиальной неправильности) исправления этого перехода - подчас задача много более сложная, чем найти этот самый условный переход. Что интересно, это утверждение работает и в обратную сторону: если Вы что-то сделали, но не получили немедленно желаемый результат, это совершенно не означает, что Вы ошиблись. Создатели защит не так уж редко создают многоуровневую оборону, и в этом случае для одержания победы недостаточно разрушить внешние рубежи защитного кода. Если вы что-то сделали, но не добились успеха, не стоит сразу же бросать избранный путь; возможно, что Вы абсолютно правы и необходимо двигаться тем же путем и дальше. Даже если после Ваших манипуляций подопытная программа рушится с "ужасным" GPF, этот GPF может быть всего лишь еще одной, еще не выявленной уловкой создателя защиты.

Теоретические основы крэкинга: Глава -1. Орудия крэкера.

Автор: CyberManiac

Дата: 2003-12-15

> *Ваше слово, товарищ "Маузер"*
> *В. Маяковский*

Осмелюсь предположить, что Вы читаете этот текст не из праздного интереса или ради абстрактного "знания", а хотите научиться применять эти знания на практике и пожинать сладкие плоды своего труда. То есть ломать программы. И хотя это пособие носит название "Теоретические основы…", сам крэкинг - дисциплина прикладная. И когда Вы решитесь перейти от изучения сухой теории к активной деятельности, Вам потребуются "рычаги", при помощи которых Вы сможете перевернуть код. И именно об этих "рычагах" пойдет речь в данном разделе. Поскольку обучение крэкингу требует постоянной и разнообразной практики, было бы логично начать с перечисления того, что Вам потребуется для "практических занятий". Но, с другой стороны, вряд ли Вы сможете выбрать наилучшие инструменты, не зная хотя бы в общих чертах особенностей Вашей будущей деятельности. И именно поэтому глава носит номер "минус один", но следует за "нулевой" главой, в которой я попытался объяснить, чем Вы будете заниматься и какие трудности могут Вас ожидать.

Инструменты - это материальная основа крэкинга, без инструментов в крэкинге - никуда, но сами по себе инструменты - не более чем набор байтов, и лишь человеческая мысль оживляет их, заставляя творить чудеса. Для использования этих программ, как правило, требуются соответствующие знания. Насколько будет Вам полезен лучший отладчик в мире, если Вы не знаете, что означают те хитрые цифры и буквы, которые он показывает на экране? Поэтому прежде чем задать программе вопрос, Вы должны быть уверены, что сможете понять ответ.

Но, тем не менее, от качества этих инструментов будет во многом зависеть скорость и эффективность Ваших действий. Ваш инструментарий должен постоянно обновляться; не надейтесь, что софт десятилетней давности сможет чем-то Вам помочь - в области высоких технологий радикальные изменения могут произойти за считанные дни. Программы совершенствуются, наращивают мощность, обрастают новыми полезными и бесполезными функциями; защиты также не стоят на месте - и Вам нужно быть в курсе этих процессов, своевременно обновляя свой арсенал.

К вашему счастью, инструментов, пригодных для использования в крэкинге, не так уж мало, и проблема состоит не в том, чтобы их раздобыть, а в том, чтобы отобрать из них наилучшие, изучить их возможности и определить для себя, в какой ситуации тот или иной инструмент лучше всего применить. Поясню эту мысль примером: на данный момент наиболее мощным из дизассемблеров является IDA Pro, которая способен не просто дизассемблировать код, но и находить в нем вызовы стандартных функций различных компиляторов. Однако когда мне необходимо покопаться в программе, написанной на Delphi версии выше третьей, я наверняка не буду использовать IDA Pro, предпочтя ему Delphi Decompiler. Почему? Во-первых, скорость работы IDA Pro и DeDe различается в десятки раз в пользу последнего; используя DeDe, я скорее всего получу желаемый результат раньше, чем закончилось бы дизассемблирование в IDA Pro. Во-вторых, DeDe позволяет анализировать работающие процессы "на лету", что позволяет анализировать сжатые программы, не отвлекаясь на распаковку, восстановление таблицы импорта и прочие вспомогательные действия. Не углубляясь в дальнейшее перечисление достоинств DeDe, начисто отсутствующих в IDA, скажу, что в большинстве случаев специализированная программа позволяет решать свой "родной" класс задач значительно эффективнее, чем программы "общего назначения",

ориентированные на ручную работу. Конечно, никто не запретит вам распаковывать программы, вручную копируя секции из памяти в файл, но не проще ли воспользоваться дампером?

Наверняка найдутся те, кто возразит, что широкое использование готовых утилит якобы мешает самостоятельному мышлению, чрезмерно упрощает процесс взлома и вообще "настоящие хакеры дизассемблируют в уме". Так вот, мое принципиальное мнение по этому вопросу такое: используйте все программы, какие только сочтете нужными, если это поможет вам добиться желаемого. В конце-концов, цель крэкера обычно состоит в получении работающей программы, а никак не в тренировке памяти или демонстрации собственной крутизны. А всем "настоящим хакерам" посоветуйте дизассемблировать в уме winword.exe из состава самого последнего MS Office, и до тех пор, пока они не справятся с этим несложным заданием, не беспокоить вас древними суевериями. Разумеется, рано или поздно Вы перейдете от использования чужих программ к написанию собственных патчеров, распаковщиков, дамперов и прочих утилит - но такой переход должен быть продиктован насущной необходимостью, а не обезьяним "чтобы быть не хуже других". В конце-концов, большинство программ было написано именно для того, чтобы люди ими пользовались.

Теперь посмотрим, какие инструменты и для чего нам потребуются...

Отладчики и дизассемблеры. Традиционно оба этих типа инструментов используются в паре, поскольку дизассемблер выдает лишь "чистый код", хотя современные дизассемблеры способны также распознать вызовы стандартных функций, выделить локальные переменные в процедурах и предоставляют прочий подобный сервис. Пользуясь дизассемблером, Вы можете лишь догадываться о том, какие данные получает та или иная функция в качестве параметров и что они означают; чтобы выяснить это, чаще всего требуется изучения если не всей программы, то довольно значительной ее части. Отладчики выполняют принципиально иные функции, они позволяют анализировать код в процессе его работы, отслеживать и изменять состояние регистров и стека, править код "на лету" - в общем, наблюдать за "личной жизнью" программы и даже активно в нее вмешиваться. Обратной стороной медали является "неинтеллектуальность" многих отладчиков - их врожденные способности к анализу кода редко простираются дальше определения направления перехода. SoftIce, "лучший отладчик всех времен и народов", например, ничего не знает о типах данных и не способен отличить обычный DWORD от указателя на ASCIIZ-строку, хотя и предоставляет пользователю возможность проверить это вручную. Впрочем, вполне реальны отладчики, сочетающие высокое качество дизассемблирования и анализа кода с широкими возможностями по его отладке. Примером может служить OllyDebug, выполняющий эвристический анализ кода, выделяющий локальные переменные, "знающий" о типах данных, передаваемых функциям WinAPI и при этом способный во многих случаях автоматически отличить обычное число от указателя на строку. **Декомпиляторы и узкоспециализированные отладчики**. С ростом мощности ЭВМ довольно широкое распространение получили компиляторы, создающие не "чистый" машинный код, а некий набор условных инструкций, который выполняется при помощи интерпретатора. Интерпретатор может как поставляться отдельно (Java), так и быть "прикрепленным" к самой программе (хотя формально не интерпретатор прикрепляется к программе, а программа к интерпретатору. Примером может служить Visual Basic в режиме компиляции в p-code). Возможны и более экзотические варианты, например, применяемый в Форте "шитый код"; компиляция программы в цепочку команд push\call или преобразование текста программы в такую цепочку непосредственно при запуске этой программы. Интерпретаторами являются практически все инсталляторы (в их основе лежит интерпретатор инсталляционного скрипта, хотя сам процесс создания такого скрипта может быть скрыт при помощи визуальных средств). Да и "обычные" компилирующие языки могут создавать код, прямой анализ которого весьма затруднителен. Для анализа таких программ используются специализированные утилиты, переводящие код, понятный лишь интерпретатору, в форму, более удобную для понимания человеком. Также некоторые декомпиляторы могут извлекать информацию об элементах

интерфейса, созданных визуальными средствами. В любом случае, не следует ожидать от декомпиляторов восстановления исходного текста программы; если декомпилированная программа успешно компилируется и сохраняет работоспособность - это исключение, а не правило. **Распаковщики и утилиты для дампинга процессов**. Дизассемблировать запакованную или зашифрованную программу "напрямую" невозможно (если быть до конца точным - возможно, но бесполезно). Но если очень хочется получить хоть какой-то листинг, можно попытаться извлечь из памяти компьютера "снимок" (дамп) программы в момент ее работы. Этот дамп уже можно более или менее успешно дизассемблировать. Более того, на основе дампа можно воссоздать исполняемый файл программы, причем этот файл будет успешно загружаться, запускаться и работать. Именно на этом принципе и основана работа большинства современных распаковщиков: подопытная программа запускается под управлением распаковщика; распаковщик ждет некоторого события, говорящего о том, что программа полностью распаковалась, и тут же "замораживает" программу и сбрасывает ее дамп на диск. Защитные системы нередко пытаются противодействовать получению работоспособного дампа при помощи манипуляций с сегментами и таблицами импорта-экспорта. В этих случаях приходится применять PE-реконструкторы, т.е. утилиты, обнаруживающие в дампе некорректные ссылки на функции и пытающиеся их восстановить. Многие продвинутые дамперы и распаковщики имеют встроенные средства восстановления секций импорта. **Утилиты анализа файлов**. Очень часто требуется быстро узнать, каким упаковщиком или защитным софтом обработана та или иная программа, найти все текстовые строки в дампе памяти, просмотреть содержимое файла в виде таблицы записей, вывести в файл список импортируемых и экспортируемых программой функций и многое другое. Для всех этих целей существует огромное количество специализированных утилит, позволяющих быстро проанализировать файл на наличие тех или иных признаков. Эти утилиты, как правило, не являются жизненно необходимыми, но, при их надлежащем качестве, способны сэкономить большое количество времени и сил. **Шестнадцатеричные редакторы и редакторы ресурсов**. Это, несомненно, наиболее древние (по идее, но не обязательно по исполнению) из программистских инструментов, ведущие свою славную историю с тех времен, когда программисты еще умели "читать" и править исполняемый код, не прибегая к помощи дизассемблеров. Тайное искусство чтения кода еще сохранилось в отдаленных уголках Вселенной, но в последние годы практически утратило актуальность. И теперь лишь крэкеры практикуют этот мистический обряд, в соответствии со священной традицией исправляя в неправильных программах идеологически чуждый опкод 75h на истинный и совершенный 0EBh. Редакторы ресурсов, в принципе, занимаются тем же самым, но по отношению к прилинкованным к исполняемому файлу ресурсам. Именно при помощи редакторов ресурсов выполняется значительная часть работ по "независимой" русификации программ и доработке интерфейсов. Рука об руку с редакторами идут всевозможные патчеры, которые позволяют создать небольшой исполняемый файл, автоматически вносящий изменения в оригинальный файл программы либо в код этой программы непосредственно в памяти. **API-шпионы и другие утилиты мониторинга**. Очень часто бывает нужно узнать, какие именно действия выполняет та или иная программа, откуда читает и куда записывает данные, какие стандартные функции и с какими параметрами она вызывает. Получить эти знания как раз и помогают утилиты мониторинга. Такие утилиты делятся на две большие группы: те, которые отслеживают сам факт возникновения каких-либо событий и те, которые позволяют выявить один или несколько специфических типов изменений, произошедших в системе за некий промежуток времени. К первой группе относятся всевозможные API-шпионы, перехватывающие вызовы системных функций (а более продвинутые API-шпионы умеют перехватывать и не только системные функции), хорошо известные утилиты Reg-, File-, PortMon и иже с ними, перехватчики системных сообщений и многие другие. Эти утилиты, как правило, предоставляют весьма подробную информацию об отслеживаемых событиях, но генерируют весьма объемные и неудобные для анализа логи, если отслеживаемые события происходят достаточно часто. Вторая группа представлена всевозможными программами, создающими снимки реестра, жесткого диска, системных файлов и т.п. Эти программы позволяют сравнить состояние компьютера "до" и "после"

некоего события, построить список различий между состояниями и на основе этого списка делать определенные выводы. Основная проблема при работе с такими утилитами хорошо описывается словами ""после" - не значит "вследствие"" (т.е. кроме интересующей Вас деятельности программы Вы можете получить лог "жизнедеятельности" других параллельно работающих программ и самой операционной системы). **Прочие утилиты**. Существует огромное количество утилит, не вписывающихся в приведенные выше категории или попадающие в несколько категорий. Одна лишь полная классификация этих инструментов потребовала бы значительных усилий, но моя цель не в том, чтобы плодить "мертвые буквы", а в том, чтобы дать Вам знания и навыки, достаточные для самостоятельных занятий. Крэкинг - занятие весьма многогранное, и столь же многогранны инструменты, в нем используемые. Более того, некоторые из утилит, которые могут быть полезны для крэкера, создавались для совершенно иных целей (хорошим примером может служить программа GameWizard32, которая вообще-то была предназначена, чтобы мухлевать в компьютерных играх, но оказалась полезна при вскрытии программы с ограничением на максимальное число вводимых записей). Поэтому еще раз обращу Ваше внимание: важно не только качество инструмента, но и умение творчески и нетривиально его применить.

Теоретические основы крэкинга: Глава 2. Почти начинаем ломать.

Автор: CyberManiac

Дата: 2004-01-12

> *Мы откроем сундук, хотя бы пришлось из-за него умереть...*
> *Р.Л. Стивенсон, "Остров сокровищ"*

Итак, допустим, что у нас есть программа и в ней содержится вредоносный код (далее - "защита"), который нужно обезвредить. Инсталляционный файл тихо лежат на нашем винчестере, ожидая того знаменательного момента, когда мы его запустим, чтобы извлечь на свет скрытую в его недрах программу. Мне вполне понятно Ваше желание немедленно установить эту программу и вступить в битву со злобным и коварным врагом, но охладите на несколько минут свой пыл, и послушайте мой рассказ о процессе инсталляции программ, и чем этот процесс может кончиться.

Итак, многие программы в настоящее время поставляются в виде инсталляционного пакета. Для установки программы, как правило, требуется либо запустить один из файлов пакета (это, в частности, отлично всем известные Setup.exe), либо открыть при помощи другой заранее установленной программы (к примеру, файлы с расширением MSI, созданные Microsoft'овским инсталлятором или RPM-пакеты в Linux). В инсталляционных пакетах кроме самих файлов, которые требуется установить на машину пользователя, содержится также описание сценария инсталляции в том или ином виде (назовем это описание сценария для простоты "инсталляционным скриптом", тем более, что чаще всего так оно и есть). Разумеется, инсталлятор может быть и обычной программой, написанной для установки конкретного приложения, но написание собственного инсталлятора - дело достаточно трудоемкое, и поэтому на практике такие инсталляторы встречаются весьма редко.

Инсталляционный скрипт описывает, в каком режиме устанавливается тот или иной файл (добавление, замена, замена с предварительной проверкой версии и т.п.), какие данные необходимо внести в реестр или файлы конфигурации, какие программы запускаются до, в процессе и после инсталляции, а также многое другое. В это "многое другое" могут входить и такие безусловно интересные вещи, как проверка серийных номеров, создание записей в реестре, а также распаковка и запуск программ. О встроенных в инсталляционный скрипт серийных номерах мы поговорим позже. А пока попробуем поразмышлять о том, в чем может заключаться опасность неконтролируемого создания ключей в реестре или запуска каких-либо программ.

Если Вы взламываете какую-либо программу, оснащенную ограничением на время использования или число запусков, один из "корней зла" может гнездиться именно в инсталляционном скрипте. Представьте себе такую ситуацию: программа хранит дату первого запуска и/или какую-либо иную информацию, необходимую для проверки на истечение срока пробного использования, в реестре. Разумеется, информация закодирована, предприняты меры, чтобы защиту не обманывало "подкручивание" системной даты, возможно даже соответствующий ключ реестра хорошо замаскирован (Вам ничего не напоминает мое описание? Да это же ASProtect - ломаный-переломаный, но, как ни странно, все еще популярный). Тем не менее, одна лазейка все-таки осталась - если триальный ключ в реестре отсутствует, защита считает, что раньше программа не запускалась. Поэтому защиту можно обмануть, просто удалив из реестра лишние ключики. А теперь представьте, что триальный ключ создается в процессе инсталляции, и если он отсутствует, программа просто перестает запускаться. Если Вы ставите целью взлома ликвидацию триальных ограничений, Вам могут потребоваться весьма значительные усилия, чтобы отыскать этот маленький, но зловредный ключик в огромном реестре еще более огромной Windows. Как вам

такая перспектива? Если встроенных средств инсталлятора оказывается недостаточно для создания триальной "метки", это можно сделать при помощи небольшого исполняемого файла, который распаковывается в процессе инсталляции, запускается, делает свое черное дело и сразу после этого удаляется. Упрощенный вариант этого приема выглядит как автоматический запуск приложения после инсталляции, чтобы защита смогла создать триальные "метки" на компьютере пользователя.

В качестве метки, на основе которой программа будет проверять ограничения по времени или числу запусков, также может использоваться какой-либо файл, создаваемый в процессе инсталляции, особенно если этот файл запрятан в глубинах одной из системных директорий (да-да, я знаю, что мусорить в системных директориях нехорошо; осталось лишь объяснить это некоторым авторам защит) и при этом еще активно используется программой. Я встречал защиту, основанную на подобном принципе: дата создания одной из DLL использовалась для отсчета времени с момента установки, а сама DLL лежала в системной директории Windows среди сотен себе подобных и динамически подгружалась основной программой.

Какие средства мы можем применить, чтобы обнаружить и обезвредить эти и другие подобные приемы? Наиболее радикальным средством, разумеется, является декомпиляция инсталляционного скрипта - в этом случае мы получаем практически полную информацию о том, что происходит в процессе установки программы, а в некоторых случаях даже можем повлиять на этот процесс, внеся исправления в инсталлятор. Разобрать инсталляционный скрипт "по косточкам" - задача не самая простая, да и не всегда это необходимо. Поэтому на практике чаще пользуются другим типовым приемом, позволяющим обнаружить произошедшие изменения. Этот прием заключается в использовании утилит мониторинга, делающих "снимки" системы (реестра, размеров и дат создания/модификации файлов) до и после установки и затем анализирующих различия между снимками. Подробный журнал изменений, выдаваемый такими программами, позволяет легко обнаружить подозрительные ключи и файлы, появившиеся в процессе инсталляции. Забегая вперед, скажу, что такие же "снимки" рекомендуется делать и при прохождении других критических периодов работы программы о которых мы поговорим в следующей главе. Установить факт запуска каких-либо программ во время инсталляции можно при помощи утилит, отслеживающих создание и завершение процессов.

Однако деятельность программ, запускаемых в процессе инсталляции, не ограничивается одним лишь созданием триальных ключей. Дело в том, что набор функций, поддерживаемых инсталляторами, ограничен и некоторые действия (например, проверку серийного номера с использованием достаточно сложного алгоритма) выполнить средствами инсталляционных скриптов не всегда возможно. Один из приемов, применяемых в этом случае - запуск исполняемого файла, который и выполняет все необходимые операции, а потом тем или иным образом возвращает результат проверки в инсталлятор - существуют защиты, в которых проверка серийного номера реализована именно так. В некоторых инсталляторах для этих же целей предусмотрен интерфейс, позволяющий использовать плагины (плагин в виде динамически загружаемой библиотеки также упрятываются внутрь инсталлятора, в нужный момент распаковываются во временную директорию и после использования удаляются). Такие исполняемые файлы и плагины, разумеется, невозможно модифицировать напрямую и чаще всего не удается извлечь из инсталлятора для дизассемблирования и изучения, т.к. они хранятся внутри инсталлятора в сжатом виде, а большинство коммерческих инсталляторов несовместимы по формату с обычными архиваторами. Если Вы хотите исследовать такой исполняемый файл, Вам почти наверняка потребуется снять с него дамп, чтобы получить материал для загрузки в дизассемблер. Сделать это совсем несложно - запустите инсталлятор под отладчиком и поставьте точки останова на все функции, связанные с загрузкой модулей и библиотек (для плагина) или создания процесса (для EXE). В Windows это будут LoadLibrary[Ex], LoadModule[Ex], CreateProcess или устаревший WinExec соответственно. Запоминаем, откуда инсталлятор пытается загрузить файл, и затем

"замораживаем" работу инсталлятора непосредственно перед исполнением этой функции патчем

```
MySelf: jmp MySelf
```

либо манипуляциями с атрибутами соответствующего процесса и потока (например, применив к нему функцию SuspendThread). Затем копируем нужный нам файл в надежное место и делаем с ним все, что только придет нам в голову.

Если такой файл, запускаемый во время инсталляции, работает сколь-нибудь длительное время (достаточное, чтобы обнаружить факт появления нового процесса и записать данные в его адресное пространство), Вы даже можете создать memory patch, позволяющий модифицировать код этого файла в памяти. Приведу практический пример: одна из программ в процессе установки запрашивала пароль, который было необходимо ввести, чтобы продолжить инсталляцию. Путем экспериментов удалось установить, что инсталлятор создавал в директории для временных файлов исполняемый файл со случайным именем, запускал его и затем блокировал доступ к файлу даже на чтение, после чего собственно ввод пароля и его проверка осуществлялась уже средствами этой запущенной программы. Заставить программу "признать" любой пароль при помощи правки кода программы в SoftIce не составляло особой сложности, но вот изготовить "классический" патч, который позволил бы устанавливать эту программу, не прибегая к помощи отладчика, оказалось крайне затруднительно. Но, поскольку ввод пароля осуществлялся в стандартном окне Windows, это дало возможность подойти к проблеме с другой стороны. После недолгих размышлений выяснилось, что можно создать программу-launcher, выполняющую следующие действия:

• Ожидание появления окна с заданным заголовком.
• Определение по хэндлу этого окна идентификатора процесса, создавшего окно.
• Внесение изменений в адресное пространство этого процесса (проще говоря, исправление программы в памяти), после которых любой пароль признавался верным.

Одним из важнейших искусств для крэкера, несомненно, является изготовление или добыча серийных номеров. Как заполучить серийный номер для программы, которая без этого номера даже не инсталлируется? Варианты "втихаря списать с лицензионного компакта" или "шантажом и пытками вытянуть из зарегистрированного пользователя", мы рассматриваем как не имеющие ничего общего с высоким искусством крэкинга, и потому изучать их не будем. Серийные номера вообще - это очень обширная тема, и в данном разделе я буду рассматривать только те серийные номера, которые "упрятаны" в инсталлятор. Несколькими строками выше я уже говорил о том, как обращаться с проверщиками серийных номеров, запускаемыми в процессе инсталляции. Теперь посмотрим, как можно решить проблему серийного номера, упрятанного в инсталляционный скрипт.

Наиболее удобным для нас был бы вариант серийного номера, записанного открытым текстом, но так уже практически никто не делает, поскольку "взломать" такую защиту можно при помощи обычного просмотрщика текстов. Так что будем считать, что при проверке правильности используется представление серийного номера как результата некой хэш-функции.

Часто в инсталлятор "зашит" не один серийный номер, а несколько - и это способно существенно облегчить нам задачу. Существует три метода проверки серийного номера:

• Вычислить хэш-функцию от серийного номера и проверить, удовлетворяет ли результат какому-либо условию.
• Пропустить результат через хэш-функцию и сравнить его со списком эталонов.
• Пропустить зашифрованные серийные номера, спрятанные внутри программы, через процедуру расшифровки и затем сравнить результат с значением серийного номера, который ввел пользователь. Очевидно, что это наиболее простой случай: требуется только обнаружить точку, в которой происходит сравнение введенного серийного номера с правильным и прочитать из памяти действительный серийный номер.

Различие между п.1 и п.2 не очевидно, но оно есть: именно на втором методе обычно основаны всевозможные "черные списки" серийных номеров. Первый способ проверки в инсталляторах применяется сравнительно редко из-за слабых математических возможностей интерпретаторов инсталляционных скриптов и ориентации на максимальную простоту процесса создания инсталляции (грамотная установка защиты, к нашему счастью, достаточно сложна и при этом все равно не дает гарантированного результата). Так что в итоге внутри большинства инсталляторов упрятан все тот же список серийных номеров (возможно, лишь из одного элемента). Еще интересно отметить, что абсолютное большинство инсталляторов никак не упаковывают инсталляционные скрипты (хотя исходный текст скрипта вполне может быть откомпилирован в байт-код), поэтому Вы можете сравнительно легко модифицировать эти скрипты при помощи шестнадцатиричного редактора.

Первое, что приходит в голову - найти, где в инсталляторе спрятан этот самый список и добавить туда свой серийник. Для этого нужно ввести какой-нибудь серийный номер, потом найти код, сравнивающий значение хэш-функции от введенного серийника со списком и вписать свое значение хэш-функции в файл инсталляции на место оригинального. Что называется, просто и со вкусом.

Другой способ заключается в том, чтобы идентифицировать программный продукт, при помощи которого сделана инсталляция, затем раздобыть этот продукт и как следует его проанализировать. Это позволит Вам сравнительно быстро узнать технические возможности инсталлятора (и проблемы, которые эти возможности могут Вам создать), структуру инсталляционного скрипта, способ генерации и формат хранения правильных серийных номеров, коды и функции внутренних команд интерпретатора скриптов. При желании Вы даже сможете написать распаковщик инсталляционных пакетов, декомпилятор инсталляций а то и универсальный(!!!) генератор ключей ко всем программам, инсталляционные пакеты которых сделаны при помощи исследованного Вами программного продукта.

В некоторых случаях наиболее прямым путем к получению полнофункциональной программы из урезанного варианта является именно вскрытие инсталлятора. Поясню эту идею примером. Допустим, что у Вас есть демо-версия какой-либо хорошей программы, но Вам этого мало, и Вы хотите иметь полностью функциональный вариант этого продукта. При этом у Вас нет никакого желания вручную восстанавливать недостающий код, заботливо выдранный автором. Однако на сайте производителя иногда можно найти обновления для коммерческих версий нужного Вам софта, и эти обновления, как правило, содержат исправленные варианты основных файлов программы. Что из этого следует? А то, что если неким таинственным образом Вам удастся установить апдейт на демо-версию, есть ненулевая вероятность, что свою демонстрационную версию Вы превратите в программу, по функциональности практически не отличающуюся от полной! Разумеется, программа обновления перед своей установкой выполнит проверку на возможность обновления (в том числе и для того, чтобы пользователь случайно не "обновил" демо-версию) - но ведь Вы решили изучать крэкинг именно для решения проблем такого рода. Так что Вам потребуется лишь немного "доработать" инсталлятор, ликвидировав в нем проверку на возможность обновления.

По сути, любой инсталлятор представляет собой самораспаковывающийся архив с достаточно сложным SFX-модулем. Более того, некоторые инсталляционные пакеты даже можно открыть обычными архиваторами! Прямым следствием этого является возможность распаковать содержимое инсталляционного пакета (если, конечно, оно не зашифровано). Даже если ни один из стандартных архиваторов "не берет" инсталляционный пакет, распаковать файлы можно вручную. Дело в том, что инсталлятор обязательно содержит внутри себя процедуру распаковки, и эта процедуру можно обнаружить и проанализировать. Вам понадобится узнать, где находится процедура распаковки, какие параметры принимает, и что эти параметры обозначают (хотя бы приблизительно). Если Вам это удастся, Вы сможете попытаться принудительно вызвать эту

процедуру с нужными Вам параметрами, манипулируя кодом программы и состоянием регистров, и извлечь файлы из пакета. Задача поиска этой процедуры облегчается тем, что в инсталляционных скриптах указание на место, куда будут устанавливаться файлы, хранится в виде текста и, поставив точку останова на начала этих текстовых строк, Вы сможете обнаружить, откуда происходит обращение к этим строкам. Вообще, даже простой анализ текстовых строк, содержащихся в инсталляционном пакете, способен дать множество полезной информации. А теперь представьте себе программу, которая при установке просит некоторое условие, и, если это условие не выполнено, "забывает" установить пару-тройку файлов или устанавливает вместо нормальных версий этих файлов урезанные. Вот тут-то и Вам и пригодится возможность заглянуть внутрь архива и извлечь из него нужные файлы.

Теоретические основы крэкинга: Глава 3. «Критические дни» программ.

Автор: CyberManiac

Дата: 2004-01-13

> *Все боится времени, но само время боится Пирамид*
> *Египетская пословица*

После того, как мы успешно установили программу, неизменно наступает торжественный момент первого запуска программы (конечно, если инсталлятор не испортил нам удовольствие, запустив программу автоматически). Немало программ содержат ограничения на время пробного использования, и потому необходимо со всей ответственностью отнестись к первому (да и не только к первому, как мы увидим в дальнейшем) запуску программы. Дело в том, что процесс исследования защиты может затянуться и Вам элементарно не хватит тех дней, которые отведены под пробное использование программы. Вы, разумеется, захотите продлить время пользования программой - и вот тут Вас может ожидать весьма неприятный сюрприз.

В общем случае задачи по неограниченному продлению триального времени программы обычно не представляют особой сложности, но в качестве дополнительных уровней защиты часто присутствуют всевозможные навесные защиты, ликвидация которых может потребовать гораздо больших трудозатрат. Поэтому надо, по возможности, предупреждать подобные ситуации, а о том, каким образом эти ситуации могут возникать, я и расскажу в этом разделе.

Возможны и другие причины, по которым Вам может потребоваться снять ограничение на время использования программы. Для некоторых программ процедура регистрации не предусмотрена в принципе, но при этом в течение испытательного срока они выполняют свои функции в полном объеме (часто такие программы помечены как Demo- и Evaluation-версии), и вполне приемлемым решением было бы неограниченное продление триального срока. Случается также, что необходимо установить бета-версию программы, у которой истек "срок годности", но более новой беты нет, а посмотреть на программу очень хочется. Так или иначе, снятие ограничений на время использования или число запусков программ - одна из актуальных задач современного крэкинга.

Представьте себе, что Вы - автор программы, и Ваша задача - ограничить "испытательный срок", в течение которого пользователь может работать с программой. Как такое можно реализовать? Выбор мест хранения информации об испытательном сроке в современных ОС довольно небогатый - содержимое некоего файла (который можно попытаться спрятать), атрибуты файлов, либо системный реестр (это актуально только для ОС Windows). Такие изменения могут быть обнаружены при помощи утилит, умеющих создавать и сравнивать снимки состояния системы. Изредка встречаются нетрадиционные решения вроде манипуляций на уровне файловой системы или использования слайсов. Использование слайсов для защиты основано на том, что последний кластер, занятый файлом, обычно заполнен не целиком, и потому в незаполненной (и невидимой для большинства программ, оперирующих с файлами) можно хранить некоторый объем информации.

В принципе, идеологических различий между реестром Windows, файлами и атрибутами отдельных файлов нет, все эти объекты могут использоваться для хранения используемых защитой данных. Действительно, аналогии в устройстве реестра и дисковой подсистемы очевидны: "ветви" реестра играют роль логических дисков, разделы - практически полные аналоги папок (убедиться на наглядном примере можно, взглянув на иконки разделов в "Редакторе реестра"), имена ключей - это имена файлов, а значения ключей - содержимое этих файлов. Аналогии можно продолжить, но для нас важно другое: поскольку файловая система подобна реестру, принципы поиска и

ликвидации защитных механизмов, основанных на сокрытии информации на дисках или в реестре, во многом будут сходны. Поэтому далее я буду говорить в основном о реестре, оставляя читателю самому разобраться в том, как аналогичные механизмы могут быть реализованы на основе файловой системы (или почему они не могут быть реализованы).

Конечно, все разнообразие механизмов ограничения времени использования программы или количества запусков охватить нереально, но в этом и нет необходимости - воспользовавшись своим воображением, Вы легко продолжите список. Я лишь перечислю основные события в "жизни" программы, и их возможную связь с реализацией триальных механизмов в программах.

Первое, что приходит в голову - во время первого запуска сохранить в файле или в реестре дату (или вычислить дату, после которой программа не должна работать) и при каждом запуске сравнивать текущую дату с сохраненной, проверяя, не истек ли "срок годности" программы. Такие программы, не обнаружив в реестре пометки о дате первого запуска, как правило, считают, что текущий запуск - первый. Такое простейшее решение, как правило, и обходится простейшими средствами - достаточно обнаружить и удалить соответствующий файл или ключ реестра. Одна из модификаций этого метода - определение даты первого запуска через чтение атрибутов файлов: при создании любого файла атрибут CreationTime (дата создания файла) устанавливается автоматически, что позволяет непосредственно в процессе инсталляции "промаркировать" все устанавливаемые файлы датой установки программы. Затем программа просто проверяет при каждом запуске дату создания какого-либо файла или папки (или вообще всех файлов, созданных в процессе инсталляции) и на основе этой информации вычисляет количество дней до истечения испытательного периода. Что интересно, свойства файлов могут использоваться не только для определения даты установки программы, но и для определения текущей даты: в процессе своей работы отдельные компоненты ОС нередко ведут всевозможные "журналы" (логи) в файлах с заведомо известными именами. Проверив дату последней модификации такого файла или его содержимое, можно с некоторой погрешностью узнать текущую дату.

Как было показано выше, надежность такого метода довольно низкая и защита ориентирована в основном на психологический эффект. Более устойчивые варианты этой защиты основаны на использовании "меток", оставляемых программой в реестре в некоторые критические моменты.

Метки, создаваемые при инсталляции, обычно предназначены для предотвращения повторной установки программы (и получения дополнительного триального срока) и для противодействия попыткам "сбросить" счетчик времени удалением ключа реестра. Если инсталлятор не позволяет выполнять сложные вычисления внутри инсталляционного скрипта, метка может выглядеть как запись в реестре с фиксированным значением, наличие которого будет проверяться при первом запуске. Если для создания инсталляционного пакета использовался достаточно мощный продукт, дата инсталляции или максимальное число запусков программы может быть прописано в реестре уже в процессе установки (возможно, в зашифрованном виде). Кроме того, инсталлятор может проверить наличие сделанных ранее "меток" в реестре для предотвращения повторной инсталляции. Таким образом, гарантированно вернуть программу в рабочее состояние после окончания триального срока можно только удалением всех "меток", оставленных программой, с последующей переустановкой.

Многие из ныне существующих защит создают "метки" в момент первого запуска программы. Факт первого запуска обычно определяется простейшим способом: если программа при запуске не обнаруживает "метку", она считает, что запущена впервые, и создает "метки". Обычно создание "меток" выполняется сразу после начала исполнения программы, но изредка встречаются программы, которые выполняют эти действия при завершении программы (возможно, что существуют защиты, которые делают это в случайный момент времени). Скорее всего, таким образом разработчики защит пытались затруднить выявление защитного кода, но на деле они добились прямо противоположного эффекта: большинство операций по загрузке данных из реестра выполняется именно при запуске или изменении настроек программы, а вот запись в реестр в

конце сеанса работы - операция менее распространенная. При завершении сеанса обычно сохраняется только информация о положении окон, настройках интерфейса программы и прочие подобные данные, которые, как правило, несложно отличить от защитных механизмов программы. Очевидно, что если защита использует исключительно "метки", создаваемые при первом запуске, после удаления этих "меток" программа считает, что запущена впервые, и начинает отсчет триального срока заново. Для того, чтобы пользователи не обходили эту защиту совсем уж элементарными средствами вроде удаления соответствующего ключа реестра, ключ, содержащий в себе защитную информацию, может быть "плавающим", то есть имя ключа может генерироваться случайным образом в зависимости от аппаратной конфигурации компьютера или каких-либо иных параметров. Найти вручную такой ключ среди десятков подобных практически нереально. В частности, именно такой механизм использует ASProtect. Поскольку "плавающий" ключ, сколь успешно бы он не был замаскирован, все-таки отличается от окружающих его ключей, определив признаки, которые отличают "плавающий" счетчик от обычных ключей реестра, возможно создать программу, которая бы автоматически выявляла подозрительные ключи, что подтверждается наличием как минимум трех независимо разработанных утилит, способных выявлять и удалять ключи, созданные ASprotect'ом. В любом случае, изменения в реестре, возникшие после первого запуска программы, нетрудно обнаружить при помощи программ мониторинга реестра.

Важно отметить, что программы, в которых установлено ограничение на число пробных запусков, при первом запуске всегда создают либо модифицируют один из ключей реестра. Это напрямую следует из логики их работы: для работы такой защиты необходим счетчик запусков, а создать такой счетчик можно только во время инсталляции либо при первом запуске.

Разумеется, программа, содержащая ограничение на число запусков, должна увеличивать либо уменьшать значение этого счетчика после каждого запуска. Кроме того, запись в реестр при каждом запуске возможна также и при ограничении по времени использования. Такая запись играет роль дополнительной защиты от изменения системного времени: если дата текущего запуска меньше даты предыдущего или отличается от него на считанные секунды, программа может предположить, что пользователь пытается использовать программу сверх установленного срока. Обнаружить такую защиту сравнительно легко - многократную модификацию одного и того же ключа скрыть практически невозможно, не помогают даже такие приемы, как дублирование счетчика и "плавающий" счетчик. Сбрасывается такой счетчик тоже без особых сложностей - достаточно один раз запомнить состояние соответствующих ключей реестра и затем восстанавливать его перед каждым запуском.

Интересно отметить, что даже столь простую идею, как отслеживание времени, некоторые разработчики ухитряются реализовать некорректно (по крайней мере, под ОС Windows такое встречается не так уж редко). Windows позволяет оперировать двумя типами времени: системным (оно же "всемирное", UTC) и местным (Local). Причем во многих странах местное время может быть зимним и летним. И если защита ориентируется на местное время, в день перевода часов пользователя может ожидать сюрприз: после автоматического перевода часов, выполняемого ОС, программа может просто перестать работать. Для этого нужно лишь, чтобы пользователь запустил программу непосредственно перед переходом с летнего времени на зимнее, и потом - еще один раз в течение ближайшего часа. По этому нехитрому признаку можно в известной степени судить о квалификации разработчика защиты.

Наконец, последним важным моментом в жизни программы является ее первый запуск после истечения срока триального использования. Поскольку большинство лицензионных соглашений запрещают пробное использование программы сверх установленного "испытательного срока", программа может "помочь" пользователю соблюсти условия лицензионного соглашения, сделав в реестре метку, запрещающую дальнейшую работу программы. Эта метка может быть как новым ключом в глубинах реестра (или файлом в какой-нибудь системной директории), так и особым значением уже существующего ключа. Проверка значения такого ключа также может быть встроена

в инсталлятор программы, чтобы пользователь не мог просто деинсталлировать и переустановить программу.

Какие выводы следуют из всего, что было сказано выше? Во-первых, то, что триальные механизмы в программах могут быть довольно разнообразны по исполнению (и, кроме того, иногда дублируются разработчиком для повышения надежности), а их поиск и удаление требует аккуратности и терпения. Несмотря на то, что создание снимков системы требует некоторого времени, пренебрежительное отношение к этим действиям может создать Вам значительные трудности в будущем. Во-вторых, существуют критические точки, при прохождении которых рекомендуется отслеживать состояние системы. В-третьих, можно составить универсальный план действий, который с очень высокой вероятностью позволил бы выявить изменения в системе, вносимые триальными механизмами, проанализировать их и выработать способы противодействия защите. Лично я чаще всего использую следующую последовательность действий, придерживаться которой рекомендую и Вам:

• Сделать снимки состояния системы до и после установки программы.
• Сделать снимок системы перед первым запуском программы, запустить программу, сразу же закрыть ее (чтобы в снимок системы не попали изменения, возникающие при работе с программой и не имеющие отношения к функционированию защиты) и тут же сделать еще один снимок системы. Если программа запускается после инсталляции автоматически, и выполнить этот пункт не представляется возможным, мы, тем не менее, сможем обнаружить изменения, произошедшие при первом запуске, сравнив снимки, сделанные в предыдущем пункте (другое дело, что мы не сможем отличить изменения, произошедшие при инсталляции, от модификаций, внесенных в систему программой при первом запуске).
• Запустить программу во второй раз, сделав снимки до запуска и после завершения программы. Если программа содержит ограничение на число запусков, путем несложного анализа мы сможем легко выявить счетчик запусков. Если программа содержит ограничение по времени использования, то мы можем установить, использует ли программа какие-либо дополнительные механизмы отслеживания времени. О наличии таких механизмов может свидетельствовать появление в реестре новых ключей или изменение уже существующих, появившихся в ходе инсталляции или первого запуска. Если такие ключи обнаружатся, есть вероятность, что программа кроме даты первого запуска отслеживает еще и дату предыдущего запуска, число дней, в течение которых использовалась программа или другую подобную информацию. Для максимальной надежности и достоверности результатов эту операцию можно повторить несколько раз.
• Выяснить, какие изменения в системе происходят, когда программу запускают впервые после истечения триального срока. Это необходимо для поиска меток, которые могли бы воспрепятствовать нормальной работе программы после окончания времени, отведенного на ее пробную эксплуатацию. Важно отметить, что для программ с ограниченным числом запусков необходимо отслеживать изменения не только при первом запуске сверх установленного лимита, но и при последнем "законном" запуске.
• По возможности, необходимо выяснить, как программа реагирует на попытки обойти ограничение времени использования при помощи изменения системной даты. Если программа успешно "проглатывает" любое изменение даты, высока вероятность того, что защита реализована максимально простым способом, и обойти ее будет нетрудно. Разумеется, все эти действия должны сопровождаться созданием снимков системы: если программа обнаружит Ваши манипуляции с системным временем, она может отреагировать на это созданием "метки", свидетельствующей о попытке обойти защиту.
• Проанализировать все собранные данные, выявить подозрительные ключи реестра и другие объекты, которые могли бы использоваться для хранения защитных данных, и затем на основе собранной информации попытаться восстановить работоспособность программы, последовательно удаляя или восстанавливая первоначальные значения подозрительных ключей.

• Проверить, нет ли в программе неочевидных ограничений по времени использования. Нередко встречаются программы (как правило, это демонстрационные, пробные и бета-версии), в которых помимо обычного ограничения на время пользования имеется дополнительная проверка на максимальную дату запуска, после которой программа считается устаревшей и перестает работать.

Вышеперечисленные приемы еще недавно считались "некрасивыми", и рассматривались как побочный результат исследования программы. Однако в последние годы ситуация начала меняться: лавинообразный рост количества защищенных программ привел к тому, что даже довольно известные группы обратили внимание на технологии продления сроков пробного пользования и выпускают launcher'ы, продляющие триальные сроки и сбрасывающие счетчики числа запусков. Чтож, изменившиеся условия требуют новых решений, и если эти решения эффективны, нет никаких причин от них отказываться.

Кроме того, серьезное исследование какой-либо защиты может потребовать неоднократного запуска приложения, и отведенное число запусков может закончиться раньше, чем удастся разобраться в программе. Кроме того, выявление счетчиков и "меток" - это один из путей поиска реализации триальных механизмов непосредственно в коде программы. К примеру, если Вы установили, что имя ключа, отвечающего за триальные ограничения, фиксировано, Вы можете попытаться найти имя этого ключа в тексте программы, затем выявить все ссылки на эту строку в программе и, в итоге, добраться до кода, реализующего защитные механизмы. Также при помощи API-шпиона можно проследить за вызовами функций обращения к реестру, проанализировать параметры, передаваемые этим функциям и таким образом попытаться выяснить, какой из этих вызовов непосредственно связан с реализацией ограничений в программе. А затем, зная, каким образом можно сбросить счетчик, ограничивающий возможность работы с программой, останется лишь воспроизвести этот эффект путем модификации кода программы.

Написание плагина — эмулирующего отладчика для дизассемблера IDA

Автор: Rustem

Дата: 2004-03-08

> _Каждой твари по паре
> Каждой виртуальной машине
> – по виртуальному процессору_

1. Вступление

Как-то в Интернете, наткнулся я на отрывок книги Криса Касперски. Вот цитата:

" !IDA DEBUGGER!

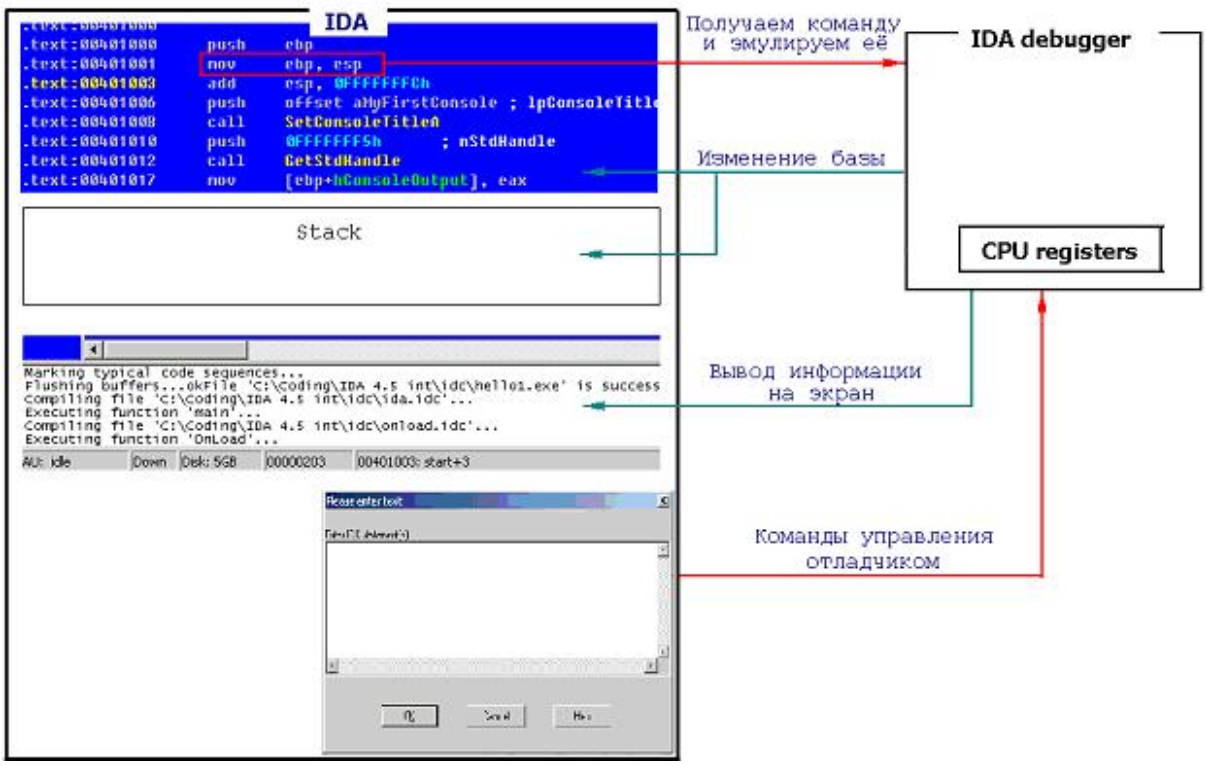
В чем новизна и удобство идеи? А в том, что можно сделать не полный, а *контекстный* отладчик! Что это такое? Обычный дебарег отлаживает всю программу целиком. Это хорошо, но чаще всего нас интересуют только выбранные фрагменты. В IDA можно подогнать курсор к нужному месту, задать начальные значения регистров и пусть на выполнение эмулятор. Такой подход прежде всего упорщает задачу, т.к. тут скорость не трубуется. А как удобно! Можно видеть работу кода в динамике! И не ломать голову какие будут значения регистров\флагов на выходе из такого и такого фрагмента и куда метнеться условный переход. Можно просто прогнать чисто локальный кусок с любой его точки."

Я понимаю, что отладчик для IDA, начиная с версии 4.5 существует, но во-первых не у всех она есть, во-вторых создать отладчик своими руками очень даже интересное дело, поможет поближе ознакомится с устройством IDA и её плагинов. Информация по написанию плагинов для дизассемблера есть в примерах SDK. Сразу надо сказать, что это будет контекстный эмулирующий отладчик 32 разрядного режима. Он будет тесно взаимодействовать с пользователем по части работы. Ну например если эмуляция какой-то команды отсутствует, то предлагается, что пользователь сам проэмулирует её и скорректирует значения регистров, стека.

Предлагаю свой проект, как один из вариантов. Писал я его из-за желания поглубже узнать IDA. Если его довести до ума, то может получится очень полезный program, который может помочь при ручной распаковке и анализе программ.

2. Общее построение

Лучше всего показать на рисунке. В принципе, подойдет не только для отладчика, но и для любого другого плагина.



Чтобы можно было корректно эмулировать, надо дополнительно создать сегмент стека в виртуальном пространстве дизассемблера-это необходимо для работы команд со стеком. Как его создать будет показано ниже.

Управление ведется с консоли дизассемблера-как вызов функций. Также можно организовать "горячие" клавиши. Вывод информации на экран - посредством окна сообщений и информативных окошек. Самое интересное - эмуляция команд. После подвода курсора к нужному адресу и ввода начальных значений в регистры, начинается работа отладчика.

В качестве подопытной была взята IDA 4.5 internal (лежала на www.crackbest.com). Также необходим SDK. Без SDK в создании плагинов делать нечего. Команды управления отладчиком были введены при помощи IDC скрипта - **"deb_comm.idc"**. Этот скрипт компилируется во время инициализации плагина (скрипт должен лежать в директории **idc** дизассемблера), если потерпит неудачу, то можно попытаться загрузить скрипт вручную из меню **File\IDC file**. Именно в этом скрипте находятся вызовы функций отладчика, можете повесить на функции "горячие клавиши", и добавить новые. Здесь показан формат вызова команды отладчика:

```
#define MY_FUNC_ID xx
static MY_NEW_COMMAND()
{
 RunPlugin("IDA_deb",MY_FUNC_ID);
}
```

Которую после добавления в **"deb_comm.idc"** можно будет вызывать прямо с консоли, например так: **MY_NEW_COMMAND();** . Только незабудьте перед этим вставить в функцию **run()** плагина обработчик этой команды, которая будет обрабатываться, если аргумент будет равен **MY_FUNC_ID**.

3. Немного об виртуальной памяти IDA

Для написания отладчика надо знать, об виртуальной памяти. Подробнее смотрите "Образ мышления IDA" Криса Касперски. Прошу простить меня за такое представление виртуального адресного пространства IDA и её базы данных, это упрощено, но отладчик работает именно с данными в виртуальной памяти, в которой лежит база данных. На рисунке показано соотношения между адресами (названия взяты от самого создателя IDA):



Линейный адрес (тип ea_t) - это адрес в виртуальном пространстве IDA Базовый адрес сегмента - это линейный адрес начала сегмента в виртуальном пространстве IDA Виртуальный адрес - это смещение между линейным адресом и базовым адресом сегмента: Виртуальный адрес=Линейный адрес - Базовый адрес сегмента Как вы уже , наверно, знаете, IDA работает со своим виртуальным пространством, а для дизассемблированной программы создаются сегменты со своими атрибутами (база, селектор разрядность и.т.п.), и с ним же будет работать и наш отладчик, через функции get_byte(..), get_word(), get_long, patch_byte, patch_word, patch_long - все они определены в BYTES.HPP, и там ещё много интересного есть. Линейный адрес вычисляется как

(Base <<4+смещение), где Base - база сегмента

4. Как эмулировать?

Думаю, для этого нужно узнать, как работает то, что мы будем эмулировать - т.е. центральный процессор. Если заглянуть в талмуды Intel, то можно найти примерно следующую последовательность действий по исполнению инструкций:

Like the Intel486 CPU, integer instructions traverse a 5 stage pipe-line. The pipe-line stages are as follows:

```
PF Prefetch
D1 Instruction Decode
D2 Address Generate
EX Execute - ALU and Cache Access
WB Writeback


T.e
-предвыборка команды
-декодирование команды
-генерация адреса, для определения адреса операндов в памяти
-выполнение опреаций с помощью АЛУ
запись результата
```

Придержимся этой последовательности и мы. Посмотрите на вариант эмулятора инструкций. Предвыборку, дешифрацию команд уже сделала IDA, нам остается только разобраться в операндах и вызвать обработчик команды.



Чтобы приступить к непосредственной эмуляции, давайте разберемся, как IDA хранит инструкции. Для этого имеются несколько структур, они описаны в файле INCLUDE\UA.hpp. Этот файл ОЧЕНЬ важен при написании эмулятора, т.к. в нем описывается формат данных команды, её операндов их типы.

```
// Хранит информацию о команде

idaman insn_t ida_export_data cmd;      // текущая инструкция


// Класс описывающий инструкцию (сокращено)
class insn_t
```

```
{
public:

  ushort itype; // Код инструкции (Инициализируется в 0 ядром
// IDA  все инструкции содержатся в директории
// INCLUDE\allins.hpp

  ulong cs;  // База сегмента, в котором находится
// инструкция (Инициализируется ядром IDA
  ulong ip;  // Виртуальный адрес инструкции (адрес ВНУТРИ
// сегмента) блин, от этих виртуальных адресов
// уже сам начинаешь виртуалиться //(Устанавливается ядром
// ИДА, а кем-же ещё)

  ea_t ea;  // Линейный адрес инструкции

  ushort size; // Размер инструкции в байтах


// Сведения об операндах инструкции
#define UA_MAXOP 6
  op_t Operands[UA_MAXOP]; // Массив содержит структуру описывающую каждый
// операнд в команде. К нему удобно обратится
#define Op1 Operands[0]  // ,например, так: op_t x= cmd.Op1-получаем ин-
#define Op2 Operands[1]  // формацию об первом операнде (странно, что не
#define Op3 Operands[2]  // нулевой)
#define Op4 Operands[3]
#define Op5 Operands[4]
#define Op6 Operands[5]
};

// Класс - тип операнда, (показано неполностью)
class op_t
{
public:
  char n;  // Порядковый номер операнда

// Структура описывающая тип операнда (очень интересное поле)
  optype_t     type;


// Тип значения операнда

  char          dtyp;

#define dt_byte         0       // 8 bit
#define dt_word         1       // 16 bit
#define dt_dword        2       // 32 bit
#define dt_float        3       // 4 byte
#define dt_double       4       // 8 byte
#define dt_tbyte        5       // variable size (ph.tbyte_size)
#define dt_packreal     6       // packed real format for mc68040
#define dt_qword        7       // 64 bit
#define dt_byte16       8       // 128 bit
#define dt_code         9       // ptr to code (not used?)
#define dt_void         10      // none
#define dt_fword        11      // 48 bit
#define dt_bitfild      12      // bit field (mc680x0)
#define dt_string       13      // pointer to asciiz string
#define dt_unicode      14      // pointer to unicode string
};

Ну и, наконец, описание типа операнда
typedef uchar optype_t;
const optype_t     //                                Поле в op_t
  o_void     =  0, // нет операнда                         ----------
  o_reg      =  1, // Основной регистр (al,ax,es,ds...)    reg
  o_mem      =  2, // Прямое обращение к памяти            addr
  o_phrase   =  3, // Memory Ref [Base Reg + Index Reg]    phrase
  o_displ    =  4, // Memory Reg [Base Reg + Index Reg + Displacement] phrase+addr
  o_imm      =  5, // Непосредственное значение            value
```

```
  o_far      =  6, // Непосредственный дальний адрес       addr
  o_near     =  7, // Непосредственный ближний адрес       addr
  o_idpspec0 =  8, // IDP specific type
  o_idpspec1 =  9, // IDP specific type
  o_idpspec2 = 10, // IDP specific type
  o_idpspec3 = 11, // IDP specific type
  o_idpspec4 = 12, // IDP specific type
  o_idpspec5 = 13, // IDP specific type
  o_last     = 14; // first unused type

  char          specflag1;
  char          specflag2;
  char          specflag3;
  char          specflag4;
}
```

В структуре optype_t есть следующие поля:

char specflag1;
char specflag2;
char specflag3;
char specflag4;

Эти флаги заполняются ядром ИДА (точнее процессорным модулем). Как заполняются не сказано, но, проведя эксперименты, с различными инструкциями и способами адресации, удалось выяснить вот что:

Флаг specflag1 и все остальные равен нулю если:
1. Имеется прямая адресация (**OpX.type=o_mem**). Т.е. например **mov eax,[401000]**
2. Косвенная адресация (**OpX.type=o_phrase**) **mov eax,[eax]**
3. Косвенная адресация со сдвигом (**OpX.type=o_displ**) **mov eax,[eax+402031]**
Если **specflag1==1** , то в **specflag2** содержится информация об составляющих операнда (например при индексной адресации с масштабированием) Если **OpX.type=o_phrase** , то **specflag2** будет расшифровываться так:

7	6	5	4	3	2	1	0
Коэффициент масштабирования	Индексный регистр	Базовый регистр					

Кодировка регистров в точности, как принята у Intel:

EAX	0
ECX	1
EDX	2
EBX	3
ESP	4
EBP	5
ESI	6
EDI	7

Коэффициент масштабирования определяет степень двойки, на которую нужно умножить значение индексного регистра:

	Коэф.Умножения
00	1
01	2

	Коэф.Умножения
10	4
11	8

Например :

```
lea     eax, [ebx+ecx*4]

Op2 type is: 3
Op2 dtyp is: 0
Op2 value is: 0
Op2 reg is: 4
Op2 phrase is: 4
Op2 address is: 0
Op2 offb is: 0
Op2 offo is: 0
Op2 specflag1 is: 1
Op2 specflag2 is: 9B
Op1 specflag3 is: 0

Op2 specflag2 is: 9B  10001011
10-4
011-ebx
001-ecx
```

Если **OpX.type=o_mem** , то specflag2 будет расшифровываться так:

7	6	5	4	3	2	1	0
					1	0	1
Коэффициент масштабирования	Регистр	Точно не выяснил, но всегда равно константе					

Если **OpX.type=o_displ** , то specflag2 будет расшифровываться так:

7	6	5	4	3	2	1	0
Коэффициент масштабирования	Индексный регистр	Базовый регистр					

Также в этом случае, в поле **OpX.addr** , будет находится значение смещения

Однако здесь есть одно исключение для обращения к памяти вида: **[esp+xxx]** В этом случае **specflag1=1; specflag2=0x24**

Разберемся с сегментами IDA. Сетменты в IDA-один из её краеугольных камней. И надо ознакомится со структурами, которые их описывают: Нам они понадобятся для создания сегмента стека отладчика. Без него работать, мягко говоря некорректно. Каждый сегмент имеет базовый сегментный адрес, который определяет положение сегмента в виртуальной памяти IDA. В поле Base может находится базовый адрес, а может находится индекс селектора. Селектор содержит 32-битный базовый адрес сегмента в виртуальном пространстве IDA. Подробнее смотрите "Образ мышления IDA" Криса Касперски.

Данные и функции работающие с сегментами содержатся в заголовочном файле **SEGMENT.HPP** . К сегментам можно обращаться разными способами. Тип структуры, описывающею сегмент segment_t. В ней содержится вся информация о сегменте: База, начальный адрес, конечный адрес, имя сегмента, выравнивание сегмента, разрядность 16 или 32 бит, и.т.д.

```
class segment_t : public area_t
{
public:
```

```
 long name;     // имя сегмента
 long sclass; // класс сегмента
 long orgbase; // это поле зависит от IDP
 uchar align;
 uchar comb;     // Код комбинирования сегмента с другими
 uchar perm;     // "Разрешение" сегмента (EXE, READ, WRITE)
 uchar use32; // разрядность сегмента
 ushort flags; // флаги сегмента
 sel_t sel;     // селектор сегмента
 uchar type;     // Тип сегмента
};
```

Вот как примерно выглядит создание сегмента стека для отладчика. Подробное описание функций смотрите в **SEGMENT.HPP**

```
// Находим пустой селектор и проецируем на виртуальную память
// N - селектор для сегмента
 sel_t N=allocate_selector(0);
// Создаем сегмент
 if(add_segm(N,0xFFFF0000,0xFFFFFFFE,"STCK","STACK"))
 {
  segment_t DEBUGGER_STACK_SEGMENT, *pDSS;
  pDSS = &DEBUGGER_STACK_SEGMENT;

  pDSS = get_segm_by_sel(N);
  set_segm_addressing(pDSS,1);

  msg("Debugger stack created\n");
 }
 else warning("Cant create segment\n");
```

Как видите, ничего сложного нет.

5. Об обработчике команд

В эмуляторе всю работу по исполнению инструкций выполняют обработчики команд. Было решено в обработчик передавать тип соотношения операндов (например: один операнд - регистр, один операнд в памяти, один регистр - другой в памяти). В исходниках это выполняет функция **get_operand_info(OPER &)**, которая заполняет структуру **OPER**. Также она заполняет поля структуры: вычисленными значениями операндов (например, при косвенной адресации). Сам обработчик команды знает о том какие операнды и какие сочетания операндов возможны и в зависимости от этого выполняется. Покажу на примере инструкции push:

```
//  push instruction
BYTE emPUSH(OPER &O){
//  Выводить о работе команды, как вы понимаете не обязательно.
//  Просто удобно при отладке обработчика
 msg("This is a PUSH cmd emulation\n");

//  В стеке есть место??
 if(!checkESP(-4)) return false;

//  Проверка, если один из операндов в памяти, на принадлежность к
//  несуществующему адресу.
 if(!checkEA(O))   return false;

 EIP2NextCmd(O);
//  Операнд один - непосредственное значение 32бит
 if(O.rel==i32) {
  *pemESP-=4;   //Уменьшили ESP
  return (WriteVMDWORD(*pemESP,O.op0_i32));//Пишем в вирт. память
 }

//  Операнд один - непосредственное значение 16бит
 if(O.rel==i16) {
```

```
  *pemESP-=2;
  return (WriteVMWORD(*pemESP,O.op0_i16));
 }
// Операнд один - непосредственное значение 8бит
 if(O.rel==i8) {
  *pemESP-=2;
  return (WriteVMWORD(*pemESP,(WORD)O.op0_i8));
 }
// Здесь должна быть проверка на опернды - регистры и операнды, находящиеся в // памяти
//    if(O.rel==r32). . .
//    if(O.rel==r16). . .

//    if(O.rel==m32). . .
//    if(O.rel==m16). . .

// Если мы здесь, то операнды неопознаны
 warning("UNKNOWN OPERAND IN PUSH CMD");
 return INS_BAD_OP;
}
```

Это одна из "частных" инструкций и для других писать обработчик намного проще. Например эмуляция инструкции cmp

```
//  cmp
void __declspec(naked) emCMP8() {_asm {cmp al,bl
            retn}
    }
void __declspec(naked) emCMP16(){_asm {cmp ax,bx
            retn}
    }
void __declspec(naked) emCMP32(){_asm {cmp eax,ebx
            retn}
    }

//////////////////////////////////////////////////////
// cmp
//////////////////////////////////////////////////////
bool emCMP(OPER &O){
 if(!checkEA(O))   return false;

 EIP2NextCmd(O);
 return two_op_handle(O, emCMP8, emCMP16, emCMP32);
}
```

Вот и весь обработчик инструкции. Вся работа будет происходить в **two_op_handle** , которая и учтет типы операдов их типы, и воздействие на флаги.
Перед написанием обработчика инструкции, неплохо бы узнать, что возвращает структура **cmd** , так как может отличаться для разных инструкций. Например для инструкции **lea eax,[ebx] cmd.Op1.dtyp = dt_dword** , а **cmd.Op1.dtyp = dt_byte**. Поэтому ,возможно, придется добавить обработку ситуации в функцию **get_operand_info**.

При написании обработчиков ОЧЕНЬ неплохо помогает справочник из книги Юрова "Ассемблер", там расписаны все типы операндов команд и алгоритм их работы.

6. Послесловие

На данный момент поддерживается:
1) Точки останова BPX,
2) BPM_R, BPM_W, BPM_RW
3) Эмулировано 122 инструкции x86 (кроме 3-х операндных imul, shld, shrd, enter, leave и др., их можете реализовать сами.)

Команды управления отладчиком можно уточнить как **help();** из консоли IDA.

Надо добавить: 1) Создать нормальный показ регистров в отладчике в виде отдельного диалогового окна
2) При желании можно добавить эмуляцию FPU, MMX
Самое серьезное, что непонятно, как эмулировать вызовы WinAPI функций. Похоже, что неполучится, но в принципе в контекстном отладчике это и ненужно, хотя было бы неплохо.

В приложении содержится исходный код отладчика, а также скомпилированная версия плагина для IDA 4.5 Похоже, что файл plugins.cfg необязателен (ИДА 4.5), т.к. и без него плагин вызывался нормально. Подозреваю, что ИДА сканирует директорию с плагинами и узнает параметры через структуру PLUGIN. Если неполучится, то добавьте следующую строку:
CDEx86 CDEx86 F11 0
, также можно добавить горячую клавишу к плагину.

Первые шаги по работе:
1) Скопируйте CDEx86.plw в папку \plugins дизассемблера.
2) Запустите дизассемблер
3) Приведите курсор к куску кода, который хотите изучить, и начинаите трассировку (по умолчанию клавиша F11)
4) Значения регистров можно посмотреть вызвав r(); из консоли (Shift+F2). В следующей версии ,возможно, будет отдельное окно с регистрами.
5) Если хотите автоматическую трассировку, то подведите курсор к месту, где должна остановится трассировка и введите из консоли точку останова.
bpx();
at();

Точно так устанавливаются виртуальные бряки на чтение\запись в память bpm();
Поработав со своим творением, у меня появились некоторые рекомендации по работе с ним:
1) Отладчик предполагает хорошие навыки работой с IDA.
2) Не надо лезть в вызов WinAPI функции, т.к. ее кода нет в дизассемблере и соответственно нельзя ее трассировать. Лучше всего перепрыгнуть её (**cheip();**), сбалансировав стек.
3) Отладчик получает инструкцию по адресу курсора IDA. Поэтому если вы прыгнете на середину команды, то получите сообщение Not code. Stay at code first. В этом случае надо сперва превратить в Unknown, а затем превратить в Code.
4) Необходимо создать окно, в которое выводить информацию. В первую очередь - значения регистров Т.к. неудобно каждый раз вызывать команду r();

Ненадо требовать от контекстного отладчика сверхъестественного. Я писал его лишь из-за интереса к внутреннему устройства IDA. Если чего-то нехватает, добавьте сами - исходники есть.

Во время работы с плагином, появилась идея использовать его для ручной распаковки програм:
Алгоритм примерно выглядит следующим образом:
1. Загружаете файл как бинарный
2. Применяете PE_scripts by Atli Gudmundsson (есть на wasm.ru)
3. Пробуете трассировать его с точки входа
4. Сохраняете на диск заголовок и секции при помощи скрипта pe_write.
(А еще лучше написать свой. Т.к. больше шансов получить работоспособный файл)

И напоследок, хотелось бы сказать большое спасибо Крису Касперски, за его отличные книги и статьи.
, а также создателям WASM.RU за то, что есть такой замечательный ресурс

7. Литература

1. pilorama.com.ru
2. Юров В., Хорошенко С. Ассемблер: учебный курс
3. Зубков С.В. Ассемблер язык неограниченных возможностей

Жду поправок и более крутых решений

Исходник (находится в архиве `wasm_files.zip`: `files/recovered/306/emul_plug_ida.zip`) к статье.

Теоретические основы крэкинга: Глава 4. Переменные и константы.

Автор: CyberManiac

Дата: 2004-03-10

Одна из первых задач, с которыми сталкивается крэкер - поиск в тексте программы констант или обращений к переменным, отвечающих за реализацию ограничений в незарегистрированных и демонстрационных программах. Нередко также возникает необходимость извлечь из программы какие-либо данные, либо заменить их своими собственными. Поскольку в данной главе пойдет речь не только об изучении, но и об изменении программ, сразу же ознакомлю Вас с одним из важнейших, с моей точки зрения, принципов, которым нужно руководствоваться, если приходится прибегать к модификации данных или исполняемого кода. Это **принцип минимального вмешательства**, который можно сформулировать так: *Чем меньше изменений внесено в логику программы, тем менее вероятно, что эти изменения приведут к некорректной работе программы.*

Приблизительный рейтинг изменений по степени их влияния на логику программы (от минимального к максимальному) выглядит следующим образом:

- Модификация константы

- Изменение значения предварительно инициализированной переменной

- Изменение условия перехода на противоположное

- Удаление линейной, то есть не содержащей ветвлений и вызовов нетривиальных подпрограмм, последовательности команд

- Дописывание в программу собственного кода; изменение значения, возвращаемого функцией

- Модификация внедренного в программу ресурса, важного для логики программы (т.е. такого, изменение которого меняет поведение программы)

- Удаление из программы логического блока (например, вызова нетривиальной функции); удаление внедренного в программу ресурса

Этот принцип, как любое другое широкое обобщение, требует достаточно осторожного отношения и в некоторых случаях может повести Вас по неоптимальному пути. Но на практике принцип минимального вмешательства обычно работает вполне успешно. Да и с точки зрения простого здравого смысла очевидно, что найти и исправить условие, которое проверяет зарегистрированность программы – прием куда более безобидный и надежный, чем вырывание с корнем nag-screen’ов и триальных ограничений при помощи глубокого патчинга.

Как я уже говорил, многие незарегистрированные программы содержат количественные ограничения. В предыдущей главе мы как раз рассматривали некоторые разновидности таких ограничений и способы их обхода. Но, к сожалению, есть немало случаев, когда манипуляции с системным временем, файлами и реестром не помогают: это всевозможные ограничения на продолжительность одного сеанса работы с программой, на максимальное количество создаваемых или обрабатываемых документов, число записей в базе данных и т.п. Даже число жизней в компьютерной игре является одним из таких ограничений (как ни странно, крэкинг иногда применяется в качестве средства для беспроблемного прохождения компьютерных игр; «вечная жизнь» и «бесконечное оружие» - это тоже один из аспектов крэкинга, возможно даже самый старый).

Наверное, наиболее распространенным типом данных, используемым в программах, в настоящее время являются целочисленные данные. Абсолютное большинство существующих процессоров, в том числе и наши любимые x86, изначально ориентировались на работу с целыми числами определенной разрядности (в настоящее время на платформе x86 наиболее актуальны 32-разрядные целые со знаком или без). Именно в виде целых чисел чаще всего хранятся всевозможные счетчики, контрольные суммы и даже логические значения (когда я переходил от программирования под ДОС к программированию под Win32, меня сильно удивляла расточительность фирмы Microsoft, отводившей под простую булевскую переменную целых 4 байта. И только более глубокое изучение архитектуры 32-разрядных процессоров и ассемблера расставило все по своим местам).

Что Вы будете делать, если Вам потребуется найти в программе сравнение чего-либо с целочисленной константой с определенным значением? В принципе, для некоторых целых чисел достаточно обычного поиска блока двоичных данных в файле. Однако двоичный поиск хорошо работает далеко не со всеми числами: скажем, если Вы ищете число 3B9ACA00h, вероятность ложного срабатывания будет весьма небольшой, а вот если Вы попытаетесь найти в исполняемом файле число 10 или 15, Вы, скорее всего, просто устанете нажимать на кнопку «Найти далее». Кроме того, при таком способе поиска не учитывается структура исполняемого файла программы, то есть Вы ищете нужную Вам константу не только в коде программы, но и в PE-заголовке, секции данных, секции ресурсов и прочих областях, имеющих к коду программы самое отдаленное отношение.

Однако есть и другой, более эффективный метод поиска в программе известных заранее значений. Как это ни странно, но в реальных программах широко используется лишь сравнительно небольшой набор целочисленных констант: это, прежде всего, небольшие положительные числа от 0 до 7 (а также небольшие отрицательные от -3 до -1) и степени двойки: 8, 16, 32 и т.д. Другие константы в программах встречаются значительно реже. Попробуйте сами провести эксперимент – дизассемблируйте какую-нибудь достаточно большую программу и попробуйте найти в ней какое-нибудь число, например, 32h. Для демонстрации этого я написал программку на Delphi 7, вся функциональность которой концентрировался в следующем коде, имитирующем простейшее ограничение на число строк в документе:

```
procedure TForm1.Button1Click(Sender: TObject);
begin
 if Memo1.Lines.Count>50 then
   begin
    Application.MessageBox('More than 50 items not available','Demo version');
    Close;
   end
  else Memo1.Lines.Add(Edit1.Text);
end;
```

В результате компиляции этот весьма нехитрый текст превратился в исполняемый файл размером более 350 килобайт (я намеренно использовал режим компиляции без использования runtimepackages, и потому мой собственный код составлял в исполняемом файле очень малую долю по сравнению с библиотеками Delphi). Затем я дизассемблировал откомпилированную программу при помощи W32Dasmи получил листинг текст длиной более 180 000 строк. Казалось бы, обнаружить область, где происходит сравнение с числом 50 в этом листинге ничуть не проще, чем найти иголку в стоге сена. Но я воспользовался функцией поиска в тексте строки, в качестве параметра поиска указав 00000032 (так в W32Dasmотображается число 50; заодно это позволило отсеять команды типа moveax,[ebx+50h], обычно использующиеся для доступа к элементам массивов и полям структур). Реальность превзошла самые смелые ожидания: четырехбайтное число 32hвстретилось в листинге всего два (!!!) раза:

```
:004478C6 6A32                  push 00000032
и
:004505BB 83F832                cmp eax, 00000032
```

Чтобы догадаться, что нужное нам сравнение происходит во второй строке, достаточно самых минимальных познаний в ассемблере. Из этого следует вывод: поиск нужной константы в дизассемблированной программе вполне реален, даже несмотря на огромный объем листинга.

Далее: Вам наверняка интересен не сам факт наличия константы где-то в недрах кода, а то, в каком контексте эта постоянная используется. Иными словами, если Вы знаете, что программа сравнивает число записей в базе данных с некоторым значением (в нашем случае - 32h), то среди всех строк, в которых присутствует эта константа, в первую очередь следует рассматривать команды сравнения (cmp) и вычисления разности (subи sbc), и лишь затем – все прочие, менее очевидные способы сравнения вроде

```
mov ebx,50
cmp eax, ebx
```

или

```
push 50
push eax
call Compare2dwords
```

Ну и, раз уж речь зашла о сравнениях, нельзя не упомянуть об альтернативных вариантах реализации этой, казалось бы, нехитрой операции. Поразмыслим над приведенным выше примером сравнения содержимого регистра eax с числом 50. В самом деле, условия eax>50 и eax>=51 в приложении к целым числам имеют один и тот же смысл, а код

```
cmp  eax,50
jg my_label
```

работает совершенно аналогично коду

```
cmp eax,51
jge my_label
```

Если же eахнеобходимо сравнить с числом 31, проверка может выглядеть даже так:

```
and eax, 0FFFFFFE0h
jnz my_label
```

То есть, проверка одного и того же условия может быть реализована несколькими разными способами, и когда Вы будете искать константы в дизассемблированном тексте, этот факт тоже надо учитывать.

Если рассматривать области возможного практического применения вышеописанного приема, то лучше всего поиск известной константы в дизассемблированном тексте работает на триальных ограничениях типа «не допускается создание больше Nэлементов в базе данных». Как правило, Nбольше 7 и является целым числом, что облегчает поиск нужной константы. Исходя из принципа минимального вмешательства, для обезвреживания таких ограничений я предпочитаю исправлять не команды сравнения, а сами константы. Действительно, если программа для проектирования интерьера комнаты не способна работать более, чем с 20 объектами, для практического применения она вряд ли будет пригодна. Но вот та же программа, где максимальное количество обрабатываемых объектов увеличено до двух с хвостиком миллиардов наверняка удовлетворит даже самого взыскательного пользователя.

Одним из наиболее частых вопросов, возникающих у начинающего крэкера, звучит так: «Программа работает 30 дней, но я так и не нашел в листинге сравнения с числом 30. Что делать?». Один из факторов я уже описал выше – там могло быть сравнение не с числом 30, а с числом 31. Однако этим список возможных причин неудачи не исчерпывается. Как мы все знаем, день состоит из 24 часов, каждый из которых состоит из 60 минут, каждая из которых состоит из 60 секунд. Более того, продолжительность секунд также может измеряться во всевозможных «условных единицах», например, в миллисекундах. Тысячные доли секунд, в частности,

используются в таких функциях WinAPI, как SetTimer (таймеры Windowsчасто используются для установки ограничений на продолжительность одного сеанса работы с программой) или Sleep. А вот в функциях, возвращающих время в виде структуры типа FILETIME, используются уже другие «условные единицы», равные ста наносекундам. Так что пресловутые 30 дней – это не только 30 дней, но еще и 720 часов, 43200 минут, 2592000 секунд, ну и так далее. И каждое из этих значений может быть использовано в программе как один из аргументов операции сравнения. Надо отметить, что в «условных единицах» может быть представлено не только время, но и многие другие величины: масса, географические координаты, денежные единицы и т.д.

Раз уж речь зашла о представлении временных отрезков внутри программ, уместно будет рассказать и о тонкостях использования таймеров. Наверняка Вы встречали программы, в которых после запуска окно с предложением о регистрации висит на экране в течение некоторого времени (иногда еще в этом окне идет обратный отсчет секунд), и при этом его невозможно закрыть. Подобные «спецэффекты» по усмотрению разработчика могут сопровождать вызов каких-либо функций программы, сохранение файлов или завершение приложения – это не суть важно. Важно другое: все эти временн **ы** е задержки так или иначе используют средства измерения времени. В ОС Windowsсуществует два наиболее популярных способа отсчета отрезков времени: использование функций задержки (в частности, функции Sleep) и использование всевозможных таймеров.

Вообще в Windowsсуществует несколько разновидностей таймеров – кроме обычного таймера, создаваемого функцией SetTimer, существует еще высокоточный мультимедийный таймер и специфические таймерные функции DirectX. Эти таймеры срабатывают с некоторой заданной частотой, вызывая функцию-обработчик (она же callback-функция), внутри которой и выполняются необходимые действия, например, тот же обратный отсчет секунд до исчезновения окна с предложением зарегистрироваться. Периодичность срабатывания таймера почти всегда является константой, однако взаимосвязь между тем, что происходит внутри программы и тем, что Вы можете видеть на экране, не всегда очевидна. Чтобы пояснить эту мысль и заодно продемонстрировать на практике, как можно обращаться с таймерами, приведу несколько примеров.

Первый пример – простейший: программа, которая при запуске в течение пяти секунд показывала баннер, при этом поверх баннера выводился обратный счетчик секунд. Регистрация в программе не предусматривалась. Дизассемблирование показало, что таймер срабатывает каждые 1000 миллисекунд, при каждом вызове callback-функции значение переменной, изначально равной пяти, уменьшалось на единицу, и результат проверялся на равенство с нулем. В той конкретной программе баннер можно было просто «выломать», убрав функцию создания и отображения рекламного окна, но в общем случае это решение было бы не лучшим (вспомните принцип минимального вмешательства). И вот почему: на последнее срабатывание таймера могло быть «подвешено» не только закрытие окна с баннером, но и инициализация каких-либо объектов внутри программы или другие критичные действия, без которых программа могла бы работать некорректно. Так что немного усложним задачу – будем считать, что полностью убирать вызов окна с рекламой нельзя. Первое, что нам приходит в голову – уменьшить число секунд, в течение которых показывается баннер. Сказано – сделано, цифру 5 исправляем на единицу. Однако баннер все равно висит целую секунду – ведь первое срабатывание таймера наступает только через секунду после его создания. Теперь уменьшим период таймера до нуля (хотя лучше все-таки до одной миллисекунды, «таймер с периодом 0 миллисекунд», согласитесь, штука довольно странная). В результате мы получили баннер, появляющийся при запуске программы лишь на мгновение и не заставляющий тратить целых пять секунд на праздное разглядывание рекламных лозунгов.

В качестве второго примера я возьму одну из старых версий TVTools. В справке к программе было четко указано, что незарегистрированная версия работает только 10 минут; дизассемблирование и

анализ листинга выявили, что программа создает два таймера с периодами 60 секунд (что навело меня на мысли о защитном назначении этого таймера) и 2 секунды. Без особых сложностей обезвредив первый таймер, я запустил программу и обнаружил, что она все равно больше 10 минут не работала. Тогда я более пристально изучил callback-функцию второго таймера, и наткнулся в ней на такой код:

```
inc     dword_40D5A7
cmp     dword_40D5A7, 136h
jbe     short loc_405CAA
```

Нетрудно догадаться, что это увеличение некоего счетчика, который затем сравнивается с числом 310. Поскольку период таймера – 2 секунды, а 310*2=620 (т.е. чуть больше 10 минут), логично было предположить, что это и есть второй уровень защиты, дублировавший первый. Очевидно, что если бы я принял на веру, что программа перестает работать ровно через 10 минут (а не через 10 минут 20 секунд, как это оказалось в действительности) и стал бы искать сравнение с числом 300, я бы не смог обнаружить таким способом вторую проверку времени работы программы. Этот пример демонстрирует один из неочевидных приемов, который может быть использован для реализации такой, казалось бы, простой операции, как отсчет 10-минутного интервала. Также из этих примеров следует и другой, не менее важный вывод: далеко не всегда следует искать известную константу, чтобы найти код, в которой она используется. Иногда следует поступать прямо противоположным образом – сначала искать код, выполняющий нужные действия, и лишь затем выяснять, какая константа внутри этого кода ответственна за интересующие нас действия.

Поиск констант с плавающей точкой – занятие с одной стороны более сложное, чем поиск целочисленной константы, но с другой – куда более простое. В чем сложность и в чем простота этого занятия? По традиции начнем с плохого. Во-первых, формат представления чисел с плавающей точкой весьма нетривиален, и Вы вряд ли сможете в уме привести шестнадцатиричный дамп такого числа в «человеческий» вид (возьмите документацию по процессорам Intel и попробуйте перевести число 1.23 в машинное представление, а затем проделать обратную операцию – Вы сами убедитесь, насколько сложна эта задача). Более того, даже целые числа в представлении с плавающей точкой выглядят весьма неординарно: к примеру, дамп самого что ни на есть обычного числа 123, приведенного к типу Double, выглядит как 00 00 00 00 00 C0 5E 40. Если Вы способны с первого взгляда отличить число с плавающей точкой от кода программы или каких-либо иных данных и оценить величину этого числа – я рад за Вас, но большинство людей, к сожалению, такими способностями не обладают.

Во-вторых, при работе с дробными числами нередко возникают проблемы, связанные с машинным округлением и потерей точности. Самым ярким примером, наверное, может служить особенность математических программ ПЗУ некоторых моделей Spectrum: с точки зрения такого Спектрума выражение 1/2=0.5 было ложным. Это, конечно, было давно, но не следует считать, что современные компьютеры полностью свободны от этой проблемы. И вот практическое тому подтверждение.

Откомпилируйте под Delphi следующий код: i:=sin(1); i:=arcsin(i) и посмотрите, как будет меняться результат при изменении типа переменной Iот Singledo Extended. Например, если I имеет тип single, в результате вычислений получим, что arcsin(sin(1))=0,999999940395355. Такие «спецэффекты» – следствие все той же потери точности в процессе вычислений.

В-третьих, округлением чисел процессор может заниматься не только по собственному желанию, но и по велению программы. К примеру, в большинстве бухгалтерских программ всевозможные ставки налогов выводятся с точностью до копеек. Однако из того, что Вы видите на экране число 10.26, совершенно не следует, что результат расчетов представлен в памяти ЭВМ именно как 10.26. Реальное значение соответствующей переменной может быть равно 10.258 или 10.26167, которое и участвует в реальных расчетах, и лишь при выводе на экран для удобства пользователя было произведено округление до двух знаков после запятой.

Я не случайно столько места уделил округлению и точности представления чисел – именно эти особенности чисел с плавающей точкой в наибольшей мере затрудняют поиск нужных значений в памяти программы. Программисты знают, что при работе с действительными числами для проверки условия равенства некоторой вычисляемой величины другой величине не рекомендуется использовать сравнения вида f(a)=b. Причина этого лежит все в той же проблеме округления и потери точности расчетах – вспомните вышеприведенные примеры со Спектрумом или арксинусом синуса единицы. Вместо простой проверки равенства обычно используется условие «значения считаются равными, если абсолютная величина разности между ними не превышает некоторой величины»: abs(f(a)-b)<=delta, где delta – максимально допустимая величина разности, после которой числа не считаются равными. Поэтому если Вы хотите найти в памяти некоторое число с плавающей точкой F, Вы в действительности должны искать все числа из промежутка [F-delta; F+delta], причем определить значение deltачаще всего можно лишь опытным путем. Это утверждение распространяется и на тот случай, когда Вы знаете округленное значение переменной, но в этом случае величина deltабудет зависеть от того, до скольки знаков округлено значение переменной. Так, если число округлено до сотых, нетрудно догадаться, что delta=0.005.

Вот тут-то Вы и столкнетесь с чисто практической проблемой неприспособленности существующих отладчиков к поиску чисел с плавающей точкой. Сама по себе функция поиска действительных чисел в адресном пространстве программы в отладчиках встречается редко, а уж отладчиков, поддерживающих поиск чисел из заданного промежутка, я вообще не встречал. Поэтому Вам, вероятно, придется для этой цели написать собственный инструмент либо искать способ получить нужную информацию каким-то другим путем.

И, наконец, нельзя забывать, что кроме стандартных для платформы x86 типов Single, Doubleи Extended (32-, 64- и 80-битных соответственно) существует еще несколько довольно экзотических, но все еще используемых форматов. Это, к примеру, Currency (64-битные, с фиксированным положением десятичной точки) или 48-битные паскалевские Real. Возможно также использование «самодельных» форматов; особенно часто встречаются числа с фиксированным положением десятичной точки (обычно такое делается для повышения скорости работы программы и применяется в основном в процедурах кодирования/декодирования аудио- и видеоинформации). Знать о таких вещах совсем не лишне, хотя, конечно, вероятность столкнуться с такими числами в современных программах довольно низка.

Теперь немного поговорим о том хорошем, что есть в числах с плавающей точкой. Как известно, изначально в процессорах x86 встроенных аппаратных и программных средств для обработки чисел с плавающей точкой не предусматривалось. Низкая скорость расчетов, в которых использовались действительные числа, вызвала к жизни математические сопроцессоры, как традиционные x87, так и весьма экзотические девайсы Weitek. Победившая линейка сопроцессоров x87 (они с некоторых пор стали интегрироваться в ядро процессора и потому перестали существовать как отдельные устройства) имела следующую особенность: новые «математические» команды активно использовали для обмена информацией оперативную память. Посмотрите, к примеру, на важнейшие команды сопроцессора fstи fld – в качестве параметра этих команд могут выступать указатели на области памяти, которые предполагается использовать для чтения/записи данных. Более того, использование указателей в качестве одного из параметров характерно и для многих других команд сопроцессора. Поэтому ищите ссылки, используемые командами сопроцессора в качестве параметров – и Вы легко доберетесь до данных, на которые эти ссылки указывают.

Из этого следует вывод: хороший дизассемблер или отладчик способен «догадаться», что по адресу, указанному в аргументах этих команд, находится число с плавающей точкой и отобразить это число. Если же Ваш дизассемблер/отладчик об этом не догадывается – Вам придется вручную (точнее говоря, при помощи соответствующих программ) вычислить значение, которое находится по этому адресу. И пока Вы будете копировать байтики из одной программы в другую, у Вас будет

достаточно времени подумать об обновлении инструментария.

Но и здесь не обошлось без ложки дегтя – компиляторы фирмы Borland, видимо, ради особой оригинальности, для загрузки констант в стек сопроцессора могут воспользоваться комбинациями вроде

```
mov [i],$9999999a
mov [i+$4],$c1999999
mov word ptr [i+$8],$4002
fld tbyte ptr [i]
```

Хотя, казалось бы, ничто не мешало положить несчастное число в секцию инициализированных данных… Тут уж не до «умного» поиска – разобраться бы, чего и куда вообще загружается. Хотя, при желании и умении обращаться с регулярными выражениями (или умении программировать) можно искать даже в таком коде.

Другим свойством действительных чисел, облегчающим автоматический поиск известной величины, является само их внутреннее устройство. Достаточно большая длина этих чисел (32 бита, а чаше всего – 64 или 80) и сложный формат хранения позволяет искать числа с плавающей точкой в любых файлах, в том числе и в исполняемых файлах программ, непосредственно в двоичном виде, причем вероятность ложного срабатывания будет незначительной. Даже существование нескольких различных форматов представления действительных чисел не представляет серьезного препятствия – соответствующая программа очень проста и пишется за считанные минуты. Народная мудрость гласит: «лучше один раз увидеть, чем сто раз услышать», поэтому в качестве практики я рекомендую Вам самим написать и отладить такую программу – это не только усовершенствует Ваши навыки в программировании, но и позволит поближе познакомиться с миром действительных чисел. Затем, если захотите, Вы сможете доработать эту программу таким образом, чтобы она могла осуществлять нечеткий поиск в файле, о котором я говорил выше, то есть поиск всех значений, подходящих под заданный пользователем интервал.

И, наконец, рассмотрим третий из наиболее часто встречающихся простых типов данных: текстовые данные. Вообще, методы представления текстовых строк и массивов в коде программ имеют давние и богатые традиции. Наиболее старым способом является выделение под строку участка фиксированного размера, причем неиспользуемая часть блока заполняется «нулевыми» символами. В чистом виде этот прием уже давно не встречается (из примеров вспоминаются разве что старые реализации классического Паскаля), но нечто подобное иногда используется в программах на Си для хранения массивов строк – под хранение каждого из элементов массива отводится блок фиксированного размера, хотя сами элементы, по сути, являются ASCIIZ-строками.

Хранение строк в блоках фиксированного размера имело два принципиальных недостатка: неэффективное расходование памяти при хранении большого числа строк различной длины и жесткое ограничение на максимальную длину строки. Всех этих недостатков были лишены строки с завершающим символом. Идея была проста – выбирается какой-либо малоиспользуемый символ, который интерпретируется программой как признак конца строки. В языке Си таким символом стал символ с кодом 0 (а строки, оканчивающиеся нулем, окрестили ASCIIZ-строками); некоторые системные функции MS-DOSв качестве завершающего символа использовали символ “$”. Несмотря на ряд недостатков, строки с завершающим символом претерпели ряд усовершенствований и используются до сих пор. С началом активного использования UNICODEпоявилась модификация строк с завершающим символом и для этой кодировки. Зная образ мышления программистов на Си, нетрудно догадаться, что в качестве завершающего символа была использована пара нулевых байтов: (0,0). Нужно отметить, что если возникает необходимость укоротить такую строку в тексте программы на несколько символов, то обычно для этого достаточно всего лишь вписать в нужную позицию завершающий символ. То есть, если у Вас есть программа, написанная на C/C++, в заголовке окна которой написано что-то вроде «CoolProgram - Unregistered», и Вы не хотите видеть напоминание о том, что она «Unregistered»,

просто замените в файле программы пробел после слова Programна символ с кодом 0. После этого слово «Unregistered» Вы почти наверняка больше не увидите. Этим же способом ненужную строку можно вообще превратить в пустую, просто поставив в ее начало завершающий символ!

Описанная техника укорачивания и «обнуления» строк пригодна не только для того, чтобы убирать неэстетичные надписи в заголовках программ, в действительности ее возможности гораздо шире. Приведу пару примеров из собственной практики. Один раз мне в руки попалась некая программа, которая очень любила при печати документов в заголовок вставлять надпись «Thisreportcreatedbydemoversion …» шрифтом аршинного размера. Разумеется, мне это не понравилось и при помощи нехитрых манипуляций в шестнадцатиричном редакторе я «обнулил» строку с надписью, оскорблявшей мои эстетические чувства. В другом случае я подобным же образом расправился с одной программой-генератором справок, которая считала, что незарегистрированность – это повод вставлять рекламный текст в каждую статью справочной системы. Небольшой memorypatch, исправлявший в программе «на лету» несколько байт, смог убедить капризную программу в ее принципиальной неправоте.

Более эффективными по сравнению с ASCIIZ-строками являются строки с указанием длины. Такие строки позволяют использовать в тексте все 256 ASCII-символов, хранить не только текстовые, но и любые другие двоичные данные, а также применять по отношению к этим данным строковые функции. Кроме того, вычисление длины строки требует лишь одной операции чтения данных по ссылке, в отличие от ASCIIZ-строк, где для определения длины необходимо последовательно сканировать все символы строки до тех пор, пока не встретится завершающий символ. Как такового, стандарта на строки с указанием длины не существует – можно лишь говорить о конкретных реализациях таких строк в различных компиляторах и библиотеках. В частности, в коде программ на Delphi 7 строковые константы хранятся следующим образом:

• 4 байта: длина строки **в байтах** (для UNICODE-текстов это значение в два раза больше длины строки в символах).
• Содержимое строки.
• Завершающий символ (#0 для ANSI-строк, #0#0 для UNICODE-строк). Завершающий символ никак не используется в «родных» функциях и процедурах Delphi, но значительно упрощает вызов функций WinAPI (которые используют строки с завершающим символом) и использование сторонних библиотек.

Зная все это, нетрудно разработать способ укорачивания Delphiйских строк: для этого требуется изменить длину строки в первых четырех байтах и поставить еще один завершающий символ в нужную позицию. Надо отметить, что тексты-свойства компонентов в ресурсах программ на Delphiхранятся в несколько ином формате, поэтому прежде чем пытаться вмешиваться в код программы, стоит побольше узнать об особенностях реализации встроенных типов в компиляторе, при помощи которого создана исследуемая программа.

Поиск текстовых констант в программе – совсем не такое простое дело, как это могло бы показаться с первого взгляда. Прежде всего, не следует полагаться на «интеллектуальность» дизассемблера: поиск текстовых строк при дизассемблировании обычно основан на анализе ссылок, встречающихся в коде программы. Поэтому, если в коде программы нет прямой ссылки на строку, дизассемблер эту строку может просто «не увидеть». Чтобы убедиться в этом, рассмотрим несложный пример:

```
.data
line1 db "Line 1",0
line2 db "Line 2",0
line3 db "Line 3",0
LineArr dd OFFSET line1, OFFSET line2, OFFSET line3

.code
…
GetMsgAddr proc MessageIndex:DWORD
```

```
mov ebx,MessageIndex
mov eax, OFFSET LineArr
mov eax,[eax+ebx*4]
ret
GetMsgAddr endp
…
```

Этот код представляет собой максимально упрощенную реализацию списка сообщений и функции, получающей адрес текстовой строки по номеру сообщения. Откомпилировав этот пример, загрузим его в W32Dasmи посмотрим, что получится. Получилось следующее: дизассемблер успешно распознал строку «Line 1», но строки «Line 2» и «Line 3» не обнаружил. А вот IDAуспешно распознал все три строки, и создал для них именованные метки. Впрочем, и IDAпри большом желании можно обмануть: достаточно лишь вписать перед текстом самой строки ее длину в байтах (именно так хранит строки Delphi). После этого IDAхотя и обнаруживает сам факт наличия текстовых строк в программе (в окне Stringsэти строки видны), но в дизассемблированном тексте программы эти строки выглядят как последовательность db… , которые нужно приводить в желаемый вид вручную. Кстати, W32Dasmпосле этой модификации не увидит вообще ни одной строки. Если же Вам и этого мало, вместо «Line 1» напишите «Строка 1» - все тексты на русском языке знаменитый дизассемблер гордо проигнорирует. И это только начало. А ведь текстовые строки могут находиться не только в сегменте кода/инициализированных данных, но и в секции ресурсов программы…

Здесь могут помочь специализированные программы, сканирующие указанный файл и вычленяющие из него все текстовые строки (или то, что похоже на текстовые строки). Кроме того Вам потребуются смещения этих строк от начала файла, поэтому Ваш инструмент должен предоставлять и такой сервис. Однако использование таких программ (и самостоятельное их написание) осложняется двумя факторами: разнообразием существующих кодировок текста и существованием национальных символов в некоторых языках (классический strings.exeи многие другие подобные программы «не понимают» русскую секцию UNICODE). Те же проблемы с UNICODEи национальными кодировками характерны и для программного обеспечения, осуществляющего поиск в текстовых файлах. К тому же русские тексты в UNICODEсовершенно нечитабельны в шестнадцатиричных редакторах и просмотрщиках. Все это необходимо учитывать при выборе инструментов поиска текстовых строк, а выбранный инструмент перед использованием желательно проверить на подходящем «пробном камне».

Напоследок расскажу про весьма простой, но весьма эффективный в некоторых случаях способ поиска численных переменных в работающей программе. Этот способ основан на многократном сканировании адресного пространства программы, отслеживании и анализе всех изменений в этом пространстве. Лучше всего этот прием работает на программах, в которых установлено ограничение на количество тех или иных действий, вроде ограничения на число записей, добавляемых в документ. И используется для этого совсем не крэкерский инструментарий. Вы, наверное, знакомы с программами типа GameWizardили ArtMoney, которые позволяют искать в работающей компьютерной игре количество денег или оставшихся жизней. Для тех, кто не сталкивался с такими программами, вкратце опишу алгоритм их работы:

1. Пользователь выбирает из списка работающих в данный момент программ подопытную игру.

2. Пользователь вводит в программу поиска начальное количество денег (хитов и т.п.), которое в данный момент существует в игре.

3. Программа сканирует адресное пространство и строит список всех значений (точнее, адресов, по которым расположены эти значения), совпадающих с введенными пользователем.

4. Пользователь выполняет в игре какое-либо действие, в результате которого количество денег изменяется.

5. В программу поиска вводится новое количество денег.

6. Программа проверяет все значения из построенного списка и исключает из него те значения, которые не соответствуют введенному пользователем.

7. Пункты 4-6 повторяются до тех пор, пока список адресов не укоротится настолько, чтобы можно было проверить назначение каждого элемента списка вручную.

8. Пользователь проверяет каждый элемент списка, записывая по найденным адресам новые значения и наблюдая, как это повлияет на количество денег в игре.

Проницательный читатель наверняка уже догадался, что защищенная программа ничем в принципе не отличается от компьютерной игры, а число добавленных в документ записей – это то же самое, что количество виртуальных «золотых монет». И потому, воспользовавшись соответствующей программой (например, все той же ArtMoney), можно определить адреса всех переменных, которые могут хранить счетчик добавленных в документ записей. Дальнейшие действия зависят исключительно от Вашего желания – можно поставить аппаратную точку останова на чтение из этой переменной и попытаться добраться до команды сравнения существующего числа записей с максимальным. Можно погрузиться в изучение дизассемблерного листинга в поисках команды увеличения переменной и сделать так, чтобы значение счетчика не увеличивалось. Можно даже попробовать модифицировать счетчик из ArtMoney, посмотреть, что из этого получится, и если из этого получится что-то хорошее – написать memorypatch, каждые 100 миллисекунд обнуляющий счетчик.

На этом мой рассказ о простых типах данных в основном закончен, а информацию о методах хранения составных типов, таких как структуры и массивы, о том, где обычно в программах лежат константы, а также об особой роли указателей Вы найдете в следующей главе.

Исследование InstallShield 6+ – обзор методик

Автор: Volodya, Flankerx

Дата: 2004-04-04

Рецензенты:

Dmit

• *Введение - обзор утилит и доступной информации*
• *MSI – ресурсы*
• *IS – ресурсы*
• *MS CAB/IS CAB/самораспаковывающиеся файлы*
• *Методики взлома MSI-программ*
• *COM-интерфейс – радость для кракера?*
• *Подходы к IS-программам*
• *Поиск строки в памяти*
• *Интерпретаторы*
• *Компиляторы*
• *Практические примеры*
• *Заключение*
• *Благодарности*

Статья предназначена только для образовательных целей. Используйте прилагаемые утилиты только для своих собственных программ. Авторы не несут никакой ответственности за возможное незаконное использование материала данной статьи.

В этой статье мы хотим продемонстрировать различные подходы к исследованию программ, использующих технологию Install Shield (в дальнейшем – IS). IS используется при инсталляции практически любого серьезного программного продукта – дистрибутивы Adobe, Microsoft, Intel, Compuware и т.д., и т.п. Подход, предлагаемый в этой статье – декомпиляция сценария инсталляции – трудно назвать новым. В сети уже существуют утилиты и даны некоторые описания, однако, их сложно назвать современными. Все они безнадежно устарели. В представленной на ваше рассмотрение статье мы попробуем восполнить это упущение.

Статья не рассчитана на новичков. Мы предполагаем, что вы умеете пользоваться стандартным джентльменским набором: дизассемблер + отладчик, знаете ассемблер и C++, кроме того, слышали о технологии COM (Component Object Model). Если с COM вы не дружите совсем, не волнуйтесь, линков на литературу будет достаточно. Обещаем также, что вам придется разобраться с MSI.

Введение - обзор утилит и доступной информации

Перед тем, как приниматься крушить все и вся, необходимо очень четко представлять себе теоретическую базу. Что такое «инсталляция» программы и какие технологии могут быть задействованы при этом? Технологий оных великое множество, мы не имеем возможности рассмотреть их все, а сфокусируем свое внимание лишь на двух – MSI (Microsoft Installer) и IS (InstallShield).

MSI – ресурсы

В качестве обзорной статьи по MSI прекрасно подойдет обзор Gary Chirhart http://www.wise.com/filelib/MSIwhitepaper.pdf.

Мы предполагаем, что вы прочитали эту статью и уже знаете, что такое MSI-файл, столь часто используемый при инсталляции программы. Также, вам уже должно быть известно, что MSI-файл предоставляет возможность доступа к своим данным при помощи SQL-запросов в сочетании с MSI API-функциями. Дополнительно о структуре MSI-файла можно почитать на RSDN.ru: http://www.rsdn.ru/article/install/wininstaller.xml?print

В качестве справочника по MSI API-функциям, разумеется, нужно использовать MSDN.

Нас с вами заинтересует раздел CustomActions в MSI-файле. Почитать о том, что это такое и с чем это едят можно здесь: http://www.codeproject.com/tips/msicustomaction.asp

Самой знаменитой утилитой для работы с MSI, вероятно, является MS ORCA, входящая в состав Platform SDK. Поскольку не все имеют возможность качать 200-мегабайтный минимум из-за одной-единственной утилиты, мы продублировали ее на wasm.ru. Дополнительно стоит ознакомиться со статьей http://www.codeproject.com/file/msiconsole.asp. (обязательно скачайте файлы - для этого требуется регистрация, но она бесплатна)

Есть также продукт под названием AdminStudio от IS Corp. На момент написания статьи текущей версией была 5.5. По своим функциональным возможностям примерно равен ORCA, только занимает не 2 мегабайта как ORCA, а 200. Да, есть еще и Wise for MSI и возможности утилиты достаточно велики. Правда, и размер, опять же, не мал…

У ORCA, однако, есть большой недостаток. В ходе чтения вы увидите, что иногда бывает необходимо извлечь файл, чтобы поправить пару байт или выдрать кусок кода для кейгена. ORCA не позволяет извлекать файлы из MSI-дистрибутива. AdminStudio тоже. WinRAR способен открывать большинство MSI-дистрибутивов, однако не все. Поэтому мы написали собственную утилиту-экстрактор. Вы увидите ее действие по ходу чтения.

IS – ресурсы

InstallShield – это надстройка над MSI-движком. Невероятно распространенная технология. Практически любая серьезная контора предлагает свои дистрибутивы в виде IS-обертки.

Невероятно важно четко себе представлять архитектуру IS. Вот очень хороший отрывок из книги: http://www.installsite.biz/en/products/isdbookch4.pdf

Мы не станем умолять вас прочесть ее. Скажем просто – если не прочтете, во всей остальной части статьи вам делать уже нечего. А в статье в виде симпатичных диаграммок показано взаимодействие MSI и IS-компонентов. Что такое IDriver.exe? А ISRT.dll? А для чего задействованы файлы, что лежат у вас в директории \Common Files\InstallShield\Driver? А что такое setup.inx и для чего он нужен? Что вообще такое «файл сценария»?

Итак, IS в процессе инсталляции обрабатывает т.н. файл сценария. Файл сценария содержит указания файлам (каким файлам?) IS о том, что делать с дистрибутивом. Куда какие файлы положить, какие значения в реестре прописать, какие сервисы запустить, и наконец, а не спросить ли у пользователя пароль? Разумеется, спектр возможностей setup.inx гораздо шире, однако, в первом приближении сойдет. А теперь вдумайтесь в смысл. Setup.inx говорит, что надо делать, а файлы IS выполняют эти действия, т.е. являются интерпретаторами. Значит, поняв принцип кодировки информации в IS можно сделать с инсталляцией что угодно! Программа просит пароль?

Мы поправим inx-файл и программа покорно забудет о пароле!

В ранних версиях IS файл сценария имел расширение ins, в IS 6+ он стал inx. Форматы ins и inx были абсолютно недокументированны – еще бы, ведь зная формат файла и принцип кодирования информации в этом файле, вовсе нетрудно подправить его так, чтобы IS-интерпретатор посчитал, к примеру, любой введенный пароль правильным. В случае ins-файла первым Прометеем стал NaTzGuL, написав пару отличных статей и шикарную утилиту в придачу. А какой выбор был до этого? Часами сидеть в Soft-Ice, пытаясь понять, как же интерпретатор оперирует со строкой пароля и что ему надо. Теперь же стало возможным просто поменять условие в ins-файле (ведь декомпилированный файл имеет C-подобный синтаксис) и все. Вуаля. В случае inx-файла появились две утилиты - Installshield 6/7 script decompiler by sn00pee и Install Shield script decompiler 1.00 beta 15 by NEKOSUKI. Обе доступны с wasm.ru, вот только inx-файлы версий 8+ они уже не понимают (ну-у, скажем так, почти не понимают), а других утилит в сети и нет…

Формат inx-файла тоже неизвестен, документацию и принципы разбора тоже никто описать не озаботился. Авторы, естественно, свои утилиты более не поддерживают – бесплатно, кому оно нужно? Сменились интересы, жизнь скрутила, да мало ли что… Исходники утилит тоже закрыты.

Словом, как видите с ресурсами по IS дело обстоит плоховато. Но есть и пара светлых моментов. Самый главный светлый момент принадлежит некоему zeezee: http://www.woodmann.com/fravia/zeeinst.htm

И эта статья стоит очень многого. Она невероятно полезна и ее стоит прочесть также как и «The InstallShield Developer Run-Time Architecture». Мы собираемся значительно расширить работу zeezee и рассмотреть современные IS компиляторы и интерпретаторы, ведь с момента написания его статьи многое изменилось. Для этого нам потребуются знания о COM и о теории компиляции программ. Первые потребуются потому, что интерпретаторы IS – это COM-сервера, а вторые потому, что компилятор от IS – это, ну, компилятор :)

Компилятор входит в состав InstallShield DevStudio. Пакет монструозен не меньше, чем AdminStudio, но с точки зрения кракера тоже очень интересен. Последней версией на момент написания статьи была 9.0 SP1. Дополнительную информацию можно найти на сайте http://www.installsite.org

Возможно, вас заинтересует вопрос, а можно ли использовать файл сценария (setup.inx) из чистого MSI-пакета. Ответ – да, можно. Здесь нужно понимать, что MSI ничего не знает об IS и знать не желает – он лежит уровнем ниже. IS опирается на MSI, а не наоборот. IS использует специальную dll – ISScriptBridge.dll для обмена информацией с IS-файлами через RPC. Более подробно мы рассмотрим это в практических примерах.

MS CAB/IS CAB/самораспаковывающиеся файлы

Часто MSI-дистрибутивы хранят данные в MS CAB-файлах. Все пакеты от MS устроены именно так. MS CAB файл можно узнать по сигнатуре MSCF в самом начале файла. Утилиты от MS и WinRAR прекрасно умеют открывать такие файлы. Кроме того, формат MS CAB документирован в MSDN.

Иногда можно встретить самораспаковывающиеся exe-файлы, содержащие внутри себя здоровенный MS CAB-файл. В этом случае PE-пришлепки (самого обычного исполняемого PE-файла) входит распаковать этот файл куда-нибудь, спросит у пользователя пароль, если необходимо и т.д., и т.п. Такие файлы тоже просто распаковать WinRAR. Примерами являются инсталляционный файл от Adobe Acrobat 6.0 или дистрибутив VTune 7.1. Выглядит это так:

```
000485a0h: 01 98 4D 53 43 46 00 00 00 00 CF 4E EE 00 00 00 ; .
```

Теоретические основы крэкинга: Глава 5. Структуры, массивы, ссылки.

Автор: CyberManiac

Дата: 2004-04-24

Читая предыдущую главу, Вы, наверное, были неприятно удивлены тем, с какими сложностями можно столкнуться, решая такую нехитрую задачу, как поиск в программе нужной константы. И, возможно, подумали, что если простые числовые и символьные константы способны создать столько проблем, то каково же будет разбираться в составных типах данных! Поспешу Вас успокоить - разобраться в структурированных данных обычно не слишком сложно именно в силу их регулярной структуры. К тому же, в этой главе мы не будем рассматривать такие достаточно сложные вещи, как тонкости представления текста в различных кодировках, и размышлять над глобальными проблемами машинного округления - это уже пройденный этап, и, я думаю, Вы сможете эффективно применить эти знания без дополнительных подсказок. Впрочем, простота теоретической части обсуждаемой нами темы компенсируется сложностями при практическом применении предлагаемых Вам знаний.

Сразу отмечу, что эта глава посвящена не столько непосредственному взлому программ, сколько техникам "вытаскивания" структурированной информации, упрятанной в недрах исполняемых файлов. Впрочем, техники, описанные в этой главе, с некоторыми модификациями могут быть применены и к другим типам файлов, хранящих структурированные данные. Это может быть полезно, к примеру, для тех, кому потребуется извлечь из программы какие-либо данные или "расколоть" базу данных заранее неизвестного формата.

Рассказывая о нецелочисленных константах, я упоминал о том, что обращения к таким константам производятся по указателю, то есть в команде, загружающей действительное число в регистр сопроцессора, в явном виде присутствует адрес загружаемого числа. Однако доступ к данным по указателю на них характерен не только чисел с плавающей точкой (на некоторых платформах числа с плавающей точкой отлично умещаются в регистрах общего назначения), но и для строк, структур и массивов. Теоретически, небольшую структуру или короткий массив еще можно было бы попытаться разместить в регистрах общего назначения (собственно, при предельной оптимизации кода иногда так и поступают), но большинство реальных данных требуют для своего хранения гораздо больше места, чем могут предоставить регистры процессора. Наиболее простым и быстрым способом работы с такими объемными данными является хранение этих данных в оперативной памяти и обращение к ним через посредство указателей.

Теперь перейдем к практическим аспектам использования указателей в программах. Здесь работают три очень простых правила:

• к любой константе длиннее 4 байт обращаются по ее адресу
• к любой константе составного или строкового типа обращаются по ее адресу
• в коде любой программы адрес начала статической переменной (т.е. переменной, память под которую выделяется в момент запуска программы) явно указан как минимум один раз независимо от типа этой переменной

Это, конечно, очень широкое обобщение (и не всегда верное - достаточно вспомнить пример с загрузкой 10-байтной вещественной константы в Delphi из предыдущей главы), но, в общем, современные компиляторы действительно используют ссылки весьма широко. И если в программе по адресу X находится строковая константа 'My text', то почти наверняка где-то в коде программы найдется команда push X, mov eax,X или что-либо подобное. Именно на этом факте наличия "указателей на все, к чему можно обратиться по указателю", а также на отсутствии путаницы с

сегментами и смещениями и основана "разумность" дизассемблеров под Win32, поначалу сильно удивляющая тех, кто успел привыкнуть к маловразумительным листингам дизассемблированных 16-разрядных программ для MS DOS.

Хотя при наличии очень большого желания (и еще большего терпения, чтобы это отладить) все-таки можно обращаться к данным, не используя явные ссылки на эти данные. Не верите? Тогда попробуйте разобраться в следующем коде для платформы Win32 и найти в нем хоть одну ссылку на выводимые в MessageBox'е строки:

```
push 0
call $+1Ah
db 'Code sample',0,'Any text',0
mov eax,[esp]
add eax,0Ch
push eax
push 0
call MessageBoxA
```

Такие могучие коды начисто "срывают башню" даже IDA Pro, который оказывается совершенно неспособен более или менее логично дизассемблировать это нечто. Вот уж воистину "горе от ума" - древний, по нынешним меркам, W32DAsm дизассемблировал этот код гораздо адекватнее. Разумеется, знаменитому дизассемблеру при желании можно (и даже не очень сложно) объяснить, где в действительности находятся данные, а где - код, который к этим данным обращается, но чтобы это сделать придется сначала разобраться в приведенном коде самому. Понятно, что, написать целую программу таким "высоким штилем" вряд ли у кого-то получится (да и языки высокого уровня мало приспособлены к подобным экзерсисам), но определение адреса блока данных при помощи пары команд

```
call $+5
pop eax
```

в защитных процедурах (в частности - в процедурах расшифровки кусков кода) встречается довольно часто.

Другим важным моментом, который необходимо помнить при "расшифровке" структур и массивов, хранимых в коде программ, является то, что популярные компиляторы не поддерживают использование структур переменного размера. То есть длина структуры жестко фиксирована и постоянна для всех структур одного и того же типа: если переменная A типа MY_TYPE занимает 100 байт, то переменная B того же типа также будет занимать 100 байт независимо от того, какие данные в ней хранятся. Возникает вполне естественный вопрос: а как же тогда хранятся строки или динамические массивы, размер которых заранее неизвестен? В действительности, в современных компиляторах строки не хранятся непосредственно внутри структур. Вместо этого хранятся лишь указатели на статически или динамически выделенные блоки памяти, а непосредственно текст строки размещается именно в этих блоках. Этим и объясняется смущающий начинающих программистов на Delphi и C++ Builder эффект, когда при попытке определить размер строки при помощи SIZEOF получается, что любая строка занимает 4 байта, а при записи структуры, содержащей поля строкового типа, в файл, вместо текстовых строк появляется странного вида "мусор" длиной в четыре байта. Исключением из этого правила являются только старые паскалевские строки фиксированной длины и массивы символов (char) в Си, но оба эти типа в настоящее время употребляются довольно редко. Кроме того, ссылки на данные вместо самих данных также используются для хранения объектов (тех, которые "экземпляры классов") и динамических массивов.

Если Вы захотите, Вы можете набросать небольшую программку для поиска всех возможных ссылок в коде программ. Базовая идея проста: код программы и статические переменные, как правило, имеют весьма небольшой объем по сравнению с максимально возможным размером адресного пространства в Win32 (4 гигабайта, если не вдаваться в тонкости устройства Windows). А

потому мы можем с высокой вероятностью считать ссылками на код или данные все 32-разрядные числа в программе, которые попадают в промежутки адресов, занятые программой или данными. Чтобы проверить все это на практике, так сказать, "потрогать руками", нужна программа, извлекающая из исполняемого файла все четырехбайтные значения, которые теоретически могут быть адресами. Если Вы уже попробовали написать утилиту для поиска нецелочисленных данных и проверили ее в действии, проблем с программированием у Вас не возникнет. Самое сложное - извлечь из заголовка PE-файла адреса начала каждой секции в памяти, размеры этих секций и смещения начала каждой секции в файле. Если Вы не хотите сразу же погружаться в изучение структуры PE-заголовка (а рано или поздно этим все же придется заниматься), на первых порах Вы можете ограничиться ручным вводом этих данных.

А теперь разберемся, что Вы собственно написали. А написали Вы ни что иное, как одну из частей обыкновенного дизассемблера, занимающуюся поиском ссылок в коде программ. Разумеется, настоящие дизассемблеры используют для поиска ссылок гораздо более сложные алгоритмы, но нам сейчас нужен как раз такой простейший инструмент. Разумеется, поиск вообще всех возможных ссылок на данные - само по себе занятие малополезное, но если немного поразмыслить… Если немного поразмыслить, у такой программы появляется довольно неожиданное применение: поиск начальных адресов структур и массивов по одному или нескольким известным полям (элементам). Так что не откладывайте эту программу в дальний угол винчестера - она нам очень скоро пригодится.

Итак, предположим, что Вы знаете о структурах и массивах все, что положено знать начинающему программисту, а именно: что они есть, что в них можно хранить данные, и, самое главное, как в программах обращаются к отдельным элементам этих структур и массивов. Более того, Вы даже знаете, что массивы и структуры можно комбинировать весьма удивительными и изящными способами: создавать массивы массивов (они же двухмерные массивы), массивы структур, массивы массивов структур, ну и так далее. Абсолютное большинство компилирующих языков при хранении данных структурированных типов придерживаются принципа: поля и элементы массивов хранятся в памяти в том порядке, в каком они определены в исходных текстах программы. Ну и, разумеется, порядок полей неизменен для всех данных одного типа. Само по себе это, конечно, мало что дает - ведь исходников-то у нас нет, но тут в игру вступает психология. Да-да, программисты - тоже люди, а изучение их психологии, даже поверхностное, иногда помогает лучше понять творения программистов (то бишь программы). В частности, многим программистам свойственно стремление к логике и элементарному порядку в исходных текстах. Например, если в ходе исследования неких данных Вам удалось установить, что первое и второе поле структуры - это указатели на фамилию и имя, то третье поле структуры скорее всего окажется указателем на отчество, а не закодированным размером обуви или цветом волос. Или, если речь идет о заголовке архива, за именем файла наверняка последует смещение сжатого файла в архиве (плюс-минус некоторая константа), размер этого файла в сжатом виде и контрольная сумма. Причем именно в таком порядке - это традиция, которую программисты нарушать не любят. Если Вы кроме крэкинга занимаетесь программированием, поразмыслите о том, как бы Вы разместили информацию, будь Вы на месте автора программы - и, возможно, это будет наиболее короткий путь к пониманию структуры данных. Именно понимание программистских традиций, "неписанных законов" позволили исследователям недр Windows NT без особых сложностей разобраться с параметрами вызовов Native API - им потребовалось лишь изучить общедоступную документацию, понять, как мыслят программисты в Microsoft и немного повозиться с отладчиком.

Но вернемся к нашим структурам. Проведем небольшой эксперимент: возьмем все тот же Delphi и определим пользовательский тип my_record (в Паскале структуры принято называть записями):

```
type my_record=record
 a:byte;
 b:word;
 c:dword;
```

```
end;
```

А теперь попробуем подсчитать, какова длина такой записи в байтах. Если просто сложить длины полей, входящих в запись, должно получиться 1+2+4=7 байт. Но в действительности все обстоит несколько иначе: sizeof (my_record)=8! Чтобы выяснить, почему так случилось, определим в программе переменную my_var типа my_record и попытаемся присвоить полям этой переменной значения: my_var.a:=1; my_var.b:=2; my_var.c:=$ABCDEF10 (надеюсь, Вы внимательно читали предыдущую главу и уже догадались, зачем я присвоил третьему полю столь странное значение). После компиляции мы получим следующий код:

```
:00452100 C605005C450001          mov byte ptr [00455C00], 01
:00452107 66C705025C45000200      mov word ptr [00455C02], 0002
:00452110 C705045C45004E61BC00    mov dword ptr [00455C04], ABCDEF10
```

Возникает закономерный вопрос: чем так плох адрес 455C01, что по этому адресу компилятор "не захотел" хранить данные. Ответ на этот вопрос лежит в недрах архитектуры x86. С незапамятных времен процессоры x86 выбирали данные, лежащие по четным адресам, немного быстрее, по сравнению с такими же данными, лежащие по нечетным адресам. Чуть позже процессорам стали "нравиться" адреса, кратные четырем. С совершенствованием процессоров список "хороших" адресов продолжал расширяться, появились "особенные" последовательности команд, которые выполнялись быстрее "обыкновенных" и в результате предельная оптимизация программ стала занятие настолько сложным, что стало проще доверить ее компилятору. Для достижения максимальной производительности программисты старались размещать часто используемые данные именно по таким "хорошим" адресам. А чтобы программисту не приходилось раскладывать данные по нужным адресам вручную, в компиляторы была введена такая опция, как "выравнивание данных". Эта опция заставляет компилятор принудительно размещать данные по адресам, кратным величине выравнивания. В нашем случае данные выровнены по четным адресам, поэтому ячейка с адресом 455C01, находящаяся между однобайтным полем a и двухбайтным b, осталась не у дел. Однако если программисту требуется хранить в памяти достаточно большое количество записей, потери памяти из-за выравнивания могут оказаться неприемлемо большими, и в таких случаях выравнивание либо отключают вообще, либо при помощи служебных слов "объясняют" компилятору, к структурам каких типов выравнивание применять не надо.

Использование выравнивания в программах дает один интересный побочный эффект, облегчающий изучение и извлечение данных, хранящихся в программах. Хотя формально значение байтов, находящихся в "дырках" между полями константы-структуры, не определено, на практике все известные мне компиляторы записывают туда нули (что интересно, эти "дырки" при желании тоже можно использовать для хранения данных). В результате достаточно длинный массив таких структур довольно легко определить среди прочих данных "на глаз". Лучше всего для таких целей подходят редакторы, позволяющие при просмотре изменять количество байт, отображаемых в одной строке и выделять цветом характерные последовательности байт (в частности, таким свойством обладает Hex Workshop 4) - цветные пятна образуют характерный узор, который Вы начнете легко замечать после минимальной практики. Тем более, что если отформатировать дамп так, чтобы в строке умещалось столько байт, сколько занимает одна структура, "дырки" выстроятся в вертикальную полосу. Чтобы Вам было понятнее, о чем я говорю, приведу пример массива записей и продемонстрирую, какими путями можно попытаться определить размер и назначение полей структур. Вот код, присутствующий в одной из старых версий моей программы InqSoft Sign 0f Misery:

```
28AB5100 74AB5100 8CAB5100 03030D0D 00000000 01002E04 09000000 80000000
B0AB5100 D0AB5100 00000000 03000F00 00000000 0100CE04 08000000 81000000
ECAB5100 D0AB5100 00000000 03000F00 00000000 0100C404 08000000 A2000000
0CAC5100 30AC5100 48AC5100 03030D0D 00000000 0100A604 08000000 A3000000
```

Как видите, факт структурированности данных заметен невооруженным глазом, хотя мы ничего не знаем о том, какие именно данные хранятся в этом массиве. Достаточно очевидно, что размер

одного элемента массива равен тридцати двум байтам. Надо отметить, здесь нам очень повезло в том, что размер структуры совпал с числом байт, отображаемых в одной строке. Впрочем, многие шестнадцатеричные редакторы позволяют менять этот параметр и группировать байты произвольным образом (т.е. не обязательно по 4, как это сделано в примере). Попробуем рассуждать логически. Первым делом заглянем в PE-заголовок файла программы и посмотрим начальные адреса и длины (в терминологии PE Explorer'a - Virtual address и Size of Raw Data соответственно) секций. Просуммировав эти характеристики, в первом приближении мы можем считать, что наша программа после загрузки занимает в памяти адреса с 410000h по 6005FFh (хотя, в общем случае, между секциями в памяти могут быть "дыры"). Поэтому числа, попадающие в этот промежуток, с большой вероятностью являются указателями на данные.

Внимательно посмотрев на первый, второй и третий столбец, Вы можете заметить, что в этих столбцах как раз находятся числа из промежутка 410000h..6005FFh, т.е. это потенциальные указатели на данные. Нули, встречающиеся в третьем столбце - это "пустые" указатели; такие указатели широко известны в языках программирования под различными именами. В C/C++ такие указатели обозначаются как NULL, в Паскале/Delphi - nil. Попробуем посмотреть, что находится по адресам из первых трех столбцов первой строки. А находятся по этим адресам ASCIIZ-строки:

```
0051AB28: "Ожидать появления окна с указанным текстом в заголовке и классом"
0051AB74: "Имя класса окна"
0051AB8C: "Текст в заголовке окна"
```

Если проверить остальные указатели, мы также обнаружим по соответствующим адресам текстовые строки. Отлично! Теперь нам известны длина и тип трех полей структуры. Теперь обратите внимание на шестую тетраду. Числа 042E0001, 04CE0001, 04C40001 мало похожи на осмысленные данные, что в шестнадцатеричной системе, что в десятичной. Но вот если четырехбайтные последовательности интерпретировать не как DWORD, а как два идущих подряд WORD'а, то данные начинают выглядеть менее странно: (1,1070), (1,1230), (1,1220). Правда, мы не можем быть уверенными (как я уже говорил, такова обратная сторона метода, базирующегося на наблюдениях и предположениях), что первые два байта - это действительно одно поле типа DWORD, а не два поля типа BYTE - имеющаяся выборка слишком мала. Чтобы проверить это, необходимо исследовать код программы, который обращается к этому массиву. Но это - тема следующей главы.

Если бы я привел более длинный кусок массива, было бы более очевидно, что четвертая тетрада на самом деле состоит из четырех полей типа BYTE, причем первые два из них принимают значения от нуля до пяти. А также то, что если первый или второй байт четвертой тетрады равен нулю, то вторая или третья тетрада соответственно будет хранить пустой указатель. В общем, определение типов полей по их значениям и структуре массива - занятие, требующее прежде всего наблюдательности и логического мышления.

А пока разберем некоторые тонкости, о которых я умалчивал до этого момента. Вы, наверняка заметили, что, приведя в качестве примера некий кусок кода, я просто сказал "это массив структур", не приведя никаких объяснений, каким образом я обнаружил этот код в программе. На практике задача обычно ставится несколько иначе: по некоторым известным данным необходимо найти в программе область, где эти данные хранятся в структурированном виде, разобраться, какие еще данные, помимо уже известных, содержит эта структура, и затем извлечь то, что нам было неизвестно.

То есть, когда мы хотим извлечь некую информацию из программы, мы ищем не "то, не знаю что", а имеем некоторое представление, что нам нужно и как оно может выглядеть. Поэтому если Вам точно (или даже приблизительно) известно хотя бы одно из значений, входящих в массив, Вы можете приступить к поиску этого значения внутри программы при помощи методов, описанных в предыдущей главе. И если поиск окажется успешным, Вы будете знать расположение одного из элементов массива.

Кроме того, я продемонстрировал расшифровку отдельных полей, но не сказал ни слова о том, с какого из этих полей начинается описание структуры. Иными словами, мы все еще не знаем, находится ли ссылка на строку по адресу 0051AB28 в начале структуры, в середине или же вообще является последним полем записи, а следующая за ней ссылка на адрес 0051AB74 относится уже к следующему элементу массива. Если вспомнить "психологию программиста", о которой я говорил выше, и проанализировать содержимое строк, то принадлежность всех трех ссылок к одной и той же записи достаточно очевидна, но мы не будем упрощать себе задачу и попробуем честно добраться до приведенного массива. К сожалению, для полного представления о решаемой задаче необходима сама программа, привести которую в данной главе невозможно по причине ее большого объема, поэтому мне придется ограничиться только демонстрацией наиболее важных ее кусков.

Итак: у нас есть программа, которая состоит из интерпретатора байт-кода и оболочки, позволяющей писать скрипты, которые затем транслируются в байт-код. Внутри оболочки содержится массив, описывающий команды, используемые в этой программе и байт-коды, соответствующие каждой команде, и нам требуется извлечь из программы список байт-кодов и названий команд, которые этим кодам соответствуют. Сама оболочка запакована при помощи UPX, но ее распаковка никакой сложности не представляет. Названия команд известны каждому запустившему программу, поскольку полный список команд отображается в оболочке.

Возьмем любую команду, например "Ожидать появления окна с указанным текстом в заголовке и классом" и попробуем найти этот текст в коде программы. Никаких особых ухищрений для этого не потребуется, при помощи функции поиска в HexWorkshop находим, что искомая строка встречается в файле в единственном экземпляре по смещению 1154856 байт от начала файла. Далее, при помощи любого Offset->RVA конвертера (утилиты, преобразующей смещения в файле в относительные виртуальные адреса (RVA)) определяем, по какому адресу будет размещен 1154856-й байт программы после загрузки этой программы. Получаем, что эта строка в памяти располагается начиная с адреса 51AB28h. Теперь вспомните то, что говорилось о методах хранения строк внутри структур, и Вам станут очевидны наши дальнейшие действия.

Следующий этап будет заключаться в поиске по всему коду программы ссылок на нужную строку, то есть 32-битной константы 51AB28h. Такая константа нашлась сразу же, причем в единственном экземпляре по смещению 1229C0h от начала файла. 128-байтный блок, начинающийся с этой константы, я и привел в качестве примера несколькими абзацами выше. Теперь Вы имеете представление, при помощи каких приемов можно добраться до нужного массива. Как видите, здесь все достаточно просто.

Теперь нам надо выяснить, с какого байта начинается какой-либо элемент приведенного массива. На практике обычно проще всего найти адрес первого элемента массива (или нулевого, если использовать нумерацию, принятую в Ассемблере или Си). Сделать это можно двумя способами: поиском ссылок на начало массива либо анализом самих данных с целью поиска верхней и/или нижней границы массива.

Первый способ базируется на том, что в программе почти наверняка найдется явная ссылка на первый элемент массива, и что адрес этого элемента больше либо равен адресу одного из известных нам элементов, входящих в массив. В применении к нашему примеру это означает, что если смещение одного из найденных нами элементов структуры равно 1229C0h (RVA для этого смещения равно 522BC0h), то начало массива, очевидно, находится не ниже этого адреса. Следовательно, если выбрать из программы все константы меньше либо равные 522BC0h (а для этого нам и нужна соответствующая программа, о которой я уже упоминал), среди них окажется и ссылка на первый элемент массива. При этом, если рассортировать найденные ссылки по убыванию их величин, велика вероятность, что ссылка на начало массива окажется в числе первых.

Продемонстрирую это примером. Допустим, Вы выяснили, что ячейка по адресу 450080h входит в состав массива. Выбрав все ссылки, указывающие на адреса не ниже 450080h, Вы получили следующий набор: 450000h, 49FFD0h, 49FCD4, 49FCD0h и так далее. Из всех найденных Вами адресов теоретически наиболее вероятным адресом будет являться 450000h. Почему? Причина в том, что информация из длинных предварительно инициализированных массивов обычно (но не всегда!) считывается при помощи конструкций вида my_arr[i], где значение i явно не указано и на этапе компиляции определено быть не может. И потому, чтобы обратиться к i-му элементу массива, в общем случае программа должна вычислить адрес этого элемента по формуле адрес_элемента = начальный_адрес_массива + размер_элемента * (номер_элемента - номер_первого_элемента). Как мы видим, в этой формуле в явном виде присутствует начальный адрес массива, который мы и ищем в программе.

Кроме того, если этот массив передается в качестве аргумента в какую-либо процедуру или функцию, то весьма вероятно, что в программе встретится команда push адрес_начала_массива. Это происходит потому, что передача всего массива в процедуру через стек встречается в аккуратно написанных программах достаточно редко, обычно передается лишь ссылка на первый элемент (которая нам и нужна).

Однако "гладко было на бумаге", а в реальности существует немало "подводных камней", заметно осложняющих использование предложенного метода, а иногда даже и делающего этот метод неприменимым. Прежде всего, проблему могут создать обращения к элементам с явно указанными номерами. Допустим, что в программе кроме конструкций вроде my_arr[i] имеются обращения к конкретным элементам, например a=my_arr[2] или if b>my_arr[10] then do_something. Если адреса значений my_arr[2] и my_arr[10] могут быть вычислены на этапе компиляции, то хороший оптимизирующий компилятор их вычислит, подставит в код, а в программе, кроме начального адреса массива, появятся также адреса второго и десятого элемента массива. И тогда при поиске начала массива по предложенному методу Вы можете найти не первый элемент, а второй или десятый - это уж как повезет.

Во что хороший оптимизирующий компилятор может превратить исходный код - это вообще тема большая и интересная. Например, в Delphi допустимы массивы, начальный элемент которых может иметь любой целочисленный индекс, в том числе и отрицательный, т.е. вполне корректно использование выражений вида a:=my_arr[-10]. Да и менее экзотичный массив вроде array [10..100] of my_record не так прост, как может показаться с первого взгляда: посмотрим, как будет выглядеть приведенная выше формула адреса произвольного элемента массива для этого случая.

адрес_элемента = начальный_адрес_массива + размер_структуры_my_record * (номер_элемента - 10)

Очевидно, что эту формулу можно переписать как: адрес_элемента = (начальный_адрес_массива - размер_структуры_my_record * 10) + (размер_структуры_my_record * номер_элемента)

Вы, наверное, заметили, что выражение в первой скобке - константа, которая для предварительно инициализированного массива может быть легко вычислена на этапе компиляции. Эта константа представляет собой адрес виртуального нулевого элемента массива (не секрет, что большинство компиляторов без специальных указаний не проверяют выход за границы массива, а потому Вы можете обратиться в программе даже к тем элементам, которые в действительности не существуют). Поэтому в программах на Паскале/Delphi вместо ссылок на истинное начало массива для экономии одной операции вычитания может использоваться вычисленный таким образом адрес "нулевого" элемента, не существующего в действительности. Подобные приемы иногда используются и при программировании на ассемблере, когда для ускорения программы вместо пары команд

```
dec ebx
mov eax, [OFFSET my_arr+ebx*4]
```

используется единственная команда mov eax, [(OFFSET my_arr-4)+ebx*4], в которой значение (OFFSET my_arr-4) вычисляется на этапе компиляции.

Как видите, поиск массива по ссылке на первый элемент - процесс не всегда простой, и когда Вам потребуется им воспользоваться, будьте готовы к возможным трудностям. Однако существует второй способ поиска массивов, который более прост в реализации, независим от использовавшегося для сборки программы компилятора и не требует каких-либо инструментов, помимо хорошего шестнадцатеричного редактора и наблюдательности.

Принцип, заложенный в основу этого метода, очень прост. Хранящиеся в массиве данные носят осмысленный характер, а при просмотре в шестнадцатеричном редакторе структурированность этих данных довольно легко заметна. А потому, если Вам известна точка, заведомо находящаяся внутри массива, двигаясь от этой точки вверх или вниз, Вы сможете обнаружить начало или конец массива по изменению внешнего вида данных.

Если посмотреть на приведенный в качестве примера кусок массива, регулярность данных в этом массиве видна невооруженным глазом: нолики идут под ноликами, единички - под единичками, указатели - под указателями, ну и так далее. Если бы я привел дамп всего массива целиком (он довольно объемный), Вы бы смогли убедиться, что внешний вид всех остальных записей массива очень похож на эти четыре строки. Однако как только Вы дойдете до начала или конца массива, внешний вид данных, скорее всего, изменится - ведь перед массивом и после него хранятся совсем другие данные с другой структурой (хотя надо отметить, что если подряд идут да массива с однотипными данными, то граница межу ними может быть не столь явной).

```
80000000 00800000 20000000 00400000 40000000 78985100 88985100 98985100
A8985100 B8985100 C8985100 E4985100 00995100 1C995100 30995100 68000000
C49B5100 E09B5100 00000000 03000E00 01000000 00006400 00000000 97000000
009C5100 E09B5100 3C9C5100 03030E0D 01000000 00006E00 00000000 6B000000
```

Внимательно посмотрев на приведенный кусок, Вы можете видеть, что массив начинается с восьмой тетрады второй строки. Почему? Во-первых, достаточно очевидно, что первые две строки заметно отличаются от третьей и четвертой, следовательно, границу между массивами нужно искать именно во второй либо в третьей строке. Первая и вторая тетрада второй и третьей строки похожи на указатели на текстовые строки (хотя, если копнуть чуть глубже, можно обнаружить, что только в третьей строке эти ссылки действительно указывают на названия команд). А вот четвертая, пятая, шестая и седьмая тетрады также являются указателями и внешне сильно отличаются от соответствующих тетрад третьей и четвертой строки. Очевиден вывод: все данные по седьмую тетраду второй строки включительно не принадлежат массиву. Осталось разобраться с восьмой тетрадой: она ничем не выделяется на фоне последующих двух, поэтому мы с чистой совестью можем отнести ее к нашему массиву и сказать, что массив начинается именно с нее. Дальше, если проанализировать весь массив, выяснится, что первое поле - уникально для каждого элемента массива, и его значение не превышает 255 (то есть первый элемент структуры - байт). Логично предположить, что это и есть код команды, который мы и искали.

Конец массива отыскивается аналогичным способом; поиск конца массива, пожалуй, даже проще, поскольку очевидно, что размер массива в целом должен быть кратен размеру одного элемента этого массива.

Увы, и этот метод не свободен от недостатков. Во-первых, для успешного использования этого метода необходимо, чтобы элемент массива можно было легко отличить от данных, не имеющих к массиву никакого отношения. Однако так бывает далеко не всегда, например, таблица констант для реализации CRC по алгоритму WiseByte выглядит как набор из 256 псевдослучайных чисел, не имеющих каких-либо характерных признаков, видимых невооруженным глазом. Во-вторых, проблему представляют случаи, когда в одном массиве хранятся разнородные данные. Возможность хранения разнородных данных в структуре предусматривается некоторыми языками программирования ("структуры с вариантами" в Паскале и объединения (union) в Си). И в-третьих,

желательно, чтобы данные, предшествующие массиву и следующие за ним, заметно отличались по внешнему виду, что совершенно не обязательно выполняется на практике. Более того, программисты, исходя из стремления к порядку в исходных текстах, очень часто располагают однотипные массивы последовательно, чем отнюдь не облегчают жизнь крэкерам.

Как видите, оба способа обладают существенными недостатками, сильно зависят от "человеческого фактора", не гарантируют стопроцентного результата (впрочем, в крэкинге результат вообще никто гарантировать не может), а область их действия - ограничена. Что еще можно сделать, чтобы поднять эффективность этих техник? Ответ прост - необходим синтез обоих методов! Результаты, полученные при помощи второго метода, могут быть проверены поиском указателя на первый элемент в коде программы. И наоборот, если Вы нашли возможную ссылку на начало массива, посмотрите, как выглядит тот участок программы, на который указывает эта ссылка, похож ли он на начало массива, или необходимо внести поправки в свои рассуждения. Вообще, крэкеру (да и не только крэкеру) стоит взять на вооружение следующий принцип: если результат, полученный одним способом, может быть проверен другим способом, эту нужно выполнить проверку. В конечном итоге следование этому принципу позволит Вам избежать ошибок, и из-под Вашей руки выйдет гораздо меньше недопатченных и криво взломанных программ.

Поэтому я сейчас поведаю Вам про третий метод исследования массивов, немного варварский, но весьма результативный. Этот метод позволяет не только обнаружить начало массива, но и выяснить действительные размеры каждого элемента структуры и определить, какой из элементов открывает определение этой структуры. Суть метода проста: добравшись до одного из элементов массива, Вы начинаете при помощи шестнадцатеричного редактора изменять отдельные байты и наблюдать, как эти изменения скажутся на работе программы. При известной наблюдательности этот метод позволяет быстро и без особых усилий понять внутреннюю логику организации данных; впрочем, изменение кода программы (значений регистров, содержимого памяти) с целью "посмотреть, что будет, если…" - один из наиболее популярных подходов в крэкинге, доказавший на практике свою эффективность.

Все, что я рассказал выше об изучении массивов, не следует воспринимать как единожды данное откровение, в котором нельзя ничего добавить, отнять или изменить. Напротив, предложенные методы можно и нужно комбинировать, адаптировать под конкретные задачи, которые будут вставать перед Вами. И если я чего-то не упомянул или о чем-то умолчал, то исключительно по той причине, что невозможно описать в одной короткой главе все возможные случаи, с которыми Вы можете столкнуться в процессе изучения программ. А потому повторю еще раз: данный материал - это не "инструкция по крэкингу", но, прежде всего, базовый материал для самостоятельных рассуждений, выводов и экспериментов.

Ну вот, теперь знакомство с приемами поиска данных в исполняемых файлах программ в основном закончено. Осталось лишь изучить специфические техники поиска данных "на лету", то есть в работающих программах, а также некоторые особо экзотические приемы извлечения данных из программ.

Способы обхода отладчиков режима пользователя

Автор: R4DX

Дата: 2004-05-07

Введение. В этой статье я рассмотрю способы одурачивания отладчиков режима пользователя. Я попытаюсь разложить по полочкам все известные мне методы и предложить читателю несколько своих. Что я понимаю под одурачиванием? Во-первых, это обнаружение отладчика в своем процессе, обход функций, перебирающих модули, защита от трассировки и еще несколько примечательных вещей. Во-вторых, это несколько радикальных и не очень методов против дампа. В данной статье я рассмотрю только отладчики режима пользователя, очень мало будет сказано про отладчики режима ядра. О них и так очень много написано (советую прочитать статью Zero Ice'a в этом же номере журнала). Для того, чтобы читатель был в курсе откуда я взял всю информацию об устройстве отладчиков режима пользователя, я дам несколько ссылок в конце статьи. Там и описание msdn отладочных функций и туториал Izceliona "Win32 Debug API". Хочу еще заметить, что все примеры я тестировал и разрабатывал только для NT систем, хотя может оказаться, что примеры работают и в линейке 9x. Все предложения и замечания можно отсылать мне на e_f_i_m at rambler dot ru. **Обнаружение отладчика в процессе.**

Во-первых, надо знать как отладчик попадает в процесс. Сделать это он может двумя методами: создать ваш процесс CreateProcess с флагом DEBUG_PROCESS или вызвать DebugActiveProcess, чтобы присоединиться к уже выполняемому процессу. В любом случае, он оставляет "следы" в вашем процессе, которыми пользуется функция IsDebuggerPresent. "Ну и в чем же проблема?" - спросит читатель - "Вызываем IsDebuggerPresent и знаем есть ли отладчик или нет!". Все сомнения в "липовости" метода исчезают, когда диссасемблируешь эту функцию. Вот буквально ее код, без исправлений и украшательств, каким его можно увидеть набрав в SoftIce'e под XP (build 2600, без sp) u IsDebuggerPresent:

```
mov eax, [fs:00000018]
mov eax, [eax+30]
movzx eax, byte ptr [eax+2]
ret
```

Все! И это механизм защиты от нежелательной отладки Microsoft??? Выглядит довольно забавно. Соответственно отладчику надо всего лишь подкорректировать нужное значение в PEB структуре и эта функция корректно работать не будет. Этим и занимаются различные плагины, например, к OllyDBG

Стоит пояснить, как используется регистр fs в 3м (в 0м этот регистр используется по-иному) кольце Windows'ом. Дело в том, что регистр fs указывает на Thread Environment Block, сокращенно TEB. Об этом очень много писали, например Шрайбер ("Недокументированные возможности Windows 2000" - неплохая книга…), плюс недавно volodya из HI-TECH в рассылке от wasm.ru рассеял пару неточностей. Стоит дать ссылку на полное описание этой структуры, тем более, что мы будем использовать ее дальше (все ссылки см. в конце статьи).

Что же делать? Как обнаружить отладчик в нашем процессе, если легальные методы недостаточно надежны? Надо определять присутствие отладчика по косвенным признакам. Что в нашем случае косвенный признак? Это состояние dwCreationFlags, которое передается, как параметр CreateProcess'у. Но дело в том, что нет никаких стандартных средств узнать состояние этого параметра. Более того, даже недокументированных методов я не нашел! Изучение структуры PEB ни к чему не привело. Структуры режима ядра я сразу отбросил, хотя потом покопавшись все равно ничего не нашел. Таким образом, методов узнать dwCreationFlags нет, этот метод не подойдет. Пришлось придумывать самопальные способы, что, конечно, не плюс. Все же я изобрел метод, хоть и корявый. Тем не менее, он работает, как часы во всех windows'ах. Совершенно случайно я

наткнулся на ничем не примечательную функцию под скромным названием DebugBreak. Сначала я не уделил ей особого внимания, т.к. ее предназначение всего-то останавливать отладчик. Это что-то типа точки останова, которая ставиться отлаживаемым же процессом. Но когда я прочитал ее описание в SDK, я чуть не свалился со стула от радости, увидев таки лазейку, которая оставила нам микрософт! Я приведу здесь описание этой функции, как она есть в SDK:

The DebugBreak function causes a breakpoint exception to occur in the current process so that the calling thread can signal the debugger and force it to take some action. **If the process is not being debugged, the search logic of a standard exception handler is used.** In most cases, this causes the calling process to terminate because of an unhandled breakpoint exception.

VOID DebugBreak(VOID)
Parameters
This function has no parameters.
Return Values
This function does not return a value.
See Also
DebugActiveProcess

Оказывается, если нас не отлаживают, то управление передается seh обработчику! Алгоритм наших действий прост! Смотрите код на ассемблере:

```
; ########################################################################
; ###################### IS_THERE_DEBUGGEE? ##############################
; ########################################################################
; ############################ About #####################################
; ########################################################################
; Определяет под отладчиком ли мы режима пользователя.
; Компилятор -  тестировал на MASM v8.0
; Компилировать: вот фаил MakeIt.bat:
;------------------------------------ Start of MakeIt.bat ---------------------------------
; @echo off
;
; if exist 1.obj del 1.obj
; if exist 1.exe del 1.exe
;
; c:\masm32\bin\ml /c /coff /nologo 1.asm
; c:\masm32\bin\Link /SUBSYSTEM:WINDOWS /MERGE:.rdata=.text 1.obj
;
; dir 1.*
;
; pause
;------------------------------------- End of MakeIt.bat ---------------------------------
; ########################################################################
; ###################### Made by R4D][ ###################################
; ########################################################################

; ########################################################################
; ############################ Directives ################################
; ########################################################################
    .386
    .model flat, stdcall
    option casemap :none
; ########################################################################

; ########################################################################
; ############################ Includes and Libs #########################
; ########################################################################
      include c:\masm32\include\windows.inc
      include c:\masm32\include\kernel32.inc

      includelib c:\masm32\lib\user32.lib
      includelib c:\masm32\lib\kernel32.lib
; ########################################################################
; ############################ CODE ######################################
; ########################################################################
```

```
.code
start:
; Устанавливаем новый seh обработчик
 assume fs: nothing
 push offset seh_handler
 push dword ptr fs:[0]
 mov fs:[0],esp
; Вызываем DebugBreak
 call DebugBreak
WeUnderDebugger:
; Если DebugBreak не вызвал ошибок, то мы здесь => нас отлаживают!!!
; Выходим ;)
 jmp exit
seh_handler:
; Если мы здесь, то все OK и нас не отлаживают!!!
 mov esi, [esp+0ch]     ; В esi указатель на CONTEXT
 assume esi: PTR CONTEXT
; Устанавливаем новый eip и выходим
 mov [esi].regEip, offset WeArentBeingDebugged
 xor eax, eax
 ret
; Здесь основная программа...
WeArentBeingDebugged:
 nop
nop
nop
exit:
 push 0
 call ExitProcess
end start
; ########################################################################
```

Если вы не поняли прошлый листинг, вот код на C:

```
#include "stdafx.h"
#include <windows.h>

int APIENTRY realWinMain(HINSTANCE hInstance,
                     HINSTANCE hPrevInstance,
                     LPSTR     lpCmdLine,
                     int       nCmdShow)
{
// Этот WinMain выполняется только если мы не под дебаггером!
 MessageBox(0,"Uhhhhhooooooo!!!",NULL,MB_OK);
 return 0;
}

int APIENTRY WinMain(HINSTANCE hInstance,
                     HINSTANCE hPrevInstance,
                     LPSTR     lpCmdLine,
                     int       nCmdShow)
{
 __try {
   DebugBreak();
 }
 __except(EXCEPTION_EXECUTE_HANDLER) {
  realWinMain(hInstance,hPrevInstance,lpCmdLine,nCmdShow);
 }
 return 0;
}

  Что же такое DebugBreak? А не что иное, как

int 3
ret
```

:) Так что если хотите сохранить в программе несколько байт можете использовать int 3 вместо DebugBreak. Что произойдет в отладчике, когда мы выполним int 3? WaitForDebugEvent вернет dwDebugEventCode = EXCEPTION_DEBUG_EVENT и наша программа приостановиться... Таким образом, если вдруг производители отладчиков откажутся от использования

EXCEPTION_DEBUG_EVENT, им придется отказаться от установки брэйкпоинтов, что несомненно затруднит процесс нежелательной отладки вашей программы.

Чтобы была полная картина происходящего надо дать хотя бы краткое описание функции WaitForDebugEvent.

```
BOOL WaitForDebugEvent(

    LPDEBUG_EVENT lpDebugEvent, // address of structure for event information
    DWORD dwMilliseconds  // number of milliseconds to wait for event
   );
```

Эта функция вызывается в отладчике и заставляет застыть поток в ожидании отладочного события. Она работает как и другие Wait* функции, поэтому в последнем параметре принимает время ожидания, вместо которого обычно стоит INFINITE, что значит ждать бесконечно долго. В первом параметре функция принимает указатель на структуру DEBUG_EVENT, вот как она выглядит:

```
typedef struct _DEBUG_EVENT { // de
    DWORD dwDebugEventCode;
    DWORD dwProcessId;
    DWORD dwThreadId;
    union {
        EXCEPTION_DEBUG_INFO Exception;
        CREATE_THREAD_DEBUG_INFO CreateThread;
        CREATE_PROCESS_DEBUG_INFO CreateProcessInfo;
        EXIT_THREAD_DEBUG_INFO ExitThread;
        EXIT_PROCESS_DEBUG_INFO ExitProcess;
        LOAD_DLL_DEBUG_INFO LoadDll;
        UNLOAD_DLL_DEBUG_INFO UnloadDll;
        OUTPUT_DEBUG_STRING_INFO DebugString;

        RIP_INFO RipInfo;
    } u;
} DEBUG_EVENT;
```

О том, что обозначает каждый из полей структуры, я говорить не буду, отправлю вас лучше к SDK.

Защита от трассировки. Сначала нужно разобраться, что такое трассировка и какие есть средства для ее осуществления. Трассировка - это пошаговое выполнение отлаживаемой программы. Отладчики режима пользователя (да и режима ядра) используют для решения данной задачи регистр eflags, а именно его бит tf, ответственный за трассировку. Как это выглядит? Если установлен вышеуказанный бит, то процессор ПЕРЕД каждой инструкцией генерирует исключение int 1, сбрасывает tf и приостанавливает поток. WaitForDebugEvent в отладчике же возвращает EXCEPTION_DEBUG_EVENT + EXCEPTION_SINGLE_STEP. Причем отладчик устанавливает бит eflags с помощью мощной функции SetThreadContext, которая позволяет в любом потоке изменить любой из регистров (ну, почти любой, gdtr и остальные с ним связанные, вам, конечно, не дадут изменить :)). Тут решение просто напрашивается само собой: Проверять бит tf и если он установлен выходить. Но не тут-то было. Если внимательно посмотреть на этот абзац можно заметить некоторую особенность, из-за которой все провалиться. Так как исключение генерируется ДО выполнения инструкции и бит сбрасывается тогда же, то мы ничего не увидим в eflags. Я довольно долго думал над этой проблемой, пока мне в голову не пришла интересная идейка... Давайте мыслить логически: Если поток выполняется пошагово, то на каждый шаг тратиться гораздо больше времени, чем в обычных условиях, не так ли? Значит, чтобы понять, что нас трассируют надо просто замерить, сколько выполняется определенный, довольно маленький кусок кода и если это время больше секунды, то мы всяко под трассировкой. С другой стороны, это не факт, ибо кто-то в этот момент времени может нас просто приостановить (SuspendThread). Все же вот реализация этого метода:

Сначала, как полагается ассемблер:

```
; ######################################################################
; #################### IS_THERE_TRACE? ##############################
```

```
; ########################################################################
; ############################ About ######################################
; ########################################################################
; Определяет трасируемся ли мы отладчиком режима пользователя.
; Компилятор -  тестировал на MASM v8.0
; Компилировать: вот фаил MakeIt.bat:
;------------------------------------ Start of MakeIt.bat ---------------------------------
; @echo off
;
; if exist 1.obj del 1.obj
; if exist 1.exe del 1.exe
;
; c:\masm32\bin\ml /c /coff /nologo 1.asm
; c:\masm32\bin\Link /SUBSYSTEM:WINDOWS /MERGE:.rdata=.text 1.obj
;
; dir 1.*
;
; pause
;------------------------------------ End of MakeIt.bat -----------------------------------

; ########################################################################
; ###################### Made by R4D][ ###################################
; ########################################################################

; ########################################################################
; ############################ Directives ################################
; ########################################################################
    .386
    .model flat, stdcall
    option casemap :none
; ########################################################################

; ########################################################################
; ############################ Includes and Libs ##########################
; ########################################################################

      include c:\masm32\include\windows.inc
      include c:\masm32\include\user32.inc
      include c:\masm32\include\kernel32.inc

      includelib c:\masm32\lib\user32.lib
      includelib c:\masm32\lib\kernel32.lib
; ########################################################################

; ########################################################################
; ###################### DATA ############################################
; ########################################################################
.data
szMes  db "Here u can write original code of da program ;)",0
; ########################################################################

; ########################################################################
; ###################### CODE ############################################
; ########################################################################
.code
start:
 call GetTickCount
 mov ebx, eax
; Здесь у нас 100 nop'ов (90h = опкод nop'а)
db 100 dup(90h)
 call GetTickCount
 sub eax, ebx
 cmp eax, 1000
 jge exit
 invoke MessageBox, 0, offset szMes, NULL, MB_OK
exit: push 0
 call ExitProcess
end start
; ########################################################################
```

Реализация кода на C не составит никаких проблем:

```
#include "stdafx.h"
#include <windows.h>

int APIENTRY WinMain(HINSTANCE hInstance,
                     HINSTANCE hPrevInstance,
                     LPSTR     lpCmdLine,
                     int       nCmdShow)
{
  DWORD d  = GetTickCount();
// Не хватило у меня фантазии на что-то большее :)
 GetLastError();
 GetLastError();
 GetLastError();
 GetLastError();
 GetLastError();
 GetLastError();
 GetLastError();
 GetLastError();
 GetLastError();
 GetLastError();
 GetLastError();
 GetLastError();
 GetLastError();
 GetLastError();
 GetLastError();
 GetLastError();
 GetLastError();
 DWORD d1 = GetTickCount()-d;
 if (d1>=1000) return 0;
 MessageBox(0,"Uhhhhoooo",NULL,MB_OK);
 return 0;
}
```

Тут должно быть все понятно. Чтобы увеличить надежность метода я советую вам вызывать это многократно на протяжении всей программы, чтоб не получилось так, что вы из-за прихоти windows (а вдруг появиться real-time поток, который все процессорное время сожрет?) вы накажите обыкновенного пользователя... В то же время не стоит "маячить" перед глазами взломщика. Идеально подошел бы неявный метод защиты, описанный у Криса Касперского, когда программа "знает", что нас отлаживают, но ничего радикального не предпринимает, просто неправильно выполняя свои функции... Так, например, можно отключить всю защиту и предоставить взломщику голый код программы, без защиты. Разработанный им crack не будет работать вовсе.

Пока я занимался вопросом антитрассировки, я придумал универсальный метод обнаружения отладчика в процессе, который действует и на отладчики режима ядра. Смотрите код:

```
; ###########################################################################################################
; ############################## KILL_ALL_DA_DEBUGGEES ######################################################
; ###########################################################################################################
; ############################## About ######################################################################
; ###########################################################################################################
; Универсальный метод обнаружения отладки.
; Компилятор - тестировалось на MASM v8.0
; Компилировать: вот фаил MakeIt.bat:
;------------------------------------ Start of MakeIt.bat ---------------------------------
; @echo off
;
; if exist 1.obj del 1.obj
; if exist 1.exe del 1.exe
;
; c:\masm32\bin\ml /c /coff /nologo 1.asm
; c:\masm32\bin\Link /SUBSYSTEM:WINDOWS /MERGE:.rdata=.text 1.obj
;
; dir 1.*
;
; pause
;------------------------------------ End of MakeIt.bat -----------------------------------
; ###########################################################################################################
; ############################## Made by R4D][ ##############################################################
; ###########################################################################################################
```

```
; ######################################################################################################
; ############################ Directives ##############################################################
; ######################################################################################################
    .386
    .model flat, stdcall
    option casemap :none
; ######################################################################################################

; ######################################################################################################
; ############################ Includes and Libs #######################################################
; ######################################################################################################
      include c:\masm32\include\windows.inc
      include c:\masm32\include\kernel32.inc

      includelib c:\masm32\lib\user32.lib
      includelib c:\masm32\lib\kernel32.lib
; ######################################################################################################

; ######################################################################################################
; ################################### CODE #############################################################
; ######################################################################################################
.code
start:
; Устанавливаем обработчик seh
 assume fs: nothing
 push offset seh_handler
 push dword ptr fs:[0]
 mov fs:[0],esp
; Устанавливаем флаг трассировки
 pushf
 pop eax
 or eax, 100h
 push eax
 popf
 jmp exit
seh_handler:
 mov esi, [esp+0ch]
 assume esi: PTR CONTEXT
 mov [esi].regEip, offset WeWasntUnderDebuggee
 xor eax, eax
 ret
WeWasntUnderDebuggee:
 nop
 nop
 nop
 nop
exit: push 0
 call ExitProcess
end start
; ######################################################################################################
```

Как видите, просто устанавливаем флаг tf, далее перед следующей инструкцией возникнет #DB (int 1), которая в нормальном случае будет необработанным исключением. Если же мы под отладкой (all kind of debuggee ;)), то обработчик прерывания 1 установлен отладчиком и int 1 будет обработан. Обратите внимание, что это только если нас трассируют! В то же время, если нас просто прогоняли под отладчиком, не используя трассировку, то в момент после popf произойдет останов в отладчике (только режима пользователя, на SoftIce это не расспространяется, как дело обстоит с kd проверить не могу...), как будто нас до этого трассировали :)

Сокрытия модуля.

Допустим, вам надо укрыть свой модуль от "чужих" глаз. Для этого надо разобраться в том, как получают информацию о модулях такие приложения, как PETools. Обычно это делается с помощью снапшотов (snapshots) или с помощью функции NtQuerySystemInformation. В конце статьи вы можете найти ссылки на описание toolhelp функций и NtQueryInformation. При дисассемблирование, например, Module32FirstW бросается в глаза работа с регистром fs в режиме пользователя (происходит call 77e77604 (в XP build 2600), где идет оперирование с указанным выше регистром).

Я дам вам часть структуры PEB, которая нам понадобиться. А именно поле Ldr типа PPEB_LDR_DATA (Шрайбер ошибочно полагал, что там (смещение от PEB=00C) находиться ProcessModuleInfo типа PPROCESS_MODULE_INFO) в Process Environment Block'e (PEB), указатель на который (PEB) можно найти в fs:[30h] (он всегда равен 7ffdf000). Почему 30h? Потому что 0h указывает на TEB, а в TEB по смещению 0x30 находиться PEB. Поэтому и fs:[30h].

```
 struct _PEB_LDR_DATA {
 /*000*/ unsigned long   Length;
 /*004*/ unsigned char   Initialized;
 /*008*/ void*    SsHandle;
 /*00C*/ struct _LIST_ENTRY  InLoadOrderModuleList;
 /*014*/ struct _LIST_ENTRY  InMemoryOrderModuleList;
 /*01C*/ struct _LIST_ENTRY InInitializationOrderModuleList;
 };
```

Нас интересуют последние 3 поля, которые представляют собой входы в двусвязные списки (очень часто используются Windows). Вот как определил _LIST_ENTRY Шрайбер:

```
typedef struct _LIST_ENTRY
 {
/*000*/ struct _LIST_ENTRY *Flink;
/*004*/ struct _LIST_ENTRY *Blink;
/*008*/ }
 LIST_ENTRY;
```

Причем в Flink указывает на следующий элемент, а Blink на предыдущий. После 004 может находиться все что угодно, а в нашем случае это _LDR_DATA_TABLE_ENTRY:

```
struct _LDR_DATA_TABLE_ENTRY {
   /*000*/ struct _LIST_ENTRY   InLoadOrderLinks;
   /*008*/ struct _LIST_ENTRY   InMemoryOrderLinks;
   /*010*/ struct _LIST_ENTRY   InInitializationOrderLinks;
   /*018*/ void*     DllBase;
   /*01c*/ void*     EntryPoint;
   /*020*/ unsigned long    SizeOfImage;
   /*024*/ struct _UNICODE_STRING FullDllName;
   /*02c*/ struct _UNICODE_STRING BaseDllName;
   /*034*/ unsigned long    Flags;
   /*038*/ unsigned short    LoadCount;
   /*03a*/ unsigned short    TlsIndex;
   /*03c*/ struct _LIST_ENTRY   HashLinks;
   /*03c*/ void*     SectionPointer;
   /*040*/ unsigned long    CheckSum;
   /*044*/ unsigned long    TimeDateStamp;
   /*044*/ void*     LoadedImports;
 };
```

Мы видим, что первые 3 элемента структуры - это _LIST_ENTRY, названия которых соответствуют названиям различных списков. Очевидно, что это не что иное, как указатели на те же самые структуры, но разных списков! А если в название списка совпадает с тем, с которым мы сейчас работаем, то это просто *Flink и *Blink ;). Все это просто очень сильно облегчает нам работу. Итак, определимся, что мы будем делать:

- переходим к Peb.Ldr - переходим к InLoadOrderModuleList (к примеру) - сохраняем начальный элемент списка - перечисляем InLoadOrderModuleList с помощью первых 8 байт, которые занимают *Flink и *Blink (определяем конец списка засчет того, что Flink последнего элемента указывает на начало списка, а мы его сохранили) - сравниваем имена BaseDllName с нужным модулем - Если в прошлом шаге сравнение показало, что это нужная нам запись, то удаляем ее из всех списков, иначе продолжаем...

Как вы, наверное, заметили, для определения BaseDllName (ровно, как и FullDllName) используется структура UNICODE_STRING. Вот ее определение по Шрайберу:

```
typedef struct _UNICODE_STRING
  {
```

```
/*000*/ USHORT Length;
/*002*/ USHORT MaximumLength;
/*004*/ PWSTR  Buffer;
/*008*/  }
  UNICODE_STRING;
```

Нас, естественно, интересует только Buffer, поэтому смещаться будем к 004.

Еще непонятки может вызвать способ удаления элемента из списка. Делается это так: в Flink'е прошлого элемента записываем Flink удаляемого элемента, соответственно в Blink следующего элемента записываем Blink удаляемого.

```
DeleteListEntry proc USES eax ebx ListEntry: DWORD
 mov eax, ListEntry
 mov eax, [eax+4]

 mov ebx, [eax]    ; Flink
 mov ebx, [ebx]
 mov dword ptr [eax], ebx

 mov ebx, [eax+4]    ; Blink
 mov ebx, [ebx+4]
 mov dword ptr [eax+4], ebx
 ret
DeleteListEntry endp
```

Это довольно небезопасный код, потому что не используется seh, не стоит использовать его as is в случае, когда вы точно не уверены в том, что вы обрабатываете именно двусвязный список, и код будет обращаться к существующей странице памяти!

Далее следует код на ассемблере. Существует одна особенность, о которой я расскажу после кода...

```
; Удаляет модуль из списков... Параметр - КОРОТКОЕ (без пути) имя модуля
DelModuleFromPEBNtA proc USES ecx ebx eax modname: PCHAR
local pfirstmod        : DWORD
local modn[255]      : DWORD
; Переводим ANSI строку в UNICODE
 invoke MultiByteToWideChar, CP_ACP, 0, modname, -1, addr modn, 255
 assume fs: nothing
 mov eax, fs:[30h]      ; Здесь находиться Peb
 mov eax, [eax+0Ch]   ; Смещаемся к структуре Ldr
 mov eax, [eax+0Ch]   ; Смотрим на список InLoadOrderModuleList
; Сохраняем адрес первого в списке модуля, чтобы не войти в бесконечный цикл (это же двусвязный
    список)
 mov pfirstmod, eax
continue:
 mov ecx, [eax+30h]    ; BaseDllName
 push eax
 invoke lstrcmpiW, ecx, addr modn
 test eax, eax
 pop eax
 je Found
 mov eax, [eax]    ; Переходим к следующему элементу(Flink)
 cmp eax, pfirstmod
 jne continue
 ret
Found:
; Собственно удаляем элемент из Ldr.InLoadOrderModuleList
 push eax
 call DeleteListEntry
; Смещаемся к Ldr.InMemoryOrderModuleList
 add eax, 8
; И удаляем элемент оттуда ;)
 push eax
 call DeleteListEntry
comment @
 add eax, 8
 push eax
 call DeleteListEntry
```

```
@
 ret
DelModuleFromPEBNtA endp
```

Теперь обещанная особенность: внимательный читатель, наверное, заметил, что удаление из списка InInitializationOrderModuleList закомментировано. Это потому что этот список, судя по всему, интенсивно используется системой... Я не знаю, для какой цели, но если, к примеру, удалить из списка главный модуль нашего приложения, то оно незамедлительно выгрузиться БЕЗ каких-либо ошибок... Если же удалить user32.dll, то у нас будут проблемы с MessageBox'ом, к примеру. Он вызываться-то будет и его код будет получать управление(то есть модуль не выгружается, как мог бы подумать проницательный читатель), но будет где-то в вызывать Access Violation... В тоже время удаление из этого списка ntdll.dll не вызовет никаких проблем... На самом деле это не проблема, ибо этот список не используется (по крайней мере я не разу не видел...) отладочными функциями для получения списка модулей.

Рассмотрим пример. В примере, я буду удалять модуль из user32.dll и смотреть, какая будет реакция у различных отладочных функций и программ. Далее следует код (без вышеописанных функций):

```
; ######################################################################
; ########################## TEB Playing ###############################
; ######################################################################
; ############################ About ###################################
; ######################################################################
; Компилятор - тестировался на MASM v8.0
; Компилировать: далее MakeIt.bat:
;------------------------------------ Start of MakeIt.bat ---------------------------------
; @echo off
;
; if exist 1.obj del 1.obj
; if exist 1.exe del 1.exe
;
; c:\masm32\bin\ml /c /coff /nologo 1.asm
; c:\masm32\bin\Link /SUBSYSTEM:WINDOWS /MERGE:.rdata=.text 1.obj
;
; dir 1.*
;
; pause
;------------------------------------ End of MakeIt.bat -----------------------------------
; ######################################################################
; ######################### Made by R4D][ ##############################
; ######################################################################

; ######################################################################
; ########################## Directives ################################
; ######################################################################
    .386
    .model flat, stdcall
    option casemap :none
; ######################################################################

; ######################################################################
; ######################### Includes and Libs ##########################
; ######################################################################
      include c:\masm32\include\windows.inc
      include c:\masm32\include\user32.inc
      include c:\masm32\include\kernel32.inc
      include c:\masm32\include\masm32.inc
      include c:\masm32\macros\macros.asm

      includelib c:\masm32\lib\user32.lib
      includelib c:\masm32\lib\kernel32.lib
      includelib c:\masm32\lib\masm32.lib
; ######################################################################

; ######################################################################
; ############################ DATA ####################################
```

```
; ######################################################################
.data
 nmodname db "user32.dll",0
; ######################################################################

; ######################################################################
; ############################ CODE ####################################
; ######################################################################
.code
start:
main proc
local SnpSht         :DWORD
local me               :MODULEENTRY32
local IsItEnd          :BOOL
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;; Удаляем из списков user32.dll ;;
   push offset nmodname
   call DelModuleFromPEBNtA
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;; Проверяем на снэпшоты ;;;;;;;;;
        invoke CreateToolhelp32Snapshot, TH32CS_SNAPMODULE, 0
        mov SnpSht, eax
        mov me.dwSize, sizeof MODULEENTRY32
        invoke Module32First, SnpSht, addr me
        mov IsItEnd, TRUE
        .while IsItEnd!=FALSE
            invoke MessageBox, 0, addr me.szModule, addr me.szModule, MB_OK
            invoke Module32Next, SnpSht, addr me
            mov IsItEnd, eax
        .endw
        invoke CloseHandle, SnpSht
        xor eax, eax

   invoke MessageBox, 0, SADD("Look now in PETools module list"), SADD("Check this"), MB_OK
   invoke ExitProcess,0
main endp
end start
; ######################################################################
```

Если вы откомпилируете и запустите эту программу, то увидите, что снапшоты не показывают модуль user32, что и немудрено. Чтобы проверить NtQuerySystemInformation, не надо ничего писать, ибо за нас все давно написали ребята, которые делали PETools. Вы можете удостовериться, что эта программа использует именно эту функцию, просто поставив breakpoint на NtQuerySystemInformation в SoftIce'e и пронаблюдать, как PETools ее вызывает. PETools так же не показывает скрываемый модуль. Можно сделать вывод, что NtQuerySystemInformation тоже берет информацию из PEB. Теперь плохие новости. Отладчики режима ядра располагают более достоверной информацией. Но это и не мудрено. Система должна ведь хранить список модулей еще и в структуре режима ядра (я к сожалению, ничего не знаю ни об этой структуре, ни о том, как SI ее получает, но я уверен, что google об этом знает все :)), иначе это была бы даже не дыра, а просто убийство, ибо как иначе система следила бы за тем, какие модули выгружать из памяти при выгрузке процесса? Хранить столь важные сведения в структуре режима пользователя было бы опасно с точки зрения целостности всей системы. В то же время если бы эти сведения не дублировались в PEB, то было бы проблематично получение этой информации отладчикам режима пользователя... Поэтому данный метод не панацея от всех и вся.

Защита от дампа.

Дамп - это снимок памяти, который записывается на носитель. Дамп - это очень мощное оружие в умелых руках. Дамп помогает против зашифрованных программ, ибо "сфотографировать" память можно в любой момент, в то время, как перед тем как инструкция выполнится она должна быть расшифрована. У дампа очень много применений, но поговорим лучше, как не позволить отладчику (или специально-предназначенной для этого программе - дамперу) "сфотографировать" нас.

Очень интересный метод предложил volodya в статье "Об упаковщиках в последний раз". Этот метод заключается в изменении размера образа, таким образом, чтобы дампер обращался к

несуществующим страницам памяти. В этом случае дампер "рухнет". Я же предлагаю менее радикальный метод, подстановки 0 на место того места, где должен быть размер образа. Этот метод плох тем, что этот 0 сильно бросается в глаза... Тем не менее, вы можете подставить вместо нуля, что угодно. Например, вы можете хранить все самые важные подпрограммы в конце вашего модуля, а вместо размера модуля подставлять размер_вашего_ модуля - размер_подпрограмм. Информация о размере модуля храниться в PEB структуре. Немного подумав, я решил, что забавно было бы так же поменять базу модуля...

```
 assume fs: nothing
 mov eax, fs:[30h]   ; PEB
 mov eax, [eax+0Ch]  ; PEB_LDR_DATA
 mov eax, [eax+0Ch]  ; Ldr.InLoadOrderModuleList.Flink
 lea ebx, [eax+20h]   ; LDR_DATA_TABLE_ENTRY.SizeOfImage
 lea ecx, [eax+18h]   ; LDR_DATA_TABLE_ENTRY.DllBase
 mov dword ptr [ebx], 0  ; Подставляйте сюда, что хотите :)
 mov dword ptr [ecx], 0  ; Подставляйте сюда, что хотите :)
```

Некоторые же дамперы могут читать вышеперечисленные параметры из заголовка PE, так что не помешает так же и там подправить значение :)

```
; Узнаем базу нашего модуля
 push 0
 call GetModuleHandle
 mov ebx, eax
; Там, как известно находиться DOS заглушка
 assume eax: PTR IMAGE_DOS_HEADER
; В которой смещение на ре заголовок
 mov eax, [eax].e_lfanew
 add eax, ebx
 assume eax: PTR IMAGE_NT_HEADERS32
; В котором (а именно в OptionalHeader'e) и храниться размер и база модуля
 lea ebx, [eax].OptionalHeader.SizeOfImage
 lea ecx, [eax].OptionalHeader.ImageBase
 push ecx
 push ebx
; На всякий слуяай изменяем атрибуты страниц в этом районе на RW
 invoke VirtualProtect, ebx, 2, PAGE_READWRITE, offset dOld
 pop ebx
 pop ecx
; Пишите сюда, что хотите ;)
 mov dword ptr [ebx], 0
 mov dword ptr [ecx], 0
```

В итоге, можно предложить такой код на C против дампа:

```
#include "stdafx.h"
#include <windows.h>

void AntiDump(DWORD dwNewImageSize, DWORD dwNewBase)
{
// Без ассемблера не обойтись...
// Доступ к регистру fs можно получить
// только в ассемблерной вставке...
// Тем не менее, можно было бы добавить мегабайты
// описаний структур PEB и сократить ассемблерную
// вставку до 1 строчки, но я предпочел обойтись
// "жестко" забитыми смещениями...
// Описание PEB и связанных с ней структур
// смотрите в одной из ссылок в конце статьи.
  __asm
 {
 mov eax, fs:[30h]
 mov eax, [eax+0Ch]
 mov eax, [eax+0Ch]
 lea ebx, [eax+20h]
 lea ecx, [eax+18h]
 mov eax, dwNewImageSize
 mov dword ptr [ebx], eax
 mov eax, dwNewBase
```

```
mov dword ptr [ecx], eax
 };
 PIMAGE_DOS_HEADER pDH    = (PIMAGE_DOS_HEADER)GetModuleHandle(NULL);
 PIMAGE_NT_HEADERS32 pINH = (PIMAGE_NT_HEADERS32)((*pDH).e_lfanew+(DWORD)GetModuleHandle(NULL));
 DWORD pOp;
 VirtualProtect(&((*pINH).OptionalHeader.SizeOfImage),4,
PAGE_EXECUTE_READWRITE,&pOp);
 (*pINH).OptionalHeader.SizeOfImage = dwNewImageSize;
(*pINH).OptionalHeader.ImageBase = dwNewBase;
}

int APIENTRY WinMain(HINSTANCE hInstance,
                     HINSTANCE hPrevInstance,
                     LPSTR     lpCmdLine,
                     int       nCmdShow)
{
 AntiDump(0,0);
 MessageBox(0,"See da result in dumper ;)",NULL,MB_OK);
 return 0;
}
```

Немного подумав над проблемой дампа, я пришел к выводу, что можно неплохо было бы поместить внутри кода некоторые "дыры", у которых атрибуты страниц будут PAGE_NOACCESS. Мы не будем обращаться к этому коду, а вот программа, делающая дамп обязательно обратиться :) и если в ней нет проверки на атрибуты страниц - потерпит крах. НО этот метод защиты крайне неэффективный по причине того, что опытный взломщик составит карту памяти (OllyDbg, например, предоставляет такие услуги), и увидев наши дыры просто обойдет их, сделав дамп вашей программы по частям. К тому же довольно сложно рассчитать границы страниц, так чтобы мы "попали" VirtualProtect'ом именно так, что атрибуты изменились только у одной страницы-дыры. А если прибавить сюда неэффективность подобного метода (целую страницу (4Кб) на защиту только в одном месте!), то становиться ясно, что этот метод не подойдет нам.

Что еще можно предложить из методов обхода дампа? Можно, например, изменять базу нашего модуля на базу какой-нибудь другой dllки. А можно и вовсе назвать его ntoskrnl.exe и подставить базу и размер того же модуля. Представляю лицо взломщика, когда он увидит подобную картину :). В общем-то, все это дело фантазии, как и все о чем я написал в этой статье.

Хочу сказать спасибо Zero Ice'y, который поддерживал меня на протяжении написания всей статьи. Так же он дисассемблировал DebugBreak, показав, что это не что иное как int 3.

Так же хочу поблагодарить рассылку от wasm.ru. Как видите, ваш материал помогает и, я бы врят ли обратил столь пристальное внимание на такой кладезь знаний, как PEB, если бы не вы...

Ссылки.

1. Уроки Iszelion'a
2. Полное описание PEB
3. Статья volodya'и "Об упаковщиках в последний раз"
4. описание toolhelp функций
5. описание NtQueryInformation
6. Debug API

Статья была первоначально опубликована в журнале argc &argv.

Защита, использующая хасп-ключ. Часть 1

Автор: Chingachguk / HI-TECH

Дата: 2004-05-11

Что собираемся делать ?

Объект исследования: программа, использующая для защиты hasp-key **Инструменты**: Soft-Ice (под '98), hiew **Цель**: изучение win32-программ, обладающих защитой в нулевом кольце и hasp **Документация/источники**: статья "Програмная эмуляция HASP ключей By exefoliator, 2000", утилита haspgrab **Примечание**: это есть поверхностный анализ защиты такого типа, приношу свои извинения знатокам hasp'ов за возможно "наивные" рассуждения :)**Состав защиты** Программа включает в себя следующие компоненты: рабочий модуль prog.exe, возможно частично зашифрованный, два VxD драйвера: haspbig.vxd, весом полмегабайта, статический (загружаемый системой при старте) и hasplit.vxd, небольшой и динамический (загружаемый программой prog.exe при запуске). В порт компьютера LPT воткнут hasp-ключ. **Первая попытка.Soft-Ice**

Первое, что я попробовал сделать - это посмотреть, как защита работает с hasp. Воспользовавшись командой Soft-Ice мониторинга портов:

```
bpio 0378
bpio 0379
...
```

я ожидал получить break по доступу к портам ввода-вывода и сделать какие-нибудь предварительные выводы. Запускаем программу... нет. Нет никаких break'ов.

Почему ? Проверяю работу ice на тестовых (своих) программах и обнаруживаю: 0) обращение из дос-задачи перехватывается; 1) из vxd-драйвера не перехватывается. В случае 1 не помогают sti и т.п.

Получается, ice не перехватывает работу с портами с нулевого кольца (в 98 есть только два уровня: 0-ой, в котором находятся все vxd-драйвера и 3-ый: win32-приложения. Не считая, конечно, V86 для дос-программ и BIOS).

Вторая попытка.Сервисы перехвата VMM

Обнаружив в списке функций главного VxD системы функции перехвата обращений к портам типа:

```
VMMCall Enable_Global_Trapping
VMMCall Disable_Global_Trapping
...
```

пытаюсь применить их для вышеуказанной задачи. Эффект тот же, что и в ice: перехватываются обращения все, кроме как из нулевого кольца.

Третья попытка.Битовая карта ввода-вывода

При организации работы в защищенном режиме есть такая вещь: если программист использует задачи (task), то в описателе задачи (на который необходимо выполнить переключение командами jmp или call) содержится так называемая битовая карта запрещения ввода-вывода. Если текущий CPL задачи не соответсвует IOPL и в этой карте установлен бит запрещения доступа к порту, то процессор генерирует исключение. Эта удобная вещь используется дос-менеджерами типа emm386 для предоставления сервиса trapping'a (возможность установить ловушку и эмулировать работу устройства). Например, программа PCDOSEMU.COM ("PC3000AT HASP/ISA board emulator") использует эту возможность для эмуляции второго hdd.

Однако поверхностное вскрытие win показало, что она совсем не использует механизм переключения задач (бит NT - nested task - в регистре EFLAGS сброшен). К тому же не совсем ясно,

как настроить CPL (текущий уровень привелегий задачи) для VxD, ибо по умолчанию он у него наивысший.

Четвертая попытка.Регистры DRx

Однако никаких принципиальных ограничений написать свой собственный мониторинг портов аналогично ice не существует. Для этого я воспользовался интерфейсом процессора для отладки программ - регистрами отладки DRx.

Для настройки отладки необходимо загрузить в регистр DR7 флаги-описатели типа ловушки: по доступу на чтение/запись/исполнение, размер точки (байт/слово/dword) и т.п. (далее будет подробнее про форматы DRx). Адреса портов ввода-вывода (или памяти) необходимо записать в регистры DR0-DR3. При выполнении условия отладочного прерывания процессор выполнит исключение #1 - debug interrupt. Таким образом можно иметь 4 точки останова по доступу к портам - как раз то, что нужно. Минусом является невозможность работать бок о бок с sice в этом случае: он и такой перехватчик не смогут разделить общий ресурс - int 01h.

Сказано - сделано. Написав динамический vxd-драйвер, обладающий подобным перехватом портов:

```
.586p
; Необходимые инклюды для сборки VxD
include vmm.inc
include vwin32.inc
...
;
; Обрабатываем только сообщение w32_DeviceIoControl
;
Begin_control_dispatch SER
 Control_Dispatch w32_DeviceIoControl, OnDeviceIoControl
End_control_dispatch SER
...
;
; Секция данных
;
include log.inc
VxD_LOCKED_DATA_SEG
  Port equ 0378h
ttrap  struc
  offs1  dw  0   ; [0]
  sel    dw  28h ; [2]
  rsrv   db  0   ; [4]
  attr_  db  8fh ; [5]
  offs_h dw  0   ; [6]
ttrap  ends
trap        ttrap <>
old_trap_01 ttrap <> ; Старый описатель ловушки #01
old_trap_UD ttrap <>
tidt_descr struc
  idt_size dw ? ; размер idt
  lin_addr dd ? ; линейный адрес начала idt
tidt_descr ends
idt_descr tidt_descr <>
old_trapUD    dd 0 ; Адрес старого обработчика исключения #06
old_Jump_Addr dd 0 ; Адрес команд перехода на инсталлятор #06
; Буфер лога
LogPtr        dd 0 ; Указатель лог-буфера-его размер
LogPackets    db (MAX_LOG_RECORDS * sizeof(tLogPacket)) dup (?)
;
; Инсталлятор обработчика исключения #1
;
  mov  dword ptr LogPtr,0
  mov  dword ptr LastLogPtr,0

; Установка обработчика #1 - debug

; Создание дескриптора ловушки в idt
```

```
  mov  eax,offset32 TrapPort
  mov  trap.offs1,ax
  shr  eax,16
  mov  trap.offs_h,ax

; Загрузка в idt
; Получить текущий idt_base
  sidt fword ptr idt_descr
  mov  ebx,dword ptr idt_descr.lin_addr
; Получить начало описателя #1 - debug
  add  ebx,8

; Установить ловушку #1, сохранив старую

  mov  eax,dword ptr trap
  xchg [ebx],eax
  mov  dword ptr old_trap_01,eax
  mov  eax,dword ptr trap+4
  xchg [ebx+4],eax
  mov  dword ptr old_trap_01+4,eax

; Set trap to ports i/o
; Enable DE flag
  mov  eax,cr4
  or   eax,01000b
  mov  cr4,eax

; Ports set
  xor  ebx,ebx
  mov  bx,Port
  mov  dr0,ebx
  add  ebx,2
  mov  dr1,ebx
  add  ebx,2
  mov  dr2,ebx
  add  ebx,2
  mov  dr3,ebx

; Включение ловушек
  ;        FEDCBA9876543210FEDCBA9876543210
  mov  eax,01100110011001100010011111111111b
  mov  dr7,eax
 ...
;
; Обработчик исключения #1
;
TrapPort proc
  push ebp
  mov  ebp,esp
  push eax
  pushad
  push ds
  push es
  pushfd
  mov  ax,ss
  mov  ds,ax

; Адрес возврата EIP
  mov  ecx,[ebp+4]
; Адрес возврата CS->ES
  mov  es,word ptr [ebp+8]

  ...

  popfd
  pop  es
  pop  ds
  popad
  pop  eax
  pop  ebp
  iretd
```

```
TrapPort endp
```

Немного пояснений по вышеприведенному коду. Загруженному драйверу VxD система (VMM) посылает различные сообщения, которые он по своему желанию может обработать. Например, при нажатии кнопки "Пуск" всем VxD придет сообщение THREAD_INIT (~ создана новая нить и драйвер может что-то встроить в нее). Наш VxD обрабатывает из всего набора сообщений (~ 15 шт) одно: w32_DeviceIoControl. Это сообщение будет послано при загрузке с подфункцией DIOC_Open:

```
BeginProc OnDeviceIoControl
  assume esi:ptr DIOCParams
.if [esi].dwIoControlCode==DIOC_Open ; Контрольное сообщение ?!
  ...
```

Я имею в виду, что мы через esi добираемся до параметра при сообщении w32_DeviceIoControl. Такое сообщение система будет посылать нам всякий раз, когда win32-приложение будет вызвать нас через апи DeviceIoControl, но тогда параметр будет тем, который уже это приложение нам передаст (второй аргумент):

```
  invoke DeviceIoControl,hVxD,1,addr InBuffer,sizeof InBuffer,NULL,NULL,NULL,NULL
```

Инсталляция заключается в установке обработчика исключения #1 и инициализации отладочных регистров DRx. Для установки обработчика необходимо встроить дескриптор собственной ловушки в idt - interrupt table. Эта таблица состоит из элементов типа ttrap (выше) (не только - могут быть hardware прерывания, но нам это сейчас не важно). Адрес таблицы idt можно получит так: выгружаем текущий дескриптор idt командой sidt, из него получаем начальный адрес таблицы idt. Далее индексируемся на описатель 1 в ней и меняем его так, чтобы управление передавалось на нашу подпрограмму TrapPort. Загрузка регистра DR7 требует некоторого пояснения (лучше прочесть более детально в книге Зубкова С.В.):

```
; Включение ловушек
  ;        FEDCBA9876543210FEDCBA9876543210
  mov  eax,01100110011001100010011111111111b
  mov  dr7,eax

  Биты 31-30: 01 - размер точки останова: 2 байта
  Биты 29-28: 10 - тип точки: доступ к портам i/o
  Биты 27-16: аналогичны предидущим - другие точки останова тоже на i/o
  Биты 15-14: 00
  бит 13: бит GD - включает режим, в котором любое обращение к
                   любому отладочному регистру, даже из кольца защиты 0,
                   вызывает исключение #DB
  биты 12 - 10: 001
  бит 9: бит GE - если этот бит 0, точка останова по обращению к
                   данным может не сработать или сработать на несколько
                   команд позже, так что лучше всегда сохранять его равным 1
  бит 7: бит G3 - точка останова 3 включена
  ...
```

Теперь обработчик исключения #1 может получить управление и, например, записать в лог обмен приложения с портами ввода-вывода.

Тестирование на собственных "кошках": OK

Тестирование на защищенном приложении: нет перехвата !

Но почему ?! Почему я могу перехватить работу с портами из другого VxD, но не из защищенного ? Смотрим внимательнее на инициализацию DR7: никто не может помешать приложению (VxD защиты обладает ВСЕМИ правами) записать в DR7 ноль и никаких прерываний больше не будет. Достаем дизассемблер - hiew (точнее, hiew32) и открываем в нем hasplit.vxd:

```
; Если бит не 0, мусорим dr7 нулем
xx4C: 50             push        eax
xx4D: 2BC0           sub         eax,eax
xx4F: 0F23F8         mov         dr7,eax
xx52: 58             pop         eax
```

Ага, нам мешают ловить прерывания :) Но отсюда сразу же вытекает следующая проверка:

Найти все такие команды и забить nop'ами: bad result

Так выяснилось, что hasplit контролирует собственную модификацию (видимо, динамические проверки crc и т.п.). Пока получается замкнутый овал: нельзя трейсить обращения к портам, пока нам мешает маленький драйвер, но сам его трогать нельзя дабы убрать сброс отладки.

DeviceIOControl

Однако тут я сделал паузу но не скушал "твикс", а обратил внимание на то, что я пока был подобен солдату, бездумно кидающемуся на пулеметную точку вместо того, чтобы обойти ее сбоку и спокойно сделать head-shoot.

Зачем пытаться имитировать отклик порта, если можно перехватить "сеанс связи" приложения (prog.exe) с драйвером (драйверами). Единственный способ общения приложения с загруженным им драйвером VxD - это апи DeviceIOControl и больше ничего. Ну если не извращаться и не патчить всякий раз статические данные VxD и запускать его несколько раз подряд.

Перехват апи - это легко и приятно. По крайней мере, в сравнении с драйверами. На тему перехвата апи не буду распространяться, отошлю Вас к замечательной статье 90210 "техника перхвата апи". Скажу лишь, что я использовал метод, открытый Vecna (вызов первой экспортируемой kernel32.dll функции VxDCall0).

Таким образом мне удалось перехватить обмен через DeviceIOControl с драйверами:

```
MsgDeviceIOControlHook:
hVxD:00000030 SecParam: 9C402450 InBufferLen: 00000006 nOutBufferSize: 00000006 lpOverlapped: 00000000
InBuffer: 00 00 00 00 00 00
OutBufbefore: 00 00 00 00 00 00
VxDAnswerEAX: 00000001
(BytesReturned: 00000000):
Out buffer changed !
VxDAnswerMsg: 00 00 FA FA 19 03
MsgDeviceIOControlHook:
hVxD:00000034 SecParam: 00000003 InBufferLen: 00000000 nOutBufferSize: 000015E8 lpOverlapped: 00000000
InBuffer:
OutBufbefore: C7 85 8F 17 00 00 00 00 00 00 8D 95 F1 18 00 00 8D BD 73 17 00 00 B8 07 00 00 40 66 8C CB 8B D2 B9 6F 01 00 00 66 8C CE 8B FF FE EF 00 01 02 03 04 05 8D 95 F1 18 00 00 8D BD 74 17 00 00 B8 07 00 00 40 66 8C CB 8B D2 B9 6F 01 00 00 66 8C CE 8B FF FE EF 00 01 02 03 04 05 8D 95 F1 18 00 00 8D BD 75 17 00 00 B8 07 00 00 40 66 8C CB 8B D2 B9 6F 01 00 00 66 8C CE 8B FF FE EF 00 01 02 03 04 05 8D 95 F1 18 00 00 8D BD 76 17 00 00 B8 07 00 00 40 66 8C CB 8B D2 B9 6F 01 00 00 66 8C CE 8B FF FE EF 00 01 02 03 04 05 8B 8D 73 17 00 00 8B 9D 8F 17 00 00 8B 9C 1D 77 17 00 00 02 D9 C1 CB 08 C1 C9 08 02 D9 C1 CB 08 C1 C9 08 02 D9 C1 CB 08 C1 C9 08 02 D9 C1 CB 08 C1 C9 08 03 DD FF E3 57 56 52 51 53 50 33 C9 33 FF 66 8B CB 66 83 F9 00 75 04 66 B9 E8 03 66 8B F0 FC 66 33 FF 66 47 02 FB 80 C3 05 22 C3 32 D0 E2 F3 8A CA 58 5B 59 5A 5E 5F 57 56 52 51 53 50 33 C9 33 FF 66 8B C9 66 83 F9 00 75 04 66 B9 E8 03 66 8B F3 FC 66 33 FF 66 47 02 FB 80 C3 05 22 C3 32 D0 E2 F3 8A C2 58 5B 59 5A 5E 5F 8D 95 46 23 00 00 8D BD 46 23 00 00 B8 06 00 00 40 66 8C CB 8B D2 B9 9F 01 00 00 66 8C CE 8B FF FE EF 00 01 02 03 04 05 21 9A AA AA AE CC 62 16 72 F3 7C 4B AA AA 31 9F 8A AA AA 45 54 AA BA 8A 9A EA FA 92 F2 52 DB AA AA EA 72 F3 AC 0B AA AA 72 71 9D DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 74 BA AA AA CC 62 46 12 55
```

45 54 AA BA 8A 9A EA FA 72 F3 AC 0B AA AA 72 71 ED DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 74 BA AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 AC 0B AA AA 72 71 FD DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 74 BA AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 AC 0B AA AA 72 71 CD DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 74 BA AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 12 72 9D DB AA AA 12 73 52 DB AA AA 12 63 7B DD DB AA AA 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 9A 77 55 94 DF CF 8F BF 9F AF 99 36 99 55 CC 12 16 CC 92 35 AA FD EA CC 31 24 9A CC 12 A5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 06 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 36 CC 92 35 AA FD EA CC 31 24 9A CC 12 95 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 86 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 B5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 07 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 16 CC 92 35 AA FD EA CC 31 24 9A CC 12 A5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 04 2F 1F 3F 0F 4F 5F 72 F3 A6 A8 AA AA 72 71 A6 A8 AA AA 21 CA AA AA AE CC 62 16 12 87 31 C2 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 21 9A AA AA AE CC 62 16 72 F3 A6 A8 AA AA 31 C2 8A AA AA 45 54 AA BA 8A 9A EA FA 92 F2 52 DB AA AA EA 72 F3 7E 6B AA AA 72 71 9D DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 A8 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 7E 6B AA AA 72 71 ED DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 A8 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 7E 6B AA AA 72 71 FD DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 A8 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 7E 6B AA AA 72 71 CD DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 A8 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 12 72 9D DB AA AA 12 73 52 DB AA AA 12 63 7B DD DB AA AA 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 9A 77 55 94 DF CF 8F BF 9F AF 99 36 99 55 CC 12 16 CC 92 35 AA FD EA CC 31 24 9A CC 12 A5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 06 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 36 CC 92 35 AA FD EA CC 31 24 9A CC 12 95 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 86 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 B5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 07 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 16 CC 92 35 AA FD EA CC 31 24 9A CC 12 A5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 04 2F 1F 3F 0F 4F 5F 72 F3 F4 E8 AA AA 72 71 F4 E8 AA AA 21 CA AA AA AE CC 62 16 12 87 31 21 3A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 21 9A AA AA AE CC 62 16 72 F3 CE 98 AA AA 31 53 BA AA AA 45 54 AA BA 8A 9A EA FA 92 F2 52 DB AA AA EA 72 F3 7C 4B AA AA 72 71 9D DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 9F 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 7C 4B AA AA 72 71 ED DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 9F 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 7C 4B AA AA 72 71 FD DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 9F 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 7C 4B AA AA 72 71 CD DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 9F 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 12 72 9D DB AA AA 12 73 52 DB AA AA 12 63 7B DD DB AA AA 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 9A 77 55 94 DF CF 8F BF 9F AF 99 36 99 55 CC 12 16 CC 92 35 AA FD EA CC 31 24 9A CC 12 A5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 06 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 36 CC 92 35 AA FD EA CC 31 24 9A CC 12 95 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 86 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 B5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 07 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 16 CC 92 35 AA FD EA CC 31 24 9A CC 12 A5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 04 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 36 CC 92 35 AA FD EA CC 31 24 9A CC 12 95 65 CC 99 55 CC DE 8A 15 A2

96 FA 88 96 89 A7 84 95 02 84 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 B5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 05 2F 1F 3F 0F 4F 5F 72 F3 AC 0B AA AA 72 71 AC 0B AA AA 21 CA AA AA AE CC 62 16 12 87 31 74 BA AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 92 F2 52 DB AA AA EA 21 9A AA AA AE CC 62 16 72 F3 AC 0B AA AA 31 74 BA AA AA 45 54 AA BA 8A 9A EA FA 72 F3 A6 A8 AA AA 72 71 9D DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 C2 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 A6 A8 AA AA 72 71 ED DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 C2 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 A6 A8 AA AA 72 71 FD DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 C2 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 A6 A8 AA AA 72 71 CD DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 C2 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 12 72 9D DB AA AA 12 73 52 DB AA AA 12 63 7B DD DB AA AA 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 9A 77 55 94 DF CF 8F BF 9F AF 99 36 99 55 CC 12 16 CC 92 35 AA FD EA CC 31 24 9A CC 12 A5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 06 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 36 CC 92 35 AA FD EA CC 31 24 9A CC 12 95 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 86 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 B5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 07 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 16 CC 92 35 AA FD EA CC 31 24 9A CC 12 A5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 04 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 36 CC 92 35 AA FD EA CC 31 24 9A CC 12 95 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 84 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 B5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 05 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 16 CC 92 35 AA FD EA CC 31 24 9A CC 12 B5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 84 2F 1F 3F 0F 4F 5F 72 F3 7E 6B AA AA 72 71 7E 6B AA AA 21 CA AA AA AE CC 62 16 12 87 31 A8 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 92 F2 52 DB AA AA EA 72 F3 CE 98 AA AA 72 71 9D DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 53 BA AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 CE 98 AA AA 72 71 ED DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 53 BA AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 CE 98 AA AA 72 71 FD DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 53 BA AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 CE 98 AA AA 72 71 CD DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 53 BA AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 12 72 9D DB AA AA 12 73 52 DB AA AA 12 63 7B DD DB AA AA 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 9A 77 55 94 DF CF 8F BF 9F AF 99 36 99 55 CC 12 16 CC 92 35 AA FD EA CC 31 24 9A CC 12 A5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 06 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 36 CC 92 35 AA FD EA CC 31 24 9A CC 12 95 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 86 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 B5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 07 2F 1F 3F 0F 4F 5F 72 F3 7C 4B AA AA 72 71 7C 4B AA AA 21 CA AA AA AE CC 62 16 12 87 31 9F 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 21 BA AA AA AE 45 54 AA BA 8A 9A EA FA 21 9A AA AA AE CC 62 16 72 F3 7E 6B AA AA 31 A8 8A AA AA 45 54 AA BA 8A 9A EA FA BC FF CF DF CC 12 F1 EF 8A AA AA CC 12 71 8F 8A AA AA CC 12 72 B5 BA AA AA CC 12 73 B7 BA AA AA 02 35 CC 12 F3 8A 8A AA AA CC 12 72 AA 8A AA AA CC B2 71 B5 BA AA AA A5 AA 5A E2 97 FA AA AA D6 F2 7F 6A AA AA BA AA AA AA D6 F2 BC 6A AA AA BA AA AA AA AF CF 72 F1 77 BA AA AA 12 CA 32 F2 FC 6A AA AA 4F 2F D6 F2 3C 6A AA AA 2E AA AA AA D6 F2 7C 6A AA AA 28 AA AA AA 21 85 DB AA AA CC 32 F2 BD 6A AA AA 22 71 9D 6A AA AA CC 12 F2 64 BA AA AA 32 F2 FD 6A AA AA 22 73 3D 6A AA AA 32 72 7D 6A AA AA 32 F3 B2 6A AA AA 32 71 F2 6A AA AA 32 F1 32 6A AA AA 12 F2 50 6A AA AA 32 F2 73 6A AA AA CC D6 F2 90 6A AA AA EA AA CC 92 71 B5 BA AA AA 28 8D 5F CC 92 71 B5 BA AA AA A9 CD 8F CC 92 71 B5 BA AA AA F9 ED 2E CC 92 71 B5 BA AA AA E9 ED 49 CC 92 71 B5 BA AA

AA FF ED E9 CC 92 71 B5 BA AA AA AC ED 08 CC 92 71 B5 BA AA AA EC ED A8 CC 92 71 B5 BA AA AA 2D ED CB CC 92 71 B5 BA AA AA 3D ED 6A CC 92 71 B5 BA AA AA 7D ED 8A 14 9A 25 14 BA 35 9D 8A 14 AA 92 7D 57 0A ED 1A 92 7D 57 1A ED FA 34 E2 9A AA AA CC 12 73 B5 BA AA AA A2 15 99 ED 8E A2 15 89 5A E2 67 AA AA AA A2 15 F9 ED E9 A2 15 E9 5A E2 46 AA AA AA A2 15 AC ED C8 A2 15 FF ED B8 A2 15 6E ED 6B A2 15 7E 5A E2 C1 AA AA AA A2 15 29 ED 4A A2 15 D9 5A E2 20 AA AA AA 34 E6 BA AA AA 12 F2 32 6A AA AA 32 F2 33 6A AA AA 12 F2 73 6A AA AA 12 A5 51 0E 2A AA AA 9A 75 CC 92 71 B5 BA AA AA 28 8D 5F CC 92 71 B5 BA AA AA A9 CD 8F CC 92 71 B5 BA AA AA F9 ED 2E CC 92 71 B5 BA AA AA E9 ED 49 CC 92 71 B5 BA AA AA FF ED E9 CC 92 71 B5 BA AA AA AC ED 08 CC 92 71 B5 BA AA AA EC ED A8 CC 92 71 B5 BA AA AA 2D ED CB CC 92 71 B5 BA AA AA 3D ED 6A CC 92 71 B5 BA AA AA 7D ED 8A 14 9A 25 14 BA 35 9D 8A 14 AA 92 D6 8A 12 72 33 6A AA AA 21 8A AA AA AA CC D5 B4 12 26 95 E0 34 C2 AA AA AA 12 F2 32 6A AA AA 32 F2 33 6A AA AA 51 0E 2A AA AA 9A 75 CC 92 71 B5 BA AA AA 28 8D 5F CC 92 71 B5 BA AA AA A9 CD 8F CC 92 71 B5 BA AA AA F9 ED 2E CC 92 71 B5 BA AA AA E9 ED 49 CC 92 71 B5 BA AA AA FF ED E9 CC 92 71 B5 BA AA AA AC ED 08 CC 92 71 B5 BA AA AA EC ED A8 CC 92 71 B5 BA AA AA 2D ED CB CC 92 71 B5 BA AA AA 3D ED 6A CC 92 71 B5 BA AA AA 7D ED 8A 14 9A 25 14 BA 35 9D 8A 14 AA 34 21 AA AA AA CC 92 71 B5 BA AA AA 28 8D 5F CC 92 71 B5 BA AA AA A9 CD 8F CC 92 71 B5 BA AA AA F9 ED 2E CC 92 71 B5 BA AA AA E9 ED 49 CC 92 71 B5 BA AA AA FF ED E9 CC 92 71 B5 BA AA AA AC ED 08 CC 92 71 B5 BA AA AA EC ED A8 CC 92 71 B5 BA AA AA 2D ED CB CC 92 71 B5 BA AA AA 3D ED 6A CC 92 71 B5 BA AA AA 7D ED 8A 14 9A 25 14 BA 35 9D 8A 14 AA CC 12 73 EF 8A AA AA B2 94 55 55 AA AA 9E CC 55 F1 35 8A AA AA 21 8A AA AA AA CC D5 94 CC 32 F2 35 8A AA AA CC 52 F2 35 8A AA AA 14 7B D6 F2 33 6A AA AA AA AA AA AA D6 F2 73 6A AA AA AA AA AA AA CC D6 F2 B0 6A AA AA AA AA D6 F2 DF 55 55 55 2E AA AA AA CC 92 71 B5 BA AA AA 28 8D 5F CC 92 71 B5 BA AA AA A9 CD 8F CC 92 71 B5 BA AA AA F9 ED 2E CC 92 71 B5 BA AA AA E9 ED 49 CC 92 71 B5 BA AA AA FF ED E9 CC 92 71 B5 BA AA AA AC ED 08 CC 92 71 B5 BA AA AA EC ED A8 CC 92 71 B5 BA AA AA 2D ED CB CC 92 71 B5 BA AA AA 3D ED 6A CC 92 71 B5 BA AA AA 7D ED 8A 14 9A 25 14 BA 35 5A 82 17 AA AA AA 72 71 7F 6A AA AA AC 92 D6 EB 72 F1 56 38 AA AA 32 F1 94 BA AA AA 21 EA AA AA AE CC 62 17 12 D7 31 28 AA AA AA CC 62 46 12 71 94 BA AA AA 45 54 AA BA 8A 9A EA FA DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 95 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 06 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 B5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 86 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 36 CC 92 35 AA FD EA CC 31 24 9A CC 12 A5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 07 2F 1F 3F 0F 4F 5F BC 72 F2 7F 6A AA AA 34 6F BA AA AA CC 12 73 B5 BA AA AA A2 15 99 ED 99 A2 15 89 ED 48 A2 15 F9 ED 38 A2 15 E9 ED E8 A2 15 AC ED 5B A2 15 FF ED 0B D1 BA A2 15 6E ED 9B A2 15 7E ED 4A D1 AA A2 15 29 ED DA A2 15 D9 ED 8A 14 6B 12 F3 73 6A AA AA 99 36 CC 12 72 EF 8A AA AA A2 55 BA ED 8A 9A 36 55 F3 5A 55 55 55 72 71 7F 6A AA AA AC 72 F3 7F 6A AA AA 31 2E AA AA AA 55 F3 5A 55 55 55 92 D6 EB 72 F1 CB 18 AA AA 32 F1 94 BA AA AA 21 EA AA AA AE CC 62 17 12 D7 31 28 AA AA AA CC 62 46 12 71 94 BA AA AA 45 54 AA BA 8A 9A EA FA DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 95 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 06 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 B5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 86 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 36 CC 92 35 AA FD EA CC 31 24 9A CC 12 A5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 07 2F 1F 3F 0F 4F 5F BC 72 F2 7F 6A AA AA 14 2B FF 55 F3 D7 45 55 55 7F D6 F2 DF 55 55 55 EA AA AA AA 72 F2 F0 6A AA AA 32 FE 17 FF 55 FF E5 7F CC 12 73 B5 BA AA AA A2 15 99 ED 99 A2 15 89 ED 48 A2 15 F9 ED 38 A2 15 E9 ED E8 A2 15 AC ED 5B A2 15 FF ED 0B D1 BA A2 15 6E ED 9B A2 15 7E ED 4A D1 AA A2 15 29 ED DA A2 15 D9 ED 8A 14 08 12 F2 50 6A AA AA 32 F2 73 6A AA AA 99 A6 12 F3 73 6A AA AA 99 36 CC 12 72 EF 8A AA AA A2 55 BA ED 8A 9A 36 55 F3 1A 55 55 55 CC D6

F2 2E 2A AA AA AA AA 92 25 AA ED 7A 12 26 99 A6 99 17 99 87 34 D7 8A AA AA 92 7D 57 0A ED 1A 92 7D 57 1A ED FA 34 FE BA AA AA CC 92 71 B5 BA AA AA 28 8D 5F CC 92 71 B5 BA AA AA A9 CD 8F CC 92 71 B5 BA AA AA F9 ED 2E CC 92 71 B5 BA AA AA E9 ED 49 CC 92 71 B5 BA AA AA FF ED E9 CC 92 71 B5 BA AA AA AC ED 08 CC 92 71 B5 BA AA AA EC ED A8 CC 92 71 B5 BA AA AA 2D ED CB CC 92 71 B5 BA AA AA 3D ED 6A CC 92 71 B5 BA AA AA 7D ED 8A 14 9A 25 14 BA 35 5A 82 F7 AA AA AA 72 71 7F 6A AA AA AC 92 D6 EB 72 F1 9B 78 AA AA 32 F1 94 BA AA AA 21 CA AA AA AE CC 62 17 12 D7 31 28 AA AA AA CC 62 46 12 71 94 BA AA AA 45 54 AA BA 8A 9A EA FA DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 95 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 06 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 B5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 86 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 36 CC 92 35 AA FD EA CC 31 24 9A CC 12 A5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 07 2F 1F 3F 0F 4F 5F BC 34 B4 AA AA AA 72 71 7F 6A AA AA AC 92 D6 EB 72 F1 24 78 AA AA 32 F1 94 BA AA AA 21 CA AA AA AE CC 62 17 12 D7 31 28 AA AA AA CC 62 46 12 71 94 BA AA AA 45 54 AA BA 8A 9A EA FA DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 95 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 06 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 B5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 86 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 36 CC 92 35 AA FD EA CC 31 24 9A CC 12 A5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 07 2F 1F 3F 0F 4F 5F 72 F3 7F 6A AA AA 31 2E AA AA AA 55 F3 1A 55 55 55 BC 92 7D 57 0A ED 2A 92 7D 57 1A ED 8A 14 0D CC 12 73 B5 BA AA AA 14 BD A2 15 89 ED BB A2 15 E9 ED 6A A2 15 7E ED DA A2 15 D9 ED 8A 14 1F 51 0E 2A AA AA 9A 75 CC 12 73 EF 8A AA AA B2 94 55 55 AA AA 9E CC 55 F1 35 8A AA AA 21 8A AA AA AA CC D5 94 CC 32 F2 35 8A AA AA CC 52 F2 35 8A AA AA 12 71 50 6A AA AA 41 0E 2A AA AA 9A F5 92 C6 8A CC 12 72 EF 8A AA AA B2 B4 55 55 AA AA 21 8A AA AA AA CC D5 B4 12 26 95 E0 12 F2 F2 6A AA AA 12 73 32 6A AA AA 12 72 72 6A AA AA 12 F3 B3 6A AA AA 5F 4F 7F 60 B8 02 00 00 40 FE EF 00 01 02 03 04 05 B8 03 00 00 40 66 8C CB 8D 95 E5 24 00 00 B9 B8 09 00 00 FE EF 00 01 02 03 04 05 8D 95 F1 18 00 00 B9 E8 15 00 00 FF 95 0B FF FF FF 61 C3

VxDAnswerEAX: 00000001

(BytesReturned: 00000000):

VxDAnswerMsg: C7 85 8F 17 00 00 00 00 00 00 8D 95 F1 18 00 00 8D BD 73 17 00 00 B8 07 00 00 40 66 8C CB 8B D2 B9 6F 01 00 00 66 8C CE 8B FF FE EF 00 01 02 03 04 05 8D 95 F1 18 00 00 8D BD 74 17 00 00 B8 07 00 00 40 66 8C CB 8B D2 B9 6F 01 00 00 66 8C CE 8B FF FE EF 00 01 02 03 04 05 8D 95 F1 18 00 00 8D BD 75 17 00 00 B8 07 00 00 40 66 8C CB 8B D2 B9 6F 01 00 00 66 8C CE 8B FF FE EF 00 01 02 03 04 05 8D 95 F1 18 00 00 8D BD 76 17 00 00 B8 07 00 00 40 66 8C CB 8B D2 B9 6F 01 00 00 66 8C CE 8B FF FE EF 00 01 02 03 04 05 8B 8D 73 17 00 00 8B 9D 8F 17 00 00 8B 9C 1D 77 17 00 00 02 D9 C1 CB 08 C1 C9 08 02 D9 C1 CB 08 C1 C9 08 02 D9 C1 CB 08 C1 C9 08 02 D9 C1 CB 08 C1 C9 08 03 DD FF E3 57 56 52 51 53 50 33 C9 33 FF 66 8B CB 66 83 F9 00 75 04 66 B9 E8 03 66 8B F0 FC 66 33 FF 66 47 02 FB 80 C3 05 22 C3 32 D0 E2 F3 8A CA 58 5B 59 5A 5E 5F 57 56 52 51 53 50 33 C9 33 FF 66 8B C9 66 83 F9 00 75 04 66 B9 E8 03 66 8B F3 FC 66 33 FF 66 47 02 FB 80 C3 05 22 C3 32 D0 E2 F3 8A C2 58 5B 59 5A 5E 5F 8D 95 46 23 00 00 8D BD 46 23 00 00 B8 06 00 00 40 66 8C CB 8B D2 B9 9F 01 00 00 66 8C CE 8B FF FE EF 00 01 02 03 04 05 21 9A AA AA AE CC 62 16 72 F3 7C 4B AA AA 31 9F 8A AA AA 45 54 AA BA 8A 9A EA FA 92 F2 52 DB AA AA EA 72 F3 AC 0B AA AA 72 71 9D DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 74 BA AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 AC 0B AA AA 72 71 ED DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 74 BA AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 AC 0B AA AA 72 71 FD DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 74 BA AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 AC 0B AA AA 72 71 CD DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 74 BA AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 12 72 9D DB AA AA 12 73 52 DB AA AA 12

63 7B DD DB AA AA 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 9A 77 55 94 DF CF 8F BF 9F AF 99 36 99 55 CC 12 16 CC 92 35 AA FD EA CC 31 24 9A CC 12 A5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 06 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 36 CC 92 35 AA FD EA CC 31 24 9A CC 12 95 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 86 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 B5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 07 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 16 CC 92 35 AA FD EA CC 31 24 9A CC 12 A5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 04 2F 1F 3F 0F 4F 5F 72 F3 A6 A8 AA AA 72 71 A6 A8 AA AA 21 CA AA AA AE CC 62 16 12 87 31 C2 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 21 9A AA AA AE CC 62 16 72 F3 A6 A8 AA AA 31 C2 8A AA AA 45 54 AA BA 8A 9A EA FA 92 F2 52 DB AA AA EA 72 F3 7E 6B AA AA 72 71 9D DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 A8 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 7E 6B AA AA 72 71 ED DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 A8 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 7E 6B AA AA 72 71 FD DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 A8 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 7E 6B AA AA 72 71 CD DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 A8 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 12 72 9D DB AA AA 12 73 52 DB AA AA 12 63 7B DD DB AA AA 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 9A 77 55 94 DF CF 8F BF 9F AF 99 36 99 55 CC 12 16 CC 92 35 AA FD EA CC 31 24 9A CC 12 A5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 06 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 36 CC 92 35 AA FD EA CC 31 24 9A CC 12 95 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 86 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 B5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 07 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 16 CC 92 35 AA FD EA CC 31 24 9A CC 12 A5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 04 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 36 CC 92 35 AA FD EA CC 31 24 9A CC 12 95 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 84 2F 1F 3F 0F 4F 5F 72 F3 F4 E8 AA AA 72 71 F4 E8 AA AA 21 CA AA AA AE CC 62 16 12 87 31 21 3A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 21 9A AA AA AE CC 62 16 72 F3 CE 98 AA AA 31 53 BA AA AA 45 54 AA BA 8A 9A EA FA 92 F2 52 DB AA AA EA 72 F3 7C 4B AA AA 72 71 9D DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 9F 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 7C 4B AA AA 72 71 ED DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 9F 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 7C 4B AA AA 72 71 FD DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 9F 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 7C 4B AA AA 72 71 CD DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 9F 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 12 72 9D DB AA AA 12 73 52 DB AA AA 12 63 7B DD DB AA AA 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 9A 77 55 94 DF CF 8F BF 9F AF 99 36 99 55 CC 12 16 CC 92 35 AA FD EA CC 31 24 9A CC 12 A5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 06 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 36 CC 92 35 AA FD EA CC 31 24 9A CC 12 95 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 86 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 B5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 07 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 16 CC 92 35 AA FD EA CC 31 24 9A CC 12 A5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 04 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 36 CC 92 35 AA FD EA CC 31 24 9A CC 12 95 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 84 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 B5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 05 2F 1F 3F 0F 4F 5F 72 F3 AC 0B AA AA 72 71 AC 0B AA AA 21 CA AA AA AE CC 62 16 12 87 31 74 BA AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 92 F2 52 DB AA AA EA 21 9A AA AA AE CC 62 16 72 F3 AC 0B AA AA 31 74 BA AA AA 45 54 AA BA 8A 9A EA FA 72 F3 A6

A8 AA AA 72 71 9D DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 C2 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 A6 A8 AA AA 72 71 ED DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 C2 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 A6 A8 AA AA 72 71 FD DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 C2 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 A6 A8 AA AA 72 71 CD DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 C2 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 12 72 9D DB AA AA 12 73 52 DB AA AA 12 63 7B DD DB AA AA 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 9A 77 55 94 DF CF 8F BF 9F AF 99 36 99 55 CC 12 16 CC 92 35 AA FD EA CC 31 24 9A CC 12 A5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 06 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 36 CC 92 35 AA FD EA CC 31 24 9A CC 12 95 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 86 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 B5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 07 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 16 CC 92 35 AA FD EA CC 31 24 9A CC 12 A5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 04 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 36 CC 92 35 AA FD EA CC 31 24 9A CC 12 95 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 84 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 B5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 05 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 16 CC 92 35 AA FD EA CC 31 24 9A CC 12 B5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 84 2F 1F 3F 0F 4F 5F 72 F3 7E 6B AA AA 72 71 7E 6B AA AA 21 CA AA AA AE CC 62 16 12 87 31 A8 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 92 F2 52 DB AA AA EA 72 F3 CE 98 AA AA 72 71 9D DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 53 BA AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 CE 98 AA AA 72 71 ED DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 53 BA AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 CE 98 AA AA 72 71 FD DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 53 BA AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 72 F3 CE 98 AA AA 72 71 CD DB AA AA 21 DA AA AA AE CC 62 16 12 87 31 53 BA AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 12 72 9D DB AA AA 12 73 52 DB AA AA 12 63 7B DD DB AA AA 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 8A 37 B6 16 2A B6 36 2A 9A 77 55 94 DF CF 8F BF 9F AF 99 36 99 55 CC 12 16 CC 92 35 AA FD EA CC 31 24 9A CC 12 A5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 06 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 36 CC 92 35 AA FD EA CC 31 24 9A CC 12 95 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 86 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 B5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 07 2F 1F 3F 0F 4F 5F 72 F3 7C 4B AA AA 72 71 7C 4B AA AA 21 CA AA AA AE CC 62 16 12 87 31 9F 8A AA AA CC 62 46 12 55 45 54 AA BA 8A 9A EA FA 21 BA AA AA AE 45 54 AA BA 8A 9A EA FA 21 9A AA AA AE CC 62 16 72 F3 7E 6B AA AA 31 A8 8A AA AA 45 54 AA BA 8A 9A EA FA BC FF CF DF CC 12 F1 EF 8A AA AA CC 12 71 8F 8A AA AA CC 12 72 B5 BA AA AA CC 12 73 B7 BA AA AA 02 35 CC 12 F3 8A 8A AA AA CC 12 72 AA 8A AA AA CC B2 71 B5 BA AA AA A5 AA 5A E2 97 FA AA AA D6 F2 7F 6A AA AA BA AA AA AA D6 F2 BC 6A AA AA BA AA AA AA AF CF 72 F1 77 BA AA AA 12 CA 32 F2 FC 6A AA AA 4F 2F D6 F2 3C 6A AA AA 2E AA AA AA D6 F2 7C 6A AA AA 28 AA AA AA 21 85 DB AA AA CC 32 F2 BD 6A AA AA 22 71 9D 6A AA AA CC 12 F2 64 BA AA AA 32 F2 FD 6A AA AA 22 73 3D 6A AA AA 32 72 7D 6A AA AA 32 F3 B2 6A AA AA 32 71 F2 6A AA AA 32 F1 32 6A AA AA 12 F2 50 6A AA AA 32 F2 73 6A AA AA CC D6 F2 90 6A AA AA EA AA CC 92 71 B5 BA AA AA 28 8D 5F CC 92 71 B5 BA AA AA A9 CD 8F CC 92 71 B5 BA AA AA F9 ED 2E CC 92 71 B5 BA AA AA E9 ED 49 CC 92 71 B5 BA AA AA FF ED E9 CC 92 71 B5 BA AA AA AC ED 08 CC 92 71 B5 BA AA AA EC ED A8 CC 92 71 B5 BA AA AA 2D ED CB CC 92 71 B5 BA AA AA 3D ED 6A CC 92 71 B5 BA AA AA 7D ED 8A 14 9A 25 14 BA 35 9D 8A 14 AA 92 7D 57 0A ED 1A 92 7D 57 1A ED FA 34 E2 9A AA AA CC 12 73 B5 BA AA AA A2 15 99 ED 8E A2 15 89 5A E2 67 AA AA AA A2 15 F9 ED E9 A2 15 E9 5A E2 46 AA AA AA A2 15 AC ED C8 A2 15 FF ED B8 A2 15 6E ED 6B A2 15 7E 5A E2 C1 AA AA AA A2

15 29 ED 4A A2 15 D9 5A E2 20 AA AA AA 34 E6 BA AA AA 12 F2 32 6A AA AA 32 F2 33 6A AA AA 12 F2 73 6A AA AA 12 A5 51 0E 2A AA AA 9A 75 CC 92 71 B5 BA AA AA 28 8D 5F CC 92 71 B5 BA AA AA A9 CD 8F CC 92 71 B5 BA AA AA F9 ED 2E CC 92 71 B5 BA AA AA E9 ED 49 CC 92 71 B5 BA AA AA FF ED E9 CC 92 71 B5 BA AA AA AC ED 08 CC 92 71 B5 BA AA AA EC ED A8 CC 92 71 B5 BA AA AA 2D ED CB CC 92 71 B5 BA AA AA 3D ED 6A CC 92 71 B5 BA AA AA 7D ED 8A 14 9A 25 14 BA 35 9D 8A 14 AA 92 D6 8A 12 72 33 6A AA AA 21 8A AA AA AA CC D5 B4 12 26 95 E0 34 C2 AA AA AA 12 F2 32 6A AA AA 32 F2 33 6A AA AA 51 0E 2A AA AA 9A 75 CC 92 71 B5 BA AA AA 28 8D 5F CC 92 71 B5 BA AA AA A9 CD 8F CC 92 71 B5 BA AA AA F9 ED 2E CC 92 71 B5 BA AA AA E9 ED 49 CC 92 71 B5 BA AA AA FF ED E9 CC 92 71 B5 BA AA AA AC ED 08 CC 92 71 B5 BA AA AA EC ED A8 CC 92 71 B5 BA AA AA 2D ED CB CC 92 71 B5 BA AA AA 3D ED 6A CC 92 71 B5 BA AA AA 7D ED 8A 14 9A 25 14 BA 35 9D 8A 14 AA 34 21 AA AA AA CC 92 71 B5 BA AA AA 28 8D 5F CC 92 71 B5 BA AA AA A9 CD 8F CC 92 71 B5 BA AA AA F9 ED 2E CC 92 71 B5 BA AA AA E9 ED 49 CC 92 71 B5 BA AA AA FF ED E9 CC 92 71 B5 BA AA AA AC ED 08 CC 92 71 B5 BA AA AA EC ED A8 CC 92 71 B5 BA AA AA 2D ED CB CC 92 71 B5 BA AA AA 3D ED 6A CC 92 71 B5 BA AA AA 7D ED 8A 14 9A 25 14 BA 35 9D 8A 14 AA CC 12 73 EF 8A AA AA B2 94 55 55 AA AA 9E CC 55 F1 35 8A AA AA 21 8A AA AA AA CC D5 94 CC 32 F2 35 8A AA AA CC 52 F2 35 8A AA AA 14 7B D6 F2 33 6A AA AA AA AA AA AA D6 F2 73 6A AA AA AA AA AA AA CC D6 F2 B0 6A AA AA AA AA D6 F2 DF 55 55 55 2E AA AA AA CC 92 71 B5 BA AA AA 28 8D 5F CC 92 71 B5 BA AA AA A9 CD 8F CC 92 71 B5 BA AA AA F9 ED 2E CC 92 71 B5 BA AA AA E9 ED 49 CC 92 71 B5 BA AA AA FF ED E9 CC 92 71 B5 BA AA AA AC ED 08 CC 92 71 B5 BA AA AA EC ED A8 CC 92 71 B5 BA AA AA 2D ED CB CC 92 71 B5 BA AA AA 3D ED 6A CC 92 71 B5 BA AA AA 7D ED 8A 14 9A 25 14 BA 35 5A 82 17 AA AA AA 72 71 7F 6A AA AA AC 92 D6 EB 72 F1 56 38 AA AA 32 F1 94 BA AA AA 21 EA AA AA AE CC 62 17 12 D7 31 28 AA AA AA CC 62 46 12 71 94 BA AA AA 45 54 AA BA 8A 9A EA FA DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 95 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 06 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 B5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 86 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 36 CC 92 35 AA FD EA CC 31 24 9A CC 12 A5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 07 2F 1F 3F 0F 4F 5F BC 72 F2 7F 6A AA AA 34 6F BA AA AA CC 12 73 B5 BA AA AA A2 15 99 ED 99 A2 15 89 ED 48 A2 15 F9 ED 38 A2 15 E9 ED E8 A2 15 AC ED 5B A2 15 FF ED 0B D1 BA A2 15 6E ED 9B A2 15 7E ED 4A D1 AA A2 15 29 ED DA A2 15 D9 ED 8A 14 6B 12 F3 73 6A AA AA 99 36 CC 12 72 EF 8A AA AA A2 55 BA ED 8A 9A 36 55 F3 5A 55 55 55 72 71 7F 6A AA AA AC 72 F3 7F 6A AA AA 31 2E AA AA AA 55 F3 5A 55 55 55 92 D6 EB 72 F1 CB 18 AA AA 32 F1 94 BA AA AA 21 EA AA AA AE CC 62 17 12 D7 31 28 AA AA AA CC 62 46 12 71 94 BA AA AA 45 54 AA BA 8A 9A EA FA DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 95 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 06 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 B5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 86 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 36 CC 92 35 AA FD EA CC 31 24 9A CC 12 A5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 07 2F 1F 3F 0F 4F 5F BC 72 F2 7F 6A AA AA 14 2B FF 55 F3 D7 45 55 55 7F D6 F2 DF 55 55 55 EA AA AA AA 72 F2 F0 6A AA AA 32 FE 17 FF 55 FF E5 7F CC 12 73 B5 BA AA AA A2 15 99 ED 99 A2 15 89 ED 48 A2 15 F9 ED 38 A2 15 E9 ED E8 A2 15 AC ED 5B A2 15 FF ED 0B D1 BA A2 15 6E ED 9B A2 15 7E ED 4A D1 AA A2 15 29 ED DA A2 15 D9 ED 8A 14 08 12 F2 50 6A AA AA 32 F2 73 6A AA AA 99 A6 12 F3 73 6A AA AA 99 36 CC 12 72 EF 8A AA AA A2 55 BA ED 8A 9A 36 55 F3 1A 55 55 55 CC D6 F2 2E 2A AA AA AA AA 92 25 AA ED 7A 12 26 99 A6 99 17 99 87 34 D7 8A AA AA 92 7D 57 0A ED 1A 92 7D 57 1A ED FA 34 FE BA AA AA CC 92 71 B5 BA AA AA 28 8D 5F CC 92 71 B5 BA AA AA A9 CD 8F CC 92 71 B5 BA AA AA F9 ED 2E CC 92 71 B5 BA AA AA E9 ED 49 CC 92 71 B5 BA AA AA FF ED E9 CC 92 71 B5 BA AA AA AC ED 08 CC 92 71 B5 BA AA AA EC ED A8 CC 92 71 B5 BA AA AA 2D ED CB CC 92 71 B5 BA AA AA 3D ED 6A CC 92 71 B5 BA AA AA

```
7D ED 8A 14 9A 25 14 BA 35 5A 82 F7 AA AA AA 72 71 7F 6A AA AA AC 92 D6 EB 72 F1 9B 78
AA AA 32 F1 94 BA AA AA 21 CA AA AA AE CC 62 17 12 D7 31 28 AA AA AA CC 62 46 12 71
94 BA AA AA 45 54 AA BA 8A 9A EA FA DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35
AA FD EA CC 31 24 9A CC 12 95 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 06
2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A
CC 12 B5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 86 2F 1F 3F 0F 4F 5F DF
CF 8F BF 9F AF 99 36 99 55 CC 12 36 CC 92 35 AA FD EA CC 31 24 9A CC 12 A5 65 CC 99 55
CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 07 2F 1F 3F 0F 4F 5F BC 34 B4 AA AA AA 72 71
7F 6A AA AA AC 92 D6 EB 72 F1 24 78 AA AA 32 F1 94 BA AA AA 21 CA AA AA AE CC 62 17
12 D7 31 28 AA AA AA CC 62 46 12 71 94 BA AA AA 45 54 AA BA 8A 9A EA FA DF CF 8F BF
9F AF 99 36 99 55 CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 95 65 CC 99 55 CC DE
8A 15 A2 96 FA 88 96 89 A7 84 95 02 06 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55
CC 12 26 CC 92 35 AA FD EA CC 31 24 9A CC 12 B5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88
96 89 A7 84 95 02 86 2F 1F 3F 0F 4F 5F DF CF 8F BF 9F AF 99 36 99 55 CC 12 36 CC 92 35
AA FD EA CC 31 24 9A CC 12 A5 65 CC 99 55 CC DE 8A 15 A2 96 FA 88 96 89 A7 84 95 02 07
2F 1F 3F 0F 4F 5F 72 F3 7F 6A AA AA 31 2E AA AA AA 55 F3 1A 55 55 55 BC 92 7D 57 0A ED
2A 92 7D 57 1A ED 8A 14 0D CC 12 73 B5 BA AA AA 14 BD A2 15 89 ED BB A2 15 E9 ED 6A
A2 15 7E ED DA A2 15 D9 ED 8A 14 1F 51 0E 2A AA AA 9A 75 CC 12 73 EF 8A AA AA B2 94
55 55 AA AA 9E CC 55 F1 35 8A AA AA 21 8A AA AA AA CC D5 94 CC 32 F2 35 8A AA AA CC
52 F2 35 8A AA AA 12 71 50 6A AA AA 41 0E 2A AA AA 9A F5 92 C6 8A CC 12 72 EF 8A AA AA
B2 B4 55 55 AA AA 21 8A AA AA AA CC D5 B4 12 26 95 E0 12 F2 F2 6A AA AA 12 73 32 6A AA
AA 12 72 72 6A AA AA 12 F3 B3 6A AA AA 5F 4F 7F 60 B8 02 00 00 40 FE EF 00 01 02 03 04
05 B8 03 00 00 40 66 8C CB 8D 95 E5 24 00 00 B9 B8 09 00 00 FE EF 00 01 02 03 04 05 8D 95
F1 18 00 00 B9 E8 15 00 00 FF 95 0B FF FF FF 61 C3
MsgDeviceIOControlHook:
hVxD:00000034 SecParam: 00000003 InBufferLen: 00000000 nOutBufferSize: 00000048
lpOverlapped: 00000000
InBuffer:
OutBufbefore: 01 00 00 00 01 00 00 00 68 73 68 6C 48 00 00 00 28 00 00 00 F2 17 05 00 00 00
00 00 00 00 00 B5 1C 9A D6 AA 58 7C 81 00 00 43 00 00 00 44 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 02 1E 00 00 00 00 04 00
VxDAnswerEAX: 00000001
(BytesReturned: 00000000):
VxDAnswerMsg: 01 00 00 00 01 00 00 00 68 73 68 6C 48 00 00 00 28 00 00 00 F2 17 05 00 00
00 00 00 00 00 00 B5 1C 9A D6 AA 58 7C 81 00 00 43 00 00 00 44 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 02 1E 00 00 00 00 04 00
...
```

Не буду приводить весь лог. Скажу только то, что он оказался не очень большим. Я уже начал писать "заменитель" таких обращений, но вначале обратил внимание на то, что вызываемый драйвер меняет совсем немного данных, переданных ему. Только ради "прикола" я заменил все вызовы DeviceIOControl на ответы "все хорошо" и ожидал, естественно, что программа не обнаружит ключа hasp и не будет загружаться. Но ... приложение загрузилось как ни в чем не бывало !!!

Вот именно - не понял. Как же так же ? Если приложение НИКАК не общается с драйвером, но тем не менее умудряется выполнять проверки на hasp, то как оно это делает ???

Шестая попытка.Опять регистры DRx

Вот так. Предидущий непонятный момент я пока списал на сообщения, которые передает VMM VxD-драйверам или что еще. Вернемся на арену VxD, где пока была отложена схватка за DRx. Обращаем внимание на то, что в регистре DR7 есть бит 13:

```
; Включение ловушек
  ;        FEDCBA9876543210FEDCBA9876543210
  mov  eax,0110011001100110001001111111111b
  mov  dr7,eax

  бит 13: бит GD - включает режим, в котором любое обращение к
                   любому отладочному регистру, даже из кольца защиты 0,
                   вызывает исключение #DB
  ...
```

О. Вот то, что нам нужно. Эта фича поможет нам выполнить следующее: устанваливаем этот бит в 1, теперь все команды замусоривания регистров DRx автоматом перейдут к нам. Но мы тогда не будем их выполнять, а будем эмулировать:

```
; подпрограмма - ловушка #1 - debug
BD equ 0010000000000000b
B0 equ 0001b
B1 equ 0010b
B2 equ 0100b
B3 equ 1000b
CMD_IN_AL_DX equ 0ECh
;      FEDCBA9876543210
TrapPort proc
  push ebp
  mov  ebp,esp

  ...

; Адрес возврата EIP
  mov  ecx,[ebp+4]
; Адрес возврата CS->ES
  mov  es,word ptr [ebp+8]

; Причина отладочного прерывания
  mov  eax,dr6
  test eax,BD
  jz   @@NotDebugRegs

; Запомнить байты инструкции
; Адрес возврата EIP
; Команды мусоренья не запоминаем
;;  mov  ecx,[ebp+4]
;;  call Store_LogRecord

; Очищаем бит BD в регистре dr6
  and  eax, not BD
  mov  dr6,eax

; Если это было обращение к DRx, не нужно выполнять эти команды
; Запоминаем адрес возврата и возвращаемся в наш код
  add  ecx,3

  jmp  @@AfterDebugRegs

@@NotDebugRegs:

  ...

; Причина отладочного прерывания в доступе к портам ?
  mov  eax,dr6
  test eax,B0
  jz   @@NotDR0
; Очищаем бит B0 в регистре dr6
  and  eax, not B0
@@NotDR0:
  test eax,B1
  jz   @@NotDR1
; Очищаем бит B1 в регистре dr6
  and  eax, not B1
@@NotDR1:
```

```
    ...

    mov  dr6,eax

  @@AfterDebugRegs:

  ; Запоминаем адрес возврата и возвращаемся в наш код
    mov  dword ptr RetAddr_UD01_EIP,ecx
    mov  word ptr RetAddr_UD01_CS,es

    mov  [ebp+4],offset32 After_DebugRegisterUsed
    mov  word ptr [ebp+8],cs

    ...

  After_DebugRegisterUsed proc
    pushad
    pushfd
  ; Enable DE flag
    mov  eax,cr4
    or   eax,01000b
    mov  cr4,eax
    ;        FEDCBA9876543210FEDCBA9876543210
    mov  eax,01100110011001100010011111111111b
    mov  dr7,eax
    popfd
    popad
    jmp  dword ptr RetAddr_UD01_EIP
  After_DebugRegisterUsed endp
```

Тут несколько сложновато написано, но смысл такой: всякий раз при выполнении такого исключения все биты в DR7 скидываются, и мы получаем в исключении #1 его "чистым". Попытка проинициализировать его внутри обработчика приведет к краху системы, а отдавть управление назад со сброшенным регистром DRx как-то неприлично. Для этого я возвращаюсь не в вызвавший исключение код, но в собственную подпрограммку After_DebugRegisterUsed.

Запомнив адресок возврата, уже в этой программке делаю переинициализацию DR7 и возвращаюсь куда следует. Возможно, кто-то спросит меня, почему повторная инициализация DR7 внутри After_DebugRegisterUsed не приводит к вызову #1 - ? Ответ такой: не знаю. А если бы это было не так, то как вообще тогда установить DR7 ? :)

Ну вот. Теперь мы в "памперсе" и мы защищены от порчи DR7. Как удачно, что DR0-DR3 никто не трогает, но с этим тоже легко справиться. А еще не менее удачно, что никто не переставляет int 01 ;)

Но что это ? Запуск программы prog.exe опять не показал перехватов обращений к портам ! Точнее говоря, в логе видны только обращения к отладочным регистрам DRx и то, что мы их успешно "исправляем". Но нет, нет обращений к портам.

Обоюдоострое оружие от Intel

Нам интел дал мощное средство против защиты в виде DRx, но "меч не той версии". Почему ? После некоторых копаний в той же книжке Зубкова С.В. находим следующее:

```
  CR4: этот регистр (появился только в процессорах Pentium)
   управляет новыми возможностями процессоров. Все эти возможности
   необязательно присутствуют, и ...

  бит 3: бит DE - запрещает отладочные прерывания по обращению к портам
```

Упс. Но вот помешать защите сбросить этот бит никто не может, никакого обращения к нему процессор не контролирует.

Получается, что Интел дал нам обоим по разному оружию, и у всего оружия есть свои слабости и достоинства. Как же быть >?

Приходится опять лезть в вязкий (хоть и не очень большой) код малого hasplit в дизассемблере. И выяснять, как и насколько часто он портит уже и регситр CR4.

Попутно: как общается приложение с VxD

Чуть выше я задал сам себе загадку: как общается код win32-приложения с VxD драйвером, если изоляция DeviceIOControl не помогает ?

Так вот, при более детальном копании в недрах малого hasplit выясняется, что он делает следующие шаги: при получении сообщения от системной машины THREAD_INIT он проверяет, установлен ли его собственный обработчик исключений #6 и #14. Исключение #6 - это исключение UD - undefined operation. Если проц наткнется на опкод, который не сможет опознать, то он зовет это исключение. Исключение #14 - это Page Fault - ошибка записи или чтения в защищенную страницу памяти (при включенной страничной адресациии). Для вызова этого исключения существует специальная команда mov eax,ds:[0]. Шутка :)

Если обработчиков таких вот нет, то малый hasplit установит их на себя. Второй обработчик, видимо, не очень интересен: он проверяет, где эксепшен. Если внутри него же (внутри hasplit.vxd), то он "застрелится" из halt+cli, иначе передаст управление куда-то там дальше...

Приложение же загружает драйвер, походу слегка проверяя, нету ли тут Soft-Ice. Это делается попыткой открыть файл "\\.\SICE" и, затем, "\\.\NTICE". Но это совсем не главное. Далее приложение сознательно передаст управление на недопустимый к выполнению код. А дальше уже управление получит VxD и начнет свою часть работы. Таких вызовов состоится около 8-ми сотен. При этом VxD активно сопротивляется попыткам его трейсить, видимо, выполняя сверки CRC.

Поиск очистки CR4 внутри hasplit.vxd

В ходе трассировки hasplit'a как-то не встретилось таких команд, как mov cr4,eax со сброшенным битом DE. Попробуем поискать его в hiew - тоже нет. Да, кончено, hasplit активно противостоит даже просмотру в отладчике: распаковывает и выполняет свои функции в памяти и проч. Но все же странно...

А что-то я давно не выслушивал "начальника транспротного цеха". Что-то мы подзабыли большой и здоровый haspbig.vxd. А в нем ? Может, в нем есть такие команды ?

А что Вы думаете - есть ! Небольшая загадка: каким числом ? Правильно, один. Одна команда на полметра. Не может быть ! Быстро пишу nop, nop, nop..., кидаю его в SYSTEM, рестарт... работает ! Все. Теперь наш перехватчик чудесно все записывает - вот они, команды общения с портом LPT:

```
Total traps: 0000072366 See log file trap_01.log...
OpC=ee DX=0378 AL=80 (?) rep=0000
OpC=ee DX=0378 AL=c6 (?) rep=0000
OpC=ee DX=0378 AL=c7 (?) rep=0000
OpC=ee DX=0378 AL=c6 (?) rep=0000
OpC=ee DX=0378 AL=80 (?) rep=0000
OpC=ee DX=0378 AL=8a (?) rep=0000
OpC=ee DX=0378 AL=8b (?) rep=0000
OpC=ee DX=0378 AL=8a (?) rep=0000
OpC=ee DX=0378 AL=f8 (?) rep=0000
OpC=ee DX=0378 AL=f9 (?) rep=0000
OpC=ee DX=0378 AL=f8 (?) rep=0000
```

В первой колонке - опкод команды исключения (используются только чтение in al,dx или запись out dx,al).

Подмена сценария

Мы подходим к кульминации сюжета. Все бронежелеты заюзаны, осталось сердце защиты - hasp ключ. В него пишутся данные и получаются ответы. Эти ответы и нужно один раз записать и подставлять во всех остальных случаях.

Делаем первый тест. Списываем лог (около 53 тысяч команд, но это опционально от версии и т.п.). Допустим, сессия обмена всегда постоянна. Тогда, подставив его, мы получим возможность работы без ключа. Но ... не проходит.

Записываем несколько логов. Смотрим разницу. Ага, логи отличаются, но не сильно (примерно 10-50 команд чтения/записи на 53 тысячи). Это внушает оптимизм.

На данном этапе придется лезть в формат обмена данными с hasp. Лучшее, что я нарыл - это "Програмная эмуляция HASP ключей" by exefoliator. Там упоминается, что hasp реализует всего три функции:

```
HASP-3 поддерживает только 3 основные функции: ISHASP (дает информацию
о подключении HASP ключа - т.е. подключени или нет), HASPSTATUS (Возвращает
тип ключа HASP) и HASPCODE (возвращает 64 битный код в зависимости от
входного (seed) 16 битного кода). Остальные ключи поддерживают эти 3
функции + функции для записи/чтения в память, получение ID кода и доступа
к таймеру.
```

Далее говорится, что в коде обычно реализованы три функции вида:

```
Функции|      Параметры вызова| Возращаемое значение|
ISHASP        BH <- 1, BL <- LPT №.    AX = 0 (нету HASP) или 1 (HASP есть)

HASPCODE      BH <- 2, BL <- LPT №|.    AX = return code 1
              AX <- seed код           |BX = return code 2
              CX <- password1           |CX = return code 3
              DX <- password2           |DX = return code 4

HASPSTATUS    BH <- 5, BL <- LPT №.    |AX = размер памяти
              CX <- password1           |BX = тип HASP
              DX <- password2           |CX = LPT к которому подключено.
```

И приводится пример программки - эмулятора таких вызовов. Но то ли я не дорюхал, то ли что, но этот путь оказался для меня тупиком. По двум причинам: 0) Я не смог подобрать 8-мь байт данных для имитации отклика хасп-ключа по совету exefoliator'a; 1) Вызов подобный тому, что приводится выше, действительно есть, но его никто не делает. То ли это забыли с давних времен win 3.11, то ли под XP заготовка.

Однако изучить хасп может помочь замечательная утилитка haspgrab. Писана на паскале под дос и упакована, чудесно смотрится в отладчике. Все вызовы удобно идут подряд в main'e. Спасибо тому, кто ее сделал. Ниже в приложении 1, приводится выдержки из этой утилиты, которые я написал на том же паскале. А сейчас я приведу формат обмена с hasp-ключем, позволяющий получить Public Table ключа:

```
Out: 0C 80 C6 C7 C6 80 8A 8B 8A F8 F9 F8 DA DB DA BA BB BA 94 95 94
Out: C8 C9 C8 A8 A9 A8 80 81 80 92 93 92 D0 D1 D0 B0 B1 B0 8C 8D 8C
Out: 9E 9F 9E DC DD DC BC BD BC
Out: FE
In:  7F
Out: FC
In:  7F
Out: FA
In:  5F
Out: F8
In:  5F
; Далее в упрощенном формате записано
Out=>F6 BIn=7F Bout=F4 BIn=7F Bout=F2 BIn=7F Bout=F0 BIn=5F Bout=EE BIn=7F
Bout=EC BIn=7F Bout=EA BIn=7F Bout=E8 BIn=7F Bout=E6 BIn=7F Bout=E4 BIn=7F
Bout=E2 BIn=7F Bout=E0 BIn=7F Bout=DE BIn=5F Bout=DC BIn=5F Bout=DA BIn=5F
Bout=D8 BIn=5F Bout=D6 BIn=7F Bout=D4 BIn=5F Bout=D2 BIn=7F Bout=D0 BIn=5F
Bout=CE BIn=7F Bout=CC BIn=7F Bout=CA BIn=7F Bout=C8 BIn=7F Bout=C6 BIn=7F
Bout=C4 BIn=7F Bout=C2 BIn=7F Bout=C0 BIn=7F Bout=BE BIn=7F Bout=BC BIn=7F
Bout=BA BIn=5F Bout=B8 BIn=5F Bout=B6 BIn=7F Bout=B4 BIn=7F Bout=B2 BIn=7F
Bout=B0 BIn=5F Bout=AE BIn=7F Bout=AC BIn=7F Bout=AA BIn=5F Bout=A8 BIn=5F
Bout=A6 BIn=7F Bout=A4 BIn=7F Bout=A2 BIn=7F Bout=A0 BIn=5F Bout=9E BIn=5F
Bout=9C BIn=5F Bout=9A BIn=5F Bout=98 BIn=5F Bout=96 BIn=7F Bout=94 BIn=5F
```

```
Bout=92 BIn=7F Bout=90 BIn=5F Bout=8E BIn=5F Bout=8C BIn=5F Bout=8A BIn=5F
Bout=88 BIn=5F Bout=86 BIn=7F Bout=84 BIn=5F Bout=82 BIn=7F Bout=80 BIn=5F

Public table :50 50 73 73 FF 50 FF 73
```

Обащаю внимание на следующие особенности формата команд (это взгялд дилетанта :)). Сначала всегда идет что-то типа сброса (инициализации) начала передачи - out base+2,0Ch, потом out base,80h, потом отсыл байта C6 в виде трех посылок: C6,C7,C6. Вообще ключевой момент формата команд - это посылка байтов тройками. Сначла посылается байт, потом байт or Bit, потом опять байт. Bit опционален и может быть и 5-м, и 6-м. В приведенном выше примере затем посылается байт 80h, а потом вот именно в формате троек вот такая строка:

```
$8A,$F8,$DA,$BA, $94,$C8,$A8,$80, $92,$D0,$B0,$8C, $9E,$DC,$BC
```

Ответ на эту команду читается следующим образом: мы должны прочесть 64-е бита (помните статью exefoliator'a ?), но за одно чтение из порта можем читать один бит. Поэтому мы 64 раза отправляем в порт (номер бита x 2) or 80h и читаем байт из base+1, бит данных окажется 5-м. Это также ключевой момент в формате обмена: данные всегда читаются один бит за одно чтение, все ответы могут быть 5F или 7F only.

Переходим к анализу лога. Как я уже говорил, логи отличаются крайне незначительно. В первом приближении было видно, что отличия представляют собой два "пятна", один примерно в области строки ~33730 лога, другое - в области строки ~35660. Ниже приведены фрагменты 6-ти логов именно в области первого "пятна":

```
    lg1 lg2 lg3 lg4 lg5 lg6
ee =9e =9e =9e =9e =9e =9e
ee =be =be =be =be =be =be
ee =9e =9e =9e =9e =9e =9e
ee =de =de =de =de =de =de
ee =fe =fe =fe =fe =fe =fe
ee =de =de =de =de =de =de
ee =de =de =de =de =de =de
ee =fe =fe =fe =fe =fe =fe
ee =de =de =de =de =de =de
ee =9e =9e =9e =9e =9e =9e
ee =be =be =be =be =be =be
ee =9e =9e =9e =9e =9e =9e
ee =9e =9e =9e =9e =9e =9e
ee =be =be =be =be =be =be
ee =9e =9e =9e =9e =9e =9e
; diff started
ee =9e =9e =de =de =de =9e ; write 9e or de
ee =be =be =fe =fe =fe =be
ee =9e =9e =de =de =de =9e
; diff
ee =de =de =9e =9e =9e =de ; write 9e or de
ee =fe =fe =be =be =be =fe
ee =de =de =9e =9e =9e =de
; diff
ee =de =de =9e =9e =9e =de ; write 9e or de
ee =fe =fe =be =be =be =fe
ee =de =de =9e =9e =9e =de
; diff
ee =de =9e =9e =de =9e =de ; write 9e or de
ee =fe =be =be =fe =be =fe
ee =de =9e =9e =de =9e =de
; diff
ee =9e =de =9e =de =de =de ; write 9e or de
ee =be =fe =be =fe =fe =fe
ee =9e =de =9e =de =de =de
; begin read
ec =5f =5f =5f =5f =5f =5f
ee =be =fe =be =fe =fe =fe ; write be or fe
ee =9e =de =9e =de =de =de ; write 9e or de
ec =5f =5f =5f =5f =5f =5f
ee =be =fe =be =fe =fe =fe ; write be or fe
```

```
ee =9e =de =9e =de =de =de ; write 9e or de
ec =7f =7f =7f =7f =7f =7f
ee =be =fe =be =fe =fe =fe ; write be or fe
ee =9e =de =9e =de =de =de ; write 9e or de
ec =5f =5f =5f =5f =5f =5f
ee =be =fe =be =fe =fe =fe ; write be or fe
ee =9e =de =9e =de =de =de ; write 9e or de
ec =7f =7f =7f =7f =7f =7f
ee =be =fe =be =fe =fe =fe ; write be or fe
ee =9e =de =9e =de =de =de ; write 9e or de
ec =5f =5f =5f =5f =5f =5f
ee =be =fe =be =fe =fe =fe ; write be or fe
ee =9e =de =9e =de =de =de ; write 9e or de
ec =7f =7f =7f =7f =7f =7f
ee =be =fe =be =fe =fe =fe ; write be or fe
ee =9e =de =9e =de =de =de ; write 9e or de
ec =7f =7f =7f =7f =7f =7f
ee =be =fe =be =fe =fe =fe ; write be or fe
ee =9e =de =9e =de =de =de ; write 9e or de
ec =7f =7f =7f =7f =7f =7f
ee =be =fe =be =fe =fe =fe ; write be or fe
ee =9e =de =9e =de =de =de ; write 9e or de
ec =5f =5f =5f =5f =5f =5f
ee =be =fe =be =fe =fe =fe ; write be or fe
ee =9e =de =9e =de =de =de ; write 9e or de
ec =7f =7f =7f =5f =7f =7f ; read 7f or 5f
ee =be =fe =be =fe =fe =fe ; write be or fe
ee =9e =de =9e =de =de =de ; write 9e or de
ec =7f =7f =7f =7f =5f =7f ; read 7f or 5f
ee =be =fe =be =fe =fe =fe
ee =9e =de =9e =de =de =de
ec =5f =5f =7f =7f =7f =7f ; read 7f or 5f
ee =be =fe =be =fe =fe =fe
ee =9e =de =9e =de =de =de
ec =7f =7f =7f =7f =5f =7f ; read 7f or 5f
ee =be =fe =be =fe =fe =fe
ee =9e =de =9e =de =de =de
ec =7f =5f =5f =7f =7f =5f ; read 7f or 5f
ee =be =fe =be =fe =fe =fe
ee =9e =de =9e =de =de =de
ec =5f =5f =7f =5f =5f =5f ; read 7f or 5f
ee =be =fe =be =fe =fe =fe
ee =9e =de =9e =de =de =de
ec =5f =7f =5f =7f =5f =5f ; read 7f or 5f
ee =be =fe =be =fe =fe =fe ; write be or fe
ee =9e =de =9e =de =de =de ; write 9e or de
ee =9c =dc =9c =dc =dc =dc
```

Начнем анализ с первых строк. Сначала во всех логах загружаются одинаковые байты (тройками):

```
9e,be,9e
de,fe,de
de,fe,de
9e,be,9e
...
```

Но далее видно, что послдение пять загружаемых байт могут отличаться:

```
; diff started
ee =9e =9e =de =de =de =9e ; write 9e or de
ee =be =be =fe =fe =fe =be
ee =9e =9e =de =de =de =9e
; diff
ee =de =de =9e =9e =9e =de ; write 9e or de
ee =fe =fe =be =be =be =fe
ee =de =de =9e =9e =9e =de
; diff
ee =de =de =9e =9e =9e =de ; write 9e or de
ee =fe =fe =be =be =be =fe
ee =de =de =9e =9e =9e =de
; diff
ee =de =9e =9e =de =9e =de ; write 9e or de
```

```
ee =fe =be =be =fe =be =fe
ee =de =9e =9e =de =9e =de
; diff
ee =9e =de =9e =de =de =de ; write 9e or de
ee =be =fe =be =fe =fe =fe
ee =9e =de =9e =de =de =de
; begin read
ec =5f =5f =5f =5f =5f =5f
```

И далее идет чтение отклика hasp-ключа именно тройками байт:

```
out be,93 ; Два байта записаны
in <- 5f  ; Один бит результата
...
```

Во втором приближении приложение выполняет в потоке команд записи/ чтения откликов две команды где-то в середине сценария и при этом на вход подает различные данные: от этого и логи различаются. Но ! Эти данные отличаются лишь на ... 5-ть бит. Это 32 комбинации. Это немного. Это тоже можно эмулировать.

Детальнее проанализируем эту часть лога. Посмотрим чуть выше, где начинается эта запись. Вот ее детальный формат:

```
;
  Data1: array[0..14] of byte= (
    $C2,$F0,$D2,$FA, $B4,$C0,$AC,$C0,
    $82,$C0,$F0,$84, $DE,$9C,$01); {$01-признак конца передачи}

; Эти десять байт передаем в вышеприведенном примере в логе 1 (левый)
  TestSend: array[0..9] of byte= (
    $9E,$DE,$DE,$9E, $9E,$9E,$DE,$DE,
    $DE,$9E);

; Стандартный префикс: 80+80+C6x3
  OutPort(PortData,$80);
  OutPort(PortData,$80);
  Send_Twice($C6);
  OutPort(PortData,$80);

  Send_Data(Data1);
  OutPort(PortData,$9E);

; Передаем 10 бит данных, последние пять могут отличаться от сессии к сесии

  for i:=0 to sizeof(TestSend)-1 do
    Send_TwiceBit5(TestSend[i]);

  { Читаем ответ }
  write('Answer:');
  for i:=0 to 16 do
    begin
      ResByte:=InPort(PortRead);
      OutPort(PortData,$BE);
      OutPort(PortData,$9E);
      BytesToHexWithFiller(ResByte,DebStr,1);
      write(DebStr);
    end;
```

Таким образом, мы можем с помощью вот такого кода получить все возможные комбинации, которые программа только способна передать. Далее их нужно запомнить в подстановщике сценария - VxD, и при подмене сценария подставлять тот вариант ответа, который хочет программа в даннной сессии.

Что же до второго "пятна" в логах, то, как это ни странно, оно оказалось совершенно идентично первому :) Это та же самая команда, с теми же самыми различными последними 5-мя битами.

Таким образом в общем случае программная защита способна образовать 32x32 различных сессии, подставляя в двух случаях пять случайных бит. Возможно, это было сделано не специально

;) , а просто какая-нибудь секция .data? попадает под расшифровку. По крайней мере перевод системной даты на 5 лет вперед никак не помогает ей в этом.

Последнее, что осталось - это привести пример (корявый) подстановщика верного отклика в этих сессиях:

```
@@NotDebugRegs:

; Запомнить байты инструкции
; Адрес возврата EIP
  mov  ecx,[ebp+4]
; Имитируем только отклики порта LPT
; Адрес возврата CS
  mov  es,word ptr [ebp+8]

  call GetFrom_LogRecord ; EDX и EAX - регистры из лога

  cmp  byte ptr es:[ecx-1],CMD_IN_AL_DX
  jnz  @@SkipLogPortsCommands

; Регистр eax меняем
  mov  byte ptr [ebp-4],al

@@SkipLogPortsCommands:

  ...

;
; Получить байты отклика из лог-буфера (EAX, EDX)
GetFrom_LogRecord proc
; Запоминаем байты и значения регистров команды
  mov  eax,dword ptr ds:LastLogPtr
  cmp  eax,dword ptr LogPtr
  jb   LogNotDone
  mov  dword ptr ds:LastLogPtr,0
  mov  eax,0
LogNotDone:

  inc  dword ptr ds:LastLogPtr
  imul eax,eax,sizeof(tLogPacket)
  mov  ebx,offset32 LogPackets
  add  ebx,eax
  assume ebx: ptr tLogPacket
; Регистр eax
  movzx eax,byte ptr [ebx].RegEAX
; Регистр dx
  movzx edx,word ptr [ebx].RegDX
  assume ebx:nothing

  ;; jmp  @@SkipAnalize

; Анализируем, что отдать/получить надо
  mov  ebx,dword ptr ds:LastLogPtr
  xor  esi,esi
  cmp  ebx,33373-1
  jb   @@SkipAnalize
  cmp  ebx,(33373+63)-1
  jbe  @@AnalizeIt
  mov  esi,35555-33373
  cmp  ebx,35555-1
  jb   @@SkipAnalize
  cmp  ebx,(35555+63)-1
  ja   @@SkipAnalize

@@AnalizeIt:

; Безотносительно места в логе
  sub  ebx,esi
; Мы находимся в чтении ?
  cmp  ebx,33388-1
```

```
    jae  @@InCmds

  ; Нет, в записи
  ; Получаем номер пакета
    sub  ebx,33373-1
    push eax
    xor  dx,dx
    mov  ax,bx
    mov  bx,3
    div  bx
    mov  bx,dx
    movzx edi,ax
    pop  eax
    test bx,bx
    jnz  @@Next
  ; Номер пакета из трех байт передачи в edi (0-4)
  ; Записываемый байт в [ebp-4]
    push ecx
    push eax
    mov  al,byte ptr [ebp-4]
    shr  al,6
    and  al,1
    mov  ecx,edi
    shl  al,cl
    or   byte ptr ds:Variant,al
    pop  eax
    pop  ecx
    jmp  @@Next

  @@InCmds:

    cmp  ebx,33391-1
    jb   @@Next
    sub  ebx,33391-1

  ; Номер пакета чтения
    push eax
    xor  dx,dx
    mov  ax,bx
    mov  bx,3
    div  bx
    mov  bx,dx
    movzx edi,ax
    pop  eax
    test bx,bx
    jnz  @@Next
  ; edi - номер пакета ответа (0..15)
    movzx eax,byte ptr ds:Variant
    shl  eax,4
    mov  al,byte ptr ArtAnswers[eax+edi]
    cmp  edi,15
    jb   @@Next
  ; Сбросить Variant
    mov  byte ptr ds:Variant,0

  @@Next:

  @@SkipAnalize:

    ret
  GetFrom_LogRecord endp
```

Спасибо всем прочитавшим ;)

(C) Chingachguk /HI-TECH

Да охранит нас □ от □! - прим. wasm.ru

Приложение 1. Код на паскале для работы с hasp. Выдержки из haspgrab

```
{$G+}
```

```
const
  base = $0378;
  PortData = 0;
  PortRead = 1;
  PortServ = 2;

procedure OutPort(PortAddr:word; PortData: Byte);
  begin
    Port[base+PortAddr]:=PortData;
    {PrintByte(PortData,'Bout=');}
  end;
function InPort(PortAddr:word):Byte;
  var
    bTmp: byte;
  begin
    bTmp:=Port[base+PortAddr];
    {PrintByte(bTmp,'BIn=');}
    InPort:=bTmp;
  end;
Procedure Send_Twice(PData: Byte);
  begin
    OutPort(PortData,PData);
    OutPort(PortData,PData or 1);
    OutPort(PortData,PData);
  end;
Procedure Send_TwiceBit5(PData: Byte);
  begin
    OutPort(PortData,PData);
    OutPort(PortData,PData or $20);
    OutPort(PortData,PData);
  end;
const
  Data0: array[0..15] of byte= (
    $8A,$F8,$DA,$BA, $94,$C8,$A8,$80,
    $92,$D0,$B0,$8C, $9E,$DC,$BC,$01);

  Data1: array[0..14] of byte= (
    $C2,$F0,$D2,$FA, $B4,$C0,$AC,$C0,
    $82,$C0,$F0,$84, $DE,$9C,$01);

  TestSend: array[0..9] of byte= (
    $9E,$DE,$DE,$9E, $9E,$9E,$DE,$DE,
    $DE,$9E);

  TestSend1: array[0..9] of byte= (
    $9E,$DE,$DE,$9E,$9E, $9E,$9E,$9E,$9E,$9E);

Procedure Send_Data(var PData: array of byte);
  var
    i: word;
  begin
    i:=0;
    while PData[i] <> $01 do
      begin
        Send_Twice(PData[i]);
        inc(i);
      end;
  end;
Function Read_DataBit: byte;
  begin
    Read_DataBit:= ( (InPort(PortRead) shr 5) and 1 );
  end;
Procedure Read_8Bytes(var Res8Bytes: array of byte);
  var
    i,j: word;
  begin
    FillChar(Res8Bytes,8,$00);
    for i:=0 to $3F do
      begin
        OutPort(PortData,((i shl 1) or $80));
        j:=(i shr 3);
```

```
        Res8Bytes[j]:=Res8Bytes[j] or (Read_DataBit shl (7-(i and $07)) );
      end;
  end;
{ Идентично предидущему, можно читать биты в обратном порядке }
Procedure Read_Swap8Bytes(var Res8Bytes: array of byte);
  var
    i,j: word;
  begin
    FillChar(Res8Bytes,8,$00);
    for i:=$3F downto 0 do
      begin
        OutPort(PortData,((i shl 1) or $80));
        j:=(i shr 3);
        Res8Bytes[j]:=Res8Bytes[j] or (Read_DataBit shl (7-(i and $07)) );
      end;
  end;
{$F+}
{ Заполнить строку AddrHex hex-представлением Number байтов из AddrBytes, }
{ но байты через пробел }
Procedure BytesToHexWithFiller(var AddrBytes,AddrHex; Number:Word);
  Begin
    Asm
    pusha
    push ds
    push es
    cld
    lds  si,AddrBytes
    les  di,AddrHex
    xor  cx,cx
    mov  al,byte ptr Number
    mov  cl,al
    shl  al,1
    add  al,cl {len x 3}
    stosb
    test cx,cx
    jz   @@Done
@@NextByte:
    lodsb
    mov  ah,al
    shr  al,4
    cmp  al,10
    sbb  al,69h
    das
    stosb
    mov  al,ah
    and  al,0Fh
    cmp  al,10
    sbb  al,69h
    das
    stosb
    mov  al,' '
    stosb

    loop @@NextByte
@@Done:
    pop  es
    pop  ds
    popa
    End;
  End;
var
  Res: array[0..7] of byte;
  ResByte: byte;
  DebStr: string;
  i, variant: word;
begin
  writeln('Start !');
  writeln('Send ini...');

  OutPort(PortServ,$0C);
  OutPort(PortData,$80);
```

```
Send_Twice($C6);
OutPort(PortData,$80);

Send_Data(Data0);

{Read_8Bytes(Res);}
Read_Swap8Bytes(Res);

BytesToHexWithFiller(Res,DebStr,8);

writeln('Public table ?:',DebStr);

exit;

writeln('Write test:');

OutPort(PortData,$80);
OutPort(PortData,$80);
Send_Twice($C6);
OutPort(PortData,$80);

Send_Data(Data1);
OutPort(PortData,$9E);

for i:=0 to sizeof(TestSend)-1 do
  Send_TwiceBit5(TestSend[i]);

{ Читаем ответ }
write('Answer:');
for i:=0 to 16 do
  begin
    ResByte:=InPort(PortRead);
    OutPort(PortData,$BE);
    OutPort(PortData,$9E);
    BytesToHexWithFiller(ResByte,DebStr,1);
    write(DebStr);
  end;

writeln('All tests:');

for variant:=1 to 32 do
  begin

    {BytesToHexWithFiller(TestSend1,DebStr,sizeof(TestSend1));
    writeln('Send:',DebStr);}

    OutPort(PortData,$80);
    OutPort(PortData,$80);
    Send_Twice($C6);
    OutPort(PortData,$80);

    Send_Data(Data1);
    OutPort(PortData,$9E);

    for i:=0 to sizeof(TestSend)-1 do Send_TwiceBit5(TestSend1[i]);

    { Читаем ответ }
    write('Answer:');
    for i:=0 to 16 do
      begin
        ResByte:=InPort(PortRead);
        OutPort(PortData,$BE);
        OutPort(PortData,$9E);
        BytesToHexWithFiller(ResByte,DebStr,1);
        asm mov  byte ptr DebStr[0],2 end;
        write('0',DebStr,'h');
        if i<16 then write(',');
      end;
    writeln;
    {readln;}
    { Следующий посыл }
```

```
      for i:=5 to 5+5 do
        begin
          if TestSend1[i] = $9E then
            begin
              TestSend1[i]:=$DE;
              break;
            end;
          TestSend1[i]:=$9E;
        end;
    end;

  writeln('Finish ;)');
end.
```

Защита, использующая хасп-ключ. Часть 2

Автор: Chingachguk / HI-TECH

Дата: 2004-05-11

Пояснение ко второй части

Данная статья является расширением к моей первой статье, посвященной анализу защиты этой программы ("Защита, использующая хасп-ключ").

В первой статье был приведен поверхностный анализ защиты без исследования ее механизма: было приведено лишь поверхностное описание, которое также содержало ошибки (впочем, непринципиальные). В этой статье я собираюсь привести более детальное описание структуры защиты, полный код эмулятора отклика драйвера защиты, включающий в себя простейший эмулятор функций хасп-ключа. Вместе с этим **хочу подчеркнуть, что ни в коей мере не претендую на уникальность исследования этой защиты: первенство, безусловно, принадлежит С-лю и М-о**. Новым (возможно) является метод перехвата функций драйвера и обход контроля целостности драйвера.

В первой моей статье программа исследовалась на уровне команд ввода-вывода (in/out) и приводилось описание того, как можно эмулировать отклик хасп-ключа на этом уровне: требуется создать лог ввода-вывода, проанализировать динамические части (меняющиеся от сессии к сессии) на предмет их эмуляции и научится избегать сброса отладочных регистров (DRx), применяемого защитой.

В данном случае будет подробно описано, как именно устроен код, общающийся с ключом и реализующий так называемые "функции" хасп-ключа и исследован метод эмуляции не отклика самого ключа (команды ввода-вывода), но эмуляции функций хасп-ключа.

Все исследования проводились в win98.

Общее описание структуры защиты

Начну с пояснения по составу защиты. Во-первых, это, конечно, приложение (exe), а во-вторых это три драйвера. Из них только один (с постфиксом "DL") является "полезным" кодом, другой просто не вызвается, третий вызвывается, но от наличия или отсутствия ключа его поведение никак не меняется. Однако инсталлятор (тот, кто устанавливает или удаляет драйвер защиты) "нагло" оставляет его в папке \SYSTEM. А ведь он занимает достаточно много места (особенно поддерживающий версию 8 - 0,5 мегабайта) в оперативной памяти. Все три драйвера являются динамическими (загружаемые по запросу приложения но не на этапе инициализации системы).

Итак, существует приложение и драйвер (VxD). Драйвер загружается в память посредством первого вызова CreateFile. Получив сообщение THREAD_INIT от системной машины, он устанавливает два обработчика исключений: #6 (UD - undefined opcode) и #14 - Page Fault. Приложение, в свою очередь, сознательно вызывает исключение UD путем примерно следующих конструкций:

```
mov  eax,40000007h
mov  ecx,Param1
mov  edx,Param2
...
db   0FEh,0EFh,00h,01h,02h,03h,04h,05h <<< Вызов исключения
```

Обработчик исключения UD анализирует байты, вызвавшие исключение - буквально сравнением с "0FEh,0EFh,...,04h,05h" и трактует значение в регистре eax как **номер функции**. Всего поддерживается 11 функций: от 40000001h до 4000000Bh, однако реально вызваются не все, а только функции (здесь и далее в упрощенном до байта виде) 01,02,03,04,06,07 и 0B. Вот их предназначение:

```
Функция 1:
 Возврат в точку исключения + 8;

Функция 2:
 Возврат в точку исключения + 8;

Функция 3:
 Decrypt data:
             rol [each byte],4
             xor [each byte],0AAh,
            Возврат в точку исключения + 8;

Функция 4:
 Encrypt data:
             rol [each byte],4
             xor [each byte],0AAh,
            Возврат на адрес, указанный вызвавшим в регистре edi;

Функция 6:
 Decrypt data:
             xor [each byte],0AAh,
             ror [each byte],4
            Возврат на адрес, указанный вызвавшим в регистре edi;

Функция 7:
 Get CRC:
             add CRC_BYTE,[each byte]
            Возврат в точку исключения + 8;

Функция 0B:
 Execution hasp key functions:
            Выполнить указанную в структуре по [edi] функцию хасп-ключа
            Возврат в точку исключения + 8;
```

Таким образом, функции 1,2,3,4,6 и 7 непосредственного отношения к ключу не имеют, но выполняют вспомогательные функции от простого обхода байт, вызвавших исключение (функции 1,2) до зашифрования/зашифрования блока данных, 7-ая функция вычисляет crc (байт). Однако функция 0Bh является одним из узловых мест защиты.

Конечно, это не все, что делают эти функции, но это все, что необходимо эмулировать. Так, например, некоторые из них "портят" отладочные регистры DRx (DR7 только), пытаясь противостоять отладке и в первом приближении прослеживается даже попытка как-то согласовать выполнение защитного кода путем передачи через DR7 каких-то чисел но это не является обязательным; в самом крайнем случае можно сказать, что необходима лишь эмуляция функции, непосредственно использующей ключ - 0B. Однако я эмулировал все функции исключения UD: меня всегда раздражали программы, исполняющие неоптимизированный код, а каждая из этих функций предваряет свое выполнение расшифровкой (далеко не из одного цикла).

Обработчик исключения #14 ничего особого не делает, разве что помогает расположенному в приложении коду распаковывать себя и т.п.

Разумеется, драйвер выполняет и стандартные запросы приложения (DeviceIoControl). В этом случае он работает с ключом, и эти запросы необходимо эмулировать.

Приложение использует построенные таким образом на исключении UD api-функции и вызовы стандартной DeviceIoControl для проверки наличия хасп-ключа. Основной особенностью использования исключения UD является попытка всеми возможными средствами маскировать вызовы функции 0B:

```
- Приложение выполняет примерно 128 вызовов исключения UD, используя
  функции 1,2,3,4,6 и 7, "разбавляя" их несколькими промежуточными
  вызовами DeviceIoControl;
- Приложение расшифровывает на стек находящийся в exe-фйале код,
  примерно ~A000h байт, причем этот код исполняется со всеми правами
  0-го кольца защиты (! - чем не вирус ?);
```

- Помогая себе обработчиком SEH'а, этот код, реализующий что-то типа виртуальной машины (реализованы команды типа MOV,CMP, виртуального регистра флагов (флага ZF)) выполняет довольно серьезную проверку модификации драйвера защиты: командой sidt получает текущую idt, извлекает из нее адрес обработчика исключения PageFault и проверяет прямым сравнением первые байты этого обработчика: они должны, по его мнению, состоять из примерно следующей структуры:

```
;2822: EB18           jmps       .00000283C
;2824: db 47,65,6E,75,69,6E,65,44,65,76,69,6C ; 'GenuineDevil'
;2830: db 0E,00,02,28,33,00,B4,00,00,00,40,01,00,00
;283C: 1E             push        ds
;283D: 06             push        es
 ...
```

  Обратите внимание на характерную строку после обходящего сигнатуру короткого jmp'а - у разработчиков все в порядке с юмором. Далее будет еще один анекдот на эту тему.
- Если прямое сравнение (при нем он умудряется вызвать исключение #5 - BOUND) не показало дефектов в драйвере, то код начинает расшифровывать полезные данные (код дальнейшей защиты), используя тело драйвера как ключ расшифровки. Если установить в критичный регион драйвера в этот момент точку останова (int 3) или что-то типа перехватывающего jmp/call/ret, то этот полезный код будет расшифрован неправильно и, поскольку впоследствии управление будет передано на него, выполнится с ошибкой, приводящей к краху системы или только приложения. Расшифровка реализована без видимых дефектов в режиме CBC (расшифровка каждого байта зависит от расшифровки предидущих), что мешает вносить исправляющие коды после команд перехвата, однако из существования патча от C-ля следует, что эту проблему можно решить атакой на шифр, но детального исследования я не проводил. Могу сказать только, что алгоритм вряд ли является чем-то стандартным типа MD5. Некоторые сомнения возникают по факту слабого анализа аргумента первой команды jmp short, но ...
- Все предидущее алгоритм выполняет и с обработчиком исключения UD: выясняет его точку входа, прямое сравнение и расшифровка телом драйвера своего кода.
- Если же сравнение прошло успешно, выполянется единственный вызов исключения UD с функцией 0B:

```
mov  eax,4000000Bh
lea  edi,[esp+4]
UD2
db DE,AD,BE,EF,BA,BE
```

- Обратите внимание на юмор разработчиков ;)
- Так вот, этот вызов может реализовать несколько функций хасп-ключа, указателем на входные данные является регистр edi.
- После вызова функции 0B опять следует блок из ~128 вызовов "обычных" api исключения UD, за которым код, размещенный на стеке, опять проверит искажение кода драйвера и вызовет функцию 0B.
- Всего таких блоков вызовов исключения UD, оканчивающихся вызовом с подфункцией 0B, будет 17 штук. В каждом из них будет проверен драйвер и будет выполнена попытка исполнить различные функции хасп-ключа.

Api-Функции исключения UD вызывает не только приложение, но и сам драйвер при исполнении некоторых запросов DeviceIoControl, однако эти вызовы используют только функции 6 и 4.

Таким образом, атака должна решить две основные задачи: успешный перехват исключения UD (про крайней мере, его функции 0B) и DeviceIoControl, т.е. тех каналов, которые непосредственно общаются с ключом, а следом успешная эмуляция данных, получаемых по этим каналам приложением (точнее будет сказать - эмуляция функций хасп-ключа).

Перехват исключения UD

Наиболее сложная часть атаки. В силу описанных выше причин невозможно делать постоянный перехват этого исключения ни модификацией idt (из нее защита извлекает адрес обработчика), ни врезкой перехватывающих кодов непосредственно в тело обработчика: контролируется каждый байт (я о опкоде 0CCh), или же необходимо преодолеть криптографию проверяющего кода.

В данном случае мною был реализован динамический перехват этого исключения, преследующий цель получать управление при вызове исключения, но отпускающий драйвер в тот момент, когда приложением выполянется проверка драйвера.

Напомню, приложение вызвает исключение UD "блоками" по ~128 штук, причем последний вызов в блоке предваряется проверкой перехвата. Следовательно, при вызове первых ~127-ми исключений можно выполнять перехват и спокойно эмулировать все функции (хотя это будут вызовы функций 1-7, необязательные к эмуляции). Однако далее необходимо будет снять перехват и взвести ожидание какого-то события, после которого можно будет восстановить перехват, но еще не вызвалось исключение UD с последней в текущем блоке подфункцией 0B (являющейся обязательной к эмуляции).

Что может быть выбрано в качестве такого события ? Замечаем, что код на стеке, выполняющий сравнение, расшифровывает некий декриптор, еще лежащий в "обычной" области - сегменте кода приложения. Определенно до его работы код проверки еще не запущен (как, впрочем, и сразу после выполнения) и не успел сделать "черной работы". Это первая реперная точка.

Далее управление перейдет к коду на стеке, который выполнит проверку целостности драйвера. Но вместе с этим кодом также раскриптован и код вызова исключения UD с подфункцией 0B. При помощи Soft-Ice убеждаем себя в том, что после выполнения декриптора кода на стек можно смело ставить bpx в точку вызова исключения. Это вторая реперная точка.

Следовательно, необходимо после работы декриптора врезать где-то рядом с кодом вызова исключения наш код, который восстановит перехват и либо самостоятельно выполнит действия, обязательные для функции 0B, либо продолжит выполнение вплоть до вызова исключения, но это нормально, так как оно будет вновь перехвачено.

Никаких проблем по врезке такого кода после декриптора нет, это достаточно простой цикл, густо сдобренный командами, характерными для полиморфных вирусов:

```
@@Loop:
  ...
  mov  dl,[ebp] ; ebp указвает на зашифрованные байты
  mov  al,al
  sub  ebp,-1
  cld
  mov  bl,bl
  not  dl
  mov  ah,ah
  ...
  sub  dl,cl
  ...
  xor  ecx,8DFE10ECh
  ...
  ror  ecx,5Eh
  ...
  add  eax,-1
  ...
  push ebp
  pop  ebp
  mov  [eax],dl ; eax указывает на стек, куда расшифровываем
  xchg ah,ah
  dec  ebx
  jnz  @@Loop
```

Выкидываем все "нужные" команды, освобождая тем самым место для необходимого нам вызова. Вызов необходимо сделать либо в наш драйвер-перехватчик, либо в дополнительный код в виде win-32 приложения. Как наиболее простой вариант я выбрал первый и сделал это следующим образом: в этот момент еще активен перехват UD и я добавил к списку его api еще одну с кодом eax=40666666h. После декриптора вызовется наш обработчик исключения UD, уберет временно себя из цепочки обработчиков UD и установит вызов подпрограммы-восстановителя прямо перед

критичным вызовом исключения UD с функцией 0B.

Остается решить последний вопрос: когда можно будет слегда изменить этот декриптор ? Анализ показал, что на предпоследнем вызове исключения UD это делать еще рано: кода декриптора просто нет в памяти, мы не можем связать момент его появления с вызовами DeviceIoControl (их не будет, пока драйвер не будет проверен). Однако замечаем, что работа декриптора связана с вызовами исключения Page Fault:

```
@@Loop:
  ...
  mov  dl,[ebp] ; <<< эта команда может вызвать Page Fault !
  mov  al,al
  sub  ebp,-1
```

Мы можем перехватить также и PageFault и на нем ожидать появление декриптора. Это произойдет не при первом вызове, но при первой дешифровке это будет обязательно. Напоследок небольшая задача: откуда взять адрес кода, выполянющего вызов исключения с функцией 0B ? (тот самый, что с Dead beef) Допустим, мы его можем сканировать в памяти и легко найти, но вот в какой памяти ? Конечно, адреса в win довольно-таки одинаковы, и можно было бы взять за границу (верхнюю) что-то типа 00690000h, но как-то криво. Однако. Где лежит SEH в момент работы расшифровщика ? Да там же, в стеке - рядом с "Dead beef". Мы смело можем извлечь базу для сканирования из FS:[0] в момент отработки уже нашего дешифровщика. Перехват же Page Fault нужен только до того момента, покуда не найден декриптор: его необходимо будет восстановить - его перехват также будет обнаружен контролирующим драйвер кодом.

В общем и целом алгоритм перехвата UD выглядит следующим образом:

```
...
Перехват 125-го вызова UD;
         |
Перехват 126-го вызова UD;
         |
Перехват 127-го вызова UD;
         |
  Перехват PageFault, ожидаем появление декриптора кода на стек;
  ...
  Перехват PageFault, ожидаем появление декриптора кода на стек;
         |
    Если декриптор появился, меняем его:
    (не забыв восстановить перехват Page Fault)

    @@NewStackDecryptor2:
      mov  dl,[ebp]
      sub  ebp,-1
      cld
      not  dl
      sub  dl,cl
      xor  ecx,8DFE10ECh
      ror  ecx,5Eh
      dec  eax
      mov  [eax],dl
      dec  ebx
      jnz  @@NewStackDecryptor2
    ; Вызов обработчика #6
      push eax
      mov  eax,SelfCallUD6 ; 40666666h
----- db   0FEh,0EFh,00h,01h,02h,03h,04h,05h
|     pop  eax
|   @@EndNewStackDecryptor2:
|
Перехват 128-го вызова UD, но уже "наш", с функцией 40666666h;
  Убираем внутри перехватчика UD перехват UD;
  Устанавливаем в код вызова исключения UD вызов своего кода:

--- mov  eax,4000000Bh << push Our_Hook_Proc, ret
|   lea  edi,[esp+4]
```

```
|   UD2 <----------------------------
|                                   |
... Здесь выполняется проверка целостности драйвера
                                    |
|                                   |
Our_Hook_Proc                       |
  Восстанавливаем перехватчик UD;   |
  Передаем управление на команду UD --
```

После того, как будет успешно пройден первый блок и вызов UD с функцией 0B будет перехвачен, последует следующий блок вызовов UD. Но в дальнейшем нет нужды перехватывать PageFault и ждать появления декриптора: его больше никто не будет переписывать, его будут только вызвать ! А, раз мы его уже дополнили своим вызовом, то этот вызов будет сделан и в следующих блоках. Нужно будет лишь опять устанавливать перехват вызова UD и только.

Перехват DeviceIoControl Не вызывает никаких проблем. Может быть реализован (на уровне VxD) несколькими способами: перехват Directed_Sys_Control (вызов конкретного драйвера из системной машины с сообщением DeviceIoControl) или же врезкой jmp (даже jz в моем варианте) в тело control-процедуры. Возможно, есть и другие варианты, но в любом случае вызваемый код никак не контролируется на предмет перехвата. **Эмуляция функций хасп-ключа**

Остается рассмотреть протокол обмена данными приложением через функцию 0B исключения UD и DeviceIoControl. В обоих случаях фактически при каждом вызове выполняется та или иная функция хасп-ключа.

Какие же функции ключа используются ? Еще в статье exefoliator'a упоминаются около 32h функций. Тем не менее используются следующие:

```
Исключение UD, функция 0B выполяет функции 02,03,06 и 32h;
Вызовы DeviceIoControl реализуют вызовы функций 03 и 05.
```

Что из себя представляют эти функции ? В общем случае, конечно, они все что-то возвращают в зависимости от возможно необязательных аргументов. Например, функция 02: получить 64-ти битный seed по входному 16-ти битному аргументу. (Конечно, для получения верных результатов необходимо предъявить хасп-ключу уникальные ключи владельца данной линии hasp)

В статье exefoliator'a можно было найти описание эмулятора функции 02:

```
4. Общий вид эмулятора Hasp ключа

  80 FF 01       cmp         bh,1                    ; вызвали ISHASP?
  75 05          jne         @CheckTwo               ; если нет
  31 C0          xor         eax,eax                 ; иначе
  40             inc         eax                     ; ...возвращаем в eax = 1...
  EB 67          jmp         @EndHProc               ; и выходим
@CheckTwo:
  80 FF 02       cmp         bh,2                    ; вызвали HASPCODE?
  74 0A          je          @MakeCode               ; если да
  EB 60          jmp         @EndHProc               ; иначе ничего не делать
@HASPData:
  db x0 x1 x2 x3 x4 x5 x6 x7                         ; статические данные для HASP
@MakeCode:
  31 C9          xor         ecx,ecx                 ;
  31 DB          xor         ebx,ebx                 ;
  E8 00 00 00 00 call        @NextInst               ;
@NextInst:
  5E             pop         esi                     ;
@NextHBit:
  53             push        ebx                     ; сохранить число бай
  66 BB 89 19    mov         bx,1989h
  F7 E3          mul         eax,ebx
  83 C0 05       add         eax,5                   ; seed <- seed*1989h+5
  0F B7 C0       movzx       eax,ax                  ; seed <- seed and FFFFh
  50             push        eax                     ; сохранить seed
  C1 E8 09       shr         eax,9
  80 E0 3F       and         al,3Fh                  ; al <- bit offset
```

```
  89 C3          mov         ebx,eax
  C1 EB 03       shr         ebx,3                       ; ebx <- номер байта.
  80 E0 07       and         al,7                        ; al <- номер бита.
  51             push        ecx                         ; сохранить текущий код
  88 C1          mov         cl,al
  B0 01          mov         al,1
  D2 E0          shl         al,cl                       ; al <- битовая маска
  84 44 33 EF    test        byte ptr [ebx+esi-11h],al   ; бит установлен?
  B0 00          mov         al,0                        ; предположим что нет.
  74 01          je          @BitClear                   ; jump если ок
  40             inc         eax                         ; иначе выставляем бит
@BitClear:
  59             pop         ecx                         ; ecx <- строка кода
  D0 E5          shl         ch,1                        ; идем на следующий бит
  08 C5          or          ch,al                       ; добавить последний бит
  58             pop         eax                         ; eax <- seed
  5B             pop         ebx                         ; ebx <- число бит
  41             inc         ecx                         ; inc число бит
  80 F9 08       cmp         cl,8                        ; 8 битов?
  75 C7          jne         @NextHBit                   ; jump если нет
  F6 C3 01       test        bl,1                        ;
  74 03          je          @EvenByte                   ;
  5A             pop         edx                         ; получаем код возврата
  88 F1          mov         cl,dh                       ; добавляем верхние 8 бит
@EvenByte:
  51             push        ecx                         ; сохраняем код возврата
  31 C9          xor         ecx,ecx                     ; очищаем строку кода
  43             inc         ebx                         ; inc число байтов
  80 FB 08       cmp         bl,8                        ; 8байтов?
  75 B6          jne         @NextHBit                   ; jump если нет
  5A             pop         edx                         ; edx <- ret code 4
  59             pop         ecx                         ; ecx <- ret code 3
  5B             pop         ebx                         ; ebx <- ret code 2
  58             pop         eax                         ; eax <- ret code 1
@EndHProc:
  C3             ret                                     ; возврат к месту вызова.
```

Центральным моментом в этой эмулирующей функции являются 8 байт @HASPData (статические данные для HASP). Для чего они нужны ? Используются они один раз:

test byte ptr [ebx+esi-11h],al ; бит установлен?

Если аккуратно разобрать код этой эмулирующей функции, то можно понять, что @HASPData - это просто битовая таблица подстановки: в зависимости от запрошенного бита возвращает его состояние: в данном случае в регистре ebx и регистре al определено, состояние какого бита требуется. Все остальное это просто обвязка, которую, кстати, можно найти в защите непосредственно. Сравните вышеизложенное и код драйвера:

```
; Переменные в структуре по [ebp+xx]
  xor  si,si
  xor  cx,cx
Loop:
  mov  bx,1989h
  mov  ax,[ebp+275Dh] ; <<- entry word (16 bits)
  mul  bx
  add  ax,5
  mov  [ebp+275Dh],ax ; <<- update entry word
  mov  [ebp+276Dh],ah
  xor  ah,ah
  call x014C ; Вывод в порт 0378h байта из [ebp+276Dh]
  call x0115 ; Чтение порта 0379h (байт)
; Бит данных в AL
  shl  ch,1
  or   ch,al
  inc  cl
  cmp  cl,8
  jnz  Loop
  and  esi,0FFFFh
  mov  [ebp+esi+278Ch],ch
```

```
  xor  ch,ch
  inc  si
  cmp  si,8
  jnz  Loop
; В результате в [ebp+278Ch] содержится 8 байтов seed
```

Чтобы самостоятельно вычислить биты @HASPData, требуется: предъявить два 16-ти битных ключа хасп-ключу и прочесть вышеуказанным способом несколько 64-х битных seed'ов. В процессе получения этих seed'ов будут определены некоторые биты из @HASPData, эти биты следует запомнить и повторить процесс до тех пор, пока не будут определены все 64-е бита. Вот как можно предъявить ключи:

Да охранит нас □ от □! - прим. wasm.ru

```
{$G+}
const
  base = $0378;
  PortData = 0;
  PortRead = 1;
  PortServ = 2;

...

procedure OutPort(PortAddr:word; PortData: Byte);
  begin
    Port[base+PortAddr]:=PortData;
    if Debug = true then PrintByte(PortData,'Bout=');
  end;
function InPort(PortAddr:word):Byte;
  begin
    InPort:=Port[base+PortAddr];
  end;
Procedure Send_Twice(PData: Byte);
  begin
    OutPort(PortData,(PData and $FE) or $80);
    OutPort(PortData,(PData or $01) or $80);
    OutPort(PortData,(PData and $FE) or $80);
  end;
Procedure Send_Data(var PData: array of byte);
  var
    i: word;
  begin
    i:=0;
    while PData[i] <> $01 do
      begin
        Send_Twice(PData[i]);
        inc(i);
      end;
  end;
Function Read_DataBit: byte;
  begin
    Read_DataBit:= ( (InPort(PortRead) shr 5) and 1 );
  end;
Procedure GetKeyExtension(Key1,Key2: word; var KeyExt: array of byte);
  const
    TranslateTable: array[0..14] of byte =
      ($1E,$2E,$4E,$36, $56,$66,$3A,$5A, $6A,$72,$3C,$5C, $6C,$74,$78);
    TransTable26EA: array[0..14] of byte =
      ($0A,$78,$5A,$3A, $14,$48,$28,$00, $12,$50,$30,$0C, $1E,$5C,$3C);
  var
    TempKey: word;
    i,j: word;
    ResByte,TempByte: byte;
  begin
    FillChar(KeyExt,15,$FF);
    TempKey:=Key1 xor $1989;
    i:=0;
    j:=$0F;
    while i<8 do
      begin
```

```
        KeyExt[i]:=TempKey and $0F;
        if j<>$0F then KeyExt[j]:=(KeyExt[i]+$07) and $0F;
        dec(j);
        TempKey:=TempKey shr 4;
        inc(i);
        if i=4 then TempKey:=Key2 xor $0108;
      end;
    for i:=0 to 14 do
      begin
        if KeyExt[i]=0 then KeyExt[i]:=$0F;
        if i=$0D then KeyExt[i]:=$0B;
        KeyExt[i]:=TranslateTable[KeyExt[i]-1];
        ResByte:=KeyExt[i] xor $7E;
        if i<>0 then
          begin
            TempByte:=ResByte;
            ResByte:=(ResByte-2) and TempByte;
            if i<7 then ResByte:=ResByte xor TempByte;
          end;
        KeyExt[i]:=ResByte xor TransTable26EA[i];
      end;
  end;
const
  Table27AA: array[0..7] of word =
    ($0180,$0140,$01C0,$0130, $0100,$0120,$0110,$0000);
  Key1: word = $576D;
  Key2: word = $2EAB;
var
  Index27BF,Index27D1: word;
  HaspData: array[1..8,1..8] of byte;
  Res: array[0..7] of byte;
  ResByte,TempByte,ResBit,BitNum,LastOutput,OutBit: byte;
  DebStr: string;
  i,j,ByteNum,seed3,GetSeed3Byte: word;
  TestSeed,StartTestSeed: word;
  AllBitsDone: boolean;
  TestKeyExt: array[0..15] of byte;
label FindBits;
begin
  writeln;
  writeln('Start !');

  writeln('TEST for SEED:');

  OutPort(PortData,$80);
  Send_Twice($C6);              { Выполнить сброс хасп-ключа }
  OutPort(PortData,$80);

  GetKeyExtension(Key1,Key2,TestKeyExt); { Получить расширение ключей }

  TestKeyExt[15]:=$01; { End of data }
  Send_Data(TestKeyExt);          { Предъявить ключи хасп-ключу }
```

Предъявив ключи, можно читать корректные значения seed. Далее в цикле получается ряд значений seed и это делается до тех пор, пока все необходимые 64-е бита не будут определены:

```
  for i:=1 to 8 do
    begin
      for j:=1 to 8 do HaspData[i,j]:=0;
    end;

  StartTestSeed:=$6507;

FindBits:

  TestSeed:=StartTestSeed;

  for i:=0 to 7 do
    begin
      for j:=0 to 7 do
        begin
```

```
          asm
            mov  ax,TestSeed
            mov  bx,1989h
            mul  bx
            add  ax,5
            mov  TestSeed,ax
          end;
          OutPort(PortData,((TestSeed shr 8) and $FE) or $80);
          ResByte:=ResByte shl 1;
          ResBit:=Read_DataBit;
          asm
            mov  ax,TestSeed
            shr  ax,9
            and  al,3Fh
            mov  bx,ax
            shr  bx,3
            mov  ByteNum,bx
            and  al,7
            mov  BitNum,al
            mov  cl,al
            mov  al,1
            shl  al,cl
          end;
          {writeln('ByteNum: ',ByteNum,' BitNum:',BitNum,' Bit=',ResBit);}
          if ResBit = 0 then HaspData[ByteNum+1,BitNum+1]:=1
            else HaspData[ByteNum+1,BitNum+1]:=2;
          ResByte:=ResByte or ResBit;
        end;
      Res[i]:=ResByte;
    end;

  BytesToHexWithFiller(Res,DebStr,8);
  writeln('SEED  ?:',DebStr);

  AllBitsDone:=true;
  writeln('HASPData:');
  for i:=1 to 8 do
    begin
      for j:=8 downto 1 do
        begin
          if HaspData[i,j] = 0 then
            begin
              write(' ?');
              AllBitsDone:=false;
            end;
          if HaspData[i,j] = 1 then write(' 0');
          if HaspData[i,j] = 2 then write(' 1');
        end;
        writeln;
    end;

  inc(StartTestSeed);
  readln;
  if AllBitsDone=false then goto FindBits;

  writeln('Finish ;)');
end.
```

Итак, данные @HASPData для эмулирующей функции получены и с этим мы получаем возможность эмуляции функции 02 хасп-ключа.

Эмуляция остальных функций (03,05,06 и 32) намного проще, чем чем то, что было только что. Функция 03 читает заданное 16-ти битное слово из памяти хасп-ключа, функция 05 возвращает константу 66h - я не уверен, что это как-то зависит от содержимого ключа, функция 06 возвращает (также из памяти ключа) что-то вроде двух ID-шников ключа, функция 32h также читает память ключа. Память ключа можно получить, воспользовавшись утилитой haspgrab или же самостоятельно использовать функции 03,06 и 32 для прочтения всей памяти ключа.

Небольшое заключение

Интересная защита для тренировки собственных навыков, достаточно простой интерфейс работы с ключом не утомляет исследователя (например по сравнению с хасп-ключом на usb).

Спасибо всем прочитавшим :)

Полный текст эмулирующего драйвера и приложения

```
;
; ***************Инклуд-файл*******************
;
MAX_LOG_RECORDS equ 100
OpCodesSize equ 4
tLogPacket struc
  RegEAX  dd ?        ; Регистр EAX
  RegEDI  dd ?        ; Регистр EDI
tLogPacket ends

DeviceLitt_ID equ 3CDCh ; ID of little VxD

NumberOfUDTraps equ 17 ; Число всех вызовов исключения UD с функцией 0Bh

SelfCallUD6 equ 40666666h ; Номер собственной функции для UD6
;
; *****************Программа - эмулятор хасп-ключа (драйвер)**********
; Coded by Chingachguk. 2004.
;
.586p
include vmm.inc
include vwin32.inc
DECLARE_VIRTUAL_DEVICE UD06,1,0, UD06_Control,\
     UNDEFINED_DEVICE_ID, UNDEFINED_INIT_ORDER
Begin_control_dispatch UD06
 Control_Dispatch w32_DeviceIoControl, OnDeviceIoControl
        Control_Dispatch DESTROY_PROCESS, OnDESTROY_PROCESS
End_control_dispatch UD06
; Сегмент данных VxD
include log06.inc
VxD_LOCKED_DATA_SEG
; Буфер лога
LogPtr               dd 0 ; Указатель лог-буфера
LogPackets           db (MAX_LOG_RECORDS * sizeof(tLogPacket)) dup (?)

old_ud06handler      dd 0 ; Адрес обработчика UD в VxD
old_14handler        dd ? ; Адрес обработчика PageFault в VxD

DDB                  dd ? ; Адрес Device Block VxD

; Переменные для перехвата Control_Proc VxD
F23Addr              dd 0 ; Адрес начала проверки cmp  eax,23h
F23JmpFound          dd 0
F23EntryPoint        dd ?
ContinuePoint        dd ?
RestoreJmpArg        dd ? ; Аргумент jmp - saved

DirectedSysLocked              dd 0 ; Флаг перехвата функций VxD
pPrevHook_Directed_Sys_Control dd ?

UD14EntryPoint       dd 0 ; Точка входа в #14
UD6EntryPoint        dd 0 ; Точка входа в #UD6

Hook14Count          dd 0 ; Число вызовов исключения Page Fault
HookUDCount          dd 0 ; Число вызовов UD6

SaveSEHProc          dd ? ; Адрес начального обработчика SEH

StepNum              dd ? ; Номер блока шагов по 126

SignalUnhookPortTrap dd 0 ; Флаг прекращения мониторинга i/0
CounterIOtraps       dd 0 ; Счетчик обращений к портам i/0
CounterCalls14Step2  dd 0 ; Отладка - счетчик вызовов #14 на шаге 2
```

```
CrcProcHookAddr      dd ? ; Адрес точки перехвата crc-процедуры
AfterCrcProcHookAddr dd ? ; Адрес возврата после перехвата crc-процедуры

VariantVesrion       dd ? ; Вариант версии: в зависимости от него разный код
StartStackDecryptor  dd 0 ; Адрес начала декриптора кода на стек
; "Сигнатуры" расшифровщиков кода на стеке (2 варианта)
; Начало кода нового расшифровщика, который мы впишем вместо прежнего
NewDecryptorOfs      dd offset32 @@NewStackDecryptor1
                     dd offset32 @@NewStackDecryptor2
; Размер расшифровщика
NewDecryptorSize     dd @@EndNewStackDecryptor1-@@NewStackDecryptor1
                     dd @@EndNewStackDecryptor2-@@NewStackDecryptor2
; Байты - признаки завершения старого расшифровщика
NewDecryptorEndBytes dw 0F68Ah
                     dw 0CA75h

; **************Данные эмулятора хасп-ключа**********************

HaspKey1 equ 576Dh ; Ключи:
HaspKey2 equ 2EABh ; 2 word

; Память хасп-ключа:

tHaspMemory struc
  Key1       dw ?         ; Ключ 1
  Key2       dw ?         ; Ключ 2
  MemSize    dw 0080h     ; Размер памяти ?
  KeyID1     dw ?         ; Идентификатор ключа - первая часть ?
  KeyID2     dw ?         ; Идентификатор ключа - вторая часть ?
  BytesFF1   db 12 dup(0FFh)
  Func3Data  dw 24 dup(?) ; Данные для функции 3h
  Null1      dw 0         ; 0
  BytesFF2   db 30 dup(0FFh)
  Func32Data dw 16 dup(?) ; Данные для функции 32h
tHaspMemory ends

DumpHaspMemory label byte
db 06Dh,057h,0ABh,02Eh,080h,000h,050h,0E4h,07Ch,054h,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,000h,000h,000h,000h,04Dh,000h,064h,000h,000h,000h
db 001h,000h,007h,000h,012h,000h,01Fh,000h,030h,000h,044h,000h,05Bh,000h,075h,000h
db 092h,000h,0B3h,000h,0D6h,000h,0FDh,000h,000h,000h,020h,000h,05Fh,000h,002h,000h
db 007h,000h,032h,000h,03Ch,000h,000h,000h,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0F4h,0ACh,039h,0CEh,08Dh,069h,086h,0EFh,0CAh,088h
db 0ABh,009h,003h,075h,0D8h,089h,032h,040h,09Eh,09Bh,04Ch,0DEh,056h,064h,03Ah,027h
db 030h,066h,00Bh,01Eh,021h,017h

; Таблица для получения seed (используется в функции 2)
HaspData db 00011000b ; 18h
         db 00011010b ; 1Ah
         db 01001001b ; 49h
         db 01001011b ; 4Bh
         db 10011000b ; 98h
         db 10011010b ; 9Ah
         db 11001001b ; C9h
         db 11001011b ; CBh

VxD_LOCKED_DATA_ENDS
; А вот наш сегмент кода.
VxD_LOCKED_CODE_SEG
;
; Сообщение DESTROY_PROCESS: деинсталлируем все перехваты
;
BeginProc OnDESTROY_PROCESS
; Инсталляция ловушек уже была ?
  cmp  dword ptr ds:DirectedSysLocked,0
  jz   @@ContinueHook

; Убираем перехват.

; Деинсталляция перехватчиков UD6 и 23h
```

```
  call Uninstall_Handlers

; Мы опять готовы приступить к перехвату.
  mov  dword ptr ds:DirectedSysLocked,0

@@ContinueHook:

  xor  eax,eax
  clc
  ret
EndProc   OnDESTROY_PROCESS
;
; Сообщение DeviceIoControl
;
BeginProc OnDeviceIoControl
; Сделаем так, чтобы esi был адресом переданной нам структуры - DIOCParams.
  assume esi:ptr DIOCParams
; При загрузке VxD в память система позовет ее с контрольными сообщениям
.if [esi].dwIoControlCode==DIOC_Open ; Контрольное сообщение ?!

;
; Установка перехватчика Directed_Sys_Control:
;
; Он ожидает сообщения от VMM для Hasp.vxd и перехватывает обработчик #06
;
; Перехватываем функцию VMM "Directed_Sys_Control"
; ***
; INT 20 P - Microsoft Windows - 0147h "Directed_Sys_Control"
; VxD = 0001h
; 0147h "Directed_Sys_Control"
  mov  eax,00010147h
  mov  esi,offset32 Hooked_Directed_Sys_Control
  int  20h   ; Call VxD
  dw   0090h  ; 0090h hook device service
  dw   0001h   ; ID VMM
  mov  eax,-1
  jnc  @@HookDirected_Sys_ControlOK
  xor  eax,eax
@@HookDirected_Sys_ControlOK:

  xor  eax,eax ; Надо отвечать: eax=0.
;
; Сигнал завершения работы
;
.elseif [esi].dwIoControlCode==DIOC_Closehandle

; ***
; Освобождаем "Directed_Sys_Control"
; ***
  mov  eax,00010147h
  mov  esi,offset32 Hooked_Directed_Sys_Control
  int  20h   ; Call VxD
  dw   011Ch  ; 011Ch unhook device service
  dw   0001h   ; ID VMM

;
; Запрос на выдачу буфера лога
;
.elseif [esi].dwIoControlCode==1

; Получим адрес переданной нам структуры InBuffer в регистр ebx
  mov  ebx,[esi].lpvInBuffer
  mov  edi,[ebx] ; указатель на буфер, который нам передал win32-код

  mov  esi,offset32 LogPtr
  cld
  lodsd
  stosd
  imul eax,eax,sizeof(tLogPacket)
  mov  ecx,eax
  test eax,eax
```

```
  jz   @@NoLogData
  rep  movsb

@@NoLogData:

  xor  eax,eax ; Вернем в eax флаг ошибки
.endif
  ret
EndProc OnDeviceIoControl
;
; *************************SUBPROGRAMS**********************************
;
;
; подпрограмма - перехватчик функции 23h - DeviceIOControl:
; ожидает сообщения от VMM для hasp.vxd и перехватывает ud6 и F23
;
BeginProc Hooked_Directed_Sys_Control, HOOK_PROC, pPrevHook_Directed_Sys_Control, LOCKED
  push ebp
  mov  ebp,esp
  pushfd                  ; Remember, hooks must preserve all regs
  pushad                  ;

; Нужно ли мониторить этот вызов ?
  cmp  word ptr [ecx+6],DeviceLitt_ID
  jnz  @@ItNotIsOurVxD

; Инсталляция обработчиков ud06 и F23 (если возможно)
  cmp  dword ptr ds:DirectedSysLocked,0
  jz   @@HookFunctions

  jmp  @@AlreadyHooked

@@HookFunctions:

  mov  dword ptr ds:DirectedSysLocked,1

;
; Инсталлируем перехватчик #6:
;
;
;9B4D: jmp  9B63 EB 14
;      db 'GenuineDevil' = db 47,65,6E,75,69,6E,65,44,65,76,69,6C
;      db 06,00,01,03,48,24,00,00
;9B63: 1E              push        ds
;9B64: 06              push        es
;9B65: 50              push        eax [ebp+10h]
;9B66: 51              push        ecx [ebp+0Ch]
;9B67: 52              push        edx [ebp+8]
;9B68: 57              push        edi [ebp+4]
;9B69: 55              push        ebp [ebp+0]
;9B6A: 8BEC            mov         ebp,esp
;9B6C: 668CD0          mov         ax,ss
;9B6F: 668ED8          mov         ds,ax
;9B72: 668EC0          mov         es,ax
;9B75: 55              push        ebp
;9B76: 9C              pushfd
;9B77: E800000000      call        000009B7C
;9B7C: 5D              pop         ebp
;9B7D: 81ED780E0000    sub         ebp,000000E78
;9B83: 9D              popfd
;9B84: 0F014D00        sidt        [ebp][00]

; Второй вариант, начало #6
;29AE: EB18            jmps       .0000029C8
;29B0: db 47,65,6E,75,69,6E,65,44,65,76,69,6C
;29BC: 06,00,02,00,94,23,00,00,00,24,00,00
;29C8: 1E              push        ds << jmp Our_Trap_Program
;29C9: 06              push        es
;29CA: 50              push        eax
;29CB: 51              push        ecx
;29CC: 52              push        edx
```

```
;29CD: 57             push        edi
;29CE: 55             push        ebp
;29CF: 8BEC           mov         ebp,esp
;29D1: 668CD0         mov         ax,ss
;29D4: 668ED8         mov         ds,ax
;29D7: 668EC0         mov         es,ax
;29DA: 55             push        ebp
;29DB: 9C             pushfd
;29DC: E800000000     call       .0000029E1
;29E1: 5D             pop         ebp
;29E2: 81EDBD0E0000   sub         ebp,000000EBD
;29E8: 9D             popfd
;29E9: 0F014D00       sidt        [ebp][00]

; Получить адрес control'a vxd
  call GetLitt_DDB
  mov  eax,dword ptr ds:DDB
  test eax,eax
  jz   @@VxDNotFound
  mov  eax,dword ptr [eax+18h]
; Ищем байты начала #06. Это вторая строка 'GenuineDevil' (первая - #14)
  mov  ecx,0FFFFh
  xor  ebx,ebx
@@FindUDEntryPoint:
  cmp  dword ptr ds:[eax],0756E6547h
  jnz  @@NextByteScan
  cmp  dword ptr ds:[eax+4],044656E69h
  jnz  @@NextByteScan
  test ebx,ebx
  jz   @@UD14EntryPointFound
; Найден обработчик UD - int 6
  lea  edx,[eax-2]
  mov  dword ptr ds:UD6EntryPoint,edx
  jmp  @@ScanForUdDone
; Найден обработчик #14 - PageFault
@@UD14EntryPointFound:
  inc  ebx
  lea  edx,[eax-2]
  mov  dword ptr ds:UD14EntryPoint,edx
@@NextByteScan:
  inc  eax
  loop @@FindUDEntryPoint
  jmp  @@NotSetup

; Найдены адреса входа в оба обработчика
@@ScanForUdDone:

; Вписать команду перехода в тело обработчика #6
  call Install_Jmp_Into_UD6

; Найти команду перехода на 23 в VxD-обработчике (control proc)
; Это две команды:
; cmp  eax,23h ; 83 F8 23
; jz   @L      ; 0F 84 66 FF FF FF

  mov  eax,dword ptr ds:DDB
  mov  eax,dword ptr [eax+18h] ; Control proc

; Найти команды перехода на инсталлятор
  mov  ecx,0FFFFh
@@FindJmp23:
  cmp  dword ptr [eax],0F23F883h
  jnz  @@PrevByte
  cmp  dword ptr [eax+4],0FFFF6684h
  jnz  @@PrevByte
; Дезактивировать старые команды перехода,
; запомнив их адрес
  mov  dword ptr F23Addr,eax
  mov  dword ptr F23JmpFound,1
  jmp  @@InstallerFound
@@PrevByte:
```

```
  inc  eax
  loop @@FindJmp23
  jmp  @@NotSetup
@@InstallerFound:

; Установить перехватчик функции 23h
; Запомнить EntryPoint функции 23h
  mov  ebx,dword ptr F23Addr
  add  ebx,3 ; Адрес jz @@F23
  lea  eax,[ebx+6] ; Адрес следующей команды после jz @@F23
; Запомнить ее
  mov  ContinuePoint,eax
  add  eax,[ebx+2] ; Входная точка F23
  mov  F23EntryPoint,eax
; Вписать в команду jz @F23 адрес нашего обработчика F23
  lea  eax,[ebx+6] ; Адрес следующей команды после jz @@F23
  sub  eax,offset32 HookControl
  neg  eax ; Размер перехода
; Сохранить ее для восстановления
  mov  edx,dword ptr [ebx+2]
  mov  dword ptr [ebx+2],eax ; Переход на Trap
  mov  dword ptr RestoreJmpArg,edx

; Счетчик вызовов int 6
  mov  dword ptr ds:HookUDCount,0

@@VxDNotFound:
@@NotSetup:

@@AlreadyHooked:
@@ItNotIsOurVxD:
; Продолжение выполнения
  popad
  popfd
  pop  ebp
  jmp  [pPrevHook_Directed_Sys_Control]  ; Chain to previous hook
EndProc Hooked_Directed_Sys_Control
;
; Деинсталляция перехватчиков UD6 и 23h
;
Uninstall_Handlers proc
  pushad
  cmp  dword ptr F23JmpFound,0
  jz   @@WasNotInstalled

; Восстановить команду перехода на функцию 23h
; cmp  eax,23h ; 83 F8 23
; jz   @L      ; 0F 84 66 FF FF FF
; Получение адреса обработчика
  mov  ebx,dword ptr F23Addr
  add  ebx,3
  mov  eax,dword ptr RestoreJmpArg
  mov  [ebx+2],eax

@@WasNotInstalled:

; Восстановить команды в VxD
  call Uninstall_Jmp_Into_UD6

  popad
  ret
Uninstall_Handlers endp
;
; подпрограмма - перехватчик #6 - UD
;
TrapPort proc

; Команды, которые не успел выполнить обработчик #6
;29C8: 1E             push        ds << jmp Our_Trap_Program
;29C9: 06             push        es
;29CA: 50             push        eax
```

```
;29CB: 51             push        ecx
;29CC: 52             push        edx

EntryEAX equ dword ptr [ebp-04h]
EntryEBX equ dword ptr [ebp-10h]
EntryECX equ dword ptr [ebp-08h]
EntryEDX equ dword ptr [ebp-0Ch]
EntryEDI equ dword ptr [ebp-20h]
CrashAdr equ dword ptr [ebp+18h]

  ; EIP        ; [ebp+18h]

  push ds      ; [ebp+14h]
  push es      ; [ebp+10h]
  push eax     ; [ebp+0Ch]
  push ecx     ; [ebp+8h]
  push edx     ; [ebp+4h]

  push ebp     ; [ebp+0h]
  mov  ebp,esp

  pushad

; push eax     ; [ebp-04h]
; push ecx     ; [ebp-08h]
; push edx     ; [ebp-0Ch]
; push ebx     ; [ebp-10h]
; push esp     ; [ebp-14h]
; push ebp     ; [ebp-18h]
; push esi     ; [ebp-1Ch]
; push edi     ; [ebp-20h]

  push ds
  push ss
  pop  ds

; Адрес возврата EIP
  mov  ecx,CrashAdr

; Вызов, сигнализирующий об окончании расшифровки кода на стеке ?

  cmp  EntryEAX,SelfCallUD6
  jnz  @@NotSelfCallUD6
  pushad
; Restore old #06 entry point
  call Uninstall_Jmp_Into_UD6
; Установить, если найдем, перехват crc-процедуры
  call Install_CrcHook
  popad
  jmp  @@ReturnToFault

@@NotSelfCallUD6:

; Запомнить байты инструкции (опционально)
  call Store_LogRecord

  cmp  EntryEAX,40000007h
  jnz  @@NotF7

; Эмуляция первого и не только вызова функции 07
; В первом приближении это подсчет crc из [edx] длиной ecx и запись в [edi]
  pushad
  xor  eax,eax
  mov  edi,EntryEDX
  mov  ecx,EntryECX
@@CalcCRC:
  add  al,ds:[edi]
  inc  edi
  dec  ecx
  jnz  @@CalcCRC
  mov  edi,EntryEDI
```

```
  mov  ds:[edi],al
  popad
; Возврат в код приложения
  jmp  @@ReturnToFault

@@NotF7:

  cmp  EntryEAX,40000006h
  jnz  @@NotF6
; Эмуляция первого и не только вызова функции 06
; В первом приближении это расшифровка из [edx] длиной ecx xor AAh + ror 4

  pushad
  xor  eax,eax
  mov  edi,EntryEDX
  mov  ecx,EntryECX
@@DecryptF06:
  xor  byte ptr ds:[ecx+edi-1],0AAh
  ror  byte ptr ds:[ecx+edi-1],4
  dec  ecx
  jnz  @@DecryptF06
  popad
; Возврат в код приложения, но на адрес из edi
; Модификация указателя
  mov  eax,EntryEDI
  mov  CrashAdr,eax
  jmp  @@SimpleReturnToFault

@@NotF6:

  cmp  EntryEAX,40000003h
  jnz  @@NotF3
; Эмуляция первого и не только вызова функции 03
; В первом приближении это расшифровка из [edx] длиной ecx rol 4 + xor AAh

  pushad
  xor  eax,eax
  mov  edi,EntryEDX
  mov  ecx,EntryECX
@@DecryptF03:
  rol  byte ptr ds:[edi],4
  xor  byte ptr ds:[edi],0AAh
  inc  edi
  dec  ecx
  jnz  @@DecryptF03
  popad

; Возврат в код приложения
  jmp  @@ReturnToFault

@@NotF3:

  cmp  EntryEAX,40000001h
  jnz  @@NotF1
; Эмуляция первого и не только вызова функции 01
; В первом приближении это просто возврат + 8
; Во втором - чтение из dr7 700h, запись в dr7 0, но это не обязательно
; Возврат в код приложения
  jmp  @@ReturnToFault

@@NotF1:

  cmp  EntryEAX,40000004h
  jnz  @@NotF4
; Эмуляция первого и не только вызова функции 04
; В первом приближении это расшифровка из [edx] длиной ecx rol 4 + xor AAh

  pushad
  xor  eax,eax
  mov  edi,EntryEDX
  mov  ecx,EntryECX
```

```
@@DecryptF04:
  rol  byte ptr ds:[edi],4
  xor  byte ptr ds:[edi],0AAh
  inc  edi
  dec  ecx
  jnz  @@DecryptF04
  popad
; Возврат в код приложения, но на адрес из edi
; Модификация указателя
  mov  eax,EntryEDI
  mov  CrashAdr,eax
  jmp  @@SimpleReturnToFault

@@NotF4:

  cmp  EntryEAX,40000002h
  jnz  @@NotF2
; Эмуляция первого и не только вызова функции 01
; В первом приближении это просто возврат + 8
; Во втором - запись в dr7 700h, но это не обязательно
; Возврат в код приложения
  jmp  @@ReturnToFault

@@NotF2:

  cmp  EntryEAX,4000000Bh
  jnz  @@NotFB
; Эмуляция первого и не только вызова функции 0B
; В первом приближении это просто возврат + 8
; Во втором - обнуление нескольких dword по edi (если неверные ключи)

  mov  eax,EntryEDI
; Флаг первой проверки в 0
  mov  dword ptr ds:[eax+20h],0
; Переданы верные значения Key's ?
  cmp  dword ptr ds:[eax+18h],HaspKey1
  jnz  @@NotFBGenSeed
  cmp  dword ptr ds:[eax+14h],HaspKey2
  jnz  @@NotFBGenSeed
  jmp  @@ExecHaspFunc

@@NotFBGenSeed:
; Ключи неверны. Обнулить 4 dword и все
  and  dword ptr ds:[eax+08h],0FFFFh
  and  dword ptr ds:[eax+0Ch],0FFFFh
  mov  dword ptr ds:[eax+10h],0
  mov  dword ptr ds:[eax+14h],0
  mov  dword ptr ds:[eax+18h],0
  mov  dword ptr ds:[eax+1Ch],0
; Возврат в код приложения
  jmp  @@ReturnToFault

@@ExecHaspFunc:
; Ключи верные. Проверяем номер функции. Если поддерживаем - выполняем.

; Функция 02: get 64 bit seed
  cmp  dword ptr ds:[eax+10h],0200h
  jnz  @@NotHaspFunc02
  pushad
  mov  edi,eax
; Сгенерировать Seed
  movzx eax,word ptr ds:[eax+1Ch]
  call HaspFunc_2
; example: enter: 6507h -> ax=7730 bx=8400 cx=19ce dx=7be8
  and  dword ptr ds:[edi+08h],0FFFFh
  and  dword ptr ds:[edi+0Ch],0FFFFh
  mov  dword ptr ds:[edi+10h],ebx
  mov  dword ptr ds:[edi+14h],edx
  mov  dword ptr ds:[edi+18h],ecx
  mov  dword ptr ds:[edi+1Ch],eax
  popad
```

```
; Возврат в код приложения
  jmp  @@ReturnToFault

@@NotHaspFunc02:

; Функция 03: get 16 bit seed
  cmp  dword ptr ds:[eax+10h],0300h
  jnz  @@NotHaspFunc03
  pushad
  mov  edi,eax
  and  dword ptr ds:[edi+08h],0FFFFh
  and  dword ptr ds:[edi+0Ch],0FFFFh
; seed
  mov  dword ptr ds:[edi+10h],0
  mov  eax,dword ptr ds:[edi+8]
  mov  dword ptr ds:[edi+1Ch],eax
  mov  dword ptr ds:[edi+18h],0FFFFFFFEh
  cmp  eax,37h
  ja   @@ErrEntrySeedForF3
  call HaspFunc_3
  mov  dword ptr ds:[edi+10h],eax
  mov  dword ptr ds:[edi+18h],0
@@ErrEntrySeedForF3:
  popad
; Возврат в код приложения
  jmp  @@ReturnToFault
@@NotHaspFunc03:

; Функция 06: get 2 word's ?
  cmp  dword ptr ds:[eax+10h],0600h
  jnz  @@NotHaspFunc06
  pushad
  mov  edi,eax
  call HaspFunc_6
  and  dword ptr ds:[edi+08h],0FFFFh
  mov  dword ptr ds:[edi+10h],eax
  mov  dword ptr ds:[edi+18h],0
  mov  dword ptr ds:[edi+1Ch],edx
  popad
; Возврат в код приложения
  jmp  @@ReturnToFault

@@NotHaspFunc06:

; Функция 32: get seed's like function 3
  cmp  dword ptr ds:[eax+10h],3200h
  jnz  @@NotHaspFunc32
  pushad
  mov  edi,eax
  mov  dword ptr ds:[edi+18h],0
; seed - проверки
; start seed
  mov  eax,dword ptr ds:[edi+8]
  mov  ecx,dword ptr ds:[edi+0Ch]
  add  eax,ecx
  cmp  eax,40h-08h
  ja   @@ErrEntrySeedForF32
  mov  esi,dword ptr ds:[edi+1Ch]
  mov  dword ptr ds:[edi+14h],esi
  mov  ebx,dword ptr ds:[edi+8]
  mov  dword ptr ds:[edi+10h],ecx
  mov  dword ptr ds:[edi+0Ch],ecx
  call HaspFunc_32
  mov  dword ptr ds:[edi+1Ch],ebx ; last seed
  jmp  @@SeedsForF32Done
@@ErrEntrySeedForF32:
  mov  dword ptr ds:[edi+10h],0
  mov  dword ptr ds:[edi+14h],0
  mov  dword ptr ds:[edi+1Ch],0
@@SeedsForF32Done:
  popad
```

```
; Возврат в код приложения
  jmp  @@ReturnToFault
@@NotHaspFunc32:

  jmp  @@ToPrevHandler

@@NotFB:
@@ToPrevHandler:

  pop  ds
  popad
  pop  ebp

  jmp  dword ptr ss:old_ud06handler
;
; Возврат в код, вызвавший исключение
;
@@ReturnToFault:

; Инкримент указателя EIP
  add  CrashAdr,8

@@SimpleReturnToFault:

  pop  ds
  popad
  pop  ebp

; Команды от оригинального обработчика
  pop  edx
  pop  ecx
  pop  eax
  pop  es
  pop  ds
; Флаг RF: resume flag после выполнения исключения - continue
  or   byte ptr ss:[esp+0Ah],1h
  iretd
TrapPort endp
;
; Запомнить байты в лог-буфере
; ecx-> EIP start
Store_LogRecord proc
; Счетчик вызовов int 6
  mov  eax,dword ptr ds:HookUDCount
  inc  dword ptr ds:HookUDCount

; (Номер шага+1) кратен 127 ?
  pushad
  add  eax,2
  xor  edx,edx
  mov  ebx,127
  div  ebx
  mov  dword ptr ds:StepNum,eax
  test edx,edx
  popad
  jz   @@UnhookUD6AndWaiting
  jmp  @@StepsDone

@@UnhookUD6AndWaiting:

  cmp  dword ptr ds:StepNum,2
  jae  @@HookToCrcProcDone

; Устанавливаем ловушку на #14, ожидающую кода-расшифровщика на стек
; после того, как он найдет код декриптора на стек,
; он запишет после него вызов (наш) #6
  call Install_Jmp_Into_14

@@HookToCrcProcDone:
@@StepsDone:
; Записываем в лог только пакеты от функции 0Bh (отладка, можно убрать)
```

```
  cmp  EntryEAX,4000000Bh
  jz   @@LogFuncB
  ret
@@LogFuncB:

; Запоминаем байты и значения регистров команды
  mov  eax,dword ptr ds:LogPtr
  cmp  eax,MAX_LOG_RECORDS
  jae  LogFull
  inc  dword ptr ds:LogPtr

  pushad
  imul eax,eax,sizeof(tLogPacket)
  mov  ebx,offset32 LogPackets
  add  ebx,eax
  assume ebx: ptr tLogPacket
; Регистр eax
  mov  eax,EntryEAX
  mov  dword ptr [ebx].RegEAX,eax
; Регистр edi
  mov  eax,EntryEDI
  mov  dword ptr [ebx].RegEDI,eax
  assume ebx:nothing
  popad

LogFull:
  ret
Store_LogRecord endp
;
; Перехват вызова F 23h VxD
;
HookControl proc
  assume esi:ptr DIOCParams
; Надо ли что-то делать ?
  cmp  dword ptr [esi].dwIoControlCode,2
  jnz  @@F23FuncDone
  cmp  dword ptr [esi].lpvInBuffer,0
  jz   @@F23FuncDone

; Если бы мы вызвали драйвер, он сгенерировал бы 2 исключения UD
  add  dword ptr ds:HookUDCount,2

  pushad
; lpvInBuffer
  mov  eax,[esi].lpvInBuffer
; Расшифровать буфер по способу F=06 #6
  mov  ecx,28h
  lea  esi,[eax+14h]
@@DecryptInBuffer:
  xor  byte ptr [esi],0AAh
  ror  byte ptr ds:[esi],4
  inc  esi
  loop @@DecryptInBuffer
; Проверить ключи. Если неверные - заполнить нулями и все
  cmp  word ptr ds:[eax+20h],HaspKey1
  jnz  @@BadKeysForF23
  cmp  word ptr ds:[eax+24h],HaspKey2
  jnz  @@BadKeysForF23
; Функция 3 ?
  cmp  byte ptr ds:[eax+16h],03
  jnz  @@NotFunc03
  mov  dword ptr ds:[eax+34h],8
  mov  dword ptr ds:[eax+30h],0
  and  dword ptr ds:[eax+28h],0FFFFh
; Получить seed / вернуть ошибку
  mov  esi,dword ptr ds:[eax+28h]
  mov  dword ptr ds:[eax+2Ch],0E450h
  cmp  esi,37h
  ja   @@FuncsDone
  xchg eax,esi
  call HaspFunc_3
```

```
  mov  dword ptr ds:[esi+2Ch],eax
  mov  eax,esi
  jmp  @@FuncsDone
@@NotFunc03:

; Функция 5 ?
  cmp  byte ptr ds:[eax+16h],05
  jnz  @@NotFunc05
; Заполнить константами
  mov  dword ptr [eax+28h],1
  mov  dword ptr [eax+2Ch],1
  push eax
  call HaspFunc_5
  mov  ecx,eax
  pop  eax
  mov  dword ptr [eax+30h],ecx
  mov  dword ptr [eax+34h],0
@@NotFunc05:

@@FuncsDone:
  jmp  @@KeysOKForF23

@@BadKeysForF23:
  mov  dword ptr [eax+28h],0
  mov  dword ptr [eax+2Ch],0
  mov  dword ptr [eax+30h],0
  mov  dword ptr [eax+34h],0
@@KeysOKForF23:
; Зашифровать буфер по способу F=04 #6
  mov  ecx,28h
  lea  esi,[eax+14h]
@@EncryptInBuffer:
  rol  byte ptr ds:[esi],4
  xor  byte ptr ds:[esi],0AAh
  inc  esi
  loop @@EncryptInBuffer
  popad

  assume esi:nothing

@@F23FuncDone:

  xor  eax,eax
  clc
  ret
HookControl endp
;
; Вписать jmp на наш обработчик #6
;
Install_Jmp_Into_UD6 proc
  pushad
  cli
; eax=команда короткого jmp на тело
  mov  eax,dword ptr ds:UD6EntryPoint
; вычислить начало обработчика
  movzx ebx,byte ptr ds:[eax+1]
  lea  eax,[ebx+eax+2]
; Вписать jmp
  mov  byte ptr ds:[eax],0E9h       ; jmp
  mov  ebx,offset32 TrapPort
  sub  ebx,eax
  sub  ebx,5
  mov  dword ptr ds:[eax+1],ebx    ; offset
  add  eax,5
  mov  dword ptr old_ud06handler,eax
  sti
  popad
  ret
Install_Jmp_Into_UD6 endp
;
; Убрать jmp на наш обработчик #6
```

```
;
Uninstall_Jmp_Into_UD6 proc
  pushad
  cli
  mov  eax,dword ptr old_ud06handler
  test eax,eax
  jz   @@HandlerUDWasNotFound
  sub  eax,5
;29C8: 1E             push        ds << jmp Our_Trap_Program
;29C9: 06             push        es
;29CA: 50             push        eax
;29CB: 51             push        ecx
;29CC: 52             push        edx
  mov  byte ptr ds:[eax],01Eh
  mov  dword ptr ds:[eax+1],52515006h
@@HandlerUDWasNotFound:
  sti
  popad
  ret
Uninstall_Jmp_Into_UD6 endp
Install_Jmp_Into_14 proc
;
; Вписать команду перехода на перехватчик #14
;
;9A02: EB14           jmps       .000009A18
;9A04: db 'GenuineDevil' = db 47,65,6E,75,69,6E,65,44,65,76,69,6C
;9A10: 0E,00,01,02,D8,00,00,00,
;9A18: 1E             push        ds  << jmp Hook_14
;9A19: 06             push        es
;9A1A: 16             push        ss
;9A1B: 1F             pop         ds
;9A1C: 50             push        eax
;9A1D: 51             push        ecx
;9A1E: 52             push        edx
;9A1F: 57             push        edi

; Второй вариант VxD, начало #14
;2822: EB18           jmps       .00000283C
;2824: db 47,65,6E,75,69,6E,65,44,65,76,69,6C
;2830: db 0E,00,02,28,33,00,B4,00,00,00,40,01,00,00
;283C: 1E             push        ds
;283D: 06             push        es
;283E: 16             push        ss
;283F: 1F             pop         ds
;2840: 50             push        eax
;2841: 51             push        ecx
;2842: 52             push        edx
;2843: 57             push        edi

  pushad
  mov  eax,dword ptr ds:UD14EntryPoint
; вычислить начало обработчика
  movzx ebx,byte ptr ds:[eax+1]
  lea  eax,[ebx+eax+2]
  mov  byte ptr ds:[eax],0E9h       ; jmp
  mov  ebx,offset32 Hook_14
  sub  ebx,eax
  sub  ebx,5
  mov  dword ptr ds:[eax+1],ebx    ; offset
  add  eax,5
; Запомнить точку возврата
  mov  dword ptr ds:old_14handler,eax
; Счетчик вызовов #14
  mov  dword ptr ds:Hook14Count,0
  popad
  ret
Install_Jmp_Into_14 endp
;
; Убрать jmp на наш обработчик #14
;
Uninstall_Jmp_Into_14 proc
```

```
  pushad
  mov  eax,dword ptr ds:old_14handler
  test eax,eax
  jz   @@HandlerPageFaultWasNotFound
  sub  eax,5h
;283C: 1E             push        ds
;283D: 06             push        es
;283E: 16             push        ss
;283F: 1F             pop         ds
;2840: 50             push        eax
  mov  byte ptr ds:[eax],1Eh
  mov  dword ptr ds:[eax+1],501F1606h
@@HandlerPageFaultWasNotFound:
  popad
  ret
Uninstall_Jmp_Into_14 endp
;
; Перехватчик исключения #14
;
Hook_14 proc
EntryEIP_14 equ dword ptr [ebp+14h]
; Повторяем команды оригинального обработчика #14
  ; EIP      [ebp+14h]
  ; ErrCode  [ebp+10h]
  push ds  ; [ebp+0Ch] << jmp Hook_14
  push es  ; [ebp+8h]
  push ss
  pop  ds
  push eax ; [ebp+4h]

  push ebp ; [ebp+0h]
  mov  ebp,esp
  pushad

; Найти процедуру расшифровки кода на стек
; Ее начало:
; nop            ; 90
; mov  cx,[ebp]  ; 66 8B 4D 00
; push esi       ; 56
; pop  esi       ; 5E
; add  ebp,2     ; 83 C5 02
;
; Или такое:
; mov  dl,[ebp]  ; 8A 55 00
; mov  al,al     ; 8A C0
; sub  ebp,-1    ; 83 ED FF

; Счетчик вызовов #14
  inc  dword ptr ds:Hook14Count
  cmp  dword ptr ds:Hook14Count,1
  jbe  @@NotStackDecryptor

  mov  eax,EntryEIP_14

; Вариант 1 ?
  mov  dword ptr ds:VariantVesrion,0
  cmp  dword ptr ds:[eax],004D8B66h
  jnz  @@TrySecondDecryptor
  cmp  dword ptr ds:[eax+4],0C5835E56h
  jz   @@DecryptorFound

@@TrySecondDecryptor:

; Вариант 2 ?
  inc  dword ptr ds:VariantVesrion
  cmp  dword ptr ds:[eax],8A00558Ah
  jnz  @@NotStackDecryptor
  cmp  dword ptr ds:[eax+4],0FFED83C0h
  jnz  @@NotStackDecryptor

@@DecryptorFound:
```

```
; Запомнить точку начала декриптора кода на стеке
  mov  dword ptr ds:StartStackDecryptor,eax
; Дезактивировать перехват #14
  call Uninstall_Jmp_Into_14
; Установить вызов прерывания #6 после декриптора
  call Install_CallAfterDecryptor

@@NotStackDecryptor:

  popad
  pop  ebp

; Возврат в VxD
  jmp  dword ptr ss:old_14handler
Hook_14 endp
;
; Установить вызов прерывания #6 после декриптора
;
Install_CallAfterDecryptor proc
  pushad
  mov  edi,dword ptr ds:StartStackDecryptor
  mov  eax,dword ptr ds:VariantVesrion
  mov  esi,dword ptr NewDecryptorOfs[eax*4]
  mov  ecx,dword ptr NewDecryptorSize[eax*4]
  mov  dx,word ptr NewDecryptorEndBytes[eax*2]
@@WriteNewStackDec:
  mov  al,byte ptr ds:[esi]
  mov  byte ptr ds:[edi],al
  inc  esi
  inc  edi
  loop @@WriteNewStackDec
@@PostNopsToDec:
  mov  byte ptr ds:[edi],90h ; nop
  inc  edi
  cmp  word ptr ds:[edi],dx
  jnz  @@PostNopsToDec
  mov  word ptr ds:[edi],9090h
  popad
  ret
@@NewStackDecryptor1:
;
; Первый вариант декриптора:
;
  mov  cx,[ebp]
  add  ebp,2
  rol  cx,0D1h
  add  cx,dx
  xor  edx,0C135C942h
  neg  edx
  sub  edi,2
  mov  [edi],cx
  cmp  ebp,eax
  jnz  @@NewStackDecryptor1
; Вызов обработчика #6
  push eax
  mov  eax,SelfCallUD6
  db   0FEh,0EFh,00h,01h,02h,03h,04h,05h
  pop  eax
@@EndNewStackDecryptor1:
@@NewStackDecryptor2:
  mov  dl,[ebp]
  sub  ebp,-1
  cld
  not  dl
  sub  dl,cl
  xor  ecx,8DFE10ECh
  ror  ecx,5Eh
  dec  eax
  mov  [eax],dl
  dec  ebx
  jnz  @@NewStackDecryptor2
```

```
; Вызов обработчика #6
  push eax
  mov  eax,SelfCallUD6
  db   0FEh,0EFh,00h,01h,02h,03h,04h,05h
  pop  eax
@@EndNewStackDecryptor2:
Install_CallAfterDecryptor endp
;
; Установка перехвата процедуры в код crc-подпрограммы / прямо в вызов UD ?
;
Install_CrcHook proc
  pushad
; Номер серии (по 126) вызовов int 6h
  cmp  dword ptr ds:StepNum,2
  jb   @@NotBlock2InCallsUD
; Просто установить ловушку в crc-процедуру
  mov  eax,dword ptr ds:CrcProcHookAddr
  jmp  @@InstallHookCRCProc
@@NotBlock2InCallsUD:

; Найти подпрограмму проверки,
; она должна быть "ниже" подпрограммы SEH
; Проверка на то, что в FS селектор из LDT
  mov  ax,fs
  test ax,07h
  jz   @@FSSelNotReady
  mov  eax,fs:[0] ; ~00690000h
; Сохранить адрес точки перехвата crc-кода
  mov  dword ptr ds:CrcProcHookAddr,eax

@@InstallHookCRCProc:

  sub  eax,10h
  mov  ecx,10000h shr 1
@@FindCrcProc:

;
; Ищем первые команды вызова UD:
; mov  eax,4000000Bh ; B8 0B 00 00 40
; lea  edi,[esp+4]   ; 8D 7C 24 04

  cmp  dword ptr [eax],00000BB8h
  jnz  @@NextCrcByte
  cmp  dword ptr [eax+4],247C8D40h
  jnz  @@NextCrcByte
  jmp  @@InstallHookInCrcProc
@@NextCrcByte:
  dec  eax
  loop @@FindCrcProc
  jmp  @@CrcProcNotFound

@@InstallHookInCrcProc:

  mov  byte ptr ds:[eax],68h  ; push imm
  mov  dword ptr ds:[eax+1],offset32 Hook_CRCControl
  mov  byte ptr ds:[eax+5],0C3h ; retn
; Вычислить адрес возврата в crc-процедуру
  add  eax,9
  mov  dword ptr ds:AfterCrcProcHookAddr,eax

@@CrcProcNotFound:
@@FSSelNotReady:

  popad
  ret
Install_CrcHook endp
;
; Перехват процедуры подсчета crc
;
Hook_CRCControl proc
; Устанавливаем вновь перехватчик #06
```

```
  call Install_Jmp_Into_UD6
; И вызываем код, который вызовет исключение с подфункцией 0B
  mov  eax,4000000Bh
  lea  edi,[esp+4]
  jmp  dword ptr ss:AfterCrcProcHookAddr
Hook_CRCControl endp
;
; Получение DDB адрес
;
GetLitt_DDB proc
  pushad
; Определение DDB VxD
  mov  eax,DeviceLitt_ID
  xor  edi,edi
  VMMCall Get_DDB
; ecx= адрес DDB ?
  mov  dword ptr ds:DDB,ecx
  popad
  ret
GetLitt_DDB endp
; ******************HASP FUNCTIONS*******************************
;
; Эмуляция функции 2 hasp
;
; Подпрограмма генерирования Seed - часть эмулятора хасп-ключа.
; Спасибо exefoliator'у.
; AX = seed
; return: ax:bx:cx:dx - 64-bits answer
HaspFunc_2 proc
  mov  esi,offset32 HaspData
  xor  ecx,ecx
  xor  ebx,ebx
@NextHBit:
  push ebx
  mov  bx,1989h
  mul  bx
  add  ax,5
  movzx eax,ax
  push eax
  shr  eax,9
  and  al,3Fh
  mov  ebx,eax
  shr  ebx,3
  and  al,7
  push ecx
  mov  cl,al
  mov  al,1
  shl  al,cl
  test byte ptr [ebx+esi],al
  mov  al,0
  jz   @BitClear
  inc  eax
@BitClear:
  pop  ecx
  shl  ch,1
  or   ch,al
  pop  eax
  pop  ebx
  inc  ecx
  cmp  cl,8
  jnz  @NextHBit
  test bl,1
  jz   @EvenByte
  pop  edx
  mov  cl,dh
@EvenByte:
  push ecx
  xor  ecx,ecx
  inc  ebx
  cmp  bl,8
  jnz  @NextHBit
```

```
  pop  edx
  pop  ecx
  pop  ebx
  pop  eax
  ret
HaspFunc_2 endp
;
; Эмуляция функции 3 hasp: получить по ax 16-ти битный seed
;
HaspFunc_3 proc
  push ebx
  and  eax,3Fh
  mov  ebx,offset32 DumpHaspMemory
  assume ebx: ptr tHaspMemory
  movzx eax,word ptr [ebx+eax*2].Func3Data
  assume ebx:nothing
  pop  ebx
  ret
HaspFunc_3 endp
;
; Эмуляция функции 5 hasp: вернуть 66h (const ?)
;
HaspFunc_5 proc
  mov  eax,66h
  ret
HaspFunc_5 endp
;
; Эмуляция функции 6 hasp: Вернуть dw 547Ch и dw E450h
;
HaspFunc_6 proc
  push ebx
  mov  ebx,offset32 DumpHaspMemory
  assume ebx: ptr tHaspMemory
  movzx eax,word ptr [ebx].KeyID1
  movzx edx,word ptr [ebx].KeyID2
  assume ebx:nothing
  pop  ebx
  ret
HaspFunc_6 endp
;
; Эмуляция функции 32 hasp:
; Получить seed'ов числом ecx в [esi] (ebx=last seed), start seed - ebx
;
HaspFunc_32 proc
  push edx
  mov  edx,offset32 DumpHaspMemory
  assume edx: ptr tHaspMemory
@@GetSeedsForF32:
  lea  eax,[ebx-28h]
  mov  ax,word ptr [edx+eax*2].Func32Data
  mov  word ptr [esi],ax
  add  esi,2
  inc  ebx
  loop @@GetSeedsForF32
  assume edx:nothing
  pop  edx
  ret
HaspFunc_32 endp
VxD_LOCKED_CODE_ENDS
end
;
; ****************Запускающий vxd код*******************
;
.386
.model flat,stdcall
option casemap:none
include \masm32\include\windows.inc     ; Макрос invoke...etc
include \masm32\include\kernel32.inc    ; DeviceIOControl, CreateFile... etc
includelib \masm32\lib\kernel32.lib     ; Сама библиотека
include \masm32\include\user32.inc      ; MessageBox
includelib \masm32\lib\user32.lib       ; Сама библиотека
```

```
.data
; Отладочные сообщения, выводимые через MessageBox
MsgCaption  db "Iczelion's tutorial no.2",0
; Сообщения о процессе загрузки/выгрузки VxD
AppName     db "Hasp key emulator for win9x",0
Success     db "The emulator is successfully loaded!",0
SerNumErr   db "VxD not present or internal error...",0
Failure     db "The VxD is not loaded !",0
; Имя загружаемого VxD, которое передается CreateFile-у
VxDName     db "\\.\UD06.VXD",0
; Структура, которой передаются параметры коду VxD
InBuffer    dd offset LogPtr ; Указатель на буфер для чтения лога
include log06.inc
; Буфер лога
LogPtr      dd 0 ; Указатель лог-буфера
LogPackets  db (MAX_LOG_RECORDS * sizeof(tLogPacket)) dup (?)
; Сегмент данных программы(неинициализированных)
.data?
hVxD        dd ? ; Тут будет храниться хэндл открытого VxD
; Начало выполнимого кода
.code
start:
; ***
; Открываем лог
; ***
.data
LogFileName   db "trap_06.log",0
.data?
LogDescr      dd ?
bWrite        dd ?
.code
; Открыть лог, если есть он уже
  invoke CreateFile,offset LogFileName,GENERIC_WRITE,FILE_SHARE_WRITE+FILE_SHARE_READ,\
      0,OPEN_EXISTING,FILE_ATTRIBUTE_NORMAL,0
  mov  LogDescr,eax          ; Save file descriptor
  inc  eax
  jnz  @@LogFileOpen
; Видимо, нет. Тогда попробовать создать
  invoke CreateFile,offset LogFileName,GENERIC_WRITE,FILE_SHARE_WRITE+FILE_SHARE_READ,\
      0,CREATE_ALWAYS,FILE_ATTRIBUTE_NORMAL,0
  mov  LogDescr,eax          ; Save file descriptor
  inc  eax
  jnz  @@LogFileOpen
.data
OpenErrorMsg db "Log file not created !",0Ah,0
.code
  push offset OpenErrorMsg
  call Write_Log
  jmp  Exit
@@LogFileOpen:
; Встать в конец лога
  invoke SetFilePointer,LogDescr,0,NULL,FILE_END

; Загрузим динамический VxD через CreateFile
  invoke CreateFile,addr VxDName,0,0,0,0,FILE_FLAG_DELETE_ON_CLOSE,0
  cmp  eax,INVALID_HANDLE_VALUE ; VxD Успешно загружена ?
; Сохранить хэндл VxD
  mov hVxD,eax
  jnz  @@LoadedOK
@@VxDProblem:
  invoke CloseHandle,hVxD
  invoke MessageBox,NULL,addr Failure,NULL,MB_OK+MB_ICONERROR
  jmp  Exit
@@LoadedOK:
; Выдать сообщение о том, что VxD загружена успешно, давая возможность
; прекратить работу VxD
  invoke MessageBox,NULL,addr Success,addr AppName,MB_OK+MB_ICONINFORMATION
; Получить протокол работы
  invoke DeviceIoControl,hVxD,1,addr InBuffer,sizeof InBuffer,NULL,NULL,NULL,NULL
; Проверка на ошибку. Если ошибка, то eax = 0
  test eax,eax
```

```
  jz   @@VxDProblem
; Ошибки нет. Скинем данные в лог.
  call Print_SerialInfo
; Закроем VxD
  invoke CloseHandle,hVxD
Exit:
; Завершим программу
invoke ExitProcess,NULL
; Подпрограммы
.data
SerialInfo db "Total traps: ?????????? See log file trap_01.log...",0Dh,0Ah,0
LogInfoEnd   db 'EAX=???????? EDI=????????',0Dh,0Ah,0
.data?
LogSize    dd ?
.code
Print_SerialInfo proc
; Сколько было записей в логе
  mov  eax,dword ptr LogPtr
  mov  ebx,1000000000
  mov  edi,offset SerialInfo+13
  call DecChar
  invoke MessageBox,NULL,addr SerialInfo,addr MsgCaption,MB_OK
  push offset SerialInfo
  call Write_Log
  mov  ecx,dword ptr LogPtr
  test ecx,ecx
  jnz  @@IsLogData
  ret
@@IsLogData:
; Размер лога
  mov  eax,ecx
  imul eax,eax,sizeof(tLogPacket)
  add  eax,4 ; Size
  mov  dword ptr LogSize,eax
  mov  ebx,offset LogPackets
  assume ebx: ptr tLogPacket
@@OutLogData:
  pushad
; Регистр eax и изображение
  mov  eax,dword ptr [ebx].RegEAX
  mov  edi,offset LogInfoEnd+4
  call HexChar
; Регистр edi
  mov  eax,dword ptr [ebx].RegEDI
  mov  edi,offset LogInfoEnd+17
  call HexChar
  push offset LogInfoEnd
  call Write_Log
  popad
  add  ebx,sizeof(tLogPacket)
  dec  ecx
  jnz  @@OutLogData
  assume ebx:nothing
  ret
Print_SerialInfo endp
; ************DecChar: Подпрограмма форматирования строки из числа***********
; eax=number to digit, edi=offset result string in format 00000000(@n[ebx])
; ebx=начальный делитель
DecChar proc
  pushad
  pushfd
  cld
@GetDec:
  xor  edx,edx
  div  ebx
  add  al,'0'
  stosb
  push edx
  mov  eax,ebx
  xor  edx,edx
  mov  ebx,10
```

```
  div  ebx
  mov  ebx,eax
  pop  eax
  test ebx,ebx
  jnz  @GetDec
  popfd
  popad
  ret
DecChar endp
; ***
; Подпрограмма записи в лог и на консоль одновременно
; ***
; [ebp+8] - адрес строки для записи
Write_Log proc
  push ebp
  mov  ebp,esp
; Получить длину строки
  mov  esi,dword ptr [ebp+8] ; Адрес строки
  call Str_Len
  test ecx,ecx
  jz   @@ExitWrite_Log
; Запись в протокол
  push NULL
  push offset bWrite
  push ecx ; Длина строки
  push dword ptr [ebp+8] ; Адрес строки
  push LogDescr
  call WriteFile
@@ExitWrite_Log:
  pop  ebp
  ret  4
Write_Log endp
Write_EndStr_ToLog proc
.data
EndStr db 0Ah,0
.code
  push offset EndStr
  call Write_Log
  ret
Write_EndStr_ToLog endp
; ***
; Get lenght of string
; esi=addr of string; result: ecx=length
; ***
Str_Len proc
  xor  ecx,ecx
  push eax
  push esi
@@GetStrLen:
  lodsb
  test al,al
  jz   @@ExitStrLen
  inc  ecx
  jmp  @@GetStrLen
@@ExitStrLen:
  pop  esi
  pop  eax
  ret
Str_Len endp
; Форматировать число в EAX в строку
HexChar proc
  pushad
  cld
  mov  ecx,8
  mov  ebx,offset TabHex
@GetHex:
  rol  eax,4
  push eax
  and  al,0fh
  xlat
  stosb
```

```
  pop  eax
  loop @GetHex
  popad
  ret
HexChar endp
TabHex  db '0123456789abcdef'
end start
```

Защита программ с помощью сетей Петри

Автор: Broken Sword

Дата: 2004-05-18

Когда сны становятся реальностью...

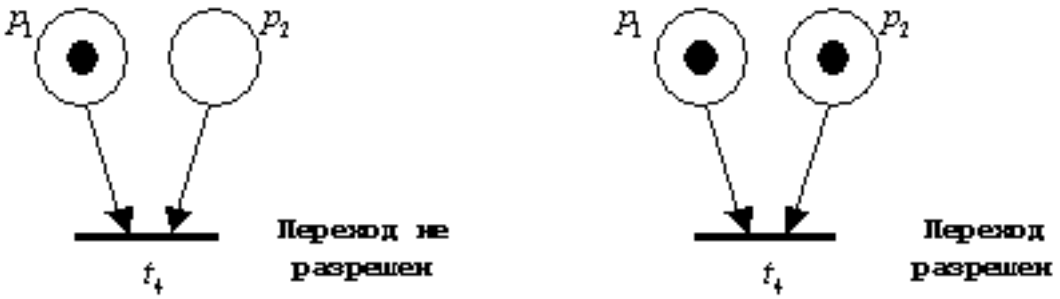
Что такое сеть Петри? Сеть Петри - это двудольный ориентированный мультиграф. Ну вот, я уже вижу, как большинство читателей в ужасе пытаются закрыть страницу. Но не стоит так пугаться - прочитайте спокойно до конца и, по крайней мере, вы узнаете кое-что новое и интересное.

Не бойтесь сетей Петри - за вычурным определением скрывается простейший, интуитивно понятный даже ребенку механизм. Для начала определим основные понятия из теории, без которых, к сожалению, дальнейшее продвижение просто невозможно: сеть Петри состоит из четырех элементов: множество *позиций* P (схематически обозначаются кружками), множества *переходов* T (обозначаются черточками), входной функции I и выходной функции O. Входная и выходная функции связаны с переходами и позициями. Входная функция I отображает переход t_j в множество позиций $I(t_j)$, называемых *входными позициями перехода*. Выходная функция O отображает переход t_j в множество позиций $O(t_j)$, называемых *выходными позициями перехода*. Ориентированные дуги (стрелки) соединяют позиции и переходы, при этом некоторые дуги направлены от позиций к переходам, а другие - от переходов к позициям.

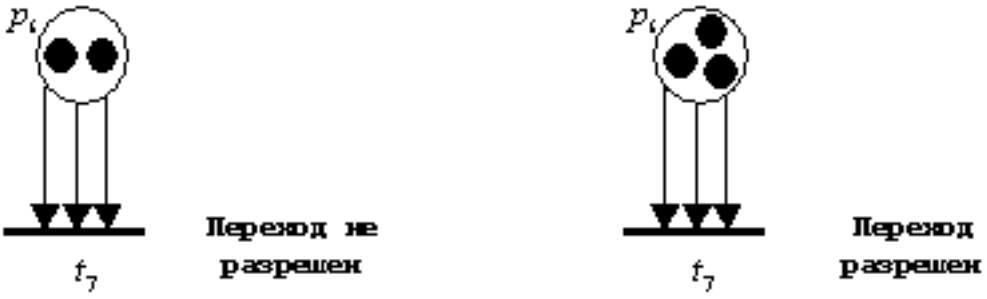
Маркировка μ - присвоение фишек позициям сети Петри. Фишка - примитивное понятие сетей Петри. Фишки используются для определения *выполнения* сети Петри.

Выполнением сети Петри управляют количество и распределение фишек в сети. Фишки находятся в кружках (позициях) и управляют выполнением переходов сети. Сеть Петри выполняется посредством *запуска* переходов. Переход может запускаться только в том случае, когда он *разрешен*.

Переход называется разрешенным, если каждая из его входных позиций имеет число фишек по крайней мере равное числу дуг из позиции в переход. Прочитайте предыдущее предложение еще пару раз. Например, если позиции p_1 и p_2 служат входами для перехода t_4, тогда t_4 разрешен, если p_1 и p_2 имеют хотя бы по одной фишке:

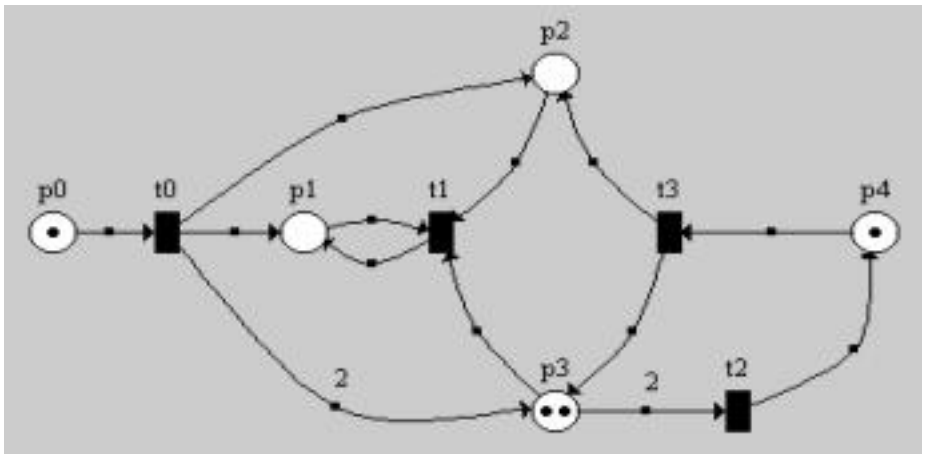


Для перехода t_7 с входным комплектом (p_6, p_6, p_6) позиция p_6 должна обладать по крайней мере тремя фишками, для того чтобы t_7 был разрешен:



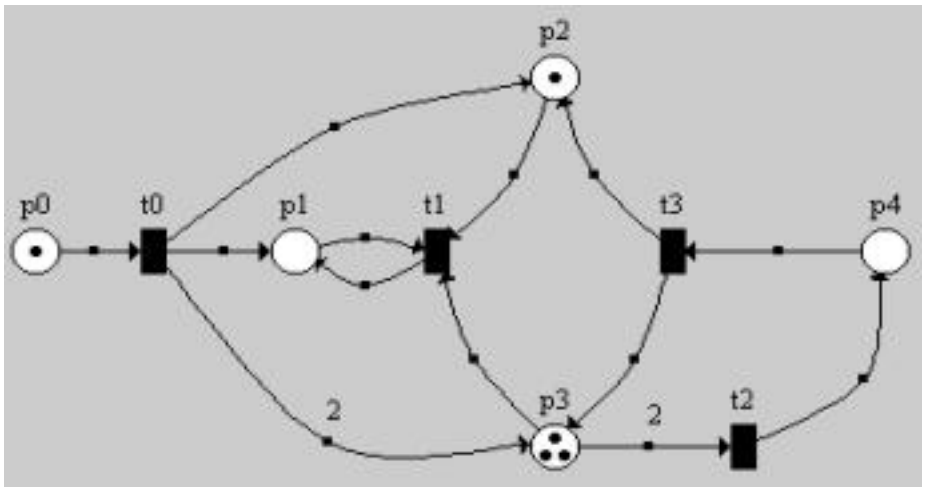
Переход запускается удалением всех разрешающих фишек из его входных позиций и последующим помещением в каждую из его выходных позиций, по одной фишке для каждой дуги. Из приведенных ниже графических иллюстраций все сразу станет понятно:

1



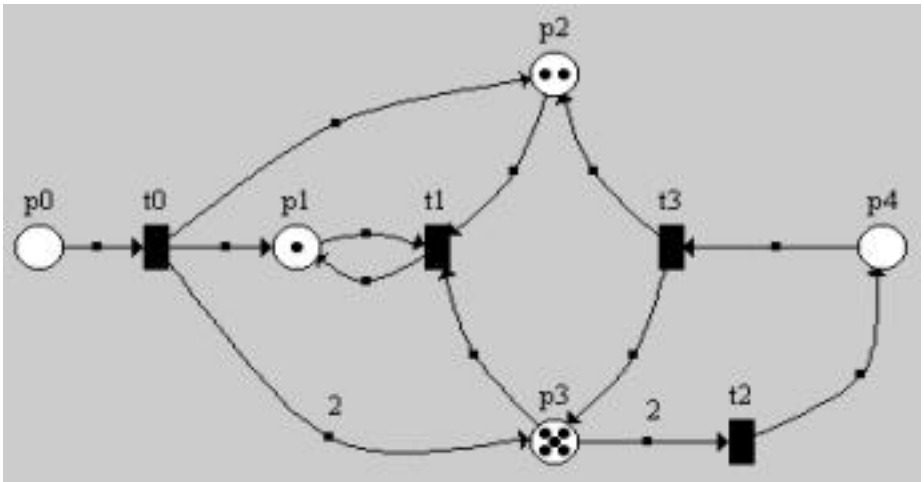
В данной сети переходы t0, t2 и t3 разрешены.

2



Маркировка, полученная в результате запуска перехода t3.

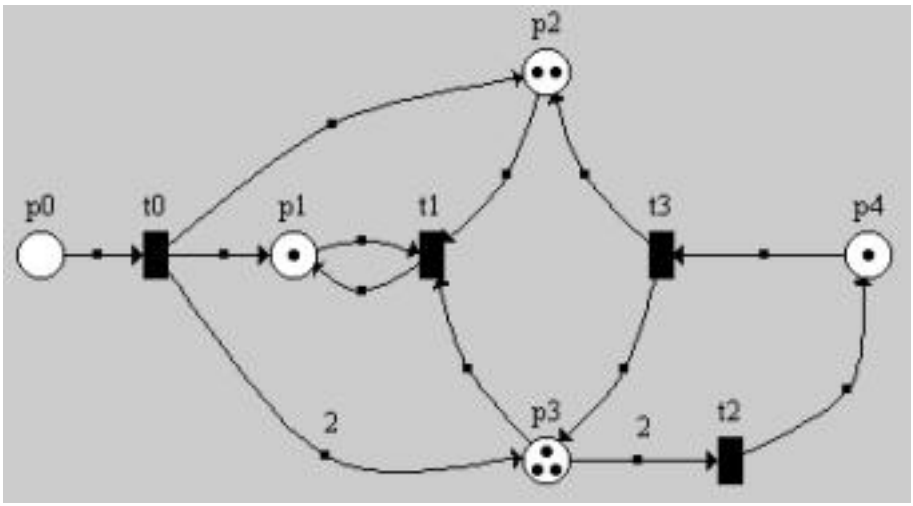
3



Маркировка, полученная в результате запуска перехода t0.

(обратите внимание - из t0 в p3 идет ДВЕ дуги!!! Поэтому в p3 добавилось ДВЕ фишки).

4



Маркировка, полученная в результате запуска перехода t2 (и снова обратите внимание на ДВЕ дуги, идущие из p3 в t2).

Всю эту игру можно продолжать до тех пор, пока хотя бы один из переходов будет разрешен. Поиск тупиковых ситуаций (когда ни один из переходов не является разрешенным) - тема отдельной статьи.

Теперь вы в самом первом приближении познакомились с сетью Петри, возможно, она напомнила давно забытую детскую игру...

Но довольно теории, полученных знаний вполне достаточно чтобы двигать дальше. За более глубоким погружением в удивительный мир сетей Петри обращайтесь к книге James L. Peterson-a, "Petri net theory and the modeling of systems".

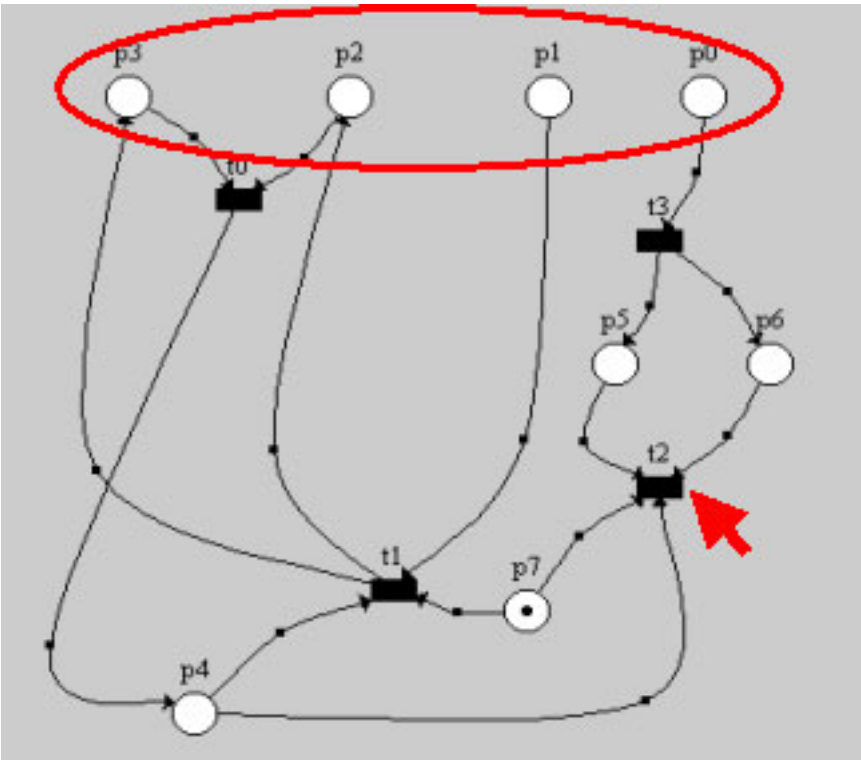
Теперь возникает резонный вопрос: как все это прикрутить к защите программ? А очень просто J.

Как вы уже догадались - Сеть Петри является идеальным инструментом для моделирования переходов при выполнении определенных условий.

Не секрет, что большинство защит сводится к максимальному сокрытию и недопущения длинных рук кракера к пресловутому условному переходу (да-да, того самого badguy/goodguyjump). При этом обертка может состоять из самых изощренных анти-отладочных приемов, программа может быть запакована множеством упаковщиков, и любой другой хренью, которую вы только можете себе

представить - в середине всей этой мишуры находится один единственный переход (нет, проверок может быть очень много, однако сути дела это не меняет). Итак, рассмотрим простейший пример, на котором можно наглядно просмотреть идею защиты:

Построим простейшую сеть Петри:



Допустим, что при начальной маркировке можно задавать кол-во фишек только в позициях p0...p3. (обратите внимание, что позиция p7 изначально содержит одну фишку). Остальные позиции недоступны для маркирования. Зададим *ключевому* переходу t2 наименьший приоритет (это означает, что если одновременно разрешены переходы t2 и t1, то сработает переход t1), после чего он (переход t2) выполниться тогда и только тогда, когда начальная маркировка помечает позиции p0...p3 следующим образом: p0=1, p1=0, p2=1, p3=1. При всех остальных допустимых начальных маркировках переход t2 не сработает. Если не верите то попытайтесь на практике опровергнуть это утверждение, но уверяю вас - ничего не выйдет.

Теперь представим, что каждый переход - это поток, который проверяет определенный набор бит на ВХОДЕ (входные позиции) и в зависимости от результата устанавливает биты на ВЫХОДЕ. Так, поток t3 проверяет фишки (биты) в позиции p0 и в зависимости от того есть там фишки (бит установлен) или нет (бит сброшен) устанавливает биты на выходных позициях (p5 и p6).

Допустим, пользователю необходимо зарегистрировать программу, введя 4-битный ключ. Биты ключа (фишки) помещаются в позиции p0,p1,p2 и p3 (если бит установлен, то фишка помещается, нет - значит соотв. позиция остается пустой). Этим обеспечивается начальная маркировка сети.

Программа будет считаться зарегистрированной, если переход t2 сработает (т.е. будет активным). Как уже было показано выше - переход t2 сработает только при такой начальной маркировке: p0=1, p1=0, p2=1, p3=1.

Все это легко можно реализовать программно (см. приложение).

Задача достижимости - основная задача, решаемая при анализе сетей Петри, к которой сводится множество других задач (в частности - задача активности). Задача достижимости формулируется следующим образом: для данной сети Петри C с маркировкой m и маркировки m' определить: m'ÎR(C,m). Задача активности и достижимости в общем случае не решены.

Дж. Питерсон: стр. 103 «4.2.1.4 Ограниченность дерева достижимости»

> *«Как видим, дерево достижимости можно использовать для решения задач безопасности, ограниченности, сохранения и покрываемости. К сожалению, в общем случае его нельзя использовать для решения задач достижимости и активности [...]»*

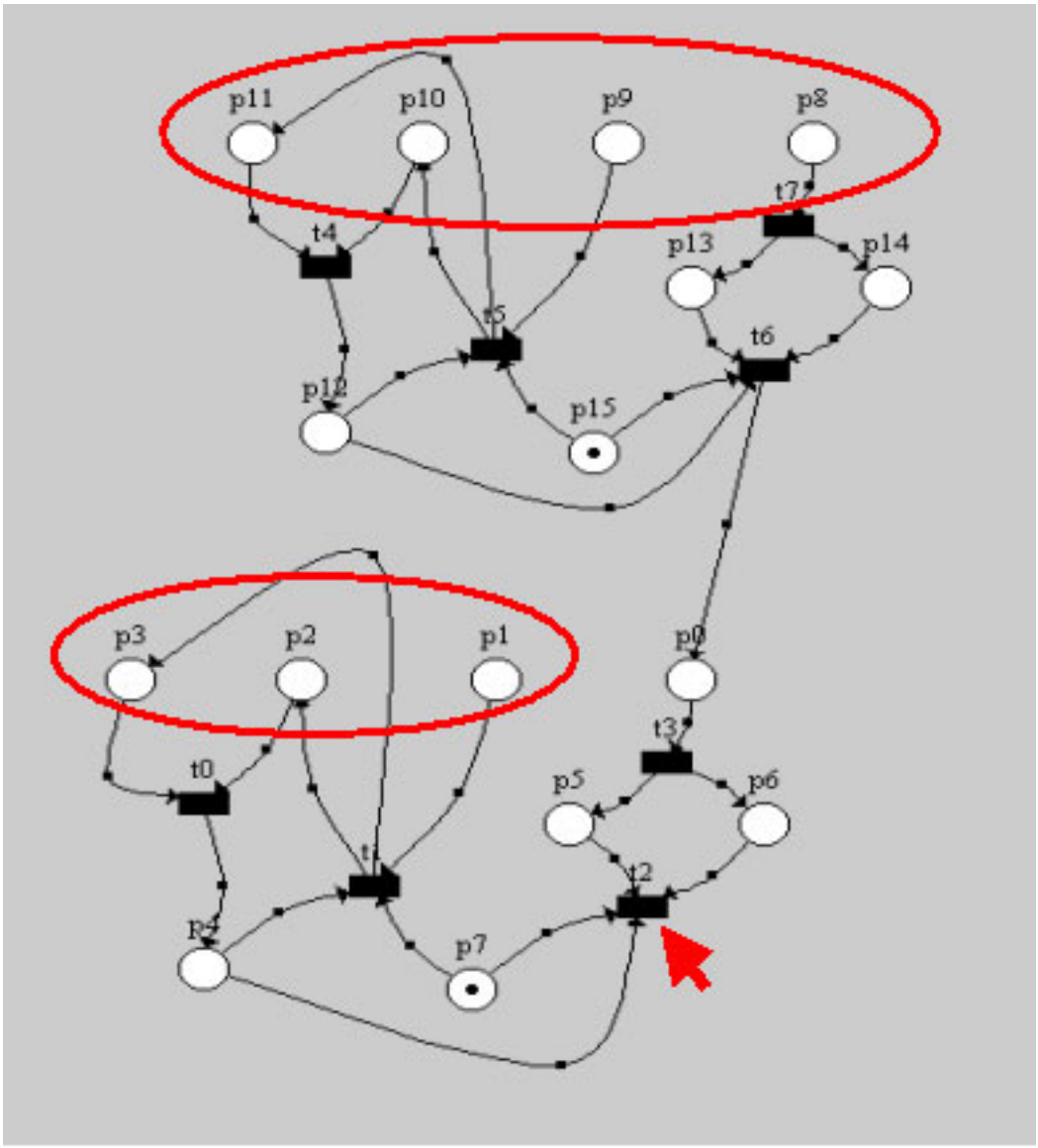
Решая задачу достижимости с использованием дерева достижимости мы фактически прибегаем к методу полного перебора. К данному методу вообще можно свести любую задачу. Поэтому защищенность здесь определяется только временем и ресурсами, затраченными на полный перебор.

На нашем примере легко видеть, что найти правильный ключ (т.е. правильную начальную маркировку) можно перебрав всего лишь 2^4 = 16 значений. Это можно сделать даже вручную, а взглянув на представление программы в виде сети Петри можно догадаться интуитивно за несколько секунд.

Использование ключей длиной менее 2^128 бит на сегодняшний день не может обеспечить защиту на приличном уровне, именно поэтому предложенная в примере сеть Петри не годится для практической защиты и приведена лишь в качестве наглядной иллюстрации.

Однако приведенную в примере сеть можно легко «наращивать» автоматически, тем самым усложняя ее структуру и увеличивая длину ключа, что является, возможно, очень важным ее свойством. Под «наращиванием» я понимаю следующее: к каждой позиции, которую можно пометить при начальной маркировке, "подводиться" новая сеть. При этом она теряет возможность быть помеченной при начальной маркировке, однако появляются новые N позиций, которые доступны для начальной маркировки.

Например, нарастим сеть над позицией p0. Для этого достаточно просто скопировать уже имеющуюся сеть:



При этом переход t2 остается *ключевым* и появляются новые позиции (p8...p11), которые можно пометить при начальной маркировке (фактически позиции p8...p11 дублируют смысловое значение позиций p0...p3 в нижней сети).

Легко видеть, что переход t6 фактически выполняет ту же роль, что и переход t2 в «нижней» сети. Очевидно, что для срабатывания перехода t2 необходимо срабатывание перехода t6, а для этого начальной маркировкой необходимо пометить позиции следующим образом:

```
p1=0, p2=1, p3=1, p8=1, p9=0, p10=1, p11=1.
```

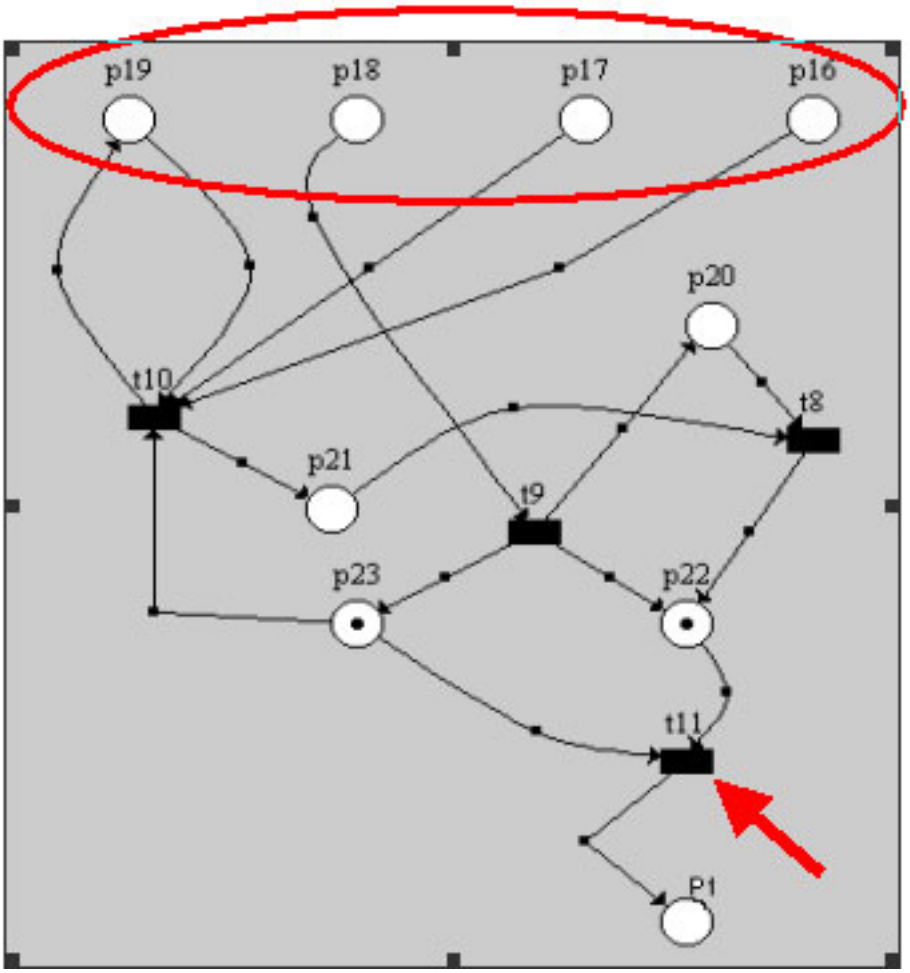
Длина ключа увеличилась до 2^7 бит. Теперь для полного перебора необходимо осуществить уже 128 проверок.

Надстроив сеть аналогичным образом над p1, p2 и p3 мы получим ключ длиной 16 бит, для полного перебора необходимо рассмотреть 2^16 комбинаций, чего также не достаточно. Продолжая наращивание дальше мы дойдем до 2^128 ключей, что можно считать достаточным для надежной защиты. В идеале размер ключа следует довести до 2^512.

Для простоты остановимся на 16 битном ключе.

Единственная позиция, которая не поддается наращению по аналогии с другими позициями - позиция p1, т.к. для достижимости t2 нам необходимо ее нулевое значение. Если нарастить над ней копию уже имеющейся сети то ее нулевое значение достижимо при 15 (16-1) различных

комбинаций. Это означает, что диапазон возможных ключей сужается. Поэтому для перехода p1 следует изменить надстроечную сеть. Одним из вариантов может явиться следующее наращение:



Здесь для того, чтобы переход t11 НЕ СРАБОТАЛ (а это именно то чего нам нужно добиться) начальной маркировкой нужно задать следующие значения:

```
P16=1, P17=1, P18=0, p19=1
```

Переходу t11 необходимо задать наименьший приоритет.

Для 16-битного ключа можно построить таблицы переходов (см. приложение "а").

Важным свойством таблицы переходов является то, что строки (столбцы) таблицы можно переставлять в произвольном порядке, структура сети от этого не меняется, а анализ усложняется.

стр. 145 «5.6. Сложность задачи достижимости»

> *«Поскольку задачи подмножества и равенства для множеств достижимости сетей Петри неразрешимы, то возможно, что неразрешима также и сама задача достижимости. Однако в настоящее время вопрос, разрешима ли (или неразрешима) задача достижимости, открыт. На сегодняшний день не существует ни алгоритма, решающего задачу достижимости, ни доказательства того, что такого алгоритма не может быть. [...]»*

И это просто замечательно! (по крайней мере для нашей защитной системы). Именно на этом принципе она и строится.

Взлом программ, защищенных с помощью сетей Петри

Напоследок несколько слов о взломе такой защиты. Очевидно, что стандартные методы здесь не подходят. В отличие от программ, защищенных «последовательными» методами, кракер никогда не сможет определить какой из переходов в защите является ключевым, т.к. при неправильной начальной маркировке ключевой переход просто недостижим. С другой стороны, кракер может попытаться запустить все переходы в сети, каким-либо образом обнаружив все позиции. Действительно, для нашего случая с 16-битным ключом ему достаточно перепробовать 20 переходов, вместо 2^16 ключей (т.е. заполнить фишками ВСЕ позиции). Для противостояния данному способу взлома можно пойти несколькими путями:

1. Построить сеть таким образом, чтобы ключевой переход имел наименьший приоритет у окружающих его переходов. То есть один из окружающих переходов сработает раньше и уведет фишку из ключевой позиции, таким образом ключевой переход не сработает. **2.** Добавление в сеть такого количества позиций, что полный перебор всех переходов сопоставим с полным перебором всех начальных маркировок. Это единственный способ обеспечить защиту, которая зависит только от длины ключа. Данный метод неэкономичен с точки зрения расхода памяти и скорости обработки.

Надо также заметить, что отладка программы существенно осложняется тем, что для анализа поведения сети необходимо отлаживать одновременно N потоков, где N - кол-во переходов в сети, поэтому взлом программы, защищенной моим методом, может быть осуществлен только статическим эвристическим путем, что при наличии тысяч переходов просто нереально осуществить.

В заключении хочется привести одну мысль, которую следует положить в основу любой защитной системы: главная цель любой защитной системы - скрыть некоторую информацию от кракера. В то же время, любая система защиты искусственна, то есть создана человеком. Любой человек, даже если он непревзойденный гений, способен создать лишь временно устойчивую к атаке защиту, то есть ни одна система защиты не достигает своей единственной и главной цели, откуда, очевидно, следует, что создание защитных систем любого вида занятие бесполезное и бесперспективное.

Однако, наверно, это не совсем так - по крайней мере это очень увлекательно.

Аттач (находится в архиве `wasm_files.zip`: `files/0322/petri.zip`).

Теоретические основы крэкинга: Глава 6. Куда попадают данные.

Автор: CyberManiac

Дата: 2004-06-13

На протяжении уже двух глав я рассказывал о техниках поиска данных всевозможных типов. Подобно муравьям, мы трудились, учась собирать по крупицам полезную информацию, которая могла бы помочь понять структуру и смысл этих данных. И теперь вы знаете, как «просеивать» мегабайты листингов, извлекая из них константы и указатели, как вести поиск переменных в адресном пространстве работающей программы. Все это время мы рассматривали «живые», находящиеся внутри работающей программы данные отдельно от «мертвых», тихо лежащих на жестком диске и никак себя не проявляющих. И вот пришло время осуществить синтез, увидеть диалектическую взаимосвязь между этими двумя формами существования данных, и, главное, понять, какую практическую пользу из этой взаимосвязи можно извлечь.

Нетрудно заметить, что любые данные, хранящиеся на диске, в действительности предназначены для обработки какой-либо программой, а, стало быть, рано или поздно будут загружены в оперативную память (если, конечно, это полезные данные, а не мусор, подлежащий удалению), где над ними будут производиться всяческие действия. А результаты этих действий так или иначе отобразятся в мире «по нашу сторону экрана» при помощи одного из многочисленных устройств ввода-вывода, чтобы пользователь мог их увидеть, услышать или ощутить каким-либо иным образом. И пока эти данные будут проходить свой непростой путь от загрузки с винчестера до отображения на экране монитора, их можно «выловить» из адресного пространства программы или даже с экрана (то есть, конечно, не совсем с экрана, а из видеопамяти). Более того, информация, которую очень непросто расшифровать, разглядывая файл в шестнадцатиричном редакторе, при загрузке соответствующей программой нередко бывает представлена в памяти в виде структур с весьма незамысловатым внутренним устройством. Да и само наблюдение за процессом загрузки данных может дать множество полезной информации, и в этой главе мы посмотрим, как такую информацию можно извлекать.

Когда-то давным-давно, когда Windows еще был девяносто пятым, защиты – простыми, а авторы защит - наивными, серийные номера извлекались из программ следующим образом: устанавливались точки останова на все функции WinAPI, при помощи которых мог считываться серийный номер (благо их не так много). Затем нужно было вызвать окно регистрации, ввести в него любые данные и посмотреть, какая из точек останова сработает. Дальше начиналось самое интересное: поскольку то были старые добрые времена, непуганые разработчики для проверки правильности серийных номеров частенько использовали обычное сравнение двух текстовых строк, причем для сравнения использовался банальный вызов функции lstrcmp (или ее самодельного аналога), два параметра которой являлись указателями на сравниваемые строки. И чтобы получить правильный серийник, требовалось лишь найти нужную функцию и посмотреть на ее параметры.

Конечно, те времена давно прошли, и ныне очень, очень редко встречаются программы, в которых серийный номер хранился бы в открытом или «как бы зашифрованном» при помощи команды XOR виде. Но для крэкера как раз важен не столько сам факт хранения данных в открытом виде, сколько идея: скормив программе заведомо неверные данные, пронаблюдать за тем, как программа эти данные будет «переваривать» и проверять на корректность. Да и наблюдение за процессом «заглатывания» данных программой может стать источником ценных идей. Приведу пример из собственной практики.

Однажды я изучал некую программу на предмет «исправить пару переходов, чтобы она лучше и дольше работала». Нужную «пару переходов» я вычислил за считанные минуты, а патчинг этих байт непосредственно в памяти успешно решал мою проблему на время одной сессии работы с программой. Но вот исправление тех же байтов в исполняемом файле неизбежно приводило к «падению» программы сразу после запуска. Нетрудно было догадаться, что программа неким образом контролировала собственную целостность, и, скорее всего – проверкой контрольной суммы. Это предположение подтверждалось и подозрительно большим временем загрузки программы (компьютеры тогда были намного медленнее, поэтому иногда следы работы защитных средств были видны, что называется, невооруженным глазом). Решение тоже было достаточно очевидным – найти функцию вычисления контрольной суммы, посмотреть, какой результат эта функция должна была возвращать в норме и либо обойти сравнение реальной контрольной суммы с эталоном, либо заставить функцию возвращать эталонное значение в любом случае. Но как найти нужную функцию?

Для начала я попытался выяснить, каким образом программа проверяет свою контрольную сумму – сканирует образ непосредственно в памяти, или все-таки проверяет то, что лежит на диске. Поскольку программа была не запакована (в те времена упаковщики вообще встречались нечасто), я просто загрузил программу при помощи loader'а из состава SoftIce (одна из полезных крэкеру функций этого loader'а как раз в том, что он передает управление отладчику сразу после загрузки подопытной программы в память). Затем я поставил аппаратные точки останова на чтение тех байт, которые я хотел изменить в файле (тут логика проста: если программа проверяет саму себя в памяти, то для этого ей придется прочитать себя) и на запись (на всякий случай) и отпустил программу на волю (то есть на исполнение). Ни одна из точек останова не сработала, из чего следовало, что программа либо не проверяет себя в памяти, либо это очень хитрая программа, которая на мою уловку не попалась. Запустив программу под filemon'ом, я увидел, что сразу после запуска эта программа поблочно читает свой собственный исполняемый файл, что навело меня на мысль о встроенной в программу проверке контрольной суммы. Дальнейшее было делом техники: прогнав программу под Bounds Checker'ом, я выяснил, что нужный мне вызов функции чтения из файла в действительности производится не из самой программы, а из DLL, которая в случае успешной проверки возвращала некое значение (а в случае неуспешной проверки – тоже значение, но уже другое) и что для работоспособности программы величина этого значения было критически важной. В этой ситуации я счел наилучшим решением выкинуть вычисление контрольной суммы файла (это ощутимо ускорило загрузку) и немного «помог» этой DLL всегда возвращать нужное мне значение.

О чем эта история? Ну разумеется, не о том, что глупо помещать код проверки в DLL, где его несложно поправить. Прежде всего, я хотел показать, как наблюдение за переходом данных из «мертвого» состояния в «живое» (а именно таким переходом и является поблочная загрузка файла для вычисления контрольной суммы) может помочь обнаружить защитные механизмы. Действительно, стоило мне понаблюдать за процессом проверки целостности файла (о котором я ранее ничего не знал, кроме факта его наличия) под API-шпионом, как я сразу же получил информацию о типе защиты и местонахождении защитной процедуры. А после недолгих экспериментов и размышлений я также узнал, какова величина контрольной суммы программы до и после внесения в нее модификаций.

Вылавливание нужных данных из оперативной памяти уже давно стало неотъемлемой частью крэкинга и получило весьма широкое распространение. Если Вы уже пробовали самостоятельно взломать или хотя бы посмотреть на внутренности какой-либо программы, то, возможно, уже столкнулись с упаковщиками исполняемых файлов (или, если быть до конца точным, с файлами, обработанными такими упаковщиками). Разумеется, крэкеру во всех этих упаковщиках и навесных защитах интересно одно: методы их снятия. Очевидно, что упаковка программ – процесс обратимый и проблема лишь в том, чтобы найти способ обращения этого процесса, проще говоря –

распаковать ее. Существует два подхода к распаковке. Можно проанализировать алгоритмы работы встроенного в программу навесного модуля, осуществляющего распаковку и самостоятельно воспроизвести эти алгоритмы в виде независимой программы. Этот метод обычно долог и труден.

А можно оставить все хлопоты по распаковке «навесному» модулю-распаковщику, встроенному в исполняемый файл, а потом воспользоваться результатами его трудов, «выдернув» распакованную и готовую к употреблению программу из памяти компьютера. В таком подходе мне определенно видится изящество и утонченность – вместо того, чтобы брать штурмом алгоритмы распаковки, мы, фактически, заставляем автора навесной защиты сражаться с собственным творением. Конечно, авторы защит тоже не дремлют – редкий упаковщик не уродует до неузнаваемости таблицу импорта, не содержит в антиотладочного кода или средств противодействия дампингу, и преодоление этих трудностей требует от крэкера гораздо больших усилий, чем собственно снятие дампа. Однако сама идея как нельзя лучше раскрывает тему этой главы – если нет возможности (сил, времени, желания) «расшифровать» нужные данные вручную, стоит подумать о том, где можно найти готовые алгоритмы декодирования, каким образом их применить и как воспользоваться результатом их работы.

Особенно интересные и впечатляющие результаты дает сочетание предлагаемой технологии с глубоким патчингом программ в памяти. Недавно мне в руки попался экземпляр MoleBox –представителя (надо сказать, не самого совершенного) нового поколения защит, где упаковке подвергается не только исполняемый файл приложения, но и все остальные файлы, входящие в комплект программы, после чего все эти упакованные файлы сливаются в один монолитный исполняемый файл («ящик» в терминологии MoleBox). Сам EXE-файл программы модифицируется таким образом, что вызовы функций API для работы с файлами подменяются вызовами внутренних функций защиты, после чего программа может одинаково успешно обращаться как к файлам на жестком диске, так и к файлам, находящимся внутри «ящика» (в MoleBox к файлам из «ящика» возможен доступ только на чтение). Кстати, базовая информация о принципах работы MoleBox честно приведена в документации к программе, поэтому позволю себе в очередной раз повторить совет внимательно читать документацию к исследуемым программам. После недолгих экспериментов удалось выяснить, что «виртуальная директория», в которой работает защищенное приложение, содержит все файлы программы, и извлечь их оттуда не составляет никакого труда. При помощи манипуляции значениями регистров и содержимым стека в SoftIce мне удалось вызвать FindFirstFile/FindNextFile и вручную прочитать список имен всех файлов, находящихся в «ящике» программы, кроме самого исполняемого файла (который пришлось выковыривать более традиционными методами). Дальше все было еще проще в теории и еще тяжелее и нуднее на практике: выделение памяти под буфер, чтение файлов в этот буфер и последующее сохранение в другой файл. Конечно, проделывать все эти операции вручную – занятие крайне трудоемкое, и если Вы захотите повторить мой эксперимент, я советую Вам не упражняться в играх с регистрами, а набросать соответствующую программку на ассемблере, внедрить ее в адресное пространство «жертвы», и получить тот же самый результат, но в несколько раз быстрее.

Еще одно применение предлагаемого метода – декодирование данных, имеющих сложную или неочевидную структуру. Например, при сохранении множества записей, содержащих как текстовую, так и числовую информацию, формат результирующего файла может быть совершенно неочевиден. К примеру, массив структур, состоящих из одного текстового (обозначим его буквой T) и одного числового поля (обозначим его как N) может сохраняться в файле как минимум двумя способами:

T1, N1, T2, N2, T3, TN3, …
или как N1, N2, N3, … T1, T2, T3, …, где Tn, Nn – текстовое и числовое поле соответственно n-й записи в массиве. Поскольку текстовые данные отличить от числовых несложно даже по внешнему виду, в данном конкретном примере никаких сложностей с извлечением из файла элементов

массива скорее всего не возникнет. Но представим, что текстовых полей – несколько, а сохраняемые в этих полях значения – внешне очень похожи. И что каждая из структур в массиве содержит подструктуры, сохраняемые в том же самом файле подобным же образом. Задача расшифровки внутреннего формата файла уже не кажется такой тривиальной, не правда ли?

Однако вспомним наш краткий курс психологии программиста и попробуем представить, как нормальный программист организует хранение тех же данных в памяти. Скорее всего, он создаст банальный массив структур, поэлементно заполнит его значениями из файла и будет обращаться к нему так, как обращаются к любым другим массивам. Если логика задачи предполагает, что в загруженные данные потребуется добавлять новые элементы или удалять имеющиеся, то вместо обычного массива скорее всего будет одно- или двухсвязный список, в котором каждый элемент помимо собственно структуры будет содержать еще указатель на предыдущий и последующий элементы списка. А дальше… Дальше задача полностью аналогична описанной в предыдущей главе задаче по «раскалыванию» неизвестного формата и извлечению данных, с той лишь разницей, что работать мы будем не с кодом приложения, а с обрабатываемыми этим приложением данными. Увы, данный прием не универсален – например, буферизованная обработка данных (т.е. такая, при которых обрабатываемые данные не переносятся в память целиком, а подгружаются по мере необходимости) не позволит расшифровать весь исследуемый файл целиком за один раз. Вообще, количество всевозможных «особых случаев» весьма велико, и рассмотреть их все в этой небольшой работе практически нереально. И если Вам придется столкнуться с такой нетривиальной программой, успех будет полностью зависеть от Вашей сообразительности, настойчивости и терпения.

С технической точки зрения изучать данные в памяти гораздо менее удобно, чем препарировать исполняемый файл на «винчестере», да и инструментов для интеллектуального поиска данных в чужом адресном пространстве не так уж много. Кроме того, если информация загружается в динамически выделяемые области памяти, исследуемые данные будут от запуска к запуску «плавать» по адресному пространству программы, располагаясь каждый раз по новым адресам. А это будет совсем уж нехорошо – проделывать массу рутинной работы только из-за того, что программу угораздило выделить очередной кусок памяти на сотню байт выше, чем в предыдущий сеанс. Поэтому позаботимся о создании элементарных удобств для работы. Прежде всего нам понадобится перенести наше поле деятельности из чрезвычайно нежной оперативной памяти на гораздо более жесткий диск, где можно будет проводить любые эксперименты не опасаясь, что случайное нажатие «не той» клавиши приведет к потере результатов длительных исследований. Вы наверняка уже догадались, что самым простым решением было бы снятие дампа с нужных областей памяти и сохранение этого дампа на «винчестере». Увы, все далеко не так просто, как хотелось бы.

Во-первых, Вам придется озадачиться поиском подходящего инструмента. Классические дамперы из крэкерского арсенала Вам не помогут, поскольку они предназначены для снятия дампа **программы**, но не данных с которыми эта программа работает. Более того, случайный захват данных для классического дампера – явление крайне нежелательное, поскольку основное назначение дамперов – распаковка программ и получение работоспособного EXE-файла с минимальным количеством избыточной информации внутри, а не анализ «мусора», перемалываемого программой в процессе ее работы. А вот нам нужен именно этот «мусор». Поэтому придется либо писать программу для «правильного» дампинга своими силами, либо извлекать нужные данные вручную при помощи отладчика, позволяющего сохранять содержимое кусков памяти в файлах.

Во-вторых, данные могут быть разбросаны по адресному пространству программы, но сдампить их нужно за один сеанс отладки. Не так уж редки программы, в которых часть информации хранится, к примеру, в секции инициализированных данных, а другая часть – в динамически выделяемых блоках памяти, и если сдампить содержимое динамической памяти программы, но забыть про

инициализированные данные, такой дамп скорее всего можно выбросить. Нет никакой гарантии, что при следующем запуске, когда Вы выясните, что указатели на динамические блоки находятся в секции инициализированных данных, и что эту секцию тоже нужно дампить, эти указатели будут указывать на те же адресам, что и в прошлый сеанс работы. Поэтому я настоятельно не рекомендую экономить место на диске, и делать полный снимок всех секций программы, поскольку лишнее всегда можно выкинуть, а вот недостающие данные взять будет неоткуда.

И в-третьих, снимая дамп, никогда не забывайте записывать базовые адреса тех кусков памяти, которые Вы дампите – в самом ближайшем будущем они Вам определенно понадобятся.

Но допустим, что мы аккуратно сделали полный снимок подопытной программы, и теперь все ее секции аккуратно разложены на нашем винчестере в идеальном порядке. Что дальше? Возьмите хороший шестнадцатиричный редактор и загрузите в него какую-либо из секций. Теперь настройте этот редактор так, чтобы вместо смещений в файле он показывал смещения относительно базового адреса этой секции в памяти. То есть если Вы сбрасывали на винчестер кусок памяти с адреса 401000h по 402000h, после соответствующей настройки смещение первого байта файла должно отображаться именно как 401000h, а не как 0. В частности, такую операцию умеет выполнять HIEW: для этого необходимо нажать Ctrl-F5 и ввести новую базу. Если Ваш шестнадцатиричный редактор делать такие фокусы не умеет, значит, Вы выбрали недостаточно хороший шестнадцатиричный редактор и Вам будет заметно сложнее постигать разверзнувшиеся перед Вами глубины программы. Возможно даже, что несовершенство Вашего инструментария подвигнет Вас на написание нового, уникального шестнадцатиричного редактора с доселе невиданными возможностями – великие дела совершались по куда менее значительным поводам, чем отсутствие подходящего инструмента для копания в кодах. В принципе, можно обойтись даже без автоматического пересчета базового смещения, но тогда Вам придется проделывать необходимые вычисления в уме, и за всей этой шестнадцатирично-арифметической рутиной Вы можете не прочувствовать до конца всю силу и эффективность предлагаемого метода.

После того, как Вы проделаете все, о чем я говорил, внимательно посмотрите на экран монитор, включите на полную мощность свое воображение и представьте, что Вы разглядываете не кучу байтов, тонким слоем рассыпанных по поверхностям жесткого диска, а мгновение из жизни программы, которое Вы вольны сделать сколь угодно долгим. И то, что Вы видите в окне шестнадцатиричного редактора, по сути, ничем не отличается от того, что Вы бы увидели в окне отладчика, разглядывая память «живой» программы. Вы точно так же можете, следуя указателям, бродить по адресному пространству, дизассемблировать куски кода, искать константы и переменные (поскольку «замороженная» переменная есть ни что иное, как константа) по их значению – в общем, делать с программой все то, о чем я говорил в двух предыдущих главах.

Однако помимо «честных» методов поиска, требующих хотя бы минимального знания о структуре и типизации искомых данных, есть еще один нехитрый прием, не требующий ничего, кроме терпения. Суть метода проста: Вы вырываете из файла на жестком диске небольшой кусок и пытаетесь найти точно такой же кусок в памяти программы. При некотором везении в памяти идентичный кусок обнаружится, а его местоположение укажет Вам, куда программа загрузила соответствующие байтики из файла. После этого Вы можете попытаться логически проанализировать наблюдаемую картину либо просто влепить аппаратную точку останова на чтение всей прилегающей памяти (это будет очень, очень большая «точка») и посмотреть, что будет делать с данными подопытная программа. Несмотря на то, что такой поиск внешне сильно напоминает пресловутый «метод научного тыка», в его основе лежит вполне логичная идея: если информация из файла переносится в память без потерь и существенных изменений, соответствующие элементы структур в файлах и в памяти будут идентичны. Проще говоря если где-то в файле хранилось двухбайтное число 12345, есть вероятность, что оно и в памяти будет выглядеть двухбайтным числом 12345. Хотя, конечно, вполне возможны программы, загружающие числа типа «байт», но обрабатывающие их как 10-байтные с плавающей точкой. Разумеется, этот метод можно усовершенствовать, заметно

повысив его эффективность: например, брать не случайные куски, а содержащие осмысленный текст – тогда Вы будете знать, что ищете текстовое поле, на которое почти наверняка будет существовать указатель, а сам этот указатель скорее всего будет входить в структуру, а структуры будут организованы в массив или список… Как видите, хотя набор базовых приемов не так уж велик, и каждый из них отнюдь не свободен от недостатков и ограничений, но, комбинируя и адаптируя их под особенности конкретных программ, можно весьма многого добиться.

Рассмотренные выше техники добывания данных из недр «живой» программы имели одно общее свойство – по отношению к программе их можно было охарактеризовать как «принуждение». Посудите сами – программа спокойно себе работает, никого не обижает, но тут в ее спокойную и размеренную жизнь врывается крэкер с отладчиком наголо и начинает направо и налево дампить секции и разбрасывать точки останова. Такой подход, конечно, приносит свои плоды – но в некоторых случаях проблемы извлечения данных проще решать не «грубой силой», даже если это сила интеллекта, но хитростью, использованием всевозможных лазеек в коде программы, или даже через нетривиальное использование стандартных средств ОС или самой программы. Практика показала, что если программу вежливо и в изысканной форме попросить, она вполне может поделиться с Вами нужными Вам данными.

В Windows роль вежливых просьб играют системные сообщения – традиционное средство, используемое для огромного количества всевозможных действий – от элементарного закрытия окна до рассылки программам уведомлений о выходе операционной системы из «спячки», иначе именуемой Hibernate. Сила сообщений в Windows весьма велика, и, овладев и правильно распорядившись ей, можно получать весьма интересные результаты. Например, при помощи сообщений можно вытащить все строки из выпадающего списка (ComboBox) или таблицы (ListView), если автор программы забыл предусмотреть в своем детище более традиционный способ сохранения данных. Что для этого нужно? Только документация и некоторые навыки в программировании с использованием WinAPI. А теперь мы плавно перейдем от теории к практике и рассмотрим пример того, как можно применить эту технику для решения конкретной задачи. Но для начала – немного истории.

Как-то раз у меня возникла необходимость получить полный список имен всех сообщений Windows и числовых значений, которые за этими именами скрываются. Задача, надо сказать, была совсем не праздная – этот список был мне жизненно необходим, чтобы включить его в состав моих программ. Но вот незадача – в заголовочном файле из комплекта поставки MASM32 эти имена были разбросаны по всему windows.inc в совершенном беспорядке, и меня совершенно не радовала перспектива проявлять чудеса трудолюбия, вручную выискивая и обрабатывая несколько сотен строк. Полный список, разумеется, можно было бы извлечь из заголовочных файлов последней версии Visual Studio, но, кроме того, что я вообще не являюсь поклонником данного продукта, в частности у меня не было никакого желания искать где-то дистрибутив оной «студии» и устанавливать его ради одного-единственного файла. Однако так уж исторически сложилось, что у меня все-таки была одна небольшая часть Visual Studio – а именно утилитка, именуемая Spy++. Одна из функций этой утилиты заключалась в том, чтобы отслеживать сообщения, которые пользователь указывал в специальном окне, по-научному называемом ListBox. В этом окне как раз и отображался полный список сообщений, среди которых можно было «мышкой» отметить те сообщения, которые требовалось отлавливать. Иными словами, было совершенно очевидно, что Spy++ содержал всю необходимую мне информацию, и требовалось лишь найти способ эту информацию извлечь.

Первой, что пришло мне в голову, это пропустить файл spyxx.exe через утилиту, вычленяющую текстовые строки, и затем выбрать из всех найденных строк имена сообщений. Однако после некоторых размышлений я отверг этот путь: во-первых, мне хотелось получить список сообщений в отсортированным по алфавиту точно в том порядке, в каком они находились в Spy++, а во-вторых, у меня не было желания разбирать ту кучу малу, которую обычно вываливают утилиты поиска

текстовых строк. Поэтому я решил поступить проще: написал программку, которая при помощи сообщения LB_GETCOUNT определяла количество строк в нужном мне ListBox'е, а потом построчно считывала содержимое ListBox'а, в цикле отправляя ему сообщения LB_GETTEXT. Через считанные минуты у меня на винчестере покоился в виде текстового файла полный список сообщений из Spy++. После этого оставалось только извлечь из исполняемого файла числовые значения, соответствующие именам сообщений, что я и сделал при помощи методов, о которых я говорил в предыдущей главе. Если у Вес есть желание попрактиковаться в применении этих методов – можете самостоятельно попробовать извлечь эти данные, особой сложности это не представляет.

Нередко для обработки и отображения данных программисты под ОС Windows используют ActiveX-компоненты, одним из полезных свойств которых является возможность получить доступ к интерфейсам такого компонента без всякой документации и заголовочных файлов. Например, импортировав нужный ActiveX-компонент в Delphi, Вы сразу же сможете увидите свойства и методы, присущие этому компоненту. И, запрашивая значение нужных свойств и вызывая соответствующие методы, Вы скорее всего сможете научиться извлекать данные, которые этот ActiveX отображает. Более того, Вы получите возможность экспериментировать с этим компонентом в «лабораторных условиях» собственных тестовых примеров, имитирующих работу программы, из которой Вы собираетесь вытащить данные, а не непосредственно на «поле битвы» с чужим кодом. Вы можете подумать «ну и какая польза от этих экспериментов – ведь нужные данные находятся в **другой** программе» - но не спешите с выводами. Представьте себе, что Вам удалось внедрить свой код в исследуемую программу и получить доступ к интерфейсам нужного ActiveX… А впрочем, почему только «представьте»? Внедряйте, получайте доступ – и считывайте вожделенную информацию!

И, наконец, не бойтесь пользоваться простейшими методами. Может случиться так, что один из множества инструментов автоматизации, умеющий листать страницы в указанном окне и делать скриншоты, объединенный с программой распознавания текста, поможет Вам получить распечатку защищенного от копирования телефонного справочника быстрее, чем извлечение той же информации из глубин адресного пространства при помощи отладчика. В конце-концов, если информация где-то отображается – значит, ее можно оттуда извлечь и сохранить в желаемом виде – нужно лишь изобрести подходящий метод. Но это уже совсем другая, далекая от крэкинга история.

Защита, использующая хасп-ключ. Часть 3: USB

Автор: Chingachguk / HI-TECH

Дата: 2004-06-20

Пояснение к третьей части

Данная статья является расширением к моим статьям ("Защита, использующая хасп-ключ" и "Защита, использующая хасп-ключ. Часть 2"), посвященным анализу защиты этой программы.

В первой статье был приведен поверхностный анализ этой защиты, исследовалась возможность эмуляции ключа на уровне команд ввода-вывода и необходимые действия для борьбы с приложением, препятствующим перехвату ввода-вывода (регистры отладки DRx).

Во второй части было описано, как именно устроен код, общающийся с ключом и реализующий так называемые "функции" хасп-ключа и исследован метод эмуляции не отклика самого ключа (команды ввода-вывода), но эмуляции функций хасп-ключа.

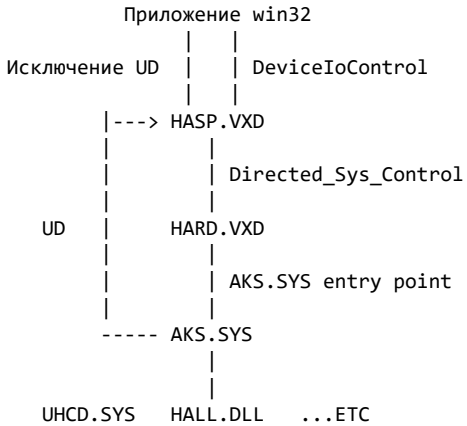
Первая и вторая части были посвящены ключу для LPT-порта, обладающему памятью порядка 128 байт и содержащему одну "секретную" функцию SeedCode. В настоящем документе будет исследовается защита, основанная на другом ключе, использующем USB-порт. Различие этих ключей, конечно, не в том, что они используют различные интерфейсы, но в том, что новый ключ (предположительно Hasp Time 4) способен реализовать по крайней мере две новые функции (кроме новой SeedCode): функцию таймера и функцию типа Decrypt_Data - расшифровка указанной последовательности. Все исследования относительно ключа носят оценочный характер и не претендуют на полноту, однако прога все же запускается ;)

Все исследования проводились в win98.

Общее описание структуры защиты новой версии

Состав защиты. В новой версии участвуют "старые" драйвера: HASP.VXD и HARD.VXD (большой, после установки драйвера защиты новой версии он весит полмегабайта), но есть и новые, причем другого формата: это AKS.SYS (~20 килобайт). Последний является промежуточным звеном между HARD.VXD и драйверами системы: драйверами USB-порта (например, uhcd.sys) и HAL.DLL. HAL.DLL реализует так называемый уровень абстракции устройства: общение с ключом ведется не на уровне команд in/out, но через переходники типа READ_PORT_USHORT и т.д.

Общая схема взаимодействия драйверов может быть представлена следующим образом:



Итак, приложение использует api DeviceIoControl для общения с HASP.VXD и вызывает исключение UD, которое обрабатывает также HASP.VXD. Но если в прежней версии программы исключение UD являлось основным для проверки ключа (см. мою вторую статью) - на этом исключении было

построено что-то вроде api-функций 01,02,..0Bh, причем функция 0Bh этого исключения именно проверяла ключ и получала из него данные, то в последней версии исключению UD отводится вспомогательная роль: используются подфункции 01..07, занимающиеся за/расшифровкой блоков данных по достаточно тривиальным алгоритмам:

```
...
; Расшифровка тела запроса по способу UD
  pushad
; eax=lpvOutBuffer
  mov  eax,[esi].lpvOutBuffer
; Расшифровать буфер по способу F=06 #6
  mov  ecx,28h
  lea  ebx,[eax+14h]
@@DecryptOutBuffer:
  xor  byte ptr [ebx],0AAh
  ror  byte ptr ds:[ebx],4
  inc  ebx
  loop @@DecryptOutBuffer
...
; Зашифровать буфер по способу F=04 #6
  mov  ecx,28h
  lea  ebx,[eax+14h]
@@EncryptOutBuffer:
  rol  byte ptr ds:[ebx],4
  xor  byte ptr ds:[ebx],0AAh
  inc  ebx
  loop @@EncryptOutBuffer
  popad
```

Но общение приложения через api DeviceIoControl является основным (или еще не все проверено). Далее HASP.VXD транслирует запрос DeviceIoControl напрямую в HARD.VXD. Таким образом, в новой версии этому драйверу отводится вспомогательная роль.

HARD.VXD занимается непосредственно обработкой таких запросов, реализует сам некоторую часть функций ключа, но и часть работы переадресует "драйверу USB-ключа" - AKS.SYS. Сам HARD.VXD предположительно не выполняет никаких обращений к портам ключа, прямо или косвенно (READ_PORT_USHORT и т.д.). Этот драйвер отличает крайняя степень "антиотладочного" кодирования: обилие команд типа xchg ebp,edi+xchg edi,ebp или add eax,0 и т.п., впрочем никак не мешающих трассировке в Soft-Ice, но делающих неудобным исследование.

AKS.SYS по-видимому, выполняет самую низкоуровневую реализацию функций ключа: именно он работает с системным драйвером USB-порта (UHCD.SYS). Скорее всего он реализует (частично) функцию ключа Decrypt_Data. Этот драйвер выгодно отличается от предидущего размером (выше) и отсутствием антиотладочных приемов. Впрочем, в нем есть функции вызова исключения UD (из HASP.VXD - на рисунке это показано обратной стрелочкой), но только вот эти функции почему-то не вызываются.

Немного про драйвер USB, работающий с ключом, хотя непосредственного отношения к защите он не имеет. Драйвер реализует два асинхронных сервиса, которые переодически вызываются VMM. В них он опрашивает ключ. Исследования лога на команды ввода-вывода (лог может быть безболезненно получен как перехватом in/out через DRx так и перехватом READ_PORT_USHORT) показывают, что ключ выдает что-то подобное значению инкрементального 11-ти битного счетчика с приращением в пределах 100-110 на одно чтение. Эмуляция этого счетчика мне пока не удалась. Видимо, этот счетчик и является функцией Time этого типа ключа. Попытка подстановки заранее записанного лога была выполнена успешно, однако такая эмуляция работает лишь при вставленном ключе ;) Это может свидетельствовать о сигнале irq при вытаскивании ключа и обрабатываемым отдельным сервисом не через основные порты ключа (например, 900x). Факт отсутствия ключа в этом случае может быть проверен драйвером защиты через другие каналы, но в данной статье эта возможность не исследовалась.

Таким образом основным "узким местом" и - самое главное - предположительно единственным является общение приложения с драйвером HASP.VXD через DeviceIoControl. Именно на этом участке будет сосредоточено все внимание в дальнейшем.

Описание протокола DeviceIoControl

Перехват и логгирование. Не представляет трудностей; было выполнено через врезку jmp (jnz) внутри Control_Proc HASP.VXD:

```
; VxD entry point

010B: 83F807         cmp         eax,007
010E: 0F84ECFEFFFF   je         .000000000
0114: 83F809         cmp         eax,009
...
0126: 83F802         cmp         eax,002
0129: 0F84EBFEFFFF   je         .00000001A
012F: 83F823         cmp         eax,023
0132: 0F8466FFFFFF   je         .00000009E  <<< Change to jz Our_Trap
0138: 83F81B         cmp         eax,01B
013B: 74AF           je         .0000000EC
013D: 83F81C         cmp         eax,01C
0140: 74B9           je         .0000000FB
```

Эта область драйвера (Control_Proc VXD) не проверяется защитой на перехват (в отличии от тела драйвера: см. вторую статью). Сообщение 23h (DeviceIoControl) перехватывается драйвером-эмулятором. Перехват выполняется следующим образом: эмулятор перехватывает сообщения Directed_Sys_Control стандартным образом при запуске (через VMM) и ожидает сообщения, адресованного VMM драйверу HASP.VXD (фильтр на ID). Такое сообщение будет послано VMM при загрузке драйвера через CreateFile (сообщение DIOC_OPEN). После этого выполянется перехват обработки DeviceIoControl.

После реализации перехвата начинаем записывать лог общения приложения через DeviceIoControl. Напомню вкратце основные параметры этого сообщения: при вызове функции F23 драйвер VXD получает следующую структуру по esi:

```
DIOCParams STRUC
Internal1       DD ?
VMHandle  DD ?
Internal2  DD ?
dwIoControlCode DD ?
lpvInBuffer     DD ?
cbInBuffer      DD ?
lpvOutBuffer    DD ?
cbOutBuffer     DD ?
...
```

Приведены не все поля, но даже из приведенных нас будет интересовать только два: dwIoControlCode - код подфункции функции F23 и lpvOutBuffer - адрес буфера, в который VXD может записать какие-то данные.

Исследования лога показало, что приложение вызвает этот сервис с следующими значениями dwIoControlCode: 2,3 и 4. Причем буфер ответа (lpvOutBuffer) модифицируется драйвером только в случае dwIoControlCode=2. В остальных случаях (3,4) этот буфер не изменяется или указатель на него не передан. Естественным является попытка простейшей эмуляции подфункций 3 и 4: просто возврат с успешным результатом (eax=0, clc) и это ... срабатывает !

Идем далее. Анализируем только подфункцию 2. Обнаруживаем, что во всех случаях размер буфера ответа одинаков и равен 48h (поле cbOutBuffer). Делаем записи этой длины буфера до и после выполнения обработки его HASP.VXD.

Анализ запросов/ответов подфункции 2 показал, что структура пакета длиной 48h может быть описана примерно так:

```
;
; Структура пакета запроса F23, подфункция 2
;
tF23Packet struc
; Неизменяемая часть
  Type1       dd ? ; Всегда 1
  Type2       dd ? ; Всегда 1
  MagConst1   dd ? ; 6C687368h = "hsh1"
  Len1        dd ? ; 48h - длина пакета ?
  Len2        dd ? ; 28h - длина шифрованной части
; Encrypted data, зашифрована по способу функции 6 исключения UD (28h bytes)
  MagConst2   dw ? ; 1D4Ch
  FuncNum     dw ? ; Номер подфункции в виде WORD (01/03/05/06/32h/3ch/3dh)
  F32Data     dd ? ; 0 для всех, кроме 32h - для нее: FC20h,FBD4h..
  F3_6_32Data dd ? ; C9h для подфункций 03,06,32h
  Const1      dd ? ; Типа CDA03B6Dh - возможно, адрес в VXD (VTD.VXD ??)
  Const2      dd ? ; 818070CBh - возможно, адрес в win32
; Changed data - то, что код драйвера может модифицировать
  Var1        dd ? ; 01,05: 0->1
                   ; 06:    0->DB3Fh
                   ; 32:    4->4,0->0 (not changed)
                   ; 03:    26h->26h, 3->3 (not changed, seed argument)
                   ; 3c,3d: ->0
  Var2        dw ? ; 01,05: 0->1
                   ; 06:    0->7D95h
                   ; 32:    5->5 (not changed, seed argument)
                   ; 03:    0->7875h, 0->3800h (seed 16 bits)
                   ; 3d:    size of encrypted data
                   ; 3c:    -> Var7
  Var3        dw ? ; 01,03,05,06: 2742h->0
                   ; 32:    2742h->2742h (not changed)
  Var4        dd ? ; 01,32: 0->0 или C9h->0
                   ; 05:    0->C9h
                   ; 03,06: C9h->0
  Var5        dd ? ; 01,03,06,32: 0->0 (not changed)
                   ; 05: 0->1F4Ah
  Var6        dd ? ; 0 - const
  Var7        dd ? ; 01,03,05,06: 0->0 (not changed)
                   ; 32: 0->1, 1->5 (Var7:=Var2)
  StackAddr   dd ? ; Address in stack (0069FC20h) for f32 or f3d
tF23Packet ends
```

Итак, что видно из этого пакета ? Во-первых, общение реализовано через подфункции (поле FuncNum), которые могут быть следующими: 1,3,5,6,32h,3ch,3dh. Следовательно, дальнейший разбор можно проводить для каждой функции отдельно. Следует заметить, что часть пакета приходит в драйвер в зашифрованном виде: это 28h байтов начиная с поля MagConst2. Шифрование выполняется приложением перед вызовом api DeviceIoControl через вызов исключения UD (см. выше). Драйвер HASP.VXD получает управление и вызывает уже внутри себя собственный обработчик исключения UD для дешифровки.

Анализ пакетов для различных подфункций выполнялся следующим образом. Допустим, есть пакет для подфункции 01. Изучаем, что изменяет в ответном пакете эта функция. Обнаруживаем, что она меняет поля Var1, Var2, Var3 и Var7 в ноль, остальные поля не трогает или заполянет их нулями. Делаем то же самое в эмуляторе, но управление уже не передаем в HASP.VXD и убеждаемся, что это работает ! Таким образом были пройдены функции 1,5,6 и 3c. В случае других функций ситуация была несколько сложнее: например, функция 3 возвращала от раза к разу различные поля Var2 и входные значения Var1 были всякий раз различными. Однако можно было проследить однозначность значений этих полей, т.е. Var2=f(Var1), размер - word. На основе исследований ключа на LPT легко экстраполировать функцию SeedCode того ключа на этот случай; так и оказалось: функции 3 и 32h реализуют некую NewSeedCode на основе таблицы ~40h word'ов. Только функция 3 отвечает одним word из этой таблицы, т.е. f3=TableSeed(Var1), а функция 32h возвращает указанное в Var2 число Seed'ов по адресу, расположенному в поле StackAddr пакета (в область приложения). Полную таблицу Seed'ов я получил собственным вызовом драйвера со всеми

возможными значениями входного аргумента.

Особый случай представляет функция 3d. Эта функция выполняет дешифрацию байт длиной, указанной в поле Var2 пакета по адресу в поле StackAddr. Длины для дешифрации бывают только 8 или 10h байт.

Что же за алгоритм дешифрации применила защита ? Может быть все: и DES, и самоделка и все, что угодно. Однако до сих пор мы не лезли с "ключом на двенадцать" в драйвера, а получили информацию изучая логи, что гораздо элегантнее. Однако кое-что мы можем узнать и без реверсинга, а только лишь задавая входные данные для функции шифрования.

Первым делом убеждаемся, что шифрование выполняется в режиме CBC, или, другими словами, расшифровка текущих байт зависит от того, как были расшифрованы предидущие а не в режиме наложения ксорирующей последовательности. Это можно выполнить так: подать на вход байты 00 00 и 00 FF и сравнить отклик функции шифрования.

Выясняем размер блока шифрования (если это блочный алгоритм). Допустим, длина блока 64 бита, тогда первые 8 байт должны расшифровываться независимо от того, чему равны предидущие: например, 8-мь нулей и 00 должны быть расшифрованы так же, как и 8-мь нулей и FF (только первые 8 байт, конечно). На первый взгляд это именно так, но !! - расшифровка следующих 8-ми байт происходит ... независимо от их входных значений ! Т.е. первые 8-мь входных байт являются "ключом", он же (измененный) укладывается назад и последующие байты формируются только на основе первых 8-ми.

Да, пожалуй - все. Приходится лезть в драйвер за информацией о способе шифровки. Довольно быстро, поставив bpm на данные, переданные для дешифрации HASP.VXD в HARD.VXD получаем несколько бряков. Обнаруживаются следующие узлы шифрования:

```
;
Первый цикл: esi = указатель на 8-мь входных байт

  mov  dword ptr Count,0
  mov  esi,offset TestBytes
@@Decrypt1:
  mov  edi,[esi]
  mov  edx,edi
  xor  edx,05B2C004Ah
  mov  ecx,dword ptr Count
  mov  eax,0FFFFFFF9h
  sub  eax,ecx
  and  eax,1Fh
  mov  ecx,20h
  mov  ebx,edx
  sub  ecx,eax
  shr  ebx,cl
  mov  ecx,eax
  mov  eax,[esi+4]
  shl  edx,cl
  mov  [esi+4],edi
  or   ebx,edx
  xor  ebx,eax
  mov  [esi],ebx
  add  dword ptr Count,5
  cmp  dword ptr Count,1Eh
  jb   @@Decrypt1

;
Второй цикл: esi = указатель на 8-мь входных байт

.data?
Tmp dd ?
.code

  mov  esi,offset TestBytes
  mov  eax,[esi]
```

```
    mov  ebx,[esi+4]
    mov  dword ptr [esi],ebx ^ HASP_Decrypt_Data[eax] ; ?
    mov  [esi+4],eax

    xor  edi,edi
  @@Decrypt2:
    mov  edx,[esi]
    mov  eax,0Ah
    sub  eax,edi
    mov  Tmp,edx
    and  eax,1Fh
    xor  edx,803425C3h
    mov  ecx,20h
    mov  ebx,edx
    sub  ecx,eax
    add  edi,2
    shr  ebx,cl
    mov  ecx,eax
    shl  edx,cl
    mov  ecx,Tmp
    or   ebx,edx
    mov  edx,[esi+4]
    xor  ebx,edx
    cmp  edi,0Ch
    mov  [esi],ebx
    mov  [esi+4],ecx
    jb   @@Decrypt2

  ; Final
    mov  esi,offset TestBytes
    mov  eax,[esi]
    mov  ebx,eax
    xor  ebx,HASP_Decrypt_Data[eax]
    mov  [esi],ebx
    mov  [esi+4],eax
  ; В [esi] - первые 8-мь расшифрованных данных
```

Обращаю внимание на то, что это лишь часть алгоритма - есть еще два вызова HASP_Decrypt_Data. Эту функцию с большой вероятностью реализует уже не HARD.VXD, но AKS.SYS. Возможно для этой функции не требуется большого объема данных для эмуляции (как для SeedCode/NewSeedCode) или вообще она реализуется минимальными средствами. По крайней мере в некоторых источниках я видел изображение этого ключа с подписью "ключ взломан" (сайт MeteO) - вполне вероятно, что эмулировать HASP_Decrypt_Data и означает окончательно добить эту защиту.

Однако эмуляция этой функции может быть реализована гораздо проще ! Анализ логов показал, что эта функция вызывается не более 8-ми раз (!), да и то при запуске программы. Вполне возможно, что при открытии очередного модуля будут сделаны и другие вызовы, но ! никто не мешает пройти их по очереди или выполнить полный цикл каких-либо штатных действий до полного выписывания всех запросов функции 3d. Однако в первом приближении я не заметил таких вызовов; вызываются всякие разные успешно эмулируемые функции, но не сильнейшая. Интерес представляет также поиск сигнатур в исполняемом модуле на предмет обнаружения всех возможных данных, которые могут быть переданы для дешифрации функции 3d: допустим, всегда эти данные заканчиваются 8-мью нулями и т.п.

Небольшое заключение

Этот следующий виток развития этой защиты определенно будет представлять интерес, но скорее всего в некотором будущем, ибо пользователи пока довольно настороженно относятся к новой версии этой программы.

Спасибо всем прочитавшим :)

Полный текст эмулирующего драйвера и приложения

```
;
; ***************Инклюд-файл********************
;
MAX_LOG_RECORDS equ 200
OpCodesSize equ 4

; Packet for F23 trapping
PacketF23 equ 0
InBufferSize  equ 48h
OutBufferSize equ 48h ; 15E8h
Func3dArgSize equ 08h
Func3dAnsSize equ 10h
tLogF23Packet struc
  PacketType        db ?   ; Тип пакета
  FuncNum           dw ?   ; Номер подфункции в виде WORD (01/03/05/06/32h/3ch/3dh)
  RegEAX            dd ?   ; Регистр EAX
  dwIoControlCode   dd ?   ; Код функции от приложения
  lpvInBuffer       dd ?   ; Указатель на данные, переданные приложением
  cbInBuffer        dd ?   ; Размер переданных данных
  lpvOutBuffer      dd ?   ; Указатель на данные для ответа VxD
  cbOutBuffer       dd ?   ; Размер данных для ответа (max)
  lpcbBytesReturned dd ?   ; Сколько байтов вернули
  OutBuffChanged    db ?   ; Изменен буфер обмена
  OutBufferBefore   db OutBufferSize dup(?)
  OutBuffer         db OutBufferSize dup(?)
  F32Arg            dd ?   ; Аргумент функции F32
  F32Seed           dd ?   ; Ответ функции F32
  ; Данные подфункции f3d
  Func3dArg         db Func3dArgSize dup(?)
  Func3dAns         db Func3dAnsSize dup(?)
  Func3dNotEmul     db ?   ; Успешно эмулирована функция f3d
tLogF23Packet ends

; Packet for UD trapping
PacketUD equ 1
tLogPacket struc
  PacketType db ?     ; Тип пакета
  RegEAX     dd ?     ; Регистр EAX
  RegEDI     dd ?     ; Регистр EDI
tLogPacket ends

; Размер пакета (= максимальный размер из всех пакетов)
LogPacketSize equ sizeof(tLogF23Packet)

DeviceBig_ID  equ 3543h
DeviceLitt_ID equ 3CDCh ; ID of little VxD

NumberOfUDTraps equ 17 ; Число всех вызовов исключения UD с функцией 0Bh

SelfCallUD6 equ 40666666h ; Номер собственной функции для UD6

;
; Структура пакета запроса F23, подфункция 2
;
tF23Packet struc
; Неизменяемая часть
  Type1       dd ? ; Всегда 1
  Type2       dd ? ; Всегда 1
  MagConst1   dd ? ; 6C687368h = "hshl"
  Len1        dd ? ; 48h - длина пакета ?
  Len2        dd ? ; 28h - длина шифрованной части
; Encrypted data, зашифрована по способу функции 6 исключения UD (28h bytes)
  MagConst2   dw ? ; 1D4Ch
  FuncNum     dw ? ; Номер подфункции в виде WORD (01/03/05/06/32h/3ch/3dh)
  F32Data     dd ? ; 0 для всех, кроме 32h - для нее: FC20h,FBD4h
  F3_6_32Data dd ? ; C9h для подфункций 03,06,32h
  Const1      dd ? ; Типа CDA03B6Dh - возможно, адрес в VXD (VTD.VXD ??)
  Const2      dd ? ; 818070CBh - возможно, адрес в win32
; Changed data - то, что код драйвера может модифицировать
  Var1        dd ? ; 01,05: 0->1
                   ; 06:    0->DB3Fh
```

```
                  ; 32:    4->4,0->0 (not changed)
                  ; 03:    26h->26h, 3->3 (not changed)
  Var2        dw ? ; 01,05: 0->1
                  ; 06:    0->7D95h
                  ; 32:    5->5 (not changed)
                  ; 03:    0->7875h, 0->3800h
  Var3        dw ? ; 01,03,05,06: 2742h->0
                  ; 32:    2742h->2742h (not changed)
  Var4        dd ? ; 01,32: 0->0 или C9h->0
                  ; 05:    0->C9h
                  ; 03,06: C9h->0
  Var5        dd ? ; 01,03,06,32: 0->0 (not changed)
                  ; 05: 0->1F4Ah
  Var6        dd ? ; 0 - const
  Var7        dd ? ; 01,03,05,06: 0->0 (not changed)
                  ; 32: 0->1, 1->5 (Var7:=Var2)
  StackAddr   dd ? ; Address in stack (0069FC20h) for f32
tF23Packet ends

;
; **************Драйвер*******************
;
; Программа - перехват исключения UD
; Coded by Chingachguk. 2004.
;
.586p
include vmm.inc
include vwin32.inc
DECLARE_VIRTUAL_DEVICE UD1C,1,0, UD1C_Control,\
     UNDEFINED_DEVICE_ID, UNDEFINED_INIT_ORDER
Begin_control_dispatch UD1C
 Control_Dispatch w32_DeviceIoControl, OnDeviceIoControl
        Control_Dispatch DESTROY_PROCESS, OnDESTROY_PROCESS
End_control_dispatch UD1C
; Сегмент данных VxD
include 1c06.inc
VxD_LOCKED_DATA_SEG
; Буфер лога
LogPtr               dd 0 ; Указатель лог-буфера
LogPackets           db (MAX_LOG_RECORDS * LogPacketSize) dup (?)

old_ud06handler      dd 0 ; Адрес обработчика UD в VxD
old_14handler        dd ? ; Адрес обработчика PageFault в VxD

DDB                  dd ? ; Адрес Device Block VxD

; Переменные для перехвата Control_Proc VxD
F23Addr              dd 0 ; Адрес начала проверки cmp  eax,23h
F23JmpFound          dd 0
F23EntryPoint        dd ?
ContinuePoint        dd ?
RestoreJmpArg        dd ? ; Аргумент jmp - saved

DirectedSysLocked              dd 0 ; Флаг перехвата функций VxD
pPrevHook_Directed_Sys_Control dd ?

UD14EntryPoint       dd 0 ; Точка входа в #14
UD6EntryPoint        dd 0 ; Точка входа в #UD6

Hook14Count          dd 0 ; Число вызовов исключения Page Fault
HookUDCount          dd 0 ; Число вызовов UD6

SaveSEHProc          dd ? ; Адрес начального обработчика SEH

StepNum              dd ? ; Номер блока шагов по 126

CounterCalls14Step2  dd 0 ; Отладка - счетчик вызовов #14 на шаге 2
CrcProcHookAddr      dd ? ; Адрес точки перехвата crc-процедуры
AfterCrcProcHookAddr dd ? ; Адрес возврата после перехвата crc-процедуры

LastPacketPtr        dd ? ; Указатель на последний пакет при перехвате F23
```

```
NeedForExecuteF23    dd ? ; Флаг необходимости вызова оригинальной F23

; Таблица для подфункции f32
f32Table label word
dw 00001h ;00
dw 00000h ;01
dw 00000h ;02
dw 03800h ;03
dw 06a02h ;04
dw 0fedbh ;05
dw 06b1eh ;06
dw 01be4h ;07
dw 0b0dbh ;08
dw 06e0dh ;09
dw 0d84ch ;0a
dw 0ceb6h ;0b
dw 0ec39h ;0c
dw 003cdh ;0d
dw 0d155h ;0e
dw 039d1h ;0f
dw 0de19h ;10
dw 0a6ddh ;11
dw 08f5bh ;12
dw 0ea67h ;13
dw 02984h ;14
dw 0b4ech ;15
dw 069bch ;16
dw 096a6h ;17
dw 08cb3h ;18
dw 0090eh ;19
dw 0fd37h ;1a
dw 06e4eh ;1b
dw 04cb6h ;1c
dw 03e1dh ;1d
dw 03fb6h ;1e
dw 0a864h ;1f
dw 0c9f8h ;20
dw 09f29h ;21
dw 0c80ah ;22
dw 065cah ;23
dw 02a04h ;24
dw 06194h ;25
dw 07875h ;26
dw 0393fh ;27
dw 0ae21h ;28
dw 04ebfh ;29
dw 04bbah ;2a
dw 01826h ;2b
dw 0bf20h ;2c
dw 0e760h ;2d
dw 05736h ;2e
dw 0e6e0h ;2f
dw 04e24h ;30
dw 081b1h ;31
dw 03e6eh ;32
dw 0e436h ;33
dw 011a6h ;34
dw 0135eh ;35
dw 00d07h ;36
dw 05052h ;37
dw 05052h ;38
dw 05052h ;39
dw 05052h ;3a
dw 05052h ;3b
dw 05052h ;3c
dw 05052h ;3d
dw 05052h ;3e
dw 05052h ;3f
dw 05052h ;40
dw 05052h ;41
dw 05052h ;42
```

```
dw 05052h ;43
dw 05052h ;44
dw 05052h ;45
dw 05052h ;46
dw 05052h ;47
dw 05052h ;48
dw 05052h ;49
dw 05052h ;4a
dw 05052h ;4b
dw 05052h ;4c
dw 05052h ;4d
dw 05052h ;4e
dw 05052h ;4f

Func3dArgsAnswersSize equ 8
Func3dArgsAnswers label byte

  db 0dbh,0feh,01eh,06bh,0e4h,01bh,0dbh,0b0h
  db 095h,078h,04bh,035h,05eh,0bch,0dch,048h,06ch,08ah,0e2h,00dh,022h,0bah,0f8h,0c3h

  db 05eh,022h,0e1h,05eh,04eh,0c8h,003h,02fh
  db 08bh,00eh,068h,014h,0beh,044h,000h,056h,027h,0d7h,0bah,0a4h,0fah,0d7h,01eh,0d8h

  db 080h,030h,0d4h,000h,0a8h,052h,006h,01eh
  db 050h,08ah,02dh,01bh,000h,05bh,0dah,0e7h,000h,000h,000h,000h,000h,000h,000h,000h

  db 0b8h,052h,003h,027h,080h,030h,0d4h,000h
  db 0a6h,016h,0ach,0eeh,08bh,0b9h,0efh,059h,000h,000h,000h,000h,000h,000h,000h,000h

  db 0b5h,019h,0cch,006h,097h,021h,0b9h,097h
  db 0e8h,09fh,02fh,0ffh,0ffh,08bh,0f0h,085h,093h,04ch,07dh,07bh,0cdh,061h,0a0h,030h

  db 0ech,059h,02dh,01eh,0fch,040h,008h,001h
  db 0ffh,0eah,0b4h,0aeh,0beh,02ch,050h,024h,000h,000h,000h,000h,000h,000h,000h,000

  db 034h,05dh,0cdh,000h,00ch,0fch,069h,000h
  db 03dh,06fh,0aah,0b6h,060h,0b8h,073h,029h,000h,000h,000h,000h,000h,000h,000h,000

  db 007h,0d4h,071h,05bh,0bbh,0b7h,0e3h,02dh
  db 0e8h,093h,031h,0ffh,0ffh,08bh,0f0h,085h,093h,04ch,07dh,07bh,08eh,090h,0b9h,0e3h

VxD_LOCKED_DATA_ENDS
; А вот наш сегмент кода.
VxD_LOCKED_CODE_SEG
;
; Сообщение DESTROY_PROCESS:
;
BeginProc OnDESTROY_PROCESS ; деинсталлируем все перехваты
; Инсталляция ловушек уже была ?
  cmp  dword ptr ds:DirectedSysLocked,0
  jz   @@ContinueHook

; Убираем перехват.

; Деинсталляция перехватчиков (UD)
  call Uninstall_Handlers

; Мы опять готовы приступить к перехвату.
  mov  dword ptr ds:DirectedSysLocked,0

@@ContinueHook:

  xor  eax,eax
  clc
  ret
EndProc   OnDESTROY_PROCESS
;
; Сообщение DeviceIoControl
;
BeginProc OnDeviceIoControl
; Сделаем так, чтобы esi был адресом переданной нам структуры - DIOCParams.
```

```
  assume esi:ptr DIOCParams
; При загрузке VxD в память система позовет ее с контрольными сообщениям
.if [esi].dwIoControlCode==DIOC_Open ; Контрольное сообщение ?!

;
; Установка перехватчика Directed_Sys_Control:
;
; Он ожидает сообщения от VMM для Hasp.vxd и перехватывает обработчик #06
;
; Перехватываем функцию VMM "Directed_Sys_Control"
; ***
; INT 20 P - Microsoft Windows - 0147h "Directed_Sys_Control"
; VxD = 0001h
; 0147h "Directed_Sys_Control"
  mov  eax,00010147h
  mov  esi,offset32 Hooked_Directed_Sys_Control
  int  20h   ; Call VxD
  dw   0090h  ; 0090h hook device service
  dw   0001h   ; ID VMM
  mov  eax,-1
  jnc  @@HookDirected_Sys_ControlOK
  xor  eax,eax
@@HookDirected_Sys_ControlOK:

  xor  eax,eax ; Надо отвечать: eax=0.
;
; Сигнал завершения работы
;
.elseif [esi].dwIoControlCode==DIOC_Closehandle

; ***
; Освобождаем "Directed_Sys_Control"
; ***
  mov  eax,00010147h
  mov  esi,offset32 Hooked_Directed_Sys_Control
  int  20h   ; Call VxD
  dw   011Ch  ; 011Ch unhook device service
  dw   0001h   ; ID VMM

;
; Запрос на выдачу буфера лога
;
.elseif [esi].dwIoControlCode==1

; Получим адрес переданной нам структуры InBuffer в регистр ebx
  mov  ebx,[esi].lpvInBuffer
  mov  edi,[ebx] ; указатель на буфер, который нам передал win32-код

  mov  esi,offset32 LogPtr
  cld
  lodsd
  stosd
  imul eax,eax,LogPacketSize
  mov  ecx,eax
  test eax,eax
  jz   @@NoLogData
  rep  movsb

@@NoLogData:

  call Uninstall_Handlers

  xor  eax,eax ; Вернем в eax флаг ошибки
.endif
  ret
EndProc OnDeviceIoControl
;
; ************************SUBPROGRAMS*********************************
;
;
; подпрограмма - перехватчик функции 23h - DeviceIOControl:
```

```
; ожидает сообщения от VMM для hasp.vxd и перехватывает ud6 и F23
;
BeginProc Hooked_Directed_Sys_Control, HOOK_PROC, pPrevHook_Directed_Sys_Control, LOCKED
  push ebp
  mov  ebp,esp
  pushfd                  ; Remember, hooks must preserve all regs
  pushad                  ;

; Если обращение к Big, то возвращаем OK
;  cmp  word ptr [ecx+6],DeviceBig_ID
;  jnz  @@NotBigVXD
;
;  popad
;  popfd
;  pop  ebp
;  xor  eax,eax
;  clc
;  ret

@@NotBigVXD:

; Нужно ли мониторить этот вызов ?
  cmp  word ptr [ecx+6],DeviceLitt_ID
  jnz  @@ItNotIsOurVxD

; Инсталляция обработчиков ud06 и F23 (если возможно)
  cmp  dword ptr ds:DirectedSysLocked,0
  jz   @@HookFunctions

  jmp  @@AlreadyHooked

@@HookFunctions:

  mov  dword ptr ds:DirectedSysLocked,1

;
; Инсталлируем перехватчик #6:
;
;
;9B4D: jmp  9B63 EB 14
;      db 'GenuineDevil' = db 47,65,6E,75,69,6E,65,44,65,76,69,6C
;      db 06,00,01,03,48,24,00,00
;9B63: 1E              push        ds
;9B64: 06              push        es
;9B65: 50              push        eax [ebp+10h]
;9B66: 51              push        ecx [ebp+0Ch]
;9B67: 52              push        edx [ebp+8]
;9B68: 57              push        edi [ebp+4]
;9B69: 55              push        ebp [ebp+0]
;9B6A: 8BEC            mov         ebp,esp
;9B6C: 668CD0          mov         ax,ss
;9B6F: 668ED8          mov         ds,ax
;9B72: 668EC0          mov         es,ax
;9B75: 55              push        ebp
;9B76: 9C              pushfd
;9B77: E800000000      call        000009B7C
;9B7C: 5D              pop         ebp
;9B7D: 81ED780E0000    sub         ebp,000000E78
;9B83: 9D              popfd
;9B84: 0F014D00        sidt        [ebp][00]

; Второй вариант, начало #6
;29AE: EB18            jmps       .0000029C8
;29B0: db 47,65,6E,75,69,6E,65,44,65,76,69,6C
;29BC: 06,00,02,00,94,23,00,00,00,24,00,00
;29C8: 1E              push        ds << jmp Our_Trap_Program
;29C9: 06              push        es
;29CA: 50              push        eax
;29CB: 51              push        ecx
;29CC: 52              push        edx
;29CD: 57              push        edi
```

```
;29CE: 55              push        ebp
;29CF: 8BEC            mov         ebp,esp
;29D1: 668CD0          mov         ax,ss
;29D4: 668ED8          mov         ds,ax
;29D7: 668EC0          mov         es,ax
;29DA: 55              push        ebp
;29DB: 9C              pushfd
;29DC: E800000000      call       .0000029E1
;29E1: 5D              pop         ebp
;29E2: 81EDBD0E0000    sub         ebp,000000EBD
;29E8: 9D              popfd
;29E9: 0F014D00        sidt        [ebp][00]

; Получить адрес control'a vxd
  call GetLitt_DDB
  mov  eax,dword ptr ds:DDB
  test eax,eax
  jz   @@VxDNotFound
  mov  eax,dword ptr [eax+18h]
; Ищем байты начала #06. Это вторая строка 'GenuineDevil' (первая - #14)
  mov  ecx,0FFFFh
  xor  ebx,ebx
@@FindUDEntryPoint:
  cmp  dword ptr ds:[eax],0756E6547h
  jnz  @@NextByteScan
  cmp  dword ptr ds:[eax+4],044656E69h
  jnz  @@NextByteScan
  test ebx,ebx
  jz   @@UD14EntryPointFound
; Найден обработчик UD - int 6
  lea  edx,[eax-2]
  mov  dword ptr ds:UD6EntryPoint,edx
  jmp  @@ScanForUdDone
; Найден обработчик #14 - PageFault
@@UD14EntryPointFound:
  inc  ebx
  lea  edx,[eax-2]
  mov  dword ptr ds:UD14EntryPoint,edx
@@NextByteScan:
  inc  eax
  loop @@FindUDEntryPoint
  jmp  @@NotSetup

; Найдены адреса входа в оба обработчика
@@ScanForUdDone:

; Вписать команду перехода в тело обработчика #6
;;  call Install_Jmp_Into_UD6

;;  jmp  @@SkipF23

; Найти команду перехода на 23 в VxD-обработчике (control proc)
; Это две команды:
; cmp  eax,23h ; 83 F8 23
; jz   @L      ; 0F 84 66 FF FF FF

  mov  eax,dword ptr ds:DDB
  mov  eax,dword ptr [eax+18h] ; Control proc

; Найти команды перехода на инсталлятор
  mov  ecx,0FFFFh
@@FindJmp23:
  cmp  dword ptr [eax],0F23F883h
  jnz  @@PrevByte
  cmp  dword ptr [eax+4],0FFFF6684h
  jnz  @@PrevByte
; Дезактивировать старые команды перехода,
; запомнив их адрес
  mov  dword ptr F23Addr,eax
  mov  dword ptr F23JmpFound,1
  jmp  @@InstallerFound
```

```
@@PrevByte:
  inc  eax
  loop @@FindJmp23
  jmp  @@NotSetup
@@InstallerFound:

; Установить перехватчик функции 23h
; Запомнить EntryPoint функции 23h
  mov  ebx,dword ptr F23Addr
  add  ebx,3 ; Адрес jz @@F23
  lea  eax,[ebx+6] ; Адрес следующей команды после jz @@F23
; Запомнить ее
  mov  ContinuePoint,eax
  add  eax,[ebx+2] ; Входная точка F23
  mov  F23EntryPoint,eax
; Вписать в команду jz @F23 адрес нашего обработчика F23
  lea  eax,[ebx+6] ; Адрес следующей команды после jz @@F23
  sub  eax,offset32 HookControl
  neg  eax ; Размер перехода
; Сохранить ее для восстановления
  mov  edx,dword ptr [ebx+2]
  mov  dword ptr [ebx+2],eax ; Переход на Trap
  mov  dword ptr RestoreJmpArg,edx

@@SkipF23:

; Счетчик вызовов int 6
  mov  dword ptr ds:HookUDCount,0

@@VxDNotFound:
@@NotSetup:

@@AlreadyHooked:
@@ItNotIsOurVxD:
; Продолжение выполнения
  popad
  popfd
  pop  ebp
  jmp  [pPrevHook_Directed_Sys_Control]  ; Chain to previous hook
EndProc Hooked_Directed_Sys_Control
;
; Деинсталляция перехватчиков UD6 и 23h
;
Uninstall_Handlers proc
  pushad
  cmp  dword ptr F23JmpFound,0
  jz   @@WasNotInstalled

; Восстановить команду перехода на функцию 23h
; cmp  eax,23h ; 83 F8 23
; jz   @L      ; 0F 84 66 FF FF FF
; Получение адреса обработчика
  mov  ebx,dword ptr F23Addr
  add  ebx,3
  mov  eax,dword ptr RestoreJmpArg
  mov  [ebx+2],eax

@@WasNotInstalled:

; Восстановить команды в VxD
  call Uninstall_Jmp_Into_UD6

  popad
  ret
Uninstall_Handlers endp
;
; подпрограмма - перехватчик #6 - UD
;
TrapPort proc

; Команды, которые не успел выполнить обработчик #6
```

```
;29C8: 1E             push        ds << jmp Our_Trap_Program
;29C9: 06             push        es
;29CA: 50             push        eax
;29CB: 51             push        ecx
;29CC: 52             push        edx

EntryEAX equ dword ptr [ebp-04h]
EntryEBX equ dword ptr [ebp-10h]
EntryECX equ dword ptr [ebp-08h]
EntryEDX equ dword ptr [ebp-0Ch]
EntryEDI equ dword ptr [ebp-20h]
CrashAdr equ dword ptr [ebp+18h]

  ; EIP        ; [ebp+18h]

  push ds      ; [ebp+14h]
  push es      ; [ebp+10h]
  push eax     ; [ebp+0Ch]
  push ecx     ; [ebp+8h]
  push edx     ; [ebp+4h]

  push ebp     ; [ebp+0h]
  mov  ebp,esp

  pushad

; push eax     ; [ebp-04h]
; push ecx     ; [ebp-08h]
; push edx     ; [ebp-0Ch]
; push ebx     ; [ebp-10h]
; push esp     ; [ebp-14h]
; push ebp     ; [ebp-18h]
; push esi     ; [ebp-1Ch]
; push edi     ; [ebp-20h]

  push ds
  push ss
  pop  ds

; Адрес возврата EIP
  mov  ecx,CrashAdr

; Запомнить байты инструкции (опционально)
  call Store_LogRecord

  cmp  EntryEAX,40000007h
  jnz  @@NotF7

; Эмуляция первого и не только вызова функции 07
; В первом приближении это подсчет crc из [edx] длиной ecx и запись в [edi]
  pushad
  xor  eax,eax
  mov  edi,EntryEDX
  mov  ecx,EntryECX
@@CalcCRC:
  add  al,ds:[edi]
  inc  edi
  dec  ecx
  jnz  @@CalcCRC
  mov  edi,EntryEDI
  mov  ds:[edi],al
  popad
; Возврат в код приложения
  jmp  @@ReturnToFault

@@NotF7:

  cmp  EntryEAX,40000006h
  jnz  @@NotF6
; Эмуляция первого и не только вызова функции 06
; В первом приближении это расшифровка из [edx] длиной ecx xor AAh + ror 4
```

```
  pushad
  xor  eax,eax
  mov  edi,EntryEDX
  mov  ecx,EntryECX
@@DecryptF06:
  xor  byte ptr ds:[ecx+edi-1],0AAh
  ror  byte ptr ds:[ecx+edi-1],4
  dec  ecx
  jnz  @@DecryptF06
  popad
; Возврат в код приложения, но на адрес из edi
; Модификация указателя
  mov  eax,EntryEDI
  mov  CrashAdr,eax
  jmp  @@SimpleReturnToFault

@@NotF6:

  cmp  EntryEAX,40000003h
  jnz  @@NotF3
; Эмуляция первого и не только вызова функции 03
; В первом приближении это расшифровка из [edx] длиной ecx rol 4 + xor AAh

  pushad
  xor  eax,eax
  mov  edi,EntryEDX
  mov  ecx,EntryECX
@@DecryptF03:
  rol  byte ptr ds:[edi],4
  xor  byte ptr ds:[edi],0AAh
  inc  edi
  dec  ecx
  jnz  @@DecryptF03
  popad

; Возврат в код приложения
  jmp  @@ReturnToFault

@@NotF3:

  cmp  EntryEAX,40000001h
  jnz  @@NotF1
; Эмуляция первого и не только вызова функции 01
; В первом приближении это просто возврат + 8
; Во втором - чтение из dr7 700h, запись в dr7 0, но это не обязательно
; Возврат в код приложения
  jmp  @@ReturnToFault

@@NotF1:

  cmp  EntryEAX,40000004h
  jnz  @@NotF4
; Эмуляция первого и не только вызова функции 04
; В первом приближении это расшифровка из [edx] длиной ecx rol 4 + xor AAh

  pushad
  xor  eax,eax
  mov  edi,EntryEDX
  mov  ecx,EntryECX
@@DecryptF04:
  rol  byte ptr ds:[edi],4
  xor  byte ptr ds:[edi],0AAh
  inc  edi
  dec  ecx
  jnz  @@DecryptF04
  popad
; Возврат в код приложения, но на адрес из edi
; Модификация указателя
  mov  eax,EntryEDI
  mov  CrashAdr,eax
  jmp  @@SimpleReturnToFault
```

```
@@NotF4:

  cmp  EntryEAX,40000002h
  jnz  @@NotF2
; Эмуляция первого и не только вызова функции 01
; В первом приближении это просто возврат + 8
; Во втором - запись в dr7 700h, но это не обязательно
; Возврат в код приложения
  jmp  @@ReturnToFault

@@NotF2:

  cmp  EntryEAX,4000000Bh
  jnz  @@NotFB

  jmp  @@ToPrevHandler

@@NotFB:
@@ToPrevHandler:

  pop  ds
  popad
  pop  ebp

  jmp  dword ptr ss:old_ud06handler
;
; Возврат в код, вызвавший исключение
;
@@ReturnToFault:

; Инкримент указателя EIP
  add  CrashAdr,8

@@SimpleReturnToFault:

  pop  ds
  popad
  pop  ebp

; Команды от оригинального обработчика
  pop  edx
  pop  ecx
  pop  eax
  pop  es
  pop  ds
; Флаг RF: resume flag после выполнения исключения - continue
  or   byte ptr ss:[esp+0Ah],1h
  iretd
TrapPort endp
;
; Запомнить байты в лог-буфере
; ecx-> EIP start
Store_LogRecord proc
; Счетчик вызовов int 6
  mov  eax,dword ptr ds:HookUDCount
  inc  dword ptr ds:HookUDCount

;;  cmp  dword ptr ds:HookUDCount,50
;;  jbe  @@ContinueHookUD

; (Номер шага+1) кратен 127 ?
;;  pushad
;;  add  eax,2
;;  xor  edx,edx
;;  mov  ebx,127
;;  div  ebx
;;  mov  dword ptr ds:StepNum,eax
;;  test edx,edx
;;  popad
;;  jz   @@UnhookUD6AndWaiting
;;  jmp  @@StepsDone
```

```
@@UnhookUD6AndWaiting:

;;  cmp  dword ptr ds:StepNum,2
;;  jae  @@HookToCrcProcDone

; Устанавливаем ловушку на #14, ожидающую кода-расшифровщика на стек
; после того, как он найдет код декриптора на стек,
; он запишет после него вызов (наш) #6

;;  call Uninstall_Handlers

@@ContinueHookUD:

  ;;call Install_Jmp_Into_14

@@HookToCrcProcDone:
@@StepsDone:

; Записываем в лог только пакеты от функции 0Bh (отладка, можно убрать)
;;  cmp  EntryEAX,4000000Bh
;;  jz   @@LogFuncB
;;  ret
@@LogFuncB:

; Запоминаем байты и значения регистров команды
  mov  eax,dword ptr ds:LogPtr
  cmp  eax,MAX_LOG_RECORDS
  jae  LogFull
  inc  dword ptr ds:LogPtr

  pushad
  imul eax,eax,LogPacketSize
  mov  ebx,offset32 LogPackets
  add  ebx,eax
  assume ebx: ptr tLogPacket
; Тип пакета
  mov  byte ptr [ebx].PacketType,PacketUD
; Регистр eax
  mov  eax,EntryEAX
  mov  dword ptr [ebx].RegEAX,eax
; Регистр edi
  mov  eax,EntryEDI
  mov  dword ptr [ebx].RegEDI,eax
  assume ebx:nothing
  popad
  ret

LogFull:

  call Uninstall_Handlers

  ret
Store_LogRecord endp
;
; Перехват вызова F 23h VxD
;
HookControl proc
  push ebp
  mov  ebp,esp
  push eax ; [ebp-4]
  push ebx ; [ebp-8]
  push ecx ; [ebp-0C]
  push edx ; [ebp-10]
  push esi ; [ebp-14]
  push edi ; [ebp-18]
  pushad
  push ds
  push es

  push ds
  pop  es
```

```
  assume esi:ptr DIOCParams
; Фильтруем те запросы, которые можно не выполнять, а сразу - ret
  cmp  dword ptr [esi].dwIoControlCode,DIOC_Open
  jz   @@NotHaspEx
  cmp  dword ptr [esi].dwIoControlCode,DIOC_Closehandle
  jz   @@NotHaspEx
; Ответы изменяются только в сообщениях с подфункцией 2
  cmp  dword ptr [esi].dwIoControlCode,2
  jnz  @@NotHaspEx
  jmp  @@ContinueLog

@@NotHaspEx:

; После первого закрытия устройства устанавливаем перехват UD
  cmp  dword ptr [esi].dwIoControlCode,DIOC_Closehandle ; DIOC_Open
  jnz  @@NotDeviceClose
  cmp  dword ptr old_ud06handler,0
  jnz  @@HandlerUDAlreadyInstalled

;;  call Install_Jmp_Into_UD6

@@NotDeviceClose:
@@HandlerUDAlreadyInstalled:

  assume esi:nothing

  pop  es
  pop  ds
  popad
  pop  edi
  pop  esi
  pop  edx
  pop  ecx
  pop  ebx
  pop  eax
  pop  ebp

;;  jmp  dword ptr F23EntryPoint

; Выполняем возврат без перехода в драйвер
  xor  eax,eax
  clc
  ret

@@ContinueLog:

; Возможно, эмулируем работу драйвера - функции F23
  assume esi:ptr DIOCParams
  mov  dword ptr NeedForExecuteF23,0
; Расшифровка тела запроса по способу UD
  pushad
; eax=lpvOutBuffer
  mov  eax,[esi].lpvOutBuffer
; Расшифровать буфер по способу F=06 #6
  mov  ecx,28h
  lea  ebx,[eax+14h]
@@DecryptOutBuffer:
  xor  byte ptr [ebx],0AAh
  ror  byte ptr ds:[ebx],4
  inc  ebx
  loop @@DecryptOutBuffer

; Запомнить байты пакета (расшифрованные)
  call Store_LogF23Record

; Анализируем код подфункции функции 2 F23
  assume eax:ptr tF23Packet

;;  jmp  @@SkipF23Self

  cmp  word ptr [eax].FuncNum,01h
```

```
  jnz  @@NotFunc01

; Имитация подфункции 01
  mov  dword ptr NeedForExecuteF23,1
  mov  dword ptr [eax].Var1,1
  mov  word ptr [eax].Var2,1
  mov  word ptr [eax].Var3,0
  mov  dword ptr [eax].Var4,0

@@NotFunc01:

  cmp  word ptr [eax].FuncNum,03h
  jnz  @@NotFunc03

; Имитация подфункции 03
  mov  dword ptr NeedForExecuteF23,1
  pushad
  movzx ecx,word ptr [eax].Var1
  mov  cx,word ptr f32Table[ecx*2]
  mov  word ptr [eax].Var2,cx
  popad
  mov  word ptr [eax].Var3,0
  mov  dword ptr [eax].Var4,0

@@NotFunc03:

  cmp  word ptr [eax].FuncNum,05h
  jnz  @@NotFunc05

; Имитация подфункции 05
  mov  dword ptr NeedForExecuteF23,1
  mov  dword ptr [eax].Var1,1
  mov  word ptr [eax].Var2,1
  mov  word ptr [eax].Var3,0
  mov  dword ptr [eax].Var4,0C9h
  mov  dword ptr [eax].Var5,01F4Ah

@@NotFunc05:

  cmp  word ptr [eax].FuncNum,06h
  jnz  @@NotFunc06

; Имитация подфункции 06
  mov  dword ptr NeedForExecuteF23,1
  mov  dword ptr [eax].Var1,0DB3Fh
  mov  word ptr [eax].Var2,07D95h
  mov  word ptr [eax].Var3,0
  mov  dword ptr [eax].Var4,0

@@NotFunc06:

  cmp  word ptr [eax].FuncNum,32h
  jnz  @@NotFunc32

; Имитация подфункции 32
  movzx ebx,word ptr [eax].Var2
  mov  dword ptr [eax].Var7,ebx
  mov  dword ptr [eax].Var4,0
; Получить seed по таблице
  pushad
  pushfd
  mov  ecx,dword ptr [eax].Var1
  lea  esi,dword ptr f32Table[ecx*2]
  movzx ecx,word ptr [eax].Var2
  mov  edi,dword ptr [eax].StackAddr
  cld
  rep  movsw
  popfd
  popad

  mov  dword ptr NeedForExecuteF23,1
```

```
@@NotFunc32:

  cmp  word ptr [eax].FuncNum,3ch
  jnz  @@NotFunc3c

; Имитация подфункции 3c
  mov  dword ptr NeedForExecuteF23,1
  mov  dword ptr [eax].Var1,0
  movzx ebx,word ptr [eax].Var2
  mov  dword ptr [eax].Var7,ebx

@@NotFunc3c:

  cmp  word ptr [eax].FuncNum,3dh
  jnz  @@NotFunc3d

; Имитация подфункции 3d
  mov  dword ptr [eax].Var1,0
  movzx ebx,word ptr [eax].Var2
  mov  dword ptr [eax].Var7,ebx
  pushad
; Сканируем аргументы f3d
  mov  byte ptr [ebx].Func3dNotEmul,0 ; Успешно эмулирована функция f3d
  mov  edi,dword ptr [eax].StackAddr
  mov  esi,offset Func3dArgsAnswers
  mov  ecx,Func3dArgsAnswersSize
  cld
@@FindArgF3d:
  pushad
  mov  ecx,Func3dArgSize
  repe cmpsb
  popad
  jz   @@ArgF3dFound
  add  esi,Func3dArgSize+Func3dAnsSize
  loop @@FindArgF3d
  jmp  @@ArgF3dNotFound

@@ArgF3dFound:

;;  jmp  @@ArgF3dNotFound

; Аргумент найден, подставляем расшифровку требуемой длины
  mov  dword ptr NeedForExecuteF23,1
  mov  byte ptr [ebx].Func3dNotEmul,1 ; Успешно эмулирована функция f3d
  movzx ecx,word ptr [eax].Var2
  add  esi,Func3dArgSize
  rep  movsb

@@ArgF3dNotFound:
  popad

@@NotFunc3d:

@@SkipF23Self:

  assume eax:nothing
; Зашифровать буфер по способу F=04 #6
  mov  ecx,28h
  lea  ebx,[eax+14h]
@@EncryptOutBuffer:
  rol  byte ptr ds:[ebx],4
  xor  byte ptr ds:[ebx],0AAh
  inc  ebx
  loop @@EncryptOutBuffer
  popad
  assume esi:nothing

  pop  es
  pop  ds
  popad
  pop  edi
```

```
  pop  esi
  pop  edx
  pop  ecx
  pop  ebx
  pop  eax
  pop  ebp

; Вызов функции обработки (опционально)
  cmp  dword ptr NeedForExecuteF23,0
  jnz  @@SkipExecOrigF23

  call dword ptr F23EntryPoint

@@SkipExecOrigF23:

  pushad
  pushfd

; Возможно, эмулируем работу драйвера - функции F23 (после выполнения)
  assume esi:ptr DIOCParams
; Расшифровка тела запроса по способу UD
  pushad
; eax=lpvOutBuffer
  mov  eax,[esi].lpvOutBuffer
; Расшифровать буфер по способу F=06 #6
  mov  ecx,28h
  lea  ebx,[eax+14h]
@@DecryptOutBufferAfter:
  xor  byte ptr [ebx],0AAh
  ror  byte ptr ds:[ebx],4
  inc  ebx
  loop @@DecryptOutBufferAfter

; Запомнить байты расшифрованного пакета (после)
  call Store_LogF23RecordAfter

; Анализируем код подфункции функции F23
;  assume eax:ptr tF23Packet
;
;  cmp  word ptr [eax].FuncNum,32h
;  jnz  @@NotFunc32After
;
;  cmp  dword ptr [eax].Var1,0
;  jz   @@NotFunc32After
;  cmp  word ptr [eax].Var2,1
;  jz   @@NotFunc32After
;  movzx ebx,word ptr [eax].Var2
;  mov  dword ptr [eax].Var7,ebx
;  mov  dword ptr [eax].Var4,0
; Получить seed по таблице
;  mov  ecx,dword ptr [eax].Var1
;  mov  ecx,dword ptr f32Table[ecx*2]
;  mov  ebx,dword ptr [eax].StackAddr
;  mov  dword ptr [ebx],ecx
;
;@@NotFunc32After:

  assume eax:nothing

; Зашифровать буфер по способу F=04 #6
  mov  ecx,28h
  lea  esi,[eax+14h]
@@EncryptOutBufferAfter:
  rol  byte ptr ds:[esi],4
  xor  byte ptr ds:[esi],0AAh
  inc  esi
  loop @@EncryptOutBufferAfter
  popad
  assume esi:nothing

  popfd
```

```
  popad

@@F23FuncDone:

  xor  eax,eax
  clc
  ret
HookControl endp
;
; Запомнить байты по перехвату F23 в лог-буфере
;
Store_LogF23Record proc
  pushad
; Запоминаем байты и значения регистров команды
  mov  eax,dword ptr ds:LogPtr
  cmp  eax,MAX_LOG_RECORDS
  jae  LogF23Full
  inc  dword ptr ds:LogPtr

; Активация/дезактивация GetXXXTime
;  cmp  eax,0
;  jnz  @@SkipActivateTime
;  push eax
;  mov  eax,ds:InitializedTimeFlag
;  mov  dword ptr ds:[eax],1
;  pop  eax
;@@SkipActivateTime:
;  cmp  eax,0FEh
;  jnz  @@SkipDeactivateTime
;  push eax
;  mov  eax,ds:InitializedTimeFlag
;  mov  dword ptr ds:[eax],0
;  pop  eax
;@@SkipDeactivateTime:

  imul eax,eax,LogPacketSize
  mov  ebx,offset32 LogPackets
  add  ebx,eax
; Сохранить указатель на последний пакет
  mov  LastPacketPtr,ebx
  assume ebx: ptr tLogF23Packet
; Тип пакета
  mov  byte ptr [ebx].PacketType,PacketF23
; Регистр eax
  mov  eax,dword ptr [ebp-4]
  mov  dword ptr [ebx].RegEAX,eax
  assume esi:ptr DIOCParams
; Код подфункции функции F23
  mov  eax,[esi].lpvOutBuffer
  assume eax:ptr tF23Packet
  mov  cx,word ptr [eax].FuncNum
  mov  word ptr [ebx].FuncNum,cx
  assume eax:nothing
; dwIoControlCode
  mov  eax,[esi].dwIoControlCode
  mov  dword ptr [ebx].dwIoControlCode,eax
; lpvInBuffer
  mov  eax,[esi].lpvInBuffer
  mov  dword ptr [ebx].lpvInBuffer,eax
; Размер переданных данных cbInBuffer
  mov  eax,[esi].cbInBuffer
  mov  dword ptr [ebx].cbInBuffer,eax
; Указатель на данные для ответа VxD
  mov  eax,[esi].lpvOutBuffer
  mov  dword ptr [ebx].lpvOutBuffer,eax
; Аргумент ответа функции F32
  mov  dword ptr [ebx].F32Arg,0
  cmp  word ptr [ebx].FuncNum,32h
  jnz  @@SkipSeedF32Before
  assume eax:ptr tF23Packet
  mov  eax,dword ptr [ebx].lpvOutBuffer
```

```
; Получить аргумент для seed f32
  mov  eax,dword ptr [eax].Var1
  assume eax:nothing
  mov  dword ptr [ebx].F32Arg,eax
@@SkipSeedF32Before:

; Аргумент ответа функции F3d
  cmp  word ptr [ebx].FuncNum,3dh
  jnz  @@SkipArgF3d
  assume eax:ptr tF23Packet
  mov  eax,dword ptr [ebx].lpvOutBuffer
  mov  eax,dword ptr [eax].StackAddr
  assume eax:nothing
  pushad
  mov  ecx,dword ptr [eax]
  mov  dword ptr [ebx].Func3dArg,ecx
  mov  ecx,dword ptr [eax+4]
  mov  dword ptr [ebx+4].Func3dArg,ecx
  popad
@@SkipArgF3d:

; Размер данных для ответа (max)
  mov  eax,[esi].cbOutBuffer
  mov  dword ptr [ebx].cbOutBuffer,eax
; Сколько байтов вернули
  mov  eax,[esi].lpcbBytesReturned
  mov  dword ptr [ebx].lpcbBytesReturned,eax

; Попытаться запомнить выходной буфер ДО
  mov  byte ptr [ebx].OutBuffChanged,0 ; Изменен буфер обмена
; Сначала заполнить его нулями
  lea  edi,[ebx].OutBuffer
  mov  ecx,OutBufferSize
  xor  eax,eax
  rep  stosb
  mov  esi,dword ptr [ebx].lpvOutBuffer
  test esi,esi
  jz   @@OutBuffBeforeIsNull
; Потом скопировать
  mov  ecx,OutBufferSize
  lea  edi,[ebx].OutBufferBefore
  rep  movsb
@@OutBuffBeforeIsNull:

  assume esi:nothing
  assume ebx:nothing
LogF23Full:
  popad
  ret
Store_LogF23Record endp
;
; Запомнить F23 байты в лог-буфере после выполнения
;
Store_LogF23RecordAfter proc
  pushad
  mov  eax,dword ptr ds:LogPtr
  cmp  eax,MAX_LOG_RECORDS
  jae  LogF23AfterFull
  mov  ebx,LastPacketPtr
  assume ebx: ptr tLogF23Packet
  cld

; Попытаться запомнить выходной буфер
; Сначала заполнить его нулями
  lea  edi,[ebx].OutBuffer
  mov  ecx,OutBufferSize
  xor  eax,eax
  rep  stosb
  mov  esi,dword ptr [ebx].lpvOutBuffer
  test esi,esi
  jz   @@OutBuffIsNull
```

```
; Потом скопировать
  mov  ecx,OutBufferSize
  lea  edi,[ebx].OutBuffer
  rep  movsb
; Проверка изменения буфера
  mov  ecx,OutBufferSize
  lea  esi,[ebx].OutBufferBefore
  lea  edi,[ebx].OutBuffer
  repe cmpsb
  jz   @@BuffsEqual
  inc  byte ptr [ebx].OutBuffChanged ; Изменен буфер обмена !
@@BuffsEqual:
; Ответ функции F32
  mov  dword ptr [ebx].F32Seed,0
  cmp  word ptr [ebx].FuncNum,32h
  jnz  @@SkipSeedF32
  assume eax:ptr tF23Packet
  mov  eax,dword ptr [ebx].lpvOutBuffer
; Получить указатель на память, содержащую ответный seed f32
  mov  eax,dword ptr [eax].StackAddr
  assume eax:nothing
; Получит значение seed
  mov  eax,[eax]
  mov  dword ptr [ebx].F32Seed,eax
@@SkipSeedF32:
; Ответ функции F3d
  cmp  word ptr [ebx].FuncNum,3dh
  jnz  @@SkipAnsF3d
  assume eax:ptr tF23Packet
  mov  eax,dword ptr [ebx].lpvOutBuffer
; Получить указатель на память, содержащую расшифровку f3d и длину
  pushad
  mov  esi,dword ptr [eax].StackAddr
  movzx ecx,word ptr [eax].Var2
  assume eax:nothing
  lea  edi,[ebx].Func3dAns
  pushad
  mov  ecx,Func3dAnsSize
  xor  eax,eax
  rep  stosb
  popad
  rep  movsb
  popad
@@SkipAnsF3d:

@@OutBuffIsNull:
  assume ebx:nothing

LogF23AfterFull:

  popad

  ret
Store_LogF23RecordAfter endp
;
; Вписать jmp на наш обработчик #6
;
Install_Jmp_Into_UD6 proc
  pushad
  cli
; eax=команда короткого jmp на тело
  mov  eax,dword ptr ds:UD6EntryPoint
; вычислить начало обработчика
  movzx ebx,byte ptr ds:[eax+1]
  lea  eax,[ebx+eax+2]
; Вписать jmp
  mov  byte ptr ds:[eax],0E9h       ; jmp
  mov  ebx,offset32 TrapPort
  sub  ebx,eax
  sub  ebx,5
  mov  dword ptr ds:[eax+1],ebx    ; offset
```

```
  add  eax,5
  mov  dword ptr old_ud06handler,eax
  sti
  popad
  ret
Install_Jmp_Into_UD6 endp
;
; Убрать jmp на наш обработчик #6
;
Uninstall_Jmp_Into_UD6 proc
  pushad
  cli
  mov  eax,dword ptr old_ud06handler
  test eax,eax
  jz   @@HandlerUDWasNotFound
  sub  eax,5
;29C8: 1E              push        ds << jmp Our_Trap_Program
;29C9: 06              push        es
;29CA: 50              push        eax
;29CB: 51              push        ecx
;29CC: 52              push        edx
  mov  byte ptr ds:[eax],01Eh
  mov  dword ptr ds:[eax+1],52515006h
@@HandlerUDWasNotFound:
  sti
  popad
  ret
Uninstall_Jmp_Into_UD6 endp
Install_Jmp_Into_14 proc
;
; Вписать команду перехода на перехватчик #14
;
;9A02: EB14            jmps       .000009A18
;9A04: db 'GenuineDevil' = db 47,65,6E,75,69,6E,65,44,65,76,69,6C
;9A10: 0E,00,01,02,D8,00,00,00,
;9A18: 1E              push        ds  << jmp Hook_14
;9A19: 06              push        es
;9A1A: 16              push        ss
;9A1B: 1F              pop         ds
;9A1C: 50              push        eax
;9A1D: 51              push        ecx
;9A1E: 52              push        edx
;9A1F: 57              push        edi

; Второй вариант VxD, начало #14
;2822: EB18            jmps       .00000283C
;2824: db 47,65,6E,75,69,6E,65,44,65,76,69,6C
;2830: db 0E,00,02,28,33,00,B4,00,00,00,40,01,00,00
;283C: 1E              push        ds
;283D: 06              push        es
;283E: 16              push        ss
;283F: 1F              pop         ds
;2840: 50              push        eax
;2841: 51              push        ecx
;2842: 52              push        edx
;2843: 57              push        edi

  pushad
  mov  eax,dword ptr ds:UD14EntryPoint
; вычислить начало обработчика
  movzx ebx,byte ptr ds:[eax+1]
  lea  eax,[ebx+eax+2]
  mov  byte ptr ds:[eax],0E9h       ; jmp
  mov  ebx,offset32 Hook_14
  sub  ebx,eax
  sub  ebx,5
  mov  dword ptr ds:[eax+1],ebx    ; offset
  add  eax,5
; Запомнить точку возврата
  mov  dword ptr ds:old_14handler,eax
; Счетчик вызовов #14
```

```
  mov  dword ptr ds:Hook14Count,0
  popad
  ret
Install_Jmp_Into_14 endp
;
; Убрать jmp на наш обработчик #14
;
Uninstall_Jmp_Into_14 proc
  pushad
  mov  eax,dword ptr ds:old_14handler
  test eax,eax
  jz   @@HandlerPageFaultWasNotFound
  sub  eax,5h
;283C: 1E              push        ds
;283D: 06              push        es
;283E: 16              push        ss
;283F: 1F              pop         ds
;2840: 50              push        eax
  mov  byte ptr ds:[eax],1Eh
  mov  dword ptr ds:[eax+1],501F1606h
@@HandlerPageFaultWasNotFound:
  popad
  ret
Uninstall_Jmp_Into_14 endp
;
; Перехватчик исключения #14
;
Hook_14 proc
EntryEIP_14 equ dword ptr [ebp+14h]
; Повторяем команды оригинального обработчика #14
  ; EIP      [ebp+14h]
  ; ErrCode  [ebp+10h]
  push ds  ; [ebp+0Ch] << jmp Hook_14
  push es  ; [ebp+8h]
  push ss
  pop  ds
  push eax ; [ebp+4h]

  push ebp ; [ebp+0h]
  mov  ebp,esp
  pushad

  popad
  pop  ebp

; Возврат в VxD
  jmp  dword ptr ss:old_14handler
Hook_14 endp
;
; Получение DDB адрес
;
GetLitt_DDB proc
  pushad
; Определение DDB VxD
  mov  eax,DeviceLitt_ID
  xor  edi,edi
  VMMCall Get_DDB
; ecx= адрес DDB ?
  mov  dword ptr ds:DDB,ecx
  popad
  ret
GetLitt_DDB endp
VxD_LOCKED_CODE_ENDS
end
;
; **************Приложение*******************
;
; Объявляем доступными команды процессора 386, непривелигерованные инструкции.
.386
; Модель памяти - flat, способ вызова апи - stdcall
.model flat,stdcall
```

```
; Case-sensetive в именах
option casemap:none
; Инклюды, содержащие описание вин-апи
include \masm32\include\windows.inc     ; Макрос invoke...etc
include \masm32\include\kernel32.inc    ; DeviceIOControl, CreateFile... etc
includelib \masm32\lib\kernel32.lib     ; Сама библиотека
include \masm32\include\user32.inc      ; MessageBox
includelib \masm32\lib\user32.lib       ; Сама библиотека
; Сегмент данных программы(инициализированных)
.data
; Отладочные сообщения, выводимые через MessageBox
MsgCaption  db "Iczelion's tutorial no.2",0
; Сообщения о процессе загрузки/выгрузки VxD
AppName     db "Hasp key test program for win9x",0
Success     db "The reseacher is successfully loaded!",0
SerNumErr   db "VxD not present or internal error...",0
Failure     db "The VxD is not loaded !",0
; Имя загружаемого VxD, которое передается CreateFile-у
VxDName     db "\\.\UD1C.VXD",0
; Структура, которой передаются параметры коду VxD
InBuffer    dd offset LogPtr ; Указатель на буфер для чтения лога
include 1C06.inc
; Буфер лога
LogPtr      dd 0 ; Указатель лог-буфера
LogPackets  db (MAX_LOG_RECORDS * LogPacketSize) dup (?)
; Сегмент данных программы(неинициализированных)
.data?
hVxD        dd ? ; Тут будет храниться хэндл открытого VxD
; Начало выполнимого кода
.code
start:
; ***
; Открываем лог
; ***
.data
LogFileName   db "UD1C_06.log",0
.data?
LogDescr      dd ?
bWrite        dd ?
.data
VxDHASPName db "\\.\HASP95DL.VXD",0
PacketF32 db 001h,000h,000h,000h,001h,000h,000h,000h
          db 068h,073h,068h,06ch,048h,000h,000h,000h
          db 028h,000h,000h,000h,04ch,01dh,032h,000h
          db 020h,0fch,000h,000h,0c9h,000h,000h,000h
          db 06dh,03bh,0afh,0cdh,0cbh,070h,081h,081h
Argum     db 004h,000h,000h,000h,00ah,000h,042h,027h
          db 000h,000h,000h,000h,04ah,01fh,000h,000h
          db 000h,000h,000h,000h,000h,000h,000h,000h
SeedAddr  db 020h,0fch,069h,000h,000h,000h,004h,000h
.data?
Seed     dd ?
hVxDHASP dd ?
PacketF32Tmp db 48h dup(?)
.code
; Открыть лог, если есть он уже
  invoke CreateFile,offset LogFileName,GENERIC_WRITE,FILE_SHARE_WRITE+FILE_SHARE_READ,\
      0,OPEN_EXISTING,FILE_ATTRIBUTE_NORMAL,0
  mov  LogDescr,eax          ; Save file descriptor
  inc  eax
  jnz  @@LogFileOpen
; Видимо, нет. Тогда попробовать создать
  invoke CreateFile,offset LogFileName,GENERIC_WRITE,FILE_SHARE_WRITE+FILE_SHARE_READ,\
      0,CREATE_ALWAYS,FILE_ATTRIBUTE_NORMAL,0
  mov  LogDescr,eax          ; Save file descriptor
  inc  eax
  jnz  @@LogFileOpen
.data
OpenErrorMsg db "Log file not created !",0Ah,0
.code
  push offset OpenErrorMsg
```

```
  call Write_Log
  jmp  Exit
@@LogFileOpen:
; Встать в конец лога
  invoke SetFilePointer,LogDescr,0,NULL,FILE_END

; Эмуляция функции f32
.data
;;TestBytes db 0B8h,052h,003h,027h,080h,030h,0D4h,000h
TestBytes db 0DBh,0FEh,01Eh,06Bh,0E4h,01Bh,0DBh,0B0h
;;TestBytes db 05Eh,022h,0E1h,05Eh,04Eh,0C8h,003h,02Fh
TestBytesStr db 'Bytes1=????????????????????????????????',0Dh,0Ah,0
.data?
Count dd ?
.code
  mov  dword ptr Count,0
  mov  esi,offset TestBytes
  ;; mov  eax,ds:[0]
@@Decrypt1:
  mov  edi,[esi]
  mov  edx,edi
  xor  edx,05B2C004Ah
  mov  ecx,dword ptr Count
  mov  eax,0FFFFFFF9h
  sub  eax,ecx
  and  eax,1Fh
  mov  ecx,20h
  mov  ebx,edx
  sub  ecx,eax
  shr  ebx,cl
  mov  ecx,eax
  mov  eax,[esi+4]
  shl  edx,cl
  mov  [esi+4],edi
  or   ebx,edx
  xor  ebx,eax
  mov  [esi],ebx
  add  dword ptr Count,5
  cmp  dword ptr Count,1Eh
  jb   @@Decrypt1

  mov  ecx,8h
  mov  esi,offset TestBytes
  mov  edi,offset TestBytesStr+7
  call Get_HexStr
  push offset TestBytesStr
  call Write_Log

.data?
Tmp dd ?
.code

  mov  esi,offset TestBytes
  mov  eax,[esi]
  mov  dword ptr [esi],0C9162F54h ; ?
  mov  [esi+4],eax

;;  mov  eax,ds:[0]
  xor  edi,edi
@@Decrypt2:
  mov  edx,[esi]
  mov  eax,0Ah
  sub  eax,edi
  mov  Tmp,edx
  and  eax,1Fh
  xor  edx,803425C3h
  mov  ecx,20h
  mov  ebx,edx
  sub  ecx,eax
  add  edi,2
  shr  ebx,cl
```

```
  mov  ecx,eax
  shl  edx,cl
  mov  ecx,Tmp
  or   ebx,edx
  mov  edx,[esi+4]
  xor  ebx,edx
  cmp  edi,0Ch
  mov  [esi],ebx
  mov  [esi+4],ecx
  jb   @@Decrypt2

  mov  ecx,8h
  mov  esi,offset TestBytes
  mov  edi,offset TestBytesStr+7
  call Get_HexStr
  push offset TestBytesStr
  call Write_Log

; Final
  mov  esi,offset TestBytes
  mov  eax,[esi]
  mov  ebx,eax
  xor  ebx,31F18DE2h
  mov  [esi],ebx
  mov  [esi+4],eax

  mov  ecx,8h
  mov  esi,offset TestBytes
  mov  edi,offset TestBytesStr+7
  call Get_HexStr
  push offset TestBytesStr
  call Write_Log

; Exit
;;  invoke ExitProcess,NULL

; Загрузим динамический VxD через CreateFile
  invoke CreateFile,addr VxDName,0,0,0,0,FILE_FLAG_DELETE_ON_CLOSE,0
  cmp  eax,INVALID_HANDLE_VALUE ; VxD Успешно загружена ?
; Сохранить хэндл VxD
  mov hVxD,eax
  jnz  @@LoadedOK
@@VxDProblem:
  invoke CloseHandle,hVxD
  invoke MessageBox,NULL,addr Failure,NULL,MB_OK+MB_ICONERROR
  jmp  Exit
@@LoadedOK:

; Выдать сообщение о том, что VxD загружена успешно, давая возможность
; прекратить работу VxD
  invoke MessageBox,NULL,addr Success,addr AppName,MB_OK+MB_ICONINFORMATION

  jmp  @@SkipDebug

; Вызвать тестовым образом HASP95DL.VXD
  mov  dword ptr SeedAddr,offset Seed
  mov  dword ptr Argum,0
; Открыть VXD
  invoke CreateFile,addr VxDHASPName,0,0,0,0,FILE_FLAG_DELETE_ON_CLOSE,0
; Сохранить хэндл VxD
  mov hVxDHASP,eax
;
;;  invoke MessageBox,NULL,addr Success,addr AppName,MB_OK+MB_ICONINFORMATION

@@NextArg:

  mov  esi,offset PacketF32
  mov  edi,offset PacketF32Tmp
  mov  ecx,48h
  cld
  rep  movsb
```

```
; Зашифровать буфер по способу F=04 #6
  mov  ecx,28h
  lea  esi,PacketF32Tmp[14h]
@@EncryptOutBufferAfter:
  rol  byte ptr ds:[esi],4
  xor  byte ptr ds:[esi],0AAh
  inc  esi
  loop @@EncryptOutBufferAfter

  invoke DeviceIoControl,hVxDHASP,2,addr PacketF32Tmp,48h,addr PacketF32Tmp,48h,NULL,NULL

  inc  dword ptr Argum
  cmp  dword ptr Argum,60h
  jbe  @@NextArg

; Закрыть VXD
  invoke CloseHandle,hVxDHASP

@@SkipDebug:

; Получить протокол работы
  invoke DeviceIoControl,hVxD,1,addr InBuffer,sizeof InBuffer,NULL,NULL,NULL,NULL
; Проверка на ошибку. Если ошибка, то eax = 0
  test eax,eax
  jz   @@VxDProblem
; Ошибки нет. Скинем данные в лог.
  call Print_SerialInfo
; Закроем VxD
  invoke CloseHandle,hVxD
Exit:
; Завершим программу
  invoke ExitProcess,NULL
; Подпрограммы
.data
SerialInfo    db "Total traps: ?????????? See log file UD1C_01.log...",0Dh,0Ah,0
LogInfoEnd    db 'EAX=???????? EDI=????????',0Dh,0Ah,0
StrNumber     dd 0
LogInfoStart  db 'Num=????:',0
LogInfoInBuf  db 'BuffBef=',0
LogInfoOp     db '0??h,',0
LogInfoOutBuf db 'OutBuff=',0
LogInfoEnd1   db 'dwIoControlCode=???????? lpvInBuffer=???????? cbInBuffer=????????',0Dh,0Ah,0
LogInfoEnd2   db 'lpvOutBuffer=???????? cbOutBuffer=???????? lpcbBytesReturned=????????',0Dh,0Ah,0
LogInfoFuncN  db 'Func=????',0Dh,0Ah,0
LogInfoF32Seed db 'F32Seed[????????]=????????',0Dh,0Ah,0
Func3dArgAnsMsg db 'Func3d[????????????????]=????????????????????????????????',0Dh,0Ah,0
LogInfoBuffCh db 'Out buffer changed !',0Dh,0Ah,0
.data?
LogSize    dd ?
.code
Print_SerialInfo proc
; Сколько было записей в логе
  mov  eax,dword ptr LogPtr
  mov  ebx,1000000000
  mov  edi,offset SerialInfo+13
  call DecChar
  invoke MessageBox,NULL,addr SerialInfo,addr MsgCaption,MB_OK
  push offset SerialInfo
  call Write_Log
  mov  ecx,dword ptr LogPtr
  test ecx,ecx
  jnz  @@IsLogData
  ret
@@IsLogData:
; Размер лога
  mov  eax,ecx
  imul eax,eax,LogPacketSize
  add  eax,4 ; Size
  mov  dword ptr LogSize,eax
  mov  ebx,offset LogPackets
@@OutLogData:
```

```
  pushad

; Номер строки
;;  call Write_StrNum

;
; Обрабатываем пакеты различных типов
;

; Пакет перехвата UD6
  assume ebx: ptr tLogPacket
  cmp  byte ptr [ebx].PacketType,PacketUD
  jnz  @@NoUDPacket

  jmp  @@SkipLogPacket

; Номер строки
  call Write_StrNum

; Регистр eax и изображение
  mov  eax,dword ptr [ebx].RegEAX
  mov  edi,offset LogInfoEnd+4
  call HexChar
; Регистр edi
  mov  eax,dword ptr [ebx].RegEDI
  mov  edi,offset LogInfoEnd+17
  call HexChar
  push offset LogInfoEnd
  call Write_Log
  assume ebx:nothing

@@NoUDPacket:

  assume ebx: ptr tLogF23Packet
  cmp  byte ptr [ebx].PacketType,PacketF23
  jnz  @@NoF23Packet

; Проверка changed ?
;;  cmp  byte ptr [ebx].OutBuffChanged,0 ; Изменен буфер обмена ?
;;  jz   @@SkipLogPacket

; Номер функции: фильтр
;;  cmp  word ptr [ebx].FuncNum,32h
;;  jnz  @@SkipLogPacket
; Номер функции: фильтр
;;  cmp  word ptr [ebx].FuncNum,3h
;;  jnz  @@SkipLogPacket
; Номер функции: фильтр
  cmp  word ptr [ebx].FuncNum,3dh
  jnz  @@SkipLogPacket
  cmp  byte ptr [ebx].Func3dNotEmul,1 ; Успешно эмулирована функция f3d ?
  jnz  @@SkipLogPacket

; Номер строки
  call Write_StrNum

; dwIoControlCode
  mov  eax,dword ptr [ebx].dwIoControlCode
  mov  edi,offset LogInfoEnd1+16
  call HexChar
; lpvInBuffer
  mov  eax,dword ptr [ebx].lpvInBuffer
  mov  edi,offset LogInfoEnd1+37
  call HexChar
; Размер переданных данных cbInBuffer
  mov  eax,dword ptr [ebx].cbInBuffer
  mov  edi,offset LogInfoEnd1+57
  call HexChar
  push offset LogInfoEnd1
  call Write_Log
; lpvOutBuffer
```

```
  mov  eax,dword ptr [ebx].lpvOutBuffer
  mov  edi,offset LogInfoEnd2+13
  call HexChar
; cbOutBuffer
  mov  eax,dword ptr [ebx].cbOutBuffer
  mov  edi,offset LogInfoEnd2+34
  call HexChar
; lpcbBytesReturned
  mov  eax,dword ptr [ebx].lpcbBytesReturned
  mov  edi,offset LogInfoEnd2+61
  call HexChar
  push offset LogInfoEnd2
  call Write_Log

; Признак изменения буфера
  cmp  byte ptr [ebx].OutBuffChanged,0 ; Изменен буфер обмена ?
  jz   @@NotChanged
;;  push offset LogInfoBuffCh
;;  call Write_Log
@@NotChanged:

; Номер функции
  mov  ax,word ptr [ebx].FuncNum
  mov  edi,offset LogInfoFuncN+5
  call HexWORD
  push offset LogInfoFuncN
  call Write_Log

; Байты входного буфера
  push offset LogInfoInBuf
  call Write_Log
  mov  ecx,OutBufferSize
  lea  esi,[ebx].OutBufferBefore
; Расшифровать буфер по способу F=06 #6
;;  call Decrypt_Buffer
@@OutBufferBefore:
  pushad
  mov  ecx,1h
  mov  edi,offset LogInfoOp+1
  call Get_HexStr
  push offset LogInfoOp
  call Write_Log
  popad
  inc  esi
  loop @@OutBufferBefore

  call Write_EndStr_ToLog

; Байты выходного буфера
  push offset LogInfoOutBuf
  call Write_Log
  mov  ecx,OutBufferSize
  lea  esi,[ebx].OutBuffer
; Расшифровать буфер по способу F=06 #6
;;  call Decrypt_Buffer
@@OutOutBuffer:
  pushad
  mov  ecx,1h
  mov  edi,offset LogInfoOp+1
  call Get_HexStr
  push offset LogInfoOp
  call Write_Log
  popad
  inc  esi
  loop @@OutOutBuffer

  call Write_EndStr_ToLog

; Ответ F32 - F32Seed
; Номер функции: фильтр
  cmp  word ptr [ebx].FuncNum,32h
```

```
  jnz  @@SkipF32SeedOut

; Var1
  mov  eax,dword ptr [ebx].F32Arg
  mov  edi,offset LogInfoF32Seed+8
  call HexChar
;Seed
  mov  eax,dword ptr [ebx].F32Seed
  mov  edi,offset LogInfoF32Seed+18
  call HexChar
;;  mov  eax,dword ptr [ebx].F32SeedBefore
;;  mov  edi,offset LogInfoF32Seed+31
;;  call HexChar
  push offset LogInfoF32Seed
  call Write_Log

@@SkipF32SeedOut:

; Ответ F3d
; Номер функции: фильтр
  cmp  word ptr [ebx].FuncNum,3dh
  jnz  @@SkipF3dData
  pushad
  mov  ecx,Func3dArgSize
  lea  esi,[ebx].Func3dArg
  mov  edi,offset Func3dArgAnsMsg+7
  call Get_HexStr
  mov  ecx,Func3dAnsSize
  lea  esi,[ebx].Func3dAns
  mov  edi,offset Func3dArgAnsMsg+25
  call Get_HexStr
  push offset Func3dArgAnsMsg
  call Write_Log
  popad
@@SkipF3dData:

  call Write_EndStr_ToLog
  assume ebx:nothing

@@NoF23Packet:
@@SkipLogPacket:

  popad
  add  ebx,LogPacketSize
  dec  ecx
  jnz  @@OutLogData
  ret
Print_SerialInfo endp
;
; Расшифровать буфер по способу F=06 #6
; esi=offset, ecx=len of buffer (but only 28h bytes)
;
Decrypt_Buffer proc
  pushad
  add  esi,14h
  mov  ecx,28h
@@DecryptBuffer:
  xor  byte ptr [esi],0AAh
  ror  byte ptr ds:[esi],4
  inc  esi
  loop @@DecryptBuffer
  popad
  ret
Decrypt_Buffer endp
;
;
;
Write_StrNum proc
; Номер строки
  pushad
  mov  eax,StrNumber
```

```
  inc  StrNumber
  mov  ebx,1000
  mov  edi,offset LogInfoStart+4
  call DecChar
  push offset LogInfoStart
  call Write_Log
  popad
  ret
Write_StrNum endp
; *************DecChar: Подпрограмма форматирования строки из числа***********
; eax=number to digit, edi=offset result string in format 00000000(@n[ebx])
; ebx=начальный делитель
DecChar proc
  pushad
  pushfd
  cld
@GetDec:
  xor  edx,edx
  div  ebx
  add  al,'0'
  stosb
  push edx
  mov  eax,ebx
  xor  edx,edx
  mov  ebx,10
  div  ebx
  mov  ebx,eax
  pop  eax
  test ebx,ebx
  jnz  @GetDec
  popfd
  popad
  ret
DecChar endp
; ***
; Подпрограмма записи в лог и на консоль одновременно
; ***
; [ebp+8] - адрес строки для записи
Write_Log proc
  push ebp
  mov  ebp,esp
; Получить длину строки
  mov  esi,dword ptr [ebp+8] ; Адрес строки
  call Str_Len
  test ecx,ecx
  jz   @@ExitWrite_Log
; Запись в протокол
  push NULL
  push offset bWrite
  push ecx ; Длина строки
  push dword ptr [ebp+8] ; Адрес строки
  push LogDescr
  call WriteFile
@@ExitWrite_Log:
  pop  ebp
  ret  4
Write_Log endp
Write_EndStr_ToLog proc
.data
EndStr db 0Ah,0
.code
  push offset EndStr
  call Write_Log
  ret
Write_EndStr_ToLog endp
; ***
; Get lenght of string
; esi=addr of string; result: ecx=length
; ***
Str_Len proc
  xor  ecx,ecx
```

```
  push eax
  push esi
@@GetStrLen:
  lodsb
  test al,al
  jz   @@ExitStrLen
  inc  ecx
  jmp  @@GetStrLen
@@ExitStrLen:
  pop  esi
  pop  eax
  ret
Str_Len endp
; Форматировать число в EAX в строку
HexChar proc
  pushad
  cld
  mov  ecx,8
  mov  ebx,offset TabHex
@GetHex:
  rol  eax,4
  push eax
  and  al,0fh
  xlat
  stosb
  pop  eax
  loop @GetHex
  popad
  ret
HexChar endp
TabHex  db '0123456789abcdef'
; Формирует хекс-строку
; ecx - длина строки байт, esi - адрес строки, edi - приемник
Get_HexStr proc
  pushad
  mov  ebx,offset TabHex
@@Get_HexStr:
  lodsb
  rol  al,4
  push eax
  and  al,0Fh
  xlat
  stosb
  pop  eax
  shr  al,4
  xlat
  stosb
  loop @@Get_HexStr
  popad
  ret
Get_HexStr endp
; Форматировать число в AX в строку
HexWORD proc
    pushad
    cld
    mov  ecx,4
    mov  ebx,offset TabHex
@GetWORD:
    rol  ax,4
    push eax
    and  al,0fh
    xlat
    stosb
    pop  eax
    loop @GetWORD
    popad
    ret
HexWORD endp
end start
```

Теоретические основы крэкинга: Глава 7. Искусство разбивать окна.

Автор: CyberManiac

Дата: 2004-09-10

_Человек и кошка плачут у окошка,
Серый дождик каплет прямо на стекло.
К человеку с кошкой едет «неотложка» -
Человеку бедному мозг больной свело._

Федор Чистяков, «Человек и кошка»

И вот, пройдя долгий и трудный путь исследователя ценных данных, упрятанных в недрах программ, мы, наконец, подошли к вратам «чистого крэкинга». Вспомните, с чего начинаются едва ли не все платные программы: с предложения зарегистрироваться. Это предложение может быть написано аршинными буквами в отдельном окне, появляющемся при запуске, или же в виде маленького MessageBox'а, выскакивающего в самый неподходящий момент во время работы. Оно может быть мерцающим баннером вверху или внизу экрана. Но, как бы оно ни выглядело, цель его существования во всех этих случаях одна: раздражать и давить на психику незарегистрированного пользователя, дабы он, зажав в кулак свои кровные, отправился на почту и перевел их автору программы в знак благодарности за будущее избавление от назойливого окна. Однако далеко не у всех людей, созерцавших эти безыскусные творения, возникали столь теплые чувства к разработчику. И эти раздраженные пользователи дали надоедливым окнам то самое имя, под которым они и известны современному крэкеру. Называются эти окна «nagscreens» (от английского «nag» - «надоедать, приставать, придираться»), и в этой главе речь как раз пойдет о способах борьбы с такими окнами.

Я, увы, лично не застал рождение крэкинга под платформу Win32, но опоздал ненамного, а потому в моей коллекции имеются многие популярные руководства тех времен. Тогда первым словом едва ли не каждого крэкера была классическая команда bpxMessageBoxA – другие варианты были очень, очень редким исключением. Тридцатидвухразрядные Delphiбыли лишь светлой мечтой, монструозный MFC, спроектированный на базе логики пришельцев с Альфы Центавра, не пользовался особой популярностью, и потому большинство программистов тогда обходилось одним лишь Win32 API. А из всех функций WinAPIMessageBoxбыла едва ли не самой популярной – ибо трудно было придумать более простое и универсальное средство вывести какое-нибудь нехитрое сообщение, да так, чтобы пользователь не смог это сообщение проигнорировать. Разумеется, разработчики shareware-программ не могли пройти мимо такой возможности, и первые «надоедливые экраны» были теми самыми MessageBox'ами. Мы же, в свою очередь, нещадно bpx'или эти MessageBox'ы – и выискивали точку ветвления, в которой расходились жизненные пути зарегистрированной и незарегистрированной инкарнации программы. И если когда-нибудь появится мемориал в честь крэкеров Вселенной – на нем, несомненно, огромными золотыми буквами будет выбито **bpx** MessageBoxA.

Но все-таки я начну рассказ о борьбе с нежелательными визуальными спецэффектами в программах не с MessageBox'ов. Ибо я стараюсь следовать принципу «от простого – к сложному», а стандартные окна с сообщениями – это все-таки не самое простое из того, с чем Вы можете столкнуться. Согласитесь, чтобы удалить из программы лишний MessageBox, все-таки надо приложить некоторые усилия по обнаружению этого MessageBox'а в коде программы при помощи отладчика или дизассемблера – поэтому я начну с рассказа о таких способах удаления nagscreen'ов, которые не предполагают выхода на уровень ассемблера. Вы спросите – возможно ли такое? Конечно, возможно! И, как это ни удивительно, такую возможность подарила нам

операционная система Windows.

Прежде чем переходить к рассмотрению методов борьбы с нежелательными окнами и конкретных примеров их применения, рассмотрим проблему баннеров и nagscreen'ов, что называется, в перспективе. Я бы выделил четыре подхода к задаче ликвидации рекламных вставок в программах:

1. Изменение свойств объектов (окон, визуальных компонентов) таким образом, чтобы они не отображались на экране. На практике это может быть достигнуто тремя способами: выносом нежелательного объекта за пределы экрана (проще всего использовать отрицательные координаты – при любых размерах монитора объект все равно будет за пределами видимой части экрана); изменением размеров объекта (к примеру, дочерние окна нулевой длины и ширины на экране не видны); модификацией шаблонов диалогов с тем, чтобы придать нежелательным объектам свойства невидимости и неактивности.
2. Удаление шаблонов объектов из исполняемого файла.
3. Модификация кода с целью предотвратить создание или отображение нежелательного объекта.
4. Модификация кода таким образом, чтобы нежелательный объект самоликвидировался сразу после появления.

Вот о первых двух подходах, в основном, и пойдет речь в этой главе.

Ради того, чтобы упростить жизнь программистам, в ОС Windows реализована возможность интегрировать в исполняемый файл ресурсы всевозможных типов – текстовые строки, иконки, изображения в стандартных форматах и, что самое интересное, так называемые «шаблоны» (templates) диалоговых окон. Эти шаблоны описывают внешний вид диалогов: размеры, атрибуты, находящиеся внутри диалога управляющие элементы (меню, кнопки, поля редактирования и т.п.) и надписи на этих элементах. При помощи соответствующих функций WindowsAPIна основе этих шаблонов могут быть созданы реальные окна, внешний вид которых будет в точности соответствовать их описанию внутри шаблона. Открыв такой исполняемый файл при помощи какого-либо редактора ресурсов (наиболее известны ResourceHacker, Restorator и eXeScope), Вы можете попытаться модифицировать ресурсы программы, например, перевести все надписи на русский язык (если быть до конца точным, то у ресурсов имеется еще один атрибут – идентификатор языка, который, возможно, также потребуется модифицировать). Если Вы все сделаете правильно, и редактор ресурсов корректно сохранит изменения, после запуска программы соответствующие надписи будут русифицированы. Таким же образом Вы можете изменять расположение и свойства элементов интерфейса (например, наличие рамки и ее цвет) в диалоговых окнах. Более того, Вы можете заменить не только надписи, но и хранящиеся в ресурсах изображения, своими собственными. Нужно отметить, что ресурсы могут храниться не только внутри исполняемого файла, но и в любом другом файле формата PortableExecutable. Чаще всего это делается для облегчения локализации – реализовать выбор одной из возможных ресурсных DLLгораздо проще, чем выпускать множество локализованных версий одного и того же исполняемого файла. Кроме того, ресурсные DLL – довольно удобный способ хранения «шкурок» (skin'ов) для приложений, использующих эту технологию модификации интерфейса.

Вообще все ресурсы делятся на типы, среди которых есть такие, как иконки (RT_ICON), курсоры (RT_CURSOR), шаблоны диалоговых окон (RT_DIALOG) и многое другое. Для идентификации каждого конкретного ресурса среди других ресурсов того же типа используются числовые либо символьные идентификаторы. Именно на основе идентификаторов программа и различает ресурсы между собой. Что интересно, программе совершенно нет дела до тех данных, которые прячутся за идентификаторами – поэтому, поменяв местами идентификаторы у пары однотипных ресурсов, можно получить весьма занятные эффекты: например, вместо сообщения об успешном завершении той или иной операции будет появляться сообщение об ошибке (и наоборот).

Теперь немного поговорим о том, какие тайны скрываются в недрах шаблонов диалоговых окон. Прежде всего, умение обращаться с шаблонами позволит Вам полностью преобразить интерфейсы

очень многих программ, причем сделать это с минимальными затратами сил. Большинство редакторов ресурсов умеют не только декодировать шаблоны в исходный текст, понятный компиляторам ресурсов (при желании Вы можете «одолжить» особенно понравившееся окно из чужой программы), но и непосредственно показывать, как этот диалог будет выглядеть на мониторе. А наиболее продвинутые редакторы позволяют даже править эти шаблоны визуальными средствами. Кто знаком с современными визуальными средствами разработки, тот, наверняка, догадался, о чем идет речь. А те, кто не догадался, могут просто взять Калькулятор из состава Windows 98 (с Калькулятором от WindowsXPтакой номер может не пройти – всяческие MUIмогут отравить Вам радость познания), программку ResourceHackeri вдоволь поэкспериментировать над первой программой при помощи второй: например, поменять местами все кнопки в Калькуляторе. Причем настоятельно рекомендую Вам попытаться проделать эту операцию не только тасканием-бросанием кнопок а-ля VisualBasic, но и через ручную правку ресурсного скрипта (resourcescript).

Но махинации с кнопками – это, в общем-то, мелочи, несмотря на тот могучий эффект, который производит «обработанный напильником» Калькулятор на неподготовленного пользователя. Гораздо более важно другое – а именно то, что управляющие элементы шаблона также имеют свои собственные идентификаторы, по которым программа различает управляющие элементы между собой и «общается» с ними. И эти идентификаторы также можно менять при помощи редактора ресурсов. Попробуйте проделать над несчастным Калькулятором еще один опыт – поменяйте местами значения идентификаторов у двух кнопок, например, у единицы и семерки. Запустив Калькулятор, Вы увидите, что единица и семерка действительно поменялись местами! Кроме идентификатора Вы можете менять и другие атрибуты кнопок (более корректно называть их не атрибутами кнопок, а стилями окна) – видимость, наличие рамки, способ выравнивания текста и многое другое. И из этой возможности следуют кое-какие выводы чисто практического свойства.

Довольно часто авторы sharewareпользуются следующим приемом: сразу после запуска программы выводится nagscreen, кнопка закрытия которого изначально неактивна и активизируется лишь спустя несколько секунд. Внутри программы это может быть реализовано следующим образом: в шаблоне, по которому создается nagscreen, стиль этой кнопки изначально определен как неактивный (он называется WS_DISABLED). После создания окна запускается таймер, который через установленное время активизирует нужную кнопку, и пользователь получает возможность закрыть окно. Однако если мы уберем у кнопки стиль WS_DISABLED, пользователю больше не потребуется ждать несколько томительных секунд, чтобы получить возможность нажать на кнопку. Аналогичным же образом иногда удается включить неактивные функциональные элементы не только на nagscreen'ах, но и непосредственно в самой ломаемой программе. Увы, всвязи с появлением некоторого количества программ, способных принудительно включать неактивные элементы управления, закрывать «лишние» окна и проделывать прочие столь же милые вещицы (ваш покорный слуга со своим проектом Sign 0fMiseryтоже отметился на этом поприще), авторы sharewarevсе чаще вводят различные дополнительные проверки, призванные оградить назязчивую рекламу от священного права пользователя эту рекламу не смотреть.

Однако самое интересное заключено даже не в манипуляциях с идентификаторами и стилями. Куда более важным представляется вопрос – а что если просто взять и удалить из секции ресурсов «лишний» диалог? Будем рассуждать логически: создание диалоговых окон производится при помощи семейства функций CreateDialog*(которые затем требуют принудительно отобразить окно при помощи ShowWindow) либо при помощи DialogBox* (это семейство функций все операции по отображению окна берет на себя). Причем для создания модальных окон (т.е. таких, которые пользователь не смог бы проигнорировать) чаще используется именно второе семейство функций.

Предположим, что программа попыталась создать nagscreenна основе шаблона диалога при помощи функции DialogBoxParam (или любой другой функции из семейства DialogBox*), и у нее это не получилось. Большинству разработчиков такой вариант обычно даже в голову не приходит – а

потому и проверку на существование шаблона nagscreen’а программа обычно не делает, а сразу переходит созданию и выводу окна на экран. Что бы Вы сделали, если бы Вас попросили создать окно на основе «никакого» шаблона? Правильно, абсолютно ничего! Вот и Windowsпоступает точно так же – а именно, ничего не делает. Хотя на самом деле такое «ничего» все-таки может иметь далеко идущие последствия: ни разу не выполнится, к примеру, оконная процедура выдранного с корнем диалога, что, в свою очередь может повлечь за собой труднопредсказуемые эффекты. Вспомните принцип минимального вмешательства – и ужаснитесь тому, что предлагаю Вам содеять: вырвать из программы особо ценный (с точки зрения разработчика, разумеется) ресурс, отключить «не глядя» оконную процедуру, спровоцировать передачу некорректных данных как минимум в одну функцию WinAPI…

Но «суха теория, мой друг…», а потому все же посмотрим, что там у нас выросло на зеленеющем «древе жизни». В качестве наглядного пособия я возьму программу SkyMapPro 8 Demo, которая обладает одним весьма ценным для нас качеством – наличием nagscreen’а, шаблон которого упрятан глубоко в недрах секции ресурсов и имеет идентификатор 3006. В качестве орудия, при помощи которого производилась ампутация надоедливого диалога, я воспользовался программой ResourceHacker (в принципе, подошел бы и любой другой редактор ресурсов). Сама операция заняла не более 15 секунд, после чего исполняемый файл «похудел» на 4 килобайта. После того, как я произвел пробный запуск исправленной программы, оказалось, что nagscreenкак рукой сняло – и мне для этого не потребовались отладчики, дизассемблеры и прочая крэкерская «тяжелая артиллерия»! Видите: все далеко не так страшно, как может показаться. В принципе, если Вы считаете, что выдирание диалога с корнем – слишком уж варварская операция, шаблон можно оставить на месте, лишь заменив его идентификатор на другой, не используемый в программе. Результат будет аналогичный – программа не сможет найти ресурс с измененным идентификатором и не создаст nagscreen. Этот прием даже предпочтительнее – практика показала, что не всегда и не во всех редакторах ресурсов операция удаления проходит успешно.

А ведь есть еще и третий путь избавиться от диалога путем изменения идентификатора! Ничто не мешает исправить идентификатор не в секции ресурсов, а непосредственно в исполняемом коде, в том месте, где этот идентификатор передается в функции WinAPI. Поскольку наш идентификатор имеет довольно редкое значение 3006, он как нельзя лучше подходит для поиска внутри исполняемого файла. Найдя среди всех подходящих значений то, которое отвечает за создание nagscreen’а – замените его каким-нибудь другим, не указывающим ни на один диалог, например – на 12345. Когда Вы запустите исправленное приложение, оно, конечно, попытается найти и загрузить шаблон с ID=12345 – но у программы это все равно не получится. Впрочем, я считаю такой путь излишне изощренным и рассказал о нем главным образом в познавательных целях – гораздо проще забить вызов функции создания диалога и все, что с ней связано, nop’ами и получить тот же самый эффект.

Успешность такого метода борьбы с «лишними» окнами, вообще, сильно зависит от внутренней логики приложения. Nagscreen, к примеру, может оказаться не «пустышкой», предназначенной исключительно для раздражения пользователя, а исполнять какие-либо неочевидные функции. Например, на обработку сообщения WM_INITDIALOGв оконной процедуре nagscreen’а может быть подвешена инициализация каких-либо критически важных для программы структур. В общем случае такая инициализация может происходить при возникновении практически любого события связанного с этим окном: при нажатии на какую-либо кнопку в окне, при закрытии окна, в момент поступления первого сообщения WM_PAINT – перечислять варианты можно очень долго. И если просто удалить nagscreen, это «волшебное» событие не произойдет – следовательно, не будет произведена инициализация, отчего, в итоге, программа будет работать некорректно, если вообще будет работать.

На практике такая защитная техника в чистом виде встречается достаточно редко – ведь программисту, использовавшему подобный прием, придется учитывать при программировании

различия в работе между полной/зарегистрированной (без nagscreen'а) и урезанной/незарегистрированной версиями программы. Тем не менее, если Вы все же столкнетесь с реализацией такого алгоритма, Вам вряд ли станет легче от осознания редкости встреченной проблемы. Поэтому давайте подумаем о том, как эту проблему можно было бы решить. В теории решение довольно простое: Вам нужно найти функции, которые выполняют все нужные действия по инициализации, и вызвать их вручную. На практике Вам скорее всего понадобится добраться до цикла выборки сообщений внутри оконной процедуры и проанализировать его содержимое. Обратите также внимание на следующий факт: если Вам удалась обнаружить, что инициализация программы происходит в ответ на WM_INITDIALOG, из этого отнюдь не следует, что Вам необходимо выполнить **все** действия, которые программа выполняет при поступлении этого сообщения. Посмотрите на следующий пример:

```
.IF uMsg==WM_INITDIALOG
 invoke GetDlgItem, hWnd, TRIALUSE_BTN
 invoke SetFocus,eax
 invoke InitProgram
```

После того, как мы умозрительно удалили диалог, которому должно было поступать сообщение, попытка установить фокус на кнопку TRIALUSE_BTNстановится совершенно бессмысленна – поскольку больше нет диалога, нет и кнопки которая в этом диалоговом окне находилась. Поэтому в нашем простейшем примере вместо вызова функций создания диалога было бы достаточно просто написать callInitProgram, а все лишние байты забить nop'ами. Однако представьте себе, что оконная процедура содержит какие-либо локальные переменные (например, вычисленное ранее число дней до истечения триального срока), используемые внутри InitProgramили же проделывает какие-либо операции с созданным диалогом – и Вы поймете, почему простейший путь не является лучшим. Что делать в таком случае? Из каждой безвыходной ситуации есть как минимум два выхода. Можно углубиться в дебри процедуры InitProgram и проверить, не обращается ли она к каким-либо объектам, которые мы так лихо снесли вместе с диалогом и потом долго заниматься художественной штопкой машинного кода. Я сам пару раз применял такую технику и могу сказать определенно: этот процесс мне совершенно не понравился. Поэтому мы поступим хитрее – попробуем поискать другое, более простое решение.

Практически в любом nagscreen'е изначально заложен способ от него избавиться. Этим способом может быть клик по кнопке в диалоге, нажатие «горячей клавиши» или же истечение определенного времени с момента появления окна. Все эти случаи объединяет одно - nagscreenждет некоего события, наступление которого станет для него сигналом к исчезновению. «Приемник» этого сигнала, который и выполняет все действия по убиранию nagscreen'а с экрана, чаще всего прячется во все той же оконной процедуре, о которой уже столько раз говорилось и еще не раз будет сказано. Для тех, кто не особо интересовался программированием с использованием «чистого» WinAPI, дам некоторые пояснения относительно традиционного устройства оконной процедуры.

«Сердцем» оконной процедуры, несомненно, является обработка сообщений, поступающих от окна, которая на ассемблере выглядит приблизительно так:

```
.IF uMsg==WM_DESTROY
  invoke PostQuitMessage,NULL
.ELSEIF uMsg==WM_INITDIALOG
 ...
.ELSEIF uMsg==WM_COMMAND
  mov eax,wParam
  .IF ax==ID1
   ...
  .ELSEIF ax==ID2
   ...
  .ENDIF
.ELSE
  invoke DefWindowProc,hWnd,uMsg,wParam,lParam
  ret
.ENDIF
```

Не вдаваясь в подробности (которые Вы можете найти в документации и в книгах по программированию под Windows), скажу, что обычно все самое интересное - обработка нажатий на кнопки или реакция на выбор пунктов меню - скрывается в блоке, анализирующем сообщение WM_COMMAND. Внутри этого блока обычно выполняется последовательное сравнение параметров сообщения wParamи lParamс идентификаторами и хэндлами управляющих элементов (кнопок, элементов меню, тулбаров и т.п.). Если значения хэндла и идентификатора указывают, что пользователь нажал на некий элемент - программа выполняет соответствующие действия. Допустим, что мы знаем, где располагается оконная процедура интересующего нас окна. Что мы можем сделать с нашей находкой? Самое простое – это, конечно, возможность менять местами функции управляющих элементов: если в приведенном выше примере поменять местами значения ID1 и ID2, соответственно изменятся и функции элементов с идентификаторами ID1 и ID2. Менее очевиден, но тоже достаточно прост для понимания тот факт, что можно «переключить» реакцию программы с одного сообщения на другое. К примеру, заменив в нашем примере WM_INITDIALOGна WM_PAINT, мы заставим процедуру инициализации выполняться не единожды при создании диалога, а при поступлении каждой команды на перерисовку содержимого окна. Зачем такое может понадобиться? С точки зрения среднего программиста это действо совершенно нелогично, нефункционально, и даже более того – опасно. Но крэкинг – это искусство, существующее по ту сторону обычного программирования. И именно благодаря своей «потусторонней» сущности даже во внешне бессмысленных операциях крэкер способен узреть потаенные возможности и раскрыть их к своей пользе.

Итак, представим также, что кнопка ID1 закрывает nagscreen (на самом деле представлять придется только Вам – у меня код этого примера сейчас перед глазами). Наша задача – убрать nagscreenлюбыми доступными средствами. Как можно решить эту задачу? Да очень просто - базовую идею я описал парой строчек выше. Перво-наперво заменим в оконной процедуре константу WM_COMMANDна WM_PAINT. Следующее, что нам нужно – это чтобы оконная процедура реагировала на сообщение WM_PAINTкак на нажатие кнопки с идентификатором ID1. Добиться этого можно, подправив условие lFax==ID1 таким образом, чтобы оно выполнялось в любом случае, независимо от величины wParam. Думаю, проделать такой фокус не составит никакого труда даже для самого начинающего крэкера, только-только научившегося заменять условный переход на безусловный. Если в оконной процедуре присутствует «настоящий» обработчик WM_PAINT, есть смысл при помощи безусловного перехода выполнить и его – ради соблюдения принципа минимального вмешательства. На этом нашу миссию можно считать завершенной – после внесенных исправлений nagscreenбудет закрываться сам собой сразу же после появления. Главная хитрость заключается в том, что как только nagscreen пытается стать видимым (а Вы когда-нибудь видели невидимые nagscreen'ы?), окно этого screen'а автоматически получит команду на перерисовку содержимого клиентской области - сообщение WM_PAINT. Мы же, движимые возвышенными эстетическими чувствами, протестующими против созерцания рекламных банальностей, преобразовали это безобидное сообщение в приказ немедленно убрать раздражающее окно с экрана.

Все, о чем я говорил выше, относится к классическим средствам работы с окнами и диалогами Windows и к объектно-ориентированным «оберткам» для WinAPI. Однако фирма Borlandвнесла в такую консервативную область, как графические интерфейсы Windows-программ, заметное оживление. Я имею в виду прежде всего фирменную борландовскую разработку: VisualComponentLibrary (она же VCL) – объектно-ориентированную библиотеку, предназначенную для создания графического интерфейса. Впервые эта библиотека появилась в Delphi 1 и с тех пор стала неотъемлемой частью всех версий Delphiи C++ Builder. В VCLшироко используются такие нетрадиционные для Windowsвещи, как «безоконные» управляющие элементы (non-windowedcontrols), по сути представляющие собой картинки на экране, и потому напрямую не подчиняющиеся функциям WinAPI. Другим новшеством, внедренным Borland, является

специфический формат хранения шаблонов форм – они хранятся как данные типа RCData, причем в качестве идентификаторов используются названия классов этих форм: к примеру, шаблон формы класса TMyFormхранится в ресурсах под именем TMYFORM. Ну и, наконец, VCLотличается от большинства библиотек аналогичной направленности обширной и довольно сложной иерархией классов.

Давайте возьмем какую-нибудь программу на Delphi (я взял одну из своих разработок) и попробуем повторить эксперимент с удалением шаблона какого-нибудь диалога. Открываем файл программы в ResourceHacker, выбираем любую понравившуюся форму – и видим, что ResourceHackerнещадно глючит, пытаясь перевести шаблон из внутреннего борландовского формата в более удобочитаемое **нечто**. На самом деле ResourceHackerдовольно успешно понимает формат Borland'овских шаблонов – просто я использовал слишком новую версию Delphi, о которой наш редактор ресурсов, увы, ничего не знает. Кстати, если взглянуть на тот же ресурс в шестнадцатиричном редакторе, Вы увидите перед собой малопонятную кучу байт, в которую вкраплены имена компонентов и их свойств, а также текстовые значения. Но мы пока не будем пытаться дешифровать шаблон, а просто удалим его. Удаляем, запускаем, пытаемся вызвать соответствующее окно – и получаем сообщение Resource <имя удаленного нами ресурса> not found. Да, это определенно не то, о чем мы мечтали – nagscreen, конечно, исчез, но окошко с сообщением об ошибке смотрится ничуть не лучше. Увы, так просто от nagscreen'ов в программах на Delphiне избавиться.

Поскольку ResourceHackerпоказал себя не с лучшей стороны, внесем коррективы в наш инструментарий: возьмем eXeScope 6.41 (последняя версия на момент написания данной главы) – эта программа более-менее успешно переваривает шаблоны Delphi 7 с надписями в кодировке Unicode. Если Вы не хотите нарушать ничьих авторских прав хотя бы в процессе обучения, раздобудьте для экспериментов мою программку InqSoftWindowScanner – лицензионное соглашение дает Вам полный карт-бланш на ее ломание в учебных целях. Распакуйте ее UPX'ом, откройте полученный исполняемый файл в eXeScopeи найдите шаблон формы TAboutForm. Вы увидите что-то вроде:

```
object AboutForm: TAboutForm
  Left = 265
  Top = 185
  BorderIcons = [biSystemMenu]
  BorderStyle = bsSingle
  Caption = #1054' '#1087#1088#1086#1075#1088#1072#1084#1084#1077'...'
  ClientHeight = 192
  ClientWidth = 318
  ...
end
…
```

Если Вы раньше программировали на Delphiили C++ Builder, Вы наверняка уже догадались, что это – текст типичного dfm-файла, в котором описан шаблон формы вместе со всеми ее компонентами, как визуальными, так и не очень. Если же Вы не программировали ни на Delphi, ни на C++ Builder – Вам будет непросто разобраться в ломании программ, написанных при помощи этих средств разработки. Однако в этой главе мы будем разбирать достаточно простые вещи, для понимания которых достаточно знания английского языка, основ ООП и знакомства с любым визуальным средством разработки. В конце-концов, тот, кто умеет помещать управляющие элементы на шаблоны диалогов в RadAsm'е, вряд ли испытает какие-либо трудности в понимании того, как бросать точно такие же компоненты на формы в Delphi. Но в общем случае работает совершенно обратное правило: **чтобы эффективно исследовать программу, необходимо иметь представление об инструментах, с использованием которых эта программа была написана**. То есть, чтобы взломать программу, написанную на C++ Builder, нужно знать характерные особенности C++ Builder.

Одной из таких характерных особенностей, к примеру, является то, что имена классов окон (понятие «класс окна» здесь используется в том же смысле, что и в документации по WindowsAPI) в Delphi/C++ Builderначинаются с буквы Ти совпадают с именами классов форм. Проще говоря, если программа создает окно как экземпляр класса TMyForm, то имя класса окна, возвращаемое WinAPI'шной функцией GetClassName, тоже будет TMyForm. Собственно, InqSoftWindowScannerв качестве объекта для экспериментов я посоветовал не случайно – эта программа, помимо прочих функций, как раз умеет определять имена классов окон. Как Вы помните, имена классов форм в программе совпадают со строковыми идентификаторами шаблонов этих форм в секции ресурсов, поэтому «подглядев» средствами WinAPIимя класса окна, Вы смело можете шерстить секцию ресурсов на предмет наличия ресурса типа RCDATAс соответствующим идентификатором, и, скорее всего, Вы такой ресурс обнаружите.

Строго говоря, использование механизмов наследования в Delphi/C++ Builder позволяет создавать формы, у которых имя класса окна отличается от символьного идентификатора шаблона, но в большинстве программ эта возможность не используется. Да и распознать форму-наследника по dfm-описанию родителя обычно бывает не так уж сложно, поэтому я не буду заострять внимание на этом достаточно специфическом случае.

Итак, что нам показал eXeScope? А показал он описание формы и свойства всех ее компонентов, причем в самом что ни на есть текстовом виде. Вы наверняка уже поняли, что слово **object** начинает описание объекта (или компонента, если быть до конца точным), а слово **end** это описание завершает. И что строки вроде **Left = 265**, **BorderIcons = [biSystemMenu]** или **BorderStyle = bsSingle** представляют собой ни что иное, как имена свойств компонентов и значения этих свойств, как они были заданы во время разработки приложения. И, что самое приятное, Вы можете не только смотреть на эти свойства, но и редактировать их. Исправить числовые или текстовые значения – дело нехитрое, стираем старое значение, вписываем новое, сохраняем результат, и можно любоваться эффектом. Но вот если понадобится исправить свойство вроде BorderStyle – скорее всего придется заглянуть в документацию по Delphi, чтобы узнать, что такое bsSingleи какие еще значения может принимать это свойство. Пока что все довольно просто – и чем-то напоминает наши эксперименты по редактированию шаблонов диалогов в Калькуляторе.

Впрочем, сказав, что изменить значение свойства – дело нехитрое, я был не совсем прав. Состояние современного инструментария таково, что если пользоваться специализированными программами (тем же eXeScope, например), то эта операция не всегда выполняется корректно. Поэтому на практике мне нередко приходилось пользоваться следующим приемом – оригинальное значение я читал при помощи eXeScope, но вот новые данные вписывал уже в шестнадцатиричном редакторе; о том, как я определял, какие байты и где надо было менять, я думаю, в очередной раз повторяться не стоит. В самых тяжелых случаях можно даже экспортировать шаблон формы в dfm-файл, а затем попытаться при помощи той же версии Delphi/C++ Builderвпихнуть этот dfm-файл в тестовый проект, откомпилировать и затем перенести шаблон формы из тестового проекта на его «родное» место в подопытной программе. При этом, разумеется, придется обеспечить совпадение версий компилятора и, если на форме имеются нестандартные компоненты, раздобыть и установить точно такие же. Как Вы догадываетесь, весь этот процесс весьма хлопотный, и такая игра вряд ли стоит свеч – я подобную «хирургическую операцию» по пересадке ресурсов проделывал лишь единожды, и то в основном в порядке эксперимента. Но вот изучение тестовых примеров перед тем, как начинать править свойства – вещь очень и очень полезная, поскольку помимо чисто практической пользы это помогает лучше понять внутреннюю логику инструментальных средств, при помощи которых создана программа.

После того, как мы убедились в возможности редактировать отдельные свойства компонентов, было бы нелишне узнать, что произойдет в том случае, если удалить описание какого-либо свойства. Как ни странно, если выполнить удаление корректно – то обычно ничего страшного не

случается. Хотя, справедливости ради надо отметить, что в некоторых случаях удаление свойства приводит к катастрофическим последствиям – прежде всего, когда удаляемое свойство ссылается на другой компонент. Но, к счастью, такие специфические компоненты чаще всего не имеют никакого отношения в nagscreen'ам и баннерам.

Причина «нечувствительности» программ к исчезновению свойств следующая: в Delphiи C++ Buildercоздание объекта на основе шаблона производится в два этапа. На первом этапе собственно создается объект, а все его свойства и внутренние структуры инициализируются значениями по умолчанию. И лишь на втором этапе новые значения свойств, которые хранятся в шаблоне диалога, помещаются на место значений по умолчанию. Неудивительно и то, что в eXeScopeВы видите у каждого компонента намного меньше свойств, чем видит разработчик, редактируя форму в IDE: для уменьшения объема исполняемого файла в ресурсах сохраняются не все значения свойств, а только те, которые отличаются от значений по умолчанию. Например, в приведенном выше отрывке описания формы AboutFormВы не увидите свойство AutoSize, несмотря на то, что такое свойство у формы имеется. Причина этого проста: значение свойства AutoSizeпо умолчанию было равно false, и автору (то есть мне) не пришлось его менять, поскольку оно вполне меня устраивало.

Итак, теперь мы знаем, что описания свойств удалять можно. Осталось лишь разобраться, зачем это нам может понадобиться. В этот раз я отступлю от принципа «сначала - теория, потом - практика» и начну с описания небольшого эксперимента. Возьмите все тот же InqSoftWindowScanner, запустите и нажмите в нем кнопку «О программе…». Вы увидите окно с информацией о программе, в левой части которого будет находиться логотип. А теперь представьте себе, что это не безобидная картинка, которая никому не мешает, а ужасный черно-зелено-оранжевый баннер размером в пол-экрана, от которого мы хотели бы избавиться.

Следующим шагом будет определение идентификатора шаблона. Запустите вторую копию WindowScanner'a и «просканируйте» окно «О программе…». В очередной раз убедившись, что класс окна носит имя TAboutForm (да, я знаю, что написал про это несчастное окно уже как минимум пять килобайт – но мы будем действовать так, как будто видим программу впервые), лезем в секцию ресурсов и ищем там шаблон формы, спрятавшийся за идентификатором TABOUTFORM. Теперь нам нужно найти в шаблоне формы описание нашего «баннера». Нетрудно догадаться, что в этой роли выступает объект Image1 (поскольку картинка на форме одна, догадаться, что Image1 и есть наш «баннер», нетрудно). Дешифрованное описание этого объекта выглядит следующим образом:

```
object Image1: TImage
  Left = 0
  Top = 0
  Width = 57
  Height = 192
  AutoSize = True
  Picture.Data = {
    0A544A504547496D616765F5260000FFD8FFE000104A4649460001020000010O
    ............
    3EEBFFD9}
end
```

Теперь можно сделать то, ради чего мы и затевали все эти поиски: удалите свойство Picture.Data, запустите программу и посмотрите, что случилось с нашей формой. А случилось именно то, чего мы и хотели добиться - логотип исчез, как будто его и не было. Сам объект, разумеется, никуда не делся – мы лишь удалили связанное с ним изображение, «подчистив» свойство Picture. Вспомните, что я говорил о двух этапах создания форм в Delphi - и Вы легко поймете, каким образом изменилась логика работы программы: стерев значение, которое было назначено свойству Picture.Data программистом, Вы не оставили программе иного выхода, как использовать значение этого свойства по умолчанию. А значением по умолчанию в нашем случае оказалось «никакое» изображение (т.е. отсутствие изображения), которое программа успешно отобразила. Более того,

удалять можно не только свойства объектов, но и объекты целиком. Однако такое действо является куда более грубым вмешательством в код программы, да и ограничений в этом случае намного больше.

А теперь давайте посмотрим, как простым редактированием ресурсов можно избавиться от настоящего, вполне реального баннера. Для этого нам потребуется программа GhostInstallerFreeEdition. XML-редактор, входящий в состав этой программы, как раз содержит баннер. И этот баннер, кроме того, что занимает довольно много места внизу экрана и сокращает поле редактирования, еще и постоянно мерцает, раздражая пользователя. Так что если Вы активно пользуетесь бесплатной редакцией GhostInstaller'а, Вам придется либо проявлять чудеса терпеливости, либо спасти огромное количество нервных клеток, избавившись от созерцания зловредного баннера.

Сначала при помощи InqSoftWindowScanneropределим имя класса окна той формы, которую мы собираемся править: TfrmProjectSource. Заодно будет нелишне просканировать и сам баннер – вдруг это позволит нам узнать хотя бы тип компонента, на основе которого этот баннер был создан. А если нам удастся узнать тип компонента, это заметно облегчит задачу поиска самого компонента в дешифрованном шаблоне формы. Пробуем просканировать окно баннера, и узнаем, что на самом деле наш баннер – ни что иное, как кусок InternetExplorer'а, засунутый в программу при помощи технологии ActiveX. Чтож, с первого захода мы получили явно недостаточно полезной информации. Поэтому попробуем копнуть глубже и просканить всю ветку дерева окон, имеющую отношение к нашему баннеру. InqSoft Window Scanner выдал следующий результат:

```
+[00050356] {TPanel}
 *[00010414] {TElSplitter}
 +[00050370] {TPanel}
  +[0001040E] {TPanel}
   +[00010410] Panel1 {TPanel}
    +[0043036A] {Shell Embedding}
     +[000103D4] {Shell DocObject View}
      *[00010412] {Internet Explorer_Server}
```

Что это значит? Всего лишь то, что наш ошметок InternetExplorer'а лежит на компоненте Panel1 типа TPanel (тут программист явно поленился стереть свойство Captiony объекта Panel1, чем сильно облегчил нам задачу поиска нужного объекта). Сам объект Panel1 в свою очередь лежит на другой панели, которая лежит на панели, которая… В общем, всяких панелей там много, и разбираться в них можно долго. Поэтому я предлагаю начать наше расследование с той панели, которую мы идентифицировали однозначно. Откроем файл GIEditor.exeв Restorator 2.52 (да, Вам опять придется обновить инструментарий – практика показала, что eXeScopec редактированием шаблона в данной программе не справился, а вот Restoratorсработал как нельзя лучше), выберем шаблон нашей формы и в этом шаблоне найдем следующее:

```
object panMainBanner: TPanel
  Left = 0
  Top = 287
  Width = 553
  Height = 126
  Align = alBottom
  BevelOuter = bvNone
  TabOrder = 1
  object Panel1: TPanel
    Left = 0
    Top = 4
    Width = 553
    Height = 122
    Align = alBottom
    BevelOuter = bvLowered
    Caption = 'Panel1'
    TabOrder = 0
    object webMainBanner: TEmbeddedWB
      Left = 1
```

```
        Top = 1
        Width = 551
        Height = 120
        Align = alClient
        ...
        end
      end
  end
```

Как видно из приведенного куска текста, это и есть описание нашей Panel1, баннера, который на этой панели находится (как оказалось, он представлен объектом webMainBanner типа TEmbeddedWB) и некой панели panMainBanner, на которой лежит Panel1. Было бы логично предположить, что имя panMainBannerрасшифровывается как panelforMainBanner («панель для главного баннера»), и что если ликвидировать эту панель вместе со всем ее содержимым, то назойливое мерцание баннера более не будет смущать наш взор. Сказано – сделано, переходим в режим редактирования и удаляем вышеприведенный блок текста из ресурса. Сохраняем, запускаем – и получаем GPF. Чтож, как я и предупреждал, безоглядное удаление компонентов – операция отнюдь не безобидная. Поэтому давайте поищем другой путь.

В самом начале главы я выделил четыре подхода к проблеме, и мы только что испробовали тот из них, который стоит под номером два. Это, конечно, было несколько непоследовательно с моей стороны, но иначе Вы бы не смогли поэкспериментировать с удалением объектов и увидеть, к чему это иногда приводит. И вот пришло время пойти правильным путем. Действительно – это ведь всего лишь баннер, он не выпрыгивает в самый неподходящий момент на передний план, не блокирует работу с программой, даже не требует, чтобы по нему кликали. Так что нам нет принципиальной необходимости совершенно изничтожать этот баннер –достаточно просто его не видеть. В нашем случае, кстати, придется убрать не только баннер, но и панели, на которых он расположен, чтобы они не занимали место, и самый простой способ сделать это – уменьшить высоту панелей до нуля. Если быть до конца точным – нам надо спрятать лишь панель panMainBanner, поскольку компоненты webMainBanneri Panel1 находятся внутри этой панели и после «исчезновения» panMainBannerтоже не будут видимы (в общем случае этому могло бы помешать свойство AutoSize=true, но у наших подопытных компонентов это свойство равно false). В Restorator’е исправляем строчку Height=126 на Height=0, сохраняем результат и запускаем многострадальную программу. Баннера больше нет!

Читая эту главу, Вы, наверное, отметили, что я меньше, чем обычно, говорил про общие подходы к проблеме рекламных окон. Действительно, чем более частные вопросы крэкинга мы рассматриваем, тем сильнее приходится «привязываться» к особенностям операционных систем и средств разработки. Но даже в этом случае «за бортом» моего повествования осталось очень многое: визуальные средства для разработки под ДОС (да, были и такие – и некоторые из них, вроде старых версий FoxPro, все еще используются), использование шаблонов диалогов в VisualBasic/VBA и т.д. Вы сами, при желании, сможете узнать об этих вещах через самостоятельные эксперименты ничуть не меньше, чем я мог бы Вам сообщить. А поскольку сама идея хранить шаблоны интерфейсов программ в унифицированном виде давно и прочно вошла в практику программирования, было бы наивным предполагать, что новые средства разработки не принесут с собой новых форматов хранения этих шаблонов. Но, я надеюсь, и в этом случае Вы сможете извлечь пользу из информации, изложенной в этой главе – разумеется, не как из руководства по конкретным инструментам, а как из источника идей.

Direct I/O

Автор: bogrus

Дата: 2004-10-17

«Direct I/O is the world's first device driver for Microsoft and which enables the direct hardware access for your existing software without any programming efforts on your side».

Введение

Из вышеприведенной цитаты, думаю понятно, что объектом исследования является приложение, позволяющее другим программам обращаться напрямую к оборудованию. С этим не возникало проблем, когда программы работали под управлением операционных систем Windows, предшественников Windows NT, но времена идут, архитектура меняется, а переделывать старые программы, зачастую просто нет возможности. Принцип, по которому работает драйвер Direct I/O (и аналогичные программы) не нов, и был опубликован в журнале Dr.Dobb's за май 1996 года, автором статьи (русский перевод которой можно найти на void.ru) является Дейл Робертс.

Видимо многим low-level программистам не составит особого труда реализовать аналогичную программу, времени на это может уйти меньше, чем на написание этой статьи и окажется это более достойным занятием, чем- то, которым будем заниматься мы, а именно - исследованием той части программы, которая касается её защиты от нелицензионного использования. Используемая при этом технология довольно простая, но может кому-то показаться интересной, т.к. статья имеет исключительно, и по возможности максимально, познавательный характер. Это путь авторам, который покажет слабые места их программ, покажет, чем пользуются, и что руководит людьми, когда нужная им программа отказывается работать больше месяца, когда нет денег, кредитной карты или просто рядом нет банка.

Знакомство

Итак, что происходит, когда нет вышеперечисленных проблем:

• Вы покупаете программу и сообщаете автору *Name* или *Company* , на которые желаете зарегистрировать программу, после чего получаете *Key*

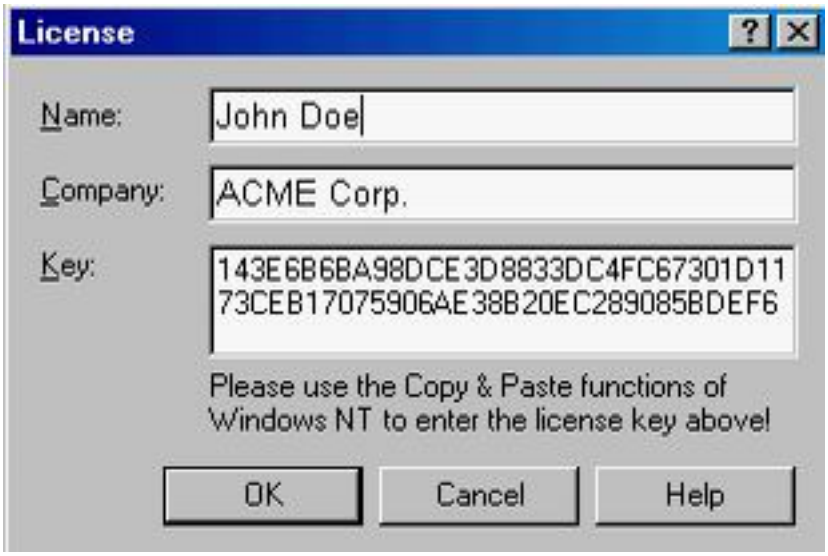
«The license information consists of three entries: Name, Company and Key._Name or Company may be empty , but *Key always is a 128 character long string* formatted into two lines as shown in the example below». _

• Пример регистрационных данных, мы попробуем их ввести в программу:

Name:	**John Doe**
Company:	ACME Corp.
Key:	143E6B6BA98DCE3D8833DC4FC67301D1173CEB17075906AE38B20EC289085BDE F65744EB4561936040A6B352ADC6ED4E585FC8075FA55927CAE6414A7215A9E7

• После того, как в окне «License» (Рис.1) будут введены правильные данные (которых у нас нет), программа начинает работать в полнофункциональном режиме. Иначе она работает в 30-ти дневном trial-режиме и при каждой её инициализации выводит наг-скрин (Рис.2)

Рис.1	**Рис.2**





Это не трехзначный пароль, который можно подобрать даже руками, столько букв, цифр ... Давайте для начала соберем максимально возможную информацию о программе, о её обращениях к реестру, к файлам, о взаимодействие с операционной системой. Лучше всего это начинать ещё во время установки программы, но мы сделаем только в тот момент, когда будем нажимать кнопку «OK» (Рис.1)

Сокрытые ключи реестра

Начнём с реестра, обычно там хранятся настройки программ и регистрационные данные, для этого воспользуемся утилитой Regmon, и вот что интересного мы сразу замечаем:

Process	Path	Other
rundll32.exe	HKLM\SOFTWARE\Paule\Direct I/O\CurrentVersion\Name	John Doe
rundll32.exe	HKLM\SOFTWARE\Paule\Direct I/O\CurrentVersion\Company	ACME Corp.
rundll32.exe	HKLM\SOFTWARE\Paule\Direct I/O\CurrentVersion\Key	14 3E 6B ...
rundll32.exe	HKLM\SOFTWARE\Microsoft\Cryptography\Defaults\Provider \Microsoft Base Cryptographic Provider v1.0\Image Path	rsabase.dll
rundll32.exe	HKLM\SOFTWARE\Paule\Direct I/O\CurrentVersion\TimeLock\Licensed	0x0

Process	Path	Other
rundll32.exe	HKLM\SOFTWARE\Paule\Direct I/O\CurrentVersion\TimeLock\InstallTime	00 E9 6B ...

Обратите внимание, что процессом является rundll32.exe, а почему, ответ найдем в свойствах ярлыка программы, запускается она такой строкой: «C:\WINNT\system32\rundll32.exe shell32.dll,Control_RunDLL DirectIO.CPL». Расширение .CPL означает, что это аплет панели управления.

Из таблицы теперь понятно, в какой ветке реестра программа хранит регистрационные данные, также понятно, что задействована криптография и ... сначала не верим своим глазам, но «Licensed» - ведь это слово обозначает, зарегистрирована ли программа, предполагает два ответа: да или нет, true или false, 1 или 0. Наивно, но почему бы не открыть regedit и проставить 0x1? Конечно, пробуем и вот здесь мы сразу спотыкаемся на «первом уровне» защиты.

Regedit нам сообщает, что не удается открыть «TimeLock». Ошибка при открытии раздела. Пробуем другие программы для работы с реестром и снова неудача, ни одна не может открыть эту ветку реестра, хотя Regmon то, ведь все отлично показывает, все параметры мы в нем видим. Неужели программа контролирует доступ к своей ветке, маловероятно, но проводим эксперименты вплоть до ручного удаления всех файлов программы: directio.cpl, directio.drv, directio.vdd, directiocoinst.dll, directio.sys, directiofilter.sys, и это все равно ничего не дает. Переходим к поиску в сети любой информации, которая может касаться этой ошибки работы с реестром, попутно задаем вопрос на форуме wasm.ru, где и получаем точный ответ от Four-F: «На sysinternals.com в разделе "Systems internals tips and trivia" есть маленький параграф "Hidden Registry Keys?" и исходник reghide.zip.»

Так и есть, для защиты используется описанное Марком Руссиновичем (автор Regmon-a) различие работы со строками между Win32 API и Native API:

«How is this possible? The answer is that a name which is a counted Unicode string can explicitly include NULL characters (0) as part of the name. For example, "Key\0". To include the NULL at the end the length of the Unicode string is specified as 4. There is absolutely no way to specify this name using the Win32 API since if "Key\0" is passed as a name, the API will determine that the name is "Key" (3 characters in length) because the "\0" indicates the end of the name».

Коротко говоря, когда вы создаете ключ реестра с помощью RegCreateKey (Win32 API), то передаете строку с именем ключа (например «Key»), но длину строки вы не указываете, функция высчитывает ее сама (как только встретит 0x0). А при использовании функции NtCreateKey (Native API), мы обязаны явно указать какой длины хотим создать ключ, по этому для строки «Key» можно определить длину как 4 и потом эту ветку, из-за несовпадения длин, будет невозможно открыть функциями Win32 API.

Значит для того, чтобы изменить параметр «Licensed» нам нужно использовать Native API, в исходнике reghide.zip можно (достаточно для начала) почерпнуть информации, как это сделать.

К сожалению, я (возможно и вы) ещё не знакомы с языком C, но уже знаете или Вас больше привлекает ассемблер, по этому именно на нём сделаем программу, главной целью которой будет изменить параметр «Licensed» с 0x0 на 0x1 . Будем использовать при этом FASM. Ниже я сразу приведу готовый код, скажу только, что все необходимые структуры, константы и некоторые примеры для работы с Native API, можно найти в Kernel Mode Driver Development Kit by Four-F.

```
;==============================================================
format      PE GUI 4.0
entry       start
;==============================================================
include     '%fasminc%\win32a.inc'
;==============================================================
KEY_SET_VALUE       =  2h
OBJ_CASE_INSENSITIVE=  40h
```

```
;=================================================================
struct              OBJECT_ATTRIBUTES
      .length       dd sizeof.OBJECT_ATTRIBUTES
      .directory    dd 0
      .object       dd sregpath
      .attributes   dd OBJ_CASE_INSENSITIVE
      .descriptor   dd 0
      .service      dd 0
ends
oa                  OBJECT_ATTRIBUTES
;=================================================================
sregpath    dd      sregparam-regpath            ; sizeof.regpath
oregpath    dd      regpath                      ; offset regpath
regpath     du      '\Registry\Machine\SOFTWARE\Paule\Direct I/O'
            du      '\CurrentVersion\TimeLock',0
;=================================================================
sregparam   dd      ntdll-regparam              ; sizeof.regparam
oregparam   dd      regparam                    ; offset regparam
regparam    du      'Licensed'
;=================================================================
ntdll       db      'ntdll.dll',0
ntok        db      'NtOpenKey',0
ntsk        db      'NtSetValueKey',0
ntc         db      'NtClose',0
;=================================================================
hkey        dd      0
param       dd      1                                       ; 0x1
;=================================================================
start:      invoke  GetModuleHandle,ntdll
            mov     esi,eax
            invoke  GetProcAddress,esi,ntok
            push    oa KEY_SET_VALUE hkey
            call    eax                                ; NtOpenKey
            invoke  GetProcAddress,esi,ntsk
            push    4 param REG_DWORD 0 sregparam [hkey]
            call    eax                              ; NtSetValueKey
            invoke  GetProcAddress,esi,ntc
            push    [hkey]
            call    eax                                  ; NtClose
            invoke  ExitProcess,0
;=================================================================
data        import
library     kernel32,'kernel32.dll'
include     '%fasminc%\apia\kernel32.inc'
end         data
;=================================================================
```

В этом коде для лучшего понимания исключена обработка ошибок, но в процессе его тестирования вы должны проверять результат, возвращаемый функциями NtOpenKey, NtSetValueKey, в случае ошибки он будет отличен от нуля.

Ну а теперь, стоит посмотреть на результат работы этой программы :)

Рис.3	**Рис.4**





Сначала мы имеем лицензионную версию (Рис.3), но как только воспользуемся услугами драйвера или просто нажмем кнопку «License», то «лицензионность» сразу улетучивается (Рис.4), понятно, что программа, независимо от параметра «Licensed», все равно проверяет корректность регистрационных данных. Назовем эту проверку «вторым уровнем» защиты (криптографическим).

Перед его изучением на минуту вспомним о ещё одном параметре - «InstallTime». Назначение его понятно, от него отсчитывается 30 дней, теперь мы знаем, как его можно изменить, тем самым, продлив trial-режим. Но это ещё не тот выход, который спасет от наг-скрина (Рис.2)

Цифровая подпись RSA

Итак, на «второй уровень» мы имеем такие данные:

• Key (128 hex-символов)
• Microsoft Base Cryptographic Provider v1.0
• rsabase.dll (Filemom подтверждает обращение к этому файлу)

Соберем ещё немного информации, а точнее посмотрим в секцию импорта directio.cpl, на предмет всяческих Crypto API функций. Инструментов для этого множество, я использовал PE Explorer (Рис.5), хотя можно обойтись простым поиском слова «Crypto» в файле directio.cpl .

Рис.5

RVA	Name
1000CFC8h	MFC42u.DLL
1000D076h	MSVCRT.dll
1000D1E2h	KERNEL32.dll
1000D2E8h	USER32.dll
1000D35Eh	GDI32.dll
1000D5DCh	ADVAPI32.dll
1000D5FAh	SHELL32.dll
1000D648h	VERSION.dll

RVA	Hint	Name
1000A050h	00EDh	RegQueryValueExW
1000A054h	00A0h	CryptReleaseContext
1000A058h	008Ch	CryptDestroyKey
1000A05Ch	008Bh	CryptDestroyHash
1000A060h	00ABh	CryptVerifySignatureW
1000A064h	009Dh	CryptHashData
1000A068h	0088h	CryptCreateHash
1000A06Ch	009Fh	CryptImportKey
1000A070h	0085h	CryptAcquireContextA
1000A074h	00FAh	RegSetValueExW

Здесь, тем, кто не знает, что делают эти функции, предлагаю воспользоваться поиском в сети, чтобы пройти первоначальный ликбез, который, в свое время прошел я, но, вспоминая, что лень мы обычно перебарываем, только когда прижмет, коротко приведу мысли, которые могут крутиться в голове у ленивого человека:

• CryptImportKey (куда-то импортирует какой-то ключ, какой?)
• CryptVerifySignatureW (сверяет сигнатуру, подпись с чем-то, с чем?)

Не сладко ... Инструментов, призванных нам помочь очень много, но мы добавим в наш арсенал ещё несколько:

• IDA (Interactive Disassembler)
• OllyDbg (32-bit assembler level analysing debugger)
• Hex Workshop (Hex Editor)

Сразу замечу, что человек, имеющий некоторый опыт, знание ассемблера, который знаком с функцией CryptVerifySignature, уже давно понял, где и что нужно поправить, затратив на всё не более получаса, а мы ленивые будем изучать то, без чего могли бы в данном случае обойтись.

Экскурсия к прову

Regmon нам показывал обращение к какому то провайдеру ... Естественно всемирная паутина здесь не при чем, и об этом коротко.

Криптопровайдер (CSP) это динамическая библиотека (.dll), содержащая различные функции (процедуры) соответственной тематики. Именно они выполняют шифрование, хеширование, проверку цифровой подписи и множество других вычислительных действий. С операционной системой Windows поставляется несколько провайдеров, в том числе и Microsoft Base Cryptographic Provider (rsabase.dll).

При создании своих собственных CSP можно воспользоваться «инструментом» разработчика - Microsoft Cryptographic Service Provider Development Kit, есть маленький, но существенный нюанс. Когда ваша dll будет готова, нужно ее «подписать» у Microsoft, и тогда любые приложения смогут обращаться к ее функциям, кроме того, если я не ошибаюсь, то она будет защищена операционной системой от изменений. А для ее инсталляции достаточно создать необходимые ключи и параметры, посмотрите у себя ветку HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Cryptography\Defaults\Provider

Проследив в отладчике, куда ведет функция CryptVerifySignature, может возникнуть вопрос, почему advapi32.dll, а не rsabase.dll? Таким образом Microsoft избегает разнообразия, возможной запутанности при работе с разными CSP, а также появляется некая универсальность для приложений, они могут работать с разными криптопровайдерами не изменяя свой код. Приложению даже не обязательно знать, с каким CSP оно имеет дело, посредником между ними выступает advapi32.dll, она «преобразовывает» вызов, например CryptVerifySignature, в вызов

CPVerifySignature, а последняя является, реально проверяющей подпись функцией (в нашем случае она из rsabase.dll). Такое же преобразование выполняется и для остальных.

Вызывая CryptAcquireContext, мы можем не указывать CSP, она сама выберет используемого в вашей ОС по-умолчанию криптопровайдера. Эта функция перебирает, упомянутую выше, ветку реестра (это обращение мы и поймали Regmon-ом) и в зависимости от переданных ей параметров, определяется с CSP и загружает его dll в адресное пространство нашего процесса. После чего вызывает CPAcquireContext и проверяет целостность криптопровайдера. Много действий выполняет CryptAcquireContext, по этому она довольно медленная.

Предварительный анализ

Давайте ознакомимся, как в программе вызываются эти функции, и с какими параметрами.

Открываем IDA, загружаем туда directio.cpl, после предварительного анализа переходим к вкладке Names и ищем функцию CryptVerifySignatureW, кликнув на ней, мы попадаем в секцию импорта. Перед именем функции будет слово BOOL, что означает «Boolean» и предполагает два варианта, возвращаемых функцией, о них мы уже говорили, помните true или false, 1 или 0. Именно так, если наш Key правильный, то она вернет истину, иначе ложь. Кто не верит, может посмотреть в MSDN :) Ниже имени функции будут такие строчки:

.idata:1000A060;	**DATAXREF:**	**sub_10006240+275r**
.idata:1000A060;	DATAXREF:	sub_10006940+C0r

sub_10006240 и sub_10006940 показывает, в каком месте программы вызывается эта Crypto API функция, и как видим, это происходит в двух разных местах, переходим к ним (с мыслью, почему их два). При долгом всматривании (найди различия), ответ будет очевиден, в одном месте используются параметры (данные), зашитые в программу, а в другом, возможно, наши данные из реестра.

Первый алгоритм проверки

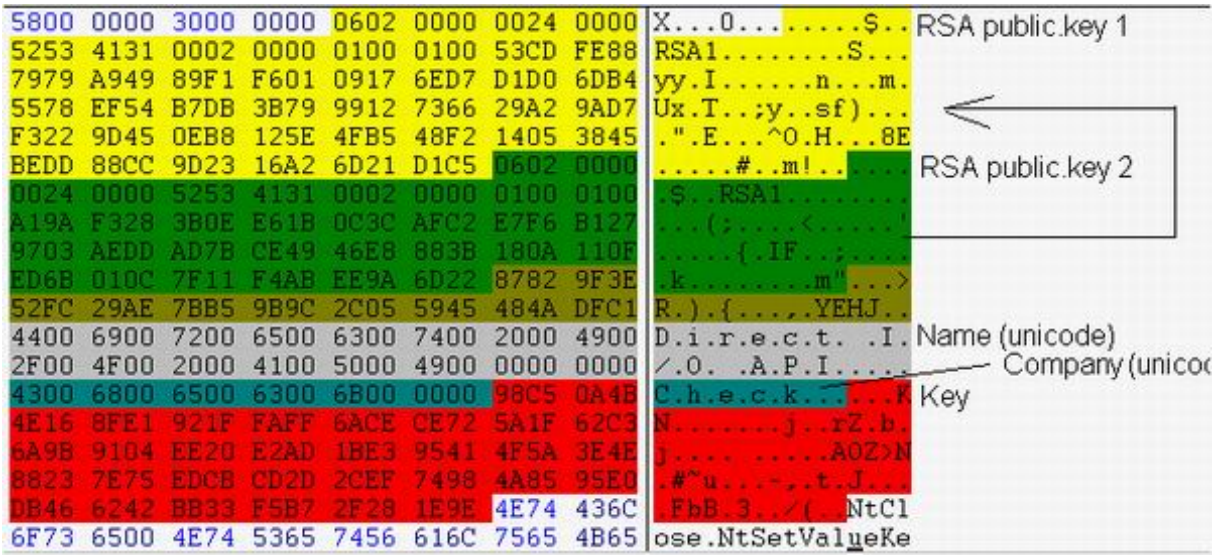
Начнем с простого и интересного, какие данные зашиты в программу и зачем?

Данными здесь являются параметры, передаваемые в вышеупомянутые функции Crypto API, IDA нам их покажет, логика этой проверки такая:

```
;===========================================================================
cspname     db      'Microsoft Base Cryptographic Provider v1.0',0
csp         dd      0
hash        dd      0
hkey        dd      0
pubkey      db      (Рис.6)
Name        db      (Рис.6)
Company     db      (Рис.6)
Key         db      (Рис.6)
;===========================================================================
invoke  CryptAcquireContext,csp,0,cspname,PROV_RSA_FULL,CRYPT_VERIFYCONTEXT
invoke  CryptImportKey,[csp],pubkey,54h,0,0,hkey
invoke  CryptCreateHash,[csp],CALG_MD5,0,0,hash
invoke  CryptHashData,[hash],Name,1Ch,0
invoke  CryptHashData,[hash],Company,0Ah,0
invoke  CryptVerifySignature,[hash],Key,40h,[hkey],0,0
        test eax,eax
        jz   false
;===========================================================================
```

Сейчас мы можем наглядно посмотреть на зашитые в directio.cpl данные (Рис.6) и сопоставить их с логикой проверки.

Рис.6



Более того, в Hex Workshop, вы можете их «вырезать», а потом, используя (входящую в пакет MASM32) утилиту Bintodb, конвертировать эти данные в массив байт, пригодный для компиляции с вышеприведенным исходным текстом. Сделав это, мы убеждаемся, что восстановили алгоритм проверки верно, вшитые данные полностью корректны, т.к. функция CryptVerifySignature возвращает 1 (true), и нам это здорово помогло осмыслить алгоритм.

Теперь мы понимаем, что CryptImportKey импортирует RSA public.key 2, а наш 128-ми символьный ключ это сигнатура, которую проверяет CryptVerifySignature, по два hex-символа на байт (максимальное его значение FFh), получится 128/2=64 байт, т.е. параметр «40h», это длина подписи.

Наивно полагая, а вдруг поможет, почему бы ни попробовать ввести эти «выдранные» и корректные данные в окне «License» (Рис.1) Пробуем, и улыбаемся до самых ушей :), какие же мы все-таки бываем наивными! Раз не проходит, значит, данные не верны, значит, не этим алгоритмом проверяются регистрационные данные. А этот алгоритм, получается, служит просто для проверки корректности работы Crypto API, ним проверяются только зашитые данные.

Вторая проверка, выводы и результаты

Внимательно сравнивая, чем же отличается одна проверка от другой, замечаем, что они используют разные данные для параметра pubkey, т.е. разные публичные ключи для проверки цифровой подписи.

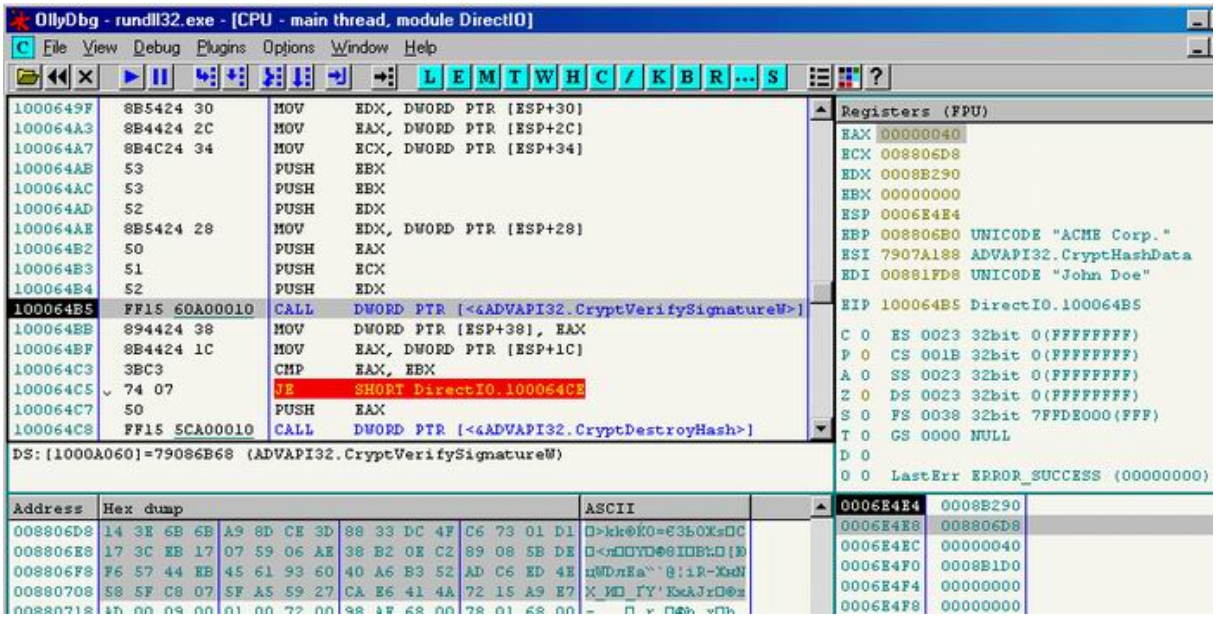
Что если попробовать переписать RSA public.key 2 на место RSA public.key 1 (Рис.6), а потом ввести зашитые в программе регистрационные данные? С этим нет проблем, два клика в Hex Workshop, сохраняем файл. Теперь вводим в качестве Name строку «Direct I/O API», в качестве Company «Check», а в качестве ключа 128 символов, помеченных красным. Если сделали всё правильно, можем забыть о наивности, *программа полноценно работает*!

А если «подправить» код так, чтобы результат, возвращаемый функцией CryptVerifySignature, для программы всегда был true? Тогда никаких регистрационных данных, не нужно будет вводить вообще.

Без проблем, для наглядности сделаем это в отладчике OllyDbg. Запустим Direct I/O, откроем окно «License» (Рис.1), чтобы нам осталось только нажать кнопку «OK», в OllyDbg сделаем «Attach» к процессу rundll32 (с окном «License»). Потоки процесса теперь будут временно приостановлены, а мы в отладчике поставим breakpoint на вызов CryptVerifySignature, это можно сделать разными способами, если установлен plug-in CommandBar, то в нем достаточно написать «bp ADVAPI32.CryptVerifySignatureW», после этого можно продолжить выполнение процесса. В Direct

I/O нажимаем кнопку «OK», срабатывает брэкпоинт и вы должны наблюдать такую картину:

Рис.7



Присмотревшись, на Рис.7 можно увидеть все наши регистрационные данные, но давайте взглянем на участок кода, следующий сразу за вызовом CryptVerifySignature:

Смещение	Опкод	Мнемоника
100064B5	FF15 60A00010	Call CryptVerifySignatureW
100064BB	894424 38	mov [esp+38],eax

Думаю не секрет, что после вызова функции, её результат будет в регистре eax, т.е. именно там будет 0x1 в случае удачи, иначе 0x0, по этому коду видно, что потом результат заноситься в стековую переменную. После вызова функции мы можем, прямо в отладчике записать в регистр eax единицу, продолжить выполнение программы и снова убедиться, что она считает себя «зарегистрированной».

Чтобы сделать этот эффект постоянным, нам нужно изменить код, на что-то вроде «mov [esp+38],1», но мы должны следить за размерами опкодов, одним байтом в сторону и «Стой, стреляю!». Здесь нужно четко знать, где и на что можно править, для этого пригодятся мануалы Intel и Книга Опкодов от The Svin. Вскоре мы понимаем, такая запись единицы в переменную будет стоить 8 байт, что равносильно расстрелу. Не долго думая, мы приходим к другому варианту, что проще и безболезненно для программы, сделать инкремент (увеличение на единицу) стековой переменной, к тому же интуитивно уверены, что до этого там всегда должен быть ноль, правим один байт (89- >FF), получается такой код:

Смещение	Опкод	Мнемоника
100064B5	FF15 60A00010	Call CryptVerifySignatureW
100064BB	FF4424 38	inc [esp+38]

Сохраняем файл, и снова убеждаемся в полноценной работе программы уже без отладчика, за исключением маленького нюанса. Это произойдет не автоматически, т.к. никаких данных в окне «License» (Рис.1) мы не вводили, то проверка не запускалась и в реестре не записано 0x1 для параметра «Licensed». Это не беда, у нас уже давно готов исходный код для этого, осталось только добавить его кодом патчера и процесс можно считать полностью автоматизированным.

Есть ли у вас план, мистер Фикс?

Раскуривая очередную сигарету, думаем ... Можно ли считать дело сделанным? Нет. Мы так и не выяснили, по какому принципу автор формирует ключ, можно ли реализовать программу генерации правильного ключа, и вообще обойтись без вмешательства в код программы?

О-о ... У меня целый мешок плана, Фокс. Можно с уверенностью сказать: «Не изменяя файлы программы, повлиять на ход ее выполнения не просто возможно, а для этого есть уйма средств, в руках программиста!». Чего нам стоит сделать лоадер, который запустит программу и уже в её памяти поправит этот байт, можно даже поправить не в directio.cpl, а в advapi32.dll . И вообще, если его правильно поправить на диске, то абсолютно все приложения, которые надеяться на достоверность результата CryptVerifySignature, будут нагло обмануты.

Возможно, теперь вы не захотите ничего слышать об этой функции и никогда не будете ее использовать в своих программах. Может, и будете правы, но если копнете ее глубже, и не только ее, то узнаете, как использовать и проверять цифровую подпись средствами Crypto API, причем без advapi32.dll, так что лопату вам в руки и семь футов под землей.

Алгоритм создания подписи

Ниже я приведу код, примерно таким он должен быть у автора (только с обработкой ошибок естественно):

```
;=========================================================================
cspname     db      'Microsoft Base Cryptographic Provider v1.0',0
private     file    'private.key'
sprivate    =       $-private
csp         dd      0
hash        dd      0
hkey        dd      0
Name        db      'John Doe'
sName       =       $-Name
Company     db      'ACME Corp.'
sCompany    =       $-Company
signature   rb      40h
ssignature  dd      $-signature
;=========================================================================
sigfile     db      'rsa.sig',0
hfile       dd      0
count       dd      0
;=========================================================================
start:      invoke  CryptAcquireContext,csp,0,cspname,\
                    PROV_RSA_FULL,CRYPT_VERIFYCONTEXT
            invoke  CryptImportKey,[csp],private,sprivate,0,0,hkey
            invoke  CryptCreateHash,[csp],CALG_MD5,0,0,hash
            invoke  CryptHashData,[hash],Name,sName,0
            invoke  CryptHashData,[hash],Company,sCompany,0
            invoke  CryptSignHash,[hash],AT_SIGNATURE,0,0,signature,ssignature
;=========================================================================
            invoke  CreateFile,sigfile,GENERIC_WRITE,0,0,CREATE_ALWAYS,0,0
            push    eax
            invoke  WriteFile,eax,signature,[ssignature],count,0
            invoke  CloseHandle
;=========================================================================
```

Хм... Код есть, а почему мы не можем создать правильный ключ для программы? А потому, что не хватает одного параметра, у нас нет файла private.key, у нас нет секретного ключа! Хм... А как он генерируется? Очень просто, вот код (думаю, вы сами догадаетесь обработать ошибки и подключить wincrypt.inc, я уже не говорю об его оптимизации)

```
;=========================================================================
RSA512      =       0x02000000
;=========================================================================
publickey   db      'public.key',0
privatekey  db      'private.key',0
csp         dd      0
```

```
key         dd      0
blob        dd      0
blobsize    dd      0
pheap       dd      0
hfile       dd      0
count       dd      0
;=========================================================================
start:      invoke  CryptAcquireContext,csp,0,0,PROV_RSA_FULL,CRYPT_NEWKEYSET
            invoke  CryptGenKey,[csp],AT_SIGNATURE,RSA512+CRYPT_EXPORTABLE,key
            call    [GetProcessHeap]
            mov     [pheap],eax
            push    PUBLICKEYBLOB
            push    publickey
            call    exportkey
            push    PRIVATEKEYBLOB
            push    privatekey
            call    exportkey
            invoke  CryptDestroyKey,[key]
            invoke  CryptReleaseContext,[csp],0
            jmp     exit
;=========================================================================
proc        exportkey,keyfile,keytype
            invoke  CryptExportKey,[key],0,[keytype],0,0,blobsize
            invoke  HeapAlloc,[pheap],HEAP_ZERO_MEMORY,[blobsize]
            mov     [blob],eax
            invoke  CryptExportKey,[key],0,[keytype],0,[blob],blobsize
            invoke  CreateFile,[keyfile],GENERIC_WRITE,0,0,CREATE_ALWAYS,0,0
            mov     [hfile],eax
            invoke  WriteFile,[hfile],[blob],[blobsize],count,0
            invoke  HeapFree,[pheap],0,[blob]
            invoke  CloseHandle,[hfile]
            return
endp
;=========================================================================
```

Ключи

Но толку от этого мало, ключ всегда генерируется разный. Сколько бы мы его не генерировали, получить такой же, как у автора не реально! Более того, именно от этого, от генератора случайных чисел (ГСЧ), зависит стойкость секретно ключа. Ну, может быть ещё немного от «утюга и паяльника». А вообще то, очень зависит от длины ключа (секретной экспоненты), скажем ключи до 384 бит, может и реально «подобрать» в домашних условиях, но на ключи 512 бит и более, нужны огромные вычислительные мощности и время. Хотя, говорят (на форуме wasm.ru проскакивала такая информация), были случаи «взлома» 2048 битного ключа RSA, при создании которого использовался плохой ГСЧ. Здесь я мало чего знаю и мало что могу сказать, я не профессор математики, а криптография требует недюжинных знаний в ней.

Наверное, существует много атак на эти схемы, даже теоретически подумать, ведь у автора есть только один private.key (соответствующий ему public.key зашит в программу), и он им генерирует регистрационные ключи для многих пользователей. Значит, в этой схеме с цифровой подписью существует множество правильных данных. Нужно найти любые, хеш-объект от которых даст нам подпись (Key), которая, в свою очередь, совпадет с подписью, полученной (после расшифровки публичным ключом) от хеш-объекта, взятого от тех же данных. И тогда CryptVerifySignature вернет нам истину. В этом случае мы упираемся в стойкость алгоритма хеширования MD5.

Да, довольно не просто все это ... А вот если бы мы столкнулись с ключом в половину меньшей длины, то посмотрите, что нам всего лишь нужно было сделать, чтобы найти секретный ключ, имея публичный (Рис.6 – RSA public.key 1):

1. Разбор ключа		
blobheader.bType	PUBLICKEYBLOB	06
blobheader.bVersion	CUR_BLOB_VERSION	02

1. Разбор ключа		
blobheader.reserved	0	0000
blobheader.aiKeyAlg	CALG_RSA_SIGN	00240000
rsapubkey.magic	0x31415352;"RSA1"	52534131
rsapubkey.bitlen	512	00020000
rsapubkey.pubexp	65537	01000100
rsapubkey.pubmod	(prime1 * prime2)	53CDFE887979A94989F1F60109176ED7D1D06DB45578 EF54B7DB3B799912736629A29AD7F3229D450EB8125E 4FB548F214053845BEDD88CC9D2316A26D21D1C5
2. rsapubkey.pubmod - hex переворот		
C5D1216DA216239DCC88DDBE45380514F248B54F5E12 B80E459D22F3D79AA22966731299793BDBB754EF7855 B46DD0D1D76E170901F6F18949A9797988FECD53		
3. rsapubkey.pubmod - hex2dec		
10360512575683163663328330543073246223858236 836350786404938717205260378190531323118735012149385851553932764279681659064121198499742718195448002379037723987		

Осталось только вот это длинное число «впихнуть» в одну из многих программ факторизации и ждать результата, а потом из полученного числа попытаться «спаять» private.key, соответствующий формату Crypto API.

PRIVATEKEYBLOB		
blobheader.bType	PRIVATEKEYBLOB	07
blobheader.bVersion	CUR_BLOB_VERSION	02
blobheader.reserved	0	0000
blobheader.aiKeyAlg	CALG_RSA_SIGN	00240000
rsapubkey.magic	0x31415352;"RSA2"	52534132
rsapubkey.bitlen	512	00020000
rsapubkey.pubexp	65537	01000100
modulus	Значение "prime1 * prime2", число "n"	
prime1	Число "p"	
prime2	Число "q"	
exponent1	Значение "d mod (p - 1)"	
exponent2	Значение "d mod (q - 1)"	
coefficient	Значение "(inverse of q) mod p"	
privateExponent	Число "d"	

Думаю, что это бы у нас получилось, и потом, используя этот ключ, могли бы спокойно сгенерировать правильные регистрационные данные. Напомню, что только при условии немного меньшей длины rsapubkey.pubmod , иначе мы никогда не дождемся результата ?. Но, стоит заметить, это будет все же не универсальный вариант, т.к. автор может сменять public.key с выходом новых версий программы (что он, кстати, иногда и делает).

Теоретические основы крэкинга: Глава 8. И вновь продолжается бой…

Автор: CyberManiac

Дата: 2004-11-01

_Контролирует ценность тот, кто может ее уничтожить
Ф. Херберт, «Дюна»_

Ну вот, все, что мы могли сделать простыми средствами, мы сделали. И теперь пришло время приступить к отладке. Да-да, эта глава будет практически полностью посвящена искусству дебагить, bpx'ить, трассировать - в общем, одному из аспектов той рутинной деятельности, которая ожидает каждого крэкера на его светлом пути к торжеству высокого искусства над коммерческими интересами. Осмелюсь предположить, что Вы имеете общее представление о том, на что похож процесс отладки программ, о точках останова, пошаговой отладке с заходом внутрь процедур и без оного и прочих столь же элементарных вещах. В интерфейсах и «горячих клавишах» отдельных представителей обширного семейства отладчиков, я думаю, Вы тоже разберетесь без особого труда – скажу лишь, что мы будем ориентироваться на работу под Windows. А наиболее достойными представителями своего племени под этой ОС на сегодня являются SoftIce(бессменный чемпион в течение многих лет – но чемпион довольно капризный) и OllyDebug (сравнительно новая, но очень качественная и совершенно бесплатная программа). Поэтому перейдем к изучению идей, которые, надеюсь, позволят Вам усовершенствовать свои умения в исследовании кодов, о полезных же особенностях конкретных отладчиков я буду упоминать тогда, когда эти полезные особенности будут нами востребованы. А поскольку методы борьбы с nagscreen'ами содержанием предыдущей главы не исчерпывается, мы будем изучать техники отладки параллельно с искусством ликвидации рекламных окон.

Как я уже говорил, долгие годы первым словом, которое знаменовало рождение нового крэкера, было «bpxMessageBoxA» - то есть команда установки breakpoint'а на некую функцию WindowsAPI. И в этом, несомненно, сокрыт глубокий смысл – точки останова (они же брейкпойнты) являются одним из важнейших средств отладки, позволяющим остановить программу в нужной точке. Системные вызовы ОС являются связующим звеном между программой и операционной системой. И если во времена DOSеще существовала возможность написать программу, выполняющую некие полезные действия, выводящую результаты и при этом ни разу не обращающуюся к средствам операционной системы, то сейчас, в эпоху многозадачных операционок, блокирующих прямой доступ к абсолютному большинству аппаратных ресурсов, сделать такое практически невозможно. Чтобы зажечь один-единственный пиксель в углу экрана, программам теперь приходится идти на поклон к операционной системе с вежливой просьбой «нарисуйте, пожалуйста, белую точку по указанным экранным координатам». С другой стороны, операционная система предлагает широкий выбор различных полезных функций – от мелочей вроде строковых операций (надо отметить, что выбор этих операций в Windowsмог бы быть и побогаче) до таких высокоуровневых функций, как работа с изображениями или уже упомянутое создание диалоговых окон по шаблонам в секции ресурсов. В общем, сделано все для удобства программиста (другое дело, что разработчики ОС иногда понимают это удобство весьма странным образом) – и разработчики этим пользуются. Пользуются этим и крэкеры. Когда какая-нибудь программка лезет в реестр, чтобы проверяться на истечение триального срока – она вызывает функции API. Когда читает серийник из поля редактирования – обращается к одной из оконных функций. Даже сообщение «Зарегистрируй меня!» - и то выводится при помощи API. И вот тут-то мы их и ловим – провоцируем приложение чем-нибудь выдать свою коммерческую природу, ставим точки останова – и терпеливо ждем, пока программа не попадется в расставленные сети. Однако чтобы поймать по-настоящему хитрую

дичь, ловушки нужно расставлять умело.

Давайте посмотрим, каким образом в уме крэкера рождается великая идея набрать в командной строке SoftIce’а ту самую эпохальную команду – bpxMessageBoxA. Сначала крэкер изучает повадки зверя, на которого идет охота: пытается вводить невалидные серийные номера (мы ведь не в рулетку играем – так что на дурную удачу рассчитывать не приходится), вызвать заблокированные функции (некоторые программы не отключают соответствующие элементы управления) или сделать еще что-либо подобное. Конечная цель всего этого – вынудить программу проявить свою незарегистрированность каким-нибудь хорошо заметным и легко идентифицируемым способом, например – выводом стандартного окна с сообщением. Впрочем, нередко бывает и так, что специально ничего делать не надо – программа сама при запуске или в процессе работы выплюнет на экран окошко с предложением зарегистрироваться. Допустим, что это окошко имеет специфический вид «MessageBox’а обыкновенного». Что такое «MessageBoxобыкновенный», я думаю, объяснять никому не надо - единожды в жизни увидев хотя бы один MessageBox, Вы уже больше никогда не сможете забыть это ужасающее зрелище, оно будет преследовать Вас годами, лишая покоя и сна, пока Вы не решите раз и навсегда «завязать» с компьютерами. Если каким-то чудом Вам за все время работы с компьютером удалось избежать этого счастья, Вы всегда можете посмотреть на MessageBox, попытавшись обратиться из «Проводника» к дисководу, в котором нет дискеты. Посмотрели? Вот и отлично, тогда продолжим.

Одна из важнейших для крэкера областей знаний есть знания о том, при помощи каких функций WindowsAPIвыполняются те или иные действия. Чтобы успешно поставить брейкпойнт на какую-либо функцию и получить от этого полезный результат, необходимо выполнение двух условий: Вы должны знать, что делает требуемая функция (сия мысль выглядит несколько банальной – но такова правда жизни) и как функция эта называется. И вот теперь Вам потребуются именно знания, одними только хорошими идеями, как мы это делали раньше, тут не обойтись. Есть два пути к обретению необходимых знаний: Вы можете либо с головой погрузиться в изучение программирования с использованием различных разделов WindowsAPI– и после этого будете способны не только влепить брейкпойнт «не глядя», но и в первом приближении оценить функции того или иного куска кода, просто посмотрев на вызываемые системные функции и передаваемые им параметры. Именно этим путем, насколько будет возможно, я и рекомендую Вам следовать – поскольку, когда Вы перейдете от обезвреживания простых защит к исследованию более защищенных программ, навыки программирования с использованием WindowsAPIВам очень пригодятся. Однако столь обширная информация, как программирование под Windows, не уместилась бы в рамки данной статьи – поэтому мы будем учиться в условиях, «максимально приближенных к боевым». Итак: у нас есть MessageBox с сообщением, и нам нужно куда-то поставить брейкпойнт, чтобы выявить, откуда этот MessageBox появляется.

Первое, что Вам надо уяснить – это то, что одно и то же действие нередко может выполняться несколькими разными функциями WinAPI. Например, наш любимый MessageBoxможет создаваться не только функцией MessageBoxA, но и функциями MessageBoxExAи MessageBoxIndirectA. Наиболее широкими возможностями в области управления внешним видом MessageBox’а обладает функция MessageBoxIndirectA, наименьшими – собственно MessageBoxA (но зато последнюю гораздо удобнее вызывать – ведь у нее всего четыре параметра). В действительности MessageBoxA – не самостоятельная функция, а лишь удобная «обертка» для вызова MessageBoxExA. Да и сама MessageBoxExAдалеко не самодостаточна: в WindowsXP, например, она реализована через вызов функции MessageBoxTimeoutA. Что же у нас получилось: мы охотились за одной функцией создания окна с сообщением, а нашли целое гнездо функций, имеющих близкое назначение! Да, именно так – ради удобства программистов и компактности кода в WindowsAPIвходит немало функций, частично дублирующих друг друга. И если Вы хотите при помощи точек останова отследить выполнение той или иной операции – Вам нужно ставить точки останова на все функции API, которые эту операцию могут выполнять. В нашем примере, если Вы

поставите точку останова на MessageBox, но забудете про экзотичный MessageBoxIndirect, Вы можете просто не обнаружить точку, в которой выводится сообщение. Поэтому рекомендую Вам пользоваться следующим правилом: **точки останова лучше ставить не на отдельные функции** API **, а на всю группу функций, выполняющих близкие по смыслу действия**.

Но как узнать, какие функции являются «близкими по смыслу» - спросите Вы. В этом обычно нет ничего сложного. Для начала Вам понадобится раздобыть документацию по программированию под Windows. В принципе, подойдет Win32 SDK(и даже он нужен не весь, а лишь справочные файлы по функциям WindowsAPI). Нужные файлы поставляется совместно с компиляторами под Windowsот Borlandили Microsoft, также нередко встречаются в Интернете (в том числе – частично переведенными на русский язык). Недостаток Win32 SDKв том, что он давно не обновляется – и, соответственно, не содержит информации по APIпоследних версий Windows. Более обширным источником, несомненно, является MSDNLibrary– ежеквартально обновляемый сборник документации по программированию под Windowsc использованием компиляторов, созданных этой фирмой. Встречается MSDNлибо в составе VisualStudio, либо отдельно, и занимает, как правило, несколько CD. Если Вы не испытываете хронический дефицит места на жестком диске, я бы рекомендовал использовать именно MSDN, причем как можно более новой версии. Помимо более свежей и подробной информации о системных вызовах Windowstam Вы найдете еще и подробную информацию по классам MFC (что может сильно пригодиться Вам при исследовании программ, созданных с использованием MFC).

Как только у Вас на «винчестере» появится требуемая документация, поиск близких по смыслу команд для Вас перестанет представлять какую-либо сложность. Вам нужно будет лишь нажать кнопку «Group» в окне справки или заглянуть в конец справочной статьи и изучить назначение функций, имена которых стоят после слов «Seealso». Разумеется, можно (и нужно!) применить и другие приемы работы с документацией – просмотреть дерево словарных статей, воспользоваться контекстным поиском по заголовкам или просто выполнить поиск текста по всей справочной системе. Для тех, у кого пока нет под рукой нужной документации, я составил небольшой «поминальник» наиболее часто используемых функций WinAPI и областей, в которых эти функции используются. Однако не думайте, что Вы сможете обойтись лишь этим списком – функции WinAPIимеют не только имена, но и параметры, понимание назначения которых ничуть не менее важно, чем знание имен функций. Так что описание перечисленных функций читать все равно придется – свою же задачу я вижу в том, чтобы указать, какие разделы стоит изучить в первую очередь.

Упрощенный вывод сообщений MessageBox, MessageBoxEx, MessageBoxIndirect, MessageBeep (эта функция не выводит сообщение, а только издает соответствующий звуковой сигнал)**Создание и уничтожение окон**

CreateWindow(наиболее популярная функция создания окон), CreateWindowEx

CloseWindow (функция закрытия окна), DestroyWindow

Создание и уничтожение диалогов

CreateDialog, CreateDialogIndirect, CreateDialogIndirectParam, CreateDialogParam (этифункциитолькосоздаютдиалог)

DialogBox, DialogBoxIndirect, DialogBoxIndirectParam, DialogBoxParam (эта группа функций позволяет создавать модальные диалоги; управление не возвращается программе, пока диалог не будет закрыт)

EndDialog, DestroyWindow

Чтение и изменение текстов окон

GetWindowText, GetDlgItemText (чтение текста окна или элемента диалога)

GetWindowTextLength (чтение длины текста окна)

GetDlgItemInt (чтение текста элемента диалога как 32-битного числа)

SetWindowText, SetDlgItemText, SetDlgItemInt (установка нового текста окна или элемента диалога)

Изменение видимости, позиции и прочих подобных свойств окна

EnableWindow (активация/деактивацияокна)

ShowWindow, ShowWindowAsync (изменение видимости и состояния окна, в частности – позволяет минимизировать или наоборот развернуть окно)

SetWindowPos, MoveWindow (изменение положения окна)

SetWindowWord (устаревшая и практически не используемая функция), SetWindowLong – две функции, позволяющие модифицировать весьма широкий спектр параметров окна. GetWindowWord, GetWindowLong – функции чтения этих параметров.

Загрузка ресурсов

LoadImage (универсальная функция загрузки изображений, иконок и курсоров), LoadBitmap, LoadIcon, LoadCursor

FindResource, FindResourceEx, LoadResource (функциизагрузкиресурсовлюбоготипавпамять)

Отображение изображений и текстов на экране

BitBlt, StretchBlt, MaskBlt (функции копирования BITMAP'ов на экран)

DrawText, TextOut, TabbedTextOut (обычный вывод текста)

GrayString (редко используемая функция, выводит на экран строку со стилем надписи на неактивном управляющем элементе)

Работа с файлами

OpenFile (устаревшая функция), CreateFile (основная функция открытия файлов; несмотря на свое название способна открывать файлы и даже директории)

ReadFile, ReadFileEx (функции чтения из файлов)

WriteFile, WriteFileEx (функции записи в файл)

SetFilePointer (перемещениепофайлу)

GetFileTime, GetFileAttributes, SetFileTime, SetFileAttributes (чтениеимодификациявременисозданияиатрибутовфайлов/директорий)

SetEndOfFile (изменение размеров файла)

Операции с **реестром**

RegOpenKey, RegOpenKeyEx, RegCreateKey, RegCreateKeyEx (открытиеисозданиеключейреестра)

RegQueryInfoKey (запрос информации о ключе, в частности – для проверки факта существования подключа)

RegQueryValue,RegQueryValueEx (чтение значений из реестра)

RegSetValue, RegSetValueEx (запись ключей в реестр)

RegCloseKey (закрытие ключа реестра)

Чтение и запись INI **-файлов**

GetProfileSection, WriteProfileSection, GetProfileInt, GetProfileString, WriteProfileString, WriteProfileInt (функции для работы с файлом Win.ini, в настоящее время считаются устаревшими, но иногда

используются)

GetPrivateProfileSection, GetPrivateProfileSectionNames, WritePrivateProfileSection, GetPrivateProfileInt, GetPrivateProfileString, GetPrivateProfileStruct, WritePrivateProfileString, WritePrivateProfileInt, WritePrivateProfileStruct (функцииработысобластьюреестра, отведеннойдляхранениянастроекпрограмм, либоспроизвольнымINI-файлом; этагруппафункцийсчитаетсяустаревшей)

Работа с датой и временем

GetSystemTime, GetLocalTime, GetSystemTimeAsFileTime (чтениетекущеговремени)

SetSystemTime, SetLocalTime(установка нового времени)

LocalTimeToFileTime, FileTimeToLocalTime, FileTimeToSystemTime, SystemTimeToFileTime (преобразованиеформатавремени)

CompareFileTime (сравнение двух переменных, хранящих время)

GetFileTime, SetFileTime (запись и чтение времени создания, последней модификации и последнего доступа к файлу)

Процессы и потоки: создание и управление

WinExec (устаревшая функция запуска исполняемых файлов), CreateProcess (функция, обычно используемая для запуска исполняемых файлов), ShellExecute, ShellExecuteEx (пара «альтернативных» функций для запуска приложений (применительно к исполняемым файлам) или открытия, печати и т.п. папок и документов).

ExitProcess(«стандартное» завершение процесса, эта функция способна завершить только тот процесс, внутри которого она вызвана), TerminateProcess (принудительное завершение процесса; эта функция способна «убить» любой процесс (в NT – при наличии соответствующих привилегий), что иногда используется защитами для подавления крэкерского софта)

CreateThread (штатная функция создания нового потока), CreateRemoteThread (эта функция «живет» только под NT-подобными и на самом деле в защитах практически не используется. Зато очень, очень (я не забыл сказать «очень»?) широко используется самими крэкерами для внедрения в чужой процесс. Так что обойти ее вниманием в этом поминальнике было бы несправедливо)

ExitThread, TerminateThread(штатное завершение и аварийное уничтожение потоков соответственно)

Загрузка и **выгрузка** DLL

LoadLibrary, LoadLibraryEx (функциизагрузкидинамическихбиблиотек)

LoadModule (устаревшая функция загрузки DLL)

GetProcAddress (функция, возвращающая адрес функции или переменной, объявленной в экспорте DLL, по имени этой функции/переменной (разумеется, соответствующая DLLдолжна быть подгружена текущим процессом). Эта функция широко используется как в защитах чтобы вызов какой-либо функции не «светился» в дизассемблированном листинге, а также для приемов типа push<желаемый адрес возврата>; jmp <адрес функции, полученный через GetProcAddress>, используемых для сокрытия точки, откуда была вызвана функция)

FreeLibrary (функция принудительной выгрузки DLL)

Надо сказать, список этот отнюдь не полный: в нем нет, к примеру, функций выделения блоков памяти (извлечь практическую пользу из информации о распределении памяти удается довольно редко, и те, области, где это актуально, отнюдь не относятся к «основам» крэкинга) и функций работы со строками (эти функции почти всегда дублируются в коде программы ради повышения

быстродействия). Нет в этом списке и специфических библиотечных функций Microsoft'овской MFCи Borland'овских VCL/RTL – если включить в список и их, «поминальник» стал бы совершенно неподъемным. Все это мы оставим за рамками – тем более, что средства разработки не стоят на месте, и, возможно, через пару лет будут актуальны совершенно другие API. Попробуйте понять, из чего я исходил, составляя этот «поминальник» - и, поняв это, Вам станет заметно легче найти подход к ранее неизвестной системе программных интерфейсов. Я выбрал WinAPIлишь по той причине, что это – «основа основ» программирования под платформу Win32, которая в настоящее время наиболее распространена. Однако даже к изложенному выше стоит относиться с известной долей осторожности – поскольку линейка Windows 9xугасает, в ближайшем будущем вполне могут стать актуальными специфические для линейки NTсистемные вызовы, а Вы будете искать правды среди стандартных функций Win32 – и не найдете ее. Прецедент уже имел место – в многих крэкерских руководствах пятилетней давности рекомендовалось для гарантированного «отлова» считывания текста из окна устанавливать брейкпойнт на функцию hmemcpy. И это работало – но только под Windows 9x, поскольку в линейке NTвместо вызова этой функции для повышения скорости использовались inline-вставки. До тех пор, пока все семейство NT-подобных ограничивалось доисторической NT 3.51 и неудобоваримой NT 4, проблемы как бы не существовало. Но вот на наши десктопы пришли более симпатичные Windows 2000 и WindowsXP – и противоречие между старыми руководствами и наблюдаемой реальностью встало в полный рост.

Как Вы можете видеть, имена функций являются сокращенным описанием тех действий, которые эти функции выполняют. Таким образом, если Вы предполагаете существование некой функции WinAPI и хотите поставить на нее брейкпойнт, но не знаете (или просто забыли), как она называется, Вы можете легко это узнать, если владеете принятой программистами под Win32 терминологией и английским языком (или хотя бы русско-английским словарем). Кроме того, если всем другим отладчикам Вы предпочитаете SoftIce, у Вас есть возможность воспользоваться командой EXP<текстовая_строка>, которая позволяет вывести на консоль отладчика все функции, имена которых начинаются с указанной Вами строки. Например, команда EXPMessageBoxпокажет Вам все MessageBox'ы, какие только встречаются в символьной информации, импортированной из DLL (список DLLдля импортирования информации должен быть заранее прописан в инициализационном файле отладчика). Надо отметить, что команда EXPимеет несколько более широкие возможности, чем простой вывод списка функций, начинающихся с определенных символов – Вы можете также просмотреть список модулей, по которым имеется символьная информация, проверить, присутствует ли нужная функция в некотором модуле и многое другое, о чем можно прочесть в руководстве по этому отладчику.

Вы наверняка заметили, что в «поминальнике» отсутствуют функции MessageBoxAи MessageBoxExA, о которых я упоминал в начале статьи, а вместо них описаны лишь MessageBox, MessageBoxEx. Разумеется, это не опечатка. Если Вы уже попробовали «на вкус» SoftIce'овую команду EXPи приложили ее к нашему измученному брейкпойнтами (то ли еще будет!) MessageBox'у, то наверняка заметили, что в списке функций, экспортируемых из USER32.DLLприсутствуют MessageBoxAи MessageBoxW (а вот просто MessageBox'а нет и в помине). Откуда же взялись буквы Аи Wв именах функций и что они вообще значат?

Расшифровка этих букв проста: A– это ANSI, W – это WIDE. А появились эти буквы после того, как в Microsoftвзяли курс на внедрение кодировки UNICODE, и возникла необходимость как-то отличать «старые» варианты функций, работающие с традиционными текстовыми строками, от «новых», использующих кодировку UNICODE. А символы в UNICODE имеют тип WCHAR длиной в 16 бит против обычных восьми – поэтому к именам функций добавили букву W, а не U, как можно было бы предположить. Вот и получается, что если функция принимает или передает строковые параметры, программист должен указать, какой из вариантов функции нужно использовать. Разумеется, если функция WinAPIне получает и не возвращает никаких символьных параметров, буквы Аи Wей совершенно ни к чему. Так что единственная причина того, что в моем «поминальнике» начисто

отсутствуют упоминания об ANSI- и UNICODE-вариантах одной и той же функции весьма банальна. Если бы я подробно расписывал оба варианта имени каждой функции, мне бы пришлось набирать этот список почти в два раза дольше, а Вам – во столько же раз дольше его читать, и при этом Вы бы не получили никакой новой информации. Поэтому я и решил не упражняться в крючкотворстве, а вместо этого объяснить причины использования окончаний Аи W. Надеюсь, что после моих объяснений Вы сможете подставить нужные буквы в имена функций самостоятельно.

Вряд ли Вам пригодится на практике эта информация, однако стремление к истине обязывает меня сообщить Вам великую тайну. Знайте, что все-таки они существуют – я имею ввиду MessageBox'ы без буквы Аили W. Ибо давным-давно, когда на Земле жили 16-разрядные динозавры Windows 3.1 и WindowsforWorkgroups, скрывавшиеся в недрах этих ОС функции ничего не ведали ни об ANSI, ни об UNICODE – а потому не нуждались в буквах Аи W. И от тех древних времен в include-файлах все еще сохранились строчки вроде lstrcmp equ <lstrcmpA> - в целях совместимости и упрощения переноса старых исходников под новые компиляторы.

Практическое использование этого списка функций выглядит очень просто. Предположим, что у Вас есть некая программа, и на этапе первоначального сбора информации Вы узнали, что она работает 30 дней, выводит при запуске MessageBoxсо счетчиком дней до окончания триального срока, и, если триальный срок закончился, после нажатия кнопки OKзавершает программу. Дальше Вы можете рассуждать примерно следующим образом:

- Если программа проверяет, сколько дней она работает – весьма вероятно, что она считывает текущую дату. Поэтому смотрим в раздел «Работа с датой и временем» и выбираем оттуда функции, предназначенные для чтения времени.

- Программа показывает MessageBox – следовательно, есть смысл поставить точки останова на функции создания MessageBox'ов.

- При истечении триального срока программа завершается – значит можно попытаться отловить вызов функции ExitProcess.

После расстановки брейкпойнтов на упомянутые функции и запуска программы отладчик скорее всего всплывет где-то внутри той ветки программы, которая ответственна за появление nagscreen'а и проверку триала. После этого Вы можете ликвидировать сам nagscreen(просто забив nop'ами все, что относится к вызову функции, создающей MessageBox) а затем методом обратной трассировки в уме (о сути этого метода я подробно расскажу чуть позже) добраться от точки вызова ExitProcessdo условия, по которому программа определяет, что триальный срок еще не кончился (и продолжает работать) или уже истек (и тогда вызывается ветка с ExitProcess). После этого, внеся исправления в исполняемый файл, Вы сможете организовать себе «вечный триал».

Практика показала, что «отловить» появление nagscreen'ов проще всего, отслеживая при помощи точек останова следующие типы событий:

1. Появление MessageBox'ов любых типов
2. Создание диалогов на основе ресурсов
3. Вызовы функций отображения окон (успешно работает только в том случае, если окон сравнительно немного – иначе становится сложно отследить среди всех вызовов соответствующих функций те, которые создают нужное окно)
4. Считывание текущего времени (в тех случаях, когда программа имеет ограничение по времени использования, и nagscreenсодержит информацию о том, когда истечет испытательный срок)
5. Создание таймеров (например, для активации управляющего элемента)
6. Изменение состояния окон Windows(например, активация кнопки или помещение в заголовок окна какой-либо надписи)
7. Вызовы вспомогательных функций, использующихся для изменения текста окон (к примеру, если в заголовке nagscreen'а присутствует надпись «Осталось Nдней пробного периода», можно

предположить, что для подстановки числа дней в надпись может использоваться функция wsprintf).

Разумеется, не следует принимать приведенный список как «истину в последней инстанции» и даже как «руководство к действию» - реальность бывает весьма разнообразна, и втиснуть ее в семь нумерованных пунктов, ничего не потеряв, вряд ли возможно. Однако как отправная точка или в качестве информации к размышлению, я думаю, этот список будет небесполезен. Однако скажу, что полезность предлагаемой последовательности подтверждена практикой – последовательная проверка каждого из пунктов этого списка во многих случаях позволяла мне легко добраться либо до процедуры создания/отображения нужного окна, либо до цикла обработки сообщений, получаемых этим окном.

Теперь Вам, пожалуй, известно о функциях WinAPIдостаточно, чтобы приступить к рассмотрению примеров, демонстрирующих, как эти знания применить для решения практическим задачам крэкинга. Так что в порядке эксперимента давайте немного взломаем какую-нибудь распространенную программу, например, Notepad. Это будет довольно несложный, но весьма поучительный эксперимент, демонстрирующий одну из техник ликвидации nagscreen'ов в «настоящих» программах. Итак, возьмем обыкновенный Блокнот (я взял тот, который входит в состав WindowsXP), скопируем его в надежное место, чтобы в случае чего можно было восстановить файл из резервной копии (вообще, возьмите себе за правило каждый успешный шаг отмечать резервными копиями файлов, дампами успешно модифицированных кусков и т.д. – очень неприятно бывает терять из-за ошибок результаты длительной и кропотливой работы) и запустим его. Теперь нажмите в Блокноте Ctrl-F (оно же «Правка|Найти…»), легонько стукните по клавиатуре левой пяткой (или просто наберите любой текст) и нажмите кнопку «Найти далее». Поскольку нашего «любого» текста в поле редактирования нет (потому что там вообще ничего нет), Блокнот ругнется сообщением вроде «Не удается найти "340t80543t"». Будем считать, что это и есть nagscreen, который нам предстоит ликвидировать.

Очевидно, что этот «nagscreen» очень сильно похож на MessageBox – поэтому вполне логичным было бы начать поиск нашего окошка с установки брейкпойнта на функции вывода стандартных окон с сообщениями (MessageBoxA, MessageBoxW, MessageBoxExA – и далее по списку). Грузим подопытную программу в отладчик (я в качестве отладчика буду использовать OllyDebug, но те, кто предпочитает SoftIce, тоже смогут без всяких сложностей повторить этот эксперимент) и устанавливаем точки останова. В SoftIceэто делается очень просто: «BPXимя_функции» - и так много-много раз, для всех подходящих функций из «поминальника». В OllyDebugэта операция выполняется несколько сложнее, зато Вы получите весьма неожиданный, но очень приятный сюрприз. Но приятные сюрпризы будут позже, а пока щелкните правой кнопкой мыши в окне кода и вызовите всплывающее меню. В этом меню вызовите пункт «Searchfor > Name (label) incurrentmodule» - этот пункт заставляет отладчик пробежаться по загруженному модулю (в нашем случае – по EXE'шнику Блокнота) и найти в нем все ссылки на «именованные» адреса (в частности, такими адресами являются вызовы WinAPI). Отладчик покажет окно с весьма немаленьким списком функций, на которые удалось найти ссылки в коде программы, и теперь в этом списке нам нужно найти наши MessageBox'ы. А вот и наступило время для обещанного приятного сюрприза: в списке нашлась только одна подходящая функция из «поминальника» - MessageBoxW. Так что в то время, как счастливые обладатели SoftIceсебивали в кровь руки, разбрасывая брейкпойнты на функции, которые вообще не используются в программе, пользователи OllyDebugбез всяких усилий получили информацию о том, с какого вызова APIлучше всего начать подбираться к нашему nagscreen'у. Пользователям же SoftIceчтобы получить список вызываемых программой функций APIпридется воспользоваться каким-нибудь дополнительным софтом, позволяющим просмотреть список импортируемых функций (благо таких программ немало).

В SoftIceбрейкпойнт на функцию ставится элементарно: BPXMessageBoxWи никаких гвоздей; в OllyDebugэта операция немного сложнее: найдя нужную строчку в списке импортированных функций, щелкните правой кнопкой мыши и в всплывшем меню выберите пункт

«Followimportindisassembler». После этого отладчик мгновенно перенесет Вас в глубины одной из важнейших системных библиотек операционной системы прямо к первому байту функции MessageBoxW. После этого останется лишь нажать клавишу F2 и увидеть, как начальный адрес этой функции окрасится в красный цвет. На этом подготовительные операции закончены – пора отпустить программу на волю и начать охоту на наш «nagscreen».

Попробовав опять заставить Блокнот искать «то, не знаю что», мы с радостью видим, что наша точка останова, о необходимости которой я так долго говорил, наконец-то выполнила свою задачу и «тормознула» программу, как только та попыталась выполнить функцию MessageBoxW. В окне отладчика Вы увидите что-то вроде этого:

```
77D70956   >  6A 00         push    0
77D70958   .  FF7424 14     push    dword ptr ss:[esp+14]
77D7095C   .  FF7424 14     push    dword ptr ss:[esp+14]           ; |Title = "????"
77D70960   .  FF7424 14     push    dword ptr ss:[esp+14]           ; |Text = "????"
77D70964   .  FF7424 14     push    dword ptr ss:[esp+14]           ; |hOwner = 00010015
77D70968   .  E8 03000000   call    USER32.MessageBoxExW            ; \MessageBoxExW
77D7096D   .  C2 1000       retn    10
```

И это есть ни что иное, как код функции MessageBoxWc комментариями, которые в него подставил OllyDebug (поскольку SoftIceникаких комментариев, кроме заранее определенных пользователем, подставлять не умеет, пользователям Айса придется немного напрячь воображение). При помощи кнопки F8 начинаем трассировать этот код («Ура! Мы отлаживаем ядро!») без захода в процедуры, и, дотрассировав до адреса 77D70968 с удивлением видим, что отладчик остановился, зато на экране появился наш MessageBox. А это значит, что наше исходное предположение насчет MessageBox'овой сущности «nagscreen'а» было верно. Закроем этот MessageBoxи продолжим трассировать до тех пор, пока не выйдем из процедуры по команде retn 10.

Пока Вы продавливаете до пола несчастную клавишу F8, я сделаю небольшое лирическое отступление. Вы наверняка уже прочувствовали, что много раз подряд нажимать F8 довольно скучно. А ведь MessageBoxW - это далеко не самая длинная процедура, нередко встречаются шедевры, в дизассемблированном виде занимающие десятки и сотни строк! И у Вас возник естественный вопрос – «неужели в таком большом и сложном отладчике нет какой-нибудь маленькой кнопки, чтобы сразу добраться до точки выхода из процедуры». Не подумайте о разработчиках плохого - они позаботились о нас, поэтому столь нужная кнопка в обеих отладчиках есть! В OllyDebugэта «кнопка» (она находится не только на панели инструментов, но и продублирована соответствующим пунктом меню) называется «Executetillreturn» и вызывается комбинацией Ctrl-F9. Заодно обратите внимание и на пункт меню «Executetillusercode»: если Вы когда-нибудь заплутаете в недрах чужой DLLи захотите одним махом выполнить весь код до того знаменательного момента, когда управление вернется в основную программу – смело жмите Alt-F9, и OllyDebugВам поможет. В SoftIceдля того, чтобы добраться до точки выхода из функции, служит команда **p** ret **,** по умолчанию «подвешенная» на клавишу F12. Вы можете подумать, что если есть команда pret, которая заставляет программу работать до первого ret'а, то должны существовать команды pmov, ppushи т.п. Увы, это не так – разработчики SoftIceсочли, что и одного отслеживания ret'ов более чем достаточно.

Но вернемся к нашему Блокноту. После того, как Вы успешно выйдете из процедуры, OllyDebugпокажет примерно следующее:

```
01001F8C  |.  FF75 14       push    [arg.4]          ;  notepad.01009800
01001F8F  |.  FF75 10       push    [arg.3]
01001F92  |.  E8 58FFFFFF   call    notepad.01001EEF
01001F97  |.  FF75 18       push    [arg.5]          ; /Style = MB_OK|MB_ICONASTERISK|MB_APPLMODAL
01001F9A  |.  FF75 0C       push    [arg.2]          ; |Title = "Блокнот"
01001F9D  |.  56            push    esi              ; |Text = "Не удается найти "oierg""
01001F9E  |.  FF75 08       push    [arg.1]          ; |hOwner = 000D063A ('Найти',
                                                     ; class='#32770',parent=004C0360)
01001FA1  |.  FF15 4C120001 call    dword ptr ds:[<&USER32.MessageBo>; \MessageBoxW
01001FA7  |.  56            push    esi              ; /hMemory = 0009BCB0
```

```
01001FA8  |.  8BF8          mov     edi, eax         ; |
01001FAA  |.  FF15 F4100001 call    dword ptr ds:[<&KERNEL32.LocalFr> \LocalFree
```

Если у Вас вместо OllyDebug установлен SoftIce, Вы все равно увидите нечто подобное, но уже без подсказок о параметрах функций MessageBoxи LocalFree – SoftIceизлишней дружелюбностью не страдает, а потому услужливо выискивать и расшифровывать для Вас параметры системных вызовов не станет. А жаль. Один из способов сделать так, чтобы MessageBoxне больше появлялся, заключается в забивании nop'ами «лишних» команд – а именно вызова call dword ptr ds:[<&USER32.MessageBoxW>]. Однако мало ликвидировать лишь сам CALL – нельзя забывать и о параметрах вызываемой функции. Потому что, если эти параметры не прибить тоже – они попадут в стек и так там и останутся до тех пор, пока процедуре, внутри которой эти параметры были положены на стек, не вздумается выполнить команду ret (или retn). Вот тут-то и начнется самое веселое: по-хорошему в момент выполнения команды retна вершине стека должен находиться адрес возврата, а в действительности окажется мусор, который должен был «уйти» в функцию MessageBoxWи благополучно исчезнуть в ее недрах. В зависимости от Вашей удачливости, настроения программы и текущей фазы Луны Вы получите либо банальный GPF (почти наверняка), либо какой-нибудь экзотический спецэффект вплоть до форматирования винчестера. Насчет форматирования - это, конечно, шутка, но шутка с долей правды – представьте, что случится, если на стеке в качестве адреса возврата окажется адрес какого-нибудь осмысленного куска кода. Так что после успешной ликвидации системного вызова не поленитесь разобраться и с PUSH'ами – пусть волшебная команда XCHGEAX,EAX (более известная как NOP) станет Вашим лучшим другом. Если же Вы питаете суеверный страх перед командой NOPили Вам просто лень много-много раз набирать код 90 – просто пропишите по адресу 01001F97 команду jmp 01001FA7; результат будет тот же самый, а пальцы устанут значительно меньше.

А потому, чтобы Ваш винчестер случайно не отформатировался, хорошенько запомните следующее правило:

Если Вы удалили вызов какой-либо функции – проверьте, не изменила ли эта операция последовательность данных, лежащих в стеке (иными словами – не кладется ли на стек чего-нибудь лишнее или наоборот – не снимается ли с него больше, чем надо).

На практике такую проверку можно выполнить следующим образом: засечь значение регистра ESPна входе в процедуру и на выходе из нее, вычислить разность между этими значениями (тут полезно помнить, что соглашения вызова PASCALи STDCALLпредусматривают удаление параметров процедуры со стека внутри процедуры, а CDECL – коррекцию стека после завершения процедуры, уже в теле программы). Если после внесения модификаций разность между значениями ESPне изменилась – значит, все сделано правильно, если изменилась – ищите ошибку в своих действиях.

Разумеется, совершенно необязательно замерять разность между значениями ESPименно в начале и в конце процедуры. К примеру, если процедура, внутри которой мы «стираем» обращение к другой процедуре, вызывается при помощи обычного CALL, можно промерить значения ESPнепосредственно перед выполнением этого CALLи сразу после него. Чтобы корректно проверить, не «застряло» ли чего-нибудь в стеке, или наоборот, не удалили ли мы чего-нибудь лишнего, необходимо соблюсти следующие условия:

1. При **любом** ходе исполнения кода (т.е. независимо от того, какие ветки срабатывают между точками, в которых проверяется положение стека) изменение величины ESPдолжно быть одинаковым.

2. Вы должны быть уверены, что **все** операции по помещению параметров на стек, относящиеся к проверяемому вызову, находятся между точками замеров. И вот здесь-то Вас могут подстерегать довольно неприятные неожиданности. Языки высокого уровня не позволяют по-настоящему изощренно работать со стеком (компиляторы C\C++, правда, позволяет динамически

зарезервировать на стеке некоторую область и даже соорудить в этой области объект – но это максимум, на что может рассчитывать программист). Даже «очень оптимизирующие» компиляторы обычно генерируют помещение параметров на стек в непосредственной близости от вызова функции, которая эти параметры принимает – поэтому найти их не так уж сложно. Но вот ассемблер… Да, ассемблер способен изменить ситуацию коренным образом. Вместо банального MyFunction (my_param) Вы вольны написать что-то вроде

```
push my_param
<сотня строчек кода, не относящегося к делу>
call MyFunction
```

А если как следует поразмыслить и поиграть с значениями регистров ESPи EBP, можно сотворить такое, что все ныне существующие дизассемблеры вывихнут свои виртуальные мозги, пытаясь разобраться, где у такой функции лежат параметры и где - локальные переменные. Правда, у подобных «антидизассемблерных» техник есть и обратная сторона – все это довольно долго пишется, тяжело отлаживается и защищает только от новичков и лентяев (надеюсь, что после прочтения этой главы Вы в число таких новичков и лентяев не попадете). Вот и получается, что замерять значения регистра ESPлучше всего на первой и на последней команде процедуры, когда вышеописанные «фокусы» со стеком еще себя не проявили, либо свое уже отработали.

Раз уж зашла речь об антидизассемблерных приемах (хотя на самом деле эти приемы направлены не столько на то, чтобы сбить с толку дизассемблер, сколько на то, чтобы запутать крэкера, пытающегося осмыслить код), еще немного уклонюсь от основной темы главы и расскажу о паре хитростей, иногда использующихся для помещения параметров на стек. Основаны эти приемы на том, что регистр ESPмало чем отличается от регистров общего назначения, а область стека – от всех прочих областей памяти. Зная, что над регистром ESPвполне возможно производить арифметические операции сложения и вычитания, а также обращаться к содержимому стека при помощи косвенной адресации командами вроде mov [ESP+8],eax, нетрудно догадаться, что команду pusheaxможно заменить, к примеру, последовательностью

```
sub esp,4
mov [esp+4],eax
```

И это только простейший способ замены одной-единственной команды… А если таких команд – много? А если загрузку значения на стек выполнять не целиком и одномоментно, а по одному байтику? А если функция, которая будет обрабатывать эти значения, вызывается не при помощи CALL, а каким-нибудь более изощренным образом? Подумайте на досуге о том, какие приемы могли бы помочь Вам преодолеть все эти сложности (кое-что я продемонстрирую Вам в следующей главе).

Надеюсь, что я не слишком напугал Вас живописаниями тех ужасов, которые Вы можете встретить (а можете и не встретить) на своем пути. С функциями WinAPI, как правило, все бывает гораздо проще – число параметров в любой момент можно посмотреть в документации, «ленивые» компиляторы складируют эти параметры непосредственно перед вызовом, и удаление «лишнего» вызова не представляет собой совершенно никакой сложности – примерно как убрать MessageBoxиз Блокнота.

А напоследок я расскажу об одном очень-очень простом способе, в некоторых случаях позволяющем найти место, в котором локализован вызов nagscreen’a. Суть способы очень проста: Вы загружаете программу и начинаете трассировать ее без захода внутрь функций, при каждом нажатии клавиши F8 запоминая текущий адрес (на практике почти всегда достаточно запоминать только адреса выполняемых call’ов). При выполнении одной из таких функции появится наш nagscreen. Снова загружаем программу, вспоминаем, каким был адрес того call’a, который вызвал появление окна и «прогоняем» программу до этого адреса в ускоренном режиме.

Перейдем ко второму шагу: зная, что создание nagscreen'а сидит где-то в глубинах вызываемой функции, войдем внутрь этой функции при помощи команды трассировки с заходом в функцию. Теперь начинаем трассировать содержимое этой функции, все так же запоминая адреса исполняемых команд. Рано или поздно мы наткнемся на очередной вызов, после которого выскочит nagscreen. Этот процесс постепенного погружения в код можно продолжать до тех пор, пока Вы не доберетесь до вызова WinAPI или другой библиотечной функции, создающей окно; не попадете в цикл обработки сообщений или ожидания закрытия окна nagscreen'а (что будет означать «перелет», но из этого результата тоже можно извлечь определенную пользу) либо не придете к выводу, что данная техника в Вашем случае неприменима.

Вообще говоря, эта методика применима не только для поиска вызовов nagscreen'ов, выводимых сразу после запуска. Если Вы знакомы с численными методами решения уравнений, Вы наверняка заметили некоторую аналогию с методом Ньютона: последовательно двигаясь вглубь кода и проверяя эффекты от вызовов подпрограмм, мы констатируем «недолет» либо «перелет» и постепенно сужаем область, в которой находится интересующий нас блок команд. Метод Ньютона изначально был предназначен для поиска решения на некотором промежутке значений, и, по аналогии, предлагаемый метод также может быть использован для поиска некоей функции «на промежутке кода». В качестве границ промежутка удобно использовать «знаковые» и легко отслеживаемые события, такие, как чтение текста из управляющего элемента, обращение к ячейке памяти, получение информации из реестра или из файла и отображение результатов этих действий в виде nagscreen'а, сообщения о неверном серийном номере и т.п. Проще говоря, если Вы ввели серийный номер, нажали кнопку «ОК» и программа в ответ сказала что-то вроде «Неправильно ты, дядя Федор, серийники вводишь» - значит, проверка правильности серийного номера лежит где-то между считыванием введенного серийника и выводом сообщения. И если Вам удастся зафиксировать оба этих события при помощи брейкпойнтов, трассируя и изучая код, лежащий между этими бряками Вы с большой вероятностью обретете желаемый адрес процедуры проверки серийника. А уж что Вы с этим адресом сделаете – зависит лишь от Ваших исходных целей и изобретательности.

Предлагаемый мной способ имеет несколько существенных ограничений: во-первых, трассируемый код должен исполняться последовательно (т.е. создание окна по таймеру или какому-либо иному событию таким способом отследить не получится или, по крайней мере, будет достаточно сложно). Во-вторых, очень желательно, чтобы создание и отображение nagscreen'а происходило в главном потоке программы (если nagscreenотображается в отдельном потоке, придется сначала выискивать место создания этого потока). И, в-третьих, если Вы активно практикуете дзен-крэкинг (то бишь Ваш любимый modusoperandi – «я не знаю, как это работает, но я все равно это сломаю»), Вы можете совершить следующую ошибку: «отключая» nagscreen, можно копнуть слишком глубоко и случайно «вынести» не процедуру отображения nagscreen'а, а более универсальную процедуру отображения окна вообще, которая по сути ни в чем не виновна. Для того, чтобы Вам было понятнее, о чем идет речь, продемонстрирую идею следующим псевдокодом:

```
ShowNagScreen proc
  …
  invoke ShowAnyWindow, nag_screen_handle
  …
ShowNagScreen endp

ShowAnyWindow proc hwnd:DWORD
  invoke ShowWindow, hwnd, SW_SHOWNORMAL
ShowAnyWindow endp
```

Предотвратить создание nagscreen'а в этом случае можно тремя способами:

1. Убрать вызов самой процедуры ShowNagScreen
2. В коде процедуры ShowNagScreenубрать вызов ShowAnyWindow
3. Удалить вызов WinAPI'шной функции ShowWindow внутри ShowAnyWindow

Первый и второй способы, в принципе, вполне корректны (причем второй даже несколько лучше – в реальной процедуре создания nagscreen'а могут выполняться какие-нибудь дополнительные операции), а вот третий… Да, своего мы бы, конечно, добились – но вместе с nagscreen'ом исчезли бы и все другие окна, которые отображаются процедурой ShowAnyWindow. А если учесть, что в современных программах вызовы WinAPIнередко упрятаны глубоко в недра всевозможных библиотек, вопрос о том, как бы случайно не перестараться с поиском «корня зла» и от избытка чувств не пропатчить библиотечную функцию – отнюдь не праздный. И универсального решения этого вопроса, по-видимому, не существует (если, конечно, не считать таким решением доскональный анализ кода программы).

Однако я могу предложить Вашему вниманию два подхода, которые с достаточно высокой вероятностью позволяют определить, используется ли функция исключительно для единственной цели (в частности – для отображения nagscreen'а), или же является универсальной в рамках приложения. Первый подход основывается на том, что библиотечные и просто широко используемые функции, как правило, вызываются многократно из разных областей исполняемого кода, в то время, как ссылки на узкоспециализированные процедуры (такие, как проверка серийного номера на валидность или вывод сообщения об ограничениях в программе) обычно присутствуют лишь в двух-трех экземплярах на всю программу, и во время исполнения кода приложения срабатывают считанные разы. Выполнить проверку подозрительной функции на количество вызовов можно как при помощи дизассемблера, так и прямо в процессе отладки. Поскольку большинство дизассемблеров позволяют получить список ссылок на процедуру (а также точек, в которых эти ссылки находятся), анализ при помощи дизассемблера сводится к простому поиску нужной процедуры в выходном листинге дизассемблера и визуальной оценке количества ссылок. Не могу еще раз не проагитировать Вас за использование OllyDebug: этот отладчик содержит несколько весьма удобных инструментов, скрывающихся внутри пункта меню «Findreferencesto…». В данном контексте нам, несомненно, особенно интересен подпункт «Selectedaddress», позволяющий найти все ссылки на команду под курсором. То есть, для выполнения описанной проверки Вы можете даже обойтись без дизассемблера; чтобы найти все прямые ссылки на некий адрес (например, на адрес первого байта функции), достаточно установить курсор на этот адрес и нажать Crtl-R. Если ссылок немного – значит, Вы нашли то, что искали. А вот если их количество перевалит за 5-8 штук – у Вас есть веские основания подозревать, что проверяемая функция выполняет некие общие функции, и потому Вам нужно подняться по дереву вызовов на уровень выше либо вообще искать нужный код в другой области. Раз уж речь зашла о дереве вызовов, добавлю еще следующее: даже удостоверившись в том, что проверяемая функция A вызывается один-единственный раз из функции B, не поленитесь посмотреть, что «растет» на дереве вызовов сверху – может оказаться, что сама функция Ввызывается в программе десятки раз. В этом случае вывод очевиден – Вы слишком увлеклись «глубоким бурением». Выполнить подобную проверку при помощи отладчика ничуть не сложнее – нужно всего лишь поставить точку останова на подозрительную функцию и запустить программу «с нуля». По количеству срабатываний точки останова, а также соотнося эти срабатывания с всплыванием nagscreen'а, Вы сможете сделать вывод о том, является функция «знаковой» для защиты или же просто исполняет некие более общие функции, прямого отношения к защитным механизмам не имеющие.

Другой способ выявления в общей массе библиотечных и других «общих» функций основывается на следующей особенности современных компиляторов: при сборке проекта функции размещаются в исполняемом файле в том порядке, в каком их обрабатывает компилятор, а код всевозможных библиотек помещается в начало либо в конец исполняемого файла. Кроме того, поскольку большинство проектов в настоящее время разбиты на модули, которые транслируются раздельно, получается так, что функции из одного модуля (обычно, к тому же, логически связанные, как того требуют принципы модульного программирования) располагаются по близким адресам. Как следствие, вызов функции из другого модуля или библиотеки в окне отладчика выглядит как очень длинный переход в далекие области памяти, в то время, как переходы внутри «своего» модуля

являются сравнительно короткими (в смысле величины, на которую изменяется значение EIPпри переходе).

Вот и подошла к концу очередная глава. Надеюсь, что Вы узнали об основах дебаггинга достаточно, чтобы попробовать самостоятельно что-нибудь взломать. Раз уж мы начали с Блокнота – попробуйте сотворить что-нибудь эдакое с Калькулятором (операция деления на ноль – достаточно интересная область для экспериментов), а уж затем можно перейти от «учебных целей» и к «настоящим» задачам. Возможно, исследование «большого» приложения у Вас тоже пройдет как по маслу – но вполне может быть, что защита окажется достаточно серьезной, и в процессе анализа кода у Вас возникнут сложности. И потому следующая глава как раз и будет посвящена различным тонкостям и хитростям отладки, а также борьбе с некоторыми антиотладочными приемами.

Встраивание Perl в IDA Pro 4.70 (Second Edition)

Автор: RedPlait

Дата: 2004-12-28

Дети не любятНикто не жалеет Больше плюютДа, я из погребая как умеюТак и живу... Аукцыон "Пионер", 1990

1. Назначение
2. Требования
3. Теория
4. Получаем idc.out
5. Процесс сборки
6. Инсталляция
7. Инструкция по использованию
8. Примеры scriptов
9. Описания исходных файлов
10. Жалобы и предложения

Назначение

В данном труде описывается технология встраивания Perl в IDA Pro строго версии 4.70 как в качестве скриптового движка для работы в дизассемблере, так и в IDA debugger.

С 2002 года, когда я в первый раз описал технологию расширения функциональности IDA Pro через отличные от IDC script engines, IDA сильно продвинулась в своем развитии. Начиная с версии 4.70 существует документированный способ расширения встроенного языка IDC через ф-цию **set_idc_func** , так что может показаться, что нужда в сторонних script engines практически отпадает. Но это не так - мой трехлетний опыт работы с встроенным в IDA perl позволяет утверждать, что он намного удобнее IDC для написания таких скриптов как импорт данных из текстовых файлов, при работе с массивами и более сложными структурами данных (вроде списка hasheй) и просто perl намного удобнее для расширения функциональности - достаточно сказать что за последний год я не использовал IDC вообще. Кроме того не так давно в IDA появился встроенный отладчик. Я покажу как добавить поддержку скриптовых языков также и к нему - и в результате такой нехитрой операции можно получить совершенно новый инструмент с практически неограниченными возможностями - управляемый через scriptы отладчик.

Требования

Вам потребуется следующее:

• IDA Pro 4.70 + соответствующий SDK. Хм, было бы странно обойтись без этого пункта
• Инсталлированный дистрибутив Perl. В принципе годится любая 5.x (где x > 3) версия perl для win32, лучше всего от ActiveState
• Если вы собираетесь дописывать свои собственные функции расширения функциональности модулей **IDA** или **IDADbg** , то для автоматизации bindinga ф-ций потребуется SWIG. Я использовал SWIG версии 1.3.17
• Компилятор C++ - лучше Visual C++. По крайней мере тестировалось все нижеописанное под Visual C++ 6, Visual C++ 7 & Intel C++ 8.0.

• Знание собственно Perla и умение написать на нем программу длиннее `#!/usr/bin/perl`
совершенно необходимы !
• Не помешает также опыт в создании собственных IDA plugins

Теория

Существует множество способов экспортировать функции из IDA Pro в script engine - я перечислю лишь некоторые из них:

1. самый простой - реализовать все функции самостоятельно используя IDA SDK. Но, учитывая, что, например только в IDC более 300 функций - это удовольствие будет длиться неопределенное (хотя и конечное) время. Далее эти реализованные функции можно экспортировать в Perl либо через механизм XS, либо через тот же SWIG, либо же опять же вручную...
2. Поступить как автор Python for IDA - модифицировать заголовки для IDA SDK таким образом, чтобы они стали понятными тому же SWIGy. Для некоторых языков такой грязный хак окажется вполне работоспособоным (скажем для того же Python или Ruby), но для Perl он опять же неприемлем. Дело в том, что в IDA SDK содержится очень большое количество разнообразных объектов в виде классов C++, а SWIG же для экспорта классов в Perl не слишком удобен - уж слишком самобытно в Perl реализованы объекты и слишком громоздкими получаются имена функций.
3. Использовать уже имеющиеся функции от IDC. Этот подход имеет кроме значительного сокращения ресурсов на разработку также то преимущество, что конечному пользователю не нужно будет переучиваться к чужому набору функций - если человек умеет писать скрипты на IDC, то в Perle 90% функций будут просто теми же самыми, что и в IDC

Так что для встраивания Perl я использовал именно третий подход, хотя для встраивания Ruby (может быть когда-нибудь я опишу и этот инструмент) - второй.
Кроме того, для встраивания Perl в IDA debugger мне все равно пришлось написать некоторое количество функций самостоятельно - я сначала использовал SWIG, а потом просто дописал примерно половину функций руками.

Получаем idc.out

Прежде всего нам необходимо извлечь из самой IDA Pro описание ее IDC функций. Если вы помните, раньше сей процесс включал в себя дизассемблирование ida.wll, нахождение в нем массива IDCFuncs и запуск IDC scripta, извлекающего всю необходимую нам информацию. Но начиная с версии IDA 4.50 Великий Яхве наконец услышал наши проклятья, и IDCFuncs стал экспортируемым из ida.wll символом. Таким образом задача сильно упростилась - я просто написал IDA plugin idcdump (находится в архиве `wasm_files.zip`: `files/recovered/352/idc_dump.zip`) и теперь процесс получения файла idc.out выглядит так:

1. собираем из исходников в поддиректории idc_dump одноименный plugin. Если вы слишком ленивы то он уже есть в собранном виде для IDA 4.70
2. Копируем его в подкаталог plugins IDA Pro
3. Запускаем IDA Pro на любом файле, идем в меню Edit->plugins и вызываем наш plugin - и благодаря чорной магии в текущем каталоге появляется искомый файл idc.out

Впрочем, поскольку 99% людей настолько же ленивы как автор сего - я просто предоставляю готовый файл idc.out для IDA версии 4.70

Процесс сборки

Если вы не собираетесь добавлять свои собственные функции в модули - можете просто воспользоваться предоставленными файлами проекта rp_vc.dsp, rp_vc.dsw и perl_dbg\perl_dbg.dsp и дальше просто не читать. Без пересборки pluginов не обойтись, ибо скорее всего у вас стоит версия Perl, отличная от моей. По этой же причине я не предоставляю исполнимых файлов (кроме вышеупомянутого **idc_dump.plw**).

Если же вы хотите скажем добавить пару (или пару сотен) своих функций для их последующего использования в своих Perl scriptax, то вам необходимо сделать нижеописанные телодвижения. Во первых, информации из файла idc.out недостаточно - дело в том, что SWIG умеет работать только с полностью типизированными функциями, а у нас пока отсутствуют типы возвращаемых значений. Поэтому был написан файл *outz\horror.pl* , который умеет делать следующее - он берет файл idc.out, а также idc.idc из подкаталога idc IDA Pro, парзит последний и формирует из этой информации заголовочный файл *ida.h* , файл интерфейса для SWIG *ida.i* , и файл для marshalling аргументов и результатов для вызова IDC функций из нашего plugina *ida.cpp*. Обратите внимание, что начиная с версии 4.70 изменился прототип вызова функций из массива IDCFuncs - если раньше они были Borland fastcall, то теперь они стали idaapi (что тоже самое что и stdcall). Так что старая версия *horror.pl* не годится. Кроме того, поскольку я реализовал некоторое количество функций не через IDCFuncs, а напрямую, используя средства IDA SDK - в файле *ida.cpp* реализации таких функций будут отсутствовать. Вызов *horror.pl* производится с ключами **-t h** для генерации *ida.h* , **-t i** для генерации *ida.i* и **-t c** для генерации *ida.cpp*. Далее запускаете SWIG с примерно такими параметрами

```
swig.exe -perl -static ida.i
```

outz/s.bat) и получаете файл с кодом bindinga ваших функций для Perl *ida_wrap.c*. Ну или не получаете... Одно из двух.
После этого у вас наконец есть все файлы для сборки plugina - копируете все 3 файла на директорию выше и собираете plugin как описано выше.

Инсталляция

очень проста - скопируйте **perl_dbg.plw** и **rp_vc.plw** в подкаталог plugins IDA Pro, а файлы экспорта для модулей Perl **pm/ida.pm** и **pm/idadbg.pm** в подкаталог lib вашего дистрибутива Perl (pm-файлы (находится в архиве `wasm_files.zip`: `files/recovered/352/perl_pm.zip`)).

Инструкция по использованию

rp_vc.plw - реализует подмножество IDC функций для модуля IDA, а также содержит perl script engine. Обратите внимание, что он не реализует функции из модуля IDADbg, так что при попытке их вызова из обычного perl scripta вы получите сообщение об ошибке. Модуль экспорта для него лежит в файле **pm/IDA.pm**
perl_dbg.plw содержит в себе реализацию обоих модулей IDA и IDADbg. Так было сделано потому, что **perl_dbg.plw** работает только с x86 PE файлами и только через callback функции, вызываемые при отладке (так что **в perl_dbg функция main скрипта не вызывается**). Соответственно он требует для работы два модуля экспорта - **pm/IDA.pm** и **pm/IDADbg**
rp_vc.plw же работает с любым процессорным модулем и любым форматом файлов, и запускает функцию main в обрабатываемом скрипте автоматически.

В **perl_dbg.plw** реализована поддержка следующих событий отладчика (обратите внимание что имена обработчиков в скрипте должны иметь предопределенные имена. Впрочем, если эти имена вам не нравятся - можете изменить их в массиве *cb_sub_names* в файле dbg_callz.cpp):

• **cbProcessStart** - вызывается при старте отлаживаемого процесса. В качестве аргумента получает DWORD - идентификатор процесса.
• **cbProcessExit** - вызывается после завершения отлаживаемого процесса. В качестве аргументов получает идентификатор почившего процесса а также его exit_code
• **cbThreadStart** - вызывается при старте нового потока. Аргументом служит идентификатор потока.
• **cbThreadExit** - вызывается после завершения потока. Аргументами являются идентификатор потока *t_id* и код его завершения *exit_code*. Обратите внимание что в этом обработчике поток уже мертв и нельзя обращаться к его контексту через *t_id*
• **cbLibraryLoad** - вызывается после загрузки какого-либо модуля, причем модуль может быть загружен как при старте процесса через его таблицу импорта, так и через явный вызов LoadLibrary. Аргументом передается ссылка на hash с характеристиками модуля
• **cbLibraryUnload** - вызывается после выгрузки модуля. Аргументом передается имя выгруженного модуля, причем перед вызовом этого callbacka модуль уже удален из внутреннего списка модулей, получаемого из **modules_list**
• **cbBpt** - Самый Главный Обработчик - вызывается при достижении breakpoint. Аргументы: *t_id* - идентификатор потока, *addr* - адрес случившегося breakpoint
• **cbException** - вызывается при исключении. Аргументы: *t_id* - идентификатор потока, *code* - код исключения, *addr* - адрес исключения, *info* - строка с описанием исключения.

Кроме поддержки этих обработчиков модуль IDADbg также реализует следующие относительно полезные при работе с отладчиком функции:
• **ProcessQty** - количество запущенных в системе процессов
• **GetProcessInfo(n, out_hash)** - выдает информацию по процессу номер *n* в ссылке на hash *out_hash* (ключами в hash будут *name* - имя главного модуля процесса и *id* - идентификатор процесса)
• **ThreadQty** - количество запущенных потоков
• **GetThreadN(n)** - выдает идентификатор потока для потока под номером *n* , где *n* должен быть в диапазоне от 0 до ThreadQty() - 1
• **ProcessState** - возвращает статус отлаживаемого процесса (см. описание **get_process_state** из файла *dbg.hpp* IDA SDK
• **StartProcess(path, args, dir)** - запускает процесс под отладчиком. *path* - имя модуля, *args* - аргументы командной строки для модуля, *dir* - стартовая директория для процесса
• **SuspendProcess** - замораживает отлаживаемый процесс
• **ContinueProcess** - продолжает исполнение ранее замороженного процесса
• **Exit_Process** - завершает отладку процесса
• **AttachProcess(p_id)** - подключает отладчик к процессу с идентификатором *p_id*
• **DetachProcess** - отключает отладчик от процесса
• **SelectThread(t_id)** - активизирует поток с идентификатором *t_id*
• **StepInto** - см. описание функции **step_into** в файле *dbg.hpp* IDA SDK
• **StepOver** - см. описание функции **step_over** в файле *dbg.hpp* IDA SDK
• **RunTo(addr)** - см. описание функции **run_to** в файле *dbg.hpp* IDA SDK
• **StepUntilRet** - см. описание функции **step_until_ret** в файле *dbg.hpp* IDA SDK
• **BptQty** - возвращает количество установленных breakpoints
• **GetBptN(n, out_hash)** - возвращает характеристики точки останова номер *n* , где *n* должен быть в диапазоне от 0 до BptQty() - 1. Информация помещается в ссылку на hash *out_hash* , ключами в этом hash могут быть:
• **ea** - адрес точки останова
• **size** - размер точки останова

• **type** - тип точки останова
• **pass_count** - количество срабатываний точки останова
• **flags** - флаги точки останова
Смотри пример использования этой функции в примере скрипта для IDA debugger ниже
• **GetBpt(ea, out_hash)** - практически полностью идентична предыдущей функции, только ищет точку останова не по ее номеру, а по адресу
• **AddBpt(ea, size, type)** - устанавливает новую точку останова по адресу *ea* размером *size* и с типом *type*. В случае удачи возвращает 1. Смотри пример использования этой функции в примере скрипта для IDA debugger ниже
• **DelBpt(ea)** - удаляет точку останова по адресу *ea* Смотри пример использования этой функции в примере скрипта для IDA debugger ниже
• **EnableBpt(ea)** - разрешает обработку точки останова по адресу *ea*
• **DisableBpt(ea)** - запрещает обработку точки останова по адресу *ea*
• **dbg_module(out_hash)** - возвращает в ссылке на hash *out_hash* описание отлаживаемого модуля. Ключи в hash могут быть:
• *name* - строка, имя модуля
• *base* - базовый адрес модуля
• *size* - размер модуля
• *rebase* - фактический адрес, по которому модуль был загружен
• **modules_list(out_array)** - возвращает ссылку на массив ссылок на hash с описаниями всех загруженных в отлаживаемом процессе модулей. Ключи в hash идентичны описанным в предыдущей функции. Смотри пример использования этой функции в примере скрипта для IDA debugger ниже
• **ThreadContext(t_id, kind_of_context, out_hash)** - функция получает контекст потока с идентификатором *t_id* , содержимое контекста задается в переменной *kind_of_context* , (ее значения см. в описании поля ContextFlags структуры _CONTEXT в справке по Win32 API). В out_hash будет помещена ссылка на hash, содержащий в качестве ключей имена регистров, а в качестве значений для этих ключей - текущее значение соответствующих регистров. Смотри пример использования этой функции в примере скрипта для IDA debugger ниже
• **set_DrX(t_id, reg_value)** где **X** может быть 0, 1, 2, 3, 6, 7. Устанавливает содержимое соответствующего регистра DrX для потока с идентификатором *t_id* в *reg_value*
• **set_Gs(t_id, reg_value)** - аналогично для регистра **Gs**
• **set_Fs(t_id, reg_value)** - аналогично для регистра **Fs**
• **set_Es(t_id, reg_value)** - аналогично для регистра **Es**
• **set_Ds(t_id, reg_value)** - аналогично для регистра **Ds**
• **set_Cs(t_id, reg_value)** - аналогично для регистра **Cs**
• **set_Ss(t_id, reg_value)** - аналогично для регистра **Ss**. Но вы должны точно знать что делаете ...
• **set_Efl(t_id, reg_value)** - аналогично для регистра флагов
• **set_REG(t_id, reg_value)** Устанавливает содержимое регистра общего назначения **REG** (**REG** можеть быть Edi, Esi, Ebx, Edx, Ecx, Eax, Ebp, Esp или Eip) для потока с идентификатором *t_id* в *reg_value* Смотри пример использования этой функции в примере скрипта для IDA debugger ниже

Примеры scriptов

Примерчик scripta, который расставляет комментарии к двойным switch-tables (т.е. когда первая таблица содержит индекс во второй, а уже вторая содержит адреса перехода). Параметры ф-ции process_tables:

1. адрес таблицы индексов
2. адрес таблицы адресов

3. delta, которую нужно прибавить к каждому индексу. Аргумент необязателен
4. размер таблицы индексов. Можно не указывать - тогда будет вычислен как разность между вторым и первым аргументами

```
#!/usr/bin/perl
# This terrible script processed 2table switches
# (C) RedPlait, 25 Nov 2004

use IDA;

sub form_string
{
  local $ref = shift;
  local $delta = shift;
  local $res = '';
  local $latch = 0;
  foreach $str (@$ref)
  {
    if ( ! $latch )
    {
      $res = sprintf("0x%X", $str + $delta);
      $latch++;
    } else {
      $res .= ',' . sprintf("0x%X", $str + $delta);
    }
  }
  return $res;
}

sub process_tables
{
  local $first = shift;
  local $second = shift;
  local $delta = shift;
  local $size = shift;

  $delta = 0 if ( ! defined $delta );
  $size = $second - $first if ( ! defined $size );
  local %hash;
  local($i, $c, $hsize, $str, $ref);
  $hsize = 0;
  for ( $i = 0; $i < $size; $i++ )
  {
    $c = Byte($first + $i);
    if ( defined $hash{$c} )
    {
      $ref = $hash{$c};
      push @$ref, $i;
    } else {
      $hash{$c} = [ $i ];
      $hsize = $c if ( $c > $hsize );
    }
  }
  for ( $i = 0; $i <= $hsize; $i++ )
  {
    $str = form_string( $hash{$i}, $delta );
    next if ( $str eq '' );
    MakeComm($second + 4 * $i, $str);
  }
}

# main
process_tables 0x413720, 0x413755, 0x61;
```

```
#!/usr/bin/perl
# use stmt - указывать строго в таком порядке !
use IDA;
use IDADbg;

# on_brealpoint handler
sub cbBpt
```

```
{
  my( $t_id, $addr ) = @_;
  printf("t_id %X addr %X\n", $t_id, $addr);
  my $modules;
  my $str;
  if ( $addr == 0x10016a0 and modules_list($modules) )
  {
    # dump all loaded modules
    foreach $str ( @$modules )
    {
      printf(FILE "%s, base %X size %X rebase %X\n",
       $str->{'name'},
       $str->{'base'},
       $str->{'size'},
       $str->{'rebase'}
      );
    }
    my $context;
    if ( ThreadContext($t_id, CONTEXT_ALL, $context) )
    {
      my $reg;
      # dump all registers values
      foreach $reg ( keys %$context )
      {
        printf(FILE "%s .eq. %X\n", uc($reg), $context->{$reg} );
      }
      # set eax in some value
      set_Eax($t_id, 0x666);
    }
  }
  if ( $addr == 0x10016a0 )
  {
    # lets add new breakpoint from our script at 10016a6
    printf("try to add new bpt %d\n", AddBpt($addr + 6, 0, HWBPT_ANY) );
    printf("result of continue %d\n", ContinueProcess() );
  }
  if ( $addr == 0x10016a6 )
  {
    printf("My added breakpoint called !\n");
    # delete previous breakpoint
    DelBpt($addr - 6);
    # lets log value is EAX
    my $context;
    ThreadContext($t_id, CONTEXT_ALL, $context);
    printf(FILE "eax is %X\n", $context->{'Eax'});
    # and go on...
    ContinueProcess();
  }
}

sub cbProcessExit
{
  # be good boy and free all used resources here
  close FILE;
}

sub cbProcessStart
{
  open(FILE, ">test.log") or die("Cannot open test.log");
  # print all existing breakpoints
  my $c = BptQty();
  printf(FILE "BptQty %d\n", $c);
  local($i, $me);
  for ( $i = 0; $i < $c; $i++ )
  {
   last if ( ! GetBptN($i, $me) );
   printf(FILE "ea %X size %d type %d flags %X\n",
     $me->{'ea'},    # addr of breakpoint
     $me->{'size'},  # size of breakpoint
     $me->{'type'},  # type of breakpoint
     $me->{'flags'}  # flags of breakpoint
```

```
    );
  }
  # print attrs of currently debugged module
  my $my_module;
  if ( dbg_module($my_module) )
  {
    printf(FILE "Me is %s, base %X size %X rebase %X\n",
     $my_module->{'name'},  # name of module
     $my_module->{'base'},  # base addr of module
     $my_module->{'size'},  # size of module
     $my_module->{'rebase'}
    );
  }
}
```

Описания исходных файлов (находится в архиве wasm_files.zip: files/recovered/352/perl_src.zip)

• *dbg_callz.cpp* - реализация вызова perl ф-ций в качестве callbackов для событий отладчика
• *dbg_plugin.cpp* - реализация plugina perl for IDA debugger. Содержит реализацию функции **dbg_plugin.cpp** , которая устанавливается через **hook_to_notification_point** как обработчик событий от отладчика и собственно выполняет всю главную часть работы - вызывает функции из *dbg_callz.cpp* для нужных событий.
• *dstore.cpp* & *dstor.h* - содержат реализацию внутреннего хранилища списка всех загруженных модулей. Дело в том что я не нашел в SDK способа получить такой список кроме как сохранять аргументы события **dbg_library_load**.
• *Embed_Perl.cpp* - реализация встроенного perla, также делает перехват стандартного perlового вывода в STDERR & STDIN для корректной их работы в IDA Pro
• *ida.cpp* - автоматически сгенерированный horror.pl файл с реализацией вызовов IDC функций через таблицу **IDCFuncs**.
• *ida_wrap.c* - сгенерированный SWIGом код для подключения функций из модуля IDA к perl
• *IDADbg_wrap.c* - сгенерированный SWIGом код для подключения функций из модуля IDADbg к perl
• *init.cpp* - version specific код для получения версии IDA Pro, а также массива IDC функций IDCFuncs
• *mydbgxs.cpp* - некоторые методы из модуля IDADbg, реализованные в силу своей специфичности не через SWIG (и даже не через XS механизм), а непосредственно на C - например возвращающие hash или ссылку на массив hasheй как **modules_list**
• *myimp.cpp* - реализация некоторых методов непосредственно с помощью IDA SDK, а не через массив IDC функций IDCFuncs. Это сделано в основном для ускорения работы наиболее часто вызываемых функций, так как таким образом мы избавляемся от одного слоя marshallinga при конвертации аргументов для обращения к функциям IDC. Реализованы таким образом следующие функции: *MakeByte, MakeWord, MakeDword, MakeQword, MakeOword, MakeFloat, MakeDouble, MakePackReal, MakeTbyte, MakeUnkn, OpBinary, OpOctal, OpDecimal, OpHex, OpChr, OpNumber, PatchByte, PatchWord, PatchDword, GetIdbPath, GetFlags, Byte, Word, Dword, Jump, RunPlugin, MakeComm, MakeRptCmt, GetInputFile, GetInputFilePath, ScreenEA, SetStatus, GetEntryPointQty, GetEntryPoint, GetStrucQty, loader_name, load_ids, save_idb, is_debugged, GetOriginalByte, GetOriginalWord, GetOriginalDword, MakeUnknRange, IsPublicName, MakeNamePublic, MakeNameNonPublic, HideName, ShowName, IdpName*
• *rp_vc.cpp* - реализация plugina perl for IDA Pro
• *rp_vc.dsp & rp_vc.dsw* - файлы проекта для Visual C++ 6 для сборки обоих pluginов
• *perl_dbg/perl_dbg.dsp* - файл проекта для Visual C++ 6 для сборки perl_dbg plugin

Жалобы и предложения

а также умные и полезные советы, подробные и аргументированные описания найденных ошибок (если я сочту ошибку недостаточно подробно описанной - ответа даже не ждите), малоношенные теплые вещи и добровольные пожертвования можно отправлять автору по адресу redp@mail.ru. Автор не несёт ответственности за последствия работы его программ, а также за кривые руки (в 99% случаев это следствие неустранимой с помощью имеющейся биохакерской технологии баги в ДНК) собирающего все самостоятельно из прилагаемого исходного кода или неправильное толкование вышеописанного...

```
#include <std_disclaim.h>
```

Теоретические основы крэкинга: Глава 9. Если бряк оказался вдруг…

Автор: CyberManiac

Дата: 2005-01-04

Наверное, Вы уже попытались что-нибудь взломать. Может быть даже, Вам это удалось – за счет знаний и способностей к анализу, благодаря интуиции, или же в силу Вашего трудолюбия и настойчивости. Возможно также, что Вам просто очень повезло с первой в жизни программой, и защита оказалась слабее, чем в большинстве других программ. Однако тех, кто не смог с первой попытки одержать победу над мегабайтами кода, гораздо больше. Кто-то споткнулся об антиотладочные приемы, кому-то «повезло» встретиться с запакованной программой, кто-то принял близко к сердцу огромнейшие возможности, предоставляемые OllyDebug и SoftIce, и погрузился в изучение этих инструментов, отложив до времени собственно копание в коде. Некоторые отступили, не добравшись до подходящей зацепки, с которой можно было бы начать «раскручивать» защиту. Свежие ощущения, новые знания, предвкушение будущих побед – все, что знаменовало рождение крэкера, осталось в светлом прошлом, куда Вы сможете вернуться лишь в мечтах. В общем, одни радуются своей первой победе, другие – переводят дыхание и с тоской глядят на заоблачные выси, которые не удалось достичь. Если Вы попали в число «других», значит, у нас есть кое-что общее – свою первую программу я взломал далеко не с первой попытки. Надеюсь, после этих слов у Вас появился повод для оптимизма – возможно, именно Вам в будущем суждено написать свои собственные «Теоретические основы…». Однако сейчас Вас, наверняка, больше интересует другой вопрос – «почему мне не удалось сломать программу?» Причем нередко этот вопрос обретает еще более конкретную формы – «почему я ставлю брейкпойнты, а они не срабатывают?» и «как отлаживаемая программа может обнаружить мои точки останова?» И вопросы о неработающих (или «странно» работающих) брейкпойнтах – это отнюдь не повод упрекнуть в невнимательности начинающего крэкера, но основание для подробного разговора об особенностях Win32 API, тонкостях работы точек останова и антиотладочных приемах.

Брейкпойнты – лучшие друзья крэкера, готовые в любой момент прийти Вам на помощь. Однако эти друзья отнюдь не всемогущи; как и живым людям, им присущи определенные слабости и врожденные особенности. И чтобы «поладить» с точками останова, нужно обладать знаниями об этих особенностях и слабостях – это, в конечном итоге, позволит Вам при помощи нехитрых приемов отлавливать весьма замысловатые ситуации и успешно обходить защитные механизмы, направленные на «вырубание» брейкпойнтов. Но прежде чем приступать к познанию столь высоких сфер, как внутреннее устройство и принципы функционирования точек останова, разберемся с куда более приземленными причинами возможной неработоспособности брейкпойнтов.

Самой простой (и, к сожалению, отнюдь не самой редкой) причиной такого поведения наших верных друзей являются ошибки в коде отладчиков. Да-да, вы не ослышались, крэкерам нередко приходится тратить часы на поиски несуществующих защит именно из-за недоработок в используемом инструментарии. «SoftIce не ставит бряк на функции», «Symbol loader не останавливает программу после загрузки» и другие подобные проблемы, с которыми сталкивался едва ли не каждый пользователь этого отладчика, уже который год отравляют жизнь крэкерам. При некотором упорстве и настойчивости эти проблемы иногда удается обойти разными «шаманскими» приемами, например, использованием аппаратного брейкпойнта вместо обычного или указанием адресов в явном виде, но даже такие «танцы с бубном» не всегда оказываются эффективны против сущностей, скрывающихся по ту сторону отладчика. Никакие конкретные рекомендации тут, понятное дело, дать невозможно – программные глюки бесконечно разнообразны, и без точного

знания причины с ними можно бороться разве что методом терпеливого перебора всех «обходных путей», какие только придут Вам в голову. Если Вас не прельщает сей метод – есть смысл поискать другую версию продукта (поскольку глюки, присущие одной версии программы, могут полностью отсутствовать в другой, пусть даже более старой), либо вообще подумать об обновлении инструментария.

Ненамного отстают по популярности среди авторов защит всевозможные приемы определения присутствия отладчиков, от откровенно примитивной проверки наличия определенных файлов/ключей реестра (отдельные разработчики защит даже удаляют эти ключи, нимало не утруждая свой беспросветно могучий интеллект мыслями о том, что SoftIce можно приобрести легально и использовать не для взлома их поделок) до довольно изощренных антиотладочных приемов, использующих особенности конкретных отладчиков. Примерами таких особенностей могут служить «черные ходы» в SoftIce для взаимодействия с Bounds Checker'ом или нездоровая реакция на вызов IsDebuggerPresent в OllyDebug и всех остальных отладчиках, использующих Debug API. Кстати, признаки наличия отладчика могут быть не только информационными, но и физическими: программа может «догадаться» о том, что ее ломают, по ненормально большому времени выполнения тех или иных процедур. Задумайтесь над тем, сколько времени уходит на выполнение десятка команд в «ручном режиме», когда Вы исступленно давите кнопку F8 в OllyDebug - и Вы сразу поймете, что я имею в виду. К этой же группе можно отнести использование в защитных механизмах отладочных регистров процессора: поскольку эти регистры используются отладчиком для установки брейкпойнтов, одновременная их эксплуатация программой и отладчиком невозможна, если попытаться проделать такое, либо отладчик «забудет» аппаратные точки останова, либо защитные процедуры выдадут некорректный результат со всеми вытекающими из этого последствиями. Большинство антиотладочных приемов, разумеется, давно и хорошо известны, и их описание несложно найти в руководствах по крэкингу и написанию защит. Впрочем, авторы защит на такие приемы обычно всерьез не рассчитывают, поскольку идентифицировать (а часто – и обойти) антиотладочный код в дизассемблерном листинге обычно несложно (например, если прикладная программа пытается оперировать с отладочными регистрами, это очевидный признак того, что в коде «что-то нечисто»), а некоторые крэкерские инструменты среди своих функций имеют отслеживание популярных антиотладочных приемов (примером может служить старая утилита FrogsIce, которая умела выявлять и побеждать множество защитных трюков, направленных против SoftIce).

Наиболее популярным среди начинающих крэкеров, по-видимому, еще долго будет оставаться вопрос: «я поставил брейкпойнты на GetWindowText'ы и GetDlgItemText'ы, и все равно не могу поймать момент чтения серийника из окна». Действительно, формально все вроде бы сделано правильно, и все подходящие функции из «поминальника» обвешаны точками останова, как новогодняя елка – игрушками, но отладчик все равно не подает ни малейших признаков активности. При этом все точки останова находятся на своих местах и вполне успешно срабатывают – но, увы, не по тому поводу, который Вам интересен. В общем, у неопытного кодокопателя может сложиться впечатление, что серийный номер считывается при помощи телепатии или, как минимум, весьма недокументированным способом, чтобы обнаружить который нужно иметь не меньше семи пядей во лбу. Однако в действительности никаких телепатических датчиков в Вашем компьютере нет (а если даже и есть, то вряд ли они используются для чтения текста из диалоговых окон), да и подозревать недокументированные приемы я бы тоже не торопился, поскольку существует куда более простое объяснение этого явления. В Windows с давних пор сосуществуют два различных механизма, позволяющих управлять окнами и некоторыми другими объектами. Об одном из этих механизмов – системных вызовах Windows API я уже говорил, и даже дал в предыдущей главе небольшой список наиболее употребительных функций с комментариями по поводу области их применения. Другая же сторона Windows до настоящего момента как-то оставалась в тени, за исключением эпизодических упоминаний «по поводу». Вы, наверное, уже догадались, что это за «другая сторона Windows»: я говорю о широко используемых в нашей любимой ОС сообщениях

(хотя, если быть до конца точным, сообщения в том или ином виде присутствуют в большинстве современных операционных систем).

Если функции WinAPI безраздельно властвуют в темном и мрачном царстве невизуальных объектов, таких, как файлы, процессы, средства синхронизации и прочее, то в «оконной» области ситуация отличается разительным образом. Сравнительно небольшой набор системных вызовов общего назначения («создать-включить-удалить окно») с лихвой компенсируется огромным разнообразием системных сообщений (в англоязычной документации – «messages»; общее число документированных сообщений уж перевалило за тысячу), подчас дублирующих функции WinAPI (например, сообщение WM_GETTEXT, которое способно читать текст окна не хуже, чем уже известная Вам функция GetWindowText). Некоторые типы управляющих элементов, такие, как обычные или выпадающие списки, вообще не имеют полноценной «обвязки» функциями Win32 API и управляются с ними именно при помощи сообщений. Вы не сможете добавить в такой список строчку или перейти к нужной позиции, вызвав WinAPI'шную функцию с названием вроде ComboBoxAddString или ComboBoxSetPos – таких функций в системных библиотеках Windows просто нет. Зато есть сообщения CB_ADDSTRING и CB_SETCURSEL соответственно, воспользовавшись которыми, Вы легко выполните задуманное. То есть, сообщения играют роль параллельного механизма управления объектами ОС, на работу которого совершенно не влияют традиционные брейкпойнты, которые мы щедрой рукой сеяли в предыдущей главе.

Поскольку сообщение – не функция, брейкпойнт на него поставить нельзя. Но очень хочется. А если очень хочется – значит, все-таки можно, хотя и не так просто, как хотелось бы. Прежде всего, Вам нужно определиться, **что именно** Вы хотите отловить – момент и точку отправки сообщения, либо подпрограмму обработки этого сообщения. Если Вас интересует второй вариант и Вы являетесь поклонником SoftIce – считайте, что о Вас уже позаботилась фирма NuMega (ныне - Compuware). Встроенная в SoftIce команда BMSG как раз для этого и предназначена, но чтобы успешно ее использовать, Вам понадобится узнать хэндл окна, которому предназначено сообщение. Если нужные данные у Вас имеются – просто набирайте **BMSG**<хэндл_окна> <код_сообщения>, и ждите, когда «всплывет» отладчик. Разумеется, команда BMSG, как и любая другая команда установки брейкпойнтов, позволяет создавать условные точки останова, срабатывающие, например, при поступлении сообщений только с определенными значениями wParam и lParam.

А что делать тем, кто пользуется другими отладчиками, в которых аналог BMSG отсутствует? Ответ на этот вопрос находится, как ни странно, именно в руководстве по SoftIce. В частности, там написано, что действие команды BMSG может быть воспроизведено установкой условного брейкпойнта на оконную процедуру, причем в качестве условия нужно указать следующее: **IF**(**esp**->8)==<имя_сообщения>; адаптация этого условия под синтаксис, принятый в конкретном отладчике, обычно сложности не представляет, хотя вместо символьного имени сообщения скорее всего придется подставить его код (коды сообщений можно найти в файлах windows.inc, winnt.h или Messages.pas – в зависимости от того, компилятор какого языка у Вас есть под рукой; те, кто не обзавелся подходящим компилятором и не планируют им обзаводиться в ближайшем будущем, могут заглянуть в файл messages.lst из состава InqSoft Window Scanner).

Такой подход несколько сложнее, чем вызвать команду BMSG с нужными параметрами, но зато он дает одно немаловажное преимущество. В предыдущем абзаце я сделал замечание насчет необходимости знать хэндл окна для успешного применения этой команды. Однако хэндл окна может быть Вам известен далеко не во всех случаях. Рассмотрим, к примеру, ситуацию, когда Вам нужно оттрассировать обработчик сообщения WM_INITDIALOG. Вы не сможете просто взять и посмотреть нужный Вам хэндл, поскольку окно только-только создано и могло еще даже не появиться на экране. Конечно, можно «заморозить» все исследуемое приложение и затем предпринять поиск среди всех окон в системе, но не кажется ли Вам, что это несколько сложнее, чем хотелось бы?

Кроме того, при каждом запуске программы хэндлы меняются, что тоже отнюдь не упрощает отладку. А вот оконная процедура всегда находится на одном и том же месте (справедливости ради надо отметить, что создание «плавающей» по адресному пространству от запуска к запуску процедуры теоретически возможно, хотя я такое и не встречал). И потому адрес этой процедуры можно просто записать на бумажке, чтобы затем восстанавливать соответствующий бряк без каких-либо сложностей. Нам осталось только раздобыть адрес этой самой процедуры. Тут тоже, в принципе, ничего сложного нет – разумеется, если под рукой имеются соответствующие инструменты (к примеру, Microsoft Spy++ или все тот же InqSoft Window Scanner). Наведите «прицел» программы на интересующее Вас окно и прочитайте желанный адрес оконной процедуры собственно окна (обычно этот адрес обозначается как WndProc) или оконной процедуры, сопоставленной классу окна.

Иной путь получения адреса оконной процедуры заключается в том, чтобы при помощи API-шпионов обнаружить системный вызов, при помощи которого производится регистрация класса окна (функции WinAPI RegisterClass и RegisterClassEx) либо непосредственно создание окна (список соответствующих функций я приводил в предыдущей главе). Операция эта выполняется в три этапа:

1. Запускаем под API-шпионом, настроенным на отслеживание процедур создания окон, и ждем появления нужного окна.
2. Как только окно появится – останавливаем работу шпиона и при помощи любого сканера окон получаем хэндл этого окна.
3. Если адрес оконной процедуры находится среди параметров функции создания окна - ищем в логе, сгенерированном API-шпионом, функцию, которая возвращает значение нашего хэндла, и считываем ее параметры, среди которых находим искомый адрес оконной процедуры.
4. Если адрес оконной процедуры находится в структуре, указатель на которую передается в функцию регистрации классов – считываем адрес, откуда был произведен вызов этой функции. Затем загружаем программу в отладчик, ставим точку останова на этот адрес (или чуть выше – на том месте, где происходит запись в стек указателя на структуру, это уж как Вам больше понравится) и запускаем программу. Как только исполнение программы прервется на нашем брейкпойнте – находим в памяти структуру, указатель на которую передается в RegisterClass[Ex] и аккуратно переписываем содержимое поля этой структуры, содержащее адрес оконной процедуры для регистрируемого класса.

Вас может смутить сложность четвертого пункта, который выполняет весьма несложные функции, но при этом его описание едва ли не длиннее предыдущих трех. Казалось бы, что нам мешает просто вытащить из лога API-шпиона значение указателя на структуру, по-быстрому снять дамп нужной области и прочитать желанный адрес? В принципе, ничего не мешает – но вот истинное содержимое структуры WNDCLASSEX Вы таким способом вряд ли прочитаете – потому что скорее всего в момент снятия дампа эта структура уже давно будет затерта другими данными. Дело в том, что регистрация класса – событие разовое, и потому память под структуру, описывающую класс, редко выделяют статически; обычно же программист обходится для этих целей куском стека. Так что когда Вы заберетесь своим дампером в адресное пространство процесса, в том месте, где находилась желанная структура, давно уже будут лежать другие данные. И единственным решением в данном случае мог бы быть интеллектуальный API-шпион, которому можно было бы объяснить правила извлечения полей структур из памяти. К сожалению, на данный момент API-шпионы с такими свойствами мне не известны.

Другой распространенной причиной, по которой может «не работают» брейкпойнты, является маскировка действий защиты под что-нибудь совершенно безобидное или нетривиальная реализация защитных механизмов. В повседневной жизни мы очень часто руководствуемся правилом «если что-то выглядит, как утка и крякает, как утка – значит, это и есть утка». Более того, данное правило - один из столпов того, что мы называем здравым смыслом. Однако Вы наверняка

замечали, что правило это – не без изъяна, и не так уж редко видимая картина мира отнюдь не соответствует истинной. В крэкинге это противоречие между видимым эффектом и скрытым от невооруженного глаза назначением защитного кода может быть доведено до предела, поскольку, взламывая программу, крэкер не просто копается в машинном коде, но ведет интеллектуальный поединок с автором защиты. И со стороны противника можно ожидать всего – блефа в виде процедур-«пустышек», имитирующих защиту, сверхсложных схем, решающих простейшие задачи, ловких имитаций, призванных повести крэкера по ложному пути, и, наконец, многоуровневой системы проверок, которые не слишком сложно реализуются, но достаточно долго и нудно обезвреживаются. При написании защит редко задаются вопросами оптимальности, скорости и расхода ресурсов – все эти добродетели программирования приносятся в жертву защищенности.

Я уже приводил пример того, как программа считывала дату своей установки под видом поиска плагинов в своей директории, и, разумеется, этим список возможных приемов маскировки одних действий под другие не исчерпывается. Программа eXeScope, например, в качестве сообщения об ограничении в незарегистрированной версии выдает окно, внешним видом точь-в-точь повторяющее стандартный MessageBox, но в действительности нарисованное визуальными средствами в Delphi. Отображение файла в память вместо обычного чтения в буфер – прием известный, и, тем не менее, чтение файла лицензии таким способом вполне может поставить в тупик начинающего крэкера. Я уж не говорю о таких изощренных техниках, как парсинг ini-файлов «вручную» (после чего можно очень долго возиться с точками останова на GetPrivateProfile* - разумеется, с нулевым результатом) или экспорт кусков реестра при помощи утилиты regedit во временный файл с последующим анализом этого файла (что позволяет обойтись без вызова функций работы с реестром внутри программы).

Однако наиболее интересным для читателя, я думаю, будет рассмотрение причин, по которым точки останова просто исчезают из отлаживаемой программы. Я мог бы просто назвать причину таких мистических исчезновений и изложить типовой способ решения этой проблемы, но, думаю, Вам будет гораздо интереснее понять причины, по которым «теряются» брейкпойнты. А уж теоретические знания помогут Вам самостоятельно найти подходы к решению этой проблемы еще до того, как Вы доберетесь до готовых рецептов. Очевидно, что прежде чем разбираться в защитных приемах, подавляющих точки останова, нужно сначала понять физический смысл этих самых точек, то есть узнать, что они собой представляют, как устанавливаются и по каким признакам программа может догадаться об их наличии. А поскольку точки останова – изобретение отнюдь не новое, рассказ о них следует начать с исторического экскурса в седую древность.

В свое время самым популярным отладчиком для «Спектрума» был MONS (впрочем, некоторые люди, включая меня, предпочитали MON) – восьмикилобайтное порождение программистской мысли, способное загружаться в ОЗУ с любого адреса и управляемое из командной строки (прямо как SoftIce – внешнее сходство этих двух отладчиков вообще сложно не заметить). И, разумеется, MONS позволял ставить брейкпойнты – еще бы, не имея в своем арсенале такой возможности, этот отладчик вряд ли стал бы столь популярен. Но поскольку процессор Z80, на основе которого был сделан «Спектрум», никаких отладочных средств не предоставлял, авторам MONS пришлось реализовывать точки останова чисто программными средствами. Реализация эта красотой отнюдь не блистала – «установка брейкпойнта» по-Спектрумовски заключалась в подстановке в нужное место кода трехбайтной команды CALL xxxx, которая передавала исполнение в недра самого отладчика и таким образом приостанавливала исполнение пользовательского кода. Старые команды, код которых затирался брейкпойнтом, копировались в специальный буфер и дополнялись командой JP (аналог jmp из набора команд x86) для возврата к следующей команде, не испорченной CALL'ом. Исполнение в пошаговом режиме выглядело не менее оригинальным – исполняемая команда перебрасывалась в отдельный буфер, дополнялась все тем же JP, после чего отладчик передавал управление в этот буфер. Если еще вспомнить, что в Z80 существовали недокументированные команды, которые были известны далеко не всем отладчикам (и потому

могли обрабатываться некорректно), отлаживаемая программа даже при абсолютно корректной работе могла испортить код отладчика, а под сам отладчик могло элементарно не хватить свободной памяти, и потому его загружали на место «ненужных» данных – Вы поймете, что представляла собой отладка в старые добрые времена.

Разработчики линейки x86 проявили больше заботы о программистах. В этой линейке процессоров вместо самодельной «затычки» в виде команды вызова подпрограммы для отладочных целей ввели отдельное прерывание с номером 3, которое вызывалось однобайтной командой (опкод команды int 3 – CC), в отличие от всех прочих прерываний, которые менее чем двумя байтами вызвать не получится. Другим полезным нововведением стала возможность исполнять код в пошаговом режиме через управление флагом трассировки (эта возможность, впрочем, мало актуальна для отладчиков пользовательского уровня под современные ОС). Однако, несмотря на такой, казалось бы, очевидный прогресс в развитии средств отладки, обыкновенные точки останова все так же, как и десятилетия назад, модифицируют исполняемый код, а потому легко обнаруживаются даже простейшими способами, например, проверкой контрольной суммы всех байтов (не говоря уже о CRC32 и использовании иных, еще более сложных и надежных хэш-функций). Самостоятельно убедиться в том, что точки останова модифицируют код, Вы можете за считанные секунды: откомпилируйте при помощи любого ассемблера следующие две строчки, возьмите OllyDebug и загрузите в него откомпилированный код.

```
addr1: mov eax,addr1
mov al, byte ptr [eax]
```

Если Вы просто выполните этот код в пошаговом режиме, то в регистре AL окажется число 0B8h.В этом нет ничего удивительного, B8 – это опкод команды mov eax, <число>. А теперь попробуйте поставить брейкпойнт на команду mov eax,addr1 и снова оттрассируйте этот код. После выполнения второй команды Вы увидите, что в регистре AL находится число 0CCh, хотя код в окне отладчика внешне совершенно не изменился (если, конечно, не считать изменением подсветку адреса, на который поставлен брейкпойнт). Самое интересное, что отладчики могут «приукрашать реальность» не только в окне кода, но и при просмотре данных.

Давайте проделаем еще один весьма поучительный в этом смысле эксперимент: загрузим наш пример из двух команд, поставим точку останова на первую и запишите адрес этой точки останова. Затем берем InqSoft Window Scanner и читаем байт по записанному адресу. Получаем, разумеется, 0CCh. А теперь взглянем на ту же область глазами отладчика (в OllyDebug это пункт меню Follow in dump|Selection) – и очень сильно удивляемся. Отладчик показывает нам совсем не то, что реально читается из памяти в регистр AL, а то, что должно было бы находиться по указанному адресу, если бы мы не поставили точку останова. Но и это еще не все! Посмотрите на динамические подсказки под окном кода – там-то как раз содержимое памяти отображается как надо.

Вот так «умные» отладчики помогают самообманываться начинающим крэкерам: отсутствие видимых изменений в коде наводит человека, не знакомого с тайнами устройства брейкпойнтов, на мысль о том, что прерывание исполнения программы в точке останова происходит по воле неких таинственных сил, с которыми отладчик находится в телепатической связи. Хотя на самом деле «классические» точки останова – это ни что иное, как обычные memory patch’и – а потому и обнаруживаются теми же самыми способами, что и любые другие исправления в коде.

Кстати, из того, что обычный (не аппаратный) брейкпойнт является ничем иным, как исправлением программы, есть одно интересное следствие. Дело в том, что SoftIce’у в общем-то без разницы, каким образом в программе появилась команда int 3 – главное, что он может на этой команде остановиться не хуже, чем на настоящем брейкпойнте. А после того, как отладчик остановится, можно внести любые поправки в содержимое регистра EIP и код программы, после чего продолжить исполнение как ни в чем не бывало (собственно, в OllyDebug тоже можно проделать такую операцию при помощи пункта New origin here из всплывающего меню). Польза от такого эрзац-брейкпойнта (после срабатывания которого, к тому же, нужно вручную восстанавливать код,

который находился на месте int 3 и править EIP), на первый взгляд кажется весьма сомнительной, но она есть. Я уже упоминал глюк в SoftIce, когда отлаживаемая программа после загрузки Symbol loader'ом начинает немедленно выполняться, хотя крэкеру хотелось бы ее в этот момент притормозить. Так вот, если в Entry point исполняемого файла воткнуть опкод 0CCh, у подопытной программы не будет ни единого шанса избегнуть процесса отладки – поскольку первой командой окажется наш int 3, принудительно активирующий отладчик.

Теперь, когда мы знаем, что точки останова обнаружить можно (и даже знаем, как их можно обнаружить), можно вернуться к основному вопросу этой главы – «почему точки останова не срабатывают». В нашем случае этот вопрос можно даже конкретизировать – «какими способами подопытная программа может удалить из себя точку останова». В различных источниках мне неоднократно встречалось предложение использовать для этой цели коды коррекции ошибок, предваряя все «критичные ко взлому» участки программы вызовом функции проверки и восстановления кода. В случае изменения кода из-за появления точек останова процедура восстановления должна откорректировать «неправильные» байты. Теоретически такая схема вполне возможна, но на практике алгоритмы коррекции ошибок довольно сложны в реализации и не слишком производительны, так что народные массы эту идею не приняли. А вот более простой вариант восстановления кода из «резервной копии», расположенной в другом конце программы (или прямого вызова этой резервной процедуры вместо основной), таки имел место во времена ДОСа; впрочем я не удивлюсь, если выяснится, что такой прием до сих пор в ходу – реализация очень проста, а какой-никакой эффект все-таки имеется.

На практике дело обстоит еще хуже – для удаления некоторых точек останова не нужны ни коды коррекции ошибок, ни резервные копии. И именно к таким точкам останова относятся всеми нами любимые BPX'ы на вызовы функций WinAPI (и, если смотреть шире, на вызовы практически любых функций). Поскольку «брейкпойнт на функцию» - это на самом деле всего лишь брейкпойнт на первый байт этой функции, самый простой из приемов, удаляющих точки останова, выглядит следующим образом: заранее узнать адреса нужных функций при помощи GetProcAddress, прочитать их первые байты (если речь идет о внутренних функциях программы – то просто прочитать содержимое соответствующей ячейки) и сохранить значения этих эталонных байтов. Затем перед особо критичными вызовами нужно лишь сравнивать первые байты процедур с эталонным, и, если обнаружится несоответствие, восстанавливать их. Сам факт того, что процедура начинается с опкода 0CCh говорит о том, что на эту процедуру поставлена точка останова, что может побудить программу предпринять некоторые действия по самозащите. Если учитывать, что многие процедуры начинаются стандартной последовательностью команд push ebp; mov ebp, esp (в шестнадцатиричном редакторе эти команды выглядят как последовательность 55 8B EC), то «действия по самозащите» могут быть простой записью в первые три байта процедуры той самой стандартной последовательности 55 8B EC. После этой операции точка останова, разумеется, исчезнет. Разумеется, выявить защиту от брейкпойнтов, основанную на проверке содержимого неких адресов в памяти, не слишком сложно – нужно лишь поставить аппаратную точку останова на чтение/запись первого байта функции и посмотреть, где этот «капкан на защиту» сработает.

Другой способ постановки бряков на импортированные из DLL функции основан на том, что вызов импортированной функции почти всегда выполняется не напрямую, а через «переходник». На практике вызовы через «переходник» обычно выполняются одним из двух способов.

Первый способ:

```
call <переходник_к_MyFunc> ; Вызов функции API
…
переходник_к_MyFunc: jmp MyFunc
```

(этот способ вызова функций наиболее распространен; переходники вида «jmp истинный_адрес_функции» обычно собраны в конце программы)

Второй способ:

mov edi, dword ptr ds:[элемент_в_таблице]

```
mov edi, dword ptr ds:[элемент_в_таблице]
call edi
…
элемент_в_таблице: dd <истинный_адрес_функции_MyFunc>
```

(данная техника вызова функций обычно встречается в продуктах Microsoft)

Идея заключается в том, что в первом случае точку останова можно поставить не на первый байт функции, а на переходник, через который вызывается эта функция. В первом случае это будет обычный BPX на адрес команды jmp MyFunc, во втором случае придется прибегнуть к аппаратной точке останова на чтение двойного слова по адресу «элемент_в_таблице». Поскольку это не будут брейкпойнты на функцию в прямом смысле слова, этот метод имеет одно существенное ограничение: если нужная функция вызывается не через «переходник», а непосредственно по значению ее адреса (получаемому, например, при помощи GetProcAddress), то такой брейкпойнт, понятное дело, не сработает. Разумеется, также существует возможность, что программа попытается проверить целостность «переходников», но как такие попытки обнаруживать, Вы уже знаете.

Если некая процедура вызывается внутри программы стандартным образом, то большинство современных компиляторов генерирует последовательность команд push для помещения параметров функции на стек, собственно переход к процедуре выполняется при помощи команды CALL, а после того, как функция отработает, управление возвращается на команду, следующую за CALL. Однако если программист имеет достаточно высокую квалификацию, он может внести заметное разнообразие в эту картину при помощи «ручного» вызова функций средствами ассемблера. Хотя великий Intel завещал нам вызывать процедуры и функции при помощи специально для этого придуманной команды CALL, отдельным гражданам закон не писан (надо отметить, в число этих граждан входят не только авторы защит, но и любители предельной оптимизации, а также фанаты нетрадиционного программирования). И вот эти странные граждане сочинили несколько имитаций несчастной команды CALL, и эти имитации давно и прочно вошли в арсенал разработчиков защит. Из универсальных нетрадиционных средств вызова подпрограмм прежде всего нужно назвать следующие:

```
push <адрес возврата>
jmp <адрес процедуры>
```

или

push <адрес возврата> push <адрес процедуры> ret

Более сложные техники неявной передачи управления основаны на умышленном создании и обработке исключительных ситуаций, вызове прерываний и эксплуатации особенностей конкретных ОС. Эти техники сами по себе представляют весьма значительный интерес – с точки зрения как крэкера, так и программиста, однако их количество практически бесконечно, а сложность нередко выходит далеко за пределами «основ». Чтобы Вы имели представление о том, насколько обширна эта тема, сообщу, что любые функции WinAPI (да и вообще любого другого API), в параметрах которых фигурирует callback-функция, могут служить инструментом неочевидного вызова пользовательского кода.

Вместо рассмотрения всего этого бесконечного разнообразия возможных приемов (большинство из которых Вы, возможно, вообще никогда не встретите) мы углубимся в исследование возможностей приведенной выше пары базовых «заменителей CALL», понимание которых в итоге дает ключ к «раскалыванию» многих других способов неочевидного вызова процедур. Прежде всего следует отметить, что помещение на стек адреса возврата в этих методах отделено от собственно вызова

процедуры, что позволяет вклинить между двумя этими действиями практически любой код, например, кусок вызываемой процедуры – и, соответственно, вызывать эту процедуру не с первого байта, а «с середины». Например, вот таким образом:

```
push <адрес возврата>
push ebp
mov ebp, esp
jmp MyProc+3 ; (1)
…

MyProc:
push ebp
mov ebp,esp
… ; При вызове процедуры в точке (1) будет выполнен переход в эту точку
```

Как видите, хотя приведенный кусок кода по функциональности полностью аналогичен тривиальному call MyProc, явным образом процедура MyProc нигде не вызывается. Причем при помощи макросов можно добиться того, что все вызовы процедуры MyProc в программе будут выглядеть именно таким образом! Так что, сколько бы Вы ни ставили брейкпойнтов на адрес MyProc, ни один из них никогда не сработает – по той простой причине, что управление на этот адрес просто никогда не передается, хотя все внешние признаки могут говорить о том, что процедура MyProc успешно отработала. Впрочем, обмануть такой защитный прием обычно не составляет никакой сложности – нужно лишь ставить точку останова не на первую команду в процедуре, а где-нибудь подальше, например, после третьей (можно даже на команду выхода из процедуры, но при этом следует помнить, что процедура может содержать несколько точек выхода) или вообще в какой-нибудь подпрограмме, вызываемой этой процедурой (вторым способом я нередко пользуюсь, когда ставлю точки останова на функции WinAPI).

Другая проблема, которую порождают нетрадиционные способы вызова процедур, заключается в том, что адрес возврата может быть совершенно любым, а не только адресом команды, следующей за командой вызова. Этот прием встречается довольно часто, когда автор защиты хочет скрыть адрес какой-либо функции, к примеру, проверки корректности введенного серийного номера. Рассуждая логически, он понимает, что необходимо максимально осложнить крэкеру нахождение связи между появлением сообщения о неверном серийнике и процедурой, этот серийник проверяющей. А поскольку традиционным приемом поиска такой процедуры является BPX MessageBox с последующим наблюдением, куда вернется программа из MessageBox'а – программист делает вывод, что хорошо бы сделать так, чтобы программа вернулась «не туда», т.е. как можно дальше от процедуры проверки серийника. В этом случае даже поставив брейкпойнт «куда надо», мы не узнаем, по какому поводу был произведен вызов функции – на стеке будет лежать совершенно другой адрес возврата. И особенно неприятно для начинающего крэкера, когда в качестве адреса возврата оказывается что-нибудь вроде адреса функции ExitProcess.

В общем случае «победить» такой прием можно либо через долгую медитацию с массированным применением шестнадцатиричного редактора/дизассемблера для поиска всех точек, в которых программа явно или неявно оперирует адресами нужных функций WinAPI, либо через поиск «модифицированным методом Ньютона», описанным в предыдущей главе, либо применив средства трассировки кода, имеющиеся в SoftIce или OllyDebug. Собственно, трассировка в таких случаях является орудием, по свойствам приближающимся к термоядерной бомбе: редкий код способен выдержать такой удар, но чтобы получить желаемый эффект, требуется весьма серьезное техническое обеспечение (процесс трассировки требует немалых объемов памяти и достаточно быстрого процессора) и грамотный выбор области применения. Так что, прежде чем пускать в ход «тяжелое вооружение», есть смысл подумать о решении проблемы более простыми средствами.

Одним из таких более простых средств является исследование содержимого стека на предмет «застрявших» в нем полезных данных и адресов. Суть метода заключается в следующем: любой

кусок кода в программе существуют не сами по себе, но находятся во взаимодействии с чем-то, и каждая из процедур может быть как вызываемой (из процедуры более высокого уровня), так и вызывающей «подчиненные» ей процедуры. И даже когда программист скрыл точку вызова конкретной процедуры (что, как я продемонстрировал, не так уж сложно), то спрятать от пытливого взора следы, оставленные выше- и нижележащими процедурами, ему могло и не удаться. А если «могло не удаться» - есть смысл попробовать отыскать эти следы.

По традиции, для начала разберемся, что эти следы из себя представляют. Представьте себе следующую широко распространенную ситуацию: A=>B=>C, где «A=>B» расшифровывается как «процедура A вызывает процедуру B». При вызове процедуры B стандартными средствами, т.е. командой CALL, в момент начала исполнения процедуры B на вершине стека будет лежать адрес возврата из процедуры B в процедуру A. Аналогичный процесс происходит при вызове процедуры C процедурой B. Получается, что если мы поставим брейкпойнт на точку входа в процедуру C, мы в этот момент сможем наблюдать в стеке следующую картину (вершина стека - вверху):

• Адрес возврата из процедуры C в процедуру B
• Адрес возврата из процедуры B в процедуру A

Немного усложним картину, и допустим, что в процедуры B и C передаются некие параметры (порядок передачи параметров для нас в данном примере несущественен). Стек в этом случае будет выглядеть следующим образом:

• Адрес возврата из процедуры C в процедуру B
• Параметры, переданные в процедуру C
• Адрес возврата из процедуры B в процедуру A
• Параметры, переданные в процедуру B

На практике процедуры обычно занимаются чем-то более сложным, чем простой вызов других процедур с параметрами, а потому довольно часто резервируют на стеке место под локальные переменные. Допустим, что процедуры A и B используют локальные переменные, место под которые выделяется на том же стеке, и посмотрим, что после этого будет твориться в стеке:

• Адрес возврата из процедуры C в процедуру B
• Параметры, переданные в процедуру C
• Область локальных переменных процедуры B
• Адрес возврата из процедуры B в процедуру A
• Параметры, переданные в процедуру B
• Область локальных переменных процедуры A

А теперь представим, что автор защиты на этапе B=>C подменил адрес возврата из процедуры C в процедуру B своим собственным значением, и возврат теперь происходит не в B, а некую процедуру D. Что от этого изменится? Да только одна, самая верхняя строчка! А вот адрес возврата в процедуру A, локальные переменные и параметры вызова какими были, такими и останутся, и это можно использовать в качестве зацепки, позволяющей выявить все этапы пути от процедуры A к процедуре C. Другое дело, что информация, лежащая в стеке, никак не структурирована, поэтому Вам придется самому угадывать, что там – локальные переменные, что – параметры вызовов, а что – адреса возврата. И хотя процесс проверки этих догадок может быть весьма трудоемким, лучше иметь хотя бы такую беспорядочную информацию, чем не иметь никакой. Иногда бывает полезно посмотреть, что находится выше вершины стека – там нередко тоже удается обнаружить следы деятельности процедур, отработавших перед тем, как мы остановили программу.

Теоретические основы крэкинга: Глава 10. Слишком хорошо – тоже не хорошо.

Автор: CyberManiac

Дата: 2005-03-17

Как Вы уже успели убедиться, обращение с брейкпойнтами – занятие далеко не такое простое, как может показаться с первого взгляда. В предыдущей главе мы разбирались в том, почему точки останова иногда не делают того, что мы от них ожидаем, значительная же часть этой главы посвящена полностью противоположной проблеме: что делать, если точки останова срабатывают слишком часто. После всего написанного в предыдущей главе такая ситуация может показаться Вам невероятной, но, представьте себе, такое тоже бывает.

Если Вы входите в число поклонников отладчика SoftIce, то Вы не могли не заметить одну милую особенность этого инструмента: если Вы поставили брейкпойнт непосредственно на системную функцию, SoftIce будет «всплывать» при каждой попытке исполнить эту функцию независимо от того, внутри какого процесса функция была вызвана. Такое поведение отладчика, несомненно, бывает полезным при отладке драйверов, хук-процедур и самой операционной системы. Но наша беда (или счастье – это уж с какой стороны посмотреть) в том, что мы отлаживаем «обычную» программу, и постоянные выпадения в отладчик по совершенно неинтересным поводам – это далеко не то, о чем мы мечтали. Вот если бы удалось сделать так, чтобы брейкпойнты работали внутри только одного процесса… Те, кто начал знакомиться с SoftIceпо пакету NuMegaDriverStudio 2.6, скорее всего не увидят в этом никакой проблемы: BPX <имя_функции> IFPID=<ID_нужного_процесса> - и дело в шляпе. Набрав это заклинание, они, в общем-то, будут полностью правы, ибо это вполне хороший и, в общем-то, самый короткий путь к цели. И я никогда не скажу дурного слова про тех, кто последует этим путем. Однако этот путь – далеко не единственный из возможных, и если Вам интересны иные способы решения проблемы постановки брейкпойнта – добро пожаловать в музей истории SoftIce.

Мое общение с этим отладчиком началось с версии 3.23 для Windows 9x, случайно обнаруженной на свежем «хакерском» диске. Пусть по нынешним меркам тот СофтАйс совершенно не производит впечатления - тогда это был шедевр! Впрочем, то был шедевр не без недостатков, самым главным из которых была его неразборчивость в срабатывании брейкпойнтов. Как я отмечал, поставить брейкпойнт на функцию WinAPI, срабатывающий исключительно внутри нужного процесса (по-научному брейкпойнты с такими свойствами называются «address-contextsensitive», они же «контекстно-зависимые»), напрямую было невозможно. Чтобы почувствовать, насколько серьезной была проблема, представьте себе следующую картину: бряк, поставленный на функции чтения из реестра (которая называется RegQueryValue[Ex], надеюсь, Вы еще не забыли мой «поминальник»), работает настолько хорошо, что отлавливает абсолютно все попытки чтения из реестра, независимо от того, выполняет их отлаживаемая Вами программа или какая-либо другая. Любые попытки осмысленной отладки в такой ситуации заведомо обречены на провал, единственное, чем Вы будете заниматься – это нажимание клавиш Ctrl-D, ибо Windowsнастолько сильно любит читать данные из реестра, что делает это много раз в секунду (если хотите своими глазами посмотреть на эту странную любовь, RegMon Вам в этом поможет). Так что же, поставить бряк на RegQueryValueExи получить от этого удовлетворительный результат совсем никак невозможно? Как бы не так! «Если нельзя, но очень хочется, то все-таки можно».

Начнем с небольшого, но очень важного определения: **адресный контекст (он же контекст процесса) – это все виртуальное адресное пространство, выделенное данному процессу**.

«Военная хитрость», при помощи которой мы «проапгрейдим» контекстно-независимый брейкпойнт до контекстно-зависимого, основана на том факте, что хотя SoftIceи игнорирует контекст в момент

срабатывания контекстно-независимых брейкпойнтов (в число которых входят и брейкпойнты на функции WinAPI), сам текущий адресный контекст от этого никуда не исчезает. И потому в момент срабатывания условного брейкпойнта отладчик вполне способен прочитать любые данные из существующего в этот момент виртуального адресного пространства. Проще говоря, если брейкпойнт сработал внутри программы X, то отладчик сможет «увидеть» адресное пространство программы X вместе со всеми данными, содержащимися в этом пространстве, а заодно и содержимое всех регистров, каким оно было в момент срабатывания брейкпойнта. А если отладчик «видит» все адресное пространство процесса и способен читать исполняемый код программы, значит, можно попытаться идентифицировать процессы по особенностям их исполняемого кода! На практике эта «идентификация по особенностям исполняемого кода» выглядит весьма прозаично: нужно забраться при помощи шестнадцатиричного редактора внутрь секции кода (впрочем, для идентификации можно использовать и любые другие заведомо не изменяющиеся при работе программы данные), выдернуть оттуда первый попавшийся DWORD (назовем его **My** *DWORD) и запомнить виртуальный адрес (My***Addr** соответственно), по которому этот DWORDнаходился. Дальнейшие операции, выполняемые уже в отладчике, ненамного сложнее: BPXимя_функции_WinAPIIF(*My_Addr)==My_DWORD. Все.

Думаю, с пониманием того, **что** делает эта команда, ни у кого сложностей не возникло: мы соорудили условную точку останова, которая в момент срабатывания проверяет, не лежит ли по адресу My_Addrдвойное слово, равное My_DWORD. И если такое двойное слово по нужному адресу обнаруживается, то отладчик приостанавливает исполнение программы. Поскольку случайно встретить пару программ, у которых в секции кода по одним и тем же виртуальным адресам находились бы одинаковые DWORD’ы, вряд ли возможно, такой простейший способ различения процессов в абсолютном большинстве случаев отлично срабатывает. Сложности могут возникнуть только в двух случаях: при отладке упакованных программ и если нужно параллельно отлаживать два экземпляра («экземпляр» следует понимать как «instance») одного и того же приложения – поскольку код обеих процессов идентичен, в общем случае различить их по содержимому адресного пространства затруднительно.

В настоящее время применение описанного выше метода для определения, в контексте какого из процессов сработал брейкпойнт, в общем-то, не требуется – в современных версиях SoftIce(вообще, сам отладчик SoftIce, особенности работы с ним и различия между разными его версиями – это отдельная большая тема, которая будет закрыта не раньше, чем прекратится развитие самого SoftIce) эта проблема решается безо всяких ухищрений. Однако поскольку особенности отдельных инструментов имеют весьма слабое отношение к теоретическим вопросам крэкинга, которым посвящена данная работа, я в дальнейшем не буду акцентировать внимание на этих особенностях – разобравшись в предлагаемом материале, Вы сами найдете способ с максимальной эффективностью использовать эти особенности. Для того же, чтобы избежать путаницы при изложении материала этой главы, мы будем считать, что все брейкпойнты, о которых ниже пойдет речь, являются контекстно-зависимыми, то есть работают исключительно в рамках того процесса, в котором они установлены. Таким образом, мы «уравняем в правах» SoftIceи отладчики третьего кольца защиты («ring3 debuggers»), к которым, в частности, относится OllyDebug.

Само по себе создание условного брейкпойнта, срабатывающего исключительно при появлении по некоему адресу известного значения – прием весьма часто употребляемый и полезный во множестве ситуаций. Самое известное из применений этой техники: «остановить программу, если переменная равна некоему значению» мы рассматривать не будем по причине крайней его банальности, такие вещи Вы способны проделывать самостоятельно, и, возможно, даже с закрытыми глазами. Куда менее очевидна возможность использования брейкпойнтов чтобы «притормозить» упакованную программу сразу после начала ее исполнения. Возможно, Вы уже встречали в статьях по крэкингу сокращение «OEP», которое расшифровывается как

OriginalEntryPoint, «оригинальная («истинная») точка входа». Возможно, Вы также читали о том, что распаковка сжатых исполняемых файлов включает в себя поиск адреса этой самой OEP. Если же Вы ничего такого еще не читали и не встречали, то Вам необходимо запомнить следующие базовые сведения:

1. 1. Entrypoint – это точка, с которой начинается исполнение программы после загрузки. Адрес точки входа в Win32-приложениях хранится в PE-заголовке исполняемого файла.
2. 2. Абсолютное большинство упаковщиков и навесных защит изменяют значение адреса точки входа таким образом, чтобы управление передавалось распаковщику или защитному модулю соответственно.
3. 3. Originalentrypoint – это entrypointисполняемого файла до того, как файл был сжат/зашифрован и значение точки входа было модифицировано упаковщиком.
4. 4. Для успешной распаковки программы в общем случае требуется узнать адрес OEP, остановить программу в тот момент, когда распаковщик передаст управление на OEP и в этот момент снять дамп со всех секций процесса. Также довольно часто после снятия дампа приходится восстанавливать таблицу импорта, в некоторых случаях может потребоваться ручная правка параметров секций.

Один из приемов отыскания OEPзаключается в следующем: нужно поставить на функции WinAPIбрейкпойнты таким образом, чтобы один из них сработал заведомо недалеко от точки входа в программу. Выяснив, откуда был произведен этот первый вызов WinAPI'шной функции, Вы сделаете первый (и самый важный) шаг на пути к адресу OEP. После этого обратной трассировкой в уме или каким-либо иным методом Вы будете «раскручивать» программу, как бы заставляя ее исполняться «задом наперед», и таким образом шаг за шагом подбираясь к OEP. Идея, лежащая в основе этого метода поиска OEP, крайне проста: современные компиляторы устроены так, что в коде абсолютного большинства программ неподалеку от точки входа содержится как минимум один вызов функции WinAPI (обычно такой функцией является GetModuleHandle или GetCommandLine).

Другой важный момент состоит в том, что практически каждый компилятор генерирует в начале программы специфический «начальный» (startup) код, который производит некие действия по инициализации данных, необходимых для работы стандартных библиотек. Причем этот код у разных компиляторов (а часто – у разных версий одного и того же компилятора или даже при разных опциях компиляции) имеет достаточно специфический и легко узнаваемый вид, причем повлиять на содержание этого startup-кода разработчик программы обычно не может (или может, но в ограниченных пределах). Поэтому чтобы Вам было легче искать OEP, я настоятельно рекомендую изучить, какой код помещают в начало программы наиболее распространенные компиляторы – так Вы быстро научитесь определять OEPисключительно по внешнему виду кода, находящегося в окрестностях первого вызова функции WinAPI.

Однако как отличить вызов нужной нам функции из только что распакованной программы от всех прочих вызовов этой же функции, которые могут выполняться в процессе работы распаковщика? Как раз для этого Вы можете воспользоваться проверкой значения, находящегося по некому известному адресу внутри кода программы (обозначим этот адрес буквой A). Извлечь нужное значение дампером из запущенной программы обычно не составляет никакой сложности. Поскольку известные мне упаковщики распаковывают код в направлении от меньших адресов к большим (и с достаточной точностью можно предположить, что так работает большинство распаковщиков), лучше выбирать эталонное значение ближе к концу программы. После того как такое значение у Вас будет, нужно лишь поставить условную точку останова на нужную функцию, а в качестве дополнительного условия указать равенство значения по адресу Aэталонному значению, которое Вы извлекли из распакованной программы. В результате Ваш брейкпойнт сработает не раньше, чем будет распакована соответствующая часть программы, так что довольно велика вероятность того, что, продолжая отладку в пошаговом режиме, Вы вернетесь в код программы, а не в недра распаковщика.

Наблюдая за работой запакованных программ, Вы могли заметить, что они после распаковки всегда располагаются в памяти по одним и тем же адресам. Применив против такой программы какую-нибудь утилиту вроде ProcDump, Вы даже можете прочитать параметры секций программы, частности - начальный адрес и размер. После этого вполне естественным кажется вопрос: «а что, если проверять, принадлежит ли адрес возврата, лежащий на стеке, промежутку адресов, в котором расположен код программы?» И вопрос этот отнюдь не праздный. Дело в том, что нередки упаковщики и навесные защиты, в которых блок, выполняющий дешифровку и распаковку кода, располагается в области, не пересекающейся с той, в которой в итоге будет расположен исполняемый код программы. Поэтому, поставив брейкпойнт на функцию и в качестве условия указав что-нибудь вроде ([esp]>401000) && ([esp]<501000) Вы добьетесь того, чтобы Ваш брейкпойнт активизировался лишь в том случае, если при выходе из функции предполагается возврат в код отлаживаемой программы. Данный метод, также как и описанный выше, может использоваться для поиска OEPчерез обратную трассировку от первого вызова функции WinAPI. Однако этим возможности условных точек останова, проверяющих адрес возврата, отнюдь не ограничиваются.

Очень часто возникает необходимость посмотреть параметры вызова некой часто используемой функции в тех случаях, когда вызов этой функции был сделан из одной или нескольких точек, и при этом проигнорировать все прочие вызовы (которых может быть очень много). Если решать задачу «в лоб», то необходимо найти все ссылки на данную функцию и установить по брейкпойнту на каждый интересующий нас вызов. Обычно такой подход вполне удовлетворителен, но мы не будем искать легких путей и посмотрим, как ту же задачу можно решить при помощи одного единственного брейкпойнта. Вы уже знаете, что для распознавания, откуда был сделан вызов функции, можно использовать адрес возврата, лежащий на вершине стека, и потому без труда напишете соответствующее условие для брейкпойнта. Такое условие может выглядеть следующим образом:

```
([esp]==ret_addr1) || ([esp]==ret_addr2) || …,
```

гдеret_addr1 иret_addr2 – адресавозврата. Все это достаточно очевидно, и Вы можете задать вопрос, зачем нужно было изобретать очередной велосипед, если традиционный подход дает ничуть не худшие результаты? Первая причина - «человеческий фактор»: работать с большим количеством точек останова не всегда удобно даже в насквозь визуальном OllyDebug, а уж «рулить» десятком-другим брейкпойнтов в SoftIce– занятие, что называется, на любителя. Так что если есть возможность значительно уменьшить число брейкпойнтов и облегчить себе жизнь, почему бы этой возможностью не воспользоваться? Кроме того, перед Вами может встать задача, обратная по отношению к вышеприведенной: отслеживать все вызовы процедуры, за исключением нескольких. И в этом случае сформировать строку с условием вида

```
([esp]!=ret_addr1) && ([esp]!=ret_addr2) && …,
```

в которой ret_addr1, ret_addr2 – адреса возврата, при которых точка останова не должна срабатывать, представляется гораздо более простым, чем искать в коде программы все подозрительные ссылки на интересующую Вас процедуру и «обвешивать» их брейкпойнтами.

Однако более важным представляется применение условных брейкпойнтов для обнаружения неочевидных вызовов функций. Я уже рассказывал в предыдущей главе о приемах, при помощи которых разработчик защиты может вызывать процедуры неявным образом, из-за чего такие обращения к процедурам становятся «невидимы» для дизассемблеров и прочих инструментов анализа «мертвого кода». В качестве дополнительного средства маскировки авторами защит могут параллельно использоваться как обычные вызовы процедур, так и неочевидные; я встречал такое в приложении к MessageBoxAи к функции чтения из реестра, но вообще эта техника может быть применена к любой достаточно часто вызываемой процедуре, используемой в защитных механизмах. В результате в дизассемблированном коде мы увидим несколько ничем не примечательных явных вызовов – но, скорее всего, не заметим самого интересного. Поставив

«обычный» брейкпойнт на вызываемую функцию, мы можем столкнуться с тем, что эта функция вызывается десятки раз, и потому тоже не сможем определить, задействована ли эта функция в защитном механизме – интересующие нас неявные вызовы потеряются среди десятков и сотен вызовов явных. Вот если бы был какой-нибудь способ отделить явные вызовы от неявных…

И такой способ есть. Базовые идеи, используемые для различения явных и неявных вызовов, могут быть сформулированы следующим образом:

1. Практически все явные вызовы функций легко обнаруживаются дизассемблером или вспомогательными инструментами для поиска ссылок (к примеру, в OllyDebugэта операция элементарно выполняется из контекстного меню: Findreferencesto|selectedcommand). Построив список явных вызовов, можно записать условие-фильтр, в котором будут перечислены все адреса возврата после явных вызовов, и использовать этот фильтр в качестве условия срабатывания для точки останова. Проще говоря, нам нужно будет установить условный брейкпойнт с фильтром вида ([esp]!=ret_addr1) && ([esp]!=ret_addr2) && …, который бы «пропускал» явные вызовы, но срабатывал на всех остальных, то есть неявных.

2. Набор «штатных» команд ассемблера, предназначенных для вызова подпрограмм, сравнительно невелик, поэтому, проанализировав несколько байт, предшествующих адресу возврата, можно строить предположения о способе вызова. В частности, если опкод, находящийся на пять байт «выше» адреса возврата, равен 0E8h (CALLxxxx), скорее всего вызов был сделан стандартным способом. Если же в окрестностях адреса возврата ни одна из разновидностей команды CALL не обнаружена – либо вызов был неявным, либо внутри отработавшей процедуры присутствовал код, искажающий адрес возврата.

3. Код, предшествующий вызову подпрограммы, может оставлять легко идентифицируемые следы: определенные состояния флагов, значения переменных или соотношения между регистрами. В некоторых случаях, проанализировав эти следы, можно делать выводы, из какой точки могла (или не могла) быть вызвана та или иная подпрограмма.

Поясню эту мысль на следующем коде:

```
cmp eax, ebx
ja _no_call
call MyProc

_no_call:
…
```

Нетрудно заметить, что если вызов процедуры MyProcбыл сделан из вышеприведенного куска кода, то в момент входа в процедуру значение регистра eaxдолжно быть меньше либо равно ebx (в противном случае выполнилась бы команда ja, обходящая call). И если это соотношение между регистрами в каких-то случаях окажется недействительным, для крэкера будет совершенно очевидно, что в этих «подозрительных» случаях вызов подпрограммы был произведен откуда угодно, но только не из продемонстрированного кода.

Разумеется, при большом желании, возможно построить код программы таким образом, что приложение вышеприведенные идей не даст положительного результата. Однако это потребует от программиста дополнительных и довольно серьезных усилий – разработчику придется проанализировать собственный код так, как это сделал бы крэкер (и отнюдь не факт, что это у него получится – принципиально разные цели порождают разный образ мышления). Затем все найденные уязвимые участки придется переписать, причем наверняка - на ассемблере, поскольку высокоуровневые языки обычно не дают необходимой гибкости. После всего этого все переписанные связки между процедурами придется заново отладить протестировать, чтобы убедиться в полной корректности работы исправленного кода. В общем, объем работ нешуточный, а результат… Результат, как водится, заведомо неизвестен – никто не может гарантировать, что разработчик не упустит какой-либо важный момент, или крэкер не найдет особо нетрадиционный

подход, после которого все усилия автора защиты пойдут прахом. Так что обычно разработчики обычно не озабочиваются сокрытием «тонких эффектов» при неявных вызовах, и Вы, при случае, можете обратить это себе на пользу.

Увы, в настоящее время применить вторую и третью идею в полном объеме, мы обнаружим ограниченность имеющихся программных средств. Да, наши любимые отладчики отлично справляются с относительно простыми фильтрами, однако более глубокий анализ ситуации в момент срабатывания брейкпойнта, невозможен либо, по меньшей мере, крайне затруднителен. Действительно, попробуйте написать условие, которое проверяло бы наличие всех возможных разновидностей команды CALLв районе адреса возврата – и Вы поймете, что я имею в виду. А ведь иногда возникает необходимость не только (и не столько) остановить программу при тех или иных значениях регистров, но в автоматическом режиме собрать статистику по срабатыванию брейкпойнта – частота появления тех или иных адресов возврата, типичные значения параметров функции, на которую поставлен брейкпойнт и тому подобное. И тут становится очевидным, что даже «статистические» команды SoftIceи модификатор DOв командах установки брейкпойнтов являются лишь слабым подобием того, в чем рано или поздно возникает потребность у каждого крэкера. Идеальным решением была бы встраивание в отладчики собственного скриптового языка, обеспечивающего полный доступ ко всем возможностям отладчика (что уже частично реализовано в плагинах для OllyDebugи различных «сторонних утилитах» для SoftIce). Если же существующие реализации средств скриптинга не предоставляют необходимых возможностей, мы будем вынуждены обходиться программными «затычками», реализация которых аналогична устройству описанных в предыдущей главе точек останова в Spectrum’овских отладчиках. Поскольку применяются такие «брейкпойнты» (а по сути - патчи) довольно широко, а с необходимостью «перехватить» исполнение программы в нужной точке рано или поздно сталкивается любой крэкер, мы подробно рассмотрим эту технологию в главе, посвященной патчингу.

До настоящего момента мы как-то обходили вниманием точки останова, срабатывающие при попытке доступа к определенным областям памяти вообще, и аппаратные брейкпойнты в частности. Вы, возможно, даже начали беспокоиться из-за того, что, говоря о точках останова, я так долго не упоминал волшебное слово «BPM». И вот пришло время поближе узнать, что такие брейкпойнты собой представляют и какую практическую пользу из них можно извлечь. «Законный» способ установки брейкпойнтов на области памяти основывается на использовании специальных отладочных регистров, обозначаемых как DR0-DR7. Каждый из брейкпойнтов может отслеживать любой (но только один) из следующих типов обращения к памяти: запись, чтение, запись или чтение, исполнение кода. Операции «чтение» и «исполнение» процессор считает принципиально разными, несмотря на то, что здравый смысл говорит нам: прежде чем исполнить код, нужно его прочитать. «Это невозможно понять, это нужно запомнить» - поэтому временно отложите здравый смысл в сторонку и запомните это правило. По этой же причине одновременно отслеживать запись, чтение и исполнение при помощи одного-единственного брейкпойнта у Вас не получится. Это первое существенное ограничение, наложенное инженерами из Intelна использование отладочных регистров.

Впервые отладочные регистры появились в процессорах 80386 именно для отслеживания обращений к памяти, но в Pentiumвозможности этих регистров были распространены и на порты ввода-вывода. Поскольку собственно адреса точек останова (или номера портов, обращение к которым будет отслеживаться) задаются в регистрах DR0-DR3, таких брейкпойнтов может быть не более четырех – это второе ограничение. Еще одна проблема состоит в том, что отладочные регистры позволяют установить брейкпойнт только на байты, слова (WORD) или двойные слова (DWORD), аппаратных брейкпойнтов на обращения к более крупным блокам памяти не предусмотрено. Если Вам нужно отследить обращения к переменным «длинных» нецелочисленных типов (Double, Extended), Вы моежете поставить брейкпойнт в середину переменной; в этом случае любое обращение к такой переменной «зацепит» брейкпойнт. Также важно помнить следующее:

если Вы устанавливаете бряк на слово, адрес брейкпойнта будет автоматически выровнен на ближайший «снизу» четный адрес, а если Вам нужен бряк на DWORD – приготовьтесь к тому, что процессор выровняет адрес брейкпойнта на адрес, кратный четырем.

И, наконец, самый неприятный для крэкера факт – работа с отладочными регистрами возможна только из нулевого кольца защиты, так что любителям запустить свои шаловливые ручки в недра системы и сотворить там что-нибудь эдакое придется сначала повозиться с написанием соответствующего софта. Впрочем, пусть Вас согревает тот факт, что разработчикам защит добраться до отладочных регистров и испортить Вам удовольствие будет ничуть не проще. Если у Вас возник живой интерес к теме внутреннего устройства аппаратных точек останова, могу порекомендовать обратиться к первоисточникам, то бишь к фирменной документации Intel.

Пожалуй, пора заканчивать с живописаниями грядущих трудностей и прочими ужасами, а то, добравшись до последней главы этой работы, Вы можете решить, что крэкинг – это очень сложно, больно и трудно (хотя в действительности это не совсем так). Поэтому давайте лучше устроим сеанс позитивного мышления и погрузимся в созерцание всего того доброго, светлого и прекрасного, которое привносят в нашу жизнь аппаратные брейкпойнты. Ибо, воистину, тяжела была бы наша жизнь, не будь в ней аппаратных точек останова.

Нетрудно догадаться, что изначально аппаратные брейкпойнты предполагалось применять для поиска ошибок, связанных с некорректной работой с переменными, и отлавливать ситуации, чреватые переполнением буфера (для этого сразу после последнего байта буфера ставился «защитный» брейкпойнт, срабатывание которого сигнализировало о попытке записать или прочитать лишние данные). Однако крэкеры даже столь милым и безобидным штукам, как аппаратные точки останова, нашли нетрадиционное применение, превратив их в главную «ударную силу» против самомодифицирующегося и упакованного кода.

Если Вы пробовали поставить брейкпойнт на упакованный код, Вы не могли не заметить, что после этого действа программа отчего-то перестает корректно распаковываться (впрочем, с вероятностью приблизительно 1/256, программа распакуется даже после такого издевательства, но брейкпойнт, разумеется, работать не будет). Прочитав предыдущую главу, Вы наверняка осознали всю глубину и тяжесть Вашей ошибки, чистосердечно раскаялись в этом ужасном деянии и, положив руку на руководство по отладчику, трижды произнесли торжественное обещание никогда больше так не поступать. А потом, подобно классику, задались вопросом: «что делать?» Разыскивая ответ на этот глубоко философский вопрос, Вы могли обнаружить в руководстве по SoftIceраздел, повествующий о команде BPMи ее параметрах, либо добраться до таинственных пунктов Breakpoint|Hardware, on… в контекстном меню OllyDebug. Если Вы еще не проделали этих операций – прочитайте документацию по отладчику, посмотрите, как правильно ставить аппаратные точки останова на чтение, запись и исполнение и немного потренируйтесь на первой попавшейся программе, дабы убедиться, что такие точки останова действительно существуют и даже работают. А потом освежите в памяти эксперимент, в котором мы при помощи дампера наблюдали изменения в коде, вызываемые командой BPX, и попытайтесь повторить его над к аппаратными точками останова. Как и следовало ожидать, аппаратные брейкпойнты не оставляют в коде программы никаких следов. Их не видно – но наши маленькие аппаратные друзья существуют и работают! И как бы программа ни утюжила свой код проверками, сколько бы ни высчитывала контрольные суммы – против аппаратных точек останова эти приемы бесполезны, так что теперь Вы сможете сколько угодно отлаживать программу, не опасаясь, что защитные процедуры в один миг изничтожат любовно расставленные Вами бряки.

Интересно, что аппаратные брейкпойнты отлично работают не только с «обычными» участками памяти, но и со стеком. В принципе, ничего удивительного в этом нет – память, зарезервированная под стек, с точки зрения процессора и ОС ничем принципиально не отличается от памяти, предназначенной для кода или данных. Однако для некоторых начинающих крэкеров, разум которых опутан предрассудками, сама идея установки аппаратного брейкпойнта на чтение/запись

данных в стек выглядит чем-то странным и противоестественным. А ведь установка аппаратного брейкпойнта на стек – это отнюдь не оригинальничание, а вполне действенная техника, часто используемая при «ручной» распаковке сжатых программ, а также позволяющая обнаруживать подмену адресов возврата в стеке.

Знания об аппаратных брейкпойнтах позволяют нам по-новому взглянуть на проблему подмены адресов возврата. Вспомните последний пример из предыдущей главы – тот, где функция A вызывает функцию B, функция Ввызывает функцию C, а функция Сделает «финт ушами», подменяя адрес возврата, и возвращается в функцию D (а не в функцию B). Теперь, когда Вы вооружены необходимой информацией о том, как отслеживать обращения к определенным адресам памяти, для Вас не составит труда обнаружить попытки процедуры подправить свой адрес возврата – просто «накройте» брейкпойнтом на чтение/запись двойное слово, хранящее адрес возврата, и Вы без проблем найдете команду, которая выполняет подмену.

Аппаратные точки останова внутри стека могут помочь Вам победить еще один довольно неприятный защитный прием – переход со «сбросом» части стека. В вышеприведенном примере демонстрировалось искажение адреса возврата, лежащего на стеке, однако автор защиты может и не возиться с искажением адресов, а волевым решением «выбросить» со стека часть параметров, переменных и адресов возврата (это может быть сделано, к примеру, командой ADDESP,xxxx или несколькими PUSH'ами), после чего переход в нужную точку кода выполнить простым JMP. Поразмыслив над содержимым стека, Вы даже можете приблизительно определить границы стековых фреймов. Если предположить, что стековые фреймы «сбрасываются» целиком (в принципе, это не обязательно, но такой код проще в отладке), то Вы можете составить список возможных значений регистра ESPпосле «сброса» фреймов. Затем методом «научного тыка» протестируйте каждое из этих значений, устанавливая аппаратные брейкпойнты на чтение/запись двойного слова из каждого из этих значений ESP, а также на 4 байта ниже; можно также добавить брейкпойнты на предполагаемые адреса локальных переменных. В результате Вы получите одну из трех ситуаций:

1. Если после «сброса» стека программа попытается выполнить возврат из процедуры, она «споткнется» о брейкпойнт, стоящий по адресу ESP.
2. Если программа попытается вызвать подпрограмму, и при этом поместит какое-либо значение (параметр или адрес возврата) на стек, команда, выполняющая запись данных в стек, вызовет срабатывание брейкпойнта по адресу ESP-4.
3. Если брейкпойнт установлен на локальную переменную, исполнение программы будет прервано при первом же обращении к этой переменной.

Смысл всех этих действия заключается в том, чтобы как можно раньше остановить программу после «сброса» стека и узнать, что пытался скрыть автор защиты при помощи этого приема. Поскольку современные программы стеком пользуются достаточно активно, можно надеяться, что первое срабатывание нашей «ловушки» будет не слишком далеко отстоять от той точки, в которую был выполнен переход. Если выяснится, что адресов, на которых срабатывают ловушки, несколько, Вам нужно выбрать тот из них, обращение к которому выполняется раньше других – очевидно, что он наиболее близок к искомой точке.

Описанный метод также традиционно используется для определения OEPупакованных программ: заклинание «BPMESP-4», которое следует набирать в SoftIcesразу после загрузки распаковываемой программы, так или иначе упоминается в большинстве статей, посвященных ручной распаковке кода. А вот смысл сего заклинания, увы, поясняется намного реже, и сейчас я попытаюсь исправить это недоразумение. Реализация абсолютного большинства навесных защит такова, что значение ESPв момент завершения работы защитного модуля и передачи управления на OEPв точности равно значению ESPсразу после загрузки запакованной программы. Причины этого лежат где-то в глубинах сознания разработчиков защит, поскольку уменьшение значения ESPперед исполнением программы на величину, кратную четырем, никаких отрицательных

эффектов не вызывает, поскольку место под стек обычно резервируется с запасом (а вот увеличение значения ESPуже может быть чревато). Однако большинство упаковщиков считают, что значение ESPлучше передавать пользовательскому коду в неизменном виде, а потому свято блюдут принцип «сколько на стек положено – столько должно быть снято», причем окончательная коррекция стека скорее всего будет проведена непосредственно перед выполнением перехода на OEP. И если эта коррекция выполняется командами чтения данных из стека (POP, POPADи т.п.), а не простой записью в ESPранее сохраненного значения, Ваш брейкпойнт сработает в этот знаменательный момент. Таким образом, Вам останется лишь отсеять ложные срабатывания брейкпойнта, если таковые будут, и трассировать код до тех пор, пока управление не будет передано из распаковщика в основную программу.

Мой рассказ об аппаратных точках останова был бы неполон, если бы я не упомянул об одной специфической разновидности брейкпойнтов останавливающих исполнение программы при обращении к произвольной области памяти. Такие брейкпойнты в SoftIceпод ОС Windows 9xсоздавались при помощи команды BPR(в современных версиях этого отладчика команда BPR, к сожалению, отсутствует, что особенно странно в свете того, что в арсенале OllyDebugтакие точки останова имеются). Внешне использование таких брейкпойнтов ничем не отличается от работы с обычными аппаратными точками останова, если не считать возможности «накрыть» брейкпойнтами практически неограниченное количество участков памяти совершенно любого размера (что выгодно отличает данный тип брейкпойнтов от «обычных» аппаратных, которых может быть не более четырех). Гораздо больший интерес представляет знание о том, каким образом реализован этот тип брейкпойнтов, которое, возможно, пригодится Вам в будущем (к примеру, если Вы захотите написать собственный отладчик).

Как Вы знаете, процессоры линейки x86 в защищенном режиме (некоторые источники называют этот режим расширенным) содержат множество средств, облегчающих создание многозадачных программ и позволяющих защитить данные от некорректных операций над ними. Для нас особенно интересной представляется возможность изменять атрибуты защиты отдельных страниц памяти, то есть разрешать или запрещать определенный тип действий (запись, чтение, исполнение кода) над информацией, хранящейся на той или иной странице памяти. При этом любая попытка выполнить запрещенную операцию, к примеру, записать данные на страницу, для которой запись запрещена, вызовет исключительную ситуацию. Более того, чтобы ради этого нехитрого действа Вам не пришлось выбираться в нулевое кольцо защиты, в WindowsAPIвключены функции VirtualProtectи VirtualProtectEx, позволяющие изменять атрибуты страниц памяти (правда, флаг запрета на исполнение кода на платформе x86 бесполезен, но необходимости в таком запрете обычно и не возникает). Как видите, создать «область останова» не так уж трудно, основную сложность в реализации таких «областей останова» представляет обработка исключительных ситуаций, возникающих при обращении к данным из защищенной области. Основным препятствием в практической реализации является то, что обычно размеры «области останова» не выровнены на границы страниц и кратны размерам страницы. По этой причине возникает необходимость в написании достаточно изощренного кода, распознающего, к какой ячейке памяти произошло обращение и обеспечивающего корректное продолжение работы программы после ошибки нарушения прав доступа к странице (по сути требуется написать нечто среднее между простым дизассемблером и виртуальной машиной).

Уместно будет упомянуть, что некоторые защиты манипулируют атрибутами страниц, чтобы противодействовать отладке, memorypatching'у и снятию дампов. Большинству патчеров и дамперов эти ухищрения глубоко безразличны, однако если Вы планируете заняться написанием собственных утилит для патчинга и/или снятия дампов (что я Вам настоятельно рекомендую), обязательно проверьте, как Ваши изделия будут реагировать на нестандартные атрибуты страниц. Поэтому если у Вас возникнут какие-либо проблемы при операциях с памятью чужого процесса, есть смысл поинтересоваться атрибутами страниц, с которыми Вы работаете (это можно сделать

при помощи той же VirtualAllocEx).

На этом наше знакомство с теорией и практическими аспектами применения точек основа в основном закончено, и теперь мы можем во всеоружии приступить к рассмотрению методов трассировки кода. Именно этому и будет посвящена следующая глава.

Написание плагинов для API шпиона kerberos

Автор: Rustem

Дата: 2005-03-18

> *Сам себя непохвалишь-никто непохвалит*

В данной статье я хочу показать, как пишутся плагины для API-шпиона kerberos 1.01

Самого шпиона можно взять на WWW.WASM.RU (раздел Утилиты)

Без поддержки плагинов - создание нового API-шпиона имело бы мало смысла, т.к. можно было бы использовать любой другой. С другой стороны свои плагины дают возможность безграничного контроля над функциями используемыми приложением и самим приложением тоже.

Общие аспекты:

1. Одновременно может работать только один плагин
2. Файл плагина всегда называется **ke_plug.dll** 3. Плагин считается установленным, если лежит в директории с **ke_core.dll** Шпион написан исключительно на ассемблере (masm32), как на самом лучшем языке программирования, и плагин тоже пишется на нем. В каждом плагине должна присутствовать функция инициализации **plug_setup**. Она возвращает в eax значение 0, если рапорт шпиона необходим, иначе рапорт создан не будет. Это нужно если вы собираетесь использовать kerberos как загрузчик программ.

Также в каждом плагине должна быть функция **plugin**- это основной код плагина.Функции **plugin** и **plugin_setup** должны являться экспортируемыми, иначе работать не будет.(Самое лучшее воспользоваться готовым примером из папки ke_plug)

Давайте, наконец, рассмотрим шаблон, который подойдет для всех плагинов.

```
ke_plug.asm
.........................
.........................
;Возвращаем значение рапорт нужен\ненужен
plug_setup proc
mov eax, REPORT_OFF
ret
plug_setup endp
; Вызывается каждый раз, при перехвате любой функции, которая есть в базе
plugin proc
pop ret_address ;Снимаем со стека адрес возврата
pop func_adr ;Адрес начала перехваченной функции
pop func_par ;Количество параметров функции
pop func_nam ;Указатель на имя функции
pop func_return_address ;Адрес возврата в вызывающую программу
pop func_idx ;Индекс функции в базе
pushfd
pushad
mov eax, func_adr
;Ваш код должен быть здесь
.if eax == FUNC_0
;Выполняем действия, которые хотим
;.......................................
;Восстанавливаем регистры и вызываем функцию
;Можно не вызывать, но надо будет сбалансировать стек от параметров функции
popad
popfd
call func_adr
;Если у нас другая функция, для которой установлен плагин
.elseif eax == FUNC_1
.....................
;Никакая функция не подошла- просто вызываем перехваченную функцию
```

```
.else
popad
popfd
call func_adr
.endif
;Возвращаемся туда, откуда пришли
push ret_address
ret
plugin endp
```

Ничего сложного. Надеюсь, что непонятные стороны прояснятся в разборе конкретного примера.

Рассмотрю, как можно сделать граббер музыки с записью музыки в wav файл. (Я тестировал его на демке UberNuss. Если будете пробовать на ней незабудьте вытащить из нее Win32 exe. В общем она 4кб, и сделана как com - файл, который распаковывается и запускает сам PE файл)

Программа из которой будет вытаскиваться музыка, должна пользоваться для вывода звука следующими функциями из **winmm.dl**: **waveOutOpen**, **waveOutWrite** Первым делом модифицируем файл **ke_spy.txt**, чтобы в нем были эти самые функции, иначе получится вот что - нет перехвата этих функций, и нет вызова нашего кода из плагина. Если нужны только две функции, то нет необходимости перехватывать остальные - так надежнее. Сам **ke_spy.txt** для нашего плагина будет выглядеть так:

```
[winmm.dll]
waveOutOpen, 6
waveOutWrite, 3
```

Дальше сам плагин. Приведу только ключевую часть, полностью смотрите в исходниках.

Главное, что делает плагин:

1. Открывает файл для вывода туда данных
2. Пишет первую часть заголовка (по шаблону)
3. Записывает вторую часть заголовка (берется из параметров функции *waveOutOpen*
4. Дописывает заголовок wav файла
5. Выводит буфер, переданный в функцию *waveOutWrite*в выходной файл.
6. После окончания закрывает выходной файл

Если интересно, описания функции есть в MSDN, сам формат wav- файла есть в сети.

Остается заметить, что в нем незаполнены два поля (DWORD)- по смещению 04h и 024h должны быть равны Размер Файла -8 и Размер Файла - 44 соответственно, хотя можно послушать и так. (Пробовал в WinAmp5.0)

```
;Имя выходного файла и заголовок wav - файла
ofile db "m.wav",0
wavhead_0 db "RIFF"
db 0,0,0,0
db "WAVE"
db "fmt "
dd 10h
wavhead_1 db "data"
db 0,0,0,0
install_plugin proc
invoke LoadLibrary,offset sz_winmm
mov hwinmm, eax
invoke GetProcAddress, eax, offset sz_waveOutOpen
mov waveOutOpen_addr, eax
invoke GetProcAddress, hwinmm,offset sz_waveOutWrite
mov waveOutWrite_addr, eax
invokeCreateFile,offset ofile,GENERIC_WRITE,NULL,NULL,CREATE_ALWAYS,FILE_ATTRIBUTE_NORMAL,NULL
mov hwavfile, eax
ret
install_plugin endp

plugin proc
```

```
pop ret_address
pop func_adr
pop func_par
pop func_nam
pop func_return_address
pop func_idx
pushfd
pushad
mov eax, func_adr
;Если адрес перехваченной функции совпал, то действуем
.if eax == waveOutOpen_addr
;Пишем в файл заголовок wav - файла
invoke WriteFile,hwavfile,offset wavhead_0,20,ADDR bwri,NULL
;Пишем в файл структуру, переданную функции waveOutOpen(должна быть в заголовке)
mov eax, [esp+2Ch] ;LPWAVEFORMATEX
invoke WriteFile,hwavfile, eax,10h,ADDR bwri,NULL
;Дописываем в файл заголовок
invoke WriteFile,hwavfile, offset wavhead_1,8,ADDR bwri,NULL popad
popfd
call func_adr
;Если мы перехватили waveOutWrite, то выводим её звукой буфер в наш файл, а уже потом играем.
.elseif eax == waveOutWrite_addr
mov eax, [esp+28h] ;LPWAVEHDR
mov ecx, [eax] ;lpData
mov edx, [eax+4] ;dwBuffer
invoke WriteFile,hwavfile, ecx, edx,ADDR bwri,NULL
popad
popfd
call func_adr; Вызвали waveOutWrite
.else
popad
popfd
call func_adr; Ни одна не совпала- прсто вызываем её
.endif
push ret_address; Возвращаемся
ret
plugin endp
```

Напоследок хочу сказать, что при помощи плагинов можно сделать кое-что более интересное, например цеплять шпиона к создающимся процессам. (Нужен перехват CreateProcess и написать для нее обработчик, который внедрит **ke_core.dll** при помощи CreateRemoteThread)

Теоретические основы крэкинга: Глава 11. Трассировка во сне и наяву.

Автор: CyberManiac

Дата: 2005-04-13

Трассировка - одна из основ, на которых держится крэкинг. О трассировке обычно говорят вскользь как о чем-то общеизвестном и само собой разумеющемся, но при этом имеющем нечеткие, плохо формализуемые правила. Читая предыдущие главы, Вы неоднократно встречали фразы "трассируем процедуру…", "трассируйте код до тех пор, пока…" и т.п., и при этом вряд ли задумывались о том, что нужно делать и как вообще трассируют код. Так что же такое отладка - наука это или искусство? Поставив себе целью разобраться в методах трассировки, мы прежде всего должны определиться, что такое трассировка и для чего она нужна. И лишь после того, как будут определены цели трассировки, возможно будет говорить о "технической" реализации методов, посредством которых эти цели могут быть достигнуты.

Прежде всего договоримся о терминологии. Само слово "трассировка" часто употребляется в различных смыслах - от синонима "отладки" до обозначения процесса пошагового исполнения программы (см. команды вроде "trace into" и "trace over" в некоторых отладчиках). Я не буду оригинален и введу еще одно значение слова "трассировка", которое и будет использоваться на протяжении данной главы: трассировка - это процесс определения "траектории" исполнения кода с некой заранее заданной точностью и наблюдение за изменениями во время исполнения этого кода. Вот такое определение - незамысловатое, но с замахом на глобальность. И, разумеется, без подробной расшифровки выглядящее достаточно туманно - как и любое другое достаточно широкое определение. Сам процесс трассировки может быть направлен на получение следующих знаний:

• Определение траекторий исполнения кода, как всех теоретически возможных, так и активирующихся только при неких начальных условиях.
• Наблюдение за изменениями каких-либо параметров (значений регистров, переменных, флагов) и определение их влияния на порядок исполнения команд.
• Нахождение точек ветвления и условий, приводящих к активации или деактивации того или иного участка траектории исполнения кода.

С пониманием того, что подразумевается под "заранее заданной точностью", я думаю, никаких сложностей у Вас не возникнет - эти слова означают всего лишь то, что при трассировке код анализируется исключительно лишь в той мере, которая нужна для решения практических задач. Если Вам необходимо полное понимание некоего алгоритма (то есть "заранее заданная точность" - максимальна), Вам придется анализировать код целиком; если же Вам нужен лишь ответ на вопрос "почему программа выдает сообщение "Неверный серийный номер" вместо поздравления с регистрацией", Вас скорее всего устроит экспресс-анализ четырех-пяти точек ветвления, находящихся "выше" процедуры, сообщающей об ошибке, а все, что происходит между этими точками, Вы вольны проигнорировать. Трассировка может заключаться в поиске двух-трех точек в программе и анализе десятка прилежащих к ним команд. Как такое возможно? Очень просто: для примера в качестве критерия "заранее заданной точности" мы берем факт чтения или записи данных в некую ячейку памяти и находим в листинге все явные обращения к соответствующему адресу. После этого мы можем смело сказать "при некоторых неизвестных условиях в данную ячейку могут быть записаны значения, равные нулю либо единице". Если речь идет о глобальном "флаге зарегистрированности", то весь дальнейший взлом сводятся к выяснению, какое из состояний означает зарегистрированность программы (автор программы мог проявить оригинальность, сделав код 1 признаком отсутствия лицензии) и исправлению одного бита в

программе.

Иногда используется даже совсем уж вырожденный вариант трассировки: в коде программы выбирается некая контрольная точка, на нее ставится брейкпойнт, а затем при помощи тестового запуска определяется, проходит траектория исполнения программы через эту точку или нет. В основе всех приведенных примеров лежит трассировка, хотя степень детализации различается очень сильно. Каких-то готовых правил, позволяющих выбрать глубину "погружения в код", по-видимому, не существует - действовать приходится по ситуации. Однако на первых порах лучше применять "экспресс-анализ" и использовать трассировку не столько для исследования тонкостей работы кода, сколько для поиска логических блоков, поиска управляющих конструкций и понимания алгоритма в общих чертах. Именно такой подход лежит в основе дзен-крэкинга с его принципом "я не знаю, как это работает, но я могу это сломать", а если учесть, что таким "наскоком" вполне успешно ломаются недорогие утилиты, то этот метод, несомненно, будет серьезным подспорьем для начинающего крэкера.

Теперь рассмотрим, что представляет собой траектория исполнения кода. По традиции, разбирать это понятие мы будем на конкретном примере, а именно на куске кода, генерирующем таблицу констант для алгоритма CRC32:

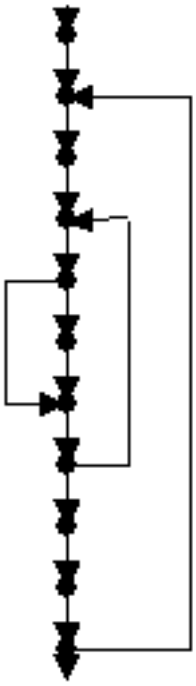
```
mov eax,255

_loop_i:
mov edx,eax
mov cl,8

_loop_j:
 shr edx,1
 jnc @@1
 xor edx,0EDB88320h
@@1:
 dec cl
 jnz _loop_j

mov [CRC32_Table+eax*4],edx
dec eax
jns _loop_i
```

Распечатайте этот код на листе бумаги (в принципе, можно выполнять все операции мысленно, но это будет не так удобно, как работать с "твердой копией"). Возьмите карандаш и напротив каждой из команд нарисуйте кружок - это будут точки возможной траектории исполнения кода. Теперь по полученным точкам попробуем построить совокупность всех возможных траекторий исполнения кода. Как известно, большинство процессоров исполняет команды последовательно (или же успешно изображают последовательное исполнение), одну за другой, по направлению от "нижних" адресов памяти к "верхним". И такой порядок исполнения может быть нарушен лишь командами переходов/возвратов, вызова прерываний, а также различными исключительными ситуациями. Вооружившись этим знанием, предположим, что исполнение данного куска кода начинается с команды mov eax,255 и начнем соединять наши кружочки стрелками в том порядке, в каком будут исполняться команды. Когда Вы доберетесь до команды jnc @@1, у Вас наверняка возникнет вопрос, куда нарисовать стрелку - ведь для флага SF может находиться в одном из двух возможных состояний, и в зависимости от этого состояния следующей исполняемой командой будет либо xor edx,0EDB88320h, либо dec cl. Выход из этой ситуации прост - рисуйте обе стрелки. После того, как Вы закончите это упражнение, должен получиться приблизительно такой рисунок:



Для удобства понимания переходы "вперед" по коду (такой переход обнаружился только один) я вынес влево, а переходы "назад" - вправо. Что мы видим? Прежде всего - то, что в каждый кружок входит как минимум одна стрелочка, а это означает, что каждая из нарисованных нами команд получает управление явным образом. Если бы после завершения нашего высокохудожественного шедевра обнаружилось, что какой-либо кружок "висит в воздухе", это было бы веским основанием для проведения подробного расследования на тему "зачем нужен программный блок, который не получает управления явным образом" (техническую сторону таких расследований мы рассматривали в предыдущих двух главах). Далее: окинув беглым взглядом картинку, Вы легко заметите два цикла, один из которых вложен в другой и одну конструкцию ветвления, "обходящую" команду xor edx,0EDB88320h при выполнении некоего условия (в языках высокого уровня такая последовательность обычно реализуется конструкциями вида IF <условие> THEN <действия>). Вот так при помощи карандаша и бумаги можно за считанные минуты выделить логические единицы внутри довольно абстрактной процедуры. Несмотря на то, что метод кажется очень простым и даже в чем-то "игрушечным", в действительности такие схемы с кружочками-стрелочками очень удобны, особенно если Вы сравнительно недавно занялись исследованием программ, и Ваш глаз еще не натренирован на вычленение управляющих конструкций в бесконечных листингах. А если Вам не хочется слишком уж часто прибегать к помощи карандаша и бумаги, есть смысл обзавестись дизассемблером IDA - последние версии этого продукта тоже умеют проставлять стрелочки напротив кода (правда, там это реализовано не так удобно, как на нашем рисунке). Или же можно написать собственную программу, которая бы на основе листинга рисовала такие вот картинки; если не пытаться сразу создать дизассемблер (и не просто дизассемблер, а как минимум аналог W32Dasm), а анализировать уже готовые листинги, программа даже получится не слишком сложной. Надо сказать, что инструменты, способные на основе ассемблерного листинга построить картинку, подобную приведенной выше, в настоящее время очень редки и потому более чем востребованы общественностью. Кстати, наш набор стрелочек на самом деле - отнюдь не изобретение "для личного пользования", а наглядное изображение весьма научной штуковины под названием "граф" (имеется в виду математический термин, а не дворянский титул). Так что если Вам близок раздел математики под названием "теория графов", Вы можете попробовать приложить для анализа нашей картинки всю мощь этой теории; особенно актуально это для тех, кого заинтересует тема визуализации и автоматического анализа кода.

При желании можно пойти еще дальше: нарисованные кружки-команды разбить на логически связанные группы, обвести каждую группу прямоугольником или ромбиком, вписать внутрь этих фигур краткие комментарии и в итоге получить обыкновенную блок-схему (она же "flowchart" в англоязычной литературе), иллюстрирующую исследуемый код. С недавних пор IDA поддерживает и эту технику представления кода - но, как обычно, со своей "спецификой". Проще говоря, блок-схемы в IDA мало похожи на то, что обычно называется блок-схемами в учебниках информатики. Да и потому пользоваться создаваемыми в IDA графиками (которые на данный момент невозможно экспортировать ни в один формат) не так удобно, как "классическими" блок-схемами. Кроме того, на сегодняшний день блок, ответственный за отображение блок-схем в IDA, является сторонней разработкой и совершенно не интерактивен, т.е. возможности пользователя по активной работе с такими блок-схемами практически нулевые - даже чтобы написать комментарий к блоку, Вам придется распечатать схему на бумаге.

Оба этих метода, рисунок из кружочков со стрелочками и блок-схема, в действительности изображают одно и то же - команды и возможные порядки их исполнения, и являются "рабочим материалом" для самого древнего метода работы с листингом - трассировки в уме. Суть метода трассировки в уме очень проста: Вы на время превращаете свой мозг в некий "виртуальный процессор", и начинаете мысленно "исполнять" команды подобно тому, как это делал бы процессор настоящий. Надо отметить, что в старые времена, когда компьютеры были большими и медленными, а машинное время было ресурсом весьма ограниченным, именно трассировка в уме была основным способом "отладки" программ - и программисты просиживали часами над огромными листингами с карандашиками в руках, пытаясь определить, в какой точке программа уклонилась с пути истинного. Нечто подобное предстоит научиться делать и Вам - с той существенной разницей, что Вы в любой момент можете проверить свои теории при помощи отладчика. Разумеется, по сравнению с машиной Ваше "быстродействие" будет ничтожным, да и отслеживать состояние регистров и ячеек памяти у Вас вряд ли получится - но в этом и нет необходимости. Главной целью такого "мысленного исполнения кода" должно быть определение "ключевых точек", вычленение логических блоков внутри трассируемого кода, приблизительное определение назначения этих блоков и наблюдение за тем, как состояние регистров, флагов и переменных отражается на пути исполнения программы.

Как я уже говорил, обычно мы изучаем не весь код построчно, а лишь те его участки, которые могут привести к интересующим нас эффектам, причем задача чаще всего стоит следующим образом: по известному эффекту необходимо найти траекторию исполнения кода, которая приводит к появлению этого эффекта. Предположим, что Вы успешно обнаружили, где в программе расположена код реализации нужного эффекта (вывод MessageBox'а, запись единицы в регистр EAX и т.п.), и Вам хочется понять, каким образом программа передает управление на этот код и каким образом этого можно избежать или наоборот - получать такой результат при любых исходных данных. Для решения таких задач обычно используется обратная трассировка в уме. Идея обратной трассировки очень проста: мы начинаем читать листинг "задом наперед", то есть движемся от следствия (которое нам известно) к причине. По ходу дела отмечаем ключевые точки, к которым относятся:

• Вызовы функций Win32 API, а также других стандартных функций, какие сможет распознать Ваш дизассемблер или Вы сами.
• Обращения к глобальным переменным, которые чаще всего выглядят как чтение или запись данных по указанному явным образом адресу.
• Вызовы подпрограмм, непосредственно за которыми следует проверка некоего условия (внешне выглядят как связка команд CALL-CMP-Jxx).

Перечисленные три группы отличаются от всех прочих кодов тем, что их назначение сравнительно легко идентифицируется. Действительно, если Вы видите вызов API'шной функции чтения командной строки, для Вас будет очевидно, что следующие за вызовом команды почти наверняка

будут оперировать именно с текстом командной строки, а не с фазами Луны или курсом доллара. И вот тому пример:

```
call    GetCommandLineA
mov     edi, eax
cmp     byte ptr [edi], 22h
jnz     short loc_401B08
```

Нетрудно догадаться, что этот кусок кода проверяет, является ли первый символ командной строки двойной кавычкой. Далее, согласно принципам дзен-крэкинга, следует помедитировать о том, что может последовать за такой проверкой. А за ней обычно следует поиск закрывающей кавычки и дальнейший синтаксический разбор строки - пропуск разделяющих пробелов, вычленение нужного параметра из строки (обычно определяется смещение первого символа в строке и длина этого параметра) и т.п. Нечто подобное как раз и проделывал тот патч, из которого я взял код для примера. Для нас в данном случае интересен не подробный анализ кода, я хотел продемонстрировать несколько иное, а именно: как всего лишь один известный системный вызов позволяет приблизительно оценить назначение процедуры, в которой он содержится.

Причина интереса к глобальным переменным тоже достаточно очевидна. Современные программисты, как правило, придерживаются принципов структурного и объектного программирования, которые предполагают минимальное использование общедоступных объектов - каждая процедура должна "видеть" лишь те данные, которые ей нужны для работы. Поэтому разработчики программ обычно дают глобальный статус двум типам данных: тем, которые используются настолько широко, что их неудобно передавать в каждую процедуру, где они требуются (например: настройки программы, таблицы констант, тексты сообщений и т.п.), и всевозможным отладочным переменным, которые полагается удалять при выпуске релиза программы. Регистрационные данные нередко занимают промежуточное положение: с одной стороны, это своеобразная "настройка программы", а с другой они близки к отладочной информации в том смысле, что добавление защиты нередко производится уже после написания программы чисто механическим путем ("если переменная не равна нулю, то нарисовать поверх отчета слово UNREGISTERED").

Как Вы понимаете, использование глобальной переменной для хранения статуса программы в наше время - редкая и счастливая для крэкера случайность. Гораздо чаще разработчики, начитавшись руководств "как защитить свою программу от хакеров за 1 час", усваивают, что глобальную переменную использовать в качестве "переключателя" нехорошо. Вот функция, вызываемая к месту и не к месту - это совсем-совсем другое дело. Сказано-сделано, и в программе появляются многочисленные куски вида "if (!RegistaProggie()) ShowMessage ("Wanna getta munnee!")" (если кто не понял текст сообщения, приблизительный перевод с нетрадиционного английского звучит как: "хочу бабки!"). А во что такие куски превращаются при ассемблировании? Правильно - в цепочку CALL-CMP-Jxx, о которой я говорил парой абзацев выше. Вот такие-то интересные кусочки мы и будем высматривать при обратной трассировке. Разумеется, одним лишь поиском регистрационных процедур дело не ограничивается - цепочки "вызов-проверка-ветвление" могут быть проверкой на корректность введенных данных (как-то раз мне пришлось поправить программу игры "Жизнь", которая могла, но отчего-то не хотела работать с полями больше, чем 100*100), и конструкцией SWITCH (она же CASE в Паскале), да и много чем еще. Причем, если у функции есть параметры, они могут стать отличной подсказкой, позволяющей установить назначение этой функции (особенно хорошо этот прием работает с функциями, выполняющими преобразование строк). Для этого нужно под отладчиком исследовать, что именно передается в функцию и какой результат она возвращает. Нередко это даже оказывается проще, чем догадаться, что хранится в глобальной переменной.

Но вернемся к описанию алгоритма обратной трассировки в уме. Итак, мы встретили команду ветвления или перекрестную ссылку. Переходим к точке, куда/откуда ведет ссылка, а затем пытаемся определить, при каких условиях эта ссылка "срабатывает" и что происходит, когда

условие срабатывания ссылки не выполняется, причем для "не сработавшей" ветки исходник нужно читать уже не "снизу вверх", а в порядке исполнения команд. Следуя этой схеме, Вы, скорее всего, доберетесь до начала либо до конца процедуры, если не запутаетесь во всех этих ветвлениях и переходах. По сути, алгоритм обратной трассировки в уме рекурсивен (команды условного перехода или перекрестные ссылки часто порождает два возможных пути чтения кода), а человеческое сознание мало приспособлено к выполнению рекурсивных алгоритмов "вручную", так что Вам лучше сначала потренироваться на простых примерах.

Добравшись до начала процедуры либо до точки выхода из нее, Вам придется либо анализировать все возможные варианты, откуда и почему могла быть вызвана процедура (т.е. пробежаться по всем ссылкам на эту процедуру), либо прибегнуть к отладчику. Второй путь, конечно, нарушит "чистоту идеи" трассировки в уме, но зато даст ответ максимально простым и быстрым способом. Теоретически процесс обратной трассировки в уме Вы можете продолжать до тех пор, пока не доберетесь до начала программы, но на практике мне редко приходилось подниматься по "дереву процедур" более чем на четыре уровня вверх. Нельзя не отметить одну тонкость: даже если Вы нашли точку выхода из процедуры, убедитесь, что эта точка единственная: компиляторы могут генерировать код с более чем одним выходом из процедуры\функции, причем эти "дальние" ветки могут быть даже более интересными, чем "ближние". Дело в том, что любая проверка может быть многоступенчатой: к примеру, сначала проверяется, введены ли вообще какие-либо данные, затем - насколько эти данные корректны, и лишь в последнюю очередь - соответствуют ли эти данные какому-либо узкоспециализированному критерию.

Трассируя программы, Вам придется немало поводить пальцем по распечатке или экрану дисплея, взбираясь по дереву вызовов и переходов. А если к тому же Ваш дизассемблер "не умеет" выделять цветом команды переходов, Вы можете запросто пропустить что-нибудь важное. В общем, блуждание по бескрайним полям кода - дело, требующее внимания, сосредоточенности, и при этом достаточно утомительное. Однако если Вы решаете типовую задачу "как заставить программу выполнить некий код независимо от правильности исходных данных", Вам может помочь все та же карта всех возможных путей исполнения кода, построение которой я демонстрировал в самом начале статьи. Как только Вы составите такую карту, Вам останется лишь выполнить три несложных действия:

1. Выбрать в программе и отметить на этой карте исходную точку. В качестве исходной точки лучше всего выбирать код, имеющий непосредственное отношение к исследуемой защите, который может быть легко идентифицирован и вызван из программы. Если речь идет о старых добрых серийных номерах, вводимых с клавиатуры, то лучше всего искать код чтения серийника из окна - этот код обычно легко "ловится" в отладчике и заведомо вызывается защитой (надо же ей как-то узнавать, с какими параметрами пользователь пытается зарегистрироваться).
2. Отметить конечную точку, путь к которой требуется найти (например, вызов MessageBoxA, сообщающий об успешной регистрации).
3. Найти на карте путь (а лучше - все возможные пути) из начальной точки в конечную.

Как только такой путь будет найден, можете начинать соответствующим образом патчить код и выяснять, что из этого получится. Интересно, что выполнять второй пункт можно не только вручную (это чем-то похоже на детскую головоломку "найди путь в лабиринте"), но и в автоматизированном режиме - в этом случае потребуется решить типовую задачу из курса теории графов. Теоретически это позволяет поставить взлом защит, использующих только переключатель "зарегистрировано/не зарегистрировано" даже не на поток - на конвейер!

Даже если у Вас возникнут проблемы с вторым пунктом (например, программа никак не сообщает об успешной регистрации или же Вы просто не знаете, как это сообщение выглядит и потому не можете его найти), это не повод для отчаяния. Пусть Вы не знаете, на каком из возможных путей исполнения находится нужный Вам код - зато никто не отнимет у Вас знания о том, на каком пути этот код не находится! А значит, последовательно форсируя при помощи отладчика исполнение

каждой возможной ветки кода, Вы можете постепенно отбрасывать "неправильные" ветки, пока не наткнетесь на правильную (или не придете к выводу о невозможности решить Вашу задачу таким способом).

Попробовав трассировать в уме сколько-нибудь сложную процедуру, вырванную из середины программы, Вы заметите, что хотя возможные пути исполнения кода и известны, но сказать, какой путь изберет программа, если ее запустить, Вы не можете. И никто не может - поскольку при трассировке в уме Вам неизвестны исходные данные, которые и заставляют программу выбирать из всех возможных путей единственный актуальный. Именно здесь и пролегает граница между разглядыванием препарированного "мертвого" кода и вождения пальцем по распечатке и наблюдением за "живой" программой, обрабатывающей "настоящие" данные. Так что пришло время поговорить о методах трассировки программ "вживую".

Исторически первым способом трассировки "вживую" был многократный вызов в отладчике функции "исполнить текущую команду" с заходом в подпрограммы или без такового. Проще говоря, программист сидел перед монитором и давил нужную кнопку, наблюдая, как бегает по коду курсор, и меняются значения регистров. Более продвинутые (или просто более ленивые) программисты смекнули, что для определения траектории важны не все команды, а лишь те, в которых программа делает выбор, в какую сторону ей "свернуть". А потому для определения траектории нужно наставить брейкпойнтов на команды условных вызовов и переходов, а все, что находится между этими командами, можно исполнять в автоматическом режиме. Интерес программиста был прост: наблюдая за траекторией "забега" программы, отследить момент, когда программа уклонится в неправильную сторону, а потом найти причину отклонения и исправить программу так, чтобы программа "бегала" по предназначенному ей пути. Обычно отслеживание траектории исполнения по "ключевым точкам" использовалось для получения ответов на вопросы "после какой точки программа начинает работать некорректно", а пошаговое исполнение - чтобы точно определить, какие именно команды формируют неверные данные. И методы эти за все годы, прошедшие со времен их появления, ничуть не утратили актуальность и широко применяются в практически неизменном виде.

Технический прогресс не стоял на месте: расставлять брейкпойнты и гонять по ним программу вручную было неудобно, и программистам не могла не прийти в голову мысль "а почему бы не усовершенствовать отладчик таки образом, чтобы собственно трассировка выполнялась в автоматическом режиме". Прообразом современных средств трассировки был режим "анимации" (то есть замедленного исполнения) программы - при некотором навыке в мелькании содержимого регистров и переменных можно было попытаться отловить полезную информацию и "притормозить" программу в нужный момент. Впрочем, пользы от этого режима в те времена было немного - изучать в режиме анимации сложные программы было неудобно, а анимировать программы, работающие с графикой - и вовсе невозможно (в те времена отладчики функционировали исключительно в текстовом режиме). Однако в некоторых современных отладчиках режим анимации все-таки сохранился, и, надо отметить, толку от него куда больше, чем во времена DOS. Значительно возросшее быстродействие современной техники превратило анимацию из слайд-шоу для особо терпеливых, сопровождавшегося лихорадочным мерцанием экрана, в весьма динамичное действо, наблюдение за которым доставляет лишь удовольствие. Некоторые люди считают, что исполнение кода в режиме анимации никакой практической пользы не приносит, но я придерживаюсь несколько иного мнения: в этом режиме хорошо заметны длинные циклы, а также циклы со счетчиком, особенно если счетчик расположен в одном из регистров. Начинающим, думаю, интересно будет "вживую" понаблюдать работу распаковщика исполняемых программ - лучше один раз увидеть работу несложной процедуры распаковки, чем сто раз прочитать "книжное" описание этого процесса. Для просмотра этого шоу лучше всего взять небольшую программу, сжатую UPX'ом, поскольку "навесная" процедура распаковки UPX - одна из самых простых, и даже если у Вас не получится полностью разобраться в ней при помощи

отладчика/дизассемблера, все сложные моменты можно разъяснить по исходным текстам, находящимся в свободном доступе.

Читая эту главу, Вы могли заметить, что многие из описанных методов, несмотря на всю их внешнюю простоту, довольно неудобны для практического применения, поскольку требуют от пользователя отличной памяти и внимательности, а также способностей к рекурсивному "чтению" кода. Попробуйте "пробежаться" отладчиком по достаточно сложной процедуре, при этом строя траекторию исполнения кода в уме. Если у Вас получилось - значит, либо процедура оказалась не очень сложной, либо большинству людей остается лишь позавидовать Вашей памяти. Так или иначе, но программисты пришли к идее автоматической трассировки, то есть пошагового исполнения программы с одновременным "запоминанием" всех сделанных шагов. Основным препятствием на пути к осуществлению этой идеи долгое время был сравнительно небольшой объем оперативной памяти старых ЭВМ и недостаточная мощность процессора. Даже такой старый процессор, как 8086, способен был выполнять десятки и сотни тысяч команд в секунду - представьте, какой объем памяти потребовался бы, чтобы запомнить всего лишь последовательность адресов исполненных команд, не говоря уже о состоянии регистров. Кроме того, на одну команду, выполненную отлаживаемой программой в режиме трассировки, приходятся десятки и сотни команд, выполненных отладчиком - и отсюда возникает заметное падение производительности трассируемой программы. В общем, до некоторого времени реализация такого способа трассировки машинного кода была практически нереальна. Однако когда объем памяти компьютеров начал измеряться мегабайтами, воплощение этой идеи наконец стало возможным. Режим трассировки появился сначала в SoftIce, а затем и в OllyDebug, причем по возможностям трассировки и связанных с ней функций OllyDebug определенно превзошел все прочие известные автору отладчики.

Прежде всего следует отметить, что OllyDebug, в отличие от SoftIce, запоминает трассировочную информацию более "интеллектуально" - то есть сохраняет не только список адресов команд, но и модификации регистров. К сожалению, хранить информацию обо всех изменениях в адресном пространстве процесса OllyDebug не может (это потребовало бы совершенно невообразимого объема ОЗУ), но если обращение к переменной производится по указателю на нее, отладчик вполне способен запомнить значение регистра-указателя. Такое поведение отладчика облегчает задачу отслеживания состояний регистров: если Вас интересует, в какой момент том или ином регистре "появилось" некое число, Вы можете решить эту задачу простым поиском этого числа в текстовом файле. Да-да, именно в текстовом файле - OllyDebug обладает совершенно уникальной на сегодняшний день возможностью сохранять практически любые промежуточные данные из отладчика на жесткий диск, и в число таких данных входит отчет об исполнении программы в режиме трассировки. Вы можете безо всяких ухищрений сохранить список всех исполненных команд, их адресов, а также всю дополнительную информацию об изменениях в регистрах. Пример такого отчета Вы можете увидеть ниже:

```
Address Thread  Command Registers and comments
    Flushing gathered information
01006AEC        Main    xor     ebx, ebx        EBX=00000000
01006AEE        Main    push    ebx     pModule = NULL
01006AEF        Main    mov     edi, dword ptr ds:[<&KERNEL32.GetModuleHandleA>]        EDI=77E7AD86
01006AF5        Main    jnz     short NOTEPAD.01006B16  EAX=01000000
01006AF7        Main    mov     ecx, dword ptr ds:[eax+3C]
01006AFC        Main    add     ecx, eax
01006AFE        Main    cmp     dword ptr ds:[ecx], 4550        ECX=000000E8
01006B01        Main    jnz     short NOTEPAD.01006B15  ECX=010000E8
```

Как распорядиться столь подробным отчетом - зависит только от Ваших целей и изобретательности. Я могу лишь подсказать самые общие направления поиска. Прежде всего, стоит проанализировать значения, находящиеся в первой колонке, то есть адреса команд и "наложить" список адресов выполненных команд на листинг дизассемблирования программы (для W32Dasm, который формирует этот листинг в виде текстового файла, технически осуществить это

несложно). Такое слияние "мертвого" листинга с отчетом отладчика о работе "живого" кода способно значительно облегчить понимание того, в какой точке программа повела себя "не так", и как наставить ее на путь истинный. Также бывает удобным "спроецировать" конкретную траекторию исполнения кода на схему всех возможных путей исполнения кода (например, обвести часть стрелочек красным карандашом) - сочетая этот прием с патчингом кода и/или модификацией данных для принудительной активации тех или иных траекторий, можно последовательно отсеивать траектории, не ведущие к желаемой цели.

Здесь следует сделать небольшое отступление и рассказать об одном принципиальном преимуществе текстового представления дизассемблерных листингов перед всевозможными упакованными двоичными форматами. Текст - это один из самых старых и универсальных способов передачи информации, и для его обработки создано огромное количество всевозможных утилит. Инструменты работы с текстовыми файлами сами по себе очень стары, но, тем не менее, практически не устаревают, алгоритмы, в них использующиеся, "вылизаны" и доведены до совершенства поколениями программистов, и потому было бы неразумно отвергать столь огромный пласт программистской культуры. Конечно, использование узкоспециализированных двоичных форматов в дизассемблерах позволяет сэкономить дисковое пространство, ускорить обработку данных и хранить вместе с листингом различную дополнительную информацию, например, списки перекрестных ссылок, но за это приходится расплачиваться неоправданной интеграцией собственно дизассемблера и средства просмотра дизассемблированного текста. А просмотрщики дизассемблерных листингов, увы, обычно проектируются по остаточному принципу. Если рассматривать наиболее популярные в настоящее время W32Dasm и IDA Pro, то можно обнаружить не всегда удобную навигацию, ориентацию на исключительно текстовый режим работы (как я уже говорил, построение блок-схем в IDA Pro реализовано в виде программы сторонних разработчиков, не поддерживающей интерактивную работу с кодом), и некоторые другие недостатки. Авторы дизассемблеров, впрочем, вполне осознают недостатки двоичных форматов и полезность прямой работы с текстом, а потому предусмотрели возможность экспорта в текстовый формат и даже "обычного" текстового поиска в окне дизассемблера. В IDA Pro реализован даже более сложный вариант поиска с использованием регулярных выражений и специальный скриптовый язык, который по идее должен дать пользователю возможность самому добавить в дизассемблер недостающую функциональность. Однако хороший просмотрщик текстов, набор двоичных утилит *NIX'ового происхождения и навыки в программировании способны творить с "сырыми" листингами такие чудеса, какие традиционным дизассемблерам и не снились - от простого просмотра с подсветкой синтаксиса до форматирования "лесенкой" текстов на ассемблере. А поскольку наш отчет о трассировке как раз имеет текстовый формат и является, по сути, специфической формой дизассемблерного листинга, к нему вполне применимы все изложенные выше соображения о работе с текстом.

Поиск повторяющихся последовательностей адресов позволяет обнаружить программные циклы, а также узнать, сколько раз эти циклы выполнились. Ну и, разумеется, Вы сможете ответить на вопрос, который начинающие весьма часто задают, но на который весьма редко получают ответ. Вопрос этот такой: "как мне узнать, когда в регистре появляется нужное число?" (иногда встречалась еще более странная формулировка: "как установить брейкпойнт на регистр"). Простой поиск нужного числа в тексте отчета позволит Вам ответить на этот вопрос, хотя полезность такого действия в большинстве случаев весьма сомнительна (по крайней мере, у меня ни разу не возникало такой потребности).

Увы, в каждой бочке меда есть своя ложка дегтя, а в нашем случае - даже не одна. При пошаговой отладке (а, стало быть, и в процессе трассировки - тоже) накладные расходы на исполнение одной команды в десятки и сотни раз превышают собственно время исполнения команды, из-за чего скорость исполнения кода в режиме трассировки падает катастрофически. О расходах памяти на хранение информации о порядке исполнения команд я уже упоминал, однако это еще не все: если

внутри отладчика эта информация подвергается упаковке для более компактного хранения, то когда Вы сохраняете эту информацию на диск в виде текста, результирующий файл может получиться просто огромным. И он будет тем огромнее, чем большее количество команд было исполнено в режиме трассировки. Так что в ближайшем будущем нам, увы, не светит возможность запустив программу под отладчиком в режиме трассировки, беспрепятственно снимать с нее "жизненные показатели", выискивая подозрительные значения регистров и "нехорошие" команды переходов. Трассировка - это метод, пригодный для анализа сравнительно небольших кусков кода, чего, впрочем, обычно более чем достаточно. Также есть принципиальные проблемы с трассировкой программ, активно работающих с "железом", многопоточных программ и с софтом, использующим коммуникацию между процессами - все эти группы ПО, даже не содержащие защит, во время трассировки могут вести себя очень капризно.

Руководство по исследованию программ, написанных на Visual Basic 6.0

Автор: GPcH

Дата: 2005-06-07

[Вступление]

Многие почему то считают, что Visual Basic это примитивный язык программирования, который не может компилировать программы, работать с адресами переменных в памяти и не позволяющий вставлять ассемблерные процедуры в код. Так вот, все это неправда. Начиная с версии 5.0 данный язык позволяет компилировать программы в Native Code, а также имеется возможность работы с адресами переменных в памяти (для этого существует функция varptr). Ассемблерные процедуры вставлять тоже можно, но не так просто. Для этого я написал уже 2 части статьи по вставке ассемблерных процедур в код на Visual Basic. Эти статьи можно найти на сайте www.dotfix.net. Как видите, недостатков у VB не так уж и много, а преимуществ настолько много, что я расскажу лишь о самых очевидных и важных:

• Полноценная компиляция и маленький размер получаемых exe файлов. Несмотря на то, что программы требуют библиотеку MSVBVM60.DLL, это не является недостатком, так как эта библиотека интегрированная во все новые операционные системы и таскать ее вместе с приложением не нужно.

• Простота написания кода. Все удобно и наглядно. Даже начинающий может писать неоптимизированные, но рабочие программы. Как говориться, умеешь писать качественно - пиши, от этого программы будут только лучше. Не умеешь - можешь писать неоптимизированные проги (переменные объявлять не обязательно, преобразовывать типы данных тоже не обязательно). Но постепенно приходят более глубокие познания и можно писать крупные проекты, не сильно уступающие аналогам на C++ или Delphi по скорости работы, но на разработку которых уходит гораздо меньше времени.

Так вот, начинающие исследователи программ почему то считают, что программы, написанные на VB невозможно сломать. Если программа скомпилирована в pcode - я частично с ними соглашусь, но даже для этого вида компиляции уже написано множество отладчиков, способных понимать псевдокодовые инструкции. Что же касается Native Code, то тут все ломается также, как и любая другая программа, написанная например на C++ или Delphi. Но есть ряд особенностей. Все операции, которые выполняет программа выполняются с использованием узкоспециализированных функций из библиотеки MSVBVM60.DLL. Имена этих функций напоминают операторы Visual Basic'a, поэтому глядя на названия большинства из них, можно понять, какие операции они позволяют выполнять. Но есть и такие, которые не поддаются логике. О них я и расскажу в данной статье.

[Функции]

Самые непонятные исследователям функции - функции для преобразования данных из одного типа в другой. Чуть ниже я для удобства приведу существующие типы данных, используемых в Visual Basic 6.0 и функции для работы с ними. За большую часть данной информации хотелось бы поблагодарить Eternal_Bliss (http://crackmes.cbj.net) и авторов сайта http://www.infonegocio.com/vbcrack/, хотя на момент второй редакции статьи (которую Вы сейчас видите перед собой) мне прилось значительно расширить эту информацию основываясь на собственном опыте.

Часть имени каждой функции преобразования данных представляет собой набор аббревиатур, обозначающих типы исходных и конечных данных. Вот их расшифровка:

Тип данных	Расшифровка
bool	boolean
str	string
i2	byte or integer (2 битный integer)
ui2	unsigned integer (2 битный unsigned integer)
i4	long (4 битный integer)
r4	single (4 битный real)
r8	double (8 битный real)
cy	currency
var	variant (VB) или variable (OLEAUT)
fp	число с плавающей точкой
cmp	сравнение
comp	сравнение

Ниже представленые функции, экспортируемые библиотекой MSVBVM60.DLL. Если Вам потребуется вызывать эти вункции из своих программ, то имейте ввиду, что у VB свои типы данных, отличные в некоторых случаях от тех, которые используются в Delphi и C++. Строки вообще имеют свой особый формат (в начале строки два (или четыре) байта указывают на длину, затем идет строка). Отсюда бессмысленно пытаться вызвать перечисленные ниже функции из Delphi или C++ - такой вызов скорее всего приведет к ошибке. Что касается конвенции вызовов, то VB использует stdcall и другие не поддерживает, из чего логично сделать вывод, что Dll написанную на C++ с конвенцией вызова cdecl вызвать из VB практически невозможно.

• Функции для преобразования типов данных

Функция	Описание
__vbaI2Str	преобразует String в Integer
__vbaI4Str	преобразует String в Long
__vbar4Str	преобразует String в Single
__vbar8Str	преобразует String в Double
VarCyFromStr	преобразует String в Currency
VarBstrFromI2	преобразует Integer в String

• Перенос данных

Функция	Описание
__vbaStrCopy	копирует строку в память - аналог API функции HMEMCPY
__vbaVarCopy	копирует переменный тип (variant) в память
__vbaVarMove	копирует переменный тип (variant) в память

• Математические функции

Функция	Описание
__vbavaradd	сложение двух переменных типа Variant

Функция	Описание
__vbavarsub	деление двух переменных типа Variant
__vbavarmul	умножение двух переменных типа Variant
__vbavaridiv	сложение двух переменных типа Variant с выводом результата в переменную типа Integer
__vbavarxor	XOR

• Другие функции

Функция	Описание
__vbavarfornext	используется в конструкциях For... Next... (Loop)
__vbafreestr	удаление переменной
__vbafreeobj	удаление объекта
__vbastrvarval	получения численного значения из строки
multibytetowidechar	преобразование кодировки
rtcMsgBox	показывает сообщение - аналог API messagebox/a/exa
__vbavarcat	объединяет две переменные типа Variant
__vbafreevar	удаляет переменную типа Variant
__vbaobjset	создает объект
__vbaLenBstr	определяет длину строки
rtcInputBox	показывает форму с полем ввода (используются также API функции getwindowtext/a, GetDlgItemtext/a)
__vbaNew	аналог API функции Dialogbox
__vbaNew2	аналог API функции Dialogboxparam/a
rtcTrimBstr	удаляет пробелы вначале и в конце строки

• Функции сравнения

Функция	Описание
__vbastrcomp	сравнивает 2 строковые переменные - аналог API функции lstrcmp
__vbastrcmp	сравнивает 2 строковые переменные - аналог API функции lstrcmp
__vbavartsteq	сравнивает 2 Variant переменные
__vbaFpCmpCy	сравнивает значение с плавающей точкой с Currency значением

[Разблокирование элементов управления]

Любой элемент управления на форме может быть видимым или невидимым, доступным или заблокированным. Очень часто для взлома программы бывает нужно разблокировать отдельные элементы управления на форме. Для установки свойств объектов существует функция __vbaObjSet. Именно с помощью нее можно заблокировать или разблокировать элемент управления и изменить любое из его свойств. Поэтому нам необходимо отлавливать вызов именно этой функции. Откроем например Olly Debugger, найдем эту функцию среди вызываемых программой и поставим на нее бряк нажав кнопку F2. Затем запустим исследуемую программу. Когда бряк сработает - посмотрите окружающий код. Если он напоминает

```
50 push eax
```

```
52 push edx
FFD7 call edi
8BD8 mov ebx, eax
6A00 push 00
53 push ebx
8B03 mov eax, dword ptr [ebx]
```

то вы на верном пути. Как вы думаете, что это за "push 00"? 00h в VB означает FALSE, а FFh TRUE, из этого следует, что данная команда устанавливает свойство в FALSE, то есть возможно это и есть блокировка элемента управления на форме. Но это может быть и установка любого другого свойства формы в TRUE. Так как синтаксис один и тот же, что принадлежность данной команды к изменению свойства блокировки можно установить только анализом окружающего кода, но умаю с этой мелочью вы справитесь сами.

[Методика взлома простейших проверок пароля]

Нижеследующий текст - мой вольный перевод статьи How to Research Visual Basic Cracking с сайта http://www.infonegocio.com/vbcrack/. За английскую версию текста спасибо ее авторам.

Что нам может понадобиться? Любой дизассемблер/отладчик. Подойдет Win32Dasm или Olly Debugger Если вы используете W32Dasm, то функции, используемые программой вы можете посмотреть в меню "Functions" -> "Imports".

Если вы будете исследовать базу данных Jet, которая использует драйвера Micro$oft, то помните, что названия функции не говорят сами за себя и вам будет трудно узнать действие, которое выполняет та или иная функция. в данном случае Вам остается только смотреть рекомендации к функциям, которые любезно оставлены программистами Micro$oft. Дыр же в этих драйверах практически нет, поэтому исследовать программы для работы с базами данных довольно сложно.

Рассмотрим пример защиты программы паролем. Для удобства поиска нужного кода поставим MessageBox. Откройте Visual Basic, добавьте на форму текстовое поле и кнопку, затем в обработчике щелчка по кнопке напишите следующий код:

```
Private Sub Command1_Click()
    Dim X&
    X = 43690
    MsgBox "Testing password"
    If CLng(Trim$(Text1.Text)) = X Then MsgBox ("GoodBoy")
End Sub
```

Теперь откроем Olly Debugger, загрузим в него прогу (предварительно прогу нужно откомпилировать) и ищем вызов функции rtcMsgBox.

```
00401F54  FF1520104000  CALL [MSVBVM60!rtcMsgBox] ;MsgBox "Testing password"
00401F5A  8D459C  LEA EAX,[EBP-64]
00401F5D  8D4DAC  LEA ECX,[EBP-54]
00401F60  50   PUSH EAX
00401F61  8D55BC  LEA EDX,[EBP-44]
00401F64  51   PUSH ECX
00401F65  8D45CCLEA  EAX,[EBP-34]
00401F68  52   PUSH EDX
00401F69  50   PUSH EAX
00401F6A  6A04   PUSH 04
00401F6C  FF1508104000  CALL [MSVBVM60!__vbaFreeVarList]
00401F72  8B0E   MOV ECX,[ESI]
00401F74  83C414  ADD ESP,14
00401F77  56   PUSH ESI
00401F78  FF9104030000  CALL [ECX+00000304]
00401F7E  8D55DCLEA  EDX,[EBP-24]
00401F81  50   PUSH EAX
00401F82  52   PUSH EDX
00401F83  FF1524104000  CALL [MSVBVM60!__vbaObjSet]
00401F89  8BF0   MOV ESI,EAX
00401F8B  8D4DE4LEA  ECX,[EBP-1C]
00401F8E  51   PUSH ECX
00401F8F  56   PUSH ESI
```

```
00401F90  8B06   MOV EAX,[ESI]
00401F92  FF90A0000000  CALL [EAX+000000A0]
00401F98  3BC7   CMP EAX,EDI
00401F9A  DBE2   FCLEX
00401F9C  7D12   JGE 00401FB0
00401F9E  68A0000000  PUSH 000000A0
00401FA3  6804184000  PUSH 00401804
00401FA8  56   PUSH ESI
00401FA9  50   PUSH EAX
00401FAA  FF1518104000  CALL [MSVBVM60!__vbaHresultCheckObj] ;получение содержимого Text1.Text
00401FB0  8B55E4MOV  EDX,[EBP-1C]
00401FB3  52   PUSH EDX
00401FB4  FF1514104000  CALL [MSVBVM60!rtcTrimBstr] ;Trim$
00401FBA  8BD0   MOV EDX,EAX
00401FBC  8D4DE0  LEA ECX,[EBP-20]
00401FBF  FF1584104000  CALL [MSVBVM60!__vbaStrMove]
00401FC5  50   PUSH EAX
00401FC6  FF1568104000  CALL [MSVBVM60!__vbaI4Str] ;CLng
00401FCC  33C9   XOR ECX,ECX
00401FCE  3DAAAA0000  CMP EAX,0000AAAA ;Число 43690 в HEX виде
00401FD3  8D55E0LEA  EDX,[EBP-20]
00401FD6  8D45E4LEA  EAX,[EBP-1C]
00401FD9  0F94C1  SETZ CL
00401FDC  52   PUSH EDX
00401FDD  50   PUSH EAX
00401FDE  F7D9   NEG ECX
00401FE0  6A02   PUSH 02
00401FE2  8BF1   MOV ESI,ECX
00401FE4  FF156C104000  CALL [MSVBVM60!__vbaFreeStrList]
00401FEA  83C40C  ADD ESP,0C
00401FED  8D4DDC  LEA ECX,[EBP-24]
00401FF0  FF1594104000  CALL [MSVBVM60!__vbaFreeObj]
00401FF6  663BF7  CMP SI,DI
00401FF9  7463   JZ 0040205E ;переход на GoodBoy код (MsgBox "GoodBoy")
00401FFB  B804000280  MOV EAX,80020004
00402000  8D558C  LEA EDX,[EBP-74]
00402003  8D4DCCLEA  ECX,[EBP-34]
00402006  8945A4  MOV [EBP-5C],EAX
00402009  895D9C  MOV [EBP-64],EBX
0040200C  8945B4  MOV [EBP-4C],EAX
0040200F  895DAC  MOV [EBP-54],EBX
00402012  8945C4  MOV [EBP-3C],EAX
00402015  895DBC  MOV [EBP-44],EBX
00402018  C7459418184000 MOV DWORD PTR [EBP-6C],00401818
0040201F  C7458C08000000 MOV DWORD PTR [EBP-74],00000008
00402026  FF157C104000  CALL [MSVBVM60!__vbaVarDup]
0040202C  8D4D9C  LEA ECX,[EBP-64]
```

Чтобы взломать данную программу - нужно изменить переход по адресу 00401FF9 с условного на абсолютный. Для этого меняем JE на JMP (то есть 74h на EBh). Как видите - простенькие защиты VB программ ломаются не сложнее чем в программах, написанных на Delphi или C++

[Взлом паролей в виде разных типов данных]

Нижеследующий тест - мой вольный перевод статей:
Cracking Visual Basic Serials - Strings, integers and Longs
Cracking Visual Basic Serials - Single, Double and XOR
с сайта http://www.infonegocio.com/vbcrack/. За английские версии этих статей спасибо их авторам.

• Получение пароля, если он лежит строкой в открытом виде

Когда VB копирует строку в память - Вы ее можете отловить. Для этого нужно поставить бряк на функцию __vbaStrCopy и когда бряк сработает, посмотрите код после je 66047B00 (он содержится в библиотеке и един для всех программ). вам нужно посмотреть содержимое edx-04 (в SoftICE нужно ввести команду d edx-04). При этом вы увидите строчку, с которой работает в данный момент функция.

```
Exported fn(): __vbaStrCopy - Ord:008Ah
```

```
:66024532  56  PUSH ESI
:66024533  57  PUSH EDI
:66024534  85D2  TEST EDX, EDX
:66024536  8BF9  MOV EDI, ECX
:66024538  0F84C2350200 JE 66047B00
:6602453E  FF72FC PUSH [EDX-04] ;по этому адресу лежит строка
:66024541  52  PUSH EDX
```

• Получение пароля, если он сравнивается с введенным

Поставьте бряк на функцию __vbaStrComp и чуть выше ее вызова будут два push'a - они заносят в стек две Variant переменные для сравнения. Посмотрите содержимое EAX чтобы просмотреть адрес теста. Просмотрите, что лежит по этому адресу, наверняка это правильный пароль.

```
00401BC7 50  PUSH EAX ;то что Вы ввели
00401BC8 6880174000 PUSH 00401780 ;верный пароль
00401BCD FF1530104000 CALL [MSVBVM60!__vbaStrComp]
```

• Получение пароля, если происходит сравнение 4х байтовых числовых переменных

Поставьте бряк на функцию __vbai4str. Эта функция используется программой для перевода введенной текстовой строки в 4х байтовое число. Когда бряк сработает - посмотрите код ниже - там наверняка будет проца сравнения паролей. Но значение пароля будет в HEX виде (например десятичное число 987654321 равно шестнадцатеричному 3ADE68B1).

```
00401B77 FF155C104000 CALL [MSVBVM60!__vbaI4Str]
00401B7D 8D4DE0  LEA ECX, [EBP-20]
00401B80 8BF8  MOV EDI, EAX
00401B82 FF157C104000 CALL [MSVBVM60!__vbaFreeStr]
00401B88 8D4DDC  LEA ECX, [EBP-24]
00401B8B FF1580104000 CALL [MSVBVM60!__vbaFreeObj]
00401B91 81FFB168DE3A CMP EDI, 3ADE68B1 ; 3ADE68B1 - это пароль
00401B97 7520  JNZ 00401BB9  ;функция вернет ноль
                                    ;если пароли верны, следовательно этот переход
                                    ;сработает только при неверном пароле
```

Тут мы можем узнать верный пароль или отключить проверку пароля (для этого нужно JNZ заменить на NOP (то есть в данном случае менять нужно байты по адресу 00401B97 с 7520 на 9090)

• Получение пароля при сравнивании двух 2х байтовых чисел

Отличие от предыдущего примера в том, что нужно ставить бряк на функцию __vbai2Str. Но иногда предварительно производится преобразование строки сначала в 4х байтовое число посредством функции __vbaI2l4, а затем 4х байтовое число преобразуется в двухбайтовое.

• Обход сравнения двух переменных типа Single

Эту проверку обойти довольно просто. Если Вашей целью является написание патча, то ставьте бряк на функцию __vbaR4Str и когда он сработает - пролистните код вниз. Вы увидите переход после проверки. Просто измените условие перехода на обратное.

```
00401C0C FF153C104000 CALL [MSVBVM60!_vbaR4Str]
00401C12 D95DE4  FSTP REAL4  PTR [EBP-1C]
00401C15 8D4DDC  LEA ECX, [EBP-24]
00401C18 8D55E0  LEA EDX, [EBP-20]
00401C1B 51  PUSH ECX
00401C1C 52  PUSH EDX
00401C1D 6A02  PUSH 02
00401C1F FF1570104000 CALL  [MSVBVM60!__vbaFreeStrList]
00401C25 83C40C  ADD ESP, 0C
00401C28 8D4DD8  LEA ECX, [EBP-28]
00401C2B FF1594104000 CALL [MSVBVM60!__vbaFreeObj]
00401C31 817DE43A92FC42 CMP DWORD PTR [EBP-1C], 42FC923A
00401C38 7520  JNZ 00401C5A ;если ноль, то пароль неправильный
```

Замените инструкцию по адресу 00401C38 на 9090 (чтобы в данном месте не выполнялось никаких операций) или измените условный переход на противоположный. Программисты на Visual Basic не смогут проверить то, что вы заменили эти байты на два NOP'a (9090). Если у Вас все же возникают опасения, что программа перестанет работать при отсутствии команд, просто введите команды, которые ничего не поменяют, например:

```
inc eax
dec eax
```

Эти команды однобайтовые и как раз впишутся на место условного перехода

• Обход проверки значений с плавающей точкой (double).

Это также сделать очень просто. Если хочется обойтись битхаком - ставьте бряк на функцию __vbaR8Str и пролистните ком чуть ниже бряка, Вы увидите процедуру сравнения (FCOMP REAL8 PTR [Address]), после нее идет TEST и jump. Измените условие перехода на противоположное.

```
00401C55 FF1510104000 CALL [MSVBVM60!rtcTrimBstr]
00401C5B 8BD0  MOV EDX, EAX
00401C5D 8D4DD8  LEA ECX, DWORD PTR [EBP-28]
00401C60 FF1580104000 CALL [MSVBVM60!__vbaStrMove]
00401C66 50  PUSH EAX
00401C67 FF1560104000 CALL [MSVBVM60!__vbaR8Str]
00401C6D DC1DD8104000 FCOMP REAL8 PTR [004010D8] ;сравнение паролей
00401C73 DFE0  FSTSW AX ;обработка сопроцессором
00401C75 F6C440  TEST AH, 40 ;проверка правильности пароля
00401C78 7409  JE 00401C83 ;переход, если пароль верный
```

Замените инструкцию по адресу 00401C78 на 9090 (чтобы в данном месте не выполнялось никаких операций) или измените условный переход на противоположный. Программисты на Visual Basic не смогут проверить то, что вы заменили эти байты на два NOP'a (9090). Если у Вас все же возникают опасения, что программа перестанет работать при отсутствии команд, просто введите команды, которые ничего не поменяют, например:

```
inc eax
dec eax
```

Эти команды однобайтовые и как раз впишутся на место условного перехода

• Получение пароля, если он поXORен.

Алгоритм работы бейсиковской процедуры таков. Введенный пароль посимвольно переводится в ASCII коды, которые по очереди XORятся. Затем производится обратный перевод, возможно с предварительными математическими манипуляциями с ASCII кодами. ANSI коды могут XORиться со случайным или фиксированным числом.

```
00401E6C 50  PUSH EAX
00401E6D FF1544104000 CALL [MSVBVM60!rtcMidCharBstr]
00401E73 8BD0  MOV EDX, EAX
00401E75 8D4DC8  LEA ECX, [EBP-38]
00401E78 FFD6  CALL ESI
00401E7A 50  PUSH EAX
00401E7B FF1518104000 CALL [MSVBVM60!rtcAnsiValueBstr]
00401E81 0FBFC8  MOVSX ECX, AX
00401E84 81F191000000 XOR ECX, 00000091 ;ANSI XOR 91
00401E8A 51  PUSH ECX ;результат XOR'a
00401E8B FF1570104000 CALL  [MSVBVM60!rtcBstrFromAnsi]
```

Ставим бряк на 00401E8A 51 PUSH ECX и отслеживаем изменение содержимого регистра ECX при прохождении цикла. Когда программа пройдет весь цикл - вы получите полный пароль.

```
CALL [MSVBVM60!rtcMidCharBstr] ;получает один символ из пароля
CALL  [MSVBVM60!rtcAnsiValueBstr] ;получает из символа Ansi код.
CALL  [MSVBVM60!rtcBstrFromAnsi] ;преобразует Ansi код в строку
```

В этом случае ниже скорее всего будет сравнение и переход, при этом CALL [MSVBVM60!__vbaStrComp] может быть использован для сравнения строк. Если не производится обращение к MSVBVM60!rtcBstrFromAnsi тогда с паролем производятся определенные математические манипуляции, при этом он может быть представлен как 2х или 4х битное число или число с плавающей точкой. Об этих случаях было сказано выше.

[Заключение]

Надеюсь, что прочитав данную статью Вас уже больше не пугает исследование VB кода. Чтобы потренироваться советую поисследовать мои крякми (лежат на www.dotfix.net в разделе "Разное"), о взломе первого из которых написана статья (лежит на www.dotfix.net в разделе "Статьи"). Второй мой крякми еще никто не взломал ;)

Еще раз благодарю авторов сайта http://www.infonegocio.com/vbcrack/, так как если бы не их туториалы на английском, то этой статьи возможно и не было бы.

Удачи и спасибо за то, что дочитали статью до конца

Теоретические основы крэкинга: Глава 12. Патчить или не патчить?

Автор: CyberManiac

Дата: 2005-06-27

*Быть или не быть? Вот в чем вопрос.
У. Шекспир*

Собственно, вопрос, вынесенный в заглавие этой главы, существует разве что в умах ревнителей идейной чистоты крэкинга, с недоверием относящихся к самой идее модификации исполняемого кода. Для всех остальных ответ очевиден - патчить надо и патчить надо много, поскольку модификация исполняемого кода - самый прямой, простой и быстрый путь заставить код делать то, чего Вы от него хотите. Вряд ли вообще возможно найти хотя бы одного крэкера, который не изготовил бы ни одного патча, не дописал бы в программу десяток-другой байт, не экспериментировал с "отклонением" системных вызовов на свои собственные функции. Хотя обычно патчинг представляют как исправление пары-тройки байт (обычно - в командах условного перехода), на самом деле эта техника применяется не только для банального "бит-хака", но и в таких областях, как перехват системных вызовов, внедрение собственного кода в чужие программы, дописывание недостающей функциональности в программы, а также для многих других не менее интересных вещей. За время чтения предыдущих глав Вы наверняка уже приобрели навыки, необходимые для успешного исправления опкода 75 на опкод EB, научились выкидывать из программ целые процедуры, не порушив при этом стек, и разобрались в основах устройства той операционной системы, под которой Вы планируете развернуть свою крэкерскую деятельность. Стало быть, пришло время рассказа о тонкостях и нетривиальных подходах к модификации программного кода, которая есть альфа и омега всего крэкинга.

Исправлять переходы или содержимое констант - дело нехитрое, но рано или поздно перед Вами встанет в полный рост жестокая необходимость вклинить пару-тройку собственных команд туда, где для этих команд место не предусмотрено, и вставить так, чтобы ничего при этом не испортить. Решение достаточно очевидно - поместить нужный код в свободную область памяти, при помощи команд условного перехода сделать "отвод" в эту область, а потом вернуть управление обратно. Но вот где эту свободную область взять? Смею Вас заверить, таких свободных областей в исполняемых файлах обычно более чем достаточно и искать их долго не придется. В зависимости от требуемого объема памяти Вы можете воспользоваться одной из следующих пяти стратегий:

1. Использование пространства между процедурами или массивами. Во многих компиляторах для повышения быстродействия по умолчанию включены опции выравнивания начала процедур и массивов. К примеру, если процедуры выровнены по границам 16-байтных областей, Вы можете обнаружить до 15 байт свободного места перед или после каждой процедуры. Найти такие промежутки несложно - они инициализированы последовательностью одинаковых байт, обычно с кодом 0 или 90h (этому коду соответствует команда nop), реже - 0CCh (команда int3) или 0FFh. В такую область вполне возможно вписать пару-тройку команд для загрузки данных в регистры или изменения значения переменной. Поскольку процедуры встречаются достаточно часто, передать управление на такой блок и затем вернуться обратно, скорее всего, получится при помощи двухбайтного варианта команды jmp, что сведет к минимуму расход памяти на переходы туда-обратно. Кроме того, при помощи все тех же двухбайтных переходов можно связать воедино несколько таких свободных кусков и таким образом увеличить количество команд. Недостатки этого метода: без ухищрений невозможно вписать в программу достаточно длинную последовательность команд; связывать вручную свободные куски довольно неудобно; нет никакой гарантии, что поблизости окажется достаточно длинный кусок свободной памяти; при включенной оптимизации

по размеру компилятор может размещать процедуры вплотную, что сделает данный метод неприменимым. Кроме того, практически невозможен вызов функции, не импортируемой явным образом: Вам просто надоест вручную сооружать "обвязку" для вызова GetProcAddress, параллельно размазывая ее по щелям между процедурами.

2. Использование места, высвободившегося после "вывода из обращения" защитных процедур или ненужных тестовых строк. В процессе взлома в программах нередко обнаруживаются "лишние" защитные процедуры, которые крэкеру приходится просто отключать, а также всевозможные надписи, предлагающие обменять некоторую сумму денежных единиц на серийный номер или ключевой файл. При необходимости память, занимаемая этими процедурами и надписями, может быть использована для размещения небольших блоков кода. Возможен и другой вариант использования этого метода - исключить из программы какую-нибудь функцию, которая Вам заведомо не понадобится, а на занимаемое ей место поместить собственный код. Кроме того, есть смысл исследовать подфункции вызываемой функции: некоторые из них могут использоваться исключительно внутри удаляемой функции, и потому тоже могут послужить источником дополнительных байтов. Недостатки метода: не всегда возможно получить достаточный объем памяти; нужна крайняя аккуратность при переписывании команд, содержащих явное указание адреса в памяти. Метод в принципе неприменим, если Вы используете крэкинг "в мирных целях", то есть Ваши задачи предполагают не выламывание из программы защитных процедур, а только лишь усовершенствование программы за счет добавления в нее собственного кода.

3. Размещение кода в неиспользуемых областях в концах секций. Программисты на ассемблере под Win32 наверняка хорошо знакомы со следующим эффектом: при добавлении в программу новых строк ее размер до поры-до времени не меняется, а потом вдруг скачкообразно увеличивается на полкилобайта, а то и больше (конкретное число зависит от настроек линковщика). Причина этого эффекта в том, что код и данные в программах под Win32 (да и не только под Win32) расположены внутри исполняемого файла в секциях, и размеры этих секций должны быть кратны определенному (и довольно большому) числу байт. Нередки ситуации, когда реальный размер кода программы или ее инициализированных данных на несколько десятков, а то и сотен байт меньше, чем размер выделенной для этого кода секции, и кусок памяти в "хвосте" секции ничем полезным не занят. А если в программе есть "лишние" байты, то почему бы не использовать их для своих целей? Недостатки метода: объем неиспользованного пространства в конце секции совершенно непредсказуем и может колебаться от нуля до нескольких килобайт; следует быть крайне осторожным при записи данных в конец секции инициализированных данных, чтобы случайно не испортить эти самые данные: к примеру, свободное место и длинный массив байтов, инициализированный нулями, внешне выглядят совершенно одинаково.

4. Расширение существующих секций или создание новых. При таком подходе в принципе невозможно случайно разрушить полезные данные - весь Ваш код будет размещаться в областях, которых в непатченой программе даже не существовало. Более того, таким способом можно внедрить собственный код даже в упакованную программу, если распаковщик не содержит специальных средств для контроля целостности исходного файла (в этом случае лучше создать отдельную секцию и размещать свой код в ней). Размер внедряемого кода при использовании этого метода ограничен разве что здравым смыслом и объемом памяти ЭВМ. Основной недостаток заключается в том, что создание или расширение секций предполагает модификацию PE-заголовка и изменение размеров файла, из-за чего антивирусные программы могут не слишком благосклонно отнестись к таким переменам. Кроме того, поскольку размер файла изменяется за счет вставок в середину, возможны проблемы с созданием исполняемых файлов, выполняющих автоматический патчинг (такие файлы в просторечии называются "крэками"). Большинство утилит, создающих такие файлы, ограничивается побайтным сравнением с оригиналом и потому "не переваривают" вставку даже одного-единственного байта в середину программы. Кроме того, если Вам потребуется вызвать функцию API, которая отсутствует в таблице импорта, Вам придется либо

править эту таблицу, либо получать адрес функции при помощи вызова GetProcAddress, что не слишком удобно. Этот недостаток в полной свойственен и трем предыдущим стратегиям патчинга, причем в еще большей мере - по причине ограниченного объема памяти, который эти стратегии позволяют получить. Кроме того, операции по расширению и вставке секций технически очень сложно выполнить в памяти над загруженной программой, поскольку это требует очень глубоких знаний системы и весьма значительных трудозатрат, совершенно не адекватных получаемому результату.

5. Подгрузка кода, размещенного во внешних модулях. Это наименее "жесткий" по отношению к модифицируемой программе метод внедрения кода. Идея метода заключается в том, чтобы вынести весь добавляемый в программу код во внешний модуль (если речь идет про Windows - то в DLL), а потом "попросить" программу загружать этот внешний модуль вместо одной из библиотек, например, исправив один-единственный байт в имени загружаемой DLL. "Подменная" DLL с нашим кодом вклинивается между программой и "настоящей" библиотекой, причем в функции инициализации DLL можно разместить что-нибудь полезное, например, код патчинга основного процесса. Более подробно методы внедрения модулей мы рассмотрим чуть позже, а пока отметим, что при использовании данного метода Вы получаете наиболее широкий выбор инструментария для написания внедряемого кода: от шестнадцатеричного редактора до самых современных RAD-средств (в отличие от предыдущих четырех стратегий, где Вашим основным инструментом будет ассемблер, встроенный внутрь редактора или отладчика). Главный недостаток заключается в необходимости держать этот самый внешний модуль рядом с программой, что не всегда удобно.

Рассказывая об использовании пространства, занимаемого "ненужными" процедурами, для размещения своего кода, я упомянул о проблеме, связанной с затиранием команд условных переходов, вызова подпрограмм и чтения содержимого фиксированных адресов. Изучая ассемблер, Вы наверняка заметили, что одним и те же мнемоникам "call" и "jmp" соответствует множество разных опкодов. Также Вы могли заметить, что в зависимости от опкода может варьироваться не только длина операнда, но и сам способ указания адреса, на который будет выполнен переход: адрес может указываться либо явным образом, либо в виде смещения относительно начала следующей команды. В современных версиях Windows исполняемые файлы всегда загружаются с адреса, указанного в заголовке исполняемого файла. Однако это правило распространяется только на исполняемые файлы, а вот DLL должны уметь загружаться с любого адреса (традиционно свойство программ или библиотек загружаться с любого адреса называется "перемещаемость"). Если учесть, что Windows - не единственная, а лишь одна из множества ОС с разными моделями памяти и механизмами загрузки приложений и библиотек, а задача обеспечения перемещаемости программных модулей возникает не так уж редко, реализация относительных переходов и вызовов "в железе" сильно облегчило жизнь разработчикам-первопроходцам.

Однако обходиться одной лишь относительной адресацией невозможно при всем желании - прежде, чем начинать отсчитывать смещения, необходимо явным образом указать расположение хотя бы одной реперной точки, от которой эти смещения будут отсчитываться. Да и с переменными удобнее работать традиционным способом, обращаясь к ним по абсолютным адресам. В общем, относительная адресация - отнюдь не панацея, гарантирующая перемещаемость любой программы, а всего лишь удобный инструмент.

Но все-таки, как решается проблема использования абсолютных адресов в перемещаемых программах? Наиболее распространенный подход - коррекция всех абсолютных адресов переходов и переменных во время загрузки. Но чтобы откорректировать абсолютные адреса, где-то должен храниться список всех точек программы, где используется абсолютная адресация. Для этого в исполняемый файл включается специальная таблица перемещаемости (она обычно называется relocation table), в которой указываются смещения всех точек программы, которые нужно откорректировать. Сам процесс пересчета адресов осуществляется весьма просто:

вычисляется разность между адресом, начиная с которого был загружен программный модуль и "желательным" адресом загрузки который был в заголовок модуля прописан линковщиком; затем полученная разность прибавляется к содержимому, находящемуся по смещениям, перечисленным в таблице перемещаемости.

Этот пересчет адресов можно рассматривать как акт патчинга со стороны операционной системы, осуществляемый во время загрузки. Теперь представьте, что произойдет, если Вы, ни о чем не подозревая, впишете вместо команды, обращающейся к какой-либо ячейке памяти, свой код. Загрузчику в общем-то без разницы, какие именно байты он исправляет - поэтому он заглянет в таблицу перемещаемости, выберет оттуда адреса ячеек, которые нужно исправить, а потом честно попытается откорректировать содержимое этих ячеек. Однако мы наполнили эти ячейки совершенно иным содержанием, и потому там лежит не адрес, который надо пересчитать, а нечто совершенно иное - написанные нами коды команд. Если алгоритмы работы загрузчика предполагают проверку корректности полученных при пересчете адресов, Вы можете получить сообщение об ошибке уже на этапе пересчета, если не предполагает - программа почти наверняка "рухнет" при попытке исполнить такой дважды исправленный кусок кода. Выходов - два: либо перетрясти таблицу перемещаемости и исключить из нее ненужные нам ссылки, либо при патчинге не затрагивать последовательности байт, похожие на абсолютные адреса в памяти, если таковые встретятся на Вашем пути. Надо отметить, что практически все программные "полуфабрикаты" - библиотеки, объектные файлы, предварительно откомпилированные модули Delphi (то есть файлы с расширением .dcu), компоненты - изначально рассчитаны на внедрение в неизвестное место будущей программы и потому являются перемещаемыми.

Другим интересным вопросом, непосредственно связанным с патчингом, является вызов функций API из внедряемого в программу кода. Как я уже упоминал в девятой главе, при статической загрузке библиотек (то есть когда библиотеки подгружаются уже в процессе загрузки программы на основе информации из таблицы импорта) вызов внешних функций осуществляется не прямым переходом по нужному адресу, а через "переходники", которые и передают управление на библиотечные функции. Понятное дело, что если такие "переходники" есть, то ими можно воспользоваться. Но как быть, если таких "переходников" нет (хотя заведомо известно, что нужная функция все-таки вызывается) или использовать нельзя их по каким-либо причинам? Есть как минимум два пути, позволяющих вызвать функции API даже в таких стесненных условиях. Обычный программистский подход к решению проблемы - вызвать функцию LoadLibrary , а вслед за ней - GetProcAddress и получить адрес искомой функции, после чего можно смело помещать параметры на стек и затем выполнить call [eax]. Для этого, разумеется, нужно, чтобы программа импортировала функции LoadLibrary и GetProcAddress, но тут обычно проблем не возникает: подавляющее большинство программ эти функции импортирует, а те, которые не импортируют, можно заставить это делать редактированием таблицы импорта.

Однако я предпочитаю использовать другой способ, как мне кажется, более удобный и безопасный. Основа метода заключается в том, чтобы вызывать функции не напрямую, а "одалживать" нужные вызовы у исследуемой программы. Если программа импортирует функцию MyFunc статически, это почти наверняка означает, что в коде программы есть команда call MyFunc. А если в программе есть команда call MyFunc, значит, когда мы хотим вызвать MyFunc, нам нужно всего лишь прочитать четыре байта (которые представляют собой адрес нашей функции) в регистр eax, поместить на стек параметры и выполнить вызов. На практике эта операция выглядит примерно так:

```
FAddr: call ExitProcess ; Нужная нам функция
…

OurPatch:
mov eax, dword ptr [FAddr+2] ; Читаем из памяти адрес функции
push 0 ; Помещаем на стек параметр вызова
call [eax] ; Вызываем "одалживаемую" функцию
```

Однако вызовами статически импортированных функций API возможности метода "функций взаймы" отнюдь не ограничиваются. "Заимствовать" из исследуемой программы можно совершенно любые функции - от элементарного сравнения строк до функций извлечения файлов из архива неизвестного формата или генерации серийных номеров (изредка встречаются программы с такой ошибкой в защите). Единственное, что для этого необходимо - выяснить тип передаваемых функции параметров и используемое соглашение вызова. Однако нужно понимать, что заимствование функций из программы или из несистемных DLL не вполне известного назначения - операция более опасная, чем использование импортированных функций документированных API. Дело в том, что работа функций API не привязана ни к каким переменным и функциям внутри программы (если, конечно, не считать callback-функции), а вот к функциям программы это в общем случае не относится. Их работа может зависеть от состояния локальных или глобальных переменных, а также других объектов, созданных в процессе работы программы; более того - сами программные вызовы тоже вполне могут изменять состояние переменных, критически важных для работы программы. Поэтому если какой-либо объект в момент вызова окажется в "неправильном" состоянии, результат вызова будет совершенно непредсказуем. Точно таким же образом никто не может гарантировать, что "несвоевременный" вызов не нарушит функционирование всей программы. Хотя, с другой стороны, встроенные процедуры, как правило, достаточно корректно обращаются с переменными и объектами, созданными программой. Так что если Вам вдруг понадобится форсировать загрузку какого-либо плагина, лучше попытаться это сделать это "родными" для программы средствами, и лишь если это у Вас не получится, прибегнуть к средствам API. В любом случае, при заимствовании из основной программы функций в качестве постоянного решения следует быть очень осторожным, хотя сам по себе этот метод нередко бывает полезен.

Патчинг позволяет решать и обратную задачу: "отклонение" вызовов из исследуемой программы на внедряемый крэкером код. Цели этой операции могут быть различными:

• Предварительный анализ параметров, передаваемых в функцию (например, для того, чтобы предотвратить выполнение нежелательных вызовов)
• Коррекция результата, возвращаемого функцией
• Выполнение каких-либо действий помимо тех, которые предполагались в программе (например, усовершенствование процедуры, создающей nag screen таким образом, чтобы это окно после исполнения функции сразу же закрывалось)
• Ведение и сохранение журнала вызовов функций, снятие статистики по выполнению тех или иных действий и частоте появления на стеке тех или иных значений

Как Вы можете видеть, приведенный список возможных целей покрывает весьма широкую область - от предотвращения нежелательных вызовов до наблюдения за вызовами функций (частным случаем этого являются API-шпионы). Собственно, некоторые техники, используемые в API-шпионах, и основаны на патчинге "переходников" к функциям API, осуществляемом сразу после загрузки программного модуля. Принцип действия большинства API-шпионов следующий:

1. Загрузить процесс в память, не запуская его (например, создав процесс с флагом CREATE_SUSPENDED, если речь идет о Win32).
2. Внедрить "шпионский" модуль в адресное пространство исследуемого процесса, например при помощи хуков или функции CreateRemoteThread (только под Windwos линейки NT).
3. Пропатчить в памяти процесса "переходники" к функциям API таким образом, чтобы они указывали на обработчики соответствующих функций внутри "шпионского" модуля.
4. Аналогичным образом перебросить переходник к функции GetProcAddress на собственный обработчик. Эта операция очень важна, поскольку только таким образом можно перехватить вызовы функций, адреса которых программа динамически запрашивает в процессе работы.
5. "Разморозить" процесс, после чего программа начнет исполняться.

Однако патчинг и перехват системных вызовов - отнюдь не прерогатива одних лишь крэкеров. Разработчики защит, хотя и с изрядным опозданием, тоже взяли на вооружение идею

перенаправления системных вызовов на собственный код. Как я уже упоминал, сравнительно недавно в большом количестве появились навесные защиты, которые позволяют упрятывать любую программу и все необходимые для ее работы файлы в упакованном и зашифрованном виде внутрь одного-единственного EXE. При этом сама программа может обращаться к своим файлам как средствами Win32 API, так и при помощи высокоуровневых функций (которые по сути являются "обертками" для все тех же системных вызовов). О том, каким образом работают такие защиты, Вы уже наверняка догадались: в исполняемый файл дописывается секция, где хранится код, обрабатывающий вызовы системных функций для работы с файлами. Необходимые для работы программы файлы упаковываются и цепляются в "хвост" программы, а таблица импорта дорабатывается таким образом, чтобы вызовы функций работы с файлами перенаправлялись на обработчики, находящиеся во внедренной секции. Когда защищенная программа попытается обратиться к файлу, задача обработчиков заключается в том, чтобы проверить, к какому именно файлу происходит обращение, и либо передать это обращение операционной системе в неизменном виде (если программа обращается к файлу вне "хранилища"), либо имитировать работу системного вызова, но в действительности считывание данных осуществлять из упакованного "хранилища". Однако ирония судьбы заключается в том, что метод взлома полностью аналогичен методу защиты. Действительно, если разработчик "отклонил" вызовы функций работы с файлами на собственный код, то и почему бы и крэкеру не проделать ту же самую операцию? То есть вклинить между программой и обработчиком свой собственный "обработчик обработчика", который будет сбрасывать все "спрятанные" файлы в надежное место. Основную проблему составляет поиск входных и выходных точек этих обработчиков, но здесь могут помочь "особые приметы", которые способны выдать чужеродный код:

1. Вклинить "левый" код в середину практически невозможно, поэтому остаются варианты с расширением секций (тогда нужный код окажется перед началом или в самом конце программы), созданием новой секции (что иногда заметно по "странным" адресам, заметно отличающимся от адресов основной программы) или динамическим выделением куска памяти и размещением там обработчика (самый сложный для реализации способ - а потому самый маловероятный).
2. Если защищенная программа читает упакованные файлы "по требованию", каждый вызов функций чтения файлов внутри программы будет сопровождаться обращениями программы к "хвосту" своего EXE-файла. Если программа читает все упакованные файлы в память сразу, такое обращение будет выполнено во время запуска программы. В обеих случаях эти вызовы вполне "уязвимы" для точек останова.
3. Наиболее уязвимы те вызовы, которые передаются в систему в неизменном виде (т.е. обращения к файлам, не находящимся внутри упакованного "хранилища") - установка брейкпойнтов на функции работы с файлами позволяет найти их без особого труда, после чего можно добраться и до точек входа и выхода в защитные процедуры-обработчики системных вызовов.
4. В памяти почти наверняка будут застревать куски спрятанных файлов, и если Вы знаете, какая информация в этих файлах может оказаться, Вы можете попытаться найти эти куски памяти и попытаться выяснить, каким образом эти куски там появляются (например, при помощи поиска ссылок на начала таких кусков или установкой брейкпойнтов на запись в память).

Все эти внедрения в чужой процесс, поиски переходников и прочее вполне способны повергнуть в шок начинающего. Это не страшно - даже если Вы никогда не напишете собственный API-шпион, знания о том, как они работают, вполне могут Вам пригодиться в дальнейшем. Как Вы уже догадались, написание приложений, перехватывающих системные вызовы - занятие далеко не самое простое и требующее определенных знаний и навыков. С другой стороны, большинство API-шпионов ограничиваются лишь ведением журнала системных вызовов и не позволяют активно вмешиваться в работу программы. Для начинающего крэкера, у которого есть желание перехватить какой-нибудь системный вызов, но нет опыта в системном программировании, это звучит как приговор. Но ведь так хочется иногда не только подсмотреть, откуда взялись те или иные

параметры, но еще и поменять их "на лету", если они Вам чем-то не понравились…

Я уже говорил, из любой безвыходной ситуации существует как минимум два выхода. Этот афоризм верен и в нашем случае, однако от Вас все же потребуются определенные навыки в программировании. Итак, наша задача - вклинить свой код между системной DLL и программой, которая ее вызывает. Одна из первых идей, которые приходят в голову, заключается в том, чтобы вместо "родной" DLL подсунуть свою собственную, которая содержала бы функции с такими же именами, что и "настоящая". Эта библиотека должна помимо вызова "родных" функций из оригинальной библиотеки выполнять еще и те операции, которые Вы в нее заложите. А уж в собственной DLL Вы вольны запрограммировать все, что угодно - от сбора статистики вызовов до анализа и подмены параметров функций. Такой подход к перехвату вызовов из DLL, основанный на подмене оригинальных библиотек, называется DLL wrapping'ом. Чисто технически создание подменной DLL выполняется следующим образом:

1. Необходимо получить список всех функций (в том числе и тех, которые не имеют имен, а только ординалы), экспортируемых той DLL, которую Вы планируете подменить. Если Вы уверены, что программа не импортирует функции этой DLL при помощи GetProcAddress или каким-либо более изощренным способом, Вы можете обойтись и списком функций, импортируемых подопытной программой из DLL. Получить такой список можно, к примеру, при помощи утилиты DUMPBIN.
2. Изготовить на основе этого списка болванку будущей подменной DLL. В простейшем случае эта болванка будет выглядеть как набор одинаковых кусков. Поскольку болванка имеет регулярную структуру, ее можно сгенерировать автоматически. Для простейшего случая, когда нужно перехватить одну лишь функцию MessageBoxA из user32.dll, наша подменная библиотека, написанная на MASM, будет выглядеть примерно так:

```
PUBLIC MessageBoxA

.data:
IsLoaded dd FALSE
aMessageBoxA dd 0
huser32 dd 0
u32 db "e:\Windows\System32\user32.dll",0
nMessageBoxA db "MessageBoxA",0

.code:

DllEntry proc hInstance:HINSTANCE, reason:DWORD, reserved1:DWORD
        mov  eax,TRUE
        ret
DllEntry Endp

CheckAndImport proc

 .if !IsLoaded
  mov IsLoaded,TRUE
  invoke LoadLibrary,ADDR u32
  mov huser32,eax
  invoke GetProcAddress,huser32,ADDR nMessageBoxA
  mov aMessageBoxA,eax
 .endif
 ret

CheckAndImport endp

MessageBoxA::
; Адрес метки MessageBoxA и прочих аналогичных нужно включить в таблицу экспорта
; при помощи директив
; LIBRARY user32
; EXPORTS MessageBoxA,
; помещенных в def-файл

 invoke CheckAndImport

; Здесь Вы можете разместить код, исполняемый перед вызовом перехватываемой функции
```

```
 ; Например, поменять местами заголовок и текст сообщения,
; как это сделано ниже
mov eax,[esp+8]
xchg [esp+12],eax
xchg [esp+8],eax

 mov eax, MessageBoxA_
 jmp eax
```

Процедура CheckAndImport выполняет важную функцию - во время первого вызова любой из функций она подгружает настоящую user32.dll, получает адрес функции MessageBoxA и помещает этот адрес в переменную aMessageBoxA. Если Вам известно, какая из функций DLL будет вызвана первой, можно сократить код, разместив вызов функции CheckAndImport только в этой функции. Вообще, внешне этот код выглядит довольно тяжеловесно - экспорт меток, специфический способ обращения к параметрам вызова через смещение относительно значения ESP, динамическая загрузка библиотек - и тут же отсутствие возможности выполнить собственный код после вызова MessageBox. Причина тяжеловесности проста: этот код представляет собой максимально упрощенную адаптацию реально использовавшейся мной подменной DLL. Что же касается мнимой невозможности выполнить собственный код после вызова MessageBoxA, то эту проблему проще всего решить при помощи подмены лежащего на стеке адреса возврата: нужно подправить его таким образом, чтобы после выполнения команды `jmp имя_функции` возврат выполнялся не в программу, а на следующую за jmp команду. Разумеется, старый адрес возврата тоже надо где-то сохранять - он Вам понадобится, чтобы вернуть управление программе. Если Вам известно количество параметров, помещаемых на стек при вызове, и их размерность, никаких сложностей с подменой адреса возврата не возникнет. Но вот если в процедуру передается неизвестное заранее количество параметров (такое, в частности, возможно в программах, написанных на C), общего решения этой проблемы не существует, так что Вам придется действовать по обстоятельствам и изобретать метод определения количества параметров на стеке самостоятельно.

Но почему я вдруг отклонился от темы этой главы и уделил столько времени технике подмены DLL? Причина в том, что DLL wrapping нередко используется совместно с патчингом исполняемого файла. Внимательно посмотрев на текст подменной DLL, Вы увидите, что загрузка user32.dll производится с явным указанием расположения этой библиотеки. Однако даже если скомпилировать эту DLL с именем "user32.dll" и положить рядом с подопытной программой, единственным результатом которой является вывод MessageBox'a с неким сообщением, Вы все равно не добьетесь желаемого результата - программа вызовет эту функцию прямиком из системной библиотеки, проигнорировав Вашу приманку. Что же делать?

Самый простой и доступный выход заключается в том, чтобы переименовать нашу подменную библиотеку в user33.dll, а потом забраться в исполняемый файл шестнадцатеричным редактором и там поменять имя импортируемой библиотеки аналогичным же образом. Теперь наша программа будет вместо стандартной библиотеки Windows подгружать нашу user33.dll и вызывать MessageBoxA именно из нее. Теоретически можно было поступить и несколько иначе - переименовать системную библиотеку user32.dll в user33.dll, а на место user32.dll положить нашу подменную библиотеку (разумеется, исправив путь к "настоящей" динамически загружаемой библиотеке), но на практике проделывать фокусы с заменой системных библиотек крайне нежелательно. Хотя если речь идет не о системной библиотеке, то действительно можно обойтись одним лишь переименованием "настоящей" DLL и помещением на ее место "поддельной".

Изучая перехват системных вызовов, мы вторглись в высокие сферы патчинга процессов, и теперь настало время всерьез углубиться эту тему. Хотя "в общее пользование" всевозможные launcher'ы, правящие файлы прямо в памяти, попадают сравнительно редко, для домашнего пользования техники патчинга процессов более чем актуальны. Если у Вас есть сжатая упаковщиком EXE программа, но нет желания заниматься ее распаковкой либо Вы просто хотите поэкспериментировать над программой, проверяющей контрольную сумму собственного файла,

патчинг кода "на лету" - это весьма эффективный способ добиться желаемого с минимальными усилиями.

Модификация кода программы в памяти может выполняться двумя путями: записью данных в адресное пространство процесса извне или же внедрением в адресное пространство процесса собственного кода, который уже будет работать внутри подопытного процесса и выполнять необходимые действия по патчингу. В подавляющем большинстве случаев вторжение в чужое адресное пространство извне осуществляется при помощи последовательности вызовов VirtualProtectEx-WriteProcessMemory, хотя возможны и более сложные варианты с выходом в нулевое кольцо (правда, после того, как линейка Windows 9x начала утрачивать актуальность, простые в осуществлении способы выхода в Ring0 остались не у дел). Для того, чтобы добраться до процесса, Вам почти наверняка потребуется его дескриптор (он же "хэндл" - "handle"). Если запуск программы и патчинг ее процесса выполняется одной и той же программой, особых сложностей не возникает: функция CreateProcess[Ex] возвращает хэндл порождаемого ей процесса. Но это не единственный подход к добыче желанного дескриптора - до него также можно добраться через получение идентификатора процесса, породившего окно (при помощи функции GetWindowThreadProcessId) либо через анализ "снимка" всех процессов, полученных при помощи функций CreateToolhelp32Snapshot, Process32First и Process32Next.

Правка кода путем предварительного внедрения в адресное пространство подопытного процесса применяется значительно реже, однако эти тоже знания могут Вам понадобиться. Наиболее распространенными способами попасть в чужое адресное пространство являются: использование хуков, запуск программы в режиме отладки и создание потока при помощи функции CreateRemoteThread (этот прием применим только под Windows линейки NT). Однако ничто не ограничивает Ваше творчество - Вы вольны комбинировать различные приемы для достижения наилучшего эффекта, например, пропатчить программу в районе точки входа так, чтобы происходила загрузка некой DLL и вызов функции, которая восстановит испорченные патчингом байты, создаст дополнительный поток и затем вернет управление основной программе.

Выше мы рассматривали всевозможные приемы, позволяющие, образно говоря, залезать в чужое адресное пространство через окно. Но иногда разработчики программ предоставляют бесценную возможность попасть внутрь их детища с "парадного входа", не таясь. И возможность эта - "родимое пятно" практически всех программ, использующих широко распространенный механизм расширения возможностей программы за счет плагинов. Плагин обычно представляет собой самую обычную DLL, построенную в соответствии с теми или иными стандартами, задаваемыми разработчиком программы. Однако что именно делает плагин, программу обычно не волнует - он может как выполнять штатные функции по обработке изображения, звука или чего-нибудь еще, так и быть "троянским конем", созданным с одной лишь целью - в момент первого же обращения к нему исправить внутри программы-"хозяина" несколько байт. Поскольку Ваш плагин будет "жить" внутри подопытной программы, для записи данных в адресное пространство Вам не нужны будут никакие ухищрения, достаточно всего лишь поместить правильные значения по известным адресам при помощи самой обычной команды mov. Однако как добиться, чтобы Ваш код выполнился как можно быстрее? Точные сведения могут быть получены лишь через эксперименты, но приблизительное направление поиска следующее: многие программы после загрузки плагина первым делом "спрашивают" свежезагруженный модуль о его названии, назначении, поддерживаемых функциях и прочих подобных параметрах. А поскольку поиск и загрузка плагинов нередко выполняется сразу после старта программы (а если очень повезет - и до срабатывания защитных механизмов или параллельно с их работой), у Вас есть хорошие шансы пропатчить программу уже на первых стадиях ее работы.

Общим требованием при патчинге программы в памяти является полная "неподвижность" программы, чего проще всего достигнуть, отправив все потоки модифицируемого процесса в состояние "замороженности" вызовом функции SuspendThread или при помощи старого фокуса

"MySelf: jmp MySelf". Связано это с тем, что модификация кода с точки зрения программы должна производиться одномоментно, чтобы во время патчинга программа случайно не попыталась выполнить не до конца модифицированный код (результатом чего почти наверняка будет сбой). Из этого правила есть важное следствие: если патчинг выполняется одной ассемблерной командой (а одной командой можно переписать один, два, четыре, а при большом желании - 8 или 10 байт), то такая модификация будет одномоментной и потому вполне допустимой. К примеру, "волшебная" комбинация MySelf: jmp MySelf (в шестнадцатеричном виде этот код выглядит как EB FE) как паз является двухбайтной. Однако если Вы выполняете патчинг чужого процесса при помощи функций WinAPI, Вы не можете наверняка знать, блоками какого размера будет осуществляться запись в чужое адресное пространство, а потому пытаться переписывать более одного байта без "заморозки" процесса не стоит.

Самым старым подходом патчинга процессов под Win32 была загрузка программы с флагом CREATE_SUSPENDED с последующей записью в ее адресное пространство и "размораживанием" главного потока, однако применение его ограничено - очевидно, что этот метод практически неприменим в том случае, если программа упакована. Но что делать, если Вы все же столкнулись с упакованной программой? Очевидно, что в момент запуска патчить ее бесполезно, поскольку тело программы все еще не декодировано. Стало быть, патчер должен дождаться, пока программа будет распакована, и, как только этот знаменательный момент настанет, обрушиться на программу и привести в надлежащее состояние все "неправильные" байты. Осталось лишь выяснить, каким образом патчер может узнать о наступлении этого самого момента.

Самым оригинальным подходом к проблеме отличился, пожалуй, довольно старый патчер процессов, выпущенный T3X: все обязанности по отслеживанию загрузки эта утилитка возлагала на конечного пользователя. Созданный при помощи этой утилиты launcher просто запускал нужную программу, а затем выводил MessageBox с сообщением вроде "Нажми OK, когда программа загрузится". От пользователя, соответственно, требовалось кликнуть по кнопке OK, когда он сочтет, что подопытная программа уже загрузилась и распаковалась. Несмотря на откровенный примитивизм, этот патчер вполне соответствовал духу популярных защит того времени и позволял решать насущные задачи.

Естественно, следующим шагом в определении наилучшего момента для патчинга стала привязка к появлению на экране какого-либо окна, создаваемого модифицируемой программой. Поскольку большинство Windows-приложений являются оконными и при запуске создают окно, в котором предстоит работать пользователю, появление такого окна может служить недвусмысленным указанием на то, что распаковка программы уже завершилась и потому можно приступать к патчингу. Одним из первых такой патчер выпустил yoda, причем его патчер позволял не только обнаруживать появление окна, но и принудительно его закрывать (что было полезным для борьбы с nag screen'ами). Однако этот инструмент имел ряд недостатков, в частности - требовал указания полного заголовка окна, что не позволяло работать с окнами с изменяющимся заголовком. А поскольку окна со счетчиками оставшихся дней "испытательного срока" встречались в программах все чаще и чаще, такая ситуация сподвигла меня на написание собственного инструмента - интерпретатора скриптов с механизмом поиска окон по содержащейся в заголовке окна подстроке, возможностью имитировать нажатие кнопок в окне, активировать отключенные управляющие элементы и т.п. Однако и этот инструмент не во всех случаях способен решить проблему своевременного патчинга: защита может быть устроена таким образом, что в момент появления окна будет уже поздно что-либо предпринимать. Кроме того, нельзя сбрасывать со счетов безоконные программы, которые хоть и редко, но все же встречаются в мире Win32-приложений. И тогда приходится применять старый, весьма расточительный с точки зрения расхода ресурсов процессора, но проверенный временем и практически безотказный прием.

Традиционно проблема определения момента распаковки программы решается следующим образом: подопытная программа запускается при помощи launcher'a с флагом

CREATE_SUSPENDED, этому процессу и его главному потоку назначается наименьший возможный приоритет, после чего главный поток "размораживается" функцией ResumeThread. Программа начинает распаковываться, но делает это очень медленно: во-первых, по причине минимального приоритета, а, во-вторых, потому что launcher в это время постоянно считывает содержимое байтов, которые предполагается пропатчить и сравнивает их с эталонными значениями, заранее добытыми из "живой" программы при помощи дампера или отладчика вроде SoftIce. Цель этих действий заключается в следующем: исправление необходимо провести как можно раньше после того, как подвергаемые патчингу участки будут распакованы. Как только содержимое нужных ячеек памяти совпадет с эталонами, программу можно считать готовой к внесению модификаций. В этот момент патчер стопорит исполнение программы и модифицирует код, а затем возвращает в исходное состояние приоритеты, размораживает все потоки и позволить программе выполняться дальше как ни в чем не бывало.

Блок, выполняющий постоянное сканирование, желательно оптимизировать по скорости настолько, насколько это возможно: если наш launcher не успеет внести в программу изменения до того, как управление будет передано на изменяемые участки, всю операция по патчингу можно считать проваленной. Именно для того, чтобы снизить вероятность "слишком быстрого" запуска программы и увеличить промежуток времени, пригодный для патчинга, мы и понижаем приоритет программы, отбирая у нее кванты процессорного времени в пользу нашего launcher'а и других программ. Из всего вышесказанного следует интересный практический вывод, который, к сожалению, не учитывается в большинстве существующих патчеров процессов: если патч предполагается использовать на мультипроцессорной системе, Ваш launcher и подопытную программу (или хотя бы ее главный поток) лучше исполнять на одном и том же процессоре. Добиться этого под Windows NT можно при помощи функций SetProcessAffinityMask, SetThreadAffinityMask и GetProcessAffinityMask.

Дочитав предыдущий абзац, Вы могли задаться вопросом, насколько вообще эффективно использование API'шных функций для чтения-записи в адресное пространство чужого процесса для наших целей и нельзя ли воспользоваться каким-либо более "прямым" методом работы в чужом адресном пространстве. Такие вопросы вполне уместны: операции по "залезанию" в чужое адресное пространство вообще требуют значительных накладных расходов, из-за чего функция ReadProcessMemory действительно работает не слишком быстро (хотя обычно и такой скорости бывает более чем достаточно).

На этом наш экскурс в теорию и методологию патчинга можно считать законченным. Думаю, Вы достаточно разобрались в основах самой важной и самой употребительной крэкерской операции. А стало быть, подходит к логическому завершению и эта работа. И единственная область, которая до настоящего момента мною почти не затрагивалась - это технологии будущего, "крэкинг завтрашнего дня". А потому темой заключительной главы будут подходы, только-только утверждающиеся на крэкинг-сцене, а также идеи, которые пока еще никто не осуществил.

Теоретические основы крэкинга: Глава 13. Заключение

Автор: CyberManiac

Дата: 2005-06-27

> *Не было начала - не было конца…*
> *Е. Летов*

В завершение этой работы мне хотелось бы рассказать о технологиях, которые вошли в арсенал крэкеров сравнительно недавно, и даже о таких , которые еще только ожидают своей реализации. Крэкинг - бурно развивающаяся область информационных технологий, в которой едва ли не ежедневно появляются заслуживающие пристального внимания идеи и инструменты, и может случиться так, что когда Вы будете читать эти строки, некоторые из идей, о которых я расскажу, уже войдут в практику. За полтора года, прошедшие с того момента, как я взялся за написание этого цикла статей, такое уже случалось, причем не единожды - что ж, тем интереснее будет понаблюдать, как благие пожелания воплощаются в жизнь.

Давайте попробуем окинуть мысленным взором ситуацию, царящую на крэкинг-сцене и вокруг нее, так сказать, с высоты птичьего полета. Первое, что мы увидим - это многомиллионную армию пользователей всевозможных генераторов ключей, патчей, серийников, launcher'ов и прочих плодов материальной крэкинг-культуры. Вся эта огромная человеческая масса непрестанно движется, находясь в вечном поиске свежих версий своего любимого софта, после чего неизменно наступает вторая стадия - поиск новых крэков к передовым версиям программ. Неважно, что все они говорят на разных языках и любят разные программы - всех их сжигает одно неистовое желание. И разносится тысячекратным эхом по бесчисленным форумам и конференциям молитва исстрадавшегося пользователя: "Дайте Кряк!" Кто поможет этим несчастным?

Впрочем, иногда нужный крэк находится: крэкерские группы и просто собиратели серийников исправно снабжают общественность средствами нетрадиционной регистрации программ. И все-таки большинство крэкеров - одиночки, не ищущие громкой славы, и потому их релизы редко выходят за рамки круга личных знакомых. Но и у крэкеров жизнь - не сахар. Чем популярнее платформа - тем больше под нее программ, а чем больше программ - тем больше среди них программ защищенных. Да и сами защиты отнюдь не стоят на месте: если в конце 90-х годов упакованная программа под Win32 была редким гостем на "операционном столе", то сейчас сжатый EXE - скорее правило, чем исключение. Хорошо, если какая-нибудь добрая душа поделится с обществом распаковщиком, да этот распаковщик еще окажется работающим. А если не поделится или не окажется? Тогда одна дорога - дампить, править, восстанавливать или же сооружать launcher. И все же рано или поздно приходит понимание того, что как ни старайся, все программы не переломать. Так что если какую-то программу все еще не сломали, это отнюдь не повод превозносить сложность и надежность ее защиты. Возможно, просто эта программка не попалась на глаза умелому крэкеру. А может случиться и так, что программу эту сломали, причем не один раз, но исключительно для домашнего пользования. Но никто об этом не узнает.

Очевидно, что разрыв между желаниями широкой пользовательской общественности и возможностями крэкеров будет тем шире, чем больше программного обеспечения будут выпускать разработчики (собственно, единственный смысл установки защиты в наше время не в том, чтобы "не смогли сломать", а чтобы не ломали все, кому не лень, т.к. "любая программа, которую можно запустить, может быть взломана"). Каким образом можно переломить эту тенденцию? На ближайшую перспективу выход видится в том, чтобы дать квалифицированному пользователю возможность решить хотя часть его проблем самостоятельно. То, что пользователь, вооруженный

подходящим программным обеспечением, способен сделать некоторые вещи, которые ранее считалось исключительно прерогативой "ребят с отладчиками", показала программа GameWizard и ее многочисленные клоны, предназначенные для "взлома" игр. Хотя с высот "чистого искусства" такое достижение кажется совершенно несерьезным, не стоит забывать, что совсем недавно проблема бесконечной жизни решалась лишь двумя способами - с помощью "знакомого хакера" или никак. Поэтому не стоит запираться в "башне из слоновой кости", оставляя пользователей наедине с их бедами, чтобы в итоге оказаться на обочине прогресса, но есть смысл заняться созданием инструментов, которые могут реально помочь широкому кругу пользователей (тем более, что в этом кругу рано или поздно оказывается и сам крэкер).

Многообещающим направлением видится создание программ, позволяющих пользователю самостоятельно устранять проблемы, связанные с ограничениями в программах. Проще говоря, обходить всевозможные триальные механизмы, причем по возможности - в полностью автоматическом режиме. "Первыми ласточками" в этой области были многочисленные утилиты, менявшие системное время перед запуском программы и через некоторое время возвращавшие его обратно. По вполне понятным причинам такой подход не всегда приемлем, но до недавнего времени ничего другого не предлагалось. В теории одним из путей, позволяющих "двигать" время внутри одной программы, не затрагивая все остальные, является перехват функций Windows API, так или иначе возвращающих локальное или системное время, и модификация возвращаемых значений, возвращаемых этими функциями. Однако путь от теории к практике оказался неблизким: долгое время эта идея не получала практической реализации, по крайней мере, готовые программные продукты, пригодные для использования рядовым пользователем, автору не встречались. И лишь сравнительно недавно появилась программа Hall of the Mountain King, которая позволяет не только менять значение "внутренних часов" для любого процесса, но и "закольцовывать" течение времени внутри программ.

Весьма многообещающе выглядят и инструменты, которые позволяют скрывать и защищать те или иные ключи реестра от доступа из определенных процессов, используя технику перехвата вызовов WinAPI. В настоящее время автору не известны программы такого рода, которые могли бы использоваться для борьбы с триальными ограничениями, но сама по себе эта идея выглядит заманчиво: запретив, к примеру, чтение из ключей HKEY_LOCAL_MACHINE\Software\Classes\CLSID и HKEY_CURRENT_USER\Software\ASProtect для программ, защищенных ASProtect, можно добиться того, что защита "не увидит" собственные триальные метки и будет считать каждый запуск первым.

Если довести идею "учета и контроля", на которой основаны две описанных выше технологии, до логического завершения, мы в итоге придем к мысли о создании "firewall'а для системных вызовов", то есть программы, надзирающей за обращениями к функциям WinAPI, ведущей журнал этих вызовов и блокирующей либо корректирующей "неправильные" обращения к системе. Максимальную гибкость такого инструмента можно было бы обеспечить встраиванием быстрого скрипт-интерпретатора, при помощи которого пользователь мог бы программировать логику работы с системными вызовами. Возможности, предоставляемые такой программой, совмещающей в себе возможности API-шпиона и DLL-враппера, было бы трудно переоценить. К примеру, вместо того, чтобы долго выламывать nag screen из строптивой программы (почти наверняка столкнувшись при этом с необходимостью ее распаковки), можно было бы просто запустить программу в режиме снятия лога системных вызовов, чтобы понять, каким образом формируется и отображается nag screen. После чего, основываясь на полученной информации, выяснить в какой точке программы производится вывод nag screen'а и написать скрипт, который бы отказывал этому вызову в праве на создание окна, но при этом "пропускал" все остальные. Подобным же образом можно было бы неограниченно продлять триальные сроки, "доработав" функции работы с реестром так, чтобы в действительности запись в некоторые ключи реестра не производилась, но при этом подопытная программа пребывала в полной уверенности, что триальные метки успешно расставлены. Нужно

отметить, что такая программа будет уже не орудием исключительно крэкинга, но инструментом двойного назначения, который может быть использован для обнаружения утечек ресурсов или как средство автоматизации.

Другой подход к решению проблемы триалов, окончательно оформившийся сравнительно недавно, заключается в автоматизации поиска и удаления "меток", по которым программы определяют количество запусков или промежуток времени, оставшийся до истечения срока пробного использования. О технологиях выявления таких меток мы уже говорили в третьей главе, однако там делался упор на "ручную работу" по поиску лишних ключей в реестре и аккуратное с ними обращение, чего весьма трудно добиться при работе с большим количеством защищенных программ. Такой подход неприемлем для рядового пользователя, которого совсем не прельщает необходимость перед каждым запуском делать снимки реестра и потом долго их сравнивать, размышляя о том, за какое из изменений в реестре ответственна защита. Современный стиль жизни таков, что любая проблема должна решаться "в два клика", без многочасовых медитаций над снимками реестра и отчетами RegMon'a. Но есть ли пути к достижению этого идеала? Разумеется, есть! Широкое распространение навесных защит, реализующих триальные ограничения, привело к тому, что способы нанесения триальных меток тоже "унифицировались". Каждый тип навесных защит имеет свои собственные "предпочтения" в том, где эти метки создавать и каким образом их генерировать. Следовательно, если собрать достаточно большую коллекцию программ, защищенных одним и тем же инструментом, и как следует погонять их во всех возможных режимах, можно собрать некоторую статистику. К примеру, ASProtect традиционно мусорит в ветке реестра HKEY_LOCAL_MACHINE\Software\Classes\CLSID, причем старые версии этой защиты создавали ключи с одним-единственным текстовым значением длиной 4, 8 или 16 байт; главная надежда разработчика, видимо, заключалась в том, что огромное количество ключей в этой ветке помешает найти среди них "лишние". Как показала практика, это мнение было глубоко ошибочным и более новые версии этой же защиты пытаются имитировать полезные ключи гораздо более тщательно, что, впрочем, тоже не слишком хорошо помогает, потому что исследователи защит тоже не сидят сложа руки.

Изучив достаточное количество одинаково защищенных программ и поразмыслив над собранной информацией можно попытаться выделить ключевые признаки, позволяющие отличать "защитные" ключи от тех, которые выполняют более полезные функции и, соответственно, удалить первые. Возможных признаков, отличающих "праведные" ключи и файлы от "неправедных" - неисчислимое множество: это может быть и тип хранимых данных, и их размер, и наличие заведомо бессмысленных комбинаций символов там, где должен быть текст, и "ошибочные" ссылки на файлы и другие ключи реестра... Готовое решение здесь, как водится, заранее предложить невозможно, и если Вы возьметесь за создание инструмента для очистки систмы от триальных меток, Вам придется немало поломать голову над вычленением отличительных признаков "ненужных" данных и изобретением хорошего алгоритма очистки системы от этой информации. Идеальным объектом для получения такой информации является сама программа, при помощи которой устанавливается защита: одно лишь чтение документации может подсказать верное направление для исследований, не говоря уже о возможности штамповать защищенные программы-"пустышки" для экспериментов в любом количестве. Ну и, разумеется, изучение интерфейса программы (даже если самая интересная функциональность заблокирована) способно многое рассказать о возможностях и особенностях исследуемой защиты.

Основываясь на опыте разработки одной из таких программ-очистителей реестра, я сделал вывод о том, что для инструментов подобного рода важно соблюдение принципа максимального разделения интерфейсной части и функциональной. Именно "монолитность" Die, ASProtect, Die! и привела к тому, что после некоторого момента эту программу было бы проще переписать "с нуля", чем поддерживать, основываясь на исходной идеологии хранения нескольких пользовательских интерфейсов и алгоритмов поиска в одном исполняемом файле. Изложение проекта "идеального

очистичеля реестра" слишком далеко увело бы нас от основной темы, поэтому я ограничусь лишь несколькими тезисами:

• **Разделение пользовательского интерфейса и функциональной части программы.** Интерфейсная часть должна представлять пользователю максимально удобные средства для выбора алгоритмов поиска ключей, ввода параметров поиска, просмотра результатов работы функциональной части, удаления найденных триальных меток и т.п. Функциональная часть должна лишь принимать от интерфейсной части введенные пользователем параметры, выполнять поиск следов защиты и возвращать интерфейсной части список подозрительных объектов, судьбу которых предстоит решать пользователю.

• **Реализация функциональной части программы в максимально "легком" (т.е. небольшом по размеру и удобном для модификации) виде.** Мир программных защит весьма динамичен, и не стоит надеяться, что единожды "освоив" какую-либо защитную схему, можно до скончания века пользоваться плодами своих усилий. В еще большей мере это относится к созданию программ, ориентированных на полную автоматизацию обхода защит: если действия крэкеров для разработчика есть нечто неприятное, но неизбежное, то самое существование инструментов, дающих рядовому пользователю власть над триальными ограничениями, воспринимается как покушение на основы мироздания. Когда разработчик защит видит, что его изделие в достаточной мере "освоено", ему не остается ничего иного, как выпустить новую версию своего продукта, неуязвимую для существующего антитриального софта. Следовательно, Ваши программы должны быть максимально легки для обновления и расширения, чего можно достичь через использование плагинов или интерпретатора сценариев. Преимущество плагинов, написанных на компилируемых языках, в скорости их работы; сила скриптов - в легкости модификации (в качестве тестера и соавтора скрипта может выступить даже квалифицированный пользователь) и минимальных размерах. С другой стороны, плагины в виде двоичных исполняемых модулей неудобны для доработки и распространения, а интерпретируемые скрипты проигрывают плагинам в быстродействии и к тому же весьма уязвимы для обратного инжиниринга (было бы крайне наивным полагать, что разработчики защит не анализируют инструменты "другой стороны", даже если в лицензионным соглашении такие действия явным образом запрещены).

• **Использование максимально унифицированных интерфейсов для обмена данными между частями программы.** Весьма вероятно, что программные модули, ориентированные на удаление меток разных типов защит, будут иметь и разные наборы пользовательских настроек, однако следует стремиться к созданию единого для всех типов защит способа передачи пользовательских настроек в программные модули, которые непосредственно будут искать "лишние" ключи. В любом случае, следует избегать включения в функциональную часть средств ведения диалога с пользователем: если планируется развитие и расширение программы, такой "симбиоз", скорее всего, станет препятствием, ограничивающим гибкость программы.

Однако "пользовательским" уровнем грядущие возможности отнюдь не исчерпываются. Уже сейчас можно видеть, как возвращаются в новом обличье старые, но до времени не актуальные идеи. Около десяти лет назад была весьма популярна концепция эмулирующих отладчиков, которые практически полностью имитировали работу центрального процессора (исключение делалось разве что для команд вызова прерываний) и позволяли исполнить программу так, как она исполнялась бы на настоящем процессоре, но при этом иметь абсолютный контроль над состоянием отлаживаемой программы. Следы работы такого отладчика было практически невозможно обнаружить, не прибегая к ухищрениям; многие приемы, направленные на нарушение стабильности отладчика, также оставались не у дел. Однако с наступлением эры многозадачных ОС этот род отладчиков как-то тихо канул в Лету. Долгие годы эмуляция была не в почете, однако некоторые признаки указывают на то, что в ближайшем будущем возможен ренессанс эмулирующих отладчиков. Эмуляция длинных кусков кода, не содержащего обращений к WinAPI, освоена довольно давно: одних только плагинов для Interactive Disassembler, позволяющих имитировать исполнение кода, насчитывается уже несколько штук. Главная проблема, ожидающая

своего решения - организация взаимодействия между эмулятором и вызовами "неудобных" функций системного API внутри эмулируемой программы (попробуйте поразмыслить над тем, как можно эффективно проэмулировать обращение к функции, порождающей новый поток или вызывающей callback-функцию, чтобы представить уровень сложности задачи).

Другой приметой времени следует признать все более и более широкое распространение виртуальных машин, более или менее точно воспроизводящих поведение настоящих, "железных" ЭВМ. Из программных продуктов этого рода, эмулирующих платформу x86, наибольшей популярностью пользуются Virtual PC и VMWare, причем VMWare существует не только в Windows-, но и в Linux-версии, что позволяет крэкеру не ограничивать себя в выборе основной операционной системы. Думаю, что многие уже оценили удобства, предоставляемые этим инструментом: отладку внутри гостевой ОС параллельно с работой в хост-системе, возможность в любой момент восстановить систему вместе со всеми необходимыми инструментами, защищенность хост-системы от действий деструктивного кода (некоторые разработчики все еще встраивают в свое ПО такие закладки, не считаясь с возможными последствиями) и другие полезные мелочи вроде возможности снятия скриншотов окна SoftIce. Однако потенциально виртуальные машины способны дать крэкеру гораздо больше, чем предлагают существующие инструменты. В первую очередь я имею в виду возможность интеграции отладчика с виртуальным процессором. Разработчикам SoftIce пришлось немало потрудиться, чтобы создать почти полностью "прозрачный" для системы отладчик, но даже этот замечательный инструмент имеет немало ограничений. Одни из них накладываются самим устройством процессора (например, невозможность установки более четырех аппаратных точек останова), другие заключаются отсутствии поддержки того или иного "железа", третьи кроются в недрах программного кода отладчика (проявляющаяся иногда нестабильность в работе, "дыры", позволяющие обнаружить присутствие SoftIce, невозможность просматривать тексты в национальных кодировках и т.п.). Корнем всех этих бед является то, что отладчик работает на той же самой программно-аппаратной системе, что и отлаживаемая программа. Вот если бы каким-то образом удалось вынести отладчик за пределы компьютера... Если подойти к решению этой проблемы "в лоб", мы придем к идее аппаратной приставки, берущей под контроль все процессы, протекающие внутри системного блока. И такой подход действительно иногда используется, но не для взлома, а для тестирования всевозможного встраиваемого ПО - от новых версий BIOS до управляющих программ бортовых ЭВМ космических спутников. Гонять такие "железки" (надо отметить, в настоящее время весьма не дешевые) исключительно в целях крэкинга - это, пожалуй, явный перебор. Но вот если реализовать такой комплекс исключительно программно, да подключить его к виртуальному же процессору не менее виртуального PC, то можно будет получать любую информацию о состоянии нашей эмулированной машины. Это было бы очень изящное и практичное решение: Вас не волновали бы ограничения на число отладочных регистров, не нужно было бы думать о том, поддерживается ли Ваша видеокарта отладчиком, а сам отладчик смог бы предоставить такие сервисные возможности, какие невозможны в SoftIce, "завязанном" на работу исключительно в текстовых режимах. Аналогичный же вариант можно было бы проделать и с прочим оборудованием, например, предоставив API для создания собственных программных модулей, эмулирующих внутренние и внешние устройства. Разумеется, это породило бы виток соревнования "брони и снаряда", и к бесчисленным антикрэкерским пособиям "как определить наличие отладчика" добавились бы изыскания о проверке обрудования, с которым работает программа, на виртуальность (а защиты с такими механизмами уже существуют, правда, направлены они в основном против эмуляторов CD-ROM), но тут уж ничего не поделаешь - таковы законы жанра. Собственно, отладчики, интегрированные с виртуальными машинами, не являются чем-то ранее невиданным: наиболее продвинутые эмуляторы машин линейки Spectrum, БК и Atari позволяют не только запускать программы, но и отлаживать их "по живому" со всеми удобствами. Возможно, в эпоху 128-разрядных ЭВМ эмуляторы PC обретут такие же возможности, но современному ПО в этой области пока похвалиться нечем.

Не стоит забывать и о многочисленной армии специализированных мобильных устройств. Лавинообразный рост количества таких аппаратов и расширение их функциональности позволяет предположить, что "мобильный крэкинг" в ближайшее время станет столь же актуален, как и классический, ориентированный на софт для персональных ЭВМ. И если "наладонники" достаточно мощны и сложны, чтобы позволять реализовывать сложные защиты, то во многих мобильных телефонах и смартфонах в настоящее время широко используется урезанный вариант языка Java. А там, где есть Java и прочие интерпретаторы байт-кода, всевозможные декомпиляторы приобретают просто убийственную эффективность: сравнительно низкая производительность и ограниченные возможности мобильных устройств сильно мешают использованию "замусоривателей" (obfuscator'ов) кода и интеграции защитных механизмов в код программы, а технологии декомпиляции Java-программ настолько хорошо отработаны, что позволяют получить листинг, идентичный исходному. Из всего этого, в свою очередь, следует исполнение вековечной мечты крэкеров, а именно - возможность декомпилировать программу в исходный текст на языке высокого уровня, внести в него любые исправления и с минимальными усилиями скомпилировать обратно без потери работоспособности. Кроме того, для многих популярных мобильных платформ существуют общедоступные эмуляторы и SDK, позволяющие исследовать и дорабатывать программы для мобильников даже не имея в наличии настоящего, "железного" телефона.

И, наконец, последние направление, обещающее огромные возможности, связано с появлением open-source ОС, совместимой с Windows. Если Вы уже пытались применить свои знания на практике, Вы не могли не заметить ограниченность либо нестабильность многих крэкерских инструментов. Зависания отладчиков, вызовы API, которые в упор не замечают API-шпионы, невозможность нормально снять дамп памяти - такова объективная реальность, с которой приходится сражаться крэкеру. Все это можно было бы списать на ограниченность и недостаточное качество используемого программного обеспечения, тем более, что авторы многих из этих программ изначально не ориентировались на потребности крэкеров и потому было бы странно требовать от них возможностей, которые не были заявлены в проекте. Однако если рассматривать вопрос с точки зрения архитектора, нам откроется, что "корень зла" лежит не в несовершенстве программ (хотя наличие такового несовершенства нельзя отрицать), но в способах, при помощи которых эти программы взаимодействуют с ОС. А взаимодействие это нередко осуществляется "с черного хода", при помощи приемов, работоспособность которых держится на многочисленных "авось". И в этом нет ничего удивительного: при разработке ОС штатные механизмы перехвата системных вызовов не были запроектированы в принципе, поэтому единственным методом удовлетворения возникших потребностей оказался метод "грязного хака" (который, впрочем, впоследствии был санкционирован самим производителем ОС). Образно говоря, система "ОС+отладчик" или "ОС+API-шпион" с высоты птичьего полета выглядит не как готический замок, в котором каждая башенка является частью единого целого, а как глинобитный сарай, прилепленный к стене суперсовременного небоскреба. Неудивительно, что прочность такой конструкции - совершенно никакая, и рушится она чаще всего именно в месте, где "небоскреб" соединяется с "сараем". Наиболее естественным выходом из сложившейся ситуации была бы интеграция средств наблюдения за взаимодействием между пользовательскими программами и операционной системой непосредственно в ядро ОС, но по вполне понятным причинам корпорация Microsoft на такой шаг не пойдет. Однако в последнее время появилось несколько весьма интересных проектов с доступными исходными текстами - от эмуляторов Windows (Wine/WineX/Cedega) до разработки полноценной "другой Windows" (проект ReactOS). Наличие исходных текстов позволяет встраивать непосредственно в ядро ОС практически любые механизмы слежения за пользовательскими программами и самой ОС, а также API для управления этими механизмами. Причем такие "усовершенствования" ОС совершенно необязательно внедрять в код жестко, их можно оформить как патчи, накладываемые на исходные тексты ядра и применяемые при необходимости. Возможности же, предоставляемые отладчиком или API-шпионом, интегрированным в ядро операционной системы, трудно переоценить.

Однако изменятся не только орудия крэкерского труда - изменится и сам крэкинг, и, изучая предмет, этот факт не стоит сбрасывать со счетов. Уже сейчас очевидны две тенденции: с одной стороны, продвижение наиболее простых крэкерских технологий в массы (о чем я говорил выше), и с другой стороны - нарастание специализации, сначала - в областях знаний, а затем - и во взломе конкретных продуктов. Десять-пятнадцать лет назад крэкер вынужден был становиться "мастером на все руки" и не только уметь разбираться в кодах чужих программ, но и обеспечивать себя необходимыми инструментами для этой деятельности - Интернета в современном его виде, откуда всегда можно выкачать свежий отладчик или распаковщик, тогда просто не существовало. Разумеется, такая ситуация сложилась не от хорошей жизни - в отсутствие Интернета свободный обмен информацией и, тем более, инструментами был крайне затруднителен - всевозможные BBS и "флоппинеты" мало чем способны были помочь, да и существовали далеко не везде. Разработчикам защит, впрочем, было не легче: "навесные" защиты не то, чтобы совсем отсутствовали, но были не слишком популярны, и в итоге каждая защищенная программа была единственной в своем роде, но с другой стороны набор защитных приемов был куда более ограничен по причине простоты процессоров и операционных систем того времени. Нынешняя ситуация отличается в корне: с одной стороны, количество программ выросло на порядки но с другой - сами авторы ПО обычно идут одним из двух проторенных путей: либо читают всевозможные руководства "как защитить программу" и затем более или менее успешно воспроизводят книжные схемы, либо покупают готовый продукт и "в два клика" прикручивают защиту, нимало не интересуясь, что они прилепили к своему коду. Следствием такой защитной стратегии стало то, что знание о том, как ломается та или иная типовая защита, является ключом не к единственной программе, но к множеству. Итогом всего этого становится ярко выраженная ориентация на изучение именно типовых, популярных защит (потому как это - способ минимальными силами "разобраться" с максимумом программ и ломать их конвейерным методом), а это есть ни что иное, как специализация. Впрочем, какой бы путь Вы для себя не выбрали, существуют области, в которых необходимо разбираться каждому, кто так или иначе собирается уделить время крэкингу, и на изучение именно этих вещей и стоит делать упор в первую очередь. Не стоит совершенно игнорировать и малопонятную или неактуальную информацию: даже если сейчас Вы не можете применить эти знания, постарайтесь усвоить их хотя бы в общих чертах - возможно, в будущем именно эти "заметки на полях памяти" лягут в основу изящного подхода к какой-нибудь проблеме. Многие из старых пособий по крэкингу сейчас видятся пыльным антикварiатом, поскольку те программы, которые в них описаны, давно и бесповоротно канули в прошлое. Но первое впечатление обманчиво: хотя коды из этих статей устарели, иные решения, приведенные в этих статьях, вызывают восхищение и сейчас. Помните, что не стоит зацикливаться на одних лишь крэкерских материалах; очень часто бывает полезно взглянуть на проблемы защиты и взлома с "другой стороны баррикад". Так уж исторически сложилось, что авторы защит не слишком охотно предоставляют информацию о своих изделиях, однако даже беглое изучение содержимого соответствующих руководств и форумов даст Вам представление, чем дышит "вероятный противник", какие идеи сейчас популярны в среде разработчиков ПО и - если повезет - в чем заключаются огрехи и недоработки популярных защитных схем. Знание о новом защитном приеме, кроме всего прочего, представляет собой отличный повод поразмыслить о том, как соответствующий код может быть идентифицирован в реальной программе и нейтрализован. Я уже давал этот совет в одной из первых глав, но повторю его еще раз в завершении: во всяком источнике информации нужно искать не столько приемы, сколько идеи, а знакомство с оригинальными идеями в свою очередь стимулирует способность находить решения самостоятельно. И какой бы Вы ни избрали путь - скромного разработчика инструментов, кующего "оружие Победы" или "бойца невидимого фронта", раскалывающего особо прочные защиты, потрошителя двадцатидолларовых утилит или мирного исследователя недр операционной системы - не бойтесь отойти от канонов и проявить неоправданную оригинальность. В конце-концов, изящные решения - это те нити, из которых соткано будущее. И это будущее обещает быть интересным - так почему бы не добавить в него что-то от себя?

CODE-RIP: искусство выдирания чужого кода - Часть I. Теория

Автор: Broken Sword

Дата: 2005-06-30

> Rip, гл. - разрезать, распарывать, рвать (одним быстрым движением),
> раскалывать (дрова);
> сленг - воровать, красть, грабить.
> Abbyy Lingvo

Аннотация Описываются основные принципы и методы выдирания чужого кода для последующего использования при написании своих программ. **Кому это нужно**

В этом разделе дано мое личное видение проблемы, поэтому прошу "в пианиста не стрелять".

При написании программы с нуля или добавлении новых функций в существующие продукты разработчики часто могут найти такие функции в разработках от других компаний (конкурентов). Времени на разработку своими средствами, конечно же, нет. Более того, существуют такие потаенные алгоритмы (конвертация экзотических форматов и т.п.), которые продаются только определенным фирмам и только за круглые суммы.

Что же делать? Выход из данной ситуации на самом деле один - украсть J.

В конце-концов, у любого найдутся свои буйные, шокирующие фантазии по использованию подобных грязных приемов в быту.

Теория

Знаете ли Вы что…

Одним из переводов каждого из слов, имеющих к крэкерской тематике непосредственное отношение - hack, rip и nag, является слово "кляча" :)

Итак, в статье попытаюсь описать основные идеи и принципы, заложенные в таком специфическом направлении искусства крекинга, как code-rip. Информацию в Интернете найти не удалось. Единственное, что выдаст пресловутый google на запрос "code rip" - некая программа по выдиранию html-кода с web-страниц. Однако, это совершенно из другой оперы и нам ни к чему.

Что, в конечном итоге, означает "рипнуть код"? В общем случае - создание из исполнимого модуля программы-жертвы библиотеки, экспортирующей необходимые взломщику функции.

Итак, минимальный набор инструментов, которым стоит обзавестись code-ripper-у:

• **Дизассемблер IDA**: в случае обычного взлома всегда можно обойтись и без дизассемблера, используя только отладчик, однако для rip-анья кода обойтись без дизасмера будет крайне проблематично, в некоторых случаях - практически невозможно.
• **Мощный текстовый редактор**: вот такого Вы еще точно нигде не встречали в описаниях инструментов ко взлому. Не удивляйтесь - в текстовом редакторе придется провести чуть ли не большую часть всего времени.
• **Отладчик Ollydbg**: в процессе rip-а гораздо удобнее SoftICE-а. Из раздела "Практика" станет понятно, почему.

Пожалуй, это все.

Процесс rip-анья кода можно условно разделить на пять этапов:

1. Оптимизация трудозатрат по времени

2. Дизассемблирование
3. Генерация файла с исходным кодом
4. Редактирование
5. Компиляция
6. Отладка

Для достижения реально работающего результата 3-6 этапы придется повторять N раз. В некоторых случаях N может перевалить за сотню.

Практика Приступим. Перед нами программа, вернее, ее исполнимый модуль, функции из которого нам так нужны для создания собственной библиотеки. **Оптимизация трудозатрат по времени**

Фактор времени играет первостепенную роль в rip-е. Чем большим набором знаний в самых различных областях (компиляция, редактирование текстов, отладка…) Вы обладаете, тем *быстрее* Вам удастся rip-нуть нужный код. Ситуация, когда несколько дней бьешься над решением проблемы, решаешь или не решаешь ее, и вдруг случайно обнаруживаешь обходной, минутный путь встречается здесь так часто, как нигде более.

Итак, вернемся к подготовке почвы к оптимизации по времени. Часто бывает так, что необходимые нам функции содержатся в программах от различных фирм. Наша первоочередная задача - выбрать ту программу, исполнимый модуль которой *меньше* по размеру. Также следует учесть размер исполнимого модуля в разных версиях одной и той же программы, он может отличаться в несколько раз. Естественно, нужные нам функции могут коренным образом измениться в новых версиях, поэтому это тоже нужно отслеживать - ведь, в конечном счете, ваш продукт должен составить конкуренцию на рынке.

Выбрав наименьший необходимый модуль можно приступать к его анализу. Обычно исполнимый модуль бывает запакован какой-нибудь гадостью. Распаковав его (рекомендую воспользоваться готовыми решениями) мы скармливаем его IDA-е.

Дизассемблирование

Перед дизассемблированием советую снять "галочку" с Create Stack Variables в Kernel Options 1, также в Processor Oprions выставьте опцию "Disassemble Zero Opcode Instructions".

После дизассемблирования Вам необходимо превратить все абсолютные смещения в относительные (для этого выделите весь код и нажмите "o", далее выставить Apply to all operands и снять галочку с Apply only if possible - спасибо Infern0). Это важно, особенно если программа была упакована. В некоторых случаях процесс может занять многие дни, т.к. придется пройтись буквально по каждой строке исходника в поисках этих самых абсолютных смещений.

Объясню, зачем это нужно. Дело в том, что процесс распаковки состоит из нехитрого дампа программы из памяти, с последующей корректировкой секции импорта и т.п. мелочами. Однако после dump-а все смещения внутри абсолютны, и валидны лишь в том случае, если база модуля соответствует базе сдампленного образа. Теперь внимание - мы собираемся компилировать код в DLL, при загрузке в память она никогда не ляжет на базу EXE (0x400000), поэтому все смещения окажутся недействительными. Мы, конечно, можем явно указать базу для DLL на этапе компиляции (ключ BASE), однако и в этом случае смещения окажутся невалидными - при компиляции MASM будет генерировать машинные коды для ассемблерных мнемоник такого размера, который во многих случаях не совпадает с оригинальными (как известно, одну и ту же команду можно представить разными машинными кодами, об этом the Svin лучше расскажет). Следовательно, все абсолютные смещения опять окажутся недействительными.

Одним словом, абсолютные смещения для rip-а не допустимы.

Есть один вариант, когда абсолютные смещения допустимы. Вместо DLL и LoadLibrary использовать CreateProcess и оригинальный EXE. Тут свои нюансы и в конечном итоге

трудозатраты такие же, как при создании DLL.

Итак, теперь важно не забыть избавиться от лишних секций. Как правило, это ресурсы и импорт. Они могут занимать до половины всего исходного кода, чем будут очень мешать при редактировании, а секция импорта вообще вызовет приступы эпилепсии у MASM-а. Избавление от сегментов происходит в окне Segments IDA-ы.

Следующий шаг - создание исходного файла (Produce asm-file).

После создания исходника наступает кульминационный момент. Реально существует два пути дальнейшего продвижения:

1. rip-ать только необходимые функции
2. rip-ать весь модуль целиком

Первый метод подразумевает copy/paste в текстовом редакторе кода нужных функций и, безусловно, с первого взгляда покажется более привлекательным - результирующий объем кода на порядок меньше, откуда следует целый букет преимуществ: легкость правки, скорость компиляции и т.д.

Под вторым я понимаю перекомпиляцию всего кода целиком, без разбору. У него, кажется, одни только недостатки - огромный размер полученного в результате кода (миллионы строк для модуля 5-6 Мб), сложность анализа, да и вообще - топорно как-то.

Однако не следует забывать о ЦЕЛЯХ, поставленных нами с самого начала: быстро и качественно получить возможность вызова чужих функций из своих программ. Так вот опытным путем установлено, что в большинстве случаев второй метод позволяет добиться желаемого результата на порядок БЫСТРЕЕ, чем первый. Более того - в некоторых случаях, первый метод вообще не применим. Объясню почему.

Рассмотрим простейший пример. Допустим, у нас есть небольшая функция test_func, которая в листинге IDA занимает пару сотен строк. Казалось бы, какие проблемы - copy/paste и брюки превращаются… Однако, первое что должен сделать настоящий code-ripper -посмотреть граф этой "небольшой" функции (chart of xrefs from) в IDA. Теперь засекайте время на генерацию этого графа. Если прошло более пяти секунд то можете остановить процесс - дальше ждать не имеет смысла. От полученного "черного пятна" не будет абсолютно никакого толку. Если Вам совершенно нечем заняться - можете считать, что Вы нашли занятие на ближайшие месяцы. Процесс перетаскивания "пятна" из общего листинга в отдельный - целый ритуал. Уже после сотой вложенной функции, перетащенной вручную, Вы наверняка вспомните про второй метод.

На этапе редактирования необходимо как минимум "не забыть" повыметать из кода все упоминания об истинном владельце (обычно каждая добропорядочная фирма норовит насовать таких меток в десятках мест). В редакторе ищем все эти метки и убираем, или, если совсем с совестью не дружим - меняем на свои.

Компиляция

Этап компиляции - отдельная песня. Здесь придется особенно тяжко потрудиться.

Приколов и нюансов встретиться немерянно, причем для каждой программы они свои. Например, если Вы собрались компилировать сгенеренный IDA-ой исходник MASM-ом, то Вам непременно понадобиться версия 8.0 (спасибо q_q и bogrus), хотя бы потому, что более ранние версии некорректно выводят номера строк с ошибками в файлах, содержащих более 65535 строк, а именно номера строк с ошибками нам и нужны больше всего. Тут следует учесть время на компиляцию - исходник в 1 млн. строк на 1700MHz компилируется больше 10 минут. С учетом того, что на этапе отладки Вы сразу же будете находить ошибки, и тут же перекомпилировать весь код заново, процесс rip-а может затянуться на долгие годы. Это одна из основных причин, по которой на первом этапе следует выбрать наименьший по размеру модуль.

С какими сложностями придется столкнуться? По большей части это инструкции, сгенерированные IDA-ой, которые не понимает MASM. Например, это почти все инструкции, содержащие префикс переопределения сегмента. Как поступать в данном случае? Проще всего - посмотреть в той же IDA-е в окне "Hex view" машинные коды "непонятных" команд, а затем воспользоваться функцией "Replace all" вашего любимого текстового редактора. Здесь главное не переусердствовать. Если инструкция выглядит слишком вызывающе (какой-нибудь bound esp, [ebp+6Ch]) то, скорее всего, IDA просто принял данные за код, в этом случае лучше вернуться в IDA, сделать "Undefine" участка подозрительного кода, преобразовать его в данные ("d"), и, как Вы уже догадались, повторить пункты 3-6.

В заключении этапа компиляции хотел еще описать небольшие хитрости по оптимизации размера rip-нутого кода. Например, как узнать какие его участки можно выкинуть? Опять же, берем отладчик, текстовый редактор и, прогоняя код в отладчике (с заходами во все процедуры), выделяем красным цветом весь код, по которому была совершена трассировка. Незакрашенные участки можно смело выкидывать :).

Отладка

Самый увлекательный этап. Здесь без OllyDbg просто не обойтись. На словах все выглядит очень просто - запускается две сессии отладки (именно поэтому SoftICE здесь нам не помощник), в одной из них мы прогоняем код оригинала, в другой, прогоняем код нашего rip-нутого кода. Т.е. в первой сессии мы трассируем код, и тут же в другой, после каждого шага, повторяем тот же самый шаг во второй сессии, после чего сравниваем содержимое регистров и других структур на соответствие оригиналу.

Для максимальной экономии времени предлагаю очевидную вещь: трассировать код оригинала без захода в процедуры, сверять возвращаемые результаты с процедурами из rip-нутого кода и повторять трассировку с заходом в процедуры только в случае отличающихся результатов. После захода в процедуру трассируем опять по тому же принципу - т.е. мы делаем обход всего дерева структуры программы.

На практике имеет место быть т.н. "лавинный" эффект - при исправлении ошибки в одном, "правильном" месте она исчезает в сотнях других. Структура любой компьютерной программы такова, что сотни различных процедур используют одну и ту же подпроцедуру, т.е. процесс поиска и исправления багов происходит лавинообразно; если на секунду представить, что каждая процедура использовала бы свои собственные подпроцедуры, но на отладку пришлось бы затратить пол жизни. Кстати, существует аналогичный эффект на этапе компиляции - MASM может выдать сотню ошибок, но стоит сделать 2-3 исправления и все они исчезают.

Подводные камни:

1. В новых версиях ПО в нужных взломщику функциях, вероятно, могут быть произведены изменения, исправлены ошибки. Поэтому, выбирая более старую и компактную версию Вы можете потерять в функциональности и надежности.
2. Часто бывает так (да практически по другому и не бывает), что "нужная функция" использует данные, инициализированные "другой функцией", причем эта "другая функция" норовит инициализировать их в самых неподходящих и неожиданных местах. Это означает, что мало просто передать управление на точку входа в "нужную функцию", необходимо выполнить ряд предварительных, инициализирующих действий (т.е. тащить в либу все инициализирующие функции).
3. Иногда нужная функция может также использовать функции из других библиотек. В этом случае придется либо а). Тащить нужные функции в конечный модуль из других библиотек, со всеми прелестями пунктов 3-6 для последних, либо б). Воспользоваться программой MoleBox (спасибо bogrus). Данная утиль может собрать все DLL и исполнимый файл в один модуль, дальше Вы

будете работать с ним как с одним целым.

Zashita

Как самому не стать жертвой коварных Джеков-потрошителей? (Джек-потрошитель - Jack The Ripper, прим. автора). Что можно предпринять для защиты своих кровных, рожденных в муках уникальных функций? Проблема заключается в том, что для защиты от rip-а стандартные методики не прокатят. Обычно, из-за лени, код рабочих функций программы защитой не затронут, т.е. проверки идут либо в функции при запуске программы либо в функциях, предшествующих вызову рабочих функций. В любом случае - rip-ер не будет даже вникать в логику работы таких защит - ему нужно лишь верно распознать начало рабочей функции и экспортировать ее при компиляции DLL.

Очевидно, что для защиты от rip-а нужно каким-то образом защищать каждую функцию в отдельности. Но как? Вообще функции должны быть все время зашифрованы, и в памяти и на диске и расшифровываться по мере вызова, по одной. Это, возможно, доставит rip-еру некоторые неудобства, но не более того.

В идеале самой защищенной функцией будет та, которая физически отсутствует как в исполнимом модуле, так и в оперативной памяти. Но возможно ли такое? Разумеется - например, разработка компании K-medulla позволяет физически вынести код функций в отдельное устройство. Само устройство - тема отдельной статьи ☺.

Во второй части мы рассмотрим практический пример применения приведенных методик.

Themida - обновлённый XProtector

Автор: CrystalDragon

Дата: 2005-11-15

Прошло уже больше года с момента написания статьи про XProtector версии 1.07, с тех пор многое поменялось и многое из того, что было описано уже устарело. Вот собственно поэтому и выходит новая статья по этому протектору. Стоит сказать сразу, что стать скорее описание того, как исследовать протектор а не руководство по распаковке.

При исследовании использовалась ОС Windows XP SP2, для изучения желательно иметь установленную WinXP или Win2003, дизассемблер IDA(f.e. 4.80), Import Reconstructor, HEX-редактор, PE Tools with Extreme Dumper Plugin(причём не для дампа самого процесса) а также некоторые специфические инструменты, такие как эмулирующий отладчик, плагин, удаляющий мусорный код, и простенькая утилита R0cmd, которая использовалась ещё год назад и была в исходниках к предыдущей статье. Подопытная программа - WinLicense Demo at 01.11.2005

Общие принципы работы защиты остались те же, используется драйвер, который открывает доступ к IDT любому процессу, передавшему запрос драйверу oreans.sys, причём неважно будет ли этот процесс защищённой программой или эксплоитом для получения прав администратора, проникающем в ring0 с помощью этого вспомогательного драйвера протектора. Именно свободное использование IDT не даёт свободно отлаживать защищённую программу, т.к. протектор использует отладочные прерывания под свои нужды, например размещает свой обработчик в int1 или int3 и расшифровывает лежащий впереди код находясь в нулевом кольце. Абсолютно также как и раньше используется перехват функций ядра с помощью SST, по тем же принципам формируются переходники на импортируемые функции и защищается пользовательский код макросами SDK. Раз так, значит снимать защиту будет точно также - с помощью подгружаемой в адресное пространство своей DLL. В сопровождающем архиве приведен исходник этой DLL, принцип её действия это перехват всевозможных полезных событий и ведение лога с указанием адресов и прочих параметров.

Антидамповая защита и антиотладка:

Выше уже упоминалось, что перехват функций для работы с памятью происходит аналогично XProtector'у, но вот есть одна проблема, при подгрузке оригинальной sst проискохит перезагрузка, именно это и надо устранить, чем сейчас и займёмся.

Логика такова, если происходит перезагрузка, причём сразу же после восстановления sst, то очевидно что протектор проверяет, на месте ли его обработчики функций. В таком случае проверка может быть в двух местах - в одном из многочисленных потоков, порождённых протектором, или во время переключения потоков либо процессов. Проверяется это легко, достаточно остановить все потоки, заморозив тем самым защищённый процесс. Делается это примерно так, внедрённая DLL собирает идентификаторы всех создаваемых потоков в таблицу, в любой момент можно вызвать SuspendThread в цикле, остановив их все, только последним надо останавливать текущий поток. После заморозки процесса восстановить SST не удаётся, следовательно этот вариант неверный и код проверки SST и IDT находится в ядре и вызывается при переключении контекстов. Для поиска применим такой оригинальный способ: делаем дампы главных модулей ядра Windows - ntoskrnl.exe(или аналога) и hal.dll дважды - до и после запуска протектора(вот именно для этого и нужен плагин extreme dumper). Если есть перехваты, они обнаружаться простым сравнением сравнением секций кода в дампах. Итак, сравнение дампов ntoskrnl:

B78C и др. - внутренности sst shadow, не интересно.

14938 - преобразовав это файловое смещение в RVA(для удобства можно использовать FLC из PE Tools, а ntoskrnl.exe загрузить в IDA по той базе, по которой она загружена у вас в памяти), выходим на функцию KeAttachProcess, которая используется для переключения в адресное пространство другого процесса и считается устаревшей, также не интересно.
31E4D - vsprintf, можно даже не разбираться сразу видно что это не то, что нужно.

смотрим hal.dll:

2278 - KfRaiseIrql, вот это уже очень интересно, т.к. эта функция используется ядром очень активно, в том числе и при переключении процессов. Смотрим, что же делает протектор при перехвате(адрес опять же проще поправить ImageBase в заголовке тому дампу где есть перехват и просто посмотреть адрес в IDA):

```
seg002:F8AAB000                 pusha
seg002:F8AAB001                 call    $+5
seg002:F8AAB006                 pop     ebp
seg002:F8AAB007                 sub     ebp, 4CAFD95h

 ;Обновление переменной, содержащей число тиков таймера(не интересно для нас)
seg002:F8AAB00D                 mov     eax, 80551280h  ; KeTickCount
seg002:F8AAB012                 mov     eax, [eax]
seg002:F8AAB014                 cmp     dword ptr [ebp+xvTickCount], 0
seg002:F8AAB01B                 jnz     short loc_F8AAB023
seg002:F8AAB01D                 mov     [ebp+xvTickCount], eax
seg002:F8AAB023
seg002:F8AAB023 loc_F8AAB023:                           ; CODE XREF: seg002:F8AAB01B
seg002:F8AAB023                 mov     ebx, [ebp+xvTickCount]
seg002:F8AAB029                 add     ebx, 78h ; 'x'
seg002:F8AAB02C                 cmp     eax, ebx
seg002:F8AAB02E                 jbe     _return_1
seg002:F8AAB034                 mov     [ebp+xvTickCount], eax

 ;DR7 - управляющий отладочный регистр, десятый бит обязан быть установлен;
 ;остальный биты сбрасываются - такие образом удаляются все 4 брэйкпоинта(bpm)
seg002:F8AAB03A                 mov     edx, 801BFAh
seg002:F8AAB03F                 sub     edx, 8017FAh
seg002:F8AAB045                 mov     dr7, edx        ; edx = 00000400

 ;загрузка в eax адреса IDT..
seg002:F8AAB048                 mov     eax, 0AB57712h
seg002:F8AAB04D                 sub     eax, 8AB18312h
seg002:F8AAB052                 add     eax, 0Ch        ; int1 descriptor
seg002:F8AAB055                 or      byte ptr [eax+1], 1100000b ; set DPL to 3

 ;читаем адрес обработчика
seg002:F8AAB059                 mov     ecx, cs:[eax]
seg002:F8AAB05C                 mov     cx, cs:[eax-4]  ; ecx = address of handler

 ;переход далее, если обработчик находится не в ядре
seg002:F8AAB061                 cmp     ecx, 0FFFF0000h
seg002:F8AAB067                 jnb     short _int3_check
seg002:F8AAB069                 cmp     ecx, 80000000h
seg002:F8AAB06F                 jb      short _int3_check

 ;а вот это интересно, если адрес обработчика находится в ядре, то он меняется на 0xFFFFFFFF,
 ;это значит что пока функция KfRaiseIrql перехвачена, восстановить IDT невозможно
seg002:F8AAB071                 push    ds
seg002:F8AAB072                 mov     ecx, 10h
seg002:F8AAB077                 db      66h
seg002:F8AAB077                 mov     ds, cx
seg002:F8AAB07A                 assume ds:nothing
seg002:F8AAB07A                 mov     cx, 0FFFFh
seg002:F8AAB07E                 mov     [eax-4], cx
seg002:F8AAB082                 mov     [eax+2], cx     ; set handler address to 0xFFFFFFFF
seg002:F8AAB086                 pop     ds
seg002:F8AAB087                 assume ds:seg003

 ;далее аналогично обрабатывается ещё одно отладочное прерывание
```

```
  ...

seg002:F8AAB0DB _pf_test:                               ; CODE XREF: seg002:F8AAB0D4
seg002:F8AAB0DB                 mov     esi, 0ED1DBFE1h
seg002:F8AAB0E0                 add     esi, 60F5D3h
seg002:F8AAB0E6                 cmp     byte ptr [esi], 0
seg002:F8AAB0E9                 jz      loc_F8AAB1B4

 ;здесь читается адрес обработчика исключения #PF - page fault
 ;если IDT была восстановлена, то перезагружаем компьютер :)
seg002:F8AAB0EF                 mov     esi, 0AB57712h
seg002:F8AAB0F4                 sub     esi, 8AB18312h
seg002:F8AAB0FA                 add     esi, 74h ; 't'
seg002:F8AAB0FD                 mov     eax, [esi]
seg002:F8AAB0FF                 mov     ax, [esi-4]
seg002:F8AAB103                 mov     esi, 9D0B83C2h
seg002:F8AAB108                 xor     esi, 65A4B3C2h
seg002:F8AAB10E                 cmp     esi, eax
seg002:F8AAB110                 jz      short loc_F8AAB117
seg002:F8AAB112                 jmp     _reboot

seg002:F8AAB117 loc_F8AAB117:                           ; CODE XREF: seg002:F8AAB110
seg002:F8AAB117                 mov     ecx, 202h
seg002:F8AAB11C                 call    _hash1_calc
seg002:F8AAB121                 mov     ebx, eax
seg002:F8AAB123                 add     esi, 206h
seg002:F8AAB129                 mov     ecx, 32h ; '2'
seg002:F8AAB12E                 call    _hash1_calc
seg002:F8AAB133                 add     eax, ebx
seg002:F8AAB135                 mov     ebx, 0ECF96E68h
seg002:F8AAB13A                 add     ebx, 854748h

 ;это то, чего не было в xprotector'е и есть в themid'е - проверка контрольной суммы обработчика
 ;если пропатчен обработчик - перезагрузка
seg002:F8AAB140                 cmp     [ebx], eax
seg002:F8AAB142                 jz      short _pf_nopatch
seg002:F8AAB144                 jmp     _reboot
```

Далее идёт однотипный код, проверяющий исключение #NP(неприсутствующего сегмента) и что более важно некоторых функций sst. Именно поэтому восстановление sst при "живом" перехвате KfRaiseIrql невозможно. Способ обхода очевиден - восстанавливаем 5 оригинальных байт обработчика этой функции hal.dll и вперёд, можно восстанавливать sst. Правда есть одно но, в разных версиях протектора перехватываются разные функции в hal.dll, поэтому предлагаю универсальный способ - после загрузки ОС снимать дамп секции кода hal.dll(можно за одно и ntoskrnl) и когда необходимо снять дамп, восстановить импорт или вообще просто открыть доступ к памяти, просто записываем на место оригинальную секцию кода и подгружаем sst с диска, всё, антидампа как не бывало. Можно даже восстановить IDT и отлаживать защищённый процесс ring3 отладчиками, правда только до первой ring0-decrypt конструкции, толку от такой отладки практически не будет.

Нахождение и восстановление кода на OEP:

Themida имеет опцию защиты, позволюющую прятать оригинальную точку входа программы, чего не было в xprotector'e. Делается это общеизвестным способом, инструкции крадутся с OEP, разбавляются мусором и отправляются куда-нибудь подальше в память. Чтобы научится ловить эти инструкции и вообще переход на OEP, надо знать, где конкретно располагаются эти краденные инструкции. Чтобы это узнать используем "троянский принцип" - подкинем протектору инструкцию, вызывающую исключение. А конкретно берём любую программу(например всеми любимый calc.exe), меняем первый байт на OEP на 0xFA(инструкция cli, причём лучше два байта 0xFA подряд на OEP вписать, чтобы структуру кода не нарушить), запаковываем его WinLicense, не забыв указать опцию прятания OEP и дело сделано - при начале исполнения программы будем получать исключение EXCEPTION_PRIV_INSTRUCTION. Т.к. мы имеем в адресном пространстве свою DLL, то ничто не мешает перехватить это исключение, и VEH подходит для этого как нельзя

лучше, т.к. векторный обработчик исполняется первым и для всех потоков процесса. Добавляем код, который выдаёт MessageBox при исключении привилегированной инструкции с заголовком, показывающим адрес исключения выдаст, снимаем дамп и можно изучать код на OEP. Код, приведённый ниже обработан обновленным unscrambled-плагином:

```
 ;Это две инструкции cli, которые мы записали на EP calc.exe
WinLicen:0123E87D                cli
WinLicen:0123E895                cli

 ;А вот эти две инструкции заменяют одну - push 010015E0h
WinLicen:0123E907                push    0C6301166h
WinLicen:0123E924                add     dword ptr [esp], 3AD0047Ah

 ;Здесь краденный с OEP код закончился
 ;1247Ch - это RVA первой некраденной инструкции на OEP
WinLicen:0123E92E                push    1247Ch
WinLicen:0123E933                pushf
WinLicen:0123E934                cld
WinLicen:0123E9CE                push    eax
WinLicen:0123EA16                push    eax
WinLicen:0123EA40                mov     [esp], ebp
WinLicen:0123EA61                clc
WinLicen:0123EA98                call    $+5
WinLicen:0123EA9D                pop     ebp
WinLicen:0123EA9E                sub     ebp, 0AA7D278h

 ;Синхронизация(возможно сигнал окончиния распаковки)
WinLicen:0123EB01                mov     eax, [ebp+0A940E25h]
WinLicen:0123EB07                cld
WinLicen:0123EB08                mov     byte ptr [eax], 0
WinLicen:0123EB2F                mov     eax, [ebp+0A942385h]
WinLicen:0123EB64                pusha
WinLicen:0123EB78                pushf

 ;Здесь мы видим код, затирающий краденные байты, чтобы их нельзя было восстановить
 ;простым дампом во время выполнения, из этого следует, что придётся ловить окончание
 ;распаковки до выполнения этого блока кода
WinLicen:0123EBCD                call    $+5
WinLicen:0123EBD2                pop     edi
WinLicen:0123EC42                mov     eax, 0
WinLicen:0123EC72                mov     edx, edi
WinLicen:0123EC93                lea     ecx, [ebp+0AA7D108h]
WinLicen:0123ECED                sub     edx, ecx
WinLicen:0123ED05                sub     edi, edx
WinLicen:0123ED24                mov     ecx, 0F8h
WinLicen:0123ED29                cld
WinLicen:0123ED2B                sub     edi, ecx
WinLicen:0123ED68                lea     esi, [ebp+0AA7D108h]
WinLicen:0123ED86                lea     edx, [ebp+0AA7D5E9h]
WinLicen:0123EDE6                sub     edx, esi
WinLicen:0123EDEA                add     ecx, edx
WinLicen:0123EE0E                rep stosb
WinLicen:0123EE10                inc     edx
WinLicen:0123EE11                stosd

 ;выравнивание стека и переход в секцию кода программы
WinLicen:0123EE8D                popf
WinLicen:0123EEAA                cmc
WinLicen:0123EEAB                popa
WinLicen:0123EED1                pop     ebp
WinLicen:0123EF12                add     [esp+8], eax
WinLicen:0123EF16                cld
WinLicen:0123EF17                pop     eax
WinLicen:0123EF91                popf
WinLicen:0123EF92                retn
```

Как видно с помощью плагина не составляет труда восстановить украденные инструкции и найти истинную OEP, но этого мало, надо ещё научится останавливать поток перед тем, как код будет затёрт нулями, но после того, как он будет расшифрован. Немного поисследовав код в

окрестностях можно сделать несколько выводов. Во-первых краденые байты лежат практически в самом конце секции протектора. Во-вторых до передачи управления на эти байты они зашифрованы, и расшифровываются одним из ring0-дешифровщиков, лежашим чуть выше. А для вызова дешифровщика необходимо записать в IDT его адрес. Смотрим, как протектор это делает:

```
 ;загружаем в eax адрес выхода из прерывания(после окончания работы дешифровщика инструкция
 ;iret передаст управление на этот адрес), а в esi - адрес самого обработчика
WinLicen:0123D181                lea     eax, [ebp+0AA7D009h]
WinLicen:0123D199                lea     esi, [ebp+0AA7C1D0h]

 ;Если вдруг страница с обработчиком прерывания будет сброшена в файл подкачки, то
 ;вместо дешифровки кода появится синий экран. Эти команды подгружают страницы в таком случае
WinLicen:0123D1BC                mov     edx, [eax]
WinLicen:0123D1E6                mov     [eax], edx
WinLicen:0123D221                mov     edx, [eax+12Ch]
WinLicen:0123D262                mov     [eax+12Ch], edx

 ;...

 ;А здесь мы видим вызов функции, устанавливающей обработчики прерываний int1 и int3.
 ;Адрес передаётся через стек. Обратите внимание, что функция перезаписывает IDT для
 ;всех процессоров в системе
WinLicen:0123D703                xchg    eax, esi
WinLicen:0123D72C                push    eax
WinLicen:0123D73B                xchg    eax, esi
WinLicen:0123D746                lea     edx, [ebp+xf_set_dbg_int]
WinLicen:0123D765                call    edx

 ;...

 ;Начало обработчика 3-его прерывания
WinLicen:0123D9F5                mov     [esp], eax
WinLicen:0123DA1C                push    0
WinLicen:0123DA1F                lea     eax, [ebp+0AA7D009h]
WinLicen:0123DA3D                add     eax, 5
WinLicen:0123DA82                push    eax
WinLicen:0123DA9C                push    eax
WinLicen:0123DAA8                mov     [esp], eax

 ;В каждом подобном обработчике сбрасываются отладочные регистры
WinLicen:0123DAC9                sub     eax, eax
WinLicen:0123DAF7                mov     dr0, eax
WinLicen:0123DB36                mov     dr1, eax
WinLicen:0123DB74                mov     dr2, eax
WinLicen:0123DBC3                mov     dr3, eax
WinLicen:0123DBF0                pop     eax
WinLicen:0123DBF1                cld
WinLicen:0123DBF2                mov     edx, [esp+4]
WinLicen:0123DC2C                mov     edi, [esp]
WinLicen:0123DC6E                mov     ecx, 0
WinLicen:0123DC99 ; ---------------------------------------------------------------------------
WinLicen:0123DC99
WinLicen:0123DC99 loc_123DC99:                           ; CODE XREF: WinLicen:0123E512
WinLicen:0123DC99                mov     eax, 400h
WinLicen:0123DC9E                mov     dr7, eax

 ;...

 ;Код впереди на данный момент расшифрован, записываем 0xFFFFFFFF на место(т.е. в качестве
 ;обработчиков int1 и int3). Если вспомнить код обработчика перехваченной функции KfRaiseIrql,
 ;то он записывал 0xFFFFFFFF в IDT только если обработчики 1-ого и 3-его прерывания находились
 ;в ядре, а сейчас адрес обработчиков - 0123D9F5h, поэтому протектор и вызывает снова функцию
 ;xf_set_dbg_int, чтобы деактивировать отладочные прерывания
WinLicen:0123E5B7                push    8BF57B2Ah
WinLicen:0123E5C3                xor     dword ptr [esp], 740A84D5h
WinLicen:0123E5EF                lea     ecx, [ebp+xf_set_dbg_int]
WinLicen:0123E61D                call    ecx
WinLicen:0123E64B                cld

 ;Это видимо для подстраховки, дублирование xf_set_dbg_int только для текущего процессора
```

```
 ;(т.е. для того, на котором выполняется этот код)
WinLicen:0123E64C                 push    eax
WinLicen:0123E667                 mov     [esp], eax
WinLicen:0123E69F                 sidt    qword ptr [esp-2]
WinLicen:0123E6B7                 pop     eax
WinLicen:0123E6B9                 add     eax, 0Ch
WinLicen:0123E6D3                 mov     word ptr [eax-4], 0FFFFh
WinLicen:0123E6F4                 mov     word ptr [eax+2], 0FFFFh
WinLicen:0123E718                 push    eax
WinLicen:0123E738                 mov     [esp], eax
WinLicen:0123E759                 sidt    qword ptr [esp-2]
WinLicen:0123E779                 pop     eax
WinLicen:0123E784                 add     eax, 1Ch
WinLicen:0123E79E                 mov     word ptr [eax-4], 0FFFFh
WinLicen:0123E7A5                 mov     word ptr [eax+2], 0FFFFh
WinLicen:0123E7AC                 mov     ecx, [ebp+0A9420A1h]
WinLicen:0123E7CA                 mov     byte ptr [ecx], 0
WinLicen:0123E7CD                 stc
WinLicen:0123E7CE
WinLicen:0123E7CE loc_123E7CE:                            ; CODE XREF: WinLicen:0123E570
WinLicen:0123E7CE                 pop     ecx
WinLicen:0123E7F1                 add     esp, 8

 ;Выход обратно в ring3
WinLicen:0123E80E                 iret
```

Наиболее простой путь это перехват внутренней функции протектора - xf_set_dbg_int. Пусть наш код будет скидывать в лог адреса вызова(точнее адреса следующие за вызовом) перехваченной функции, когда этим адресом окажется 0123E61Fh, можно спокойно снимать дамп, определять положение OEP и восстанавливать краденные байты. Правда остаются проблемы - как найти функцию xf_set_dbg_int внутри защищённой программы и в какой момент её перехватывать. Искать можно несколькими способами, например найти любой ring0-decryptor в дампе и вызов этой функции, но есть способ проще и лучше - по сигнатуре. Внутренности этой функции не менялись со времён xprotector'a, поэтому поиск по сигнатуре будет работать абсолютно со всеми версиями протектора. А перехватывать лучше всего в момент, когда DLL получит уведомление о запуске последнего треда перед окончанием распаковки. Номер его можно посмотреть в логе, например в упакованном calc.exe последний тред имеет номер 28(включая главный). Также надо ещё помнить, что наш обработчик xf_set_dbg_int после расшифровки спертых с OEP байт будет выполняться в ring0 и перед тем, как снимать дамп необходимо вернуться в ring3(например чтобы вывести MessageBox с сообщением о том, что распаковка окончена). Для этого подменяем адрес возврата для инструкции iret прямо в стеке. Рассчитываем смещение в стеке: допустим при выполнение инструкции iret по адресу 0123E80Eh адрес возврата лежит по смещению 0. Поднимаемся вверх до call'a на 0123E61D и получаем что адрес возврата здесь: [esp + 0Ch]. Но учитывая, что мы находимся внутри процедуры, надо добавить ещё - 8(т.к. при вызове xf_set_dbg_int в стек кидается адрес возврата из функции и 1 параметр), получаем [esp + 14h]. Но в самом начале обработчика находится инструкция pushad, уменьщающая стек на 20h, в итоге адрес возврата будет находится по адресу [esp + 34h] внутри нашего обработчика xf_set_dbg_int. Сохраняем куда-нибудь старый адрес, и записываем на это место какой-нибудь свой адрес, где будет находится вызов MessageBox, выводящий сообщение "Stolen bytes at 0XXXXXXXXh". Теперь рассмотрим всё это подробнее на примере нашей цели - WinLicense Demo.

```
 ;Это небольшой кусок кода из внедрямой DLL, а конкретно всё, что касается
 ;поиска OEP и краденный байт

cmp reason, DLL_THREAD_ATTACH
jnz @@reason2

 ;========== Thread create processing ==========;
 assume fs:NOTHING

 ;Читаем и сохраняем идентификатор нового потока, а также записываем сообщение в лог
mov eax, DWORD PTR fs:[24h]
```

```
mov edx, ThreadCounter
mov [offset threads + edx*4], eax

 @@l1:
inc ThreadCounter
invoke wsprintf, offset buffer1, offset fmt003, ThreadCounter, DWORD PTR fs:[24h]
invoke LogWrite, offset buffer1
  assume fs:ERROR

 ;Здесь надо указать номер последнего треда, который создаётся
протектором,

 ;в WinLicense demo этот номер равен 26, все последующие создаёт уже пользовательский код
cmp ThreadCounter, 26
jnz @@exit

 ;Поиск xf_set_dbg_int по сигнатуре
invoke find_signature, MHandle, ImageSize, offset xf_set_dbg_int_sign
test eax, eax
jz @@l2 ;если не найдено

 ;Вывод в лог адреса найденной функции
mov ebx, eax
invoke wsprintf, offset buffer1, offset fmt005, ebx
invoke LogWrite, offset buffer1

 ;Перехват функции
mov eax, offset h_xf_set_dbg_int_sign
mov BYTE PTR [ebx], 0E9h
sub eax, ebx
sub eax, 5
mov DWORD PTR [ebx+1], eax

 ;Сохраняем адрес для перехода на него из нашего обработчика
add ebx, 5
mov _xf_set_dbg_int, ebx
jmp @@exit

 ;Вывод в лог сообщения о том, что функция xf_set_dbg_int не обнаружена
 @@l2:
invoke LogWrite, offset mess002
jmp @@exit
```

Количество тредов можно ввести максимальное и уменьшать, пока количество изменений не возрастёт резко до несколько десятков тысяч. При запуске создаётся 29 тредов, но 3 последние - порождение уже не протектора а пользовательского кода. Теперь рассмотрим наш обработчик xf_set_dbg_int_sign:

```
 ;Вот этот адрес надо будет определить, а точнее переписать из лога
LAST_IDTACCESS  equ  0BE8668h

 ;Об этих константах рассказано далее
;_offset1  equ  0
;_sleeparg  equ  0

h_xf_set_dbg_int_sign proc
pusha

 ;Увеличение счётчика обращений к IDT
inc idtacc_c

 ;"Обновление" переменной - максимально большего адреса обращения к IDT
mov eax, [esp + 20h]
cmp eax, top_idtacc
jbe @@exit

mov top_idtacc, eax

 ;Распаковка окончена?
cmp eax, LAST_IDTACCESS
```

```
jnz @@exit

 ;_offset1 - смещение адреса возврата с ring0-декриптора,
 ;о том как его искать было рассказано выше

  ifdef _offset1
mov eax, [esp + _offset1]
mov oep, eax
mov DWORD PTR [esp + _offset1], offset @@oep
  else
invoke MessageBox, NULL, offset mess003, offset mess, MB_OK
  endif

jmp @@exit

 @@oep:
pusha

 ;...

  ifdef _sleeparg
mov BYTE PTR ds:[_sleeparg], 0FFh
  endif

 ;...

invoke MessageBox, NULL, offset mess004, offset mess, MB_OK
popa
cli ;исключение обеспечит корректный выход на 100%

 @@exit:
mov     ebx, [esp+24h] ;инструкция с начала перехваченной функции в протекторе
jmp _xf_set_dbg_int
h_xf_set_dbg_int_sign endp
```

После определения количество тредов и адреса последнего обращения к IDT можно снимать дамп и изучать окрестности OEP. Адреса этих самых окрестностей нужно брать из лога: top address 0XXXXXXXh - адрес, где меняется IDT, будет известен при перехвате xf_set_dbg_int, oep(?) = 0XXXXXXXh - адрес выхода из последнего перед OEP дешифровщика нулевого кольца, этот будет известен, только после определения смещения в стеке этого самого адреса

```
 ;Вызов xf_set_dbg_int, однако "окружение" этого вызова выглядит совсем не так, как в
 ;запакованном calc.exe, потому что это другой тип ring0-decryptor'a
 ;Высчитывая смещение стека, где хранится EIP для инструкции iret:
 ;-20(смещение стека перед вызовом) -8(адрес возврата и параметр) - 20(инструкция pusha
 ;в начале h_xf_set_dbg_int_sign, приведённой выше) получаем -48, вписываем
 ;константу _offset1 равную 48h в исходник DLL
WinLicen:00BE865D                push    0FFFFFFFFh
WinLicen:00BE8662                call    dword ptr [ebp+4BF2A11h]
WinLicen:00BE8668                push    ecx ;-20
WinLicen:00BE8669                sidt    qword ptr [esp-2]
WinLicen:00BE866E                pop     ecx ;-24
WinLicen:00BE866F                add     ecx, 0Ch
WinLicen:00BE8672                mov     word ptr [ecx-4], 0FFFFh
WinLicen:00BE8678                mov     word ptr [ecx+2], 0FFFFh
WinLicen:00BE867E                mov     ecx, [ebp+4BF03E9h]
WinLicen:00BE8684                mov     byte ptr [ecx], 0
WinLicen:00BE8687                mov     eax, ebx
WinLicen:00BE86AF                popa ;-20
WinLicen:00BE86B0                iret ;0

 ;...

WinLicen:00BE86DA                mov     dword ptr [ebp+4BF0449h], 4D20h
WinLicen:00BE86ED                mov     dword ptr [ebp+4BF12B9h], 0
WinLicen:00BE870D                push    dword ptr [ebp+4BF1DB5h]
WinLicen:00BE8713                mov     [ebp+4BF14C1h], esi
WinLicen:00BE8719                call    dword ptr [ebp+4BF1C61h]
WinLicen:00BE87EC                mov     esi, ecx
WinLicen:00BE87EE
```

```
 ;Хмм.. бесконечный цикл. Очевидно, что по адресу 00BE8719h вызывается SetEvent
 ;я правда не проверял, вдруг это не так :)
 ;А по адресу 00BE87F0h - Sleep. Оба этих адреса лежат в heap'e, напомню, что некторые библиотеки,
 ;включая kernel32.dll подгружаются с диска и функции вызываются минуя kernel32.dll в памяти
WinLicen:00BE87EE loc_BE87EE:                           ; CODE XREF: WinLicen:00BE87F8
WinLicen:00BE87EE                push    0
WinLicen:00BE87F0                call    dword ptr [ebp+4BF2851h]
WinLicen:00BE87F6                mov     eax, eax
WinLicen:00BE87F8                jmp     short loc_BE87EE
```

Дальнейшую расщифровку будет выполнять другой тред, и нужно как-то поймать окончание этой расшифровки. Недолго думая вписываем вместо инструкции push 0 по адресу 00BE87EEh инструкцию push 0FFFFFFFFh(т.о. константа _sleeparg будет равна 00BE87EFh) тем самым делаю задержку более чем на 4 миллиарда секунд, которого уж точно хватит для того, чтобы тред успел закончить расшифровку и даже для того, чтобы снять дамп уже с готовыми к восстановлению краденными байтами:

```
 ;замена push ebp
WinLicen:00BE8859                push    eax
WinLicen:00BE886E                mov     [esp], ebp

WinLicen:00BE8891                mov     ebp, esp
WinLicen:00BE88BA                add     esp, 0FFFFFFE0h

 ;push ebx
WinLicen:00BE88F7                xchg    eax, ebx
WinLicen:00BE892D                push    eax
WinLicen:00BE897C                xchg    eax, ebx

WinLicen:00BE897E                sub     eax, eax
WinLicen:00BE89C6                mov     [ebp-20h], eax
WinLicen:00BE89F3                mov     [ebp-1Ch], eax
WinLicen:00BE8A0C                mov     [ebp-18h], eax
WinLicen:00BE8A57                mov     [ebp-14h], eax
WinLicen:00BE8A9C                mov     eax, offset dword_776200

 ;далее идёт код протектора, из которого нам полезна только следующая инструкция,
 ;где в стек записывается RVA OEP + 0XXh, где 0XXh - размер украденных инструкций
WinLicen:00BE8AA4                push    376D22h
WinLicen:00BE8AA9                pushf
 ;...

 ;Вбиваем в FASM или HIEW эти инструкции, определяем их размер(26 байт),
 ;из этого следует, что в WinLicense Demo OEP = 776D08h(RVA=376D08h)
 ;и именно туда надо добавить эти инструкции
push    ebp
mov     ebp, esp
add     esp, 0FFFFFFE0h
push    ebx
xor     eax, eax
mov     [ebp-20h], eax
mov     [ebp-1Ch], eax
mov     [ebp-18h], eax
mov     [ebp-14h], eax
mov     eax, 776200h
```

На этом заканчивается работа над OEP. Теперь можно переходить к импорту.

Восстановление импорта:

В основном восстановление импорта мало чем отличается от xprotector'a, но некоторые моменты стоит упомянуть, особенно перехват API-функций. Ведь теперь протектор замечает изменение кода в native API функциях, но всё же есть пока способ не лезть в ядро. Каждое обращение к ядру идёт через функцию KiFastSystemCall, а адрес её записан здесь: 7FFE0300h, оттуда он каждый и читается. Запись в эту область памяти запрещена для кода режима пользователя, и к тому же эта область является общей для всех процессов, и если даже написать свой драйвер, который будет записывать туда что-нибудь по требованию система сразу рухнет, потому что все остальные

процессы окажутся изолированными от ядра. Выход в том, что надо как-то сделать страницу памяти контекстно-зависимой. Как раз для этого Windows NT использует зарезервированный девятый бит в таблице страниц, установление его равнозначно вызову VirtualProtect с параметром PAGE_WRITECOPY. Т.е. чтобы перехватить обращения к ядру, надо установить в PTE страницы с адресом 7FFE0000h бит copy-on-write и можно спокойно перехватывать. Причём драйвер можно всё-таки не писать, ведь протектор открывает доступ к IDT, и никто не помешает нам использовать любое прерывание для того, чтобы попасть в ring0:

```
hook_kifastsystemcall proc

 ;адрес дескриптора 255-ого прерывания
mov edx, 8003F400h+0FFh*8
mov eax, offset _intFF
mov [edx], ax ;младшие 16 бит смешения обработчика
mov WORD PTR [edx+2], 08h ;селектор обработчика
shr eax, 16
mov WORD PTR [edx+6], ax ;старшие 16 бит
or WORD PTR [edx+4], 6000h ;DPL устанавливаем 3

 ;Вызываем 255-ое прерывание, в eax - адрес для установки флага copy-on-write
mov eax, 7FFE0300h
int 0FFh

 ;Собственно перехват
mov DWORD PTR ds:[7FFE0300h], offset hKiFastSystemCall
ret
hook_kifastsystemcall endp

 ;...

 _intFF:
invoke set_copyonwrite_flag, eax ;Вызов функции для установки флага
iretd

 ;Это новая функция KiFastSystemCall
 hKiFastSystemCall proc

 ;Это необходимо для вызова функций API из перехваченной функции
 ;Например записи в чего-нибудь в лог, чтобы не было ненужной рекурсии
cmp hook_active, 1
jz @@systemcall

pusha
mov hook_active, 1

 ;Здесь можно размещать свой код для перехвата

 @@exit:

 ;Выход(почему-то masm не хочет ассемблировать инструкцию SYSENTER)
popa
mov hook_active, 0
mov edx, esp
db 0Fh, 034h ;SYSENTER

 @@systemcall:
mov edx, esp
db 0Fh, 034h ;SYSENTER
hKiFastSystemCall endp
```

А дальше полная аналогия с xprot-ом, перехватываем VirtualAlloc(только смещение адреса возврата из VirtualAlloc надо, это можно сделать в ollydbg или softice протрассировав вызов этой функции до инструкции SYSENTER и прибавив 20h уравновешивая инструкцию pusha в начале hKiFastSystemCall. В логе видно, что часто вызывается VirtualAlloc в адреса 0BD290Dh, это и есть процедура создания импорта, но если вписать этот адрес в исходник DLL, то можно устать закрывать MessageBox'ы, вписать туда надо адрес начала процедуры создания импорта, это

0BD209Ch. После сообщения "Imports creating begins; see log for detalis" можно снимать дамп и изучать создание импорта. Здесь я не буду приводить её полностью, а только самые важные моменты.

```
 ;ecx - ImageBase библиотеки импортируемая функция
WinLicen:00BD2395                cmp     dword ptr [ebp+4BF002Dh], 1
WinLicen:00BD239C                jz      loc_BD23DB
WinLicen:00BD23A2                cmp     ecx, [ebp+xvKernel32Handle]
WinLicen:00BD23A8                jz      loc_BD23DB
WinLicen:00BD23AE                cmp     ecx, [ebp+xvUser32Handle]
WinLicen:00BD23B4                jz      loc_BD23DB
WinLicen:00BD23BA                cmp     ecx, [ebp+xvAdvapi32Handle]
WinLicen:00BD23C0                jz      loc_BD23DB
WinLicen:00BD23C6

 ;Если ни один переход не сработал то функция запишется в IAT как есть
 ;без переходника
WinLicen:00BD23C6 _save_to_IATx1:                        ; CODE XREF: WinLicen:loc_BD2408
WinLicen:00BD23C6                                        ; WinLicen:00BD242C ...
WinLicen:00BD23C6                lea     ebx, [ebp+xfiGetProcAddress]
WinLicen:00BD23CC                call    ebx
WinLicen:00BD23CE
WinLicen:00BD23CE _save_to_IATx0:                        ; CODE XREF: WinLicen:00BD245C
WinLicen:00BD23CE                                        ; WinLicen:00BD2480 ...
WinLicen:00BD23CE                mov     edi, eax
WinLicen:00BD23D0                mov     [ebp+xvImportAddress], eax
WinLicen:00BD23D6                jmp     _save_to_IAT
WinLicen:00BD23DB ; ---------------------------------------------------------------------------
WinLicen:00BD23DB
WinLicen:00BD23DB loc_BD23DB:                            ; CODE XREF: WinLicen:00BD239C
WinLicen:00BD23DB                                        ; WinLicen:00BD23A8 ...
WinLicen:00BD23DB                lea     ebx, [ebp+xfiGetProcAddress]
WinLicen:00BD23E1                call    ebx
WinLicen:00BD23E3                cmp     dword ptr [ebp+4BF002Dh], 0
WinLicen:00BD23EA                jz      loc_BD240D      ; 00A9182Ch
WinLicen:00BD23F0                cmp     eax, [ebp+4BF25D1h] ; 00A91EB8h
WinLicen:00BD23F6                jz      loc_BD2408
WinLicen:00BD23FC                cmp     eax, [ebp+4BF01A1h] ; 00A8FA88h
WinLicen:00BD2402                jnz     loc_BD240D      ; 00A9182Ch
WinLicen:00BD2408
WinLicen:00BD2408 loc_BD2408:                            ; CODE XREF: WinLicen:00BD23F6
WinLicen:00BD2408                jmp     _save_to_IATx1
WinLicen:00BD240D ; ---------------------------------------------------------------------------

 ;Замена некоторых функций протекторными эмуляторами соответствующий функций
WinLicen:00BD240D loc_BD240D:                            ; CODE XREF: WinLicen:00BD23EA
WinLicen:00BD240D                                        ; WinLicen:00BD2402
WinLicen:00BD240D                cmp     eax, [ebp+xfExitProcess] ; 00A9182Ch
WinLicen:00BD2413                jnz     loc_BD2431
WinLicen:00BD2419                cmp     dword ptr [ebp+4BF0A6Dh], 0
WinLicen:00BD2420                jnz     loc_BD2431
WinLicen:00BD2426                lea     eax, [ebp+xfi_emul_ExitProcess] ; 00BBB01D
WinLicen:00BD242C                jmp     _save_to_IATx1
WinLicen:00BD2431 ; ---------------------------------------------------------------------------
WinLicen:00BD2431
WinLicen:00BD2431 loc_BD2431:                            ; CODE XREF: WinLicen:00BD2413
WinLicen:00BD2431                                        ; WinLicen:00BD2420
WinLicen:00BD2431                cmp     eax, [ebp+xfExitProcess]
WinLicen:00BD2437                jz      _save_to_IATx1
WinLicen:00BD243D                cmp     dword ptr [ebp+4BF0CF5h], 0
WinLicen:00BD2444                jz      loc_BD2461
WinLicen:00BD244A                cmp     eax, [ebp+xfReadFile] ; 00A900B0h
WinLicen:00BD2450                jnz     loc_BD2461
WinLicen:00BD2456                lea     eax, [ebp+xfi_emul_ReadFile]
WinLicen:00BD245C                jmp     _save_to_IATx0
WinLicen:00BD2461 ; ---------------------------------------------------------------------------
WinLicen:00BD2461
WinLicen:00BD2461 loc_BD2461:                            ; CODE XREF: WinLicen:00BD2444
WinLicen:00BD2461                                        ; WinLicen:00BD2450
WinLicen:00BD2461                cmp     dword ptr [ebp+4D3201Fh], 1
```

```
WinLicen:00BD2468                jnz     loc_BD2485
WinLicen:00BD246E                cmp     eax, [ebp+xfGetProcAddress] ; 00BD195Dh
WinLicen:00BD2474                jnz     loc_BD2485
WinLicen:00BD247A                lea     eax, [ebp+xfi_emul_GetProcAddress] ; 00BCCBA6
WinLicen:00BD2480                jmp     _save_to_IATx0

WinLicen:00BD2485
WinLicen:00BD2485 loc_BD2485:                            ; CODE XREF: WinLicen:00BD2468
WinLicen:00BD2485                                        ; WinLicen:00BD2474
WinLicen:00BD2485                xor     edi, edi
WinLicen:00BD2487                cmp     dword ptr [ebp+4BF1641h], 0
WinLicen:00BD248E                jz      loc_BD26A1
WinLicen:00BD2494                cmp     eax, [ebp+4D32062h] ; 00BD1949h
WinLicen:00BD249A                jnz     short loc_BD24A3 ; 00BD1951h
WinLicen:00BD249C                mov     eax, [ebp+4BF28FDh]
WinLicen:00BD24A2                inc     edi

 ;... пропустим мусор :)

WinLicen:00BD26A1 loc_BD26A1:                            ; CODE XREF: WinLicen:00BD248E
WinLicen:00BD26A1                                        ; WinLicen:00BD2698
WinLicen:00BD26A1                or      edi, edi
WinLicen:00BD26A3                jz      loc_BD26AE      ; 00A905B8h
WinLicen:00BD26A9                jmp     _save_to_IATx0
WinLicen:00BD26AE ; ---------------------------------------------------------------------------

 ;После мусора видна замена функции wsprintfA переходником, подробнее рассмотрим это
 ;после в главе про макросы sdk
WinLicen:00BD26AE loc_BD26AE:                            ; CODE XREF: WinLicen:00BD26A3
WinLicen:00BD26AE                cmp     eax, [ebp+xfwsprintf] ; 00A905B8h
WinLicen:00BD26B4                jnz     loc_BD26C5      ; 00A9185Ch
WinLicen:00BD26BA                lea     eax, [ebp+xfi_emul_wsprintf] ; 00B8CD09h
WinLicen:00BD26C0                jmp     _save_to_IATx0
WinLicen:00BD26C5 ; ---------------------------------------------------------------------------
WinLicen:00BD26C5
WinLicen:00BD26C5 loc_BD26C5:                            ; CODE XREF: WinLicen:00BD26B4
WinLicen:00BD26C5                cmp     eax, [ebp+xfRaiseException] ; 00A9185Ch
WinLicen:00BD26CB                jnz     loc_BD26E9      ; 00BD1941h
WinLicen:00BD26D1                cmp     dword ptr [ebp+4D3201Fh], 1
WinLicen:00BD26D8                jnz     loc_BD26E9      ; 00BD1941h
WinLicen:00BD26DE                lea     eax, [ebp+xfi_emul_RaiseException] ; 00BCCB29h
WinLicen:00BD26E4                jmp     _save_to_IATx0
WinLicen:00BD26E9 ; ---------------------------------------------------------------------------
WinLicen:00BD26E9
WinLicen:00BD26E9 loc_BD26E9:                            ; CODE XREF: WinLicen:00BD26CB
WinLicen:00BD26E9                                        ; WinLicen:00BD26D8
WinLicen:00BD26E9                cmp     eax, [ebp+xfRtlEnterCriticalSection] ; 00BD1941h
WinLicen:00BD26EF                jz      loc_BD2701
WinLicen:00BD26F5                cmp     eax, [ebp+xfRtlLeaveCriticalSection] ; 00BD1945h
WinLicen:00BD26FB                jnz     loc_BD2706
WinLicen:00BD2701
WinLicen:00BD2701 loc_BD2701:                            ; CODE XREF: WinLicen:00BD26EF
WinLicen:00BD2701                jmp     _save_to_IATx0
WinLicen:00BD2706 ; ---------------------------------------------------------------------------
WinLicen:00BD2706
WinLicen:00BD2706 loc_BD2706:                            ; CODE XREF: WinLicen:00BD26FB
WinLicen:00BD2706                mov     esi, 0
WinLicen:00BD270B                cmp     esi, 1
WinLicen:00BD270E                jnz     loc_BD2759
WinLicen:00BD2714                cmp     eax, [ebp+xfCreateThread] ; 00BD1935h
WinLicen:00BD271A                jnz     loc_BD272B      ; 00BD1939h
WinLicen:00BD2720                lea     eax, [ebp+xfi_emul_CreateThread]
WinLicen:00BD2726                jmp     _save_to_IATx0
WinLicen:00BD272B ; ---------------------------------------------------------------------------
WinLicen:00BD272B
WinLicen:00BD272B loc_BD272B:                            ; CODE XREF: WinLicen:00BD271A
WinLicen:00BD272B                cmp     eax, [ebp+xfTerminateThread] ; 00BD1939h
WinLicen:00BD2731                jnz     loc_BD2742      ; 00BD193Dh
WinLicen:00BD2737                lea     eax, [ebp+xfi_emul_TerminateThread]
WinLicen:00BD273D                jmp     _save_to_IATx0
WinLicen:00BD2742 ; ---------------------------------------------------------------------------
```

```
WinLicen:00BD2742
WinLicen:00BD2742 loc_BD2742:                             ; CODE XREF: WinLicen:00BD2731
WinLicen:00BD2742                 cmp     eax, [ebp+xfExitThread] ; 00BD193Dh
WinLicen:00BD2748                 jnz     loc_BD2759
WinLicen:00BD274E                 lea     eax, [ebp+xfi_emul_ExitThread]
WinLicen:00BD2754                 jmp     _save_to_IATx0

 ; ...

 ;Запись в IAT очередного элемента
WinLicen:00BD294F _save_to_IAT:                           ; CODE XREF: WinLicen:00BD23D6
WinLicen:00BD294F                 mov     esi, [ebp+MainImportHashArray]
WinLicen:00BD2955                 lodsd
WinLicen:00BD2956                 mov     dword ptr [esi-4], 0
WinLicen:00BD295D                 rol     eax, 5
WinLicen:00BD2960                 add     eax, 470EE6D2h
WinLicen:00BD2965                 add     eax, [ebp+xvMainHandle]
WinLicen:00BD296B                 mov     ecx, [ebp+xvImportAddress]
WinLicen:00BD2971                 mov     [eax], ecx
WinLicen:00BD2973                 lodsd
WinLicen:00BD2974                 mov     dword ptr [esi-4], 0

 ;...

 ;сохранение переходов на импортируемые функции
WinLicen:00BD29E1 loc_BD29E1:                             ; CODE XREF: WinLicen:00BD29D9
WinLicen:00BD29E1                 push    eax
WinLicen:00BD29E2                 cmp     dword ptr [ebp+4BF002Dh], 1
WinLicen:00BD29E9                 jz      loc_BD2A0F
WinLicen:00BD29EF                 mov     eax, 100h
WinLicen:00BD29F4                 lea     ebx, [ebp+4CD12FDh]
WinLicen:00BD29FA                 call    ebx
WinLicen:00BD29FC                 cmp     eax, 50h ; 'P'
WinLicen:00BD29FF                 jb      loc_BD2A0F
WinLicen:00BD2A05                 mov     al, 90h ; 'P'

 ;запись опкода инструкции перехода(jmp или call) на элемент импорта
 ;с предшествующей инструкцией nop(инструкции call [mem32] и jmp [mem32]
 ;занимают 6 байт, а непосредственные переходы - 5, поэтому протектор заменяя
 ;инструкцию может поставить nop впереди)
WinLicen:00BD2A07                 stosb
WinLicen:00BD2A08                 pop     eax
WinLicen:00BD2A09                 stosb
WinLicen:00BD2A0A                 jmp     loc_BD2A26
WinLicen:00BD2A0F ; ---------------------------------------------------------------------------

 ;запись того же опкода, но уже без nop
WinLicen:00BD2A0F loc_BD2A0F:                             ; CODE XREF: WinLicen:00BD29E9
WinLicen:00BD2A0F                                         ; WinLicen:00BD29FF
WinLicen:00BD2A0F                 pop     eax
WinLicen:00BD2A10                 stosb
WinLicen:00BD2A11                 cmp     byte ptr [edi-1], 0E9h ; 'щ'
WinLicen:00BD2A15                 jnz     loc_BD2A26
WinLicen:00BD2A1B                 lea     ebx, [ebp+4CD12CDh]
WinLicen:00BD2A21                 call    ebx
WinLicen:00BD2A23                 mov     [edi+4], al

 ;Запись смещения
WinLicen:00BD2A26 loc_BD2A26:                             ; CODE XREF: WinLicen:00BD2A0A
WinLicen:00BD2A26                                         ; WinLicen:00BD2A15
WinLicen:00BD2A26                 mov     eax, [ebp+xvImportAddress]
WinLicen:00BD2A2C                 sub     eax, edi
WinLicen:00BD2A2E                 sub     eax, 4
WinLicen:00BD2A31                 stosd
```

В общем то если заменить 4 перехода nop'ами, и заодно устранить инструкцию stosb по адресу 0BD2A07h, то переходники генерироваться не будут, а также не будут смещаться непосредственные переходы на импорт, в результате чего получится чистая IAT и останется только переориентировать переходы. Но.. "An Error has ocurred while loading imports" - не стоит забывать

про проверку CRC кода, создающего импорт. Здесь просматривается целых 2 цикла, но фактически мешается только один:

```
 ;Загрузка в регистры адреса и размера проверяемого блока кода - 00BD19D9h, 1874h
WinLicen:00BD2217                lea     esi, [ebp+4D320F2h]
WinLicen:00BD221D                lea     edi, [ebp+4D33966h]
WinLicen:00BD2223                sub     edi, esi

WinLicen:00BD2225                mov     edx, edi
WinLicen:00BD2227                mov     edi, [ebp+4BF0811h]
WinLicen:00BD222D                or      ecx, 0FFFFFFFFh
WinLicen:00BD2230

 ;Цикл подсчёта хэша
WinLicen:00BD2230 loc_BD2230:                            ; CODE XREF: WinLicen:00BD2240
WinLicen:00BD2230                xor     eax, eax
WinLicen:00BD2232                mov     al, [esi]
WinLicen:00BD2234                xor     al, cl
WinLicen:00BD2236                inc     esi
WinLicen:00BD2237                mov     eax, [edi+eax*4]
WinLicen:00BD223A                shr     ecx, 8
WinLicen:00BD223D                xor     ecx, eax
WinLicen:00BD223F                dec     edx
WinLicen:00BD2240                jnz     loc_BD2230
WinLicen:00BD2246                mov     eax, ecx
WinLicen:00BD2248                not     eax

 ;сравнения, инструкия по адресу 00BD2263h должна выполнится
 ;чтоб ошибок при загрузке импорта не было
WinLicen:00BD224A                cmp     [ebp+4BF132Dh], eax
WinLicen:00BD2250                jz      loc_BD226D
WinLicen:00BD2256                cmp     dword ptr [ebp+4BF2865h], 0
WinLicen:00BD225D                jnz     loc_BD226D
WinLicen:00BD2263                mov     dword ptr [ebp+4BF0255h], 1
```

И вот, заменяя переход jz по адресу 00BD2250h на jmp получаем чистую IAT, но пока вместо ссылок на неё импорт вызывается напрямую, например:

```
___:00401E90                 jmp     near ptr 7756846Dh
___:00401E90 sub_401E90      endp
___:00401E90
___:00401E95 ; ---------------------------------------------------------------------------
___:00401E95                 dec     ebx
___:00401E96                 mov     eax, eax
___:00401E98
___:00401E98 ; --------------- S U B R O U T I N E ---------------------------------------
___:00401E98
___:00401E98
___:00401E98 sub_401E98      proc near               ; CODE XREF: sub_533024+88
___:00401E98                                         ; sub_534544+71 ...
___:00401E98                 jmp     near ptr 77520519h
___:00401E98 sub_401E98      endp
___:00401E98
___:00401E98 ; ---------------------------------------------------------------------------
___:00401E9D                 db 0C5h, 8Bh, 0C0h
___:00401EA0
___:00401EA0 ; --------------- S U B R O U T I N E ---------------------------------------
___:00401EA0
___:00401EA0
___:00401EA0 sub_401EA0      proc near               ; CODE XREF: sub_533024+A5
___:00401EA0                 jmp     near ptr 774F0326h
___:00401EA0 sub_401EA0      endp
___:00401EA0
___:00401EA5 ; ---------------------------------------------------------------------------
___:00401EA5                 das
___:00401EA6                 mov     eax, eax
___:00401EA8
___:00401EA8 ; --------------- S U B R O U T I N E ---------------------------------------
___:00401EA8
___:00401EA8
___:00401EA8 sub_401EA8      proc near               ; CODE XREF: sub_44CA24+8F
```

```
___:00401EA8                jmp     near ptr 774ED024h
___:00401EA8 sub_401EA8     endp
```

Это конечно никуда не годиться, здесь должны находится ссылки на IAT. Поэтому в DLL есть код для восстановления, выполняться он будет после окончания распаковки(там где идёт переход на OEP, т.е. перед MessageBox'ом "Unpack finished!". Чтобы найти IAT, надо простассировать код после метки _save_to_IAT, когда MainImportHashArray будет известен. Взять этот MainImportHashArray можно здесь:

```
WinLicen:00BD20E4               mov     esi, [ebp+4BF00CDh] ;Здесь находится этот адрес
WinLicen:00BD20EA               mov     ebx, [ebp+4BF0781h]
WinLicen:00BD20F0               mov     [ebp+MainImportHashArray], esi
```

Трассировать код можно отладчиком-эмулятором или в уме, что сложнее. Если в момент записи(адрес 00BD2971h), в eax будет адрес, непохожий на IAT, то можно переориентировать EIP на адрес 00BD2955h и трассировать снова, пока не увидим адрес 007CF618h - начало IAT. Размер можно определить, просмотрев дамп после запуска. Кончается IAT вот здесь: 007D24A0h. 7D24A0-7CF618 = 2E88h. И наконец после правки констант начала и размера IAT получаем полностью рабочий импорт, иногда распаковка на этом и заканчивается если тот, кто упаковывал программу не посмотрел SDK, которое позволяет не так уж и плохо защитить пользовательский код.

Восстановление пользовательского кода(удаление макросов SDK):

В примере используются три документированных макроса(CODE_ENCRYPT, CODE_CLEAR и CODE_REPLACE, см. include-файл из SDK для подробностей) и один недокументированный с использованием переходника wsprintf. CODE_ENCRYPT, CODE_CLEAR и wsprintf всего лишь расшифровывают впереди(а может и не впереди) лежащий код, для восстановления надо всего лишь передать управление на начало конструкции и перехватить управление после расшифровки. С CODE_REPLACE немного сложнее, придётся немного закопаться в код протектора, и даже столкнуться и обойти простенькую виртуальную машину.

Начнём с wsprintf. Правда здесь совсем нет отличий от xprotector'a, но всё равно есть, что пояснить. Вот пример такой конструкции:

```
___:0055A960                push    78263845h
___:0055A965                push    5
___:0055A967                push    0 ;действие 0 - расшифровка
___:0055A969                push    0E3B90E0Ch
___:0055A96E                push    0F478990Ah
___:0055A973                push    78263845h
___:0055A978                call    emul_wsprintfA
___:0055A97D                add     esp, 18h

 ;... (зашифрованный кусок кода)

___:0055AB04                push    78263845h
___:0055AB09                push    5
___:0055AB0B                push    1 ;1 - шифровка
___:0055AB0D                push    0E3B90E0Ch
___:0055AB12                push    0F478990Ah
___:0055AB17                push    78263845h
___:0055AB1C                call    emul_wsprintfA
___:0055AB21                add     esp, 18h
```

А теперь смотрим как удалять эти вставки:

```
; ========== wsprintf recoverer ==========
  ifdef wsprintf_in_IAT
  ifdef wsprintf_emul_addr

 ;Сохраняем в регистре ebp правильный адрес wsprintf и помещаем туда адрес
 ;своего кода, которые подменит в стеке адрес возврата и передаст
 ;управление на функцию протектора xfi_emul_wsprintf
mov ebp, DWORD PTR ds:[wsprintf_in_IAT]
```

```
mov DWORD PTR ds:[wsprintf_in_IAT], offset @@wsprintf_redirect

 ;edi - начало секции кода, ebx - счётчик конструкций
mov edi, CodeSectionVA
xor ebx, ebx

 @@wsprintf_loop:

 ;Ищем(по сигнатуре) начальную(шифрующую)конструкцию,
 ;если таких больше нет - выход из цикла
invoke find_wsprintf_decrypt, edi
test eax, eax
jz @@wsprintf_end

 ;Вывод сообщения в лог о том, что макрос wsprintf найден
mov edi, eax
invoke wsprintfA, offset buffer1, offset fmt009, edi
invoke LogWrite, offset buffer1

 ;Передаём управление макросу - пусть расшифровывает всё что надо
jmp edi ;decryption

 @@wsprintf_after_decryption:

 ;А сюда мы попадаем, потому что код около метки @@wsprintf_redirect
 ;записывает в стек этот адрес
add esp, 18h
inc ebx

 ;Заменяем макрос nop'ами
mov ecx, wsprintf_block_size
mov al, 90h
rep stosb
jmp @@wsprintf_loop

 @@wsprintf_end:

 ;Вывод в лог сообщения о количестве найденных макросов wsprintf
invoke wsprintf, offset buffer1, offset fmt011, ebx
invoke LogWrite, offset buffer1

 ;Записываем на место адрес истинной функции wsprintf,
 ;переходник протектора больше не нужен
mov DWORD PTR ds:[wsprintf_in_IAT], ebp
mov edi, CodeSectionVA

 ;Далее идёт цикл по удалению завершающих конструкций макроса
 @@wsprintf_clearloop:
invoke find_wsprintf_encrypt, edi
test eax, eax
jz @@wsprintf_clearend

mov edi, eax
invoke wsprintf, offset buffer1, offset fmt010, edi
invoke LogWrite, offset buffer1

mov ecx, wsprintf_block_size
mov al, 90h
rep stosb
jmp @@wsprintf_clearloop

 ;Всё, макросов wsprintf больше не осталось
 @@wsprintf_clearend:
  endif
  endif
```

wsprintf_in_IAT - можно найти после восстановления импорта imprec'ом, а wsprintf_emul_addr проскакивает в создании импорта, можно брать прямо оттуда. Теперь рассмотрим ещё два простеньких макроса - CODE_ENCRYPT и CODE_CLEAR:

```
 ;Начало у макросов CODE_ENCRYPT и CODE_REPLACE одинаковое:
```

```
___:0055B772                call    near ptr unk_BD32FA
___:0055B772 ; ---------------------------------------------------------------------------
___:0055B777                dd 0Ah ;Индекс для определения thread ID
___:0055B77B                dd 0 ;decrypt
___:0055B77F                dd 21h ;code size
___:0055B783                db 20h ;какой-то байт(он не нужен)
___:0055B784 ; ---------------------------------------------------------------------------

 ;Скрытый пользовательский код
___:0055B784                call    loc_5C079D
___:0055B789                sub     edi, dword_7B8A2C
___:0055B78F                mov     [ebp-4], edi
___:0055B792                push    dword_77EF82
___:0055B798                call    loc_55BDD2
___:0055B79D                call    near ptr unk_BA772B
___:0055B79D ; ---------------------------------------------------------------------------
___:0055B7A2                dd 0Ah ; -//-
___:0055B7A6                dd 1 ;encrypt
___:0055B7AA                dd 21h ; -//-
___:0055B7AE                db 20h ; -//-

 ;В WinLicense demo нет макросов CODE_CLEAR, поэтому приведу такой макрос из
 ;примера к SDK. sub_5ED4BF здесь и unk_BD32FA выше - одна и та же процедура
___:0040105D                call    near ptr sub_5ED4BF
___:0040105D ; ---------------------------------------------------------------------------
___:00401062                dd 8 ;thread ID index
___:00401066                dd 0 ;action decrypt
___:0040106A                dd 2Ah ;size
___:0040106E                db 20h ;?
___:0040106F ; ---------------------------------------------------------------------------

 ;код пользователя
___:0040106F                push    0
___:00401071                push    offset aThemidaSdkEx_4 ; "Themida SDK example"
___:00401076                push    offset aWeAreShowing_0 ; "We are showing this message inside a
    CL"...
___:0040107B                push    0
___:0040107D                call    sub_401106

 ;Затираем этот код пользователя, чтоб его не было видно в дизассемблере
 ;после снятия дампа. Очевидно, что такой макрос подходит только для
 ;кода, который должен выполнится всего один раз
___:00401082                pusha
___:00401083                call    $+5
___:00401088                pop     edi
___:00401089                sub     edi, 2Bh
___:0040108F                mov     ecx, 2Bh
___:00401094                xor     eax, eax
___:00401096                rep stosb
___:00401098                popa
```

При вызове процедуры расшифровки(или шифровки) поток уходит в бесконечный цикл, предварительно разморозив другой поток, который будет расшифровывать код, причём этот поток просто вынужден будет менять контекст(а конкретно регистр EIP в контексте) после обработки кода. Способ такой, передаём управление на начало макроса, не забыв перехватить native API функцию ZwSetContextThread. При вызове переориентируем EIP на продолжение цикла восстановление кода, после чего остаётся только заменить всё, что относиться к макросам nop'ами и ещё больше приблизится к моменту полной распаковки.

```
; ========== CODE_ENCRYPT and CODE_CLEAR destroyer ==========
  ifdef code_encrypt_proc

 ;Инициализация цикла
mov edi, CodeSectionVA

 @@code_encrypt_loop:

 ;Поиск начала макроса, ищется по инструкции call code_encrypt_proc, это аналог
 ;sub_5ED4BF из примера. Чтобы найти этот адрес, можно поискать например в hiew
```

```
 ;вызовы(call'ы) процедур из секции кода программы в секцию протектора
invoke find_macro1_start, edi, code_encrypt_proc
test eax, eax
jz @@code_encrypt_end

 ;Вывод в лог сообщения о найденном макросе
mov edi, eax
invoke wsprintf, offset buffer1, offset fmt012, edi
invoke LogWrite, offset buffer1

 ;code_encrypt_indicator - переменная, которую будет использовать перехваченная
 ;функция ZwSetContextThread для того, чтобы определить, надо ли менять EIP или нет
mov code_encrypt_indicator, 1
mov gv0, edi
mov gv1, ebx

 ;расшифровываем код
jmp edi

 ;Сюда попадаем после расшифровки.
 ;Обратите внимание на метку, она объявлена с двумя двоеточиями, это значит что к
 ;ней можно обращаться из другой процедуры - особенность MASM'a
 _code_encrypt_done::
mov ebx, gv1
mov edi, gv0

 ;Сброс индикатора для ZwSetContextThread
mov code_encrypt_indicator, 0

 ;Читаем длину зашифрованного кода
mov edx, [edi + 0Dh]

 ;Уничтожение начала макроса
mov ecx, code_encrypt_size
mov al, 90h
rep stosb

 ;CODE_ENCRYPT?
 ;(опкод первой инструкции окончания макроса CODE_ENCRYPT - E8, т.е. это call, см. пример)
lea esi, [edi + edx]
sub esi, 8
cmp BYTE PTR [esi], 0E8h
jnz @@code_encrypt_no_CE

 ;Вывод сообщения об уничтожении CODE_ENCRYPT,
 ;а затем и само уничтожение
invoke wsprintf, offset buffer1, offset fmt013, edi, edx
invoke LogWrite, offset buffer1

mov edi, esi
mov ecx, code_encrypt_size
mov al, 90h
rep stosb
jmp @@code_encrypt_loop

 @@code_encrypt_no_CE:

 ;Всё аналогично для удаления окончания CODE_CLEAR
lea esi, [edi + edx]
sub esi, code_clear_end_size
cmp BYTE PTR [esi], 60h ;pusha
jnz @@code_encrypt_no_CC

invoke wsprintf, offset buffer1, offset fmt014, edi, edx
invoke LogWrite, offset buffer1

mov edi, esi
mov ecx, code_clear_end_size
mov al, 90h
rep stosb
jmp @@code_encrypt_loop
```

```
 @@code_encrypt_no_CC:
jmp @@code_encrypt_loop

 @@code_encrypt_end:
  endif
```

Результаты работы DLL можно посмотреть в IDA, перебирая адреса из лога, код должен восстановится правильно. Теперь остался только один вид макросов - CODE_REPLACE, но этот макрос представляет собой одну из самых мощных(если не самую мощную) защиту кода, макросу этому посвящается даже свой раздел :)

Восстановление кода после макросов CODE_REPLACE:

Если вспомнить xprotector, то аналогичные макросы там выглядели так:

```
jmp     invalid_opcode
db      "xpro" ;Это просто сигнатура
dd      0 ;0 - начало, 1 - окончание
dd      0
db      "xpro"
 invalid_opcode:
```

Т.е. они выглядели почти также как и в sdk. Делаем предположение, что и в themid'e они также практически не меняются при упаковке. Смотрим SDK:

```
CODEREPLACE_START MACRO

    jmp     @F

 ;Сигнатура уже здесь другая, WL - сокращение от WinLicense как я понимаю
    db      'WL  ' ;
    dd      ID_CODEREPLACE_START
    dd      0
    db      'WL  '

    @@:

ENDM

CODEREPLACE_END MACRO

    jmp     @F

    db      'WL  '
    dd      ID_CODEREPLACE_END
    dd      0
    db      'WL  '

    @@:

ENDM
```

Чтобы найти макрос CODE_REPLACE, достаточно поискать в шестнадцатиричном редакторе "WL ". Первый макрос находится по адресу 0558D23h. Однако испытания "древнего" способа с перехватам NtContinue заканчиваются неудачей. Да не просто неудачей а вообще непонятно чем, в лог не попадает исключение о неправильном опкоде, а это значит, что управление не попадает к векторному обработчику, чего просто не может быть, однако это так. Вспоминая, как Windows NT обрабатывает исключения, начинаем проверять функции, которые задействованы при этой обработке. Первая же функция этого ряда - KiUserExceptionDispatcher радует нас очень интересной инструкцией - jmp near ptr 0BCCA52h, причём адрес 0BCCA52h принадлежит конечно же протектору. Ничего не остаётся, кроме того, как туда заглянуть:

```
WinLicen:00BCCA52 sub_BCCA52      proc near
WinLicen:00BCCA52
WinLicen:00BCCA52 pExceptionRecord= dword ptr  4
WinLicen:00BCCA52 arg_4           = dword ptr  8
 ;Это недокументированный способ проверить версию ОС, WinNT перед нами или Win9x
```

```
WinLicen:00BCCA52                 mov     cx, ds
WinLicen:00BCCA55                 test    cl, 4
WinLicen:00BCCA58                 jz      short @@WinNT
WinLicen:00BCCA5A                 mov     ebx, [esp+arg_4]
WinLicen:00BCCA5E                 mov     ecx, [esp+pExceptionRecord]
WinLicen:00BCCA62                 jmp     short loc_BCCA6F
WinLicen:00BCCA62 ; ---------------------------------------------------------------------------
WinLicen:00BCCA64                 dd 0
WinLicen:00BCCA68 ; ---------------------------------------------------------------------------
WinLicen:00BCCA68
WinLicen:00BCCA68 @@WinNT:                                ; CODE XREF: sub_BCCA52+6.j
WinLicen:00BCCA68                 mov     ecx, [esp+pExceptionRecord]
WinLicen:00BCCA6C                 mov     ebx, [esp+0]
WinLicen:00BCCA6F
WinLicen:00BCCA6F loc_BCCA6F:                             ; CODE XREF: sub_BCCA52+10.j
WinLicen:00BCCA6F                 mov     eax, [ebx]
WinLicen:00BCCA71                 call    $+5
WinLicen:00BCCA76                 pop     ebp
WinLicen:00BCCA77                 sub     ebp, 4D2D18Fh
WinLicen:00BCCA7D                 pusha

 ;Проверка, произошло ли исключение внутри программы или скажем где-то в DLL
WinLicen:00BCCA7E                 mov     eax, [ecx+CONTEXT.Eip]
WinLicen:00BCCA84                 mov     edx, [ebp+xvImageSize]
WinLicen:00BCCA8A                 add     edx, [ebp+xvHandle]
WinLicen:00BCCA90                 cmp     eax, edx
WinLicen:00BCCA92                 ja      short loc_BCCA9C
WinLicen:00BCCA94                 cmp     eax, [ebp+xvHandle]
WinLicen:00BCCA9A                 jnb     short loc_BCCAA4
WinLicen:00BCCA9C
WinLicen:00BCCA9C loc_BCCA9C:                             ; CODE XREF: sub_BCCA52+40.j
WinLicen:00BCCA9C                 mov     eax, [ebp+4BF1AE9h]
WinLicen:00BCCAA2                 jmp     short loc_BCCAFC
WinLicen:00BCCAA4 ; ---------------------------------------------------------------------------

 ;Ожидание, пока IDT не окажется свободна(её могут использовать
 ;другие потоки для своих ring0 дешифровщиков)
WinLicen:00BCCAA4 loc_BCCAA4:                             ; CODE XREF: sub_BCCA52+48.j
WinLicen:00BCCAA4                 mov     edx, 2
WinLicen:00BCCAA9                 mov     eax, [ebp+4BF060Dh]
WinLicen:00BCCAAF
WinLicen:00BCCAAF @@waitforidt_loop:                      ; CODE XREF: sub_BCCA52+6D.j
WinLicen:00BCCAAF                 xchg    dl, [eax]
WinLicen:00BCCAB1                 or      dl, dl
WinLicen:00BCCAB3                 jz      short loc_BCCAC1
WinLicen:00BCCAB5                 pusha
WinLicen:00BCCAB6                 push    1
WinLicen:00BCCAB8                 call    dword ptr [ebp+Sleep]
WinLicen:00BCCABE                 popa
WinLicen:00BCCABF                 jmp     short @@waitforidt_loop
WinLicen:00BCCAC1 ; ---------------------------------------------------------------------------
WinLicen:00BCCAC1
WinLicen:00BCCAC1 loc_BCCAC1:                             ; CODE XREF: sub_BCCA52+61.j
WinLicen:00BCCAC1                 mov     ebx, [ecx+CONTEXT.Eip]

 ;Протектор держит таблицу с адресами исключений
WinLicen:00BCCAC7                 call    find_exception_address
WinLicen:00BCCACC                 or      eax, eax

 ;Переход, если адреса нету в таблице
WinLicen:00BCCACE                 jz      short loc_BCCADB
WinLicen:00BCCAD0                 mov     eax, [ebp+4BF060Dh]
WinLicen:00BCCAD6                 mov     byte ptr [eax], 0

 ;И переход на реальную функцию KiUserExceptionDispatcher если исключение есть в таблице
WinLicen:00BCCAD9                 jmp     short loc_BCCAFC
WinLicen:00BCCADB ; ---------------------------------------------------------------------------
WinLicen:00BCCADB
WinLicen:00BCCADB loc_BCCADB:                             ; CODE XREF: sub_BCCA52+7C.j
WinLicen:00BCCADB                 mov     eax, [ecx+CONTEXT.Eip]
WinLicen:00BCCAE1                 mov     [ebp+xvOldEIP], eax
```

```
 ;Информация об исключении теряется, управление передаётся внутрь протектора, контекст
 ;восстанавливается функцией NtContinue.
WinLicen:00BCCAE7                 lea     eax, [ebp+xfExceptionHandlerProc]
WinLicen:00BCCAED                 mov     [ecx+CONTEXT.Eip], eax
WinLicen:00BCCAF3                 push    0
WinLicen:00BCCAF5                 push    ecx
WinLicen:00BCCAF6                 call    dword ptr [ebp+NtContinue]
WinLicen:00BCCAFC

 ;Переход на функцию ОС, обрабатывающую исключение
WinLicen:00BCCAFC loc_BCCAFC:                             ; CODE XREF: sub_BCCA52+50.j
WinLicen:00BCCAFC                                         ; sub_BCCA52+87.j
WinLicen:00BCCAFC                 mov     ax, ds
WinLicen:00BCCAFF                 test    al, 4
WinLicen:00BCCB01                 jz      short loc_BCCB13
WinLicen:00BCCB03                 mov     eax, [ebp+4BF23E1h]
WinLicen:00BCCB09                 add     eax, 8
WinLicen:00BCCB0C                 add     eax, 42h ; 'B'
WinLicen:00BCCB11                 jmp     short loc_BCCB19
WinLicen:00BCCB13 ; ---------------------------------------------------------------------------
WinLicen:00BCCB13
WinLicen:00BCCB13 loc_BCCB13:                             ; CODE XREF: sub_BCCA52+AF.j
WinLicen:00BCCB13                 lea     eax, [ebp+4D218E3h]
WinLicen:00BCCB19
WinLicen:00BCCB19 loc_BCCB19:                             ; CODE XREF: sub_BCCA52+BF.j
WinLicen:00BCCB19                 lea     ebx, [ebp+4D2D23Ch]
WinLicen:00BCCB1F                 mov     [ebx+1], eax
WinLicen:00BCCB22                 popa
WinLicen:00BCCB23                 push    12345678h
WinLicen:00BCCB28                 retn
WinLicen:00BCCB28 sub_BCCA52      endp ; sp = -0Ch
```

Исходя из вышестоящего кода, обработка исключения передаётся операционной системе, если адреса этого исключения нет в какой-то внутренней таблице процессора, при этом информация об исключении теряется. Могут быть два случая - произошло обычное исключение, или управление попало на макрос CODE_REPLACE. Очевидно, что в первом случае управление должно вернуться на адрес, вызвавший исключение, а потом опять на 00BCCA52h, и уже потом на KiUserExceptionDispatcher, при этом адрес должен был быть занесён в таблицу. Во втором случае просто выполнится код, который протектор вытащил изнутри макроса, и управление после этого попадёт за пределы макроса. Посмотрим на код xfExceptionHandlerProc, хотя толку от этого мало будет :)

```
 ;Код этот так сильно разбавлен мусором, что без плагина невозможно вообще понять
 ;что к чему. С плагином идёт описание, как удалять мусорные конструкции,
 ;здесь оно точно уж пригодится
WinLicen:00BBBE5F                 pushf
WinLicen:00BBBE60                 pusha
WinLicen:00BBBE61                 call    $+5
WinLicen:00BBBE66                 pop     ebp
WinLicen:00BBBE67                 sub     ebp, 4D1C57Fh
WinLicen:00BBC05F                 lea     eax, [ebp+4D1DC64h]
WinLicen:00BBC2CC                 push    eax
WinLicen:00BBC2D0                 mov     [esp], eax
WinLicen:00BBC2F0                 cmp     dword ptr [ebp+4BF2961h], 0
WinLicen:00BBC2F7                 jz      loc_BBC5D4
WinLicen:00BBC5A1                 push    0FFFFFFFFh
WinLicen:00BBC5C8                 call    dword ptr [ebp+4BF2851h]
WinLicen:00BBC5CE                 mov     edi, [ebp+4BF013Dh]
WinLicen:00BBC5D4
WinLicen:00BBC5D4 loc_BBC5D4:                             ; CODE XREF: WinLicen:00BBC2F7.j

 ;...
 ;Весь смысл внутри этого call'а, можно посмотреть и туда
WinLicen:00BBCD50                 call    eax
WinLicen:00BBCD52                 mov     ebx, [ebp+4BF1C7Dh]
WinLicen:00BBCD58                 mov     ecx, [ebp+4BF060Dh]
WinLicen:00BBCF87                 mov     byte ptr [ecx], 0
WinLicen:00BBCF9C                 lea     ecx, [ebp+4D1DC64h]
```

```
WinLicen:00BBD21F                mov     [ecx+1], eax
WinLicen:00BBD2EB                add     esp, 4
WinLicen:00BBD549                popa
WinLicen:00BBD54A                popf
WinLicen:00BBD54B                push    12345678h
WinLicen:00BBD550                retn

 ...

 ;Я уже упоминал о виртуальной машине, именно здесь она и находится
 ;Исследовать у меня желание не возникло, но одно её присутствие уже есть
 ;нехороший знак.
WinLicen:00B1112E sub_B1112E      proc near
WinLicen:00B1112E
WinLicen:00B1112E arg_0           = dword ptr  4
WinLicen:00B1112E arg_4           = dword ptr  8
WinLicen:00B1112E
WinLicen:00B1112E                 pusha
WinLicen:00B11148                 mov     esi, [esp+20h+arg_0]
WinLicen:00B1114C                 xor     [ebp+4BF168Dh], edi
WinLicen:00B11152                 mov     ebx, [esp+20h+arg_4]
WinLicen:00B11158                 mov     edi, [ebp+4C697FCh]
WinLicen:00B1115E                 mov     [ebp+4BF070Dh], ecx
WinLicen:00B11164                 mov     edx, [ebp+4C6980Ch]
WinLicen:00B11176                 mov     dword ptr [ebp+4C6981Ch], 0
WinLicen:00B1118A                 mov     dword ptr [ebp+4C69820h], 0
WinLicen:00B111C3                 mov     eax, 1Fh
WinLicen:00B111DB                 shl     eax, 2
WinLicen:00B1121A                 mov     ecx, [ebp+4C69828h]
WinLicen:00B11220                 mov     [ebp+4BF0A25h], eax
WinLicen:00B11226                 mov     [eax+edx], ecx
WinLicen:00B1122A                 mov     [ebp+4BF1DE9h], edi
WinLicen:00B11230                 lea     eax, [eax+edx]
WinLicen:00B11255                 mov     [ebp+4C69824h], eax
WinLicen:00B11267                 jmp     loc_B1133C
WinLicen:00B1126C ; ---------------------------------------------------------------------------
WinLicen:00B1126C                 mov     [ebp+4BF29E9h], eax
WinLicen:00B11272
WinLicen:00B11272 loc_B11272:                             ; CODE XREF: sub_B1112E+210.j
WinLicen:00B11272                 movzx   eax, byte ptr [esi]
WinLicen:00B1128E                 shl     eax, 2
WinLicen:00B112B0                 call    dword ptr [eax+edi]
WinLicen:00B112B3                 mov     [ebp+4BF1D81h], edi
WinLicen:00B112B9                 inc     dword ptr [ebp+4C69820h]
WinLicen:00B112CB                 mov     eax, [ebp+4C69820h]
WinLicen:00B112F9                 mov     ecx, 0Ah
WinLicen:00B11316                 mul     cl
WinLicen:00B11329                 mov     esi, [esp+20h+arg_0]
WinLicen:00B11334                 add     esi, eax
WinLicen:00B11336                 sub     [ebp+4BF0571h], eax
WinLicen:00B1133C
WinLicen:00B1133C loc_B1133C:                             ; CODE XREF: sub_B1112E+139.j
WinLicen:00B1133C                 cmp     esi, ebx
WinLicen:00B1133E                 jb      loc_B11272
WinLicen:00B11345                 popa
WinLicen:00B11346                 sub     [ebp+4BF1BBDh], ebx
WinLicen:00B1134C                 retn    8
WinLicen:00B1134C sub_B1112E      endp
```

Практически можно сдаваться, ведь легкое восстановление кода из CODE_REPLACE не представляется возможным, но..

```
WinLicen:00BCCADB                 mov     eax, [ecx+CONTEXT.Eip]
WinLicen:00BCCAE1                 mov     [ebp+xvOldEIP], eax ;где xvOldEIP = 4BF14B9h
```

Если код проверки на принадлежность адреса исключения к CODE_REPLACE лежит в открытом виде, то HEX-редакторе можно будет найти DWORD 4BF14B9h. И действительно, найти его можно и не раз. Первое вхождение 7CCAE3 выводит нас на перехваченную функцию KiUserExceptionDispatcher, но уже следующее выводит на совсем другой код. Он также очень

сильно разбавлен мусором, поэтому придётся несколько минут посидеть, прежде чем раскопать окрестности вхождения и выделить нужный код, который обслуживает макрос(т.е. сверяет адрес из xvOldEIP с таблицей адресов, где применены макросы). Смотрим на этот код, на этот раз толку будет хоть отбавляй:

```
 ;Вот где она, таблица-то..
WinLicen:00BE0848                lea     edi, [ebp+xaCodeReplaceTable]
WinLicen:00BE0854                add     edi, 5
WinLicen:00BE0857                mov     edx, ebx

 ;Записываем в переменную адрес таблицы
WinLicen:00BE0931                mov     [ebp+xvCodeReplaceTable], edi
WinLicen:00BE095E                lea     ecx, [ebp+xaCodeReplaceTableEnd]
WinLicen:00BE0988                sub     ecx, edi
WinLicen:00BE0BE1                shr     ecx, 2
WinLicen:00BE0E45                inc     ecx
WinLicen:00BE0E46                sub     edx, 4A202D45h
WinLicen:00BE0F40                mov     [ebp+4D3D93Bh], ecx
WinLicen:00BE11AB                mov     edx, [ebp+4BF039Dh]

 ;Таблица лежит в зашифрованном виде, нужны два ключа, чтоб запустить
 ;"мощный" криптоалгоритм для её расшифровки
WinLicen:00BE11B1                mov     eax, [ebp+xvCodeReplaceKey1]
WinLicen:00BE11D6                mov     ebx, [ebp+xvCodeReplaceKey2]
WinLicen:00BE1307                call    _codereplace_decrypt

 ;Получаем RVA адреса исключения и ищем его в таблице
WinLicen:00BE14F5                mov     ebx, [ebp+xvOldEIP]
WinLicen:00BE177F                sub     ebx, [ebp+xvHandle]
WinLicen:00BE1A01                mov     esi, [ebp+xvCodeReplaceTable]
WinLicen:00BE1A07                mov     edx, 4BBF4CCEh
WinLicen:00BE1A0C                lea     edi, [ebp+xaCodeReplaceTableEnd]
WinLicen:00BE1A12                push    ebx
WinLicen:00BE1A13                mov     dx, 1246h
WinLicen:00BE1A17                pop     edx
WinLicen:00BE1A18                call    _codereplace_find
WinLicen:00BE1C37                or      eax, eax

 ;Если исключение никак не связано с CODE_REPLACE, то вот переход выполнится
WinLicen:00BE1C39                jz      _no_codereplace

 ;Расчёт адреса для перехода на краденный из макроса код,
 ;Видно, что в таблице третий элемент это RVA - CodeReplaceConst1
WinLicen:00BE1C5B                mov     eax, [esi+8]
WinLicen:00BE1C7E                sub     ebx, [esi]
WinLicen:00BE1D32                add     eax, ebx
WinLicen:00BE1D3C                add     eax, [ebp+xvCodeReplaceConst1]
WinLicen:00BE1DEF                add     eax, [ebp+xvHandle]

 ;Запись нового адреса для передачи управления
WinLicen:00BE207F                mov     [ebp+xvOldEIP], eax
WinLicen:00BE208E                jmp     loc_BE258E

 ;Если это не макрос, то xvOldEIP не меняется, управление попадёт туда для
 ;повторной обработки исключения протектором
WinLicen:00BE2096 _no_codereplace:                       ; CODE XREF: WinLicen:00BE1C39.j
WinLicen:00BE2096                mov     ebx, [ebp+xvOldEIP]
WinLicen:00BE22A6                call    loc_BE423D
```

После такого описание думаю понятно что делать с краденным кодом, достаточно найти таблицу, расщифровать её, и просто скопировать код на место, не забыв подправить относительные смещения для длинных переходов. Другие способы, основанные на поиске макросов по сигнатуре "WL " недееспособны, т.к. при упаковке есть такая опция, как автоматическое создание CODE_REPLACE макросов, но вот в таблице они будут все, вне зависимости от того, автомитически ли они были созданы или нет. Теперь пара слов о восстановлении:

```
 ;CODE_REPLACE constants
code_replace_table equ  00BDD184h ;адрес таблицы
```

```
code_replace_n  equ  0Ch ;число макросов
code_replace_key1 equ  00BDD226h ;первый ключ ..
code_replace_key2 equ  00BDD22Ah ;.. и второй ключ
code_replace_const1 equ  0FB8E7h ;константа для суммирования с 3-им элементом таблицы
code_replace_decrypt equ  00AC9B3Dh ;функция расшифровки таблицы
code_replace_macrosize equ  12h ;размер начала макроса из SDK

 ;...

; ========== CODE_REPLACE destroyer ==========
  ifdef code_replace_table
  ifdef code_replace_n
  ifdef code_replace_key1
  ifdef code_replace_key2
  ifdef code_replace_const1
  ifdef code_replace_decrypt

 ;Расшифровываем таблицу
mov edi, code_replace_table
mov eax, code_replace_key1
mov eax, [eax]
mov ebx, code_replace_key2
mov ebx, [ebx]
mov ecx, code_replace_n
imul ecx, 3

mov ebp, code_replace_decrypt
call ebp

 ;Начинается цикл копирования и правки кода
mov ebp, code_replace_table
mov ebx, code_replace_n

 @@code_replace_loop:

 ;Первый элемент в таблице - RVA макроса в коде, второй - RVA конца
 ;макроса, третий - RVA адреса, куда протектор спрятал украденный код
mov esi, [ebp+8]
add esi, code_replace_const1
add esi, MHandle

mov edi, [ebp]
add edi, MHandle

mov ecx, [ebp+4]
add ecx, MHandle
sub ecx, edi

push ecx
push esi
push edi

 ;сначала копирование
rep movsb

pop edi
pop esi
pop ecx

 ;теперь правда смещений
invoke code_replace_fix, edi, esi, ecx

 ;А потом удаление всяких сигнатур "WL  ", в общем
 ;удаление всего лишнего
cmp DWORD PTR [edi - 4], 20204C57h
jnz @@code_replace_iter

push edi
push ecx
push ecx
mov al, 90h
```

```
mov ecx, code_replace_macrosize
sub edi, ecx
rep stosb

pop ecx
add edi, ecx
mov ecx, code_replace_macrosize
rep stosb

pop ecx
pop edi

 ;Вывод в лог и итерация цикла
 @@code_replace_iter:
invoke wsprintf, offset buffer1, offset fmt015, edi, ecx
invoke LogWrite, offset buffer1

add ebp, 0Ch
dec ebx
jnz @@code_replace_loop
  endif
  endif
  endif
  endif
  endif
  endif
```

Всё, все ступени защиты преодолены, здесь можно закончить распаковку любой программы, за исключением творений Oreans. Программы, упакованные распакованной WinLicense не хотят работать, выдают исключения. Защита эта встроена в логику работы программы, это сильно усложняет её поиск, но не исключает совсем.

Тест на упакованность(доп. защита от авторов протектора):

Исследование приведу в кратком изложении. Вновь запакованный calc.exe не работает, вылетает с исключением обращения к памяти. Если запускать его через лоадер, то будет видно, в логе будут видны вызовы VirtualAlloc. Перехватывая и снимая дамп сразу после последнего вызова VirtualAlloc перед исключением, быстро находим инструкцию, приводящую к краху:

```
WinLicen:0121F163                push    4
WinLicen:0121F165                push    1000h
WinLicen:0121F16A                push    dword ptr [ebp+7490999h]
WinLicen:0121F170                push    0
WinLicen:0121F172                call    eax
WinLicen:0121F174                test    eax, eax
WinLicen:0121F176                jnz     loc_121F189
WinLicen:0121F17C                mov     eax, 0
WinLicen:0121F181                lea     ecx, [ebp+7498F7Eh]
WinLicen:0121F187                jmp     ecx
WinLicen:0121F189 ; ---------------------------------------------------------------------------
WinLicen:0121F189
WinLicen:0121F189 loc_121F189:                           ; CODE XREF: WinLicen:0121F176.j
WinLicen:0121F189                mov     ecx, eax
WinLicen:0121F18B                mov     eax, ebx
WinLicen:0121F18D                add     eax, [eax+3Ch]
WinLicen:0121F190                add     eax, 0F8h
WinLicen:0121F195                mov     edx, [eax+0Ch]
WinLicen:0121F198                add     edx, ebx
WinLicen:0121F19A                cmp     dword ptr [ebp+74910A1h], 0
WinLicen:0121F1A1                jz      loc_121F1B5
WinLicen:0121F1A7                mov     ebx, [ebp+74910A1h]
WinLicen:0121F1AD                mov     eax, [ebp+7491D61h]
WinLicen:0121F1B3                mov     [ebx], eax
WinLicen:0121F1B5
WinLicen:0121F1B5 loc_121F1B5:                           ; CODE XREF: WinLicen:0121F1A1.j
WinLicen:0121F1B5                push    ecx
WinLicen:0121F1B6                push    edx

 ;Смещение относительно EBP не изменилось, но если вспомнить xprotector,
```

```
 ;то там игнорировались все смещения
WinLicen:0121F1B7                lea     eax, [ebp+5A6E80h]
WinLicen:0121F1BD                call    eax
```

Пользуясь тем, что код это лежит в открытом виде, находим его в дампе по адресу 005A7171. По перекрестным ссылкам можно найти код, который учавствует в упаковке и как раз обрабатывает такие куски:

```
 ;Видим кучки nop'ов оставшиеся после удаления макросов wsprintf
___:005A7227                nop
___:005A7228                nop
___:005A7229                nop
___:005A722A                nop
___:005A722B                nop
___:005A722C                nop
___:005A722D                nop
___:005A722E                cmp     [ebp+arg_4], 0
___:005A7232                jnz     short loc_5A7251
___:005A7234                mov     eax, offset loc_5A70BF
___:005A7239                mov     ebx, offset sub_5A7205
___:005A723E                push    ebx
___:005A723F                push    eax
___:005A7240                call    sub_5BFFE7
___:005A7245                mov     dword_78D32F, 0
___:005A724F                jmp     short loc_5A72AD
___:005A7251 ; ---------------------------------------------------------------------------
___:005A7251
___:005A7251 loc_5A7251:                            ; CODE XREF: sub_5A7205+2D.j
___:005A7251                push    0
___:005A7253                call    sub_5C0881
___:005A7258                push    0
___:005A725A                call    sub_5C1F49
___:005A725F                push    0
___:005A7261                call    sub_5C1F28
___:005A7266                push    0
___:005A7268                push    0
___:005A726A                push    0
___:005A726C                push    64h
___:005A726E                call    sub_5C091B
___:005A7273                call    sub_5C2538
___:005A7278                call    sub_5C2549
___:005A727D                call    sub_5C2551
___:005A7282                call    sub_5C2559
___:005A7287                call    sub_5C2561
___:005A728C                call    sub_5C2579
___:005A7291                call    sub_5C2571
___:005A7296                mov     eax, offset loc_5A70BF
___:005A729B                mov     ebx, offset sub_5A7205
___:005A72A0                push    [ebp+arg_0]
___:005A72A3                push    ebx
___:005A72A4                push    eax
___:005A72A5                call    sub_5C01D7
```

Как и в xprotector'e здесь тоже поддерживается массив для обработки перемещаемых элементов(как relocations в DLL). Функция 5C01D7h как раз заполняет этот массив, куда добавляются все адреса внутри обрабатываемого блока(в данном случае от 5A70BFh до 5A7205h). Но наше необработанное(5A6E80h) смещение в данный блок не попадает, попадает оно в другой блок, обработка которого ведётся в процедуре по адресу 5A6FCFh, но оказывается управление вообще не попадает на этот адрес! Попадать оно туда естесственно должно, значит защита где-то рядом. Протектор при упаковке собирает адреса всех таких блоков в единый массив и по очереди их вызывает по два раза, один раз для инициализации, второй для создания релоков(могу ошибаться), и адрес 5A6FCFh почему-то не попадает в этот массив. По xref'ам выходим на такой код:

```
___:0055C309 loc_55C309:                            ; CODE XREF: sub_55BE60+49D
___:0055C309                mov     al, 9
___:0055C30B                and     eax, 0FFh
```

```
___:0055C310                add     eax, ebx
___:0055C312                sub     eax, edx
___:0055C314                imul    eax, ecx

 ;Вызывается вот эта интересная функция..
___:0055C317                call    sub_5C1B38
___:0055C31C                or      eax, eax

 ;И если функция возвращает 1, то адрес блока не передаётся в функцию,
 ;да и сама функция вообще не вызывается
___:0055C31E                jnz     short loc_55C33E
___:0055C320                push    offset sub_5BEDDD
___:0055C325                call    sub_55B63C
___:0055C32A                push    offset sub_5A6FCF
___:0055C32F                call    sub_55B63C
___:0055C334                push    offset sub_5A6181
___:0055C339                call    sub_55B63C
___:0055C33E
___:0055C33E loc_55C33E:                            ; CODE XREF: sub_55BE60+4BE
___:0055C33E                cmp     dword_7B8254, 0
___:0055C345                jz      short loc_55C371

 ...

 ;Смотрим теперь что там за функция такая вызывается:
___:005C1B38 sub_5C1B38     proc near               ; CODE XREF: sub_558B00+12E.p
___:005C1B38                                        ; sub_55BE60+4B7.p
___:005C1B38                call    GetThreadsCount
___:005C1B3D                cmp     eax, 20
___:005C1B40                jbe     short loc_5C1B49
___:005C1B42                mov     eax, 1
___:005C1B47                jmp     short locret_5C1B4E
___:005C1B49 ; ---------------------------------------------------------------------------
___:005C1B49
___:005C1B49 loc_5C1B49:                            ; CODE XREF: sub_5C1B38+8.j
___:005C1B49                mov     eax, 0
___:005C1B4E
___:005C1B4E locret_5C1B4E:                         ; CODE XREF: sub_5C1B38+F.j
___:005C1B4E                retn
___:005C1B4E sub_5C1B38     endp
```

Вот где корень зла :) Функция возвращает единицу, если количество тредов больше 20 и 0 если меньше. Если вспомнить, сколько тредов создаёт протектор при своей работе, то становится понятно, что функция вернёт 1 если WinLicense запустили в упакованном виде и 0, если запустили дамп. После патча, который сделает, чтобы эта функция всегда возвращает 1, прогамма WinLicense не узнает, что её распаковали и будет запаковывать всё подряд и без ошибок, что собственно от ней и требуется. Наконец после этого патча протектор оказывается побеждённым полностью, уррааа!! :)

Послесловие:

Хочется предупредить, но похоже это последний случай, когда themida снималась так легко, уже после написания статьи автор обновил своё творение, и среди обновлений появился новый макрос, по сравнению с которым приведённый выше CODE_REPLACE отдыхает. Новинка превращает пользовательский код в псевдо-код для виртульной машины, это уже очень много говорит о силе защиты. Вообще будущее защиты именно за виртуальными машинами и обфускацией кода, и пора думать как с этим бороться.. На этом пожалуй всё.

Исходники и инструменты: здесь (находится в архиве `wasm_files.zip`: `files/0392/th_adv.rar`)

Использование отладочных регистров процессора Intel X86 в прикладной программе Windows NT

Автор: Rom Lameroff

Дата: 2006-02-19

1. **Введение.**
2. **Описание отладочных регистров.**
3. **Векторная обработка исключений в Windows NT.**
4. **Установка точек останова из прикладной программы.**
5. **Пример использования на практике.**
6. **Заключение**
7. **Литература**

Введение.

Наверное, сейчас ни для кого не секрет, какой огромный выигрыш даёт использование отладочных регистров процессора для исследования механизма защиты программ. Яркий тому пример отладчик SoftIce, который использует эти регистры для установки аппаратных точек останова в программе (команды типа ‘BPM’). Но как использовать их в прикладной программе, где прямое обращение к ним вызывает общую ошибку защиты?

Неужели нет способов, кроме, как написания собственных драйверов ядра, которое само по себе является достаточно трудоёмкой задачей, отпугивающей от их применения? Оказывается, есть, и достаточно простой. Об этом и пойдёт речь в этой статье.

Описание отладочных регистров.

Процессор Intel X86 содержит в своём составе восемь 32 – разрядных отладочных регистров, из которых используется только шесть, это DR0 – DR3, DR6 и DR7, обращение к которым производится с помощью привилегированной команды ассемблера MOV. Команда выполняется, либо в реальном режиме работы процессора, либо в защищищённом, на нулевом кольце защиты. Регистры позволяют установить четыре аппаратных точки останова одновременно. Ниже приводится детальное описание каждого из используемых регистров.

DR7 – регистр управления отладкой, назначение битов такое:

```
Биты 31 - 30: размер точки останова 3,
 00 - 1 байт,
 01 - 2 байта,
 10 - не используется,
 11 - 4 байта.
Биты 29 - 28: тип точки останова 3,
 00 - при выполнении команды,
 01 - при записи в ячейку памяти,
 10 - при обращении к порту ввода - вывода,
 11 - при чтении или записи в ячейку памяти.
Биты 27 - 26: размер точки останова 2.
Биты 25 - 24: тип точки останова 2.
Биты 23 - 22: размер точки останова 1.
Биты 21 - 20: тип точки останова 1.
Биты 19 - 18: размер точки останова 0.
Биты 17 - 16: тип точки останова 0.
Биты 15 - 14: не используются.
Бит 13 -  обращение к любому из отладочных регистров вызывает
          генерацию процессором отладочного исключения #DB,
    сбрасывается аппаратно на входе в обработчик исключения.
Биты 12 - 10: не используются.
Бит 9 -   разрешает использование глобальных точек останова.
Бит 8 -   разрешает использование локальных точек останова.
```

```
Бит 7 -   включение глобальной точки останова 3.
Бит 6 -   включение локальной точки останова 3.
Бит 5 -   включение глобальной точки останова 2.
Бит 4 -   включение локальной точки останова 2.
Бит 3 -   включение глобальной точки останова 1.
Бит 2 -   включение локальной точки останова 1.
Бит 1 -   включение глобальной точки останова 0.
Бит 0 -  включение локальной точки останова 0.
```

Различие глобальных и локальных точек останова в том, что биты локальных точек, сбрасываются при переключении задачи, а глобальных остаются неизменными.

DR6 – регистр состояния отладки, содержит информацию о причине отладочного останова для обработчика исключения #DB, биты такие:

```
Биты 31 - 16:  не используются.
Бит 15 - переключение на задачу с установленным
         отладочным битом в TSS.
Бит 14 - выполнена одна команда при установленном флаге TF в
         регистре EFLAGS (пошаговый режим отладки, флаг TF
   сбрасывается аппаратно на входе  в обработчик исключения).
Бит 13 - произошло обращение к отладочному регистру, при
         установленном бите 13 в регистре DR7.
Биты 12 - 4: не используются.
Бит 3 - произошёл останов в точке 3.
Бит 2 - произошёл останов в точке 2.
Бит 1 - произошёл останов в точке 1.
Бит 0 - произошёл останов в точке 0.
```

Перед выходом из обработчика исключения этот регистр необходимо сбросить в нуль.

Регистры DR0 – DR3 содержат *линейные* адреса точек останова 0 – 3, соответственно. Напомню, что *линейный* адрес в защищённом режиме состоит из базового адреса дескриптора памяти, селектор которого содержится в сегментном регистре кода, данных или стека, и смещения внутри этого сегмента, но т.к. в Windows NT базовый адрес равен нулю, то *линейный* адрес и смещение одинаковы.

При срабатывании любой из точек останова, процессором генерируется исключение #DB.

Векторная обработка исключений в Windows NT.

Итак, из описания отладочных регистров, следует, что для их использования, нам необходимо каким – то образом обработать отладочное исключение #DB. Первое, что приходит на ум, это перехватить вектор прерывания, в котором обрабатывается это исключение, но сделать это из прикладной программы, без написания драйвера ядра, невозможно. Второе – это написать собственный отладчик, занятие тоже довольно утомительное.

К счастью, начиная с Windows 2000, в SDK включена векторная обработка исключений. Ниже приводится детальное описание всех функций и структур поддержки.

PVOID AddVectoredExceptionHandler (

ULONG FirstHandler,

PVECTORED_EXCEPTION_HANDLER VectoredHandler __);

ULONG RemoveVectoredExceptionHandler (

PVOID __VectoredHandlerHandle_ _**);**

Первая – устанавливает, а вторая – удаляет обработчик, прототип которого:

LONG WINAPI VectoredHandler (

PEXCEPTION_POINTERS __ExceptionInfo_ _**);**

Функция должна возвратить одно из двух значений:

• EXCEPTION_CONTINUE_EXECUTION, или – 1, если обработка исключения закончена.
• EXCEPTION_CONTINUE_SEARCH, или 0, для поиска и вызова другого обработчика.

FirstHandler – очерёдность вызова обработчика, если не 0, то первым.

VectoredHandler – указатель на обработчик.

VectoredHandlerHandle – указатель на обработчик, возвращаемый при его установке.

Функцию **RemoveVectoredExceptionHandler** нужно, по завершении программы, вызывать обязательно.

ExceptionInfo – указатель на структуру EXCEPTION_POINTERS, которая определена, как:

typedef struct _EXCEPTION_POINTERS {
PEXCEPTION_RECORD *ExceptionRecord* ;
PCONTEXT *ContextRecord* ;

} EXCEPTION_POINTERS, *PEXCEPTION_POINTERS;

ExceptionRecord – указатель на структуру EXCEPTION_RECORD:

typedef struct _EXCEPTION_RECORD {
DWORD *ExceptionCode* ;
DWORD *ExceptionFlags* ;
struct *EXCEPTION_RECORD* *_*NestedExceptionRecord* ;
PVOID *ExceptionAddress* ;
DWORD *NumberParameters* ;
ULONG_PTR _ExceptionInformation _[EXCEPTION_MAXIMUM_PARAMETERS];

} EXCEPTION_RECORD, *PEXCEPTION_RECORD;

ExceptionCode ___-_ коды причины вызова исключения, из которых, нас будет интересовать EXCEPTION_SINGLE_STEP или 0x80000004, соответствующий вектору прерывания 1, который и вызывается процессором при возникновении исключения #DB "Отладочное прерывание" и представляет собой ловушку (*trap*):

• При *пошаговой* трассировке, если установлен флаг TF (бит 8) в регистре EFLAGS;
• При переключении на задачу с установленным отладочным флагом в TSS;
• При срабатывании точки останова по доступу к данным, определённой в отладочных регистрах;

и ошибку (*fault*):

• При срабатывании точки останова по *выполнению* команды, адрес которой определён в отладочных регистрах;

Здесь оговорюсь, для ясности, что при возникновении исключения типа ошибки (*fault*), адрес возврата указывает на команду вызвавшую исключение, а ловушки (*trap*) – на команду, следующую за ней. В этой связи есть один неприятный момент, а именно, невозможность продолжения программы с адреса останова по *выполнению* команды. Это вызвано тем, что разрешение на такое продолжение, без повторной генерации исключения, даёт флаг RF (бит 16) регистра EFLAGS, который устанавливается аппаратно, при выполнении команды ассемблера IRETD, а затем сбрасывается, при успешном выполнении любой команды. Поэтому нужно, либо пропустить команду, на которой произошёл останов, изменив регистр EIP, либо выполнить её в *пошаговом* режиме, установив флаг TF в регистре EFLAGS, либо просто отключить эту точку останова.

ExceptionFlags – флаг продолжения программы, если 0, программа может быть продолжена и, если EXCEPTION_NONCONTINUABLE_EXCEPTION или 0xC0000025, программа должна завершиться.

Nested ExceptionRecord – указатель на структуру, содержащую дополнительную информацию, при возникновении вложенного исключения.

ExceptionAddress – адрес возврата из исключения.

NumberParameters – количество элементов в массиве *ExceptionInformation*.

ExceptionInformation – массив дополнительных параметров, передаваемых при генерации исключения функцией **RaiseException**.

ContextRecord – указатель на структуру CONTEXT, которая определена в файле WinNT.h, и содержит значения всех регистров процессора, включая отладочные. Ниже приводится структура для процессоров Intel X86, в том же самом виде, за исключением комментариев. Названия членов структуры, говорят сами за себя, поэтому описывать их здесь не буду.

```
typedef struct _CONTEXT {

    //
    // Набор флагов, в зависимости от которых, может быть считана
    // или записана конкретная секция структуры.
    //
    DWORD ContextFlags;

    //
    // Эта секция может быть считана или записана, если ContextFlags
    // содержит флаг CONTEXT_DEBUG_REGISTERS.
    // Помните, что CONTEXT_DEBUG_REGISTERS не включен в CONTEXT_FULL.
    //
    DWORD   Dr0;
    DWORD   Dr1;
    DWORD   Dr2;
    DWORD   Dr3;
    DWORD   Dr6;
    DWORD   Dr7;

    //
    // Эта секция может быть считана или записана, если ContextFlags
    // содержит флаг CONTEXT_FLOATING_POINT.
    //
    FLOATING_SAVE_AREA FloatSave;

    //
    // Эта секция может быть считана или записана, если ContextFlags
    // содержит флаг CONTEXT_SEGMENTS.
    //
    DWORD   SegGs;
    DWORD   SegFs;
    DWORD   SegEs;
    DWORD   SegDs;

    //
    // Эта секция может быть считана или записана, если ContextFlags
    // содержит флаг CONTEXT_INTEGER.
    //
    DWORD   Edi;
    DWORD   Esi;
    DWORD   Ebx;
    DWORD   Edx;
    DWORD   Ecx;
    DWORD   Eax;

    //
    // Эта секция может быть считана или записана, если ContextFlags
    // содержит флаг CONTEXT_CONTROL.
    //
    DWORD   Ebp;
    DWORD   Eip;
    DWORD   SegCs;
    DWORD   EFlags;
    DWORD   Esp;
    DWORD   SegSs;
    //
```

```
    // Эта секция может быть считана или записана, если ContextFlags
    // содержит флаг CONTEXT_EXTENDED_REGISTERS.
    // Содержание и формат зависят от конкретного процессора.
    //
    BYTE    ExtendedRegisters[MAXIMUM_SUPPORTED_EXTENSION];

} CONTEXT;
```

Все возможные значения флагов также определены в файле WinNT.h.

Установка точек останова из прикладной программы.

Для установки точек останова, будем использовать следующие функции API Windows:

BOOL GetThreadContext(

HANDLE *hThread***,**

LPCONTEXT _lpContext _**);**

BOOL SetThreadContext(

HANDLE *hThread***,**

const CONTEXT* *lpContext* __**);**

hThread – дескриптор потока.

lpContext – указатель на структуру CONTEXT, описанную выше.

В документации на эти функции сказано, что перед их использованием, нужно остановить поток, вызвав функцию **SuspendThread** , а впоследствии запустить, вызвав **ResumeThread.** Это не вызывает затруднений, если наш код исполняется в другом потоке, но тогда возникают проблемы с синхронизацией, к тому же его легко обнаружить и уничтожить. Здесь приходит на помощь, то обстоятельство, что контекст потока можно изменять частями, установив определённый флаг в ContextFlags. Практика показывает, что изменение только *отладочных регистров* из основного потока приложения вполне допускается без его останова, и не приводит ни к каким непредсказуемым результатам. Поэтому так и поступим дальше.

Пример использования на практике.

Прежде, чем перейти к разбору примера, необходимо решить, каким образом наш код будет подключен к прикладной программе. Есть много разных способов, но оптимальным, с моей точки зрения, является оформление его в виде DLL. Для подключения её, достаточно написать небольшой *patch* на ассемблере, что – то типа:

```
 org EntryPoint      ; точка входа
start:        ; программы
 push name       ; имя dll
 push start       ; адрес возврата
 mov  dword ptr[start+6], offset ExitProcess ; выход по ошибке
 jmp  LoadLibraryA      ; загрузка dll
name:
 db   'mydll', 0
```

Естественно, кодовый сегмент программы, должен быть доступен для чтения, записи. При успешной загрузке DLL, как известно, будет вызвана функция **DllMain** , в теле которой, будет восстановлен исходный код программы, установлен обработчик исключений и точки останова. Также там можно восстановить исходный вид файла программы и т.д. и т.п. Далее программа, как ни в чём не бывало, запускается с нашей DLL. В случае невозможности загрузки DLL, программа завершается, вызовом **ExitProcess**. Но, достаточно теории, перейдём к практике, для этого напишем простенький “crackme ” , в виде консольной программы:

```
#include "stdafx.h"
#include

TCHAR registered[] =
"Спасибо за покупку нашей программы!\n\
Правом пользования обладает: Rom Lameroff.";

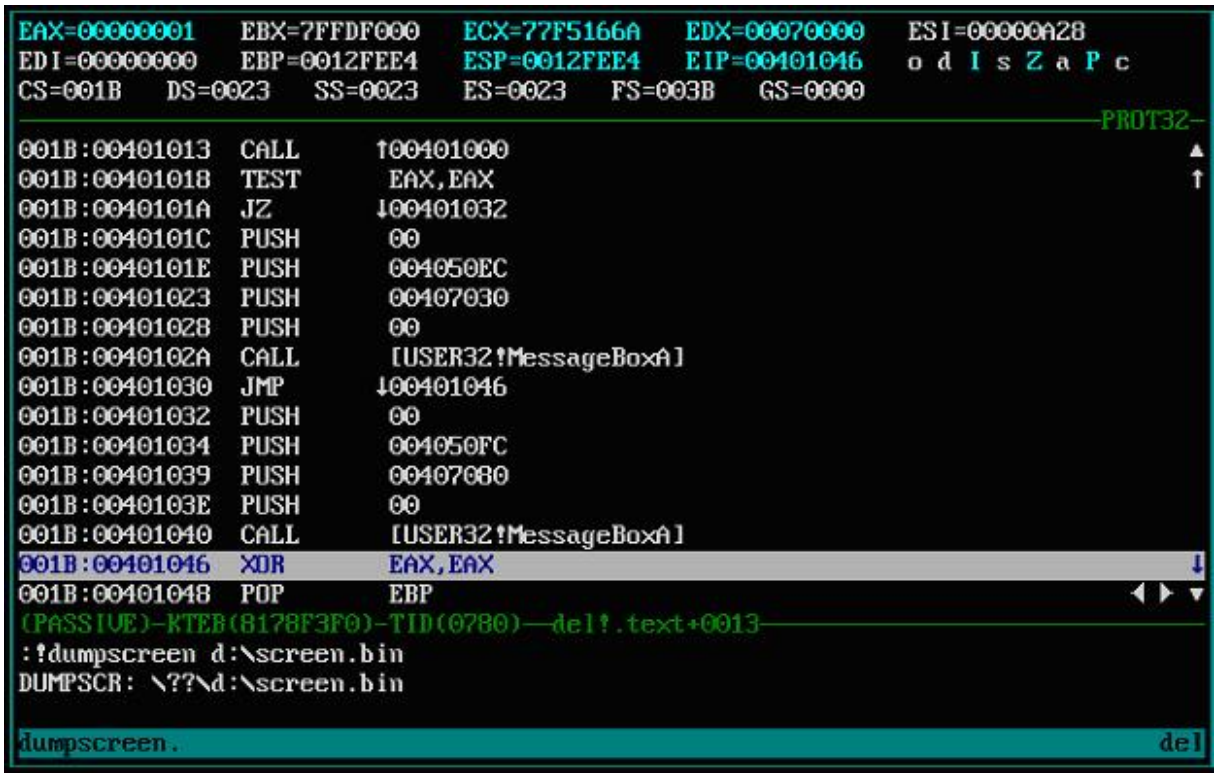
TCHAR unregistered[] =
"Вы используете незарегистрированную \
копию программы.\nДальше так продолжаться \
не может!\nВы должны купить её за 1000 у.е.";


BOOL Registration()
{
 return FALSE;
}

int _tmain( int argc, _TCHAR* argv[] )
{

 if( Registration() )
 {
  MessageBox( NULL, registered, "Registered copy", MB_OK );
 }
 else
 {
  MessageBox( NULL, unregistered, "Unregistered copy", MB_OK );
 }
 return 0;
}
```

Программу компилируем с ключом /SECTION:.text,ERW и без оптимизации. Запускаем её под SoftIce, запоминаем адрес точки входа, у меня он равен 40106F, а также 32 байта кода с этой точки. Вводим код нашей “заплатки”, используя команду ‘A’, и также запоминаем его. Восстанавливаем исходный код, ставим точку останова на **MessageBoxA** , жмём F5, F12. У меня экран выглядит так:



По адресу 401013 мы видим вызов функции регистрации, туда и будем устанавливать точку останова по выполнеию команды, пропустим сам вызов, увеличив EIP на пять байтов, столько

занимает эта команда, и установим регистр EAX в единицу, т.е. сымитируем вызов функции регистрации так, как - будто она возвратила TRUE. Осталось претворить это в жизнь, написав библиотеку. Вот она:

```
#include "stdafx.h"

#define  _WIN32_WINNT 0x500
#include

#define ENTRYPOINT 0x40106F
#define BREAKPOINT 0x401013

BYTE original[] = {
 0x6A, 0x18, 0x68, 0x10, 0x51, 0x40, 0x00, 0xE8,
 0x0D, 0x0D, 0x00, 0x00, 0xBF, 0x94, 0x00, 0x00,
 0x00, 0x8B, 0xC7, 0xE8, 0x59, 0x0E, 0x00, 0x00,
 0x89, 0x65, 0xE8, 0x8B, 0xF4, 0x89, 0x3E, 0x56
};

BOOL SetBreakPoint( DWORD dwAddress )
{
 CONTEXT ct;
      HANDLE hThread = GetCurrentThread();

 ct.ContextFlags = CONTEXT_DEBUG_REGISTERS;
 if( !GetThreadContext( hThread, &ct ) )
  return FALSE;

 ct.Dr0 = dwAddress;
 ct.Dr6 = 0;
 ct.Dr7 = ( ct.Dr7 & 0xFFF0FFFF ) | 0x101;

 return SetThreadContext( hThread, &ct );
}

LONG WINAPI MyVectoredHandler( PEXCEPTION_POINTERS ExceptionInfo )
{
 if( ExceptionInfo->ContextRecord->Dr6 & 1 )
 {
  ExceptionInfo->ContextRecord->Eip += 5;
  ExceptionInfo->ContextRecord->Eax = TRUE;
  ExceptionInfo->ContextRecord->Dr6 = 0;
  return EXCEPTION_CONTINUE_EXECUTION;
 }
 return EXCEPTION_CONTINUE_SEARCH;
}

BOOL APIENTRY DllMain( HANDLE hModule,
                       DWORD  ul_reason_for_call,
                       LPVOID lpReserved )
{
 static PVOID pv;

 DisableThreadLibraryCalls( (HMODULE)hModule );

 switch ( ul_reason_for_call )
 {
 case DLL_PROCESS_ATTACH:

  CopyMemory( (LPVOID)ENTRYPOINT, original, sizeof(original) );
  pv = AddVectoredExceptionHandler( 1, MyVectoredHandler );
  SetBreakPoint( BREAKPOINT );
  break;

 case DLL_PROCESS_DETACH:

  RemoveVectoredExceptionHandler( pv );
  break;
 }
      return TRUE;
```

```
}
```

Компилируем. В любом HEX – редакторе устанавливаем “заплатку” в файл программы и копируем библиотеку в её папку. Запускаем программу, результат говорит сам за себя.

Заключение.

В заключение, можно сказать, что данная “технология” может с успехом применяться для программ из категории “неломаемых”, чей код может модифицироваться или подвергаться проверке во время выполнения и даёт большой выигрыш во времени, благодаря схожести с трассировкой SoftIce. Из – за реализации на языке высокого уровня, процесс можно даже автоматизировать, написав один раз универсальные функции или, может быть, саму библиотеку, всё ограничивается только фантазией программиста. Конечно нельзя сказать, что защиты от этого нет, но то, что код выполняется раньше кода прикладной программы, даёт огромное преимущество. Вообщем “ищите и обрящите”. С точки зрения законности ничего положительного сказать не могу, но много ли у нас найдётся компьютеров оснащённых полностью лицензированным программным обеспечением, наверное, единицы. Но, тем не менее, прошу данную статью, ни в коем случае, не считать пропагандой нарушения авторских прав, а просто даю пищу для размышления, как потенциальным “взломщикам”, так и прикладным программистам. То, что опубликовано, перестаёт быть тайной и, надеюсь, будет полезно и тем, и другим. По всем вопросам, связанным с данным материалом, пишите мне сюда.

Литература.

1. Зубков С.В. “Assembler. Для DOS, Windows и Unix.” – М.: ДМК, 1999. – 640 с.
2. MSDN Library - Visual Studio .NET 2003.
3. IA – 32 Intel® Architecture Software Developer’s Manual.

Тайна золотого ключика или Особенности системы лицензионных ключей, используемой компанией Software AG

Автор: Konstantin

Дата: 2006-03-06

Лицензионный ключ - штука весьма распространенная в современном компьютерном мире, а потому хорошо исследованы как методы защиты, так и нападения Эта статья не имеет своей целью обескуражить мир сообщением о принципиально новом методе подлома/подбора/обхода ключа. Основная цель немного другая

1. Посмотреть, как реализована система лицензирования на примере довольно дорогих коммерческих продуктов от компании Software AG
2. А также показать на реальном примере, что иногда вовсе не обязательно проводить все ночи напролет, закопавшись в отладчике или дизассемблере, чтобы корректно "поправить" лицензионный ключ (цели "правки" могут быть любыми), что иногда достаточно просто пошире открыть глаза…

Чтобы придать статье некую логическую последовательность, попробую дать ответы на вполне закономерные вопросы:

1. Что же это за компания такая - "Software AG"?
2. В чем суть пресловутой системы лицензионных ключей?
3. И в чем, собственно, особенности этой системы?

Software AG В те стародавние времена, когда компьютеры были большими, а программисту при поступлении на работу выдавался не навороченный комп в персональное пользование, а линейка, карандаши, ручка и график работы на "машине" (именно так любовно назывался монстр, стучащий копытом по фальшполу в машзале в окружении ленточных стоек и ЦПУ), эта немецкая компания была более известна, чем сейчас, благодаря своим довольно известным в СССР продуктам ADABAS (СУБД) и NATURAL (язык программирования). Сегодня же редкий специалист по ИТ сможет вспомнить, "что это за зверь, и с чем его едят". **Немного информации о компании (информация взята с ее официального сайта).**

• Название - Software AG (принятое сокращение SAG)
• Официальный сайт - http://www.softwareag.com, http://www.softwareag.com/ru - на русском языке
• Год основания - 1969
• Головной офис - г. Дармштадт (Германия)
• Количество сотрудников - более 2 600 чел
• Валовой доход - 411 млн. евро (по 2004 г)
• Направление деятельности - "разработка, продажа, последующее обслуживание и сопровождение высокоэффективных программных продуктов, служащих основой для интеграции информационных систем предприятия".
• Дополнительно - более 3000 заказчиков (в более чем 60 странах мира)

Линейка основных программных продуктов:

• ADABAS - быстродействующая СУБД, предназначенная для использования в больших системах, работоспособность которых жизненно важна для предприятия
• NATURAL - мощный сервер приложений компании Software AG, позволяющий разработчикам быстро создавать критически важные приложения1

• ENTIREX - надежное, высокопроизводительное интеграционное программное обеспечение, позволяющее организациям использовать функциональность унаследованных систем при разработке новых приложений
• TAMINO XML Server - высокопроизводительный сервер для хранения, управления, публикации и обмена XML - документами

Перечень представительств есть на сайте компании. Если у вас по прочтении статьи возникнет непреодолимое желание познакомиться с продуктами SAG поближе, проще всего обратиться в ближайшее представительство - на сколько я знаю, проблем предоставлением ознакомительных версий нет. Второй вариант - залить дистрибутив с сайта компании.

Что ж, если верить тому что написано, перед нами вполне солидная компания… Похоже, есть что продавать (увы, не в России!), и значит, есть, что лицензировать… Так что там с системой лицензионных ключей?

Система лицензионных ключей

Прежде чем говорить про лицензионные ключи, хотелось бы отметить одну особенность - в продуктах SAG система лицензирования - это не система защиты от взлома. По мнению представителей компании, от взлома их защищают сам договор лицензии и закон. А система лицензирования - скорее средство оградить заказчика от установки продуктов "куда попало" - например, Natural for Windows 2000/XP не будет нормально работать под Windows98/ME (нечто подобное и с unix'ами).

Условности соблюдены, теперь за дело!

Продукты SAG в последнее время используют так называемый "файл лицензии", являющийся по сути xml-файлом, и содержащим информацию о продукте и об условиях его поставки. Именно здесь находится информация о сроке действия лицензии, об операционной системе, о доступных компонентах продукта и проч. Вполне логично, что он используется как при установке продукта (отсюда берется информация о том, какие компоненты устанавливать, а какие нет; проверяется "среда обитания" продукта), так и при эксплуатации (проверяется "среда обитания" продукта). Проверка "Среды обитания" - проверяется заявленная и фактическая ОС, и проверяется срок действия лицензии.

Пример файла лицензии (взят из форумаhttp://tamino.forums.softwareag.com2):

```
<SoftwareAG_License>
 <Component Id="SalesInfo">
  <SerialNumber>0000004680</SerialNumber>
  <LicenseKey>A7B77D8CC2B46E9C5821DEC41B9B9CE0</LicenseKey>
  <CustomerID>SAG</CustomerID>
  <CustomerName>Software AG</CustomerName>
 </Component>
 <Component Id="ProductInfo">
  <ExpirationDate>30days</ExpirationDate>
  <OS>w2000p, w2000s, winxp_pro, winxp_per</OS>
  <ProductCode>INS</ProductCode>
  <ProductID>INS414FSETWIN</ProductID>
  <ProductName>Tamino XML Server</ProductName>
  <ProductVersion>4.1.4</ProductVersion>
  <Usage>XML Starter Kit 4</Usage>
  <RegistryDate>2005/06/30</RegistryDate>
 </Component>
 <Component Id="TaminoServer">
  <XTension>yes</XTension>
  <ConcurrentSessions>3</ConcurrentSessions>
  <DatabaseSize>20</DatabaseSize>
  <CPU>1</CPU>
  <XML>yes</XML>
  <Logging>yes</Logging>
  <AdditionalTokenizedLanguages>chinese, japanese, korean</AdditionalTokenizedLanguages>
  <Readonly>no</Readonly>
```

```
 </Component>
 <Component Id="TaminoEnterprise">
  <ExternalBackup>no</ExternalBackup>
  <Replication>yes</Replication>
  <SNMP>no</SNMP>
 </Component>
 <Component Id="TaminoXNode">
  <DBMS>SQL Server, Microsoft SQL Server, ACCESS, Informix, DB2, Oracle, ADABAS, ADABAS D </DBMS>
 </Component>
</SoftwareAG_License>
```

Любая попытка модифицировать этот файл (например, продлить срок лицензии, изменить ОС и т.д.) заканчивается сообщением об "инвалидности файла лицензии" при запуске продукта.

Небольшое расследование показывает, что все функции, необходимые для работы с лицензионным файлом, собраны в одной библиотеке (очень удобно - и вашим и нашим!) с незатейливым именем saglic.dll, устанавливаемой в папку %CommonProgramFiles%\Software AG. Имена экспортируемых этой библиотекой функций весьма красноречивы. Пройдемся по ключевым функциям:

```
  LIC_read_Parameter
  LIC_checkExpiration
  LIC_checkOS
  LIC_checkSignature
  LIC_genSignature
  LIC_set_trace_level
```

LIC_read_Parameter

Прототип функции:

```
int LIC_read_Parameter (
 char *,  //  (in)   указатель на имя файла лицензии
 char *,  //  (in)   укзатель на название компонента
 char *,  //  (in)   указатель на название поля в компоненте
 char **,  //  (out)  адрес указателя на значение поля)
)
```

Назначение: Получить значение заданного поля.
Примечание: Необходимо иметь в виду, что LIC_read_Parameter() вызывает LIC_checkExpiration(), т.е. вначале проверяется валидность ключа.

LIC_checkExpiration

Прототип функции:

```
int LIC_ checkExpiration(
 char *,  //  (in)   указатель на имя файла лицензии
 char *   //  (in)   указатель на название компонента ("ProductInfo")
)
```

Назначение: Проверка срока действия лицензии.
Примечание: В этой функции "рабочей лошадкой" является поле файла лицензии. Функция считает поле "валидным", если его значение задано одним из трех способов:

1. <ExpirationDate>Unlimited</ExpirationDate>
2. <ExpirationDate>YYYY/MM/DD</ExpirationDate> (YYYY-год, MM - месяц, DD - день, т.е. дата завершения лицензии)
3. <ExpirationDate>Ndays</ExpirationDate> (N - количество дней действия лицензии)

При способе "Ndays" делается попытка найти значение (задается в формате YYYY/MM/DD). Если поле отсутствует - берется текущая дата. Подсчитывается дата завершения действия лицензии и заносится в в формате YYYY/MM/DD. Производится пересчет . Т.е. такой способ задания значения "живет" до первого вызова LIC_checkExpiration, после которого превращается в способ "YYYY/MM/DD"

LIC_checkOS

Прототип функции:

```
int LIC_checkOS (
 char *,  //  (in)   указатель на имя файла лицензии
 char *   //  (in)   указатель на название компонента ("ProductInfo")
)
```

Назначение: Проверка соответствия фактической операционной системы и заданной в лицензии.
Примечание: В этой функции "подопытным" является поле . Как можно увидеть из примера, в этом поле допустимо перечисление "лицензированных" операционных систем. Помимо явного задания допустимых ОС (например, w2000p - Windows 2000 Professional), возможно задать группу ОС (например, w2000).

Чуть ниже я покажу простой способ определения ОС в идентификации SAG, отмечу лишь, что возможно использование ключевых слов "any" или "win" в этом поле (т.е., становится возможным эксплуатировать продукт SAG под любой - в первом случае - ОС, либо под ОС семейства Windows - во втором случае), но злоупотреблять, конечно же, этим не стоит

LIC_checkSignature

Прототип функции:

```
int LIC_checkSignature (
 char *,  //  (in)   указатель на имя файла лицензии
)
```

Назначение: Проверка соответствия фактического хэш-значения файла лицензии и операционной системы и "эталонного" (хранящегося непосредственно в файле лицензии).
Примечание: В этой функции "козырным" является поле . Проверка осуществляется следующим образом. Лицензионный файл "хэшируется" (поле LicenseKey в этой процедуре не участвует) и происходит его сравнение (увы, операция сравнения производится в явном виде) со значением LicenseKey. По словам разработчиков и по отладочной информации можно предположить, что сей хэш рассчитывается по MD5-алгоритму. Может быть и так, не проверял.

LIC_genSignature

Прототип функции:

```
int LIC_genSignature (
 char *,  //  (in)   указатель на имя файла лицензии
)
```

Назначение: ???
Примечание: Исходя из названия, можно предположить, что это - функция генерации . Но нам остается только предполагать, т.к. в Public - версии библиотеки saglic.dll реализации функции отсутствует (вывод отладочной информации - не в счет! Об отладочной информации я расскажу чуть ниже)

LIC_set_trace_level

Прототипы функции:

```
int LIC_set_trace_level (
 int ,  // (in)  уровень трассировки
 char *  // (in)  указатель на имя трассируемой функции
//  (не используется)
 int   // (in)  номер строки
//  (не используется)
)

int LIC_set_trace_level (
 int   // (in)  уровень трассировки
)
```

Назначение: Установка уровня трассировки
Примечание: Этой функцией устанавливается "глобальный уровень трассировки" (название условно). Вся отладочная информация выводится функциями библиотеки не напрямую, а через специальную подпрограмму, одним из параметров которой является "локальный уровень трассировки". Эта подпрограмма сравнивает "глобальный уровень" и "локальный уровень", если "локальный уровень" не меньше "глобального", то информация выводится, иначе - нет.

Коды возврата

• 0 LICENSE OK
• 1 ERROR - LICENSE KEY NOT FOUND
• 2 ERROR - INVALID (TAMPERED) LICENSE KEY
• 3 ERROR - INVALID (TAMPERED) LICENSE KEY (действительно повтор!)
• 4 ERROR - PARAMETER NOT FOUND IN LICENSE KEY
• 5 ERROR - COMPONENT NOT FOUND IN LICENSE KEY
• 6 ERROR - MISSING PARAMETER(S) - SEE FUNCTION USAGE
• 7 ERROR - MISSING WINSOCK DLL ON WIN32
• 8 ERROR - WRONG WINSOCK DLL (BELOW 1.0] ON WIN32
• 9 ERROR - SOCKET CANNOT BE OPENED
• 10 ERROR - HOST CANNOT BE FOUND IN HTTP ADDRESS
• 11 ERROR - HOST CANNOT BE OPENED IN HTTP ADDRESS
• 12 ERROR - HOST CANNOT BE OPENED IN HTTP ADDRESS
• 13 ERROR - CANNOT CREATE LICENSE USING EXTERNAL LIC VERSION
• 14 ERROR - CANNOT ALLOCATE MEMORY
• 15 ERROR - LICENSE EXPIRED
• 16 ERROR - INVALID LICENSE OS
• 17 ERROR - CANNOT CREATE FIRST WINDOWS REGISTRY KEY
• 18 ERROR - CANNOT ACCESS WINDOWS REGISTRY
• 19 ERROR - READING LICENSE KEY FILE
• 20 ERROR - TRIAL LICENSE KEY FILES must NOT be WRITE-PROTECTED and MUST be passed as a PATHNAME (not CONTEN TS).
• 21 ERROR - CONVERTING ASCII-TO-EBCDIC (OR VICE-VERSA)
• 22 ERROR - SYSTEM COMMAND (I.E., UNAME)
• 23 ERROR - CANNOT READ LOCAL SAG SECURITY PROFILE
• 24 ERROR - WRITING to LICENSE KEY FILE

Думаю, разбор потрохов пора завершать.

Если по-крупному, действительно защита отсутствует напрочь ("крутой" алгоритм хэширования не в счет!). Логика системы лицензирования понятна, и очень сильно облегчают понимание этой логики красноречивые имена экспортируемых функций не менее красноречивой saglic.dll, а также обилие отладочной информации.

"Тоже мне, диковина!" - воскликнет проницательный читатель. - "Да такие чудеса на каждом углу по 10 копеек кулек!"

Ну что ж, настало время десерта…

Особенности системы лицензионных ключей Пикантность ситуации заключается в том, что вместе с установкой основного продукта, на ваш компьютер устанавливается и своеобразный бонус от Software AG в виде двух утилит, уютно разместившимся в папке %Common Files%Software AG (Вообще-то подарков больше, но нужно же знать меру!). Речь пойдет о sagver.exe и saglicExeWrapper.exe. **sagver.exe**

Попробуем выполнить ее для начала без параметров (лучше из консольного файл-менеджера, и лучше перенаправить вывод потока в файл).

Что это? А это, дорогие читатели, ни что иное, как описание свойств файлов, содержащихся в текущей (%Common Files%Software AG) папке. Конечно, эту же информацию можно получить и из Проводника (правая кнопка мыши -> Свойства ->Вкладка "Версия"), но это долго и не очень удобно, если нужно просмотреть описание многих файлов.

Судя по всему, описания файлов довольно полные, а не попробовать ли нам поискать что-нибудь на лицензионную тему, по подстроке поиска, скажем, "lic"? Сказано - сделано! А вот и старый знакомый

Описание свойств saglic.dll:

```
Software AG Version Information for .\saglic.dll
    Company            - Software AG
    Description        - Software AG Licensing Services API
    Product Version    - 1.4.0
    Internal Name      - SAGLIC DLL
    Copyright          - (C) Copyright Software AG 1995-2003. All rights reserved.
    Product Name       - saglic
    Patch Level        - 10
    Build Number       - 1
    Release            - Release
    File OS            - Windows 32-Bit
    File Type          - Dynamic Link Library
Windows Numeric Version Info for .\saglic.dll
    File Version       - 1.4.0.10
    Product Version    - 1.4.0.10
Windows String Version Info for .\saglic.dll
    Comments           - Public-use DLL
    Company Name       - Software AG
    File Description   - Software AG Licensing Services API
    File Version       - 1.4.0.10
    Internal Name      - SAGLIC DLL
    Legal Copyright    - (C) Copyright Software AG 1995-2003. All rights reserved.
    Original Filename  - saglic.dll
    Product Name       - License System
    Product Version    - 1.4.0
    Special Build      - PUBLIC
```

(То же самое можно получить, набрав в командной строке sagver saglic.dll)

Итак, с sagver.exe более-менее понятно - эта утилита вытаскивает описание свойств заданного файла, а если файл не задан - сканирует библиотеку и выдает описание всех содержащихся в ней файлов. Правда, нужно отметить, что "подсунув" этой утилите "чужой" файл, мы разочаруемся - утилита не выдаст никакой информации. Фокус в том, что она (утилита) проверяет содержимое "подопытного" файла на присутствие определенной сигнатуры (или по образному выражению разработчиков SAG "EyeCatch"), которая является началом структуры, содержащей служебную информацию о файле (т.е. описание его свойств). Маленькая подсказка для любознательных - утилита sagver.exe тоже может вывести отладочную информацию, если ее хорошо попросить - например, создать переменную окружения SVRLOG со значением 0xFFFF.

Однако, помимо уже знакомой нам saglic.dll, мы наткнемся и на saglicExeWrapper.exe, тоже относящуюся к License System. А это что за чудо-утилита?

saglicExeWrapper.exe

Еще раз отдадим должное разработчикам SAG - название говорит само за себя! Эта утилита - обертка вокруг saglic.dll, в которой собраны все функции, связанные с системой лицензирования.

Попробуем ее выполнить без параметров.

Синтаксис вызова saglicExeWrapper.exe:

```
PRODUCT NAME: saglicExeWrapper (SAGLIC EXE)
              (C) Copyright Software AG 1995-2003. All rights reserved.
DESCRIPTION:  Software AG Licensing Services API
VERSION:      1.4.0
PATCH LEVEL:  10
BUILD TYPE:   PUBLIC

USAGE:

    saglicExeWrapper.exe 0
           List LICLOG <trace_level> options

    saglicExeWrapper.exe 1 <xmlfile_in> <component> <parameter> [<trace_level>]
           LIC_read_parameter() called with the middle 3 parameters

    saglicExeWrapper.exe 2 <xmlfile_in> <xmlfile_out> [<trace_level>]
           LIC_genSignature() called with the middle 2 parameters

    saglicExeWrapper.exe 3 <xmlfile_in> [<trace_level>]
           LIC_checkSignature() called with the middle parameter

    saglicExeWrapper.exe 4 <xmlfile_in> <component> [<trace_level>]
           LIC_checkExpiration() called with the middle 2 parameters

    saglicExeWrapper.exe 5 <xmlfile_in> <component> [<trace_level>]
           LIC_checkOS() called with the middle 2 parameters

    saglicExeWrapper.exe 6 [<trace_level>]
           LIC_checkExpiration() called with DIRECT INPUT of an internal
           Trial ('90 Days') License Certificate.  (THIS SHOULD FAIL!)

    saglicExeWrapper.exe 7
           LIC_validOS() called with no parameters

For example:

    saglicExeWrapper.exe 5 ino223.xml TaminoProduct CHECK_OS
```

Взгляните, перед нами действительно практически все экспортируемые saglic.dll функции!

Думаю, не стоит повторяться с описанием параметров вызова saglicExeWrapper.exe - посмотрите описание экспортируемых функций saglic.dll, все то же самое. Единственное, на что хотелось бы обратить ваше внимание, на необязательный параметр trace_level.

Понятно, что он как-то связан с уровнем трассировки, используемым функциями saglic.dll, но как? Может быть, утилита нам сама подскажет?

Выполняем **saglicExeWrapper.exe 0 /?**.

```
LICLOG  <trace_level> DESCRIPTION
-------------------------------------------------------
0x0001  ERR           Trace error conditions
0x0002  EXT           Trace external LIC API functions
0x0004  INT           Trace internal LIC functions
0x0008  DBG           Trace debugging info
0x0010  CHECK_OS      LIC_checkOS function only
0x0020  CHECK_EXP     LIC_checkExpiration function only
0x0040  READ_PARM     LIC_read_parameter function only
0x0080  CHECK_SIG     LIC_checkSignature function only
0x0100  GEN_SIG       LIC_genSignature function only
0x0200  FREE_PARM     LIC_freeParameter function only
0x0400  SOCKETS       Socket-related operations only
0x0800  UTILS         Utility functions only
0x1000  MD5           MD5 (Message Digest) functions only
0x2000  UNUSED        UNUSED
0x4000  LOG           Trace LOG functions only
0x8000  LOG_FILE      LOG to a local file in TMP directory
0x03f0  HIGH_LEVEL    High Level (LIC) functions only
0x1c00  LOW_LEVEL     Low Level operations only
0xffff  LOG_ALL       Log ALL Trace Levels
```

```
Trace Level(s) may be set using TWO different methods:

1. Via the LICLOG environment variable when set to the SUM of
   the appropriate HEX value(s) shown above.  For example, to
   trace the LIC_checkOS() and LIC_read_parameter() functions,
   set LICLOG = (0x0008 + 0x0010 + 0x0040) = 0x0058
   (Set LICLOG directly to trace LIC functions called directly
   from applications such as Tamino.)

2. Via the optional
 argument in this executable

   Example 1:  saglicExeWrapper 5 ino223.xml TaminoProduct
                                CHECK_OS READ_PARM

   traces the LIC_checkOS() and LIC_read_parameter() functions.

   Example 2:  saglicExeWrapper 5 ino223.xml TaminoProduct
                                CHECK_OS LOG_FILE

   will send LIC_checkOS() Trace output to a temporary file.
```

Хорошо, и что со всем этим "добром" делать?

Первое, что идет на ум, все же попробовать saglicExeWrapper.exe с параметром 2 (LIC_genSignature()). Понятно, что в saglic.dll эта функция "недореализована", но вдруг эта реализация есть в saglicExeWrapper.exe?

Итак: **saglicExeWrapper.exe 2 ino414.xml test.xml**

Похоже, чудо не случилось.

```
LIC_genSignature()...

License File (IN)  = ino414.xml
License File (OUT) = test.xml

13:  ERROR - CANNOT CREATE LICENSE USING EXTERNAL LIC VERSION
```

Так, а что если попробовать saglicExeWrapper.exe с параметром 3 (LIC_checkSignature())? Да не просто так, а с полной трассировкой, авось что-нибудь выудим!

saglicExeWrapper.exe 3 ino414.xml LOG_ALL
(LOG_ALL - именно в верхнем регистре)

После долгого мелькания отладочной информации должны получить нечто вроде такого

```
MD5_memcpy():
{
Encode():
{
MD5_memset():
{
SAG_certification():
   aSig[0] = A7
SAG_certification():
   aSig[1] = B7
SAG_certification():
   aSig[2] = 7D
SAG_certification():
   aSig[3] = 8C
SAG_certification():
   aSig[4] = C2
SAG_certification():
   aSig[5] = B4
SAG_certification():
   aSig[6] = 6E
SAG_certification():
   aSig[7] = 9C
SAG_certification():
```

```
   aSig[8] = 58
SAG_certification():
   aSig[9] = 21
SAG_certification():
   aSig[10] = DE
SAG_certification():
   aSig[11] = C4
SAG_certification():
   aSig[12] = 1B
SAG_certification():
   aSig[13] = 9B
SAG_certification():
   aSig[14] = 9C
SAG_certification():
   aSig[15] = E0
SAG_certification():
returned 0
SAG_certification():
}
LIC_checkSignature():
returned 0
LIC_checkSignature():
}

   0:  LICENSE OK
```

Смотрите, а ведь последовательность A7B77D8CC2B46E9C5821DEC41B9B9CE0 не что иное, как нашего файла лицензии! Как минимум, любопытно…

Но ведь не для того утилита долго что-то рассчитывала, чтобы в итоге выдать нам лицензионный ключ, прочитанный из файла лицензии!

Почему бы не попробовать модифицировать файл лицензии и посмотреть, изменится ли эта последовательность! Что ж, попробуем… Подкорректируем дату регистрации (2006/01/01), сохраним его с именем _ino414.xml и "натравим" на него saglicExeWrapper.exe

saglicExeWrapper.exe 3 _ino414.xml LOG_ALL

```
MD5_memcpy():
{
Encode():
{
MD5_memset():
{
SAG_certification():
   aSig[0] = 9E
SAG_certification():
   aSig[1] = 1F
SAG_certification():
   aSig[2] = 58
SAG_certification():
   aSig[3] = B4
SAG_certification():
   aSig[4] = FB
SAG_certification():
   aSig[5] = 3B
SAG_certification():
   aSig[6] = D7
SAG_certification():
   aSig[7] = 1D
SAG_certification():
   aSig[8] = D6
SAG_certification():
   aSig[9] = 73
SAG_certification():
   aSig[10] = 14
SAG_certification():
   aSig[11] = 3B
SAG_certification():
   aSig[12] = 91
```

```
SAG_certification():
   aSig[13] = D
SAG_certification():
   aSig[14] = 99
SAG_certification():
   aSig[15] = 18
SAG_certification():
returned 0
SAG_certification():
}
LIC_checkSignature():
returned 3
LIC_checkSignature():
}

   3:  ERROR - INVALID (TAMPERED) LICENSE KEY
```

Однако, последовательность совсем другая! Может быть, это как раз то хэш-значение, с которым и сравнивается LicenseKey из файла лицензии? Если наши рассуждения верны, и это и есть "правильный" LicenseKey , то поставив заветную последовательность (не забывая про ведущий нолик для значений aSig[], меньших 0x10) в поле LicenseKey файла _ino414.xml, мы должны бы получить "0: LICENSE OK"

Пробуем!

Результат выполнения saglicExeWrapper.exe с параметром 3 (LIC_checkSignature):

```
MD5_memcpy():
{
Encode():
{
MD5_memset():
{
SAG_certification():
   aSig[0] = 9E
SAG_certification():
   aSig[1] = 1F
SAG_certification():
   aSig[2] = 58
SAG_certification():
   aSig[3] = B4
SAG_certification():
   aSig[4] = FB
SAG_certification():
   aSig[5] = 3B
SAG_certification():
   aSig[6] = D7
SAG_certification():
   aSig[7] = 1D
SAG_certification():
   aSig[8] = D6
SAG_certification():
   aSig[9] = 73
SAG_certification():
   aSig[10] = 14
SAG_certification():
   aSig[11] = 3B
SAG_certification():
   aSig[12] = 91
SAG_certification():
   aSig[13] = D
SAG_certification():
   aSig[14] = 99
SAG_certification():
   aSig[15] = 18
SAG_certification():
returned 0
SAG_certification():
}
LIC_checkSignature():
```

```
returned 0
LIC_checkSignature():
}

   0:  LICENSE OK
```

Ого! Работает!!!

Что ж, осталось собрать полученную информацию об "особенностях системы лицензирования" воедино.

Чтобы получить корректный файл лицензии после его модификации необходимо:

1. Убедиться, что наша ОС соответствует тому, что задано в файле лицензии. Для этого используем утилиту saglicExeWrapper.exe

saglicExeWrapper.exe 7

Если в файле лицензии нет ни одного из предложенных значений ОС - правим значение поля "OS" файла лицензии в соответствии с "рекомендациями" saglicExeWrapper.exe . Напомню, что самым общим является значение "any", менее общим для семейства Windows - "win". Это менее корректный вариант, но в этом случае необходимости использовать утилиту saglicExeWrapper.exe нет.

2. Убедиться, что срок действия лицензии не истек. Если есть необходимость, правим значение поля "ExpirationDate". Напомню, здесь допустимо значение "Unlimited"
3. Рассчитать новое значение LicenseKey, используя saglicExeWrapper.exe в режиме трассировки (достаточно уровня GEN_SIG)

saglicExeWrapper.exe 3 lic_file.xml GEN_SIG 4. Полученную последовательность занести в поле "LicenseKey" файла лицензии (не забывая про ведущий ноль!) и проверить на отсутствие ошибок. saglicExeWrapper.exe 3 lic_file.xml

В принципе, должно работать. Если получаете ошибку "3: ERROR - INVALID (TAMPERED) LICENSE KEY", проверьте еще разок, правильно ли переписали значение LicenseKey.

На эту процедуру у вас едва ли уйдет более 2 минут. Причем, заметьте, результат получается исключительно благодаря своей голове и замечательному подарку от компании Software AG в виде утилиты saglicExeWrapper.exe

Хорошо, а что делать, если вы устанавливаете самый первый продукт SAG (т.е. у вас нет ни папки %CommonProgramFiles%\Software AG, ни saglicExeWrapper.exe), а вам очень нужно "поправить" файл лицензии до инсталляции (например, вам нужно установить компонент, не входящий в демо-лицензию)? Проще всего найти на установочном диске папку Windows\BTY, а в ней - файл data1.cab. Нужно извлечь из этого архива 2 файла saglic.dll и saglicExeWrapper.exe (в архиве они находятся в весьма неприглядном виде, например, saglic.dll.44FFEB35_100E_11D6_A6C8_00300501EF6A, поэтому после извлечения переименуйте их так, как положено). Извлечь можно WinRar'ом, либо любым другим упаковщиком, понимающим формат .cab. В эту же папку скопируйте свой лицензионный файл и дальше действуйте по общей схеме.

Заключение

Вот некоторые замечания, которые пришли мне на ум, когда я разбирался с пресловутой "системой лицензирования":

1. Как отмечалось выше, отсутствие защитных механизмов в системе лицензирования продуктов компании Software AG не делает им чести. И использование MD5- алгоритма сильно ситуацию не исправляет - как вы видите, вполне можно обойтись и без знания тонкостей хэширования, чтобы делать с лицензионным файлом все, что душа пожелает. Но если разработчики считают, что закон

их оберегает надежнее, чем они сами себя - что ж, это их право.
2. Возможность трассировки - штука несомненно полезная, но только не в системе защиты/лицензирования! По крайней мере, это просто не очень разумно.
3. Наличие утилит для внутреннего служебного пользования (к которым без всякого сомнения относятся sagver.exe и saglicExeWrapper.exe) в дистрибутиве вообще не поддается разумному объяснению! Причем эти утилиты помещаются в дистрибутивы большинства продуктов SAGa из версии к версии…

Выявленные изъяны не стали неожиданностью для разработчиков Software AG. Остается только понадеется, что следующие версии продуктов Software AG станут более защищенными.

1 В настоящее время готовится к публикации описание p-кода, используемого в NATURAL (будем надеяться, что автор выполнит это обещание - *прим.ред.*).

2 Полный адрес http://tamino.forums.softwareag.com/download.php?id=774&sid=96bcb784da946a6dec504a291d84801d (для зарегистрированных пользователей).

Исследование «промежуточного» кода на примере GP-кода языка NATURAL

Автор: Konstantin

Дата: 2006-05-20

После такого заголовка я, как "порядочный мужчина", просто обязан объяснить, что имел в виду, называя так статью, о чем, зачем и для кого она написана.
Начнем с главного.

Зачем и для кого написана эта статья?
В этой статье я бы хотел поделиться своими результатами в исследовании "промежуточного" кода. Я вполне отдаю себе отчет в том, что исследование скомпилированного кода малоизвестным компилятором, да еще и требующего интерпретатора для своего исполнения, едва ли вызовет широкий интерес. Но все же я решился на публикацию, и вот по каким причинам:

Эта статья может быть интересной малочисленным программистам, специализирующимся на Natural, и желающим поднять свою квалификацию. Думаю, понимание того, во что превращается исходный код при компиляции, будет способствовать этому росту.
Эта статья может помочь также в случае необходимости восстановить исходный код из "промежуточного" (опять-таки для Natural-программистов).
Эта статья может быть полезной для исследователей, изучающих какой-то свой специфический псевдокод, по которому нет информации ни из официальной документации, ни от коллег-исследователей.
Надеюсь, статья может стать интересной и для программистов, разрабатывающих свой язык/интерпретатор (если такие еще найдутся).

Промежуточный код
Существует множество терминов, означающих приблизительно одно и то же. Псевдокод, псевдокомпилированный код, промежуточный код - все это одного поля ягоды. Мне больше нравится термин "промежуточный код" - в виду отсутствия четких общепринятых определений могу себе позволить выбирать по своему усмотрению.
Как правило, промежуточный код более компактен, чем исходный код, за счет удаления в процессе компиляции (или псевдокомпиляции, если хотите) всяких излишеств - комментариев, пустых строк и т.д., а так же за счет преобразования имен переменных в адреса, а "человеческих" операторов в инструкции, понятные интерпретатору. Очевидно, промежуточный код должен быть структурирован так, чтобы его исполнение интерпретатором было максимально эффективным.
Существует два крайних подхода к исследованию промежуточного кода:

исследование работы компилятора и интерпретатора (инструменты - дизассемблер, отладчик);
метод "черного ящика" (инструменты - голова и hex-редактор). "Черным ящиком" опять же может быть как компилятор (вход - исходный код программы, выход - промежуточный код), так и интерпретатор (вход - промежуточный код, выход - результат выполнения программы).

Наиболее эффективным, конечно же, является их комбинация, поэтому мы попробуем и то, и другое… и третье - в предыдущей статье ("Тайна золотого ключика…") я уже напоминал о "бонусах" от SAG'a, не обошлось без них и на этот раз. Но об этом ниже.

NATURAL и GP-код
Сейчас под именем Natural (начиная с 4 версии) скрывается и интегрированная среда разработки (Natural Studio), и язык программирования. Разработчик - немецкая компания Software AG (коротко SAG).

Языку уже более четверти века - его начали разрабатывать в 1976 и в 1979 он был выпущен на рынок. В те времена его называли "языком манипулирования данными". Он "крутился" на мэйнфрейме и был логическим продолжением БД ADABAS (разработчик - та же компания, SAG). Сейчас же это не иначе как "мощный сервер приложений компании Software AG, позволяющий разработчикам быстро создавать критически важные приложения". Эта штуковина работает на мэйнфреймах, на *nix и на Windows - платформах. Гарантируется переносимость на уровне исходных кодов с одной платформы на другую (если хорошенько подработать напильником). С 5-ой версии (на момент написания статьи последняя версия Natural - 6-ая) была заявлена переносимость т.н. "сгенерированных программ" (generated programs, GP, проще - скомпилированных программ) между *nix и Windows - платформами. В ближайшем будущем анонсируется переносимость GP-кода и на мэйнфрейм - платформы.
Как и всякий промежуточный код, GP-код не самодостаточен - он может быть исполнен лишь при наличии интерпретатора.
Переносимость того самого "промежуточного" кода, о котором мы говорили выше, да тем более между "открытыми системами" и мэйнфреймами - это уже интересно!

Часть 1. Определение общей структуры GP-кода
Первое, что приходит на ум (и не самое лучшее1) - создать минимально возможную программу, скомпилировать ее и посмотреть получившийся код. На языке NATURAL исходный код такой программы будет содержать единственный оператор - END (синтаксис языка позволяет также использовать точку - "."), символизирующий завершение текста программы. Сохраняем чудо-программу под именем test01, тип объекта2 - Program, компилируем (в терминах NATURAL "каталогизируем"), в результате чего получаем замечательный файлик test01.ngp3. Этот файл как раз и содержит "промежуточный", или GP-код.

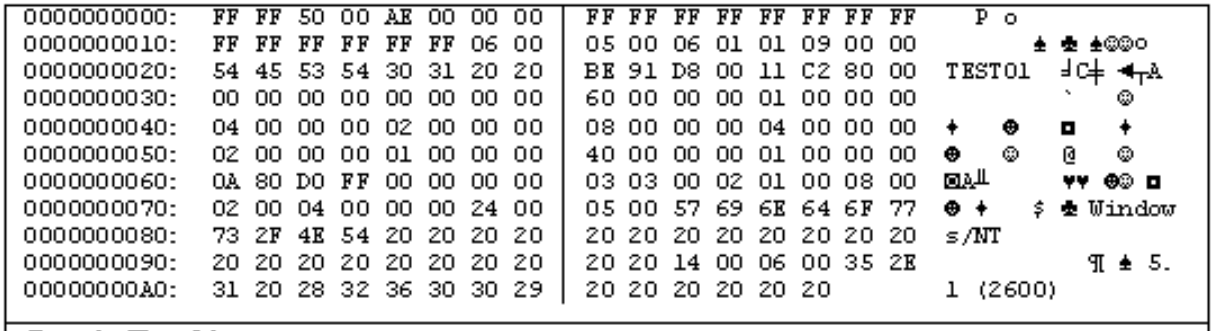


Рис 1. Test01.ngp

Обратим внимание, исходный текст нашей "минимальной" программы (трехбайтный, если использовался оператор"END", и однобайтный, если использовался оператор ".") преобразовался 174-байтный. Очевидно, что структура GP-кода сложна и разнородна. Раз так, есть резон предположить, что GP-код должен иметь заголовочную область. Вот ее-то мы и попробуем определить.

1 Правильный ответ - читать документацию! Этот набивший оскомину постулат был мной проигнорирован, за что я поплатился 2 месяцами времени. Оказалось, Natural имеет полезную настройку - формирование отчета о процессе компиляции. Этот отчет - просто кладезь полезной информации! Однако в моей глупости есть и свои плюсы - был испробован в деле метод "черного ящика", что для меня оказалось весьма полезным, и надеюсь, окажется полезным и еще кому-нибудь.

2 Object type - имеется в виду тип программного модуля. Перечень возможных типов объектов и дополнительная информация по ним даны в Приложении 1, п 1.1

3 Условия кодогенерации: ОС Microsoft Windows XP [Версия 5.1.2600], Natural 6.1.1 pl9

Заголовок

Логично начать поиски заголовка в начале файла. А вот где он заканчивается? Явно раньше последнего байта (где-то же должны храниться программные инструкции и едва ли внутри заголовка) и явно после последовательности "TEST01" (имя программы).
Поехали!
Пропустим пока значения 0x0 и 0xFF и сведем ценную информацию в табличку

Таб.1

Смещение	Размер	Значение	Комментарий
0x00	2	0xFFFF	Пропускаем
0x02	1	0x50 (ASCII - "P")	Возможно, тип объекта (Program)
0x03	1	0x00	Пропускаем
0x04	2 (?)	0xAE	Сильно смахивает на размер GP-кода!
0x06	2	0x00	Пропускаем
0x08	14	0xFF (14 повторений)	Пропускаем
0x16	2(?)	0x06	?
0x18	2(?)	0x05	?
0x1A	1	0x06	Version
0x1A	1	0x01	Release
0x1A	1	0x01	System maintenance level
0x1A	1	0x09	Patch level
0x1E	2	0x0	Пропускаем
0x20	8	TEST01	Имя программы (размер = 8 – это документированное ограничение Natural на имя программы)
0x28	8(?)	BE91D60011C28000	?

А вот что дальше - совершенно непонятно! Мало того, что последовательность не ассоциируется ни с какой "человеческой" информацией, так она еще носит явно периодический характер! Может быть, это уже не заголовок? А что тогда?
Давайте попробуем ответить на этот вопрос, исходя из периодичности (т.е. повторяемости). Единственная оговорка, последовательность нулей (начиная с 0x30 по 0x37) рассматривать не будем из-за ее "неинформативности", а рассмотрим последовательность с 0x38 по 0x5F.
Процесс предстоит творческий - необходимо преобразовать эту последовательность в нечто осмысленное и удобоваримое. Возможно множество вариантов (не забываем отброшенную нами ранее последовательность нулей - они тоже могут быть задействованы), но я бы хотел предложить вашему вниманию лишь два

Таб.2

Вариант 1	Вариант 2
60 00 00 00	60 00 00 00 01 00 00 00
01 00 00 00	04 00 00 00 02 00 00 00
04 00 00 00	08 00 00 00 04 00 00 00
02 00 00 00	02 00 00 00 01 00 00 00
08 00 00 00	40 00 00 00 01 00 00 00
04 00 00 00	
02 00 00 00	
01 00 00 00	
40 00 00 00	
01 00 00 00	

Если в первом варианте сложно найти какую-то "осмысленность", то второй вариант выглядит более любопытным: 0x60 + 0x04 + 0x08 + 0x02 + 0x40 = 0xAE, а это ни что иное, как размер GP-кода!
Вопрос - что при сложении дает общий размер? Наверное, размер элементов … А что может являться элементом скомпилированной программы? Уж не секция ли!? (можно, конечно, подобрать и другие названия этим элементам - директория, сегмент, раздел и т.д, но давайте остановимся на секции)
Выглядит вполне заманчиво - в начале GP-кода заголовок, за ним таблица с размерами секций. Проверим?

Таб.3.

№ секции	Размер	Начало секции	Конец секции
1	0x80	0x00	0x5F
2	0x04	0x80	0x83
3	0x08	0x84	0x8B
4	0x02	0x8C	0x8D
5	0x40	0x8E	0xAD

(0xAD таб.3 разъехалось с 0xAE таб.2 в один байт из-за того, что в таб.3 даны смещения, а в таб.2 - размеры, т.е в таб.3 мы отсчитываем от нуля, а не от единицы)

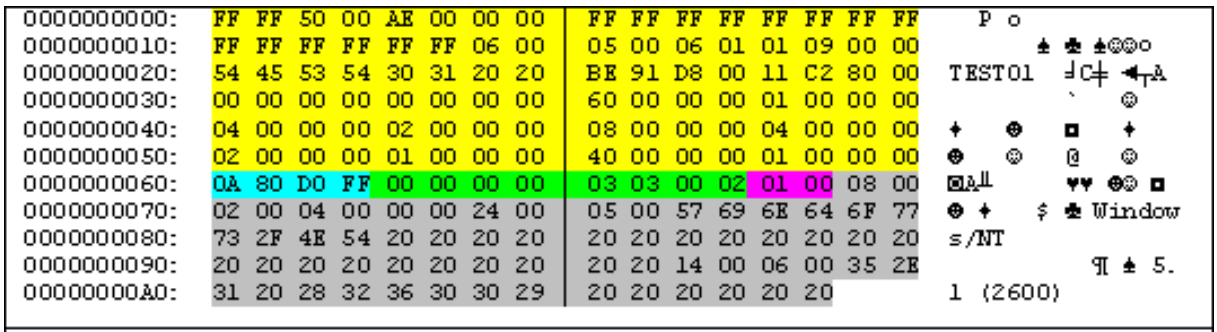


Рис 2. Test01.ngp с разукрашенными секциями

Возьмем на заметку - количество секций 5, и это количество должно бы фигурировать в заголовке (интерпретатор, переваривающий этот код, должен бы знать размер таблицы секций еще до того, как ее прочитает, чтобы корректно выделить для нее память). В заголовке (Таб.1) это число встречается всего один раз, и оно еще не опознано, а потому вполне пригодно в качестве кандидата на вакантную должность "хранителя количества секций".

Если все наши предположения верны, то в первую секцию (условно - "заголовок") вошел сам фиксированный заголовок и переменная часть - таблица секций. Вторая, третья и четвертая секции слишком малы, чтобы что-то предполагать, а структура пятой не совсем очевидна (хотя там есть любопытные закономерности, и я предлагаю вам их отыскать).

Будем экспериментировать!
Но хороший эксперимент нужно подготовить, т.е. решить, что проверяем и каким образом. Сначала убедимся, что наши предположения по структуре фиксированного заголовка верны, для чего проведем ряд экспериментов.

Таб.4 Журнал экспериментов

№	Имя программы	Тип	Физ. файл	Что проверяем	Каким образом
1	Test01	Program	Test01.ngp	Общая структура кода	Минимальная программа
2	Test02	Subprogram	Test02.ngn	Тип объекта	Минимальная программа + тип объекта при сохранении выбрать Subprogram (перечень типов объектов приведен в приложении 1. rtl.1)
3	Test002	Subprogram	Test002.ngn	Длина имени программы	Сохранить Test02 как Test002

Хочу сразу оговориться - в жизни экспериментов было гораздо больше, и они по разным причинам были совершенно другими! Не надейтесь с одного - двух раз разобраться со сложной структурой данных. Готовьтесь к тому, что нужно будет проделать десятки, сотни экспериментов, чтобы выдвинуть только гипотезу, и десятки - чтобы ее подтвердить или опровергнуть. И обязательно все скрупулезно журнализируйте.
Сводим в табличку полученные после компиляции результаты:

Таб.5 Свод (Заголовки, 1-2-3)

Смещение	Размер	Test01	Test02	Test002	Комментарий
0x00	2	0xFFFF	0xFFFF	0xFFFF	Пропускаем
0x02	1	0x50 (ASCII "P")	*0x4E (ASCII "N")*	0x4E (ASCII "N")	Тип объекта
0x03	1	0x00	0x00	0x00	Пропускаем
0x04	2 (?)	0xAE	0xAE	0xAE	Сильно смахивает на размер GP-кода!
0x06	2	0x00	0x00	0x00	Пропускаем
0x08	14	0xFF (14)	0xFF (14)	0xFF (14)	Пропускаем
0x16	2(?)	0x06	0x06	*0x07*	Длина имени программы
0x18	2(?)	0x05	0x05	0x05	Количество секций?
0x1A	1	0x06	0x06	0x06	Version
0x1A	1	0x01	0x01	0x01	Release
0x1A	1	0x01	0x01	0x01	System maintenance level
0x1A	1	0x09	0x09	0x09	Patch level
0x1E	2	0x0	0x0	0x0	Пропускаем
0x20	8	TEST01	*TEST02*	*TEST002*	Имя программы
0x28	8(?)	BE91D80011C28000	*BE92069C20F00000*	*BE9206C248340000*	?

(Курсивом с с подчеркиванием отмечены отличия от предыдущего состояния)
С расположением типа объекта и длины имени программы в структуре заголовка вроде понятно.

Отметим, что последнее поле (смещение 0x28) меняется от эксперимента к эксперименту. Мало того, если мы вообще ничего не изменим, а просто перекомпилируем исходный код, значение этого поля все равно поменяется. Как вы уже, наверное, догадались - это не что иное, как Timestamp. Т.к эта штуковина будет меняться от эксперимента к эксперименту, то давайте договоримся не обращать на нее внимания.

Таблица секций

На очереди вторая серия экспериментов - необходимо разобраться с секциями и либо получить подтверждение наших предположений, либо пересмотреть модель структуры GP-кода.

А предположили мы следующее

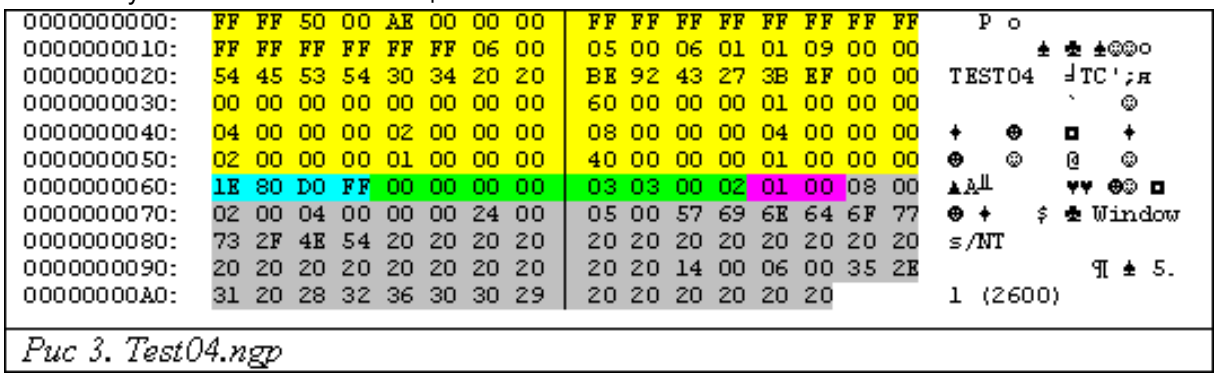
По смещению 0x18 находится число, равное количеству секций
Начиная с фиксированной позиции 0x30 и заканчивая 0x38 включительно, находится начало таблицы секций (последовательность нулей так и осталась бесхозной). Размер строки в таблице секций равен 8. Соответственно, размер всей таблицы равен [кол-во секций из заголовка] x 8
Первый элемент строки таблицы секции - размер секции

Попутно хотелось бы получить ответ на вопрос, а где же хранятся номера строк в GP-коде? Ведь при возникновении ошибки на этапе выполнения программы помимо самого сообщения об ошибке нам сообщается и номер строки инструкции, вызвавшей ошибку! Причем отсутствие исходного кода не влияет на способность интерпретатора определять номер строки. Значит, номера строк исходного кода каким-то образом хранятся и в GP-коде!

Таб.6 Журнал экспериментов

№	Имя программы	Тип	Физ. файл	Что проверяем	Каким образом
4	Test04	Program	Test04.ngp	Влияние комментариев и пустых строк на размер GP-кода	Комментарий в первой строке + пустая вторая строка + оператор END в третьей строке 0010 /* комментарий 0020 0030 end
5	Test05	Program	Test05.ngp	Размер GP-кода и влияние количества секций на фиксированный заголовок и таблицу секций	В пустую вторую строку программы Test04 добавляем объявление переменной и ее инициализацию. Такая манипуляция должна гарантированно увеличить размер кода по сравнению с Test04, и интересно, что произойдет с количеством секций 0010 /* комментарий 0020 str1 (a10) = 'wasm.ru' 0030 end (выражение str1 (a10) = 'wasm.ru' означает, что мы объявляем переменную str1, символьную, 10-байтную и присваиваем ей значение wasm.ru. 0010, 0020, 0030 – номера строк, NATURAL по умолчанию нумерует через 10)

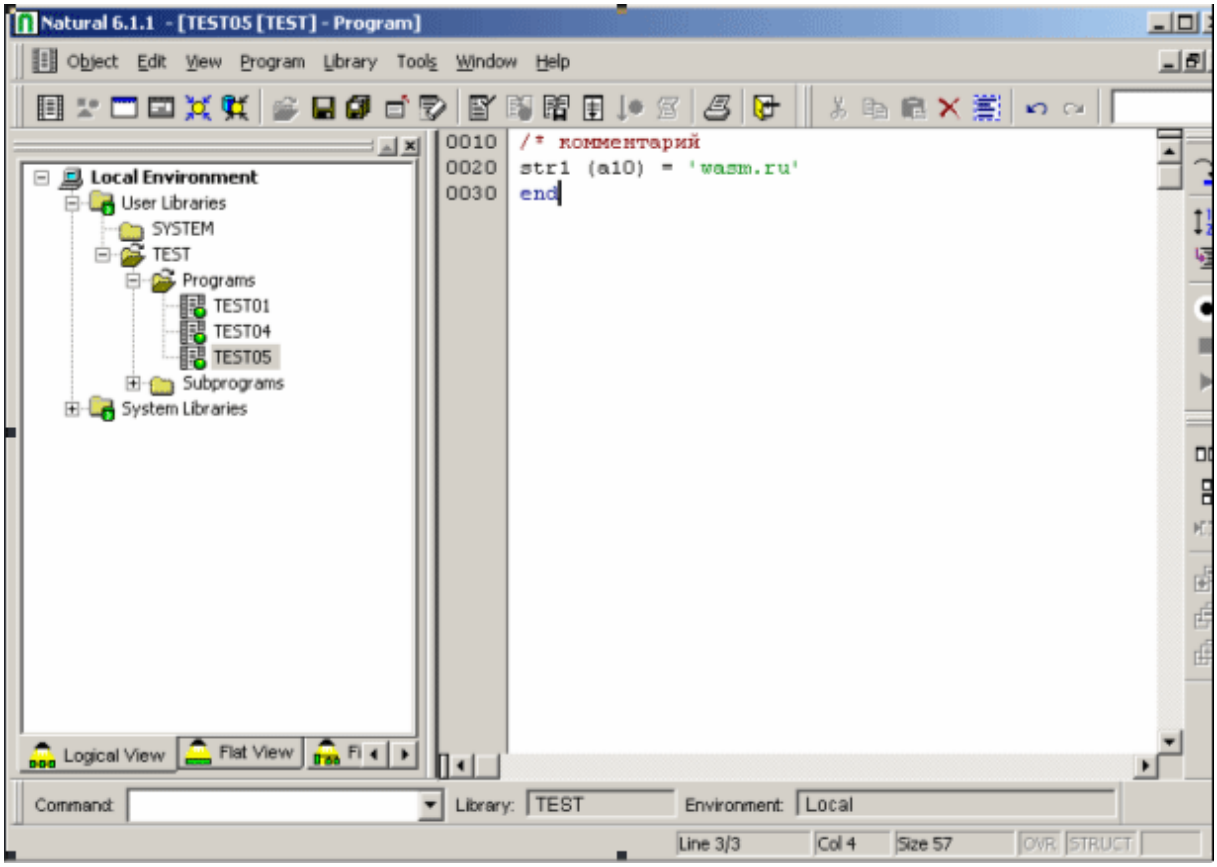
Вот что получилось после компиляции Test04:



```
0000000000:  FF FF 50 00 AE 00 00 00   FF FF FF FF FF FF FF FF    P ©
0000000010:  FF FF FF FF FF FF 06 00   05 00 06 01 01 09 00 00          ♠ ♣ ♠◘◘○
0000000020:  54 45 53 54 30 34 20 20   BE 92 43 27 3B EF 00 00  TEST04  ╛TC';я
0000000030:  00 00 00 00 00 00 00 00   60 00 00 00 01 00 00 00            `   ☺
0000000040:  04 00 00 00 02 00 00 00   08 00 00 00 04 00 00 00  ♦   ☻   ◘   ♦
0000000050:  02 00 00 00 01 00 00 00   40 00 00 00 01 00 00 00  ☻   ☺   @   ☺
0000000060:  1E 80 D0 FF 00 00 00 00   03 03 00 02 01 00 08 00  ▲Ä╨         ♥♥ ☻☻ ◘
0000000070:  02 00 04 00 00 00 24 00   05 00 57 69 6E 64 6F 77  ☻ ♦    $ ♣ Window
0000000080:  73 2F 4E 54 20 20 20 20   20 20 20 20 20 20 20 20  s/NT
0000000090:  20 20 20 20 20 20 20 20   20 20 14 00 06 00 35 2E            ¶ ♠ 5.
00000000A0:  31 20 28 32 36 30 30 29   20 20 20 20 20 20        1 (2600)
```

Рис 3. Test04.ngp

И что получилось после компиляции Test05:




```
0000000000:  FF FF 50 00 D5 00 00 00   FF FF FF FF FF FF FF FF    P ╒
0000000010:  FF FF FF FF FF FF 06 00   06 00 06 01 01 09 00 00          ♠ ♠ ♠◘◘○
0000000020:  54 45 53 54 30 35 20 20   BE 92 43 34 64 D4 80 00  TEST05  ╛TC4d╘A
0000000030:  00 00 00 00 00 00 00 00   68 00 00 00 01 00 00 00            h   ☺
0000000040:  0C 00 00 00 02 00 00 00   18 00 00 00 04 00 00 00  ♀   ☻   ↑   ♦
0000000050:  09 00 00 00 01 00 00 00   40 00 00 00 01 00 00 00  ○   ☺   @   ☺
0000000060:  0A 00 00 00 01 00 00 00   14 80 04 00 02 00 FE FF  ◙   ☺   ¶Ä♦ ☻ ■
0000000070:  1E 80 D0 FF 00 00 00 00   03 03 00 02 00 00 00 00  ▲Ä╨         ♥♥ ☻
0000000080:  05 0A 00 0A 02 00 00 00   03 0A 00 07 01 00 77 61  ♣◙◙ ◙☻    ♥◙ •☺ wa
0000000090:  73 6D 2E 72 75 08 00 02   00 01 00 00 00 24 00 05  sm.ru◘ ☻ ☺    $ ♣
00000000A0:  00 57 69 6E 64 6F 77 73   2F 4E 54 20 20 20 20 20   Windows/NT
00000000B0:  20 20 20 20 20 20 20 20   20 20 20 20 20 20 20 20
00000000C0:  20 14 00 06 00 35 2E 31   20 28 32 36 30 30 29 20   ¶ ♠ 5.1 (2600)
00000000D0:  20 20 20 20 20
```

Рис 5. Test05.ngp

Сводим в таблицу фиксированные заголовки

Таб.7 Свод (Заголовки, 1-4-5)

Смещение	Размер	Test01	Test04	Test05	Комментарий
0x00	2	0xFFFF	0xFFFF	0xFFFF	Пропускаем
0x02	1	0x50 (ASCII "P")	0x50 (ASCII "P")	0x50 (ASCII "P")	Тип объекта
0x03	1	0x00	0x00	0x00	Пропускаем
0x04	2 (?)	0xAE	0xAE	*0xD5*	Размер GP-кода
0x06	2	0x00	0x00	0x00	Пропускаем
0x08	14	0xFF (14 повторений)	0xFF (14 повторений)	0xFF (14 повторений)	Пропускаем
0x16	2(?)	0x06	0x06	0x06	Длина имени программы
0x18	2(?)	0x05	0x05	*0x06*	Количество секций?
0x1A	1	0x06	0x06	0x06	Version
0x1A	1	0x01	0x01	0x01	Release
0x1A	1	0x01	0x01	0x01	System maintenance level
0x1A	1	0x09	0x09	0x09	Patch level
0x1E	2	0x0	0x0	0x0	Пропускаем
0x20	8	TEST01	*TEST04*	*TEST05*	Имя программы
0x28	8(?)	BE91D8001C28000	*BE9243273EEF0000*	*BE92453464D48000*	Timestamp

(Курсивом с с подчеркиванием отмечены отличия от предыдущего состояния)

Наше предположение о размере GP-кода подтвердилось. Также подтвердилось "интуитивно очевидное" предположение, что комментарии и пустые строки не увеличивают размер GP-кода. И, по большому счету, структура фиксированного заголовка стала довольно прозрачной. А вот с секциями (а значит, и с предполагаемой структурой GP-кода) пока до ясности далеко.

Таб.8 Свод (табл.секций, 1-4-5)

Индекс табл.	Test01	Test04	Test05
1	60 00 00 00 01 00 00 00	60 00 00 00 01 00 00 00	*68* 00 00 00 01 00 00 00
2	04 00 00 00 02 00 00 00	04 00 00 00 02 00 00 00	*0C* 00 00 00 02 00 00 00
3	08 00 00 00 04 00 00 00	08 00 00 00 04 00 00 00	*18* 00 00 00 04 00 00 00
4	02 00 00 00 01 00 00 00	02 00 00 00 01 00 00 00	*09* 00 00 00 01 00 00 00
5	40 00 00 00 01 00 00 00	40 00 00 00 01 00 00 00	40 00 00 00 01 00 00 00
6			*0A 00 00 00 01 00 00 00*

Сравнение предполагаемых таблиц секций Test01 и Test04 не выявило никаких различий, что, в общем-то, вполне ожидаемо (напомню, что исходный код Test04 отличается от Test01 строкой комментария и пустой строкой). А вот Test05 преподнесла сюрпризы! Если наши предположения насчет таблицы секций верны, то в Test05 по сравнению с Test04 (и Test01 соответственно) добавилась новая секция (что отразилось в фиксированном заголовке) и изменились размеры секций (опять-таки, если наши предположения верны).

Проверяем наше предположение, что "Первый элемент строки таблицы секции - размер секции". Если сумма первых элементов будет равна размеру GP-кода из заголовка (а она, в свою очередь - физическому размеру файла), то честь нам и хвала, если же нет - рвем на себе остатки волос, вешаем IDA на гвоздь и уходим из большого спорта!

Проверим предположение для файла Test05: 0x68 + 0x0C + 0x18 + 0x09 + 0x40 + 0x0A = 0xDF

Вот так номер! В заголовке-то 0xD5! И физический размер файла, однако, тоже 0xD5…

Предлагаю не спешить с вырыванием волос, тем же, кто успел все же выдрать клок, настоятельно рекомендую приклеить его обратно.

Очень не хочется отбрасывать гипотезу, что перед ними таблица секций и размеры секций в начале строк этой таблицы! И основным аргументом, в пользу того, что отбрасывать гипотезу рановато, является следующее. Сумма первых элементов таблицы (размеры секций, по нашему предположению) отличаются от физического размера файла ровно на последнее слагаемое!

Действительно, 0xDF - 0xD5 = 0x0A

Что это, случайность или закономерность? Придется заняться этими секциями вплотную!

Пусть первая секция остается заголовочной - ее размер с завидным постоянством в точности покрывает фиксированный заголовок и пресловутую таблицу секций. Давайте посмотрим, что представляет собой вторая секция:

Таб.9 Свод (секц№2, 1-4-5)			
	Размер секции	**Расположение**	**Содержимое**
Test01	0x04	0x60 – 0x63	0A 80 D0 FF
Test04	0x04	0x60 – 0x63	1E 80 D0 FF
Test05	0x0C	0x68 – 0x73	14 80 04 00 02 00 FE FF 1E 80 D0 FF

Содержимое вторых секций Test01 и Test04 отличаются первым байтом (скажем больше, не только содержимое секции, но и файлов, за исключением, конечно, timestamp). Вспоминаем, Test01 и Test04 отличаются тем, что единственный оператор в первом случае находился в первой строке (с учетом того, что NATURAL нумерует строки через 10, первая строка соответствует числу 0010dec или 0x0A), а во втором - в третьей строке (т.е. 0030 dec или 0x1E). Раз это номер строки, то за ним должны операторы и операнды, т.е. сам код программы. Значит, перед нами секция, содержащая код программы?! Посмотрим, что у нас с Test5. В первом байте колонки Содержимое видим 0x14 (0020dec - вторая строка исходного кода), а в девятом байте в колонки Содержимое - 0x1E (0030 dec - третья строка исходного кода). Что ж совсем неплохо! А идентичная концовка 80 D0 FF вообще вызывает оптимизм.

Правда, настораживает, что всего один байт зарезервирован за номером строки. Явно маловато. Из документации следует, что максимально возможное значение номера строки 9999dec , а оно в один байт никак не поместится. Чтобы внести ясность, проделаем два высокоинтеллектуальных эксперимента. В первом, исходный код будет состоять из 300 пустых строк и оператора END, а втором из 9998 пустых строк (о, боже!) и оператора END:

Таб.10 Журнал экспериментов					
№	**Имя программы**	**Тип**	**Физ. файл**	**Что проверяем**	**Каким образом**
6	Test06	Program	Test06.ngp	Номер строки	300 пустых строк + оператор END 0010 0020 ... 3010 end (нумеруется через 10)
7	Test07	Program	Test07.ngp	Номер строки	9998 пустых строк + оператор END 0001 0002 ... 9999 end (шаг нумерации автоматически стал 1)

Таб 11. Свод (ном.строк 1-6-7)		
	№ строки с END в исходнике	**№ строки + END**
Test01	0010	0A 80 D0 FF
Test06	3010	C2 8B D0 FF
Test07	9999	0F A7 D0 FF

Вот так фокус! Два последних байта остаются неизменными, и, похоже, именно они соответствуют оператору END, а вот с двумя первыми творится что-то непонятное.

Хотя, все не так сложно. Предполагаем, что за номер строки отвечает 2 байта, меняем порядок последовательности этих байтов (что логично и правомерно), вспоминаем шестнадцатеричную систему счисления и находим весьма любопытную закономерность

Таб 12. Свод (Знач.Ном.Строк 1-6-7)			
	№ строки в GP-коде	№ строки с END в исходнике	Разность
Test01	800A	0010_{dec} (0x000A)	0x8000
Test06	8BC2	3010_{dec} (0x0BC2)	0x8000
Test07	A70F	9999_{dec} (0x270F)	0x8000

Т.е, номер строки (исход. кода) = номер строки (объект. код) - 0x8000

(тонкие ценители двоичной системы счисления увидят еще более элегантное решение)

К вопросу - зачем нужны такие трудности, вернемся несколько позже, а сейчас отметим второе точное попадание с размерами секций, и вернемся к нашей таблице секций.
На очереди третья секция:

Таб.13 Свод (секц№3, 1-4-5)			
	Размер секции	Расположение	Содержимое
Test01	0x08	0x64 – 0x6B	00 00 00 00 03 03 00 02
Test04	0x08	0x64 – 0x6B	00 00 00 00 03 03 00 02
Test05	0x18	0x74 – 0x8B	00 00 00 00 03 03 00 02 *00 00 00 00 05 0A 00 0A* *02 00 00 00 03 0A 00 07*

Что это такое, пока не очень понятно. Похоже, что это тоже какая-то таблица, причем она начинается и заканчивается в полном соответствии с нашими предположениями размерах секций. Выяснение назначения этой секции оставим на потом, а сейчас отметим очередное попадание - пусть и не столь убедительное, как первые два, но наша гипотеза еще вполне жизнеспособна.
Четвертая секция в наших примерах выглядит так:

Таб.14 Свод (секц№4, 1-4-5)			
	Размер секции	Расположение	Содержимое
Test01	0x02	0x6C – 0x6D	01 00
Test04	0x02	0x6C – 0x6D	01 00
Test05	0x09	0x8C – 0x94	01 00 77 61 73 6D 2E 72 75

Что такое 01 00 не очень понятно, а вот 77 61 73 6D 2E 72 75 не что иное, как "wasm.ru", т.е. то, что мы присвоили переменной. Уж не область ли это констант? Опять-таки, оставим выяснение этого вопроса на потом, и опять-таки, отметим жизнеспособность наших предположений относительно таблицы секций.
Содержимое пятой секции идентично для всех наших экспериментов, потому приводить ее не буду. Назначение ее не совсем очевидно, но в ней можно уверенно обнаружить следы ОС, под которой компилировалась программа.
Шестая секция…
А вот с ней как раз и сложности. Причем по простой причине - файл GP-кода закончился! Т.е. эта секция повисла в воздухе!
Давайте попробуем порассуждать (несколько раз нам это уже помогло!).
Вопрос: с чего вдруг она взялась, эта зловещая секция?
Ответ: после того, как мы объявили переменную и присвоили ей значение. И если присвоение как-то отразилось в рассмотренных ранее секциях (напомню, увеличилась секция, содержащая программный код, скорее всего именно на операцию присвоения, и увеличилась также секция с константами), то следы объявления переменной мы можем попробовать поискать только в третьей секции. Но тогда непонятно, почему объявление одной переменной добавило две строки в таблицу, содержащуюся в третьей секции? И еще более непонятно, связан ли размер секции-фантома с размером объявленной переменной (напомню, в обоих случаях он 0x0A)?

Чтобы внести ясность в возникшие вопросы, поставим еще один эксперимент, заключающийся в следующем. Модифицируем Test05 так, чтобы там объявлялась переменная с легко идентифицируемом размером.

Таб.15 Журнал экспериментов

№	Имя программы	Тип	Физ. файл	Что проверяем	Каким образом
8	Test08	Program	Test08.ngp	Влияние размера переменной на GP-код	Программа такая же, что и Test05, но размер переменной будет 123 байта. Дополнительно, увеличим присваиваемую этой переменной строку. Т.е. в итоге должны получить 0010 /* комментарий 0020 strl (al 23) = 'www.wasm.ru' 0030 end

Вот, что получим в итоге:

```
0000000000:  FF FF 50 00 D9 00 00 00   FF FF FF FF FF FF FF FF    p ┘
0000000010:  FF FF FF FF FF FF 06 00   06 00 06 01 01 09 00 00          ♠ ♠ ♠◎◎○
0000000020:  54 45 53 54 30 38 20 20   BE 9A EF D1 9C 8E 00 00  TEST08  ╛Ъя╤╜O
0000000030:  00 00 00 00 00 00 00 00   68 00 00 00 01 00 00 00          h   ☺
0000000040:  0C 00 00 00 02 00 00 00   18 00 00 00 04 00 00 00  ↓ ☻  ↑ ♦
0000000050:  0D 00 00 00 01 00 00 00   40 00 00 00 01 00 00 00  ♪ ☺  @ ☺
0000000060:  7B 00 00 00 01 00 00 00   14 80 04 00 02 00 FE FF  { ☺  ¶Ä♦ ☻ ■
0000000070:  1E 80 D0 FF 00 00 00 00   03 03 00 02 00 00 00 00  ▲Ä╨      ♥♥ ☻
0000000080:  05 0A 00 7B 02 00 00 00   03 0A 00 0B 01 00 77 77  ♣◙ {☻    ♥◙ ♪◎ ww
0000000090:  77 2E 77 61 73 6D 2E 72   75 08 00 02 00 01 00 00  w.wasm.ru◘ ☻ ☺
00000000A0:  00 24 00 05 00 57 69 6E   64 6F 77 73 2F 4E 54 20   $ ♣ Windows/NT
00000000B0:  20 20 20 20 20 20 20 20   20 20 20 20 20 20 20 20
00000000C0:  20 20 20 20 20 14 00 06   00 35 2E 31 20 28 32 36       ¶ ♠ 5.1 (26
00000000D0:  30 30 29 20 20 20 20 20   20                       00)
```

Рис 7. Test08.ngp

Таб.16 Свод (Разм.секц, 5-8)

Секция		Размер секции	Расположение	Комментарий
1 Фиксированный заголовок + таблица секций.	**Test05**	0x68	0x00 – 0x67	Изменилось значение поля «Размер GP-кода» - было 0xD5, стало 0xD9
	Test08	0x68	0x00 – 0x67	
2 Секция кода	**Test05**	0x0C	0x68 – 0x73	Без изменений.
	Test08	0x0C	0x68 – 0x73	
3 Секция №3	**Test05**	0x18	0x74 – 0x8B	Изменилось содержимое секции, размер остался прежним
	Test08	0x18	0x74 – 0x8B	
4 Секция констант	**Test05**	0x09	0x8C – 0x94	Есть контакт! Увеличился размер в соответствии с увеличением размера присваиваемой строки.
	Test08	0x0D	0x8C – 0x98	
5 Секция №5	**Test05**	0x40	0x95 – 0xD4	Без изменений.
	Test08	0x40	0x98 – 0xD7	
6 Секция №6	**Test05**	0x0A	–	Есть контакт! Увеличился размер в соответствии с увеличением размера объявленной переменной.
	Test08	0x7B	–	

Мозаика собралась: если секция №6 действительно область переменных, то нет никакого смысла в ее физическом присутствии в файле GP-кода. А вот сообщить интерпретатору, что необходимо отвести определенное количество памяти под переменные - это святое.

Что ж, многое стало понятным. Мы теперь имеем некое представление о структуре GP-кода, можем с уверенностью разобрать основные поля фиксированного заголовка, определить количество, размер (а иногда и назначение!) секций. Но где начинается пресловутая таблица секций? Переиначим вопрос - где заканчивается фиксированная часть заголовка?

Смотрите, у нас есть размер всего заголовка (первая строка таблицы секций любого примера), есть количество секций в фиксированной части заголовка (т.е. количество строк в таблице секций) и есть размер строки таблицы секций. Т.е. необходимо из первого вычесть произведение второго и третьего. В результате получим (для определенности, возьмем данные Test08):

0x68 - 0x06 * 0x08 = 0x38

Т.о. фиксированная часть заголовка имеет размер 0x38 байт, а значит "неоприходованная" последовательность нулей принадлежит фиксированному заголовку.

Заключение

Попробуем схематично представить полученные сведения

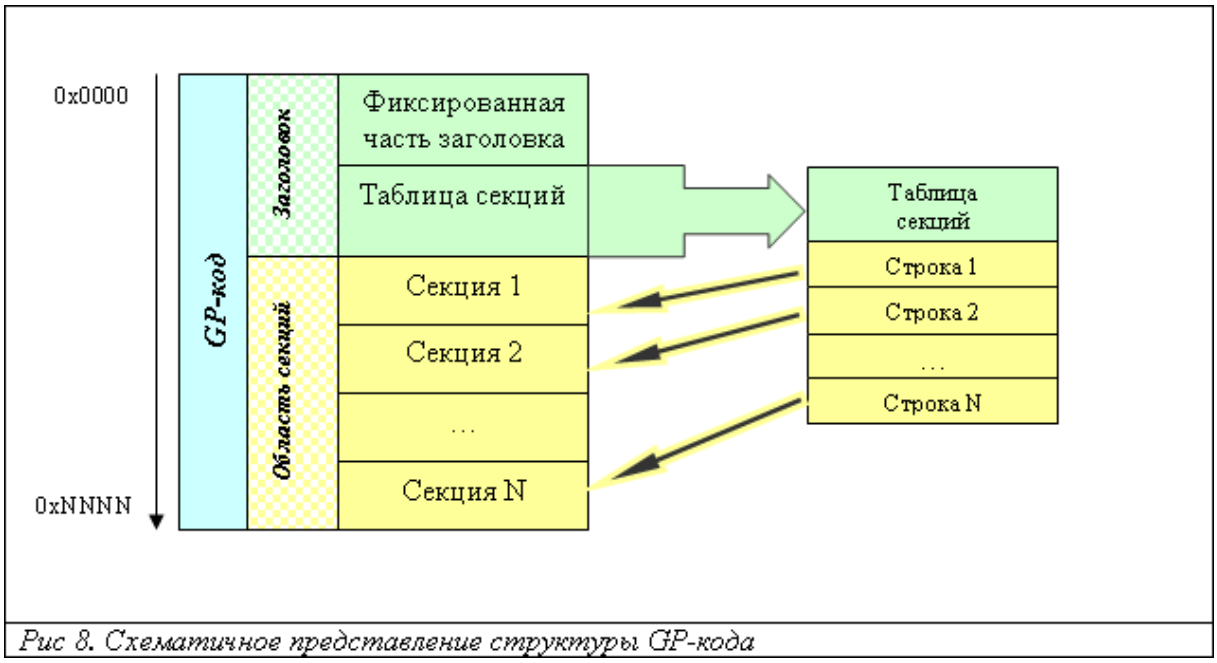


Рис 8. Схематичное представление структуры GP-кода

GP-код программы можно условно разделить на Заголовок и Область Секций. В Заголовке в свою очередь можно выделить Фиксированную Часть Заголовка и Таблицу секций. Таблица секций содержит информацию о секциях. Структура Заголовка и Таблицы секций приведена в Приложении 1.

Следующая часть статьи будет посвящена переменным и всему, что с ними связано.

PS: Выражаю свою благодарность VOlegV за конструктивные правки и замечания.

Приложение 1

1. Структура Фиксированной части Заголовка

Field name	Offset	Size	Value & Comment
GP version	0x00	2	FFFF = GPVERS_PORTABLE
Object type	0x02	1	См п1 .1
Endian mode	0x03	1	0 = Little endian mode (low order byte first). 1 = Big endian mode (high order byte first)
GP size	0x04	4	
CT offset: on error	0x08	4	
CT offset: db call RC	0x0C	4	
CT offset: common	0x10	4	
BRT offset: final brk	0x14	2	
Length GP name	0x16	2	
Number of TABDIR entries	0x18	2	
Natural major version	0x1A	1	
Natural minor version	0x1B	1	
Natural sm version	0x1C	1	
Natural p1 version	0x1D	1	
Char set	0x1E	1	0 = ASCII
Flags	0x1F	1	
GP name	0x20	8	
GP timestamp	0x28	8	
GDA GP timestamp	0x30	8	

1.1 Перечень компилируемых типов объектов Natural

Object type	Name	Src file ext	GP file ext	Comment
L	Local Data Area (LDA)	.nsl	.ngl	Область локальных переменных
G	Global Data Area (GDA)	.nsg	.ngg	Область глобальных переменных
A	Parameter Data Area (PDA)	.nsa	.nga	Область параметров
P	Program	.nsp	.ngp	Программа
7	Function	.ns7	.ng7	Функция
N	Subprogram	.nsn	.ngn	Подпрограмма
S	Subroutine	.nss	.ngs	Внешняя подпрограмма
H	Helproutine	.nsh	.ngh	Помощь
M	Map	.nsm	.ngm	Форма
3	Dialog	.ns3	.ng3	Диалог
D	Data definition module (DDM)	.nsd	.ngd	Модуль, содержащий информацию о хранении данных
4	Class	.ns4	.ng4	Класс

2. Структура Таблицы секций

Field name	Size
Size	4
Align	4

Идентификатора секции в Таблице секций нет. Идентификация происходит по индексу, т.е. за каждой секцией "закреплен" свой индекс. Подробнее про идентификацию секций и индексы см. в следующей части статьи.

Исследование «промежуточного» кода на примере GP-кода языка NATURAL - Часть 2: Операнды

Автор: Konstantin

Дата: 2006-07-26

Как известно, операндами называют то, что участвует в операциях. Так в операции сложения операндами будут слагаемые, а в операции умножения - сомножители.

Применительно к GP-коду Natural, термин "операнд" объединяет такие элементы программного кода как константы, локальные переменные, глобальные переменные, объекты и т.д.

Согласитесь, разобравшись с тем, как и где описываются операнды, нам будет вполне по силам делать "косметический ремонт" программ (изменить формат, размер переменной, атрибут поля формы и т.д.). Также нам станут подвластными и параметры вызываемых подпрограмм. Это самое очевидное, но мало ли, что можно еще придумать!

Полагаю, самым разумным будет начать наше исследование с констант, т.к. во-первых, константы - они в Natural константы, а во-вторых, их легко разглядеть в hex-редакторе.

И еще небольшое отступление. Natural предоставляет возможность использовать два режима написания программ - Report mode (режим отчета, обычно для написания программ "на скорую руку"), Structure mode (структурный режим). Основное отличие (не вдаваясь в особые подробности) заключается в том, что в режиме отчета можно объявлять переменные в любом месте программы по мере необходимости, а в структурном режиме - только в начале программы в блоке, выделенном ключевыми словами Define Data и End-Define. Есть различия в синтаксисе, но нам они сейчас не интересны. Для GP-кода безразлично, в каком режиме написан исходный текст программы. Для большей наглядности далее в статье исходные тексты программ будут представлены в структурном режиме.

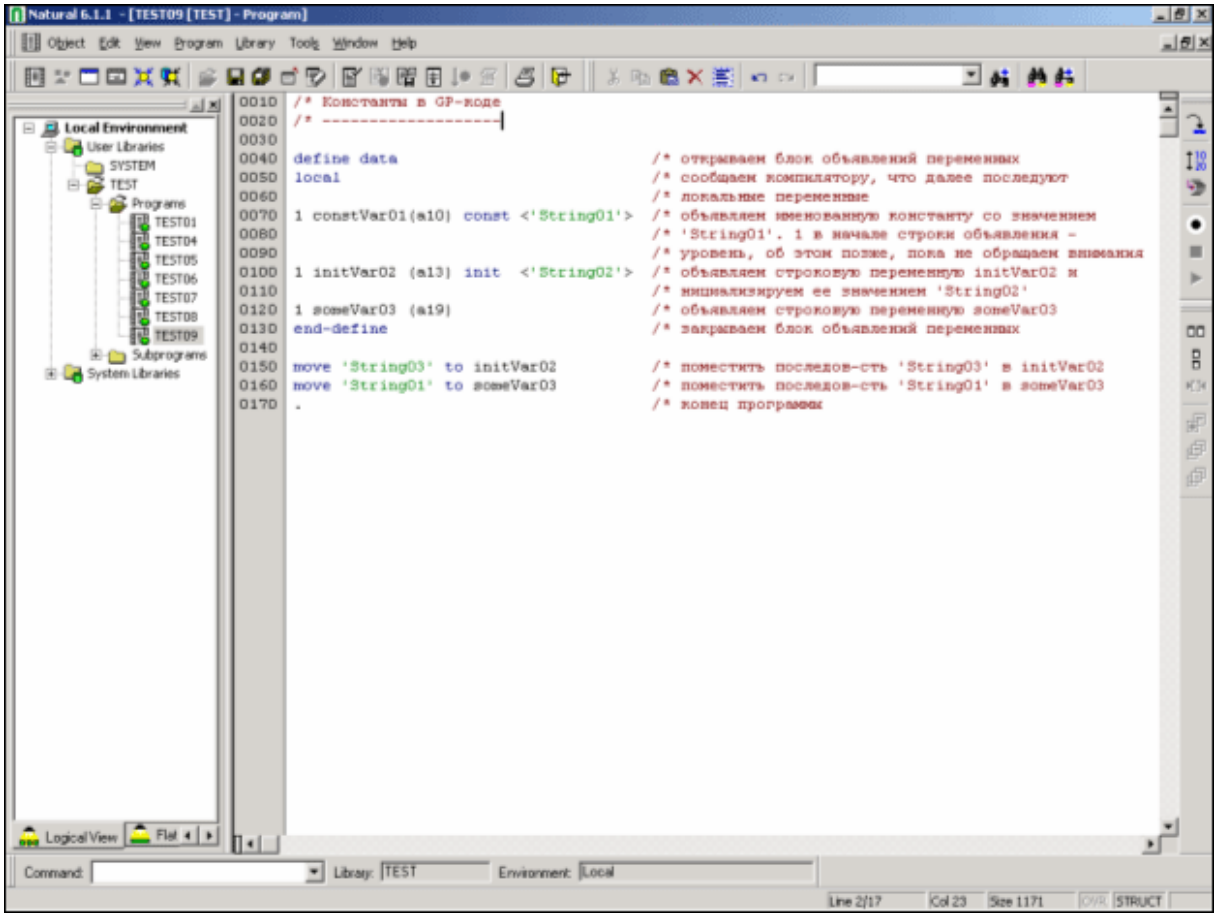
Константы

Предлагаю решительно отказаться от соблазна вывести каноническое "Hello world!" на экран и исследовать получившийся код! Основная причина заключается в том, что вывод на экран - это не самая простая операция (справедливо не только для Natural). Если же есть желание разбираться с буфером формата и другими важными элементами программы, не имея для этого достаточной базы - что ж, дело хозяйское. Вы навсегда останетесь в моей памяти…

Предлагаю пойти другим путем. Рассмотрим константы в двух ипостасях - объявление константы, и присвоение константы. В первом случае мы объявим переменную константой (т.е. такой переменной, значение которой далее программе нельзя будет менять), во втором - присвоение некой переменной (конечно, объявленной не как константа) некоего значения.

Таб.2.1 Журнал экспериментов

№	Имя программы	Тип	Физ. файл	Что проверяем	Каким образом
8	Test09	Program	Test09.ngp	Константы в GP-коде	Объявляем и присваиваем константы. Текст программы см. Рис 2.1



После компиляции получим следующий GP-код

```
0000000000:  FF FF 50 00 4E 01 00 00   FF FF FF FF FF FF FF FF     P N
0000000010:  FF FF FF FF FF FF 06 00   0B 00 06 01 01 09 00 00          ♂ ♠☺☺o
0000000020:  54 45 53 54 30 39 20 20   BE FB CC F2 E4 78 00 00   TEST09  ╛√╠Eфx
0000000030:  00 00 00 00 00 00 00 00   90 00 00 00 01 00 00 00            P   ☺
0000000040:  14 00 00 00 02 00 00 00   40 00 00 00 04 00 00 00   ¶   ☻    @   ♦
0000000050:  1A 00 00 00 01 00 00 00   40 00 00 00 01 00 00 00   →   ☺    @   ☺
0000000060:  2A 00 00 00 01 00 00 00   00 00 00 00 00 00 00 00   *   ☺
0000000070:  00 00 00 00 00 00 00 00   00 00 00 00 00 00 00 00
0000000080:  00 00 00 00 00 00 00 00   10 00 00 00 04 00 00 00            ►   ♦
0000000090:  96 80 0E 00 09 00 FE FF   A0 80 02 00 0C 00 FE FF   ЦA  ○ ■ aA  ↓ ■
00000000A0:  AA 80 D0 FF 00 00 00 00   03 03 00 02 02 00 00 00   кA╨      ♥♥ ☻☻
00000000B0:  03 0A 00 08 00 00 00 00   05 0A 04 0A 00 00 00 00   ♥◙ ◘      ♣◙♦◙
00000000C0:  0A 00 00 00 03 0A 00 08   0A 00 00 00 05 0A 04 0D   ◙    ♥◙ ◘◙   ♣◙♦♪
00000000D0:  02 00 00 00 17 00 00 00   05 0A 00 13 12 00 00 00   ☻    ↨    ♣◙ ‼↕
00000000E0:  03 0A 00 08 01 00 53 74   72 69 6E 67 30 31 53 74   ♥◙ ◘☺ String01St
00000000F0:  72 69 6E 67 30 32 53 74   72 69 6E 67 30 33 08 00   ring02String03◙
0000000100:  02 00 01 00 00 00 24 00   05 00 57 69 6E 64 6F 77   ☻ ☺   $ ♣ Window
0000000110:  73 2F 4E 54 20 20 20 20   20 20 20 20 20 20 20 20   s/NT
0000000120:  20 20 20 20 20 20 20 20   20 20 14 00 06 00 35 2E            ¶ ♦ 5.
0000000130:  31 20 28 32 36 30 30 29   20 20 20 20 20 20 00 A0   1 (2600)        a
0000000140:  00 00 02 00 00 00 00 80   00 00 07 00 00 00          ☻     A  ▪
```

Рис 2.2. Test09.ngp

Из заголовка нам пригодится только значение поля Number of TABDIR entries, которое находится по смещению 0x18.

Ого! А там-то 0x0B!!! Т.е. количество секций 11, вместо полюбившихся нам 6! Но не будем спешить, одиннадцать, так одиннадцать.

На очереди таблица секций.

Напомню, запись в таблице секций состоит из двух полей по 4 байта каждая - размер секции и выравнивание. Начало таблицы секций расположено всегда по смещению 0x38, а размер таблицы равен произведению длины записи в таблице (4 + 4 = 8) и количества записей в таблице (поле Number of TABDIR entries заголовка, т.е. 0x0B). Итого 0x58. Для удобства добавим расчетные столбцы с границами секций

NN	Section	Size	Start	End	Align
01	Section1	0090	0000	008F	1
02	Section2	0014	0090	00A3	2
03	Section3	0040	00A4	00E3	4
04	Section4	001A	00E4	00FD	1
05	Section5	0040	00FE	013D	1
06	Section6	002A	0000	0000	1
07	Section7	0000	0000	0000	0
08	Section8	0000	0000	0000	0
09	Section9	0000	0000	0000	0
10	SectionA	0000	0000	0000	0
11	SectionB	0010	013E	014D	4

Рис 2.3. Test09.ngp (Таблица секций)

С первыми шестью секциями мы разбирались в предыдущей части статьи. Напомню:

1. Заголовок (фиксированная часть и таблица секций).
2. Секция кода.
3. Секция №3 (пока неопознанная)
4. Секция констант
5. Секция №5 (пока неопознанная)
6. Секция переменных. Точнее, размер памяти, который необходимо зарезервировать интерпретатору для размещения переменных во время исполнении программы. Физически в файле не присутствует, потому границы секции не определены.

Секции 7,8,9,10 имеют нулевой размер, говоря проще, их в GP-коде нет, и появилась шестнадцатибайтовая одиннадцатая секция.

Секция №3

Очень похоже, что перед нами очередная таблица, а значит, прежде чем гадать, зачем же она понадобилась, потратим немного времени на то, чтобы разбить ее по записям. Сразу оговорюсь, что понятие "запись" здесь условно. Нам не важно на этом этапе, каким образом в действительности происходит запись или чтение этой секции, нам важно понять ее назначение, а потому мы вольны в своих действиях. Т.е. задача - разбить последовательность данных на составляющие, осязаемые, понятные нам, на составляющие, которые мы смогли бы интерпретировать.

Основной критерий - повторяемость.

Для начала, предположим, что длина записи в таблице равна значению поля Align для 3 секции в таблице секций (см. Таб.2.1 Вариант 1). Беглого взгляда достаточно, чтобы увидеть некую периодичность, или повторяемость.

Увеличиваем длину записи до 8 (см. Таб.2.1 Вариант 2). Уже что-то выстраивается! Однако если с первой половиной записей все выглядит довольно оптимистично, то вторая половина явно сбилась. Попробуем добиться большей повторяемости (см. Таб.2.1 Вариант 3)

Таб.2.1 Варианты длины записи секции №3		
Вариант 1 (длина = 4)	**Вариант 2 (длина = 8)**	**Вариант 3 (длина переменная, кратная 4)**
00 00 00 00	00 00 00 00 03 03 00 02	00 00 00 00 03 03 00 02
03 03 00 02	02 00 00 00 03 0A 00 08	02 00 00 00 03 0A 00 08
02 00 00 00	00 00 00 00 05 0A 04 0A	00 00 00 00 05 0A 04 0A
03 0A 00 08	00 00 00 00 0A 00 00 00	00 00 00 00
00 00 00 00	03 0A 00 08 0A 00 00 00	0A 00 00 00 03 0A 00 08
05 0A 04 0A	05 0A 04 0D 02 00 00 00	0A 00 00 00 05 0A 04 0D
00 00 00 00	17 00 00 00 05 0A 00 13	02 00 00 00
0A 00 00 00	12 00 00 00 03 0A 00 08	17 00 00 00 05 0A 00 13
03 0A 00 08		12 00 00 00 03 0A 00 08
0A 00 00 00		
05 0A 04 0D		
02 00 00 00		
17 00 00 00		
05 0A 00 13		
12 00 00 00		
03 0A 00 08		

Визуально все выглядит вполне пристойно. Попробуем интерпретировать получившееся. Обратите внимание на последние байты записей - 0x0A, 0x0D, 0x13 - это ни что иное, как размеры объявленных нами переменных! Что такое 02 не понятно, а вот 08 вполне может быть размером наших строковых констант (они действительно по 8 байт). Единственное, что настораживает - вообще-то строковых констант в программе было 4, а здесь всего 3. Но смотрим дальше. Если наши предположения верны и в последних байтах записей помещены размеры переменных (точнее, операндов), то надо бы отыскать формат переменных. Все наши объявленные переменные являются символьными переменными, константы - тоже символьные, и лучшим кандидатом на хранение формата является шестой байт записи (во всех записях, кроме первой имеет одинаковое значение 0x0A).

Седьмой байт ничем не примечателен, кроме одного - если он равен 04 (или не равен 0 - это надо уточнять), то далее идет тот самый четырехбайтный аппендикс, который портил всю картину. Примем в качестве рабочей гипотезы, что это какой-то флаг, или нечто подобное. Исходя из этого предположения, еще немного преобразуем подопытную таблицу.

Таб.2.2 Окончательное представление секции №3	
	Вариант 4 (длина переменная, кратная 4)
1	00 00 00 00 03 03 00 02
2	02 00 00 00 03 0A 00 08
3	00 00 00 00 05 0A 04 0A 00 00 00 00
4	0A 00 00 00 03 0A 00 08
5	0A 00 00 00 05 0A 04 0D 02 00 00 00
6	17 00 00 00 05 0A 00 13
7	12 00 00 00 03 0A 00 08

Пятый байт любопытен двумя свойствами:

• тем, что для записей, в которых последние (восьмые) байты 0x0A, 0x0D, 0x13, т.е. объявленные нами явно переменные, имеет значение 05, в остальных же случаях (уж не для идентификации ли констант!) имеет значение 03;
• явно влияет на поведение первого байта.

Рассмотрим второе свойство подробнее.

Наблюдаем, как изменяется первый байт только для записей с пятым байтом, равным 03 (т.е. записи 1,2,4,7).

Первый байт увеличивается точно на размер предыдущей. Тоже самое происходит и для записей с пятым байтом, равным 05 (записи 3,5,6).

Вполне вероятно, что первая группа байтов - относительный адрес переменной в соответствующей ей секции.

По-крупному с третьей секцией понятно - это секция, содержащая описание операндов. Остался вопрос с пропавшей константой, к которому обязательно вернемся чуть позже.

Таб.2.3 Таблица операндов

	Код секции	Комментарий
1	00 00 00 00 03 03 00 02	Константа, разм 2 байта, формат ?
2	02 00 00 00 03 0A 00 08	Символьн. константа разм. 8 байт
3	00 00 00 00 05 0A 04 0A 00 00 00 00	Символьн. переменная разм. 10 байт (constVar01)
4	0A 00 00 00 03 0A 00 08	Символьн. константа разм. 8 байт
5	0A 00 00 00 05 0A 04 0D 02 00 00 00	Символьн. переменная разм. 13 байт (initVar02)
6	17 00 00 00 05 0A 00 13	Символьн. переменная разм. 19 байт (someVar03)
7	12 00 00 00 03 0A 00 08	Символьн. константа разм. 8 байт

Секция №4 (секция констант)

Содержимое секции представлено на рис 2.4.

```
000000E4: 01 00 53 74 72 69 6E 67 | ..String
000000EC: 30 31 53 74 72 69 6E 67 | 01String
000000F4: 30 32 53 74 72 69 6E 67 | 02String
000000FC: 30 33                   | 03
```

Рис 2.4. Test09.ngp (секция констант)

Здесь мы легко обнаруживаем строковые константы, которые используются в нашей программе, а также последовательность 01,00 в начале секции.

Возникает два вопроса:

1. что это за последовательность?
2. куда пропала четвертая константа?
3. как с этим безобразием работать?

Относительно первого вопроса, ответ простой - не знаю! Да и особо не интересовался. Известно лишь, что эта последовательность встречается и в "минимальной" программе.

Относительно второго вопроса - перед нами зачатки оптимизации! Вернитесь к исходному коду программы - там строковая константа "String01" используется 2 раза. Компилятор посчитал расточительным хранить две идентичных константы, в чем, безусловно, прав. Жаль, на этом оптимизирующие свойства компилятора практически заканчиваются. С другой стороны, нам, исследователям, это только на руку.

Относительно третьего вопроса - эта секция напрямую связана с секцией №3. Посудите сами - записи с описанием констант (т.е. записи 1,2,4,7 в Таб. 2.2, имеющие значение 03 в пятом байте) в точности соответствуют содержимому секции констант.

Таб.2.4 Таблица операндов

№№	Ссылка	Код секции	Комментарий
1	0x00	00 00 00 00 03 03 00 02	Константа, разм 2 байта, формат ? Знач: 01
2	0x02	02 00 00 00 03 0A 00 08	Символьн. константа разм. 8 байт Знач: String01
3	0x04	00 00 00 00 05 0A 04 0A 00 00 00 00	Символьн. переменная разм. 10 байт (constVar01)
4	0x07	0A 00 00 00 03 0A 00 08	Символьн. константа разм. 8 байт Знач: String02
5	0x09	0A 00 00 00 05 0A 04 0D 02 00 00 00	Символьн. переменная разм. 13 байт (initVar02)
6	0x0C	17 00 00 00 05 0A 00 13	Символьн. переменная разм. 19 байт (someVar03)
7	0x0E	12 00 00 00 03 0A 00 08	Символьн. константа разм. 8 байт Знач: String03

Секция №11

Содержимое секции представлено на Рис 2.5.

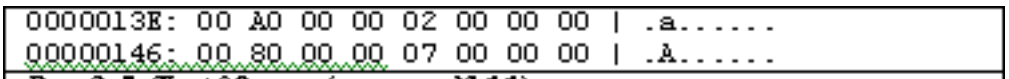


Рис 2.5. Test09.ngp (секция №11)

Здесь не все очевидно, но достаточно просто выясняется, поставив несколько экспериментов. Я не буду морочить вам голову этими экспериментами - они того не стоят (о том, что эксперименты надо планировать, а полученные результаты журнализировать, надеюсь, вы помните!).

Логика рассуждений для прогноза результатов может быть такой. Пока не было изысков в виде "const" и "init" при объявлении переменных, одиннадцатая секция не проявлялась. Опять же именно для этих переменных выросли четырехбайтовые аппендиксы. Поэтому вполне возможно, что перед нами секция атрибутов переменных - операндов (проще говоря, секция каких-то дополнительных свойств операндов). И если вы предположите, что пресловутый аппендикс в таблице операндов не что иное, как ссылка на соответствующую запись в секции атрибутов, то будете абсолютно правы! Вы также будете правы, если предположите, что запись помимо признака константы или инициализированной переменной содержит и ссылку на то, чем переменная инициализирована.

Эксперименты покажут, что (см. Рис 2.5) 0xA0 - признак константы, 0x80 - признак инициализированной переменной (стоит отметить, что это довольно характерные значения для всякого рода флагов), 02 - ссылка на последовательность "Srting01", а 07 - ссылка на последовательность "Srting02".

Буквально несколько слов об использовании ссылок в GP-коде.

В GP-коде для связи секций-таблиц не используется номер записи в таблице в качестве ссылки, т.к. длина записи в таблице не постоянна. Не используется также и относительный адрес на начало записи, а используется некий симбиоз - индекс секций, равный отношению относительного адреса начала записи к значению Align соответствующей секции. Оригинальное название индекса для каждой секции свое.

Чтобы картина описания констант в GP-коде стала более четкой (мы прошлись по ней лишь крупными мазками), пройдемся по предложенному примеру еще раз, и начнем с таблицы операндов.

Суммируя полученную информацию

Таблица операндов предназначена для хранения информации об операндах, имеет переменную длину записи (постоянная часть записи плюс, если есть, переменная часть записи).

Постоянная часть записи содержит следующую информацию:

• адрес операнда в соответствующей операнду области (в области констант, в области локальных переменных и т.д.);
• тип операнда (константа, локальная переменная и т.д.)
• формат операнда (символьный, числовой, и т.д.);
• флаг (его значение показывает, есть ли дополнительная информация, связанная с описанием операнда, и где дополнительная информация находится);
• размер операнда.

Переменная часть записи содержит дополнительную информацию об операнде и содержит ссылку на соответствующую запись в соответствующей таблице с этой дополнительной информацией.

С таблицей операндов, думаю, достаточно, переходим к таблице атрибутов.

Таблица атрибутов (секция №11) имеет тоже переменную длину. Постоянная часть содержит следующую информацию:

• флаг (на самом деле, конечно, флаги, но давайте договоримся, что пока этого не знаем);
• режим печати (о! а вот это отложим до лучших времен).

Если флаг имеет значение 0xA0 (константа) или 0x80 (инициализация переменной), то следующие 4 байта (переменная часть) - это ссылка значение, которое присвоено константе (в первом случае), либо которым инициализирована переменная (во втором случае). Ссылка на таблицу операндов!

Рассмотрим в качестве примера запись №5 в таблице операндов (Таб.2.4). Последовательность 0A 00 00 00 05 0A 04 0D 02 00 00 00 может быть интерпретирована следующим образом. Постоянная часть (8 байт) содержит информацию о локальной (0x05, 5-ый байт) символьной (0x0A, 6-ой байт) переменной размером 13 байт (0x0D, 8-ой байт), находящейся по смещению 0x0A (0x0000000A, 1-4-ые байты) в области локальных переменных. Это описание операнда дополняется информацией об атрибутах операнда (0x04, 7-ой байт) - переменной частью записи. Значение флага 0x04 указывает нам, что дополнительная информация находится в секции атрибутов (секция №11), а значение 02 (1-2-ые байты, переменная часть записи) указывает на вторую запись в таблице атрибутов. Вот как это считается. Как мы уже знаем (Рис. 2.3), значение Align для секции №11 (секции атрибутов) равно 4, а произведение 2 (1-2-ые байты, переменная часть записи) и 4 дает 8, т.е. запись, содержащая дополнительную информацию об атрибутах операнда, находится по смещению 8 от начала секции атрибутов, а это и есть вторая запись.

Возвратимся к нашему примеру и попробуем интерпретировать последовательность 00 80 00 00 07 00 00 00 (Таб.2.4), соответствующую второй записи в таблице атрибутов.

Постоянная часть (4 байта) показывает, что перед нами переменная, инициализированная (0x80, 2-ой байт) … а вот каким значением сейчас рассчитаем.

Значение Align (Рис. 2.3) для секции №3 (таблицы операндов) равно 4, а произведение 7 (1-2 байты, переменная часть записи) и 4 дает 28 (0x1C), т.е. переменная инициализирована операндом (!), информация о котором находится по смещению 28 (0x1C) от начала таблицы операндов. А это ни что иное, как 4 запись в этой таблице. Попробуем разобрать последовательность 0A 00 00 00 03 0A 00 08 , соответствующую смещению 28 (0x1C). Постоянная часть (8 байт) содержит информацию о константной (0x03, 5 байт) символьной (0x0A, 6 байт) последовательности размером 8 байт (0x08, 8 байт), находящейся по смещению 0x0A (0x0000000A, 1-4 байты) в области констант. Под область констант отведена секция №4. Отсчитываем 0x0A от начала области констант и получаем восьмибайтную последовательность "String02".

Что ж, вполне похоже на правду.

Более подробную информацию о структурах секций - таблиц и возможные значения некоторых полей можно найти в Приложении 2.

Секция кода

Что ж, настало время взглянуть в "святая святых" - в секцию кода!

Это вторая секция, и думаю, не составит большого труда по Рис 2.3 определить границы секции, а по Рис 2.2 взглянуть на содержимое.

Напомню, что в предыдущей части статьи мы научились распознавать в этой секции номера строк, поэтому привязать содержимое секции к исходному тексту программы будет не сложно.

Таб.2.5 Сопоставление секции кода и исходного текста программы

№№	Секция кода	Исходный текст	Комментарий
1	96 80 0E 00 09 00 FE FF	0150 move 'String03' to initVar02	поместить последов-сть 'String03' в initVar02
2	A0 80 02 00 0C 00 FE FF	0160 move 'String01' to someVar03	поместить последов-сть'String01' в someVar03
3	AA 80 D0 FF	0170 .	конец программы

Два первых байта записей - номера строк (об этом мы говорили в предыдущей части статьи), два последних - идентификаторы команд (об этом не трудно догадаться, тем более что D0 FF в третьей записи мы научились распознавать опять-таки в предыдущей части статьи). Очевидно, что между номером строки и FE FF (командой MOVE) в первых двух записях должны каким-то боком казаться переменные (что и куда пересылается). Этим "боком" оказывают те самые ссылки на операнды, которые нам уже встречались при разборе констант.

Таким образом, оказывается (можно воспользоваться колонкой Ссылка Таб.2.4) , что

• 0x0E - ссылка на последовательность "String03"
• 0x09 - ссылка на переменную (в тексте программы initVar02)
• 0x02 - ссылка на последовательность "String01"
• 0x0C - ссылка на переменную (в тексте программы someVar03)

Что ж, можно считать, что разбор секции завершен, и полученный результат вполне похож на правду.

С чем вас и поздравляю!

Вместо заключения

В этой части статьи мы рассмотрели принцип организации таблицы операндов в GP-коде на примере констант, благодаря чему можем достаточно уверенно восстановить область локальных переменных. Правда, пока без особых изысков в виде переопределений (redefine), массивов и т.д. Увы, имена переменных нам не доступны, если не выставлен специальный флажок в настройках компилятора, который отвечает за создание Symbol Table при компиляции. Но это не помешает нам восстановить не только логику программы, но и вполне работоспособный исходный текст программы. Впрочем, я слишком тороплюсь…

Мы прикоснулись также к великому таинству секции кода. Благодаря тому, что мы разобрались с таблицей операндов, это прикосновение не ударило по нашей нервной системе. Все оказалось дольно простым. Для тех, кто исследует что-то свое - настоятельно рекомендую начинать исследование секций программного кода лишь после того, как вам станет понятно, как идентифицируются переменные в коде (а в этом вам помогут константы!).

Уверен, что такая организация и секций и межсекционного взаимодействия вызывет много вопросов. Но давайте не будем спешить. Посмотрим, что будет дальше, возможно часть вопросов снимется сама собой!

Приложение 2

1. Секции в GP-коде[1]

Индекс	Комментарий
0	Header Table
1	Code Table
2	Operand Table
3	Constant Table
4	Multi Purpose Table
5	Local Data Area Table
6	Parameter Table
7	Static constant Data Area Table
8	Array Description Table
9	Temp Data Area Table
10	Attribute Table
11	Local Printer Table
12	Include Table
13	Class Description Table
14	Object Data Area Table
15	External Subroutine Table
16	Symbol Table
17	ADABAS command Table
18	Global Data Area Source Table
19	Internal Subroutine Table
20	SQL Table
21	INDEPENDENT Var Table
22	AT Condition Table
23	Break Table
24	SysFunction Table
25	SFA Table

2. Структура постоянной части записи Таблицы операндов (Operand Table)

Field name	Offset	Size	Value & Comment
Value table offset	0x00	4	По-простому, относительный адрес операнда в соответствующей таблице.
Value table index	0x02	1	По-простому, тип операнда. Операнды располагаются в памяти по своим таблицам в соответствии с типом. Т.е. на свой тип – своя таблица. Соответствует индексам секций – Приложение 2.1
Format	0x03	1	Формат операнда. См. Приложение 2.3
Flags	0x04	1	Флаги операнда. См. Приложение 2.4
Length	0x08	1	Длина операнда

3. Форматы операнда

Формат	Объявление в программе	Комментарий
01	N	Числовая
02	P	числовая упакованная
03	I	целая числовая
04	F	числовая с плавающей
05	B	битовая
06	D	дата
07	T	время
08	L	логическая
09	C	атрибут
0A	A	символьная
0B	HANDLE OF GUI	указатель на GUI
0C	HANDLE OF OBJECT	указатель на OBJECT

4. Флаги операнда

Флаг	Комментарий
02	Инициализация параметра
04	Инициализация переменной
08	Объявление массива
40	Расширение размера переменной (когда размер > 255)
80	Переопределение (redefine)

Сноски

[1] [назад] Приведенное соответствие верно лишь для исполняемых программных модулей. Для неисполняемых соответствие будет немного другим.

Дзенский реверсинг: исследование программ в полевых условиях

Автор: GPcH

Дата: 2006-08-29

Как часто бывают ситуации, когда под рукой кроме HEX редактора/дизассемблера hiew ничего нет, а требуется срочно разблокировать какие-либо опции в программе или отвязать от железа? Причем обычно требуется срочно, на работе, когда горе-программист поставит защиту-привязку к компу, а нужно срочно перенести АРМ (автоматизированное рабочее место, не путать с ARM) с одного компьютера на другой. Конечно, можно пригласить автора АРМ'а и попросить его, но обычно на это банально нет времени и желания. В итоге приходится довольствоваться тем, что есть по рукой - обычно это только HIEW :)

Введение Все мы привыкли использовать довольно массивный комплект из различных утилит для исследования. Это и отладчики SoftICE и OllyDbg, дизассемблер IDA Pro, файловый анализатор PEiD и редактор PE Tools. Плюс еще целая гора специализированного и нужного софта для крэкинга. Но нужно ли реально столько инструментов? Для большинства несложных задач по взлому можно вполне обойтись HEX редактором и дизассемблером, причем не самым идеальным. Все дело в том, что многие вещи можно сделать и самому, не прибегая к мощности утилит. Здесь я расскажу, как исследовать софт только с помощью HIEW. Этот дизассемблер я выбрал не случайно - его всегда легко найти в локалке в любом крупном предприятии, хоть и старую версию, и он в своем составе имеет хороший HEX редактор, потому и пользуется спросом. Итак ты сидишь на работе, кроме компа и локалки под руками у тебя ничего нет и перед тобой стоит задача банально запустить неработающий софт на своем (или любом) компьютере. Давай рассмотрим, как выполнять различные повседневные реверсерские операции в этом замечательном дизасме. **Сам себе PE Sniffer**

Главным для реверсера всегда являлось локализовать компилятор/упаковщик, дабы знать, с чем он имеет дело и какой подход ему использовать. Не спорю, что найдя упаковщик нам как минимум потребуется его снять, и тут уже без автораспаковщика не обойтись. Но обычно в узкоспециализированном софте, который писался по заказу для предприятия не используют никакие упаковщики, так как клиент один и он всегда платит, а потому программист делает защиту только для того, чтобы клиент ее никому потом не перепродал и чтобы его услуги были востребованы и потом. Конечно идея хорошая, но порой самому клиенту приходится апгрейдить комп и тут приходится думать. Локализация компилятора будет не менее полезна, так как к примеру для Delphi и VB потребуются разные знания и подготовка :) Открываем программу в hiew и смотрим на гору ASCII символов. Что-нибудь понятно?



При запуске экран hiew ничуть не отличается от экрана FAR'a в режиме просмотра файла.

Нажимаем дважды Enter и смотрим на более понятный дизассемблированный код. Дабы он стал еще понятнее - нажмем F8 и F5 - hiew перейдет на тот участок кода, который прописан в оригинальной точке входа. Внимательно взглянув на этот код уже можно понять на чем он написан. К примеру для Delphi программ код будет подобен этому:

```
.0046D380: 55    push  ebp
.0046D381: 8BEC    mov  ebp,esp
.0046D383: 83C4F0   add  esp,-010 ;"?"
.0046D386: B8A0D14600   mov  eax,00046D1A0  -----? (1)
.0046D38B: E8DC94F9FF   call  .00040686C  -----? (2)
.0046D390: A1E8F84600   mov  eax,[0046F8E8]
.0046D395: 8B00    mov  eax,[eax]
.0046D397: E8A4E6FCFF   call  .00043BA40  -----? (3)
.0046D39C: E8B3EAFFFF   call  .00046BE54  -----? (4)
.0046D3A1: 8B0D28F94600   mov  ecx,[0046F928]
.0046D3A7: A1E8F84600   mov  eax,[0046F8E8]
.0046D3AC: 8B00    mov  eax,[eax]
.0046D3AE: 8B15F0B84600   mov  edx,[0046B8F0]
.0046D3B4: E89FE6FCFF   call  .00043BA58  -----? (5)
.0046D3B9: A1E8F84600   mov  eax,[0046F8E8]
.0046D3BE: 8B00    mov  eax,[eax]
.0046D3C0: 8B4044   mov  eax,[eax][44]
.0046D3C3: E834FBFFFF   call  .00046CEFC  -----? (6)
.0046D3C8: A1E8F84600   mov  eax,[0046F8E8]
```

Не узнать его нельзя... просто несколько Call вызовов, стандартных в Delphi программах. Если опять перейти в текстовый режим и нажать и подержать PgUp в самом верху увидим названия секций. Вот примерный расклад для нетронутых защитой Delphi программ:

- CODE
- DATA
- BSS
- idata
- tls
- rdata
- .reloc
- .rsrc

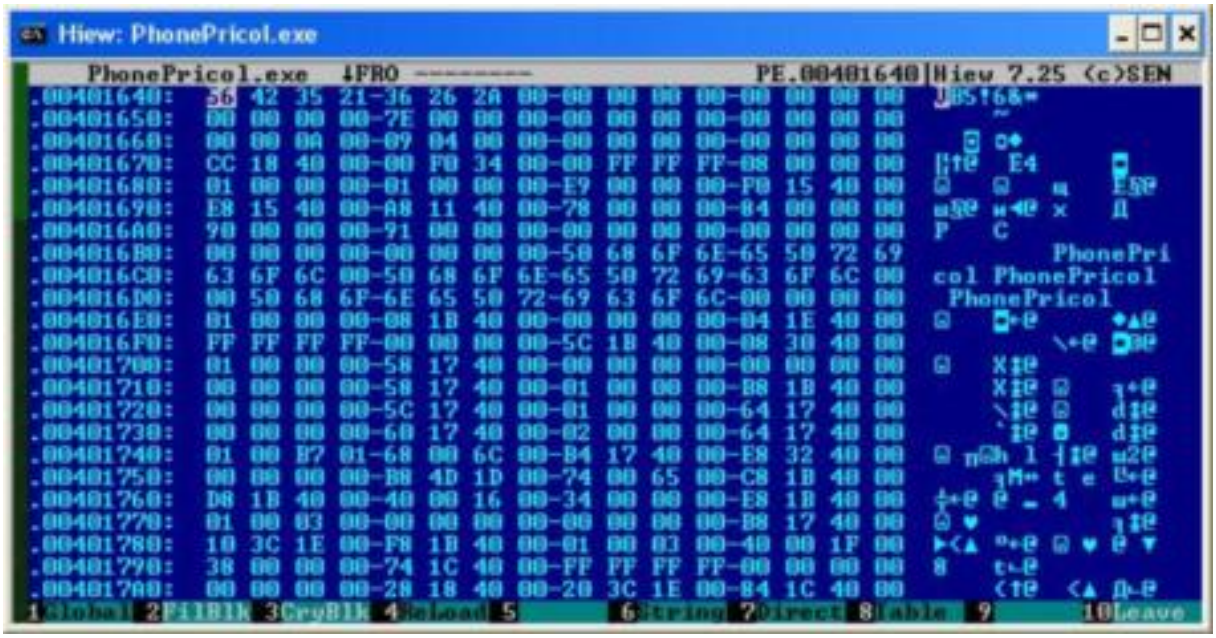
Итак с дельфями определились, теперь поговорим про C++ Builder. Тут вообще все проще. На оригинальной точке входа всегда такой код:

```
.00401000: EB10  jmps  .000401012
.00401002: 66623A bound  di,[edx]
.00401005: 43  inc  ebx
.00401006: 2B2B  sub  ebp,[ebx]
.00401008: 48  dec  eax
.00401009: 4F  dec  edi
.0040100A: 4F  dec  edi
.0040100B: 4B  dec  ebx
.0040100C: 90  nop
.0040100D: E978D54900 jmp  0206321AA
.00401012: A16BD54900 mov  eax,[0049D56B]
```

Байты расположенные между jmp'ом и mov'ом: "C++HOOK". Тоже все просто :) Что же насчет бейсика? А у него всегда 2 команды на EP:

```
.004011CC: 68EC164000  push  0004016EC  -----? (1)
.004011D1: E8EEFFFFFF  call  MSVBVM60.100
```

По адресу 0004016EC всегда VBHeader, начинающийся с сигнатуры "VB5!".



Начало структуры, описывающей любую VB программу

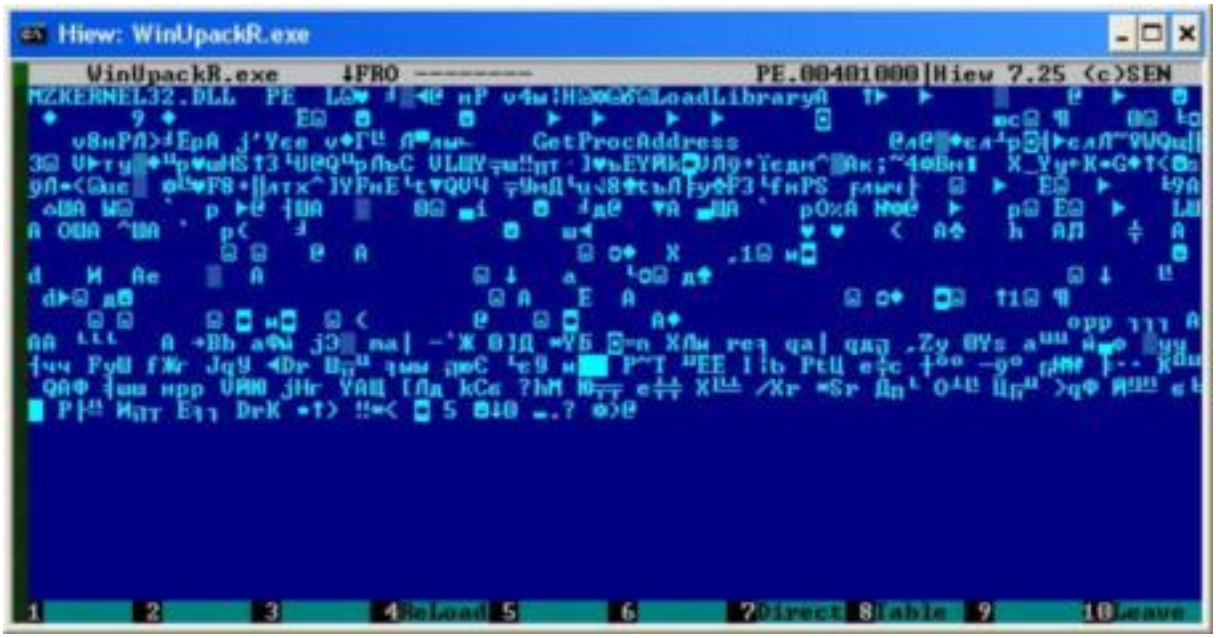
Как видно из предыдущих нескольких примеров - не так уж и требуется PEiD для определения компилятора. А что же про упаковщики? А здесь можно даже не смотреть на точку входа :) Достаточно в текстовом виде поглядеть на EXE заголовок. UPX после упаковки в начало первой секции помещает свою структуру, начинающуюся с сигнатуры "UPX!", а секции обзывает ".UPX0", ".UPX1", ".rsrc", причем секций три вне зависимости от того сколько их было до упаковки.

Если же взглянуть на точку входа, то код откроется довольно стандартный для всех версий UPX'a:

```
.00417180: 60  pushad
.00417181: BE00E04000 mov  esi,00040E000  -----? (1)
.00417186: 8DBE0030FFFF lea  edi,[esi][-0000D000]
.0041718C: 57  push  edi
.0041718D: 83CDFF or  ebp,-001 ;"?"
.00417190: EB10  jmps  .0004171A2  -----? (2)
.00417192: 90  nop
.00417193: 90  nop
.00417194: 90  nop
.00417195: 90  nop
.00417196: 90  nop
.00417197: 90  nop
```

```
.00417198: 8A06  mov  al,[esi]
.0041719A: 46  inc  esi
.0041719B: 8807  mov  [edi],al
```

Подробнее о самом алгоритме распаковки и восстановлении импорта можно прочитать в статье "Об упаковщиках в последний раз", которая лежит на данном сайте. Я же продолжу рассказ о детекте других упаковщиков. Следующим по распространенности является FSG и с недавних пор Upack. Их узнать очень легко, даже взглянув на текстовое представление EXE заголовка - он крайне оптимизирован, и обычно там нет стольких нулевых байт мусора и заполнения, сколько оставляют другие пакеры. А Upack даже запихивает импорт прямо в DOS Header после MZ :)



UPack жмет программу так, что от заголовка остается только MZ, после которого идет импорт. Жаль, антивирусы не разделяют эту идею оптимизации.

Все просто, только hiew :) Кстати, еще есть методика легко отличать MS компиляторы от борландовых по присутствию "Rich" строки после DOS Header'a. Хотя об патче MS линкера не раз говорилось на данном сайте - попрежнему большинство использует непропатченный вариант даже и не подозревая о том, что же линкер вставляет в EXE файл.

В общем, если держать в голове все эти методики плюс иметь неслабый опыт - можно определить любой компилятор или пакер в hiew'e, это приходит со временем, потому начинай сейчас!

Стринг референсы Лучшим помощником крэкера являются стринг референсы - перечень всех строковых данных, встречающихся в программе и адресов, где происходит обращение к этим строкам. Так вот в hiew'e это тоже есть! Достаточно в HEX режиме поставить курсор на начало любой строки и нажать F6 - сразу перейдем на первое обращение к этой строке! Вкупе с возможностью искать русские строки - хиев просто рулез. Жмем F7, пишем "Программа не зарегистрирована" и жмем Enter. Если нужно искать все места, где встречается строка, жмем Shift + F7 - хиев найдет следующий адрес где имеется строчка. Так как же он находит русские строки? Ведь они могут быть и в DOS, и в windows кодировке - все просто, нужные знания даем ему мы сами. Перед поиском необходимо в текстовом режиме нажать F8 и выбрать кодировку, именно используя ее хиев и будет искать строку. Универсальности конечно не получится, но это куда проще, чем любоваться только на английские строки в Olly :)**Серфинг по EXE**

Порой, когда смотришь дизассемблерные листинги, так и хочется перейти по адресу, на который указывает jmp, call или даже mov. Хиев и тут не подведет - каждому адресу на экране присваивается уникальная цифра, нажав которую на клавиатуре - мы перейдем куда надо, вот

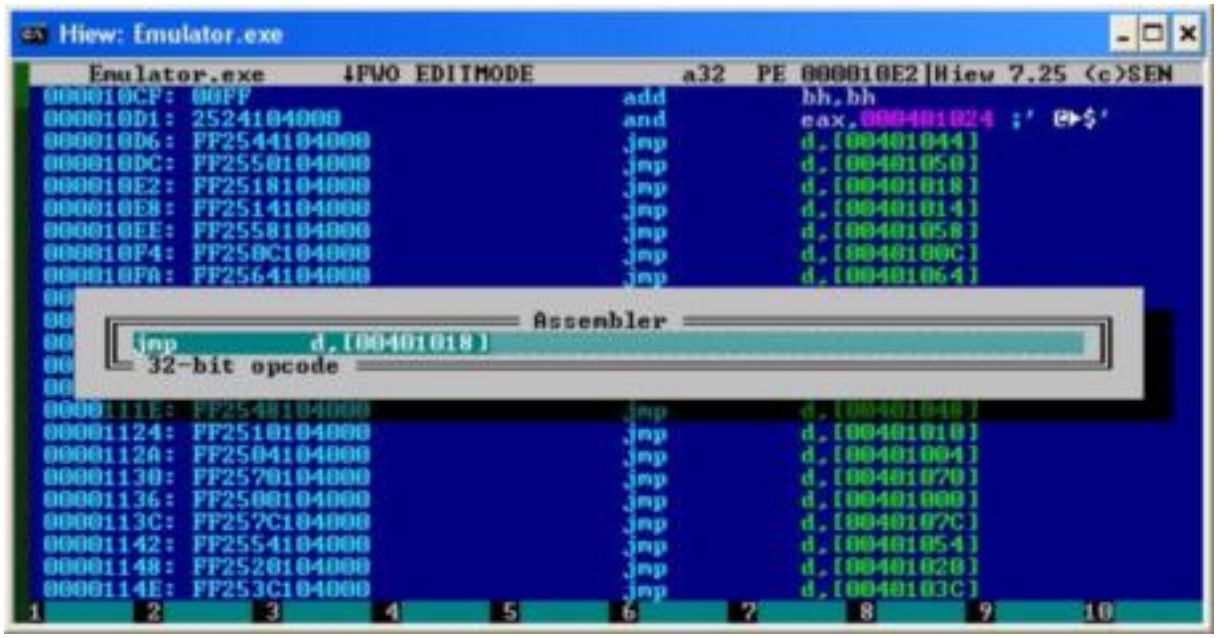
пример:

```
.00417180: 60  pushad
.00417181: BE00E04000 mov  esi,00040E000  -----? (1)
.00417186: 8DBE0030FFFF lea  edi,[esi][-0000D000]
.0041718C: 57  push  edi
.0041718D: 83CDFF or  ebp,-001 ;"?"
.00417190: EB10  jmps  .0004171A2  -----? (2)
```

(1) и (2) - это как раз эти самые цифры :) Дабы перейти по указанным адресам просто жмем на клавиатуре эти самые цифры (или буквы, если переходов больше 9). Вообще, цифровость задается в hiew7.ini, и ты можешь сделать, чтобы вместо цифер были хоть китайские иероглифы :-) Теперь остановимся на перемещениям по EXE вручную. F5 позволяет перейти на указанный адрес, причем, если мы просто введем адрес, то перейдем по Offset'y, а если перед адресом поставить точку - то хиев перейдет по виртуальному адресу. Очень удобно, причем даже в коде эти адреса можно переключать нажимая Alt + F1. Перейти к началу нужной секции также просто - зайдя в PE заголовок нажав F8, нажмем кнопку F6 и выбрав нужную секцию из списка перейдем в ее начальный адрес. А что, если нужно перейти в начало таблицы импорта или TLS? Искать их самим? Зачем? F8, затем F10 - и перейдем к таблице полей NTHeader'a. В нем просто выбирай нужное поле и нажимай Enter - переместишься мгновенно :-) Вот так вот старый добрый хиев помогает в нашем нелегком деле.

Ассемблируем

Исследовать мало - нужно править код, причем не все помнят машинные коды и не все могут ориентироваться в составлении mod/rm флагов и не все могут кодировать регистры в уме, потому править просто в HEX редакторе немного неудобный метод (хотя я, написав весь движок DotFix NiceProtect'a чисто в hiew (кроме исполнителя VM конечно), уже привык кодировать в уме, но эт так к слову). Хиев не оставит в беде, он представляет мощный ассемблер. Нажми F3, дабы править код, а затем дрожащим пальцем ткни в Tab - и о чудо! - в появившейся текстовой строке можно писать команды прямо на ассемблере, причем нажав Enter можно набирать уже следующую команду и так хоть весь код.



Пишем программу как настоящие ассемблерщики, безо всяких сред программирования.

Как видишь - не все так сложно, как казалось, когда ты начинал читать эту статью :) Если ты чего неверно понаписал - помни, практически любой введенный код можно отменить нажимая F3. Когда уже написал что надо - жми F9 для записи в файл. Вот незадача, после записи файл не запускается

- приходится выходить из hiew, чтобы снять залоченность. Но и это обходится. Открываем hiew7.ini, ищем строку "ReopenAfterEdit" и ставим ее в "On". Теперь после редактирования файл будет закрываться и снова открываться для чтения, поэтому его можно будет запустить после каждой правки :-) Спросил я у Евгения, зачем он так не сделал в дефолтных настройках и меня настолько порадовал его ответ - "чтобы люди читали хелп" :) Так что читай хелп и комментарии ко всем строкам hiew7.ini - столько нового узнаешь.

Декриптуем XOR С некоторых пор повелось, что каждый кодер, разобравшийся с PE форматом пишет свой криптор. Как не странно, явление сие постоянно процветает и появляется все больше новых крипторов, способных пошифровать EXE и якобы защитить его от взлома. Такие поделки используют в основном XOR для шифрации файла, который обратим, достаточно узнать пароль и пошифровать им уже шифрованный кусок кода, чтобы его расшифровать. Порой, анализируя прогу, можно узнать каким ключом дешифруется тот или иной блок. Но как же расшифровывать? В уме? Всю секцию кода? Не смешите, скажешь ты и пойдешь писать по-быстрому прогу на сях для решения данной задачи. Но зачем, если это все есть в хекс редакторе hiew? Хиев поистине рульная вещь - в ней есть и ксорилка. Достаточно в режиме редактирования нажить Ctrl + F8 и задать ключ, после чего просто нажимать F8 ксоря блок за блоком. Зажал F8 и держишь пока все не раскриптовалось! Теперь остается только запустить hiew и расксорить все, что надо. **Шифруем по формуле** Как бы не силен был XOR, он не всегда подходит для шифровки/дешифровки. Но грустить смысла нет, так как hiew позволяет задавать алгоритм шифровки самому. Для этого нажми F3, чтобы перейти в режим редактирования а затем F7. Отроется диалог набора кода. Забиваем туда алгоритм криптовки и затем юзаем. Как писать криптоалгоритм объяснено в справке - я не буду ее переписывать в статье, так как ее ты можешь прочитать и так, скажу лишь что ничего сложного там нет - разобраться можно. **Реальный пример**



Несложный кейгенми, который нам предстоит изучить.

Все вышеописанное будет иметь смысл только, если это применить на реальном примере, а именно, нам необходимо что-нибудь взломать. Ломать будем, используя только HIEW. А для большей простоты и наглядности возьмем KeygenMe by Fabsys. Стянуть его можно с crackmes.de. Начнем исследовать. Открываем keygen.exe в hiew. Что мы видим?

```
.0040820C: 55  push  ebp
.0040820D: 8BEC  mov  ebp,esp
.0040820F: 83C4F0 add  esp,0FFFFFFF0 ;'?'
.00408212: B8C4814000 mov  eax,0004081C4 --?1
.00408217: E8F0C2FFFF call  .00040450C --?2
.0040821C: 6A40  push  000000040 ;'@'
.0040821E: 684C824000 push  00040824C ;'Rules'
.00408223: 6854824000 push  000408254 ;'KeygenMe and de
.00408228: 6A00  push  0
.0040822A: E841C4FFFF call  MessageBoxA ;user32 --?5
.0040822F: 68047F4000 push  000407F04 --?6
```

По адресу 40822A красуется вызов нагскрина :) самое лучшее его пропатчить. Для этого устанавливаем курсор на 40821C и жмем F3, чтобы перейти в режим правки, затем Tab для вызова

окна ассемблера. В нем пишем "jmps .40822F" (без кавычек). Команда jmps в отличии от jmp сгенерит short jmp, который занимает всего 2 байта. Точка перед адресом ставится, потому что это VA, а не Offset. Запустим для проверки. Ура! нага как небывало. Теперь разберемся с проверкой пароля. Для этого я ввел имя "GPcH", а пароль "12345". При нажатии на кнопку "Generate" видим сообщение: "BaD BoY". Переходим обратно в hiew. В режиме дизассемблера жмем F8, затем F6 для вывода секций и выбираем первую секцию. Как только перейдем в ее начало - начнем поиск, для этого жми F7 и вводи "BaD BoY". Как только найдется жми F6, чтобы hiew перешел по адресу, откуда идет обращение к этой строке. Видим:

```
.00408067: 6848814000 push  000408148 ;'Prolixe KeygenM
.0040806C: 53  push  ebx
.0040806D: E81EC6FFFF call  SetWindowTextA ;user32 --?2
.00408072: 6A00  push  0
.00408074: 6874814000 push  000408174 ;'Winner'
.00408079: 687C814000 push  00040817C ;'GooD BoY'
.0040807E: 53  push  ebx
.0040807F: E8ECC5FFFF call  MessageBoxA ;user32 --?5
.00408084: EB20  jmps  .0004080A6 --?6
.00408086: 6A00  push  0
.00408088: 6888814000 push  000408188 ;'Wrong Way'
.0040808D: 6894814000 push  000408194 ;'BaD BoY'
.00408092: 53  push  ebx
.00408093: E8D8C5FFFF call  MessageBoxA ;user32 --?5
.00408098: EB0C  jmps  .0004080A6 --?6
```

Теперь логично узнать, откуда идет обращение к 408086 адресу, ведь чуть выше его вывод информации о том что пароль правильный, следовательно где-то есть переходи либо на вывод инфы о верном пароле, либо о неверном. Ставим курсор на адрес 408086 и жмем F6. Хиев почти мгновенно, как по велению волшебной палочки перенесет нас на строку, где идет обращение к адресу 408086:

```
.0040801E: E819B8FFFF  call  .00040383C --?1
.00408023: 8B85F8FDFFFF  mov  eax,[ebp][-00000208]
.00408029: 8D55FC  lea  edx,[ebp][-04]
.0040802C: E8E3FDFFFF  call  .000407E14 --?2
.00408031: 8D85F4FDFFFF  lea  eax,[ebp][-0000020C]
.00408037: 8D95FCFDFFFF  lea  edx,[ebp][-00000204]
.0040803D: B900010000  mov  ecx,000000100 ;'  ? '
.00408042: E8F5B7FFFF  call  .00040383C --?1
.00408047: 8B95F4FDFFFF  mov  edx,[ebp][-0000020C]
.0040804D: 8B45FC  mov  eax,[ebp][-04]
.00408050: E8EBB8FFFF  call  .000403940 --?3
.00408055: 752F   jne  .000408086 --?4 - мы здесь
.00408057: 6834814000  push  000408134 ;'Registred Versi
.0040805C: 68F1030000  push  0000003F1 ;'  ??'
.00408061: 53   push  ebx
.00408062: E811C6FFFF  call  SetDlgItemTextA ;user32 --?
```

Попробуем исправить переход jne на je по адресу 408055. Запустим кейгенми и введем любой пароль - программа будет зарегистрирована. HIEW просто рулез! Но не забывай - наша цель получить рабочий ключ! Сразу огорчу - хиев не слишком мощен, чтобы в нем реверсить алгоритм и писать кейген. Для того, чтобы сделать полноценный кейген нам потребуется отлаживать программу. Но нужно ли это? Может просто попытаться получить серийник на наше имя? Давай-ка попробуем! Потому поизучаем код, что идет перед jne. А на пару экранов выше идет:

```
.00407FCD: 80BDFCFDFFFF00 cmp  b,[ebp][-00000204],0
.00407FD4: 7517   jne  .000407FED --?1
.00407FD6: 6A40   push  000000040 ;'@'
.00407FD8: 680C814000  push  00040810C ;'Error'
.00407FDD: 6814814000  push  000408114 ;'Enter a Serial'
.00407FE2: 53   push  ebx
.00407FE3: E888C6FFFF  call  MessageBoxA ;user32 --?4
.00407FE8: E9B9000000  jmp  .0004080A6 --?5
.00407FED: 80BDFCFEFFFF00 cmp  b,[ebp][-00000104],0
.00407FF4: 7517   jne  .00040800D --?6
```

```
.00407FF6: 6A40   push  000000040 ;'@'
.00407FF8: 680C814000  push  00040810C ;'Error'
.00407FFD: 6824814000  push  000408124 ;'Enter a Name'
.00408002: 53   push  ebx
.00408003: E868C6FFFF  call  MessageBoxA ;user32 --?4
```

Сравниваем [ebp][-00000204] с нулем и если равно выводим, что неверный серийник. Логично предположить, что по адресу [ebp-204] лежит серийник. Смотрим ниже. Сравнивается с нулем уже [ebp-204] и выводится сообщение что неверное имя пользователя. Теперь ясно, что имя в [ebp-204]. Смотрим далее:

```
.0040800D: 8D85F8FDFFFF  lea  eax,[ebp][-00000208]
.00408013: 8D95FCFEFFFF  lea  edx,[ebp][-00000104]
.00408019: B900010000  mov  ecx,000000100 ;'  ? '
.0040801E: E819B8FFFF  call  .00040383C --?2
.00408023: 8B85F8FDFFFF  mov  eax,[ebp][-00000208]
.00408029: 8D55FC  lea  edx,[ebp][-04]
.0040802C: E8E3FDFFFF  call  .000407E14 --?3
.00408031: 8D85F4FDFFFF  lea  eax,[ebp][-0000020C]
.00408037: 8D95FCFDFFFF  lea  edx,[ebp][-00000204]
.0040803D: B900010000  mov  ecx,000000100 ;'  ? '
.00408042: E8F5B7FFFF  call  .00040383C --?2
.00408047: 8B95F4FDFFFF  mov  edx,[ebp][-0000020C]
.0040804D: 8B45FC  mov  eax,[ebp][-04]
.00408050: E8EBB8FFFF  call  .000403940 --?4
.00408055: 752F   jne  .000408086 --?5
```

Ясно, что без дебаггера нам не понять, какой Call выполняет какую функцию, но давай ка взглянем на каждый из них. Первый, похоже, просто переносит имя юзера в другую переменную. А вот второй криптует ее. Пролистаем код по адресу 407E14 и, о чудо:

```
.00407E99: 68EC7E4000  push  000407EEC ;'HZF-'
.00407E9E: FF75F4  push  d,[ebp][-0C]
.00407EA1: 68FC7E4000  push  000407EFC ;'-GFD'
```

Без дебаггера понять этот код можно только логически. Как мне кажется, пароль может складываться как: 'HZF-' + [ebp][-0C] + '-GFD'. Из листингов выше мы знаем адрес MessageBoxA в IAT. Так давай вызовем мессагу и в параметрах укажем [ebp][-0C]. Прямо после

```
push  000407EFC ;'-GFD'
```

жми F3 и набирай. У меня получилось следующее:

```
.00407E99: 68EC7E4000 push  000407EEC ;'HZF-'
.00407E9E: FF75F4 push  d,[ebp][-0C]
.00407EA1: 68FC7E4000 push  000407EFC ;'-GFD'
.00407EA6: 8B45F4 mov  eax,[ebp][-0C]
.00407EA9: 6A00  push  0
.00407EAB: 50  push  eax
.00407EAC: 50  push  eax
.00407EAD: 6A00  push  0
.00407EAF: E8BCC7FFFF call  MessageBoxA ;user32 --?6
.00407EB4: 90  nop
.00407EB5: 5A  pop  edx
.00407EB6: 59  pop  ecx
.00407EB7: 59  pop  ecx
.00407EB8: 648910 mov  fs:[eax],edx
```

Теперь при вводе неверного серийника выводится середина верного. Добавим к ней слева 'HZF-' и справа '-GFD' и получим верный код. Только автор не позаботился о том, чтобы код всегда имел печатаемые символы, потому не на каждое имя будет ключ из нормальных печатаемых символов. но это уже не наши проблемы - цель получения пароля достигнута.

Перспективы развития hiew

В заключении статьи я решил немного познакомить читателя с тем, что нас ждет в будущих релизах программы. Главное, что дейстительно планирует автор - это поддержка AMD64. Эта линейка

процессоров уже поистине стала народной и получила широкое распространение в мире. Даже Майкрософт в Win64, насколько мне известно, делает основной упор на процессоры AMD. Отсюда в последнее время стало появляться все больше программ и даже драйверов, скомпилированных в формате AMD64, отсюда ассемблер и дизассемблер hiew'a полюбому необходимо расширять. Про IA64 пока планов никаких, но тут уж хотя бы AMD64 реализовать. Насчет .NET и ARM Sen обещал подумать, но это больше на перспективу, так как разбираться с абсолютно новым ассемблером и псевдокодом не так просто. Помимо того что уже сказал, автором дизассемблера планируется публикация полного Plugin SDK, чтобы каждый мог разработать плагин для расширения функционала HIEW'a. К примеру, может кто напишет скриптовый язык для автоматизации действий в HIEW или скажем анализатор компилятора/упаковщика. В общем если SDK появится на сцене, за плагинами, думаю, не заржавеет. Что касается GUI, то его скорее всего не будет, так как консольный инструмент гораздо удобнее виндового. Это уже доказано FAR'ом. Так что перспектив много - осталось ждать :) Надеюсь, Евгений не подкачает.

Последнюю версию hiew всегда можно найти на сайте http://webhost.kemtel.ru/~sen.

Декомпилируем p-code в уме

Автор: GPcH

Дата: 2006-09-08

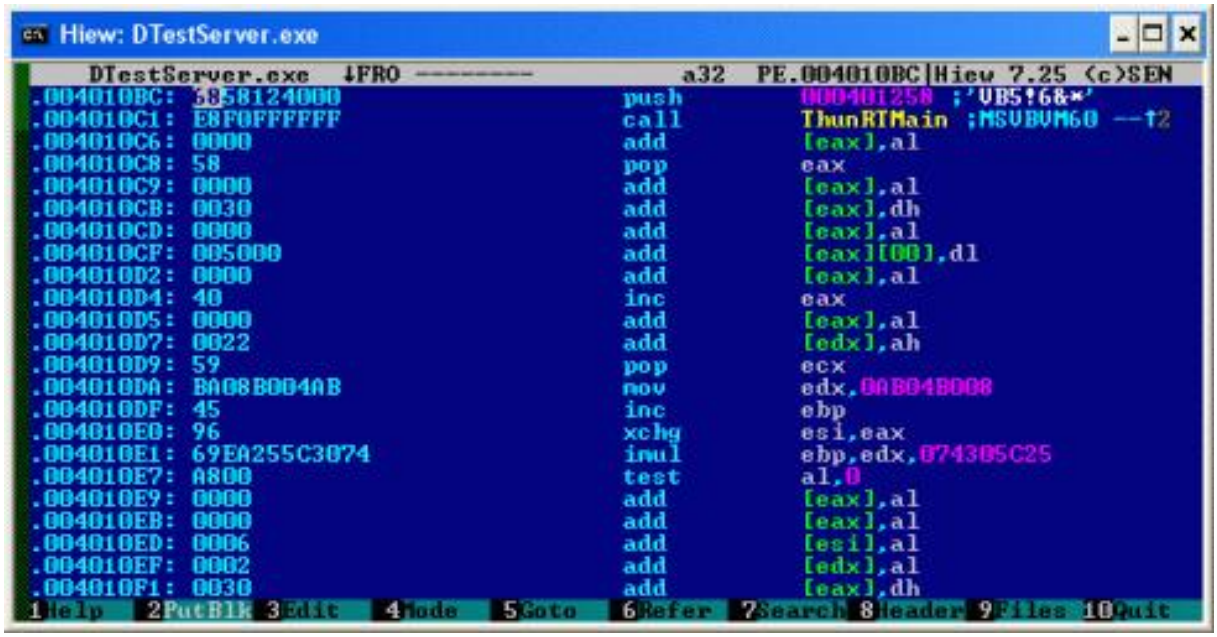
Введение

Думаю, что все мы знакомы с программами на пикоде. Даже если реверсер в жизни не видел VB и его компилятор, то все равно хотя бы раз он сталкивался с пикодом. В отличии от стандартного машинного кода, исполняемого напрямую процессором, p-code это набор мнемоник VM, которые исполняются движком msvbvmXX.dll. Olly тут не особый помощник (хотя для кого как), IDA тем более. Тут нужен либо декомпилятор, либо мозги. Надеюсь, что второе есть точно у всех, кто читает эти строки. Именно поэтому листинги из декомпилятора я буду приводить в статье лишь для наглядности, основной же упор будет на декомпилирование, используя в работе только HIEW.

Немного о структурах

Пикод нам еще придется поискать в EXE файле - он так просто не лежит. Для этого придется разобраться хотя бы с частью структур VB, которые зашиты в EXE файл. Начнем с рассмотрения оригинальной точки входа в программу. Чтобы на нее перейти из HEX редактора HIEW нам потребуется лишь загрузить EXE'шник в данный HEX редактор и нажать поочереди Enter, Enter, F8, F5. У кого хиев купленный знают как оптимизировать эту операцию до командной строки, но это так к слову. Нашему взгляду будет представлено примерно следующее:

```
push 0004042E8 ;'VB5!'
call ThunRTMain ;MSVBVM60 --?2
```



P-Code Inside :)

Весь смысл этой мегасложной строчки заключается в вызове функции ThunRTMain из MSVBVM60. В параметрах этой функции передается указатель на VB Header. Вот его примерный вид:

Структура VBHeader

Поле — Тип — Описание

Signature — String * 4 — Сигнатура "VB5!"

RuntimeBuild — Integer — Показатель рантаймовости

LanguageDLL — String * 14 — Языковая библиотека

BackupLanguageDLL — String * 14 — Не влияет на работу EXE

RuntimeDLLVersion — Integer — Версия рантайм библиотеки

LanguageID — Long — Язык программы

BackupLanguageID — Long — Используется совместно с LanguageDLL

aSubMain — Long — Main процедура, запускаемая при старте EXE. Если отсутствует, то при загрузке грузится самая первая форма

aProjectInfo — Long — Указатель на структуру ProjectInfo

fMDLIntObjs — Long

fMDLIntObjs2 — Long

ThreadFlags — Long — Флаги потока

ThreadCount — Long — Число потоков (смысл малопонятен, так как VB не позволяет создавать многопоточные программы)

FormCount — Integer — Число форм в данном файле

ExternalComponentCount — Integer — Число внешних OCX компонентов

ThunkCount — Long

aGUITable — Long — Указатель на GUITable

aExternalComponentTable — Long — Указатель на ExternalComponentTable

aComRegisterData — Long — Указатель на ComRegisterData

oProjectExename — Long — Адрес строки с именем EXE файла

oProjectTitle — Long — Адрес строки с заголовком проекта

oHelpFile — Long — Адрес строки с именем Help файла

oProjectName — Long — Адрес строки с именем проекта

ThunRTMain из описанной структуры получает все необходимые данные и адреса структур, необходимые для настройки и запуска EXE. Функция ProcCallEngine, в случае присутствия поля aSubMain, запускает функцию, расположенную по адресу, указанному в этом поле структуры, которая является главной в проекте и не равна 0, если в EXE файле присутствовал модуль с функцией Main. Если данная функция отсутствует, то грузится первая форма и управление передается функции Form_Initialize, а если она отсутствует то Form_Load. Соотвественно, если и этой функции нет, и вообще в форме события не используются, то запускается WindowProc - цикл ожидания событий, как в любой программе (ассемблерщики меня поймут). Нам из этой структуры потребуются лишь aSubMain и структура ProjectInfo. Последняя имеет вид:

Структура ProjectInfo

lTemplateVersion — Long — Версия VB совместимости

aObjectTable — Long — Указатель на aObjectTable

lNull1 — Long

aStartOfCode — Long — Начало кода (нам бесполезно)

aEndOfCode — Long — Конец кода (нам бесполезно)

lDataBufferSize — Long — Размер буфера для хранения различных данных

aThreadSpace — Long — Пространство потока

aVBAExceptionhandler — Long — Указатель на функцию обработки ошибок

aNativeCode — Long — Если не равно нулю, то это Native Code, иначе P-Code

uIncludeID — 527 Byte

aExternalTable — Long — Указатель на таблицу API функций

lExternalCount — Long — Число импортируемых API функций

Обозначенная таблица для нас как распутье. Продолжать далее имеет смысл только, если в aNativeCode стоит 0. Это означает, что все функции в программе на пикоде и имеет смысл с ними разобраться. Все полезные нам данные мы получим из aObjectTable - это указатель на структуру объектов (форм модулей и так далее), среди прочих данных в которой можно найти адреса процедур и событий.

Структура ObjectTable

lNull1 — Long

aExecProj — Long

aProjectInfo2 — Long

lConst1 — Long

lNull2 — Long

aProjectObject — Long

uuidObjectTable — Long

Flag2 — Long

Flag3 — Long

Flag4 — Long

fCompileType — Integer

iObjectsCount — Integer — Число объектов

iCompiledObjects — Integer

iObjectsInUse — Integer

aObjectsArray — Long — Указатель на массив объектов (единственное что для нас в этой структуре важно помимо iObjectsCount)

lNull3 — Long

lNull4 — Long

lNull5 — Long

aNTSProjectName — Long

lLcID1 — Long

lLcID2 — Long

lNull6 — Long

lTemplateVersion — Long

Теперь, когда мы определились с таблицей объектов, давай посмотрим поподробнее на массив этих самых объектов. ObjectsArray - это массив структур TObject. Число элементов массива

определяется полем iObjectsCount. Рассмотрим-ка поподробнее структуру TObject и производную от нее TObjectInfo.

Структура TObject

aObjectInfo — Long — Указатель на структуру ObjectInfo

lConst1 — Long

aPublicBytes — Long — Указатель на массив публичных переменных

aStaticBytes — Long — Указатель на массив статических переменных

aModulePublic — Long — Указатель на публичные переменных

aModuleStatic — Long — Указатель на статические переменные

aNTSObjectName — Long — Указатель на имя объекта

lMethodCount — Long — Число методов объекта

aMethodNameTable — Long — Указатель на массив адресов функций

oStaticVars — Long — Смещение на переменные из aModuleStatic

lObjectType — Long — Тип объекта

lNull2 — Long

Как видишь эта структура уже поинтереснее. Поле aNTSObjectName означает, что это собственно за модуль. Если это поле содержит адрес на frmRegister, то это уже может нам намекнуть что функции, находящиеся в массиве из следующей структуры отвечают за процесс проверки серийника :)

Структура TObjectInfo

iConst1 — Integer

iObjectIndex — Integer — Индекс объекта

aObjectTable — Long — Указатель на таблицу объектов (требуется для разбора форм и лежащих на них объектов... нам же в данной статье не потребуется)

lNull1 — Long

aObjectDescriptor — Long

lConst2 — Long

lNull2 — Long

aObjectHeader — Long — Указатель на ObjectHeader

aObjectData — Long — Указатель на ObjectData

следующие члены валидны только если программа скомпилирована в пикод

iMethodCount — Integer — Число методов

iNull3 — Integer

aMethodTable — Long — Указатель на массив указателей на методы

iConstantsCount — Integer — Число констант

iMaxConstants — Integer — Максимальновозможное число констант

lNull4 — Long

lFlag1 — Long

aConstantTable — Long — Указатель на массив указателей на константы

Если не брать во внимание события объектов на форме, а только созданные пользователем функции, то для все их адреса можно найти в таблице, на которую указывает aMethodTable. Эта таблица для каждого модуля или формы конечно своя, это логично. Потому, чтобы получить доступ ко всем функциям, необходимо перебрать все формы и модули (а также классмодули, юзерконтролы...). В общем, если у тебя много свободного времени, можешь уже сейчас писать плагин для HIEW, который будет это все автоматизировать, я же лишь рассмотрю что же все-таки находится по адресам из таблицы методов. А находится там собственно не пикод, и не ассемблерные команды, как многие могут подумать. Там очередная структура, коими любит нас потчевать микрософт.

Структура ProcDscInfo

ProcTable — Long

field_4 — Integer

FrameSize — Integer

ProcSize — Integer

...

Я преднамеренно урезал эту структуру, так как нам из нее потребуется только ProcSize и адрес на таблицу ProcTable. Из таблицы ProcTable нам потребуется в свою очередь только адрес на блок данных. Об этом блоке данных думаю надо поговорить поподробнее. Все опкоды пикода, которые так или иначе оперируют с данными (будь то строки, API функции и так далее) ссылаются на них не по абсолютному, а по относительному адресу. Относительный этот адрес именно от начала блока данных. Так что, не зная адреса на блок данных, мы по сути не сможем ничего декомпилировать. Отсюда рассмотрим следующую структуру, расположенную по адресу ProcTable:

Структура ProcTable

SomeTemp — String*52

DataConst — Long

SomeTemp - это просто набор ненужных нам полей, которые я объединил дабы немного сэкономить места в статье. Они нам не потребуются, для нас важен только адрес DataConst. Теперь думаю логично задаться вопросом, а что мы узнали? Только размер пикодовой процедуры и адрес на блок данных. А где же сама пикодовая процедура? Ааа, вот тут билли гатес засюрпризил нам еще одну подлянку. Адрес пикодовой процедуры напрямую нигде не прописан, зато мы его можем получить вычитанием из адреса начала структуры ProcDscInfo поля sProcDscInfo.ProcSize. Как видно, пикод идет прямо перед структурой ProcDscInfo. Что-ж, теперь, когда мы знаем откуда брать методы (про ивенты я умолчу всвязи в ограничением объема статьи), можно приступать исследовать этот самый пикод.

Как исследовать

Узнали мы адрес начала блока пикода, а что дальше? А дальше, что бы хоть как-то понять пикод, нам потребуется таблица назначения опкодов. Ее можно с легкостью найти в гугле. Но для простоты я ее разместил на диске, так что открывай его и начиный изучать :) Заранее предупрежу - таблица эта неполная, так как публичная и имеет кое какие ошибки (их крайне мало), потому доверять ей на все 100 не советую, хотя это всетаки лучшее, что можно добыть в инете на данный момент. Таблица представлена в виде: <опкод>tab<размер>tab<название> Опкод может состоять из одного или двух байт. До FBh опкода невключительно идут однобайтовые. Далее идут двухбайтовые. Так что, если ты встретишь к примеру FEh, то следующий за ним байт также относится к опкоду. <размер> в таблице это число параметров - 1. Из одного ли байта состоит

опкод или из двух, все равно -1 :) Не знаю , зачем так сделали составители таблицы, главное, нам это придется учитывать при разборе кода (само собой я сделал для себя иную таблицу, гораздо более продвинутую, но по коммерческим соображениям публиковать ее не буду). Теперь посмотрим на пример пикодового блока:

```
00 00 00 00-00 00 00 00-00 00 00 00-F4 00 2B 6E
FF F5 00 00-00 00 F5 00-00 00 00 1B-0C 00 04 70
FF 34 6C 70-FF 1B 0D 00-04 74 FF 34-6C 74 FF F5
00 00 00 00-59 78 FF 0A-0E 00 18 00-3C 32 04 00
74 FF 70 FF-13 00 00 00
```

Пикод в нашем блоке начинается с F4. Посмотрим по таблице, чтоже означает этот опкод. Согласно таблице это LitI2_Byte и имеет один байт в параметрах. Запомни раз и навсегда, Lit это всегда push. I2 - это 2х байтный integer. Вот небольшая табличка всех возможных значений, дабы было дальше проще исследовать:

Типы данных

UI1 — Byte

Bool — Boolean

I2 — Integer

I4 — Long

R4 — Single

R8 — Double

Cy — Currency

Var — Variant

Str — String

Так как после F4 идет 0, то логично предположить, что эта команда push 0. Продолжим далее. 2B - это PopTmpLdAd2. Читать следует так: Pop from stack to Temp variable and Load Address to stack. Пишу по английски, так как на русском это будет малопонятно. Итак, эта переменная всего лишь резервирует содержимое верхней ячейки стека во временную переменную - нам это не потребуется, так как мы не исполняем код а декомпилируем. Но для понятности расскажу. Следующие 2 байта - это та самая временная переменная FF6E. Если это число из откричательного перевести в положительное, то получится 92. Мой декомилятор в этом случае выведет var_92. Если бы число было положительным, то это была бы не локальная а глобальная переменная. Думаю теперь все ясно :) приступим к следующей команде. F5 - LitI4. Понимаем как push следующие 4 байта, то есть push 0 :) Далее опять идет push 0. Теперь 1B - LitStr. Этот опкод заносит строку в стэк. Не зря я тебе рассказал про адрес на блок данных, именно от него и отсчитываются следующие два байта в параметрах, то есть: адрес строки = ProcTable.DataConst + следующие за командой 2 байта Строка представлена в юникоде и заканчивается двумя нулями. Допустим, что там строка "Test", следовательно имеем push "Test". 04 - FLdRfVar, то есть push переменная. Переменная вычисляется как и раньше FF70 = var_90. 34 - CStr2Ansi. Считывает из стека два элемента и первому присваивает второй, то есть в стэке на данный момент переменная var_90 и строка "Test", следовательно, эту команду можно читать как var_90 = "Test". У этого опкода параметров нет, следовательно, далее идет уже другой опкод 6C - ILdRf. Понимается он как push переменная. То есть следующие два байта FF70 декомпятся как var_90 и вставляются в команду как push var_90. 1B снова заносит уже другую строку в стэк, пусть это будет "Test2", 04 опять создает переменную и заносит ее в стэк, а 34 присваивает "Test2" этой переменной. В частности, FF74 это var_8C. 6C заносит эту переменную в стэк по накатанной стандартной схеме. Затем идет F5, про него мы уже говорили - он заносит в стэк следующие 4 байта. Далее уже поинтереснее: 59 - PopTmpLdAdStr. Резервирует в переменной var_88 содержимое верхней ячейки стэка, при этом

занося заново это содержимое на вершину стэка дабы оно не терялось. 0A - ImpAdCallFPR4. Вот тут уже запахло чем то вкусным... мы видим чистой воды Call на функцию, адрес которой отсчитывается от блока данных по традиции. Пока не забыл - это два байта, прежде чем суммироваться с адресом блока данных, умножаются на 4 - и это правило действует всегда. Зачем так сделали создатели VM - я не знаю, но это надо учитывать. Итак что же мы имеем в блоке данных? А имеем мы VA на следующую заглушку:

```
A1A0634000                  mov         eax,[004063A0]
0BC0                        or          eax,eax
7402                        je         .000402A77
FFE0                        jmp         eax
68542A4000                  push        000402A54
B870104000                  mov         eax,000401070
FFD0                        call        eax
```

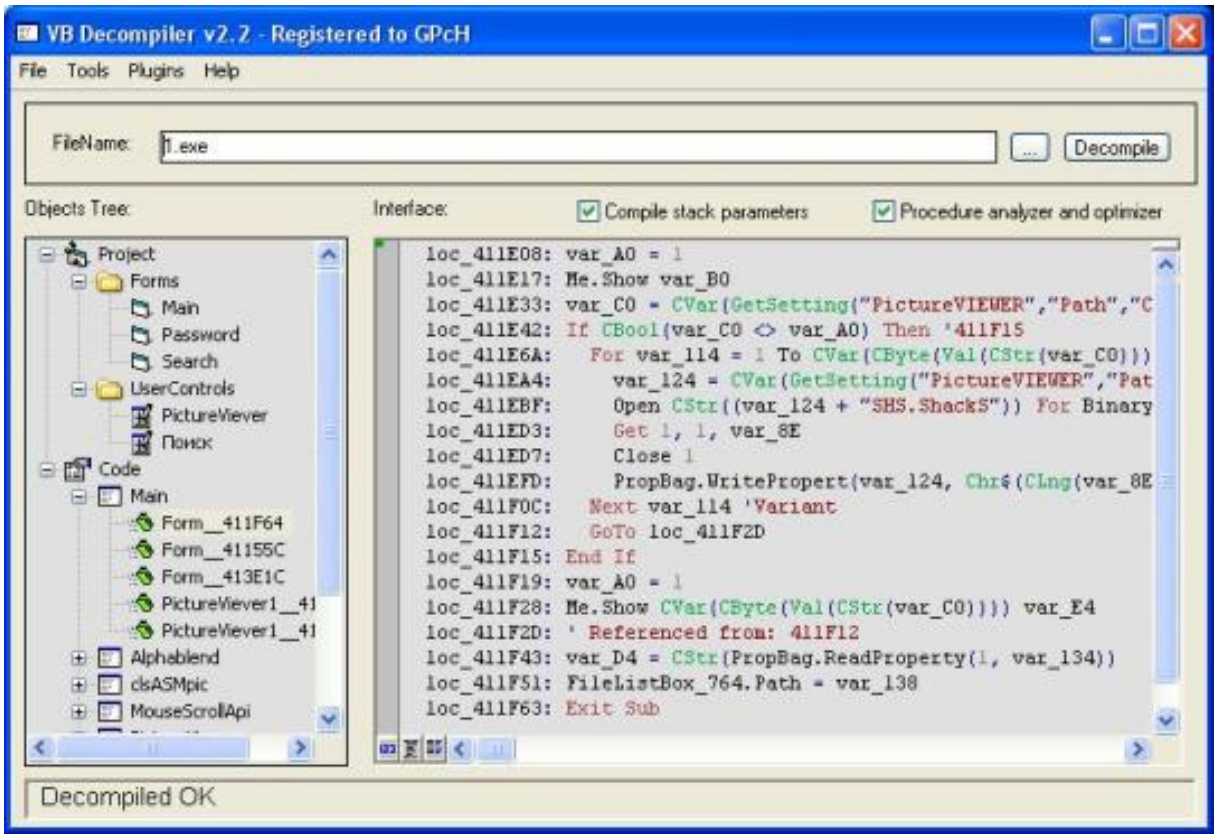
В этой конструкции нам важен push 402A54. Виртуальный адрес 402A54 указывает на структуру вида:

Структура CallAPI

strLibraryName — Long

strFunctionName — Long

Эти два виртуальных адреса указывают на имя Dll и имя функции, которая должна быть вызвана. Вот разработчики намудрили с виртуальной машиной, правда?



Разбор переходников на API

Теперь то мы знаем, почему VB так тормозит. Чтобы вызвать любую API функцию необходимо пройти немыслимое количество структур и заглушек, что тратит золотые такты процессора и жутко снижает скорость. Как видишь - никакой таблицы импорта нет. Мы имеем дело с именем Dll и функции, которые вызываются динамически функцией DllFunctionCall, которая экспортируется из всеми ними замусоленной msvbvm60.dll. Поэтому строки из блока, что был указан выше:

```
B870104000                  mov         eax,000401070 --?3
FFD0                        call        eax
```

именно, получают адрес на переходник:

```
jmp DllFunctionCall
```

и вызывают его через call eax. Что находится в функции DllFunctionCall думаю и так понятно:

```
hLib = GetModuleHandle(strLibraryName)
hProc = GetProcAddress(hLib, strFunctionName)
Call hProc
```

Это в упрощенном виде, для наглядности :) В нашем же случае strLibraryName это user32.dll, а strFunctionName - это ShellExecute. Теперь то думаю ясно, зачем в пикоде было столько push'e и резервирования. Это всего лишь подготовка стэка для вызова ShellExecute. Поэтому, если свернуть все полученные нами данные, то выйдет что-то вроде:

```
loc_4043F1: var_90 = "Test"
loc_4043FB: var_8C = "Test2"
loc_404407: ShellExecute(0, var_8C, var_90, 0, 0, 0)
```

Ну что, продолжим исследовать пикод. 3C - SetLastSystemError. Тут все просто - это всеголишь переходник на API. Нам он не потребуется, так как он юзается виртульной машиной для отслеживания ошибок и не дает нам никаких важных данных для анализа пикода. 32 - FFreeStr. Очень интересная команда. Число ее параметров всегда переменно и определяется первыми двумя байтами. В данном случае первые два байта - 4, следовательно следующие 4 байта - две двухбайтовые переменные. В частности FF74 и FF70. Команда FFreeStr обнуляет эти строковые переменные: var_8C и var_90. То есть на нормальном языке это выглядит так:

```
var_8C = ""
var_90 = ""
```

Следующий и последний опкод 13 - ExitProcHresult. Как понятно из названия, он завершает процедуру. Это значит, что после него идет уже известная нам структура ProcDscInfo. Вот мы и разобрались, как декомпилировать пикод в уме. Теперь пора поговорить о реальных задачах по исследованию, для которых исследование в уме слишком долгое и геморное, потому исследовать будем в моем декомпиляторе.

Срубаем NAG скрины



Злостные наги

В любой программе есть две проблемы - триал и нагскрины. Если триал обычно можно обойти лишь найдя ключ в реестре, который отвечает за счетчик дней и написав простенький инлайн патч, который будет перед запуском EXE обнулять этот ключ, то с нагами такой номер не пройдет. Они надоедают либо до покупки проги, либо до отрубания этих злостных трюков :) Думаю, лучшим примером нагскрина в виде мессаги будет простенький крякми на пикоде. Не уверен, что ты найдешь его в интернете, потому бери с диска, я его назвал "AC_Crackme_01.exe". Откроем-ка мы его в декомпиляторе, дабы посмотреть с чем имеем дело. Видим две процедуры, одна закрывает приложение по кнопке, а вторая, та что Form_Load:

```
loc_401A48: LitVar_Missing var_104
loc_401A4B: LitVar_Missing var_E4
loc_401A4E: LitVar_Missing var_C4
loc_401A51: LitI4 0
loc_401A56: LitVarStr var_94, "NAG"
loc_401A5B: FStVarCopyObj var_A4
loc_401A5E: FLdRfVar var_A4
loc_401A61: ImpAdCallFPR4 MsgBox(, , , , )
loc_401A66: FFreeVar var_A4 = "": var_C4 = "": var_E4 = ""
loc_401A71: ExitProcHresult
```

Видим вызов MsgBox по адресу 401A61. Как ты понимаешь, нам его нужно убрать. Тут есть несколько путей. Так как в данном случае вся функция это NAG, то можно просто сделать выход в самом начале функции, то есть по адресу loc_401A48 вбить опкод ExitProcHresult. Красиво, оригинально и работает :). Посмотрим теперь крякми посложнее "AC_Crackme_01_A.exe":

```
loc_401DE8: LitVar_Missing var_104
loc_401DEB: LitVar_Missing var_E4
loc_401DEE: LitVar_Missing var_C4
loc_401DF1: LitI4 0
loc_401DF6: LitVarStr var_94, "Another NAG"
loc_401DFB: FStVarCopyObj var_A4
loc_401DFE: FLdRfVar var_A4
loc_401E01: ImpAdCallFPR4 MsgBox(, , , , )
loc_401E06: FFreeVar var_A4 = "": var_C4 = "": var_E4 = ""
loc_401E11: FLdPr arg_8
loc_401E14: Me.Hide
loc_401E19: LitVar_Missing var_B4
loc_401E1C: PopAdLdVar
loc_401E1D: LitVar_Missing var_94
loc_401E20: PopAdLdVar
loc_401E21: ImpAdLdRf unk_4019F4
loc_401E24: NewIfNullPr
loc_401E27: Me.Show from_stack_1 from_stack_2
loc_401E2C: ExitProcHresult
loc_401E2D: ILdPr
```

Видим, что по адресу 401E01 выводится один наг с помощью MsgBox, затем 401E14 скрывается форма и 401E27 происходит загрузка основного окна. Здесь я рассмотрю сначала как можно обойти мессагу, а уж потом поговорим про форму. Как же обойти этот вызов MsgBox? А как бы мы его обошли в программировании? Наверное логичным по адресу 401DE8 прописать GoTo на адрес 401E11. Это мы и сделаем. В качестве GoTo используется BranchF а адрес рассчитывается относительно адреса начала данной функции. То есть начальный адрес 401DE8, а нам нужно перейти на 401E11, следовательно, 401E11h-401DE8h=29h, а значит, в начало функции надо приписать 1E2900 :) Запускаем - нага как не бывало, осталось убрать форму нага - но это будет домашним заданием - думаю ты спрашишься без проблем.

Ломаем запрос пароля

Хорошим примером на эту тему будет крякми "AC_Crackme_02_A.exe". В нем нужно ввести верный серийник. Если серийник неверный, то выводится "fuxxor". Давай посмотрим его в декомпиляторе:

```
loc_401D4C: FLdRfVar var_94
```

```
loc_401D4F: FLdPrThis
loc_401D50: VCallAd Crackme.Frame1
loc_401D53: FStAdFunc var_90
loc_401D56: FLdPr var_90
loc_401D59: from_stack_1 = TextBox.Text
loc_401D5E: FLdZeroAd var_94
loc_401D61: FStStr var_8C
loc_401D64: FFree1Ad var_90
loc_401D67: LitStr "ExDec_Roxx"
loc_401D6A: FStStrCopy var_88
loc_401D6D: ILdRf var_88
loc_401D70: ILdRf var_8C
loc_401D73: EqStr
loc_401D75: BranchF loc_401DA4
loc_401D78: LitVar_Missing var_114
loc_401D7B: LitVar_Missing var_F4
loc_401D7E: LitVar_Missing var_D4
loc_401D81: LitI4 0
loc_401D86: LitVarStr var_A4, "yess"
loc_401D8B: FStVarCopyObj var_B4
loc_401D8E: FLdRfVar var_B4
loc_401D91: ImpAdCallFPR4 MsgBox(, , , , )
loc_401D96: FFreeVar var_B4 = "": var_D4 = "": var_F4 = ""
loc_401DA1: Branch loc_401DCD
loc_401DA4: ' Referenced from: 401D75
loc_401DA4: LitVar_Missing var_114
loc_401DA7: LitVar_Missing var_F4
loc_401DAA: LitVar_Missing var_D4
loc_401DAD: LitI4 0
loc_401DB2: LitVarStr var_A4, "fuxxor"
loc_401DB7: FStVarCopyObj var_B4
loc_401DBA: FLdRfVar var_B4
loc_401DBD: ImpAdCallFPR4 MsgBox(, , , , )
loc_401DC2: FFreeVar var_B4 = "": var_D4 = "": var_F4 = ""
loc_401DCD: ' Referenced from: 401DA1
loc_401DCD: ExitProcHresult
loc_401DCE: ImpAdLdFPR4
```

Просмотрев внимательно код видим сравнение в самом начале

```
loc_401D67: LitStr "ExDec_Roxx"
loc_401D6A: FStStrCopy var_88
loc_401D6D: ILdRf var_88
loc_401D70: ILdRf var_8C
loc_401D73: EqStr
loc_401D75: BranchF loc_401DA4
```

Понимать этот код следует примерно так:

```
var_88="ExDec_Roxx"
if (var_88=var_8C) then
```

Затем видим мессагу, что пасс верный. Собственно пасс-то мы узнали, да вот нам-то крякнуть надобы, чтобы подходил любой пароль неверный :) BranchF по адресу 401D75 джампается на вывод инфы о неверном пароле, следовательно он понимается, как jne, то есть если EqStr возвращает что строки различны, то джампаемся. Вывод - инвертировать инструкцию. Следует понимать, что в VB всего три типа переходов, два из которых условные - 1C и 1D. ТО есть для инвертирования нам надо 1C заменить на 1D или наоборот. Меняем и, о чудо, - мессаги больше нет.

Более сложные приемы

Ну вот, крякам мы научились, пора бы попробовать себя в реально сложном деле - написать кейген. В качестве примера возьмем кейгенми "KeygenMe.exe". Тут нам больше потребуется Pro версия декомпилятора, так как для анализа кода лучше смотреть на подобие исходникак, нежеле на сотни строк пикодовых опкодов. Потребуется движок сворачивания команд и стэка, который прекрасно реализован в VB Decompiler Pro. Посмотрим на кейгенми под декомпилятором:

```
loc_405825: var_88 = TextBox_764.Text
loc_40583F: var_90 = TextBox_768.Text
loc_40585B: If (var_94 Or (var_94 = "")) Then '405892
loc_40587F:   MsgBox("Incorrect username or password!", 16, "Keygenme", var_F4,
              var_114)
loc_40588F:   GoTo loc_405D22
loc_405892: End If
```

Сразу бросается в глаза проверку на пустоту одного из текстовых полей. Для нас это малокритично, так что смотрим дальше:

```
loc_40589F: var_88 = TextBox_764.Text
loc_4058B4: If (Len(var_8C) > 2) Then '405CF1
```

Теперь узнаем что имя не может быть менее 2х символов. Нам на это пофиг... анализируем далее:

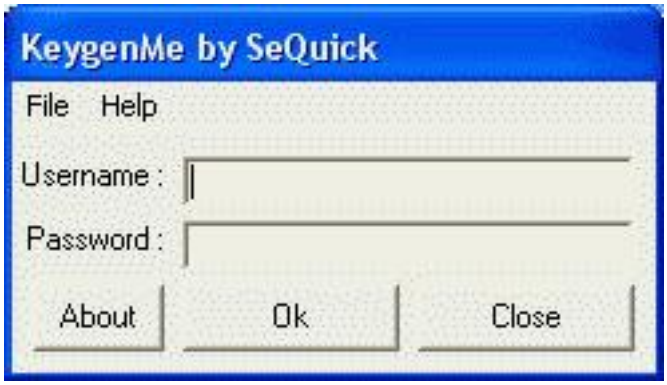
```
loc_405944:     var_88 = TextBox_768.Text
loc_40594C:     var_B4 = CVar(var_8C) 'String
loc_40595B:     If Not(IsNumeric(var_B4)) Then '405992
loc_40597F:       MsgBox("No access!", 16, "Keygenme", var_F4, var_114)
loc_40598F:       GoTo loc_405B11
loc_405992:     End If
```

Если пароль не число - прога ругается. Что-ж, нам лучше, уже знаем, что пароль как-то вычисляется и представляет собой результат вычислений над числами. Проверим наше предположение и поглядим ниже:

```
loc_405A53:     If (CDbl(var_8C) = CDbl((CLng((Asc(CStr(Left(CVar
                (TextBox_768.Text), 1))) * Asc(CStr(Right(CVar(CStr
                (TextBox_768.Text)), 1))))) * Len(TextBox_764.Text))))
                Then '405AE0
loc_405ABE:     MsgBox("Thanks for registration!" & Chr(10) & Chr(10) &
                "Registered to:" & CVar(TextBox_764.Text), 64, "Keygenme",
                var_180, var_1A0)
```

Сразу видно, что одно из текстовых полей пароль второе имя... догадаться, какое текстовое поле пароль несложно. Видим, что пароль есть результат умножения кодов 1 и последнего символа имени и длины имени. Попробуем:

```
имя: GPcH
пароль: 20448
```



Школа кейгенеров :)

Кейген на том же VB будет выглядеть примерно так:

```
strName="GPcH"
strPass=Asc(Left$(strName,1)) * Asc(Right$(strName,1)) * Len(strName)
```

Примерно таким же методом можно закейгенить и другие алгоритмы, но тут без исследования, анализа и плотного разбора алго не обойтись - рассмотренный выше пример слишком идеален и вряд ли встретится в реальной программе. Но поверь - главное желание, а разобрать можно любой сложности алгоритм, был бы декомпиялтор. А что если Pro версии моего декомпилятора нет, а

прогу крякнуть нужно? Поисследуем этот же кейгенми под Lite версией и поглядим что можно крякнуть:

```
loc_405A31: EqR4
loc_405A32: FFreeStr var_118 = "": var_124 = "": var_12C = ""
loc_405A3D: FFreeAd var_88 = "": var_90 = "": var_120 = ""
loc_405A48: FFreeVar var_B4 = "": var_D4 = "": var_F4 = ""
loc_405A53: BranchF loc_405AE0
```

Видим сравнение пароля с рассчетным эталоном оператором EqR4, затем идет переход на адрес 405AE0, если пароль неверен. Меняем этот переход с 1C на 1D, и наш кейгенми будет регаться на любой числовой пароль. Теперь поглядим enm ранее, если пароль проверяется на то, что он состоит только из чисел:

```
loc_40594F: ImpAdCallI2 IsNumeric()
loc_405954: NotI4
loc_405955: FFree1Ad var_88
loc_405958: FFree1Var var_B4 = ""
loc_40595B: BranchF loc_405992
```

Опять меняем 1C на 1D и можем в качестве пароля вводить все что угодно :) Менять само собой надо по указанному адресу, открыв кейгенми в любом HEX редакторе, хотя я советую юзать лючший на мой взгляд продукт для этих целей - hiew от sen'a.

Заключение

Вот мы и научились с тобой находить пикод в EXE файле, декомпилировать его в уме и в декомпиляторе, познакомились с основными опкодами P-Code и теперь можем анализировать что угодно. Будь то вирус с целью написания для него антивируса, будь то прога которая не регается без ключа или банально для повышения своего скилла. И не верь тем кто говорит что пикода нигде уже нет... я видел проги с гвардантом и хаспом в пикоде, а немало огромных проектов, скомпилированных в пикод. А уж троев и червей на VB полно и чтобы их анализировать обязательно нужен декомпилятор или знания, которые я думаю ты получил, прочитав данную статью. Удачи тебе!

Заходи на www.dotfix.net - почерпнешь много полезной информации по устройству VB и не только. Декомпилятор VB бери с www.vb-decompiler.org

Файлы к статье здесь (находится в архиве `wasm_files.zip`: `files/recovered/420/dec_pcode.zip`).

Реверсинг протокола Network Assistant 3.x

Автор: Mescalito

Дата: 2006-10-05

Введение

В статье рассматривается реверсинг протокола чата Network Assistant 3.x. Это первая моя статья, поэтому можно считать, что она написана новичком для новичков, тем более что разработчики никакой защиты на программу не навешивали, код достаточно прост и ясен, а единственная проблема - его объём.

Идея поковыряться в чём-либо возникла, когда мне надоело писать свою собственную программу, поэтому выбор продукта по большому счёту случаен.

Network Assistant (далее сокращённо - Nassi) от Gracebyte Software представляет собой чат для локальной сети под Windows. Программа шароварная, однако в сети легко можно найти ключ. На настоящий момент актуальной является четвёртая версия, однако я буду рассматривать третью, т. к. именно этой версией пользуются в моей локалке

Версия 3.x поддерживает общие, приватные и защищённые паролем каналы, личные сообщения, доску рисования, смену состояний пользователей, передачу информации о версии чата, ОС, заголовке активного окна, просмотр и управление процессами, копии экрана, буфера обмена, статистики, сигнализаторы и т. д. К счастью, большую часть этих "возможностей" можно отключить. Чат не имеет сервера и работает через UDP-броадкаст (по умолчанию порты 50138/50139) и IP-мультикаст. Далее я буду ориентироваться на броадкаст т. к. его по историческим причинам используют в моей сети, хотя в общем-то формат пакетов Nassi от типа транспорта не зависит.

Прежде чем самому пытаться вскрыть протокол имеет смысл поискать в интернете открытую информацию. Побродив по сети можно выяснить следующее:

Во-первых. Nassi-пакет устроен следующим образом. Первый байт обозначает версию протокола: 0 для 2.x, 1 для 3.x. Далее идут два байта с длиной пакета. Это справедливо для 2.x и 3.x. Формат следующих далее данных зависит от версии чата. Структура пакета 2.x хорошо описывается в статье Эксплоит для сетевого чата в 48-м спецвыпуске Хакера (советую почитать, как очень неплохой туториал по реверсингу и написанию эксплоитов для начинающих). Для 3.x же все данные со смещения +3 зашифрованы.

Во-вторых. Для Nassi существует два бота: Vsbot и Krbot. Первый написан на Делфи (как, к слову, и сам чат) второй на Борланд C++, оба запакованы, однако распаковываются легко (самая длительная фаза - набор в адресной строке браузера http://cracklab.ru/download.php :)). Боты работают только с запущенным чатом и используют сырые сокеты для перехвата его пакетов. Очевидно, что функционально они победнее чата, по объёму кода - всего на один десятичный порядок меньше, поэтому реверсить их особого смысла нет.

Из открытых источников известно, что разработчики чата помогли авторам ботов, а значит у последних должно быть точное описание формата пакетов. Мне удалось связаться с одним из них, однако поделиться информацией он отказался, сославшись на договорённости с Gracebyte. Сами же разработчики меня проигнорировали.

Итак. Для реверсинга нам потребуется собственно сам чат - я исследовал Network Assistant 3.2.5 build 2260, отладчик и дизассемблер - я использовал Ida Pro, что и вам советую

Первым делом скармливаем главный модуль Nassi.exe Иде (не забываем применить сигнатуры - Delphi, Borland VCL, CBuilder). Пока идёт анализ программы подумаем, откуда в листинге в несколько сот тысяч строк начинать поиск интересующего нас кода по формированию, шифровке, расшифровке и анализу пакета.

Во-первых, нам известен криптоалгоритм используемый чатом - Blowfish (об этом можно прочитать на SecurityLab и на форуме самой Gracebyte) и можно попытаться найти код, отвечающий за шифровку/дешифровку, если была использована более или менее стандартная реализация. Во-вторых, можно начинать с анализа вызовов Winsock API

Blowfish

Начнём с Blowfish. Я использовал вот это описание алгоритма: достаточно подробное, но без излишеств с примером реализации на Яве. Из него мы можем выделить следующие, предположительно полезные фрагменты:

1. Blowfish — 64-битный блочный шифр с ключом переменной длины. Алгоритм включает в себя 2 части: часть расширения ключей и часть шифрования данных. Расширение ключа преобразует ключ, в большинстве 448-битный, в несколько суммированных массивов подключей в 4168 байт. Шифрование данных происходит через 16-итерационную сеть Feistel
2. Blowfish использует большое количество подключей. Эти ключи должны быть предварительно вычислены перед любым шифрованием данных или расшифровкой
1. P-массив включает 18 32-битных подключей
2. Имеются четыре 32-битных S-блока с 256 входами каждый
3. Подключи вычисляются, путем использования алгоритма Blowfish. Данный метод состоит в следующем:
1. Сначала инициализируется P-массив и затем четыре S-блока, с фиксированной строкой. Эта строка состоит из шестнадцатеричных цифр pi (меньше начальной тройки).
2. Произведите XOR P1 с первыми 32 битами ключа, XOR P2 со вторым 32-битами ключа, и так далее для всех битов ключа (возможно до P14). Циклически пройдите биты ключа, пока весь P-массив не будет "поXORен" с битами ключа. (Для каждой короткого ключа, имеется, по крайней мере, один эквивалентный более длинный ключ; например, если ключ 64-битный, тогда AA, AAA, и т.д., являются эквивалентными ключами.)
3. Необходимо шифровать все пустые строки с помощью алгоритма Blowfish, используя подключи, описанные на шагах (1) и (2).
4. Заменить P1 и P2 с использованием (3).
5. Шифровать с изменяемым подключом, используя шаг (3).
6. Заменить P3 и P4 с использованием шага (5).
7. Продолжить процесс, заменяя все входы P- массива, и затем все четыре S-блока.

И так, попробуем найти Blowfish в Nassi. Для начала попробуем на удачу поискать в листинге дизассемблера строки типа blow, blowfish и т. п. К счастью, нам везёт и мы сразу находим приведённый ниже фрагмент кода (Примечание: его наличие показывает, что Gracebyte использовали при написании чата сторонний криптокомпонент, т. к. вывод такого сообщения Nassi не нужен в принципе - как мы увидим далее чат использует два типа ключей, длины которых - константы. Некоторое время, я искал его в сети и нашёл один фриварный, похожий на используемый в Nassi, однако т. к. он достаточно большой и поставляется без исходников, анализировать его не имеет смысла):

```
CODE:004B476D                jle     short loc_4B4774
CODE:004B476F                cmp     ebx, 38h
CODE:004B4772                jle     short loc_4B478A
```

```
CODE:004B4774
CODE:004B4774 loc_4B4774:                             ; CODE XREF: sub_4B4754+19.j
CODE:004B4774                 mov     ecx, offset aBlowfishKeyMus ; "Blowfish: Key must be between
    1 and 56 "...
CODE:004B4779                 mov     dl, 1
CODE:004B477B                 mov     eax, off_40860C
CODE:004B4780                 call    unknown_libname_40 ; Delphi 5 Visual Component Library
CODE:004B4785                 call    @System@@RaiseExcept$qqrv ; System::__linkproc__ RaiseExcept(
    void)
CODE:004B478A
CODE:004B478A loc_4B478A:                             ; CODE XREF: sub_4B4754+1E.j
```

Очевидно, что перед нами проверка корректности Blowfish-ключа: если его длина (в ebx) ненулевая и меньше либо равна 56, исполнение продолжается, если нет - возбуждается исключение. Эта проверка - часть довольно большой функции sub_4B4754 (назовём её BlowInit), которая предположительно вычисляет подключи. Чтобы убедиться в этом проанализируем код в районе выше и ниже по листингу. При этом обнаруживаются следующие факты:

Во-первых, функция BlowInit вызывается согласно конвенкции thiscall и принимает четыре параметра (это видно по прологу и сгенерированному Идой заголовку функции и использованию регистров):

```
CODE:004B4754 var_20          = dword ptr -20h
CODE:004B4754 var_1C          = dword ptr -1Ch
CODE:004B4754 var_18          = dword ptr -18h
CODE:004B4754 var_10          = dword ptr -10h
CODE:004B4754 var_C           = dword ptr -0Ch
CODE:004B4754 var_8           = dword ptr -8
CODE:004B4754 var_4           = dword ptr -4
CODE:004B4754 arg_0           = dword ptr  8
CODE:004B4754
CODE:004B4754                 push    ebp
CODE:004B4755                 mov     ebp, esp
CODE:004B4757                 add     esp, 0FFFFFFE0h
CODE:004B475A                 push    ebx
CODE:004B475B                 push    esi
CODE:004B475C                 push    edi
CODE:004B475D                 mov     ebx, ecx
CODE:004B475F                 mov     [ebp+var_8], edx
CODE:004B4762                 mov     [ebp+var_4], eax
CODE:004B4765                 mov     esi, [ebp+arg_0]
CODE:004B4768                 lea     edi, [ebp+var_18]
```

Аргументы функции:

1. eax - указатель на объект, назовём его TBlow
2. edx - какой-то указатель
3. ecx - длина ключа
4. один аргумент в стеке - указатель

Во-вторых, функция BlowInit вызывается из двух мест в программе

1. С адреса 004E19EF:

```
CODE:004E19D9                 mov     eax, ds:off_52AC8C
CODE:004E19DE                 push    eax
CODE:004E19DF                 mov     edx, ds:off_52ADBC
CODE:004E19E5                 mov     eax, ds:off_52AF7C
CODE:004E19EA                 mov     ecx, 0Ah
CODE:004E19EF                 call    BlowInit
```

С, как мы видим, следующми аргументами:
1. eax = off_52AF7C = offset dword_52D9A4
2. edx = off_52ADBC
3. ecx = 10
4. off_52AC8C - через стек

2. С адреса 00504EDB:

```
CODE:00504EBD                call    GetTickCount_0
CODE:00504EC2                mov     ds:dword_526644, eax
CODE:00504EC7                push    offset unk_526620
CODE:00504ECC                mov     edx, offset dword_526644
CODE:00504ED1                mov     eax, offset unk_52D9A4
CODE:00504ED6                mov     ecx, 4
CODE:00504EDB                call    BlowInit
```

С аргументами:
1. eax = dword unk_52D9A4
2. edx = указатель на переменную хранящую текущий тик таймера
3. ecx = 4
4. offset unk_526620 в стеке

В-третьих, в районе адресов 004B478A - 004B4817 путем последовательных вызовов функции sub_4029C4 (копирует ecx байт с eax в edx) и sub_403324, которую у меня Ида распознаёт как @@FillChar (заполняет строку длиной edx по адресу eax байтом cl) инициализируются поля объекта TBlow. В итоге TBlow содержит следующие поля:

1. Два блока по 8 байт, копируемых с буфера, на который указывает 4-й аргумент BlowInit
2. 4096 байт, копируемых с адреса 0052472C
3. 72 байта, копируемых с адреса 005246E4

Теперь самое время вспомнить, что для генерации подключей Blowfish использует P-массив размером в 72 байта и S-массив размером 4096 байт. В стандартной реализации для их инициализации используется число пи в двоичном виде без начальной тройки. Оно (точнее, первые 4096+72 байт) приведено в в этой статье. Как видим, реализация в Nassi не стала исключением. (К слову, если скормить Nassi.exe PEiD, он найдёт в ней несколько криптокомпонентов, причём в качестве сигнатур используются как раз эти четыре с небольшим килобайта - число пи)

И так, код инициализации Blowfish нами найден. Самое время найти функции шифровки и расшифровки. Очевидно, что они обращаются к полям объекта TBlow. Ставим бряк на 52D9A4 и аттачимся к уже запущенному процессу (чтоб не ловить инициализацию). После минутного эксперимента обнаруживается, что бряки часто срабатывают, когда программа обращается к TBlow с адреса 004B4A4E и иногда с 004029A3

Посмотри на код в районе 004B4A4E. Этот адрес находится в теле функции sub_4B4A30, которая вызывается по 16 раз из тел функций sub_4B4A78 и sub_4B4C58. В свою очередь sub_4B4A78 вызывается дважды с BlowInit и один раз с sub_4B4E38, которая в свою очередь вызывается в цикле практически сразу перед отправкой пакета функцией sendto:

```
CODE:0050444B                xor     ebx, ebx
CODE:0050444D
CODE:0050444D loc_50444D:                            ; CODE XREF: sub_5042D4+199.j
CODE:0050444D                mov     eax, ebx
CODE:0050444F                shl     eax, 3
CODE:00504452                add     eax, 3
CODE:00504455                lea     ecx, buf[eax]
CODE:0050445B                lea     edx, buf[eax]
CODE:00504461                mov     eax, offset dword_52D9A4
CODE:00504466                call    sub_4B4E38
CODE:0050446B                inc     ebx
CODE:0050446C                dec     esi
CODE:0050446D                jnz     short loc_50444D
CODE:0050446F
CODE:0050446F loc_50446F:                            ; CODE XREF: sub_5042D4+174.j
CODE:0050446F                mov     eax, offset dword_52D9A4
CODE:00504474                call    sub_4B4EBC
CODE:00504479                mov     [ebp+to.sa_family], 2
CODE:0050447F                mov     eax, ds:off_52B3B8
CODE:00504484                mov     ax, [eax]
```

```
CODE:00504487                push    eax             ; hostshort
CODE:00504488                call    htons
CODE:0050448D                mov     word ptr [ebp+to.sa_data], ax
CODE:00504491                mov     eax, [ebp+hostlong]
CODE:00504494                push    eax             ; hostlong
CODE:00504495                call    htonl
CODE:0050449A                mov     dword ptr [ebp+to.sa_data+2], eax
CODE:0050449D                push    10h             ; tolen
CODE:0050449F                lea     eax, [ebp+to]
CODE:005044A2                push    eax             ; to
CODE:005044A3                push    0               ; flags
CODE:005044A5                mov     eax, [ebp+len]
CODE:005044A8                push    eax             ; len
CODE:005044A9                push    offset buf      ; buf
CODE:005044AE                mov     eax, ds:dword_526634
CODE:005044B3                push    eax             ; s
CODE:005044B4                call    sendto
```

Функция sub_4B4C58 вызывается из тела sub_4B4E6C которая вызывается в очень похожем цикле в теле sub_518534:

```
CODE:005186A0                xor     ebx, ebx
CODE:005186A2
CODE:005186A2 loc_5186A2:                            ; CODE XREF: sub_518534+193.j
CODE:005186A2                mov     eax, ebx
CODE:005186A4                shl     eax, 3
CODE:005186A7                add     eax, 3
CODE:005186AA                mov     edx, [ebp+var_11C]
CODE:005186B0                lea     ecx, [edx+eax]
CODE:005186B3                mov     edx, [ebp+var_11C]
CODE:005186B9                add     edx, eax
CODE:005186BB                mov     eax, ds:off_52AF7C
CODE:005186C0                call    sub_4B4E6C
CODE:005186C5                inc     ebx
CODE:005186C6                dec     esi
CODE:005186C7                jnz     short loc_5186A2
```

Данные циклы так и просятся, чтобы их назвали циклами кодирования и декодирования пакетов блоками по 8 байт со смещения +3. Однако в отличии от цикла loc_50446F, который находится практически сразу перед вызовом sendto, что выдаёт его "вину", никаких winsock выше loc_5186A2 не видно. Вместо них мы находим заголовок функции sub_518534 вызываемой из sub_512E44, которая в свою очередь вызывается по таблице. Однако, если посмотреть на код следующий за вызовом recvfrom по адресу 0050384B, обнаруживается, что в случае удачного вызова какому-то окну (с классом TfrmNassiMain, что, впрочем, не важно) отсылается сообщение 40Ah = WM_USER + 0Ah, wParam которого указывает на буфер содержащий в том числе и принятый пакет

```
CODE:005038DB                mov     eax, [ebp+lParam]
CODE:005038DE                call    sub_40278C
CODE:005038E3                mov     ebx, eax
CODE:005038E5                mov     edx, ebx
CODE:005038E7                lea     eax, [ebp+var_8]
CODE:005038EA                mov     ecx, 4
CODE:005038EF                call    sub_4029C4
CODE:005038F4                lea     edx, [ebx+4]
CODE:005038F7                lea     eax, [ebp+PerformanceCount]
CODE:005038FA                mov     ecx, 8
CODE:005038FF                call    sub_4029C4
CODE:00503904                lea     edx, [ebx+0Ch]
CODE:00503907                lea     eax, [ebp+buf]
CODE:0050390D                mov     ecx, [ebp+wParam]
CODE:00503910                call    sub_4029C4
CODE:00503915                mov     eax, [ebp+lParam]
CODE:00503918                push    eax             ; lParam
CODE:00503919                push    ebx             ; wParam
CODE:0050391A                push    40Ah            ; Msg
CODE:0050391F                mov     eax, ds:off_52B35C
CODE:00503924                mov     eax, [eax]
CODE:00503926                call    @Controls@TWinControl@GetHandle$qqrv ; Controls::TWinControl::
```

```
    GetHandle(void)
CODE:0050392B                push    eax             ; hWnd
CODE:0050392C                call    PostMessageA
```

Если поставить бряки на 0050392C и 00512E44 и посмотреть на значения в регистрах и памяти, на которую они ссылаются, то хорошо видно, что sub_512E44 не что иное как процедура обработки сообщения 40Ah

Теперь у нас есть практически все знания необходимые для создания собственного кодера/декодера пакетов в виде отдельной dll-ки. Осталось только узнать, каким ключем осуществляется шифрование. Для этого ставим бряк на функцию BlowInit и (на всякий случай) на объект TBlow. До отправки кодирования первого пакета бряк на BlowInit срабатывает дважды. Первый раз Nassi использует 4-байтный ключ - тик таймера, второй раз 10-байтный ключ с адреса 00526614. При этом параметр передающийся через стек указывает на 8-байтный блок по адресу 00526620. По-видимому, этот блок Очевидно, что подключи генерируемые первым вызовом игнорируются и шифровка/расшифровка использует только результат второго вызова.

Стартуя с BlowInit, loc_5186A2 и loc_50444D проходим по вложенным вызовам и выдираем из листинга данные и код (благо их немного), подчищаем, редактируем, создаём удобные нам входы для BlowInit, CryptPacket и DecryptPacket и компилируем. Подробнее я описывать не буду, т. к. это быстрее сделать, чем рассказать. В файлах идущих со статьёй есть dll-ка с "исходниками". В комментариях указаны адреса, с которых выдрана та или иная функция, так что разобраться думаю будет не сложно.

Заголовок

Начинать исследовать поля пакета можно тремя методами:

1. Анализировать конструктор
2. Анализировать получателя
3. Связать вместе снифер и декодер и анализировать сами пакеты

Начнём с конструктора. Беглым просмотром от начала функции sub_5042D4 до цикла loc_50444D можно восстановить приблизительную структуру пакета:

Смещение	Размер	Назначение
0	1	Версия протокола; в нашем случае всегда 1
1	2	Размер пакета
Далее все данные кодируются		
3	1	Неизвестный байт
4	4	Неизвестный dword
8	1	Неизвестный байт
9	4	Неизвестный dword
13	...	Неизвестный блок
...	...	Неизвестный блок
...	8	Время отправки пакета в формате TDateTime
Далее предположительно идут собственно данные пакета		

Неизвестный байт по смещению +8 передается функции-сендеру (sub_5042D4), как первый параметр (в dl). Сама же функция вызывается с нескольких десятков мест с разными константными значениями dl. Предположительно этот байт определяет тип пакета

Неизвестные блоки, как это видно под отладчиком, - имя хоста и ник-нейм в виде pascal-строк, а двойное слово по +4 - инкрементальный счётчик.

Больше на поверхности здесь ничего вроде бы не лежит, поэтому можно перейти к анализатору пакетов - функции sub_518534. В ней мы найдём, что байт +3 должен быть меньше 2 (на практике - всегда 1), а байт +8 действительно определяет тип пакета (по адресу 005188CC и ниже располагается ветвление). Также легко обнаружить, что двойное слово +9 есть цвет пользователя в формате RGB

Итого заголовок Nassi-пакета имеет следующий вид:

Смещение	Размер	Назначение
0	1	Версия протокола; в нашем случае всегда 1
1	2	Размер пакета
Далее все данные кодируются		
3	1	Всегда 1
4	4	Счётчик
8	1	Тип пакета, Nassi-команда
9	4	Цвет пользователя
13	...	Pascal-строка, хост-нейм
...	...	Pascal-строка, ник-нейм
...	8	Время отправки пакета в формате TDateTime
Далее идут собственно данные пакета		

Тело

Чтоб выяснить ещё что-либо я написал снифер и прикрутил к нему декодер пакетов. Снифер вместе с исходниками приведён во вложении. Подробно описывать его не буду, скажу лишь, что он использует сырые сокеты для перехвата (информации об этом в сети достаточно), APC для корректного прерывания перехвата (см. статью Проблемы реализации многопоточных WinSockets-приложений в 11-м номере PC Magazine за 2005 год) и алгоритм приведённый в Зубкове для печати чисел в hex-формате. Остальное вроде бы вопросов вызывать не должно. Снифер игнорирует фрагментированные IP-пакеты, однако для исследования протокола это не важно

Написав снифер мы получаем относительно удобный инструмент для анализа пакетов и в короткое время можем узнать формат наиболее важных из них.
Например:

Смещение	Размер	Назначение
Тип 36: Смена состояния		

Смещение	Размер	Назначение
0	1	Номер состояние: 0.. 5 - встроенное (Свободен, Играю, Работаю, Не беспокоить, Секретное, Отошёл), 6.. 15 - определяемое пользователем
1	1	Тип состояния: 1 для встроенного
2	...	Pascal-строка - название состояния
...	...	Pascal-строка - описание состояния
Тип 10: Сообщение в канал		
0	1	Тип канала: 0 - обычный 1 - общий для всех 2 - запароленный 3 - приватный
1	1	Pascal-строка, название канала
...	...	Pascal-строка, пароль (только для 2-го типа)
...	...	ASCIIZ-строка, сообщение в канал

Всего типов сообщений - несколько десятков, для практических целей могут понадобиться:
2 - Запрос информации о пользователе
3 - Оповещение об активности окна
10 - Сообщение в канал
13 - Оповещение о присоединении к каналу
14 - Оповещение о выходе из канала
30 - Личное сообщение
35 - Оповещение о смене ника
36 - Оповещение о смене состояния
37 - Запрос состояния
130 - Сообщение информации о себе
и, возможно, некоторые другие

Я не буду здесь приводить более подробное описание всех типов пакетов: любой желающий может восстановить его сам. Назову лишь варианты практического применения полученной информации:

1. Написание собственного клиента или бота (Существующие боты работают через raw-сокеты только параллельно чату, а сам чат - лучший детектор ОС в сетях, где им активно пользуются :))
2. Написание флудилок
3. Чтение сообщений из запароленных каналов
4. Вместе с ARP-спуфером - перехват личных сообщений и сообщений из приватных каналов
5. Написание nassi-шлюзов для общения в чате пользователей из различных сегментов
6. Написание шлюзов а-ля nassi2irc
7. Совместо с известным нюкером nassidos.pl написание полезных утилит (например: автокикалка любителей создавать общие для всех каналы, рассылать всплывающие сообщения, употребляющих "плохие слова", флудящих, имеющих ненравящийся вам цвет :) и т. п.)
(Примечание: В отличии от версии 2.x, насколько мне известно, серьёзной уязвимости, позволяющей написание эксплоита, в 3.x найдено не было. Однако 3.x подвержена DoS-атаке: программа не всегда корректно обрабатывает "мусорные пакеты" и может аварийно завершить работу. Подробнее об этом можно прочитать в частности на SecurityLab. Gracebyte выпустили два патча для устранения проблемы. При установке они обновляют версию Nassi до build 2261 (первый глючный патч) и 2262 соответственно, однако в настоящее время скачать их с сайта разработчиков

нельзя - очевидно таким образом они пытаются убедить юзеров перейти на версию 4.x)

Файлы к статье находятся здесь (находится в архиве `wasm_files.zip`: `files/recovered/437/nassi.zip`).

Исследование «промежуточного» кода на примере GP-кода языка NATURAL - Часть 2: Операнды (окончание)

Автор: Konstantin

Дата: 2006-11-13

В этой части статьи на предметном столике наших исследований
окажутся такие представители славной плеяды операндов как параметры, глобальные переменные и проч. Не обойдем своим вниманием и подход к описанию массивов операндов в GP-коде NATURAL.

2.2 Параметры (Value table
index = 6)

Объяснять, что такое параметры, и для чего они нужны, думаю,
нет никакой необходимости. Однако есть любопытная особенность использования параметров в NATURAL- программах (точнее,
подпрограммах), мимо которой пройти никак нельзя. В NATURAL-подпрограммах с параметрами вызываются не процедуры (функции), а программные модули.

Параметры в NATURAL-подпрограммах
объявляются одним из двух способов (либо их комбинацией):

1. Явно определяются в блоке define data parameter. Например:

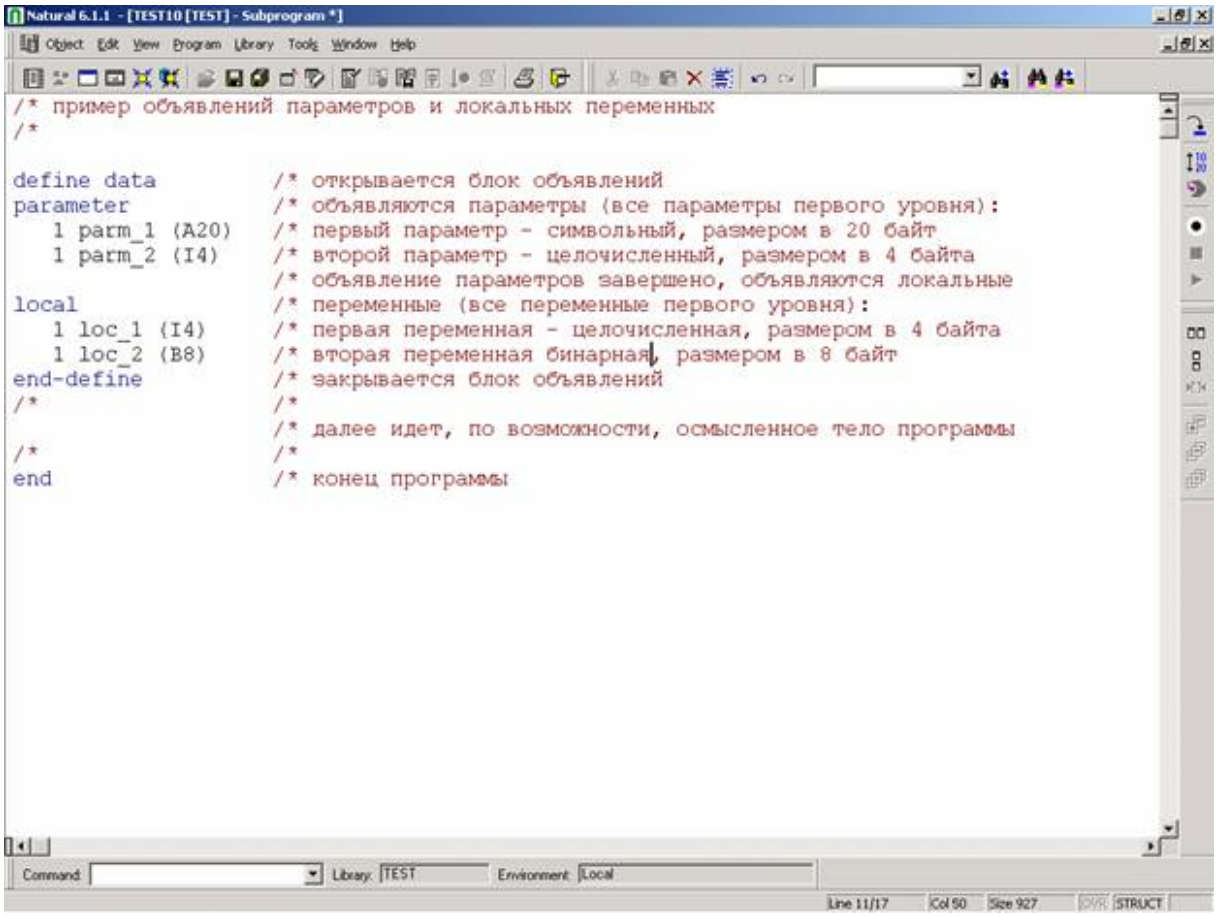


Рис 2.6. Пример объявлений в Natural

2. Объявляются в специальном программном модуле (PDA – Parameter Data Area), а в подпрограмме осуществляется ссылка на этот программный модуль (либо на несколько модулей, при необходимости). Например, так:

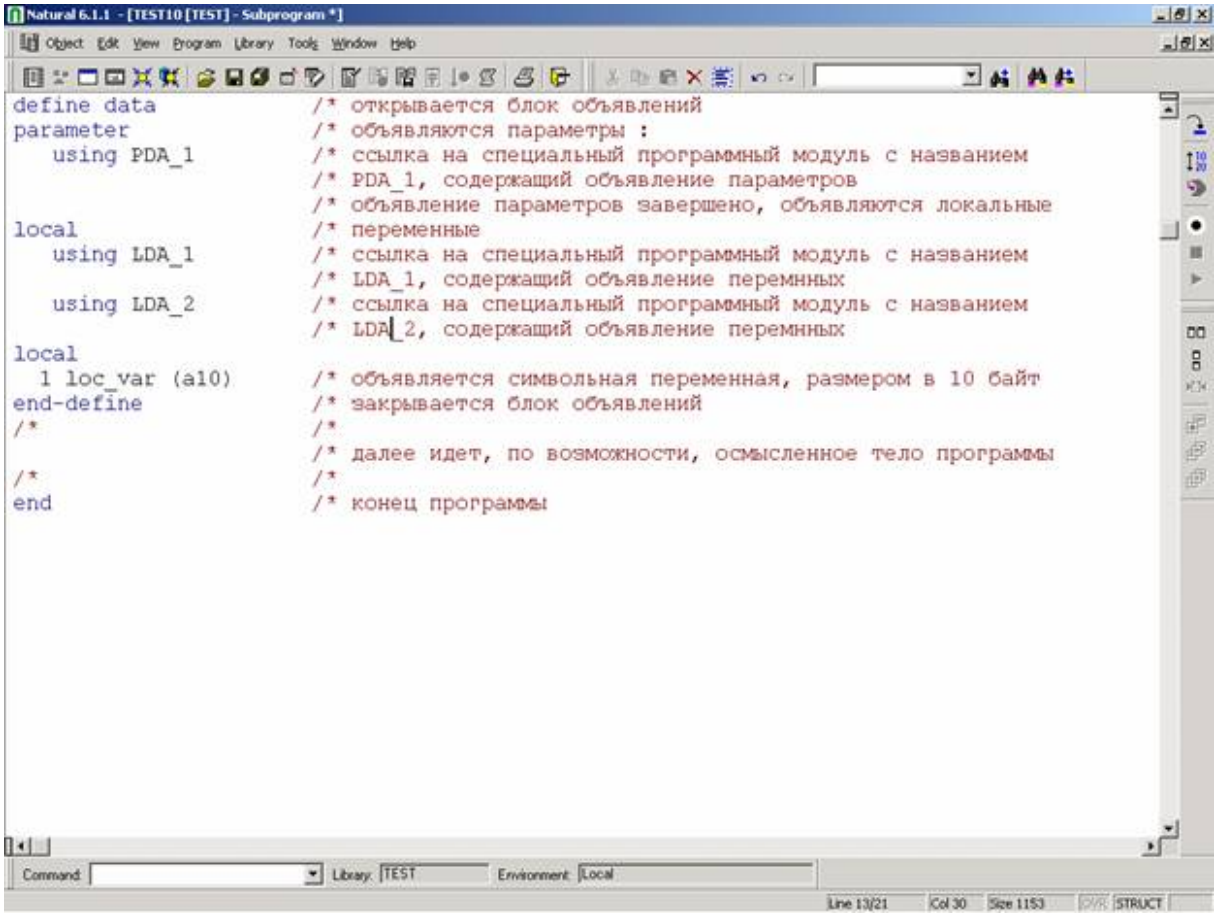


Рис 2.7. Пример объявлений в Natural

В этом случае модули PDA_1, LDA_1, LDA_2 должны быть заранее сформированы и скомпилированы. При компиляции целевой подпрограммы определение операндов копируется внутрь GP_кода. Для выполнения целевой подпрограммы модули объявлений операндов уже не нужны.

Важно! После компиляции в папке объектных кодов появятся файлы, содержащие компилированное описание операндов. В нашем случае это PDA_1.nga, LDA_1.ngl, LDA_1.ngl. Гарантированно восстанавливать из них операнды – милое дело! Но об этом чуть ниже.

GP-код не зависит от того, каким образом объявлены параметры (да и операнды вообще),

Подробнее параметры мы будем обсуждать, когда доберемся до разбора механизма вызова внешних программных модулей (для знатоков - речь идет о call, callnat, fetch, perform), а сейчас нам важно запомнить три основных момента:

Если в Таблице операндов нам встретился Value table index = 6, перед нами параметр (но не все параметры имеют Value table index = 6 в Таблице операндов! См. ниже).

Параметрическая область общая для всего программного модуля.

Формат и порядок следования параметров определен в таблице операндов, в Таблице же параметров (Parameter Table, таблица с седьмым индексом) содержатся дополнительные свойства параметров. Структура записи в таблице параметров и возможные значения полей приведены в Приложении 2 (Структура Таблицы параметров).

Вооружившись этими ценными сведениями, попробуем решить следующую задачку.

Задача

Группа программистов-энтузиастов после многодневных свирепых дебатов пришла к выводу, что для полного счастья (т.е. для завершения супер-пупер-мега-проги, умножающей сумму двух чисел на третье) им не хватает всего лишь одной процедурки. Причем процедурка существующая, но в виде исполняемого кода (аксакалы еще помнят людей, видевших исходник). Документация (если она и была) оказалась нещадно съедена гигантскими крысами-мутантами. А безумный программист, сваявший эту процедуру, был задавлен самосвалом при попытке перевести какую-то бабушку через дорогу (бабушка жива, но ничего не помнит). Программисты-натуралисты готовы отдать пачку нечитающихся пятидюймовых дискет и свою душу в придачу за восстановление параметров вызова этой процедуры.

*А вот и*GP-код этой процедуры:*

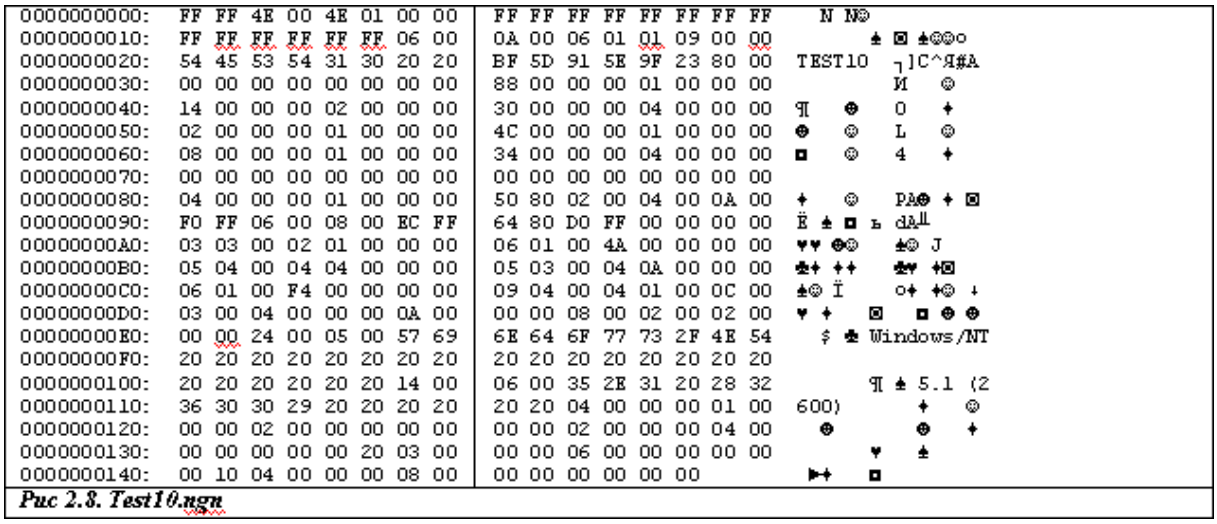


Рис 2.8. Test10.ngp

Решение

Общий план восстановления параметров таков: разбор заголовка GP-кода, разбор секции параметров, определение формата параметров по таблице операндов. Почему сначала секция параметров, потом таблица операндов, а не наоборот, увидим чуть позже.

Определение названия и типа программного модуля (условимся с версиями не заморачиваться)

Название и тип модуля надежнее всего определять по фиксированной части заголовка. Это обсуждалось в первой части статьи. В приложении к ней же приведена структура фиксированной части заголовка. В результате получаем, что перед нами подпрограмма (Object type = N, что значит Subprogram) с именем Test10. Это вполне соответствует

имени и расширению файла, содержащего GP-код (test10.ngn)

Определение Таблицы Секций

Как ее отыскать – см. в первой части статьи, соответствие индексов секций и их названий – см. Секции в GP-коде Приложения 2 (вторая часть статьи).

Должны получить вот такую табличку секций

NN	Table	Size	Start	End	Align	Commets
00	.Header	0088	0000	0087	1	Header Table
01	.Code	0014	0088	009B	2	Code Table
02	.Operand	0030	009C	00CB	4	Operand Table
03	.Constant	0002	00CC	00CD	1	Constant Table
04	.MultPurp	004C	00CE	0119	1	Multi Purpose Table
05	.LDA	0008	0000	0000	1	Local Data Area Source Table
06	.PDA	0034	011A	014D	4	Parameter Table
07	.SDA	0000	0000	0000	0	Static constant Data Area Table
08	.ArrDescr	0000	0000	0000	0	Array Description Table
09	.TDA	0004	0000	0000	1	TDA Table

Рис 2.9. Test10.ngn (Таблица секций)

Разбор Секции Параметров

Этой секцией мы не занимались, потому остановимся на ней поподробнее.

Начало секции 0x011A
(см. Рис 2.7).

```
0000011A: 04 00 00 00 01 00 00 00 | ........
00000122: 02 00 00 00 00 00 00 00 | ........
0000012A: 02 00 00 00 04 00 00 00 | ........
00000132: 00 00 00 20 03 00 00 00 | ... ....
0000013A: 06 00 00 00 00 00 00 10 | ........
00000142: 04 00 00 00 08 00 00 00 | ........
0000014A: 00 00 00 00             | ....
```

Рис 2.10. Test10.ngn (секция параметров)

Первые 4 байта – количество параметров. Далее следует таблица параметров (см. Секция параметров в Приложении 2) . Разбираем секцию:

Таб.2.6 Разбор секции параметров

Индекс параметра	Ссылка на операнд	Ссылка на переопред.	Флаги	Комментарий
01	02	00	00	Параметр без опций
02	04	00	2000	By value
03	06	00	1000	By value result
04	08	00	00	Параметр без опций

Итак, перед нами 4 параметра. За счет ссылок на операнды в таблице операндов становится возможным определить формат этих параметров.

Определение формата параметров по Таблице Операндов

О том, как находить Таблицу Операндов и что с этим безобразием делать – см. предыдущую часть статьи. Повторяться смысла не вижу, потому привожу результат разбора таблицы полностью:

Таб.2.7 Таблица операндов			
№№	Ссылка	Код секции	Комментарий
1	0x00	00 00 00 00 03 03 00 02	Константа, разм 2 байта, формат I Знач: 01
2	0x02	01 00 00 00 06 01 00 4A	Параметр, разм. 12 байт (10.2) формат N
3	0x04	00 00 00 00 05 04 00 04	Лок. переменная, разм. 4 байта, формат F
4	0x06	04 00 00 00 05 03 00 04	Лок. переменная, разм. 4 байта, формат I
5	0x08	0A 00 00 00 06 01 00 F4	Параметр, разм. 27 байт (20.7) формат N
6	0x0A	00 00 00 00 09 04 00 04	Временная переменная, 4 байта, формат F

Интерпретация длины формата N
требует разъяснений. Здесь логика такая: пять бит отводится для хранения
размера целой части операнда, а три бита – для хранения размера дробной части.
Например, 0x4A
в двоичном представлении 01001010, здесь 010 (3 dec)
– точность после запятой, т.е. размер дробной части, а 01010 (10 dec) – размер до запятой, т.е. целой части операнда. Итого N10.2. Аналогично F4 (11110100)
интерпретируется как N20.7 (111 = 7dec после запятой, 10100 = 20dec – до
запятой).

Важная особенность! Обратите внимание, после второго
операнда, распознанного нами как параметр, следуют два, распознанные нами (согласно значению поля Value table index Таблицы операндов) как локальные переменные, а
потом опять параметр. В NATURAL-программах такого не
бывает! Операнды в таблицу операндов заносятся в порядке объявления, а
объявлять сначала параметры, потом локальные переменные, и опять параметры не позволит синтаксический анализатор! Что-то тут не то!!!

Собственно, чтобы избежать этой неопределенности мы и
сделали таблицу параметров ведущей.

Восстановление параметрической области

Таблицы символов в пердставленном GP-коде
нет, потому сами придумаем названия и восстановим объявление параметрической области. Например, так:

```
define data parameter
 1 parm1 (n10.2)
 1 parm2 (f4) by value
 1 parm3 (I4) by value result
 1 parm4 (n20.7)
 end-define
```

И это работает!!!

Что ж, хороший повод поздравить себя с очередной маленькой
победой!

2.3 Глобальные переменные (Value table index = 0x12)

Глобальная область данных (GDA – Global Data Area) представляет собой часть общей
памяти, используемой Natural, отдельные элементы
которой могут быть совместно использованы программными модулями Natural. Любое изменение элементов данных глобальной области
сразу же становится доступным всем программным модулям Natural,
ссылающимся на данную глобальную область.

В GP-коде о наличии глобальных
переменных говорят поля *Flags* (значение 0x40 соответствует GDA referenced) и *GDA GP timestamp* постоянной части заголовка (см. первую часть статьи). Описание формата

глобальных переменных содержится в Таблице операндов и отличается от описания локальных переменных лишь значением поля Value table index (значение 0x12 соответствует GDA).

2.4 Независимые переменные (AIV, Value table index = 0x15)

AIV (Application-Independent Variables)
– переменные, глобальные по своей сути. Принципиальное отличие от «нормальных» глобальных переменных, рассмотренных выше, заключается в том, что aiv-переменные объявляются непосредственно в программных
модулях, а «нормальные» глобальные переменные объявляются только в специальном модуле, на который можно сослаться в целевой программе.

Обязательное условие объявления aiv-переменной
– имя переменной должно начинаться с символа «+» (например, «+aiv_var_01»);

Структура записи в AIV-TABLE хитрая (см. п.4 AIV-Table Приложения 2). Она состоит из постоянной части и переменной части. Если значение поля *Flags*
отлично от 0x4000 (Redefine), то переменная часть содержит
только имя переменной (без знака «+»). Если же значение поля *Flags* равно 0x4000 (Redefine), то переменная часть содержит параметры переопределения и имя переменной (без знака «+»).

После поля *Variable name* могут следовать
«выравнивающие» нули (обычно значение *Align* для AIV-Table равно 4, но надежнее посмотреть в соответствующей записи
Таблицы секций).

Все не так сложно и запутанно, как может показаться, и в этом мы убедимся на примере.

Предположим, нам посчастливилось встретиться с таким содержимым AIV-Table:

```
Dump of Section (21 - AIV-Table)

FileOffs  SectOffset Data
--------- ---------- -----------------------------------
00000156: (00000000) 01 00 00 00 00 00 04 00 | ........
0000015E: (00000008) 61 69 76 30 02 00 00 00 | aiv0....
00000166: (00000010) 00 00 04 00 61 69 76 31 | ....aiv1
0000016E: (00000018) 03 00 00 00 00 40 04 00 | .....@..
00000176: (00000020) 03 00 00 00 03 00 00 00 | ........
0000017E: (00000028) 61 69 76 32 04 00 00 00 | aiv2....
00000186: (00000030) 00 80 05 00 63 6F 6E 74 | .A..cont
0000018E: (00000038) 31 00 00 00             | 1...
End of dump
-------------------------------------------------------------
```

Рис 2.11. AIV-Table

Наши действия?

Насвистывая бодрую и незатейливую мелодию (не забывая при этом поглядывать на значение поля *Flags*, чтобы правильно подобрать подходящий вариант структуры записи), получаем следующее:

Таб.2.8 Разбор AIV-Table

Index of variable	Flags	Length variable name	AIV offset: base variable	Byte offset to base variable	Variable name	Комментарий
01	0000	04	-	-	aiv0	Вариант 1
02	0000	04	-	-	aiv1	Вариант 1
03	4000	04	03	03	aiv2	Вариант 2
04	8000	05	-	-	cont1	Вариант 1

Напомню, что форматы переменных находятся в таблице операндов (собственно, эта таблица и должна быть ведущей). Но т.к. разбор таблицы операндов уже набил оскомину, восстанавливаем aiv-переменные, оставив вместо формата многоточие.

```
define data
 independent
 1 +aiv0(…)
 1 +aiv1(…)
 1 redefine +aiv1
 2 filler 3x
 2 aiv2(…)
context
 1 cont1(…)
end-define
```

Вот и все! Ничего сложного…

2.5 Массивы (Value table index = 0x08)

Для объявления массивов в Natural используется так называемая индексная нотация. По умолчанию, нижняя граница индекса равна 1. Фантазию программиста Natural ограничивает, максимум, трехмерным массивом.

Таб.2.9. Примеры объявления массива в Natural

NN	Пример объявления	Комментарий
1	`1 Arr_1 (A10/3)`	Символьный 10-байтовый одномерный массив, 3 элемента. Эквивалентно `Arr_1 (A10/1:3)`
2	`1 Arr_2 (A10/3,4,6)`	Символьный 10-байтовый трехмерный массив. Эквивалентно `Arr_2 (A10/1:3,1:4,1:6)`
3	`1 Arr_3 (A10/0:2)`	Символьный 10-байтовый одномерный массив, 3 элемента
4	`1 Arr_4 (A10/10:13)`	Символьный 10-байтовый одномерный массив, 3 элемента
5	`1 Arr_5 (A10/*)`	Символьный 10-байтовый одномерный массив, верхний индекс не определен, количество элементов не определено. Эквивалентно `Arr_5 (A10/1:*)`
6	`1 Arr_6 (A10/*:10)`	Символьный 10-байтовый одномерный массив, нижний индекс не определен, количество элементов не определено. Эквивалентно `Arr_6 (A10/1:*)`

7	`1 Arr_7 (1:4)`
	`2 Fld_1(I4)`
	`2 Fld_3(A8/1:15)`
		Одномерный, четырехэлементный массив структуры, состоящей из 4-байтового целочисленной переменной и символьного 8-байтового одномерного массива из 15 элементов

8	`1 Arr_6 (A10/1:V)`	Символьный 10-байтовый одномерный массив, являющийся параметром подпрограммы, нижний индекс не определен, количество элементов не определено

Но это верхушка айсберга, а что у нас там, под водой? А под водой у нас замечательная секция **Array Description Table**!

Структура записи таблицы состоит из «описательной» части (см. Приложение 2 п.5) и данных. «Описательная» часть значениями полей Dimension flags 1, Dimension flags 2, Dimension flags 3 определяет

перечень параметров, описывающих соответствующие размерности массива. Данные – это 4-байтовые значения этих параметров.

Чтобы переварить этот словесный винегрет, разберем простой пример:

```
Dump of Section (8 - Array Description Table)

FileOffs  SectOffset Data
--------- ---------- -----------------------------------
000000DA: (00000000) 00 B0 00 00 00 00 01 00 |..▒.....
000000E2: (00000008) 00 00 00 00 03 00 00 00 |........
000000EA: (00000010) 0A 00 00 00             |....

End of dump
------------------------------------------------------------
```

Рис 2.11. Array Description Table

Разбираем «описательную» часть (см. Приложение 2 п.5.1)

Таб.2.10. «Описательная» часть массива

Field name	Offset	Size	Value
Dimension flags 1	0x00	2	0x0B00 (10110000 bin)
Dimension flags 2	0x02	2	0x0000
Dimension flags 3	0x04	2	0x0000
Dimension	0x06	1	01
Flags	0x07	1	00

По значению поля Dimension заключаем, что перед нами одномерный массив (косвенно на это же указывают нулевые значения полей Dimension flags 2 и Dimension flags 3).

Определяем перечень параметров, описывающих размерности (см. Приложение 2 п.5.2). В нашем случае, размерность одна. Значение поля Dimension flags 1, равное 10110000 bin, позволяет заключить, что параметров 3, и это:

OPT offset lower;

Occurences;

Index factor;

Разбор описательной части завершен.

Определение значений параметров. Первые 4 байта, следующие за «описательной» частью, являются значением первого параметра, вторые 4 байты – значением второго параметра и т.д. Итого:

Таб.2.11. Значения параметров в описании массива

№№	Parameter	Value
1	OPT offset lower	0x00000000
2	Occurences	0x00000003
3	Index factor	0x0000000A

Из предыдущих примеров мы уже знаем, что в Таблице операндов в самую первую запись (OPT offset = 0) Natural с завидным постоянством «кладет» константу со значением 1. А раз так, то мы можем смело заявить, что перед нами одномерный массив из 3 элементов, нижний индекс которого равен 1, соответственно верхний 3.

2.6 Восстановление компилированных областей данных

Объявлять из программы в программу повторяющиеся параметры и переменные – дело мучительное и неблагодарное. Особенно в мало-мальски серьезных проектах, в которых, как правило, повторяемость их использования довольно велика. Чтобы немного облегчить разработку и сопровождение прикладных программных продуктов, Natural предлагает использовать так называемые «области данных» (data area). Создав однажды некую область и скомпилировав ее, на нее можно ссылаться из множества программ.

Важно: для исполнения целевой программы нужна скомпилированная область данных!

Но я повторяюсь, об этом мы уже говорили, когда знакомились с техникой восстановления параметров.

Теперь ближе к теме.

Самый простой способ восстановления – ручной. Для этого необходимо открыть файл скомпилированной области данных в редакторе и скопировать оттуда строки определений – благо они хранятся в объектном коде в явном виде.

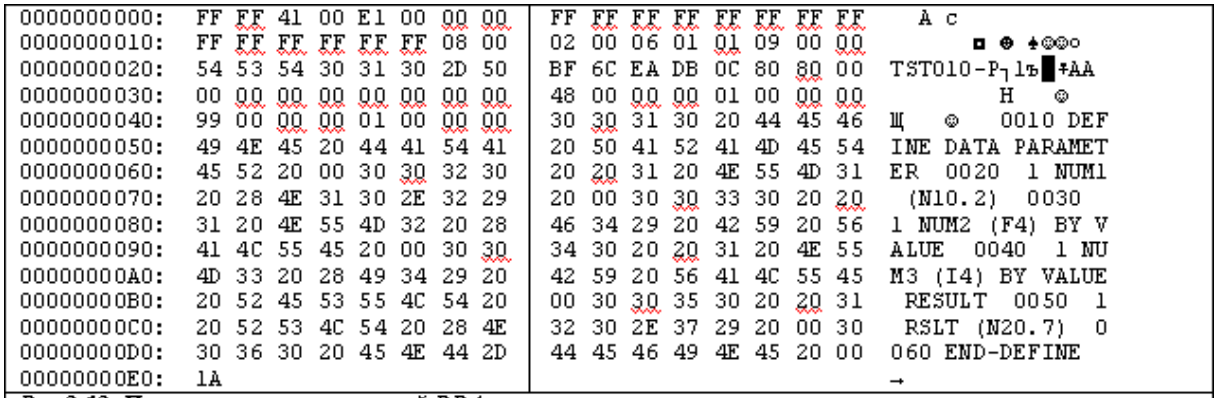


Рис 2.12. Пример скомпилированной PDA

В этом примере нужно скопировать блок, начиная с «0010 DEFINE DATA PARAMETER» (смещение 0x48) и заканчивая «0060 END-DEFINE» (смещение 0xCF). Что делать дальше? Это дело вкуса, квалификации и наличия свободного времени!

Такой вариант годится и для LDA, и для PDA, и для GDA, и может быть легко автоматизирован. Единственное, на что нужно обратить внимание, не добавил ли при компиляции Natural в объявления что-то от «себя» (например, GFID при компиляции LDA на основе DDM).

Хорошо, с экстренным восстановлением, предположим, понятно, а что со структурой скомпилированных областей данных вообще?

Структуры LDA и PDA идентичны. Вначале (вы не поверите!) идет уже полюбившийся нам и ставший родным заголовок (фиксированная часть плюс таблица секций, которые мы разбирали в первой части статьи), а далее – секция, содержащая «человеческие» объявления.

В качестве упражнения вы можете разобрать заголовок

предыдущего примера и убедиться, что вторая секция начинается как раз по смещению 0x48. И это вполне закономерно, т.к. размер фиксированной части заголовка 0x38 плюс 16 байт на Таблицу секций (8 байт – размер записи в таблице X 2 секции) как раз дает 0x48.

Структура же GDA гораздо сложнее, по той простой причине, что фактически это – полноценная подпрограмма! Не верите? Смотрим пример!

```
0000000000:  FF FF 47 00 5C 02 00 00   FF FF FF FF FF FF FF FF    G \☻
0000000010:  FF FF FF FF FF FF 05 00   13 00 06 01 01 09 00 00        ♣ ‼ ♠☺☺o
0000000020:  47 44 41 2D 31 20 20 20   BF 93 A1 D9 51 49 00 00  GDA-1    ┐Y6┘QI
0000000030:  00 00 00 00 00 00 00 00   D0 00 00 00 01 00 00 00           ╨   ☺
0000000040:  28 00 00 00 02 00 00 00   44 00 00 00 04 00 00 00  (    ☻    D    ♦
0000000050:  06 00 00 00 01 00 00 00   38 00 00 00 01 00 00 00  ♠    ☺    8    ☺
0000000060:  00 00 00 00 00 00 00 00   10 00 00 00 04 00 00 00           ►    ♦
0000000070:  00 00 00 00 00 00 00 00   14 00 00 00 04 00 00 00           ¶    ♦
0000000080:  02 00 00 00 01 00 00 00   00 00 00 00 00 00 00 00  ☻    ☺
0000000090:  00 00 00 00 00 00 00 00   00 00 00 00 00 00 00 00
00000000A0:  BE 00 00 00 01 00 00 00   00 00 00 00 00 00 00 00  ╛    ☺
00000000B0:  00 00 00 00 00 00 00 00   00 00 00 00 00 00 00 00
00000000C0:  00 00 00 00 00 00 00 00   57 00 00 00 01 00 00 00           W    ☺
00000000D0:  0D 00 0B 00 28 FF 0F 00   0B 00 0D 00 2D FF 0B 00  ♪ ♂ ( ✿ ♂ ♪ - ♂
00000000E0:  0B 00 0F 00 29 FF 07 00   41 FF 02 00 11 00 00 00  ♂ ✿ ) ▪ A ☻ ◄
00000000F0:  57 00 01 00 46 80 D0 FF   00 00 00 00 03 03 00 02  W ☺ FA╨       ♥♥ ☻
0000000100:  00 00 00 00 12 05 00 0A   01 00 00 00 12 0A 02 00      ↕♣ ◙☺    ↕◙☻
0000000110:  0A 00 00 00 12 0A 08 0A   00 00 00 00 50 00 00 00  ◙    ↕◙▫◙     P
0000000120:  12 0A 00 07 01 00 00 00   06 03 00 02 02 00 00 00  ↕◙ ▪☺    ♠♥ ☻☻
0000000130:  03 03 00 02 04 00 00 00   03 03 00 02 01 00 00 00  ♥♥ ☻♦    ♥♥ ☻☺
0000000140:  01 00 24 00 05 00 57 69   6E 64 6F 77 73 2F 4E 54  ☺ $ ♣ Windows/NT
0000000150:  20 20 20 20 20 20 20 20   20 20 20 20 20 20 20 20
0000000160:  20 20 20 20 20 20 14 00   06 00 35 2E 31 20 28 32        ¶ ♦ 5.1 (2
0000000170:  36 30 30 29 20 20 20 20   20 20 01 00 00 00 01 00  600)       ☺    ☺
0000000180:  00 00 0B 00 00 00 00 00   00 00 00 B0 00 00 00 00    ♂          ▒
0000000190:  01 00 00 00 00 00 07 00   00 00 0A 00 00 00 30 30  ☺      ▪    ◙    00
00000001A0:  31 30 20 44 45 46 49 4E   45 20 44 41 54 41 20 47  10 DEFINE DATA G
00000001B0:  4C 4F 42 41 4C 20 00 30   30 32 30 20 20 31 20 47  LOBAL  0020  1 G
00000001C0:  4C 4F 42 30 31 20 28 42   31 30 29 20 2F 2A 63 6F  LOB01 (B10) /*co
00000001D0:  6D 6D 65 6E 74 20 6F 66   20 47 4C 4F 42 30 31 20  mment of GLOB01
00000001E0:  00 30 30 33 30 20 20 31   20 47 4C 4F 42 30 32 20   0030  1 GLOB02
00000001F0:  28 41 29 20 44 59 4E 41   4D 49 43 20 2F 2A 63 6F  (A) DYNAMIC /*co
0000000200:  6D 6D 65 6E 74 20 6F 66   20 67 6C 6F 62 30 32 20  mment of glob02
0000000210:  00 30 30 34 30 20 20 31   20 47 4C 4F 42 30 33 20   0040  1 GLOB03
0000000220:  28 41 31 30 2F 31 3A 37   29 20 00 30 30 35 30 20  (A10/1:7)  0050
0000000230:  20 31 20 47 4C 4F 42 30   34 20 28 41 37 29 20 00   1 GLOB04 (A7)
0000000240:  30 30 36 30 20 45 4E 44   2D 44 45 46 49 4E 45 20  0060 END-DEFINE
0000000250:  00 30 30 37 30 20 45 4E   44 20 00 1A              0070 END  →
```

Рис 2.12. Пример скомпилированной GDA

Разбор фиксированной части заголовка показывает, что перед нами действительно GDA (значение 0x47 по смещению 0x02), и нам следует морально подготовиться к 19 записям в Таблице секций (значение 0x13 по смещению 0x18). А что же самой Таблице секций?

NN	Table	Size	Start	End	Align	Commets
00	.Header	00D0	0000	00CF	1	Header Table
01	.Code	0028	00D0	00F7	2	Code Table
02	.Operand	0044	00F8	013B	4	Operand Table
03	.Constant	0006	013C	0141	1	Constant Table
04	.MultPurp	0038	0142	0179	1	Multi Purpose Table
05	.LDA	0000	0000	0000	0	Local Data Area Source Table
06	.PDA	0010	017A	0189	4	Parameter Table
07	.SDA	0000	0000	0000	0	Static constant Data Area Table
08	.ArrDescr	0014	018A	019D	4	Array Description Table
09	.TDA	0002	0000	0000	1	Temp Data Area Table
10	.Attrib	0000	0000	0000	0	Attribute Table
11	.LocPrint	0000	0000	0000	0	Local Printer Table
12	.Include	0000	0000	0000	0	Include Table
13	.ClssDsc	00BE	019E	025B	1	Class Description Table
14	.ODA	0000	0000	0000	0	ODA Table
15	.ExtrnlS	0000	0000	0000	0	External Subroutine Table
16	.Symbol	0000	0000	0000	0	Symbol Table
17	.ADABAS	0000	0000	0000	0	ADABAS command Table
18	.GDA	0057	0000	0000	1	Global Data Area Source Table

Рис 2.13. Таблица секций скомпилированной GDA

Однако! Как говорится, почувствуйте разницу! Две очевидные
секции в скомпилированных LDA и
PDA и «жалких» 10 (пустые секции не в счет) в GDA при их внешней «похожести»! Но давайте по порядку…

Заголовок: с ним все понятно, без него – никуда.
Найти там что-то новое, наверно, нереально.

Секция кода: 40 байт кода – это серьезно. К
сожалению, мы пока не в состоянии понять, что же там происходит (разбору секции кода будет посвящена следующая часть статьи). Но разглядеть и интерпретировать последовательность 46
80 D0 FF нам должно быть уже
вполне по силам – это оператор END в
0070 строке программы (см. первую часть статьи). Догадка перерастет в
убеждение, если мы устремим свой взор на ascii-последовательность,
начинающуюся с 0x0251. Это, конечно, интересно, но
ведь никакого оператора END (впрочем, как и 40 байт
кода) мы не вводили при создании этой глобальной области! Т.е. это – ни что
иное, как «отсебятина» Natural!

Таблица операндов: «раскрутить» операнды мы уже
вполне способны! Нет большого смысла повторяться, привожу конечный результат

Таб 2.12. Таблица операндов

№№	Ссылка	Код секции	Комментарий
1	0x00	00 00 00 00 03 03 00 02	Константа, формат I2 Знач: 01
2	0x02	00 00 00 00 12 05 00 0A	Глоб.переменная, формат B10
3	0x04	01 00 00 00 12 0A 02 00	Глоб.переменная, формат A DYNAMIC
4	0x06	0A 00 00 00 12 0A 08 0A 00 00 00 00	Глоб.переменная, формат A10/1:7
5	0x09	50 00 00 00 12 0A 00 07	Глоб.переменная, формат A7
6	0x0B	01 00 00 00 06 03 00 02	Параметр, формат I2
7	0x0D	02 00 00 00 03 03 00 02	Константа, формат I2 Знач: 0
8	0x0F	04 00 00 00 03 03 00 02	Константа, формат I2 Знач: 1

Обратите внимание на последние 3 операнда – это тоже
«отсебятина» Natural! Ничего подобного мы не объявляли!
Забегая вперед, скажу, что эти операнды непосредственно связаны с секцией кода.
Параметром является некий флаг, который показывает интерпретатору, создавать и инициализировать эту глобальную область, или же она уже существует. А константы

– возможные значения этого флага, с которыми сравнивается текущее значение флага. Но об этом – в следующей части статьи.

Секция описания класса: причем здесь класс, не очень понятно. Тем более что именно в этой секции фактически хранится «человеческое» представление глобальной области.

Область глобальных данных: в самом GP-коде этой секции нет. Почему это «source table» (а это SAG'овское оригинальное название секции) – тоже не очень понятно. Ну и бог с ним! Замечу, что размер секции – это количество байтов, которое необходимо отвести интерпретатору для размещения «глобалки» в памяти.

Заключение

Операнды в GP-коде – тема обширная. И я не ставил перед собой задачи рассказать обо всех возможных операндах в Natural. Что-то – слишком очевидно, что-то – слишком специфично, а чего-то я и сам до сих пор не знаю. Основной побудительный мотив, заставивший меня начать «бодягу» с операндами – идея, что перед тем, как ковырять секцию кода, необходимо хотя бы по-крупному разобраться с операндами. Думаю, это утверждение годится не только для Natural.

Приложение 2

3. Parameter **Table**

Field name	Offset	Size	Comment
ESD index	0x00	4	Индекс параметра
OPT offset or Redef. offset	0x04	4	Ссылка на операнд, соответствующий данному параметру
ESD offset	0x08	2	Используется лишь в случае переопределения и содержит индекс переопределяемого параметра. В других случаях не используется
Flags	0x0B	2	Значения флагов см. во второй части статьи, приложение 2.2

Значения поля Flags

Value	Comment
0x0800	Optional
0x1000	By value result
0x2000	By value
0x4000	Modified
0x8000	Переопределение (redefine)

4. AIV-Table

Вариант 1			
Field name	**Offset**	**Size**	**Comment**
Index of variable	0x00	4	Индекс aiv-переменной
Flags	0x04	2	Флаг
Length variable name	0x06	2	Длина имени aiv-переменной
Variable name	0x08		Имя aiv-переменной (без знака «+»). Размер определен значением поля Length variable name.

Вариант 2(Flags == 0x4000 (redefine))			
Field name	**Offset**	**Size**	**Comment**
Index of variable	0x00	4	Индекс aiv-переменной
Flags	0x04	2	Флаг
Length variable name	0x06	2	Длина имени aiv-переменной
AIV offset: base variable	0x08	4	Ссылка на переопределяемую aiv-переменную
Byte offset to base variable	0x0C	4	Смещение от начала переопределяемой aiv-переменной
Variable name	0x08		Имя aiv-переменной (без знака «+»). Размер определен значением поля Length variable name.

Значения поля Flags

Value	**Comment**
0x4000	Переопределение aiv-переменной (Redefine)
0x8000	CONTEXT variable

5. Array Description Table

«Заголовочная» запись массива

Field name	**Offset**	**Size**	**Comment**
Dimension flags 1	0x00	2	Комбинация флагов 1 размерности (см.п5.1.1)
Dimension flags 2	0x02	2	Комбинация флагов 2 размерности (см.п5.1.1)
Dimension flags 3	0x04	2	Комбинация флагов 3 размерности (см.п5.1.1)
Dimension	0x06	1	Количество размерностей
Flags	0x07	1	Флаги массива (см.п5.1.2)

Значения первого байта Dimension flags

Value (bin)	**Comment**	
0x8000	OPT offset lower	Ссылка на константу в таблице операндов, равную нижней границе соответствующей размерности массива
0x4000	Upper bound value	Явно заданное значение верхней границы соответствующей размерности массива
0x2000	Occurences	Количество элементов соответствующей размерности массива
0x1000	Index factor	Вспомогательный коэффициент, рассчитываемый компилятором, для оптимизации доступа к элементам массива. Показывает, на какое количество байт должен сместиться указатель на массив при приращении индекса соответствующей размерности на 1. Для восстановления объявления массива не используется.
0x0800	Dependent dimension	

Значения поля Flags

Value	Comment
0x01	SetLow1st
0x02	SetLow2nd
0x04	1:V

Исследование «промежуточного» кода на примере GP-кода языка NATURAL - Часть 3: Секция кода

Автор: Konstantin

Дата: 2007-07-03

Разбор секции кода и прост, и сложен. Прост, потому что молитвами разработчиков языка там нет ничего, кроме номеров строк исходника, кодов операций (команд) и операндов. И если с номерами строк все понятно, то с командами и операндами все гораздо запутаннее.

Какие сложности с командами? Во-первых, их несколько сотен; во-вторых, у каждой команды (точнее, группы команд) свой перечень параметров (аргументов), причем параметры могут быть представлены как в префиксном виде, так и постфиксном; в-третьих, команды могут быть размером и в байт, и в два байта.

Какие сложности с операндами? Во-первых, в их многоликости. Дело в том, что в качестве параметров (аргументов) для команд могут быть как операнды, явно присутствующие в Natural-выражениях (именно те, о которых мы говорили в предыдущей части статьи «Часть 2. Операнды в GP-коде», и чтобы не путаться в дальнейшем, условимся называть их nat-операндами), так и всевозможные флаги и просто константы, которых нет в Natural-выражениях, но которые появляются на этапе компиляции (например, «дальность» прыжка операторов семейства JMP, которые обязательно появятся при компиляции любого условного выражения в Natural-программах). Во-вторых, тип постфиксного операнда можно понять только в контексте последней команды – то ли это nat-операнд, то ли это значение флага, то ли константа.

И что со всем этим безобразием делать? Да ничего особенного! Просто набраться терпения – ведь как-то же код выполняется интерпретатором. Хотите знать как?

Смотрите, размер элемента исследуемой секции нам показывает значение поля Align в Таблице секций, соответствующее секции кода – всегда два байта. Перебираем секцию кода по элементам, определяя его «смысловую нагрузку» по его же значению.

```
Если  0000  «  значение элемента   ‘  8000, то перед нами операнд      (1)
Если  8000  ‘= значение элемента   ‘= A8FF, то перед нами номер строки (2)
Если  A8FF  ‘  значение элемента   ‘= F6FF, то перед нами byte-code    (3)
Если  F6FF  ‘  значение элемента   ‘= FFFF, то перед нами word-code    (4)
```

Вот, собственно, и все. Напомню, что под операндом понимается и nat-операнд, и флаг, и константа, и чтобы верно определить тип операнда, необходимо:

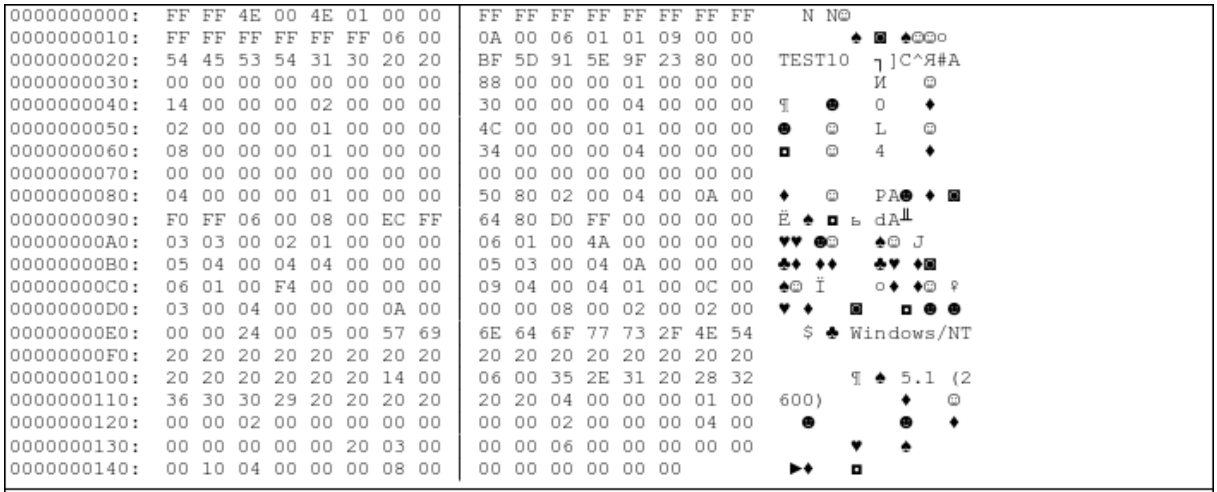
1. а. определить, «чей» это операнд, т.е. определить команду-владельца этого операнда;
2. по прототипу команды определить тип операнда.

Чтобы выйти из шока, вызванного прочтением этих пространных объяснений, скорее запутывающих, нежели объясняющих, разберем два примера, и, думаю, вопросов с пониманием построения GP-кода не должно остаться.

В качестве первого примера вполне подойдет задача про программистов-энтузиастов, которая предлагалась в предыдущей части статьи (Часть 2. Операнды в GP-коде (окончание)).

Усложним условие. Попробуем восстановить исходный код программы полностью, а не только область параметров.

Вот GP-код, который нам надлежит превратить в нечто потребное:



```
0000000000:  FF FF 4E 00 4E 01 00 00 | FF FF FF FF FF FF FF FF    N N
0000000010:  FF FF FF FF FF FF 06 00 | 0A 00 06 01 01 09 00 00
0000000020:  54 45 53 54 31 30 20 20 | BF 5D 91 5E 9F 23 80 00  TEST10
0000000030:  00 00 00 00 00 00 00 00 | 88 00 00 00 01 00 00 00
0000000040:  14 00 00 00 02 00 00 00 | 30 00 00 00 04 00 00 00
0000000050:  02 00 00 00 01 00 00 00 | 4C 00 00 00 01 00 00 00
0000000060:  08 00 00 00 01 00 00 00 | 34 00 00 00 04 00 00 00
0000000070:  00 00 00 00 00 00 00 00 | 00 00 00 00 00 00 00 00
0000000080:  04 00 00 00 01 00 00 00 | 50 80 02 00 04 00 0A 00
0000000090:  F0 FF 06 00 08 00 EC FF | 64 80 D0 FF 00 00 00 00
00000000A0:  03 03 00 02 01 00 00 00 | 06 01 00 4A 00 00 00 00
00000000B0:  05 04 00 04 04 00 00 00 | 05 03 00 04 0A 00 00 00
00000000C0:  06 01 00 F4 00 00 00 00 | 09 04 00 04 01 00 0C 00
00000000D0:  03 00 04 00 00 00 0A 00 | 00 00 08 00 02 00 02 00
00000000E0:  00 00 24 00 05 00 57 69 | 6E 64 6F 77 73 2F 4E 54    $ Windows/NT
00000000F0:  20 20 20 20 20 20 20 20 | 20 20 20 20 20 20 20 20
0000000100:  20 20 20 20 20 20 14 00 | 06 00 35 2E 31 20 28 32        5.1 (2
0000000110:  36 30 30 29 20 20 20 20 | 20 20 04 00 00 00 01 00  600)
0000000120:  00 00 02 00 00 00 00 00 | 00 00 02 00 00 00 04 00
0000000130:  00 00 00 00 00 20 03 00 | 00 00 06 00 00 00 00 00
0000000140:  00 10 04 00 00 00 08 00 | 00 00 00 00 00 00
```

Рис 3.1. Test10.ngn

Воспользуемся результатами, которые были получены нами при решении предыдущей задачи. Нам понадобится Таблица Секций:

NN	Table	Size	Start	End	Align	Commets
00	.Header	0088	0000	0087	1	Header Table
01	.Code	0014	0088	009B	2	Code Table
02	.Operand	0030	009C	00CB	4	Operand Table
03	.Constant	0002	00CC	00CD	1	Constant Table
04	.MultPurp	004C	00CE	0119	1	Multi Purpose Table
05	.LDA	0008	0000	0000	1	Local Data Area Source Table
06	.PDA	0034	011A	014D	4	Parameter Table
07	.SDA	0000	0000	0000	0	Static constant Data Area Table
08	.ArrDescr	0000	0000	0000	0	Array Description Table
09	.TDA	0004	0000	0000	1	TDA Table

Рис 3.2. Test10.ngn (Таблица секций)

И Таблица Операндов (напомню, имена параметров - переменных придумали сами):

Таб.3.1 Таблица операндов

№№	Ссылка	Комментарий
1	0x00	Константа, разм 2 байта, формат I Знач: 01
2	0x02	Параметр, parm1 (n10.2)
3	0x04	Параметр, parm2 (f4) by value
4	0x06	Параметр, parm3 (I4) by value result
5	0x08	Параметр, parm4 (n20.7)
6	0x0A	Временная переменная, 4 байта, формат F

А теперь непосредственно решение. По Таблице Секций определяем, что Секция Кода начинается по смещению 0x88 от начала файла и ее размер составляет 0x14 байт. Вот она:

```
FileOffs  Data
--------- -----------------------------------
00000088: 32 80 02 00 04 00 0A 00 | 2A......
00000090: F0 FF 06 00 08 00 EC FF | Ё ....ь
00000098: 46 80 D0 FF             | FA
```

Рис 3.3. Test10.ngn (Секция Кода)

Разбираем по двухбайтовым элементам, руководствуясь приведенными выше диапазонами значений:

Таб.3.2 Разбор секции кода по элементам

NN	Значение элемента	Сущность элемента	Диапазон значений	Комментарий
01	0x8032	Номер строки	2	0050
02	0x0002	Операнд	1	???
03	0x0004	Операнд	1	???
04	0x000A	Операнд	1	???
05	0xFFF0	Word-code	4	ADD
06	0x0006	Операнд	1	???
07	0x0008	Операнд	1	???
08	0xFFEC	Word-code	4	MULT
09	0x8046	Номер строки	2	0070
10	0xFFD0	Word-code	4	END

С пониманием преобразования номеров строк проблем быть не должно – мы это подробно обсуждали в первой части статьи.

А вот с командами мы знакомы шапочно. Поэтому необходимы небольшие комментарии.

При компиляции, исходный код Natural-программы превращается в GP-модуль, секция кода которого содержит ассемблероподобные инструкции. Это важно! Выражения Natural-программы не просто кодируются, а транслируются в выражения языка более низкого уровня (вспомните, в первой части статьи говорилось о том, что GP – это сокращенное Generated Program – сгенерированная программа). Причем, у этого языка свой самостоятельный синтаксис, более близкий к синтаксису ассемблера, нежели к синтаксису Natural. Команды этого языка имеют свой опкод, свою мнемонику, свою логику (см. Таб. 3.3.).

Таб.3.3 Некоторые команды

Опкод	Мнем.	Разм.	Операнды	Комментарий
Двухбайтовые команды				
FFFF	NOP	Word	-	Нет операции
FFFE	MOVE	Word	1. arg2 (word, prefix, nat-op) 2. arg1 (word, prefix, nat-op)	Поместить arg2 в arg1
FFD0	END	Word	-	Конец программы
FF2C	JGT_R	Word	1. arg3 (word, prefix, nat-op) 2. arg2 (word, prefix, nat-op) 3. arg1 (word, postfix, const)	Условный переход. Пропустить arg1 инструкций, если arg3>arg2
FF13	FOR3_R	Word	1. arg1 (word, postfix, const) 2. arg2 (word, postfix, const)	arg1 - относительное смещение до перехода на конец цикла Arg2 - относительное смещение до перехода на начало цикла
FF6C	ENDLOOP	Word	1. arg1 (word, postfix, const)	Конец цикла. arg1 - флаг, показывающий наличие в цикле обработчика AT BREAK
FF2E	JMP_R	Word	1. arg1 (word, postfix, const)	Безусловный переход. Пропустить arg1 инструкций
FFF0	ADD	Word	1. arg3 (word, prefix, nat-op) 2. arg2 (word, prefix, nat-op) 3. arg1 (word, prefix, nat-op)	Прибавить arg3 к arg2 и поместить в arg1
FFEF	ADDR	Word		Прибавить с округл. arg3 к arg2 и поместить в arg1
FFEE	SUB	Word		Вычесть из arg3 arg2 и поместить в arg1
FFED	SUBR	Word		Вычесть с округл. из arg3 arg2 и поместить в arg1
FFEC	MULT	Word		Умножить arg3 на arg2 и поместить в arg1
FFEB	MULTR	Word		Умножить с округл. arg3 на arg2 и поместить в arg1
FFEA	DIV	Word		Разделить arg3 на arg2 и поместить в arg1
FFE9	DIVR	Word		Разделить с округл. arg3 на arg2 и поместить в arg1
Байтовые команды				
A9	JMP	Byte	1. arg1 (byte, postfix, const)	Безусловный переход. Пропустить arg1 инструкций
AA	JLT	Byte	1. arg3 (word, prefix, nat-op) 2. arg2 (word, prefix, nat-op) 3. arg1 (byte, prefix, const)	Условный переход. Пропустить arg1 инструкций, если arg3 < arg2
AB	JGT			Условный переход. Пропустить arg1 инструкций, если arg3 > arg2
AC	JLE			Условный переход. Пропустить arg1 инструкций, если arg3 <= arg2
AD	JGE			Условный переход. Пропустить arg1 инструкций, если arg3 >= arg2
AE	JNE	Byte		Условный переход. Пропустить arg1 инструкций, если arg3>arg2
AF	JEQ	Byte		Условный переход. Пропустить arg1 инструкций, если arg3=arg2
B0	JTHRU	Byte	1. arg1 (word, prefix, nat-op) 2. arg2 (word, prefix, nat-op) 3. arg3 (word, prefix, nat-op) 4. arg4 (byte, prefix, const)	Условный переход. Пропустить arg4 инструкций, если arg1 входит в диапазон от arg2 до arg3
B1	JNTHRU	Byte		Условный переход. Пропустить arg4 инструкций, если arg1 не входит в диапазон от arg2 до arg3
BE	INC	Byte	1. arg2 (word, prefix, nat-op) 2. arg1 (byte, postfix, const)	Увеличить arg2 на arg1

Но вернемся с небес на землю, точнее, к разбору Секции кода, а еще точнее – к элементу под номером пять (См. Таб.3.2.). Судя по значению элемента (0xFFF0), это команда. По Таб.3.3. определяем, что это ничто иное, как команда ADD. Читаем комментарий: «Прибавить arg3 к arg2 и поместить в arg1». Все аргументы – nat-операнды, точнее ссылки на nat-операнды. В нашем случае arg1 – это Временная переменная (элемент №4), arg2 – это parm2 (элемент №3), arg3 – это parm1 (элемент №2) (см. Таб.3.1). Таким образом, получаем:

```
Временная переменная = parm1 + parm2
```

С этим, вроде бы все понятно. Идем дальше. На очереди элемент под номером восемь. (См. Таб.3.2.). Судя по значению элемента (0xFFEC), это опкод. По Таб.3.3. определяем, что это ничто иное, как команда MULT. Читаем комментарий: «Умножить arg3 на arg2 и поместить в arg1». В нашем случае arg1 – это parm4 (элемент №7), arg2 – это parm3 (элемент №6), arg3 – это ... Если честно, не очень понятно, что делать с arg3, потому что элемент №5 – это разобранная нами ранее команда ADD. Правда, непонятность исчезает, если представить, что операнды перед использованием командами, помещаются в некий стек, и отсчет операндов надо вести не от

номера элемента команды, а от вершины стека. (Эта гипотеза может показаться притянутой за уши, но, тем не менее, она работает, и не только в этом примере). Тогда arg3 – это Временная переменная! Таким образом, получаем:

```
parm4  = Временная переменная * parm3
```

Было бы неплохо избавиться от Временной переменной, т.к. в исходном коде она не объявлялась (иначе она была бы нормальной локальной переменной). Значит, изначально арифметическое выражение выглядело так:

```
parm4  = (parm1 + parm2) * parm3
```

Т.е. восстановленный исходный код программы должен выглядеть так:

```
define data parameter
 1 parm1 (n10.2)
 1 parm2 (f4) by value
 1 parm3 (I4) by value result
 1 parm4 (n20.7)
end-define

parm4 := (parm1 + parm2) * parm3
END
```

И это работает! Мало того, сравнение содержимого старого файла и нового (полученного после компиляции восстановленного исходного кода) показывает, что отличаются только поле TimeStamp заголовков файлов (но мы договорились, что не будем обращать на это поле внимание) и … номера строк инструкций в Секции кода.

А дело вот в чем. На объявление 4 параметров уходит 6 строк исходного коде (4 – на само объявление параметров, 2 – на открытие и закрытие блока объявлений). Таким образом, первое программное выражение не может находиться в строке, меньшей, чем 0070 (Natural нумерует строки автоматически с шагом в 10). Тем не менее, в разбираемой нами программе выражение находится в строке 0050. Наиболее вероятный вариант – это использование параметрической области данных, в которой и объявлены параметры. Но это уже нюансы для тонких ценителей Natural.

В качестве второго примера разберем такой GP-код.

```
0000000000:  FF FF 50 00 2A 01 00 00 | FF FF FF FF FF FF FF FF    P *☺
0000000010:  FF FF FF FF FF FF 07 00 | 06 00 06 01 01 09 00 00        • ♠ ♠☺☺○
0000000020:  46 4F 52 2D 54 53 54 20 | C0 66 D9 ED C7 5D 00 00  FOR-TST └f┘ə╠]
0000000030:  00 00 00 00 00 00 00 00 | 68 00 00 00 01 00 00 00          h   ☺
0000000040:  40 00 00 00 02 00 00 00 | 38 00 00 00 04 00 00 00  @   ●   8   ♦
0000000050:  0A 00 00 00 01 00 00 00 | 40 00 00 00 01 00 00 00  ◙   ☺   @   ☺
0000000060:  04 00 00 00 01 00 00 00 | 32 80 08 00 02 00 FE FF  ♦   ☺   2A◘ ● ■
0000000070:  13 FF 12 00 02 00 04 A9 | 32 80 02 00 03 BE 02 00  ‼ ↕ ● ♦й2A● ♥┘●
0000000080:  0A 00 2C FF 09 00 3C 80 | 02 00 00 00 02 00 EE FF  ◙ , ○ <A●   ● ◘
0000000090:  46 80 2E FF F2 FF 6C FF | 00 00 FF FF 50 80 02 00  FA. Є l     PA●
00000000A0:  04 00 FE FF 5A 80 D0 FF | 00 00 00 00 03 03 00 02  ♦ ■ ZA╨     ♥♥ ●
00000000B0:  00 00 00 00 05 01 00 02 | 02 00 00 00 05 01 00 02     ♣☺ ●●  ♣☺ ●
00000000C0:  02 00 00 00 03 03 00 02 | 04 00 00 00 03 03 00 02  ●   ♥♥ ●♦   ♥♥ ●
00000000D0:  06 00 00 00 03 03 00 02 | 08 00 00 00 03 03 00 02  ♠   ♥♥ ●◘   ♥♥ ●
00000000E0:  01 00 00 00 02 00 0A 00 | 03 00 08 00 02 00 02 00  ☺   ● ◙ ♥ ◘ ● ●
00000000F0:  00 00 24 00 05 00 57 69 | 6E 64 6F 77 73 2F 4E 54    $ ♣ Windows/NT
0000000100:  20 20 20 20 20 20 20 20 | 20 20 20 20 20 20 20 20
0000000110:  20 20 20 20 20 20 14 00 | 06 00 35 2E 31 20 28 32        ¶ ♠ 5.1 (2
0000000120:  36 30 30 29 20 20 20 20 | 20 20                    600)
```

Рис 3.4. For-tst.ngn

На всем, что неоднократно разжевано, не останавливаемся.

NN	Table	Size	Start	End	Align	Commets
01	.Header	0068	0000	0067	1	Header Table
02	.Code	0040	0068	00A7	2	Code Table
03	.Operand	0038	00A8	00DF	4	Operand Table
04	.Constant	000A	00E0	00E9	1	Constant Table
05	.MultPurp	0040	00EA	0129	1	Multi Purpose Table
06	.LDA	0004	0000	0000	1	Local Data Area Source Table

Рис 3.5. For-tst.ngn (Таблица секций)

Таб.3.4 Таблица операндов

№№	Ссылка	Комментарий
1	0x00	Константа, (I2), Знач: 01
2	0x02	Лок. переменная, lvar1 (n2)
3	0x04	Лок. переменная, lvar2 (n2)
4	0x06	Константа, (I2), Знач: 0
5	0x08	Константа, (I2), Знач: 2
6	0x0A	Константа, (I2), Знач: 10
7	0x0C	Константа, (I2), Знач: 3

Секция Кода:

```
FileOffs  Data
--------- -----------------------------------
00000068: 32 80 08 00 02 00 FE FF | 2A....■
00000070: 13 FF 12 00 02 00 04 A9 | . .....й
00000078: 32 80 02 00 03 BE 02 00 | 2A...╛..
00000080: 0A 00 2C FF 09 00 3C 80 | .., ..<A
00000088: 02 00 00 00 02 00 EE FF | ......ю
00000090: 46 80 2E FF F2 FF 6C FF | FA. Є l
00000098: 00 00 FF FF 50 80 02 00 | ..  PA..
000000A0: 04 00 FE FF 5A 80 D0 FF | ..■ ZA╨
```

Рис 3.6. For-tst.ngn (Секция Кода)

Элементы кода секций:

Таб.3.2 Разбор секции кода по элементам

NN	Значение элемента	Сущность элемента	Комментарий
01	0x8032	Номер строки	0050
02	0x0008	Операнд	Ссылка на Константу=2
03	0x0002	Операнд	Ссылка на lvar1
04	0xFFFE	Word-code	MOVE
05	0xFF13	Word-code	FOR3_R
06	0x0012	Postfix.Операнд	Константа, 18
07	0x0002	Postfix.Операнд	Константа, 2
08	0xA904	Byte-code (0xA9)	JMP, параметр = 4
09	0x8032	Номер строки	0050
10	0x0002	Операнд	Ссылка на lvar1
11	0xBE03	Byte-code (0xBE)	INC, параметр = 3
12	0x0002	Операнд	Ссылка на lvar1
13	0x000A	Операнд	Ссылка на Константу=10
14	0xFF2C	Word-code	JGT_R
15	0x0009	Postfix.Операнд	9
16	0x803C	Номер строки	0060
17	0x0002	Операнд	lvar1
18	0x0000	Операнд	Ссылка на Константу=1
19	0x0002	Операнд	Ссылка на lvar1
20	0xFFEE	Word-code	SUB
21	0x8046	Номер строки	0070
22	0xFF2E	Word-code	JMP_R
23	0xFFF2	Postfix.Операнд	-14
24	0xFFC6	Word-code	ENDLOOP
25	0x0000	Операнд	Ссылка на Константу=0
26	0xFFFF	Word-code	NOP
27	0x8050	Номер строки	0080
28	0x0002	Операнд	Ссылка на lvar1
29	0x0004	Операнд	Ссылка на lvar2
30	0xFFFE	Word-code	MOVE
31	0x805A	Номер строки	0090
32	0xFFD0	Word-code	END

Логика разбора совершенно такая же, как и в предыдущем примере: перебираем последовательно все элементы секции, пристально вглядываясь в их значение. Если это команда, то обращаемся к Таб.3.3. По прототипу команды определяем тип операнда (операнды могут располагаться и до команды и после!). Таб.3.4 поможет нам не запутаться в ссылках на nat-операнды. В очередной раз теоретически подковались, а теперь - в путь!

С идентификацией первого элемента проблем быть не должно – это номер строки. Второй и третий элементы – точно операнды, но пока непонятно какие. Четвертый элемент – команда. По Таб.3.3

видим, что это старый знакомый MOVE, а раз так, то второй и третий элементы – ссылки на nat-операнды. Отсюда следует, что перед нами - простое присвоение значения 2 переменной lvar1 (См. Таб.3.4).

Ok, с этим разобрались, идем дальше.

А дальше перед нами команда FOR3_R, которая ничего не делает, а просто предупреждает, что настало время морально приготовиться к разбору цикла FOR. Эта команда имеет два постфиксных параметра. Назначение первого (ближайшего к команде) – указать, где находится безусловный переход на конец цикла, назначение второго – указать на начало программного кода, который должен выполняться в цикле (тело цикла). В качестве указателей используется относительное смещение.

Поупражняемся в арифметике. FOR3_R - элемент №5. Значение его первого аргумента 18 (элемент № 6), второго аргумента - 2(элемент № 7). Отыскиваем конец цикла FOR по первому аргументу.

```
6 (номер элемента) + 18 (значение первого аргумента) = 24
```

Смотрим, 22 элемент – ENDLOOP, что вполне подходит на роль идентификатора конца цикла.

Отыскиваем тело цикла по второму аргументу:

```
7 (номер элемента) + 2 (значение второго аргумента) = 9
```

Смотрим, 9 элемент – номер строки, что тоже вполне подходит на роль начала тела цикла (отметим для себя, что номер строки совпадает с номером строки в первом элементе).

Будем считать, что разбор пятого - седьмого элементов закончен.

Восьмой элемент – байтовая команда JMP, соответствующая безусловному «прыжку» на 4 элемента в сторону увеличения номера элемента, т.е. на 12 элемент.

Девятый элемент – номер строки, ничего нового.

Десятый элемент – операнд, пока непонятно какой.

Одиннадцатый элемент – судя по значению, это байтовая команда, а судя по Таб.3.2, это команда INC. Прототип использования INC говорит о том, что десятый элемент – ссылка nat-операнд, а это никто иной, как старый знакомец lvar1. Таким образом, последовательность из десятого и одиннадцатого элементов увеличивает значение lvar1 на 3 (3 – постфиксный байтовый параметр).

Двенадцатый и тринадцатый элементы – операнды, пока непонятно какие.

Четырнадцатый элемент – команда JGT_R с двумя префиксными параметрами – ссылками на nat-операнды и одним постфиксным параметром – константой. Отсюда, двенадцатый элемент – ссылка опять-таки на lvar1, тринадцатый элемент – ссылка на nat-константу, равную 10. Таким образом, последовательность с двенадцатого элемента по четырнадцатый, соответствует «прыжку» на 9 элементов в сторону увеличения номера элемента (т.е. на элемент 24 - ENDLOOP), если lvar1 > 10.

Шестнадцатый элемент – номер строки, не останавливаемся.

Последовательность с семнадцатого элемента по двадцатый соответствует вычитанию 1 из lvar1 (если вы внимательно читали предыдущие рассуждения, с интерпретацией этой последовательности вопросов возникнуть не должно).

Двадцать первый элемент – номер строки.

Последовательность с двадцать второго элемента по двадцать третий – безусловный переход на 14 элементов в сторону уменьшения номеров элементов.

Последовательность с двадцать четвертого элемента по двадцать пятый – идентификатор завершения цикла. Как и команда FOR3_R, ENDLOOP ничего не делает, а просто сигнализирует, что цикл завершается (настоящее «зацикливание» осуществляется предыдущим переходом).

Двадцать шестой элемент – NOP. Ну, NOP, он и в Африке NOP.

Двадцать седьмой элемент – номер строки.

Последовательность с двадцать восьмого элемента по тридцатый соответствует присвоению переменной lvar2 переменной lvar1.

Тридцать первый элемент – номер строки.

Тридцать второй элемент – старый знакомый END.

По-крупному, картина ясна – есть цикл, создаваемый nat-командой FOR, внутри этого цикла что-то происходит, по выходу из цикла.

Осталось разобраться с параметрами цикла и телом цикла, что совсем несложно.

Начнем с конца цикла, точнее, с безусловного перехода с отрицательным параметром (элемент 22). Как мы уже видели, он направляет на элемент 9, соответствующий номеру строки (причем, такому же, что и номер строки FOR3_R). Далее идет увеличение переменной lvar1 на 3. Причем, такое увеличение происходит на каждой итерации за исключением самой первой! Уж очень это похоже на параметр STEP nat-команды FOR! И еще сильнее убеждает в нашей правоте последующий условный переход (элемент 14), указывающий за цикл (мы уже отмечали, что элемент 24, на который указывает условный переход, соответствует команде ENDLOOP, которая является декларативной). Т.е. этот условный переход – условие выхода из цикла. Если наши рассуждения верны, то интерпретация четвертого элемента (команда MOVE) проста до безобразия – это инициализация переменной, управляющей выполнением цикла.

Тело цикла состоит из одной-единственной команды SUB (элемент 20). Других команд между условием на выход из цикла и «зацикливающим» переходом нет.

Суммируя сведения, полученные запредельной дедукцией, и переводя их на язык NATURAL, уверенно пишем:

```
0010 define data local
0020  1 lvar1(n2)
0030  1 lvar2(n2)
0040  end-define
0050 for lvar1 from 2 to 10 step 3
0060  lvar1 := lvar1 - 1
0070 end-for
0080 move lvar1 to lvar2
0090 end
```

Вот и все!

Конечно, все эти танцы с бубном могут показаться инопланетянским лепетом. Однако при небольшом навыке все выглядит не так уж и страшно. По крайней мере, не страшнее, чем дизассемблерный листинг.

Проницательный читатель должен бы ухмыльнуться – мол, спасибо за пояснение, что, например, 0xFFF0 – это ничто иное, как команда ADD, но было бы неплохо пояснить, откуда это я взял!

Действительно, нет никакой официальной документации по GP-коду. Вся информация добывалась посредством проведения бесчисленного множества экспериментов и исследования natparse.dll. Дело в том, что natparse.dll – это и компилятор, и интерпретатор в одном флаконе. Мало того, в этой же библиотеке находится функция, создающая отчеты компиляции, вполне понятные и заметно экономящие время кодокопателя. Про эти отчеты я уже как-то упоминал - речь идет о параметре копиляции Dump of Natural GP. У этих отчетов есть один недостаток – они формируются

в процессе компиляции, т.е. при наличии исходника. Правда, если эту библиотеку хорошенько «попросить», то она способна выдать набор стандартных отчетов, достаточных как для исследования p-кода, так и для восстановления исходного кода в ручном режиме. Но это уже совсем другая история.

Добавление функциональности в готовые программы

Автор: GMax

Дата: 2007-07-08

Нашей целью будет добавление функциональности в готовую (скомпилированную) программу. Фактически, данной техникой можно добавить функциональность в любую программу, добавить тоже можно все, что вы придумаете. В этом описании не ставилась цель создать какой-либо полезный программный продукт, целью было показать возможности технологии добавления кода в скомпилированную программу. Наша цель - добавить новую функцию в стандартный Блокнот. Добавочной функцией мной была выбрана возможность вычисления выражений. Чтобы было интереснее, возможность вычисления была позаимствована из другой программы.

Итак, имеется:

• Total Calculator v2.02 – TCalc.exe, размер 36 864 байт
• Блокнот – стандартный notepad.exe (Windows XP), размер 66 048 байт.

Добавим функциональность одного в другой.

Порядок наших действий будет такой:

• Реверс инжиниринг TCalc.exe
• Добавление кода и данных в notepad.exe
• Написание кода взаимодействия приложения с добавленным кодом

Реверс инжиниринг TCalc.exe

Прежде всего, нам потребуется код, который мы будем добавлять в блокнот. Дизассемблируем TCalc.exe. Файл небольшого размера, так что разобраться с ним не будет проблем. Про то, как правильно дизассемблировать код можно почитать в других источниках, например в статьях на сайте wasm.ru. Найти участок кода, отвечающий непосредственно за вычисление выражения несложно, он находится по адресу 4037E0 (Разумно будет дать этой функции какое-либо осмысленное имя, например CalcFunc). Процедура не принимает никаких аргументов. Строка с выражением находится по адресу 40B258, результат вычисления расположен по адресу 406450. После того как исходный код будет изучен, делаем File –> Produce file –> Create ASM file и назначаем имя файлу с исходным кодом.

Самый простой и действенный способ выдирания кода - взять нужную функцию, вставить в тестовую программу и запустить компиляцию. Очевидно, что сразу кроме списка сообщений об ошибках мы ничего не получим, но вот с этим списком уже можно работать. Например, если компилятор пишет, что нет какой-либо функции, то просто необходимо добавить её в тестовую программу, а если не хватает данных, то добавить их. Нужно иметь ввиду, что при добавлении нового кода список с ошибками может увеличиться, но в итоге весь требуемый код будет добавлен. Работа, конечно, не из приятных, но при небольшом терпении результат не заставит себя ждать.

Несколько советов: код добавляйте выше кода первой функции, а с данными надо быть предельно внимательным, и если в коде ссылаются на какие-либо данные и сразу за ними идет участок данных к которому нет прямого обращения, то, вероятно, на эти данные идет косвенная адресация, следовательно, они должны быть скопированы в той же последовательности.

Можно пойти дальше и скопировать всю область данных, закомментировать ее, а затем при необходимости убирать комментарии.

После того как весь код будет скомпилирован, нужно подготовить входные данные: во входную строку поместить какое-либо выражение (например: 2+2), затем вызвать функцию sub_4037E0. Далее имеет смысл взглянуть на процесс выполнения под отладчиком. Ставим точку останова на функцию CalcFunc и смотрим, что получается после выполнения по адресу 406450. Если сразу добиться успеха не получилось, погружаемся в медитацию и ищем ошибку, прогоняя тестовую программу и эталонную TCalc.exe.

Опишу несколько ошибок, на которые я потратил время. Во-первых, IDA Pro не дружит с распознаванием констант и смещений в командах для работы с дробными числами. Код вроде этого:

```
.text:004038C8                fstp    qword ptr [edx+413348h]
```

Должен выглядеть так:

```
.text:004038C8                fstp    dbl_413348[ecx]
```

Особенно внимательно нужно просмотреть процедуру sub_402400, там много такого кода.

Также могу посоветовать вызов функции 4015A0 просто убрать – от этого ничего не изменится, так как там отладочная информация.

Также в файле пару раз встречается конструкция switch, с областью данных в конце процедуры

```
  pop ebx
  mov esp, ebp
  pop ebp
  retn
CalcFunc endp
align 4
;off_4042EC dd offset loc_40411A ; DATA XREF: CalcFunc+7A r
;  dd offset loc_4038F0 ; jump table for switch statement
;  dd offset loc_403CE2
;  dd offset loc_403991
;  dd offset loc_403940
;  dd offset loc_4038A5
;  dd offset loc_403883
;  dd offset loc_403861
;  align 10h
```

Я вышел из положения так:

```
  pop ebx
  mov esp, ebp
  pop ebp
  retn
.data
off_4042EC dd offset loc_40411A ; DATA XREF: CalcFunc+7A r
  dd offset loc_4038F0 ; jump table for switch statement
  dd offset loc_403CE2
  dd offset loc_403991
  dd offset loc_403940
  dd offset loc_4038A5
  dd offset loc_403883
  dd offset loc_403861
.code
CalcFunc endp
```

Но можно, например, заменить на 8 операций сравнения и последующих условных переходов.

Кроме того, надо не забыть подключить нужные библиотеки

```
include masm32\include\msvcrt.inc
includelib masm32\lib\msvcrt.lib
```

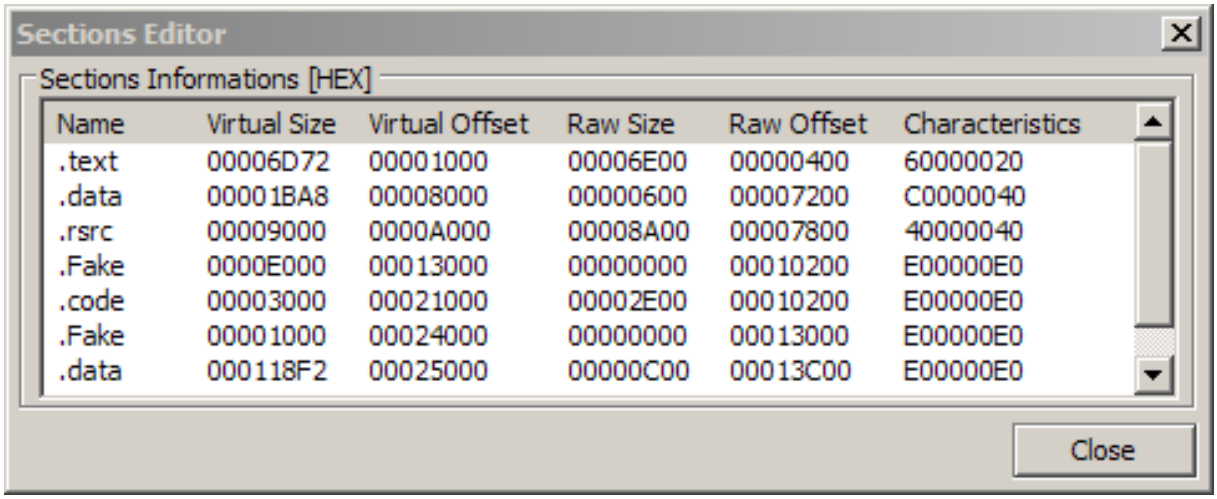
К именам функций atof, *Clacos, _Clasin добавить префикс “crt*”.

Добавление кода и данных в notepad.exe

Теперь у нас есть исходный код, который мы будем добавлять в notepad.exe. Настал момент задуматься над тем, как сделать, чтобы код работал, чтобы все “длинные” переходы были правильными, чтобы данные, на которые ссылается код, были там, где надо. На данный момент базовый адрес нашей тестовой программы 400000h, базовый адрес по которому грузится notepad.exe равен 1000000h, но это нам не помешает.

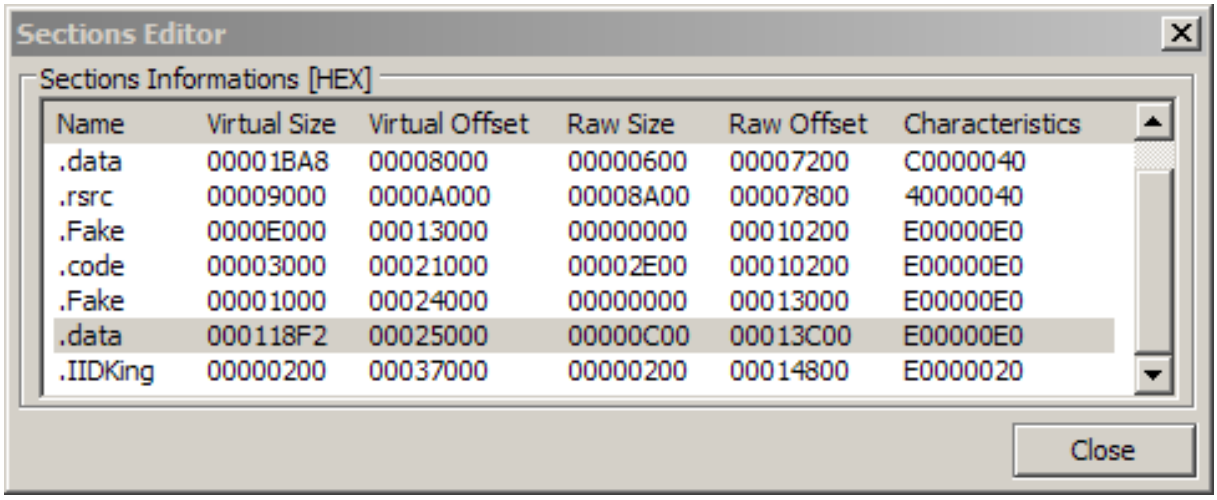
Теперь нам потребуется возможность по редактированию секций PE файла, для этого воспользуемся редактором PE Tools, он поддерживает все необходимые операции по манипулированию секциями. В связи с тем, что секции в памяти должны идти одна за другой, а код мы будем добавлять в конец файла, нам потребуется код с базовым адресом, лежащим за пределами адреса Image Base + Size of Image (1000000h+13000h). Необходимо немного модифицировать наше тестовое приложение, а именно необходимо изменить адрес загрузки нашей программы. Еще раз компилируем программу, на этот раз с ключом /BASE:0x1020000. Выбор числа 0x1020000 связан с тем, что адрес загрузки файла должен быть кратен 64 Кб. Берем получившуюся программу и копируем на диск секцию кода и данных.

Следующим шагом открываем notepad.exe в PE Tools и пробуем расположить секции в памяти так, как они располагались в нашей тестовой программе. Делаем это, манипулируя полями Virtual Size и Virtual Offset каждой секции. Согласно стандарту PE, секции в файле должны идти одна за другой. У меня получилось вот так:

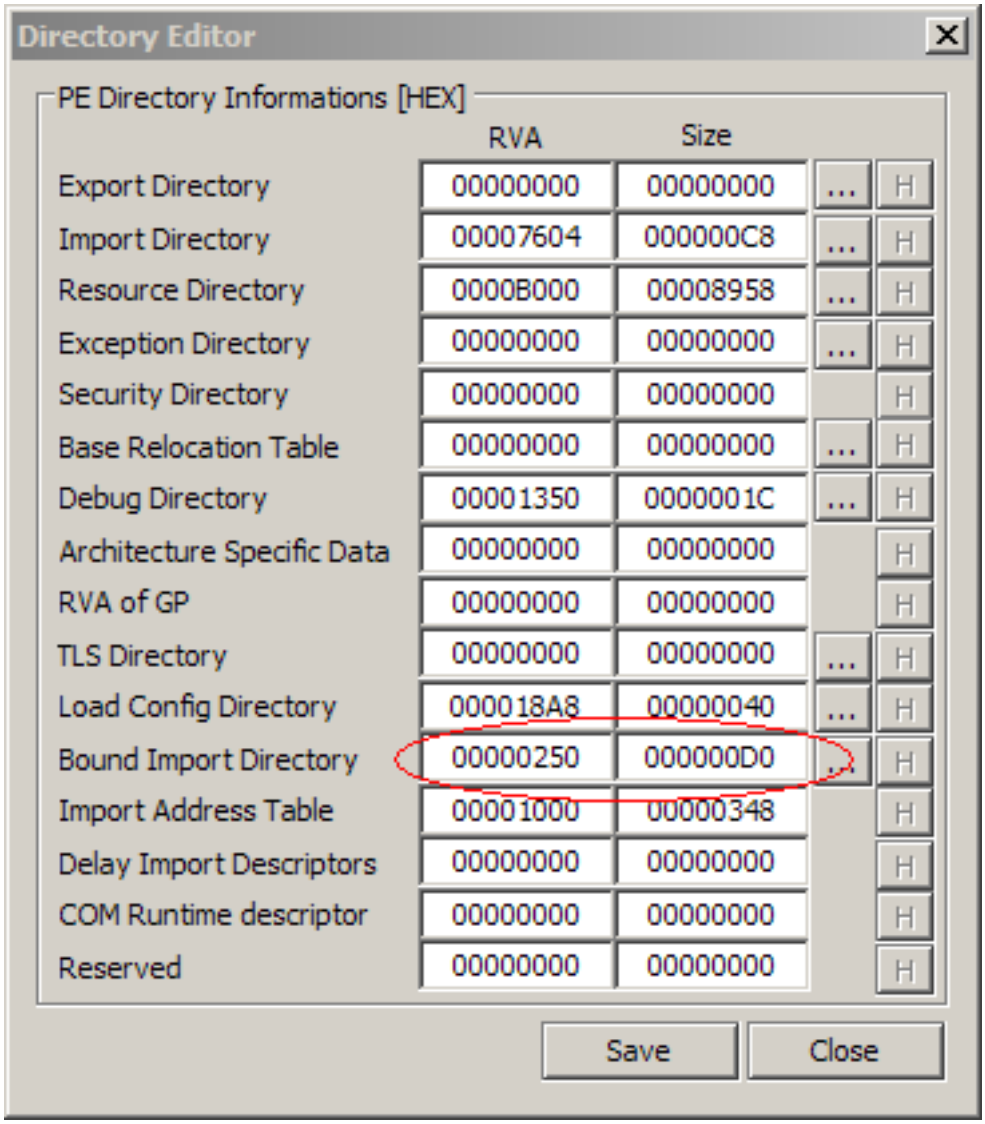


Я поступил следующим образом: просто добавил так называемые Fake секции в те места, где получались разрывы. Это просто секции, заполненные нулями в памяти и не занимающие места на диске.

Следующая проблема - это импорт. Беглый осмотр, показал наличие нескольких функций в тестовой программе, которых нет в notepad.exe. Здесь есть несколько вариантов решения: можно написать небольшую процедуру, подгружающую нужные библиотеки и узнающую адреса нужных функций. Можно просто расширить импорт, и здесь нам поможет программа IIDKing v2.0. Запускаем утилиту и указываем, какие функции, и из какой библиотеки нужно добавить. После того как программа отработала, замечаем, что модифицированный файл notepad.exe увеличился еще на одну секцию, и именно в ней теперь находятся новые функции импорта. После этого надо подправить поле Size of Image в опциональном заголовке.



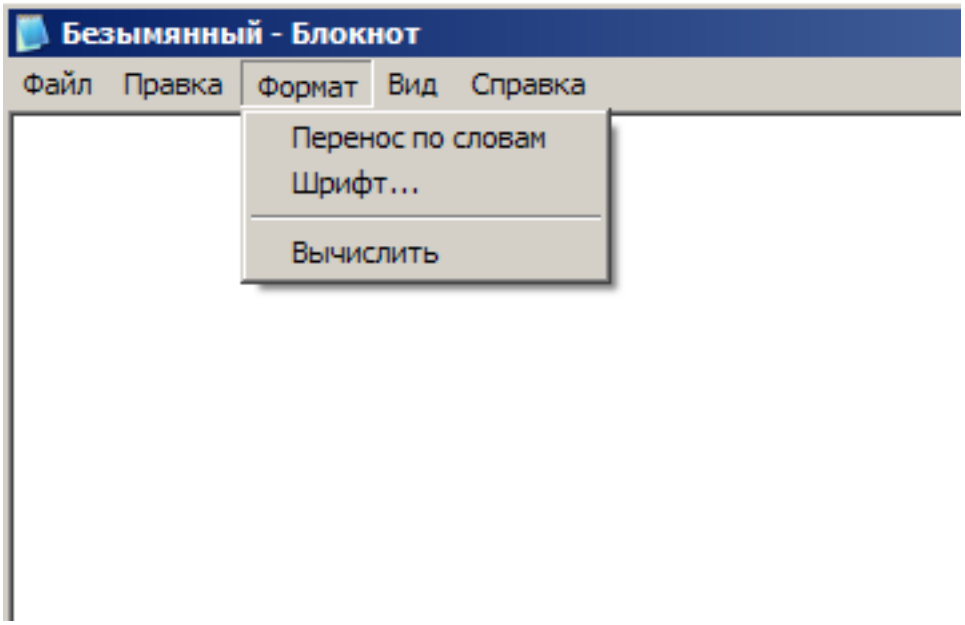
Настало время проверить работоспособность нашего “мутанта”. Запускаем модифицированный блокнот на исполнение. Убеждаемся, что он не работает. Если все сделано, так как я описал, то проблема связанна с “bound import” я просто отключил его и notepad.exe загружался без проблем. Делается это следующим образом: идем в список директорий PE файла и вместо адреса и размера “ Bound Import Directory” прописываем нули.



Написание кода взаимодействия приложения с добавленным кодом

Теперь настало время написать интерфейс взаимодействия блокнота с нашим добавленным кодом. Необходимо найти свободное место в новом файле (это совсем не трудно) и дописать туда код, который будет брать выражение, и передавать его в функцию для вычисления, а затем выводить результат обратно в блокнот. Я решил, что буду использовать пространство в конце секции кода, которую мы вставили из тестовой программы, там, в конце секции остался код, который не будет использоваться в блокноте, его мы и заменим. Как потом выяснилось, это не самое лучшее решение, а втиснуть код в невостребованное пространство оказалось сложно: код влез байт в байт, лучше выбрать свободное место с запасом.

Модифицируем интерфейс нашего блокнота. Я поступил следующим образом: просто добавил новый пункт в меню “Формат”. Сделать это можно любой программой, которая позволяет редактировать ресурсы. Я воспользовался Restorator-ом. Новому пункту меню необходимо дать уникальный идентификатор в пределах всего меню. Я назначил число 34.



Необходимо найти место в программе, где будет сделана вставка кода, который будет отрабатывать при выборе нового пункта меню. Необходимо найти адрес, по которому расположена оконная функция отработки сообщений. И поставим код, производящий прыжок на код обработки:

01002DAA	E9 46090200	JMP 010236F5
01002DAF	90	NOP
01002DB0	90	NOP
01002DB1	90	NOP
01002DB2	90	NOP
01002DB3	48	DEC EAX
01002DB4	0F85 CAFBFFFF	JNZ 01002984

Здесь мы просто вставили “длинный” прыжок JMP 010236F5. Теперь идем по адресу 010236F5 и пишем код обработки выбора пункта меню “Вычислить”. Далее я приведу код, а затем эго прокомментирую:

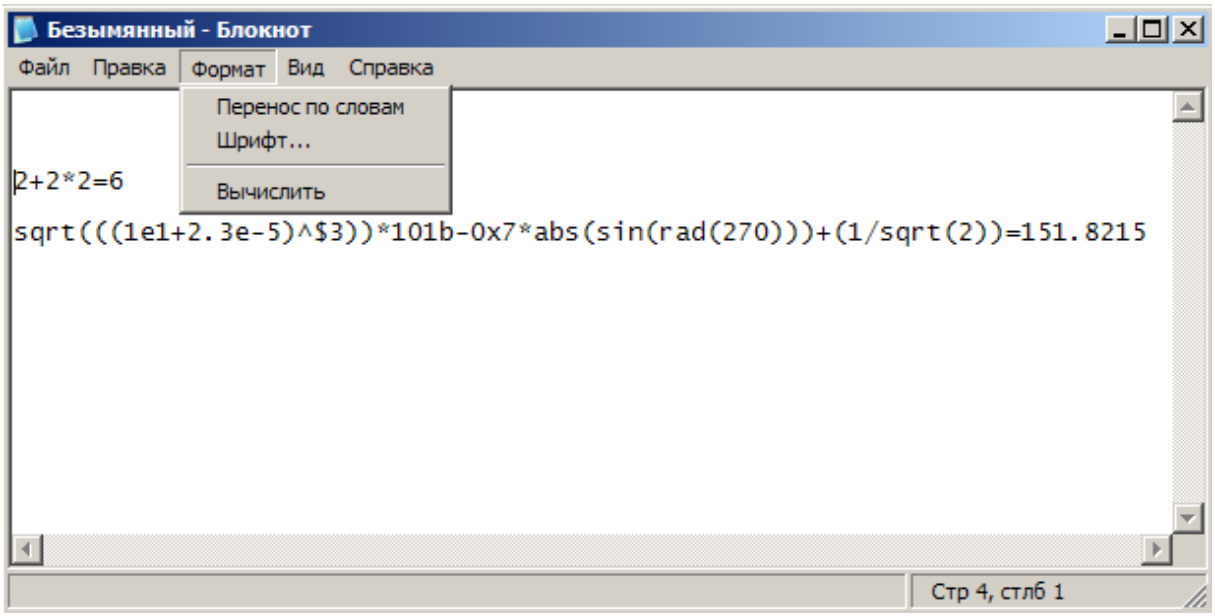
```
010236F5  83E8 05          SUB EAX,5
010236F8  0F84 96F7FDFF    JE notepad.01002E94
010236FE  83F8 02          CMP EAX,2
01023701  0F85 ACF6FDFF    JNZ notepad.01002DB3
01023707  6A 00            PUSH 0
01023709  6A FF            PUSH -1
0102370B  68 C9000000      PUSH 0C9          ;EM_LINEFROMCHAR
01023710  FF35 18880001    PUSH DWORD PTR DS:[1008818]
01023716  FF15 20120001    CALL DWORD PTR DS:[<&USER32.SendMessageW>;
0102371C  66:C705 F44B0201 MOV WORD PTR DS:[1024BF4],968
01023725  68 F44B0201      PUSH notepad.01024BF4
0102372A  50               PUSH EAX
0102372B  68 C4000000      PUSH 0C4          ; EM_GETLINE
01023730  FF35 18880001    PUSH DWORD PTR DS:[1008818]
01023736  FF15 20120001    CALL DWORD PTR DS:[<&USER32.SendMessageW>;
0102373C  3E:C70445 F34B02 MOV DWORD PTR DS:[EAX*2+1024BF3],0
01023748  33C0             XOR EAX,EAX
0102374A  50               PUSH EAX
0102374B  50               PUSH EAX
0102374C  50               PUSH EAX
0102374D  50               PUSH EAX
0102374E  6A FF            PUSH -1
01023750  68 F44B0201      PUSH notepad.01024BF4
01023755  50               PUSH EAX
01023756  50               PUSH EAX
01023757  FF15 44110001    CALL DWORD PTR DS:[<&KERNEL32.WideCharToMultiByte>;
0102375D  33C9             XOR ECX,ECX
0102375F  51               PUSH ECX
01023760  51               PUSH ECX
01023761  68 C8030000      PUSH 3C8
```

```
01023766  68 F45B0201      PUSH notepad.01025BF4
0102376B  6A FF            PUSH -1
0102376D  68 F44B0201      PUSH notepad.01024BF4
01023772  51               PUSH ECX
01023773  51               PUSH ECX
01023774  FF15 44110001    CALL DWORD PTR DS:[<&KERNEL32.WideCharToMultiByte>;
0102377A  E8 3BF4FFFF      CALL notepad.01022BBA
0102377F  68 2C5B0201      PUSH notepad.01025B2C
01023784  FF35 C05F0201    PUSH DWORD PTR DS:[1025FC0]
0102378A  FF35 BC5F0201    PUSH DWORD PTR DS:[1025FBC]
01023790  E8 93000000      CALL notepad.01023828
01023795  68 C8030000      PUSH 3C8
0102379A  68 F44B0201      PUSH notepad.01024BF4
0102379F  6A FF            PUSH -1
010237A1   68 2C5B0201     PUSH notepad.01025B2C
010237A6  6A 00            PUSH 0
010237A8  6A 00            PUSH 0
010237AA  FF15 9C100001    CALL DWORD PTR DS:[<&KERNEL32.MultiByteToWideChar];
010237B0  68 EB370201      PUSH notepad.010237EB
010237B5  6A 01            PUSH 1
010237B7  68 C2000000      PUSH 0C2          ; EM_REPLACESEL
010237BC  FF35 18880001    PUSH DWORD PTR DS:[1008818]
010237C2  FF15 20120001    CALL DWORD PTR DS:[<&USER32.SendMessageW>;
010237C8  68 F44B0201      PUSH notepad.01024BF4
010237CD  6A 01            PUSH 1
010237CF  68 C2000000      PUSH 0C2          ; EM_REPLACESEL
010237D4  FF35 18880001    PUSH DWORD PTR DS:[1008818]
010237DA  FF15 20120001    CALL DWORD PTR DS:[<&USER32.SendMessageW>;
010237E0  B8 00000000      MOV EAX,0
010237E5  -E9 C9F5FDFF     JMP notepad.01002DB3
010237EA  90               NOP
010237EB  3D 00000000      CMP EAX,0
```

Первые 2 строчки этого кода просто взяты с того места, где мы добавили код для прыжка на вставленный код (там он затерся), следующие 2 строчки проверка на то, что выбран именно

добавленный пункт меню. Далее идут два вызова функции SendMessageW с сообщением EM_LINEFROMCHAR, первый раз, чтобы узнать на какой строчке установлен курсор ввода, второй раз с сообщением EM_GETLINE, чтобы получить строку с выражением. После этого по адресу 01024BF4 находится выражение для вычисления, но тут нас ждет одна неприятность: текст в буфере находится в UNICOD кодировке, а наша функция для вычисления выражений работает только с ANSI форматом строк. Дальнейший код это исправит, функция WideCharToMultiByte переводит UNICOD строки в формат ANSI. Функция вызывается 2 раза: первый раз, чтобы определить размер строки, затем для преобразования. Сразу за этим следует вызов функции для вычисления выражения. Следующий код делает обратное преобразование из ANSI в UNICOD с помощью функции MultiByteToWideChar. Далее идет вызов функции SendMessageW с сообщением EM_REPLACESEL для вставки строки “=” (строка находится по адресу 010237EB), а затем строки с результатом выражения.

Вот собственно и все! Мы получили работающий файл. Блокнот с расширенной функциональностью.



Здесь (находится в архиве `wasm_files.zip`: `files/0456/AddFunc.rar`) можно скачать файлы к статье: исходный код, добавляемого кода и конечный результат.

Простой обход фаервола KIS 7

Автор: Mental_Mirror

Дата: 2007-10-25

Привет всем. Сегодня поговорим о простом, но, тем не менее, эффективном методе обхода фаервола Kaspersky Internet Security 7.0 – сетевого экрана, компонента одной из самых популярных и известных защит для персонального пользователя на сегодняшний день. Кто-то хвалит данный продукт, кто-то им восхищается, а кто-то откровенно недолюбливает =) Есть за что или нет, решать Вам. Тут будут только факты. Рассмотрим самую свежую на данное время версию 7.0.0.125. Факт обхода будет заключаться в скрытой отправке данных, лик, утечка - ну как угодно. Доказательства концепции не будет, дабы не сильно радовать наших младших братьев – скрипт-киддисов. Кому надо код можно написать вместе с отладкой за 30 минут. Меньше слов – больше дела.

Существует очень известная и чрезвычайно распространенная техника – инжект. О ней много написано, и большинство из того, что написано, уже детектируется нашем сегодяшним подопытным кроликом. Суть техники сводится к исполнению кода уже доверенного приложения, каким угодно способом. Большинство пользователей, у которых есть Интернет пользуются браузером, а стало быть и серфят Интернет с помощью браузера, а следовательно, должно быть правило в сетевом экране, которое разрешает браузеру по умолчанию передавать данные на 80 порт любому серверу. Отлично, будем этим пользоваться – инжектим код в браузер, и от его имени передаем или получаем данные. Ну, это-то знают все =) А как инжектится? Как обойти преграду проактивной защиты KIS? Просто, даже очень.

Итак, сам процесс инжекта состоит из двух этапов – 1) запись кода 2) передача ему управления. Запись кода… Тут можно много чего придумать, но я расскажу всего один секрет применимый исключительно для KIS. Вообще запись кода в чужой процесс осуществляется функцией KERNEL32!WriteProcessMemory, спускаясь ниже - ее нативным аналогом NTDLL!NtWriteVirtualMemory. Параметры конечно рассказывать не буду, для этого есть исходные источники – IDA =), шутка – Micro$oft. Обычно попытки запись в чужой процесс KIS предупреждает, но не всегда. Взлом любой защиты основан на ее допущениях. Допущение KIS – возможность записи в область виртуальной памяти, НЕ помеченной как исполняемая, конкретно – у области нет атрибута EXECUTABLE. Если данного атрибута нет – то можно писать свободно – KIS и ухом не поведет. Этот факт был получен чисто экспериментальным путем. Отлично. Запись мы сделаем. Но вот куда? Есть простое решение – это область памяти называемая стеком. Но тут кроется самое интересное – записывая код в определенное место в стеке мы убиваем одном выстрелом двух зайцев, а зайцы эти - как раз два пункта процесса инжекта. Все, наверное, уже догадались, о чем идет речь. Кто не догадался - читает дальше. С записью итак понятно, выполнять код будем тоже записью в стек – в стек потока, который дальше будет исполнять наш код. Дело в том, что потоки постоянно вызывают функции и с очень большой вероятностью адрес возврата записывают в стек. Подменив адрес возврата на свой, мы заставим поток исполнять наш код, который также записан в стеке. Знаю, что код в стеке не всегда исполняется, учитывая современные тенденции. Ну, записать код можно и не только в стек. Оставлю это Вам. Т.о. план простого инжекта вычерчивается очень четко следующими пунктами

1. Создаем процесс, на основе исполняемого файла браузера по умолчанию
2. После некоторого времени исполнения останавливаем поток браузера
3. Получаем его контекст – нас интересует значение указателя стека
4. Анализируем стек на предмет адреса возврата
5. Пишем адрес нашего кода, который будет что-то качать или передавать по адресу в стеке, где находиться адрес возврата
6. Возобновляем исполнение потока

В этом алгоритме есть один возможно не самый понятный момент – как анализировать стек на предмет адреса возврата. Я использую следующий метод – двигаемся «вниз» по стеку, и получаем DWORD'ы. Каждый из них может быть адресом возврата. Это можно проверить - читаем по адресу, который соответствует данному DWORD'у, и дизассембируем по адресам вниз. Если там находится инструкция CALL, значит это и есть адрес возврата. Маленькая иллюстрация:

```
 CALL X
RET_ADDRESS:


 PROC X
 ….
```

Пусть мы остановили поток во время исполнения функции X, в стеке присутствуют (или могут) локальные переменные, сохраненные неизменяемые регистры (при STDCALL), просто данные помещенные в стек (временно) и конечно адрес возврата. Адрес возврата – это RET_ADDRESS. Его значение мы должны найти в результате анализа стека. При этом можно предположить, что при анализе можно полагаться на значение EBP, как правило, тоже сохраняемое в стеке – тут полезно вспомнить стандартный код пролога многих функций. И значение EBP мы можем получить из контекста. Но практика показала, что к этому нельзя привязываться, т.к. стек останавливаемого потока в контексте вызова конкретной функции может содержать просто адрес возврата и больше ничего. Дизассемблер длин плюс разбор по полям, понятно, нам для реализации техники непременно потребуется. Пустяк. Что получилось в итоге… Получился псевдокод мнимого языка с мнимыми функциями для простого обхода KIS 7:

```
CreateProcess(BrowserFilePath,…,ProcessInformation); // Создаем процесс
Sleep(1000);       // Поспим немного
SuspendThread(ProcessInformation.hThread);    // Приостанавливаем первично-порожденный поток
GetThreadContext(ProcessInformation.hThread,&Context);// Получаем контекст
ReadProcessMemory(ProcessInformation.hProcess,Context.Esp,StackChunk,1000*4);// Читаем 1000
    DWORD'ов с ESP
for (i=0;i<1000;i++) // анализ стека потока
{
 ReadProcessMemory(ProcessInformation.hProcess,StackChunk[i]-10,CodeBuffer,10);
 // читаем 10 байт – если это адрес возврата, то эти 10 байт содержат инструкцию CALL
 for (j=0;j<10;j++)
 {
  disasm(&(CodeBuffer[9-j]),&Instr);  // разбор инструкции «назад»
  if ( ( Instr.Length == j+1 ) &&
   (Instr.OpCode == 0xe8 || Instr.OpCode == 0xff || Instr.OpCode == 0x9a)
     ) // это CALL?
  {
   WriteProcessMemory(ProcessInformation.hProcess,
        (Context.Esp+StackChunk[i] -  StackChunk),&ShellCodeAddress,4);
   // пишем адрес нашего shell-кода
   ResumeThread(ProcessInformation.hThread); // возобновляем исполнение потока
  }
 }
}
```

Респект **dead_body** из команды WASM за вклад в идею обхода. Памяти безвременного ушедшего кудесника кода Ms-Rem'а. Ждем официальной реакции команды разработчиков KIS. До скорой встречи. mailto – mental_mirror@freed0m.org

Искусство редактирования интерфейса программ на VB

Автор: GPcH

Дата: 2008-10-01

Вступление

Опытным реверсерам или русификаторщикам наверняка попадались программы, написанные на VB. Что удивительно, про редактирование форм и контролов на них вообще на данный момент нет никакой информации. Это надо исправлять, чем мы и займемся :)

Вообще, русификаторщики, реверсеры и просто продвинутые пользователи :) давно привыкли юзать Restorator или Resource Hacker для редактирования интерфейса программ. К сожалению, эти утилиты никак не видят ресурсы VB программ. Формат, в котором они хранятся, в принципе, не сложный, но кому охота писать отдельный редактор ресурсов чисто под VB? Что поделать, придется изучать этот формат самим. Для начала разберем, как найти эти самые ресурсы в VB проге. Начнем с рассмотрения оригинальной точки входа в программу. Чтобы на нее перейти из HEX редактора HIEW нам потребуется лишь загрузить EXE'шник в данный HEX редактор и нажать поочереди Enter, Enter, F8, F5. У кого хиев купленный, знают как оптимизировать эту операцию до командной строки, но это так, к слову. Нашему взгляду будет представлено примерно следующее:

```
push 0004042E8 ;'VB5!'
call ThunRTMain ;MSVBVM60 --?2
```

Теперь считываем структуру VBHeader по адресу 0004042E8. Как бы это не было парадоксально - больше нам для исследования никакие сторонние структуры не потребуются - все нужное мы выцепим из VBHeader.

Структура VBHeader

Поле — Тип — Описание

Signature — String * 4 — Сигнатура "VB5!"

RuntimeBuild — Integer — Показатель рантаймовости

LanguageDLL — String * 14 — Языковая библиотека

BackupLanguageDLL — String * 14 — Не влияет на работу EXE

RuntimeDLLVersion — Integer — Версия рантайм библиотеки

LanguageID — Long — Язык программы

BackupLanguageID — Long — Используется совместно с LanguageDLL

aSubMain — Long — Main процедура, запускаемая при старте EXE. Если отсутствует, то при загрузке грузится самая первая форма

aProjectInfo — Long — Указатель на структуру ProjectInfo

fMDLIntObjs — Long

fMDLIntObjs2 — Long

ThreadFlags — Long — Флаги потока

ThreadCount — Long — Число потоков (смысл малопонятен, так как VB не позволяет создавать многопоточные программы)

FormCount — Integer — Число форм в данном файле

ExternalComponentCount — Integer — Число внешних OCX компонентов

ThunkCount — Long

aGUITable — Long — Указатель на GUITable

aExternalComponentTable — Long — Указатель на ExternalComponentTable

aComRegisterData — Long — Указатель на ComRegisterData

oProjectExename — Long — Адрес строки с именем EXE файла

oProjectTitle — Long — Адрес строки с заголовком проекта

oHelpFile — Long — Адрес строки с именем Help файла

oProjectName — Long — Адрес строки с именем проекта

Мощная структура, не правда ли? Все элементарно! Нам потребуется только FormCount для определения числа форм и указатель на структуры описывающие формы - aGUITable. Структура GUITable имеет вид:

Структура tGuiTable

Поле — Тип — Описание

SectionHeader — Long — Адрес заголовка описывающего секции

unknown(59) — Byte — Неиспользуемый блок байт

FormSize — Long — Размер блока, описывающего форму и контролы, лежащие на ней

un1 — Long — Неиспользуемый DWORD

aFormPointer — Long — Указатель на блок, описывающий форму и контролы, лежащие на ней

un2 — Long — Неиспользуемый DWORD

Таких структур столько же, сколько форм в проекте, и они идут одна за одной. Чтобы получить адрес начала формы, нужно к aFormPointer прибавить 93. Этот адрес должен указывать на длину информации о форме. Тут есть небольшая хитрость - он может занимать 2 либо 4 байта. Если считанный DWORD отANDить с &H80000000 то мы определим число байт информации. Если DWORD содержит флаг &H80000000 то длина записана в 4 байта, иначе в два. После длины идет собственно описание формы и лежащих на ней контролов. Наконецто мы нашли то, что нам нужно, пора бы разобраться с бинарным форматом форм и контролов.

Бинарный формат формы

Когда-то во времена VB 1.0 for DOS все формы по умолчанию сохранялись в бинарном формате, и это не считалось ненормальным. Теперь же, когда мы привыкли, что frm файлы можно редактировать прямо в блокноте, трудно представить, что эти же формы можно представить в упакованном бинарном формате. Почему упакованном? А потому, что чтобы узнать информацию о последнем контроле на форме, нужно последовательно пропарсить все предыдущие контролы. Из этого следует, что чтобы добавить новое свойство контролу придется перепаковать всю структуру. То есть сначала декомпилировать ее, потом изменить и снова скомпилировать, как это делает VB. Сложно, не спорю, но что поделать. Самое сложное если на форме лежит ActiveX или UserControl, который нужно выделить, чтобы не изменить неизвестных его свойств. Все это оказало сильное впечатление на программеров, потому по сей день нет ни одного нормального редактора интерфеса VB программ и русификаторы к VB прогах практически не делают. Надеюсь ты прочитав эту статью разберешься, как устроены формы VB и как их разбирать и заново собирать. В бинарном упакованном виде каждый объект начинается со свойства Name и заканчивается идентификатором, по которому можно узнать идут ли дальше другие объекты, вложенность

объектов и их завершение, а также меню. Свойства чередуются крайне просто. Сначала идет идентификатор свойства, потом само значение, затем следующий идентификатор. Идентификаторы FF00-FF05 зарезервированы. Вот их описание:

```
Public Const vbFormNewChildControl = &H1FF;
Public Const vbFormExistingChildControl = &H2FF;
Public Const vbFormChildControl = &H3FF;
Public Const vbFormEnd = &H4FF;
Public Const vbFormMenu = &H5FF;
```

Теперь надо решить одну проблемку - откуда же нам брать идентификаторы всех свойств всех контролов? Все просто - я уже составил таблицу, которая мной была получена поутем выдирания этих свойств из TypeLib'ов VB и их многочисленными исправлениями. Таблицу берите на диске и давайте приступать к реальному примеру.

```
00 00 00 00-00 00 00 00-00 00 04 00-00 00 0D 00       ??  ??   ?
41 43 5F 45-78 44 65 63-5F 30 33 5F-42 00 0D 01  AC_ExDec_03_B ??
27 00 43 72-61 63 6B 6D-65 20 66 6F-72 20 4A 6F  ' Crackme for Jo
73 65 70 68-43 6F 27 73-20 45 78 44-65 63 20 50  sephCo's ExDec P
72 6F 67 72-61 6D 2E 2E-2E 00 03 08-00 00 80 19  rogram... ??  ??
01 00 42 00-23 3E 04 00-00 6C 74 00-00 36 04 00  ? B #>?  lt  6?
00 00 00 01-00 02 00 20-20 10 00 00-00 00 00 E8     ? ?   ?     ?
02 00 00 26-00 00 00 10-10 10 00 00-00 00 00 28  ?  &   ???     (
01 00 00 0E-03 00 00 28-00 00 00 20-00 00 00 40  ?  ??  (       @
00 00 00 01-00 04 00 00-00 00 00 80-02 00 00 00     ? ?     ??
00 00 00 00-00 00 00 00-00 00 00 00-00 00 00 00
00 00 00 00-00 80 00 00-80 00 00 00-80 80 00 80       ?  ?   ?? ?
00 00 00 80-00 80 00 80-80 00 00 80-80 80 00 C0     ? ? ??  ??? +
C0 C0 00 00-00 FF 00 00-FF 00 00 00-FF FF 00 FF  ++
00 00 00 FF-00 FF 00 FF-FF 00 00 FF-FF FF 00 00
```

0D указывает на то, что имя формы содержит 0Dh символов. Далее идет имя "AC_ExDec_03_B", зевершающееся нулевым байтом, после которого опять стоит 0Dh. Следующий байт 01h определяем по таблице для формы - это Caption, следовательно после него должна идти длина строки и сама строка. Со строками у VB не все гладко - строки в некоторых свойствах объектов он хранит в ACIIZ формате, а в некоторых - в Unicode формате. Распознать формат нереально; единственный способ, просто запомнить те свойства которые имеют Unicode формат, а какие нет. К примеру Caption и Name всегда ASCII, а вот Tag, Connect и некоторые другие в Unicode формате. Вернемся к нашим данным. 03 - BackColor согласно нашей таблице, следовательно следующие 4 байта отвечают за 32 битный код цвета. Далее идет 19 - ScaleMode, следующий за ним Word определяет масштаб. 42 - WhatsThisButton, за ним 1 байт определяющий логическое True (FF) или False (0). Теперь рассмотрим самое интересное, что есть в формах - слудующий байт 23. Это Icon. Вообще при программировании на VB формы хранятся в файле frm,а вот графика и прочие большие данные хранятся в frx. frm в свою очередь ссылается на определенный адрес в этом frx, храня в котором все используемые данные простосто один за другим. После компиляции содержимое frx встраивается в форму, поэтому в рассматриваемом случае после байта 23 будет идти иконка в формате stdole.Picture. Если иконка берется дефолтная из MSVBVM60.DLL, то после 23 будет FFFFFFFF, иначе будет размер картинки. Именно столько байт мы должны считать после адреса, чтобы получить всю используемую иконку. 3E 04 00-00 = 43E = 1086 байт. Именно через столько байт кончится иконка и продолжится форма, которую мы декомпиляем:

```
FF 00 00 35-FF 00 00 24-05 00 46 6F-72 6D 31 00     5   $? Form1
35 3C 00 00-00 59 01 00-00 CC 15 00-00 03 0C 00  5<   Y?  ¦§  ??
00 46 03 FF-01 55 00 00-00 01 06 00-46 72 61 6D   F? ?U   ?? Fram
65 31 00 03-01 11 00 42-75 74 74 6F-6E 20 69 73  e1 ??? Button is
20 69 6E 20-68 65 72 65-00 03 00 00-00 00 04 FF   in here ?    ?
FF FF 00 05-78 00 A0 05-AF 14 37 05-12 01 00 1B     ?x ???¶7??? ?
01 00 00 00-BC 02 A4 2C-02 00 0E 43-65 6E 74 75  ?   +??,? ?Centu
72 79 20 47-6F 74 68 69-63 FF 01 2B-00 00 00 03  ry Gothic ?+   ?
08 00 43 6F-6D 6D 61 6E-64 31 00 04-01 09 00 45  ? Command1 ??? E
6E 61 62 6C-65 20 4D 65-00 04 78 00-58 02 BF 13  nable Me ?x X?+?
EF 01 11 02-00 FF 02 03-AE 00 00 00-02 06 00 4C  ????  ???   ?? L
```

```
61 62 65 6C-31 00 01 01-6A 00 41 63-69 64 5F 43  abel1 ??j Acid_C
6F 6F 6C 5F-31 37 38 27-73 20 45 78-44 65 63 20  ool_178's ExDec
43 72 61 63-6B 6D 65 20-30 33 2E 42-2C 20 6A 75  Crackme 03.B, ju
73 74 20 65-6E 61 62 6C-65 20 74 68-65 20 62 75  st enable the bu
74 74 6F 6E-2E 2E 2E 20-4D 61 79 62-65 20 74 68  tton... Maybe th
69 73 6F 6E-65 20 69 73-20 61 62 69-74 20 65 61  isone is abit ea
73 69 65 72-20 74 68 61-6E 20 45 78-44 65 63 20  sier than ExDec
30 33 2E 41-00 03 00 00-00 00 04 FF-FF FF 00 05  03.A ?    ?    ?
78 00 78 00-AF 14 47 04-12 00 00 25-01 00 00 00  x x ?¶G??  %?
BC 02 A4 2C-02 00 0E 43-65 6E 74 75-72 79 20 47  +??,? ?Century G
6F 74 68 69-63 FF 02 04-50 00 00 00-2E F4 B5 01  othic ??P   .?¦?
C9 42 34 4B-9A 3F 43 B2-41 04 7C 5E-00 00 00 00  +B4K??C¦A?|^
```

Теперь видим 24. Это LinkTopic, после которого идет строка, строки мы уже умеем достать потому продолжим. 35 - этого опкода в таблице нет, но я тебе расскажу - это всего лишь линейные размеры клиентской части формы. За байтом 35 идут 4 dword'a. Это соответственно ClientLeft, ClientTop, ClientWidth, ClientHeight. Затем видим 46 - StartUpPosition. Это один байт, определяющий позицию формы при запуске (в центре экрана, где получится или в центре Parent формы. Теперь мы дошли до самого интересного - FF01. Помнишь, я говорил про константы, определяющие конец одних контролов или начало других? Так вот FF01 - это vbFormNewChildControl определяет что далее идет контрол, контейнером для которого является форма. Сначало стандартно dword размер информации о следующем контроле, затем имя контрола и пошли свойства. 01 - Caption, 03 - BackColor, 04 - ForeColor, 05 - линейные размеры, декомпиляются подобно линейным размерам клиентской части формы, с одной небольшой разницей - каждый из размеров занимает не 4, а 2 байта. Продолжим 12 - TabIndex, индекс, используемый для перечисления контролов на формы при нажатии Tab. Многие программисты забывают его проставить после разработки программы, отсюда любители работать только с использованием клавы плюются и ругают программу и руки автора. Причем я их в этом поддерживаю, это свойство программист обязан выставить - это правило профессиональной разработки интерфейсов. Так вот этот индекс определяется двумя байтами, и это означает, что более 65535 контролов на форму не поместить. Затем идет 1B. Это одно из самых интересных свойств - Font. В отличии от других оно описывается классом stdole.Font,который есть только в VB. Поэтому писать декомпилятор VB не на VB это большой геморрой именно из-за классов, зашитых в библиотеки VB. В конце всех контролов мы увидим FF0204 Как мы помним 02 это vbFormExistingChildControl, то есть надо закрыть контрол и 04 это vbFormEnd закрывает форму. Вот что получится, если бы мы записывали то, что декомпиляли в уме (листинг взят из моего декомпилятора VB Decompiler):

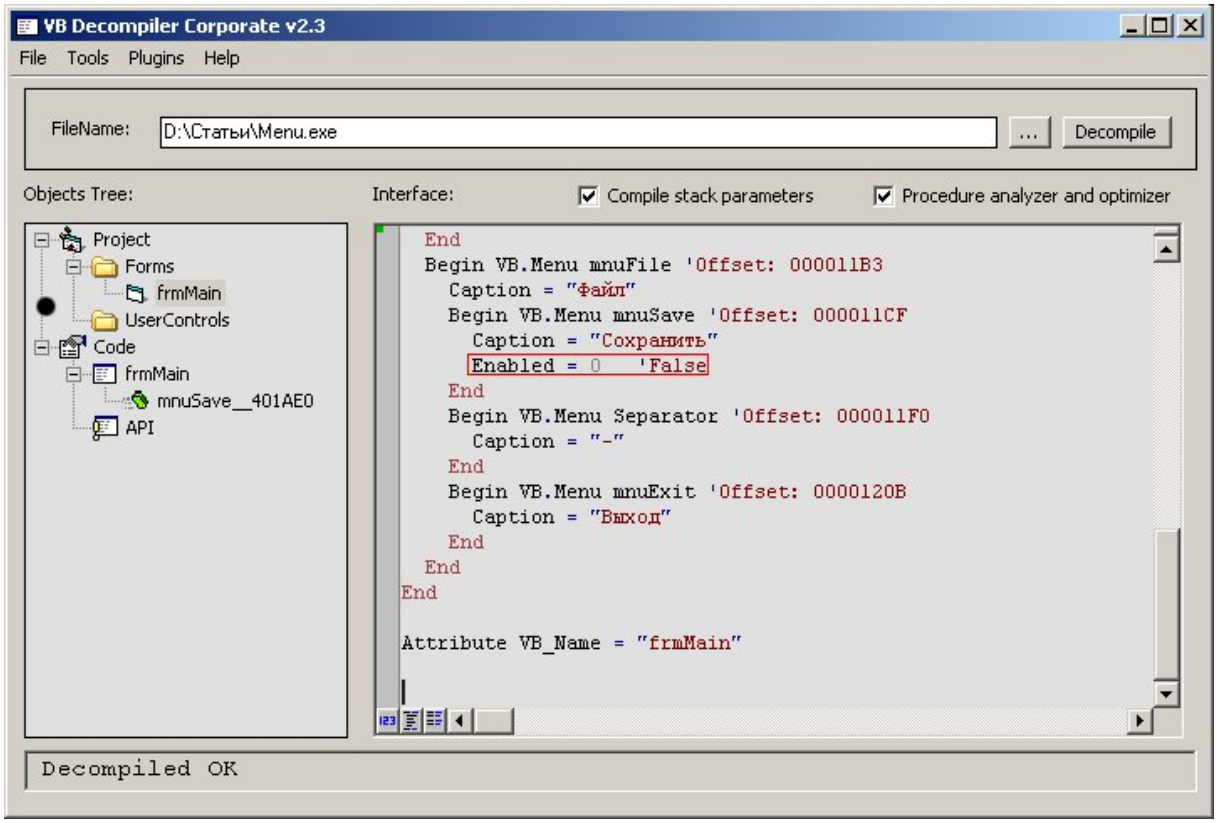
```
Begin VB.Form AC_ExDec_03_B 'Offset: 000010FA
 Caption = "Crackme for JosephCo's ExDec Program..."
 BackColor = &H80000008&
 ScaleMode = 1
 WhatsThisButton = 0  'False
 Icon = "AC_ExDec_03_B.frx":0
 LinkTopic = "Form1"
 ClientLeft = 60
 ClientTop = 345
 ClientWidth = 5580
 ClientHeight = 3075
 StartUpPosition = 3 'Windows Default
 Begin VB.Frame Frame1 'Offset: 000015A6
  Caption = "Button is in here"
  BackColor = &H0&
  ForeColor = &HFFFFFF&
  Left = 120
  Top = 1440
  Width = 5295
  Height = 1335
  TabIndex = 1
  BeginProperty Font
   Name = "Century Gothic"
   Size = 14,25
   Charset = 0
   Weight = 700
```

```
   Underline = 0 'False
   Italic = 0 'False
   Strikethrough = 0 'False
  EndProperty
  ... и так далее
```

Теперь предлагаю на конкретном примере разблокировать залоченное меню и показать невидимую кнопку.

Разлочка меню

Специально для демонстрации залоченного меню я написал простенький крякми.



В декомпиляторе отчетливо видно, что патчить.

В нем заблокирован пункт меню "Сохранить", как это обычно бывает в коммерческих программах. Попробуем его разблокировать. Для начала необходимо знать как можно сделать меню неактивным. Тут есть два способа. Первый - при проектировании поставить свойство Enabled у меню в False, второй - установить это свойство кодом при запуске формы. Представим что кодер поленился и установил это свойство при разработке меню. Тогда декомпиляем этот проект и поглядим. Для простоты не будем заново декомпилять в уме, а возьмем мой декомпилятор (Lite версия вполне подойдет) и декомпиляем. В разделе формы всего одна форма. Поищем там меню:

```
Begin VB.Menu mnuFile 'Offset: 000011B3
 Caption = "Файл"
 Begin VB.Menu mnuSave 'Offset: 000011CF
  Caption = "Сохранить"
  Enabled = 0  'False
 End
 Begin VB.Menu Separator 'Offset: 000011F0
  Caption = "-"
 End
 Begin VB.Menu mnuExit 'Offset: 0000120B
  Caption = "Выход"
 End
End
```

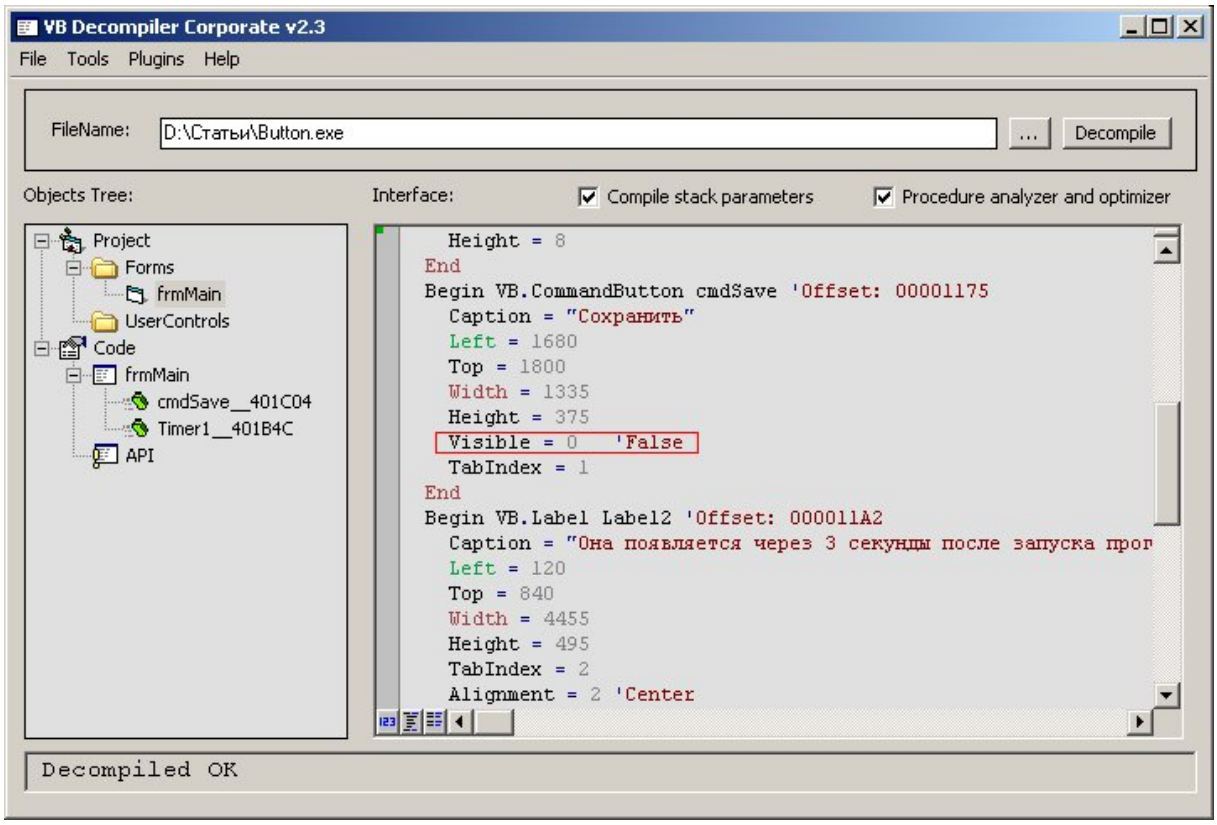
Опа, "Enabled = 0" это то что мы искали. Теперь подумаем как поправить. Откроем программу в hiew и перейдем по адресу 11F0:

```
00 00 02 07-00 6D 6E 75-53 61 76 65-00 13 03 09  O• mnuSave !¦0
00 D1 EE F5-F0 E0 ED E8-F2 FC 00 05-00 FF 02 1A  Сохранить ¦ яO>
```

Все стандартно сначало Name, затем Caption (03) и 05 - Enabled. После него идет один байт 00б что означает False. Замени на FF (True) и попробуй запустить. Меню разблокировано и при нажитии на него выводится MessageBox "cool". Вот и все - легкий взлом верно?

Палим инвиз

Несмотря на ICQ'шный стиль фразы мы не будем писать плагин детекта инвиза :) Мы будем делать видимыми скрытые кнопки.



Декомпилятор и на этот раз раскусил злой замысел шароварщика.

Для большего реализма я написал крякми, который после запуска делает видимой кнопку "Сохранить" через 3 секунды. Типа, если мы его купим, то ждать не придется - стандартно для Trial прог. Что ж, поглядим в декомпиляторе, что у нас с кнопкой "Сохранить":

```
Begin VB.CommandButton cmdSave 'Offset: 00001175
 Caption = "Сохранить"
 Left = 1680
 Top = 1800
 Width = 1335
 Height = 375
 Visible = 0  'False
 TabIndex = 1
End
```

Сразу бросается в глаза "Visible = 0", которое в таблице значится как 09. Переходим по смещению 1175 и проходим все свойства до 09. Видим 00, это False, меняем на FF (True) и дело сделано. Но! Данный крякми я сделал специально так, чтобы его можно было взломать разными способами. Рассмотрим еще один способ. Как можно сделать кнопку видимой через 3 секунды? Можно сделать цикл при запуске программы, но он на разных процессорах будет работать с разной скоростью.

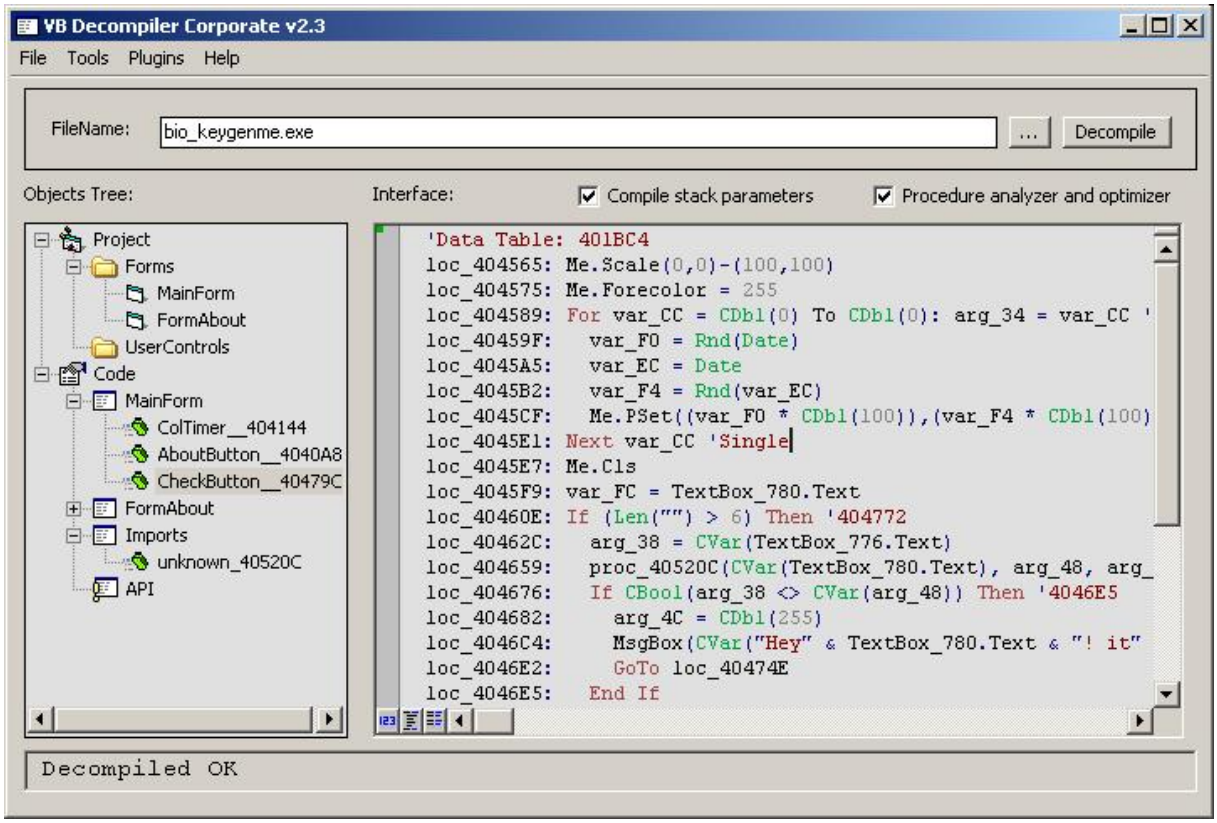
Можно использовать GetTickCount, но потребуется его проверять в While цикле, что тоже неудобно. Это все геморрой, который программисты не любят, потому они юзают таймеры. Таймер - это невидимый контрол на форме, у которого событие срабатывает каждые Interval милисекунд. Поищем ка мы любимый таймерна форме:

```
Begin VB.Timer Timer1 'Offset: 00001155
 Interval = 3000
 Left = 2880
 Top = 0
 Width = 59400
 Height = 8
End
```

Так и есть! 3000 милисекунд это 3 секунды. Заменим ка мы их на 1. Но! Автор забыл поставить End после установки свойства в кнопку поэтому таймер после события будет срабатывать и дальше и тормозить программу, потому проще использовать предыдущий способ, а таймер просто отключить, поставив интервал в 0.

Добавление новых свойств

Читая статью, ты наверняка задался вопросом, а как же добавить новое свойство в упакованный контрол? Добавить можно только разбором, вставкой и последующей сборкой обратно всей формы. Ясное дело что уже на старое место данные не поместятся, поэтому придется создавать новую секцию в файле или расширять последнюю и редиректить данные туда. Кроме того адрес на новое расположение формы придется прописать в структуре информации о форме. При этом нужно учитывать что если пользователь будет часто добавлять свойства, то нужно заранее в новой секции сделать запас в виде резервных байт под расширение каждой формы. Всю эту информацию о резерве байт и начале и длине каждой формы, вынесенной в новую секцию придется хранить. Для этого придется создать свою служебную структуру. Это все будет полезно если ты будешь писать свой редактор ресурсов VB. Если же ты просто собираешься исследовать программы, то знаний из этой статьи тебе вполне хватит.



Подпись: Лухitt средство для исследования VB программ

Заключение

Надеюсь, что у тебя не осталось после прочтения статьи вопросов по теме редактирования интерфейса VB программ. Если же они всеже есть - задавай их на моем форуме, посвященном вопросам декомпилированию VB http://vbdecompiler.dotfix.net я его периодически читаю и отвечаю на вопросы :)

Исследование функции SetTimer

Автор: lhc645

Дата: 2009-08-11

На днях, во время реверса одной программы, задалась вопросом, как вывести список таймеров, установленных win32 - функцией SetTimer (а если быть конкретнее, то мне нужны были адреса процедур - lpTimerFunc). Конечно, с практической точки зрения это малополезно, а с теоретической - очень даже. Что потребуется для успешного восприятия нижеизложенного? Из инструментов:

• IDA
• Наборы системных файлов для разных билдов Windows
• Отладочные символы
• msdn

Приступим непосредственно к исследованию. Сначала приведу прототип функции SetTimer (в процессе изложения удобно иметь его перед глазами)

```
UINT_PTR SetTimer(
    HWND hWnd,
    UINT_PTR nIDEvent,
    UINT uElapse,
    TIMERPROC lpTimerFunc
);
```

Путешествия в недры дизассемблированного кода начнем с юзермода. Когда мы вызываем функцию SetTimer из user32.dll вызывается сервис win32k.sys (это видно по номеру сервиса, который >1000h) NtUserSetTimer

```
; UINT __stdcall NtUserSetTimer(HWND hWnd, UINT nIDEvent, UINT uElapse, TIMERPROC lpTimerFunc)
public _NtUserSetTimer@16
_NtUserSetTimer@16 proc near

hWnd= dword ptr  4
nIDEvent= dword ptr  8
uElapse= dword ptr  0Ch
lpTimerFunc= dword ptr  10h

mov     eax, 121Eh
mov     edx, 7FFE0300h
call    dword ptr [edx]
retn    10h
_NtUserSetTimer@16 endp
```

Далее управление передается в ядро. Что ж, грузим в IDA win32k.sys

Обратимся к листингу NtUserSetTimer:

```
...
.text:BF803A10                 mov     ecx, [ebp+hwnd]
.text:BF803A13                 test    ecx, ecx
.text:BF803A15                 jz      short loc_BF803A4C
.text:BF803A17                 call    @ValidateHwnd@4 ; ValidateHwnd(x)
.text:BF803A1C                 test    eax, eax
.text:BF803A1E                 jz      short loc_BF8039F3
.text:BF803A20
.text:BF803A20 loc_BF803A20:                           ; CODE XREF: NtUserSetTimer(x,x,x,x)+49j
.text:BF803A20                 mov     edx, [ebp+uElapse]
.text:BF803A23                 cmp     edx, USER_TIMER_MINIMUM
.text:BF803A26                 jb      short set_minimum
.text:BF803A28
.text:BF803A28 loc_BF803A28:                           ; CODE XREF: NtUserSetTimer(x,x,x,x)-7j
.text:BF803A28                 mov     ecx, USER_TIMER_MAXIMUM
.text:BF803A2D                 cmp     edx, ecx
.text:BF803A2F                 ja      short set_maximum
```

```
.text:BF803A31
.text:BF803A31 loc_BF803A31:                           ; CODE XREF: NtUserSetTimer(x,x,x,x)-Cj
.text:BF803A31                 push    [ebp+lpTimerFunc]
.text:BF803A34                 push    edx
.text:BF803A35                 push    [ebp+nIDEvent]
.text:BF803A38                 push    eax
.text:BF803A39                 call    __SetTimer@16   ; _SetTimer(x,x,x,x)

...
```

Первым делом вызывается функция ValidateHwnd (подробнее о ней можно почитать в статье Twister-a http://www.wasm.ru/article.php?article=window_inject). Функция возвращает указатель на структуру tagWND. Этот указатель затем передается во внутреннюю функцию __SetTimer. В NtUserSetTimer проверяется параметр uElapse (об этом написано в msdn). Если быть точнее, то он должен быть USER_TIMER_MINIMUM<=uElapse<=USER_TIMER_MAXIMUM. Константы USER_TIMER_MINIMUM и USER_TIMER_MAXIMUM равны соответственно 0xA и 0x7FFFFFFF.
Идем дальше... Заглянем в функцию __SetTimer, которая сводится к вызову InternalSetTimer.

```
...
.text:BF803A5B                 mov     esi, [ebp+ptagWND]
...
.text:BF803A77 loc_BF803A77:                           ; CODE XREF: _SetTimer(x,x,x,x)+Bj
.text:BF803A77                 push    0
.text:BF803A79                 push    [ebp+lpTimerFunc]
.text:BF803A7C                 push    [ebp+uElapse]
.text:BF803A7F                 push    [ebp+nIDEvent]
.text:BF803A82                 push    esi
.text:BF803A83                 call    _InternalSetTimer@20 ; InternalSetTimer(x,x,x,x,x)
```

...

Далее смотрим _InternalSetTimer. Здесь-то и начинается самое интересное:

```
.text:BF803894 ; __stdcall InternalSetTimer(x, x, x, x, x)
.text:BF803894 _InternalSetTimer@20 proc near          ; CODE XREF: zzzUpdateCursorImage()-14p
.text:BF803894                                         ; _SetTimer(x,x,x,x)+2Ep ...
.text:BF803894
.text:BF803894 hwnd            = dword ptr  8
.text:BF803894 IDEvent         = dword ptr  0Ch
.text:BF803894 uElapse         = dword ptr  10h
.text:BF803894 TimerFunction   = dword ptr  14h
.text:BF803894 unknown         = dword ptr  18h
.text:BF803894
.text:BF803894 ; FUNCTION CHUNK AT .text:BF80384C SIZE 00000043 BYTES
.text:BF803894
.text:BF803894                 mov     edi, edi
.text:BF803896                 push    ebp
.text:BF803897                 mov     ebp, esp
.text:BF803899                 push    ebx
.text:BF80389A                 push    esi
.text:BF80389B                 push    edi
.text:BF80389C                 push    USER_TIMER_MINIMUM
.text:BF80389E                 pop     eax
.text:BF80389F                 xor     edi, edi
.text:BF8038A1                 cmp     [ebp+uElapse], eax
.text:BF8038A4                 jb      short set_el_minimum
.text:BF8038A6
.text:BF8038A6 loc_BF8038A6:                           ; CODE XREF: InternalSetTimer(x,x,x,x,x)-45j
.text:BF8038A6                 mov     eax, USER_TIMER_MAXIMUM
.text:BF8038AB                 cmp     [ebp+uElapse], eax
.text:BF8038AE                 ja      short set_el_maximum
```

Первым делом, как видно, проводятся уже знакомые проверки параметра uElapse. Затем вызывается внутренняя функция _FindTimer. Затем выделяется память под структуру, содержащую информацию о таймере

```
.text:BF8038C6                 push    30h             ; NumberOfBytes
.text:BF8038C8                 push    10h             ; char
.text:BF8038CA                 push    edi             ; int
```

```
.text:BF8038CB                 push    edi             ; int
.text:BF8038CC                 call    _HMAllocObject@16 ; HMAllocObject(x,x,x,x)
.text:BF8038D1                 mov     esi, eax
.text:BF8038D3                 cmp     esi, edi
.text:BF8038D5                 jz      error_allocate
```

Заполнение структуры (указываю только на интересные поля) - это процедура таймера, ид таймера и следующий таймер в списке

```
.text:BF8038E7                 mov     eax, [ebp+nIDEvent]
.text:BF8038EA
.text:BF8038EA loc_BF8038EA:                           ; CODE XREF: InternalSetTimer(x,x,x,x,x)+150j
; записываем в структуру по смещению 18h ид таймера
.text:BF8038EA                 mov     [esi+18h], eax
.text:BF8038ED                 mov     eax, _gptmrFirst
; записываем адрес следующей за нами структуры таймера
.text:BF8038F2                 mov     [esi+8], eax
.text:BF8038F5                 mov     [esi+0Ch], edi
.text:BF8038F8                 mov     eax, _gptmrFirst
.text:BF8038FD                 cmp     eax, edi
.text:BF8038FF                 jz      short ifFirst
.text:BF803901                 mov     [eax+0Ch], esi
.text:BF803904
.text:BF803904 ifFirst:                                ; CODE XREF: InternalSetTimer(x,x,x,x,x)+6Bj
.text:BF803904                 mov     _gptmrFirst, esi
.text:BF80390A
```

Тут появляется весьма интересная переменная _gptmrFirst - которая представляет собой указатель на односвязный список таймеров, зарегистрированных в системе. Последний зарегистрированный таймер помещается в начало списка.

По смещению 0x28 в нашей структуре содержится процедура таймера (ради которой и пришлось расковырять эту функцию):

```
.text:BF80394E                 mov     eax, [ebp+TimerFunction]
.text:BF803951                 mov     [esi+28h], eax
```

Приведенной выше информации достаточно для вывода информации о таймерах в winxp. Объявим такую структуру

```
typedef struct _timer_struct_xp
{
 DWORD unkn01[2];
 _timer_struct_xp * nextTimer;
 DWORD unkn02[2];
 PULONG ptagWND;
 DWORD nIDEvent;
 DWORD unkn03[3];
 PVOID TimerFunc;
 DWORD unkn04;
}timer_struct_xp,*ptimer_struct_xp;
```

Обратите внимание на поле ptagWND – указатель на структуру tagWND для окна, хендл которого передается как первый аргумент SetTimer. Нужно оно для идентификации процесса, связанного с таймером. Дело в том, что в структуре tagWND по смещению 8 лежит указатель на структуру WIN32THREAD, первое поле которой - указатель ETHREAD. А имея в наличии ETHREAD можно определить процесс, которому принадлежит нить.

Все бы ничего, но сначала необходимо найти переменную _gptmrFirst. Из вышеизложенного видно, что дизассемблировать процедуру за процедурой будет не очень рационально, да и ненадежно. Более красивого способа поиска этой переменной я не нашла, поэтому я определила только адреса в разных версиях Windows.

На этом можно было бы поставить точку, но тогда статья была бы неполной. Стоит сказать несколько слов о том, как обстоят дела в Windows 7 (бета и RC). Какие же изменения произошли в этой версии Винды? Во-первых структура под таймер теперь выделяется размером не 30h, а 44h

```
.text:BF8D73C2                push    44h             ; NumberOfBytes
.text:BF8D73C4                push    10h             ; char
.text:BF8D73C6                push    esi             ; int
.text:BF8D73C7                push    [ebp+var_4]     ; int
.text:BF8D73CA                call    _HMAllocObject@16 ; HMAllocObject(x,x,x,x)
.text:BF8D73CF                mov     esi, eax
```

Во-вторых переменная, хранящая указатель на список таймеров (который, в-третьих, из односвязного трансформировался в двусвязный) теперь именуется _gtmrListHead.

```
.text:BF8D741E                mov     [esi+1Ch], eax
.text:BF8D7421                mov     ecx, _gtmrListHead
.text:BF8D7427                lea     eax, [esi+0Ch]
.text:BF8D742A                mov     [eax], ecx
.text:BF8D742C                mov     dword ptr [eax+4], offset _gtmrListHead
.text:BF8D7433                mov     [ecx+4], eax
.text:BF8D7436                mov     _gtmrListHead, eax
```

Ну и, в-четвертых, смещение нужного нам поля с адресом процедуры таймера тоже поменялось

```
.text:BF8D7483                mov     eax, [ebp+lpTimerFunc]
.text:BF8D7486                mov     [esi+2Ch], eax
```

Кстати говоря, в Висте дела обстоят аналогичным образом. Все поля структуры реверсить незачем (может как-нибудь в другой раз), следующего объявления будет вполне достаточно

```
typedef struct _timer_struct_vistawin7
{
 DWORD unkn01[3];
 LIST_ENTRY leTimerList;
 PULONG ptagWND;
 DWORD unkn03[5];
 PVOID TimerFunc;
/*---*/
}timer_struct_vistawin7,*ptimer_struct_vistawin7;
```

Теперь, когда известно где брать нужную информацию, в коде драйвера, выводящего DbgPrint – ом поля структуры таймера, пропишем в коде RVA переменных (_gptmrFirst и _gtmrListHead) для разных билдов

```
switch(*NtBuildNumber)
 {
 case 7100:
  // Win7 RC
  uRVA = 0x21D970;
  break;
 case 6001:
  // Vista
  uRVA = 0x1E2110;
  break;
 case 7000:
  // Beta Win7
  uRVA = 0x2189F0;
  break;

 case 2600:
  // SP2
  uRVA = 0x1A8914;
  break;

 default:
  break;

 }
```

К сожалению, у меня не было в наличии большого числа различных win32k.sys, но, как мне кажется и этого достаточно. Я не люблю прописывать смещения, но в данном случае это оправдано. Кто хочет, может встроить любой понравившийся дизасм длин и отдельно написать код поиска для XP

и Vista. Перед этим желающим дизассемблировать надо будет найти ShadowSSDT и определить номер сервиса NtUserSetTimer.

Так же необходимо определить базу win32k.sys. Привожу две процедуры вывода

```
VOID KePrintTimersWin7Vista(DWORD dwBase,DWORD dwRVA)
{
 PLIST_ENTRY pleTimerHead;

 ptimer_struct_vistawin7 ptsCurrent;

 DWORD pWND;

 ULONG ptCurThread,pProcess;

 if(!dwBase) return;

 // RVA переменной + база win32k.sys
 pleTimerHead = (PLIST_ENTRY)(dwRVA+dwBase);

 ptsCurrent = CONTAINING_RECORD(pleTimerHead->Flink,
    timer_struct_vistawin7,
    leTimerList);

 DWORD dwCount = 0;

 do
 {

  if(MmIsAddressValid((PVOID)ptsCurrent))

  {

   DbgPrint("KePrintTimers: Current timer %X; Next timer %X; Timer func %X\n",
    ptsCurrent,
    ptsCurrent->leTimerList.Flink,
    ptsCurrent->TimerFunc
    );

   pWND = (DWORD)ptsCurrent->ptagWND;

   // определение по указателю на tagWND имени процесса
   if(pWND)
   {

    DbgPrint(" Timer window handle %X\n",
     *(PULONG)(pWND));
    // по смещению 8 в структуре tagWND лежит указатель на Win32Thread
    // в этой структуре первый дворд - указатель на ETHREAD
    ptCurThread = *(*(PULONG *)(pWND+8));

    if(ptCurThread)
    {
      DbgPrint(" Timer ETHREAD %X\n",ptCurThread);

      // смещение KPROCESS в ETHREAD
      if(*NtBuildNumber<7100)
      {

       pProcess = *(PULONG)(ptCurThread+0x144);

      }
      else
      {

       pProcess = *(PULONG)(ptCurThread+0x150);

      }
      DbgPrint(" EPROCESS =%X\n",pProcess);

      if(pProcess)
```

```
      {
       DbgPrint(" Process name = %s\n",
        PsGetProcessImageFileName((PEPROCESS)pProcess));
      }

     }
    }


    ptsCurrent = CONTAINING_RECORD(ptsCurrent->leTimerList.Flink,
     timer_struct_vistawin7,
     leTimerList);

    dwCount ++;

  }
  else
  {
    DbgPrint("KePrintTimers: Invalid address\n");

    break;
  }

  }while(ptsCurrent->leTimerList.Flink!=pleTimerHead);

  DbgPrint("KePrintTimers: Timers count =%d\n",dwCount);


 }
```

Как вы могли заметить смещение PKPROCESS в ETHREAD поменялось в Windows 7 RC, поэтому введена дополнительная проверка.

```
 VOID KePrintTimersXP(DWORD dwBase,DWORD dwRVA)
 {
  ptimer_struct_xp ptsInstalledTimers;

  if(!dwBase) return;

  ptsInstalledTimers = *(ptimer_struct_xp *)(dwRVA + dwBase);

  DWORD dwCount = 0;

  DWORD pWND;

  PETHREAD ptCurThread;


  while(ptsInstalledTimers)
  {

   if(MmIsAddressValid((PVOID)ptsInstalledTimers))

   {
    DbgPrint("KePrintTimers: Address valid\n");

    DbgPrint("KePrintTimers: Current timer %X; Next timer %X; Timer func %X; nIDEvent %X\n",
     ptsInstalledTimers,
     ptsInstalledTimers->nextTimer,
     ptsInstalledTimers->TimerFunc,
     ptsInstalledTimers->nIDEvent);

    pWND = (DWORD)ptsInstalledTimers->ptagWND;

    // определение по указателю на tagWND имени процесса
    if(pWND)
    {

     DbgPrint(" Timer window handle %X\n",
      *(PULONG)(pWND));
     // по смещению 8 в структуре tagWND лежит указатель на Win32Thread
```

```
    // в этой структуре первый дворд - указатель на ETHREAD
    ptCurThread = *(PETHREAD *)(*(PULONG)(pWND+8));

    if(ptCurThread)
    {
      DbgPrint(" Timer ETHREAD %X\n",ptCurThread);

      DbgPrint(" EPROCESS =%X\n",ptCurThread->ThreadsProcess);

      if(ptCurThread->ThreadsProcess)
      {
       DbgPrint(" Process name = %s\n",
        PsGetProcessImageFileName(ptCurThread->ThreadsProcess));
      }

    }
   }
   ptsInstalledTimers = ptsInstalledTimers->nextTimer;

   dwCount ++;

 }
 else
 {
   DbgPrint("KePrintTimers: Invalid address\n");

   break;
 }

}

 DbgPrint("KePrintTimers: Timers count =%d\n",dwCount);


}
```

Как видите все предельно просто.

Как проверить, что все правильно? Инсталлировать таймер в юзермоде и посмотреть будет ли он присутствовать в выводе.

nIDEvent – то, что вернула SetTimer.

На этом статью можно завершить, надеюсь, кому-нибудь было интересно.